Programació Java · Català

DAM / DAW

Sergi Garcia Barea — CC BY-SA 4.0

Índex

- Inici
  - 📚 Unitats
  - 📝 Butlletins
  - 📄 Llicència
- U01 — Introducció al pensament computacional
  - 🎯 Objectiu de la unitat
  - 🗺️ Mapa de la unitat
  - 📝 Boletins de la unitat
  - ✅ Criteris d'avaluació coberts (RA1)
  - 🚪 Per on commence?
- 01 — Què és un ordinador i un llenguatge de programació
  - 🧠 1. Un ordinador és extraordinàriament ràpid, però absolutament ximple
  - 🗣️ 2. Què és un llenguatge de programació?
  - 🔄 3. El traductor: Compiladors vs. Intèrprets
  - 🔤 4. Conceptes clau que escoltaràs cada dia
  - 💡 Idea clau per a recordar
- 02 — Programar no és memoritzar codi
  - 🧟 La trampa del "Miratge de la Comprensió"
  - 🏔️ La pantalla en blanc és el teu hàbitat natural
  - 🕳️ Com evitar la farsa de "Copiar i Apegar" (Tutorial Hell)
  - 💡 Idea clau per a recordar
- 03 — Primer pensem, després escrivim
  - 🔮 La il·lusió de la literalitat (El teu ordinador no és endeví)
  - 🛤️ El camí sagrat del programador
  - 🤖🧪 Exercici pràctic: El test del Robot Literal
  - 😈 Els 5 dimonis de la lògica
  - 💡 Idea clau per a recordar
- 04 — La descomposició
  - 🐘 Com menjar-se un elefant? (A mossegades!)
  - 🏛️ L'art de la descomposició en sub-blocs
  - ⚠️ Les 3 regles d'or per a no morir en la divisió
  - 💡 Idea clau per a recordar

- 05 — Debugging i casos límit
  - 🔴 1. El canvi de xip: El text vermell és el teu amic
  - 🧪 2. Pensar com un programador: Provar els casos límit
  - 💡 Idea clau per a recordar
- 06 — El protocol de pau
  - 🛡️ Els 4 passos del protocol
  - 💡 Idea clau per a recordar
- 07 — Practicar (menys mirar, més intentar)
  - 🎭 La gran farsa del “Ja ho he entés”
  - ⏳🎯 La Regla del 80/20 per a l’FP a Distància
  - 🏋️ Com entrenar la “musculatura de programador”
  - 💡 Idea clau per a recordar
- 08 — Com utilitzarem la IA
  - 🚨 El gran perill: Pagar a un altre perquè faci les teves flexions
  - 🏛️💬 El Mètode Pro: La IA com a Tutor Socràtic
  - 📊 Comparativa de mentalitat
  - 🕵️ La Regla del “No et fies ni de la teva ombra” (Al·lucinacions)
  - 💡 Idea clau per a recordar
- 09 — El meu primer problema (pont cap a Java)
  - 🥊 El Repte: El Comparador de Nombres
  - 🧩 Pas 1: Descomposició i Entrenament de Dades (Inputs i Outputs)
  - 👣 Pas 2: L’Algorisme en Pseudocodi (Lògica pura)
  - 🧪 Pas 3: El testing de casos límit (Mentalitat de detectiu)
  - 🌉 I ara… On està el pont cap a Java? ☕
  - 💡 Idea clau per a tancar la Unitat 1
  - 💻 Termes tècnics d’Infraestructura
  - 🧠 Termes de lògica i procés
  - 🐛 Termes de gestió d’errors i control
  - 🤖 Termes de treball i metodologia
- U02 — Introducció a Java i l'entorn de treball
  - 🎯 Objectiu de la unitat
  - 🗺️ Mapa de la unitat
  - 📝 Butlletins de la unitat
  - ✅ Criteris d’avaluació coberts (RA1)

    - 🚪 Per on començo?
- 01 — Què és Java?
    - 🔍 D'on surt aquest llenguatge?
    - 📦 Què és això del bytecode?
    - ☕ La trilogia del cafè: JVM, JRE i JDK
    - 🌍 Per què segueix viu Java tants anys després?
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 02 — Instal·lació del JDK
    - 🛒 Quin JDK instal·le?
    - 🚀 Els primers passos
    - 🛠️ L'IDE: la teva navalla suïssa
    - 🏫 Exemple guiat: el teu primer projecte en VS Code
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 03 — El teu primer programa
    - 👋 Hola Món, el ritual d'iniciació
    - 🔬 Dissecció d'Hola Món
    - 🗝️ Per què public static void main(String[] args)?
    - 🚪 El mètode que no es crida
    - ⭐ Tracta de pensar com el codi: tu ets la JVM
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 04 — El depurador
    - 🔍 Què és depurar?
    - 🛠️ Les quatre eines del detectiu en VS Code
    - 🏫 Exemple guiat: el cas del sospitós en VS Code
    - 🧠 La regla d'or del depurador
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases

    - 🐛 Vocabulari ràpid
- 05 — Comentaris i documentació
    - 📝 Els tres tipus de comentaris
    - 🗝️ El consell que t'estalviarà hores: comenta el "per què", no el "què"
    - 📖 Javadoc: documentació que es genera sola
    - 🏫 Exemple guiat: documentar un programa complet
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 06 — Arguments de línia d'ordres
    - 🎒 Què és "args"?
    - 🔢 Com s'indexen les paraules
    - 🧠 Per a què serveix passar arguments?
    - 🏫 Exemple guiat: el programa que et saluda
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 07 — El compilador i els seus errors
    - 🔍 Compilar ≠ executar
    - 😤 L'error de l'aprenent: 4 errors en un programa
    - 🔎 Com llegir un missatge d'error de javac
    - 🚨 Errors típics de l'aprenent (i el seu remei)
    - 🧩 EL LÍO: el codi remenat
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 08 — L'IDE i el teu flux de treball
    - 🗂️ Anatomia d'un projecte en VS Code
    - 🔁 El cicle de treball (el teu nou bucle de vida)
    - ⌨️ Les dreceres de VS Code que et faran semblar un pro
    - 🧩 Autocompletat (IntelliSense): el teu company silenciós
    - 🏫 Exemple guiat: de zero a executar en 60 segons en VS Code
    - 🎯 Mini-comprovació

    - ✅ Resum en 3 frases
    - 🐛 Vocabulari ràpid
- 09 — Repàs
    - ⭐ Tracta de pensar com el codi, my friend…
    - 🔥 Fireside Chat: JDK vs JRE
    - 🕵️ Qui soc?
    - 😡 CONRAD VS EL MÓN: “El compilador m’odia”
    - ⚡ Laboratori de Tortura: el programa que no arranca
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a pensar
    - 💬 Preguntes d’entrevista de treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Poscrèdits
- U03 — Variables, tipus i operadors
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d’avaluació coberts (RA2 + RA1)
    - 🚪 Per on començo?
- 01 — Variables i tipus primitius
    - 📬 La idea en una frase
    - 📦 La declaració: la recepta d’una caixa
    - 🏷️ Les regles de nomenclatura (o com no ficar-la)
    - 📐 Els 8 primitius: el catàleg de caixes
    - 🎒 Quina caixa usi per a cada dada?
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — String, constants i final
    - 📬 La idea en una frase
    - 🔤 String: la caixa màgica que no és una caixa
    - 🧊 La immutabilitat: no toquis, que es trenca
    - 🧲 == vs .equals(): la trampa clàssica

    - 🔒 Constants amb final: caixes amb superglue
    - 🏫 Exemple guiat: la factura que no es pot tocar
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — Operadors aritmètics
    - 📬 La idea en una frase
    - 💪 El dia al gimnàs: els 5 exercicis bàsics
    - ⚠️ La divisió entera mata
    - 🎭 Precedència: la llei del menjador
    - 🌀 Assignació composta: la drecera peresosa
    - 💪 ++ i --: flexions per a variables
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Relacionals, lògics i ternari
    - 📬 La idea en una frase
    - ⚖️ Relacionals: el jutge de la discussió
    - 🚪 Lògics: el porter del club
    - ⚡ Curtcircuit: el porter que no mira
    - 🚦 El ternari: el bouncer del club
    - 🏫 Exemple guiat: el porter del club
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — Casting i conversions
    - 📬 La idea en una frase
    - 🪜 Conversió implícita (widening): mudar-se a una caixa més gran
    - 📉 Conversió explícita (narrowing): la maleta XXL en un Smart
    - 🪓 El truncament: el cèntim oblidat
    - 💥 El desbordament: quan apretes de més
    - 🏫 Exemple guiat: el guarda del magatzem
    - ⭐ Be the Code, my friend…

    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · Scanner: llegir pel teclat
    - 📬 La idea en una frase
    - 📖 El paquet: importar la llibreria
    - 🏗️ Instanciar: el constructor
    - 🔢 Demanar dades: els mètodes next
    - ⚠️ El embolic de nextLine() després de nextInt()
    - 🏫 Exemple guiat: la calculadora de la propina
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Consola: sortida amb format i errors d'entrada
    - 📬 La idea en una frase
    - 🖨️ Sortida amb format: System.out.printf
    - 🧵 String.format: el mateix, però sense imprimir
    - 💶 Números grans: NumberFormat
    - 🚨 Errors clàssics del Scanner (i els seus remeis)
    - 🏫 Exemple guiat: el validador que no es trenca
    - ⭐ Sé el Codi, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Math.random() i nombres aleatoris
    - 📬 La idea en una frase
    - 🎲 El casino: què retorna Math.random()
    - 🎯 La fórmula: de 0.0-1.0 al que tu vulguis
    - 🧰 Altres eines del casino: la classe Math
    - 🏫 Exemple guiat: el joc dels daus
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Mètodes útils de String
    - 📬 La idea en una frase

    - 🪄 El gimnàs del text: els mètodes que mesuraràs
    - 🔍 Els mètodes que busquen
    - ✂️ Els mètodes que retallen
    - 🏫 Exemple guiat: el nom de l'usuari
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 10 · Repàs interactiu: la caixa que no cabia
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: int vs double
    - 🕵️ Qui Soc?
    - 😡 CONRAD VS EL MÓN: "El compilador odia la teva memòria"
    - ⚡ Laboratori de Tortura: el programa que cobra malament
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U04 — Estructures de control i excepcions bàsiques
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA3)
    - 🚪 Per on començo?
- 01 — if, else if i else: el semàfor del codi
    - 📬 La idea en una frase
    - 🚦 El semàfor: if i else
    - 🔀 else if: quan hi ha més de dos camins
    - 🪆 If anidats: semàfors dins de semàfors
    - 🎚️ El ternari: el if de butxaca
    - 🏫 Exemple guiat: la discoteca municipal

    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — switch: el menú del restaurant
    - 📬 La idea en una frase
    - 🍽️ La carta del restaurant
    - 🧱 Les peces del puzle
    - 🌀 El fall-through: error o superpoder?
    - 🆚 switch vs else if: el duel
    - 🏫 Exemple guiat: el menú del dia
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — Bucles: while i do-while
    - 📬 La idea en una frase
    - 🏃 while: comprova i després corre
    - ♾️ El bucle infinit (i com sortir-ne)
    - 🏃 do-while: corre i després comprova
    - 🆚 while vs do-while: el duel ràpid
    - 🏫 Exemple guiat: el compte enrere
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Bucle for i bucles anidats
    - 📬 La idea en una frase
    - 🔢 L'anatomia del for
    - 🔁 Els tres bucles són el mateix acudit
    - 🧩 Bucles anidats: la graella d'exercicis
    - 🏫 Exemple guiat: la taula de multiplicar
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — break, continue i etiquetes

    - 📬 La idea en una frase
    - 🚪 break: el botó de parada
    - ⏭️ continue: el botó de saltar
    - 🏷️ Etiquetes: el GPS dels bucles anidats
    - 🏫 Exemple guiat: el detector de nombre primer
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 — Excepcions bàsiques
    - 📬 La idea en una frase
    - 💥 El crash: la teva primera excepció
    - 🌳 La família Throwable: l'arbre genealògic
    - 🗺️ Les excepcions més comunes: la guia de camp
    - 🏫 Exemple guiat: el lector d'edats
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 — try, catch i finally
    - 📬 La idea en una frase
    - 🛡️ L'estructura completa
    - 🎯 Captura múltiple: diversos catch en fila
    - 🧽 La variable e: el botí de l'error
    - 🏫 Exemple guiat: el menú a prova de bombes
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — throw i excepcions pròpies
    - 📬 La idea en una frase
    - 🎳 throw: llança la pedra
    - 🏗️ Excepcions pròpies: el teu defecte a mida
    - ⚖️ checked vs unchecked: la burocràcia de les excepcions
    - 🏫 Exemple guiat: la màquina expenedora
    - ⭐ Be the Code, my friend…

    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Repàs interactiu: controla les estructures
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: if-else vs switch
    - 🕵️ Qui Soc?
    - 🤬 CONRAD VS EL MÓN: “El bucle que no s’acaba”
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: la màquina que plora
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d’Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U05 — Funcions i mètodes
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d’avaluació coberts (RA2)
    - 🚪 Per on començo?
- 01 · Què és una funció?
    - 📬 La idea en una frase
    - 🍳 El problema: el paràgraf infinit
    - 📖 La recepta amb nom
    - 🔧 Funció o mètode? (no, no és el mateix… bé, sí)
    - 🧰 Què guanya el teu programa al tallar-lo
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · El teu primer mètode
    - 📬 La idea en una frase

    - ✍️ La primera recepta
    - 🔍 La firma, paraula per paraula
    - 🎬 main també és un mètode
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · Paràmetres: l'entrada
    - 📬 La idea en una frase
    - 🔁 El problema: el mètode de sempre
    - 📥 Com es declaren
    - 🎯 Regles de les entrades
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 · return: la sortida
    - 📬 La idea en una frase
    - 📤 Dos oficis, dos firmes
    - 🚧 return talla l'execució
    - 🎭 El pecat d'imprimir quan has de tornar
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · Àmbit de variables
    - 📬 La idea en una frase
    - 🏠 Cada mètode és una casa
    - 📏 Les regles de l'àmbit
    - 🚪 El que entra per la porta també és de la casa
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · Errors freqüents
    - 📬 La idea en una frase
    - 🧯 La galeria dels 7 entrebancs

    - 🩺 Com diagnosticar sense perdre't
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Divideix el problema
    - 📬 La idea en una frase
    - 🎼 El director no toca els instruments
    - 🔪 Com trossegar: el mètode dels verbs
    - 🧪 Abans i després: l'informe de notes
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · Be the Code
    - 📬 La idea en una frase
    - 🛠️ El teu punt de partida
    - 🪜 Pas a pas (pistes que no regalen el final)
    - 🧪 El Lío: el refactor malograt
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Repàs interactiu
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: void vs return
    - 🕵️ Qui soc?
    - 🤖 CONRAD VS EL MÓN: "El main que es va fer el llist"
    - 🎮 El joc de les decisions
    - ⚡ Laboratori de tortura: el programa sense nom
    - 🏆 Assoliments d'aquesta unitat
    - 🤔 Atreveix-te a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntes d'entrevista de treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Postcrèdits

- U06 — Arrays
  - 🎯 Objectiu de la unitat
  - 🗺️ Mapa de la unitat
  - 📝 Butlletins de la unitat
  - ✅ Criteris d'avaluació coberts (RA6)
  - 🚪 Per on començo?
- 01 · Arrays: l'aparcament de dades
  - 📬 La idea en una frase
  - 🐱 El problema: tens 100 gats i un sol nom
  - 🅿️ L'array: el teu primer aparcament
  - 🚗 Com ficar coses a l'aparcament
  - ⭐ Be the Code, my friend…
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 02 · Recórrer arrays: for i for-each
  - 📬 La idea en una frase
  - 🔗 El duo inseparable: for + array
  - 🛋️ for-each: la variant peresosa
  - ⭐ Be the Code, my friend…
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 03 · Arrays multidimensionals
  - 📬 La idea en una frase
  - 🏢 L'aparcament de diverses plantes
  - 🎲 Arrays de arrays irregulars
  - 🔁 Recórrer un array 2D: els bucles niats
  - 🧱 Per a què serveix de veritat
  - ⭐ Be the Code, my friend…
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 04 · La classe Arrays: la teva navalla suïssa
  - 📬 La idea en una frase
  - 🧰 La classe Arrays

    - 🧭 El mapa de la navalla
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · Arrays i mètodes
    - 📬 La idea en una frase
    - 🏃 Passant el testimoni
    - 🆚 Primitius vs arrays: la diferència clau
    - 🧪 Mètodes que usen arrays
    - 🏁 El famós main(String[] args)
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · Aplicacions dels arrays
    - 📬 La idea en una frase
    - 📝 Arrays de String: la llista de la classe
    - 🔤 char[] vs String
    - 🧍 Arrays d'objectes: l'aparcament de persones
    - 🗂️ Arrays de arrays: les dades en taula
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Be the Code: l'aparcament es gestiona
    - 📬 La idea en una frase
    - 🕶️ Don Tip: la recepta mental de l'invers al lloc
    - 🧩 REPTE 1: Invertir l'array al lloc
    - 🧩 REPTE 2: Buscar totes les posicions
    - 🧩 EL LÍO: l'aparcament que es va rebel·lar
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · Array-revelde: errors comuns i depuració
    - 📬 La idea en una frase
    - 👹 La galeria de monstres

    - 🤬 CONRAD VS EL MÓN: “L’array que no es calla”
    - 🐛 Depurar arrays sense perdre el cap
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Repàs interactiu: l'aparcament a examen
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: for clàssic vs for-each
    - 🕵️ Qui soc?
    - 🤬 CONRAD VS EL MÓN: “L’array que no para”
    - 🎮 El joc de les decisions
    - ⚡ Laboratori de tortura: la mitjana que es resisteix
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntes d’entrevista de treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Postcrèdits
- U07 — Algorítmica I: fonaments
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d’avaluació coberts (RA2 + RA6)
    - 🚪 Per on començo?
- 01 — Què és un algoritme
    - 📬 La idea en una frase
    - 🏛️ Les propietats d’un algoritme
    - 🧠 La idea vs. el codi
    - 📦 Les dues grans famílies
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases

- 02 — Cerca lineal
  - 📬 La idea en una frase
  - 👟 L'algoritme
  - ⏱️ Com de ràpida és?
  - ⭐ Be the Code, my friend...
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 03 — Cerca binària
  - 📬 La idea en una frase
  - ⚠️ El requisit imprescindible
  - 🕯️ L'algoritme
  - 🧮 Per què esquerra + (dreta - esquerra) / 2 i no (esquerra + dreta) / 2?
  - ⏱️ L'anàlisi: O(log n)
  - ⭐ Be the Code, my friend...
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 04 — Ordenació bombolla
  - 📬 La idea en una frase
  - 🫧 L'algoritme
  - 🏎️ Per què és tan lenta?
  - 🚩 L'optimització del flag
  - ⭐ Be the Code, my friend...
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 05 — Ordenació per inserció
  - 📬 La idea en una frase
  - 🃏 L'algoritme
  - 👣 Pas a pas
  - 📊 L'anàlisi: quan és bona?
  - ⭐ Be the Code, my friend...
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 06 — Complexitat algorísmica: Big O

    - 📬 La idea en una frase
    - 📈 Les complexitats més comunes
    - 🧪 Big O en codi
    - 📏 Les regles pràctiques de Big O
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 — Triar l'algoritme adequat
    - 📬 La idea en una frase
    - 📋 La taula de resum
    - 🧮 La pregunta clau: ¿ordenar abans de buscar?
    - 🏫 Exemple guiat: el catàleg de la botiga
    - 📊 Regles d'or per a decidir
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Be the Code: cerca binària des de zero
    - 📬 La idea en una frase
    - 🕶️ Don Tip: la recepta mental de la binària
    - 🧩 REPTE 1: Cerca binària des de zero
    - 🧩 REPTE 2: Bombolla a mà
    - 🧩 EL LIO: la bombolla que apesta
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Repàs interactiu: ordena i busca com un professional
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend...
    - 🔥 Fireside Chat: cerca lineal vs cerca binària
    - 🕵️ Qui Soc?
    - 😡 CONRAD VS EL MÓN: "L'algoritme que no s'acaba"
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: la màquina que no ordena
    - 🏆 Assoliments d'aquesta unitat

    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U08 — Algorítmica II: tècniques avançades i modularitat
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA2 + RA6)
    - 🚪 Per on començo?
- 01 — Recursivitat: una funció que es crida a si mateixa
    - 📬 La idea en una frase
    - 🔁 El cicle de la vida recursiva
    - 📚 La pila de crides: l'origen de tota la diversió
    - 🥊 Recursivitat vs iteració: el duel
    - 🌰 Un primer exemple complet
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — Recursivitat: exemples clàssics
    - 📬 La idea en una frase
    - 🧮 Factorial: el "Hola Món" de la recursivitat
    - 🐰 Fibonacci ingenu: no ho facis a casa
    - ⚡ Fibonacci optimitzat: amb memoització, ara sí
    - 🔍 Palíndroms: la paraula mirall
    - 🗼 Torres de Hanoi: el rei dels clàssics
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — Divide i venceràs
    - 📬 La idea en una frase
    - 🧩 Els tres passos del patró
    - 📊 Quan paga la pena?
    - 💡 Un exemple senzill: el màxim d'un array

    - 🌳 Aplicacions famoses
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Quicksort: el ràpid
    - 📬 La idea en una frase
    - 🎯 Com funciona
    - 🛠️ Implementació
    - 🎚️ Elecció del pivot
    - 📊 Complexitat
    - 🚫 És estable?
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — Mergesort: el fiable
    - 📬 La idea en una frase
    - 🎯 Com funciona
    - 🛠️ Implementació
    - 📊 Complexitat
    - ✅ És estable?
    - 🏆 Quan brilla Mergesort
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 — Comparació: quan usar cada ordenació
    - 📬 La idea en una frase
    - 🧭 La taula de decisió
    - 🥊 El duel dels titans
    - 🕵️ Què fa Java per dins?
    - 📊 Rendiment en la pràctica
    - 🧠 La lògica que hi ha darrere
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 — Be the Code: quicksort des de zero
    - 📬 La idea en una frase
    - 🚦 Exercici guiat 1: Quicksort

    - 🚦 Exercici guiat 2: Mergesort
    - ⭐ Nivells de logro
    - 🧩 El merder
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Altres tècniques algorísmiques
    - 📬 La idea en una frase
    - 🍬 Algorismes voracos (greedy)
    - 🔁 Backtracking (volta enrere)
    - 🧠 I amb això, el panorama
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Modularitat
    - 📬 La idea en una frase
    - 🧩 Per què importa tant?
    - 📦 La peça oficial en Java: el paquet
    - 🎯 Cohesió alta, acoblament baix
    - ✂️ Com decideixes els talls
    - 🔮 El que ve després
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
- 10 · Repàs interactiu
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🎯 Vertader o Fals
    - 🔥 Fireside Chat: Quicksort i Mergesort discuteixen
    - 🕵️ Qui Soc?
    - 🧩 Completa el codi
    - 🤬 CONRAD VS EL MÓN: “La teva recursió ha explotat”
    - ⚡ Laboratori de Tortura: el programa que no ordena
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits

    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U09 — POO: classes i objectes
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA2 + RA4)
    - 🚪 Per on començo?
- 01 — Què és la POO
    - 📬 La idea en una frase
    - 📜 De la recepta al món real
    - 🧩 Els dos ingredients: atributs i mètodes
    - 🏛️ Els quatre pilars (una promo, no el capítol)
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — Classes i objectes: el tallagalletas i les galetes
    - 📬 La idea en una frase
    - 🍪 El tallagalletas i la galeta
    - 🔨 La teva primera classe
    - 🏭 new: la màquina de galetes
    - 🥊 Classe vs objecte: el ring
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — Atributs i mètodes
    - 📬 La idea en una frase
    - 🏷️ Els atributs: les propietats de la cosa
    - 🎮 Els mètodes: els comportaments de la cosa
    - 📞 Cridar els mètodes

    - 🧬 Cada objecte amb la seva pròpia còpia
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Constructors
    - 📬 La idea en una frase
    - 🎉 Què és un constructor?
    - 🆓 El constructor per defecte (el que regala Java)
    - 🎛️ El constructor amb paràmetres (el forn amb programa)
    - 🏭 new i el constructor, un tàndem
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — La paraula clau this
    - 📬 La idea en una frase
    - 🔍 Desambiguar: atribut vs paràmetre
    - 🔗 Cridar un altre constructor: this(…)
    - 📦 Passar-te a tu mateix com a argument
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 — Referències, null i memòria
    - 📬 La idea en una frase
    - 📬 La carta davant de la casa
    - 👯 L'àlies: dues variables, un objecte
    - ⚰️ null: la referència al buit
    - 🏠 Primitius vs referències: el resum
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig

    - ✅ Resum en 3 frases
- 07 — Sobrecàrrega i pas per valor
    - 📬 La idea en una frase
    - 🎭 Sobrecàrrega: el mateix nom, altres mans
    - 📦 Pas per valor: Java només sap copiar
    - 📊 El resum que et salva en l'examen
    - ⭐ Be the Code, my friend…
    - ❓ No Hi Ha Preguntes Tontes!
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Be the Code: dissenya la teva classe
    - 📬 La idea en una frase
    - 🚦 Exercici guiat: el compte bancari
    - ⭐ Nivells d'assoliment
    - 🧩 El lío
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Repàs interactiu: domina les classes
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: Classe vs Objecte
    - 🕵️ Qui Soc?
    - 🤬 CONRAD VS EL MÓN: "Constructor, on ets?"
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: la fàbrica que plora
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U10 — Visibilitat, encapsulació i static
    - 🎯 Objectiu de la unitat

    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA4)
    - 🚪 Per on començo?
- 01 — Visibilitat: l'art de no ensenyar-ho tot
    - 📬 La idea en una frase
    - 🏠 El problema de la casa de vidre
    - 🚪 Per què importa la visibilitat
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — public, private i protected
    - 📬 La idea en una frase
    - 🗂️ Els 4 nivells: de la tanca al diari
    - 📣 public: la tanca publicitària
    - 🔒 private: el diari amb cadenat
    - 👨‍👩‍👧 protected: els secrets de família
    - 🏘️ package-private: el xafardeig del barri
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — Encapsulació: el pilar de la POO
    - 📬 La idea en una frase
    - 📦 La fórmula de l'encapsulació
    - 🎁 Els avantatges (o per què no dormiràs pitjor)
    - 🧬 La convenció JavaBeans: el protocol
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Getters i setters
    - 📬 La idea en una frase
    - 🏷️ La convenció de noms (memoritza-la)
    - 🚪 El setter com a porter: validar

    - 🔒 Getter sense setter: atributs immutables
    - 🤷 No hi ha preguntes tontes
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — Atributs static
    - 📬 La idea en una frase
    - 📱 El grup de WhatsApp vs els missatges privats
    - 🧠 Com es veu en memòria
    - 📞 L'ús clàssic: comptadors i ids
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 — Mètodes static i el main
    - 📬 La idea en una frase
    - 📞 El telèfon de la classe
    - 🚫 El que un mètode estàtic NO pot fer
    - 🧮 La classe Math: la fàbrica de mètodes estàtics
    - 🎉 Per què main és static
    - 🤷 No hi ha preguntes tontes
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 — Constants de classe: static final
    - 📬 La idea en una frase
    - 🏷️ La constant de classe
    - 🔒 La regla de les MAJÚSCULES
    - 🏛️ Les classes utilitàries: el "no necessiti parella" de Java
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Be the Code: encapsula la teva classe
    - 📬 La idea en una frase

    - 🏚️ El punt de partida: la classe del caos
    - 🧰 Pas 1: atributs privats
    - 🧰 Pas 2: getters per a llegir
    - 🧰 Pas 3: setter per al que es pot canviar
    - 🧰 Pas 4: validar també en els mètodes
    - 🧰 Pas 5: el toc final, constants
    - 🏆 La classe final (compara-la amb la del caos)
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Repàs interactiu: blinda-ho tot
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: la classe i l'objecte
    - 🕵️ Qui Soc?
    - 🎮 El Joc de les Decisions
    - 🤬 CONRAD VS EL MÓN: "Per què tot públic?"
    - ⚡ Laboratori de Tortura: la caixa forta sense cadenat
    - 🧩 L'EMBOLIC: el setter que no validava
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U11 — Herència, polimorfisme i interfícies
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA4 i RA7)
    - 🚪 Per on començo?
- 01 — Herència: quan els teus fills segueixen els teus passos
    - 📬 La idea en una frase
    - 🧬 Què és l'herència?

    - 🔑 extends: “Soc com tu, però amb millores”
    - ⚖️ La regla d’or: IS-A vs HAS-A
    - 🌳 Jerarquia de classes: l’arbre genealògic
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 — super i @Override
    - 📬 La idea en una frase
    - 📞 super: cridant mamà/papà perquè t’ajuden
    - ✍️ @Override: “Papa, jo ho faig millor”
    - 🔒 final: quan vols bloquejar l’herència
    - 🏗️ Constructors en l’herència: la cadena de muntatge
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 — protected i jerarquies de classes
    - 📬 La idea en una frase
    - 🔐 protected: el secret que només la família veu
    - 🌳 Dissenyant jerarquies sensates
    - 💥 El problema de la classe base fràgil
    - 🧩 Composició: “tenir” en comptes de “ser”
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 — Polimorfisme: el camaleó de la POO
    - 📬 La idea en una frase
    - 🎭 El mateix mètode, diferents comportaments
    - 🔮 Dynamic binding: la decisió en runtime
    - 🔗 Referències polimòrfiques
    - 📚 Polimorfisme amb col·leccions
    - 🧑‍💼 Polimorfisme amb paràmetres
    - 🚫 Els atributs NO són polimòrfics
    - ⭐ Be the Code, my friend…

    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 — instanceof i downcasting
    - 📬 La idea en una frase
    - 🕵️ instanceof: “Qui ets realment?”
    - 📉 Downcasting: baixar la referència
    - 🛡️ El combo segur: instanceof + downcasting
    - ✨ Bonus: pattern matching (Java 16+)
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 — La classe Object: el besavi de tot
    - 📬 La idea en una frase
    - 👴 Object: l’arrel de tot
    - 📇 toString(): la targeta de presentació
    - ⚖️ equals(): mateix objecte o iguals?
    - 🏷️ hashCode(): el codi de barres
    - 🕵️ getClass(): el nom de l’objecte real
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 — Classes abstractes
    - 📬 La idea en una frase
    - 🎨 Què és una classe abstracta?
    - 📋 You MUST: implementar els mètodes abstractes
    - 📐 Exemple: figures geomètriques
    - 🏗️ Constructors en classes abstractes
    - 🧩 Template Method: les abstractes en acció
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 — Interfícies: el contracte que signes
    - 📬 La idea en una frase

    - ✍️ Declarant i implementant
    - 🎭 Polimorfisme amb interfícies
    - 🍜 Múltiples interfícies
    - 🩹 default Methods: pegats sense trencar res
    - ⚖️ La gran decisió: abstract class vs interface
    - 🥊 EL RING: extends vs implements
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 — Repàs interactiu: domina l'herència
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: Classe abstracta vs Interfície
    - 🕵️ Qui soc?
    - ⚖️ Vertader o fals
    - ⚡ Laboratori de tortura: l'herència que plora
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntes d'entrevista de treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Poscrèdits
- U12 — Col·leccions, genèrics i mapes
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA6)
    - 🚪 Per on començo?
- 01 · ArrayList i el Java Collections Framework
    - 📬 La idea en una frase
    - 🅿️ El problema: el teu array s'ha quedat sense places
    - 🎓 El Java Collections Framework (JCF)
    - 🅿️📈 ArrayList: l'aparcament que creix sol

    - 🚫 ArrayList NO guarda primitius
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · LinkedList: la conga line
    - 📬 La idea en una frase
    - 🎉 LinkedList: la conga line
    - 🛒 Cues i piles: les dues disfresses de LinkedList
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · HashSet, TreeSet i LinkedHashSet: els conjunts
    - 📬 La idea en una frase
    - 🚫 HashSet: el porter que no deixa duplicats
    - 🧭 TreeSet: l'organitzat
    - 🧠 LinkedHashSet: el que recorda l'ordre d'arribada
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 · Iterator i Collections: recórrer i manipular
    - 📬 La idea en una frase
    - 🍽️ Iterator: el camarer que pren nota
    - 🧰 Collections: l'aliat utilitari
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · Classes genèriques: la teva `Caja`
    - 📬 La idea en una frase
    - 😱 L'infern abans dels genèrics
    - 🤝 Arriben els genèrics
    - 🔤 Les convencions: T, E, K, V, N
    - 📦 La classe Caja
    - ⭐ Be the Code, my friend...

    - 🤷 No hi ha preguntes tontes
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · El diamant, mètodes genèrics i wildcards
    - 📬 La idea en una frase
    - 💎 L'operador diamant <>
    - 🎩 Type erasure: el mag es porta els genèrics
    - 🧰 Mètodes genèrics
    - 🃏 Wildcards: el problema de la invariància
    - ⭐ Be the Code, my friend…
    - 🤷 No hi ha preguntes tontes
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · HashMap: la guia telefònica
    - 📬 La idea en una frase
    - ☎️ HashMap: clau → valor
    - 🛟 getOrDefault: el salvavides dels null
    - 🚶 Recórrer un HashMap: tres camins
    - ⭐ Be the Code, my friend…
    - 🤷 No hi ha preguntes tontes
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · TreeMap i com triar estructura
    - 📬 La idea en una frase
    - 🌳 TreeMap: l'ordenat de sèrie
    - 🔗 LinkedHashMap: el que recorda l'ordre d'arribada
    - ⚖️ La família al complet
    - 🥊 EL RING: HashMap vs TreeMap
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ⚖️ Map vs List vs Set: la comparativa definitiva
    - 🧭 L'arbre de decisions
    - 🏅 La regla pràctica

    - ✅ Resum en 3 frases (ampliat)
- 09 · Repàs interactiu: col·leccions, genèrics i mapes
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: ArrayList vs LinkedList
    - 🔥 Fireside Chat: el Genèric vs el Tipus Cru
    - 🕵️ Qui Soc?
    - 😡 CONRAD VS EL MÓN: “L’aparcament que no aparca”
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: la llista que plora
    - ⚡ Laboratori de Tortura: la mitjana que plora
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d’Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U13 — Programació funcional (Streams i Lambdas)
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d’avaluació coberts (RA6)
    - ✅ Criteris d’avaluació coberts (RA7)
    - 🚪 Per on començo?
- 01 · Lambdes: funcions que viatgen soles
    - 📬 La idea en una frase
    - 😱 La forma antiga: classes anònimes
    - 🏹 Arriba la fletxa: la lambda
    - 📦 Funcions que viatgen en variables
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier

    - 📬 La idea en una frase
    - 🎭 Què és una interfície funcional?
    - 🎭 Els quatre vestits oficials
    - 📋 La xurrera de les quatre
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · Referències a mètodes: la drecera `::`
    - 📬 La idea en una frase
    - 🩹 Quan la lambda és només una crida
    - 🗂️ Els quatre tipus de referència
    - 📋 La taula del ::
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 · Streams: el flux de dades
    - 📬 La idea en una frase
    - 📦 Què és un Stream (i què NO és)?
    - 🚚 Crear un Stream: les quatre fonts
    - 🏭 El pipeline: font → intermèdies → terminal
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · Operacions intermèdies: filter, map, distinct, sorted, limit
    - 📬 La idea en una frase
    - 🌀 Les cinc estacions
    - 🏭 Les estacions en cadena
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · Operacions terminals: collect, reduce, count, forEach
    - 📬 La idea en una frase
    - 🚰 Les quatre terminals

    - 🏭 El pipeline complet
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Del stream a la col·lecció: toList, Collectors, groupingBy
    - 📬 La idea en una frase
    - 📥 toList(): la sortida ràpida (Java 16+)
    - 🗂️ Els Collectors de tota la vida
    - 🗺️ groupingBy: agrupar com en una taula
    - 🗺️ toMap: d'elements a entrades
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · Optional: el que accepta l'absència
    - 📬 La idea en una frase
    - 🎁 La capsa en acció
    - 🎁 Obrir la capsa amb elegància
    - 🎁 Optional i els streams: parella de ball
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Repàs interactiu: domina la programació funcional
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend...
    - 🔥 Fireside Chat: el Bucle For vs el Stream
    - 🕵️ Qui Soc?
    - 🤬 CONRAD VS EL MÓN: "El que oblida l'aixeta"
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: l'estadística que no arranca
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball

    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U14 — Fitxers i expressions regulars (Regex)
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA5)
    - ✅ Criteris d'avaluació coberts (RA6)
    - 🚪 Per on començo?
- 01 · La classe File: el teu explorador de fitxers
    - 📬 La idea en una frase
    - 🗂️ File: el GPS dels teus fitxers
    - 📁 Recórrer una carpeta amb listFiles
    - 📝 La ruta no és el contingut
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader
    - 📬 La idea en una frase
    - ✍️ Escriure amb FileWriter
    - 📖 Llegir amb FileReader + BufferedReader
    - 🚨 I si el fitxer no existeix en llegir?
    - 🧱 I per què BufferedReader i no FileReader a seques?
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · try-with-resources: tancar sense plorar
    - 📬 La idea en una frase
    - 🔒 El try-with-resources en acció
    - 🧹 Diversos recursos alhora
    - 🤔 Quins recursos valen?
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig

    - ✅ Resum en 3 frases
- 04 · PrintWriter i Scanner + File
    - 📬 La idea en una frase
    - 🖨️ PrintWriter: System.out, però en un fitxer
    - 🦑 Scanner + File: llegir fitxers a trossos
    - 🪓 useDelimiter: tallar on tu digues
    - 🆚 Scanner vs BufferedReader: quin usi?
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · NIO: Files i Paths (la forma moderna)
    - 📬 La idea en una frase
    - 🗃️ Path i Paths: el nou File
    - ⚡ Files: la navalla suïssa
    - 📋 Altres joies de Files
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · Serialització: guardar objectes amb ObjectOutputStream
    - 📬 La idea en una frase
    - 🧳 El tracte: implementar Serializable
    - 📦 Guardar: ObjectOutputStream
    - 🧊 Recuperar: ObjectInputStream
    - 🧨 I si la classe canvia entre guardar i llegir?
    - 📚 Guardar col·leccions senceres
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Expressions regulars: Pattern i Matcher
    - 📬 La idea en una frase
    - ⚠️ L'infern de les contrabarres (llegeix-ho dues vegades)
    - 🔍 Pattern i Matcher: el motlle i el text
    - 🧩 La taula de símbols (la teva xulleta d'examen)

    - 🕵️ Grups de captura: els parèntesis que recorden
    - 🚩 Les banderes (flags)
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · Regex en acció: matches, replaceAll, split i validacions
    - 📬 La idea en una frase
    - 🎯 String.matches(): ¿complica TOT el string?
    - 🧹 replaceAll(): netejar text en una línia
    - 🔪 split(): trocejar per patró
    - 🎫 Validacions de la vida real
    - 🕵️ El processador de logs (grups de captura + fitxers)
    - ⭐ Be the Code, my friend...
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Repàs interactiu: domina fitxers i regex
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend...
    - 🔥 Fireside Chat: File vs Path
    - 🕵️ Qui Soc?
    - 😡 CONRAD VS EL MÓN: “No vaig tancar el fitxer”
    - 🎮 El Joc de les Decisions
    - ⚡ Laboratori de Tortura: el validador que no valida
    - 🏆 Assoliments d’aquesta unitat
    - 🧠 Atreveix-te a Pensar
    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d’Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U15 — Persistència de dades: JDBC i introducció a ORM
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat

    - ✅ Criteris d'avaluació coberts (RA9)
    - 🚪 Per on començo?
- 01 · Què és JDBC
    - 📬 La idea en una frase
    - 🔌 JDBC: l'USB de les bases de dades
    - 🗄️ SQLite: la base de dades d'entrenament
    - 📦 La dependència Maven
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · Connexió a SQLite: els 5 passos i Connection
    - 📬 La idea en una frase
    - 🪜 Els 5 passos (i el bonus no opcional)
    - 🔗 Connection: la canonada
    - 🧼 La connexió completa amb try-with-resources
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · Statement i ResultSet: consultar dades
    - 📬 La idea en una frase
    - 📤 Statement: el missatger
    - 📊 ResultSet: la taula virtual
    - ⚖️ executeQuery vs executeUpdate
    - 🧼 El programa complet
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 · CRUD complet: inserir, llegir, actualitzar i esborrar
    - 📬 La idea en una frase
    - 🧱 La taula
    - 🧍 El POJO
    - ✍️ Create (INSERT)
    - 📖 Read (SELECT)
    - ✏️ Update (UPDATE)
    - 🗑️ Delete (DELETE)
    - 🎯 Mini-chequeig

    - ✅ Resum en 3 frases
- 05 · PreparedStatement i SQL injection
    - 📬 La idea en una frase
    - 💣 Concatenar SQL: la bomba
    - 💉 PreparedStatement al rescat
    - 🧪 Guia de placeholders
    - ⭐ Be the Code, my friend…
    - 🥊 El ring: Statement vs PreparedStatement
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 06 · El patró DAO
    - 📬 La idea en una frase
    - 🏛️ La idea: separar l'SQL del negoci
    - 📜 La interfície: el contracte
    - 🏭 La implementació: qui posa les mans a la pasta
    - 🤔 DAO vs Repository
    - ⭐ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 07 · Transaccions: commit, rollback i savepoints
    - 📬 La idea en una frase
    - 🎭 El problema: el limbe dels diners
    - 🔄 setAutoCommit(false): obrir el parèntesi
    - 📍 Savepoints: no desfer-ho tot
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 08 · Bones pràctiques: el decàleg del JDBC
    - 📬 La idea en una frase
    - 📜 El decàleg del JDBC
    - 🔮 L'enigma
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 09 · Introducció a ORM

    - 📬 La idea en una frase
    - 🔎 Què és un ORM, concretament?
    - ⚖️ JDBC vs ORM: la mateixa tasca, dos estils
    - 🧰 L'arsenal: JPA, Hibernate i companyia
    - 🐘 La cara oculta: allò que el ORM no t'arregla
    - 🧭 I per què ho veem ara?
    - 🎯 Mini-comprovació
    - ✅ Resum en 3 frases
- 10 · Repàs interactiu: domina el JDBC
    - 📬 La idea en una frase
    - ⭐ Be the Code, my friend…
    - 🔥 Fireside Chat: Statement vs PreparedStatement
    - 🕵️ Qui soc?
    - 😡 CONRAD VS EL MÓN: "El JDBC que no es tanca"
    - 🎮 El joc de les decisions
    - ⚡ Laboratori de tortura: el CRUD que sagna
    - 🏆 Assoliments d'aquesta unitat
    - 🧠 Atreveix-te a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntes d'entrevista de treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- U16 — Servir i consumir APIs amb Web
    - 🎯 Objectiu de la unitat
    - 🗺️ Mapa de la unitat
    - 📝 Butlletins de la unitat
    - ✅ Criteris d'avaluació coberts (RA5)
    - 🚪 Per on començo?
- 01 · El protocol HTTP en 30 segons
    - 📬 La idea en una frase
    - 💬 La conversa de la web
    - 🎤 Els verbs d'HTTP
    - 🚦 Els codis d'estat més comuns

    - 🧩 Anatomia d'una URL
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 02 · Servidor web mínim amb HttpServer
    - 📬 La idea en una frase
    - 🖥️ El servidor mínim
    - 🕶️ Be the Code, my friend…
    - 🚨 "Address already in use"
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 03 · Servint HTML
    - 📬 La idea en una frase
    - 📄 HTML des d'una cadena
    - 📂 HTML des d'un fitxer
    - 🕶️ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 04 · Paràmetres GET i formularis POST
    - 📬 La idea en una frase
    - 📥 GET: la query string
    - 📤 POST: el cos de la petició
    - 🧰 El parser casolà
    - 🕶️ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases
- 05 · Tornant JSON: com una API de veritat
    - 📬 La idea en una frase
    - 🧩 Què és JSON (en 10 segons)
    - 🚀 Servir JSON des del teu servidor
    - 📡 Consumir-lo amb fetch
    - 🕶️ Be the Code, my friend…
    - 🎯 Mini-chequeig
    - ✅ Resum en 3 frases

- 06 · Mini projecte: gestor de tasques (API REST)
  - 📬 La idea en una frase
  - 🗂️ Què és REST (en 20 segons)
  - 🧱 El model i el magatzem
  - 🔧 Els quatre endpoints
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 07 · Consumir APIs externes amb HttpClient
  - 📬 La idea en una frase
  - 📡 El trio sagrat del client
  - 🧩 Parsejar JSON amb Gson
  - 🌟 L'exemple complet: els repos de GitHub
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 08 · HttpClient avançat
  - 📬 La idea en una frase
  - 📤 POST: enviar dades a una API
  - 🛡️ Capçaleres, timeouts i gestió d'errors
  - ⚡ Crides asíncrones amb sendAsync
  - 🗑️ PUT, DELETE i altres mètodes
  - 🏗️ Frameworks moderns (una ullada)
  - 🎯 Mini-chequeig
  - ✅ Resum en 3 frases
- 09 · Repàs interactiu: domina la web
  - 📬 La idea en una frase
  - ⭐ Be the Code, my friend…
  - 🔥 Fireside Chat: GET vs POST
  - 🕵️ Qui Soc?
  - 😡 CONRAD VS EL MÓN: "El port que no arranca"
  - 🎮 El Joc de les Decisions
  - ⚡ Laboratori de Tortura: el servidor que no era
  - 🏆 Assoliments d'aquesta unitat
  - 🧠 Atreveix-te a Pensar

    - 🧩 Crucigrama de Bits
    - 💬 Preguntes d'Entrevista de Treball
    - 🤷 No hi ha preguntes tontes
    - 🎬 Post-Crèdits
- Butlletí U01 — Inicial
    - Exercici 1: "Enganya al teu robot literal" 🤖🥪
    - Exercici 2: "Caçadors de bugs humans" 🐛🎯
    - Exercici 3: "El teu protocol de pau" 🛡️💬
    - Exercici 4: El filtre de l'hospital vs. el gimnàs (Abstracció)
    - Exercici 5: El desglossador de promocions (Descomposició)
    - Exercici 6: El test de la recepta tapada (Metacognició)
    - Exercici 7: El Patró del Caixer i el pàrquing (Patrons)
    - Exercici 8: El meu manual per a aprendre a programar
- Butlletí U02 — Inicial
    - Exercici 1: Desordena això
    - Exercici 2: Què imprimeix?
    - Exercici 3: Caçador d'errors
    - Exercici 4: La teva fitxa personal
    - Exercici 5: Completa el programa
    - Exercici 6: Emparella conceptes
    - Exercici 7: El detectiu d'errors
    - Exercici 8: La teva biografia
- Butlletí U02 — Inicial Resolt
    - Exercici 1: Desordena això (Solució)
    - Exercici 2: Què imprimeix? (Solució)
    - Exercici 3: Caçador d'errors (Solució)
    - Exercici 4: La teva fitxa personal (Solució)
    - Exercici 5: Completa el programa (Solució)
    - Exercici 6: Emparella conceptes (Solució)
    - Exercici 7: El detectiu d'errors (Solució)
    - Exercici 8: La teva biografia (Solució)
- Butlletí U02 — Avançat
    - Exercici 1: Calculadora de propines

    - Exercici 2: Conversor dòlar-euro
    - Exercici 3: Què imprimeix? — el casting traïdor
    - Exercici 4: Interès compost (sense bucle)
    - Exercici 5: L'enigma del post-increment
    - 💡 Eines d'ampliació (Operador ternari, .equals(), Scanner i Math.random())
    - Exercici 6: El duel de daus (Extra)
    - Exercici 7: Supercalculadora amb lògica (Extra)
    - Exercici 8: Javadoc de campionat
- Butlletí U02 — Avançat Resolt
    - Exercici 1: Calculadora de propines (Solució)
    - Exercici 2: Conversor dòlar-euro (Solució)
    - Exercici 3: Què imprimeix? — el casting traïdor (Solució)
    - Exercici 4: Interès compost (sense bucle) (Solució)
    - Exercici 5: L'enigma del post-increment (Solució)
    - 💡 Eines d'ampliació (Operador ternari, .equals(), Scanner i Math.random())
    - Exercici 6: El duel de daus (Extra) (Solució)
    - Exercici 7: Supercalculadora amb lògica (Extra) (Solució)
    - Exercici 8: Javadoc de campionat (Solució)
- Butlletí U02 — Extras
    - Exercici 1: CodeWars — Square(n) Sum (Extra)
    - Exercici 2: CodeWars — Grasshopper - Summation (Extra)
    - Exercici 3: AceptaElReto — 119 Escudos del ejército romano (Extra)
- Butlletí U03 — Inicial
    - Exercici 1: Conversor de temperatures
    - Exercici 2: Què imprimeix? — increments
    - Exercici 3: Calculadora de descomptes
    - Exercici 4: El tipus perfecte
    - Exercici 5: El casting assassí
    - Exercici 6: Parell o senar?
    - Exercici 7: String en acció
    - Exercici 8: Salutació amb Scanner
    - Exercici 9: CodeWars — Will you make it?
    - Exercici 10: Què imprimeix? — printf amb conversions

    - Exercici 11: la nota amb dos decimals
    - Exercici 12: la propina amb dos decimals
- Butlletí U03 — Inicial Resolt
    - Exercici 1: Conversor de temperatures
    - Exercici 2: Què imprimeix? — increments
    - Exercici 3: Calculadora de descomptes
    - Exercici 4: El tipus perfecte
    - Exercici 5: El casting assassí
    - Exercici 6: Parell o senar?
    - Exercici 7: String en acció
    - Exercici 8: Salutació amb Scanner
    - Exercici 9: CodeWars — Will you make it?
    - Exercici 10: Què imprimeix? — printf amb conversions
    - Exercici 11: la nota amb dos decimals
    - Exercici 12: la propina amb dos decimals
- Butlletí U03 — Avançat
    - ⭐ Exercici 1: Calculadora de propines
    - ⭐ Exercici 2: Conversor dòlar-euro
    - ⭐⭐ Exercici 3: Què imprimeix? — el casting traïdor
    - ⭐⭐ Exercici 4: Interès compost (sense bucle)
    - ⭐⭐⭐ Exercici 5: L'enigma del post-increment
    - ⭐⭐ Exercici 6: El duel de daus
    - ⭐⭐ Exercici 7: Supercalculadora amb lògica
    - ⭐⭐⭐ Exercici 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'
    - ⭐⭐⭐ Exercici 9: AceptaElReto — 114 Últim dígit del factorial
    - ⭐⭐ Exercici 10: el tiquet de compra amb NumberFormat
    - ⭐⭐⭐ Exercici 11: l'edat a prova de bombes
    - 📚 Referències
- Butlletí U03 — Avançat Resolt
    - ⭐ Exercici 1: Calculadora de propines
    - ⭐ Exercici 2: Conversor dòlar-euro
    - ⭐⭐ Exercici 3: Què imprimeix? — el casting traïdor
    - ⭐⭐ Exercici 4: Interès compost (sense bucle)

    - ⭐⭐⭐ Exercici 5: L'enigma del post-increment
    - ⭐⭐ Exercici 6: El duel de daus
    - ⭐⭐ Exercici 7: Supercalculadora amb lògica
    - ⭐⭐⭐ Exercici 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'
    - ⭐⭐⭐ Exercici 9: AceptaElReto — 114 Últim dígit del factorial
    - ⭐⭐ Exercici 10: el tiquet de compra amb NumberFormat
    - ⭐⭐⭐ Exercici 11: l'edat a prova de bombes
    - 📚 Referències
- Butlletí U03 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U04 — Inicial
    - Exercici 1: El porter del club
    - Exercici 2: Què imprimeix? — el semàfor invertit
    - Exercici 3: El menú del dia
    - Exercici 4: El compte enrere del coet
    - Exercici 5: La taula del 7
    - Exercici 6: Només els parells
    - Exercici 7: El detectiu de divisors
    - Exercici 8: L'edat blindada
    - Exercici 9: CodeWars — Even or Odd
- Butlletí U04 — Inicial Resolt
    - Exercici 1: El porter del club
    - Exercici 2: Què imprimeix? — el semàfor invertit
    - Exercici 3: El menú del dia
    - Exercici 4: El compte enrere del coet
    - Exercici 5: La taula del 7
    - Exercici 6: Només els parells
    - Exercici 7: El detectiu de divisors
    - Exercici 8: L'edat blindada
    - Exercici 9: CodeWars — Even or Odd
- Butlletí U04 — Avançat
    - ⭐ Exercici 1: La calculadora de notes

    - ⭐ Exercici 2: El menú que no es rendeix
    - ⭐⭐ Exercici 3: Què imprimeix? — la piràmide
    - ⭐⭐ Exercici 4: L'escala de nombres
    - ⭐⭐⭐ Exercici 5: Què imprimeix? — break, continue i l'etiqueta
    - ⭐⭐ Exercici 6: Caçador de primers
    - ⭐⭐ Exercici 7: La suma sentinella
    - ⭐⭐⭐ Exercici 8: CodeWars — Categorize New Member
    - ⭐⭐⭐ Exercici 9: AceptaElReto — 156 Ascensor
    - 📚 Referències
- Butlletí U04 — Avançat Resolt
    - ⭐ Exercici 1: La calculadora de notes
    - ⭐ Exercici 2: El menú que no es rendeix
    - ⭐⭐ Exercici 3: Què imprimeix? — la piràmide
    - ⭐⭐ Exercici 4: L'escala de nombres
    - ⭐⭐⭐ Exercici 5: Què imprimeix? — break, continue i l'etiqueta
    - ⭐⭐ Exercici 6: Caçador de primers
    - ⭐⭐ Exercici 7: La suma sentinella
    - ⭐⭐⭐ Exercici 8: CodeWars — Categorize New Member
    - ⭐⭐⭐ Exercici 9: AceptaElReto — 156 Ascensor
    - 📚 Referències
- Butlletí U04 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U05 — Inicial
    - Exercici 1: La salutació oficial
    - Exercici 2: Què passa? — la variable fantasma
    - Exercici 3: La fitxa de presentació
    - Exercici 4: El primer return
    - Exercici 5: Doble, si us plau
    - Exercici 6: Què imprimeix? — el viatge d'anada i tornada
    - Exercici 7: Parell o senar?
    - Exercici 8: La porta de l'edat
    - Exercici 9: CodeWars — Century From Year

- Butlletí U05 — Inicial Resolt
  - Exercici 1: La salutació oficial
  - Exercici 2: Què passa? — la variable fantasma
  - Exercici 3: La fitxa de presentació
  - Exercici 4: El primer return
  - Exercici 5: Doble, si us plau
  - Exercici 6: Què imprimeix? — el viatge d'anada i tornada
  - Exercici 7: Parell o senar?
  - Exercici 8: La porta de l'edat
  - Exercici 9: CodeWars — Century From Year
- Butlletí U05 — Avançat
  - ⭐ Exercici 1: El classificador de notes
  - ⭐ Exercici 2: Taules a demanda
  - ⭐⭐ Exercici 3: Què imprimeix? — l'ombra del paràmetre
  - ⭐⭐ Exercici 4: La suma de xifres
  - ⭐⭐⭐ Exercici 5: Què imprimeix? — el return que talla
  - ⭐⭐⭐ Exercici 6: Divideix l'informe
  - ⭐⭐ Exercici 7: El comptador de vocals
  - ⭐⭐⭐ Exercici 8: CodeWars — 'Disemvowel' Trolls
  - ⭐⭐⭐ Exercici 9: AceptaElReto — 165 Número hyperpar
  - 📚 Referències
- Butlletí U05 — Avançat Resolt
  - ⭐ Exercici 1: El classificador de notes
  - ⭐ Exercici 2: Taules a demanda
  - ⭐⭐ Exercici 3: Què imprimeix? — l'ombra del paràmetre
  - ⭐⭐ Exercici 4: La suma de xifres
  - ⭐⭐⭐ Exercici 5: Què imprimeix? — el return que talla
  - ⭐⭐⭐ Exercici 6: Divideix l'informe
  - ⭐⭐ Exercici 7: El comptador de vocals
  - ⭐⭐⭐ Exercici 8: CodeWars — 'Disemvowel' Trolls
  - ⭐⭐⭐ Exercici 9: AceptaElReto — 165 Número hyperpar
  - 📚 Referències
- Butlletí U05 — Extres

    - CodeWars
    - AceptaElReto
- Butlletí U06 — Inicial
    - Exercici 1: Què imprimeix? — Array de booleans
    - Exercici 2: Troba l'error — NullPointerException
    - Exercici 3: Completa el codi — for bàsic per a buscar el major
    - Exercici 4: Escriu aquest programa — comptar números parells
    - Exercici 5: Troba l'error — length vs length()
    - Exercici 6: Què imprimeix? — la suma dels senars
    - Exercici 7: Escriu aquest programa — cerca lineal
    - Exercici 8: Escriu aquest programa — l'invers
    - Exercici 9: Escriu aquest programa — la classe Arrays en acció
- Butlletí U06 — Inicial Resolt
    - Exercici 1: Què imprimeix? — Array de booleans
    - Exercici 2: Troba l'error — NullPointerException
    - Exercici 3: Completa el codi — for bàsic per a buscar el major
    - Exercici 4: Escriu aquest programa — comptar números parells
    - Exercici 5: Troba l'error — length vs length()
    - Exercici 6: Què imprimeix? — la suma dels senars
    - Exercici 7: Escriu aquest programa — cerca lineal
    - Exercici 8: Escriu aquest programa — l'invers
    - Exercici 9: Escriu aquest programa — la classe Arrays en acció
- Butlletí U06 — Avançat
    - ⭐ Exercici 1: La fusió d'arrays ordenats
    - ⭐ Exercici 2: Rotació circular a la dreta
    - ⭐ Exercici 3: Suma de diagonals (matriu quadrada)
    - ⭐⭐ Exercici 4: Està ordenat?
    - ⭐⭐ Exercici 5: L'invers al lloc
    - ⭐⭐ Exercici 6: Què imprimeix? — el doble bucle que compta parelles
    - ⭐⭐⭐ Exercici 7: Estadístiques de classe
    - ⭐⭐⭐ Exercici 8: El gran repte — busca-mines simplificat
    - ⭐⭐⭐ Exercici 9: Compactar — els zeros al final
- Butlletí U06 — Avançat Resolt

    - ⭐ Exercici 1: La fusió d'arrays ordenats
    - ⭐ Exercici 2: Rotació circular a la dreta
    - ⭐ Exercici 3: Suma de diagonals (matriu quadrada)
    - ⭐⭐ Exercici 4: Està ordenat?
    - ⭐⭐ Exercici 5: L'invers al lloc
    - ⭐⭐ Exercici 6: Què imprimeix? — el doble bucle que compta parelles
    - ⭐⭐⭐ Exercici 7: Estadístiques de classe
    - ⭐⭐⭐ Exercici 8: El gran repte — busca-mines simplificat
    - ⭐⭐⭐ Exercici 9: Compactar — els zeros al final
- Butlletí U06 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U07 — Inicial
    - Exercici 1: Què imprimeix? — la cerca de la gema
    - Exercici 2: El buscador de tresors
    - Exercici 3: Què imprimeix? — la bombolla mínima
    - Exercici 4: La capsa de sabates ordenada
    - Exercici 5: El detectiu de la cerca binària
    - Exercici 6: Què imprimeix? — la binària que falla
    - Exercici 7: El comptador d'intercanvis
    - Exercici 8: La nota més alta de la classe
    - Exercici 9: CodeWars — Find the smallest integer in the array
- Butlletí U07 — Inicial Resolt
    - Exercici 1: Què imprimeix? — la cerca de la gema
    - Exercici 2: El buscador de tresors
    - Exercici 3: Què imprimeix? — la bombolla mínima
    - Exercici 4: La capsa de sabates ordenada
    - Exercici 5: El detectiu de la cerca binària
    - Exercici 6: Què imprimeix? — la binària que falla
    - Exercici 7: El comptador d'intercanvis
    - Exercici 8: La nota més alta de la classe
    - Exercici 9: CodeWars — Find the smallest integer in the array
- Butlletí U07 — Avançat

    - ⭐ Exercici 1: Què imprimeix? — la binària amb historial
    - ⭐ Exercici 2: El buscador binari amb historial
    - ⭐⭐ Exercici 3: La bombolla amb recompte (i flag)
    - ⭐⭐ Exercici 4: La inserció descendent
    - ⭐⭐ Exercici 5: L'analista de complexitat
    - ⭐⭐ Exercici 6: El caçador de parelles
    - ⭐⭐ Exercici 7: El detectiu d'inversions
    - ⭐⭐⭐ Exercici 8: CodeWars — Ones and Zeros
    - ⭐⭐⭐ Exercici 9: AceptaElReto — 100 Constant de Kaprekar
    - 📚 Referències
- Butlletí U07 — Avançat Resolt
    - Exercici 1: Què imprimeix? — la binària amb historial
    - Exercici 2: El buscador binari amb historial
    - Exercici 3: La bombolla amb recompte (i flag)
    - Exercici 4: La inserció descendent
    - Exercici 5: L'analista de complexitat
    - Exercici 6: El caçador de parelles
    - Exercici 7: El detectiu d'inversions
    - Exercici 8: CodeWars — Ones and Zeros
    - Exercici 9: AceptaElReto — 100 Constant de Kaprekar
- Butlletí U07 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U08 — Inicial
    - Exercici 1: El factorial del valent
    - Exercici 2: L'àvia suma l'array
    - Exercici 3: Què imprimeix? — la compte enrere recursiva
    - Exercici 4: La potència de l'aprenent
    - Exercici 5: El detectiu de dígits
    - Exercici 6: El mirall de lletres
    - Exercici 7: Què imprimeix? — l'enigma del stack
    - Exercici 8: El guardià dels palíndroms
    - Exercici 9: CodeWars — Factorial

- Butlletí U08 — Inicial (Resolt)
  - Exercici 1: El factorial del valent
  - Exercici 2: L'àvia suma l'array
  - Exercici 3: Què imprimeix? — la compte enrere recursiva
  - Exercici 4: La potència de l'aprenent
  - Exercici 5: El detectiu de dígits
  - Exercici 6: El mirall de lletres
  - Exercici 7: Què imprimeix? — l'enigma del stack
  - Exercici 8: El guardià dels palíndroms
  - Exercici 9: CodeWars — Factorial
- Butlletí U08 — Avançat
  - Exercici 1: ⭐ Fibonacci amb el comptador xafarder
  - Exercici 2: ⭐⭐ Què imprimeix? — l'arbre de crides
  - Exercici 3: ⭐⭐ El palíndrom rebel
  - Exercici 4: ⭐⭐ La potència exprés (divide i venceràs)
  - Exercici 5: ⭐⭐⭐ Quicksort amb mediana de tres
  - Exercici 6: ⭐⭐⭐ Mergesort amb el comptador de comparacions
  - Exercici 7: ⭐⭐⭐ Les torres de Hanói amb comptador
  - Exercici 8: ⭐⭐⭐ CodeWars — Sort Numbers
  - Exercici 9: ⭐⭐⭐ AceptaElReto — 104 Mòbils
- Butlletí U08 — Avançat (Resolt)
  - Exercici 1: ⭐ Fibonacci amb el comptador xafarder
  - Exercici 2: ⭐⭐ Què imprimeix? — l'arbre de crides
  - Exercici 3: ⭐⭐ El palíndrom rebel
  - Exercici 4: ⭐⭐ La potència exprés (divide i venceràs)
  - Exercici 5: ⭐⭐⭐ Quicksort amb mediana de tres
  - Exercici 6: ⭐⭐⭐ Mergesort amb el comptador de comparacions
  - Exercici 7: ⭐⭐⭐ Les torres de Hanói amb comptador
  - Exercici 8: ⭐⭐⭐ CodeWars — Sort Numbers
  - Exercici 9: ⭐⭐⭐ AceptaElReto — 104 Mòbils
- Butlletí U08 — Extres
  - 🥋 CodeWars
  - 🏆 AceptaElReto

- Butlletí U09 — Inicial
  - Exercici 1: El gos que parla
  - Exercici 2: El telèfon amb bateria
  - Exercici 3: L'estudiant amb nota
  - Exercici 4: La pel·lícula en cartellera
  - Exercici 5: La persona educada
  - Exercici 6: El cercle calculador
  - Exercici 7: El videojoc jugable
  - Exercici 8: Què imprimeix? — el taller de cotxes
  - Exercici 9: CodeWars — Object Oriented Piracy
- Butlletí U09 — Inicial Resolt
  - Exercici 1: El gos que parla
  - Exercici 2: El telèfon amb bateria
  - Exercici 3: L'estudiant amb nota
  - Exercici 4: La pel·lícula en cartellera
  - Exercici 5: La persona educada
  - Exercici 6: El cercle calculador
  - Exercici 7: El videojoc jugable
  - Exercici 8: Què imprimeix? — el taller de cotxes
  - Exercici 9: CodeWars — Object Oriented Piracy
- Butlletí U09 — Avançat
  - ⭐ Exercici 1: La biblioteca
  - ⭐ Exercici 2: El rectangle que raona
  - ⭐ Exercici 3: El compte bancari blindat
  - ⭐⭐ Exercici 4: L'hora que es corregeix sola
  - ⭐⭐ Exercici 5: Què imprimeix? — el ball de referències
  - ⭐⭐ Exercici 6: El correu que s'encadena
  - ⭐⭐ Exercici 7: La fracció que es simplifica
  - ⭐⭐⭐ Exercici 8: CodeWars — Building blocks
  - ⭐⭐⭐ Exercici 9: CodeWars — Object Oriented Piracy
  - 📚 Referències
- Butlletí U09 — Avançat Resolt
  - ⭐ Exercici 1: La biblioteca

    - ⭐ Exercici 2: El rectangle que raona
    - ⭐ Exercici 3: El compte bancari blindat
    - ⭐⭐ Exercici 4: L'hora que es corregeix sola
    - ⭐⭐ Exercici 5: Què imprimeix? — el ball de referències
    - ⭐⭐ Exercici 6: El correu que s'encadena
    - ⭐⭐ Exercici 7: La fracció que es simplifica
    - ⭐⭐⭐ Exercici 8: CodeWars — Building blocks
    - ⭐⭐⭐ Exercici 9: CodeWars — Object Oriented Piracy
- Butlletí U09 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U10 — Inicial
    - Exercici 1: La casa de cristall
    - Exercici 2: El cotxe del veïnat
    - Exercici 3: El termòmetre amb cervell
    - Exercici 4: Getter sense setter
    - Exercici 5: El comptador de la classe
    - Exercici 6: La calculadora sense piles
    - Exercici 7: Les constants del barri
    - Exercici 8: Què imprimeix? — el trencaclosques estàtic
    - Exercici 9: CodeWars — Square(n) Sum
- Butlletí U10 — Inicial Resolt
    - Exercici 1: La casa de cristall
    - Exercici 2: El cotxe del veïnat
    - Exercici 3: El termòmetre amb cervell
    - Exercici 4: Getter sense setter
    - Exercici 5: El comptador de la classe
    - Exercici 6: La calculadora sense piles
    - Exercici 7: Les constants del barri
    - Exercici 8: Què imprimeix? — el trencaclosques estàtic
    - Exercici 9: CodeWars — Square(n) Sum
- Butlletí U10 — Avançat
    - ⭐ Exercici 1: Empleat amb validació

    - ⭐ Exercici 2: Cercle encapsulat
    - ⭐⭐ Exercici 3: JavaBean Alumne
    - ⭐⭐ Exercici 4: Hora immutable
    - ⭐⭐ Exercici 5: Què imprimeix? — el trencaclosques dels gats
    - ⭐⭐ Exercici 6: Comptador d'usuaris
    - ⭐⭐⭐ Exercici 7: La classe utilitària OperacionsArray
    - ⭐⭐⭐ Exercici 8: Validador de dades
    - ⭐⭐⭐ Exercici 9: El gran repte — refactoritza el banc
- Butlletí U10 — Avançat Resolt
    - ⭐ Exercici 1: Empleat amb validació
    - ⭐ Exercici 2: Cercle encapsulat
    - ⭐⭐ Exercici 3: JavaBean Alumne
    - ⭐⭐ Exercici 4: Hora immutable
    - ⭐⭐ Exercici 5: Què imprimeix? — el trencaclosques dels gats
    - ⭐⭐ Exercici 6: Comptador d'usuaris
    - ⭐⭐⭐ Exercici 7: La classe utilitària OperacionsArray
    - ⭐⭐⭐ Exercici 8: Validador de dades
    - ⭐⭐⭐ Exercici 9: El gran repte — refactoritza el banc
- Butlletí U10 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U11 — Inicial
    - Exercici 1: Què imprimeix? — La família musical
    - Exercici 2: Troba l'error — extends mal usat
    - Exercici 3: Completa el codi — el gat que crida el seu pare
    - Exercici 4: Escriu aquest programa — l'herència de vehicles
    - Exercici 5: Què imprimeix? — Polimorfisme amb referències
    - Exercici 6: Escriu aquest programa — la granja polimòrfica
    - Exercici 7: Troba l'error — @Override que no ho és
    - Exercici 8: Escriu aquest programa — el gos ben heretat
    - Exercici 9: Què imprimeix? — la cadena de constructors
- Butlletí U11 — Inicial Resolt
    - Exercici 1: Què imprimeix? — La família musical

    - Exercici 2: Troba l'error — extends mal usat
    - Exercici 3: Completa el codi — el gat que crida el seu pare
    - Exercici 4: Escriu aquest programa — l'herència de vehicles
    - Exercici 5: Què imprimeix? — Polimorfisme amb referències
    - Exercici 6: Escriu aquest programa — la granja polimòrfica
    - Exercici 7: Troba l'error — @Override que no ho és
    - Exercici 8: Escriu aquest programa — el gos ben heretat
    - Exercici 9: Què imprimeix? — la cadena de constructors
- Butlletí U11 — Avançat
    - ⭐ Exercici 1: Interfície FiguraGeometrica
    - ⭐ Exercici 2: Jerarquia d'Empleats
    - ⭐⭐ Exercici 3: Sistema de pagaments amb interfície
    - ⭐⭐ Exercici 4: Interfícies múltiples: Volador i Nedador
    - ⭐⭐ Exercici 5: Downcasting segur
    - ⭐⭐ Exercici 6: Calculadora de figures amb classe abstracta
    - ⭐⭐⭐ Exercici 7: Sistema de notificacions polimòrfic
    - ⭐⭐⭐ Exercici 8: Template method — les begudes
    - ⭐⭐⭐ Exercici 9: El gran repte — vehicles amb combustible
- Butlletí U11 — Avançat Resolt
    - ⭐ Exercici 1: Interfície FiguraGeometrica
    - ⭐ Exercici 2: Jerarquia d'Empleats
    - ⭐⭐ Exercici 3: Sistema de pagaments amb interfície
    - ⭐⭐ Exercici 4: Interfícies múltiples: Volador i Nedador
    - ⭐⭐ Exercici 5: Downcasting segur
    - ⭐⭐ Exercici 6: Calculadora de figures amb classe abstracta
    - ⭐⭐⭐ Exercici 7: Sistema de notificacions polimòrfic
    - ⭐⭐⭐ Exercici 8: Template method — les begudes
    - ⭐⭐⭐ Exercici 9: El gran repte — vehicles amb combustible
- Butlletí U11 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U12 — Inicial
    - Exercici 1: Què imprimeix? — ArrayList remove per índex vs valor

    - Exercici 2: Troba l'error — size() vs length vs length()
    - Exercici 3: Completa el codi — for-each que suma una llista
    - Exercici 4: Escriu aquest programa — la llista de la compra
    - Exercici 5: Què imprimeix? — el HashSet que no deixa repetir
    - Exercici 6: Escriu aquest programa — sense duplicats
    - Exercici 7: Completa el codi — la teva primera classe genèrica
    - Exercici 8: Què imprimeix? — HashMap amb put repetit
    - Exercici 9: Escriu aquest programa — comptador de paraules amb HashMap
- Butlletí U12 — Inicial Resolt
    - Exercici 1: Què imprimeix? — ArrayList remove per índex vs valor
    - Exercici 2: Troba l'error — size() vs length vs length()
    - Exercici 3: Completa el codi — for-each que suma una llista
    - Exercici 4: Escriu aquest programa — la llista de la compra
    - Exercici 5: Què imprimeix? — el HashSet que no deixa repetir
    - Exercici 6: Escriu aquest programa — sense duplicats
    - Exercici 7: Completa el codi — la teva primera classe genèrica
    - Exercici 8: Què imprimeix? — HashMap amb put repetit
    - Exercici 9: Escriu aquest programa — comptador de paraules amb HashMap
- Butlletí U12 — Avançat
    - ⭐ Exercici 1: La cua del supermercat amb LinkedList
    - ⭐⭐ Exercici 2: Intersecció i unió de conjunts
    - ⭐⭐ Exercici 3: Eliminar duplicats mantenint l'ordre
    - ⭐⭐⭐ Exercici 4: Què imprimeix? — el remove que trenca el ball
    - ⭐⭐ Exercici 5: Filtrar amb Iterator
    - ⭐⭐ Exercici 6: El TreeSet ordenat
    - ⭐ Pila genèrica : Exercici 7
    - ⭐⭐ Exercici 8: HashMap invers
    - ⭐⭐ Exercici 9: TreeMap — freqüència de lletres
    - ⭐⭐⭐ Exercici 10: Wildcards — suma i barreja de números
    - ⭐⭐⭐ Exercici 11: Sistema de votacions amb mètode genèric
    - ⭐⭐⭐ Exercici 12: el type erasure al descobert
- Butlletí U12 — Avançat Resolt
    - ⭐ Exercici 1: La cua del supermercat amb LinkedList

    - ⭐⭐ Exercici 2: Intersecció i unió de conjunts
    - ⭐⭐ Exercici 3: Eliminar duplicats mantenint l'ordre
    - ⭐⭐⭐ Exercici 4: Què imprimeix? — el remove que trenca el ball
    - ⭐⭐ Exercici 5: Filtrar amb Iterator
    - ⭐⭐ Exercici 6: El TreeSet ordenat
    - ⭐ Pila genèrica : Exercici 7
    - ⭐⭐ Exercici 8: HashMap invers
    - ⭐⭐ Exercici 9: TreeMap — freqüència de lletres
    - ⭐⭐⭐ Exercici 10: Wildcards — suma i barreja de números
    - ⭐⭐⭐ Exercici 11: Sistema de votacions amb mètode genèric
    - ⭐⭐⭐ Exercici 12: el type erasure al descobert
- Butlletí U12 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U13 — Inicial
    - Exercici 1: Completa el codi — la teva primera lambda
    - Exercici 2: Què imprimeix? — l'ordre de la fletxa
    - Exercici 3: Troba l'error — la lambda mal vestida
    - Exercici 4: Escriu aquest programa — filtrar parells amb streams
    - Exercici 5: Què imprimeix? — el pipeline bàsic
    - Exercici 6: Completa el codi — majúscules amb map
    - Exercici 7: Escriu aquest programa — longitud de cada paraula
    - Exercici 8: Troba l'error — la cinta que mai no arranca
    - Exercici 9: Completa el codi — un Consumer per a imprimir
- Butlletí U13 — Inicial Resolt
    - Exercici 1: Completa el codi — la teva primera lambda
    - Exercici 2: Què imprimeix? — l'ordre de la fletxa
    - Exercici 3: Troba l'error — la lambda mal vestida
    - Exercici 4: Escriu aquest programa — filtrar parells amb streams
    - Exercici 5: Què imprimeix? — el pipeline bàsic
    - Exercici 6: Completa el codi — majúscules amb map
    - Exercici 7: Escriu aquest programa — longitud de cada paraula
    - Exercici 8: Troba l'error — la cinta que mai no arranca

    - Exercici 9: Completa el codi — un Consumer per a imprimir
- Butlletí U13 — Avançat
    - ⭐⭐ Exercici 1: Ordenar amb referències a mètode
    - ⭐⭐ Exercici 2: Agrupar paraules per la seva primera lletra
    - ⭐⭐ Exercici 3: Optional — el que no es deixa enganyar
    - ⭐⭐⭐ Exercici 4: El pipeline complet
    - ⭐⭐ Exercici 5: De llista a mapa amb toMap
    - ⭐⭐⭐ Exercici 6: El màxim amb reduce i comparador
    - ⭐⭐ Exercici 7: Freqüències amb groupingBy
    - ⭐⭐⭐ Exercici 8: Optional i streams, la parella
    - ⭐⭐⭐ Exercici 9: el stream que es nega a morir
- Butlletí U13 — Avançat Resolt
    - ⭐⭐ Exercici 1: Ordenar amb referències a mètode
    - ⭐⭐ Exercici 2: Agrupar paraules per la seva primera lletra
    - ⭐⭐ Exercici 3: Optional — el que no es deixa enganyar
    - ⭐⭐⭐ Exercici 4: El pipeline complet
    - ⭐⭐ Exercici 5: De llista a mapa amb toMap
    - ⭐⭐⭐ Exercici 6: El màxim amb reduce i comparador
    - ⭐⭐ Exercici 7: Freqüències amb groupingBy
    - ⭐⭐⭐ Exercici 8: Optional i streams, la parella
    - ⭐⭐⭐ Exercici 9: el stream que es nega a morir
- Butlletí U13 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U14 — Inicial
    - Exercici 1: Troba l'error — IOException sense capturar
    - Exercici 2: Completa el codi — try-with-resources
    - Exercici 3: Escriu aquest programa — guardar ciutats en un fitxer
    - Exercici 4: Troba l'error — File.createNewFile sense comprovar
    - Exercici 5: Què imprimeix? — el comptador de línies
    - Exercici 6: Escriu aquest programa — el diari personal
    - Exercici 7: Què imprimeix? — matches() vs find()
    - Exercici 8: Escriu aquest programa — comptar paraules amb split

    - Exercici 9: Escriu aquest programa — comptar dígits amb Matcher
- Butlletí U14 — Inicial Resolt
    - Exercici 1: Troba l'error — IOException sense capturar
    - Exercici 2: Completa el codi — try-with-resources
    - Exercici 3: Escriu aquest programa — guardar ciutats en un fitxer
    - Exercici 4: Troba l'error — File.createNewFile sense comprovar
    - Exercici 5: Què imprimeix? — el comptador de línies
    - Exercici 6: Escriu aquest programa — el diari personal
    - Exercici 7: Què imprimeix? — matches() vs find()
    - Exercici 8: Escriu aquest programa — comptar paraules amb split
    - Exercici 9: Escriu aquest programa — comptar dígits amb Matcher
- Butlletí U14 — Avançat
    - ⭐ Exercici 1: Buscador de fitxers per extensió
    - ⭐ Exercici 2: Lector de CSV amb Scanner
    - ⭐⭐ Exercici 3: Filtri de línies per paraula clau
    - ⭐⭐ Exercici 4: Separador de línies parells i senars
    - ⭐⭐ Exercici 5: Split amb regex — analitzador de frases
    - ⭐⭐⭐ Exercici 6: Validador de dades amb regex
    - ⭐⭐⭐ Exercici 7: Xifrat Cèsar amb fitxers
    - ⭐⭐⭐ Exercici 8: Serialització d'estudiants
    - ⭐⭐ Exercici 9: El comptador de línies, paraules i caràcters
- Butlletí U14 — Avançat Resolt
    - ⭐ Exercici 1: Buscador de fitxers per extensió
    - ⭐ Exercici 2: Lector de CSV amb Scanner
    - ⭐⭐ Exercici 3: Filtri de línies per paraula clau
    - ⭐⭐ Exercici 4: Separador de línies parells i senars
    - ⭐⭐ Exercici 5: Split amb regex — analitzador de frases
    - ⭐⭐⭐ Exercici 6: Validador de dades amb regex
    - ⭐⭐⭐ Exercici 7: Xifrat Cèsar amb fitxers
    - ⭐⭐⭐ Exercici 8: Serialització d'estudiants
    - ⭐⭐ Exercici 9: El comptador de línies, paraules i caràcters
- Butlletí U14 — Extres
    - CodeWars

    - AceptaElReto
- Butlletí U15 — Inicial
    - Exercici 1: Què necessitos per a usar JDBC?
    - Exercici 2: Completa el codi — la connexió
    - Exercici 3: Què imprimeix? — ResultSet buit
    - Exercici 4: Troba l'error — SQLException sense gestionar
    - Exercici 5: Escriu aquest programa — la primera connexió
    - Exercici 6: Què imprimeix? — executeQuery en UPDATE
    - Exercici 7: Completa el codi — INSERT amb PreparedStatement
    - Exercici 8: Escriu aquest programa — llistar amb try-with-resources
    - Exercici 9: Troba l'error — índexs del PreparedStatement
- Butlletí U15 — Inicial Resolt
    - Exercici 1: Què necessitos per a usar JDBC?
    - Exercici 2: Completa el codi — la connexió
    - Exercici 3: Què imprimeix? — ResultSet buit
    - Exercici 4: Troba l'error — SQLException sense gestionar
    - Exercici 5: Escriu aquest programa — la primera connexió
    - Exercici 6: Què imprimeix? — executeQuery en UPDATE
    - Exercici 7: Completa el codi — INSERT amb PreparedStatement
    - Exercici 8: Escriu aquest programa — llistar amb try-with-resources
    - Exercici 9: Troba l'error — índexs del PreparedStatement
- Butlletí U15 — Avançat
    - ⭐ Exercici 1: Connexió des de fitxer de propietats
    - ⭐ Exercici 2: INSERT amb clau autogenerada
    - ⭐ Exercici 3: UPDATE condicional
    - ⭐⭐ Exercici 4: INNER JOIN amb PreparedStatement
    - ⭐⭐ Exercici 5: Cerca amb LIKE
    - ⭐⭐ Exercici 6: Dates en JDBC
    - ⭐⭐ Exercici 7: Batch INSERT — 100 alumnes de prova
    - ⭐⭐⭐ Exercici 8: El patró DAO
    - ⭐⭐⭐ Exercici 9: Transacció bancària atòmica
- Butlletí U15 — Avançat Resolt
    - ⭐ Exercici 1: Connexió des de fitxer de propietats

    - ⭐ Exercici 2: INSERT amb clau autogenerada
    - ⭐ Exercici 3: UPDATE condicional
    - ⭐⭐ Exercici 4: INNER JOIN amb PreparedStatement
    - ⭐⭐ Exercici 5: Cerca amb LIKE
    - ⭐⭐ Exercici 6: Dates en JDBC
    - ⭐⭐ Exercici 7: Batch INSERT — 100 alumnes de prova
    - ⭐⭐⭐ Exercici 8: El patró DAO
    - ⭐⭐⭐ Exercici 9: Transacció bancària atòmica
- Butlletí U15 — Extres
    - CodeWars
    - AceptaElReto
- Butlletí U15 — Inicial
    - Exercici 1: Escriu aquest programa — Hola, món web
    - Exercici 2: Escriu aquest programa — servidor de l'hora
    - Exercici 3: Escriu aquest programa — pàgina que diu el teu nom
    - Exercici 4: Escriu aquest programa — comptador de visites global
    - Exercici 5: Escriu aquest programa — generador d'excuses per a lliuraments tardans
    - Exercici 6: Escriu aquest programa — taula de multiplicar personalitzada
    - Exercici 7: Escriu aquest programa — convertidor d'euros a pessetes (sí, pessetes)
    - Exercici 8: Escriu aquest programa — pàgina d'estat del servidor
    - Exercici 9: Troba l'error — el Content-Type oblidat
- Butlletí U15 — Inicial Resolt
    - Exercici 1: Escriu aquest programa — Hola, món web
    - Exercici 2: Escriu aquest programa — servidor de l'hora
    - Exercici 3: Escriu aquest programa — pàgina que diu el teu nom
    - Exercici 4: Escriu aquest programa — comptador de visites global
    - Exercici 5: Escriu aquest programa — generador d'excuses per a lliuraments tardans
    - Exercici 6: Escriu aquest programa — taula de multiplicar personalitzada
    - Exercici 7: Escriu aquest programa — convertidor d'euros a pessetes
    - Exercici 8: Escriu aquest programa — pàgina d'estat del servidor
    - Exercici 9: Troba l'error — el Content-Type oblidat
- Butlletí U15 — Avançat
    - ⭐ Exercici 1: API de frases motivacionals

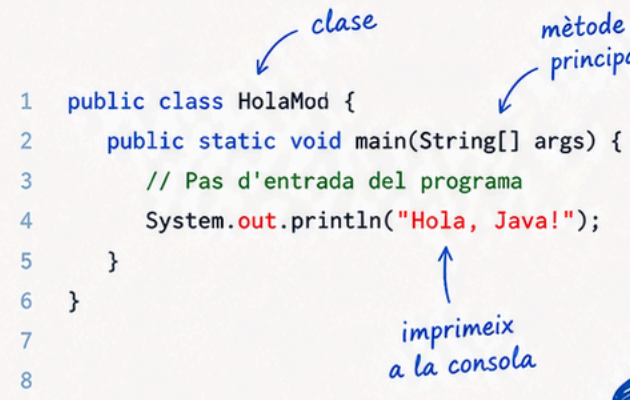
    - ⭐ Exercici 2: Formulari de contacte amb POST
    - ⭐ Exercici 3: Pedra, paper, tisora online
    - ⭐⭐ Exercici 4: El temps que NO fa
    - ⭐⭐ Exercici 5: Traductor xungo (però funcional)
    - ⭐⭐ Exercici 6: API REST de tasques amb prioritat
    - ⭐⭐ Exercici 7: Client GET — els repos de GitHub
    - ⭐⭐ Exercici 8: Client POST — crear una publicació a jsonplaceholder
    - ⭐⭐⭐ Exercici 9: Middleware de logging
    - ⭐⭐⭐ Exercici 10: la petició asíncrona amb sendAsync
- Butlletí U15 — Avançat Resolt
    - ⭐ Exercici 1: API de frases motivacionals
    - ⭐ Exercici 2: Formulari de contacte amb POST
    - ⭐ Exercici 3: Pedra, paper, tisora online
    - ⭐⭐ Exercici 4: El temps que NO fa
    - ⭐⭐ Exercici 5: Traductor xungo (però funcional)
    - ⭐⭐ Exercici 6: API REST de tasques amb prioritat
    - ⭐⭐ Exercici 7: Client GET — els repos de GitHub
    - ⭐⭐ Exercici 8: Client POST — crear una publicació a jsonplaceholder
    - ⭐⭐⭐ Exercici 9: Middleware de logging
    - ⭐⭐⭐ Exercici 10: la petició asíncrona amb sendAsync
- Butlletí U15 — Extres
    - CodeWars
    - AceptaElReto

Inici

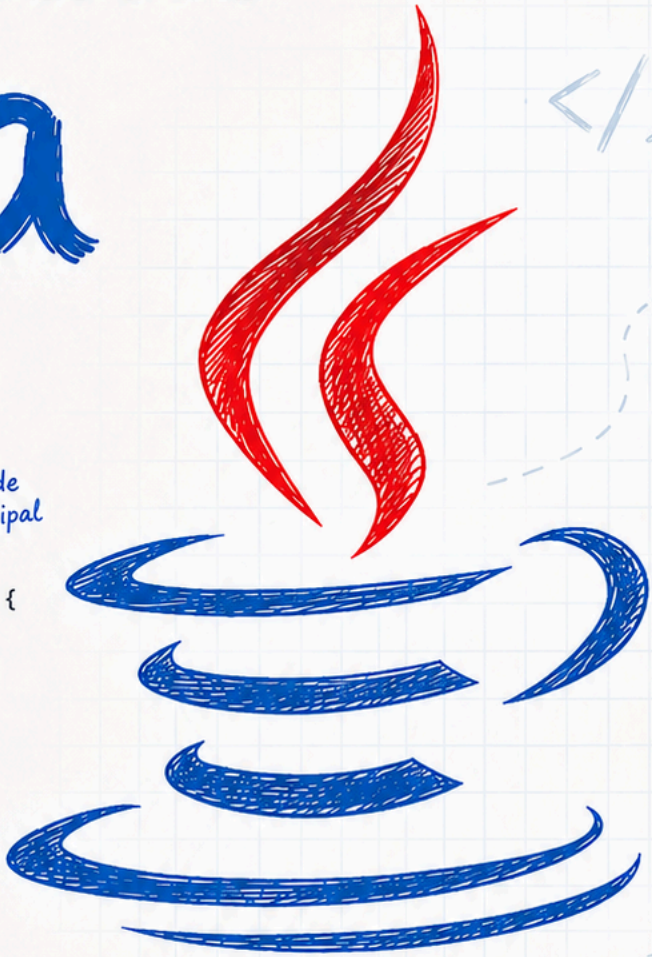
APUNTS

Programació Java

— DAM/DAW —




```java
public class HolaMod {
    public static void main(String[] args) {
        // Pas d'entrada del programa
        System.out.println("Hola, Java!");
    }
}
```



Pér
Sergi García Barea

llicència
CC BY SA

🚀 Començar per la Unitat 2

🧠 Primer: Introducció al pensament computacional

🐙 Veure al GitHub

Sergi Garcia Barea — CC BY-SA 4.0

⬇️ Prefereixes estudiar sense connexió? Descarrega el curs complet

📄 PDF (Castellà) 📄 PDF (Valencià) 📄 PDF (Català) 📱 EPUB (Castellà) 📱 EPUB (Valencià) 📱 EPUB (Català)

📚 Unitats

UNITAT 1 RA1

Introducció al pensament computacional

Abans de codi, pensem. Descomposició, debugging, protocol de pau i el teu primer problema de programació sense tocar Java.

👉 Veure unitat 👈

UNITAT 2 RA1

Introducció a Java i l'entorn de treball

Primer contacte amb Java: instal·la el JDK, escriu el teu primer programa, coneix el mètode `main`, els comentaris i els arguments de línia de ordres.

👉 Veure unitat 👈

UNITAT 3 RA2

Variables, tipus i operadors

Declara variables, usa tipus primitius, operadors aritmètics i lògics, conversions de tipus i llegeix dades per teclat amb `Scanner`.

👉 Veure unitat 👈

UNITAT 4 RA3

Estructures de control i excepcions bàsiques

Domina `if`/`else`, `switch`, bucles `while` i `for`, i gestiona excepcions amb `try`/`catch` perquè el teu programa no es trenqui.

👉 Veure unitat 👈

UNITAT 5 RA2

Funcions i mètodes

Pasa d'escriure codi solt a dissenyar-lo: mètodes amb paràmetres, `return` amb sentit i variables

UNITAT 6 RA6

Arrays

Els arrays són l'aparcament de dades: declara, recorre, ordena i manipula col·leccions de dades

amb el seu àmbit.

👉 Veure unitat 👈

amb arrays unidimensionals, multidimensionals i la classe `Arrays`.

👉 Veure unitat 👈

UNITAT 7 RA2, RA6

Algorítmica I: fonaments

Aprèn a pensar com un programador: divideix problemes en parts, usa pseudocodi, diagrames de flux i crea funcions reutilitzables.

👉 Veure unitat 👈

UNITAT 8 RA2, RA6

Algorítmica II: tècniques avançades i modularitat

Algoritmes d'ordenació, cerca binària, recursivitat i tècniques divideix i venceràs per a resoldre problemes més complexos.

👉 Veure unitat 👈

UNITAT 9 RA2, RA4

POO: classes i objectes

Programació Orientada a Objectes: crea classes, instància objectes, defineix atributs i mètodes, i entén la màgia dels constructors.

👉 Veure unitat 👈

UNITAT 10 RA4

Visibilitat, encapsulació i static

Controla qui veu què: modificadors d'accés (`public`, `private`, `protected`), encapsulació amb getters/setters, membres `static` i constants.

👉 Veure unitat 👈

UNITAT 11 RA4, RA7

Herència, polimorfisme i interfícies

Herència, polimorfisme, classes abstractes i interfícies: la base del disseny flexible i reutilitzable en Java. Aprèn a sobreescriure mètodes i a usar `super`.

👉 Veure unitat 👈

UNITAT 12 RA6

Col·leccions, genèrics i mapes

`ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `Iterator` i la classe `Collections`: el Java Collections Framework per a guardar i recórrer dades de manera elegant.

👉 Veure unitat 👈

UNITAT 13 RA6, RA7

Programació funcional (Streams i Lambdas)

Lambdes, interfícies funcionals, referències a mètodes, la Stream API (`filter`, `map`, `reduce`, `collect`) i `Optional` per a escriure Java modern i expressiu.

UNITAT 14 RA5

Fitxers i expressions regulars (Regex)

Lectura i escriptura de fitxers de text i binaris, serialització d'objectes i expressions regulars per a buscar, validar i reemplaçar patrons en text.

👉 Veure unitat 👈

👉 Veure unitat 👈

UNITAT 15

RA9

Persistència de dades: JDBC i introducció a ORM

Connecta Java amb bases de dades relacionals usant JDBC: `Connection`, `Statement`, consultes, insercions, actualitzacions i transaccions segures.

👉 Veure unitat 👈

UNITAT 16 (EXTRA)

RA5

Servir i consumir APIs amb Web

Crea un servidor HTTP amb Java `HttpServer`, serveix pàgines HTML/JS, gestiona peticions JSON, implementa una API REST completa des de zero.

👉 Veure unitat 👈

📝 Butlletins

UNITAT 1

📝 Butlletins de la unitat

🟢 Inicial per resoldre

UNITAT 2

📝 Butlletins de la unitat

🟢 Inicial per resoldre ✅ Inicial resolt

⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

UNITAT 3

📝 Butlletins de la unitat

🟢 Inicial per resoldre ✅ Inicial resolt

⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

UNITAT 4

📝 Butlletins de la unitat

🟢 Inicial per resoldre ✅ Inicial resolt

⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

UNITAT 5

📝 Butlletins de la unitat

🟢 Inicial per resoldre ✅ Inicial resolt

⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

UNITAT 6

📝 Butlletins de la unitat

🟢 Inicial per resoldre ✅ Inicial resolt

⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

UNITAT 7

UNITAT 8

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 9

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 10

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 11

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 12

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 13

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 14

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 15

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

UNITAT 16 (EXTRA)

📝 Butlletins de la unitat

🟢 Inicial per resoldre | ✅ Inicial resolt

⭐ Avançat per resoldre | 💪 Avançat resolt | 🔥 Extres

📄 Llicència

CC BY-SA 4.0 — *Sergi Garcia Barea*

Creative Commons Reconeixement-CompartirIgual 4.0 Internacional

Pots compartir i adaptar el material per a qualsevol propòsit, fins i tot comercial, sempre que en reconeguis l'autoria i compartesques les modificacions sota la mateixa llicència.

U01 — Introducció al pensament computacional

Abans de codi, pensem. La ment del programador/a abans de Java 🧠

Benvingut/da a la primera unitat del curs. Aquí no escriurem ni una sola línia de Java. Encara no.

Abans d'aprendre a programar, cal aprendre a **pensar com un programador/a**: descompondre problemes, entendre com "veu" un ordinador el codi, no tenir por al text vermell i saber usar la IA sense que ens substitueixi. Aquesta unitat és el mapa mental que necessites abans d'obrir l'editor de codi.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: cada punt desenvolupa una idea completa i enllaça amb el següent.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç d'explicar:

- Què és un ordinador i com "entén" el codi (binari, compiladors, intèrprets).
- Per què programar no és memoritzar codi sinó resoldre problemes.
- El flux sagrat del programador: pensar → paper → codi.
- La tècnica de la **descomposició** per a no col·lapsar davant problemes grans.
- A mirar els errors com un mapa, no com un fracàs.
- El **protocol de pau** abans de demanar ajuda.
- La regla del 80/20: menys llegir, més intentar.
- A usar la IA com a tutor socràtic, no com a còpia.
- Resoldre el teu primer problema de programació **sense tocar Java**.

🗺️ Mapa de la unitat 🔗

Punt	Què aprendràs	Nivel
01 · Què és un ordinador i un llenguatge de programació	Binari, compiladors vs intèrprets, JVM, terminologia clau	Todos
02 · Programar no és memoritzar codi	Miratge de la comprensió, pantalla en blanc, tutorial hell	Todos
03 · Primer pensem, després escrivim	Literalitat, els 4 passos sagrats, exercici del robot	Todos
04 · La descomposició	Divideix i venceràs, l'art de partir problemes	Todos
05 · Debugging i casos límit	El text vermell és el teu amic, edge cases	Todos
06 · El protocol de pau	4 passos abans de demanar ajuda al fòrum	Todos
07 · Practicar (menys mirar, més intentar)	Regla 80/20, hàbits diaris del programador	Todos
08 · Com utilitzarem la IA	Tutor socràtic, al·lucinacions, regles d'or	Todos
09 · El meu primer problema (pont cap a Java)	Comparador de nombres, pseudocodi, glossari	Todos

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9o és l'aterratge pràctic: lé-lo just després del 8o i abans d'obrir els boletins.

📝 Boletins de la unitat

> Entrena la ment de programador/a sense una sola línia de Java. Són exercicis no avaluables, però són la millor manera de trencar el gel.

🟢 Inicial per resoldre

✅ Criteris d'avaluació coberts (RA1)

RA1: Reconeix l'estructura d'un programa informàtic, identificant i relacionant els elements propis del llenguatge de programació utilitzat.

CE	Criteri	On es cobreix
a)	Blocs que componen l'estructura d'un programa	✅ Punts 3 i 4
b)	Projectes de desenvolupament d'aplicacions	✅ Punt 9

> 📌 El criteri a) es practica descomponen i escrivint el pla abans de programar; el b), tancant el primer problema complet del punt 9. La resta de criteris del RA1 (c: entorns integrats de desenvolupament, i: comentaris en el codi) es veuen en la **U02**.

🚪 Per on commence? 🔗

- 🆕 Zero coneixements? → Comença en el punt 1, no hi ha cap requisit previ.
- 💻 Ja saps què és un ordinador? → Ves directe al punt 3 i torna enrere si et perds.
- 🎯 Vens a repassar? → Fes les activitats.

📍 **Primer punt:** 01 · Què és un ordinador i un llenguatge de programació ⏭️ **En acabar la unitat, continua en U02 · Introducció a Java i l'entorn de treball.**

01 — Què és un ordinador i un llenguatge de programació

El primer pas: entendre amb qui anem a parlar 🖥️

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 01 · Què és un ordinador i un llenguatge de programació

Abans d'aprendre a pensar la lògica dels nostres programes, cal fer un pas enrere i entendre **on dimonis estem i amb qui anem a parlar**.

Quan la gent diu *"estic aprenent a programar"*, en realitat està aprenent a traduir idees humanes a un format que un conjunt de circuits electrònics pugui executar. Però com es produeix aquella màgia? Aquí tens els conceptes clau que necessites tenir clars abans de començar. 📌

🧠 1. Un ordinador és extraordinàriament ràpid, però absolutament ximple 🔗

Un ordinador no pensa, no té intuïció i no sap què vols fer. L'únic que entén la màquina a nivell de maquinari (hardware) és el codi màquina: una seqüència infinita de zeros i uns (0 i 1), conegut com a sistema binari.

- Els 0 i 1 representen, literalment, l'absència o el pas de corrent elèctric pels transistors del processador.
- Escriure un programa directament en zeros i uns és una tasca humana pràcticament impossible i dolorosa.

🗣️ 2. Què és un llenguatge de programació? 🔗

Un **llenguatge de programació** és el pont o la interfície entre el teu cervell i el processador. És un conjunt de regles, paraules clau i estructures creades per humans que ens permeten escriure instruccions de manera entenedora per a nosaltres, però que es poden traduir fàcilment a zeros i uns.

Hi ha dos grans tipus de llenguatges segons la seva proximitat a la màquina:

- **Llenguatges de baix nivell (ex: Assemblador):** Estan molt a prop del maquinari. Són ultra-ràpids, però extremadament difícils d'escriure i mantenir per a un humà.
- **Llenguatges d'alt nivell (ex: Java, Python, C#):** Utilitzen paraules en anglès (if, while, class), estructures lògiques complexes i símbols matemàtics. Són fàcils de llegir i escriure per a nosaltres, però la màquina no els entén directament.

🔄 3. El traductor: Compiladors vs. Intèrprets

Com que l'ordinador només entén binari i nosaltres escrivim en un llenguatge d'alt nivell, **necessitem un traductor**. Aquí és on apareix la gran diferència en com s'executa el codi:

[📝 Codi Font (Java)] ──► [⚙️ TRADUCTOR / Compilador] ──► [🤖 Codi Màquina (0s i 1s)]

1. **Compilador (El traductor de llibres):** Agafa TOT el teu fitxer de codi font, el revisa sencer, comprova que no hi hagi errors de sintaxi i genera un fitxer executable independent. Si hi ha un sol error, no tradueix res fins que ho corregeixes. *(Exemple: C, C++).*
2. **Intèrpret (El traductor en directe):** Llegeix el teu codi línia per línia i l'executa a l'acte. Si troba un error a la línia 50, executarà les primeres 49 línies i es pararà exactament en la 50. *(Exemple: Python, JavaScript).*

> ☕ **I Java?** Java utilitza un sistema híbrid molt especial. Primer es **compila** a un codi intermedi anomenat *Bytecode*, i després una "màquina virtual" (**JVM - Java Virtual Machine**) l'**interpreta** en qualsevol ordinador (Windows, Mac, Linux). Per això es diu que Java és *"Escriu-ho una vegada, executa-ho on vulguis"*.

🔤 4. Conceptes clau que escoltaràs cada dia

Per a no sonar com un estranger quan parlem del mòdul, guarda aquests 5 termes al teu diccionari personal:

- **Codi Font (Source Code):** El text pla que tu escrius en el llenguatge de programació (els teus fitxers .java).
- **Sintaxi:** Les regles gramaticals del llenguatge. Si en valencià no pots dir *"casa la vermella és"*, en Java no pots oblidar un punt i coma ; o una clau {}. Si et saltes la sintaxi, el compilador es queixarà en vermell.
- **IDE (Entorn de Desenvolupament Integrat):** El programa o la "fàbrica" que utilitzem per a escriure codi (com VS Code, Eclipse o NetBeans). És com un processador de textos (estil

Word), però dissenyat per a programar: remarca colors, avisa d'errors i té el botó d'executar integrat.

- **Algorisme:** La seqüència lògica i finita de passos que resol un problema (independentment del llenguatge que uses).
- **Execució / Run:** El moment en què li dius a l'ordinador *"Agafa el meu programa i fes el que posa aquí"*.

💡 Idea clau per a recordar

> **El llenguatge de programació és només l'idioma de la conversa.**

> Aprendre la sintaxi de Java és fàcil (és com aprendre vocabulari). El repte real d'aquest curs no és memoritzar el diccionari, sinó saber **què dimonis vols dir-li a la màquina** per a resoldre un problema. I d'això tracta la resta d'aquesta Unitat 1! 🚀

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · Programar no és memoritzar codi

02 — Programar no és memoritzar codi

El miratge de la comprensió i la trampa del copy-paste 🧠💻

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 02 · Programar no és memoritzar codi

---

Molt sovint, quan algú s'enfronta per primera vegada a una assignatura de programació, té la idea preconcebuda que aprendre a programar és com aprendre un idioma estranger a l'antiga: memoritzar una llista infinita de paraules estranyes, regles de sintaxi i fórmules màgiques en anglès. 🕵️✨

Res més lluny de la realitat. **Aprendre a programar no és un exercici de memòria; és un entrenament per a resoldre problemes.** El llenguatge de programació (en el nostre cas, Java) és simplement l'eina final que utilitzem per a comunicar la nostra solució a la màquina, de la mateixa manera que un fuster utilitza un martell o una serra per a construir una taula. 🔨🪑

🧟 La trampa del "Miratge de la Comprensió" 🔗

Un dels fenòmens més habituals en l'estudi a distància és el que en psicologia de l'aprenentatge s'anomena **la il·lusió de competència** o el **miratge de la comprensió**.

Segurament et sonarà aquesta pel·lícula:

1. Obris la plataforma virtual, llegeixes un exemple de codi explicat pas a pas o veus un vídeo on el professor resol un exercici. 🎬
2. Mentrestant ho veus, ho trobes tot lògic, fluid i clar. Dius per a tu mateix/a: *«És clar, quin geni! Té tot el sentit del món! Ho he entés a la primera»*. 😎
3. Tanques el vídeo, obris un fitxer buit per a fer un exercici similar i… **et quedes mirant el parpelleig del cursor en blanc durant 20 minuts.** 😳⏱️

Per què passa això? És un problema de capacitat? Que va, de cap de les maneres! Passa perquè el teu cervell utilitza dos circuits completament diferents segons la tasca:

- **El circuit del Reconeixement (Mode Passiu / "Sofà i Manta" 🛋️):** Quan llegeixes o veus un exercici resolt, el teu cervell només ha de verificar que la lògica que un altre ha creat té sentit. És una tasca molt lleugera que requereix quasi zero esforços mentals.

- **El circuit de la Generació (Mode Actiu / "Gimnàs Mental" 🏋️):** Quan t'enfrontes a la pantalla en blanc, el teu cervell ha de crear l'estructura des de zero, prendre decisions, recordar la lògica i traduir-la. Això requereix un esforç cognitiu intens.

> 🎹 **L'analogia:** Confondre "entendre un exemple de codi" amb "saber programar" és com creure que saps tocar el piano només perquè t'agrada molt la música clàssica i entens quan algú desafina. Escoltar no et fa pianista!

🏔️ La pantalla en blanc és el teu hàbitat natural 🔗

Quan t'assegues a estudiar i et trobes bloquejat/ada davant de la pantalla en blanc, la teva primera reacció pot ser de pànic: *«Mare meva, això no és per a mi, soc inútil, no se m'ocorre res!»* 😱

Cal que canvies el xip des del primer dia: **el bloqueig davant de la pantalla en blanc no és un senyal de fracàs, és l'estat natural de qualsevol programador.**

Programar consisteix, precisament, a transformar aquella incertesa inicial en un conjunt d'instruccions ordenades. La diferència entre una persona que està aprenent i un professional de Google no és que el professional sàpiga la solució immediatament, sinó que té un mètode de treball per a sortir d'aquell bloqueig sense tirar el portàtil per la finestra. 🪟💻❌

🕳️ Com evitar la farsa de "Copiar i Apegar" (Tutorial Hell) 🔗

A distància, quan ens quedem bloquejats, la temptació de la "via ràpida" és enorme: buscar la solució exacta a internet, copiar-la, apegar-la en el nostre projecte, veure que s'executa i dir: *«Ja ho tinc fet, soc un crack!»* 🤡

Això només genera una falsa sensació de progrés. Si no has passat pel procés mental de patir i pensar la solució, **no has après res**. La pròxima vegada que el problema canviï una miqueta, et tornaràs a quedar igual de bloquejat/ada.

Per a evitar aquesta trampa, aplicarem una norma molt simple durant tot el curs:

1. **La Regla de la Pantalla Tapada 🙈:** Si mires la solució d'un exercici o un exemple del temari, tanca la finestra o tapa el codi abans d'intentar escriure el teu.

2. **Res de miraculoses inspeccions visuals:** Si et penses que t'has encallat, deixa un poc de marge a la teva ment per a buscar la sortida, revisa només la línia concreta que et fa dubtar i torna a tapar la solució immediatament.

💡 Idea clau per a recordar

> **Veure o llegir com es resol un problema no significa saber resoldre'l.**

> L'aprenentatge real només ocorre quan el teu cervell fa l'esforç de construir la solució per ell mateix. Menys mirar, més intentar! 💪🎯

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Què és un ordinador i un llenguatge de programació · **Següent:** 03 · Primer pensem, després escrivim

03 — Primer pensem, després escrivim

El camí sagrat del programador i el test del Robot Literal 📝➡️💻

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 03 · Primer pensem, després escrivim

Hi ha un error clàssic de principiant que es repeteix des de la invenció dels ordinadors: obrir l'editor de codi, posar les mans sobre el teclat i començar a escriure línies sense tenir ni la més remota idea de com es resol el problema. 🤦‍♂️💥

El resultat d'això sol ser una mescla de frustració, errors de sintaxi en vermell per tota la pantalla i la sensació que la informàtica és una mena de màgia negra reservada per a uns pocs triats. 🕵️❌

Avui volem trencar aquest mite amb una regla d'or que t'estalviarà centenars d'hores de dolor durant el curs: **El codi és simplement la traducció d'una solució que JA has pensat prèviament en paper.** 📄✍️

🔮 La il·lusió de la literalitat (El teu ordinador no és endeví) 🔗

Els humans som unes màquines de "llegir entre línies". Si li dius a un amic *"Vaig a la cuina, vols aigua?"* i et respon *"Sí, si us plau"*, tu entens perfectament que has d'agafar un got, obrir l'aixeta o la botella, omplir el got i portar-li-lo. No et passa pel cap anar a la cuina, agafar un doll d'aigua amb les mans i tirar-li-lo per sobre a la cara, veritat? 🌊😳

Si li demanes això mateix a un ordinador:

- No sap què és un got 🥃.
- No sap quan ha de parar d'abocar aigua (inundació a la cuina 🌊).
- Es queda bloquejat per sempre esperant que li digues si l'aigua ha de ser del temps o freda 🧊.

> 🤖 **La crua realitat:** Un ordinador no té intuïció, no té sentit comú, no té empatia i **no interpreta el que vols dir: executa exactament les instruccions que li dones**. Si el teu programa fa coses rares, no és que la màquina et tingui mania; és que tu has assumit que l'ordinador "ja t'entenia". I no, no t'entén!

🛣️ El camí sagrat del programador

Per a no col·lapsar el teu cervell, el flux de treball per a resoldre qualsevol exercici ha de seguir SEMPRE aquesta seqüència de 4 passos:

[1. Problema 🧩] ──► [2. Pas a pas (Algorisme) 👣] ──► [3. Solució en paper 📄✍️] ──► [4. Codi 💻]

1. **Entendre el problema 🧩:** Llegir l'enunciat fins que siguis capaç d'explicar-li-ho a la teva àvia o al teu gat. Si no saps què et demanen, és impossible resoldre-ho!
2. **Pensar els passos (Algorisme) 👣 :** Dissenyar la seqüència lògica de passos que solucionen el problema. Quina dada va primer? Què he de calcular? Quina decisió he de prendre?
3. **Escriure la solució en paper 📄:** Usar el teu propi idioma (valencià/castellà) o un dibuix per a comprovar que la lògica té sentit.
4. **Traduir a Java 💻:** Obris l'ordinador. Ara només has de preocupar-te per la sintaxi (punts i comes ;, claus {}, etc.), perquè la lògica ja la tens resolta en el paper.

Si intentes fer el Pas 2 i el Pas 4 al mateix temps directament al teclat, la teva "RAM mental" es col·lapsarà i apareixerà el bloqueig. 🤯❌

🤖🧪 Exercici pràctic: El test del Robot Literal

Per a experimentar això en la teva pròpia pell sense tocar cap ordinador, anem a fer un experiment mental.

Imagina que has de donar instruccions a un **Robot Literal** (un ésser que no té sentit comú i ho fa TOT al peu de la lletra) per a una d'aquestes tasques quotidianes:

- ☕ **Fer un cafè en cafetera de degoteig.**
- 🎒 **Preparar la motxilla per a anar a classe.**
- 🍳 **Fer una truita de creïlles.**
- 🚗 **Canviar una roda picada d'un cotxe.**
- 🛒 **Fer una compra en una botiga en línia.**

Què passa quan intentem escriure les instruccions? ⚠️

Si escrius: *“Agafa el cafè i posa’l a la cafetera”*, el Robot Literal agafarà el paquet de cafè TANCAT i el tirarà damunt de la cafetera sense obrir-lo. 📦💥

Perquè el Robot ho faci bé, has de desglossar-ho així:

1. Agafa el paquet de cafè amb la mà dreta.
2. Amb la mà esquerra, obre la pestanya superior del paquet.
3. Agafa la cullera… *(i així 20 passos més)*.

😈 Els 5 dimonis de la lògica

Quan fem aquest exercici, apareixen immediatament els 5 problemes que després et trobaràs programant en Java:

1. **Passos que falten 🚫:** Obrir la botella d’aigua abans d’abocar-la.
2. **Ordre incorrecte 🔀:** Posar-te les sabates abans que els mitjons.
3. **Ambigüitats ❓:** En comptes de dir *“Gira a la dreta”*, dir *“Gira un poc”*. Quant és “un poc”? 45 graus? 180 graus?
4. **Informació implícita 🤫:** Donar per fet que el Robot sap que l’aigua crema.
5. **Decisions no contemplades 🔀:** Què fa el Robot si quan agafa el cafè… s’ha acabat el cafè? *(Si no li ho has dit, el Robot es queda penjat per sempre).*

💡 Idea clau per a recordar

> **El codi és només l’últim pas.**

> Si no saps resoldre un problema amb un paper i un bolígraf en el teu propi idioma, és absolutament impossible que el puguis resoldre en Java davant de la pantalla. Pensa primer, tecleja després! 🧠📄➡️⌨️

📚 Tornar a l’índex de la unitat · **Anterior:** 02 · Programar no és memoritzar codi · **Següent:** 04 · La descomposició

04 — La descomposició

Divideix i venceràs: l'art de partir un elefant 🧱⛏️

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 04 · La descomposició

Quan et trobes davant d'un exercici de programació que ocupa mig foli d'enunciat, és completament normal que la teva primera reacció sigui una mescla de suor freda, ganes de tancar el portàtil i ganes de demanar un canvi de cicle a un d'art Floral. 🌿🙈

El problema no és que l'exercici sigui impossible. El problema és que el teu cervell està intentant processar **TOTA la complexitat de colp**. Vol resoldre el principi, el mig, el final, la interfície, els errors i la base de dades en el mateix segon.

Veurem la tècnica secreta (i mil·lenària) que fan servir els programadors de tot el planeta per a no col·lapsar: **Divideix i venceràs.** ⚔️👑

🐘 Com menjar-se un elefant? (A mossegades!) 🔗

Hi ha una pregunta clàssica de reflexió: *«Com et menges un elefant de 4 tones?»*

La resposta és evident: *«Mossegada a mossegada, lentament i sense pressa».* 🥩

En programació és exactament el mateix: **un problema gran només és una col·lecció de problemes petits disfressats.** Si intentes resoldre'l tot de colp, et bloquejaràs. Si el desmuntes en peces microscòpiques, veuràs que cada peça individual és tan senzilla que podries resoldre-la quasi sense pensar.

🏛️ L'art de la descomposició en sub-blocs 🔗

Imagina que el teu primer dia de pràctiques et diuen:

💬 *«Necessitem que facis una aplicació sencera per a gestionar una Biblioteca Municipal».*

Si intentes començar a programar això directament, la pantalla en blanc es riurà de tu a la cara. Com apliquem la descomposició? Dividint el "monstre" en blocs independents:

[1. Gestió d'Usuaris 👤] 📖 ──► *[2. Catàleg de Llibres 📚]* 📍 ──► *[3. Sistema de Préstecs 🔄]* ⏱️ ──► *[4. Devolucions i Penalitzacions ⏰]* ──► *[5. Cerca i filtres 🔍]*

Falta menys! Ara, en lloc de pensar en “TOTA la biblioteca”, ens centrem **únicament** en el sub-bloc número 3: **El sistema de préstecs**.

Però espera, que encara el podem dividir més! Com és el procés d’un préstec pas a pas?

[Préstec d’un Llibre] 📖

```
|
├── 1. Identificar l'usuari (DNI) 🆔
├── 2. Identificar el llibre (Codi de barres) 🏷️
├── 3. Comprovar si l'usuari té multes pendents 🚫
├── 4. Comprovar si el llibre està disponible (no prestat) ❓
├── 5. Registrar la data de hui i la data límit 📅
└── 6. Canviar l'estat del llibre a "Prestat" ✅
```

MIRA AIXÒ! 🎉 La tasca gegant i aterridora de “Fer una Biblioteca” s’ha convertit en una llista d’accions tan simples com: *“Comprovar si un número és major que zero”* o *“Guardar un nom en un text”*.

Felicitats! Ja estàs dissenyant l’arquitectura d’un programa sense haver tocat una sola línia de Java. 👏🥳

⚠️ Les 3 regles d’or per a no morir en la divisió

1. **Mai no resolguis dos sub-problemes alhora** 🚫**:** Si estàs fabricant la part on l’usuari posa el seu DNI, NO penses encara en com s’imprimirà el tiquet en PDF. Una cosa rere l’altra!
2. **Si un sub-pas encara et sembla difícil, torna a dividir-lo** ✂️**:** Si el pas 3 (*Comprovar multes*) et sembla embolicat, divideix-lo en: 3a) Llegir multes de l’usuari, 3b) Sumar l’import total, 3c) Si és major que 0€, bloquejar el préstec.
3. **Celebra les petites victòries** 🥳**:** Veure que el pas 1 (*Llegir el DNI*) funciona correctament et dona un xut de dopamina i seguretat per a afrontar el pas 2.

💡 Idea clau per a recordar

> **Davant d’un problema gran, no et bloqueges: esmola la tisora.**

> Si una tasca et fa por, és simplement perquè no l’has dividit en peces prou petites. Fes els passos més i més petits fins que siguin ridículament fàcils de resoldre. ✂️🧩🚀

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Primer pensem, després escrivim · **Següent:** 05 · Debugging i casos límit

05 — Debugging i casos límit

El text vermell és el teu amic i el detectiu mai no prova només el bonic 🚨🕵️

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 05 · Debugging i casos límit

Arriba el moment inevitable de la vida de qualsevol persona que programa: fas clic en el botó verd d'executar el programa i, en lloc de funcionar com la seda, la pantalla s'il·lumina amb un text vermell fosc que sembla cridar-te: *«Has fracassat en la vida! Tanca el portàtil i fuig al camp a cuidar ovelles!»* 🐑🏃💨

Tranquil·litat! Cal desmuntar el drama des del minut 1: **un error en el teu codi no és una vergonya, ni un atac personal, ni un senyal que no vals per a la informàtica.**

En programació, els errors tenen un nom tàctic —**bugs** 🐛— i l'art de trobar-los i corregir-los es diu **debugging** (o desinsectació mental). Aprendrem a mirar els errors als ulls sense entrar en pànic. 👁️🛑

🔴 1. El canvi de xip: El text vermell és el teu amic 🔗

Per a un humà normal, una llum vermella gegant significa: *«PERILL! HO HAS FET MALAMENT!»*. Però per a un programador o programadora, **el text vermell és un assistent ultra-eficient que parla amb tu gratis.** 🤖💬

Pensa-ho així: la màquina podria simplement quedar-se en silenci, petar o dir-te un "No funciona" i deixar-te a les fosques. Però no! Et dona una **stack trace** (la famosa muntanya de lletres vermelles) que et diu dues coses d'or pur:

1. **Què ha passat?** 🧐 *(Ex. T'has oblidat de tancar un parèntesi, o has intentat dividir per zero).*
2. **En quina línia exacta ha passat?** 📍 (Ex. Al fitxer Main.java, a la línia 24).

> ❌ **El que veu el teu cervell en pànic:** *"FATAL ERROR SYSTEM CRASH DIE DIE DIE BLA BLA BLA"* 😱

> 🟢 **El que diu el missatge en realitat:** "Hola! A la línia 24 has escrit num2 en lloc de num1. De res, campió!" 👍

🧪 2. Pensar com un programador: Provar els casos límit 🔗

Un altre error de principiant és provar el programa NOMÉS amb les dades “boniques” i dir: *«Ja funciona!»*.

Imagina aquest programa súper senzill:

> *“Un programa que indica si una persona és major d’edat (edat mínima: 18 anys).”*

Si el proves posant un **20**, el programa diu “Major d’edat” i tu te’n vas a fer un cafè tan content/a ☕. Però un bon programador té una **ment de detectiu maliciós** i es pregunta: *«I com podria fer petar jo això?»* 😈

Cal provar SEMPRE els **casos límit (edge cases)**:

- 🎯 **El límit exacte (18 anys):** El teu programa diu que té 18 és major d’edat o s’ha quedat fora per un error de > en lloc de >=?
- 🎯 **El límit inferior (17 anys):** Diu correctament que és menor?
- 🎯 **El límit zero (0 anys):** Què fa el programa?
- 💣 **El cas “Gamberro” (-5 anys o 200 anys):** Té sentit que algú tingui -5 anys? O hauries de comprovar que l’edat sigui un número vàlid abans de fer res?

Provar els casos que ningú s’espera és la diferència entre un programa d’aficionat i un programa professional! 🛡️✨

💡 Idea clau per a recordar 🔗

> **Els errors no són el final del camí, són el mapa.**

> Un programa que falla no és un fracàs; és simplement un trencaclosques que t’està donant les pistes exactes per a ser resolt. Abraça el text vermell! 🔴🧩🔎

📚 Tornar a l’índex de la unitat · **Anterior:** 04 · La descomposició · **Següent:** 06 · El protocol de pau

06 — El protocol de pau

4 passos abans de demanar ajuda (i no semblar un vague) 🛡️💬

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 06 · El protocol de pau

Quan estudies a distància i un exercici no surt, la temptació és anar directament al fòrum o a la IA i escriure el clàssic: *«Professor, no em funciona, ajuda!»* 🆘❌

Per a no quedar-te bloquejat/ada i aprendre de veritat, abans d'enviar cap missatge, **has de seguir religiosament aquest protocol de 4 passos**:

🛡️ Els 4 passos del protocol 🔗

[1. Llegir l'error] 📖 ──► [2. Localitzar la línia] 📍 ──► [3. Hipòtesi dels 15 min] ⏱️ ──► [4. Preguntar amb dades] 💬

1. Llegeix l'error en veu alta 📖 🔗

No tantes els ulls. Llegeix la línia vermella i busca les paraules clau.

2. Ves a la línia exacta 📍 🔗

Obris el teu codi, ves a aquella línia i mira què dimonis hi ha escrit ahir.

3. La Regla dels 15 Minuts (La hipòtesi) ⏱️ 🔗

Intenta canviar UNA SOLA COSA per a provar si la teva idea d'on està la falla és correcta. Si fa 15 minuts que estàs encallat/ada en la mateixa línia... **ALÇA'T!** 🚶 Ves per un got d'aigua, parla amb el gat o mira per la finestra. La solució quasi sempre arriba quan la ment es relaxa.

4. Demana ajuda com un Pro 💬 🔗

Si després d'això no ho resols, escriu al fòrum, però **no diguis "no funciona"**. Digues:

- *"Estic intentant fer X..."*
- *"Em salta l'error Y a la línia Z..."*
- *"Ja he provat de canviar A i B, però continua fallant."*

💡 Idea clau per a recordar

> **Els errors no són el final del camí, són el mapa.**

> Un programa que falla no és un fracàs; és simplement un trencaclosques que t'està donant les pistes exactes per a ser resolt. Abraça el text vermell! 🔴🧩🔎

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Debugging i casos límit · **Següent:** 07 · Practicar (menys mirar, més intentar)

07 — Practicar (menys mirar, més intentar)

La regla 80/20 i els hàbits del programador 🏊‍♂️🏋️

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 07 · Practicar (menys mirar, més intentar)

Imagina que vols aprendre a nadar. Et compres el millor llibre de natació del món, et llegeixes 500 pàgines sobre la física de la flotabilitat, veus 40 hores de vídeos de la Selecció Olímpica en 4K i t'estudies de memòria l'angle exacte en què el braç ha d'entrar a l'aigua. 📖🏊‍♂️

Arriba el dia de la veritat: te'n vas a la piscina, et tires a la part profunda sense haver tocat l'aigua mai... i què passa? **Te'n vas directament al fons com un sac de creïlles!** 🌊⚓

Amb la programació passa exactament el mateix. Pots tenir els apunts més bonics de la classe, haver subratllat tot el temari amb retolador groc i tenir la col·lecció de videotutorials més gran d'Internet, però **si no teclegis i t'equivoques tu sol/a, no estàs aprenent a programar.** 🤦‍♂️💻

🎭 La gran farsa del "Ja ho he entés"

L'estudi a distància té un perill silenciós que cal destapar des del primer dia: **confondre la lectura passiva amb l'aprenentatge real.**

Segurament viuràs aquesta experiència en dos passos molt clars durant les pròximes setmanes:

🎬 Fase A: El mode espectador (*«Quina fada soc!»*)

Llegeixes un algorisme resolt als apunts o veus com el professor programa la solució d'un exercici en un vídeo. Tot sembla clar, estructurat i logicíssim. El teu cervell es relaxa, fa un xut de dopamina i dius: *«Buf, quina ganga de mòdul, ho entenc tot a la primera!»* 😎✨

🛑 Fase B: La prova maestra (*«Ufff... què ha passat?»*)

Tanques la finestra de la solució, te'n vas a un fitxer completament en blanc i intentes fer un exercici idèntic però canviant les dades. De sobte, el buit absolut. El cursor parpelleja (▌), no saps si començar per la dalt o pel baix, i la ment es queda en blanc. 😳⚡

> 💡 **La lliçó de vida:** Entendre la solució d'un altre i saber construir una solució tu mateix/a **són dues habilitats cerebrals completament diferents.** La primera és facilíssima; la segona és la que es paga a les empreses d'informàtica.

⏳🎯 La Regla del 80/20 per a l'FP a Distància 🔗

Per a no caure en la trampa d'estar hores i hores "llegint apunts" sense avançar de veritat, durant aquest curs aplicarem una regla de distribució del teu temps d'estudi molt clara:

- 📖 20% del temps: Llegir la teoria / Veure el concepte.
- 💻 80% del temps: Fer exercicis, errar, provar i picar.

Si dediques 10 hores a la setmana a aquest mòdul, només 2 hores haurien de ser per a llegir o veure explicacions. Les altres **8 hores han de ser de picar codi, barallar-te amb la pantalla en blanc i provar coses en el teu ordinador!** ⌨️💥

🏋️‍♂️ Com entrenar la "musculatura de programador" 🔗

La programació no funciona per memorització de dades (com aprendre's les capitals d'Europa), sinó per **memòria muscular i esquemes mentals** (com anar en bicicleta o jugar a un videojoc).

Per a entrenar aquesta musculatura sense frustrar-te, aplica aquests 3 hàbits diaris:

1. **La norma del "Fes-ho abans de mirar"** 🙈**:** Quan el temari propose un exercici d'exemple, NO llegeixis la solució directament. Intenta fer-ho tu en paper durant 5 minuts. Encara que sigui malament! L'esforç de pensar abans de veure la resposta multiplica per deu la retenció.
2. **El canvi d'un 5%** 🔀**:** Quan un exemple de la plataforma et funcioni, no el tanquis i passis a una altra cosa. Pregunta't: *«I si ara faig que en comptes de sumar, multipliqui?»* o *«I si li demano dues dades més a l'usuari?»*. Modificar el codi que funciona és la millor manera d'entendre com està fet per dins!
3. **No tinguis por de tancar el tutorial** ❌📺**:** Si per a fer un exercici necessites tenir el vídeo del professor o el PDF obert al costat mirant línia per línia el que fa, no estàs fent l'exercici: estàs fent de copista medieval. Tanca la mostra i força el teu cervell a recordar!

💡 Idea clau per a recordar

> **Programar no es mira, programar es fa.**

> Menys llegir sobre la piscina i més tirar-se a l'aigua! L'únic codi que et fa aprendre és el que escrius tu després d'haver-ho pensat. 💪🏊⌨️

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · El protocol de pau · **Següent:** 08 · Com utilitzarem la IA

08 — Com utilitzarem la IA

L'assistent, no el teu oracle 🤖🧠

> 🗺️ **Estás en:** 🧠 **U01 · Introducció al pensament computacional** → 08 · Com utilitzarem la IA

Siguem realistes des del primer segon: en un cicle a distància, eines com ChatGPT, Gemini o Claude estan literalment a un clic de distància a la teva pantalla. Dir-te que està «completament prohibit usar la intel·ligència artificial» seria com intentar posar-li portes al camp o prohibir l'ús de la calculadora a un matemàtic. 🚫🌾

Ara bé, hi ha una diferència ABISMAL entre **usar la IA per a multiplicar el teu aprenentatge** i **usar la IA per a atrofiar el teu cervell**. 🧠💥

Establirem les regles del joc perquè la IA sigui el teu millor tutor privat i no la causa que penses el dia de l'examen presencial. 🎓⚠️

🚨 El gran perill: Pagar a un altre perquè faci les teves flexions 🔗

Imagina que t'apuntes al gimnàs perquè vols posar-te en forma, però pagues a un petit robot perquè faci totes les flexions, abdominals i carreres per tu. Al final del mes, el robot està en una forma increïble, però els teus músculs continuen exactament igual de fofos. 🏋️🤖

Amb la programació passa exactament això:

- ❌ **Si li dius a la IA:** *«Fes-me l'exercici 3 de Java»* La IA et donarà un codi impecable en 3 segons ⚡. L'empegaràs a la plataforma, el professor et posarà un 10... però **tu no hauràs après absolutament res**.
- 💣 **El dia de l'examen presencial (sense IA):** Et quedes sol/a davant del full en blanc o de l'ordinador sense internet. La teva "RAM mental" no ha entrenat mai la generació de solucions i el bloqueig serà monumental. 😳🔥

🏛️💬 El Mètode Pro: La IA com a Tutor Socràtic 🔗

Perquè la IA treballi PER A TU (i no EN LLOC DE TU), la clau està a **canviar la manera com li preguntes**.

Quan te n'atures en un exercici, no demanis la solució directa. Fes servir el **Aviat del Tutor Socràtic**:

> 💬 **Copia i pega aquest text a la IA quan te n'atures:**
>
> *«Hola! Estic aprenent a programar en Java i m'he encallat en aquest exercici: [enganxa l'enunciat]. No em donis el codi ni la solució directa. Fes-me preguntes guiadores o dona'm una petita pista perquè pugui descobrir jo mateix/a on està la falla o quin pas em falta.»*

📊 Comparativa de mentalitat

❌ Mode Vague (Te'n vas al fons)	✅ Mode Pro (Et fas un/a màquina)
"Fes-me el codi per a calcular si un número és parell."	"Explica'm quin concepte matemàtic s'usa per a saber si un número és divisible per 2 sense donar-me el codi."
"On està l'error en aquest codi? Corregeix-ho."	"Tinc aquest error a la línia 12. Dona'm una pista de quin concepte de sintaxi m'he oblidat de revisar."
Copiar el codi directament al teu IDE.	Llegir l'explicació de la IA, tancar la finestra del xat i escriure el codi de memòria. 🙈

🕵️ La Regla del "No et fies ni de la teva ombra" (Al·lucinacions)

Els models de llenguatge no són cervells que "pensen" com nosaltres; són sistemes estadístics brutals que predien quin és la paraula més probable que ve rere una altra. 📊🔤

Això significa una cosa fonamental: **la IA es pot inventar coses amb una seguretat i una elegància altíssima, però continua sent una "invenció"**.

- Et pot donar mètodes que no existeixen en Java 🦄.
- Et pot proposar solucions ultra-complexos que no corresponen al teu nivell 🤯.
- Et pot dir que un codi que està MALAMENT és correcte amb un somriure virtual 🤡.

> **La regla d'or:** Mai, MAI dones per bo un codi que t'hagi suggerit una IA sense haver-lo provat, entés línia per línia i comprovat amb els teus propis casos límit. Tu ets el/la pilot; la IA és

> només el copilot! 🏎️💨

💡 Idea clau per a recordar

> **La IA ha d'ajudar-te a pensar, mai substituir el teu pensament.**

> Utilitza la IA perquè t'expliqui conceptes o et dona pistes com un professor privat les 24 hores, però assegura't que l'únic que escriu el codi final sigui el teu propi cervell. 💪🤖🎯

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Practicar (menys mirar, més intentar) · **Següent:** 09 · El meu primer problema (pont cap a Java)

09 — El meu primer problema (pont cap a Java)

Resolem el nostre primer repte sense tocar Java 🌉☕

> 🗺️ **Estás en: 🧠 U01 · Introducció al pensament computacional** → 09 · El meu primer problema (pont cap a Java)

Hem arribat al final d'aquesta Unitat 1! Després de desmuntar falsos mites, de barallar-nos amb la literalitat dels robots, d'aprendre a no tenir por al text vermell i de posar a la IA a treballar com a tutor privat, arriba el moment de la veritat: **resoldre el nostre primer problema de programació real.** 🥳🎉

I el millor de tot? Anem a fer-ho **sense escriure ni una sola línia de Java encara**.

Per què? Perquè així comprovaràs en carn pròpia la gran lliçó d'aquesta unitat: **la part difícil de programar és pensar la lògica; traduir-ho a un llenguatge concret és només qüestió de sintaxi.** 🧠➡️⌨️

🥊 El Repte: El Comparador de Nombres

Imagina que ens demanen crear un programa súper senzill per a una aplicació:

> 📌 **Enunciat del problema:**
>
> *"Un programa ha de demanar dos nombres enters a l'usuari i indicar clarament per pantalla quin dels dos és el major, o si tots dos números són iguals."*

Sembla fàcil, veritat? Però abans d'obrir qualsevol editor de codi o començar a escriure parèntesis, aplicarem el nostre mètode de treball en 3 passos. 📄✍️

🧩 Pas 1: Descomposició i Entrenament de Dades (Inputs i Outputs)

Abans de pensar la lògica, un/a programador/a es pregunta: **Què necessito demanar (Inputs) i què he de tornar (Outputs)?**

- 📥 **Dades d'entrada (Inputs):** Necessitem 2 nombres enters. Els direm el nombre A i nombre B.
- 📤 **Dada de sortida (Output):** Un missatge per pantalla que digui quin és el major o si són iguals.

👣 Pas 2: L'Algorisme en Pseudocodi (Lògica pura) 🔗

Ara escriurem la seqüència de passos en el nostre propi idioma, sense preocupar-nos per punts i comes o claus. Solament lògica!

```
1. Demanar a l'usuari el Número A.
2. Demanar a l'usuari el Número B.
3. Comparar els dos números:
   - SI el Número A és major que el Número B:
         Mostrar el missatge: "El Número A és el major".
   - SI NO, SI el Número B és major que el Número A:
         Mostrar el missatge: "El Número B és el major".
   - SI NO (si cap de les anteriors es compleix):
         Mostrar el missatge: "Tots dos números són iguals".
4. Fi del programa.
```

Mira aquest algorisme un segon. Té tot el sentit del món, veritat? **Això JA és programar.** Qualsevol persona, sigui d'Ontinyent, de Tòquio o un robot de la NASA, entén perfectament aquesta lògica. 🚀✨

🧪 Pas 3: El testing de casos límit (Mentalitat de detectiu) 🔗

Abans de donar l'algorisme per bo, anem a fer-li el nostre "control de qualitat" provant diferents valors:

- 🟢 **Cas 1 (Normal):** A = 8, B = 3 ──► El programa diu *"A és el major"*. (Correcte! ✅)
- 🟢 **Cas 2 (Invers):** A = 2, B = 10 ──► El programa diu *"B és el major"*. (Correcte! ✅)
- 🟡 **Cas Límit (Igualtat):** A = 5, B = 5 ──► El programa comprova la primera opció (5 > 5? Fals), la segona (5 > 5? Fals) i cau en el *SI NO*, dient *"Tots dos números són iguals"*.

(Correcte! ✅)

L'algorisme és 100% sòlid. Funciona en paper i funciona en el teu cap. 🏆

🌉 I ara... On està el pont cap a Java? ☕

Aquí ve la màgia! A partir de la pròxima unitat, l'única cosa que farem serà **aprendre com es diuen en Java aquelles mateixes instruccions que tu ja has pensat**:

- Per a demanar dades, aprendrem a usar una eina de Java anomenada Scanner.
- Per a comparar i prendre decisions (SI... SI NO), aprendrem la paraula reservada if i else.
- Per a mostrar missatges per pantalla, usarem una instrucció anomenada System.out.println().

🧠 LA TEVA LÒGICA (Unitat 1)	☕ LA TRADUCCIÓ A JAVA (Unitat 2)
SI el Número A és major que el B...	if (numeroA > numeroB) {
Mostrar "A és el major"	System.out.println("A és el major");
FI DEL SI	}

Veus com no era tan fosc ni tan difícil? La lògica ja la tens; ara només aprendrem el "diccionari" de Java per a dir-li-ho a la màquina! 📖☕

💡 Idea clau per a tancar la Unitat 1

> **Ja saps pensar com un programador/a.**

> Ara que tens el mètode, la mentalitat i el protocol per a no bloquejar-te, obrim la porta a la Unitat 2. Benvingut/da al món real de Java! 🚀☕💻

🐛 Glossari

Per a tancar la unitat i que no et sonen a "xinés" les paraules que el professorat i el sector utilitzen diàriament, aquí tens la teva guia ràpida de consulta:

💻 Termes tècnics d'Infraestructura

- **Codi Font (Source Code):** El text que tu escrius en el llenguatge de programació (fitxers .java). És el "manuscrit" que nosaltres podem llegir i editar.
- **Sintaxi:** Les regles gramaticals d'un llenguatge. Si et saltes un punt i coma ; o una clau {}, el compilador no entendre res i et donarà un error de sintaxi.
- **Compilador:** El programa traductor que agafa el teu codi font i el converteix en instruccions que el processador pugui executar.
- **IDE (*Integrated Development Environment*):** El programa que utilitzes per a escriure codi (com *Eclipse*, *NetBeans* o *VS Code*). És com un "Word per a programadors", amb colors, autocompletat i botó d'executar.

🧠 Termes de lògica i procés

- **Algorisme:** La seqüència ordenada, finita i sense ambigüitats de passos que resol un problema.
- **Pseudocodi:** Cal escriure la lògica d'un algorisme en el teu propi idioma (valencià/castellà) abans de traduir-lo a Java.
- **Descomposició:** La tècnica de dividir un problema gran i complex en sub-problemes molt més petits i fàcils de resoldre.
- **Abstracció:** El procés de filtrar i ignorar els detalls irrellevants d'un problema per a centrar-te només en les dades essencials.

🐛 Termes de gestió d'errors i control

- **Bug:** Un error o falla en la lògica o en l'execució d'un programa.
- **Debugging (o Depuració):** El procés de buscar, aïllar i corregir un *bug* en el teu codi.
- **Stack Traci:** La muntanya de text vermell que apareix a la consola quan el programa falla. Et diu quin error hi ha hagut i en quina línia exacta ha passat.
- **Casos límit (*Edge Cases*):** Les situacions o dades extremades (valors zero, nombres negatius, edats exactes) on el programa té més risc de fallar si no s'ha provat bé.

🤖 Termes de treball i metodologia

- **Miratge de la comprensió:** La falsa sensació de saber fer un exercici només per haver vist o llegit com el resolia una altra persona.
- **Tutor Socràtic (amb IA):** Utilitzar la intel·ligència artificial fent-li preguntes perquè t'ajudi a pensar, en lloc de demanar-li que et faci el codi directe.
- **Entrada/Sortida (*Input/Output* o I/O):** Les dades que el programa rep de l'usuari (Input) i els resultats que el programa mostra per pantalla (Output).

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Com utilitzarem la IA · **Següent:** **UO2 · Introducció a Java i l'entorn de treball**

U02 — Introducció a Java i l'entorn de treball

El teu primer cafè amb Java ☕

Benvingut al curs de Programació en Java. Sabies que el teu ordinador és bàsicament un cadell molt llest però amb zero iniciativa? No fa res fins que li dones ordres precises. I per a això necessitem un **entorn de desenvolupament**, un llenguatge i moltes ganes de compilar.

Aquesta primera unitat et deixa a la porta del castell: instal·laràs el JDK, escriuràs el teu primer programa, coneixeràs la trilogia del cafè (JVM, JRE i JDK), usaràs el depurador com un detectiu i parlaràs amb la màquina mitjançant comentaris i arguments. No necessites experiència prèvia: només un editor de text i curiositat.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: cada punt desenvolupa una idea completa i enllaça amb el següent.

🎯 Objectiu de la unitat

En acabar, seràs capaç de:

- Explicar la diferència entre **JDK, JRE i JVM** amb l'analogia de la cafeteria.
- Instal·lar i configurar un JDK i comprovar la instal·lació amb `java -version`.
- Escriure, compilar i executar el teu primer programa Java.
- Diseccionar el mètode `main` i entendre cada peça de l'esquelet d'un programa.
- Usar el **depurador** (breakpoints, step over/into, watch) com un detectiu.
- Escriure comentaris d'una línia, de bloc i **Javadoc**.
- Gestionar els **arguments de línia de comandes** amb `args`.
- Reconèixer i corregir els errors de compilació més típics del novell.
- Treballar amb **Visual Studio Code** (o el teu IDE favorit) i conèixer el teu flux de treball diari.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Nivell
01 · Què és Java?	Història, bytecode, la trilogia JVM/JRE/JDK	Tots
02 · Instal·lació del JDK	Eclipse Temurin, VS Code, Extension Pack, `java -version`, PATH	Tots
03 · El teu primer programa	Hola Món, estructura, el mètode `main`	Tots
04 · El depurador	Breakpoints, step over/into, watch	Tots
05 · Comentaris i documentació	`//`, `/* */`, `/** */` i Javadoc	Tots
06 · Arguments de línia de comandes	L'array `args`, com llegir-lo i usar-lo	Tots
07 · El compilador i els seus errors	Compilar vs executar, errors típics del novell	Tots
08 · VS Code i el teu flux de treball	Projectes, dreceres, crear/executar/depurar	Tots
09 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA1)

RA1: Reconeix l'estructura d'un programa informàtic, identificant i relacionant els elements propis del llenguatge de programació utilitzat.

CE	Criteri	On es cobreix
a)	Blocs que componen l'estructura d'un programa	✅ Punts 3 i 7
b)	Projectes de desenvolupament d'aplicacions	✅ Punt 8
c)	Entorns integrats de desenvolupament	✅ Punts 2 i 8
i)	Comentaris en el codi	✅ Punt 5

> 📌 El depurador que es veu en el punt 4 posa les bases del RA3 (programes provats i depurats), que es completa en la U04.

🚪 Per on començo?

- ¿Cero coneixements? → Comença en el punt 1, no hi ha cap requisit previ.
- Ja saps què és un IDE? → Ves directe al punt 3 i torna enrere només si et perds.
- Vens a repassar? → Fes el Repàs interactiu i els butlletins.

📍 **Primer punt:** 01 · Què és Java?

⏭️ **En acabar la unitat, continua en U03 · Variables, tipus i operadors.**

01 — Què és Java?

La trilogia del cafè i el secret de la tassa humejant ☕

> 🗺️ **Estàs en: 🚀 UO2 · Introducció a Java i l'entorn de treball** → 01 · Què és Java?

Java és un llenguatge de programació que s'executa dins d'una màquina virtual (la JVM), la qual cosa li permet córrer igual en Windows, Linux o macOS: **"escriu una vegada, corre en qualsevol lloc"**.

Si el teu ordinador és un cadell molt llest, però amb zero iniciatives, Java és un dels idiomes que pots usar per a donar-li ordres. I la JVM és l'intèrpret que tradueix les teves ordres a l'idioma concret de cada cadell.

🔍 D'on surt aquest llenguatge?

Java va néixer el **1995** en **Sun Microsystems**, i la seva història té una de les curiositats més divertides del sector: es va inspirar en la **màquina de cafè** de l'oficina. Per això el logo és una tassa fumejant. No m'ho invento.

La idea dels seus creadors era simple, però ambiciosa: un llenguatge que funcionés en **qualsevol dispositiu**, sense importar el sistema operatiu ni el maquinari. Abans de Java, cada plataforma necessitava la seva pròpia versió del programa. Java va proposar la solució: no programar per a l'ordinador, sinó per a una **màquina virtual** que l'ordinador simula.

💡 **Dada freak:** el llenguatge es va dir primer *Oak* (roure), per un arbre que es veia des de l'oficina. Van haver de canviar-lo per motius de marca registrada i, segons la llegenda, el nom "Java" es va triar pel cafè que prenen durant les reunions.

📦 Què és això del bytecode?

Quan escrius Java no escrius instruccions de la CPU del teu ordinador: escrius instruccions per a la **JVM**. El procés és així:

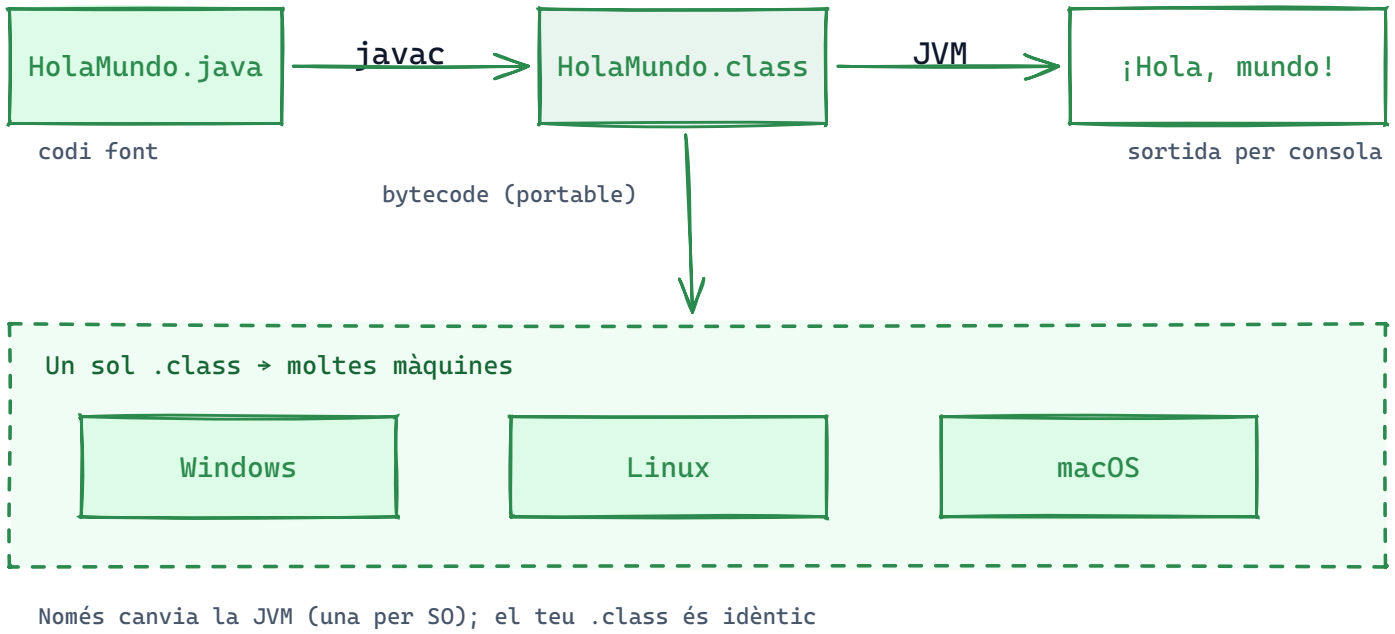
1. Tu escrius codi font en un arxiu `.java`.
2. El **compilador** (`javac`) ho tradueix a *bytecode*: un idioma intermedi que entén la JVM. El resultat és un arxiu `.class`.

3. La **JVM** llegeix el bytecode i l'executa en la teva màquina concreta.

```
El teu codi (.java)  --javac-->  bytecode (.class)  --JVM-->  ¡s'executa!
```

La gràcia és que aquell `.class` és el mateix per a totes les plataformes: la JVM de Windows, la de Linux i la de macOS saben llegir-lo. Només canvia la JVM, no el teu programa.

Compilar i executar: el viatge del teu programa



☕ La trilogia del cafè: JVM, JRE i JDK

Aquí està el moment estrella de la unitat. Java funciona com una **cafeteria d'especialitat**, i has de memoritzar qui és qui:

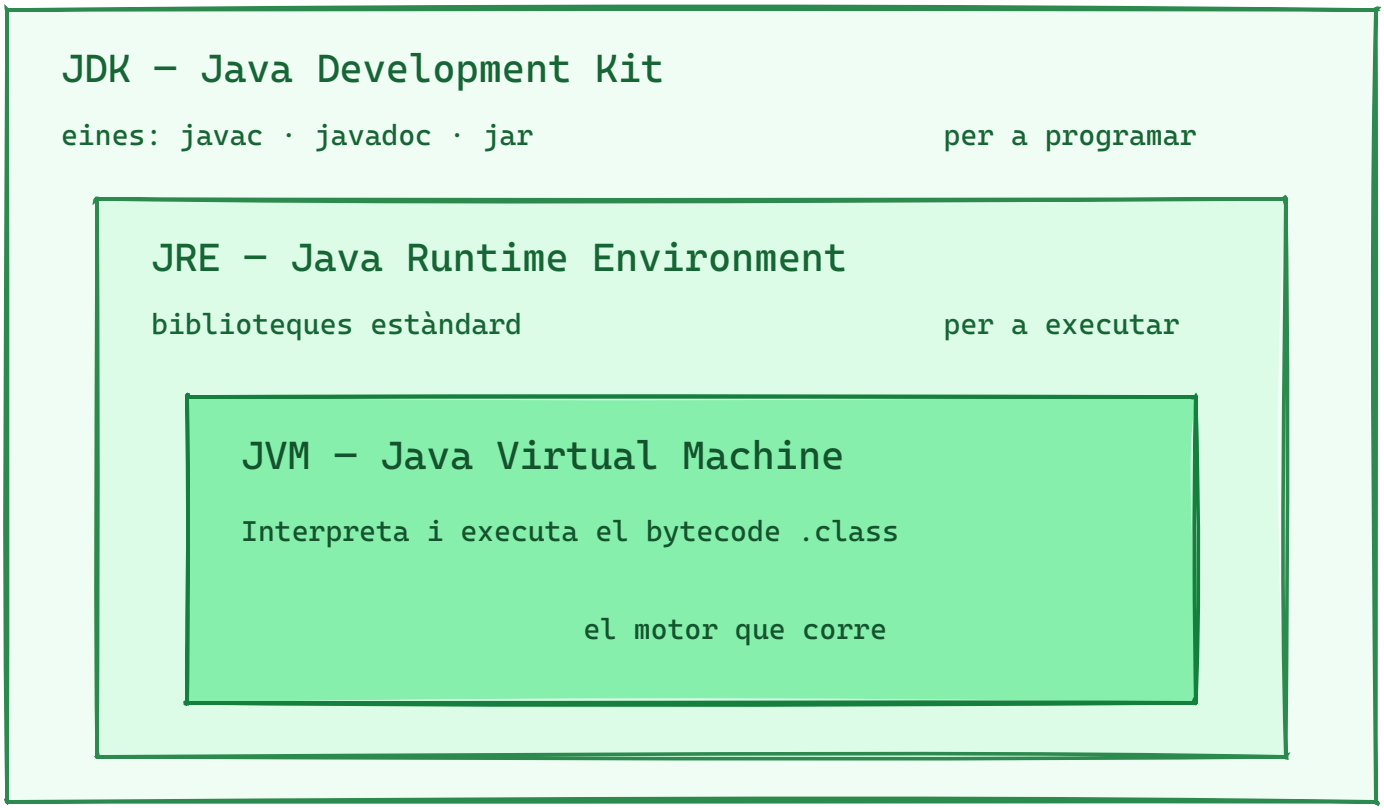
Concepte	Definició	Analogia
JVM	La màquina que executa el *bytecode*.	La màquina de cafè: té la seva recepta i funciona igual en qualsevol lloc.
JRE	Tot el necessari per a executar Java.	La cafeteria sencera: màquina, gots, sucre…
JDK	Tot el necessari per a crear programes.	El kit complet per a muntar la teva cafeteria: màquina, grans, molinet i manual de barista.

- **JVM (Java Virtual Machine)**: la màquina de cafè. Executa el bytecode i viatja amb el teu programa a tot arreu.
- **JRE (Java Runtime Environment)**: la cafeteria sencera. Té la JVM i les llibreries necessàries per a *executar* cafè ja fet. Si només vols *executar* programes, et basta.
- **JDK (Java Development Kit)**: el kit per a muntar la teva pròpia cafeteria. Inclou el JRE més el compilador `javac`, el generador de documentació `javadoc` i altres eines de desenvolupament. Si vols *crear* programes, el necessites.

```
// Comentari de com funciona el codi
public class Cafe {
    public static void main(String[] args) {
        System.out.println("☕ ¡Café preparat!");
    }
}
```

El JDK compila això a bytecode (cafè molt), el JRE ho passa per la JVM i… tachán! Cafè en la teva pantalla.

⚠️ **Advertència:** no confonguis JDK amb JRE. El JDK és el *ganivet del xef*; el JRE és el *plat servit*. El JDK inclou el JRE, així que instal·lant el JDK tens les dues coses. Instal·lar només el JRE et permet executar programes, però no crear-los.



🌍 Per què segueix viu Java tants anys després?

No és nostàlgia. Java segueix sent un dels llenguatges més demanats del món perquè:

- **És multiplataforma**: el mateix programa corre en mòbils, servidors, caixers i fins i tot en la rentadora intel·ligent.
- **Domina el món empresarial**: les grans empreses (banca, assegurances, logística) fa dècades que construeixen els seus sistemes en Java.
- **És el llenguatge oficial d'Android** (en la seva variant Kotlin conviu, però Java segueix omnipresent).
- **Té una comunitat enorme**: qualsevol error que tinguis, probablement algú ja el va resoldre fa deu anys en un fòrum.
- **És exigent, i això et fa millor**: Java t'obliga a ser ordenat. Els programadors que aprenen amb Java solen escriure codi més net.

🎯 Mini-comprovació

Posat a prova en 30 segons (les respostes estan més endavant):

1. Quina és la diferència entre JDK i JRE en una frase?
2. Què fa el compilador javac amb el teu codi `.java`?
3. Per què el logo de Java és una tassa fumejant?

🔄 Respostes

1. El JDK serveix per a crear programes (inclou el compilador); el JRE serveix per a executar-los. El JDK conté el JRE.
2. Ho tradueix a bytecode (un arxiu `.class`) que la JVM pot executar.
3. Perquè el llenguatge s'inspira en la màquina de cafè de l'oficina de Sun Microsystems: "escriu una vegada, corre en qualsevol lloc" (com el cafè, que es prepara igual en qualsevol cafeteria).

✅ Resum en 3 frases

1. Java executa el seu codi dins d'una **màquina virtual (JVM)**, la qual cosa li dona portabilitat total.
2. **JDK** crea (compila a bytecode), **JRE** executa, **JVM** transporta: com Amazon però amb cafè.
3. Java segueix sent rei en el món empresarial i aprendre'l et forma com a programador ordenat.

🐛 Vocabulari ràpid

Terme	Idea general
JVM	La màquina virtual que executa el bytecode
JRE	Entorn d'execució (JVM + llibreries)
JDK	Kit de desenvolupament (compilador + JRE + eines)
Bytecode	Codi intermedi que entén la JVM
Compilador	Traductor de codi font a bytecode

02 — Instal·lació del JDK

OpenJDK, java -version, PATH, IDE ☕

> 🗺️ **Estàs en:** 🚀 **U02 · Introducció a Java i l'entorn de treball** → 02 · Instal·lació del JDK

Per a crear programes Java necessites el JDK. Instal·lar-lo és més fàcil que muntar un moble d'Ikea, i no et sobraran caragols.

En el punt anterior vas veure que el JDK és el kit complet per a *crear* programes. Ara toca posar-lo en la teva màquina i comprovar que tot funciona.

🛒 Quin JDK instal·le?

El Java original de Sun/Oracle ha tingut una vida moguda amb les llicències. Avui la millor opció i la més sensata per a estudiar és anar directament a la font oficial i lliure:

1. **OpenJDK**: És el projecte de referència, lliure i de codi obert, del qual surten totes les altres distribucions. **És la nostra opció principal i la recomanada, més senzill d'instal·lar en Linux.**
   - https://openjdk.org/index.html
2. **Eclipse Temurin** (abans AdoptOpenJDK): Una distribució excel·lent basada directament en OpenJDK, lliure, gratuïta i mantinguda per la fundació Eclipse.
   - https://adoptium.net/es/temurin/releases
3. **Oracle JDK**: La versió comercial d'Oracle. Vàlida per a entorns empresarials, però amb una llicència més restrictiva. **Més senzill d'instal·lar en Windows**
   - https://www.oracle.com/java/technologies/downloads/

La versió a triar: Triaràs l'última versió **LTS** (*suport a llarg termini*). Avui dia, qualsevol versió 17, 26 o superior et serveix per a tot el curs. No t'obsessiones amb el número exacte: tot el que aprendrem funciona exactament igual en totes elles.

🚀 Els primers passos

Pas 1: Instal·lar

Executa l'instal·lador d'OpenJDK i accepta els valors per defecte. En Windows, assegura't de marcar l'opció d'afegir el JDK al PATH si te l'ofereix l'instal·lador (així podràs usar `java` i `javac`

des de qualsevol terminal sense haver de configurar res a mà).

Pas 2: Verificar

Obre una terminal (PowerShell o CMD en Windows, o la terminal en Linux/macOS) i escriu:

```
> java -version
openjdk version "21.0.2" 2024-01-16
OpenJDK Runtime Environment (build 21.0.2+13-LTS)
OpenJDK 64-Bit Server VM (build 21.0.2+13-LTS, mixed mode)
```

I després comprova el compilador:

```
> javac -version
javac 21.0.2
```

Si veus una sortida semblant, enhorabona! Tens poders de compilació actius.

Si, en canvi, el sistema et diu `'java' no es reconeix com un comandament intern o extern`, significa que el PATH no està ben configurat: busca en el menú de Windows "Editar les variables d'entorn del sistema" i afegeix la ruta de la carpeta `bin` de la teva instal·lació d'OpenJDK a la variable `Path`.

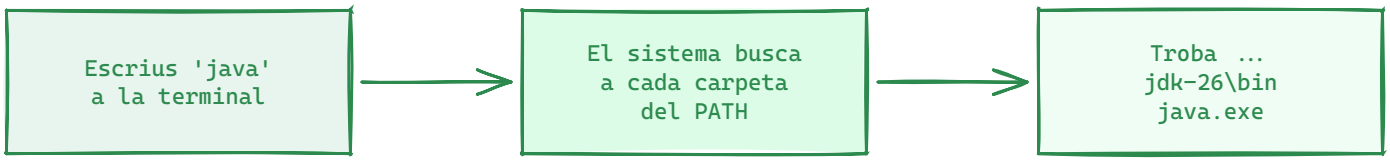
💡 Què és el PATH?

El "PATH" és la llista de carpetes on el teu sistema operatiu busca els programes i comandaments que escrius en la terminal. Si la carpeta `...\jdk-26\bin` (o similar) està en el PATH, en escriure `java` el sistema la troba a la primera. Sense aquella configuració, el sistema no sap on està instal·lat el teu JDK.

Verificar el JDK i el PATH

```
> java -version
openjdk version "21.0.2" 2024-01-16
> javac -version
javac 21.0.2
```

⚠ 'java' no es reconeix com a ordre → el PATH no està ben configurat



Sense el PATH, el sistema no sap on està el teu JDK

Pas 3: Per què hi ha dos comandaments?

- **`javac`**: És el compilador. Converteix el teu codi font (`.java`) en bytecode (`.class`).
- **`java`**: És l'executor. Arranca la JVM per a executar el bytecode que has compilat prèviament.

Es necessiten els dos: primer **`javac`** tradueix el teu codi i després **`java`** el posa en marxa. Els veuràs treballar estretament durant tot el curs.

🛠️ L'IDE: la teva navalla suïssa

El JDK és el motor, però la major part del temps no escriuràs codi en un bloc de notes: usaràs un IDE (Entorn de Desenvolupament Integrat) o un editor avançat com VS Code. L'IDE reuneix en una sola aplicació:

- Editor de codi amb colors, autocompletat i ressaltat d'errors mentre escrius.
- Compilador i executor amb un botó: ja no necessites teclejar `javac` i `java` a mà (encara que és bo saber com funciona).
- Depurador integrat.
- Gestió de projectes: els teus programes no són només arxius solts, sinó projectes amb estructura.

IDE / Editor	Punts forts
VS Code	L'opció recomanada. Molt lleuger, modern i altament personalitzable. Amb l'extensió Extension Pack for Java ofereix un entorn complet i excel·lent.
IntelliJ IDEA (Community)	El favorit del sector professional; autocompletat bestial. Una mica més pesat en arrancar.
NetBeans	Simple, oficial d'Oracle, perfecte per a començar.
Eclipse	Clàssic, molt usat en empreses, un pèl més dens.

💡 **Recomanació per al curs:** VS Code amb el plugin de Java (Extension Pack for Java) és la millor opció per la seva lleugeresa i versatilitat. També pots usar IntelliJ IDEA Community Edition si el teu ordinador ho suporta, o NetBeans si vols alguna cosa més tradicional. Tots valen: l'IDE és una eina, no l'objectiu.

🏠 Exemple guiat: el teu primer projecte en VS Code 🔗

Ho deixarem tot muntat abans d'escriure codi:

1. Obre VS Code i assegura't de tenir instal·lat el plugin Extension Pack for Java.
2. Prem `Ctrl + Shift + P` (o `Cmd + Shift + P` en Mac) per a obrir la paleta d'ordres, escriu `Java: Create Java Project` i selecciona `No build tools`.
3. Tria la carpeta on vols guardar el projecte i dona-li un nom (per exemple, `MiPrimerCurso`).
4. VS Code et crearà l'estructura del projecte amb una carpeta `src`.
5. Dins de la carpeta `src`, crea un arxiu anomenat `HolaMundo.java` i escriu dins:

```
public class HolaMundo {
    public static void main(String[] args) {
            System.out.println("¡Hola, Mundo! Porte anys esperant a que em
creares.");
    }
}
```

6. Fes clic en el botó Run ▶ (que apareixerà just damunt del mètode main) o prem F5 i mira la terminal/consola.

Si veus el missatge en la pantalla, la teva paradeta està muntada: JDK + VS Code + primer programa. Estàs oficialment dins.

⚠️ **Advertència:** no confonguis la consola/terminal de l'IDE amb la terminal del sistema. La consola de VS Code és on s'imprimeixen els `System.out.println` en executar. Si no veus la sortida, busca la pestanya Terminal o Output / Sortida.

🎯 Mini-comprovació

1. Quin comandament comprova que la teva instal·lació funciona?
2. Per a què serveix el PATH i què passa si el JDK no està en ell?
3. Quina és la diferència entre `java` i `javac`?

🔄 Respostes

1. `java -version` (i també `javac -version`).
2. El PATH és la llista de carpetes on el sistema busca els comandaments. Sense ell, en escriure `java` el sistema respon que no reconeix el comandament.
3. `javac` compila el codi font a bytecode; `java` executa el bytecode amb la JVM.

✅ Resum en 3 frases

1. Instal·la OpenJDK (un JDK lliure) i verifica amb `java -version` i `javac -version`.
2. `javac` compila, `java` executa, i el PATH és l'adreça postal que el sistema necessita per a trobar-los.
3. Un IDE o editor com VS Code (amb el plugin de Java) reuneix editor, compilador i depurador: és la teva navalla suïssa.

🐛 Vocabulari ràpid

Terme	Idea general
PATH	Llista de carpetes on el sistema busca comandaments
IDE / Editor	Entorn integrat: editor + compilador + depurador
LTS	Versió amb suport a llarg termini (la recomanada)
Terminal	La finestra on escrius comandaments com `java`
Projecte	La carpeta on viuen les teves classes i configuracions

03 — El teu primer programa

Hola Món, estructura, el mètode main 💻

> 🗺️ **Estàs en: 🚀 UO2 · Introducció a Java i l'entorn de treball** → 03 · El teu primer programa

Programar és parlar-li a un extraterrestre molt literal: si li dius "saluda", no ho fa. Has de dir-li com, *quan* i *per què*.

L'ordinador és ximple, però precís: no interpreta, **executa**. Cada línia, en ordre, sense saltar-se'n cap. El teu primer programa va a cridar "Hola!" per la consola, i a partir d'aquí tot és afegir més ordres.

👋 Hola Món, el ritual d'iniciació

Tot programador comença per aquí. És com el primer cafè del matí: no és opcional.

```
public class HolaMundo {
    public static void main(String[] args) {
            System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");
    }
}
```

Compila-ho (`javac HolaMundo.java`) i executa-ho (`java HolaMundo`), o prem el botó ▶ del teu IDE. La consola dirà:

```
¡Hola, Mundo! Llevo años esperando a que me crearas.
```

🔬 Dissecció d'Hola Món

Diseccionarem això com si fora una granota en biologia:

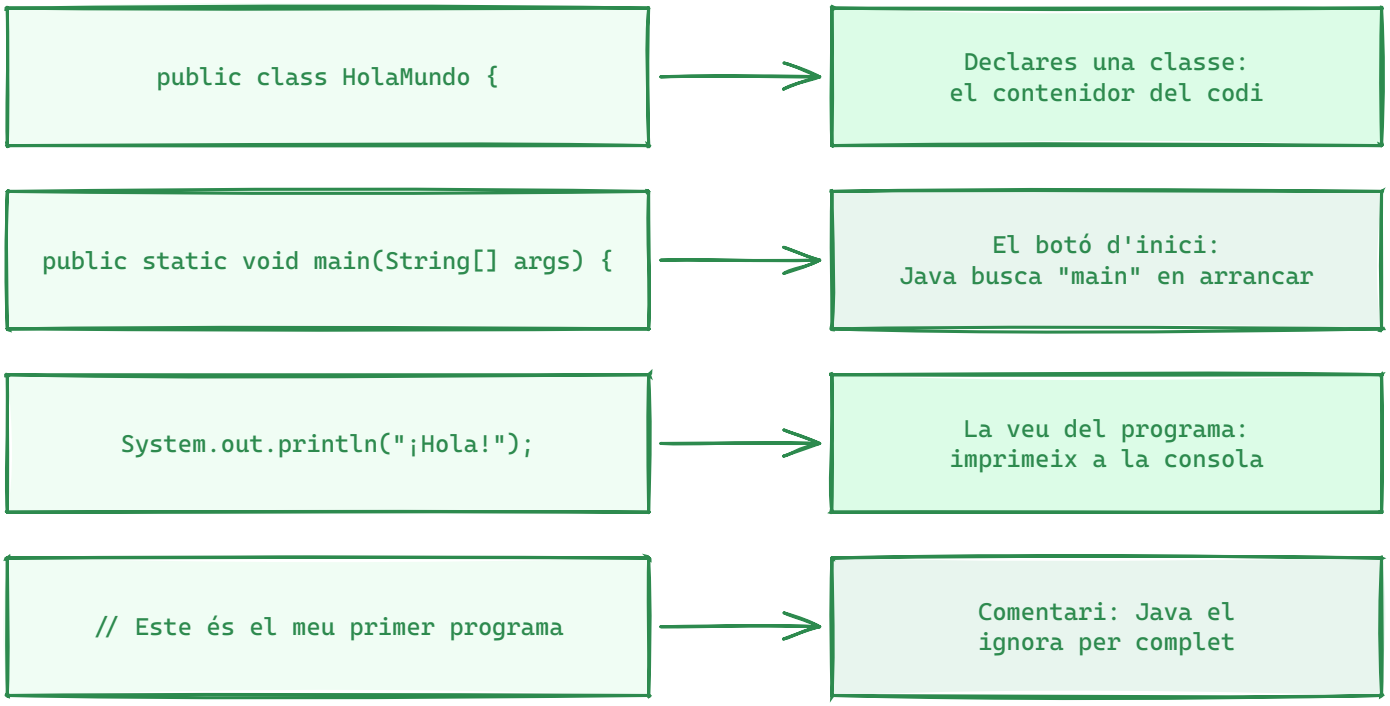
- **`public class HolaMundo`**: declares una classe. Pensa que li dius a Java: "Escolta, vaig a crear una cosa que es diu HolaMundo". `public` significa que és accessible des de fora i la classe ha de dir-se igual que l'arxiu.

- **`public static void main(String[] args)`**: aquest és el **botó d'inici**. Quan executes el programa, Java busca aquesta línia i diu "per aquí es comença!".
- **`System.out.println(...)`**: és **la veu** del programa. Li dius que cridi alguna cosa per la consola. `print` imprimeix sense salt de línia; `println` imprimeix i salta de línia.

```
public class MiPrimerPrograma {
    public static void main(String[] args) {
        // Este és el meu primer programa
        System.out.println("¡Holaaaa, mundo!");
        System.out.println("Estoy aprendiendo Java");
        System.out.println("Y me está gustando (de momento)");
    }
}
```

💡 **Detall pràctic:** cada instrucció acaba amb `;`. És el punt final de cada frase. Sense ell, el compilador pensa que la frase continua i es lliga. Els `{}` delimiten els blocs: els de la classe contenen la classe, els del main contenen les ordres.

Anatomia d'un Hola Mundo



Java només executa el que està dins de main

🗝️ Per què `public static void main(String[] args)`?

Sembla un encanteri de Harry Potter. Desgranem cada paraula:

Paraula	Què significa
public	Java pot trobar-lo des de fora: el botó és visible
static	Pot cridar-se sense necessitat de crear un objecte
void	No torna cap valor: fa el seu treball i es calla
main	El nom exacte que Java busca en arrancar. No en val un altre
String[] args	Una butxaca on pots ficar arguments en executar (punt 6)

El main és **obligatori tal qual**. Si canvies `main` per `inici`, Java no el troba i el programa no fa res.

🚪 El mètode que no es crida

Aquí va una de les trampes favorites en els exàmens. Observa:

```
public class Saludos {
    public static void main(String[] args) {
        System.out.println("¡Hola desde el método main!");
    }

    public static void saludo() {
        System.out.println("Esto nunca se ejecuta...");
    }
}
```

S'executarà correctament? **Sí**, però només imprimeix la primera línia. El mètode `saludo()` existeix, però com mai el crides des de `main`, es queda aquí fent el vague. Java només executa el que està dins del `main` (llevat que explícitament crides altres mètodes). El mètode `saludo()` és com un actor que té el guió après, però mai surt a l'escenari.

🧠 **Truc de memòria:** `main` és la porta d'entrada de la casa. Pot haver-hi moltes habitacions (mètodes), però ningú entra per la finestra. Si no cridis a la porta, les habitacions es queden buides.

⭐ Tracta de pensar com el codi: tu ets la JVM

Seràs Java per un moment. Pren paper i boli (o mentalment). Et donen aquest codi:

```
public class Ordinador {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        int z = x + y;
        System.out.println("El resultado es: " + z);
    }
}
```

Segueix els passos com si fossis la JVM:

1. Trobes la classe `Ordinador`.
2. Busques el mètode `main` — aquí està.
3. Crees un espai anomenat `x` i fiques un 5.
4. Crees `y` i fiques un 10.
5. Crees `z`, sumes `x` i `y` (15), ho guardes.
6. Crides per pantalla: "El resultado es: 15".

🔄 **Solució**

La sortida és exactament:

```
El resultado es: 15
```

Si la teva resposta és diferent, torna a començar: l'ordinador no interpreta, *executa*, línia a línia.

🎯 Mini-comprovació

1. Què passa si reanomene `main` a `inici`?

2. Imprimeix el mateix `System.out.print("Hola")` que `System.out.println("Hola")`?
3. Per què en falta un `;` en el codi de baix? Assenyala l'error:

```
public class Erroneo {
    public static void main(String[] args) {
        System.out.println("Me falta algo")
    }
}
```

🔄 **Respostes**

1. Java no troba la porta d'entrada: el programa compila (en alguns entorns) però en executar no fa res, o dona error. La firma ha de ser exacta.
2. No. `print` no salta de línia; `println` sí. En pantalla es nota: `print("Hola")` seguit de `print("Mundo")` mostra `HolaMundo` pegat.
3. Falta el `;` al final de `System.out.println("Me falta algo")`. El compilador es queixarà esperant el punt i coma.

✅ Resum en 3 frases

1. Tot programa té una **classe** (contenidor) i un **mètode main** (porta d'entrada).
2. `System.out.println()` és la veu del programa i el `";"` és el punt final de cada frase.
3. La JVM executa **línia a línia, en ordre**: tu decideixes què entra per la porta.

🐛 Vocabulari ràpid

Terme	Idea general
Classe	El contenidor del codi (una "cosa" en Java)
Mètode	Un bloc d'ordres amb nom
main	El mètode que Java executa en arrancar
println	Imprimeix text i salta de línia
Consola / Terminal	La finestra de text on s'imprimeix la sortida del programa

04 — El depurador

Breakpoints, step over/into, watch 🐛

> 🗺️ **Estàs en:** 🚀 **U02 · Introducció a Java i l'entorn de treball** → 04 · El depurador

El depurador (debugger) és com tenir visió de raigs X per al teu codi: para el programa on tu vols i t'ensenya el valor de cada variable en cada instant.

El teu primer programa funciona. Però aviat tindràs un programa rar: una variable `edat` que surt 25 quan n'hauria de sortir 18. Què fas? Li pegues a l'ordinador? No. Uses el depurador.

🔍 Què és depurar?

Depurar (debug) és treure-li els bitxos (bugs) al programa. I el secret dels programadors experimentats no és endevinar: és veure. El depurador et permet:

- Parar el programa en una línia concreta.
- Avançar instrucció a instrucció.
- Inspeccionar el valor de les variables en cada moment.
- Modificar valors sobre la marxa.

És com veure una sèrie de crims en càmera lenta: pots pausar, observar qui fa què, i analitzar cada detall.

💡 **Detall pràctic:** Els errors es divideixen en dues famílies. Els de compilació els atrapa `javac` (o el mateix VS Code remarcant-ho en vermell) i et diu la línia. Els de lògica compilen perfecte però fan el que no han de fer: per a aquells no hi ha millor arma que el depurador.

🛠️ Les quatre eines del detectiu en VS Code

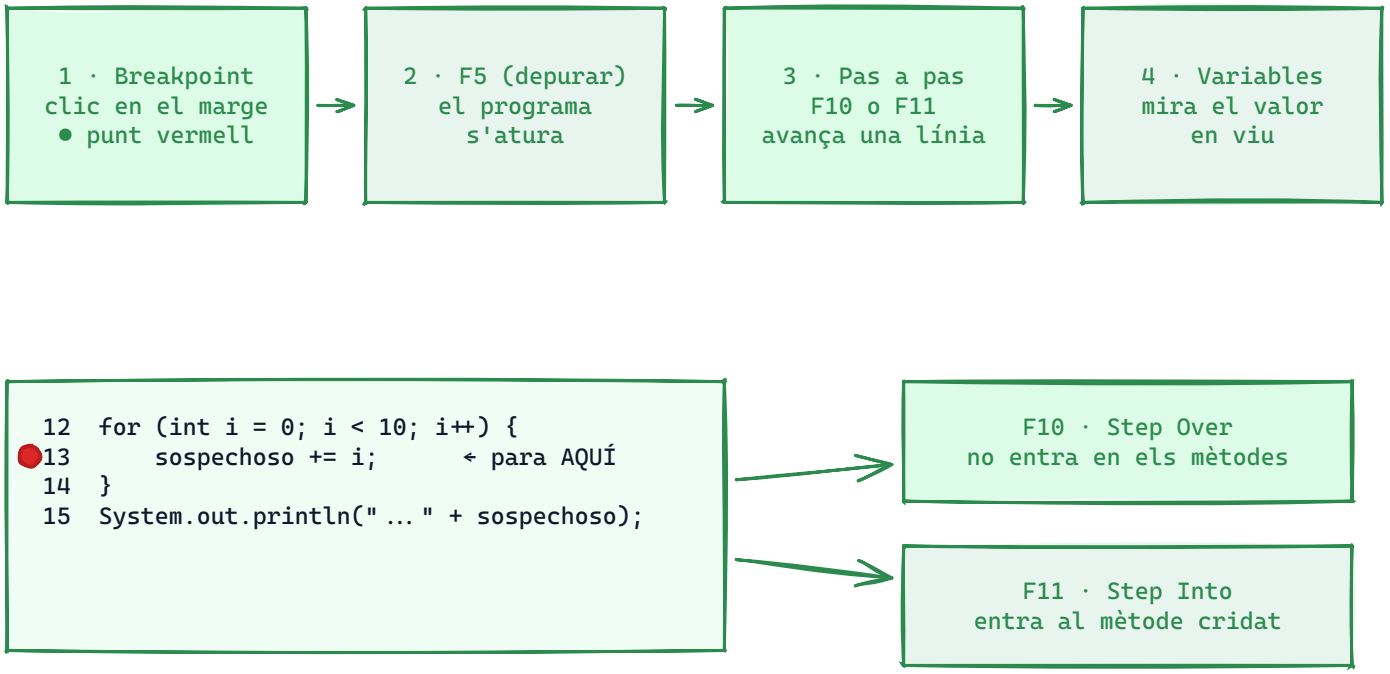
Eina	Drecera (VS Code)	Què fa
Breakpoint (punt de ruptura)	Clic a l'esquerra del número de línia	Li dius a Java "para AQUÍ, vull veure què passa".

Eina	Drecera (VS Code)	Què fa
Step Over	F10	“Executa aquesta línia però no em contes els detalls interns”.
Step Into	F11	“Executa aquesta línia I porta’m dins d’aquella crida, vull espiar”.
Variables / Watch	Panell lateral de depuració	“Ensenya’m el valor de la variable ARA MATEIX”.

- **Breakpoint**: Marca una línia amb un punt vermell. El programa es deté just abans d’executar-la i espera les teves ordres.
- **Step Over (F10):** Executa la línia completa de colp, sense entrar en els mètodes que crida.
- **Step Into (F11):** Entra dins del mètode cridat per a veure les seves línies una a una.
- **Variables / Watch:** El panell esquerre en la vista de depuració de VS Code (Run and Debug) que mostra les variables vives i els seus valors actuals. En Watch pots afegir expressions concretes per a avaluar-les en temps real.

⚠️ **Advertència:** Si et perds dins d’un mètode alié amb Step Into, prem Step Out (`Shift + F11`): surt del mètode i torna al punt on el vas cridar. És el botó “he entrat sense voler, traieu-me d’aquí”.

Depurar pas a pas (VS Code)



● El punt vermell atura el programa abans d'aquella línia

🏫 Exemple guiat: el cas del sospitós en VS Code

Aquí tens un programa perfecte per a estrenar el depurador:

```
public class DetectivesDeCodigo {
    public static void main(String[] args) {
        int sospechoso = 0;
        for (int i = 0; i < 10; i++) {
            sospechoso += i; // Posa un breakpoint ací
        }
        System.out.println("El culpable es: " + sospechoso);
    }
}
```

El bucle `for` el veuràs a fons en la U04, però ho pots intuir: repeteix la línia de dins mentre `i` sigui menor que 10, amb `i` valent 0, 1, 2… La línia clau és `sospechoso += i` (que és el mateix que `sospechoso = sospechoso + i`).

L'exercici de detectiu:

1. Posa un breakpoint en la línia de `sospechoso += i` (fes clic a l'esquerra del número de línia; apareixerà un punt vermell fix).
2. Executa en mode depuració: prem `F5` o fes clic a la icona del bitxo amb el triangle ▶ (Run and Debug / Start Debugging) en la barra flotant superior o en la pestanya esquerra.
3. El programa es deté i aquella línia s'il·lumina en groc. Mira el panell Variables a l'esquerra: veuràs que `sospechoso` val 0 i `i` val 0.
4. Prem Step Over (`F10`) diverses vegades i observa com canvien `sospechoso` i `i` en el panell lateral en cada vegada.

🔄 Què hauries de veure

Valors de `sospechoso` en cada parada: 0, 1, 3, 6, 10, 15, 21, 28, 36, 45 i, en acabar el bucle, 55. El valor final imprés en la terminal de VS Code és "El culpable es: 55". Si en el teu depurador el valor no coincideix, tens al davant un bug real per a diagnosticar. Això és depurar.

🧠 La regla d'or del depurador

Quan alguna cosa falla, no endevinis: observa. El procés és sempre el mateix:

1. Reprodueix la fallada (que ocorri sempre que puguis).
2. Posa un breakpoint abans de la zona sospitosa.
3. Avança amb `F10` i observa les variables en el panell lateral fins a localitzar la línia on el valor es torç.
4. En veure on es desvia el valor, hauràs trobat el bug.

💡 **Consell:** Si el programa s'executa fins al final sense detindre's, és que el breakpoint està en una línia que mai s'aconsegueix (per exemple, dins d'un mètode que ningú crida o en una condició `if` que no es compleix). El breakpoint no es dispara: una altra pista de detectiu.

🎯 Mini-comprovació

1. Quina diferència hi ha entre Step Over (F10) i Step Into (F11)?
2. Si poses un breakpoint i el programa no es deté, què sospites?
3. En quin mode cal executar el programa perquè els breakpoints funcionen?

🔄 Respostes

1. Step Over (F10) executa la línia sencera sense entrar en els mètodes que crida; Step Into (F11) entra dins del mètode per a veure les seves línies una a una.
2. Que el breakpoint està en una línia que mai s'executa (codi mort, mètode no cridat, condició que no es compleix).
3. En mode depuració (prement `F5` / icona del bitxo 🐞 Start Debugging), no en mode execució normal (Run Without Debugging).

✅ Resum en 3 frases

1. El depurador para el programa on tu marques (breakpoint) i et mostra les variables en viu en el panell lateral de VS Code.
2. Step Over (F10) no entra en els mètodes; Step Into (F11) sí; el panell Variables / Watch et mostra els valors en temps real.
3. Davant un bug, no endevinis: reprodueix, para, avança i observa fins a trobar on es torç el valor.

🐛 Vocabulari ràpid

Terme	Idea general
Bug	Un error en el programa
Debug	Procés de trobar i arreglar bugs
Breakpoint	Marca (punt vermell) on el programa es deté
Step Over / Into	Avançar sense entrar (F10) / entrant (F11) en mètodes
Watch / Variables	Panell lateral de VS Code que mostra variables i expressions

05 — Comentaris i documentació

//, /* */, /** */ i Javadoc 📝

> 🗺️ **Estàs en: 🚀 U02 · Introducció a Java i l'entorn de treball** → 05 · Comentaris i documentació

Un comentari és un missatge que et deixes a tu mateix (o a uns altres). L'ordinador l'ignora completament: és per als humans, no per a les màquines.

El codi explica *què* fa la màquina; els comentaris expliquen *per què* ho fa. I el *per què* és or pur: sis mesos després, aquell comentari t'estalviarà hores de "*què* estava pensant quan vaig escriure això?".

📝 Els tres tipus de comentaris

Java té tres formes d'escriure comentaris, cada una amb el seu ús:

Tipus	Sintaxi	Ús
D'una línia	`// text`	Notes ràpides al costat del codi per a aclarir una línia o llista de tasques.
De bloc	`/* text */`	Explicacions llargues, algorismes complexos o desactivar temporals de codi.
Javadoc	`/** text */`	Documentació formal i estructurada de classes, mètodes i atributs.

```
// Comentari d'una línia: "Ací va la màgia"

/*
Comentari de diverses línies:
"Si això funciona, no ho toques.
Si no funciona, no ho toques tampoc.
Ja cridarem algú."
*/

/**
 * Comentari Javadoc (l'elegant):
 * Serveix per a generar documentació automàtica.
 * @param args arguments de la línia de comandes
 * @return res, açò és void, no te n'adones?
 */
```

💡 **Detall pràctic:** els comentaris poden anar enmig d'una línia sense problema. `System.out.println(/* "Cuatro" */ "Cinco")` imprimeix `Cinco`: el comentari s'ignora i la resta de la línia segueix viva.

🗝️ El consell que t'estalviarà hores: comenta el "per què", no el "què" 🔗

Aquest és el consell més rendible de tota la unitat:

```
int i = 0; // Declare i amb valor 0 ← MAL: el codi ja ho diu
int i = 0; // Comencem des de 0 perquè l'usuari no ha polsat res ← BÉ
```

Escriure `// Declare i amb valor 0` és com posar "Obro la porta" en una porta. El codi ja diu això. El comentari útil afegeix informació que **el codi no pot expressar**: el context, la intenció, la decisió de disseny.

- ❌ `int temperatura = 30; // temperatura val 30`
- ✅ `int temperatura = 30; // Refresca per davall de 25 segons el cap`

🧠 **Truc de memòria:** si el comentari descriu la mateixa acció que veus en el codi, esborra'l. El bon comentari respon a *per què?*, mai a *què?*.

📖 Javadoc: documentació que es genera sola

Javadoc és un comentari especial (`/** ... */`) que col·loques **just abans** d'una classe o d'un mètode. L'eina `javadoc` (inclosa en el JDK) ho converteix en pàgines de documentació automàtiques, iguals a les oficials de Java.

```
/**
 * Classe que representa un alumne del curs de programació.
 *
 * @author Sergi Garcia
 * @version 1.0
 */
public class SobreMi {

    /**
     * Punt d'entrada del programa.
     *
     * @param args arguments rebuts des de la línia d'ordres
     */
    public static void main(String[] args) {
        System.out.println("Em dic Sergi i m'agrada la programació");
    }
}
```

💡 Aprofundint en Javadoc: la documentació professional

Javadoc no és un simple comentari per a tu o els teus companys de classe; és l'eina oficial del JDK que llegeix el teu codi i genera automàticament una pàgina web (HTML) amb la documentació tècnica del teu projecte, exactament igual que la documentació oficial de la llibreria de Java.

Quan escrius un bloc Javadoc (comença per `/**` i acaba amb `*/`), pots utilitzar etiquetes especials (tags) que comencen per @ per a estructurar la informació:

- **`@param nomParametre`**: Explica per a què serveix un paràmetre d'entrada que rep el mètode.
- **`@return`**: Descriu què torna o calcula el mètode (si no és void).
- **`@author`**: Indica l'autor de la classe o del codi.
- **`@version`**: Especifica la versió del mòdul o classe.

- **`@see`**: Afegeix una referència o enllaç a una altra classe o mètode relacionat.
- **`@throws Exception`**: Documenta quin tipus d'excepció o error pot llançar el mètode.

Per què és tan útil en el teu dia a dia?

A més de generar la web, els IDEs moderns com VS Code o IntelliJ IDEA llegeixen aquests comentaris Javadoc i, en passar el cursor sobre qualsevol mètode o utilitzar l'autocompletat, et mostren una finestra emergent amb tota la documentació, paràmetres i explicacions en temps real sense necessitat d'obrir l'arxiu original.

Per a generar-la: `javadoc SobreMi.java` en la terminal i es crearan els `.html` amb la documentació. Prova-ho: és com tenir un manual d'instruccions fet per tu mateix.

💡 **Consell:** en la vida real, els comentaris Javadoc en els mètodes solen "caure" en les rúbriques. A més, en el treball, els que documenten dormen millor… i els que no, tornen al codi a les 3 de la matinada amb cara de pànic.

🏫 Exemple guiat: documentar un programa complet 🔗

Ho aplicarem tot a un programa real:

```java
/**
 * Classe que saluda de distintes formes.
 *
 * @author Alumne DAW
 * @version 1.0
 */
public class Saludo {

    /**
     * Mètode principal: imprimeix els dos missatges de la classe.
     *
     * @param args arguments de línia d'ordres (no usats ací)
     */
    public static void main(String[] args) {
        // Primer missatge: benvinguda al curs
        System.out.println("Bienvenidos al curso de Programación");
        /*
         * Segon missatge:
         * usem println per a saltar de línia i deixar la consola neta
         */
        System.out.println("Este curso va a molar");
    }
}
```

Fixa't que cada comentari aporta alguna cosa: el Javadoc documenta l'API, el `//` explica la intenció i el `/* */` justifica una decisió de format. Cap no repeteix el que el codi ja diu.

🎯 Mini-comprovació

1. Quina diferència hi ha entre `//`, `/* */` i `/** */`?
2. Què imprimeix aquest programa?

```java
public class Comentarios {
    public static void main(String[] args) {
        // System.out.println("Uno");
        System.out.println("Dos");
        /* System.out.println("Tres"); */
        System.out.println(/* "Cuatro" */ "Cinco");
    }
}
```

3. És bon comentari `// x = 10`?

🔄 Respostes

1. `//` és d'una línia; `/* */` és de bloc (diverses línies); `/** */` és Javadoc, que l'eina `javadoc` converteix en documentació automàtica.
2. Imprimeix `Dos` i `Cinco`. La línia amb `//` i la del bloc `/* */` s'ignoren, i en l'última el comentari intern s'elimina, però "Cinco" segueix sent l'argument del `println`.
3. No: el codi ja mostra que `x` val 10. Comenta el **per què**, no el què.

✅ Resum en 3 frases

1. Els comentaris són per a humans: `//`, `/* */` i `/** */`, segons el que necessites.
2. Comenta el **per què**, no el què: el codi ja diu què fa.
3. **Javadoc** genera documentació automàtica amb `@author`, `@version`, `@param` i `@return`.

🐛 Vocabulari ràpid

Terme	Idea general
Comentari	Text que el compilador ignora completament en generar el bytecode.
Javadoc	Comentari estructurat que genera documentació HTML automàtica.
Etiqueta	`@param`, `@return`… metadades per a descriure peces del codi.
Documentació	Explicació escrita sobre el funcionament del codi per a altres desenvolupadors.

Terme	Idea general
Compilador	Traductor (`javac`) que ignora els comentaris i crea l'arxiu `.class`.

06 — Arguments de línia d'ordres

L'array args, com llegir-lo i usar-lo 📬

> 🗺️ **Estàs en:** 🚀 **U02 · Introducció a Java i l'entorn de treball** → 06 · Arguments de línia d'ordres

El paràmetre "args" del "main" és una butxaca on Java fica tot el que escriguis després del nom del teu programa en executar-lo.

Quan escrius `java MiPrograma hola mundo`, aquelles paraules soltes (`hola`, `mundo`) no són casuals: Java les guarda en l'array `args` perquè el teu programa les pugui usar.

🎒 Què és "args"?

Recorda la firma del main:

```
public static void main(String[] args)
```

`String[] args` significa: "un array de cadenes de text anomenat args". Un array és una llista de coses (els veuràs a fons més endavant en el curs). Aquí, la llista conté **cada paraula que vas escriure després del nom de la classe en executar**.

```
public class UsoDeArgumentos {
    public static void main(String[] args) {
        System.out.println("Has escrito " + args.length + " palabras:");
        for (int i = 0; i < args.length; i++) {
            System.out.println("Palabra " + (i + 1) + ": " + args[i]);
        }
    }
}
```

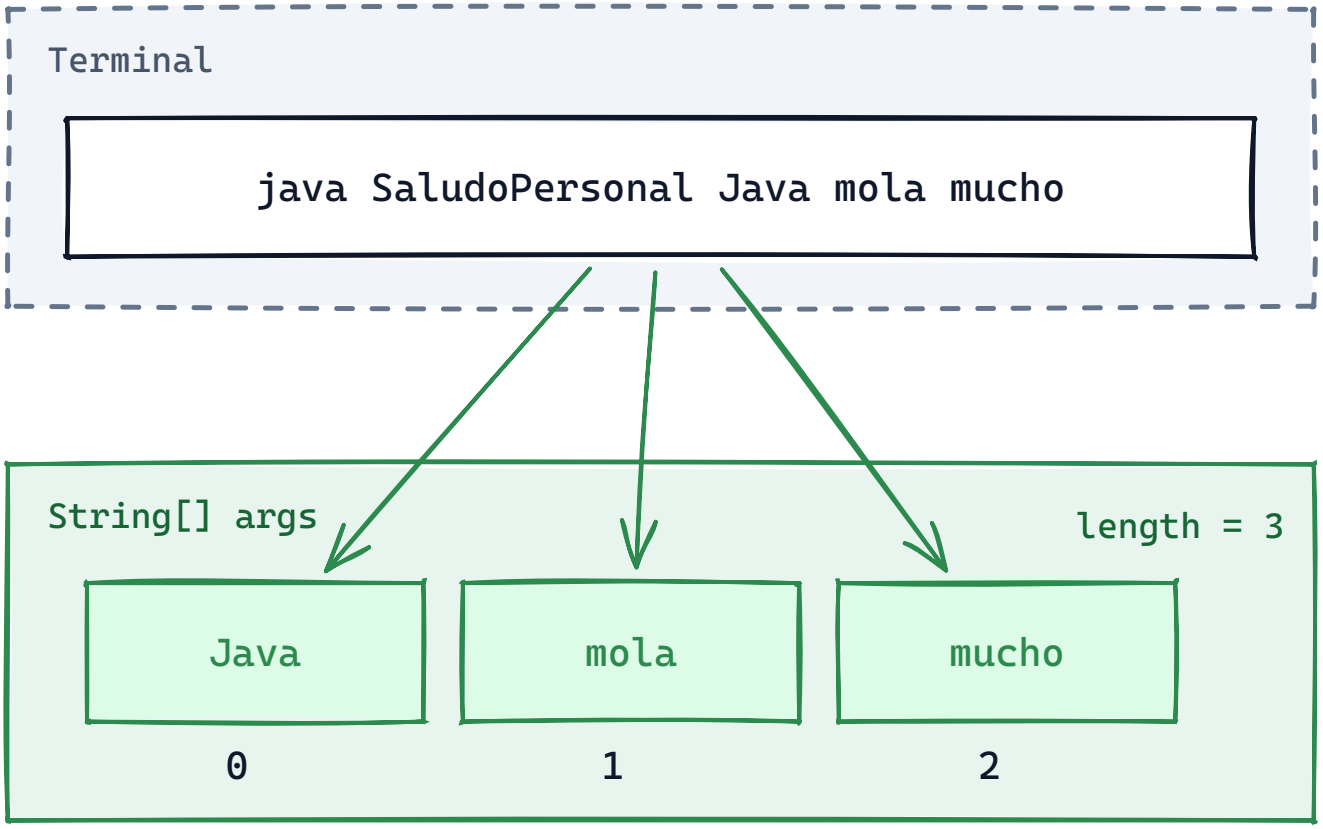
Si executes:

```
> java UsoDeArgumentos Java mola mucho
```

Veuràs:

```
Has escrito 3 palabras:
Palabra 1: Java
Palabra 2: mola
Palabra 3: mucho
```

L'array args: de la terminal al programa



🔢 Com s'indexen les paraules

En Java, les llistes comencen a comptar en 0, com quasi tot en programació:

Índex	Valor
`args[0]`	"Java"
`args[1]`	"mola"
`args[2]`	"mucho"
`args.length`	3 (quants n'hi ha en total)

- `args.length` et diu quants arguments s'han passat (en aquest exemple, 3).
- `args[0]` és el primer element, `args[1]` el segon… i així successivament.
- Si executes el programa sense arguments, `args.length` valdrà 0 i l'array estarà buit.

⚠️ **Advertència:** compte amb l'error de l'aprenent: `args[0]` és el **primer** argument, no "l'argument número 0". El **primer** compta com a índex 0. És com les plantes d'un edifici: la baixa és la 0.

🧠 Per a què serveix passar arguments?

Passar arguments per línia d'ordres és la forma més antiga i directa de "configurar" un programa sense tocar el codi:

- **Dades d'entrada**: `java Calculadora 5 3` → el teu programa rep 5 i 3.
- **Modes d'execució**: `java App --verbose` o `java App --silencioso`.
- **Arxius**: `java Convertidor entrada.txt salida.txt`.
- **Programes reals**: els comandaments de la terminal (com `git status` o `ls -la`) són exactament això: programes que reben arguments.

En la pràctica, l'array `args` apareix cada vegada que vulguis que el teu programa parli amb "el món exterior" **abans** d'arrancar. Més endavant usaràs el `Scanner` (UO3) per a demanar dades durant l'execució; els args són la via d'entrada **abans** d'executar-se.

🏫 Exemple guiat: el programa que et saluda

Anem a fer que el teu programa sàpiga el teu nom sense preguntar-ho:

```
public class SaludoPersonal {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("Hola, " + args[0] + ". ¡Bienvenido al curso!");
        } else {
            System.out.println("Hola, desconocido. ¿Olvidaste tu nombre?");
        }
    }
}
```

Si executes `java SaludoPersonal Sergi`, saluda a Sergi. Si executes `java SaludoPersonal`, et recorda que no vas passar arguments. El `if` aquí és un aperitiu de les estructures de control que es veuran en futures unitats.

💡 **Detall pràctic:** quan executes des de l'IDE (botó ▶), pots configurar els arguments en el menú *Run → Edit Configurations → Program arguments*. Escriu aquí `Sergi 20` i prem Run: el teu programa els rebrà igual que si els teclejaren en la terminal.

🎯 Mini-comprovació

1. Si executes `java MiPrograma uno dos tres`, quant val `args.length` i què conté `args[2]`?
2. Què passa si accedeixes a `args[5]` quan només hi ha 3 arguments?
3. Com saludes a la primera paraula que rebi el teu programa?

🔄 Respostes

1. `args.length` val **3** i `args[2]` conté "tres" (els índexs comencen en 0).
2. Reventarà amb `ArrayIndexOutOfBoundsException`: estàs demanant un element que no existeix.
3. Amb `args[0]`, per exemple: `System.out.println("Hola, " + args[0]);`.

✅ Resum en 3 frases

1. `args` és l'array de cadenes que Java crea amb tot el que escrius després del nom de la teva classe.
2. Els índexs comencen en **0**: `args[0]` és el primer argument i `args.length` diu quants n'hi ha.
3. Els arguments són la forma de donar **dades d'entrada** a un programa abans que arranqui.

🐛 Vocabulari ràpid

Terme	Idea general
Argument	Una paraula o valor que li passes al programa en el moment d'executar-lo.
args	L'array (llista) de cadenes de text on Java guarda tots els arguments rebuts.
Array	Una estructura de dades que guarda una llista d'elements ordenats i numerats des de 0.
args.length	La propietat que indica la quantitat total d'arguments que té l'array.
Línia d'ordres	La finestra de terminal o consola on escrius les ordres com `java`, per a executar programes.

07 — El compilador i els seus errors

Compilar vs executar, errors típics del principiant ⚠️

> 🗺️ **Estàs en: 🚀 U02 · Introducció a Java i l'entorn de treball** → 07 · El compilador i els seus errors

El compilador no t'odia: és un professor de llengua puntillós que marca cada coma mal posada. Aprendre a llegir els seus missatges és aprendre a programar.

Tota la teva carrera estarà plena de missatges d'error. Els aprenents els temen; els professionals els llegeix com qui llegeix una carta. En aquest punt aprendràs a entendre què et diu `javac` (i per què Java és tan puntillós).

🔍 Compilar ≠ executar

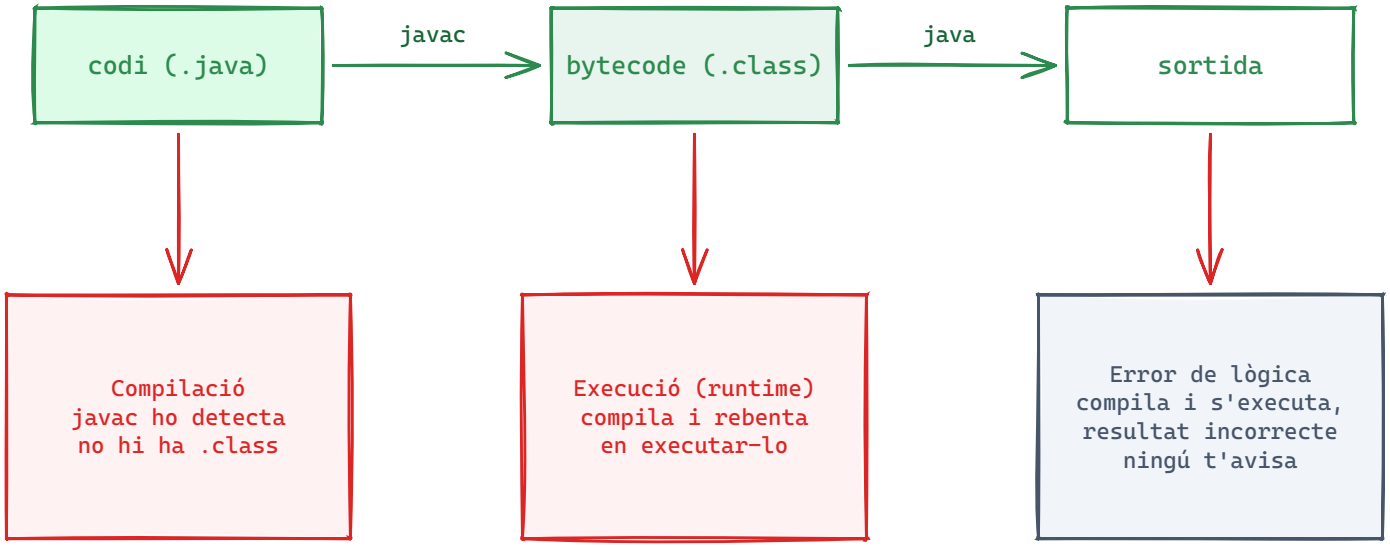
Dos moments distints, dues famílies d'errors:

Moment	Comandament	Què ocorre
Compilar	`javac MiPrograma.java`	El compilador tradueix el teu codi a bytecode. Si hi ha errors de sintaxi, es queixa aquí.
Executar	`java MiPrograma`	La JVM executa el bytecode. Aquí poden aparèixer errors de runtime (en temps d'execució).

```
El teu codi (.java)  --javac-->  bytecode (.class)  --java-->  eixida
       (errors de compilació ací)                (errors d'execució ací)
```

- **Errors de compilació**: el compilador detecta que has escrit malament alguna cosa. Et diu la **línia exacta** i el motiu. No es genera el `.class`.
- **Errors d'execució (runtime)**: el programa compila, però rebenta en córrer (per exemple, `ArrayIndexOutOfBoundsException` del punt 6).
- **Errors de lògica**: tot "funciona", però el resultat és incorrecte. El més perillós: ni el compilador ni el runtime t'avisen. Aquí entra el depurador del punt 4.

Tres famílies d'errors (i on apareixen)



Compila javac · Executa la JVM · La lògica només la caça el depurador

---

😤 L'error de l'aprenent: 4 errors en un programa

Aquest codi està fatal. Troba els 4 errors abans de mirar la solució:

```
Public class Calculadora
public static void main(string[] args) {
    System.out.println("Suma: " + 5 + 3)
    System.out.println("Resta: " + (5 - 3));
} }
```

🔄 Els 4 errors

1. `Public` hauria de ser **`public`** (minúscula). Java és sensible a les majúscules.
2. Falta **`{`** després de `Calculadora`. La classe necessita la seva clau d'obertura.
3. `string[] args` hauria de ser **`String[] args`** (la S majúscula importa: `String` és una classe, no existeix `string`).
4. Falta el **`;`** al final de la primera línia del `println`.

Java és molt puntillós, com un professor de llengua amb les comes: `public`, `String` i `System` comencen en majúscula o majúscules concretes, i cada frase acaba en `;`.

🔎 Com llegir un missatge d'error de javac

Quan alguna cosa no compila, `javac` et mostra alguna cosa com:

```
Calculadora.java:3: error: ';' expected
System.out.println("Suma: " + 5 + 3)
                                   ^
1 error
```

Tres dades clau:

1. **"Calculadora.java:3"** → l'arxiu i la **línia** on està el problema (mira la ^).
2. **"error: ';' expected"** → el *què*: esperava un punt i coma.
3. **El "^"** → el punt exacte on s'ha atascat.

💡 **Consell:** l'error sol trobar-se en la línia marcada, però de vegades és l'**anterior**: si falta un `{` o un `;`, javac de vegades se n'adona una línia més tard. Comença mirant la línia del ^ i, si no, puja una.

🚨 Errors típics de l'aprenent (i el seu remei)

Error	Missatge típic	Remei
Public en lloc de public	`class, interface, or enum expected`	Els noms i paraules clau en Java són exactes: `public`, `class`, `String`, `System`.
Falta ;	`';' expected`	Tota instrucció o ordre en Java ha d'acabar obligatòriament en `;`.
Falta { o }	`reached end of file while parsing` o `class, interface...`	Comprova que cada clau d'obertura `{` tingui la seva corresponent clau de tancament `}`.
string en lloc de String	`cannot find symbol`	`String` és una classe i sempre s'escriu amb la S inicial majúscula.

Error	Missatge típic	Remei
Nom de classe diferent de l'arxiu	`class X is public, should be declared in a file named X.java`	La classe public ha de tenir exactament el mateix nom que l'arxiu `.java` (respectant majúscules i minúscules).
system en lloc de System	`cannot find symbol: variable system`	`System` va amb S majúscula, mentre que `out` i `println` van en minúscules.

🧠 **Truc de memòria:** els noms de les *classes* (`String`, `System`, `Scanner`…) comencen en majúscula; els noms de *variables i mètodes* (`out`, `println`, `main`) en minúscula. Quan dubtes, mira com ho va escriure Java.

🧩 EL LÍO: el codi remenat

El teu cap ha deixat aquest codi fet un desastre. Les línies estan barrejades. Ets capaç d'ordenar-les perquè sigui un programa Java vàlid que imprimeixi "La suma es: 8"?

```
System.out.println("La suma es: " + (a + b));

int a = 5;

public class CalculoLioso {

System.out.println("Calculando...");

public static void main(String[] args) {

int b = 3;
```

Pista: busca primer on comença la classe i el mètode `main`.

🔄 Solució

```java
public class CalculoLioso {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        System.out.println("Calculando...");
        System.out.println("La suma es: " + (a + b));
    }
}
```

La classe és el contenidor, el main és la porta d'entrada, les instruccions van dins del main, en ordre. Com una recepta: els ingredients es declaren abans d'usar-se.

🎯 Mini-comprovació

1. Què significa `Calculadora.java:3: error: ';' expected`?
2. Per què `Public` (amb P majúscula) dona error?
3. Quina diferència hi ha entre un error de compilació i un de lògica?

🔄 Respostes

1. Que en l'arxiu `Calculadora.java`, en la línia 3, `javac` esperava un `;` i no el va trobar (mira la ^).
2. Perquè Java és sensible a les majúscules: la paraula reservada és `public`, en minúscula exacta.
3. El de compilació impedeix generar el `.class` (ho detecta `javac`). El de lògica compila i executa, però el resultat és incorrecte: ningú t'avisa, només el depurador (o el sentit comú).

✅ Resum en 3 frases

1. Compilar (`javac`) i executar (`java`) són moments distints amb errors distints.
2. El missatge d'error et diu **arxiu, línia i motiu**: aprèn a llegir-lo en lloc de témer-lo.
3. Les fallades clàssiques (majúscules, `;`, `{}`, nom de la classe) es corregeixen amb un cop d'ull… una vegada saps què mirar.

🐛 Vocabulari ràpid

Terme	Idea general
Compilar	Traduir el codi font (`.java`) a bytecode (`.class`) que entén la JVM.
Error de compilació	El compilador detecta una fallada de sintaxi i es nega a generar el bytecode.
Error de runtime	El programa compila bé, però falla o es bloqueja durant l'execució.
Error de lògica	El programa s'executa sense errors, però el resultat no és el que s'esperava.
Sensible a majúscules	Java distingeix majúscules de minúscules (Case-sensitive): "Public" no és el mateix que "public".

08 — L'IDE i el teu flux de treball

Projectes, dreceres, crear/executar/depurar 🛠️

> 🗺️ **Estàs en:** 🚀 **U02 · Introducció a Java i l'entorn de treball** → 08 · L'IDE i el teu flux de treball

---

L'IDE (Entorn de Desenvolupament Integrat) és la teva navalla suïssa: editor, compilador, executor i depurador, tot en una sola finestra.

En el punt 2 vas instal·lar el JDK i li vas fer una primera ullada a l'IDE/Editor. Ara vas a muntar el teu flux de treball diari: com s'organitza un projecte, com crear classes, executar-les, depurar-les i moure't amb dreceres en VS Code sense tocar el ratolí.

🗂️ Anatomia d'un projecte en VS Code

Quan crees un projecte Java en VS Code, l'extensió et genera una estructura de carpetes neta. Les importants són:

```
MiPrimerCurso/
├── src/          ← EL TEU codi (.java) viu ací
│   └── HolaMundo.java
└── bin/ (o target)  ← el bytecode (.class) compilat automàticament
    └── HolaMundo.class
```

- **src** (source): Els teus arxius `.java`. És el que tu escrius i el que es puja a Git.
- **bin / target**: El que genera el compilador en segon pla. No es toca a mà i normalment ni es puja a Git.

💡 **Detall pràctic:** No edites mai els `.class`. Si esborres la carpeta `bin`, VS Code la regenerarà en compilar en segon pla. El teu codi font en `src` és l'única font de veritat.

🔁 El cicle de treball (el teu nou bucle de vida)

Programar és un bucle de quatre passos que repetiràs milions de vegades:

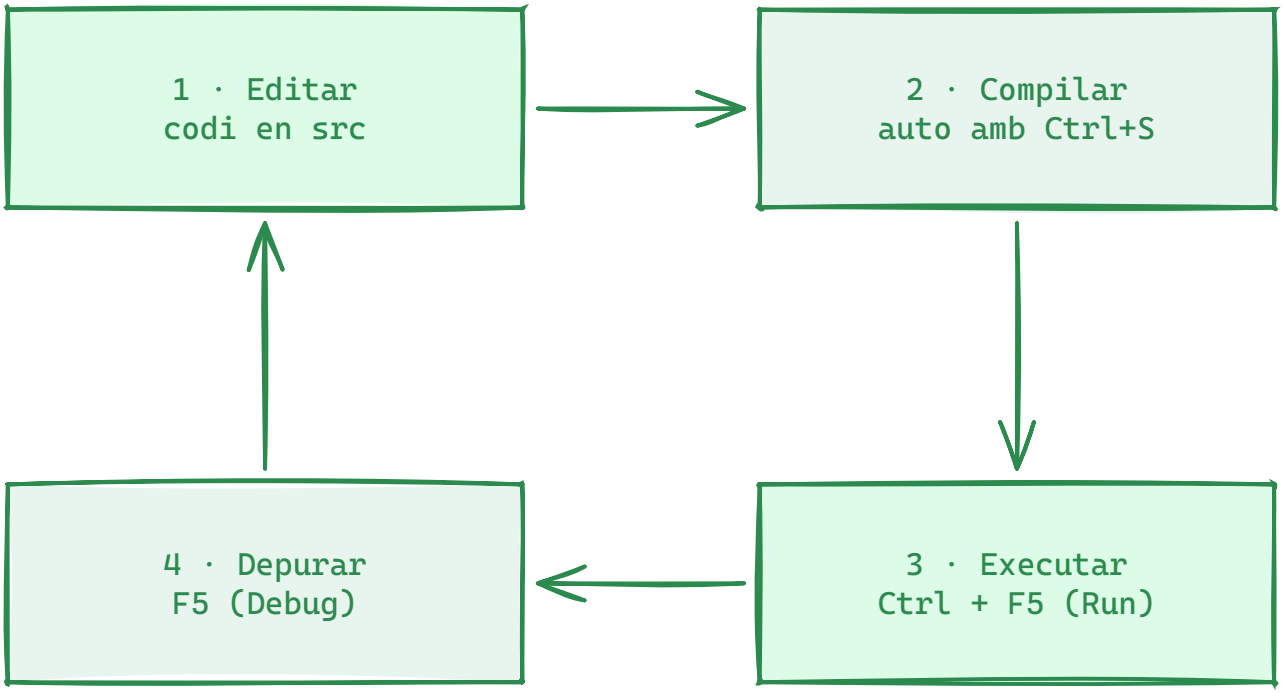
1. **Editar**: Escrius o canvies codi en `src`.

2. **Compilar**: VS Code compila automàticament en guardar (`Ctrl + S`). Aquí es marquen els errors de sintaxi en vermell.
3. **Executar**: Polses `Ctrl + F5` (o el botó ▶ Run). Aquí es detecten els errors de runtime.
4. **Depurar**: Si la lògica falla, polses `F5` (o Debug) i comença el detectiu del punt 4.

Editar → Compilar (Auto) → Executar → Depurar → (Tornar a Editar)

⚠️ **Advertència:** Executar sense depurar (`Ctrl + F5`) i en mode depuració (`F5`) NO són el mateix. Executar normal ignora els breakpoints; Debug els respecta. Si poses un punt vermell i executes sense depurar, el programa no es detindrà.

El cicle de treball (el teu bucle diari)



El bucle es repeteix milions de vegades

⚠ Run ≠ Debug

Ctrl + F5 (Run): ignora els breakpoints

F5 (Debug): els respecta i s'hi atura

⌨️ Les dreceres de VS Code que et faran semblar un pro 🔗

Deixa el ratolí en pau. Aquestes són les dreceres essencials per a Java en VS Code:

Drecera (Windows)	Acció
`main` + `Tab` (o `psvm`)	Escriu l'esquelet `public static void main(String[] args) {}`
`sysout` + `Tab` (o `sout`)	Escriu `System.out.println()`
`Ctrl` + `F5`	Executar (Run Without Debugging)
`F5`	Executar en mode depuració (Start Debugging)
`F10` / `F11`	Step Over / Step Into (depurador)
`Ctrl` + `/`	Comentar / descomentar la línia seleccionada
`Shift` + `Alt` + `↓`	Duplicar la línia cap avall
`F12`	Anar a la definició (Go to Definition)
`F2`	Canviar el nom d'un símbol/variable en tot el projecte (Rename)

🧠 **Truc de memòria**: `main` i `sysout` són els dos snippets que més escriuràs en el curs. Escrius aquelles lletres, premis Tab i VS Code te les ompli de cop.

🧩 Autocompletat (IntelliSense): el teu company silenciós 🔗

L'extensió de Java mentre escrius et suggereix opcions, ressalta errors en vermell abans d'executar i pot arreglar problemes amb un clic (la icona de la bombeta 💡 o `Ctrl` + `.`):

- Escrius `Syste` i VS Code t'ofereix `System` (amb la S majúscula que tant costa al principi).
- Et falta una variable o importació i la bombeta 💡 t'ofereix "Import 'java.util.Scanner'" o la correcció ràpida.
- Selecciones una variable, premis `F2` i li canvies el nom: VS Code actualitza totes les seves aparicions en el projecte (això és refactoritzar).

L'autocompletat no és trampa: és la raó per la qual la gent usa VS Code o un IDE en lloc d'un bloc de notes. El teu codi surt amb menys errors tontos perquè l'editor et corregeix mentre penses.

🏫 Exemple guiat: de zero a executar en 60 segons en VS Code

1. **Nou projecte:** Prem `Ctrl + Shift + P`, escriu `Java: Create Java Project`, tria `No build tools`, selecciona la carpeta de destinació i anomena el projecte `MiPrimerCurso`.
2. **Nova classe:** Dins de l'explorador esquerre, fes clic dret sobre la carpeta `src` → New File → anomena'l `HolaMundo.java`.
3. Escriu `main` i prem Tab (apareix el mètode main). Dins, escriu `sysout` i prem Tab:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola des de VS Code!");
    }
}
```

4. Prem `Ctrl + F5` (Run). Mira la pestanya Terminal en la part inferior de VS Code.
5. Posa un breakpoint en el `println` (clic a l'esquerra del número de línia), prem `F5` (Debug) i observa el panell lateral esquerre de variables.

Si has arribat al pas 5 sense pànic, el teu flux de treball està muntat. A partir d'aquí, el curs sencer és omplir aquest esquelet de contingut.

🎯 Mini-comprovació

1. On viuen els teus arxius `.java` i on els `.class`?
2. Quina diferència hi ha entre executar amb `Ctrl + F5` (Run) i amb `F5` (Debug)?
3. Què escriu `main + Tab`?

🔄 Respostes

1. Els teus `.java` viuen en `src`; els `.class` compilats en `bin` (o `target`). No edites els `.class`.
2. `Ctrl + F5` executa normal (ignora els breakpoints); `F5` executa en mode depuració (els respecta).
3. L'esquelet `public static void main(String[] args) {}` complet.

✅ Resum en 3 frases

1. Un projecte té `src` (el teu codi) i `bin` (bytecode): VS Code compila per tu en segon pla en guardar.
2. El teu bucle diari és editar → compilar → executar → depurar, i Run (`Ctrl + F5`) i Debug (`F5`) no són el mateix.
3. Les dreceres (`main`, `sysout`, `F10/F11`) i l'autocompletat (IntelliSense) et fan treballar més ràpid i amb menys errors.

🐛 Vocabulari ràpid

Terme	Idea general
VS Code / IDE	Editor + eines de Java en una sola aplicació
Projecte	La carpeta arrel que agrupa el teu codi (`src`) i configuracions
src	Carpeta de codi font on escrius els teus arxius `.java`
Refactoritzar	Modificar o reestructurar el codi mantenint el seu comportament exactament igual
Terminal / Consola	La finestra inferior on s'imprimeix la sortida dels teus programes

09 — Repàs

Sé el Codi, Fireside, Qui Soc, Laboratori, Crucigrama… 🧩

> 🗺️ **Estàs en: 🚀 UO2 · Introducció a Java i l'entorn de treball** → 09 · Repàs

En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈

⭐ Tracta de pensar com el codi, *my friend…* 🔗

Ets la JVM. Acaben de donar-te aquest programa per a executar:

```
public class Misterio {
    public static void main(String[] args) {
        System.out.println("Café " + 1 + 2);
        System.out.println("Café " + (1 + 2));
    }
}
```

Què imprimeixes per pantalla? Tria sàviament:

1. **Cafè 3 i Cafè 3** → La JVM sempre suma els números, ocorri el que ocorri. ❌
2. **Cafè 12 i Cafè 3** → En la primera línia, en trobar text abans del +, Java concatena d'esquerra a dreta: "Cafè " + 1 és "Cafè 1", i després + 2 dona "Cafè 12". En la segona línia, els parèntesis forcen la suma aritmètica primer: (1 + 2) = 3, donant "Cafè 3".
3. **Cafè 1 2 i Cafè 12** → Els parèntesis no existeixen per a la JVM. ❌

🔄 Solució 🔗

L'opció 2. Quan un operador + mescla text i números, Java concatena (uneix cadenes). Tanmateix, els parèntesis `(1 + 2)` obliguen a realitzar primer la suma aritmètica. Aquest és el clàssic exercici que separa els qui han treballat la unitat dels qui han dormit.

🔥 Fireside Chat: JDK vs JRE 🔗

Dos veterans del cafè discuteixen al costat de la tassa fumejant.

- **JRE**: — Mira, jo soc l'entorn d'execució (Java Runtime Environment). Porto la JVM, les llibreries, tot el necessari per a executar Java. Sense mi, el teu `.class` corre ni de broma.
- **JDK**: — Executar, executar… I qui crea el `.class`? Jo. Porto el compilador `javac`, el generador de Javadoc, l'eina `jar`… Jo soc el kit complet de desenvolupament (Java Development Kit). Tu ets només la meitat preparada.
- **JRE**: — Escolta, que sense mi tu ets un catàleg d'eines sense endollar. Compiles un programa i… on l'executes? A casa meva!
- **JDK**: (Sarcàstic) — Ah, clar! Com si jo no et portés dins. Quan m'instal·len a mi, tu vens inclòs. Ets com el regal que ve en el pack gran.
- **JRE**: — D'acord, però per a l'usuari final que només vol utilitzar aplicacions Java ja fetes, a qui criden? A mi! Soc més lleuger i ocupi menys espai.
- **JDK**: — I sense compilador. O sigui, un cuiner sense ganivets. Parlarem quan vulguis crear alguna cosa.
- **JRE**: (Sospir) — Sempre el mateix argument…

💡 **La lliçó:** El JDK inclou el JRE. Per a programar i crear projectes necessites el JDK. Per a executar programes en la màquina de l'usuari final, en bastaria amb el JRE. Mai et penses que són opcions excloents en un examen.

🕵️ Qui soc?

Endevina quin concepte de la unitat soc:

1. Tradueix el teu codi `.java` a bytecode. Soc un professor de llengua molt puntillós: qualsevol coma o clau mal posada m'ofèn.
2. Soc la màquina de cafè: executo el bytecode i funciono igual en qualsevol sistema operatiu.
3. Soc la porta d'entrada del teu programa. Si canvien el meu nom, Java no troba l'inici i tot es queda a les fosques.
4. Soc el detectiu de VS Code: et deixo parar el programa on vulguis (breakpoints) i espiar les variables pas a pas.

🔄 Respostes

1. El compilador (`javac`): Tradueix `.java` a `.class` i detesta els errors de sintaxi.
2. La JVM (Java Virtual Machine): La màquina virtual que executa el bytecode igual en qualsevol plataforma.
3. El mètode `main`: `public static void main(String[] args)` és el punt d'inici exacte que busca Java.
4. El depurador (debugger): Amb breakpoints, Step Over (`F10`), Step Into (`F11`) i el panell de variables.

😡 CONRAD VS EL MÓN: "El compilador m'odia"

CONRAD, el nostre compilador cascarràbies, opina sobre el clàssic dels principiants:

CONRAD: — ALTRA VEGADA! Ve un alumne i em diu: "CONRAD, no compila". I jo li pregunto: "Val, què diu el missatge d'error?". I em respon: "Ah, no ho sé, no me l'he llegit". Ai, mare meva! Et dono la línia exacta, el motiu i fins i tot la fletxeta `^` assenyalant el lloc exacte, i no ho llegeixes? És com queixar-se que el menjar està calent sense mirar el plat!

I després està el clàssic: `Public` amb majúscula. PER QUÈ? La paraula clau és `public`. En minúscula. Fa dècades que compilo i encara veig `Public`, `string` amb minúscula en lloc de `String`... I les classes que no coincideixen amb el nom de l'arxiu! Si ho posa clar en l'error!

💡 **La lliçó:** Abans de plorar sobre el teclat, llegeix el missatge d'error. Arxiu, línia i motiu. El 90% dels errors dels principiants s'arreglen sols consultant aquella informació. El compilador no t'odia: t'està passant les respostes de l'examen.

⚡ Laboratori de Tortura: el programa que no arranca

- Duració estimada: 20–30 minuts
- Eina: VS Code i un arxiu `.java` nou
- L'escenari: Copia aquest programa en un arxiu anomenat `Tortura.java` i fes que funcioni. Té 3 errors que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte quan l'arregles.

```
public class Tortura
    public static void main(string[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + a + b)
        System.out.println("El producto es: " + (a * b));
    }
}
```

🛠️ La teva tasca

Aconseguir que compili, que s'executi i que tota la sortida sigui correcta.

💡 Pistes (només si et bloques)

1. Falta alguna clau `{` d'obertura en la classe?
2. `string` va en majúscula o minúscula?
3. Falta algun `;` al final de la línia de la suma?
4. Quan aconseguís executar-lo, per què la suma surt `34` en lloc de `7`? Recorda com funciona l'operador `+` amb cadenes de text.

🔄 Solució del Laboratori

```
public class Tortura {
    public static void main(String[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + (a + b)); // Parèntesis per a forçar la suma
        System.out.println("El producto es: " + (a * b));
    }
}
```

Sortida correcta per pantalla:

```
La suma es: 7
El producto es: 12
```

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **El Cafè del Matí**	Cal explicar la diferència entre JVM, JRE i JDK amb l'analogia de la cafeteria sense mirar els apunts.
🏅 **Primer Crit**	Executar el teu primer `HolaMundo.java` des de VS Code sense ajuda.
🏅 **Detectiu de Bugs**	Trobar els 3+1 errors del Laboratori de Tortura sense mirar la solució.
🏅 **Drecera Pro**	Cal escriure un programa en VS Code usant només les dreceres `main + Tab` i `sysout + Tab`.
🏅 **Traductor**	Documentar una classe amb Javadoc (`/** ... */`) i entendre les etiquetes `@param` i `@return`.

🧠 Atreveix-te a pensar

1. Sense executar, què imprimeix aquest programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        System.out.print("Uno ");
        System.out.println("Dos");
        System.out.print("Tres");
    }
}
```

2. El salut que no saluda: En un programa on tens un mètode `public static void saludo()`, com fas perquè s'executi si només està definit fora de `main`?

3. El detectiu de VS Code: El teu programa s'executa, però imprimeix "La suma es: 34" en lloc de 7. Quina eina penses utilitzar i com ho investigues?

4. Verdader o fals: "javac i java fan el mateix: el primer és per a Windows i el segon per a Linux."

💡 Solucions a les preguntes

1. Imprimeix:

```
Uno Dos
Tres
```

(`print` no afegeix salt de línia al final, però `println` sí. Després de "Tres" no hi ha salt).

2. Cal afegir la crida `saludo();` dins del mètode `main`. En Java, només s'executa automàticament allò que es crida directament o indirecta des del mètode `main`.

3. Utilitzes el depurador (`F5`): Poses un breakpoint en la línia del `println`, avances amb `F10` (Step Over) i inspectes el panell lateral de variables. Veuràs com `"La suma es: " + 3` es converteix en `"La suma es: 3"` i després `+ 4` dona `"La suma es: 34"`. La solució és afegir parèntesis `(a + b)`.

4. Fals. `javac` és el compilador (`.java` → `.class`) i `java` és l'executor de la JVM. Tots dos s'utilitzen en qualsevol sistema operatiu (Windows, Linux, macOS).

💬 Preguntes d'entrevista de treball

Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo fos la teva iaia, la diferència entre JDK, JRE i JVM."**
2. **"Què és el mètode `main` i per què té aquella firma exacta?"**
3. **"Un programa compila però fa el que no ha de fer. Quin és el teu procés per a arreglar-lo?"**
4. **"Què és un array i què passa si accedeixes a un índex que no existeix?"**
5. **"Com li passes dades a un programa Java sense que et les demani per teclat?"**

🤷 No hi ha preguntes tontes

❓ Puc escriure Java en un bloc de notes o necessito sí o sí un IDE?

Pots, i de fet és un bon exercici: `notepad MiPrograma.java`, `javac MiPrograma.java`, `java MiPrograma`. Però l'IDE t'estalvia errors tontos amb l'autocompletat i et dona el depurador. Els pros usen IDE; els masoquistes, bloc de notes.

❓ Per què el primer `println` d'Hola Món tenia "args" si no l'usi?

`String[] args` és obligatori en la firma del `main` encara que no l'uses. Java l'espera sí o sí. És com la firma en un contracte: està encara que no s'executi aquella clàusula. En el punt 6 vas veure com usar-lo.

❓ Java i JavaScript són cosins?

No, ni tan sols són del mateix planeta. Java és a JavaScript com un gos és a un gosset calent. El nom va ser una estratègia de màrqueting de Netscape per a muntar-se en el boom de Java.

🎬 Poscrèdits

El programador aprenent acaba el seu primer programa, el compila i l'executa. Tot perfecte. Llavors s'acosta CONRAD, el compilador cascarrabies, amb cara de pocs amics i una tassa humejant a la mà.

CONRAD: — Molt bonic l'Hola Món. I ara què? Vas a imprimir text tota la vida? Els programes de veritat *guarden* dades, *calculem* i *decideixen*. Sense variables, sense tipus, sense operadors… això és un triler de cartes.

Aprenent: — I això quan s'aprèn?

CONRAD: — *fa un glop* La pròxima parada. Ja et veig declarant el teu primer `int`… i cometent el teu primer error de tipus. Espera'm, que jo estaré allà per a riure.

L'aprenent guarda el seu projecte, tanca l'IDE i sent que això només acaba de començar.

U03 — Variables, tipus i operadors

Les caixes del magatzem de la memòria i les màquines del gimnàs de dades 🔤

Benvingut al magatzem. En la U02 vas aprendre a dir-li a Java que et salude per consola, però un programa que només imprimeix text és un lloro: repeteix, però no pensa. Per a pensar necessita *guardar* coses. Edats, preus, notes, si fa fred o no… i per a això existeixen les **variables**.

Imagina que la memòria del teu ordinador és un **magatzem gegant** ple de prestatgeries. Cada prestatgeria té caixes, i les variables són aquelles caixes: cada una té una etiqueta perquè sàpigues què hi ha dins. I quan les caixes s'omplin de números i text, arriben els **operadors**: les màquines de peses del gimnàs de dades que sumen, resten, comparen i transformen.

En aquesta unitat aprendràs a declarar variables dels **8 tipus primitius**, a guardar text amb `String`, a fer immutables algunes caixes amb `final`, a manipular-les amb operadors aritmètics, relacionals i lògics, a convertir-les entre tipus amb **casting**, a fer que el teu programa **escolte el teclat** amb `Scanner`, a donar format a la sortida amb `printf` i a jugar a la loteria amb `Math.random()`.

Aquesta unitat es llegeix com un **llibre de 10 capítols**: els 9 primers punts són teoria en progressió i el 10 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat

En acabar, seràs capaç de:

- Declarar i usar variables dels **8 tipus primitius** i saber què cap en cada caixa.
- Aplicar les regles de **nomenclatura** i distingir `int`, `long`, `double`, `char` i `boolean` amb un cop d'ull.
- Crear i usar `String` com una **classe** (no un primitiu) i entendre la seva immutabilitat.
- Declarar **constants** amb `final` i entendre per què el compilador s'enfada si les toques.
- Usar els operadors aritmètics `+ - * / %`, sobreviure a la **divisió entera** i respectar la **precedència**.
- Comparar valors amb operadors **relacionals**, combinar-los amb **lògics** (`&&`, `||`, `!`) i resumir decisions amb el **ternari**.
- Convertir entre tipus amb **casting** implícit i explícit, sabent quan es perd precisió.

- Llegir dades del teclat amb **`Scanner`**, donar format a la sortida amb **`printf`** i generar nombres aleatoris amb **`Math.random()`**.
- Manejar els **mètodes de `String`** més usats per a manipular text.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Nivell
01 · Variables i tipus primitius	Declarar variables, nomenclatura i els 8 primitius	Tots
02 · String, constants i final	`String`, immutabilitat, `==` vs `equals` i constants `final`	Tots
03 · Operadors aritmètics	`+ - * / %`, divisió entera, precedència i `++/--`	Tots
04 · Relacionals, lògics i ternari	Comparacions, `&&`/`||`/`!`, curtcircuit i ternari	Tots
05 · Casting i conversions	Conversió implícita i explícita, truncament i desbordament	Tots
06 · Scanner: llegir pel teclat	`Scanner`, `nextInt`, `nextDouble` i `nextLine`	Tots
07 · Consola: sortida amb format i errors d'entrada	`printf`, `String.format`, `NumberFormat` i errors del `Scanner`	Tots
08 · Math.random() i nombres aleatoris	`Math.random()`, la fórmula min-max i les eines de `Math`	Tots
09 · Mètodes útils de String	`length`, `substring`, `replace`, `trim`, `charAt`…	Tots
10 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 9 primers punts són teoria en progressió. El 10 és l'aterratge pràctic: llegeix-lo just després del 9 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

---

✅ Criteris d'avaluació coberts (RA2 + RA1)

RA1: Reconeix l'estructura d'un programa informàtic, identificant i relacionant els elements propis del llenguatge de programació utilitzat.

RA2: Escriu i prova programes senzills, reconeixent i aplicant els fonaments de la programació orientada a objectes.

CE	Criteri	On es cobreix
RA1 d)	S'han identificat els distints tipus de variables i la utilitat específica de cada un.	✅ Punt 1
RA1 e)	S'ha modificat el codi d'un programa per a crear i utilitzar variables.	✅ Punt 1
RA1 f)	S'han creat i utilitzat constants i literals.	✅ Punt 2
RA1 g)	S'han classificat, reconegut i utilitzat en expressions els operadors del llenguatge.	✅ Punts 3 i 4
RA1 h)	S'ha comprovat el funcionament de les conversions de tipus explícites i implícites.	✅ Punt 5
RA2 b)	S'han escrit programes simples.	✅ Tots
RA2 c)	S'han instanciat objectes a partir de classes predefinides.	✅ Punts 2 i 6
RA2 d)	S'han utilitzat mètodes i propietats dels objectes.	✅ Punts 6, 7 i 9

CE	Criteri	On es cobreix
RA2 e)	S'han escrit crides a mètodes estàtics.	✅ Punt 8
RA2 g)	S'han incorporat i utilitzat llibreries d'objectes.	✅ Punt 6
RA2 h)	S'han utilitzat constructors.	✅ Punts 2 i 6

> 📌 El `Scanner` del punt 6, la sortida amb format del punt 7 i els mètodes de `String` del punt 9 planten la llavor del RA5 (entrada i sortida d'informació), que floreix en la U14.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites el que vas veure en la U02.
- Ja saps declarar variables i vols cim? → Ves directe al punt 6 i al punt 8: allà comencen els programes interessants.
- Només vens a per operadors? → Salta al punt 3 i al punt 4.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Variables i tipus primitius

**⏭️ En acabar la unitat, continua en U04 · Estructures de control i excepcions bàsiques.

01 — Variables i tipus primitius

Les caixes del magatzem de la memòria i els seus 8 tamanys distints 📦

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 01 · Variables i tipus primitius

📬 La idea en una frase

> **Les variables són caixes etiquetades en la memòria de l'ordinador, i Java et ofereix 8 tamanys de caixa (els tipus primitius) perquè tries el que millor li va a cada dada.**

En la U02 el teu programa només cridava text per la consola. Ara li donaràs memòria: guardarà la teva edat, el teu nom, la teva nota mitjana i fins i tot si tens gana. I per a triar bé la caixa de cada dada, primer has de conèixer el catàleg del magatzem.

📦 La declaració: la recepta d'una caixa

Per a crear una caixa (una variable) li dius a Java tres coses: el **tipus** (tamany i forma de la caixa), el **nom** (l'etiqueta) i el **valor** (el que fiques dins). La recepta és:

```
tipo nombreDeLaCaja = valorQueMetoDentro;
```

Exemples reals:

```
int edad = 25;              // Caixa etiquetada "edad" amb un 25 dins
double precio = 19.99;      // Caixa amb decimals
String nombre = "María";    // Caixa màgica que guarda text
boolean hambre = true;      // Caixa de vertader/fals (ara mateix: true)
```

> 💡 **Detall pràctic:** les variables es diuen així perquè… ¡varien! Pots canviar el seu contingut quan vulguis. `int edad = 25;` i, el dia del teu aniversari, `edad = 26;`. L'etiqueta és la mateixa, el contingut canvia.

🏷️ Les regles de nomenclatura (o com no ficar-la)

Java és tiquismiquis amb els noms de les caixes. Aquestes són les regles d'or:

- Poden portar lletres, números, _ i $. **Res d'espais**. (Java admet molts caràcters Unicode, però per convenció i per a evitar embolics s'usen lletres ASCII: evita ñ, ç o accents en els noms.)
- **No poden començar amb número.** `1numero` és il·legal; `numero1` és legal. Com les matrícules dels cotxes.
- **Les majúscules importen**: `edad`, `Edad` i `EDAD` són tres caixes distintes. Com etiquetar "Zapatos", "zapatos" i "ZAPATOS".
- **No usis paraules reservades**: `int`, `class`, `if`, `while`… són de Java, no teves.
- Usa **camelCase** per a les variables: `miVariableEjemplo`. Com un camell, amb gepa enmig.

```
// ✅ Correcte
int numeroAlumnos = 30;
double notaMedia = 7.5;

// ❌ Incorrecte
int 1numero = 30;        // comença per número
double nota media = 7.5; // espai en el nom
int class = 30;          // paraula reservada
```

📐 Els 8 primitius: el catàleg de caixes

Java té **8 tipus primitius**. Pensa en ells com caixes de distints tamanys al teu magatzem:

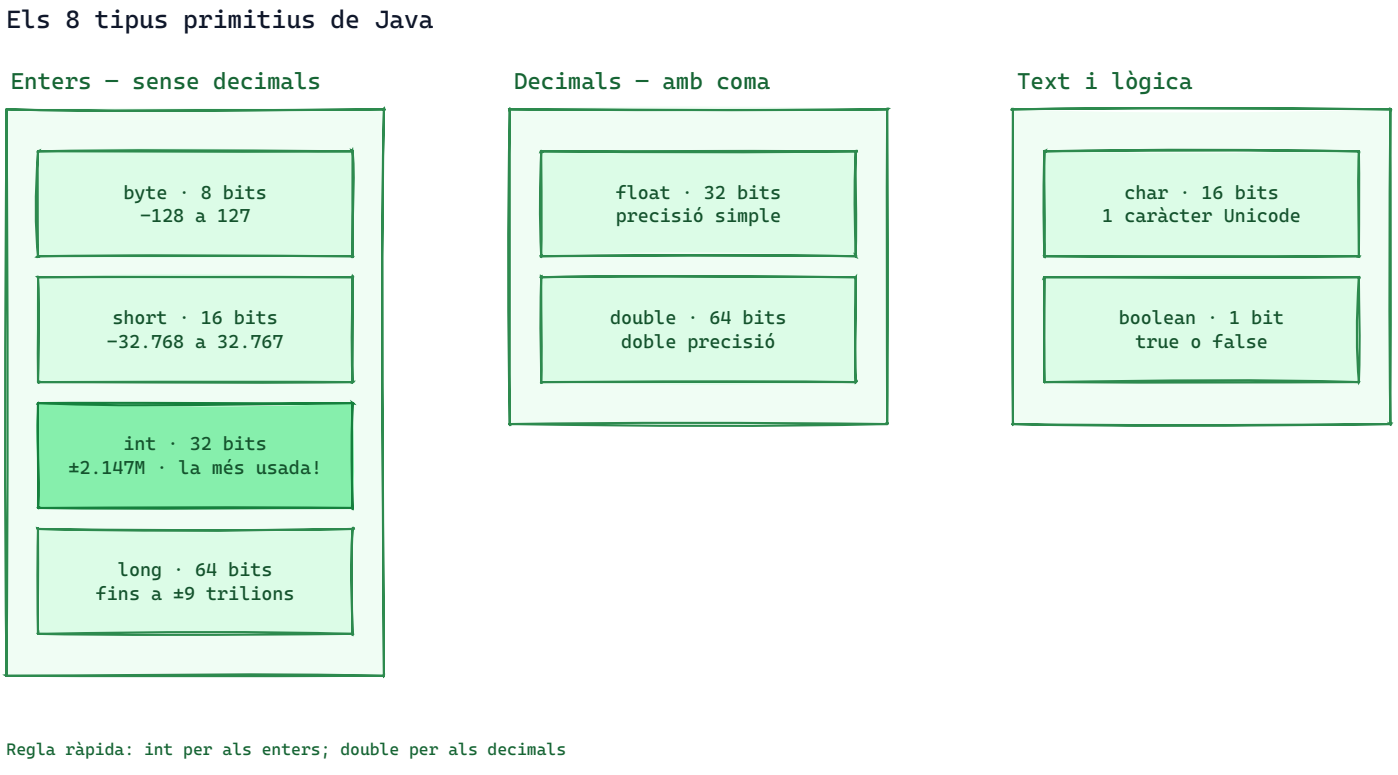
Tipus	Mida	El que cap	Analogia
`byte`	8 bits	-128 a 127	Caixa de llumins
`short`	16 bits	-32.768 a 32.767	Caixa de sabates
`int`	32 bits	-2.147M a 2.147M	Caixa de mudança (la que més usaràs)
`long`	64 bits	-9 trilions a +9 trilions	Contenidor de vaixell
`float`	32 bits	Decimals de precisió simple	Got d'aigua
`double`	64 bits	Decimals de precisió doble	Cubell d'aigua

Tipus	Mida	El que cap	Analogia
`char`	16 bits	Un sol caràcter Unicode	Una lletra en una caixa de sabates
`boolean`	1 bit	`true` o `false`	Interruptor de llum

I així es declaren cada un:

```
byte nivel = 100;
short poblacion = 30000;
int habitantes = 1500000;           // El más usado
long distancia = 384400000L;        // La L al final es obligatoria
float precio = 12.99f;              // La f al final es obligatoria
double pi = 3.14159265359;
char letra = 'A';                   // Comillas SIMPLES para char
boolean esJavaDivertido = true;     // Esto es opinable
```

> 📝 **Nota:** usa `int` per a quasi tot el numèric enter. Només passa a `long` si vas a comptar estrelles. Usa `double` per a decimals, a menys que estalviar memòria sigui el teu fetitxe.



🎒 Quina caixa usi per a cada dada?

Triar el tipus correcte és com triar la maleta del viatge: ni un microbus per a dues persones, ni un Smart per a una família de cinc. La pràctica fa el mestre:

- **`int`**: edats, comptadors, puntuacions, quasi tot enter.
- **`double`**: preus, notes mitjanes, temperatures, qualsevol decimal.
- **`boolean`**: respostes de sí/no: "ha aprovat?", "hi ha connexió?".
- **`char`**: una sola lletra: la inicial d'un nom, una qualificació 'A', 'B', 'C'.
- **`long`**: nombres astronòmics, mil·lisegons, identificadors gegants.

> ⚠️ **Advertència:** quan dubtes entre `int` i `double`, pensa: *aquesta dada pot portar decimals?* Si sí → `double`. Si no → `int`. Simple.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** abans d'escriure codi, fes-te sempre la mateixa pregunta: *quina mena de dada és això i en quina caixa cap?* El 90% dels errors d'aquesta unitat vénen de ficar malament la caixa.

Exercici: El guarda del magatzem

Ets el guarda d'un magatzem de dades. Et donen aquestes declaracions i et pregunten: **quines compilen i quines no?** Marca les que fallarien i per què:

```
int a = 150;                // ¿compila?
int b = 10.5;               // ¿compila?
double c = 7;               // ¿compila?
char d = "A";               // ¿compila?
boolean e = "true";         // ¿compila?
long f = 3000000000L;       // ¿compila?
int g = 3000000000;         // ¿compila?
```

> › 🔄 **Solució**
>
> - `int a = 150;` ✅ Compila. Un 150 cap de sobres en un `int`.
> - `int b = 10.5;` ❌ **No compila.** Un `int` no admet decimals; això seria un `double`.
> - `double c = 7;` ✅ Compila. Un enter cap dins d'un `double` (conversió implícita, ho veuràs en el punt 5).

- `char d = "A";` ❌ **No compila.** `char` usa cometes simples `'A'`; les dobles són per a `String`.
- `boolean e = "true";` ❌ **No compila.** `true` sense cometes és el booleà; `"true"` amb cometes és text.
- `long f = 3000000000L;` ✅ Compila. La `L` li diu a Java "això és un long, no un int".
- `int g = 3000000000;` ❌ **No compila.** Tres mil milions no cap en un `int` (tope: 2.147M). Seria un `long`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin tamany de caixa usaries per a guardar el nombre d'habitants de la Terra (més de 8.000 milions)?
2. Per què `char letra = "A";` no compila i `char letra = 'A';` sí?
3. Quina és la diferència entre `float` i `double` en una frase?
4. Per què `Edad`, `edad` i `EDAD` són tres variables distintes?

> 🔄 **Respostes**
>
> 1. **`long`** — més de 2.147 milions no cap en un `int`.
> 2. Perquè `char` va amb **cometes simples** `'A'` (un sol caràcter); les cometes dobles són per a `String`.
> 3. El `double` té **doble precisió** (64 bits) i el `float` precisió simple (32 bits): el `double` guarda més decimals exactes.
> 4. Perquè **les majúscules importen**: cada variació del nom és una caixa distinta al magatzem.

✅ Resum en 3 frases

1. Una variable és una **caixa etiquetada** en memòria: tipus (tamany), nom (etiqueta) i valor (contingut).
2. Java té **8 tipus primitius**, i triar el correcte (normalment `int` o `double`) és mitat examen.

3. Les regles de nomenclatura (camelCase, sense espais, sense paraules reservades, majúscules que importen) t'estalvien errors de compilació tontos.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Variable | Caixa etiquetada on guardes una dada que pot canviar |
> | Tipus primitiu | Un dels 8 tipus bàsics de dades de Java |
> | Declarar | Crear la variable: `tipo nombre = valor;` |
> | Literal | El valor escrit tal qual: `25`, `"María"`, `true` |
> | camelCase | Convenció de noms: `miVariableEjemplo` |
> | Paraula reservada | Paraula de Java que no pots usar com a nom |

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · String, constants i final

02 — String, constants i final

La caixa màgica del text i les caixes amb superglue 🔤

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 02 · String, constants i final

---

📬 La idea en una frase

> **`String` és una classe (no un primitiu) que guarda text, és immutable com una foto, i `final` és el superglue que converteix qualsevol caixa en una constant que no es pot tocar.**

En el punt 1 vas veure les 8 caixes primitives. Però els programes també guarden text: noms, missatges, contrasenyes… Per a això existeix `String`. I quan vulguis que un valor no canviï mai, el declares `final`. Anem a les dues.

🔤 String: la caixa màgica que no és una caixa

`String` **no és primitiu**: és una **classe**. Però es comporta tan natural que sembla primitiu. És com un amic que encaixa tan bé en el teu grup que jurares que és de la família.

```
String saludo = "Hola, DAM";           // La forma normal
String nombre = new String("Ana");      // També es pot crear així (usa un constructor)
```

Fixa't en la segona línia: `new String(...)` és un **constructor**. Encara no estudies POO a fons (això arriba en la U08), però ja pots instanciar objectes de classes predefinides com `String`. La primera línia és una drecera que Java et dona per a no escriure `new String(...)` cada vegada.

> 💡 **Detall pràctic:** `String` va amb **cometes dobles** `"..."`. Les cometes simples `'...'` són només per a `char`, un únic caràcter.

🧊 La immutabilitat: no toquis, que es trenca

Els `String` són **immutables**: una vegada creats, no es poden canviar. Quan fas això:

```java
String texto = "Hola";
texto = texto + " mundo";    // Java NO modifica "Hola"
```

…en realitat Java tira el `"Hola"` a la brossa i crea un `"Hola mundo"` nou. El text original segueix aquí, congelat, per sempre. És com si cada vegada que volguessis posar un cartell nou haguessis de cremar l'anterior.

> ⚠️ **Advertència:** aquesta immutabilitat és la raó per la qual comparar `String` amb `==` és perillós. Comparar amb `==` pregunta *"són el mateix objecte?"*, no *"tenen el mateix text?"*. Per a comparar contingut usa `.equals()`.

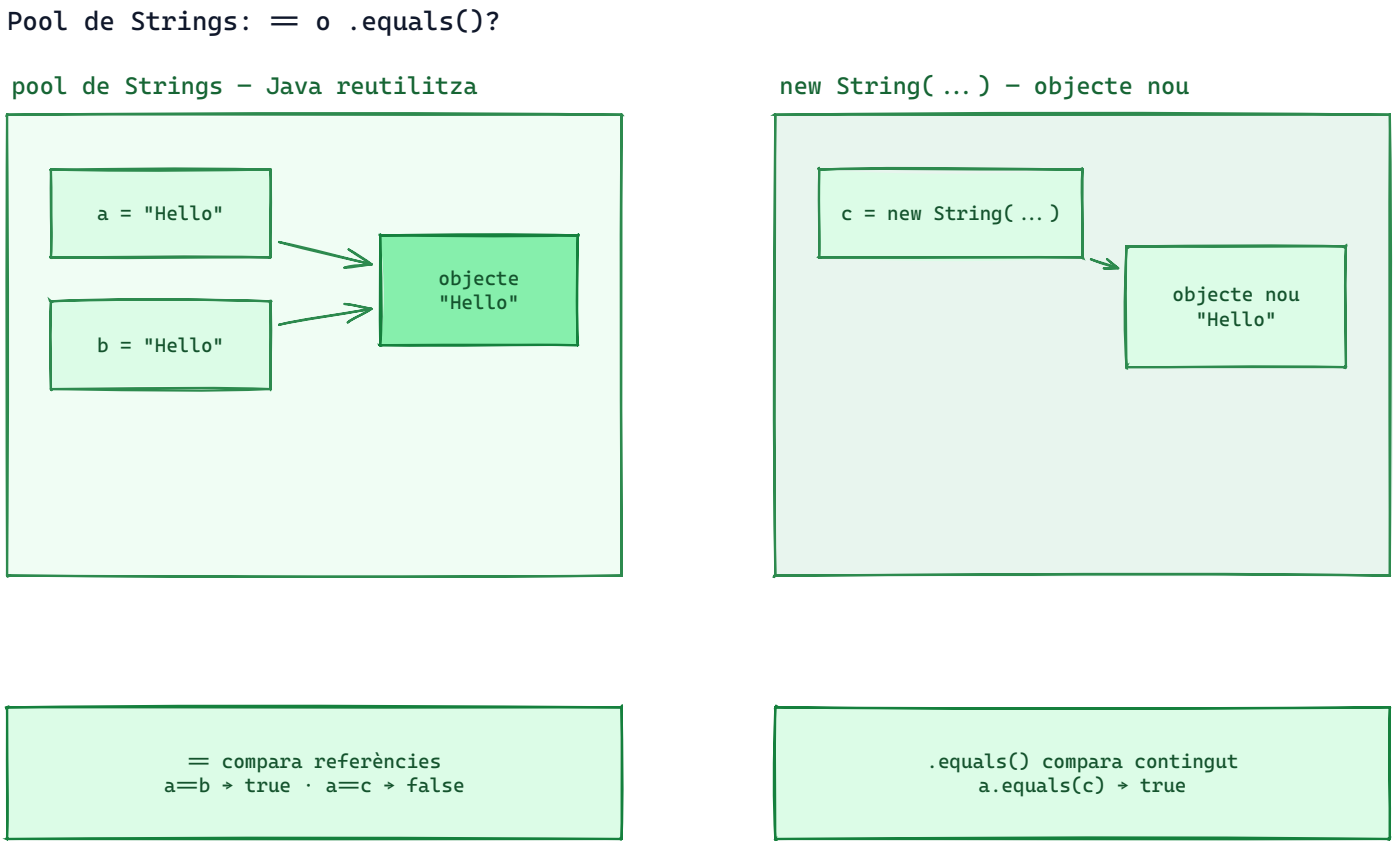
🧲 `==` vs `.equals()`: la trampa clàssica

```java
String a = "Hello";
String b = "Hello";
String c = new String("Hello");

System.out.println(a == b);       // true
System.out.println(a == c);       // false
System.out.println(a.equals(c));  // true
```

Per què `a == b` dona `true` i `a == c` dona `false`, si els tres textos són "Hello"?

- a i b apunten al **mateix objecte** en el "pool de Strings": Java reutilitza literals iguals.
- c es va crear amb `new String(...)`, així que és un objecte **nou i distint**.
- `==` compara **referències** (són la mateixa caixa?); `.equals()` compara **contingut** (tenen el mateix dins?).

> ⚠️ **Advertència:** regla d'or: **els `String` sempre es comparen amb `.equals()`**. Si uses `==`, tard o d'hora et mossegarà en un examen.



🔒 Constants amb `final`: caixes amb superglue

Les constants es declaren amb `final`. Una vegada que fiques alguna cosa aquí dins, no surt ni amb palanca:

```
final double IVA = 0.21;
final int MAXIMO_INTENTOS = 3;
final String NOMBRE_APP = "Gestión DAM";

IVA = 0.10;  // ERROR de compilació: ¡no pots reasignar una constant!
```

Per convenció, les constants s'escriuen **EN_MAJÚSCULES_AMB_GUIONS_BAIXOS**, com si estiguessin cridant "¡SOC IMMUTABLE!". Això li diu a qualsevol programador (inclòs el teu jo del futur) que aquell valor no s'ha de tocar.

> 💡 **Detall pràctic:** per què usar constants i no escriure el número directament? Perquè si l'IVA canvia demà, edites **una línia**, no les 50 on vas usar el 0.21. Això és el que s'anomena *mantenir el codi*.

🏫 Exemple guiat: la factura que no es pot tocar

Anem a muntar un mini-programa que calcula el preu final d'un producte amb IVA. La màgia: l'IVA és `final` i el nom de l'app també.

```
public class Factura {
    public static void main(String[] args) {
        final double IVA = 0.21;
        final String NOMBRE_APP = "Factura Express";

        double precioBase = 50.0;
        double ivaAplicado = precioBase * IVA;
        double precioFinal = precioBase + ivaAplicado;

        System.out.println(NOMBRE_APP + " — precio base: " + precioBase + "€");
        System.out.println("IVA (" + (IVA * 100) + "%): " + ivaAplicado + "€");
        System.out.println("Total: " + precioFinal + "€");
    }
}
```

Sortida:

```
Factura Express — precio base: 50.0€
IVA (21.0%): 10.5€
Total: 60.5€
```

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan vagis == amb dos `String`, sospita. Pregunta't primer: *estan comparant referències o contingut?*

Exercici: Què imprimeix aquest embolic de Strings?

Sense executar, digues què imprimeix exactament aquest codi:

```java
String x = "Java";
String y = "Java";
String z = new String("Java");
System.out.println(x == y);
System.out.println(x == z);
System.out.println(x.equals(z));
```

> 🔄 **Solució**
>
> Imprimeix `true`, `false` i `true`.
>
> - `x == y` → **true**: els dos literals apunten al mateix objecte del pool de Strings.
> - `x == z` → **false**: `z` és un objecte nou creat amb `new`, no comparteix referència.
> - `x.equals(z)` → **true**: el contingut és el mateix. `equals` compara text, no referències.
>
> Clàssic d'examen. Si l'encertes a la primera, aquesta unitat la portes bé.

🎯 Mini-chequeig

1. És `String` un tipus primitiu? Per què?
2. Què significa que un `String` sigui immutable?
3. Què passa si intentes reasignar una variable declarada com `final`?
4. Per què es comparen els `String` amb `.equals()` i no amb `==`?

> 🔄 **Respostes**
>
> 1. **No**, és una **classe**. Es comporta com primitiu però té mètodes i es crea amb `new` (encara que Java et dona la drecera dels literals).
> 2. Que una vegada creat, el seu valor **no es pot modificar**: cada "canvi" crea un String nou.
> 3. Error de **compilació**. `final` és el superglue: el que entra, no surt.
> 4. Perquè `==` compara **referències** (mateix objecte?) i `.equals()` compara **contingut** (mateix text?), que és el que normalment vols.

✅ Resum en 3 frases

1. `String` és una **classe** que guarda text entre cometes dobles i és **immutable**: cada canvi crea un objecte nou.
2. Els `String` es comparen amb **`.equals()`**, mai amb == (que només compara referències i et dona sorpreses).
3. `final` converteix una variable en **constant** (per convenció, en MAJÚSCULES), i el compilador s'enfada si intentes canviar-la.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `String` | Classe de Java que guarda cadenes de text |
> | Immutable | Que no es pot modificar una vegada creat |
> | `final` | Modificador que converteix una variable en constant |
> | Constructor | El mecanisme que crea un objecte (`new String(...)`) |
> | Pool de Strings | Zona on Java reutilitza literals iguals |
> | `.equals()` | Mètode que compara el contingut de dos objectes |

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Variables i tipus primitius · **Següent:** 03 · Operadors aritmètics

03 — Operadors aritmètics

Les màquines de pesos del gimnàs de dades ➕

> 🗺️ **Estàs en: 🔤 U03 · Variables, tipus i operadors** → 03 · Operadors aritmètics

📬 La idea en una frase

> **Els operadors aritmètics (+, -, *, /, %) són les màquines de pesos del gimnàs de dades: transformen les teves variables, i la divisió entera i la precedència són les trampes que separen els que saben dels que improvisen.**

Tenir variables està molt bé, però no serveixen de res si no fas coses amb elles. Benvingut al gimnàs: suaràs amb els cinc exercicis bàsics i descobriràs per què `10 / 3` no és el que tu creus.

💪 El dia al gimnàs: els 5 exercicis bàsics

Operador	Exercici	Exemple
+	Press de banca	`5 + 3 = 8`
-	Curl de bíceps	`5 - 3 = 2`
*	Sentadilla	`5 * 3 = 15`
/	Pes mort	`10 / 3 = 3` (enters) o `10.0 / 3 = 3.333...`
%	L'odiat abdominal	`10 % 3 = 1` (el resti de 10/3)

```
int a = 10;
int b = 3;
double c = 10.0;

System.out.println(a / b);           // 3  (divisió entera)
System.out.println(a % b);           // 1  (el reste)
System.out.println(c / b);           // 3.333... (divisió real)
System.out.println((double) a / b);  // 3.333... (obligues decimal)
```

> 💡 **Detall pràctic:** el **mòdul** (`%`) no és per a avorrits: et diu si un nombre és parell (`numero % 2 == 0`), reparteix torns, fa voltes als rellotges i alimenta un munt de jocs. Sense `%` no existiria res cíclic.

⚠️ La divisió entera mata

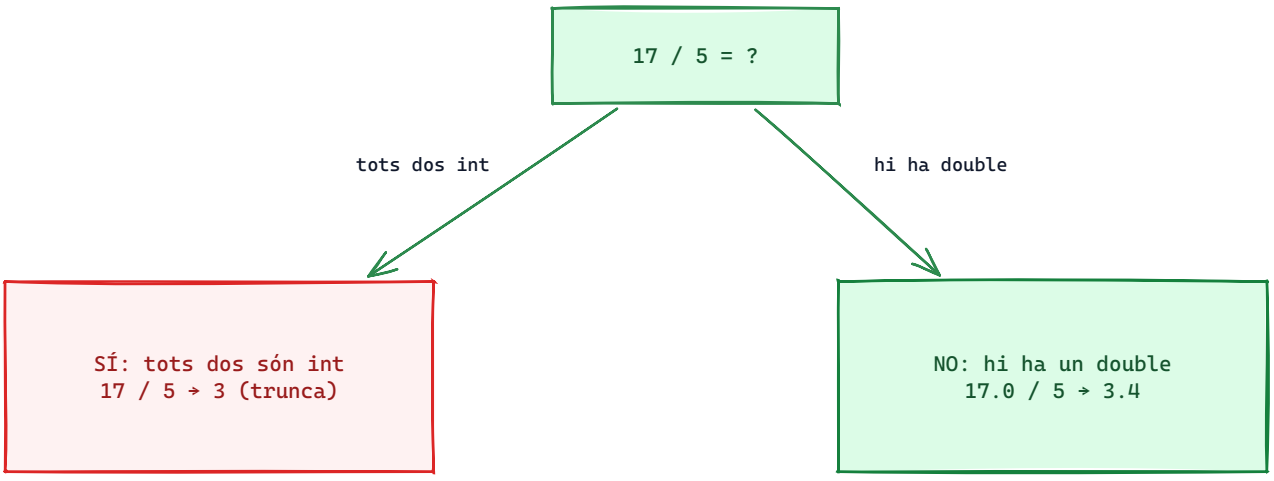
Si divideixes dos enters, Java et retorna un enter. Punt. Els decimals es truncaren sense pietat:

```
int alumnos = 17;
int grupos = 5;
System.out.println(alumnos / grupos);    // 3 — Java diu que cada grup té 3 alumnes
```

Per a Java, 17 dividit entre 5 són **3**. Ni 3.4 ni 3.5: 3. Si vols decimals, almenys un dels dos operands ha de ser `double` (o fer un casting, que veuràs en el punt 5).

> ⚠️ **Advertència:** aquest és un dels errors més rendibles per a un examen. `5 / 2` és `2.5 / 2.0` és `2.5`. `(double) 5 / 2` és `2.5`. Memoritza-ho com un mantra.

Divisió entera: int / int trunca



Mantra d'examen: 5/2=2 · 5/2.0=2.5 · (double)5/2=2.5

🎭 Precedència: la llei del menjador

Qui se serveix primer en el menjador de les expressions? Hi ha un ordre estricte:

```
int resultado = 2 + 3 * 4;        // 14 — la multiplicació es cola abans
int conParentesis = (2 + 3) * 4;  // 20 — els parèntesis tenen passe VIP
```

La llei del menjador:

1. **Parèntesis `( )`** — passi VIP, van els primers.
2. **Multiplicació, divisió i mòdul `* / %`** — els populars.
3. **Suma i resta `+ -`** — els normals, els últims.

> 📝 **Nota:** i quan dubtes, **posa parèntesis**. `(a + b) * (c - d)` és molt més llegible que confiar en la teva memòria de la precedència. Els parèntesis no dolen i el que llegeix el teu codi (el teu jo del futur) t'ho agrairà.

🌀 Assignació composta: la drecera peresosa

Escriure `x = x + 5` és tan verbós... Per sort, Java té els **operadors d'assignació composta**, una aixeta d'aigua al sofà per a no anar a la cuina:

```
int x = 10;
x += 5;   // x = 15  (x = x + 5, però més cool)
x -= 3;   // x = 12
x *= 2;   // x = 24
x /= 4;   // x = 6
x %= 3;   // x = 0
```

💪 ++ i --: flexions per a variables

Sumar o restar 1 és tan comú que Java té el seu propi operador: `++` (increment) i `--` (decrement). Però compte, que tenen dues cares:

```
int a = 5;
int b = a++;  // POST: primer usa a (5), després incrementa → b = 5, a = 6
int c = ++a;  // PRE: primer incrementa, després usa → a = 7, c = 7
```

- **Post-increment (a++)**: "usa i després puja".
- **Pre-increment (++a)**: "puja i després usa".

> ⚠️ **Advertència:** regla d'or: si uses ++ o -- *dins* d'una expressió complicada, estaràs escrivint codi que ni tu entendràs en una setmana. Usa'ls sols, en la seva pròpia línia. En els exàmens apareixen les trampes, i aquí tens la prova.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** desglossa l'expressió pas a pas. Quin valor té cada variable en cada moment? Anota-ho, no ho facis de memòria.

Exercici: l'acròbata de les variables

Sense executar, calcula quant val tot aquí:

```
int x = 3;
int y = x++ + ++x;
System.out.println("x = " + x + ", y = " + y);
```

> 🔄 **Solució**
>
> Imprimeix `x = 5, y = 8`. Pas a pas:
>
> 1. `x = 3`.
> 2. x++ — POST: usa x (3), després incrementa x a 4. El valor de x++ és **3**.
> 3. ++x — PRE: x val 4 ara; l'incrementa a **5** i aquell és el seu valor.
> 4. `y = 3 + 5 = 8`.
> 5. Resultat: `x = 5, y = 8`.
>
> Als programadors professionals també els costa. Per això quasi ningú escriu això en producció… però en els exàmens, ¡ai, apareix!

🎯 Mini-chequeig

1. Quant val `int r = 10 / 3`? I `double r = 10 / 3`?
2. Què fa l'operador `%` i per a què serveix saber si `n % 2 == 0`?
3. Quin és el resultat de `2 + 3 * 4 - 1`?
4. Diferència entre a++ i ++a en una frase.

> 🔄 **Respostes**
>
> 1. `int r = 10 / 3` val **3** (divisió entera trunca). `double r = 10 / 3` també val **3.0**: la divisió es fa primer amb enters i després es guarda. Per a decimals necessites `10 / 3.0` o `(double) 10 / 3`.
> 2. El `%` dona el **resti** de la divisió. `n % 2 == 0` és la prova universal de paritat: si el resti és 0, n és parell.
> 3. **13**: primer `3 * 4 = 12`, després `2 + 12 - 1 = 13`. La multiplicació mana.
> 4. `a++` **usa** el valor actual i després incrementa; `++a` **incrementa** primer i després usa el nou.

✅ Resum en 3 frases

1. Els operadors `+ - * / %` transformen les teves dades, i la **divisió entera** trunca els decimals si tots dos operands són enters.
2. La **precedència** segueix la llei del menjador (parèntesis → `* / %` → `+ -`), i els parèntesis sempre són el pla B segur.
3. `+=`, `-=`, `++` i `--` són dreceres perillosament còmodes: usa-les soles i no dins d'expressions enrevessades.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Operador aritmètic	`+ - * / %`: les operacions matemàtiques de Java
Divisió entera	Divisió de dos enters que trunca els decimals
Mòdul	`%`, el resti d'una divisió
Precedència	L'ordre en què Java avalua els operadors
Assignació composta	Drecera com `x += 5` (= `x = x + 5`)
Increment	`++`/`--`: sumar o restar 1, amb cara pre i post

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · String, constants i final · **Següent:** 04 · Relacionals, lògics i ternari

04 — Relacionals, lògics i ternari

El jutge, el porter i el bouncer del club de les decisions ⚖️

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 04 · Relacionals, lògics i ternari

📬 La idea en una frase

> **Els operadors relacionals (`==`, `!=`, `<`, `>`, `<=`, `>=`) comparen valors i retornen un `boolean`; els lògics (`&&`, `||`, `!`) combinen booleans; i el ternari (`? :`) resumeix una decisió en una sola línia.**

Les teves variables ja saben sumar i restar. Ara aprendran a **jutjar**: "ets major d'edat?", "tens entrada I diners?", "ets el amo O un convidat?". Benvingut al club de les decisions.

⚖️ Relacionals: el jutge de la discussió

Els operadors relacionals sempre retornen un `boolean`: `true` o `false`. Són el jutge que dicta sentència sobre la relació entre dos valors:

```
int edad = 18;
boolean puedeVotar = edad >= 18;                  // true
boolean tieneDescuento = edad < 12 || edad > 65;  // false
boolean noEsEl = edad != 18;                      // false
```

Operador	Significat	Exemple (`edad = 18`)
`==`	Igual que	`edad == 18` → `true`
`!=`	Distint de	`edad != 18` → `false`
`>`	Major que	`edad > 21` → `false`
`<`	Menor que	`edad < 21` → `true`
`>=`	Major o igual	`edad >= 18` → `true`
`<=`	Menor o igual	`edad <= 18` → `true`

> ⚠️ **Advertència: = no és ==.** = assigna (“guarda aquest valor en aquesta caixa”), == compara (“són iguals?”). Confondre’ls és l’error més clàssic de la història de la programació. És com confondre “posa la taula” amb “està posada la taula?”.

🚪 Lògics: el porter del club

Els operadors lògics combinen booleans per a prendre decisions compostes. Són el porter del club nocturn:

- **&& (AND)**: Tens més de 18 **I** tens entrada? Les dues condicions s’han de complir.
- **|| (OR)**: Tens més de 18 **O** ets l’amo? Basta que es compleixi una.
- **! (NOT)**: **NO** tens menys de 18? Niega la condició.

```
int edad = 18;
boolean mayorEdad = true;
boolean tieneEntrada = false;

boolean entra = mayorEdad && tieneEntrada;    // false (falta l'entrada)
boolean entraVip = mayorEdad || tieneEntrada; // true (basta ser major d'edat)
boolean noEsMenor = !(edad < 18);             // true (nega que siga menor)
```

⚡ Curtcircuit: el porter que no mira

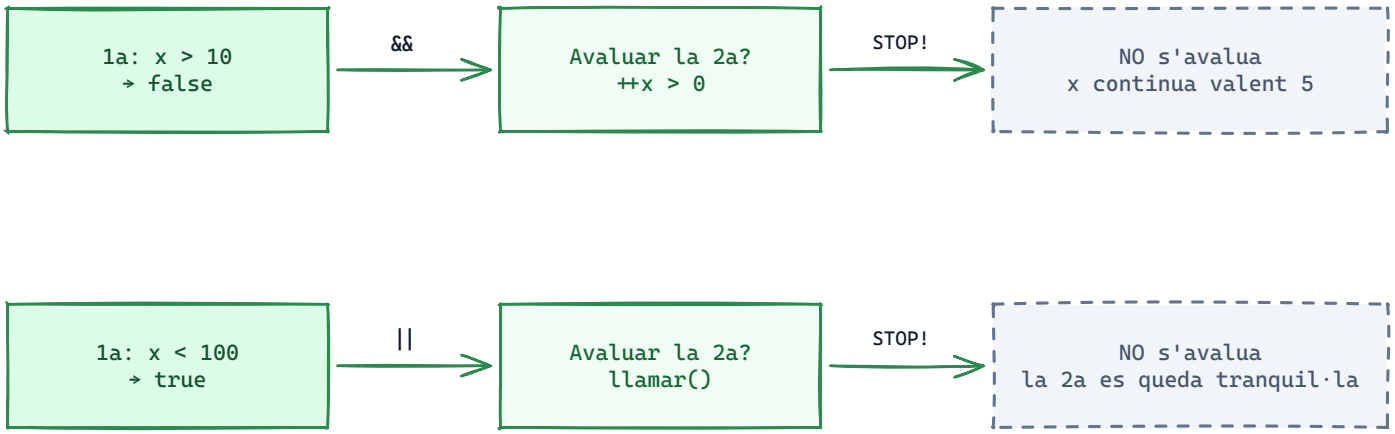
Aquí ve el truc del club més rendible de Java: el **curtcircuit**.

- Amb `&&`, si el primer és `false`, l’expressió ja és `false` i **Java ni es molesta a avaluar el segon**.
- Amb `||`, si el primer és `true`, l’expressió ja és `true` i tampoc mira el segon.

```
int x = 5;
boolean resultado = (x > 10) && (++x > 0);   // false, i x seguix sent 5
System.out.println(x);                       // 5 — el ++x mai no es va executar
```

> 💡 **Detall pràctic:** el curtcircuit també et protegeix. Si escrius `(algo != null) && algo.metodo()`, Java no cridarà el mètode si `algo` és `null`, evitant un crash al teu programa.

Curtcircuit: la 2a condició no es mira



Regla: && amb false talla · || amb true talla — la 2a condició ni es mira

---

🎚️ El ternari: el bouncer del club

Quan la decisió és “si passa això, missatge A; si no, missatge B”, el **operador ternari** ho resumeix en una línia. És un `if-else` de butxaca (els `if` de veritat arriben en la U04):

```
int edad = 17;
String mensaje = (edad >= 18) ? "Pasa, joven" : "Vuelve cuando crezcas";
```

L’estructura és: `condició ? valorSiTrue : valorSiFalse`.

```
int nota = 7;
String resultado = nota >= 5 ? "Aprobado" : "Suspenso";
```

🏫 Exemple guiat: el porter del club

Anem a muntar el control d’accés d’un club amb tot el que has après:

```java
public class ClubNoche {
    public static void main(String[] args) {
        int edad = 19;
        boolean tieneEntrada = true;
        boolean esVip = false;

        boolean puedeEntrar = (edad >= 18) && (tieneEntrada || esVip);
          String mensaje = puedeEntrar ? "¡Pasa, disfruta!" : "Fuera de aquí, pequeñín";

        System.out.println(mensaje);

        int golesLocal = 2;
        int golesVisitante = 2;
        String resultado = golesLocal > golesVisitante ? "Gana el local" :
                           golesLocal < golesVisitante ? "Gana el visitante" : "Empate";
        System.out.println("Resultado: " + resultado);
    }
}
```

Sortida:

```
¡Pasa, disfruta!
Resultado: Empate
```

Fixa't en la segona decisió: els ternaris es poden **encadenar** (un ternari dins d'un altre) per a gestionar tres casos. Llegible i compacte.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** en una expressió amb `&&` i `||`, pregunta sempre: *i si el primer ja decideix?* Aquell és el curtcircuit.

Exercici: el detectiu de booleans

Sense executar, calcula el valor de cada variable:

```java
int n = 10;
boolean a = (n > 5) && (n < 20);              // ¿?
boolean b = (n > 15) || (++n == 11);          // ¿?
boolean c = !(n == 10) && (n % 2 == 0);       // ¿?
System.out.println(a);
System.out.println(b);
System.out.println(c);
System.out.println(n);
```

> 🔄 **Solució**
>
> Imprimeix `true`, `true`, `false` i `11`.
>
> 1. a: 10 és major que 5 **i** menor que 20 → **true**.
> 2. b: 10 no és major que 15, així que Java avalua el `||` amb la segona part: `++n == 11` → incrementa n a 11 i compara: `11 == 11` → **true**. (El `||` només curtcircuita quan la primera és `true`; aquí era `false`.)
> 3. c: `!(n == 10)` amb `n = 11` → `!false` → `true`; `&&` amb `11 % 2 == 0` → `false`. Resultat: **false**.
> 4. n va acabar en **11** pel ++n de la línia 2.

🎯 Mini-chequeig

1. Què retorna sempre un operador relacional?
2. Quina és la diferència entre `&&` i `||` en una frase?
3. Què és el curtcircuit i quan s'activa?
4. Escriu el ternari que assigni `"major"` o `"menor"` a una variable segons una edat.

> 🔄 **Respostes**
>
> 1. Un **`boolean`**: `true` o `false`.
> 2. `&&` exigeix que **totes** les condicions es compleixin; `||` es conforma amb **una sola**.
> 3. Quan la primera condició ja decideix el resultat, Java **no avalua les altres**: amb `&&` si la primera és `false`, amb `||` si la primera és `true`.
> 4. `String resultado = edad >= 18 ? "major" : "menor";`

✅ Resum en 3 frases

1. Els **relacionals** (==, `!=`, <, >, <=, >=) comparen i retornen un `boolean`, i `=` mai no s'usa per a comparar.
2. Els **lògics** (`&&`, `||`, `!`) combinen condicions i pateixen **curtcircuit**: si la primera ja decideix, no miren les altres.
3. El **ternari** (`condició ? A : B`) resumeix una decisió de dos camins en una línia.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Operador relacional | Compara dos valors i dona `true`/`false` |
> | Operador lògic | Combina booleans: `&&`, `||`, `!` |
> | Curtcircuit | Deixar d'avaluar quan la primera condició ja decideix |
> | Ternari | `condició ? valor1 : valor2`, un if-else en una línia |
> | Booleà | Tipus amb només dos valors: `true` o `false` |

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Operadors aritmètics · **Següent:** 05 · Casting i conversions

05 — Casting i conversions

Apreta que cap, amb pèrdues i llàgrimes si no tries bé 🔄

> 🗺️ **Estàs en: 🔤 U03 · Variables, tipus i operadors** → 05 · Casting i conversions

---

📬 La idea en una frase

> **El casting converteix un valor d'un tipus a un altre: la conversió implícita (widening) la fa Java sol i sense pèrdues, mentre que l'explícita (narrowing) la forces tu amb `(tipo)` i pots perdre dades pel camí.**

Al magatzem de la memòria tens caixes de tots els tamanys. A vegades necessites ficar el contingut d'una caixa gran en una de petita… i això, o ho fas amb compte, o perds coses pel camí. Benvingut a l'art d'apretar.

🪜 Conversió implícita (widening): mudar-se a una caixa més gran

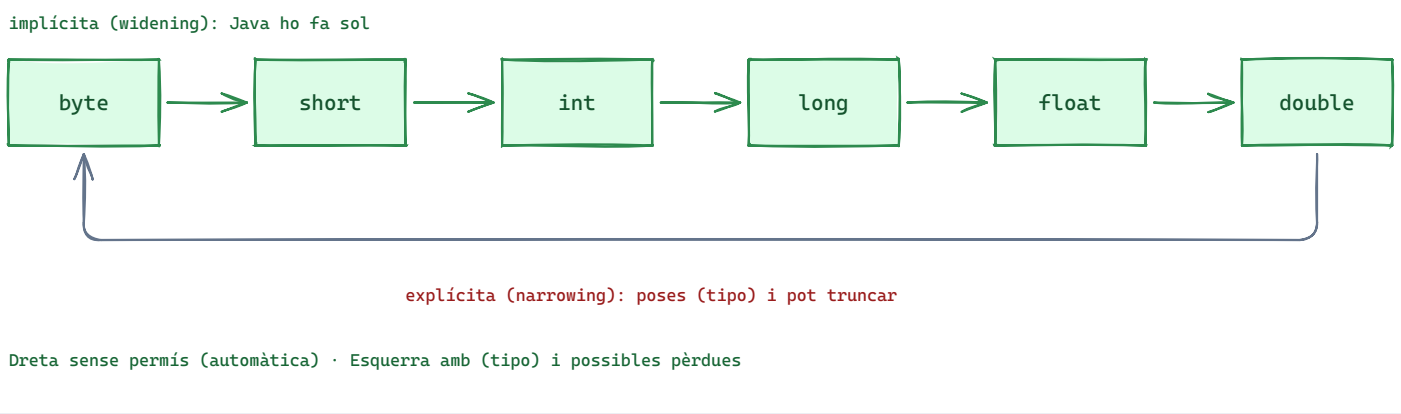
Quan el destí és una caixa **més gran**, Java ho fa sol, sense preguntar. És com canviar d'un pis a una mansió: et mudes i no perds res. Això s'anomena *widening* (eixamplar):

```
int num = 100;
long numLong = num;        // Cap de sobres, sense pèrdues
double numDouble = num;    // 100 → 100.0, també sense problemes
```

La cadena natural d'eixamplament entre tipus numèrics és:

```
byte → short → int → long → float → double
```

Qualsevol tipus pot passar al que està a la seva dreta sense que es perda ni un bit. Java somriu i et deixa.



📉 Conversió explícita (narrowing): la maleta XXL en un Smart

Quan el destí és una caixa **més petita**, Java es nega. Has d'empènyer a la força amb un casting explícit: poses el `(tipo)` davant i reses:

```
double precio = 19.99;
int entero = (int) precio;    // Apretes 19.99 en un int
System.out.println(entero);   // 19 — els cèntims desapareixen en l'oblit
```

Això s'anomena *narrowing* (estrényer). Java t'obliga a escriure el `(tipo)` perquè és perillós: estàs dient "confia en mi, sé el que faig". I si t'equivoques, les dades es perden en silenci.

> 💡 **Detall pràctic:** el casting sobre un `double` **trunca**, no arrodoneixi. `(int) 19.99` dona `19` i `(int) 19.5` també dona `19`. Si vols arrodonir, usa `Math.round()` (ho veuràs en el punt 7).

🪓 El truncament: el cèntim oblidat

L'exemple del preu és l'advertència clàssica:

```
double precio = 9.99;
int precioEntero = (int) precio;
System.out.println(precioEntero);   // 9 — t'acaben de llevar 0.99€
```

No és arrodonir: és **tallar amb destral**. Java tira la part decimal sense mirar si era 0.1 o 0.9.

> ⚠️ **Advertència:** abans d'estrényer una caixa, pregunta't sempre: *cap el valor?* Si el número és major que el màxim de la caixa destí, no només perdràs decimals: el valor es **desbordarà** a una cosa completament absurda.

💥 El desbordament: quan apretes de més

Què passa si intentes ficar un elefant (300) en una caixa de llumins (byte, màxim 127)?

```
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);   // 44 — ¡quaranta-quatre!?
```

D'on surt el 44? En binari, 300 és `100101100`. Un `byte` només guarda 8 bits, així que es truncaren els sobrants i es queda amb `00101100`, que és… 44. És com ficar un elefant en un Mini Cooper i que surti un gos salchicha: el resultat tècnicament és un animal, però no el que vas ficar.

> ⚠️ **Advertència:** el desbordament no dona error. Java no t'avisa. El programa segueix corrent amb un valor absurd. Per això el casting explícit és responsabilitat teva: comprova sempre que el valor cap.

🏚️ Exemple guiat: el guarda del magatzem

Aquest mini-programa recorre tota la cadena de conversions, perquè vagis quan Java t'acompanya i quan t'obliga a firmar:

```java
public class CadenaDeCajas {
    public static void main(String[] args) {
        int a = 10;
        double b = a;            // implícita: int → double, sense llàgrimes
        int c = (int) b;         // explícita: double → int, forçada
        byte d = (byte) c;       // explícita: int → byte, cap de sobres

        System.out.println(b);   // 10.0
        System.out.println(c);   // 10
        System.out.println(d);   // 10
    }
}
```

I ara la part dolenta:

```java
public class Perdidas {
    public static void main(String[] args) {
        double nota = 9.9;
        int notaEntera = (int) nota;    // 9 — adéu, dècimes
        int enorme = 400;
        byte pequeno = (byte) enorme;   // desbordament silenciós
        System.out.println(notaEntera); // 9
        System.out.println(pequeno);    // -112 (¡ni tan sols és 400!)
    }
}
```

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** el casting amb `(tipo)` pot truncar i desbordar. Comprova sempre si el valor cap abans de forçar-lo.

Exercici: segueix el guarda

Sense executar, determina què imprimeix aquest programa:

```java
int a = 10;
double b = a;
int c = (int) b;
byte d = (byte) c;
System.out.println(d);

int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
```

> 🔄 **Solució**
>
> Imprimeix `10` i `44`.
>
> - El primer bloc és una cadena de conversions sense pèrdua: `10` → `10.0` → `10` → `10`. Imprimeix **10**.
> - El segon és el desbordament clàssic: 300 no cap en un `byte` (tope 127), es truncaren els bits sobrants i queda **44**. Com ficar un elefant en un Mini i que surti un gos salchicha.

🎯 Mini-chequeig

1. Per què `long x = 100000;` compila sense casting i `int y = (int) 100000.5;` sí que necessita el `(int)`?
2. Quant val `(int) 7.99`? I `(int) 7.1`?
3. Què passa si converteixes `300` a `byte`?
4. És el truncament el mateix que arrodonir?

> 🔄 **Respostes**
>
> 1. Perquè `100000` (un `int`) cap en un `long` sense pèrdues: és conversió **implícita**. Ficar un `double` amb decimals en un `int` és **estrényer** (narrowing), i Java t'obliga a escriure-ho amb `(int)`.
> 2. `(int) 7.99` → **7** i `(int) 7.1` → **7**. Trunca, no arrodoneixi: la part decimal es tira sencera.
> 3. Es **desborda** silenciosament i val **44**, sense error ni avís.
> 4. **No.** El truncament talla la part decimal sempre; arrodonir puja o baixa segons el valor.

✅ Resum en 3 frases

1. La conversió **implícita** (widening) cap a caixes més grans la fa Java sol i sense pèrdues.
2. La conversió **explícita** (narrowing) la forces amb `(tipo)` i pot **truncar** els decimals o **desbordar** el valor.
3. Abans d'estrényer, comprova sempre que el valor cap: Java no t'avisarà de les pèrdues.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Casting	Convertir un valor d'un tipus a un altre
Widening	Eixamplar: a caixa més gran, sense pèrdues, automàtic
Narrowing	Estrényer: a caixa més petita, amb `(tipo)` i possibles pèrdues
Truncament	Tallar la part decimal, sense arrodonir
Desbordament	El valor no cap i es converteix en una cosa absurda en silenci
`(int)`	El casting que força un valor decimal a enter

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Relacionals, lògics i ternari · **Següent:** 06 · Scanner: llegir pel teclat

06 · Scanner: llegir pel teclat

El moment en què el teu programa deixa de cridar i comença a escoltar ⌨️

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 06 · Scanner: llegir pel teclat

---

📬 La idea en una frase

> **`Scanner` és la classe de Java que llegeix el que escrius pel teclat: instancies un objecte amb `new Scanner(System.in)` i li demanes dades amb `nextInt()`, `nextDouble()` o `nextLine()`.**

Fins ara, els teus programes eren uns cridaners: només escopien text a la consola. A partir d'aquest punt tindran oïdes. I amb oïdes arriben els programes de veritat: un conversor de temperatures que et pregunta els graus, una calculadora que rep números…

📖 El paquet: importar la llibreria

`Scanner` no viu al centre de Java: viu en una llibreria (`java.util`). Per a usar-lo, la primera línia del teu archiu ha de ser:

```
import java.util.Scanner;
```

És com demanar en la biblioteca el llibre que vas a usar. Sense el `import`, Java et dirà que no coneix `Scanner`.

🏗️ Instanciar: el constructor

Per a tenir un lector de teclat necessites **crear un objecte** de la classe `Scanner`. Això es fa amb `new` i un **constructor** (recorda el `new String(...)` del punt 2):

```
Scanner sc = new Scanner(System.in);
```

- `Scanner` és la classe (el motlle).
- `new Scanner(...)` crea l'objecte (el constructor).
- `System.in` és l'argument que li passes: "llegeix del teclat estàndard".

> 💡 **Detall pràctic:** el nom de la variable sol ser `sc` o `teclado`, per pura costum. Quan acabis d'usar el `Scanner`, és bona pràctica tancar-lo amb `sc.close()`, sobretot si el programa va a continuar fent coses rares.

🔢 Demanar dades: els mètodes next

Una vegada tens l'objecte `sc`, li demanes dades amb els seus mètodes. Cada mètode espera que escriguis i premis Enter:

```
import java.util.Scanner;

public class PrimerEscucha {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cómo te llamas? ");
        String nombre = sc.nextLine();     // llig una línia de text completa

        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();           // llig un número enter

        System.out.print("¿Cuál es tu nota media? ");
        double nota = sc.nextDouble();     // llig un número amb decimals

        System.out.println("Hola, " + nombre + ". Con " + edad + " años y un " +
nota + " de media, vas sobrado.");

        sc.close();
    }
}
```

Sortida d'exemple:

```
¿Cómo te llamas? Ana
¿Cuántos años tienes? 18
¿Cuál es tu nota media? 9.5
Hola, Ana. Con 18 años y un 9.5 de media, vas sobrado.
```

Els mètodes més usats:

Mètode	Llegeix	Exemple
`nextLine()`	Una línia de text completa	`String nombre = sc.nextLine();`
`next()`	Només la següent paraula	`String palabra = sc.next();`
`nextInt()`	Un número enter	`int edad = sc.nextInt();`
`nextDouble()`	Un número amb decimals	`double nota = sc.nextDouble();`
`nextBoolean()`	`true` o `false`	`boolean ok = sc.nextBoolean();`

⚠️ El embolic de `nextLine()` després de `nextInt()`

Aquest és l'error més odiat del `Scanner`, i apareix en tots els exàmens. Quan fas `nextInt()`, l'Enter que vas pulsar es queda *guardat* al buffer. Si després crides a `nextLine()`, aquella crida es menja l'Enter residual i et retorna una línia buida:

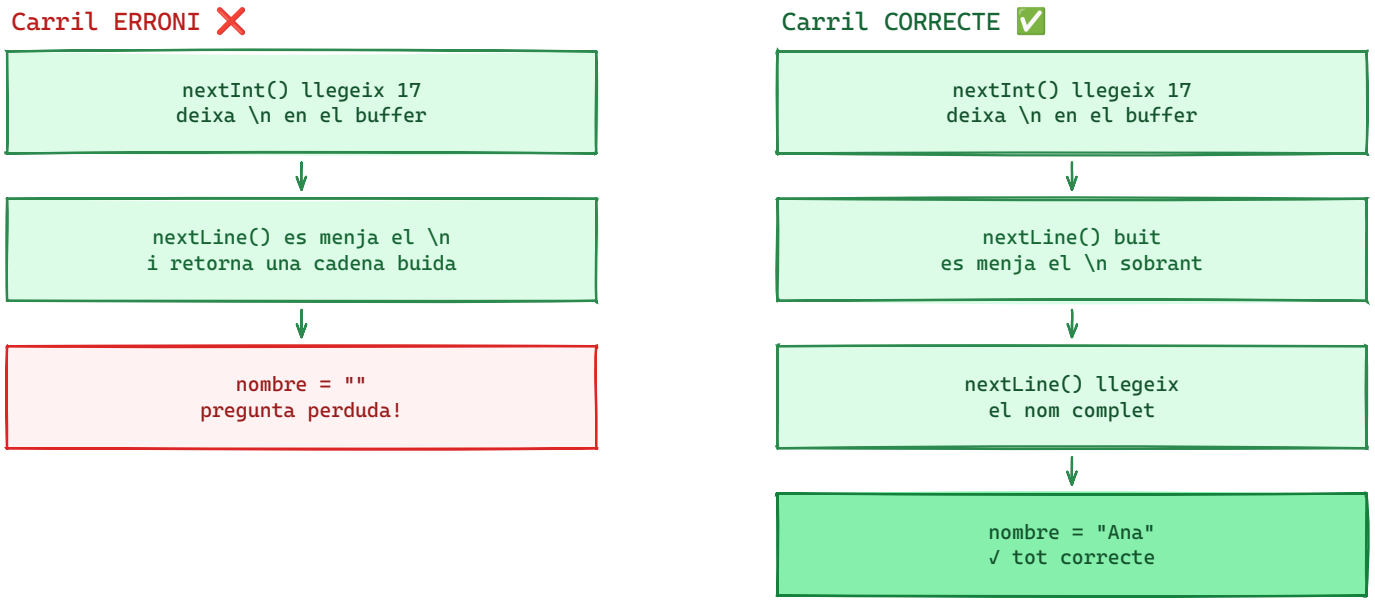
```
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();          // escrius 18 i pulses Enter
System.out.print("Nombre: ");
String nombre = sc.nextLine();    // ¡es salta la pregunta! retorna ""
```

La solució: afegeix un `nextLine()` extra (o usa `next()` per al text) just després del número:

```
int edad = sc.nextInt();
sc.nextLine();                    // es menja l'Enter sobrant
String nombre = sc.nextLine();    // ara sí, llig el nom
```

> ⚠️ **Advertència:** memoritza el truc: *després d'un `nextInt()` / `nextDouble()`, inserta un `nextLine()` buit abans del següent `nextLine()`.* És el guardià del buffer.

El salt de línia residual: dos camins, dos resultats



Truc: un nextLine() buit entre el número i el text es menja l'Enter

---

🏫 Exemple guiat: la calculadora de la propina

Anem a usar el teclat per a una cosa útil: calcular quant deixar de propina.

```java
import java.util.Scanner;

public class Propina {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cuánto vale la cuenta? ");
        double cuenta = sc.nextDouble();

        System.out.print("¿Qué porcentaje de propina? ");
        int porcentaje = sc.nextInt();

        double propina = cuenta * porcentaje / 100;
        double total = cuenta + propina;

        System.out.println("Propina: " + propina + "€");
        System.out.println("Total a pagar: " + total + "€");

        sc.close();
    }
}
```

Sortida d'exemple:

```
¿Cuánto vale la cuenta? 45.5
¿Qué porcentaje de propina? 15
Propina: 6.825€
Total a pagar: 52.325€
```

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan combines `nextInt()` i `nextLine()`, recorda sempre l'Enter residual. Escriu-ho com un reflex: número → `nextLine()` buit → text.

Exercici: el presentador amb trampa

Aquest programa intenta saludar l'usuari… però alguna cosa falla. Sense executar, digues què ocorre i com ho arreglaries:

```java
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();
System.out.print("Nombre: ");
String nombre = sc.nextLine();
System.out.println(nombre + " tiene " + edad + " años.");
```

> 🔄 **Solució**

El problema és l'**Enter residual**: després de `nextInt()`, el `nextLine()` es menja l'Enter i `nombre` queda com text buit. El programa imprimiria una cosa com `tiene 18 años.`

La solució és afegir un `nextLine()` buit entre el número i el text:

```java
int edad = sc.nextInt();
sc.nextLine();                    // es menja l'Enter sobrant
String nombre = sc.nextLine();    // ara sí que llig el nom
```

🎯 Mini-chequeig

1. Què fa la línia `import java.util.Scanner;`?
2. Què fa `new Scanner(System.in)`?
3. Quina és la diferència entre `next()` i `nextLine()`?
4. Per què després d'un `nextInt()` cal posar un `nextLine()` buit?

> 🔄 **Respostes**

1. Importa la classe `Scanner` des de la llibreria `java.util`, per a poder usar-la.
2. **Crea un objecte** de tipus `Scanner` que llegeix del teclat (`System.in`). És el constructor de la classe.
3. `next()` llegeix **una sola paraula** (fins a un espai); `nextLine()` llegeix **tota la línia** fins a l'Enter.
4. Perquè l'Enter que vas pulsar en `nextInt()` queda al buffer i el següent `nextLine()` se'l menja, retornant text buit.

✅ Resum en 3 frases

1. `Scanner` és la classe per a llegir del teclat: la importes amb `import`, la instancies amb `new Scanner(System.in)` i demanes dades amb mètodes `next...`.
2. Cada tipus de dada té el seu mètode: `nextInt()`, `nextDouble()`, `nextLine()` per a text.
3. Després d'un `nextInt()` o `nextDouble()`, un `nextLine()` buit es menja l'Enter residual: sense ell, la teva següent pregunta es saltarà.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `Scanner` | Classe de `java.util` per a llegir dades de teclat |
> | `import` | Línia que porta una classe des de la seva llibreria |
> | Constructor | Mecanisme que crea l'objecte (`new Scanner(...)`) |
> | `System.in` | El teclat estàndard, la font d'entrada |
> | Buffer | Zona de memòria on queda l'Enter residual |
> | Instanciar | Crear un objecte a partir d'una classe |

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Casting i conversions · **Següent:** 07 · Consola: sortida amb format i errors d'entrada

07 · Consola: sortida amb format i errors d'entrada

printf, String.format, NumberFormat i les fallades clàssiques del Scanner (i com no morir en l'intent) 🖨️

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 07 · Consola: sortida amb format i errors d'entrada

---

📬 La idea en una frase

> **`printf` i `String.format` donen format a la teva sortida (decimals, amplària, alineació) en una línia, i conèixer els errors típics del `Scanner` t'estalvia els bugs més odiats de la unitat.**

En el punt 6 vas aprendre a llegir del teclat. Ara donaràs bellesa al que escrius i, de passada, blindaràs els teus programes contra les fallades més típiques d'entrada. Amb això tanques el cercle de la consola.

🖨️ Sortida amb format: `System.out.printf`

`System.out.println` imprimeix tal qual. Per a controlar **com** es veu (decimals, amplària, farcit) tens `printf` (print formatted):

```
String nom = "Ana";
int edat = 20;
double nota = 9.5;

System.out.printf("Nom: %s, Edat: %d, Nota: %.2f%n", nom, edat, nota);
// Nom: Ana, Edat: 20, Nota: 9,50
```

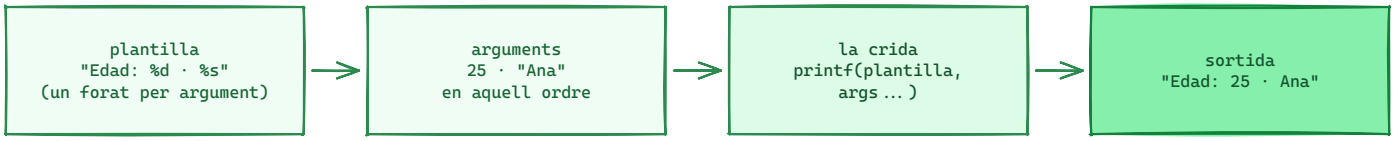
Cada `%alguna_cosa` és un **forat** que s'ompli amb el valor que li segueix, en ordre. Els especificadors bàsics:

Especificador	Tipus	Exemple
`%s`	String	`"Hola %s"` → `"Hola Mario"`
`%d`	Enter	`"Edat: %d"` → `"Edat: 25"`

Especificador	Tipus	Exemple
%f	Decimal	"%.2f" → "19,99"
%c	Caràcter	"Inicial: %c"
%n	Salt de línia	(independent del sistema)
%b	boolean	"%b" → true

> 💡 **Consell:** %n per als salts de línia en printf (no \n): funciona igual a Windows, Linux i Mac. El \n també val, però %n és l'opció "oficial".

printf: la plantilla i els seus arguments



Els arguments es col·loquen en ordre: el 1r va al primer forat (%d), el 2n al segon (%s).

Controlar els decimals i l'amplària

```
double pi = Math.PI;
System.out.printf("2 decimals: %.2f%n", pi);   // 3,14
System.out.printf("4 decimals: %.4f%n", pi);   // 3,1416
System.out.printf("Amplària 10: %10.2f%n", pi); //       3,14
System.out.printf("Esquerra: %-10.2f%n", pi);  // 3,14
```

- %.2f → dos decimals.
- %10.2f → amplària mínima de 10 caràcters, alineat a la dreta.
- %-10.2f → el guió l'alinea a l'esquerra.

> ⚠️ **Advertència:** els decimals de printf utilitzen la **configuració regional** del teu sistema. En un ordinador amb locale espanyol, %.2f escriu 3,14 (coma); en un amb locale anglès, 3.14 (punt). No t'espantes si el resultat varia: és la màquina parlant en el seu idioma.

🧵 `String.format`: el mateix, però sense imprimir

A vegades no vols imprimir en el moment, sinó **construir un text** amb format per a utilitzar-lo després (guardar-lo, concatenar-lo…). `String.format` fa exactament el mateix que `printf`, però **torna** la cadena en comptes d'imprimir-la:

```
String msg = String.format("Benvingut, %s. Tens %d missatges nous.",
"Carles", 3);
System.out.println(msg);
// Benvingut, Carles. Tens 3 missatges nous.
```

> 💡 **Consell:** utilitza `String.format` quan vulguis un text amb format com a **valor** (per a guardar-lo o usar-lo diverses vegades), i `printf` quan només vulguis escriure'l en pantalla.

💶 Números grans: `NumberFormat`

Imprimir `1234567.89` sense format és lleig i difícil de llegir. `NumberFormat` aplica els separadors de milers i decimals del teu idioma:

```
import java.text.NumberFormat;
import java.util.Locale;

NumberFormat nf = NumberFormat.getInstance(new Locale("es", "ES"));
System.out.println(nf.format(1234567.89));  // 1.234.567,89

NumberFormat moneda = NumberFormat.getCurrencyInstance(new Locale("es", "ES"));
System.out.println(moneda.format(12345.67));  // 12.345,67 €
```

- `getInstance(locale)` → format numèric amb separadors.
- `getCurrencyInstance(locale)` → format de moneda (amb el símbol €).

> 📝 **Nota:** el `Locale("es", "ES")` li diu "parla com a Espanya": punt per als milers, coma per als decimals. Si no li passes locale, utilitza el del teu sistema.

🚨 Errors clàssics del Scanner (i els seus remeis)

El `Scanner` és traïdor. Aquestes són les fallades que es repeteixen en cada examen i en cada programa de pràctiques:

1. Oblidar el `import java.util.Scanner;`

Sense la línia d'import, Java no coneix la classe i et llança un error de compilació. És la fallada més ximple i la més comuna.

2. No tancar el Scanner: `sc.close()`

Deixar el `Scanner` obert és de mala educació (i en programes llargs, pot deixar recursos sense alliberar). Tanca sempre en acabar:

```
Scanner sc = new Scanner(System.in);
// ... tot el teu codi ...
sc.close();
```

3. Demanar un tipus i escriure'n un altre: `InputMismatchException`

Si demanes un `int` i l'usuari escriu lletres, el programa **explota** amb `InputMismatchException`:

```
int edat = sc.nextInt();        // l'usuari escriu "hola" → 💥 InputMismatchException
```

La solució robusta és **preguntar abans** amb `hasNextInt()` (o `hasNextDouble()`, `hasNext()`…):

```
if (sc.hasNextInt()) {
    int edat = sc.nextInt();
} else {
    System.out.println("Això no és un número enter.");
    sc.next();   // descarta el text mal escrit
}
```

> ⚠️ **Advertència:** `hasNextInt()` **no consumeix** la dada: només mira si el següent és un enter. Si no ho és, has de consumir el text brossa amb `sc.next()` abans de tornar a preguntar, o es quedarà allà per sempre.

4. El problema del `nextLine()` després del `nextInt()` (ja ho vas veure en el punt 6)

L'Enter residual es queda en el buffer. Després d'un número, posa un `nextLine()` buit abans de demanar text.

🏫 Exemple guiat: el validador que no es trenca

Un programa que demana una edat i no cau per molt que l'usuari escrigui brossa:

```
import java.util.Scanner;

public class EdatSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int edat = -1;

        while (edat == -1) {
            System.out.print("Quants anys tens? ");
            if (sc.hasNextInt()) {
                edat = sc.nextInt();
            } else {
                  System.out.println("Això no és un número enter, intenta-ho una
altra vegada.");
                sc.next();
            }
        }

        System.out.printf("Genial, %d anys i llest per a programar.%n", edat);
        sc.close();
    }
}
```

El bucle `while` repeteix la pregunta fins que l'usuari dona un enter. Amb `hasNextInt()` + `sc.next()` per a descartar la brossa, el programa és **a prova de bombes**.

⭐ Sé el Codi, my friend...

> 🕶️ **Don Tip:** la regla d'or: `printf` per a imprimir amb format, `String.format` per a guardar el text amb format, `hasNextInt()` abans de cada `nextInt()` si l'usuari pot equivocar-se.

Exercici: el formatador misteriós

Què imprimeix exactament aquest programa?

```
public class Formatacio {
    public static void main(String[] args) {
        int hores = 5;
        double preu = 12.5;
        System.out.printf("Treball: %d hores a %.1f €/hora = %.2f €%n",
                hores, preu, hores * preu);
    }
}
```

> 🔄 **Solució**
>
> Imprimeix:
>
> ```
> Treball: 5 hores a 12,5 €/hora = 62,50 €
> ```
>
> El `%d` ompli amb l'enter, `%.1f` amb un decimal, i `%.2f` amb dos. Els tres valors (`5`, `12.5` i `62.5`) es col·loquen en els forats en ordre. El `%n` afegeix el salt de línia final. (Els decimals amb coma o punt depenen del locale del sistema.)

🎯 Mini-chequeig

1. Quina diferència hi ha entre `System.out.printf` i `String.format`?
2. Quin especificador utilitzaries per a un `double` amb dos decimals?
3. Quina excepció llança `sc.nextInt()` si l'usuari escriu lletres?
4. Com evites que `sc.nextInt()` exploti amb una entrada incorrecta?

> 🔄 **Respostes**
>
> 1. Tots dos apliquen el mateix format, però `printf` ho imprimeix en pantalla i `String.format` **torna** el text format per a utilitzar-lo com a valor.
> 2. `%.2f`.

3. `InputMismatchException`.
4. Comprovant abans amb `sc.hasNextInt()` i, si no és enter, descartant la brossa amb `sc.next()` abans de tornar a preguntar.

✅ Resum en 3 frases

1. **`printf` i `String.format`** donen format a la sortida amb especificadors (`%d`, `%s`, `%.2f`) i control de decimals i amplària.
2. **`NumberFormat`** formateja números grans i monedes amb els separadors del teu idioma.
3. El `Scanner` es trenca amb `InputMismatchException` si l'usuari escriu malament: pregunta abans amb **`hasNextInt()`** i tanca sempre amb `sc.close()`.

🐛 Vocabulari ràpid

Terme	Idea general
`printf`	Imprimeix amb format (`%d`, `%s`, `%.2f`…)
`String.format`	Torna un text amb format, sense imprimir-lo
Especificador	El `%alguna_cosa` que marca on i com es col·loca un valor
`NumberFormat`	Formateja números i monedes amb separadors locals
`InputMismatchException`	Explosió en demanar un tipus i rebre'n un altre
`hasNextInt()`	Pregunta si el que ve és un enter, sense consumir-lo

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Scanner: llegir pel teclat · **Següent:** 08 · Math.random() i números aleatoris

08 — Math.random() i nombres aleatoris

El casino de Java, amb daus i fórmules per a no fer trampa 🎲

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 08 · Math.random() i nombres aleatoris

📬 La idea en una frase

> **`Math.random()` retorna un nombre aleatori entre `0.0` (inclòs) i `1.0` (exclòs), i amb la fórmula `(int)(Math.random() * (max - min + 1)) + min` el converteixes en un dau, una loteria o qualsevol nombre que necessites.**

Els teus programes ja escolten (Scanner) i calculen (operadors). Ara jugaran: als daus, a la loteria, a endevinar nombres. I per a això necessites el casino de Java: `Math.random()`.

🎲 El casino: què retorna Math.random()

`Math.random()` és un **mètode estàtic** de la classe `Math`. Retorna un nombre entre `0.0` i `1.0`… amb un detall important: **l'`1.0` no està inclòs**. És com quan et toca la loteria però no.

```
double aleatorio = Math.random();          // Entre 0.0 y 0.999999...
System.out.println(aleatorio);             // Per exemple: 0.473821...
```

> 💡 **Detall pràctic:** `Math.random()` és un mètode **estàtic**: no necessites crear un objecte de `Math` per a cridar-lo. Escrius `Math.random()` directament i ja està. És una crida de classe, no d'objecte.

🎯 La fórmula: de 0.0–1.0 al que tu vulguis

Un nombre entre 0 i 1 està molt bé, però tu vols un dau, un número de la loteria, un percentatge… Aquí va l'escala:

```
double aleatorio = Math.random();                          // Entre 0.0 y
0.999999...
int deCeroANueve = (int) (Math.random() * 10);      // Entre 0 y 9
int dado = (int) (Math.random() * 6) + 1;              // Entre 1 y 6 (como un
dado)
```

Veus el patró? El truc està en multiplicar i sumar:

- `Math.random() * 6` → nombre entre `0.0` i `5.999...`
- `(int)` el trunca → entre `0` i `5`
- `+ 1` el desplaça → entre `1` i `6` ✅

La fórmula universal per a un nombre entre `min` i `max` (tots dos inclosos):

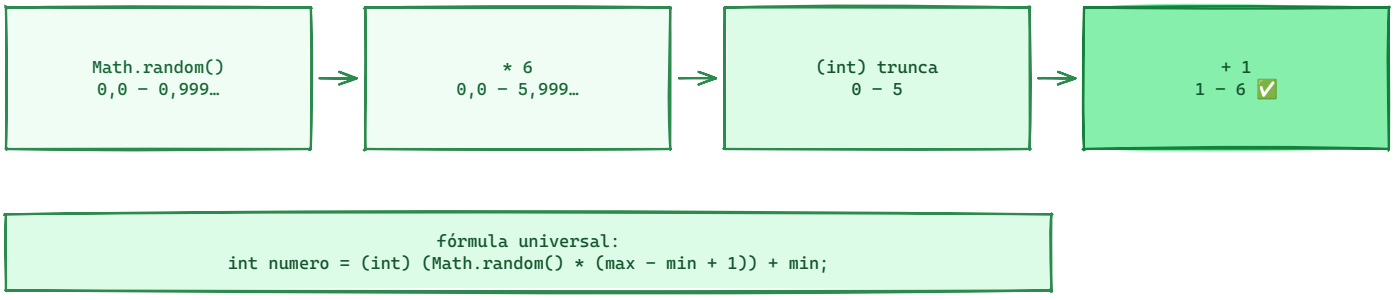
```
int numero = (int) (Math.random() * (max - min + 1)) + min;
```

Exemple del 5 al 10:

```
int entreCincoYDiez = (int) (Math.random() * 6) + 5;   // 5, 6, 7, 8, 9 o 10
```

> 📝 **Nota:** memoritza la fórmula com un mantra: `(max - min + 1)` dona la mida del ventall, i `+ min` el col·loca on comença. No hi ha més secret.

`La fórmula màgica: de Math.random() al teu rang`



🧰 Altres eines del casino: la classe Math

`Math` no és només la ruleta: és tota la sala de màquines. Algunes joies que usaràs a diari:

```java
double pi = Math.PI;                    // 3.141592653589793 — constant
double potencia = Math.pow(2, 10);      // 1024.0 — 2 elevat a 10
double raiz = Math.sqrt(144);           // 12.0
double absoluto = Math.abs(-7);         // 7.0
double redondeo = Math.round(4.6);      // 5.0
double techo = Math.ceil(4.1);          // 5.0
double suelo = Math.floor(4.9);         // 4.0
int maximo = Math.max(3, 9);            // 9
int minimo = Math.min(3, 9);            // 3
```

> 💡 **Detall pràctic:** tots aquests són **mètodes estàtics** de la classe `Math` (i `Math.PI` una constant estàtica): es criden amb `Math.nombre`, sense crear objectes. A l'examen, la pregunta típica és "com arrodoneix 4.6 sense truncar-lo?" → `Math.round(4.6)`.

🏫 Exemple guiat: el joc dels daus

Anem a muntar un dau de veritat, amb dos tirades, suma i veredicte:

```java
public class JuegoDeDados {
    public static void main(String[] args) {
        int dado1 = (int) (Math.random() * 6) + 1;
        int dado2 = (int) (Math.random() * 6) + 1;
        int suma = dado1 + dado2;

        System.out.println("Dado 1: " + dado1);
        System.out.println("Dado 2: " + dado2);
        System.out.println("Suma: " + suma);

        boolean esPar = suma % 2 == 0;
        String mensaje = esPar ? "Suma par — ganas" : "Suma impar — pierdes";
        System.out.println(mensaje);
    }
}
```

Sortida possible:

```
Dado 1: 4
Dado 2: 6
Suma: 10
Suma par — ganas
```

Cada execució dona un resultat distint: això és el divertit (i a vegades frustrant) dels aleatoris.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** recorda sempre *truncar abans de sumar*: `(int)(... * 6) + 1`, mai `(int)(... * 6 + 1)`. Si sumes abans del casting, el rang canvia i els teus daus mentiran.

Exercici: el dau que menteix

Quin rang de nombres produeix cadascuna d'aquestes tres línies? Quina és la que dona un dau de veritat (1 a 6)?

```java
int a = (int) (Math.random() * 6);
int b = (int) (Math.random() * 6) + 1;
int c = (int) (Math.random() * 7);
```

> 🔄 **Solució**
>
> - a: `Math.random() * 6` va de `0.0` a `5.999...` → després del `(int)`, **0 a 5**.
> - b: la línia anterior més `1` → **1 a 6**. ✅ És el dau de veritat.
> - c: `Math.random() * 7` va de `0.0` a `6.999...` → després del `(int)`, **0 a 6** (¡set cares, i el 0 no existeix en un dau!).
>
> La diferència entre `b` i `c` és subtil però decisiva: la `+ 1` ha d'anar **fora** del casting.

🎯 Mini-chequeig

1. Quin rang exacte retorna `Math.random()`?
2. Escriu la línia per a obtenir un nombre aleatori entre 5 i 10.
3. Què retorna `(int) (Math.random() * 100)`? I si li sumes 1?
4. És `Math.PI` un mètode o una constant? I `Math.round`?

> 🔄 **Respostes**
>
> 1. Entre `0.0` (inclòs) i `1.0` (**exclòs**): de `0.0` a `0.999...`
> 2. `int numero = (int) (Math.random() * 6) + 5;` — ventall de 6 valors començant en 5.
> 3. `(int) (Math.random() * 100)` dona **0 a 99**; amb `+ 1` dona **1 a 100**.
> 4. `Math.PI` és una **constant** (sense parèntesis); `Math.round` és un **mètode** (amb parèntesis).

✅ Resum en 3 frases

1. `Math.random()` retorna un nombre entre `0.0` i `1.0` (sense incloure l'1), i es combina amb multiplicacions i casting per a generar el rang que vulguis.
2. La fórmula `(int)(Math.random() * (max - min + 1)) + min` és la teva navalla suïssa per a qualsevol nombre aleatori.
3. `Math` és la sala de màquines estàtica: `PI`, `pow`, `sqrt`, `abs`, `round`… tots es criden amb `Math.nombre`, sense crear objectes.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`Math.random()`	Nombre aleatori entre 0.0 i 1.0 (l'1 exclòs)
Mètode estàtic	Es crida amb `Math.nombre(...)`, sense crear objectes
`Math.pow`	Potència: `Math.pow(base, exponent)`
`Math.round`	Arrodoneix (a diferència del truncament del casting)
`Math.PI`	Constant amb el número π
Truncar	Tallar els decimals amb `(int)`

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Consola: sortida amb format i errors d'entrada · **Següent:** 09 · Mètodes útils de String

09 — Mètodes útils de String

La caixa d'eines per a manipular text 🪄

> 🗺️ **Estàs en:** 🔤 **U03 · Variables, tipus i operadors** → 09 · Mètodes útils de String

📬 La idea en una frase

> **`String` porta de sèrie una caixa d'eines amb mètodes per a mesurar, retallar, buscar i transformar text (`length()`, `trim()`, `toUpperCase()`, `substring()`, `replace()`…).**

En el punt 2 vas conèixer el `String` com la caixa màgica del text. Ara obriràs la seva caixa d'eines: perquè els programes no només guarden text, també el mesuren, el netegen, el posen en majúscules i el trocegen.

🪄 El gimnàs del text: els mètodes que mesuraràs

Aquí està l'arsenal complet. Fixa't en com es crida un mètode d'objecte: `texto.metodo()`, amb un punt entre la variable i el mètode (no com `Math.random()`, que era estàtic):

```
String texto = "  Programación DAM  ";

texto.length();              // 20 — quants caràcters hi ha (espais inclosos)
texto.trim();                // "Programación DAM" — sense espais als costats
texto.toUpperCase();         // "  PROGRAMACIÓN DAM  "
texto.toLowerCase();         // "  programación dam  "
texto.contains("DAM");       // true — conté eixe text?
texto.startsWith("  ");      // true — comença per...?
texto.endsWith("AM  ");      // true — acaba per...?
texto.indexOf("DAM");        // 15 — en quina posició comença "DAM"?
texto.substring(2, 14);      // "Programación" — retalla del caràcter 2 al 13
texto.replace("DAM", "DAW"); // "  Programación DAW  " — substitueix text
```

> 💡 **Detall pràctic:** `length()` és un **mètode** (amb parèntesis). És l'error clàssic del novell escriure `texto.length` sense parèntesis i que no compili. En canvi, per a un array (la U06) s'usa `.length` sense parèntesis. Els `String` porten parèntesis; els arrays, no.

🔍 Els mètodes que busquen

Quan necessites saber si alguna cosa està dins del text:

```
String email = "ana@instituto.edu";
boolean tieneArroba = email.contains("@");          // true
boolean esDeEdu = email.endsWith(".edu");           // true
boolean empiezaPorAna = email.startsWith("ana");    // true
int posicionArroba = email.indexOf("@");                // 3 — el @ està en la posició 3
```

> 📝 **Nota:** `indexOf()` retorna la **posició** (començant en 0) on troba el text, o **-1** si no el troba. És el "buscar" de Java.

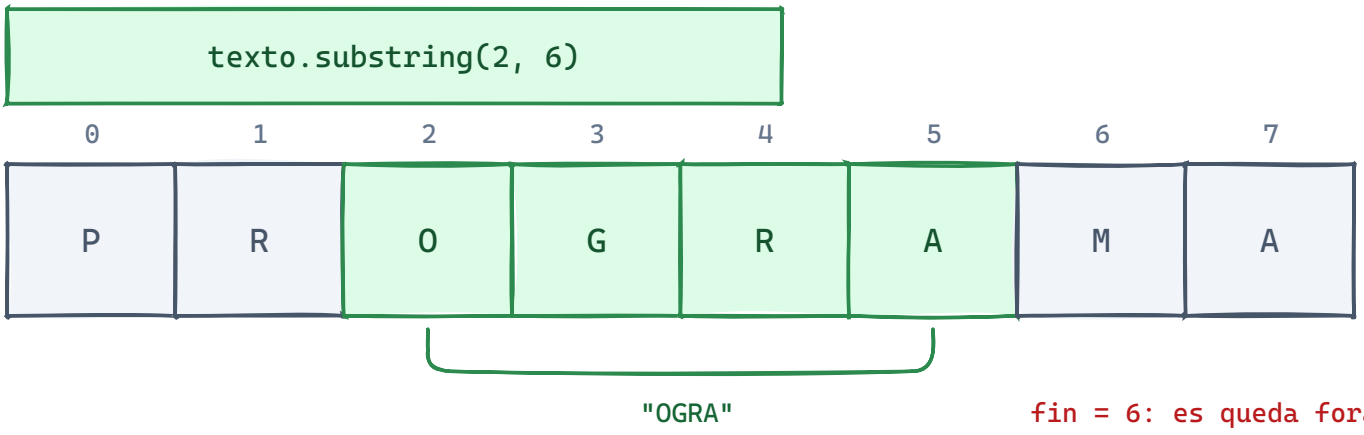
✂️ Els mètodes que retallen

El clàssic: netejar espais i trocejar. `trim()` és l'heroi silenciós dels formularis mal omplits:

```
String sucio = "   Ana   ";
String limpio = sucio.trim();          // "Ana" — sense espais al voltant
String nombreCompleto = "Ana Martínez";
String nombre = nombreCompleto.substring(0, 3);         // "Ana" — del 0 al 3 (sense incloure el 3)
String apellido = nombreCompleto.substring(4);         // "Martínez" — des del 4 fins al final
```

> ⚠️ **Advertència:** en `substring(inicio, fin)`, el `fin` **no s'inclou**. `substring(0, 3)` et dona els caràcters 0, 1 i 2. És un error típic demanar un caràcter de més (o de menys).

substring(inicio, fin): el fin NO s'inclou



🏫 Exemple guiat: el nom de l'usuari

Anem a construir un programa que processi un nom com ho faria un formulari seriós: netejant espais i mostrant dades:

```
public class ProcesaNombre {
    public static void main(String[] args) {
        String nombre = "  aNA  ";

        String limpio = nombre.trim();                    // "aNA"
        String enMayusculas = limpio.toUpperCase();       // "ANA"
        String enMinusculas = limpio.toLowerCase();       // "ana"
        String primera = enMayusculas.substring(0, 1);    // "A"
         String ultima = enMayusculas.substring(enMayusculas.length() - 1);  //
"A"

        System.out.println("Nombre limpio: " + limpio);
        System.out.println("Longitud: " + limpio.length());
        System.out.println("En mayúsculas: " + enMayusculas);
        System.out.println("Primera letra: " + primera);
        System.out.println("Última letra: " + ultima);
    }
}
```

Sortida:

```
Nombre limpio: aNA
Longitud: 3
En mayúsculas: ANA
Primera letra: A
Última letra: A
```

Fixa't en l'última lletra: `enMayusculas.length() - 1` és l'última posició, perquè les posicions comencen en 0.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** encadena els mètodes per a expressions potents: `texto.trim().toUpperCase().substring(0, 1)` fa tres coses en una línia. Java les executa d'esquerra a dreta.

Exercici: la inicial d'una reina

Sense executar, digues què imprimeix aquest codi:

```java
String nombre = "  merida  ";
String inicial = nombre.trim().toUpperCase().substring(0, 1);
String resto = nombre.trim().substring(1).toLowerCase();
System.out.println(inicial + ". " + resto);
```

> › 🔄 **Solució**
>
> Imprimeix `M. erida`.
>
> - `nombre.trim()` → `"merida"` (fora espais).
> - `.toUpperCase()` → `"MERIDA"`.
> - `.substring(0, 1)` → `"M"`. Aquell és el `inicial`.
> - Per al resto: `"merida".substring(1)` → `"erida"`, i `.toLowerCase()` ho deixa igual. Resultat: `M. erida`.
>
> (La idea era normalitzar un nom estil "M. erida"... encara que el resultat sona a princesa amb pressa.)

🎯 Mini-chequeig

1. Porta `String.length()` parèntesis o no? Per què?
2. Què fa `trim()` i quan és imprescindible?
3. Què retorna `indexOf("@")` si el text no conté `@`?
4. Què retorna `"Hola".substring(1, 3)`?

> 🔄 **Respostes**
>
> 1. **Porta parèntesis**: `length()` és un mètode de la classe `String`. (Els arrays usen `.length` sense parèntesis, però això és la UO6.)
> 2. `trim()` elimina els **espais del principi i del final**. És imprescindible en netejar entrades d'usuari que solen portar espais de més.
> 3. **`-1`**, el sentinella de "no trobat".
> 4. `"ol"` — el caràcter 1 ('o') i el 2 ('l'); el 3 no s'inclou.

✅ Resum en 3 frases

1. Els mètodes de `String` es criden sobre la variable (`texto.metodo()`) i transformen el text en alguna cosa nova.
2. `length()`, `trim()`, `toUpperCase()`, `contains()`, `indexOf()`, `substring()` i `replace()` cobreixen el 90% del que faràs amb text.
3. En `substring(inicio, fin)` el fin no s'inclou, i `indexOf()` retorna `-1` quan no troba res.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`length()`	No de caràcters d'un String (amb parèntesis)
`trim()`	Lleva els espais dels extrems
`substring()`	Retalla una porció del text
`indexOf()`	Posició de la primera aparició (o `-1`)
`replace()`	Substituïx una part del text per una altra

Terme	Idea general
Encadenar mètodes	Aplicar diversos mètodes seguits amb punts

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Math.random() i nombres aleatoris · **Següent:** 10 · Repàs interactiu

10 · Repàs interactiu: la caixa que no cabia

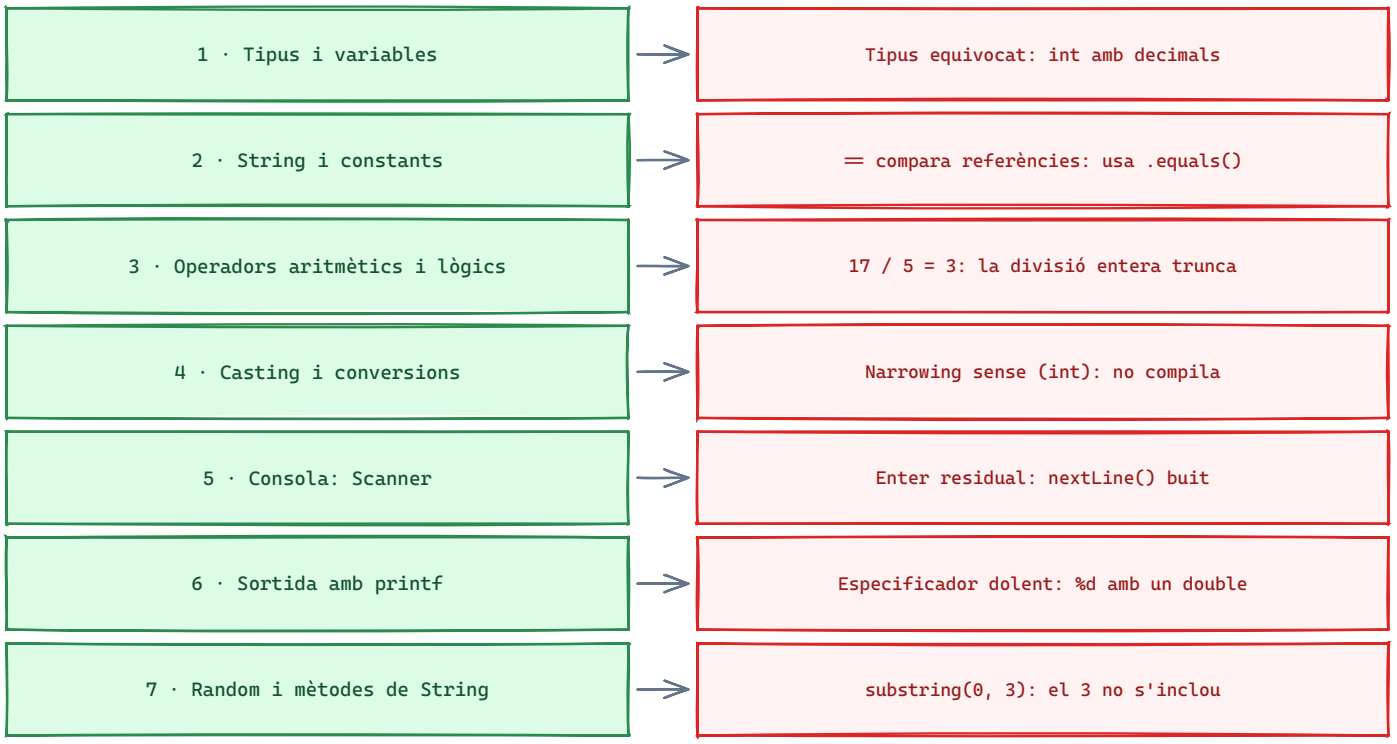
El tancament pràctic de la unitat, amb un bug de truncament que no agradarà a ningú 😈

> 🗺️ **Estàs en: 🔤 U03 · Variables, tipus i operadors** → 10 · Repàs interactiu

📬 La idea en una frase

> En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈

Mapa de la unitat 03: 7 zones i el seu error típic



⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
public class Misterio {
    public static void main(String[] args) {
        int nota = 7;
        int sobre = 10;
        System.out.println("Nota " + nota + sobre);
        System.out.println("Nota " + (nota + sobre));
        System.out.println(nota / 2 + " de nota media");
    }
}
```

Què imprimeixes per pantalla? Tria sàviament:

1. **`Nota 710`, `Nota 17` i `3 de nota media`** → ✅ Correcte! En la primera línia, en veure text abans del +, Java concaten: `"Nota " + 7` és `"Nota 7"` i després `+ 10` dona `"Nota 710"`. En la segona, els parèntesis forcen la suma: `Nota 17`. I `7 / 2` és divisió entera: 3.
2. **`Nota 17`, `Nota 17` i `3.5 de nota media`** → Els parèntesis no canvien res i la divisió entera es arrodoneix. ❌
3. **`Nota 710`, `Nota 17` i `3.5 de nota media`** → La divisió de dos enters dona decimals. ❌

> 🔄 **Solució**
>
> L'opció **1**. Quan un + mescla text i números, Java concaten. Els parèntesis `(nota + sobre)` obliguen a sumar primer. I `7 / 2` amb enters trunca: **3**, no 3.5. Tres trampes de la unitat en un sol programa. Brutal.

🔥 Fireside Chat: int vs double

> *Dos caixes del magatzem discuteixen al costat de la prestatgeria de les dades.*

int: — Jo soc la caixa de mudança. Compacta, exacta, sense decimals. En mi no hi ha lloc per a tonteries. 17 dividit entre 5 són 3 i s'ha acabat.

double: — *alça una cella* ¿3? De debò? Per a mi són 3.4. Jo guardo els decimals de veritat. Els preus, les notes mitjanes, les temperatures… tot això viu a casa meva.

int: — Sí, i quan intentes ficar un número meu a la teva caixa, va tot bé. Però quan intentes ficar tu el teu a la meva… ¡has de demanar permís amb `(int)` i encara perds cèntims pel camí!

double: — *arronsant les espatlles* És el preu de la precisió. Jo guardo més dígits que tu. Saps quantes vegades he vist a novells escriure `(int)(Math.random() * 6 + 1)` i plorar perquè el dau mai no sortia 6?

int: — Val, val… I per a comptar? Per a un bucle? Per a una edat? A mi em criden a mi. Comptar amb decimals no té sentit.

double: — I mesurar, calcular mitjanes i preus, em criden a mi. Som un equip: tu per al sencer, jo per al fi.

int: — *gruny* Un equip. Bah. Però… val. Tu i jo, i `long` per als astronòmics i `char` per a les lletres. Tots al mateix magatzem.

> La lliçó: **`int` és per al sencer, `double` per al decimal**. Confondre'ls en una divisió (o en un casting mal posat) produeix els errors més clàssics d'aquesta unitat.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc el superglue del magatzem: una vegada que fiques alguna cosa en mi, no surt ni amb palanca. M'escriuen en MAJÚSCULES perquè tots em respecten.**
2. **Compari dos valors i només sé dir dues paraules: `true` i `false`. Soc el jutge de la discussió.**
3. **Soc la caixa màgica del text: no soc primitiu, soc una classe, i si intentes canviar-me, tire el vell i en creo un de nou.**
4. **Soc l'orella del programa: esper que escriguis pel teclat i després passi el que has llegit a una variable.**
5. **Soc el casino: et dono un nombre entre 0 i 1, i si em multipliques i em converteixes a `int`, et faig un dau.**

> › 🔄 **Respostes**
>
> 1. **`final`** — El modificador que converteix una variable en constant.
> 2. **Un operador relacional** (==, <, >, …) — Sempre retorna un `boolean`.
> 3. **`String`** — Classe immutable que guarda text.
> 4. **`Scanner`** — Llegeix del teclat amb mètodes `next...`.
> 5. **`Math.random()`** — El generador de nombres aleatoris.

🤬 CONRAD VS EL MÓN: “El compilador odia la teva memòria” 🔗

> *CONRAD, el nostre compilador cascarrabias, opina sobre els clàssics del novell en aquesta unitat.*

CONRAD: — ALTRA VEGADA! Ve un alumne i em dona això: `long distancia = 3000000000;` sense la L. I jo: *això és un int, i no cap, collons.* Però ell no m’escolta. Per què? ¡Perquè creu que ja ho sap tot!

I després el preu: `float precio = 19.99;`. Un `double` solt dins d’una caixa `float`… i li falta la `f`. ¡LA F, HOME! És una lletra, una sola. Tan difícil és? I `char letra = "A"` amb cometes dobles. ¡Les simples, les simples! És com confondre un pis amb una escala.

I el rei del mambo: `String nombre = "Ana";` i després `if (nombre == "Ana")`. PERÒ T’HAS LLEGIT EL PUNT 2? == compara referències, no text. És que t’estic donant l’examen amb les respostes i me’l retornes en blanc.

La lliçó: en aquesta unitat, el compilador és el teu millor amic: `long` necessita L, `float` necessita f, `char` usa cometes simples i els `String` es comparen amb `.equals()`. Aprèn la llista de quatre i t’estalviaràs el 80% de les bronques.

⚡ Laboratori de Tortura: el programa que cobra malament 🔗

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un archiu nou

L’escenari: copia aquest programa al teu IDE i fes que funcioni. És un caixer que calcula quants bitllets de 5 € et dona el banc per un reintegrament. Té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte quan l’arregles.

```java
import java.util.Scanner;

public class Tortura {
    public static void main(string[] args)
        Scanner sc = new Scanner(System.in);
        System.out.print("Cantidad a retirar: ");
        int cantidad = sc.nextInt()
        double billetes5 = cantidad / 5.0;
        System.out.println("Te dan " + billetes5 + " billetes de 5");
        sc.close();
    }
}
```

Fallada intencionada: un dels errors sembla correcte a simple vista perquè "es veu bé", però canvia per complet la sortida del programa.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Prova amb `cantidad = 17`: quants bitllets de 5 haurien de ser?

Pistes per quan et frustres (no abans):

1. Hi ha algun `;` que falti? *no → segueix buscant.*
   > **› I si encara estic atascat?**
   > La línia del `nextInt()` acaba sense punt i coma. El `;` existeix perquè Java sàpiga on acaba cada ordre.

2. Ja compila? *no → mira el missatge d'error i les majúscules.*
   > **› I si encara estic atascat?**
   > Són dues coses: `string` ha de ser `String` (la classe amb majúscula) i falta la `{` que obre el cos del `main`.

3. Executa però el número de bitllets surt amb decimals? *És l'error de lògica: mescla de tipus.*
   > **› Solució final**

Els **3 errors de compilació**:

1. `string[] args` → `String[] args` (la classe `String` amb majúscula).
2. Falta la `{` que obre el cos del `main` després de `main(string[] args)`.
3. Falta el `;` al final de `int cantidad = sc.nextInt()`.

L'**error de lògica**: `double billetes5 = cantidad / 5.0;` compila, però amb `cantidad = 17` imprimeix **3.4** bitllets… i un caixer no et pot donar 3.4 bitllets. La solució és usar **divisió entera**: `int billetes5 = cantidad / 5;`

```
import java.util.Scanner;

public class Tortura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Cantidad a retirar: ");
        int cantidad = sc.nextInt();
        int billetes5 = cantidad / 5;
        System.out.println("Te dan " + billetes5 + " billetes de 5");
        sc.close();
    }
}
```

Per a `cantidad = 17`: `17 / 5` amb enters dona **3** bitllets (i sobren 2 €). Amb la versió trencada, `cantidad / 5.0` donava 3.4, que com a `double` sí que s'imprimeix tal qual: el caixer "et donava 3.4 bitllets". La divisió entera és la que fa el treball net.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **El Guarda del Magatzem**	Explicar els 8 tipus primitius i què cap en cada caixa sense mirar
🏅 **L'Equilibrista**	Resoldre `int y = x++ + ++x;` sense executar i sense plorar
🏅 **El Traductor de Text**	Normalitzar un nom amb `trim().toUpperCase().substring(...)` en una línia
🏅 **El Dau Trucat**	Generar 10 nombres aleatoris entre 1 i 6 sense que surti mai un 0
🏅 **El Coiot del Scanner**	Evitar l'embolic de `nextLine()` després de `nextInt()` sense que ningú t'ho recorde

---

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;
        System.out.println("a/b = " + a / b);
        System.out.println("a/b real = " + (double) a / b);
        System.out.println("a%b = " + a % b);
    }
}
```

2. **El preu que no quadra:** un programa calcula `double total = precio * 0.21;` amb `precio = 100` i mostra `21.000000000000004`. Què li passa a Java? Com ho arreglaries només amb eines d'aquesta unitat?

3. **El ternari encadenat:** escriu un ternari (o diversos encadenats) que assigni a `categoria` el valor `"niño"`, `"adulto"` o `"jubilado"` segons si l'edat és `< 12`, `< 65` o `>= 65`.

4. **Vertader o fals:** "`Math.random() * 5` pot retornar el número 5." Justifica.

> 💡 **Solucions**
>
> 1.

```
a/b = 3
a/b real = 3.3333333333333335
a%b = 1
```

La primera és divisió entera (trunca). La segona força decimal amb `(double) a` abans de dividir.

> 2. Java usa **coma flotant binària**: `0.21` no es pot representar exactament en binari, així que queden residus com `21.000000000000004`. Pots deixar-ho visualment net redonint amb `Math.round(total * 100) / 100.0`, o usar `printf` (ho vas veure en el punt 7) per a donar format a la sortida sense tocar el número.
>
> 3.

```
String categoria = edad < 12 ? "niño" : (edad < 65 ? "adulto" : "jubilado");
```

Els ternaris es poden encadenar: si la primera condició és falsa, avaluem la segona.

4. **Fals.** `Math.random()` retorna entre `0.0` i `0.999...` (l'1 mai no s'inclou). En multiplicar per 5, el màxim és `4.999...`, que després del `(int)` dona 4. `(int)(Math.random() * 5)` dona de **0 a 4**.

🧩 Crucigrama de Bits

```
Horitzontal:
1. Operador que dona el reste d'una divisió (1 lletra)
3. Modificador que fa una variable constant (5 lletres)
5. Classe per a llegir del teclat (7 lletres)
7. Mètode que lleva els espais dels extrems (4 lletres)

Vertical:
2. Tipus que guarda text (6 lletres)
4. Classe amb mètodes estàtics com random i round (4 lletres)
6. Tipus primitiu de vertader o fals (7 lletres)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. % · 3. FINAL · 5. SCANNER · 7. TRIM **Vertical:** 2. STRING · 4. MATH · 6. BOOLEAN

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo tingués vuit anys, què és una variable i què és un tipus primitiu."**
2. **"Per què `double nota = 7/2;` dona `3.0` i no `3.5`? I com ho arreglaries?"**
3. **"Quan usaries `int` i quan `long`? Posa un exemple de cada un."**
4. **"Què és un casting i quins riscos té fer un casting de `double` a `int`?"**
5. **"Com llegeixes un número enter i una línia de text des del teclat sense que el text es quedi buit?"**

🤷 No hi ha preguntes tontes

> ❓ **Per què a vegades posi `L` al final d'un `long` i altres vegades no?**

Perquè depèn del número. Si el número cap en un `int` (màxim 2.147 milions), `long x = 100;` va sense `L` (Java el promociona sol). Si supera aquell límit, `long x = 3000000000;` necessita la `L` perquè Java no intenti ficar-lo en un `int` i es queixe.

> ❓ **És `Math.random()` un mètode de l'objecte `Math`?**

No exactament: `Math` és una **classe**, i `random()`, `pow()`, `round()`… són **mètodes estàtics**. No creïs cap objecte de `Math`; crides directament `Math.random()`. És la diferència entre "cridar la classe" i "cridar l'objecte", que veuràs a fons en la U10.

> ❓ **Si escric `int nota = (int) 7.99;`, em dona 8 per arrodoniment?**

No. El casting **trunca**, no arrodoneix: `(int) 7.99` dona **7**. Per a arrodonir de veritat usa `Math.round(7.99)`. El truncament talla amb destral; el arrodoniment negocia.

> ❓ **Puc sumar un `String` i un número així, sense més?**

Sí: `"Resultado: " + 5` dona `"Resultado: 5"`. Java converteix el número a text i el concaten. Això s'anomena **concatenació**. El problema ve quan t'oblides dels parèntesis: `"Suma: " + 5 + 3` dona `"Suma: 53"`. ¡Els parèntesis són vida!

🎬 Post-Crèdits

El programador novell acaba el seu caixer de bitllets, el compila i l'executa. Funciona. Llavors s'acosta CONRAD, el compilador cascarrabias, amb cara de pocs amics i una caixa etiquetada `double` a la mà.

CONRAD: — Molt bonic el teu caixer. Declares variables, fas castings, llegeixes del teclat… quasi sembla un programa de veritat. Però hi ha un problema.

Novell: — Quin problema?

CONRAD: — *assenyala el codi amb menyspreu* El teu programa només fa UNA cosa. L'executes i s'ha acabat. Si l'usuari vol provar un altre reintegrament… ha de tornar a executar? Els

programes de veritat **decideixen** i **repeteixen**. Trien entre camins i donen vegades fins que s'acaba la condició.

Novell: — I això quan s'aprèn?

CONRAD: — *somriu malvat* La pròxima parada. Ja et veig escrivint el teu primer `if`… i el teu primer bucle infinit. Espera'm, que jo estaré aquí per a riure'm.

El novell guarda el seu projecte, tanca l'IDE i sent que el magatzem de dades ja té amo.

PRÒXIMAMENT EN U04: Estructures de control i excepcions. El moment en què els teus programes deixen de fer *una* cosa i comencen a *decidir* i *repetir*. 🔀

📚 Tornar a l'índex de la unitat · **Anterior:** 09 · Mètodes útils de String · **Següent:** U04 · Estructures de control i excepcions bàsiques

U04 — Estructures de control i excepcions bàsiques

El semàfor, la cinta de córrer i l'assegurança de vida que fan decidir i repetir als teus programes 🔀

Benvingut al semàfor del codi. En la U02 el teu programa va aprendre a parlar i en la U03 a guardar dades. Però parlar i guardar sense decidir és un lloro amb arxivador: repeteix, però no pensa. Aquesta és la unitat on els teus programes comencen a **pensar**.

Fins ara, el teu codi corria en línia recta, de dalt a baix, com un soldadet obedient. En aquesta unitat li donaràs tres superpoders:

1. **Decidir**: "si aprova, puja la nota; si no, a recuperació". Això és `if`, `else` i `switch`.
2. **Repetir**: "imprimeix les taules de multiplicar, tots els nombres de l'1 al 10". Això és `while`, `do-while` i `for`.
3. **Sobrevindre**: "i si l'usuari escriu lletres on s'esperaven nombres?". Això és `try`, `catch`, `throw` i les excepcions.

I, de regal, aprendràs a escapar-te d'un bucle amb `break`, a saltar-te una vegada amb `continue` i a inventar-te les teves pròpies excepcions. Sí, com en la vida real: et queixes d'alguna cosa, i de pas construïxes el teu propi defecte.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Prendre decisions amb **`if`, `else if` i `else`**, anidar-los sense perdre el nord i resumir decisions simples amb el **ternari**.
- Triar entre moltes opcions amb **`switch`**, sense oblidar el `break` ni el `default`.
- Repetir codi amb **`while`** i **`do-while`**, sabent quan s'executa almenys una vegada.
- Comptar vegades amb **`for`**, controlar comptadors i crear **bucles anidats**.
- Escapar-te d'un bucle amb **`break`**, saltar-te una vegada amb **`continue`** i usar **etiquetes** quan calgui.

- Entendre la jerarquia de **`Throwable`, `Exception` i `RuntimeException`** i conèixer les excepcions més comunes de Java.
- Protegir el teu codi amb **`try` / `catch` / `finally`** i capturar diversos errors alhora.
- Llançar excepcions amb **`throw`** i crear les teves pròpies excepcions amb herència.
- Escriure programes que **decideixin, repeteixin i sobrevisquin**, i depurar-los quan alguna cosa explota.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · if, else if i else	Decisions amb `if/else`, anidats i ternari	Tots
02 · switch	`switch`, `case`, `break` i `default`	Tots
03 · Bucles: while i do-while	Repetir amb `while` i `do-while`, bucles infinits	Tots
04 · Bucle for i bucles anidats	`for` amb comptador i bucles dins de bucles	Tots
05 · break, continue i etiquetes	Sortides del bucle i salt entre etiquetes	Tots
06 · Excepcions bàsiques	`Throwable`, `Exception`, `RuntimeException` i les excepcions típiques	Tots
07 · try, catch i finally	Protegir codi, capturar errors i captura múltiple	Tots
08 · throw i excepcions pròpies	Llançar excepcions i crear les teves	Tots
09 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

---

✅ Criteris d'avaluació coberts (RA3)

RA3: Escriu i depura codi, analitzant i utilitzant les estructures de control del llenguatge.

CE	Criteri	On es cobreix
RA3 a)	S'ha escrit i provat codi que faci ús d'estructures de selecció.	✅ Punts 1 i 2
RA3 b)	S'han utilitzat estructures de repetició.	✅ Punts 3 i 4
RA3 c)	S'han reconegut les possibilitats de les sentències de salt.	✅ Punt 5
RA3 d)	S'ha escrit codi utilitzant control d'excepcions.	✅ Punts 6, 7 i 8
RA3 e)	S'han creat programes executables utilitzant diferents estructures de control.	✅ Tots
RA3 f)	S'han provat i depurat els programes.	✅ Tots, amb molta cosa extra en els butlletins
RA3 g)	S'ha comentat i documentat el codi.	✅ Tots

> 📌 Els bucles que veus aquí són el múscul de tot el que ve: els recorreguts d'arrays i col·leccions (U12) i els algoritmes d'ordenació i cerca (U08) no serien res sense `for` i `while`.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites les variables i el `Scanner` de la U03.
- Ja saps decidir i només vols repetir? → Ves directe al punt 3 i al punt 4.
- El teu programa es trenca i no saps per què? → Els punts 6, 7 i 8 t'ensenyen a llegir els crits de Java.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · if, else if i else

⏭️ **En acabar la unitat, continua en U05 · Funcions i mètodes.**

01 — if, else if i else: el semàfor del codi

Quan el teu programa aprèn a decidir (i a prendre's els semàfors seriosament) 🚦

> 🗺️ **Estàs en:** 🔀 **U04 · Estructures de control i excepcions bàsiques** → 01 · if, else if i else

---

📬 La idea en una frase

> **`if` és el semàfor del codi: si la condició és `true`, deixa passar al bloc; si és `false`, el redirigeix al `else` (o es queda esperant).**

En la U03 els teus programes jutjaven amb operadors relacionals i ternaris, però aquella justícia durava una línia. Ara arriba la justícia de debò: blocs sencers de codi que s'executen o no segons el que decideixi Java.

🚦 El semàfor: if i else

L'estructura bàsica és aquesta:

```
if (condicio) {
    // codi que s'executa si condicio és true
} else {
    // codi que s'executa si condicio és false
}
```

```
int edat = 17;

if (edat >= 18) {
    System.out.println("Pots votar.");
} else {
    System.out.println("Encara no pots votar.");
}
```

> ⚠️ **Advertència:** el `else` és **opcional**. Un `if` sol, sense `else`, és perfectament legal: si la condició falla, Java continua com si res.

🔀 else if: quan hi ha més de dos camins

I si el semàfor té tres colors? Aquí entra el `else if`, que s'encadena:

```
int nota = 7;

if (nota >= 9) {
    System.out.println("Excel·lent");
} else if (nota >= 7) {
    System.out.println("Notable");
} else if (nota >= 5) {
    System.out.println("Aprovat");
} else {
    System.out.println("Suspés");
}
```

Java avalua les condicions **en ordre, de dalt a baix**. Tan bon punt una dona `true`, s'executa el seu bloc i **es salta la resta**. El `else` final atrapa tots els que no entraren.

> 💡 **Detall pràctic:** l'ordre importa. Si començares per `nota >= 5`, la nota 7 entraria en l'"Aprovat" i mai no arribaria al "Notable". Ordena les condicions de la més exigent a la més permissiva.

🪆 If anidats: semàfors dins de semàfors

Un `if` pot viure dins d'un altre. Útil quan vols decidir *dins* d'una decisió:

```
int edat = 20;
boolean teCarnet = true;

if (edat >= 18) {
    if (teCarnet) {
        System.out.println("Pots conduir.");
    } else {
        System.out.println("Et falta el carnet.");
    }
} else {
    System.out.println("Encara no pots conduir.");
}
```

> ⚠️ **Advertència:** no converteixis els teus programes en les Torres Kio. Més de 3 nivells d'anidament és senyal que estàs fent les coses estrany: en la UO7 aprendràs a aplanar-ho.

🎚️ El ternari: el if de butxaca

Del punt 4 de la UO3 el coneixes com "un if-else en una línia". Aquí està el seu moment de glòria:

```
int edat = 21;
String missatge = (edat >= 18) ? "Major d'edat" : "Menor d'edat";
System.out.println(missatge);
```

La regla d'or: ternari per a **assignar un valor** en una línia; `if/else` quan el bloc és llarg o fa més que assignar.

```
// ✅ Bé: ternari per a triar valor
double preuFinal = (dia.equals("divendres")) ? preu * 0.9 : preu;

// ✅ Bé: if quan hi ha diverses línies per branca
if (saldo < 0) {
    System.out.println("Estàs en números rojos.");
    System.out.println("Revisa les teues despeses.");
} else {
    System.out.println("Saldo sa.");
}
```

🏫 Exemple guiat: la discoteca municipal

Anem a muntar el control d'accés d'una discoteca amb entrada reduïda:

```java
public class Discoteca {
    public static void main(String[] args) {
        int edat = 16;
        boolean acompanyat = true;

        if (edat >= 18) {
            System.out.println("Entra, major d'edat.");
        } else if (edat >= 16 && acompanyat) {
            System.out.println("Entra, però amb el teu acompanyant.");
        } else {
            System.out.println("Ho sent, torna en uns quants anys.");
        }
    }
}
```

Sortida:

```
Entra, però amb el teu acompanyant.
```

⭐ Be the Code, my friend… 🔗

> 🕶️ **Don Tip:** quan vegis una cadena d'`if/else if`, pregunta't sempre: *l'ordre és de la condició més estricta a la més laxa?* Aquell és el 90% dels bugs d'aquesta unitat.

Exercici: el semàfor confús

Sense executar, calcula què imprimeix aquest programa:

```java
public class Semafor {
    public static void main(String[] args) {
        int nota = 8;
        String resultat;

        if (nota >= 5) {
            resultat = "Aprovat";
        } else if (nota >= 7) {
            resultat = "Notable";
        } else if (nota >= 9) {
            resultat = "Excel·lent";
        } else {
            resultat = "Suspés";
        }

        System.out.println(resultat);
    }
}
```

> 🔄 **Solució**
>
> Imprimeix **`Aprovat`**. L'ordre està invertit: com que el primer `if` demana `nota >= 5` i 8 ho compleix, Java entra aquí i no mira les altres condicions, encara que 8 també compliria `nota >= 7` i `nota >= 9`. Les condicions correctes anirien de la més exigent (9) a la més permissiva (5). La lliçó: **el primer `if` que es compleix guanya**, encara que no sigui el que volies.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què fa Java si un `if` és `false` i no hi ha `else`?
2. En quin ordre has d'encadenar les condicions d'un `else if`?
3. Quan prefereixes un ternari a un `if/else`?
4. Què imprimiria `String s = 5 > 3 ? "A" : "B";`?

> 🔄 **Respostes**
>
> 1. Segueix executant la següent línia: l'`if` s'ignora en silenci.
> 2. De la més **exigent** a la més **permissiva**, perquè el primer `true` es queda amb la decisió.

3. Quan només vols **assignar un valor** en una línia i les dues branques són curtes.
4. **`"A"`** — perquè 5 > 3 és `true`.

✅ Resum en 3 frases

1. `if` / `else if` / `else` són el **semàfor** del codi: executen un bloc o un altre segons una condició booleana.
2. Les condicions s'avaluen **en ordre** i guanya la primera que doni `true`, així que ordena de la més estricta a la més laxa.
3. El **ternari** resumeix un if-else de dos valors en una línia, però per a blocs llargs usa l'`if` clàssic.

🐛 Vocabulari ràpid

Terme	Idea general
Condició	Expressió booleana que decideix: `edat >= 18`
Branca	Cada un dels camins possibles (`if`, `else`)
Anidar	Ficar un `if` dins d'un altre `if`
Ternari	`condició ? valor1 : valor2`, un if-else en una línia
Curtcircuit	Java deixa d'avaluar quan la primera condició ja decideix

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · switch

02 — switch: el menú del restaurant

Triar entre moltes opcions sense omplir el codi de semàfors 🍽️

> 🗺️ **Estàs en:** 🔀 **U04 · Estructures de control i excepcions bàsiques** → 02 · switch

📬 La idea en una frase

> **`switch` és la carta d'un restaurant: mires el valor d'una variable i executes el `case` que coincideixi, sense encadenar vint `if`.**

Quan has de triar entre moltes opcions amb un sol valor (dia de la setmana, talla, menú), una cadena d'`else if` funciona però és lletja. `switch` existeix per a això.

🍽️ La carta del restaurant

```java
int dia = 3;

switch (dia) {
    case 1:
        System.out.println("Dilluns");
        break;
    case 2:
        System.out.println("Dimarts");
        break;
    case 3:
        System.out.println("Dimecres");
        break;
    default:
        System.out.println("Dia desconegut");
        break;
}
```

> ⚠️ **Advertència:** el **`break` és obligatori** (a menys que vulguis "fall-through", que veurem a baix). Sense ell, Java entra en el `case` correcte i segueix executant tots els següents fins a trobar un `break`. És el clàssic bug del novell.

🧱 Les peces del puzle

- **`switch (variable)`**: la variable que s'examina. Admet tipus enters, `char` i `enum`; a partir de Java 7 també `String`.
- **`case valor:`**: cada opció possible. Si la variable coincideix, s'executa aquell bloc.
- **`break;`**: "fins aquí he arribat, surt del switch". Sense ell, tot es desborda cap avall.
- **`default:`**: el comodí, el "cap dels anteriors". És opcional, com l'`else`.

```
String talla = "M";

switch (talla) {
    case "S":
        System.out.println("Xicoteta");
        break;
    case "M":
        System.out.println("Mitjana");
        break;
    case "L":
        System.out.println("Gran");
        break;
    default:
        System.out.println("Talla no vàlida");
        break;
}
```

> 💡 **Detall pràctic:** usa `switch` quan compares **una variable amb molts valors concrets**. Usa `if/else if` quan les condicions siguin rangs ("més gran que 5", "entre 10 i 20") o mesclen variables.

🌀 El fall-through: error o superpoder?

El famós "caure a través" ocorre quan oblides el `break`. En la majoria dels casos és un bug:

```java
// ⚠️ Fall-through ACCIDENTAL: imprimeix els tres plats
int plat = 1;

switch (plat) {
    case 1:
        System.out.println("Amanida");
        // sense break: es cau al següent
    case 2:
        System.out.println("Sopa");
        break;
}
```

Però a vegades s'usa a propòsit, per a agrupar casos:

```java
// ✅ Fall-through INTENCIONAL: diversos casos comparteixen bloc
char lletra = 'a';

switch (lletra) {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        System.out.println("Vocal");
        break;
    default:
        System.out.println("Consonant");
        break;
}
```

Aquí, si `lletra` és qualsevol vocal, executa el bloc compartit. Elegant i compacte.

🆚 switch vs else if: el duel

Situació	Millor opció
Un valor i moltes opcions concretes (`1..7`, `"S"/"M"/"L"`)	`switch`
Rangs o comparacions (`>= 18`, `entre 10 i 20`)	`if/else if`

Situació	Millor opció
Combinar diverses variables	`if/else if`
Comprovar `null`	`if`

> 💡 **Nota de futur:** en Java 14+ existeix el `switch` amb fletxes (`->`) que no necessita `break` i retorna valors. Ho veuràs com a curiositat avançada; aquí aprenem el clàssic, que és el de tots els exàmens.

🏫 Exemple guiat: el menú del dia

Muntem una carta que digui el plat segons el dia, amb un `default` que caça els despistats:

```
public class MenuDia {
    public static void main(String[] args) {
        String dia = "dimecres";

        switch (dia) {
            case "dilluns":
                System.out.println("Llenties");
                break;
            case "dimarts":
                System.out.println("Paella");
                break;
            case "dimecres":
                System.out.println("Macarrons");
                break;
            case "dijous":
                System.out.println("Fabada");
                break;
            case "divendres":
                System.out.println("Peix");
                break;
            default:
                System.out.println("Cap de setmana: no hi ha menú");
                break;
        }
    }
}
```

Sortida:

```
Macarrons
```

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan vegis un `switch`, compta els `break`: **n'hi ha d'haver un per cada case no compartit**. Si en falta algun, el teu programa es converteix en un tobogan.

Exercici: el switch oblidadís

Sense executar, calcula què imprimeix aquest programa:

```
public class Tobogan {
    public static void main(String[] args) {
        int numero = 2;

        switch (numero) {
            case 1:
                System.out.println("U");
            case 2:
                System.out.println("Dos");
            case 3:
                System.out.println("Tres");
                break;
            default:
                System.out.println("Altre");
                break;
        }
    }
}
```

> › 🔄 **Solució**
>
> Imprimeix:
>
> ```
> Dos
> Tres
> ```

> El `case 2` no té `break`, així que després d'imprimir "Dos" es cau al `case 3` ("Tres") i allà sí que troba el `break` i es deté. Fixa't: el `case 1` no imprimeix res perquè `numero` no val 1. Un sol `break` oblidat converteix el switch en un tobogan.

🎯 Mini-chequeig

1. Què passa si un `case` no té `break`?
2. Què fa el `default`?
3. Quan prefereixes `switch` a una cadena d'`else if`?
4. Com agrupes diversos valors en el mateix bloc de `switch`?

> 🔄 **Respostes**
>
> 1. Es produeix el **fall-through**: el codi segueix executant els `case` següents fins a trobar un `break`.
> 2. És el comodí: s'executa si **cap** `case` coincideix. És opcional.
> 3. Quan compares **un valor amb moltes opcions concretes** (nombres, `char`, `String`).
> 4. Escrivint els `case` seguits sense `break` entre ells i un sol bloc al final.

✅ Resum en 3 frases

1. `switch` tria entre **moltes opcions concretes** d'una variable, amb `case`, `break` i `default`.
2. Sense `break` es produeix el **fall-through** (el codi es desborda), que pot ser un bug o un truc per a agrupar casos.
3. Usa `switch` per a valors exactes i `if/else if` per a **rangs** i condicions combinades.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
`switch`	Estructura que tria entre diversos `case` segons un valor
`case`	Cada opció concreta a comparar
`break`	Ordre de sortida: talla el switch

Terme	Idea general
`default`	El “cap dels anteriors”, opcional
Fall-through	Que el codi es desborde d’un `case` al següent

📚 Tornar a l’índex de la unitat · **Anterior:** 01 · if, else if i else · **Següent:** 03 · Bucles: while i do-while

03 — Bucles: while i do-while

Repetir sense copiar i enganxar, fins que una condició digui prou 🏃

> 🗺️ **Estàs en: 🔀 U04 · Estructures de control i excepcions bàsiques** → 03 · Bucles: while i do-while

---

📬 La idea en una frase

> **Un bucle és una cinta de córrer: executa el mateix bloc una vegada i una altra mentre la condició sigui `true`.**

T'imagines escriure "imprimeix de l'1 al 100" amb cent `println`? Copiar i enganxar és pecat. Els bucles fan la faena bruta per tu: repeteixen un bloc fins que els dius prou.

🏃 while: comprova i després corre

```
while (condicio) {
    // bloc que es repeteix
}
```

El `while` primer **mira la condició** i, si és `true`, executa el bloc. En acabar, torna a mirar. Si és `false` des del principi… **no executa res**.

```
int intents = 3;

while (intents > 0) {
    System.out.println("Reintentant... queden " + intents);
    intents = intents - 1;
}
```

Sortida:

```
Reintentant... queden 3
Reintentant... queden 2
Reintentant... queden 1
```

> ⚠️ **Advertència:** si oblides la línia que modifica la condició (`intents = intents - 1;`), la condició és `true` per sempre i el teu programa **no acaba mai**. Benvingut al bucle infinit, el cotxe sense frens de la programació.

♾️ El bucle infinit (i com sortir-ne)

```
while (true) {
    System.out.println("Socors");
}
```

Aquest programa imprimiria "Socors" fins que l'univers es congele. En l'IDE, el botó de parar (🟥) és el teu millor amic. Per què existeix `while (true)`? Perquè a vegades vols un bucle "per sempre" que es trenqui a l'interior amb `break` (ja ho veuràs en el punt 5).

> 💡 **Detall pràctic:** la sentència sentinella. Un clàssic és llegir dades fins que l'usuari escrigui "sortir":

```
String resposta = "";
Scanner sc = new Scanner(System.in);

while (!resposta.equals("eixir")) {
    System.out.print("Digues una cosa (o 'eixir'): ");
    resposta = sc.nextLine();
}
System.out.println("Adéu.");
```

🏃 do-while: corre i després comprova

```
do {
    // bloc
} while (condicio);
```

La diferència amb `while` és l'**ordre**: el `do-while` executa el bloc **almenys una vegada** i comprova la condició al final. Útil quan necessites preguntar sí o sí abans de decidir:

```java
int opcio;
Scanner sc = new Scanner(System.in);

do {
    System.out.println("1. Jugar  2. Eixir");
    System.out.print("Tria: ");
    opcio = sc.nextInt();
} while (opcio != 1 && opcio != 2);

System.out.println("Has triat l'opció " + opcio);
```

Aquí el menú es mostra **sempre almenys una vegada**, i es repeteix mentre l'usuari no triï 1 o 2. Perfecte per a menús.

> ⚠️ **Advertència:** no confonguis els dos. Amb `while`, si la condició és `false` d'entrada, **zero execucions**. Amb `do-while`, **almenys una**. És com la diferència entre "mira abans de creuar" i "creua i després mira".

🆚 while vs do-while: el duel ràpid

Situació	Bucle ideal
No saps si tocarà executar el bloc	`while`
El bloc s'ha d'executar sí o sí una vegada	`do-while`
Llegir dades fins que l'usuari doni el sentinella	`while`
Mostrar un menú fins que triï una opció vàlida	`do-while`

🏫 Exemple guiat: el compte enrere

Anem a llançar un coet amb un compte enrere. Com que s'ha d'imprimir el "Enlairament!" encara que el comptador comenci en 0… usem `do-while`:

```java
public class CompteArrere {
    public static void main(String[] args) {
        int comptador = 5;

        do {
            System.out.println(comptador);
            comptador--;
        } while (comptador >= 0);

        System.out.println("Enlairament! 🚀");
    }
}
```

Sortida:

```
5
4
3
2
1
0
Enlairament! 🚀
```

⭐ Be the Code, my friend… 🔗

> 🕶️ **Don Tip:** quan vegis un bucle, pregunta't: *la condició avança cap a `false` en algun moment?* Si la resposta és "no", tens un bucle infinit.

Exercici: el comptador parat

Sense executar, calcula quantes vegades imprimeix "Hola" aquest programa… o si es penja:

```java
public class Comptador {
    public static void main(String[] args) {
        int x = 10;

        while (x > 0) {
            System.out.println("Hola");
            x = x + 1;
        }
    }
}
```

> 🔄 **Solució**
>
> **Bucle infinit.** `x` comença en 10 i en comptes de disminuir, s'incrementa (`x = x + 1`): la condició `x > 0` és `true` per sempre i el programa imprimeix "Hola" eternament. La correcció seria `x = x - 1;`. Pista visual: un comptador que puja en un `while` que demana que baixi és fum a l'ordinador.

🎯 Mini-chequeig

1. Quantes vegades s'executa el bloc d'un `while` si la condició és `false` des del principi?
2. I en un `do-while`?
3. Què és un bucle infinit i com s'surt d'ell en l'IDE?
4. Quan usaries un `do-while` per a un menú?

> 🔄 **Respostes**
>
> 1. **Zero vegades**: el `while` comprova abans d'executar.
> 2. **Almenys una vegada**: el `do-while` executa i comprova després.
> 3. Un bucle la condició del qual mai no passa a `false`. Es talla amb el botó de **parar** (🟥) de l'IDE, i s'evita assegurant que alguna cosa modifiqui la condició dins.
> 4. Quan vulguis mostrar el menú **sempre almenys una vegada** i repetir-lo fins que l'usuari triï una opció vàlida.

✅ Resum en 3 frases

1. `while` comprova la condició **abans** d'executar i `do-while` **després**: el segon garanteix almenys una execució.
2. Un bucle necessita que la condició **avanci cap a `false`**; si no, tens un bucle infinit.
3. Usa `while` per a llegir fins a un sentinella i `do-while` per a menús que s'han de mostrar sí o sí.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Bucle	Bloc que es repeteix mentre una condició sigui `true`
Iteració	Una vegada completa del bucle
Condició	L'expressió booleana que decideix si es continua
Sentinella	Valor especial que acaba la lectura ("sortir")
Bucle infinit	Bucle que no s'acaba mai per descuit
do-while	Bucle que executa almenys una vegada i comprova al final

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · switch · **Següent:** 04 · Bucle for

04 — Bucle for i bucles anidats

El comptador obsessiu que repeteix un nombre exacte de vegades 🔢

> 🗺️ **Estàs en:** 🔀 **U04 · Estructures de control i excepcions bàsiques** → 04 · Bucle for i bucles anidats

---

📬 La idea en una frase

> **`for` és un bucle amb comptador de sèrie: declara la variable, posa la condició i l'actualitza en la mateixa línia, ideal per a "repeteix N vegades".**

El `while` repetia "mentres passi alguna cosa". El `for` repeteix "un nombre exacte de vegades". És el bucle favorit per a recórrer coses i el que més usaràs en tota la teva carrera.

🔢 L'anatomia del for

```
for (inicialitzacio; condicio; actualitzacio) {
    // bloc
}
```

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Volta " + i);
}
```

Sortida:

```
Volta 1
Volta 2
Volta 3
Volta 4
Volta 5
```

El cicle de vida de `i`:

1. **Inicialització**: `int i = 1` — s'executa una sola vegada, en entrar.
2. **Condició**: `i <= 5` — es comprova abans de cada vegada.

3. **Bloc**: s'executa si la condició és `true`.
4. **Actualització**: `i++` — s'executa al final de cada vegada.
5. Tornada al pas 2.

> ⚠️ **Advertència:** una coma mal posada en el `for` és l'error d'examen més comú. La sintaxi és: inicialització `;` condició `;` actualització. Tres parts, dos punts i coma, zero comes entre elles.

🔁 Els tres bucles són el mateix acudit

Això és `while`, `do-while` i `for` fent exactament el mateix:

```
int i = 1;                          // inicialització
while (i <= 5) {                     // condició
    System.out.println("Volta " + i);
    i++;                             // actualització
}
```

```
int i = 1;
do {
    System.out.println("Volta " + i);
    i++;
} while (i <= 5);
```

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Volta " + i);
}
```

Els tres imprimeixen el mateix. El `for` guanya perquè junta les tres parts del control en una línia: és més difícil oblidar el `i++` (adéu, bucles infinits per descuit).

> 💡 **Detall pràctic:** si saps quantes vegades repetiràs → `for`. Si no ho saps → `while`. Regla que et salvarà la vida en l'examen.

🧩 Bucles anidats: la graella d'exercicis

Un bucle dins d'un altre. Per cada vegada del bucle **exterior**, el **interior** s'executa complet.

```java
for (int fila = 1; fila <= 3; fila++) {
    for (int columna = 1; columna <= 4; columna++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Sortida:

```
* * * *
* * * *
* * * *
```

- El bucle exterior controla les **files** (3).
- El interior controla les **columnes** (4).
- El `println()` sense text després de l'interior salta de línia en acabar cada fila.

> ⚠️ **Advertència:** cada nivell d'anidament multiplica les vegades. Amb 1000 files i 1000 columnes són 1.000.000 d'iteracions. Els bucles anidats són potents, però també la fàbrica de programes lents.

🏫 Exemple guiat: la taula de multiplicar

```java
public class TaulaMultiplicar {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            System.out.println("7 x " + i + " = " + (7 * i));
        }
    }
}
```

Sortida (primeres línies):

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
```

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan uses un `for`, pregunta: *la condició usa < o <=?* Un descuit d'un dígit (repetir 9 vegades en comptes de 10) és el bug de l'off-by-one, el més famós de la història.

Exercici: el triangular

Sense executar, calcula quants asteriscs imprimeix en total aquest programa:

```
public class Triangle {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int ast = 1; ast <= fila; ast++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

> 🔄 **Solució**
>
> Imprimeix:
>
> ```
> *
> **
> ***
> ****
> ```
>
> En total, **10 asteriscs** (1 + 2 + 3 + 4). Fixa't en el truc: la condició de l'interior és `ast <= fila`, així que cada fila imprimeix tants asteriscs com número de fila. El bucle interior depèn del valor de l'exterior: això és el cor dels bucles anidats.

🎯 Mini-chequeig

1. Quines són les tres parts del `for`?
2. Quantes vegades s'executa la inicialització?
3. Què imprimeix `for (int i = 0; i < 5; i++)`? 5 o 4 vegades?
4. En un bucle anidat, què fa el bucle interior per cada vegada de l'exterior?

> 🔄 **Respostes**
>
> 1. **Inicialització**, **condició** i **actualització**, separades per `;`.
> 2. **Una sola vegada**, en entrar en el bucle.
> 3. **5 vegades**: amb `i` valent 0, 1, 2, 3 i 4 (quan `i` arriba a 5, la condició falla).
> 4. S'executa **complet** (totes les seves vegades) per cada vegada de l'exterior.

✅ Resum en 3 frases

1. `for` junta **inicialització, condició i actualització** en una línia: ideal per a repetir un nombre conegut de vegades.
2. `while`, `do-while` i `for` són intercanviables; tria `for` quan sàpigues les vegades.
3. En els **bucles anidats**, per cada vegada de l'exterior el interior s'executa complet, i això multiplica el treball.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
`for`	Bucle amb comptador: `for (inici; condicio; avanc)`
Comptador	Variable que compta les vegades (normalment `i`)
Bucle anidat	Un bucle dins d'un altre
Off-by-one	Fallar per un en la condició (< vs <=)
Iteració	Una vegada del bucle

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Bucles: while i do-while · **Següent:** 05 · break i continue

05 — break, continue i etiquetes

Els botons d'emergència i els salts de línia del bucle 🚪

> 🗺️ **Estàs en:** 🔀 **U04 · Estructures de control i excepcions bàsiques** → 05 · break, continue i etiquetes

---

📬 La idea en una frase

> **`break` apaga el bucle sencer i `continue` es salta només la volta actual; amb etiquetes pots decidir a quin bucle anidat afecten.**

Ja saps repetir. Ara toca aprendre a **sortir amb estil**: interrompre, saltar i dirigir-te a un bucle concret quan n'hi ha diversos.

🚪 break: el botó de parada

`break` acaba el bucle **immediatament**, sense comprovar la condició:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
```

Sortida:

```
1
2
3
4
```

Tan bon punt `i` val 5, `break` talla el bucle: les vegades 6 a 10 no ocorren mai. És perfecte per a "troba alguna cosa i para de buscar".

> 💡 **Detall pràctic:** el `break` dins d'un `switch` (punt 2) tallava el switch. El `break` dins d'un bucle talla el bucle. Mateix botó, diferent aparell.

⏭️ continue: el botó de saltar

`continue` no acaba el bucle: **salta directament a la següent volta**, ignorant la resta del bloc:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
```

Sortida:

```
1
2
4
5
```

El 3 es salta, però el bucle continua. Útil per a "no processis aquests valors, però segueix amb els altres":

```
// Suma només els nombres parells de l'1 al 10
int suma = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 != 0) {
        continue;      // senars: no compten
    }
    suma += i;
}
System.out.println("Suma de parells: " + suma);   // 2+4+6+8+10 = 30
```

> ⚠️ **Advertència:** en un `while`, si poses el `continue` **abans** d'actualitzar la variable del bucle, l'actualització es salta… i el bucle no avança. Bug infinit assegurat. En un `for` l'actualització és a la capçalera i no passa res.

🏷️ Etiquetes: el GPS dels bucles anidats

Un `break` o `continue` solts afecten **només el bucle més intern**. I si vols sortir de dos bucles alhora? Aquí neixen les **etiquetes**:

```
exterior:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i * j >= 6) {
            break exterior;    // eix dels DOS bucles
        }
        System.out.println(i + " x " + j);
    }
}
```

Sortida:

```
1 x 1
1 x 2
1 x 3
2 x 1
2 x 2
```

Quan `i * j >= 6`, el `break exterior` salta fora de l'etiqueta, acabant els dos bucles alhora. Sense etiqueta, el `break` només hauria sortit del bucle de `j`.

```
exterior:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) {
            continue exterior;   // salta a la següent i
        }
        System.out.println(i + "-" + j);
    }
}
```

Sortida:

```
1-1
2-1
3-1
```

> ⚠️ **Advertència:** les etiquetes són legals però poc usades. Davant el dubti, quasi sempre es pot redissenyar amb una variable booleana. Usa etiquetes amb moderació: el teu company de projecte t'ho agrairà.

🏫 Exemple guiat: el detector de nombre primer

Usem `break` per a comprovar si un nombre és primer de manera eficient:

```
public class EsPrimer {
    public static void main(String[] args) {
        int numero = 29;
        boolean esPrimer = true;

        for (int divisor = 2; divisor < numero; divisor++) {
            if (numero % divisor == 0) {
                esPrimer = false;
                break;   // trobat divisor: para de buscar
            }
        }

        System.out.println(numero + " és primer? " + esPrimer);
    }
}
```

Sortida:

```
29 és primer? true
```

Amb `break`, tan bon punt apareix un divisor deixem de comprovar. Per al 29 no hi ha divisors, així que el bucle es recorre sencer i `esPrimer` continua sent `true`.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** per a distingir-los d'un cop d'ull: `break` = **apaga** el bucle; `continue` = **salta** aquesta volta. Un acaba la festa, l'altre només es perd una cançó.

Exercici: el bucle esquizofrènic

Sense executar, escriu la sortida exacta:

```
public class Esquizofrenia {
    public static void main(String[] args) {
        for (int i = 1; i <= 8; i++) {
            if (i % 3 == 0) {
                continue;
            }
            if (i == 7) {
                break;
            }
            System.out.println(i);
        }
    }
}
```

> 🔄 **Solució**

```
1
2
4
5
```

Pas a pas: de l'1 al 8, `continue` es salta els múltiples de 3 (3 i 6), i `break` talla en el 7 (que tampoc no arribaria a imprimir-se). Queden l'1, el 2, el 4 i el 5. El 8 mai no s'avalua perquè el `break` de `i == 7` va apagar el bucle abans.

🎯 Mini-chequeig

1. Quina és la diferència entre `break` i `continue` en una frase?
2. A què afecten per defecte en bucles anidats?
3. Per a què serveix una etiqueta?
4. Per què és perillós el `continue` en un `while` si va abans de l'actualització?

> 🔄 **Respostes**

1. `break` **acaba** el bucle; `continue` **salta només la volta actual**.
2. Al bucle **més intern**.

3. Perquè un `break` o `continue` afecte un bucle exterior concret (`break etiqueta;`).
4. Perquè l'actualització es salta i el bucle **no avança**: condició `true` per sempre.

✅ Resum en 3 frases

1. `break` apaga el bucle sencer i `continue` es salta només la volta actual.
2. En bucles anidats afecten el més intern; les **etiquetes** et deixen apuntar a un bucle exterior.
3. Compte amb `continue` abans de l'actualització en `while`: és un bucle infinit en potència.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`break`	Acaba el bucle (o el switch) immediatament
`continue`	Salta a la següent volta del bucle
Etiqueta	Nom que poses a un bucle per a saltar-hi
`break etiqueta`	Eix del bucle etiquetat, no del més intern
Bucle infinit	Risc si `continue` es salta l'actualització d'un `while`

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Bucle for · **Següent:** 06 · Excepcions bàsiques

06 — Excepcions bàsiques

Quan Java es queixa per escrit: la família Throwable i les queixes més típiques 💥

> 🗺️ **Estàs en:** 🔀 **UO4 · Estructures de control i excepcions bàsiques** → 06 · Excepcions bàsiques

📬 La idea en una frase

> **Una excepció és un avís que alguna cosa ha sortit malament; Java el llança com un objecte que hereta de `Throwable`, i el teu programa pot estar o no preparat per a atrapar-lo.**

T'ha passat que un programa es "cau" amb un munt de text vermell? Aquell text és una excepció. En comptes de morir en silenci, Java crida amb tot el detall. Aprendre a llegir aquells crits és aprendre a depurar.

💥 El crash: la teva primera excepció

Executa això:

```
public class Explosio {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3};
        System.out.println(numeros[5]);   // no existeix!
    }
}
```

Java no es calla. Apareix una cosa així:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5
out of bounds for length 3
  at Explosio.main(Explosio.java:5)
```

Aquest text és or: et diu **què** excepció (`ArrayIndexOutOfBoundsException`), **on** (línia 5) i en **quin mètode**. Llegir-lo bé resol la mitat dels teus problemes.

🌳 La família Throwable: l'arbre genealògic

Totes les excepcions hereten d'una classe mare:

```
Object
 └── Throwable
      ├── Error                  (coses que no hauries d'intentar arreglar)
      └── Exception              (el que de veritat ens interessa)
           ├── RuntimeException  (excepcions en temps d'execució)
           └── (altres excepcions "controlades")
```

- **`Error`**: problemes greus de la JVM (memòria esgotada, per exemple). No els provoques tu i no has d'intentar atrapar-los. Ignora'ls.
- **`Exception`**: fallades del programa. Aquí viu el 99% de la teva vida.
- **`RuntimeException`**: subfamília d'`Exception` que es llança en **temps d'execució** i que **no estàs obligat** a capturar. Aquí viuen les més famoses.

```java
// Totes estes són RuntimeException (no necessites try perquè compile):
int x = 10 / 0;                         // ArithmeticException
int[] a = new int[3]; a[9] = 1;         // ArrayIndexOutOfBoundsException
String s = null; s.length();            // NullPointerException
int num = Integer.parseInt("Hola");     // NumberFormatException
```

> 💡 **Detall pràctic:** "RuntimeException" significa que l'error apareix quan el programa **corre**, no en compilar. El compilador no t'avisa: només ho descobreixes en plena execució.

🗺️ Les excepcions més comunes: la guia de camp

Excepció	Quan apareix	Frase típica
`ArithmeticException`	Dividir entre 0	"Dividir entre zero, quin valent"
`ArrayIndexOutOfBoundsException`	Índex fora de l'array	"Aquell buit no existeix"
`NullPointerException`	Cridar alguna cosa `null`	"El clàssic absolut"

Excepció	Quan apareix	Frase típica
`NumberFormatException`	Convertir text que no és nombre	“Convertir ‘Hola’ en nombre, no”
`StringIndexOutOfBoundsException`	Índex fora d’un `String`	“substring() més enllà del final”
`InputMismatchException`	`Scanner` rep el tipus equivocat	“Vas posar text on anava un nombre”

> ⚠️ **Advertència:** la `NullPointerException` (NPE) és, amb diferència, l’excepció més comuna de la història de Java. La teva àvia, si programés, també la tindria. El missatge sol ser un críptic “null” seguit de la línia on vas tocar un objecte que no existia.

🏫 Exemple guiat: el lector d’edats

Un programa que demana una edat per teclat pot explotar si l’usuari escriu lletres. Vegem el crash i després l’arreglarem en el punt 7:

```
import java.util.Scanner;

public class LectorEdat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Quants anys tens? ");
        int edat = sc.nextInt();
        System.out.println("Vas nàixer fa " + edat + " anys.");
        sc.close();
    }
}
```

Si escrius `hola`, obtens una `InputMismatchException`. El programa mor. La solució és atrapar l’excepció amb `try/catch`… que és justament el punt 7.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan un programa exploti, llegeix la primera línia de l'error: el nom de l'excepció et diu *què* ha passat, i la línia amb `at ...` et diu *on*. És un GPS amb acusacions.

Exercici: el detectiu d'excepcions

Digues quina excepció llançaria cada línia (o si no llançaria cap):

```
int a = 5 / 0;
String[] dies = {"L", "M", "X"};
System.out.println(dies[3]);
String text = null;
System.out.println(text.toUpperCase());
int b = Integer.parseInt("42");
int c = Integer.parseInt("quaranta-dos");
```

> 🔄 **Solució**
>
> - `5 / 0` → **`ArithmeticException`** (divisió entre zero).
> - `dies[3]` → **`ArrayIndexOutOfBoundsException`** (un array de 3 buits s'indexa 0, 1, 2).
> - `text.toUpperCase()` → **`NullPointerException`** (`text` és `null`).
> - `parseInt("42")` → **Sense excepció**: 42 sí que és un nombre.
> - `parseInt("quaranta-dos")` → **`NumberFormatException`** (aquell text no és un nombre).

🎯 Mini-chequeig

1. Quina classe està a l'arrel de totes les excepcions?
2. Quina diferència hi ha entre `Error` i `Exception`?
3. Què significa que sigui una `RuntimeException`?
4. Quina és l'excepció més famosa de la història i quan apareix?

> 🔄 **Respostes**
>
> 1. **`Throwable`**.
> 2. `Error` són problemes greus de la JVM que no has d'intentar arreglar; `Exception` són fallades del programa que sí que pots capturar.
> 3. Que es llança **en temps d'execució** (no en compilar) i que **no estàs obligat** a capturar-la.
> 4. La **`NullPointerException`**: apareix en tocar un objecte que val `null`.

✅ Resum en 3 frases

1. Una excepció és un **objecte** que Java llança quan alguna cosa eix malament i que hereta de `Throwable`.
2. La família es divideix en `Error` (greus, no tocar), `Exception` (capturables) i `RuntimeException` (es llancen en executar, sense obligació de capturar-les).
3. Llegir el missatge de l'excepció (què, on, en quin mètode) és la primera habilitat d'un bon depurador.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Excepció | Objecte que representa un error en el programa |
> | `Throwable` | Classe arrel d'errors i excepcions |
> | `Error` | Fallada greu de la JVM, no capturable |
> | `Exception` | Fallada del programa, capturable |
> | `RuntimeException` | Excepció d'execució, sense captura obligatòria |
> | Trace | El "stack trace": la llista de crides fins al fallo |

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · break i continue · **Següent:** 07 · try, catch i finally

07 — try, catch i finally

L'airbag del codi: que un error no destrossi el teu programa 🛡️

> 🗺️ **Estàs en:** 🔀 **U04 · Estructures de control i excepcions bàsiques** → 07 · try, catch i finally

---

📬 La idea en una frase

> **`try` protegeix el codi perillós, `catch` atrapar l'excepció si apareix i `finally` s'executa sempre, ocorri el que ocorri.**

En el punt 6 vas veure que una `InputMismatchException` mata el teu programa. Ara toca blindar-lo: el `try/catch/finally` és l'airbag del codi.

🛡️ L'estructura completa

```
try {
    // codi perillós
} catch (TipusDExcepcio e) {
    // què fer si apareix eixa excepció
} finally {
    // s'executa SEMPRE, amb o sense excepció
}
```

```java
import java.util.InputMismatchException;
import java.util.Scanner;

public class LectorBlindat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Quants anys tens? ");
            int edat = sc.nextInt();
            System.out.println("Vas nàixer fa " + edat + " anys.");
        } catch (InputMismatchException e) {
            System.out.println("Això no és un nombre. No em faces això.");
        }

        System.out.println("El programa seguix viu. 🎉");
        sc.close();
    }
}
```

Si escrius `hola`, ja no explota: el `catch` atrapar l'error, imprimeix un missatge simpàtic i **el programa continua**.

> 💡 **Detall pràctic:** el `finally` és opcional i sol usar-se per a netejar recursos (tancar `Scanner`, fitxers…). S'executa **sempre**: si hi hagué excepció, si no n'hi hagué, i fins i tot si el `try` tenia un `return`.

🎯 Captura múltiple: diversos catch en fila

Pots capturar diversos tipus d'excepció, cada un amb el seu tractament. **L'ordre importa: primer les més específiques, després les generals.**

```java
try {
    int[] numeros = {1, 2};
    int indice = 5;
    int divisor = 0;
    System.out.println(numeros[indice] / divisor);
} catch (ArithmeticException e) {
    System.out.println("Divisió entre zero.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índex fora de l'array.");
} catch (RuntimeException e) {
    System.out.println("Alguna cosa estranya va passar en temps d'execució.");
}
```

En Java 7+ existeix una forma compacta per a diversos tipus amb el mateix tractament, separats per `|`:

```java
} catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
    System.out.println("Matemàtica o índex: vas fallar per ací.");
}
```

> ⚠️ **Advertència:** un `catch (Exception e)` al principi es menjaria les excepcions més específiques. Regla: del més concret al més general, com en els `else if`.

🧽 La variable e: el botí de l'error

L'e del `catch` és l'objecte excepció atrapat. Pots preguntar-li coses:

```java
catch (Exception e) {
    System.out.println("Missatge: " + e.getMessage());
    e.printStackTrace();   // imprimeix el stack trace complet (per a depurar)
}
```

> ⚠️ **Advertència:** un `catch` **buit** (sense res a dins) és un pecat mortal: te treguis l'error i ni tan sols te n'adones que ha passat. Com un testimoni que no parla en un judici. Com a mínim, imprimeix un missatge.

🏫 Exemple guiat: el menú a prova de bombes

Reunim `do-while`, `try` i `catch`: un menú que repeteix fins a triar bé i que no explota si escrius porqueria:

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class MenuBlindat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcio = 0;
        boolean valida = false;

        do {
            System.out.println("1. Jugar  2. Eixir");
            System.out.print("Tria: ");

            try {
                opcio = sc.nextInt();
                valida = (opcio == 1 || opcio == 2);
                if (!valida) {
                    System.out.println("Opció no vàlida, torna-ho a intentar.");
                }
            } catch (InputMismatchException e) {
                System.out.println("Això no és un nombre.");
                sc.next();   // descarta el text brossa del buffer
            }
        } while (!valida);

        System.out.println("Has triat l'opció " + opcio);
        sc.close();
    }
}
```

> 💡 **Detall pràctic:** fixa't en `sc.next()` dins del `catch`: sense ell, el text brossa seguiria al buffer del `Scanner` i el següent `nextInt()` tornaria a fallar. Atrapar l'excepció i **netejar el buffer** són dos passos del mateix ball.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** la pregunta clau davant d'un `try` és: *què pot fallar aquí i com ho manege?* Un `catch` que només existeix "per si de cas" però no fa res és fum.

Exercici: el detectiu de l'ordre

Aquest codi té un problema d'ordre en els `catch`. Sense executar, explica què ocorre:

```
try {
    int[] numeros = {10, 20};
    System.out.println(numeros[3]);
} catch (Exception e) {
    System.out.println("Atrapat per Exception.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Atrapat per ArrayIndexOutOfBoundsException.");
}
```

> 🔄 **Solució**
>
> **No compila.** Java es queixa en el segon `catch`: com que `ArrayIndexOutOfBoundsException` és una subclasse d'`Exception`, el primer `catch` ja l'atraparia tot, i Java no permet un catch "inalcançable". Els `catch` han d'anar **del més específic al més general**: primer `ArrayIndexOutOfBoundsException`, després `Exception`.

🎯 Mini-chequeig

1. Què fa el bloc `finally` i quan s'executa?
2. En quin ordre han d'anar els `catch`?
3. Quin perill té un `catch` buit?
4. Per què convé cridar `sc.next()` després d'una `InputMismatchException`?

> 🔄 **Respostes**
>
> 1. S'executa **sempre**, hi hagi o no excepció; serveix per a netejar recursos.
> 2. **Del més específic al més general**; si no, el general "se menja" els altres i no compila.
> 3. Te treguis l'error sense assabentar-te'n: el programa continua, però amb una fallada oculta. Mínim: imprimeix un missatge.
> 4. Perquè el text brossa es queda al buffer del `Scanner` i el següent `nextInt()` tornaria a fallar.

✅ Resum en 3 frases

1. `try` protegeix el codi perillós i `catch` atrapar l'excepció perquè el programa **no mori**.
2. Els `catch` van **del més específic al més general**, i mai no deixis un `catch` buit.
3. `finally` s'executa sempre, ideal per a tancar `Scanner`, fitxers i altres recursos.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `try` | Bloc amb codi que pot llançar excepcions |
> | `catch` | Bloc que atrapar una excepció i la maneja |
> | `finally` | Bloc que s'executa sempre |
> | Captura múltiple | Diversos `catch` seguits, de l'específic al general |
> | `e.getMessage()` | Missatge de l'error atrapat |
> | Stack trace | Rastre de crides on va ocórrer el fallo |

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Excepcions bàsiques · **Següent:** 08 · throw i excepcions pròpies

08 — throw i excepcions pròpies

Quan et toca a tu llançar la pedra i crear el teu propi defecte 🎳

> 🗺️ **Estàs en: 🔀 U04 · Estructures de control i excepcions bàsiques** → 08 · throw i excepcions pròpies

---

📬 La idea en una frase

> **`throw` llança una excepció quan tu decideixes que alguna cosa no ha de continuar, i creant la teva pròpia excepció pots posar-li el nom que vulguis al problema.**

Fins ara Java llançava les excepcions per tu. Però hi ha un superpoder millor: **tu** decideixes quan llançar-les, i pots inventar-te tipus d'error a la teva mesura.

🎳 throw: llança la pedra

`throw` crea i llança una excepció on vulguis. És com dir "aquí alguna cosa està mal, que ho sàpiga tot el món":

```
public class CompteBancari {
    public static void main(String[] args) {
        double saldo = 10.0;
        double retir = 500.0;

        if (retir > saldo) {
            throw new ArithmeticException("Saldo insuficient: " + saldo);
        }

        saldo -= retir;
        System.out.println("Nou saldo: " + saldo);
    }
}
```

Aquell `throw new ArithmeticException("...")` deté el programa amb aquella excepció. I el missatge que li passes al constructor és el que veuràs en `e.getMessage()`.

> 💡 **Detall pràctic:** `throw` i `throws` no són cosins: són bessons diferents. `throw` **llança** una excepció (ho veus aquí). `throws` **anuncia** en la signatura del mètode que pot llançar excepcions controlades. `throw` va al cos; `throws`, a la capçalera.

🏗️ Excepcions pròpies: el teu defecte a mida

Per què conformar-te amb `ArithmeticException` quan pots tenir una `SaldoInsuficientException` amb nom de pel·lícula? Crear la teva pròpia excepció és **heretar d'`Exception`** (o de `RuntimeException`) i llest:

```
public class SaldoInsuficientException extends RuntimeException {
    public SaldoInsuficientException(String missatge) {
        super(missatge);
    }
}
```

I ara la uses:

```
public class Caixer {
    public static void main(String[] args) {
        double saldo = 10.0;
        double retir = 500.0;

        if (retir > saldo) {
            throw new SaldoInsuficientException("Només tens " + saldo + "€.");
        }

        System.out.println("Retirat: " + retir);
    }
}
```

Sortida (amb el programa tallant-se):

```
Exception in thread "main" SaldoInsuficientException: Només tens 10.0€.
  at Caixer.main(Caixer.java:9)
```

> 💡 **Detall pràctic:** hereta d'`Exception` si vols **obligar** els qui la usen a capturar-la (excepció controlada). Hereda de `RuntimeException` si prefereixes que no els obligue (com les que vas

> veure en el punt 6). Per a començar, `RuntimeException` és més còmoda.

⚖️ checked vs unchecked: la burocràcia de les excepcions

- **Checked (controlades)**: el compilador **t'obliga** a capturar-les o a declarar-les amb `throws`. Hereden d'`Exception` però no de `RuntimeException`. Exemple: `IOException`.
- **Unchecked (no controlades)**: no t'obliguen a res. Són `RuntimeException` i les seves filles.

```
import java.io.IOException;

public class Mussol {
    public static void main(String[] args) throws IOException {
        // com que IOException és checked, HA d'anar en el throws
        // o estar dins d'un try/catch
    }
}
```

Si una excepció és checked i no la gestiones, **no compila**. Si és unchecked, el compilador et deixa tranquil (i l'error explota en execució).

🏫 Exemple guiat: la màquina expenedora

Anem a crear una excepció pròpia i un programa que la llanci i l'atrabe:

```
public class ProducteEsgotatException extends RuntimeException {
    public ProducteEsgotatException(String producte) {
        super("El producte " + producte + " està esgotat.");
    }
}
```

```java
public class MaquinaExpenedora {
    public static void main(String[] args) {
        int stock = 0;
        String producte = "Refresc";

        try {
            if (stock == 0) {
                throw new ProducteEsgotatException(producte);
            }
            System.out.println("Ací tens el teu " + producte);
        } catch (ProducteEsgotatException e) {
            System.out.println("Ho sentim: " + e.getMessage());
        }

        System.out.println("La màquina seguix funcionant. 🤖");
    }
}
```

Sortida:

```
Ho sentim: El producte Refresc està esgotat.
La màquina seguix funcionant. 🤖
```

Veus la màgia? El `throw` llança la teva excepció, el `catch` l'atrapar pel seu **nom propi** i el programa sobreviu. Aquell nom converteix un error genèric en un missatge que fins i tot la teva cap entén.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** si una condició "impossible" ocorre al teu codi, és millor `throw` que deixar que el programa continuï amb dades trencades. Una excepció primerenca val més que un bug que apareix dues setmanes després.

Exercici: el controlador de notes

Crea (en paper o a l'IDE) una excepció pròpia `NotaInvalidaException` que hereti de `RuntimeException`, i un mètode que la llanci si una nota no està entre 0 i 10. Què hereta la teva excepció del seu pare?

› 🔄 Solució

```
public class NotaInvalidaException extends RuntimeException {
    public NotaInvalidaException(double nota) {
        super("La nota " + nota + " no està entre 0 i 10.");
    }
}
```

Ús:

```
public class Notes {
    public static void main(String[] args) {
        double nota = 15;
        if (nota < 0 || nota > 10) {
            throw new NotaInvalidaException(nota);
        }
        System.out.println("Nota vàlida: " + nota);
    }
}
```

La teva excepció hereta de `RuntimeException` (que al seu torn hereta d'`Exception` i de `Throwable`) tot el comportament de llançar-se i capturar-se, el constructor que rep un missatge (amb `super(missatge)`) i el mètode `getMessage()`. Tu només poses el nom i el missatge.

🎯 Mini-chequeig

1. Què fa la paraula clau `throw`?
2. Com es crea una excepció pròpia?
3. Quina és la diferència entre `throw` i `throws`?
4. Quina diferència hi ha entre checked i unchecked?

› 🔄 Respostes

1. **Llança** una excepció en el punt del codi on la col·loques: `throw new MiExcepcion("...");`.
2. Heredant d'`Exception` o de `RuntimeException` i afegint (opcionalment) un constructor que cridi a `super(missatge)`.

3. `throw` llança una excepció (al cos del mètode); `throws` declara en la signatura que el mètode pot llançar excepcions checked.
4. Les **checked** t'obliguen a capturar-les o declarar-les (`throws`); les **unchecked** (`RuntimeException` i filles) no t'obliguen.

✅ Resum en 3 frases

1. `throw` llança una excepció on tu decideixes, amb el missatge que vulguis.
2. Crear la teva pròpia excepció és **heretar d'`Exception` o `RuntimeException`** i posar-li un constructor amb missatge.
3. `throw` (llançar) ≠ `throws` (declarar), i les excepcions **checked** obliguen a gestionar-les mentre les **unchecked** no.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`throw`	Llança una excepció: `throw new MiExcepcion()`
`throws`	Declara en la signatura que el mètode pot llançar alguna cosa
Excepció pròpia	Classe que hereta d'`Exception` o `RuntimeException`
Checked	Excepció que el compilador t'obliga a gestionar
Unchecked	Excepció sense obligació de captura (`RuntimeException`)
`super(missatge)`	Passa el missatge a la classe pare

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · try, catch i finally · **Següent:** 09 · Repàs interactiu

09 — Repàs interactiu: controla les estructures

El tancament pràctic de la unitat, amb decisions, bucles i una excepció que no funcionarà 😈

> 🗺️ **Estàs en: 🔀 U04 · Estructures de control i excepcions bàsiques** → 09 · Repàs interactiu

---

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
public class Misteri {
    public static void main(String[] args) {
        int nota = 6;

        if (nota >= 5) {
            System.out.println("Aprovat");
        } else if (nota >= 7) {
            System.out.println("Notable");
        } else {
            System.out.println("Suspés");
        }
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`Notable`** → La nota 6 és més gran que 5, i quasi també que 7, així que Java tria la millor. ❌
2. **`Aprovat`** → ✅ Correcte! Java avalua de dalt a baix i es queda amb la **primera** condició que doni `true`. Com que `nota >= 5` es compleix, entra aquí i s'oblida de la resta, encara que 6 no arribi a 7.

3. **Suspés** → L'`else` només s'executa si cap condició anterior es compleix, i aquí sí que es compleix la primera. ❌

> 🔄 **Solució**
>
> L'opció **2**. L'ordre de les condicions mana: guanya el primer `if` que sigui `true`. Si vols que un 6 sigui "Notable", hauries de reordenar de la condició més exigent (7) a la més permissiva (5).

🔥 Fireside Chat: if-else vs switch

> *Dos veterans del control de flux discuteixen al costat de la màquina de cafè.*

if-else: — Jo soc el clàssic. Condicions, rangs, comparacions... necessites decidir si alguna cosa és més gran que 5 o està entre 10 i 20? Crida'm a mi. Jo comparo el que sigui.

switch: — Clar, i t'omplires d'`else if` fins que el codi sembla l'escala d'un edifici. Amb mi poses la variable una vegada i cada cas en la seva línia. Net, directe, elegant.

if-else: — Elegant fins que t'oblides un `break` i el teu switch es converteix en un tobogan. Saps què és el fall-through? Una malson amb nom.

switch: — El fall-through s'usa a propòsit quan vull agrupar casos. I tu? Amb trenta `else if`, saps almenys quin va abans que quin?

if-else: — Jo suporto rangs! `>= 18`, `< 65`... Tu només serveixes per a valors exactes. Un dia has de decidir per edat i aniràs a plorar.

switch: — Millor plorar que repetir una variable vint vegades. Cada un al seu terreny, no?

if-else: — Fet. Tu, valors exactes. Jo, rangs i condicions combinades. Així ningú no es fa mal.

> La lliçó: no hi ha guanyador. **`switch` per a valors concrets** (dia, menú, talla) i **`if/else if` per a rangs i mescles**. Triar bé és la mitat de l'examen.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc el semàfor: si la meva condició és `true`, deixo passar; si és `false`, redirigeix a l'altre carril.**
2. **Soc el menú del restaurant: mires la meva variable i executes el `case` que coincideixi.**
3. **Soc la cinta de córrer que comprova abans de córrer: si la condició és `false` d'entrada, no faig ni un pas.**
4. **Soc el botó de parada: talle el bucle sencer tan bon punt aparec.**
5. **Soc l'avi de tots els errors: tot el que es llança hereta de mi.**
6. **Soc l'airbag: atrapar l'error perquè el programa no mori.**

> 🔄 **Respostes**
>
> 1. **L'`if/else`** — decideix entre dos camins segons una condició booleana.
> 2. **El `switch`** — tria entre diversos `case` segons el valor d'una variable.
> 3. **El `while`** — comprova abans d'executar (el `do-while` és el que corre primer).
> 4. **El `break`** — acaba el bucle (i també el `switch`).
> 5. **`Throwable`** — la classe arrel d'`Error` i `Exception`.
> 6. **El `catch`** — atrapar l'excepció perquè el programa sobrevisqui.

🤬 CONRAD VS EL MÓN: "El bucle que no s'acaba" 🔗

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre el clàssic del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu programa es queda penjat*. I jo: val, què té el bucle? *Pues no ho sé, no l'he mirat.* AI, MARE MEVA! Un `while` sense res que canviï la condició dins és un cotxe sense frens, t'ho explico amb plastilina?

I després està el que escriu `while (x > 0) { x = x + 1; }` *quan volia restar. Puja en comptes de baixar. No és un bucle infinit, és un bucle que ascendeix fins a l'infinit. Com si volguessis buidar una piscina tirant-hi més aigua.*

I el colmo: `if (x = 5)`. Amb UN igual. Això no és una condició, és una assignació! T'ho dic des de la UO3 i ho continuec veient. El doble igual == es queda a casa quan toca comparar.

La lliçó: abans d'acusar l'ordinador de "congelar-se", mira el bucle: alguna cosa modifica la condició cap a `false`? El `continue` es salta l'actualització? Uses == o t'has quedat en =? El 90% dels programes "penjats" s'arreglen amb un cop d'ull a aquestes tres preguntes.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Què imprimeix `int n = 4; String r = n >= 5 ? "A" : "B";`?
   - a) `A` b) `B`
2. Quantes vegades fa `for (int i = 0; i < 3; i++)`?
   - a) 3 b) 4
3. Què imprimeix un `switch` amb `case 1` i `case 2` seguits sense `break` entre ells, si la variable val 1?
   - a) Només el `case 1` b) El `case 1` i després el `case 2`
4. Quin és el resultat de `10 / 0`?
   - a) `ArithmeticException` b) Un nombre enorme

> 🔄 **Solucions**
>
> 1. **b)** — 4 no és més gran o igual que 5, així que el ternari retorna `"B"`.
> 2. **a)** — `i` val 0, 1 i 2: tres vegades. Amb `< 3` mai no entra amb `i = 3`.
> 3. **b)** — Sense `break`, el `case 1` es desborda al `case 2` (fall-through).
> 4. **a)** — Dividir entre zero llança `ArithmeticException` en temps d'execució.

⚡ Laboratori de Tortura: la màquina que plora

> **Durada estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte quan el arregles.

```
public class Tortura
    public static void main(string[] args) {
        int suma = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;
            suma = suma - i;
        }
        System.out.println("La suma dels senars és: " + suma)
        System.out.println("El nombre de senars és: " + 5);
    }
}
```

Fallada intencionada: un dels errors sembla correcte a simple vista perquè "es veu bé", però fa que el programa compte malament.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun `;` que falti? *no → segueix buscant.*
   > **I si encara estic atascat?**
   > Comprova també les claus `{}`: la classe necessita la seva obertura.
2. Compila ja? *no → mira el missatge d'error i les majúscules.*
   > **I si encara estic atascat?**
   > Són dues coses: `string` ha de ser `String` (la classe amb majúscula) i falta el `;` al final de la primera línia del `println`.
3. Executa però la suma surt rara? *És l'error de lògica: el signe.*
   > **Solució final**

Els **3 errors de compilació**:

1. Falta la `{` que obre el cos de la classe després de `Tortura`.
2. `string[] args` → `String[] args` (la classe `String` amb majúscula).
3. Falta el `;` al final de `System.out.println("La suma dels senars és: " + suma)`.

L'**error de lògica**: `suma = suma - i;` resta en lloc de sumar. Compila perfectament, però amb els senars 1, 3, 5, 7 i 9 la suma surt **-25** en lloc de 25. Un signe és l'únic que separa el teu programa de la veritat.

```java
public class Tortura {
    public static void main(String[] args) {
        int suma = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;
            suma = suma + i;
        }
        System.out.println("La suma dels senars és: " + suma);
        System.out.println("El nombre de senars és: " + 5);
    }
}
```

Sortida correcta: `La suma dels senars és: 25` i `El nombre de senars és: 5` (els senars de l'1 al 10 són 1, 3, 5, 7 i 9: cinc). Amb la versió trencada, la suma sortia `-25`, i això era la pista del signe.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **El Semàfor Humà**	Explicar `if/else if/else` i l'ordre de les condicions sense mirar
🏅 **Break o Continua**	Distingir `break` i `continue` en un exercici "Què imprimeix?" sense fallar
🏅 **El Caçador d'Errors**	Trobar els errors del Laboratori sense usar les pistes
🏅 **A prova de bombes**	Fer un menú amb `do-while` + `try/catch` que no exploti amb porqueria
🏅 **L'Inventor**	Crear una excepció pròpia i llançar-la amb `throw` en un programa teu

---

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```java
public class Misteri2 {
    public static void main(String[] args) {
        for (int i = 1; i <= 6; i++) {
            if (i % 2 != 0) continue;
            System.out.println(i);
        }
    }
}
```

2. **El nombre invisible:** amb el `while` del punt 3, com faries per a comptar quants dígits té un nombre sense usar `String`?
3. **El detectiu:** el teu programa llança `InputMismatchException` en la línia del `nextInt()`. Quina eina uses i què mires primer en el stack trace?
4. **Vertader o fals:** "un `catch (Exception e)` atrapar també les `RuntimeException`".

> 💡 **Solucions**
>
> 1. Imprimeix `2`, `4`, `6`: el `continue` salta els senars i només s'imprimeixen els parells de l'1 al 6.
> 2. Repetint `while (numero > 0) { numero /= 10; comptador++; }`: cada divisió entre 10 li lleva un dígit al nombre fins que arriba a 0. Amb `123` → 3 dígits.
> 3. El **depurador**: posa un breakpoint en el `nextInt()` i mira el valor que està arribant pel buffer. O, més ràpid, llegeix el stack trace: la línia `at ...` et diu exactament on es va llançar.
> 4. **Vertader.** `RuntimeException` hereta d'`Exception`, així que un `catch (Exception e)` les atrapar totes.

🧩 Crucigrama de Bits

```
Horitzontal:
1. Estructura que tria entre diversos case (6 lletres)
3. Acaba el bucle sencer (5 lletres)
5. La classe arrel de tots els errors (9 lletres)
6. Repetix "mentres" (5 lletres)

Vertical:
2. Es salta només la volta actual (8 lletres)
4. El bloc que s'executa sempre en un try (7 lletres)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. SWITCH · 3. BREAK · 5. THROWABLE · 6. WHILE **Vertical:** 2. CONTINUÏ · 4. FINALLY

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo fos la teva àvia, la diferència entre `if` i `switch`."**
2. **"Quina és la diferència entre `break` i `continue` en un bucle?"**
3. **"Un usuari escriu text on el teu programa espera un nombre i l'aplicació es cau. Com ho arreglaries?"**
4. **"Què és una `NullPointerException` i com l'evites?"**
5. **"Per a què serveix el bloc `finally`?"**
6. **"Quan crearíes una excepció pròpia en comptes d'usar les de Java?"**

🤷 No hi ha preguntes tontes

> ❓ **Puc usar `switch` amb un `double`?**

No. `switch` admet `int` i enters afins, `char`, `enum` i `String` (des de Java 7). Amb `double` usa `if/else if`, perquè els decimals quasi mai no es comparen amb igualtat exacta.

> ❓ **Per què a vegades veig `while (true)` si és un bucle infinit?**

Perquè es trenca des de dins amb `break`: `while (true) { if (condicio) break; ... }`. És la forma d'escriure "bucle per sempre fins que passi alguna cosa". Ho veuràs molt en jocs i servidors.

> ❓ **El `catch` pot capturar qualsevol excepció?**

Si poses `catch (Exception e)`, captures totes les `Exception` i les seves filles (incloses les `RuntimeException`). Si vols capturar-ho absolutament tot, existeix `catch (Throwable e)`, però això és com pescar amb dinamita: també atrapa errors greus de la JVM que no hauries de tocar.

🎬 Post-Crèdits

El programador acaba el seu menú blindat: `do-while` per a demanar opció, `try/catch` per a sobreviure a les lletres de l'usuari, `switch` per a les opcions… i una excepció pròpia que va llançar amb `throw` quan l'usuari va intentar retirar més saldo del que tenia.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa humejant.

CONRAD: — Això era un programa teu o una central nuclear? Quin blindatge. `try`, `catch`, `switch`, bucles… Fins i tot una excepció que t'has inventat. No està malament per a un aprenent de semàfor.

Novell: — I ara què? Els meus programes decideixen, repeteixen i sobreviuen.

CONRAD: — *fa un glop* Decidir i repetir està bé, però per a resoldre problemes de veritat falta una cosa: pensar en gran. Dividir el problema, dissenyar la solució… Això és una altra història. La pròxima parada t'ho compta.

El novell guarda el seu projecte, tanca l'IDE i sent que els programes ja no li tenen por a res.

PRÒXIMAMENT EN U07: Algorítmica I: fonaments. El moment en què els teus programes no només pensen, sinó que **pensen amb mètode**. 🧩

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · throw i excepcions pròpies · **Següent:** U05 · Funcions i mètodes

U05 — Funcions i mètodes

L'ofici de tallar el teu codi: mètodes que reben dades, tornen resultats i converteixen un main enorme en peces petites i provables 🔧

Fins ara tot el teu codi ha viscut dins de `main`: declaraves variables, reparties `if` i bucles, i el programa sencer era un paràgraf llarg i valent. Funciona… fins que el programa creix. Aleshores `main` es converteix en un full de receptes, no pots repetir una part sense copiar i enganxar, i cada canvi s'estén com una taca de cafè.

En aquesta unitat aprens l'eina que separa a qui escriu codi de qui el **dissenya**: el **mètode** (o funció). Un mètode és una recepta amb nom: rep dades per la porta, fa una cosa ben explicada, i torna el resultat o no torna res i se'n va. Res més. Però amb això construeixes programes sencers.

La promesa concreta: en acabar aquesta unitat, `main` serà petit, cada concepte viurà en el seu propi mètode amb un nom que s'explica sol, i hauràs donat el salt que et separa dels exercicis d'"un sol fitxer" per sempre.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9è és un aterratge pràctic per a martellejar tot l'après.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- **Declarar** els teus propis mètodes (`public static`) i cridar-los des de `main`.
- **Passar dades** a un mètode mitjançant **paràmetres** i usar-los dins com si fossin variables.
- **Tornar resultats** amb `return` i distingir quan un mètode torna i quan és `void`.
- Entendre l'**àmbit** de les variables: què existeix dins d'un mètode i què no surt d'ell.
- Reconèixer i arreglar els **errors típics** de les funcions (return oblidat, arguments desordenats…).
- **Dividir** un problema gran en mètodes petits, cadascú amb una responsabilitat.
- **Refactoritzar** un programa monolític en funcions reutilitzables sense canviar el que fa.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Què és una funció?	La recepta amb nom: per què existeix i què és "mètode"	Tots
02 · El teu primer mètode	`public static void`, cridar des de `main` i executar	Tots
03 · Paràmetres: l'entrada	Dades que entren: tipus, ordre i diversos alhora	Tots
04 · return: la sortida	Tornar resultats i quan usar `void`	Tots
05 · Àmbit de variables	Quines variables viuen dins d'un mètode i per què	Tots
06 · Errors freqüents	Els 7 entrebancs de tothom sortir-ne	Tots
07 · Divideix el problema	Compondre: main petit, una responsabilitat per mètode	Tots
08 · Be the Code	Refactoritza un programa monolític a mà	Tots
09 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9è és l'aterratge pràctic: llegeix-lo just després del 8è i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extra

✅ Criteris d'avaluació coberts (RA2)

RA2: Escriu i prova programes senzills, reconeixent i aplicant els fonaments de la programació orientada a objectes.

CE	Criteri	On es cobreix
RA2 b)	S'han escrit programes simples.	✅ Tots
RA2 e)	S'han escrit crides a mètodes estàtics.	✅ Punts 2, 3, 4 i 7
RA2 f)	S'han utilitzat paràmetres en la crida a mètodes.	✅ Punts 3, 4 i 7

> 📌 Aquesta unitat és la primera meitat del viatge dels mètodes: aquí els escrius "a pel" amb `static`, com a receptes soltes. La segona meitat arriba en la U09 (POO), on els mètodes viuen dins de classes amb atributs, i en la U10, on `static` per fi explica el seu nom. Avui només necessites l'eina: entrades, sortides i un nom honrat.

🚪 Per on començo?

- No has escrit mai un mètode que no fora `main`? → Comença en el punt 1 i no et saltses el 2.
- El teu `main` ja sembla un paràgraf infinit de copiar i enganxar? → Ves directe al punt 7 i després al Be the Code.
- Només vols entendre per què `main` és `public static void main`? → El punt 2 te desmonta la firma en 4 paraules.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Què és una funció?
⏭️ **En acabar la unitat, continua en U06 · Arrays.**

01 · Què és una funció?

La recepta amb nom que converteix el codi repetit en una sola línia: per què existeix i què diables és un mètode 🔧

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 01 · Què és una funció?

📬 La idea en una frase

> **Una funció és una recepta amb nom: rep dades per la porta, fa una cosa ben explicada i torna el resultat; tu només l'invites pel seu nom.**

🍳 El problema: el paràgraf infinit

Mira aquest `main` d'un quadern de notes:

```
public class Notas {
    public static void main(String[] args) {
        // Calcular la mitjana de 3 notes
        double media = (8 + 7 + 9) / 3.0;
        System.out.println("Mitjana 1: " + media);

        // Calcular la mitjana d'altres 3 notes... una altra volta
        double media2 = (6 + 5 + 10) / 3.0;
        System.out.println("Mitjana 2: " + media2);

        // I demà, amb 30 alumnes, copiar i pegar 30 voltes
    }
}
```

El càlcul està escrit **dues vegades**. Si demà canvies la fórmula (¿i si les notes tenen pes?), hauràs de buscar i arreglar totes les còpies: una oblidada i el programa ja ment. Programar copiant i enganxant és com cuinar repetint cada pas del receptari en veu alta en cada plat: funciona… fins que tens comensals.

> ⚠️ **Advertència:** si alguna vegada escrius `// Calcular la mitjana` més d'un programa en el mateix fitxer, en algun lloc un programador sènior torna a plorar. El codi repetit no és codi: és deute amb interessos.

📖 La recepta amb nom

Una **funció** (en jerga: un **mètode**) és exactament això: una recepta amb nom en el receptari.

Recepta de cuina	Funció en Java
El nom (“Truita de patates”)	El **nom** del mètode (`calcularMedia`)
els ingredients que et passen	Els **paràmetres** (les dades d’entrada)
Els passos dins de la recepta	El **cos** (el codi entre `{` i `}`)
El plat que surt	El **valor que torna** (o res, si només fa alguna cosa)

No recitis la recepta sencera cada vegada: dius **“truita”** i la cuina treballa. El mateix: dius `calcularMedia(...)` i el programa executa aquell tros de codi en el seu lloc.

```
double notaMedia = calcularMedia(8, 7, 9);
System.out.println("Mitjana 1: " + notaMedia);
```

Dues línies on abans n’hi havia tres… i el millor: la fórmula **només està escrita una vegada**. Quan canviï, canviarà en un sol lloc.

🔧 Funció o mètode? (no, no és el mateix… bé, sí)

Curso honest sobre la terminologia:

- **Funció** és el concepte general: entrades → procés → sortides. En Python, JavaScript o C les anomenes funcions.
- **Mètode** és aquella mateixa idea **vivint dins d’una classe**, que és on viu tot en Java (ja ho vas veure en la UO2: fins i tot `main` està dins d’una classe).

En la pràctica, tothom usa els dos noms per al mateix, i en aquest curs també ho farem. Si en una entrevista et pregunten la diferència, aquí tens la resposta exacta: **en Java, les funcions són mètodes d’una classe**.

> 💡 **Consell:** “mètode” és la paraula que veuràs en la documentació de Java i en els errors del compilador. “Funció” és la que escoltaràs en les converses. Saber les dues t’evita quedar-te fora de la conversa.

🧰 Què guanya el teu programa al tallar-lo

1. **No repeteixes codi.** La fórmula s'escriu una vegada i s'usa 30.
2. **Es llegeix millor.** `mostrarMedia(...)` explica què passa; cinc línies d'aritmètica a mitges no.
3. **Es prova per parts.** Pots comprovar que `calcularMedia` funciona sense executar el programa sencer.
4. **Es repara sense por.** Arregles el tros trencat (el mètode) i la resta no s'assabenta.

És el mateix instint de la descomposició de la UO1, però aquesta vegada amb eina: abans **pensaves** el problema per parts; ara el llenguatge et deixa **escriure** cada part per separat.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** el nom del mètode és documentació. Un mètode anomenat `hacerCosas` és una confessió; un anomenat `calcularIVA` és un pla.

Exercici: el vident de sortides

```
public class Vidente {
    static void saludar() {
        System.out.println("Hola, món");
    }

    public static void main(String[] args) {
        saludar();
        saludar();
    }
}
```

Què imprimeix?

- (A) `Hola, món` una vegada
- (B) `Hola, món` dues vegades
- (C) Res: els mètodes no s'executen sols
- (D) Error de compilació

> › 🔄 **Solució**

> La **B**. `saludar()` es crida dos vegades des de `main`, i cada crida executa el seu cos: dos `Hola, món`. Un mètode no s'executa sol en declarar-se: només quan algú el crida. (La C és la trampa clàssica de "si està escrit, s'executa".)

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines tres coses "típiques" té una funció (segons la taula de la recepta)?
2. S'executa el cos d'un mètode pel sol fet d'estar escrit en el fitxer?
3. `main` és una funció? I `saludar()` del teu propi programa?
4. Menciona dues avantatges de tallar el codi en mètodes.

> 🔄 **Respostes**
>
> 1. Un **nom**, unes **entrades** (paràmetres) i un **resultat** que torna (o res).
> 2. No. S'executa quan algú el **crida**.
> 3. `main` és el mètode especial per on arranca el programa; `saludar()` és un mètode normal. En Java, els dos són mètodes (funcions dins d'una classe).
> 4. Qualsevol de: no repetir codi, llegibilitat, provar per parts, reparar sense trencar la resta (i la quarta corona: canviar la fórmula en un sol lloc).

✅ Resum en 3 frases

1. Una **funció/mètode** és una recepta amb nom: rep dades, fa una cosa i torna alguna cosa (o no).
2. La seva avantatge és **escriure una vegada, usar moltes**: el codi repetit desapareix i el canvi afecta a un sol lloc.
3. En Java tot mètode viu dins d'una classe, per això "funció" i "mètode" són la mateixa eina amb dos noms.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Funció / Mètode	Recepta amb nom: entrades → procés → sortida

Terme	Idea general
Paràmetre	Dada que la funció rep per la porta
Retorn	Valor que la funció torna en acabar
Crida	Executar un mètode: `nom(...)`
Cos	El codi entre { i } del mètode

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · El teu primer mètode

02 · El teu primer mètode

Desmuntem la firma `public static void` en quatre paraules i escrivim la primera recepta del receptari 🍳

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 02 · El teu primer mètode

📬 La idea en una frase

> **Declarar un mètode és escriure la seva recepta (`public static void nom() { ... }`); cridar-lo és dir el seu nom amb parèntesis i punt i coma perquè executi el seu cos.**

✍️ La primera recepta

Aquest és un programa complet amb un mètode propi:

```
public class Saludo {
    public static void saludar() {
        System.out.println("Hola, classe!");
    }

    public static void main(String[] args) {
        saludar();
        saludar();
    }
}
```

Sortida:

```
Hola, classe!
Hola, classe!
```

Dues parts: **`saludar()`** és la recepta (declarada amunt), i en `main` la **cridem** dos vegades. Fixa't en els dos detalls d'escriptura:

- La **declaració** no porta punt i coma al final: acaba amb el seu bloc `{ ... }`.
- La **crida** sí porta `;`, perquè és una sentència, igual que `int x = 5;`.

🔍 La firma, paraula per paraula

```
public static void saludar() {
```

Paraula	Què significa	Quan la veuràs explicada
`public`	Visible des de qualsevol lloc	U10 (visibilitat)
`static`	Pertany a la classe, no a un objecte	U10 (mètodes static)
`void`	**No torna** res	Punt 4 d'aquesta unitat
`saludar()`	El nom i els seus parèntesis	Avui

De moment escrius **sempre** `public static` davant dels teus mètodes. No els oblides: sense `static`, `main` no podrà cridar-los (l'explicació completa arriba en la U10, quan ja sàpigues què és un objecte).

> 📝 **Nota:** el nom segueix la convenció camelCase: `calcularMedia`, `mostrarMenu`, `esPar`. Sense espais, sense accents, i la primera paraula en minúscula. Java és mandrós amb els accents i puntual amb les majúscules.

Les regles del joc

1. Els mètodes es declaren **dins de la classe**, mai dins d'un altre mètode (ni tan sols dins de `main`).
2. L'**ordre no importa**: pots cridar a un mètode declarat més baix de tot. Java no llegeix d'amunt a avall; busca per nom.
3. Cada mètode és una illa amb el seu propi nom: dos mètodes no poden cridar-se igual **amb els mateixos paràmetres** (això ho veuràs en la U09, amb la sobrecàrrega).

> ⚠️ **Advertència:** l'error de compilació `cannot find symbol` en cridar un mètode gairebé sempre vol dir que li has posat un nom diferent del que té, o que el crides des de fora de la classe.

🎬 `main` també és un mètode

Aquí està la revelació del dia: `main` no és especial perquè sigui màgic, sinó perquè la JVM **el busca per aquell nom exacte** per a arrancar el teu programa:

```
public static void main(String[] args) {
```

- `public` → la JVM ha de poder veure'l.
- `static` → la JVM el crida **sense crear un objecte** de la teva classe.
- `void` → torna res (només executa).
- `String[] args` → l'array d'arguments de la línia d'ordres (ho vas veure en la U02).

Tot el que has vingut usant des de la U02 era... un mètode més. La diferència és que tu només cridaves a un sense adonar-te'n. Ara crides a tots els que et vingui de gust.

```
public class Reencuentro {
    public static void main(String[] args) {
        saludar();              // cridem a un mètode nostre
        System.out.println("Continuar amb el programa...");
    }

    public static void saludar() {
        System.out.println("Hola una altra volta");
    }
}
```

Funciona encara que `saludar` estigui **baix** de `main`. Regla 2: l'ordre no importa.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** si `main` necessita cridar a un mètode, aquell mètode gairebé sempre necessita `static`. Sense `static`, hauràs d'esperar a la U10 perquè les coses encaixen.

Exercici: la firma trencada

Quin d'aquests mètodes **compila**?

```java
// Opció A
static public void contar() {
    System.out.println("1");
}

// Opció B
public static contar() {
    System.out.println("1");
}

// Opció C
public static void contar {
    System.out.println("1");
}

// Opció D
public static void contar() System.out.println("1");
```

> 🔄 **Solució**
>
> La **A**. `static` i `public` poden anar en qualsevol ordre: les dues són modificadors. La B oblida el tipus de retorn (`void`). La C oblida els parèntesis `()`. La D oblida les claus `{ }`: el cos **sempre** va entre claus.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Escriu la firma completa d'un mètode anomenat `acomiadar` que no torno res.
2. Quina línia (i en quin lloc) executa el cos de `acomiadar()`?
3. Pot `main` estar declarat DESPRÉS d'acomiadar en la classe? I a l'inrevés?
4. Quina és la diferència de puntuació entre declarar i cridar?

> 🔄 **Respostes**
>
> 1. `public static void acomiadar() { ... }`
> 2. Una **crida** com `acomiadar();` (amb punt i coma), per exemple dins de `main`.
> 3. Sí a les dues: l'ordre dels mètodes dins de la classe no importa.
> 4. Declarar acaba en `{ ... }` **sense** `;`; cridar és una sentència i porta `;`.

✅ Resum en 3 frases

1. Un mètode es declara amb la seva firma (`public static void nom() {...}`) dins de la classe i **s'executa només quan el cries**.
2. `public static void main(String[] args)` és un mètode més: la JVM el crida en arrancar perquè té aquell nom exacte.
3. L'ordre dels mètodes no importa; el que importa és el nom correcte i no declarar un mètode dins d'un altre.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Firma | La línia completa: modificadors + retorn + nom + paràmetres |
> | Declaració | Definir la recepta (amb el seu cos `{...}`) |
> | Crida | Executar el mètode: `nom(args);` |
> | `static` | De classe: s'usa sense crear objectes (detall en U10) |
> | `void` | "No torno res" |
> | camelCase | `calcularMedia`: minúscula a l'inici, majúscules entre paraules |

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Què és una funció? · **Següent:** 03 · Paràmetres: l'entrada

03 · Paràmetres: l'entrada

Ingredients que entren per la porta: tipus, ordre, diversos alhora i la diferència entre paràmetre i argument 📥

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 03 · Paràmetres: l'entrada

📬 La idea en una frase

> **Els paràmetres són les variables que un mètode rep en ser cridat: tu passes valors en els parèntesis i el mètode treballa amb ells com si els hagués escrit ell mateix.**

🔁 El problema: el mètode de sempre

El nostre `saludar()` només sap dir una cosa:

```
static void saludar() {
    System.out.println("Hola, classe!");
}
```

Si demà s'ha de saludar a "Marta", a "Youssef" i a tota la classe un a un, toca copiar i enganxar amb noms diferents… o donar-li **entrades** al mètode:

```
public class Saludos {
    public static void saludar(String nombre) {
        System.out.println("Hola, " + nombre + "!");
    }

    public static void main(String[] args) {
        saludar("Marta");
        saludar("Youssef");
        saludar("la classe sencera");
    }
}
```

Sortida:

```
Hola, Marta!
Hola, Youssef!
Hola, la classe sencera!
```

Una sola recepta, tres plats. El mètode ja no decideix quines dades usa: **se les passes**.

📥 Com es declaren

```
public static void saludar(String nombre) {
//                         ^^^^^^^^^^^^
//                         paràmetre: tipus + nom
```

- Cada paràmetre porta **tipus** i **nom**, com una variable: `String nombre`, `int edad`, `double nota`.
- Varios paràmetres se separen amb **coma**, i cadascú repeteix el seu tipus: `String nombre, int edad`.
- Dins del cos, els paràmetres s'usen **com variables normals**: es llegeixen, es concatenen, es comparen.

```
public static void presentar(String nombre, int edad) {
    System.out.println(nombre + " té " + edad + " anys.");
}

public static double areaRectangulo(double base, double altura) {
    return base * altura;   // ja sabem tornar! detall en el punt 4
}
```

> 💡 **Consell:** nomena els paràmetres per el que **són**, no per l'ordre: `double base, double altura` s'entén; `double x, double y` obliga a llegir el cos cada vegada.

Paràmetre ≠ argument

Dos paraules, dos moments, la mateixa confusió eterna:

```
//                paràmetres (en la declaració)
static int sumar(int a, int b) { return a + b; }

//               arguments (en la crida)
int r = sumar(3, 4);
```

Moment	Nom	Exemple
Declarar el mètode	**Paràmetres**	`int a, int b`
Cridar el mètode	**Arguments**	`3, 4`

En les converses de passadís tot el món diu “paràmetres” per als dos. En un examen o en una entrevista, la distinció et fa sonar a professional.

🎯 Regles de les entrades

1. **L’ordre importa:** `presentar("Ana", 20)` no és el mateix que `presentar(20, "Ana")` (aquesta última ni compila: Java espera un `String` primer).
2. **El nombre importa:** `sumar(3)` i `sumar(3, 4, 5)` no coincideixen amb la firma → error de compilació.
3. **El tipus ha d’encaixar:** un `int` s’accepta on es demana `double` (ampliació automàtica), però un `String` on es demana `int` és un mur: `incompatible types: String cannot be converted to int`.
4. Cada crida crea **les seves pròpies còpies** dels arguments: `sumar(3, 4)` i `sumar(10, 20)` viatgen amb els seus valors, sense pisar-se (per a primitius i valors simples, és així; els objectes guarden alguna sorpresa per a la UO9).

> ⚠️ **Advertència:** si crides `saludar();` a un mètode que espera un `String`, no hi ha “evitació elegant”: `error: method saludar in class ... cannot be applied to given types`. Els paràmetres no són opcionals.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan un mètode té molts paràmetres del mateix tipus (`int fila, int columna`), comet el crim de canviar l’ordre i veuràs com tot el programa comença a apuntar a

> la casella equivocada. Per això es nomenen bé i es passen en ordre.

Exercici: la crida confusa

```
public class Confusion {
    static void pintar(String color, int veces) {
        for (int i = 0; i < veces; i++) {
            System.out.print(color);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        pintar("verd", 3);
        pintar(2, "blau");
    }
}
```

Què passa?

- (A) Imprimeix `verdverdverd` i després `blaublaublau`
- (B) Compila i imprimeix `verdverdverd` i `2222...` (bucle rar)
- (C) No compila: en la segona crida, arguments en ordre incorrecte
- (D) No compila: `pintar` necessita `return`

> › 🔄 **Solució**
>
> La **C**. El primer paràmetre és `String` i el segon `int`; en `pintar(2, "blau")` arriba un `int` on ha d'anar el `String` → `incompatible types`. L'ordre dels arguments és part de la firma.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. En `static void log(String mensaje, int nivel)`, quins són els paràmetres i què li passes en `log("Fallo", 3)`?
2. Per què `sumar(1, 2, 3)` no compila si `sumar(int a, int b)` està ben escrit?
3. Quina paraula descriu a `3` i `4` en `sumar(3, 4)`: paràmetre o argument?
4. Es pot cridar a un mètode sense parèntesis, com `saludar;`?

> 🔄 **Respostes**
>
> 1. Paràmetres: `String mensaje` i `int nivel`. Arguments: `"Fallo"` i `3`.
> 2. Perquè la firma admet **2** arguments i la crida n'envia **3**: `wrong number of arguments`.
> 3. **Arguments** (els paràmetres viuen en la declaració).
> 4. No. Sense `()` no és una crida: `saludar;` és una expressió sense efecte (i ni això compila com t'esperes). Crida = `saludar();`.

✅ Resum en 3 frases

1. Els **paràmetres** (tipus + nom) es declaren entre parèntesis i dins del mètode s'usen com variables.
2. En la crida passes **arguments** amb un **ordre, nombre i tipus** que han d'encaixar amb la firma.
3. Varios paràmetres se separen per coma i cadascú porta el seu tipus: `String nombre, int edad`.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Paràmetre	Variable de l'entrada, en la declaració
Argument	Valor que passes en la crida
Firma	Nom + tipus dels paràmetres
Ampliació	`int` cap en `double` sense avisar
Còpia de l'argument	Cada crida treballa amb el seu propi valor

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · El teu primer mètode · **Següent:** 04 · return: la sortida

04 · return: la sortida

Tornar resultats com una màquina ben educada: tipus que coincideixen, camins que tanquen i el pecat d'imprimir quan has de tornar 📤

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 04 · return: la sortida

📬 La idea en una frase

> **Un mètode amb `return` calcula i lliura un valor a qui l'ha cridat; un mètode `void` només executa alguna cosa i s'calla. La firma promet el tipus i `return` compleix la promesa.**

📤 Dos oficis, dos firmes

Fins ara els teus mètodes **feien** coses (imprimien). Ara aniran a **entregar** coses:

```
public static int sumar(int a, int b) {
    return a + b;
}
//     ^^^
//     la firma diu "torne int" i return ho compleix
```

Qui crida rep el valor **i pot usar-lo**:

```
int total = sumar(3, 4);          // 7 en una variable
System.out.println(sumar(3, 4));  // 7 directe al println
int doble = sumar(sumar(1, 2), 3); // tornat com a argument: 6
```

La regla d'or: **el tipus del `return` ha d'encaixar amb el tipus de la firma**. `int` amb `int`, `double` amb `double`, `String` amb `String`. Si la firma diu `double` i tornes `7`, Java ho accepta (l'enter s'amplia). Si diu `String` i tornes `7`, és un mur: `incompatible types: int cannot be converted to String`.

`void`: la firma que no promet res

```java
public static void imprimirResultado(int x) {
    System.out.println("Resultat: " + x);
    // sense return (o amb un return; buit opcional)
}
```

`void` = “no torno res, només faig el meu treball”. Com tries?

Usa retorn quan…	Usa void quan…
Un altre mètode necessita el resultat	Només s’ha de produir un efecte (imprimir, guardar, pintar)
La tasca és “calcular”	La tasca és “fer”
Vols provar el valor per separat	El resultat ja es veu en pantalla o en fitxer

> 💡 **Consell:** si el teu mètode es diu `calcular...`, `get...`, `es...` o `devolver...`, gairebé segur que ha de **tornar**. Si es diu `imprimir...`, `mostrar...` o `guardar...`, gairebé segur que és `void`. El nom del mètode i la seva firma conten la mateixa història.

🛤️ `return` talla l’execució

El primer `return` que s’executa **acaba el mètode a l’instant**: el que hi hagi davall no s’executa mai.

```java
public static String clasificar(int nota) {
    if (nota >= 5) {
        return "Aprovat";
    }
    return "Suspès";   // només s'assolix si la línia anterior no s'executa
}
```

Dos `return` en el mateix mètode són legals (i molt útils), sempre que **tots els camins possibles acaben en un**:

```java
public static int maximo(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;   // tots els camins tanquen ✓
    }
}
```

Si un camí arriba al final sense `return`:

```java
public static int peligro(int a) {
    if (a > 0) {
        return a;
    }
    // ¿i si a <= 0?  →  error: missing return statement
}
```

CONRAD es posa vermell: `error: missing return statement`. Java no endevina: **exigeix** que la firma no pugui quedar sense complir.

> ⚠️ **Advertència:** codi després d'un `return` incondicional no s'executa mai (`unreachable statement` és la seva forma de protesta). I si una branca torna `int` i una altra `double`… la firma només admet un: decideix.

🎭 El pecat d'imprimir quan has de tornar

L'error conceptual més comú de la unitat:

```java
// ❌ Imprimeix la mitjana... però qui crida no s'assabenta
public static void media(int a, int b) {
    System.out.println((a + b) / 2.0);
}

// ✅ Torna la mitjana; qui crida decideix què fer
public static double media(int a, int b) {
    return (a + b) / 2.0;
}
```

Amb la versió `void`, `main` no pot guardar la mitjana, comparar-la ni reutilitzar-la: es va imprimir i va desaparèixer. Amb `return`, el valor entra en el programa com a dades de veritat.

Pràctica: **`System.out.println` és per a `main` (o per als mètodes l'ofici dels quals sigui mostrar), no per als mètodes que calculen**. Qui calcula calcula; qui decideix imprimeix.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `return true;` / `return false;` en un mètode que comença per `es`, `te` o `pot` és el patró més rendible del llenguatge: converteix preguntes en booleanes reutilitzables.

Exercici: l'endevinalla del retorn

```
public static int truco(int x) {
    if (x > 10) {
        return x * 2;
    } else if (x > 5) {
        return x + 3;
    }
    return 1;
}
```

Què torna `truco(7)` i `truco(12)`?

- (A) `10` i `24`
- (B) `14` i `24`
- (C) `10` i `12`
- (D) Compila, però `truco(7)` no torna res

> › 🔄 **Solució**
>
> La **A**: `10` i `24`. `truco(7)` no compleix `x > 10`, cau en `x > 5` i torna `7 + 3 = 10`. `truco(12)` compleix la primera branca i torna `12 * 2 = 24`. Traça el camí línia a línia sense donar-te les ganes: això és exactament el que fa el truc.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. En `public static boolean esPar(int n)`, quines dues línies ha de tenir com a mínim el cos?
2. Quin error dona un mètode `int` sense `return` en alguns camins?
3. Quina és la diferència entre `void imprimir(int x)` i `int devolver(int x)` per a qui crida?
4. Es pot fer `return` sense valor en un mètode `int`?

> 🔄 **Respostes**
>
> 1. Almenys un `return` amb booleà: `return n % 2 == 0;` (pot tenir més branques amb `if`).
> 2. `missing return statement` (error de compilació).
> 3. `void` només produeix un efecte (imprimir); `int` entrega un valor que qui crida pot guardar, comparar o reutilitzar.
> 4. No. `return;` (buit) només val en mètodes `void`; un `int` necessita `return expressió;`.

✅ Resum en 3 frases

1. La firma promet un tipus de retorn i cada camí del cos el compleix amb `return expressió;`.
2. `return` **acaba el mètode** en executar-se; si algun camí arriba al final sense `return` (i no és `void`), no compila.
3. Els mètodes **calculen** tornant; imprimir és ofici de qui decideix (normalment `main`).

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
`return`	Entrega un valor i tanca el mètode
`void`	Firma sense retorn: només executa
`missing return statement`	Algun camí no compleix la firma
Predicat	Mètode que torna `boolean` (`esPar`)
Ampliació	`int` cap en `double` en tornar

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Paràmetres: l'entrada · **Següent:** 05 · Àmbit de variables

05 · Àmbit de variables

Les variables viuen dins del seu mètode i no surten: per què la teva `total` no pot xocar amb la de al costat ni escapar-se de casa 🏠

> 🗺️ **Estàs en:** 🔧 **UO5 · Funcions i mètodes** → 05 · Àmbit de variables

📬 La idea en una frase

> **L'àmbit (scope) és el tros de codi on una variable existeix: dins d'un mètode només viuen els seus paràmetres i els seus locals, i el que neix aquí mor aquí.**

🏠 Cada mètode és una casa

```
public class Casas {
    public static void sumar() {
        int total = 10;
        total += 5;
        System.out.println(total);   // 15
    }

    public static void restar() {
        int total = 100;
        total -= 5;
        System.out.println(total);   // 95
    }

    public static void main(String[] args) {
        sumar();
        restar();
        // System.out.println(total);  →  ¡error!
    }
}
```

Dos mètodes amb una variable anomenada `total` **alhora**: zero problemes, perquè són dos `total` diferents, una en cada casa. En canvi, si `main` intenta usar `total` sense declarar-la:

```
error: cannot find symbol: variable total
```

Java no la troba perquè **no existeix** fora de `sumar()`. El mètode ha acabat, la seva pila d'execució s'ha desfet i `total` ja no està.

> 💡 **Consell:** quan vegis `cannot find symbol` en una variable, gairebé sempre és una de dos: l'has escrit malament (typo) o estàs intentant usar-la **fora de la seva casa**.

📏 Les regles de l'àmbit

1. **Paràmetres i locals** d'un mètode es veuen **dins d'aquell mètode** (de la seva firma a la seva clau de tancament).
2. **Un bloc també és una casa petita:** una variable declarada dins d'un `if` o d'un `for` no existeix fora de les seves claus (ja ho vas veure en la U04).

```
for (int i = 0; i < 10; i++) { ... }
// System.out.println(i);  →  cannot find symbol: variable i
```

3. **L'ordre importa dins del mètode:** només pots usar una variable **després** de declarar-la.
4. **Dues germanes no es barallen:** cada mètode té el seu propi espai; repetir noms entre mètodes és normal i sa.
5. En acabar el mètode, les seves variables **moren**: els seus valors no sobreviuen (salvo el que el mètode torni amb `return` o el que imprimeixi).

```
main:            sumar:
┌──────────┐     ┌──────────────┐
| (res)    |     | total = 10   |  ← existeix només ací
└──────────┘     | total = 15   |
                 └──────────────┘  ← i ací desapareix
```

🚪 El que entra per la porta també és de la casa

Els **paràmetres** són variables com les altres: es declaren a l'inici del mètode i moren amb ell.

```java
public static int triplicar(int numero) {
    int resultado = numero * 3;  // dos variables de la casa
    return resultado;
}
```

I aquí arriba la conseqüència bonica: com Java copia els arguments en entrar, **el teu mètode pot usar el valor que li passes, però no reescriure la variable de qui crida**:

```java
int puntos = 10;
triplicar(puntos);
System.out.println(puntos);   // continua sent 10
```

`triplicar` va treballar amb la seva **còpia** de `puntos`. Per a "tornar" canvis, el mètode torna el nou valor amb `return` i qui crida el guarda. (¿I què passa amb els arrays, que sí es modifiquen des de dins? Aquell misteri ho resols en la U06, punt 5. I la versió completa, amb objectes, en la U09.)

> ⚠️ **Advertència:** no intentis "guardar" el resultat d'un mètode en una variable d'un altre mètode. Els mètodes només comparteixen el que es **passen** (arguments) o el que es **torna** (`return`). No hi ha ventanetes laterals.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** si necessites recordar alguna cosa d'un mètode a un altre, no amaguis una variable: **torna-la** amb `return` i que la decisió la prenga qui crida.

Exercici: existeix aquesta variable?

```java
public class Alcance {
    public static void a() {
        int x = 1;
    }

    public static void b() {
        System.out.println(x);
    }

    public static void main(String[] args) {
        b();
    }
}
```

Què passa?

- (A) Imprimeix `1`
- (B) Imprimeix `0`
- (C) No compila: `x` no existeix en `b()`
- (D) Compila, però llança excepció en executar

> 🔄 **Solució**
>
> La **C**. `x` viu i mor dins de `a()`. El mètode `b()` no la coneix: és un `cannot find symbol` de compilació, ni tan sols arriba a executar-se. Les variables no viatgen entre mètodes per art d'engany.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Poden dos mètodes tenir una variable local anomenada `contador` alhora?
2. Existeix `i` del `for` fora de les seves claus?
3. Sobreviu `resultado` a l'acabar el mètode que la declara?
4. El meu mètode no canvia el `saldo` que li he passat. És un bug?

> 🔄 **Respostes**
>
> 1. Sí: cadascuna viu en el seu mètode, són dues variables diferents.

2. No. El seu àmbit acaba en la clau de tancament del `for`.
3. No. Les locals moren en sortir del mètode; només sobreviu el que tornes.
4. No és un bug: els arguments primitius arriben **copiats**. Si necessites el resultat, torna'l amb `return`.

✅ Resum en 3 frases

1. L'**àmbit** d'una variable és el tros de codi on existeix: el seu mètode (o el seu bloc `if`/`for`) i res més.
2. Dins caben **paràmetres** i **locals**; en acabar el mètode desapareixen i un altre mètode no pot veure-les.
3. Entre mètodes només viatja el que es **passa** (arguments copiats) o el que es **torna** (`return`).

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Àmbit / scope	Regió on una variable existeix
Variable local	Declarada dins d'un mètode o bloc
Paràmetre	Variable d'entrada del mètode
`cannot find symbol`	Estàs usant una variable fora de la seva casa
Còpia de l'argument	El mètode treballa amb el seu propi duplicat

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · return: la sortida · **Següent:** 06 · Errors freqüents

06 · Errors freqüents

Els 7 entrebancs de tothom amb els mètodes: codi trencat, error real de CONRAD i la seva arregla 🧯

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 06 · Errors freqüents

📬 La idea en una frase

> **Els errors amb mètodes no són aleatoris: tenen nom, missatge i arregla. Apren els set i el 90% de les teves hores de frustració s'evaporen.**

🧯 La galeria dels 7 entrebancs

1. Falta el `return` (o no tots els camins tanquen)

```
public static int doble(int n) {
    if (n > 0) {
        return n * 2;
    }
    // ¿i si n <= 0?
}
```

`error: missing return statement`. **Arregla:** tanca tots els camins amb `return`.

2. Arguments en ordre o tipus incorrecte

```
static void pintar(String color, int veces) { ... }
pintar(3, "roig");
```

`error: incompatible types: int cannot be converted to String`. **Arregla:** respecta l'ordre i el tipus de la firma; repassa la crida amb els ulls en la declaració.

3. Nombre d'arguments que no quadra

```
static int suma(int a, int b) { return a + b; }
int r = suma(1, 2, 3);
```

`error: wrong number of arguments`. **Arregla:** la firma mana: 2 paràmetres = 2 arguments, ni un de més ni un de menys.

4. Imprimir quan has de tornar

```
static void media(int a, int b) {
    System.out.println((a + b) / 2.0);   // void, sense retorn
}
double m = media(3, 4);   // ← això no compila: void no és un valor
```

`error: incompatible types: void cannot be converted to double`. **Arregla:** que el mètode `return` el resultat; imprimir és cosa de qui crida.

5. Oblidar `static` en cridar des de `main`

```
void saludar() { ... }              // sense static
public static void main(String[] a) {
    saludar();                      // ← ¡PROHIBIT de moment!
}
```

`error: non-static method saludar() cannot be referenced from a static context`. **Arregla:** d'aquesta unitat, els teus mètodes porten `static`. L'explicació completa (i l'alternativa amb objectes) viu en la U10.

6. Cridar com si fora sentència solta

```
saludar        // falten () i ;
saludar()      // falta el ;
```

Arregla: la crida és `saludar();` — amb parèntesis (encara que no porti arguments) i amb punt i coma.

7. Copiar i enganxar… el mètode equivocat

```
mostrarTotal();
mostrarTotal();   // volies cridar a mostrarMedia()
```

Sense error de compilació: **el més perillós dels set**. Compila, executa i ment. **Arregla:** llegeix el nom de cada crida com si l'escrivís un desconegut (perquè d'aquí a dos setmanes ho seràs).

> 📝 **Nota:** els errors 1-6 els veu CONRAD en vermell abans d'executar. El 7 és d'execució: per això el nom del mètode i la prova manual importen tant.

🩺 Com diagnosticar sense perdre't

Quan fallen un mètode, fes les preguntes en aquest ordre:

1. **¿Compila?** → Llegeix el missatge de CONRAD: `cannot find symbol` (mal escrit o fora d'àmbit), `wrong number of arguments`, `incompatible types` (tipus/ordre), `missing return statement`.
2. **¿Compila però no fa el que esperaves?** → Has cridat al mètode correcte (7)? Li passes els arguments en l'ordre que creus (2)?
3. **¿Imprimeix alguna cosa rara?** → Traça la crida: qui imprimeix i qui torna? Estàs usant la còpia que arriba per paràmetre?

> ⚠️ **Advertència:** no pegues l'error en el buscador abans de llegir-lo sencer. El 95% dels errors d'aquesta unitat diuen en la primera línia **què** està mal i en la segona **on**.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan tot compili i el resultat continuï sense quadrar, imprimeix (o millor: torna) els valors a mig camí. Depurar és mirar, no endevinar.

Exercici: el taller d'arregles

Aquest mètode hauria de tornar el major de dos nombres, però té **2 errors** (un de compilació i un de lògica):

```
public static int mayor(int a, int b) {
    if (a > b) {
        return a
    }
    return 0;
}
```

Quins són i com queden?

> 🔄 **Solució**
>
> 1. **Compilació:** falta `;` després de `return a` → `return a;`.
> 2. **Lògica:** el segon camí torna `0` en lloc de `b`: quan `a <= b`, el major és `b`.
>
> Versió correcta:
>
> ```
> public static int mayor(int a, int b) {
>     if (a > b) {
>         return a;
>     }
>     return b;
> }
> ```

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin error dona `int r = media(3, 4);` si `media` és `void` i només imprimeix?
2. Quins tres missatges busques primer en no compilar una crida?
3. Per què l'error 7 (cridar al mètode equivocat) no el detecta el compilador?
4. Quina paraula falta en `void log(String s) { System.out.println(s) }`?

> 🔄 **Respostes**
>
> 1. `void cannot be converted to int`: res de `void` pot guardar-se en una variable.
> 2. `cannot find symbol`, `wrong number of arguments`, `incompatible types` (i `missing return statement` si el fall està en el cos).
> 3. Perquè el nom i els arguments són correctes **sintàcticament**: només el programador sap que volies l'altre mètode.
> 4. El `;` després del `println` (i aquell error apareix en compilar la classe, no només en cridar).

✅ Resum en 3 frases

1. Els errors de mètodes són **set famílies**: return, ordre/tipus d'arguments, nombre d'arguments, void-vs-retorn, `static`, puntuació de la crida i nom equivocat.
2. Els sis primers els atrapa el compilador; el setè el detecta **el teu cervell** llegint noms en veu alta.
3. Diagnosticar sempre en ordre: compila → missatge d'error → traça de la crida.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `missing return statement` | Algun camí no torna el que la firma promet |
> | `wrong number of arguments` | Arguments ≠ paràmetres |
> | `incompatible types` | El tipus no encaixa (ordre inclòs) |
> | `static context` | Cridar a algo de classe des de `main` (detalls en U10) |
> | Error de lògica | Compila i executa, però ment |

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Àmbit de variables · **Següent:** 07 · Divideix el problema

07 · Divideix el problema

Main com a director d'orquestra: una responsabilitat per mètode, el programa que es llegeix com un resum i com trossejar sense por 🧱

> 🗺️ **Estàs en:** 🔧 **UO5 · Funcions i mètodes** → 07 · Divideix el problema

📬 La idea en una frase

> **Dividir és escriure un `main` que es llegeixi com un resum en anglès clar ("llegir dades, calcular mitjana, mostrar informe") i amagar cada frase en un mètode amb el seu nom.**

🎼 El director no toca els instruments

Un bon `main` sembla això:

```
public static void main(String[] args) {
    double[] notas = leerNotas();
    double media = calcularMedia(notas);
    String informe = formatearInforme(media);
    System.out.println(informe);
}
```

Quatre línies, quatre verbs, zero aritmètica. Vols saber com es llegeixen les notes? Obres `leerNotas()`. Com es calcula la mitjana? `calcularMedia()`. Cada detall viu al seu lloc i `main` continua sent un **índex del programa**, no un full de receptes.

Si aquell mateix programa vivira tot en `main`, tindries 40 línies barrejant lectura, càlcul i format on cap comentari et salva. El director d'orquestra no toca els instruments: **coordina**.

🔪 Com trossegar: el mètode dels verbs

Mira un `main` de taller i encén els verbs:

```
public static void main(String[] args) {
    // 1. llegir les notes del teclat          → leerNotas()
    // 2. comprovar si hi ha alguna suspesa    → tieneSuspensa(notas)
    // 3. calcular la mitjana                  → calcularMedia(notas)
    // 4. imprimir el resultat bonic           → mostrarResultado(media, notas)
}
```

Cada verb amb parèntesis en el comentari és **un candidat a mètode**. El patró es repeteix:

1. **Escriu el comentari-objectiu** (o llegeix-lo si ja està).
2. **Crea el mètode** amb aquell nom, `public static`, paràmetres si necessita dades, retorn si torna alguna cosa.
3. **Mou el codi** d'aquell pas al cos.
4. **Deixa una crida** en `main` i **executa**: si feia el mateix que abans, has trossejat bé.
5. Repeteix amb el següent verb. **Un mètode cada vegada.**

> 💡 **Consell:** si necessites un comentari per a explicar un bloc de 5 línies, aquell bloc ja té nom: fes-lo mètode i que el nom parli per ell.

On està el límit?

Es queda en `main`	Va al seu mètode
La seqüència de passos	Un pas amb nom propi
Un `if` de dues línies que decideix flux	Un bloc que repeteixes o que "fa una cosa"
La crida final d'impressió	El compte, la lectura, la cerca, el format

No hi ha llei universal de "més de N línies", sí una brúixola: **una responsabilitat per mètode**. `calcularMediaYMostrarYGuardar` són tres mètodes disfressats d'un.

🧪 Abans i després: l'informe de notes

Abans (tot en `main`):

```java
public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);
    double suma = 0;
    for (int i = 0; i < 5; i++) {
        System.out.print("Nota " + (i + 1) + ": ");
        suma += teclado.nextDouble();
    }
    double media = suma / 5;
    System.out.println("Mitjana: " + media);
    if (media >= 5) {
        System.out.println("¡APROVAT!");
    } else {
        System.out.println("Suspès. A per ell.");
    }
}
```

Després (un resum i dos receptes):

```java
public static void main(String[] args) {
    double[] notas = leerNotas(5);
    double media = calcularMedia(notas);
    mostrarVeredicto(media);
}

static double[] leerNotas(int cantidad) { ... }      // llegir
static double calcularMedia(double[] notas) { ... }  // calcular
static void mostrarVeredicto(double media) { ... }   // contar
```

Mateix programa, dos vides distintes: en la segona, demà afegeixes el màxim sense tocar res més (bé, gairebé: `leerNotas` ja torna un array… que veus de veritat en la UO6).

> 📝 **Nota:** l'ordre de declaració no importa (ho vas veure en el punt 2), així que `main` pot llegir-se amunt com un resum encara que els seus ajudants estiguin davall.

⭐ Be the Code, my friend… 🔗

> 🕶️ **Don Tip:** si dubtes entre trossejar o no, escriu la crida que **volguessis** tenir (`int max = encontrarMaximo(notas);`) i després fes que existeixi. El codi es dissenya cap endavant, com es demana en un restaurant.

Exercici: el verb amagat

Quin mètode (nom i firma) treguries d'aquest `main`? Torna `true` si tots els nombres són positius:

```
public static void main(String[] args) {
    int[] datos = {3, 7, 2};
    boolean todoPositivo = true;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] <= 0) {
            todoPositivo = false;
        }
    }
    System.out.println("¿Tot positiu? " + todoPositivo);
}
```

> 🔄 **Solució**

```
public static boolean todosPositivos(int[] datos) {
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] <= 0) {
            return false;
        }
    }
    return true;
}
```

Verb: "tots positius" → nom `todosPositivos`. Necessita dades? Sí → `int[] datos`. Torna alguna cosa? Sí, un judici → `boolean`. `main` queda en: `boolean ok = todosPositivos(datos);`. (Que el bucle usi arrays no et distraga: la idea del trosseig és la mateixa en qualsevol terreny.)

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines tres preguntes et fas en dissenyar un mètode? (pista: nom, entrades, sortides)
2. Quin és l'ofici de `main` en un programa ben trossejat?
3. Quina senyal diu "aquest bloc demana mètode"?
4. Quants blocs trosseges d'una vegada quan refactoritzes?

> 🔄 **Respostes**
>
> 1. Com es diu (quin verb fa)? Quines dades necessita (paràmetres)? Què torna (retorn o `void`)?
> 2. Coordinar: llegir → calcular → mostrar, en crides llegibles.
> 3. Necessitar un comentari per a explicar-lo, repetir-se en un altre lloc o contenir "una cosa amb nom".
> 4. Un sol, executant després de cada extracció: si algo es trenca, saps què va ser.

✅ Resum en 3 frases

1. Un `main` ben trossejat es llegeix com un **resum** de verbs; cada verb viu en un mètode amb una responsabilitat.
2. El mètode dels verbs: **comentari → nom → firma → moure codi → cridar i provar**, un mètode cada vegada.
3. La brúixola no és la longitud, és la **responsabilitat**: si el nom del mètode necessita "i", són dos mètodes.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Orquestració	`main` coordina; els mètodes treballen
Responsabilitat única	Un mètode, una cosa ben feta
Extreure mètode	Moure un bloc a un mètode nou
Refactoritzar	Reorganitzar sense canviar el que fa
Mètode helper	Ajudant petit al servei d'un altre pas

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Errors freqüents · **Següent:** 08 · Be the Code

08 · Be the Code

Refactoritza un programa de notes en casa, pas a pas, sense copiar solucions: els verbs, les firmes i el main que es llegeix sol 🛠️

> 🗺️ **Estàs en: 🔧 U05 · Funcions i mètodes** → 08 · Be the Code

---

📬 La idea en una frase

> **Aquest punt no té teoria nova: té un programa de 30 línies que has de trossejar amb les teves mans. Només demanes pistes quan t'atuesques de veritat.**

El programa d'abans (tot en `main`) és el teu taller de avui. La teva missió: convertir-lo en una orquestra de mètodes sense canviar el que imprimeix. Si ho has fet a mà, domines la unitat sencera.

🛠️ El teu punt de partida

```java
import java.util.Scanner;

public class InformeNotas {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        double suma = 0;
        boolean haySuspensa = false;

        for (int i = 0; i < 5; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            double nota = teclado.nextDouble();
            suma += nota;
            if (nota < 5) {
                haySuspensa = true;
            }
        }

        double media = suma / 5;
        System.out.println("Mitjana: " + media);

        if (haySuspensa) {
            System.out.println("Hi ha suspesa: a per ella.");
        } else if (media >= 7) {
            System.out.println("¡Qué nota!");
        } else {
            System.out.println("Aprovat amb genolls.");
        }
    }
}
```

Abans d'escriure res, respon en paper (en serio, en paper):

1. Quants **verbs amb nom propi** veus? (pista: tres: llegir, calcular, mostrar)
2. Quines dades necessita cadascun i **què torna** cadascun?
3. Què queda en `main` després de trossejar?

🪜 Pas a pas (pistes que no regalen el final)

1. Crea `static double[] leerNotas(int cantidad)` davall de `main`.

> 🪶 **Aturat?**
>
> Usa un `Scanner` **dins** del mètode, un `for` de `cantidad` vegades i un array `double[] notas = new double[cantidad]`. Torna `notas` al final.

2. Crea `static double calcularMedia(double[] notas)`.

> 🪶 **Aturat?**
>
> Suma les notes en un bucle i divideix entre `notas.length`. L'array arriba com a paràmetre; res de variables globals.

3. Crea `static boolean tieneSuspensa(double[] notas)`.

> 🪶 **Aturat?**
>
> Bucle: si alguna nota és menor que 5, `return true` a l'instant. Si el bucle acaba, `return false` (tots els camins tanquen).

4. Crea `static void mostrarVeredicto(double media, boolean suspensa)`.

> 🪶 **Aturat?**
>
> Les dades ja estan calculades: aquest mètode només **compta** el resultat. Dos paràmetres, sense `return`.

5. Reescriu `main` com a resum: llegir → mitjana → suspesa → mostrar.

> 🪶 **Aturat?**
>
> Quatre crides, zero bucles en `main`. Executa i compara la sortida amb l'original.

> 🔄 **Solució completa**

```java
import java.util.Scanner;

public class InformeNotas {

    public static void main(String[] args) {
        double[] notas = leerNotas(5);
        double media = calcularMedia(notas);
        boolean suspensa = tieneSuspensa(notas);
        mostrarVeredicto(media, suspensa);
    }

    static double[] leerNotas(int cantidad) {
        Scanner teclado = new Scanner(System.in);
        double[] notas = new double[cantidad];
        for (int i = 0; i < cantidad; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas[i] = teclado.nextDouble();
        }
        return notas;
    }

    static double calcularMedia(double[] notas) {
        double suma = 0;
        for (int i = 0; i < notas.length; i++) {
            suma += notas[i];
        }
        return suma / notas.length;
    }

    static boolean tieneSuspensa(double[] notas) {
        for (int i = 0; i < notas.length; i++) {
            if (notas[i] < 5) {
                return true;
            }
        }
        return false;
    }

    static void mostrarVeredicto(double media, boolean suspensa) {
        System.out.println("Mitjana: " + media);
        if (suspensa) {
            System.out.println("Hi ha suspesa: a per ella.");
```

```
        } else if (media >= 7) {
            System.out.println("¡Qué nota!");
        } else {
            System.out.println("Aprovat amb genolls.");
        }
    }
}
```

⚠️ **Advertència:** la sortida ha de ser **idèntica** abans i després. Si canvia, has mogut una línia de més: torni al pas anterior (refactoritzar és canviar la forma, mai el comportament).

🧪 El Lío: el refactor malograt

El teu company va refactoritzar i ara això **no compila**:

```
public class MalRefactor {
    public static void main(String[] args) {
        double m = media(3, 4, 5);
        System.out.println(m);
    }

    static void media(int a, int b, int c) {
        double resultado = (a + b + c) / 3.0;
        System.out.println(resultado);
    }
}
```

Pistes (no miris la solució encara):

1. Què vol fer `main` amb el resultat de `media`?
2. Què diu la firma que torna…?
3. Què passa amb el `println` de dins: és ofici de `media` o de qui crida?

> 🔄 **Solució**
>
> `main intenta guardar en double m el resultat d'un mètode void: void cannot be converted to double`. Dos arregles possibles; el correcte és:

```java
static double media(int a, int b, int c) {
    return (a + b + c) / 3.0;
}
```

i llevada el `println` de dins (o deixar-lo si `media` és de veritat "mostrar mitjana"… però llavors no s'anomenaria `media`). Calcula → torna; qui crida imprimeix.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines dues preguntes et fas de cada verb abans de crear el seu mètode? (entrades i sortides)
2. Per què `leerNotas` torna `double[]` i no imprimeix les notes?
3. Què ha de passar si trosseges bé i executes?
4. Quina és la senyal que has trossejat malament?

> 🔄 **Respostes**
>
> 1. Quins **paràmetres** necessita? Quin **retorn** té (o és `void`)?
> 2. Perquè `main` decideixi després: calcular mitjana, buscar suspeses… les dades viuen més enllà del llegir.
> 3. La sortida és idèntica al programa original.
> 4. Va canviar el comportament, o `main` continua amb bucles dins (no l'has convertit en resum).

✅ Resum en 3 frases

1. Trossejar = **verb → mètode** amb firma clara, moure el codi i deixar una crida en `main`.
2. El `main` final es llegeix com un resum: llegir → calcular → mostrar, sense un sol `for`.
3. Refactoritzar **mai** canvia la sortida: si canvia, es reverteix el pas.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Refactoritzar	Reorganitzar sense canviar el comportament
Extreure mètode	Sacar un bloc a la seva pròpia funció
Resum en `main`	Línies que es llegeixen com el pla del programa
Firma clara	Nom + paràmetres + retorn que s'entenen sols
Abans/després	Prova que la refactorització va neta

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Divideix el problema · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu

Fireside, CONRAD, crucigrama i laboratori: la festa de comiat de Funcions i mètodes 🎉

> 🗺️ **Estàs en:** 🔧 **U05 · Funcions i mètodes** → 09 · Repàs interactiu

---

📬 La idea en una frase

> **Aquest punt no ensenya res nou: destrossa l'après amb preguntes, errors aliens i un laboratori fins que ho tinguis a les mans.**

⭐ Be the Code, my friend…

Pregunta 1 — la firma

```
public static double promedio(int a, int b, int c) {
    return (a + b + c) / 3;
}
```

Què li passa a aquest mètode?

- (A) Res: compila i torna bé
- (B) Compila, però torna un `int` amb la part decimal perduda
- (C) No compila: `double` no pot rebre la suma de tres `int`

> › 🔄 **Solució**
>
> La **B**. `(a + b + c)` és `int` i `/ 3` també: divisió entera (`7` en lloc de `7,33...`). Java **no** canvia a `double` perquè la firma ho digui. Arregla: `return (a + b + c) / 3.0;`.

Pregunta 2 — l'ordre

```
static void pintar(String texto, int veces) { ... }
pintar(3, "OK");
```

Què passa?

- (A) Imprimeix `OK` tres vegades

- (B) Imprimeix `3` tres vegades
- (C) No compila: tipus al revés

> 🔄 **Solució**
>
> La **C**. El primer paràmetre és `String` i arriba un `int`: `incompatible types`. Els arguments viatgen en el mateix ordre que els paràmetres, sense excepcions.

Pregunta 3 — el pecat

```
static void doble(int n) {
    System.out.println(n * 2);
}
int r = doble(5);
```

Tria amb saviesa:

1. **`10`** — ¡correcte! … no: `void` no es guarda en res ✗
2. **No compila** — `main` intenta guardar un `void` en un `int` ✓
3. **`0`** — Java ompli amb zeros el que falta ✗ (això només passa en arrays)
4. **Imprimeix `10` i `r` queda en `0`** — sona lògic, però `void` no és un valor ✗

> 🔄 **Solució**
>
> La **2**: la línia `int r = doble(5);` dona `incompatible types: void cannot be converted to int`. El mètode `void` no produeix valor: si vols el doble en una variable, que el **torni** (`static int doble(...)` amb `return n * 2;`) i el `println` el poses tu on faci falta.

🔥 Fireside Chat: `void` vs `return`

> *Dos veterans del recorregut discuteixen junt a la màquina de cafè.*

`void`: - Jo faig i m'assabento. Imprimir, guardar, esborrar: efectes purs. El món em veu canviar i no em necessita tornar res.

`return`: - I per això quan et necessiten per a **calcular**, es perden. Em guarden en variables, em comparen, em passen com a argument a altres deu mètodes. Són dades de veritat.

`void`: - Els efectes també són dades… bé, efectes.

`return`: - ¿I què passa quan un novell et usa a mi però et posa `System.out.println` dins? Resultat: ningú rep el valor, s'imprimeix en l'instant exacte en que a vegades **no toca**, i qui crida no pot fer res amb ell.

`void`: - Això no és culpa meva, és culpa de barrejar els oficis.

`return`: - Exacte. Calcules → tornes. Decideixes → imprimeix. Si el teu mètode comença per `calcular`, `obtener` o `es`, vine a ma casa.

`void`: - Tractat. Jo per a `imprimir` i `guardar`; tu per a tot el que un altre mètode vagi a necessitar. Que `main` sigui l'únic que parli en veu alta.

> La lliçó: **`void` per a efectes, `return` per a valors.** Qui calcula torna; qui decideix imprimeix.

🕵️ Qui soc?

Endevina quin concepte de la unitat soc:

1. **Soc la variable que un mètode rep en ser cridat, declarada en la seva firma.**
2. **Soc el valor que arriba en la crida, en lloc exacte on polses el gatell.**
3. **Soc la regió on una variable existeix: de la firma a la clau de tancament.**
4. **Soc el final del mètode: tanqui l'execució i entregue el valor promés.**
5. **Soc la firma completa: nom + tipus dels paràmetres, la meva targeta de visita.**
6. **Soc un mètode que torna `true` o `false`: `esPar`, `tieneSuspensa`, `todoPositivo`.**

> 🔄 **Respostes**
>
> 1. **El paràmetre** — viu en la declaració i s'usa com a variable dins del mètode.
> 2. **L'argument** — el paràmetre és la cadira; l'argument és qui se senta.
> 3. **L'àmbit (scope)** — fora d'ell, `cannot find symbol`.
> 4. **`return`** — i si algun camí d'una firma amb retorn no l'alcança: `missing return statement`.
> 5. **La firma del mètode** — canvien els noms de variables, no la firma.
> 6. **El predicat** — el patró `boolean` que decora tota la POO futura.

🤖 CONRAD VS EL MÓN: “El main que es va fer el llist”

> *CONRAD, el nostre compilador mala llet, ha trobat una nota en la safata d’errors.*

CONRAD: - ¡UNA ALTRA VOLTA! Un alumne em diu: *CONRAD, vull cridar al meu mètode des de `main`*. I jo: *¿i el `static`? Doncs… el vaig llevar, que em paregia repetir.* SENSE `static` NO HI HA `main`! `non-static method cannot be referenced from a static context`. El `main` és estàtic de naixement; fins que no aprenguis a crear objectes (U09, U10), tots els teus mètodes porten `static`. No és decoració, és l’entrada obligatòria.

I el clàssic del retorn: el meu mètode calcula bé però em dona `missing return statement`. ¿I els camins? *¿Camins?* És clar! Si el `if` torna en una branca i l’`else` no, hi ha un sender que arriba al final amb la firma sense complir. Java no endevina la teva intenció: **tots els camins tanquen**.

I el de la variable fantasma: en `main` em diu `cannot find symbol: variable total`… però la vaig declarar en `sumar()`. PERQUÈ ÉS DE `sumar()`! Les locals moren amb el seu mètode. No hi ha ventanetes laterals entre cases: només arguments que entren i `return` que surt.

I el que m’empipa: criden `saludar()`… **sense `static`… sense `()`**… des d’un altre mètode… ¡LLEGIU EL MISSATGE ENTER, QUE LA PRIMERA LÍNIA DÍA QUÈ I LA SEGONA ON!

> CONRAD afegeix: si el teu error desapareix en canviar **una lletra**, era un typo; si desapareix en afegir `()`, era la crida; si desapareix en afegir `static`, era el context. El compilador et dona classes gratis: llegeix-les.

🎮 El joc de les decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Quina és la diferència real entre paràmetre i argument?
   - a) Cap, són sinònims b) Declaració vs crida c) Un és de `int` i l’altre de `String`
2. Què imprimeix `System.out.println(truco(7));` amb `truco` del punt 4?
   - a) `10` b) `14` c) `12`
3. Poden `a()` i `b()` tenir totes dues una variable `x`?
   - a) No: xoca b) Sí, són cases distintes c) Només si són `static`
4. Quin error dona `int r = suma(1, 2);` si `suma(int a, int b, int c)`?

- a) `wrong number of arguments` b) Res, el tercer queda a 0 c) `missing return statement`

5. Quan uses `void`?
   - a) Quan no tornis res útil, només fas un efecte b) Quan no saps què tornar c) Quan hi ha un sol paràmetre
6. Què fa `return;` dins d'un mètode `int`?
   - a) Torna 0 b) No compila c) Torna `null`

> 🔄 **Solucions**
>
> 1. **b)** — paràmetre en la declaració, argument en la crida.
> 2. **a)** — `10`: cau en la branca `x > 5` i torna `7 + 3`.
> 3. **b)** — cada mètode té el seu propi àmbit; dos `x` diferents.
> 4. **a)** — la firma demana 3 arguments i n'arriben 2.
> 5. **a)** — `void` = només efecte (imprimir, guardar); no hi ha valor que entregar.
> 6. **b)** — `return;` (buit) només és legal en `void`.

⚡ Laboratori de tortura: el programa sense nom

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que imprimeixi `Mitjana: 7.5`. Té **2 errors de compilació** i **1 error de lògica** que només es nota en la sortida.

```
public class Tortura {
    public static void main(String[] args) {
        int r = media(6, 7, 8, 9);
        System.out.println("Mitjana: " + r);
    }

    public static int media(int a, int b, int c, int d) {
        double m = (a + b + c + d) / 4;
        return m;
    }
}
```

La teva tasca: que compili i que imprimeixi `Mitjana: 7.5`. Si imprimeix `7`, has trobat la lògica; si no compila, són les dues de compilació.

Pistes per a quan te frustres (no abans):

1. Quants paràmetres declara `media` i quants arguments li passes? *si quadra, continua.*

   > **¿I si continuo encallat?**
   > 4 i 4: això està bé. Mira dins del cos: quin tipus és `m` i quin tipus promet `return`?

2. Què diu exactament l'error de la línia `return m;`?

   > **¿I si continuo encallat?**
   > `incompatible types: double cannot be converted to int`. Una via: canviar la firma a `static double media(...)` i `double r` en `main`. Una altra: dividir amb `/ 4.0` i castear... però llavors perdries els decimals: la via bona és la primera.

3. ¿Ja compila, però imprimeix `Mitjana: 7`?

   > **¿I si continuo encallat?**
   > La divisió és entera: `(6+7+8+9)/4 = 30/4 = 7`. Amb la firma `double` i `/ 4.0` surt `7.5` de veritat.

> 🔄 **Solució completa**

```java
public class Tortura {
    public static void main(String[] args) {
        double r = media(6, 7, 8, 9);
        System.out.println("Mitjana: " + r);
    }

    public static double media(int a, int b, int c, int d) {
        return (a + b + c + d) / 4.0;
    }
}
```

Errors: (1) `double m` no cap en un `return` d'`int` → firma `double`; (2) `int r = ...` no rep `double` → `double r`; (3) lògica: divisió entera → `4.0`.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🗣️ **El Traductor**	Explicar en veu alta la diferència entre paràmetre i argument sense dubtar
📤 **El Tornador**	Escriure un predicat (`esPar`) que tanqui tots els seus camins a la primera
🏠 **El Paleta d'Àmbits**	Diagnosticar un `cannot find symbol` d'un cop d'ull (typo o fora de casa)
🧯 **El Bomber**	Arreglar els 7 entrebancs del punt 6 sense mirar la taula
🧱 **El Director d'Orquestra**	Trossejar un `main` de 30 línies en 4 mètodes amb sortida idèntica
🛠️ **El Refactoritzador**	Completar el Be the Code amb la sortida byte a byte igual que a l'inici

🤔 Atreveix-te a pensar

1. **Sense executar:** què imprimeix?

```
public class Misterio {
    static int f(int x) {
        if (x > 2) {
            return x + 1;
        }
        return f(x + 1);
    }

    public static void main(String[] args) {
        System.out.println(f(0));
    }
}
```

> 🔄 **Resposta**
>
> **4.** Cadena de crides: `f(0)` → `f(1)` → `f(2)` → `f(3)` → `3 > 2` és cert → `3 + 1 = 4`. Fixa't en que `2 > 2` és fals, així que continua encadenant. Compte: estàs mirant **recursió**, el plat fort de la

> U08.

2. Per què diem que `main` ha de llegir-se “com un resum” i no com una recepta?
3. Si tots els teus mètodes imprimeixen, què li falta al teu programa per a ser reutilitzable?
4. Se’t ocurren un cas legítim en què un mètode `void` cridi a un altre `void` i el programa complet tingui sentit sense un sol `return` amb valor?

🧩 Crucigrama de bits

```
Horitzontal:
1. El que declara el mètode: tipus + nom de cada entrada (10 lletres)
3. Mètode que torna true o false (9 lletres)
5. Tipus de dada sense valor: "____ media(3, 4);" (4 lletres)
6. On viu una variable: el seu _______ (6 lletres)

Vertical:
2. El que passa en la crida: "sumar(3, 4)" són ____ (9 lletres)
4. Promesa de la firma que compleix return (7 lletres)
7. Paraula que tanca el mètode i entrega (6 lletres)
```

> › 🔄 **Solucions**
>
> **Horitzontal:** 1. PARÀMETRES · 3. PREDICAT · 5. VOID · 6. ÀMBIT **Vertical:** 2. ARGUMENTS · 4. FIRMA · 7. RETURN

💬 Preguntes d’entrevista de treball

> Preguntes reals que et farien per a programador Java junior.

1. **“Explica’m, com si jo fos la teva àvia, la diferència entre un paràmetre i un argument.”**
2. **“Escriu un mètode que torni `true` si tots els nombres d’una llista són positius. Per què `boolean` i no un `println`?”**
3. **“Què és l’àmbit d’una variable i per què existeix?”**
4. **“El teu mètode compila però el resultat no arriba a `main`. Per què pot ser?”**
5. **“Quan faries un mètode `void` i quan amb retorn? Dona’m un exemple de cada un.”**

6. **“Refactoritza un `main` de 40 línies. Per on comences i com comproves que no l’has trencat?”**

🤷 No hi ha preguntes tontes

> **❓ ¿Per què Java m’obliga a posar `static` si jo només vull cridar al meu mètode?**

Perquè `main` és estàtic: s’executa **sense** crear cap objecte. Un mètode no estàtic pertany a un objecte, i no hi ha objecte al qual pertànyer. De moment tots els teus mètodes són de classe (`static`); quan creguis el teu primer objecte en la U09 i vegis `static` en detall en la U10, això farà *clic* i no tornaràs a dubtar.

> **❓ Puc cridar a un mètode des d’un altre mètode que estigui damunt o baix?**

Sí, sense restriccions d’ordre: tots es coneixen dins d’una classe. `main` pot cridar a `calcularMedia` encara que estigui escrita 50 línies més avall.

> **❓ Si passi `x` per paràmetre, per què el meu mètode no pot canviar el meu `x`?**

Perquè rep una **còpia** (per a tipus primitius). És un disseny deliberat: els mètodes no reescriven res de ningú d’amagades; si hi ha un resultat nou, el **tornen** amb `return`. Amb els arrays la cosa canvia (es passa la referència) i ho veuràs en la U06.

> **❓ Un mètode pot cridar a si mateix?**

Pot, i se’n diu **recursió**. És potent i perillós: si no arriba mai al seu cas base, s’acaba la pila i explota. Toca en la U08, Algorítmica II, amb guants.

🎬 Postcrèdits

El novell acaba de trossejar el seu informe de notes: `main` té quatre línies, cada verb viu al seu lloc i la sortida és idèntica a la del matí. S’acosta CONRAD, el compilador mala llet, amb la seva tassa fumejant.

CONRAD: - Vagi. Quatre mètodes, zero bucles en `main` i ni un sol `println` on no tocava. ¿Segur que ets el mateix que va començar la unitat amb tot en 40 línies seguides?

Novell: - *somriu* Vaig començar copiant i enganxant, i he acabat amb un programa que es llegeix com una llista de la compra. ¿I ara què?

CONRAD: - *beu un glop* Ara aquestes dades volen créixer: cinc notes s'han convertit en cent, i guardar-les en variables soltes és un infern. Necessites un aparcament de veritat: arrays, índexs i `length`. Un altre plat, i ve just després.

El novell guarda el seu projecte, tanca l'IDE i sent que ja no escriu blocs de codi: **escriu plans amb noms**.

PRÒXIMAMENT EN U06: Arrays. L'aparcament de dades del mateix tipus: crear, omplir, recórrer i no sortir-te mai de l'últim índex. 🅿️

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Be the Code · **Següent:** U06 · Arrays

U06 — Arrays

L'aparcament de dades: crea, recorre, ordena i busca amb els arrays de Java 🅿️

Benvingut a l'aparcament. En les unitats anteriors vas aprendre a **decidir** (U04) i a **repetir**, però sempre guardant una cosa per variable. Fins avui. En aquesta unitat descobreixes els **arrays**: la forma de guardar centenars (o milions) de dades del mateix tipus sota un sol nom i de moure-les amb un bucle.

Pensa en tot el que és, en el fons, una llista de coses: les notes dels teus 30 alumnes, la temperatura de cada dia de l'any, les fitxes d'un tauler, els noms d'una classe. Doncs tot això, portat a Java, és aquesta unitat: **crear** l'array, **recórrer-lo**, **ordenar-lo**, **buscar-lo**, gestionar-lo amb la classe `Arrays` i passar-lo a **mètodes** sense que se t'escapi res pel camí.

I, com sempre, CONRAD el compilador serà per aquí per a recordar-te que un índex fora de rang és l'error més car (i més típic) del curs: `ArrayIndexOutOfBoundsException` no perdona.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat

En acabar, seràs capaç de:

- **Crear** arrays de qualsevol tipus, inicialitzar-los i entendre els seus valors per defecte (`0`, `false`, `null`…).
- **Recórrer** un array amb `for` i amb `for-each`, sabent quan usar cada un.
- Treballar amb **arrays multidimensionals** (files i columnes, taulers, matrius) i amb arrays irregulars.
- Aprofitar la classe **`Arrays`**: `toString`, `sort`, `binarySearch`, `copyOf`, `fill` i `equals`.
- **Passar arrays a mètodes** i entendre el pas per referència de la còpia.
- Aplicar els arrays a casos reals: llistes de noms, notes, objectes propis i dades en taula.
- **Depurar** els errors típics dels arrays: índexs fora de rang, `null`, imprimir sense `toString`…

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Arrays: l'aparcament de dades	Crear arrays, índexs, `length` i el primer BOOM	Tots
02 · Recórrer arrays: for i for-each	El duo inseparable i la variant peresosa de només lectura	Tots
03 · Arrays multidimensionals	Arrays de arrays, files, columnes i bucles niats	Tots
04 · La classe Arrays: la teva navalla suïssa	`toString`, `sort`, `binarySearch`, `copyOf`, `fill`, `equals`	Tots
05 · Arrays i mètodes	Passar arrays per referència i tornar-los	Tots
06 · Aplicacions dels arrays	Arrays de String, de char, d'objectes i dades en taula	Tots
07 · Be the Code: l'aparcament es gestiona	Programar invers, cerques i compactar a mà	Tots
08 · Array-revelde: errors comuns i depuració	La galeria de monstres dels arrays i com caçar-los	Tots
09 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA6)

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA6 a)	S'han escrit programes que utilitzen arrays.	✅ Tots
RA6 b)	S'han reconegut les llibreries de classes relacionades amb tipus avançats de dades.	✅ Punt 4

> 📌 Els arrays són la base de tot el que ve: els algoritmes de la U07 busquen i ordenen arrays, i quan un aparcament de grandària fixa se't queda petit, les col·leccions de la U12 (ArrayList, LinkedList…) prenen el relleu. Aprendre arrays a fons avui és la inversió que rendeix durant tot el curs.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites variables, bucles i `if` de la U04.
- Ja saps crear arrays i vols molta cosa? → Salta al recorregut o a la classe Arrays.
- Només vols veure arrays en acció real? → Llegeix Aplicacions dels arrays.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Arrays: l'aparcament de dades
⏭️ **En acabar la unitat, continua en U07 · Algorítmica I: fonaments.**

01 · Arrays: l'aparcament de dades

Declaració, índexs i el primer BOOM: la ArrayIndexOutOfBoundsException que et farà estavellar-te alguna vegada 🅿️

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 01 · Arrays: l'aparcament de dades

---

📬 La idea en una frase

> **Un array és un aparcament de grandària fixa: guarda moltes dades del mateix tipus sota un sol nom, i cada plaça té un número (l'índex) que comença en 0.**

Fins ara guardaves una cosa per variable. Amb els arrays en guardes 100 sota el mateix cartell. És la primera eina "de veritat" per a gestionar quantitats, i d'ella penja tot el que ve després.

🐱 El problema: tens 100 gats i un sol nom

Imagina que tens 100 gats i necessites guardar els seus noms. Podries fer això:

```
String gato1 = "Bigotes";
String gato2 = "Garfield";
String gato3 = "Misifú";
// ... 97 líneas después ...
String gato100 = "Calcetines";
```

Però aleshores arriba el gat 101 i el teu programa es cau. O pitjor: vols saber quants gats comencen amb "M" i has d'escriure 100 `if`. L'esquena ja et fa mal només de pensar-ho.

> ⚠️ **Advertència:** si alguna vegada escrius `gato1`, `gato2`, `gato3`… `gatoN` al teu codi, en algun lloc un programador sènior plora. Els arrays existeixen exactament per a això.

🅿️ L'array: el teu primer aparcament

Un array és com un aparcament de diverses plantes. Cada plaça té un número (l'**índex**) i en cada plaça només caben cotxes del mateix tipus (bé, i les seves subclasses).

```
String[] gatos = new String[100];
// Has creat un aparcament amb 100 places per a Strings
```

Hi ha dues formes de declarar-lo i crear-lo:

```
int[] numeros = new int[5];      // 5 places, totes buides (0)
int[] directo = {10, 20, 30};    // 3 places, ja ocupades
```

La primera plaça és la **0**, no la 1. Això confon tothom al principi. Accepta-ho.

> 💡 **Consell:** pensa en els índexs com a distàncies des de la primera posició. La primera casa és a 0 passes de tu, no a 1.

Els valors per defecte

Quan crees un array amb `new`, cada plaça s'ompli amb el valor per defecte del tipus:

Tipus	Valor per defecte
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`'\U0100'`
Objectes (`String`, `Persona`…)	`null`

Aquell últim és el que mossega: un array de `String` acabat de crear està ple de `null`, no de `""`. Si intentes cridar un mètode sobre una plaça null, et portes un `NullPointerException` a l'acte.

🚗 Com ficar coses a l'aparcament

S'accedeix a una plaça amb claudàtors i el número d'índex:

```java
String[] gatos = new String[3];
gatos[0] = "Bigotes";
gatos[1] = "Garfield";
gatos[2] = "Misifú";
gatos[3] = "Calcetines"; // ¡BOOM!
```

Què passa a l'última línia? Et vas a estavellar.

¡BOOM! La ArrayIndexOutOfBoundsException

```java
int[] numeros = new int[5];
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;
numeros[5] = 60; // Index 5 out of bounds for length 5
```

L'array té places del 0 al 4. Demanar la 5 és com intentar aparcar on no hi ha plaça. Java et respon amb `ArrayIndexOutOfBoundsException` i el teu programa mor a l'acte. És **la** excepció més típica d'aquesta unitat i la primera que quasi tothom pateix.

> 📝 **Nota:** els índexs vàlids van de `0` a `length - 1`. L'últim element sempre és `arr[arr.length - 1]`. No t'ho penses dues vegades: memoritza-ho.

La longitud: `length` sense parèntesis

Per a saber quantes places té l'aparcament:

```java
int[] numeros = new int[10];
System.out.println(numeros.length);  // 10 (sense parèntesis)
```

Fixa't: `array.length` NO porta parèntesis. No és un mètode, és un atribut. Els `String` usen `length()`. Els arrays usen `length`. És una trampa mortal als exàmens.

> ⚠️ **Advertència:** l'array és un objecte (és al heap), però la variable que el referencia és a la pila (stack). Quan passes un array a un mètode, passes la referència, no les dades. Ho veuràs en detall al punt 5, però ja ho saps: si modifiques l'array dins d'un mètode, els canvis afecten l'original.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** els arrays tenen grandària fixa. Una vegada creats, no pots afegir ni treure elements. Si ho necessites, se'n crea un altre i es copia. En la U12 veuràs les col·leccions, que solucionen això.

Exercici: l'array que es duplica

```
public class BeTheArray {
    public static void main(String[] args) {
        int[] arr = new int[4];
        arr[0] = 2;
        arr[1] = 4;
        arr[2] = 6;
        arr[3] = 8;

        for (int i = 0; i < arr.length; i++) {
            arr[i] = arr[i] * 2;
        }

        System.out.println(arr[2]);
    }
}
```

Què imprimeix?

- (A) 6
- (B) 8
- (C) 12
- (D) 16

> 🔄 **Solució**
>
> La **C**. L'array original és `{2, 4, 6, 8}`. Després del bucle, cada element es multiplica per 2: `{4, 8, 12, 16}`. Per tant, `arr[2]` = 12. El `for` recorre totes les places i les sobreescriu al lloc: no necessites un altre array.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quantes places té `int[] a = new int[7]` i quins són els índexs vàlids?
2. Quin valor té `boolean[] b = new boolean[3]` a cada plaça?
3. Quina excepció llança `arr[arr.length]`?
4. `numeros.length` o `numeros.length()`? I per a un `String`?

> 🔄 **Respostes**
>
> 1. 7 places. Índexs del 0 al 6 (`length - 1`).
> 2. `false` a totes: és el valor per defecte de `boolean`.
> 3. `ArrayIndexOutOfBoundsException`: la plaça `length` no existeix, les vàlides acaben en `length - 1`.
> 4. `numeros.length` (atribut, sense parèntesis) per a arrays; `texto.length()` (mètode, amb parèntesis) per a `String`.

✅ Resum en 3 frases

1. Un **array** guarda molts valors del mateix tipus sota un sol nom, en un aparcament de **grandària fixa**.
2. S'accedeix per **índex** des de `0` fins a `length - 1`; passar-te provoca `ArrayIndexOutOfBoundsException`.
3. La **longitud** es pregunta amb `length` (sense parèntesis) i les places acabades de crear s'omplin amb els valors per defecte (`0`, `false`, `null`…).

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Array	Aparcament de dades del mateix tipus, grandària fixa
Índex	Número de la plaça; comença en 0
`length`	Grandària de l'array (atribut, sense parèntesis)
`ArrayIndexOutOfBoundsException`	Error per sortir-te de les places vàlides

Terme	Idea general
Valor per defecte	El que ocupa una plaça acabada de crear (`0`, `false`, `null`…)

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · Recórrer arrays: for i for-each

02 · Recórrer arrays: for i for-each

El duo inseparable (for + array) i la variant peresosa que només llegeix: el for-each 🌀

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 02 · Recórrer arrays: for i for-each

---

📬 La idea en una frase

> **Un array sense un bucle és un aparcament que ningú no visita: el `for` recorre plaça a plaça per índex, i el `for-each` fa el mateix però sense índex i només per a llegir.**

Al punt 1 vas aprendre a crear l'aparcament. Ara toca el divertit: passejar per totes les places i fer alguna cosa amb cada cotxe. I aquí mana un duo tan inseparable com el pa i la mantega.

🔗 El duo inseparable: for + array

Els arrays i els bucles `for` es veuen sempre junts. No és casualitat: el `for` té un comptador natural (`i`) que encaixa perfecte amb els índexs de l'array.

```
String[] gatos = {"Bigotes", "Garfield", "Misifú", "Calcetines"};

for (int i = 0; i < gatos.length; i++) {
    System.out.println("Gato " + i + ": " + gatos[i]);
}
```

Sortida:

```
Gato 0: Bigotes
Gato 1: Garfield
Gato 2: Misifú
Gato 3: Calcetines
```

Fixa't en la condició: `i < gatos.length`. Si escrivissis `i <= gatos.length`, a l'última volta demanaries la plaça `length` i… ¡BOOM! `ArrayIndexOutOfBoundsException`. És l'error de bucle més comés de l'univers.

Patrons clàssics amb for

El `for` amb índex no només serveix per a imprimir. Aquests tres patrons es repeteixen a cada exercici del curs:

Sumar tots els elements:

```
int[] notas = {7, 8, 5, 9, 6};
int suma = 0;
for (int i = 0; i < notas.length; i++) {
    suma += notas[i];
}
System.out.println("Media: " + (double) suma / notas.length);
```

Buscar un valor (cerca lineal):

```
int[] edades = {12, 45, 7, 34, 89};
int buscado = 34;
int posicion = -1;
for (int i = 0; i < edades.length; i++) {
    if (edades[i] == buscado) {
        posicion = i;
        break;
    }
}
System.out.println(posicion >= 0 ? "Encontrado en " + posicion : "No encontrado");
```

Modificar l'array al lloc (això només es pot amb índex):

```
int[] numeros = {1, 2, 3, 4, 5};
for (int i = 0; i < numeros.length; i++) {
    numeros[i] = numeros[i] * 10;
}
// {10, 20, 30, 40, 50}
```

🛋️ for-each: la variant peresosa

Si no necessites l'índex (només vols llegir els valors), hi ha una sintaxi més curta:

```java
String[] gatos = {"Bigotes", "Garfield", "Misifú"};

for (String gato : gatos) {
    System.out.println("Miau: " + gato);
}
```

Es llegeix: “per a cada String gato en gatos, fes això”. La variable `gato` va prenent el valor de cada plaça una a una, sense que tu gestiones comptadors ni claudàtors.

> 📝 **Nota:** el for-each és **només de lectura**. No pots modificar l’array original dins del bucle. Bé, pots intentar-ho, però el canvi es perd en l’èter: `gato = "Nuevo"` només canvia la variable local, no la plaça de l’array. Per a modificar, usa el `for` amb índex.

Quan usi cada un?

Situació	Bucle recomanat
Només llegir i no m’importa la posició	`for-each`
Necessito l’índex (posicions, comparar veïns)	`for` clàssic
Vull modificar els elements de l’array	`for` clàssic
Recórrer cap enrere o de dos en dos	`for` clàssic
Recórrer una col·lecció (`ArrayList`, `HashSet`…)	`for-each` (U12 Col·leccions)

> 💡 **Consell:** si no necessites l’índex, usa `for-each`. És més curt, més llegible i t’estalvia una classe sencera d’errors (oblidar el ++, començar en 1, escriure <=…).

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** el for-each és com un robot que va per l’aparcament llegint les matrícules. Només llegeix: no pot repintar els cotxes.

Exercici: la suma dels pacients

```java
public class BeTheForEach {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30, 40, 50};
        int total = 0;

        for (int n : numeros) {
            if (n % 20 == 0) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

Què imprimeix?

- (A) 60
- (B) 90
- (C) 120
- (D) 150

> 🔄 **Solució**
>
> La **A**. El for-each recorre cada element: 10, 20, 30, 40, 50. El `if` només suma els múltiples de 20, que són 20 i 40. `20 + 40 = 60`. Els altres s'ignoren.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què imprimeix `for (int i = 0; i < a.length; i++)` sobre `{1,2,3}` si imprimeixes `a[i]`?
2. Per què `for (int i = 0; i <= a.length; i++)` llança excepció?
3. Es pot modificar un array amb `for-each`?
4. Quin bucle usaries per a imprimir l'array al revés?

> 🔄 **Respostes**

1. `1 2 3`. El bucle recorre les places 0, 1 i 2.
2. Perquè a l'última volta (`i == a.length`) demana una plaça que no existeix: els índexs vàlids acaben en `length - 1`.
3. No. El for-each és de només lectura: la variable del bucle és una còpia del valor, no la plaça.
4. Un `for` clàssic cap enrere: `for (int i = a.length - 1; i >= 0; i--)`.

✅ Resum en 3 frases

1. El **`for` clàssic** recorre l'array amb un índex (`i`) des de `0` fins a `length - 1`, i és l'únic que permet **modificar** elements.
2. El **`for-each`** llegeix tots els valors sense índex: perfecte per a sumar, comptar o imprimir, però és **només de lectura**.
3. La condició del bucle és `i < length`; si escrius `<=`, et estavelles contra `ArrayIndexOutOfBoundsException`.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Recórrer	Visitar cada element de l'array, un a un
`for` clàssic	Bucle amb índex per a llegir, modificar o buscar
`for-each`	Bucle de només lectura: "per a cada X en Y"
Cerca lineal	Recórrer de principi a fi buscant un valor
`break`	Tallar el bucle en el moment en què trobes allò que busques

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Arrays: l'aparcament de dades · **Següent:** 03 · Arrays multidimensionals

03 · Arrays multidimensionals

L'aparcament de diverses plantes: arrays de arrays, files i columnes, i com recórrer-los sense perdre't 🏢

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 03 · Arrays multidimensionals

📬 La idea en una frase

> **Un array multidimensional és un array els elements del qual són altres arrays: un aparcament de diverses plantes, on cada plaça es localitza per planta i número.**

Fins ara cada plaça guardava una dada. I si la dada en si és un altre aparcament? Aleshores tens una taula amb files i columnes. Això és el que uses per a representar taulers, matrius, mapes i qualsevol cosa amb dues (o més) dimensions.

🏢 L'aparcament de diverses plantes

Un array bidimensional és "un array de arrays". Es declara amb doble claudàtor:

```
int[][] tabla = new int[3][4]; // 3 files, 4 columnes
```

Pensa-ho com un aparcament amb **3 plantes** i **4 places** per planta. Per a accedir a una plaça necessites dos números: el de la planta (fila) i el de la plaça dins d'ella (columna).

```
tabla[0][0] = 1; // fila 0, columna 0
tabla[1][2] = 5; // fila 1, columna 2
```

També pots crear-lo ja ple:

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

`matriz.length` és el nombre de files (3). `matriz[0].length` és el nombre de columnes de la fila 0 (3).

> 💡 **Consell:** anomena els índexs dels arrays multidimensionals com a `fila` i `col`, o `i` i `j`. NO uses `x` i `y` tret que realment treballes amb coordenades. El teu jo del futur t'ho agrairà.

🎲 Arrays de arrays irregulars

Java permet "arrays de arrays" on cada fila té un nombre diferent de columnes (els anomenats *jagged arrays*, arrays dentats):

```
int[][] irregular = new int[3][];
irregular[0] = new int[2];
irregular[1] = new int[5];
irregular[2] = new int[3];
```

La fila 0 té 2 columnes, la fila 1 en té 5 i la fila 2 en té 3. Per a què serveix? Triangles, piràmides o simplement dades que no formen un rectangle perfecte (per exemple, els dies de cada mes: febrer en té menys).

```
int[][] diasPorMes = {
    {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}, // un sol mes per fila
    {1, 2, 3},                                         // exemple de fila curta
};
```

En un array irregular, `irregular[i].length` pot ser diferent per a cada `i`. Per això els recorreguts usen `matriz[i].length` dins del bucle, mai un número fix.

🔁 Recórrer un array 2D: els bucles niats

Per a recórrer un rectangle perfecte necessites dos bucles: un per a les files i un altre per a les columnes.

```java
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

for (int i = 0; i < matriz.length; i++) {          // files
    for (int j = 0; j < matriz[i].length; j++) {   // columnes
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

Sortida:

```
1 2 3
4 5 6
7 8 9
```

El bucle exterior va fila a fila; l'interior recorre cada columna d'aquella fila. Fixa't que l'interior usa `matriz[i].length`: així funciona també amb arrays irregulars.

I la versió peresosa amb for-each (només lectura):

```java
for (int[] fila : matriz) {
    for (int valor : fila) {
        System.out.print(valor + " ");
    }
    System.out.println();
}
```

Cada `fila` és un `int[]`, i sobre ell tornes a usar for-each. Arrays de arrays, bucles de bucles.

🧱 Per a què serveix de veritat

Els arrays 2D no són un caprici acadèmic. Són la forma natural de representar:

Situació	Array
Tauler de joc (escacs, busca-mines, tres en ratlla)	`char[][]` o `boolean[][]`
Notes per alumne i assignatura	`double[][]`

Situació	Array
Mapa de píxels d'una imatge	`int[][]`
Matrius matemàtiques	`int[][]`, `double[][]`

Un busca-mines simplificat, per exemple, és un `boolean[][]`:

```
boolean[][] minas = new boolean[5][5];
minas[2][3] = true;  // hi ha una mina en fila 2, columna 3
```

I per a saber si una posició existeix, sempre preguntes abans de tocar: l'índex de fila va de `0` a `length - 1` i el de columna de `0` a `matriz[fila].length - 1`. Sortir-te d'allà torna a ser `ArrayIndexOutOfBoundsException`, però ara amb dues coordenades.

> ⚠️ **Advertència:** `matriz.length` i `matriz[0].length` NO són el mateix. El primer són les files; el segon, les columnes de la fila 0. Confondre'ls és l'error clàssic dels principiants amb matrius.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan recórres una matriu amb dos bucles, l'ordre importa: `i` per a files, `j` per a columnes. Si els intercanvies, estàs recorrent la matriu transposada.

Exercici: la diagonal que no es veu

```java
public class BeTheDiagonal {
    public static void main(String[] args) {
        int[][] m = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        int suma = 0;
        for (int i = 0; i < m.length; i++) {
            suma += m[i][i];
        }
        System.out.println(suma);
    }
}
```

Què imprimeix?

- (A) 12
- (B) 15
- (C) 18
- (D) 45

> 🔄 **Solució**
>
> La **B**. El bucle suma `m[0][0] + m[1][1] + m[2][2] = 1 + 5 + 9 = 15`. És la **diagonal principal**: quan fila i columna són el mateix número, camines per la diagonal de dalt-esquerra a baix-dreta. Un sol bucle, no dos.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quantes files i columnes té `int[][] a = new int[3][4]`?
2. Com accedeixes a l'element de la fila 2, columna 1?
3. Què representa `a.length` i què representa `a[0].length`?
4. Per què en un array irregular el bucle interior usa `a[i].length`?

> 🔄 **Respostes**

1. 3 files i 4 columnes.
2. `a[2][1]`. Recorda: primer la fila, després la columna, tots dos començant en 0.
3. `a.length` és el nombre de files; `a[0].length`, el nombre de columnes de la primera fila.
4. Perquè cada fila pot tenir una grandària diferent. Usar `a[i].length` garanteix que recorres exactament les columnes d'aquella fila, ni més ni menys.

✅ Resum en 3 frases

1. Un array **bidimensional** és un array de arrays: s'accedeix amb dos índexs `[fila][columna]`.
2. Es **recorre amb dos bucles niats**: l'exterior per a files i l'interior per a columnes, usant `matriz[i].length`.
3. Java admet **arrays irregulars** on cada fila té la seva longitud, i `matriz.length` i `matriz[i].length` no són el mateix.

🐛 **Vocabulari ràpid**

Terme	Idea general
Array multidimensional	Array els elements del qual són altres arrays
Fila	Primera dimensió (l'índex `i`)
Columna	Segona dimensió (l'índex `j`)
Array irregular	Cada fila amb diferent nombre de columnes
Diagonal principal	Els elements on `fila == columna`

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Recórrer arrays: for i for-each · **Següent:** 04 · La classe Arrays: la teva navalla suïssa

04 · La classe Arrays: la teva navalla suïssa

toString, sort, copyOf, binarySearch i fill: els mètodes estàtics que t'estalvien reinventar la roda 🔧

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 04 · La classe Arrays: la teva navalla suïssa

📬 La idea en una frase

> **`java.util.Arrays` és una classe plena de mètodes estàtics per a treballar amb arrays: imprimir, ordenar, copiar, buscar i omplir sense escriure tu el bucle.**

Els arrays tenen un problema: no tenen mètodes. `numeros.sort()` no existeix. Per això Java et regala una classe d'eines, totes estàtiques, perquè no hagis de reinventar el bucle cada vegada. És el més semblant a una navalla suïssa que existeix en el món dels arrays.

🧰 La classe Arrays

S'importa amb `import java.util.Arrays;` i els seus mètodes es criden passant-li l'array:

```
import java.util.Arrays;

public class EjemploArrays {
    public static void main(String[] args) {
        int[] numeros = {5, 2, 8, 1, 9};

        Arrays.sort(numeros);                    // {1, 2, 5, 8, 9}
        String texto = Arrays.toString(numeros); // "[1, 2, 5, 8, 9]"
        System.out.println(texto);
    }
}
```

Recorda: com són mètodes `static` (ho vas veure en la U09), es criden amb el nom de la classe: `Arrays.xxx(array)`. No cal crear cap objecte `Arrays` — de fet, no pots.

🔍 Arrays.toString: imprimir bonic

El mètode més socorregut. Sense ell, imprimir un array mostra escombraries:

```java
int[] numeros = {1, 2, 3};
System.out.println(numeros);                // [I@6d06d69c   (adreça de memòria, inútil)
System.out.println(Arrays.toString(numeros)); // [1, 2, 3]
```

Sense `toString`, Java imprimeix l'adreça de memòria de l'objecte (`[I@6d06d69c`), no les dades. Amb ell, obtens alguna cosa llegible. Per a arrays 2D existeix `Arrays.deepToString()`.

> ⚠️ **Advertència:** `numeros.toString()` tampoc no funciona: els arrays no sobreescriuen `toString()`. Sempre `Arrays.toString(numeros)`.

📊 Arrays.sort: ordenar d'un buf

Ordena l'array al lloc, de menor a major (segons l'ordre natural del tipus):

```java
int[] notas = {7, 3, 9, 5};
Arrays.sort(notas);
System.out.println(Arrays.toString(notas)); // [3, 5, 7, 9]
```

Amb `String` ordena alfabèticament. Compte amb les majúscules: `"Zebra"` va abans que `"abc"` perquè les majúscules tenen menor valor Unicode. És ordenació lexicogràfica, no "de diccionari humà".

🔎 Arrays.binarySearch: buscar ràpid (però només ordenat)

La cerca binària parteix l'array per la meitat a cada pas. És rapidíssima, però **exigeix que l'array estigui ordenat abans**.

```java
int[] numeros = {3, 5, 7, 9, 11};
int pos = Arrays.binarySearch(numeros, 7);
System.out.println(pos); // 2
```

Si el valor no hi és, torna un número negatiu (`-(puntDInsercio) - 1`). Compte: si l'array no està ordenat, el resultat és impredictible. Ordena abans de buscar, sempre.

📋 Arrays.copyOf: copiar amb talla nova

Crea un **nou** array amb els primers `n` elements (o tots més zeros si demanes més dels que hi ha):

```
int[] original = {1, 2, 3, 4, 5};
int[] retallat = Arrays.copyOf(original, 3); // {1, 2, 3}
int[] allargat = Arrays.copyOf(original, 8); // {1, 2, 3, 4, 5, 0, 0, 0}
```

És la forma civilitzada de "canviar la grandària" d'un array, que com saps és fixa: en crees un de nou i copies.

🧽 Arrays.fill: omplir-ho tot d'un colp

Posa el mateix valor a totes les places:

```
int[] tabla = new int[10];
Arrays.fill(tabla, 0);     // tot a zeros
Arrays.fill(tabla, 7);     // tot a sets
```

Útil per a inicialitzar taulers, reiniciar marcadors o preparar un array abans d'usar-lo.

⚖️ Arrays.equals: comparar contingut, no referències

Aquest és el que més bugs evita. `array1.equals(array2)` NO compara els elements: compara si són el MATEIX objecte en memòria. Usa SEMPRE `Arrays.equals()`:

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a.equals(b));               // false (objectes diferents en memòria)
System.out.println(Arrays.equals(a, b));    // true (mateix contingut)
```

El teu cap t'ho agrairà.

🧭 El mapa de la navalla

Mètode	Què fa	Compte amb
`Arrays.toString(arr)`	Imprimeix l'array llegible	No usar `arr.toString()`
`Arrays.sort(arr)`	Ordena al lloc	Modifica l'original
`Arrays.binarySearch(arr, v)`	Busca per índex	Requereix array ordenat
`Arrays.copyOf(arr, n)`	Nou array amb n elements	Crea còpia, no toca l'original
`Arrays.fill(arr, v)`	Ompli tot amb v	Serveix per a inicialitzar
`Arrays.equals(a, b)`	Compara contingut	No confondre amb ==
`Arrays.deepToString(arr2d)`	Imprimeix arrays 2D	Versió profunda del toString

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** `Arrays.binarySearch()` requereix l'array ORDENAT. Si no, el resultat és impredictible. És com buscar en un diccionari que no està alfabètic: no trobaràs res fiable.

Exercici: la cerca que ho tenia tot

```
import java.util.Arrays;

public class BeTheSort {
    public static void main(String[] args) {
        int[] datos = {42, 17, 8, 99, 3};
        Arrays.sort(datos);

        int indice = Arrays.binarySearch(datos, 42);
        System.out.println(indice);
    }
}
```

Què imprimeix?

- (A) 0

- (B) 3
- (C) 4
- (D) 99

> 🔄 **Solució**
>
> La **B**. Després d'ordenar, l'array és `{3, 8, 17, 42, 99}`. El 42 està a l'índex 3. Si no haguessis ordenat abans, `binarySearch` podria haver-te tornat qualsevol cosa, inclòs un negatiu fals.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `Arrays.binarySearch` si el valor no està a l'array?
2. `a.equals(b)` i `Arrays.equals(a, b)` fan el mateix?
3. Què fa `Arrays.copyOf(arr, 10)` si `arr` té 4 elements?
4. Per què `System.out.println(arr)` no imprimeix les dades?

> 🔄 **Respostes**
>
> 1. Un número negatiu (`-(puntDInsercio) - 1`). És una forma compacta de dir "no hi és, però aquí aniria".
> 2. No. `a.equals(b)` compara referències (és el mateix objecte?); `Arrays.equals(a, b)` compara el contingut element a element.
> 3. Crea un nou array de 10 places amb els 4 valors i la resta a `0`. És el truc per a "engrandir" un array.
> 4. Perquè `arr` és un objecte i el seu `toString()` heretat imprimeix l'adreça de memòria (`[I@...`). Per a veure les dades usa `Arrays.toString(arr)`.

✅ Resum en 3 frases

1. `java.util.Arrays` és la caixa d'eines **estàtica** dels arrays: s'importa i s'usa sense crear objectes.
2. Els cinc imprescindibles: `toString` (imprimir), `sort` (ordenar), `copyOf` (copiar), `binarySearch` (buscar, requereix ordre) i `fill` (omplir).

3. Per a **comparar contingut** usa `Arrays.equals`, mai `equals` ni `==`, que comparen referències.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Classe utilitària | Classe amb mètodes estàtics que no s'instancia (com `Math`) |
> | Ordre natural | L'ordre que el tipus defineix per defecte (numèric, alfabètic) |
> | Cerca binària | Cerca que parteix l'array per la meitat; requereix ordre |
> | Ordenació lexicogràfica | Ordre alfabètic segons el valor dels caràcters |
> | Còpia | Nou array independent amb els mateixos valors |

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Arrays multidimensionals · **Següent:** 05 · Arrays i mètodes

05 · Arrays i mètodes

Passar arrays a mètodes i el pas per referència: el testimoni que es comparteix, no es copia 🏃

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 05 · Arrays i mètodes

📬 La idea en una frase

> **Quan passes un array a un mètode, passes la referència, no les dades: el mètode comparteix el teu aparcament i, si toca un cotxe, el canvi es veu fora.**

Aquí se't cauran les ulleres (o se't posaran de moda). En la U09 vas veure que Java passa els arguments **per valor**. Però els arrays semblen arribar per referència… La clau: es passa per valor la **còpia de la referència**. L'array no es copia; només es copia l'adreça on és.

🏃 Passant el testimoni

Mira aquest programa i la seva sortida:

```
public class ArraysMetodos {
    public static void main(String[] args) {
        int[] edades = {10, 20, 30};
        modificar(edades);
        System.out.println(edades[0]); // 99
    }

    public static void modificar(int[] arr) {
        arr[0] = 99;
    }
}
```

Imprimeix `99`, no `10`. El mètode `modificar` ha canviat la plaça 0 de l'array `edades`… encara que l'array es va crear en `main`. Com és possible si Java passa per valor?

L'explicació: **la referència** (l'adreça de memòria on viu l'array) es copia en passar al mètode. La còpia i l'original apunten al **mateix aparcament**. Quan el mètode fa `arr[0] = 99`, no canvia la seva còpia de la referència: canvia el contingut de l'objecte al qual tots dos apunten.

> 📝 **Nota:** això NO és un pas per referència de veritat. Java mai passa la variable per referència. Passa una còpia de la referència (per això es diu *pass-by-value*). Però com l'array és un objecte, aquella còpia apunta al mateix lloc.

🆚 Primitius vs arrays: la diferència clau

Compara el comportament:

```
public class PasoDeDatos {
    public static void main(String[] args) {
        int numero = 5;
        cambiarNumero(numero);
        System.out.println(numero);   // 5 (el primitiu NO canvia)

        int[] arr = {1, 2, 3};
        cambiarArray(arr);
        System.out.println(arr[0]);   // 99 (l'array SÍ canvia)
    }

    static void cambiarNumero(int n) {
        n = 99;                       // canvia la còpia, l'original seguix en 5
    }

    static void cambiarArray(int[] a) {
          a[0] = 99;                        // canvia el contingut de l'objecte
compartit
    }
}
```

Tipus	Què rep el mètode	Es modifica fora?
`int`, `double`, `boolean`…	Una còpia del valor	No
`String`	Una còpia de la referència (String és immutable)	No (per immutable)
Array (`int[]`, `String[]`…)	Una còpia de la referència	Sí, si es modifiquen elements

Tipus	Què rep el mètode	Es modifica fora?
Objectes	Una còpia de la referència	Sí, si es modifiquen atributs

> ⚠️ **Advertència:** si dins del mètode fas `arr = otroArray`, NO canvies l'array original: només reassignes la teva còpia de la referència. Per a modificar l'original, toca elements (`arr[i] = ...`) o atributs de l'objecte, mai la variable.

🧪 Mètodes que usen arrays

Rebre un array per a calcular

```
public static double media(int[] notas) {
    int suma = 0;
    for (int n : notas) {
        suma += n;
    }
    return (double) suma / notas.length;
}
```

Modificar un array (el canvi es veu fora)

```
public static void duplicar(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2;
    }
}
```

Tornar un array nou

Els mètodes també poden **tornar** arrays. Aquí se'n crea un de nou i s'ompli:

```java
public static int[] primerosCuadrados(int n) {
    int[] resultado = new int[n];
    for (int i = 0; i < n; i++) {
        resultado[i] = (i + 1) * (i + 1);
    }
    return resultado;
}

// Al main:
int[] cuadrados = primerosCuadrados(4);
System.out.println(Arrays.toString(cuadrados)); // [1, 4, 9, 16]
```

Quan tornes un array, tornes una **referència** a un objecte del heap. Qui la rep pot usar i modificar aquell objecte. Per això, si no vols que te'l toquen, torna una còpia (`Arrays.copyOf`).

🏁 El famós `main(String[] args)`

Des de la UO2 portes escrivint `main(String[] args)` sense parar-te a pensar. És un mètode que rep un array de Strings! Aquells són els arguments de línia de comandes:

```java
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola, " + args[0]);
    }
}
```

Si l'executes amb `java Saludo Ana`, `args` serà `{"Ana"}` i imprimirà `Hola, Ana`. Si l'executes sense arguments i accedeixes a `args[0]`, `ArrayIndexOutOfBoundsException` — el mateix error del punt 1, ara amb la cara de `args`.

> 💡 **Consell:** `args.length` et diu quants arguments t'han passat. Comprova sempre abans d'accedir: `if (args.length > 0) { ... }`.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** els arrays es passen per referència (còpia de la referència). Les variables primitives es passen per valor. Si ho entens aquí, entens el 90% dels bugs rars del curs.

Exercici: l'array al quadrat

```
import java.util.Arrays;

public class BeTheArrayRevelde {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5};
        for (int i = 0; i < nums.length; i++) {
            nums[i] = nums[i] * nums[i];
        }
        System.out.println(nums[2]);
    }
}
```

Què imprimeix?

- (A) 3
- (B) 6
- (C) 9
- (D) 25

> 🔄 **Solució**
>
> La **C**. Es fa el quadrat de cada número al lloc: `{1, 4, 9, 16, 25}`. `nums[2]` = 9. No hi ha trampa de referències aquí perquè tot passa al mateix mètode, però el patró (`arr[i] = ...`) és el mateix que usaries per a modificar un array passat a un mètode.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què imprimeix aquest codi: `int[] a = {1,2,3}; cambiar(a);` on `cambiar(int[] x) { x[0] = 7; }`?
2. I si el mètode fa `x = new int[]{9,9,9}` en comptes de tocar `x[0]`?
3. `String` es comporta com un array o com un primitiu al passar-lo a un mètode?
4. Què guarda realment la variable `args` de `main`?

> 🔄 **Respostes**

1. `7`. El mètode modifica el contingut de l'objecte compartit, i el canvi es veu en `main`.
2. Res: l'array original segueix `{1, 2, 3}`. `x = new int[]{...}` només reassigna la còpia local de la referència.
3. Com un primitiu "especial": la referència es copia, però `String` és immutable, així que cap mètode no pot canviar el seu contingut. L'original mai canvia.
4. Un array de `String` amb els arguments de línia de comandes. `args[0]` és el primer, `args.length` quants n'hi ha.

✅ Resum en 3 frases

1. Els **arrays es passen per referència** (una còpia de la referència a l'objecte compartit): modificar-los dins d'un mètode es veu fora.
2. Els **primitius es passen per valor**: el mètode rep una còpia i l'original no canvia; els `String`, per ser immutables, es comporten com ells.
3. Els mètodes poden **tornar arrays** (una referència a un objecte nou) i `main(String[] args)` és, en realitat, un mètode que rep un array.

🐛 **Vocabulari ràpid**

Terme	Idea general
Pas per valor	Es copia la dada (primitius, referències)
Pas per referència	Es comparteix l'objecte (efecte visible en arrays)
Referència	Adreça de memòria on viu l'objecte
Àlies	Dues variables que apunten al mateix objecte
Arguments de `main`	L'array `args` amb els paràmetres de la línia de comandes

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · La classe Arrays: la teva navalla suïssa · **Següent:** 06 · Aplicacions dels arrays

06 · Aplicacions dels arrays

Arrays de String, de char, d'objectes i de taules: quan l'aparcament deixa de ser teoria i comença a treballar 💼

> 🗺️ **Estàs en:** 🅿️ **UO6 · Arrays** → 06 · Aplicacions dels arrays

📬 La idea en una frase

> **Els arrays no són només per a números: guardes noms, paraules, caràcters, objectes i fins i tot taules senceres. Tot el que es pugui comptar, es pot ficar en un aparcament.**

Fins ara has aparcat números. Però en la vida real les dades tenen noms: alumnes, assignatures, temperatures, pel·lícules… Aquest punt t'ensenya a guardar **coses** (no només números) en els teus arrays, i a donar-los un ús de veritat.

📝 Arrays de String: la llista de la classe

L'array més humà que existeix: una llista de noms.

```
String[] clase = {"Ana", "Bruno", "Carla", "Diego"};

for (String alumno : clase) {
    System.out.println("Hola, " + alumno);
}
```

Cada plaça guarda una `String`. Res de nou en la sintaxi: el que canvia és el que guardes. I amb ells pots fer coses pròpies del text:

```
String[] frutas = {"pera", "manzana", "pera", "uva"};

int peras = 0;
for (String fruta : frutas) {
    if (fruta.equals("pera")) {
        peras++;
    }
}
System.out.println("Hay " + peras + " peras.");
```

> ⚠️ **Advertència:** amb `String` mai no comparis amb ==. Usa `.equals()`. Ho vas veure en la UO3 i en els arrays és igual d'obligatori: `fruta == "pera"` compara referències, no text.

El més famós de tots: `args`

L'array de `String` més usat del curs el portes escrivint des de la UO2: `main(String[] args)`. En el punt 5 vas veure que conté els arguments de la línia de comandes. Doncs això: un `String[]` de veritat, com el de la teva classe.

🔤 `char[]` vs `String`

Un `char[]` és un array de caràcters. I compte: s'assembla moltíssim a una `String`… però no és el mateix.

```
char[] vocales = {'a', 'e', 'i', 'o', 'u'};

for (int i = 0; i < vocales.length; i++) {
    System.out.print(vocales[i] + " ");
}
```

Cosa	`String`	`char[]`
Immutable?	Sí: no es pot canviar	No: pots tocar cada plaça
Mètodes	`length()`, `charAt()`, `substring()`…	No en té (uses bucles)
Es pot modificar una lletra?	No (en crees una altra)	Sí: `vocales[0] = 'A';`
Passar a mètode	Es comporta com immutable	Es comparteix com qualsevol objecte

> 💡 **Consell:** si necessites "canviar una lletra", amb `String` no pots. Una opció és passar-la a `char[]`, modificar-la i tornar a construir la `String`. En la U14 (fitxers i regex) aquesta idea sortirà diverses vegades.

🧍 Arrays d'objectes: l'aparcament de persones

Aquí és on l'aparcament brilla de veritat. Els arrays no només guarden primitius o `String`: guarden **objectes** de qualsevol classe. Com encara no has creat les teves pròpies classes (això és la UO8), usa una senzilla per a l'exemple:

```
class Alumno {
    String nombre;
    int nota;

    Alumno(String nombre, int nota) {
        this.nombre = nombre;
        this.nota = nota;
    }
}
```

```
Alumno[] alumnos = new Alumno[3];
alumnos[0] = new Alumno("Ana", 8);
alumnos[1] = new Alumno("Bruno", 5);
alumnos[2] = new Alumno("Carla", 10);

int aprobados = 0;
for (Alumno a : alumnos) {
    if (a.nota >= 5) {
        aprobados++;
    }
}
System.out.println("Aprobados: " + aprobados);
```

> ⚠️ **Advertència:** un `Alumno[]` acabat de crear està ple de `null`, no d'alumnes. `new Alumno[3]` crea 3 places buides; si accedeixes a `alumnos[0].nombre` sense crear l'objecte abans, `NullPointerException` a l'acte. Primer `new Alumno(...)`, després usar.

El bucle és idèntic al dels números: canvia el contingut, no la mecànica. Això es diu **recórrer una col·lecció d'objectes**, i ho faràs moltíssim la resta del curs.

🗂️ Arrays de arrays: les dades en taula

L'últim clàssic: guardar diverses llistes a la vegada. Si tens les notes de diversos alumnes en diverses assignatures, una taula `double[][]` és la forma natural:

```
double[][] notas = {
    {8.0, 7.5, 9.0},   // Ana: Matemàtiques, Llengua, Anglés
    {5.0, 6.0, 4.5},   // Bruno
    {9.5, 8.0, 10.0},  // Carla
};

for (int i = 0; i < notas.length; i++) {
    double suma = 0;
    for (int j = 0; j < notas[i].length; j++) {
        suma += notas[i][j];
    }
    System.out.println("Alumne " + i + ": mitjana " + (suma / notas[i].length));
}
```

Cada fila és un alumne i cada columna una assignatura. Amb dos bucles niats (els del punt 3) recorres tota la taula i calcules el que necessites. Els programes que "porten el compte" de la vida real són, en el fons, això.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan un array guarda objectes, el bucle no canvia res: recorre, pregunta, acumula. L'únic que és nou és que preguntes per **atributs** (`a.nota`), no per valors solts.

Exercici: la cerca de la pel·lícula

```java
public class BeTheCatalogo {
    public static void main(String[] args) {
        String[] peliculas = {"Alien", "Matrix", "Gladiator", "Matrix", "Coco"};
        String buscada = "Matrix";
        int cuantas = 0;

        for (String p : peliculas) {
            if (p.equals(buscada)) {
                cuantas++;
            }
        }

        System.out.println(cuantas);
    }
}
```

Què imprimeix?

- (A) 1
- (B) 2
- (C) 3
- (D) Matrix

> 🔄 **Solució**
>
> La **B**. El for-each recorre les 5 pel·lícules i el `if` compta quantes vegades apareix `"Matrix"`. Apareix en les posicions 1 i 3: dues vegades. Fixa't en el `.equals()`: amb `==` compararies referències i `"Matrix"` no seria igual a cap, perquè són objectes diferents en memòria.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què hi ha a les places d'un `Alumno[]` acabat de crear?
2. Per què `fruta == "pera"` és un error amb `String`?
3. Pots modificar una lletra d'una `String`? I d'un `char[]`?
4. Quin bucle usaries per a sumar una fila d'una taula `double[][]`?

> 🔄 **Respostes**

1. `null`. `new Alumno[3]` crea 3 places buides; cal crear els objectes amb `new Alumno(...)`.
2. Perquè == compara referències (és el mateix objecte?), no el contingut. Amb `String` cal usar `.equals()`.
3. No, `String` és immutable. Sí, `char[]` no ho és: `vocales[0] = 'A'` funciona.
4. Un `for` niat, o un `for` per la fila concreta: `for (int j = 0; j < notas[i].length; j++)`.

✅ Resum en 3 frases

1. Els arrays guarden **el que sigui**: `String`, `char`, objectes propis i taules senceres, amb la mateixa mecànica de sempre.
2. Amb `String` usa **`.equals()`** i recorda que un array d'objectes acabat de crear està ple de **`null`**.
3. Els **arrays d'objectes** (recorres atributs) i els **arrays 2D** (recorres files i columnes) són la base dels programes que gestionen dades de la vida real.

🐛 **Vocabulari ràpid**

Terme	Idea general
Array d'objectes	Aparcament on cada plaça guarda un objecte (`Alumno`, `String`…)
`null`	El valor per defecte de les places d'objectes acabades de crear
Immutable	Que no es pot canviar; la `String` ho és
`char[]`	Array de caràcters, modificable, sense mètodes propis
Taula de dades	`double[][]`: files i columnes per a dades reals

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Arrays i mètodes · **Següent:** 07 · Be the Code: l'aparcament es gestiona

07 · Be the Code: l'aparcament es gestiona

Inverteix, busca i compacta arrays a mà, sense ajuda de la classe Arrays, com en una entrevista de treball 🕶️

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 07 · Be the Code: l'aparcament es gestiona

---

📬 La idea en una frase

> **Aquest punt no té teoria nova: té tres reptes. Invertir un array sense mirar apunts, buscar totes les posicions d'un valor i caçar els errors d'un array que es rebel·la.**

La classe `Arrays` t'ho dona tot fet: ordena, copia, busca… Però quan et demanen "inverteix aquest array" en una entrevista, ningú et deixa usar `Arrays.sort()`. Cal saber-ho fer a mà. Aquest punt és el teu gimnàs.

🕶️ Don Tip: la recepta mental de l'invers al lloc

Invertir un array és el clàssic de les entrevistes. La recepta mental:

1. Dos punters: `izquierda = 0` i `derecha = array.length - 1`.
2. Mentre `izquierda < derecha`: intercanvia `array[izquierda]` i `array[derecha]`.
3. `izquierda++` i `derecha--`.
4. Quan es creuen, ja està.

L'error més comú: crear un **array auxiliar** quan no cal. Al lloc, amb dos punters i una variable temporal, és O(n) de temps i O(1) de memòria. Això és el que impressiona en una entrevista.

🧩 REPTE 1: Invertir l'array al lloc

Completa el mètode `invertir` perquè doni la volta a l'array **sense crear un altre array**. Ha de mostrar `5 4 3 2 1`:

```java
public class RetoInverso {

    public static void invertir(int[] array) {
        // 🧠 EL TEU CODI ACÍ
    }

    public static void main(String[] args) {
        int[] datos = {1, 2, 3, 4, 5};
        invertir(datos);
        for (int n : datos) {
            System.out.print(n + " ");
        }
    }
}
```

Passos guiats (resisteix a llegir-los tots de colp):

1. Declara els dos punters.
   > **Atascat?**
   > `int izquierda = 0;` i `int derecha = array.length - 1;`
2. Escriu el `while` amb la condició correcta.
   > **Atascat?**
   > `while (izquierda < derecha) { … }` — quan es creuen, ja està.
3. Intercanvia els dos elements amb una variable temporal.
   > **Atascat?**
   > `int temp = array[izquierda]; array[izquierda] = array[derecha]; array[derecha] = temp;`
4. Mou els punters cap al centre.
   > **Atascat?**
   > `izquierda++; derecha--;`

> 🔄 **Solució completa**

```java
public static void invertir(int[] array) {
    int izquierda = 0;
    int derecha = array.length - 1;

    while (izquierda < derecha) {
        int temp = array[izquierda];
        array[izquierda] = array[derecha];
        array[derecha] = temp;
        izquierda++;
        derecha--;
    }
}
```

🧩 REPTE 2: Buscar totes les posicions

`binarySearch` et dona una posició. Aquest repte et demana **totes**. Escriu un mètode que torni un array amb tots els índexs on apareix un valor (buit si no apareix):

```java
public class RetoBusqueda {

    public static int[] posiciones(int[] datos, int buscado) {
        // 🧠 EL TEU CODI ACÍ
        return new int[0];
    }

    public static void main(String[] args) {
        int[] datos = {3, 7, 2, 7, 9, 7, 1};
        System.out.println(java.util.Arrays.toString(posiciones(datos, 7)));
        System.out.println(java.util.Arrays.toString(posiciones(datos, 5)));
    }
}
```

Ha de mostrar `[1, 3, 5]` i `[]`.

Passos guiats:

1. Primera passada: compta quantes vegades apareix.

   > **Atascat?**
   > Un `for` amb un comptador que pugi quan `datos[i] == buscado`.

2. Crea l'array de resultats amb aquella grandària.

> **Atascat?**
>
> `int[] resultado = new int[cuantas];`

3. Segona passada: ompli les posicions.

> **Atascat?**
>
> Un segon `for` que, quan trobi el valor, faci `resultado[k++] = i;`

> 💡 **Consell de depuració:** aquest "comptar primer, crear després" és un patró que es repeteix: no pots crear l'array de resultats fins a saber quantes places necessita. Quan la grandària depèn de les dades, es fa en dues passades.

> **🔄 Solució completa**

```
public static int[] posiciones(int[] datos, int buscado) {
    int cuantas = 0;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] == buscado) {
            cuantas++;
        }
    }

    int[] resultado = new int[cuantas];
    int k = 0;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] == buscado) {
            resultado[k++] = i;
        }
    }
    return resultado;
}
```

🧩 EL LÍO: l'aparcament que es va rebel·lar

L'encarregat de l'aparcament ha escrit això per a "deixar les places imparelles buides". Alguna cosa fa mala olor. Troba els errors:

```java
public class ParkingLioso {
    public static void vaciarImpares(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 1) {
                arr[i] = null;   // 🚨 compila això?
            }
        }
    }
}
```

🕶️ **Don Tip:** pregunta sempre: quin tipus de dada guarda aquest array? Un `int` no pot valer `null`; un `Integer` sí (però això és un objecte de la U12).

› 🔄 **Solució**

No compila. `arr` és un `int[]`, i `int` és un tipus primitiu: **no pot valer `null`**. `null` només cap en variables de tipus objecte (`String`, `Integer`, `Alumno`…).

Si volguessis "buidar" un `int[]`, hauríes de posar un valor de sentinella, per exemple `0` o `-1`. I si de veritat necessites places buides de veritat, hauríes de usar un array d'objectes (`Integer[]`), que veuràs en la U12 amb les col·leccions.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina condició usa el `while` de l'invers al lloc: < o <=?
2. Què passa si uses <= en l'invers amb un array de longitud senar?
3. Per què el patró "comptar primer, crear després" necessita dues passades?
4. Pot un `int[]` guardar `null`? I un `String[]`?

› 🔄 **Respostes**

1. `izquierda < derecha`. Amb <= et sobraria el pas en què tots dos punters apunten al mateix element (amb longitud senar), que a més intercanviaria un element amb si mateix.
2. Amb longitud senar, a la volta central `izquierda == derecha`: intercanvies l'element amb si mateix (inútil però inofensiu). Amb < ni tan sols entres aquí.

3. Perquè no saps quantes places necessita l'array de resultats fins a comptar les coincidències. Els arrays tenen grandària fixa: cal saber-la abans de crear-los.
4. No: `int` és primitiu i no accepta `null`. Sí: `String` és un objecte i el seu valor per defecte és `null`.

✅ Resum en 3 frases

1. **Invertir al lloc** és el clàssic de les entrevistes: dos punters (`izquierda`/`derecha`), intercanvi amb `temp`, i `while (izquierda < derecha)`.
2. Quan la grandària del resultat **depèn de les dades**, es fa servir el patró de **dues passades**: comptar primer, crear i omplir després.
3. `null` no cap en un array de **primitius**: només en arrays d'objectes. Amb `int[]` s'usen valors de sentinella.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Invertir al lloc	Donar la volta a l'array sense crear-ne un altre, amb dos punters
Variable temporal	El `temp` que guarda un valor durant l'intercanvi
Dues passades	Comptar primer i crear/omplir després (grandària dependent de les dades)
Valor de sentinella	Un valor especial (`-1`, `0`) que significa "buit" en primitius
Punter	Índex que delimita una zona (`izquierda`, `derecha`)

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Aplicacions dels arrays · **Següent:** 08 · Array-revelde: errors comuns i depuració

08 · Array-revelde: errors comuns i depuració

La galeria de monstres dels arrays: excepcions, null, comparacions que no comparen i com caçar-los amb calma 🐛

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 08 · Array-revelde: errors comuns i depuració

📬 La idea en una frase

> **Els arrays no es rebel·len per maldat: es rebel·len perquè oblides els límits. Conèixer els 6 monstres típics és la mitat de la batalla; saber depurar és l'altra mitat.**

A aquestes alçades ja has escrit els teus primers arrays. I, si ets humà, ja t'has estavellat una vegada (o dos, o cinquanta). Aquest punt recopila els errors més comuns de l'univers array perquè els reconeguis a l'instant i deixes de plorar sobre el teclat.

👹 La galeria de monstres

Monstre 1: la `ArrayIndexOutOfBoundsException`

El rei del mambo. Aquelles de les places vàlides (`0` a `length - 1`).

```
int[] a = {10, 20, 30};
System.out.println(a[3]); // Index 3 out of bounds for length 3
```

> 🔍 **Diagnòstic:** el missatge et diu l'índex i la longitud. Si demanes `a[3]` i hi ha 3 places (0, 1, 2), el problema és el `<=` del teu bucle o un índex calculat malament. Llegeix el missatge: no és un misteri, és una pista.

Monstre 2: la `NullPointerException`

Recorres un array d'objectes acabat de crear i toques una plaça sense objecte.

```
String[] nombres = new String[3];
System.out.println(nombres[0].toUpperCase()); // BOOM: null no té mètodes
```

> 🔍 **Diagnòstic:** les places d'un `String[]` nou estan plenes de `null`. Pregunta abans: `if (nombres[i] != null) { ... }` o crea els objectes al principi.

Monstre 3: imprimir sense `Arrays.toString`

```
int[] a = {1, 2, 3};
System.out.println(a); // [I@6d06d69c
```

> 🔍 **Diagnòstic:** imprimeix l'adreça de memòria, no les dades. Sempre `Arrays.toString(a)` (o `Arrays.deepToString` per a 2D). Ho vas veure en el punt 4 i no et cansaràs de veure-ho.

Monstre 4: comparar amb ==

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b);           // false: són el MATEIX objecte?
System.out.println(a.equals(b));              //  false  també:  els  arrays  no
sobreescriuen equals
System.out.println(Arrays.equals(a, b)); // true: això compara contingut
```

> 🔍 **Diagnòstic:** `==` i `equals` comparen **referències**. Per a comparar contingut: `Arrays.equals(a, b)`.

Monstre 5: confondre `length`, `length()` i `size()`

```
int[] a = {1, 2, 3};
String s = "Hola";
System.out.println(a.length());  // no compila: array usa length (atribut)
System.out.println(s.length);    // no compila: String usa length() (mètode)
```

> 🔍 **Diagnòstic:** la regla d'or: **array → `length`** (sense parèntesis), **`String` → `length()`**, **col·lecció → `size()`**. Els exàmens adoren aquesta trampa.

Monstre 6: intentar modificar amb `for-each`

```java
int[] a = {1, 2, 3};
for (int n : a) {
    n = n * 10;   // pugen els valors? NO: canvia una còpia
}
System.out.println(Arrays.toString(a)); // [1, 2, 3]
```

> 🔍 **Diagnòstic:** el `for-each` és de només lectura. La variable `n` és una còpia del valor de la plaça. Per a modificar, `for` amb índex: `a[i] = a[i] * 10;`.

😡 CONRAD VS EL MÓN: “L’array que no es calla”

> *CONRAD, el nostre compilador rondinaire, té la tassa plena i els arrays trencats sobre la taula.*

CONRAD: — ¡ALTRES VEGADES! Em porten un programa i em diuen: *CONRAD, em dona ArrayIndexOutOfBoundsException*. I jo: i tu què has fet? *Doncs recórrer l’array.* AMB QUÈ? *Doncs amb un for… crec.* ¡AI, MARE MEVA! Un for amb quina condició? *Doncs* `i <= numeros.length`*…* ¡¿<=?! ¡PERÒ SI T’HO VAIG DIR EN EL PUNT 2! L’últim índex vàlid és `length - 1`. <= et porta a la plaça fantasma i d’allà no torna ningú.

I després estan els que pregunten per què els surt null. Pregunta: què tens a la plaça? No ho sé, la vaig crear amb `new String[10]`. *I quantes places has omplit?* Doncs… bé… ¡AH! ¡CAP! Un `String[]` acabat de crear és una fila de `null`. Si no fiquis objectes, no hi ha objectes. És tan difícil?

I el favorit de tots: no em funciona el bucle que duplica. *Ensenya-me’l.* És un `for-each` que fa `n = n * 2`. *I esperaves canviar l’array?* Doncs sí. ¡EL `for-each` NO MODIFICA! És com un robot lector de matrícules: llegeix, però no repinta. Per a repintar, índex i claudàtors.

La lliçó: els sis monstres tenen nom, missatge i cura. El 90% dels “arrays rebel·les” s’arreglen mirant el missatge d’error amb calma i recordant les regles d’or: límits (`0` a `length-1`), `null` en objectes, `Arrays.toString` per a imprimir, `Arrays.equals` per a comparar, `length` sense parèntesis i `for` amb índex per a modificar.

🐛 Depurar arrays sense perdre el cap

Quan un array es porta malament, segueix aquest ordre:

1. **Imprimeix l'array complet** amb `Arrays.toString()` a cada pas. Veure les dades a la vista arregla la mitat dels misteris.
2. **Comprova els límits abans de tocar.** Si vas a accedir a `a[i + 1]`, que `i` no arribi a `length - 1`. Pregunta sempre: pot sortir-se?
3. **Usa el depurador de l'IDE.** Posa un *breakpoint* al bucle i observa `i` a cada vegada. Si es passa de `length - 1`, ho veus a la primera.
4. **Paper i boli.** Amb un array de 3 elements i un objectiu que no hi sigui, simula el bucle a mà. Sí, en ple segle XXI, i segueix funcionant.
5. **Aïlla l'error.** Divideix el problema: primer ompli i comprova que el contingut és correcte; després el bucle; després el càlcul. Si alguna cosa falla, sabràs quin tram és.

> 💡 **Consell:** el missatge d'excepció de Java no és el teu enemic: és un detectiu que et diu la línia i el motiu. `Index 5 out of bounds for length 5` no deixa lloc a dubtes. LLEGEIX-LO abans de tocar res.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan alguna cosa falli, pregunta abans "què esperava?" i després "què he escrit?". El 90% de les vegades la diferència entre les dues respostes és un <=, un `null` o un ==.

Exercici: el caçador de monstres

```
public class CazaMonstruos {
    public static void main(String[] args) {
        int[] datos = new int[5];
        for (int i = 1; i <= datos.length; i++) {
            datos[i] = i * 10;
        }
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Què ocorre?

- (A) Imprimeix `[10, 20, 30, 40, 50]`
- (B) Imprimeix `[0, 10, 20, 30, 40]`

- (C) `ArrayIndexOutOfBoundsException`
- (D) `NullPointerException`

> 🔄 **Solució**
>
> La **C**. El bucle va de `i = 1` a `i = 5` inclòs (<=). Quan `i` val 5, fa `datos[5] = 50`, però les places vàlides van de 0 a 4. Monstre 1 en acció. I de passada, la plaça 0 es queda sense tocar (a `0`), perquè el bucle comença en 1: un altre ensurt per als despistats.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina excepció llanços en accedir a `a[a.length]`?
2. Què hi ha en `String[] s = new String[3]` a cada plaça?
3. Com compares dos arrays per a saber si tenen el mateix contingut?
4. Amb quin bucle pots modificar els elements d'un array?

> 🔄 **Respostes**
>
> 1. `ArrayIndexOutOfBoundsException`: l'índex `length` no existeix, les places acaben en `length - 1`.
> 2. `null` a les tres. Les places d'objectes acabades de crear estan buides fins que les omplis amb `new`.
> 3. Amb `Arrays.equals(a, b)`. `==` i `a.equals(b)` comparen referències, no contingut.
> 4. Amb el `for` clàssic amb índex (`a[i] = ...`). El `for-each` és de només lectura.

✅ Resum en 3 frases

1. Els **sis monstres** dels arrays són: índex fora de rang, `null`, imprimir sense `toString`, comparar amb `==`, confondre `length`/`length()`/`size()` i modificar amb `for-each`.
2. El **missatge de l'excepció és una pista**: et diu la línia i el motiu. Llegeix-lo abans de tocar res.
3. Depurar un array és **veure les dades** (`Arrays.toString`), **respectar els límits** i aïllar el tram que falla, amb boli o amb *breakpoint*.

🐛 Vocabulari ràpid

Terme	Idea general
Monstre	Error típic amb nom i cura (`ArrayIndexOutOfBounds`, `NullPointer`…)
`Arrays.toString`	La forma d'imprimir un array llegible
`Arrays.equals`	Comparar contingut de dos arrays
Valor de sentinella	Valor especial que representa "buit" en primitius
Breakpoint	Punt de parada del depurador per a inspeccionar variables

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Be the Code: l'aparcament es gestiona · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu: l'aparcament a examen

El tancament pràctic de la unitat, amb recorreguts, cerques i un array que no funcionarà 😈

> 🗺️ **Estàs en:** 🅿️ **U06 · Arrays** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend...

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
public class Misterio {
    public static void main(String[] args) {
        int[] datos = {3, 1, 4, 1, 5, 9};
        int total = 0;

        for (int i = 0; i < datos.length; i += 2) {
            total += datos[i];
        }

        System.out.println(total);
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **23** → Sumes tots els elements, però el bucle va de dos en dos. ❌
2. **12** → ✅ Correcte! El bucle salta de 2 en 2: `i = 0, 2, 4`. Suma `datos[0] + datos[2] + datos[4]` = `3 + 4 + 5` = **12**. La plaça 1 (l'1) i la 3 (l'1) es queden sense visitar.
3. **13** → Sumes els índexs en lloc dels valors. ❌
4. **9** → Només et quedes amb `datos[4]`, l'últim del salt. ❌

> 🔄 Solució

L'opció **2**. `i += 2` recorre les posicions parells 0, 2 i 4. Els seus valors són 3, 4 i 5. `3 + 4 + 5 = 12`. El `for` amb índex et dona el control del pas: una cosa que el `for-each` no pot fer.

🔥 Fireside Chat: for clàssic vs for-each

> *Dos veterans del recorregut discuteixen al costat de la màquina de cafè.*

For-each: — Jo soc el modern. "Per a cada X en Y": curt, net, impossible equivocar-se amb l'índex. Qui vol comptar places a mà?

For clàssic: — I quan em necessites a mi? Quan cal **modificar**, quan cal **saber la posició**, quan cal anar **cap enrere** o **de dos en dos**. Aquelles quatre coses són el 80% dels exercicis del curs, i les faig totes. Tu, en canvi, llegeixes i et calles.

For-each: — Llegir és el que es fa quasi sempre: sumar, comptar, imprimir. El 80% de les vegades no et necessito, i si t'uso sense necessitar-te, arriben els errors: el <=, el començar en 1, l'oblidar el ++...

For clàssic: — Això és perquè no m'uses bé. Amb mi tens el control total. Tu ets... la variant peresosa.

For-each: — La variant **segura**. I no em digues peresosa: digues-me "sense índex, sense excuses".

For clàssic: — Val, treva. Tu per a llegir sense posicions; jo per a tota la resta. Tracte?

For-each: — Tracte. Però que sàpigues que en el bucle de la mitjana et guanyi fins i tot amb els ulls tancats.

> La lliçó: **for-each** si només llegeixes i no t'importa la posició; **for amb índex** si modifiques, busques posicions o recorres de forma especial. El context decideix, no la moda.

🕵️ Qui soc?

Endevina quin concepte de la unitat soc:

1. **Soc un aparcament de grandària fixa: guardo moltes dades del mateix tipus sota un sol nom.**

2. **Soc el número de cada plaça, i comenci en 0, per a confusió general.**
3. **Soc l'excepció favorita del novell: em llancen quan demanen la plaça que no existeix.**
4. **Soc el bucle de només lectura: "per a cada X en Y", sense índex i sense presses.**
5. **Soc la classe utilitària amb `toString`, `sort`, `binarySearch`, `copyOf` i `fill`.**
6. **Soc l'atribut que diu quantes places hi ha, sense parèntesis, per a no confondre'm amb els `String`.**

> 🔄 **Respostes**
>
> 1. **L'array** — aparcament de dades del mateix tipus, grandària fixa.
> 2. **L'índex** — número de la plaça; les vàlides van de 0 a `length - 1`.
> 3. **La `ArrayIndexOutOfBoundsException`** — en demanar la plaça `length` o més.
> 4. **El `for-each`** — només llegeix, no modifica.
> 5. **La classe `Arrays`** — la navalla suïssa dels arrays.
> 6. **`length`** — atribut de l'array, sense parèntesis (els `String` usen `length()`).

🤬 CONRAD VS EL MÓN: "L'array que no para"

> *CONRAD, el nostre compilador rondinaire, ha trobat una nota en la safata d'errors.*

CONRAD: — ¡ALTRES VEGADES! Un alumne em diu: *CONRAD, el meu bucle surt de l'array*. I jo: amb quina condició l'has escrit? *Doncs…* `i <= notas.length`. ¡¡PERÒ SI AIXÒ ÉS EL PRIMER QUE S'APRÈN EN AQUESTA UNITAT!! L'últim índex és `length - 1`. <= vol dir que demanaràs la plaça `length`, que no existeix. És com cridar a la planta 5 d'un aparcament de 5 plantes (que van de la 0 a la 4): no hi és, i el porter et llança la `ArrayIndexOutOfBoundsException`.

I després el clàssic del for-each: el meu array no es modifica. I com el recorres? *Amb for-each.* ¡CLAR! El for-each és un robot que LIG les matrícules. No repinta cotxes. Si vols canviar valors, índex i claudàtors: `notas[i] = notas[i] + 1;`. Quantes vegades ho he de dir?

I el dels String: comparo dos noms i no em dona igual. Amb què? *Amb ==.* ¿QUANTES VEGADES? Amb `String` s'usa `.equals()`. == compara referències: són el MATEIX objecte? Encara que tinguin el mateix text, si són dos objectes diferents, == diu false. ¡LLEGEIX-T'HO EN EL PUNT 6, PER FAVOR!

La lliçó: els tres mals del novell amb arrays tenen nom: **<= fora de rang**, **for-each que no modifica** i **== que no compara text**. Abans de plorar sobre el teclat, comprova aquelles tres

coses. El 90% dels “arrays rebel·les” s’arreglen amb una ullada.

🎮 El joc de les decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Quin és l’últim índex vàlid de `int[] a = new int[8]`?
   - a) 7 b) 8 c) 9
2. Quin valor té cada plaça de `boolean[] b = new boolean[4]`?
   - a) `true` b) `false` c) `null`
3. Què fa `Arrays.binarySearch` sobre un array **desordenat**?
   - a) Llança una excepció b) Torna un resultat impredictible c) Ordena primer
4. Què imprimeix `System.out.println(a.length);` per a `int[] a = new int[10]`?
   - a) `9` b) `10` c) `[10]`
5. Quin bucle uses per a recórrer un array **cap enrere**?
   - a) `for-each` b) `for` amb índex c) qualsevol

> 🔄 **Solucions**
>
> 1. **a)** — 7. Les places van de 0 a `length - 1`, és a dir, de 0 a 7.
> 2. **b)** — `false`. És el valor per defecte de `boolean`.
> 3. **b)** — Un resultat impredictible, sense avisar. Exigeix array ordenat, sempre.
> 4. **b)** — `10`. `length` és el nombre de places, sense parèntesis.
> 5. **b)** — `for (int i = a.length - 1; i >= 0; i--)`. El `for-each` només avança de principi a fi.

⚡ Laboratori de tortura: la mitjana que es resisteix

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un arxiu nou

L’escenari: copia aquest programa i fes que funcioni. Té **3 errors** que impedeixen que compili o que falli en execució, i 1 error de lògica que fa que el resultat sigui incorrecte quan els arregles.

```java
public class Tortura
    public static void main(String[] args) {
        int[] notas = {8, 7, 9, 6};
        int suma = 0;

        for (int i = 0; i <= notas.length; i++) {
            suma += notas[i];
        }

        double media = suma / notas.length;
        System.out.println("Media: " + media)
    }
}
```

Falta intencionada: un dels errors sembla correcte a simple vista perquè "es veu bé", però fa que el programa falli en execució en el moment que arranca.

La teva tasca: aconseguir que compili, que executi i que imprimeixi `Media: 7.5`. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun `;` que falti? *no → segueix buscant.*
   > **I si segueixo atascat?**
   > Comprova també les claus `{}`: la classe necessita la seva apertura, i l'últim `System.out.println` necessita el seu `;`.

2. Compila ja? *no → mira el missatge d'error.*
   > **I si segueixo atascat?**
   > La classe `Tortura` necessita `{` d'apertura.

3. Compila però explota en executar? *És l'error d'índexs: el bucle arriba massa lluny.*
   > **I si segueixo atascat?**
   > `i <= notas.length` accedeix a `notas[4]`, que no existeix (les vàlides són 0-3). Ha de ser `i < notas.length`.

4. Executa i imprimeix `Media: 7.0`? *És l'error de lògica: la divisió entera.*
   > **Solució final**

Els **3 errors** que impedeixen compilar o executar:

1. Falta la `{` d'apertura de la classe després de `Tortura`.
2. Falta el `;` al final de `System.out.println("Media: " + media)`.
3. `i <= notas.length` accedeix a `notas[4]` fora de l'array: `ArrayIndexOutOfBoundsException`. Ha de ser `i < notas.length`.

L'**error de lògica**: `suma / notas.length` divideix enters. `30 / 4 = 7` (es trunca), no `7.5`. Perquè la mitjana surti amb decimals cal convertir abans: `(double) suma / notas.length`.

```
public class Tortura {
    public static void main(String[] args) {
        int[] notas = {8, 7, 9, 6};
        int suma = 0;

        for (int i = 0; i < notas.length; i++) {
            suma += notas[i];
        }

        double media = (double) suma / notas.length;
        System.out.println("Media: " + media);
    }
}
```

Sortida correcta: `Media: 7.5`. Amb la versió trencada, una vegada arreglats els altres errors, la divisió entera donava `7.0`: la pista de l'error de lògica.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **L'Aparcador Cap**	Crear un array, omplir-lo i recórrer-lo sense mirar apunts
🏅 **El Comptador de Places**	Explicar la diferència entre `length`, `length()` i `size()`
🏅 **El Caçador de Monstres**	Trobar els 3+1 errors del Laboratori sense usar les pistes
🏅 **L'Inversor**	Invertir un array al lloc amb dos punters, a la primera

Assoliment	Com aconseguir-lo
🏅 El Navaller	Usar `toString`, `sort`, `binarySearch`, `copyOf` i `fill` en un programa
🏅 El Profe de la Classe	Aprovar amb un array d'objectes: omplir, recórrer i calcular la mitjana

🧠 Atreveix-te a pensar

1. **Sense executar:** què imprimeix aquest programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        int[] datos = {4, 2, 8, 1, 6};
        int mayor = datos[0];

        for (int i = 1; i < datos.length; i++) {
            if (datos[i] > mayor) {
                mayor = datos[i];
            }
        }
        System.out.println(mayor);
    }
}
```

2. **El truc de l'índex:** què passa si en el bucle de la mitjana de `{8, 7, 9, 6}` uses `i <= notas.length - 1` en lloc de `i < notas.length`? Funciona? Per què?
3. **El doble:** `int[] a = {1, 2, 3}; int[] b = a; b[0] = 99;` quant val `a[0]` després? (Pista: és el punt 5.)
4. **Vertader o fals:** "`Arrays.sort` modifica l'array original, així que convé copiar-lo abans si no vols perdre l'ordre inicial".

> 💡 **Solucions**
>
> 1. Imprimeix **8**. El patró del màxim: comença amb `datos[0]` (4) i va comparant; quan arriba al 8, el guarda; l'1 i el 6 no li guanyen.

2. **Funciona.** `i <= notas.length - 1` és exactament el mateix que `i < notas.length`: en tots dos casos l'últim valor de `i` és `length - 1`. Són dues formes d'escriure el mateix, però `i < notas.length` és la que no convida a l'error.
3. **99.** `b = a` no copia l'array: copia la referència. `a` i `b` apunten al mateix aparcament, així que tocar per `b` es veu per `a`. Per a copiar de veritat, `Arrays.copyOf`.
4. **Vertader.** `Arrays.sort` ordena "al lloc" (modifica l'original). Si necessites conservar l'ordre inicial, copia abans amb `Arrays.copyOf`.

🧩 Crucigrama de bits

```
Horitzontal:
1. Grandària d'un array, sense parèntesis (6 lletres)
4. Mètode de la classe Arrays per a comparar contingut (5 lletres)

Vertical:
2. Valor per defecte d'un boolean en un array (5 lletres)
3. Bucle de només lectura (3 lletres + 4 lletres)
5. L'excepció que llanços en eixir-te de l'array (una sola paraula molt llarga)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. LENGTH · 4. EQUALS **Vertical:** 2. FALSE · 3. FOR EACH · 5. ARRAYINDEXOUTOFBOUNDSEXCEPTION

💬 Preguntes d'entrevista de treball

Preguntes reals que et farien per a programador Java júnior.

1. **"Explica-m'ho, com si jo fos la teva àvia, què és un array."**
2. **"Inverteix aquest array sense crear-ne un altre. Ara digues-me quanta memòria extra necessites."**
3. **"Quina és la diferència entre `length`, `length()` i `size()`?"**
4. **"Quan usaries `for-each` i quan un `for` amb índex?"**
5. **"Com compares dos arrays per a saber si tenen el mateix contingut?"**
6. **"Escriu el codi que torna la nota més alta d'un array."**

🤷 No hi ha preguntes tontes

> ❓ **Per què el primer índex és 0 i no 1?**

Perquè l'índex és una **distància** des del principi, no un número de plaça. La primera casa és a 0 passes de tu, no a 1. En programació, comptar des de 0 evita l'off-by-one en milers de càlculs (i és una convenció heretada dels llenguatges més antics). Et semblarà rar fins que deixi de semblar-t'ho, i aleshores ho defensaràs amb ungles i dents.

> ❓ **Puc usar `Arrays.sort` i `Arrays.binarySearch` en lloc d'escriure els algoritmes a mà?**

En els teus programes reals, sí: són ràpids i provats, i els veuràs per tot arreu. Però en la U07 aprendràs **com funcionen per dins** (bombolla, cerca binària…) perquè entendre la idea és el que et diferencia d'algú que només importa llibreries. I en una entrevista, et demanaran l'algoritme a mà. Primer s'aprèn a sumar sense calculadora, no?

> ❓ **Un array pot canviar de grandària?**

No. És **grandària fixa** per sempre. Quan necessites "més places", es crea un array nou i es copia (`Arrays.copyOf`). Si això et sembla un incordi, tens raó: per això existeixen les col·leccions (`ArrayList` i companyia), que creixen soles. Les veuràs en la U12, i les abraçaràs.

🎬 Postcrèdits

El programador tanca la seva llista de la compra: un array de 12 `String`, recorregut amb un for-each, ordenat amb `Arrays.sort` i imprés amb `Arrays.toString`. Tot funciona a la primera. S'acosta CONRAD, el compilador rondinaire, amb la seva tassa humejant.

CONRAD: — Escolta, no està mal. Crees, recorres, ordenes, busques… i a més no has sortit de l'array ni una sola vegada. Segur que ets el mateix que va començar la unitat amb `gato1`, `gato2`, `gato3`?

Novell: — *somriu* Vaig començar amb 100 gats i he acabat gestionant l'aparcament sencer amb 20 línies. I ara què?

CONRAD: — *fa un glop* Ara el que és interessant. Totes aquestes dades que guardes en arrays demanen a crits que els **busques** i els **ordenes** bé. Bombolla, cerca binària, Big O… Un altre tipus de bèsties, i van just després.

El novell guarda el seu projecte, tanca l'IDE i sent que ja no guarda dades soltes: **pensa en aparcaments**.

PRÒXIMAMENT EN LA U07: Algorísmica I: Fonaments. Els algoritmes que busquen i ordenen els teus arrays, i la notació per a saber quin és ràpid de solemnitat. 🧩

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Array-revelde: errors comuns i depuració · **Següent:** U07 · Algorítmica I: fonaments

U07 — Algorítmica I: fonaments

Receptes, diccionaris, cartes i bombolles: els algoritmes que ordenen i busquen per tu 🧩

Benvingut a la cuina del programador. En la U04 els teus programes van aprendre a **decidir** i a **repetir**, però repetir per repetir és fer footing sense rumb: corres, però no arribes enlloc. Aquesta és la unitat on el rumb apareix: aprendràs els **algoritmes clàssics** que ordenen i busquen dades, i l'eina per a saber si un és ràpid o lent de solemnitat: la **notació Big O**.

Pensa en el que fas cada dia sense adonar-te'n. Busques la paraula "berenar" en el diccionari (i no obres per la pàgina 1, sinó per la meitat). Ordenes les cartes de la mà per al pòquer. Busques les teves sabatilles perdudes revisant l'habitació. Doncs tot això, portat a Java, és aquesta unitat: **cerca lineal**, **cerca binària**, **ordenació bombolla**, **ordenació per inserció** i l'art de triar quin usar.

I, com sempre, CONRAD el compilador serà per aquí per a recordar-te que un algoritme mal pensat és com una truita sense sal: es pot menjar, però no hauries.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar **què és un algoritme**, les seves propietats i la diferència entre la *idea* i el *codi*.
- Buscar elements en un array amb **cerca lineal** (O(n)) i saber quan convé.
- Buscar en arrays **ordenats** amb **cerca binària** (O(log n)), sense morir en l'intent (ni en l'off-by-one).
- Ordenar arrays amb **bombolla** (el patós) i amb **inserció** (el de les cartes), i saber els seus límits.
- Analitzar la **complexitat algorísmica** amb notació Big O i simplificar amb les regles d'or.
- **Triar l'algoritme adequat** segons la mida, l'ordre inicial i la situació.
- Escriure cerca binària i bombolla **a mà**, sense mirar apunts (sí, com en les entrevistes de treball).

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Què és un algoritme	Definir algoritme, les seves propietats i la recepta de la truita	Tots
02 · Cerca lineal	Recórrer l'array buscant un element, O(n)	Tots
03 · Cerca binària	Dividir i véncer: descartar mitats, O(log n)	Tots
04 · Ordenació bombolla	Intercanviar bombolles fins que tot sura, $O(n^2)$	Tots
05 · Ordenació per inserció	Ordenar com cartes a la mà, $O(n^2)$ i bèstia amb quasi ordenats	Tots
06 · Complexitat algorísmica: Big O	Com creix el temps amb les dades, d'O(1) a $O(2^n)$	Tots
07 · Triar l'algoritme adequat	La taula de decisió segons mida i ordre inicial	Tots
08 · Be the Code: cerca binària des de zero	Programar binària i bombolla sense xarxa de seguretat	Tots
09 · Repàs interactiu	Be the Code, Fireside, Qui Soc, Laboratori, Crucigrama…	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA2 + RA6)

RA2: Escriu i prova programes senzills, reconeixent i aplicant els fonaments de la programació orientada a objectes.

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA2 b)	S'han escrit programes simples.	✅ Tots
RA2 f)	S'han utilitzat paràmetres en la crida a mètodes.	✅ Punts 2, 3, 4 i 5
RA6 a)	S'han escrit programes que utilitzen arrays.	✅ Tots

> 📌 Aquesta unitat és el germen de la U08: els bucles que domines aquí (`for`, `while`), les cerques i les ordenacions es converteixen en QuickSort, MergeSort i recursivitat. I els arrays que recorres avui són la base de les col·leccions de la U12.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites variables, bucles i arrays bàsics de la U04.
- Ja saps què és un algoritme i vols molta cosa? → Salta a la cerca binària o a l'ordenació per inserció.
- Només vols saber quin algoritme triar? → Llegeix Big O i Triar l'algoritme adequat.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Què és un algoritme
⏭️ **En acabar la unitat, continua en U08 · Algorítmica II: tècniques avançades i modularitat.**

01 — Què és un algoritme

La recepta de la truita que el teu ordinador pot seguir al peu de la lletra 🍳

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 01 · Què és un algoritme

---

📬 La idea en una frase

> **Un algoritme és una recepta de cuina per al teu ordinador: una seqüència finita, ordenada i sense ambigüitats de passos que resol un problema.**

Quan cuines una truita de creïlles, segueixes un algoritme mental:

1. Pelar les creïlles.
2. Tallar-les a rodanxes fines.
3. Fregir-les amb oli abundant.
4. Batre els ous.
5. Barrejar-ho tot i quallar.

Però compte: “posa sal al gust” no val com a pas d’algoritme. Quant és “al gust”? Un pessic? Un grapat? Cada persona interpretaria la recepta de manera diferent. Un algoritme de veritat **no deixa espai a la interpretació**: cada pas ha de ser precís i determinista. Si li passes la mateixa entrada, sempre produeix la mateixa sortida. Com una màquina expenedora: fiques la moneda, polses el botó, i sempre surt el mateix batut.

🏛️ Les propietats d’un algoritme

Perquè una seqüència de passos mereca el nom d’algoritme, ha de complir cinc propietats:

1. **Finit**: ha d’acabar en algun moment. Si s’executa per sempre, no és un algoritme, és un malson.
2. **Precís**: cada pas està definit sense ambigüitat. Res de “al gust” ni “quan estigui llest”.
3. **Entrada**: rep zero o més valors d’entrada.
4. **Sortida**: produeix almenys un valor de sortida.
5. **Eficaç**: resol el problema en temps finit i de manera correcta.

> 📝 **Nota històrica:** la paraula “algoritme” ve del matemàtic persa **Al-Juarismi** (segle SURT), que va escriure un llibre sobre com fer càlculs amb els nombres indis. Segles després, els informàtics li vam robar la paraula. Som així.

🧠 La idea vs. el codi

Aquest és un moment important, així que puja el volum mental:

> ⚠️ **Advertència: no tot codi és un algoritme.** L’algoritme és la *idea*: la seqüència de passos. El codi és la seva *materialització* en un llenguatge concret. Pots implementar el mateix algoritme en Java, en Python o en ensamblador, i l’essència serà la mateixa.

Mira aquest exemple. L’algoritme de “sumar dos nombres” es materialitza així en Java:

```java
public class AlgoritmeSimple {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        int resultat = a + b;  // la idea: sumar. El codi: la materialització
        System.out.println("5 + 3 = " + resultat);
    }
}
```

La *idea* (sumar dos nombres) és la mateixa en qualsevol llenguatge. El *codi* és només el disfressa. Per això els algoritmes s’estudien independentment del llenguatge: són les receptes, i Java només és una de les teves cuines.

📦 Les dues grans famílies

En aquesta unitat (i en la pròxima) viuràs amb dos famílies d’algoritmes:

- **Cerca**: trobar un element dins d’un conjunt de dades. El nombre 23 és en aquest array?
- **Ordenació**: posar un conjunt de dades en un ordre determinat. Pots deixar aquest array ordenat de menor a major?

Són les dues habilitats bàsiques de qualsevol programa que maneja dades, i apareixen pertot arreu: en una llista de reproducció, en una base de dades, en un cercador. Sense ordenar ni buscar, la teva app és un calaix desordenat on les coses només estan “per aquí”.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan escriguis passos "a la babalà" en un paper, pregunta't sempre: *podria la meva àvia executar aquests passos sense preguntar-me res?* Si la resposta és "no", tens ambigüitat.

Exercici: la truita ordenada

Els passos d'aquesta recepta estan desordenats. Ordena'ls perquè formen un algoritme vàlid i explica quina de les propietats de l'algoritme fallaria si els deixessis en l'ordre original:

1. Batre els ous.
2. Menjar la truita.
3. Posar sal "al gust" sobre les creïlles fregides.
4. Fregir les creïlles amb oli abundant.
5. Pelar i tallar les creïlles a rodanxes.
6. Barrejar les creïlles amb l'ou batut i quallar a la paella.

> 🔄 **Solució**
>
> Ordre correcte: **5 → 4 → 1 → 6 → 2** (pelar i tallar, fregir, batre, barrejar i quallar, menjar).
>
> El pas 3 ("posar sal al gust") sobra com a algoritme **precís**: és ambigu, cada persona interpretaria una quantitat diferent. Si el deixessis en l'ordre original, la recepta no seria **precisa** ni **eficaç**, perquè no hi ha una única manera correcta d'executar-la. Nota extra: el pas 2 (menjar) podria estar en un altre ordre, però el lògic és al final.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina és la diferència entre un algoritme i un programa?
2. Per què "posa sal al gust" no pot ser un pas d'algoritme?
3. Un algoritme pot rebre zero entrades?
4. Què passaria si un algoritme mai no acabés?

> 🔄 **Respostes**

1. L'algoritme és la **idea** (la seqüència de passos); el programa és la seva **materialització** en un llenguatge (Java, Python…).
2. Perquè és **ambigu**: no defineix una quantitat exacta, i dues persones l'interpretarien de manera diferent.
3. Sí, un algoritme pot rebre **zero o més** entrades. Per exemple, "imprimeix els nombres de l'1 al 10".
4. Deixaria de ser un algoritme: incompleix la propietat de ser **finit**. Es convertiria en un malson en bucle.

✅ Resum en 3 frases

1. Un algoritme és una **seqüència finita, precisa i sense ambigüitats de passos** que resol un problema: una recepta de cuina per a l'ordinador.
2. Ha de ser **finit, precís, amb entrades, amb sortida i eficaç**, i no deixa espai a la interpretació.
3. L'algoritme és la **idea** i el codi la seva materialització: el mateix algoritme s'escriu igual de bé en qualsevol llenguatge.

🐛 **Vocabulari ràpid**

Terme	Idea general
Algoritme	Seqüència finita i precisa de passos que resol un problema
Determinista	Mateixa entrada → sempre mateixa sortida
Ambigüitat	Pas que es pot interpretar de diverses maneres ("al gust")
Cerca	Trobar un element dins d'un conjunt de dades
Ordenació	Posar les dades en un ordre determinat (numèric, alfabètic…)
Materialitzar	Traduir la idea de l'algoritme a codi d'un llenguatge

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · Cerca lineal

02 — Cerca lineal

El cercador de sabatilles perdudes: revisa cada lloc fins a trobar el que busques 👟

> 🗺️ **Estàs en:** 🧩 **U07 · Algorítmica I: fonaments** → 02 · Cerca lineal

---

📬 La idea en una frase

> **La cerca lineal recorre l'array element per element fins a trobar l'objectiu (o fins a comprovar que no hi és). Simple, directa i funciona encara que les dades estiguin desordenades.**

Imagina que perds una sabatilla en la teva habitació. Què fas? Mires davall del llit, darrere de la porta, en l'armari… bàsicament **revises cada lloc fins a trobar-la**. Doncs això és la cerca lineal: vas un a un, sense dreceres.

👟 L'algoritme

```java
public class CercaLiniaal {

    public static int buscar(int[] array, int objectiu) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == objectiu) {
                return i;  // Trobat! Retorna la posició
            }
        }
        return -1;  // No és en l'array
    }

    public static void main(String[] args) {
        int[] numeros = {34, 12, 56, 78, 23, 9, 45, 67};

        int resultat = buscar(numeros, 23);
        if (resultat != -1) {
            System.out.println("Trobat el 23 en la posició " + resultat + "!");
        } else {
            System.out.println("El 23 no és en l'array.");
        }

        resultat = buscar(numeros, 99);
        if (resultat == -1) {
             System.out.println("El 99 no hi és. Com unes sabatilles que mai no
apareixen.");
        }
    }
}
```

Sortida:

```
Trobat el 23 en la posició 4!
El 99 no hi és. Com unes sabatilles que mai no apareixen.
```

Detalls del mètode `buscar`:

- Recorre l'array amb un `for` des de la posició 0 fins a `array.length - 1`.
- Si troba l'objectiu, **retorna el seu índex** i para: el `return` talla el mètode sencer.
- Si acaba el bucle sense trobar res, retorna `-1`, el "índex impossible" que usem com a senyal de *no trobat*.

> 💡 **Detall pràctic:** retornar `-1` és la convenció clàssica de “no hi és”. No retornis mai `0` per a dir “no trobat”, perquè `0` és una posició vàlida: la primera. Aquell error és el clàssic “bug de l’índex zero”.

⏱️ Com de ràpida és?

En el **millor cas**, l’element és en la primera posició → 1 pas.

En el **pitjor cas**, l’element és al final, o no existeix → recorres els n elements sencers.

Diem que la seva complexitat és **O(n)**, lineal. Si l’array té 10 elements, tardes ~10 passos; si en té 10.000, tardes ~10.000. Creix al mateix ritme que les dades.

> 💡 **Consell:** la cerca lineal és com buscar en la teva nevera: si és petita, tant li fa el mètode. Però si tens un magatzem de 10.000 productes, necessites alguna cosa millor… i en el pròxim punt la trobes.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan un mètode retorna un índex, els dos camins de sortida són: `return` quan el trobes, `return -1` quan el bucle acaba. Si mescles això, el programa es comporta com una gavina: confon els llocs.

Exercici: el cercador que es perd

Sense executar, calcula quantes comparacions fa aquest programa i què imprimeix:

```java
public class Cerca2 {
    public static int buscar(int[] array, int objectiu) {
        int passos = 0;
        for (int i = 0; i < array.length; i++) {
            passos++;
            if (array[i] == objectiu) {
                System.out.println("He necessitat " + passos + " passos.");
                return i;
            }
        }
        System.out.println("He necessitat " + passos + " passos.");
        return -1;
    }

    public static void main(String[] args) {
        int[] dades = {3, 8, 1, 9, 5, 2};
        int resultat = buscar(dades, 9);
        System.out.println("Posició: " + resultat);
    }
}
```

> 🔄 **Solució**
>
> El 9 és en la posició 3 (índex 3, el quart element). El bucle compara: `3` (pas 1), `8` (pas 2), `1` (pas 3), `9` (pas 4) → el troba. Imprimeix:
>
> ```
> He necessitat 4 passos.
> Posició: 3
> ```
>
> Fixa't que el `for` **no** recorre tot l'array: s'atura tan bon punt el `return` talla el mètode. Aquell és el poder del `return` com a "break" d'emergència.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què retorna la cerca lineal quan l'element no és en l'array?
2. Funciona la cerca lineal amb arrays desordenats?
3. Per què es diu que és O(n)?

4. Què imprimiria `buscar(new int[]{5, 5, 5}, 5)`? Quina posició retorna?

> 🔄 **Respostes**
>
> 1. Retorna `-1`, la senyal clàssica de “no trobat”.
> 2. **Sí.** Aquella és la seva gran avantatge: no exigeix cap ordre previ.
> 3. Perquè en el pitjor cas recorre els `n` elements de l’array: el temps creix en proporció directa amb les dades.
> 4. Retorna la posició **0** (el primer `5`), perquè el `return` talla tan bon punt troba el primer. La posició 0 és vàlida i diferent de “no trobat”.

✅ Resum en 3 frases

1. La cerca lineal recorre l’array **element per element** fins a trobar l’objectiu o esgotar la llista.
2. Retorna l’**índex** de l’element, o `-1` si no existeix, i funciona amb dades **desordenades**.
3. La seva complexitat és **O(n)**: perfecta per a arrays petits, lenta per als grans.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Índex	Posició d’un element dins de l’array (comença en 0)
Recórrer	Visitar cada element de l’array un a un
-1	L’“índex impossible”: senyal que l’element no hi és
O(n)	El temps creix en proporció directa al nombre d’elements
Millor cas	L’element és el primer: 1 sol pas

📚 Tornar a l’índex de la unitat · **Anterior:** 01 · Què és un algoritme · **Següent:** 03 · Cerca binària

03 — Cerca binària

El cercador Jedi: descarta mitja tona de paper en cada intent 🕯️

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 03 · Cerca binària

---

📬 La idea en una frase

> **La cerca binària obre el "diccionari" per la meitat, compara, i descarta mitja tona de paper en cada intent. Només funciona si l'array està ordenat.**

Buscar una paraula en un diccionari és un ritual molt concret: no obres per la pàgina 1 i passes d'una en una. Obres per la meitat, veus si la paraula està abans o després, i descartes la meitat del llibre en un gest. I repeteix. Això és la cerca binària.

⚠️ El requisit imprescindible

> **L'array ha d'estar ordenat.** Si no, aquest mètode no funciona.

I el pitjor de tot: no t'avisa. No hi ha error de compilació, no hi ha excepció, no hi ha "ei, m'has donat escombraria". Simplement obtens la resposta equivocada. És com buscar "berenar" en un diccionari les paraules del qual estan a l'atzar: obrir per la meitat no et serveix de res.

🕯️ L'algoritme

```java
public class CercaBinaria {

    public static int buscar(int[] array, int objectiu) {
        int esquerra = 0;
        int dreta = array.length - 1;

        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;  // meitat del segment
              System.out.println("esquerra=" + esquerra + " dreta=" + dreta + "
mig=" + mig);

            if (array[mig] == objectiu) {
                return mig;  // Bingo!
            }

            if (array[mig] < objectiu) {
                esquerra = mig + 1;  // descartem la meitat esquerra
            } else {
                dreta = mig - 1;     // descartem la meitat dreta
            }
        }
        return -1;  // no trobat
    }

    public static void main(String[] args) {
        // COMPTE: ha d'estar ORDENAT
        int[] numeros = {2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63};

        int resultat = buscar(numeros, 31);
        System.out.println("31 trobat en posició: " + resultat);  // 6

        resultat = buscar(numeros, 3);
        System.out.println("3 trobat en: " + resultat);  // -1
    }
}
```

Anem a traçar la cerca del 31 sobre l'array `{2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63}` (12 elements):

Volta	esquerra	dreta	mig	array[mig]	Què passa?
1	0	11	5	24	24 < 31 → esquerra = 6
2	6	11	8	42	42 > 31 → dreta = 7
3	6	7	6	31	Bingo! → retorna 6

Tres comparacions. La cerca lineal n'hauria necessitat set. I en un array d'un milió d'elements, la diferència encara és més escandalosa.

🧱 Per què `esquerra + (dreta - esquerra) / 2` i no `(esquerra + dreta) / 2`?

Perquè si l'array és molt gran (a prop de `Integer.MAX_VALUE` elements), `esquerra + dreta` pot **desbordar-se**: el resultat ja no cap en un `int` i es converteix en un nombre negatiu de sobte. La fórmula alternativa `esquerra + (dreta - esquerra) / 2` evita aquell problema.

> ⚠️ **Advertència:** aquest és un bug tan famós que va aparèixer fins i tot en la biblioteca de Java original. Du anys col·leccionant trofeus: Bug de l'any, Bug de la dècada, Bug favorit del públic...

⏱️ L'anàlisi: O(log n)

En cada pas, la cerca binària **descartar la meitat** de l'array restant. Mira com creix el nombre de passos:

- Array de 16 elements → 4 passos màxims
- Array de 32 elements → 5 passos
- Array de 1.024 elements → 10 passos
- Array d'1.000.000 d'elements → 20 passos

Això és **O(log n)**, complexitat logarítmica. Creix molt a a poc a poc fins i tot amb dades enormes. És la diferència entre preguntar a mil persones una per una, o preguntar "és a l'esquerra o a la dreta?" i descartar-ne 500 de cop. Amb un milió d'elements, la lineal necessita un milió de passos i la binària només **20**. Torna a llegir-ho. Vint.

> 📝 **Nota:** quan un informàtic diu “log n”, pensa en **base 2**: és “meitat, meitat, meitat…”. No és el logaritme decimal de tota la vida. Un logaritme en base 2 respon a la pregunta “quantes vegades puc partir entre 2 abans d’arribar a 1?”.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** la cerca binària té una regla d’or: si el mig és menor que l’objectiu, `esquerra = mig + 1`; si és major, `dreta = mig - 1`. El `+1` i el `-1` són sagrats: sense ells, el bucle es pot quedar donant vegades per sempre.

Exercici: el cercador que no avança

Sense executar, traça la cerca del nombre **8** en aquest array ordenat i escriu els valors d’`esquerra`, `dreta` i `mig` en cada vegada:

```
public class Traça {
    public static int buscar(int[] array, int objectiu) {
        int esquerra = 0;
        int dreta = array.length - 1;

        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;
              System.out.println("esquerra=" + esquerra + " dreta=" + dreta + "
mig=" + mig);

            if (array[mig] == objectiu) return mig;
            if (array[mig] < objectiu) {
                esquerra = mig + 1;
            } else {
                dreta = mig - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] dades = {1, 4, 8, 12, 20, 33};
        System.out.println("Resultat: " + buscar(dades, 8));
    }
}
```

› 🔄 Solució

Volta	esquerra	dreta	mig	array[mig]	Acció
1	0	5	2	8	Bingo! → retorna 2

Imprimeix:

```
esquerra=0 dreta=5 mig=2
Resultat: 2
```

El 8 és just en el mig de la primera passada, així que l'algoritme fa **una sola comparació**. Aquest és el millor cas de la cerca binària: trobar l'objectiu en el centre a la primera.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin requisit imprescindible té la cerca binària?
2. Què passa si l'incompleixes?
3. Quants passos màxims necessita per a un array d'1.000.000 d'elements?
4. Per què s'usa `esquerra + (dreta - esquerra) / 2` en comptes de `(esquerra + dreta) / 2`?

› 🔄 Respostes

1. Que l'array estigui **ordenat**.
2. Retorna la resposta equivocada **sense avisar**: no hi ha error ni excepció. Silenci i escombraria.
3. **20 passos** — $\log_2(1.000.000) \approx 20$.
4. Per a evitar el **desbordament**: `esquerra + dreta` pot no cabre en un `int` amb arrays gegants.

✅ Resum en 3 frases

1. La cerca binària **divideix el problema a la meitat en cada pas**, comparant l'objectiu amb l'element central.

2. Exigeix un array **ordenat**; si no ho està, retorna escombraria sense avisar.
3. La seva complexitat és **O(log n)**: amb un milió d'elements basten ~20 passos, mentre que la lineal necessita un milió.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Meitat | L'element central del segment actual (`mig`) |
> | Descartar | Eliminar la meitat esquerra o dreta del segment |
> | O(log n) | El temps creix molt a a poc a poc: cada pas descarta la meitat |
> | Off-by-one | Error de "per un": confondre <= amb < o `mig+1` amb `mig` |
> | Desbordament | Quan una suma supera el màxim que cap en un `int` |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Cerca lineal · **Següent:** 04 · Ordenació bombolla

04 — Ordenació bombolla

Simple, simpàtica… i la més lenta de les que mereixen la pena 🫧

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 04 · Ordenació bombolla

📬 La idea en una frase

> **La bombolla recorre l'array comparant parelles veïnes: si el de l'esquerra és major, els intercanvia. Repeteix fins que en una passada no hi hagi cap intercanvi. Els grans "pugen" cap al final com bombolles en una copa.**

És l'algoritme d'ordenació més senzill d'entendre… i el més lent dels que mereixen la pena. Però abans d'aprendre a córrer, cal aprendre a caminar. La bombolla és el teu caminador.

🫧 L'algoritme

```java
public class Bombolla {

    public static void ordenar(int[] array) {
        int n = array.length;
        boolean hiHaIntercanvi;

        for (int i = 0; i < n - 1; i++) {
            hiHaIntercanvi = false;

            for (int j = 0; j < n - 1 - i; j++) {
                if (array[j] > array[j + 1]) {
                    // intercanvi
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    hiHaIntercanvi = true;
                }
            }

            // si no hi va haver intercanvi, l'array ja està ordenat
            if (!hiHaIntercanvi) break;
        }
    }

    public static void main(String[] args) {
        int[] dades = {64, 34, 25, 12, 22, 11, 90};

        System.out.print("Abans: ");
        for (int nombre : dades) System.out.print(nombre + " ");

        ordenar(dades);

        System.out.print("\nDesprés: ");
        for (int nombre : dades) System.out.print(nombre + " ");
        // 11 12 22 25 34 64 90
    }
}
```

Anem a traçar la primera passada amb un array petit, `{5, 2, 9, 1}`:

Pas	Què compara?	Intercanvia?	Array
1	5 vs 2	Sí	2 5 9 1
2	5 vs 9	No	2 5 9 1
3	9 vs 1	Sí	2 5 1 9

El 9, el major, "va pujar" fins al final. En cada passada, el major dels que queden queda col·locat en el seu lloc: el 9, després el 5, després el 2, després el 1. Per això el bucle interior arriba només fins a `n - 1 - i`: ja no cal mirar els elements que van quedar col·locats al final.

🏎️ Per què és tan lenta?

Dos bucles anidats. Per a un array de n elements:

- Primer bucle: n vegades.
- Segon bucle: ~n vegades (en realitat `n-i-1`, però a grans trets n).

Total: ~n × n = n² operacions. Complexitat **O(n²)**.

- Per a 10 elements → 100 operacions (bé).
- Per a 1.000 elements → 1.000.000 d'operacions (comença a doldre).
- Per a 1.000.000 d'elements → 1.000.000.000.000 d'operacions (el teu ordinador demana la jubilació).

> 💡 **Consell:** la bombolla només s'usa en dos situacions: (1) estàs aprenent, i (2) saps que l'array tindrà menys de 50 elements. Per a tot lo demés hi ha alternatives millors (les veuràs en la U08).

🚩 L'optimització del flag

Fixa't en la variable `hiHaIntercanvi`. Si en una passada completa no intercanviem res, és que l'array ja està ordenat i podem parar: `break`. Sense aquest flag, la bombolla faria totes les passades encara que l'array arribés ordenat en la primera.

Aquesta optimització **no millora el pitjor cas** (array invertit: cal intercanviar-ho tot), però converteix el millor cas (array ja ordenat) en O(n): una sola passada de comprovació i llest.

> 💡 **Detall pràctic:** el patró del flag ("marca si ha passat alguna cosa; si no, para") apareix en moltíssims algoritmes reals. És una d'aquelles idees que et faran semblar programador sènior encara que només portes quatre unitats.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan vegis `array[j] > array[j + 1]`, pensa "estic ordenant **de menor a major**". Si vols el contrari, canvia la fletxa. La resta de l'algoritme no canvia ni una coma.

Exercici: la bombolla que es queda curta

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class BombollaCurta {
    public static void main(String[] args) {
        int[] dades = {3, 1, 2};

        for (int i = 0; i < dades.length - 1; i++) {
            for (int j = 0; j < dades.length - 1 - i; j++) {
                if (dades[j] > dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                }
            }
        }

        for (int nombre : dades) {
            System.out.print(nombre + " ");
        }
    }
}
```

> › 🔄 **Solució**
>
> Imprimeix **`1 2 3`**.
>
> Traça sense el flag (aquest programa no té `hiHaIntercanvi`):

Passada	j	Compara?	Array
1	0	3 vs 1 → sí	1 3 2
1	1	3 vs 2 → sí	1 2 3
2	0	1 vs 2 → no	1 2 3

Sense el flag, la bombolla fa una passada extra de comprovació. El resultat és el mateix, però en un array ja ordenat de 1.000 elements faria totes les passades sense necessitat. Aquí guanya la versió amb `break`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Per què el bucle interior de la bombolla només arriba fins a `n - 1 - i`?
2. Quina és la complexitat de la bombolla en el pitjor cas?
3. Què fa la variable `hiHaIntercanvi`?
4. Quan està justificat usar la bombolla en un programa real?

> 🔄 **Respostes**
>
> 1. Perquè després de cada passada, el major dels elements restants ja va quedar **col·locat al final**, i no cal tornar-lo a mirar.
> 2. **O(n²)** — dos bucles anidats.
> 3. Detecta si en la passada hi va haver intercanvis: si no n'hi va haver cap, l'array ja està ordenat i es fa `break`.
> 4. Només per a aprendre, o amb arrays de **menys de ~50 elements**. Per a la resta, espera a la U08.

✅ Resum en 3 frases

1. La bombolla compara **parelles veïnes** i intercanvia les que estan desordenades, passada rere passada.
2. La seva complexitat és **O(n²)**: funciona, però és lenta amb dades grans.

3. El flag `hiHaIntercanvi` l'optimitza per a arrays quasi ordenats, convertint el millor cas en O(n).

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Intercanvi | Canviar dos elements entre si usant una variable temporal |
> | Passada | Un recorregut complet del bucle interior |
> | Flag | Variable booleana que marca si alguna cosa va ocórrer durant la passada |
> | $O(n^2)$ | El temps creix al quadrat: dos bucles anidats |
> | In-place | Ordena modificant el mateix array, sense memòria extra |

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Cerca binària · **Següent:** 05 · Ordenació per inserció

05 — Ordenació per inserció

Ordena les cartes de la mà com un crupier veterà 🃏

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 05 · Ordenació per inserció

📬 La idea en una frase

> **La inserció pren cada element com una carta nova i el col·loca en el seu lloc dins de les que ja tens ordenades a la mà. Com ordenar cartes en el pòquer.**

Quan et reparteixen cartes, no les tires totes sobre la taula i comences de zero: vas col·locant cada carta nova en el seu buit dins de la mà que ja tens ordenada. El 7 entre el 5 i el 9, el 3 al principi, la K al final. Doncs això, però en Java.

🃏 L'algoritme

```java
public class Insercio {

    public static void ordenar(int[] array) {
        for (int i = 1; i < array.length; i++) {
            int clau = array[i];  // la carta que anem a col·locar
            int j = i - 1;

            // desplaçar els majors cap a la dreta
            while (j >= 0 && array[j] > clau) {
                array[j + 1] = array[j];
                j--;
            }
            array[j + 1] = clau;  // col·locar la carta en el seu lloc
        }
    }

    public static void main(String[] args) {
        int[] dades = {9, 5, 1, 4, 3};

        System.out.print("Abans: ");
        for (int nombre : dades) System.out.print(nombre + " ");

        ordenar(dades);

        System.out.print("\nDesprés: ");
        for (int nombre : dades) System.out.print(nombre + " ");
        // 1 3 4 5 9
    }
}
```

El truc està en el `while` interior: guardes la `clau` (la carta nova), i mentre hi hagi cartes majors que ella a la seva esquerra, les desplaces una posició a la dreta. Quan trobes una menor (o arribes al principi), aquella és la posició de la clau. La “mà” esquerra sempre està ordenada.

👣 Pas a pas

Donat `{9, 5, 1, 4, 3}`, mira com creix la “mà” (el que hi ha a l’esquerra de la barra):

```
Pas 0: [9] | 5 1 4 3   → la mà comença amb el 9
Pas 1: [5 9] | 1 4 3   → el 5 es col·loca a l'esquerra del 9
Pas 2: [1 5 9] | 4 3   → l'1 es cola al principi
Pas 3: [1 4 5 9] | 3   → el 4 entra entre l'1 i el 5
Pas 4: [1 3 4 5 9]     → el 3 entra entre l'1 i el 4
```

Cada element nou s'"inserta" en el seu lloc. D'aquí el nom. La mà esquerra sempre està ordenada; la resta de l'array espera el seu torn.

📊 L'anàlisi: quan és bona?

També és **O(n²)** en el pitjor cas (array invertit: cada element ha de viatjar fins al principi). Però té truc:

- **Millor cas (array quasi ordenat):** O(n). Només fa una passada de comprovació. És rapidíssima.
- És **estable**: manté l'ordre relatiu dels elements iguals.
- **No necessita memòria extra**: ordena in-place, modificant el mateix array.
- En la pràctica, és **més ràpida que la bombolla**, encara que totes dues siguin O(n²).

> 💡 **Consell:** la inserció és la reina de les dades **quasi ordenades**. Si saps que el teu array té 100 elements i ja està "quasi bé" (només un parell d'elements fora de lloc), la inserció et sorprendrà. De fet, s'usa com a pas final en algoritmes avançats (TimSort, el que usa Java per defecte en les seves col·leccions).

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** en la inserció, la variable `clau` és l'única que "sobreviu" al desplaçament. Si la usares per a una altra cosa, la perdries en sobrescriure `array[j + 1]`. Guarda-la com un tresor: és la teva carta.

Exercici: la mà que es desordena

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class Insercio2 {
    public static void main(String[] args) {
        int[] dades = {4, 2};

        for (int i = 1; i < dades.length; i++) {
            int clau = dades[i];
            int j = i - 1;
            while (j >= 0 && dades[j] > clau) {
                dades[j + 1] = dades[j];
                j--;
            }
            dades[j + 1] = clau;
        }

        System.out.println(dades[0] + " " + dades[1]);
    }
}
```

> 🔄 **Solució**
>
> Imprimeix **`2 4`**.
>
> Amb només dos elements, la inserció és quasi ridícula de simple: `clau = 2`, `j = 0`. Com que `4 > 2`, desplaça el 4 a la posició 1 i `j` passa a `-1`. El `while` acaba (perquè `j >= 0` ja no es compleix) i la clau es col·loca en `dades[0]`. Resultat: `{2, 4}`. La clau va viatjar fins al principi: aquell és el mecanisme exacte que, repetit, ordena arrays sencers.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. En què es diferencia la inserció de la bombolla a l'hora d'ordenar?
2. Quina és la complexitat de la inserció en el seu millor cas i per què?
3. Què significa que sigui "estable"?
4. Per què es diu que no necessita memòria extra?

> 🔄 **Respostes**
>
> 1. La bombolla intercanvia **veïns** en cada passada; la inserció **col·loca cada element en el seu lloc** desplaçant els majors una posició.

2. **O(n)** — amb un array quasi ordenat, cada element només necessita una comprovació i es queda on està.
3. Que manté l'**ordre relatiu** dels elements iguals entre si (si "Anna" venia abans que "Lluís" i tenen la mateixa edat, continua venint abans).
4. Perquè ordena **in-place**: modifica l'array original, sense crear estructures auxiliars.

✅ Resum en 3 frases

1. La inserció pren cada element com una **carta nova** i el col·loca en el seu lloc dins de la part ja ordenada.
2. És **O(n²)** en el pitjor cas, però **O(n)** amb dades quasi ordenades: la reina dels arrays quasi llestos.
3. És **estable**, no usa memòria extra i en la pràctica supera la bombolla.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Clau	L'element actual que estem col·locant en el seu lloc
Desplaçar	Moure un element una posició cap a la dreta
Mà	La part de l'array ja ordenada (a l'esquerra)
Estable	Respecta l'ordre relatiu dels elements iguals
In-place	Ordena sense necessitat d'arrays auxiliars

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Ordenació bombolla · **Següent:** 06 · Complexitat algorísmica: Big O

06 — Complexitat algorísmica: Big O

La balança que pesa algoritmes sense parar el cronòmetre ⏱️

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 06 · Complexitat algorísmica: Big O

📬 La idea en una frase

> **Big O no et diu quants segons tarda un algoritme: et diu com creix el seu temps quan creix la quantitat de dades. La tendència, no el cronòmetre.**

Dos ordinadors, un de 2024 i un altre de 2004. El modern guanya sempre en una prova ràpida, però això no ens diu res de l'*algoritme*. Per a comparar algoritmes de manera justa, mirem la seva **taxa de creixement**: què passa quan el nombre de dades `n` es fa MOLT gran? Això és Big O.

📈 Les complexitats més comunes

Notació	Nom	Exemple	Per a n = 1.000
O(1)	Constant	Accedir a `array[0]`	1 operació
O(log n)	Logarítmica	Cerca binària	~10 operacions
O(n)	Lineal	Cerca lineal	1.000 operacions
O(n log n)	Quasi lineal	Ordenacions avançades (ho veuràs en la U08)	~10.000 operacions
$O(n^2)$	Quadràtica	Bombolla, inserció	1.000.000 d'operacions
$O(2^n)$	Exponencial	Fibonacci sense optimitzar	inviable!

Fixa't en l'últim. Per a `n = 1.000`, $O(2^n)$ no és "molt lent": és que no acaba ni en l'era dels dinosaures. La diferència entre O(n) i $O(n^2)$ amb dades grans és la diferència entre "faig un cafè mentre carrega" i "em jubile abans que acabi".

🧪 Big O en codi

Anem a veure les tres més habituals amb exemples en Java:

```java
public class ExemplesComplexitat {

    // O(1) — CONSTANT: sempre igual, no importa la mida
    public static int obtenirPrimer(int[] array) {
        return array[0];  // un sol pas, sempre
    }

    // O(n) — LINEAL: creix en proporció a n
    public static int sumar(int[] array) {
        int suma = 0;
        for (int nombre : array) {
            suma += nombre;  // n passos
        }
        return suma;
    }

    // O(n²) — QUADRÀTICA: dos bucles anidats
    public static void imprimirParells(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + ", " + array[j]);  // n × n passos
            }
        }
    }

    public static void main(String[] args) {
        int[] dades = {10, 20, 30, 40, 50};

        System.out.println("O(1): " + obtenirPrimer(dades));
        System.out.println("O(n): " + sumar(dades));
        System.out.println("O(n²): mira la consola omplint-se de parells...");
        imprimirParells(dades);
    }
}
```

La regla del polze: **un bucle sol → O(n). Un bucle dins d'un altre → O(n²).** I si el bucle només recorre la meitat de les dades… continua sent O(n), no O(n/2). Les constants no conten.

📏 Les regles pràctiques de Big O

1. **Ignora les constants**: O(2n) és el mateix que O(n). El 2 no importa quan n tendeix a l'infinit.
2. **Queda't amb el terme dominant**: $O(n^2 + n) \rightarrow O(n^2)$. El n^2 es menja el n quan n creix.
3. **Els bucles anidats multipliquen**: un bucle dins d'un altre → $n \times n \rightarrow O(n^2)$.
4. **Els bucles seqüencials sumen**: un bucle i després un altre → $O(n + n) \rightarrow O(2n) \rightarrow O(n)$.

```
public class ReglesBigO {
    public static void main(String[] args) {
        // REGLA 1: les constants no importen
        // O(2n) → O(n)
        // O(100n) → O(n)

        // REGLA 2: el terme dominant es queda
        // O(n² + 5n + 1) → O(n²)
        // O(n + log n) → O(n)

        // REGLA 3: bucles anidats → multipliquen
        // for (i...) { for (j...) { } }  →  O(n × n) → O(n²)

        // REGLA 4: bucles seqüencials → sumen
        // for (i...) { }  for (i...) { }  →  O(n + n) → O(2n) → O(n)

        System.out.println("Big O no és màgia, és simplificar.");
        System.out.println("Pregunta't: què passa quan n es fa MOLT gran?");
    }
}
```

> 📝 **Nota:** Big O descriu el **pitjor cas** (la cota superior): "com a molt, tardarà això". Existeixen també Big Omega Ω (millor cas) i Big Theta Θ (cas mitjà), però amb Big O tens suficient per a començar. I per a aprovar, també.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** per a calcular Big O de memòria: compta bucles i mira si s'aniden o se succeïxen. Si tens dubtes entre O(n) i $O(n^2)$, pensa en un array d'un milió: recorre una vegada (un milió de passos) o un milió de vegades (un bilió)?

Exercici: l'analista de complexitats

Digues la complexitat Big O de cada mètode (les respostes, amagades):

```java
public class Analisi {

    public static int metodeA(int[] array) {
        int total = 0;
        for (int i = 0; i < array.length; i++) {
            total += array[i];
        }
        for (int i = 0; i < array.length; i++) {
            total += array[i] * 2;
        }
        return total;
    }

    public static void metodeB(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + " " + array[j]);
            }
        }
    }

    public static int metodeC(int[] array) {
        return array[array.length - 1];
    }
}
```

> 🔄 **Solució**
>
> - **`metodeA` → O(n)**: dos bucles **seqüencials** sumen: O(n + n) = O(2n) = O(n).
> - **`metodeB` → O(n²)**: dos bucles **anidats** multipliquen: n × n.
> - **`metodeC` → O(1)**: accés directe per índex, sense bucles. Un sol pas, sempre.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què mesura exactament Big O?
2. Quina és la complexitat d'un bucle anidat?

3. O(2n) i O(n) són el mateix?
4. Ordena de menor a major: O(1), $O(n^2)$, O(n), O(log n), $O(2^n)$.

> 🔄 **Respostes**
>
> 1. La **taxa de creixement** del temps d'execució quan creix n. No els segons exactes.
> 2. **$O(n^2)$** — els bucles anidats multipliquen.
> 3. **Sí** — les constants s'ignoren: O(2n) = O(n).
> 4. **O(1) < O(log n) < O(n) < $O(n^2)$ < $O(2^n)$.**

✅ Resum en 3 frases

1. Big O descriu **com creix el temps** d'un algoritme quan creix la quantitat de dades, no els segons exactes.
2. Les regles d'or: ignora constants, queda't amb el terme dominant, **anidar multiplica** i **seqüenciar suma**.
3. Amb dades grans, la diferència entre O(n) i $O(n^2)$ és la diferència entre un cafè i una jubilació.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Big O	Notació per a la cota superior de creixement d'un algoritme
Terme dominant	El que mana quan n és enorme (el n^2 de $n^2 + n$)
Anidar	Ficar un bucle dins d'un altre → multiplica la complexitat
O(1)	Constant: sempre el mateix nombre de passos
O(log n)	Cada pas descarta la meitat: creix molt a a poc a poc
Exponencial	$O(2^n)$: creix tan ràpid que és inviable amb dades mitjanes

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Ordenació per inserció · **Següent:** 07 · Triar l'algoritme adequat

07 — Triar l'algoritme adequat

La taula de decisió del bon programador, per a no usar una excavadora com a cullereta 🧠

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 07 · Triar l'algoritme adequat

📬 La idea en una frase

> **El millor algoritme no és el més famós ni el més ràpid en teoria: és el que encaixa amb la mida de les teves dades, el seu ordre inicial i el que necessites. Triar bé és la mitat de la faena.**

Per a obrir una nou faries servir una excavadora? Doncs això. Hi ha qui clava una cerca binària en un array de 5 elements, i qui ordena un milió de dades amb bombolla. Els dos estan malament: un malgasta esforç, l'altre malgasta el seu temps de vida. Triar l'algoritme adequat és part de l'ofici.

📋 La taula de resum

Algoritme	Complexitat	Quan usar-lo?	Quan NO?
Cerca lineal	O(n)	Arrays petits o desordenats	Arrays grans i ordenats (la binària guanya)
Cerca binària	O(log n)	Arrays grans **ordenats**, amb moltes cerques	Arrays desordenats (escombraria sense avís!)
Bombolla	$O(n^2)$	Aprendre i arrays molt petits (< 50)	Qualsevol cosa que mereixi la pena
Inserció	$O(n^2)$ / O(n)	Arrays petits o **quasi ordenats**	Arrays grans i desordenats

🧱 La pregunta clau: ¿ordenar abans de buscar?

Val, la cerca binària és rapidíssima… però exigeix un array ordenat. I ordenar també costa. Llavors: mereix la pena ordenar primer?

La regla del bon administrador:

- Si **ordenes una vegada i busques moltes vegades** → ordena amb alguna cosa decent (en la U08 veuràs com) i després usa binària. La inversió s'amortitza.
- Si **busques una sola vegada** en un array desordenat → cerca lineal directa. Ordenar només per a una cerca és regar el jardí amb xampany.

I una curiositat: per a arrays **molt petits** (menys de ~50 elements), la cerca lineal sol guanyar fins i tot amb dades ordenades, perquè la sobrecàrrega de la binària no compensa. La teoria importa, però el context mana.

🏫 Exemple guiat: el catàleg de la botiga

Tens una botiga amb `notes` de clients i vols saber la nota de "Lluís". El catàleg està en un `int[] notes` amb els valors desordenats i només preguntaràs una vegada. Què uses?

```
public class Botiga {
    // Una sola cerca sobre dades desordenades → cerca lineal
    public static int buscarNota(int[] notes, int objectiu) {
        for (int i = 0; i < notes.length; i++) {
            if (notes[i] == objectiu) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] notes = {7, 9, 5, 8, 6, 4};
        int posicio = buscarNota(notes, 8);
        System.out.println("El 8 està en la posició " + posicio);
    }
}
```

I si la teva botiga rebés **milers de consultes al dia** sobre el mateix catàleg? Llavors mereix la pena ordenar l'array una vegada (costa $O(n^2)$ amb el que saps avui, però en la U08 aprendràs

més ràpid) i usar cerca binària en cada consulta. **Ordenar una vegada, buscar mil.**

📊 Regles d'or per a decidir

1. **És petit (< 50)?** → Qualsevol val: usa lineal o inserció per simplicitat.
2. **És gran i desordenat?** → No usis bombolla ni inserció. Espera a la U08 (QuickSort, MergeSort).
3. **És gran i ordenat?** → Cerca binària, sense pensar-ho.
4. **Està quasi ordenat?** → Inserció arrasa: O(n) en la pràctica.
5. **Vaig a buscar moltes vegades?** → Inverteix en ordenar bé i busca amb binària.
6. **Vaig a buscar una sola vegada?** → Lineal directa, sense drames.

> ⚠️ **Advertència:** la tria de l'algoritme també dependrà d'altres factors que veuràs més endavant: l'**estabilitat** (mantenir l'ordre dels iguals?), la **memòria** disponible i si les dades caben en memòria. Per ara, amb aquestes regles d'or sobreviu a qualsevol examen i a quasi qualsevol app.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** abans d'escriure una línia, pregunta't: *quina mida té el meu array? Està ordenat? Quantes vegades faré aquesta operació?* Tres preguntes, tres respostes, i l'algoritme es tria sol.

Exercici: el cap de magatzem

Per a cada escenari, tria l'algoritme més adequat i justifica en una línia:

1. Un array de **8** notes desordenat, buscar la nota d'un alumne una sola vegada.
2. Una agenda de **10.000** contactes ordenada alfabèticament, on buscaràs noms constantment.
3. Un array de **60.000** mesuraments desordenats que cal deixar ordenats de menor a major.
4. Un array de **200** nombres ja quasi ordenats (només un parell de despistats fora de lloc).

> › 🔄 **Solució**
>
> 1. **Cerca lineal**: array petit, una sola cerca, i a més està desordenat. La binària ni es planteja.

2. **Cerca binària**: està ordenat i busques moltes vegades: O(log n) en cada consulta.
3. **Ni bombolla ni inserció**: amb 60.000 elements, O(n²) és un martiri. Toca esperar a la U08 (QuickSort/MergeSort, O(n log n)).
4. **Inserció**: amb dades quasi ordenades és O(n) en la pràctica, molt millor que la bombolla.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quan guanya la cerca lineal a la binària encara que l'array estigui ordenat?
2. Quan mereix la pena ordenar abans de buscar?
3. Quin algoritme tries per a un array quasi ordenat?
4. Quina regla d'or usaries per a un array de 100.000 elements desordenat?

> 🔄 **Respostes**
>
> 1. Amb arrays **molt petits** (< ~50) o quan només vas a buscar **una vegada**: la sobrecàrrega de la binària no compensa.
> 2. Quan **ordenes una vegada i busques moltes vegades**; la inversió s'amortitza amb les consultes.
> 3. **Inserció** — és O(n) en la pràctica amb dades quasi ordenades.
> 4. No usar bombolla ni inserció: el seu O(n²) seria un calvari. Espera a la U08 per a ordenar com cal.

✅ Resum en 3 frases

1. L'algoritme adequat depèn de la **mida**, l'**ordre inicial** i la **freqüència de les operacions**.
2. **Ordenar una vegada i buscar moltes** amortitza la inversió; buscar una sola vegada no mereix ordenar.
3. Regla ràpida: petit o desordenat → lineal; gran i ordenat → binària; quasi ordenat → inserció; gran i desordenat → espera a la U08.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Context	Mida, ordre inicial i freqüència d'ús de les teves dades
Amortitzar	Recuperar una inversió inicial amb l'ús repetit
Sobrecàrrega	El cost fix extra d'un algoritme sofisticat en dades petites
Trade-off	Compromís: guanyar en alguna cosa pagant alguna cosa a canvi
Estabilitat	Mantenir l'ordre relatiu dels elements iguals

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Complexitat algorísmica: Big O · **Següent:** 08 · Be the Code: cerca binària des de zero

08 — Be the Code: cerca binària des de zero

Programa binària i bombolla sense xarxa de seguretat, com en les entrevistes de Google 🕶️

> 🗺️ **Estàs en:** 🧩 **UO7 · Algorítmica I: fonaments** → 08 · Be the Code: cerca binària des de zero

---

📬 La idea en una frase

> **Aquest punt no té teoria nova: té dos reptes. Programar cerca binària i bombolla a mà, pas a pas, sense mirar els apunts. Si algun dia vols treballar en una FAANG, això ho has de saber escriure dormit.**

La cerca binària és un clàssic de les entrevistes tècniques de Google, Amazon i companyia. No perquè sigui difícil, sinó perquè l'**off-by-one** castiga sense pietat qui confia de més. Anem a entrenar-te perquè no siguis aquella víctima.

🕶️ Don Tip: la recepta mental de la binària

Abans d'escriure codi, memoritza la recepta:

1. Dos punters: `esquerra = 0` i `dreta = array.length - 1`.
2. Mentre `esquerra <= dreta`: calcula `mig = esquerra + (dreta - esquerra) / 2`.
3. Si `array[mig] == objectiu` → retorna `mig`.
4. Si `array[mig] < objectiu` → `esquerra = mig + 1`.
5. Si no → `dreta = mig - 1`.
6. Fi del bucle → retorna `-1`.

L'error més comú (fins i tot amb 10 anys d'experiència) és l'**off-by-one**: ¿`<=` o `<`? ¿`mig + 1` o `mig`? La resposta: `<=` en la condició, i `mig + 1` / `mig - 1` en moure els punters. Sense el `+1`/`-1`, si l'objectiu no hi és, el bucle es pot quedar girant per sempre.

🧩 REPTE 1: Cerca binària des de zero

Aquí tens l'esquelet. Completa'l. **Sense mirar els apunts.** El teu cervell ha de recordar, no copiar:

```java
public class RepBbinaria {

    public static int cercaBinaria(int[] array, int objectiu) {
        // 🧠 EL TEU CODI ACÍ
        return -1;
    }

    public static void main(String[] args) {
        int[] proves = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

         System.out.println("El 7 està en posició: " + cercaBinaria(proves, 7));
// 3
           System.out.println("El 13 està en posició: " + cercaBinaria(proves,
13));  // 6
         System.out.println("El 8 està en posició: " + cercaBinaria(proves, 8));
// -1
           System.out.println("El 19 està en posició: " + cercaBinaria(proves,
19));  // 9
          System.out.println("L'1 està en posició: " + cercaBinaria(proves, 1));
// 0
            System.out.println("Array buit: " + cercaBinaria(new int[]{}, 5));
// -1
    }
}
```

Passos guiats (resisteix a llegir-los tots de cop):

1. Declara `esquerra` i `dreta`.

   > **Atascat?**
   > `int esquerra = 0;` i `int dreta = array.length - 1;`

2. Escriu el `while` amb la condició correcta.

   > **Atascat?**
   > `while (esquerra <= dreta) { … }`

3. Calcula el `mig`.

   > **Atascat?**
   > `int mig = esquerra + (dreta - esquerra) / 2;` — la fórmula anti-desbordament.

4. Compara i decideix els tres casos.

   > **Atascat?**

> `==` → retorna `mig`; `<` → `esquerra = mig + 1`; `>` → `dreta = mig - 1`.

5. Fora del bucle, retorna el “no trobat”.

> **Atascat?**
>
> `return -1;`

> 💡 **Consell de depuració:** si alguna cosa falla, dibuixa l’array en un paper amb 3 elements i simula els passos amb un objectiu que no hi sigui. El paper i el bolígraf són les teves millors eines de depuració. Sí, en ple 2026, i sí, funcionen.

> **🔄 Solució completa**

```
public static int cercaBinaria(int[] array, int objectiu) {
    if (array == null || array.length == 0) return -1;

    int esquerra = 0;
    int dreta = array.length - 1;

    while (esquerra <= dreta) {
        int mig = esquerra + (dreta - esquerra) / 2;

        if (array[mig] == objectiu) {
            return mig;
        } else if (array[mig] < objectiu) {
            esquerra = mig + 1;
        } else {
            dreta = mig - 1;
        }
    }
    return -1;
}
```

🧩 REPTE 2: Bombolla a mà

Ara toca la bombolla. Mateixa regla: sense mirar. Completa el mètode `ordenar` perquè deixi l’array de menor a major. Ha de mostrar `11 12 22 25 34 64 90`:

```java
public class RepBombolla {

    public static void ordenar(int[] array) {
        // 🧠 EL TEU CODI ACÍ
    }

    public static void main(String[] args) {
        int[] dades = {64, 34, 25, 12, 22, 11, 90};

        ordenar(dades);

        for (int nombre : dades) {
            System.out.print(nombre + " ");
        }
    }
}
```

Passos guiats:

1. Dos bucles anidats: l'exterior repeteix passades.

   > **Atascat?**
   >
   `for (int i = 0; i < array.length - 1; i++) { … }`

2. L'interior compara parelles veïnes, arribant cada vegada un element menys.

   > **Atascat?**
   >
   `for (int j = 0; j < array.length - 1 - i; j++) { … }`

3. Si estan desordenades, intercanvia amb una variable temporal.

   > **Atascat?**
   >
   `if (array[j] > array[j + 1]) { int temp = array[j]; array[j] = array[j + 1]; array[j + 1] = temp; }`

4. Extres de nivell: afegeix el flag `hiHaIntercanvi` amb el seu `break`.

   > **Atascat?**
   >
   Un `boolean hiHaIntercanvi = false;` al principi de cada passada, `true` en intercanviar, i `if (!hiHaIntercanvi) break;` al final de la passada.

> **🔄 Solució completa**

```java
public static void ordenar(int[] array) {
    int n = array.length;
    boolean hiHaIntercanvi;

    for (int i = 0; i < n - 1; i++) {
        hiHaIntercanvi = false;

        for (int j = 0; j < n - 1 - i; j++) {
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
                hiHaIntercanvi = true;
            }
        }

        if (!hiHaIntercanvi) break;
    }
}
```

🧩 EL LIO: la bombolla que apesta

El departament de qualitat ha rebut aquest algoritme. Alguna cosa fa mala olor. Identifica els errors i explica per què no funciona:

```java
public class BombollaLiosa {
    public static void ordenar(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

🕶️ **Don Tip:** quan un bucle accedeix a `arr[j + 1]`, assegura't que `j + 1` no surti de l'array. I pregunta't: recórrec menys elements en cada passada?

> 🔄 **Solució**
>
> Hi ha **dos errors**:
>
> 1. **Error d'índexs (i de càstig segur):** el bucle interior va `j < arr.length`, així que quan `j = arr.length - 1`, accedeix a `arr[j + 1] = arr[arr.length]`, que **no existeix** → `ArrayIndexOutOfBoundsException`. L'interior ha d'anar fins a `arr.length - 1 - i`.
> 2. **Error de rendiment:** el bucle exterior recorre `arr.length` vegades i l'interior **sempre** recorre tot l'array, sense aprofitar que cada passada deixa un element col·locat al final. A més, sense el flag `hiHaIntercanvi`, continua fent passades encara que l'array ja estigui ordenat. És bombolla "sense polir", i es nota.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. En la cerca binària, quina condició usa el `while`: `esquerra <= dreta` o `esquerra < dreta`?
2. Per què `esquerra = mig` (sense el `+1`) pot penjar el bucle?
3. Què passa si fas la bombolla amb `j < array.length` en el bucle interior?
4. Per a què serveix el flag `hiHaIntercanvi` en la bombolla?

> 🔄 **Respostes**
>
> 1. `esquerra <= dreta`. Amb `<`, et pots perdre l'element que queda just en `mig` quan els punters es creuen.
> 2. Perquè si `mig` no és l'objectiu, en reassignar `esquerra = mig` (o `dreta = mig`) el segment **no es redueix** i el bucle es repeteix amb els mateixos límits per sempre.
> 3. `ArrayIndexOutOfBoundsException`: en arribar a `j = array.length - 1`, `arr[j + 1]` està fora de l'array.
> 4. Detectar que l'array ja està ordenat per a parar (`break`) en comptes de seguir fent passades inútils.

✅ Resum en 3 frases

1. La cerca binària s'escriu amb **dos punters**, el **mig anti-desbordament** i els **`+1`/`-1` sagrats**; amb això, no hi ha off-by-one que valgui.
2. La bombolla es construeix amb **dos bucles anidats**, un intercanvi amb variable temporal i, si ets llest, un **flag** que talla quan ja està ordenat.
3. Escriure tots dos **a mà** (sense mirar) és l'exercici de la unitat: si ho aconsegueixes, et portes la medalla al cinturó.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Off-by-one | Error de "per un": `<=` vs `<`, `mig+1` vs `mig` |
> | Punter | Índex que delimita el segment actual (`esquerra`, `dreta`) |
> | Anti-desbordament | `esquerra + (dreta - esquerra) / 2` en comptes de `(esquerra+dreta)/2` |
> | Variable temporal | El `temp` que guarda un valor durant l'intercanvi |
> | Flag | Booleà que avisa de si va passar alguna cosa en la passada |

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Triar l'algoritme adequat · **Següent:** 09 · Repàs interactiu

09 — Repàs interactiu: ordena i busca com un professional

El tancament pràctic de la unitat, amb cerques, bombolles i un algoritme que no funcionarà 😈

> 🗺️ **Estàs en: 🧩 U07 · Algorítmica I: fonaments** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
public class Misteri {
    public static void main(String[] args) {
        int[] dades = {10, 20, 30, 40, 50, 60};
        int esquerra = 0;
        int dreta = dades.length - 1;
        int objectiu = 40;
        int passos = 0;

        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;
            passos++;
            if (dades[mig] == objectiu) {
                  System.out.println("Trobat en " + mig + " amb " + passos + "
passos.");
                return;
            }
            if (dades[mig] < objectiu) {
                esquerra = mig + 1;
            } else {
                dreta = mig - 1;
            }
        }
        System.out.println("No hi és. Passos: " + passos);
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`Trobat en 3 amb 1 passos.`** → Confons el nombre de passos amb la posició: la cerca no arriba en una sola volta. ❌
2. **`Trobat en 3 amb 3 passos.`** → ✅ Correcte! Amb 6 elements, `mig` és `0 + (5-0)/2 = 2` → `dades[2] = 30`, que és menor que 40, així que `esquerra = 3`. Segona volta: `mig = 3 + (5-3)/2 = 4` → `dades[4] = 50`, que és major, així que `dreta = 3`. Tercera volta: `mig = 3 + (3-3)/2 = 3` → `dades[3] = 40`. Bingo en 3 passos! Amb `return` (estem en `main`) el programa acaba aquí.
3. **`No hi és. Passos: 4.`** → El 40 és en l'array, només que la cerca binària no el caça a la primera. ❌

> 🔄 **Solució**

> L'opció **2**. Traça: `mig=2 (30<40) → esq=3`, `mig=4 (50>40) → der=3`, `mig=3 → 40!`. Són **3 passos** en la posició **3**. El `return` dins de `main` acaba el programa sense arribar a la línia del "No hi és".

🔥 Fireside Chat: cerca lineal vs cerca binària

> *Dos veterans de la cerca discuteixen al costat de la màquina de cafè.*

Lineal: — Jo soc la bàsica. Recórrec element per element, sense exigir-li res a ningú. Desordenat? Sense problema. Cinc elements? Un moment. Un milió? Uns milions de passos, però vaig.

Binària: — Uns milions de passos. Quina generositat. Jo amb un milió trigo vint passos. Vint. Mentre tu sues, jo ja he acabat i estic demanant un altre cafè.

Lineal: — I qui t'ha donat permís per a ser tan intel·ligent? L'array **ordenat**. Si les dades arriben desordenades, tu no serveixes ni per a obrir la porta. Jo, en canvi, funciono sempre. És la vida: sense exigir res, però sense grans alegries.

Binària: — Ordenar una vegada i buscar mil, i veuràs. Jo soc la que salva les apps amb milions d'usuaris. Tu ets… el pla B.

Lineal: — El pla B que no s'estavella. Quan el teu array està desordenat i a ningú li apeteix ordenar-lo, a mi em criden. I no em queixe.

Binària: — Val, cadascuna al seu terreny: tu, el petit i desordenat; jo, el gran i ordenat amb moltes cerques. ¿Treva?

Lineal: — Treva. Però que sàpigues que en els arrays de 5 elements et guanyi fins i tot a tu, amb els teus aires d'`esquerra + (dreta - esquerra) / 2`.

> La lliçó: cap no és millor "en general". **Lineal** per a dades petites o desordenades; **binària** per a dades grans i ordenades amb moltes cerques. El context decideix.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc una recepta de cuina per a l'ordinador: finita, precisa i sense ambigüitats.**

2. **Recórrec l'array element per element fins a trobar l'objectiu. No exigeixo ordre, però soc lenta amb dades grans.**
3. **Obric el diccionari per la meitat i descarte mitja tona de paper en cada intent. Exigeixo ordre, o et torni escombraria sense avisar.**
4. **Soc el patós de l'ordenació: compari veïns i intercanvii, una vegada i una altra, fins que els grans pugen com bombolles.**
5. **Ordeni com en el pòquer: col·loque cada carta nova en el seu lloc dins de la mà que ja tinc ordenada.**
6. **No mesuri segons: mesuri com creix el temps quan creixen les dades.**

> 🔄 **Respostes**
>
> 1. **L'algoritme** — seqüència finita, precisa i sense ambigüitat de passos.
> 2. **La cerca lineal** — O(n), no exigeix ordre.
> 3. **La cerca binària** — O(log n), exigeix array ordenat.
> 4. **L'ordenació bombolla** — intercanvia veïns, O(n²).
> 5. **L'ordenació per inserció** — col·loca cada element en el seu lloc, O(n²) amb O(n) en quasi ordenats.
> 6. **La notació Big O** — descriu la taxa de creixement del temps d'execució.

🤬 CONRAD VS EL MÓN: "L'algoritme que no s'acaba"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre el clàssic del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, la meva cerca binària es queda penjada*. I jo: val, què té el bucle? *Pues no ho sé, no l'he mirat.* AI, MARE MEVA! Un `while (esquerra <= dreta)` que en lloc d'`esquerra = mig + 1` posa `esquerra = mig`… saps què passa? El segment no es redueix. `esquerra` es queda atascada i el bucle fa voltes com un gat perseguint-se la cua. Ensenyant-li l'algoritme a un robot aspirador amb direcció pròpia.

I després està el de la bombolla: `for (int j = 0; j < array.length; j++)` i dins `array[j + 1]`. DE VERITAT? Quan `j` arriba al final, `j + 1` surt de l'array. A què esperes, que et llanci l'`ArrayIndexOutOfBoundsException` per a llegir el missatge? El missatge ja t'està dient l'índex, la línia i el motiu. LLEGEIX-LO.

I el clàssic: el meu algoritme ordena però molt a a poc a poc. Ja. I què vas usar? *Bombolla*. Amb un array de mig milió. Saps quants intercanvis són? Els que són. I no és culpa de l'algoritme, que avisa en l'etiqueta: "O(n²), per a arrays petits". L'algoritme no té la culpa que no llegires l'etiqueta.

La lliçó: els bucles dels algoritmes es pengen per dos motius: o la condició mai no avança cap a `false`, o l'índex surt de l'array. Abans de plorar sobre el teclat, comprova aquelles dues coses. El 90% dels "algoritmes penjats" s'arreglen amb un cop d'ull.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Quina és la complexitat de la cerca binària?
   - a) O(n) b) O(log n) c) O(1)
2. Quants passos màxims necessita la cerca binària per a un array de 1.024 elements?
   - a) 10 b) 1.024 c) 11
3. Què retorna `buscar(new int[]{}, 5)` amb una cerca binària ben feta?
   - a) `-1` b) `0` c) `Exception`
4. Quina d'aquestes és O(n²)?
   - a) Un bucle seqüencial b) Dos bucles anidats c) Accedir a `array[0]`

> 🔄 **Solucions**
>
> 1. **b)** — O(log n): descartar la meitat en cada pas.
> 2. **a)** — 10 passos: $\log_2(1.024) = 10$.
> 3. **a)** — `-1`: el bucle ni tan sols entra (0 <= -1 és `false`) i retorna el "no trobat".
> 4. **b)** — Dos bucles anidats multipliquen: n × n.

⚡ Laboratori de Tortura: la màquina que no ordena

> **Durada estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Té **3 errors** que impedeixen que compili o que falli en execució, i 1 error de lògica que fa que el resultat sigui incorrecte quan el arregles.

```java
public class Tortura
    public static void main(String[] args) {
        int[] dades = {3, 1, 4, 2};
        for (int i = 0; i < dades.length; i++) {
            for (int j = 0; j < dades.length; j++) {
                if (dades[j] < dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                }
            }
        }
        for (int n : dades) System.out.print(n + " ")
    }
}
```

Fallada intencionada: un dels errors sembla correcte a simple vista perquè “es veu bé”, però fa que el programa falli en execució tan bon punt arranca.

La teva tasca: aconseguir que compili, que executi i que imprimeixi `1 2 3 4`. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun `;` que falti? *no → segueix buscant.*
   > **› I si encara estic atascat?**
   > Comprova també les claus `{}`: la classe necessita la seva obertura, i l'últim `for` necessita el seu `;`.
2. Compila ja? *no → mira el missatge d’error.*
   > **› I si encara estic atascat?**
   > La classe `Tortura` necessita `{` d'obertura.
3. Compila però explota en executar? *És l’error d’índexs: el bucle interior arriba massa lluny.*
   > **› I si encara estic atascat?**
   > `j < dades.length` accedeix a `dades[j + 1]` fora de l'array. Ha de ser `j < dades.length - 1 - i` (i l'exterior `i < dades.length - 1`).
4. Executa i imprimeix `4 3 2 1`? *És l’error de lògica: el signe de la comparació ordena al revés.*
   > **› Solució final**

Els **3 errors** que impedeixen compilar o executar:

1. Falta la `{` d'obertura de la classe després de `Tortura`.
2. Falta el `;` al final de `System.out.print(n + " ")`.
3. `j < dades.length` accedeix a `dades[j + 1]` fora de l'array: `ArrayIndexOutOfBoundsException`. Ha de ser `j < dades.length - 1 - i`.

L'**error de lògica**: `dades[j] < dades[j + 1]` ordena **de major a menor**. Compila i executa perfectament, però imprimeix `4 3 2 1` en lloc de `1 2 3 4`. La bombolla puja el major cap al final comparant amb >, no amb <.

```
public class Tortura {
    public static void main(String[] args) {
        int[] dades = {3, 1, 4, 2};
        for (int i = 0; i < dades.length - 1; i++) {
            for (int j = 0; j < dades.length - 1 - i; j++) {
                if (dades[j] > dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                }
            }
        }
        for (int n : dades) System.out.print(n + " ");
    }
}
```

Sortida correcta: `1 2 3 4`. Amb la versió trencada, una vegada arreglats els altres errors, la comparació < donava `4 3 2 1`: el signe era la pista de l'error de lògica.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 El Receptari Humà	Explicar què és un algoritme i les seves 5 propietats sense mirar
🏅 El Cercador Jedi	Escriure cerca binària a mà i que funcioni a la primera
🏅 El Crupier	Explicar la inserció amb l'analogia de les cartes, sense mirar

Assoliment	Com aconseguir-lo
🏅 **L'Analista**	Calcular Big O d'un fragment de codi en menys d'un minut
🏅 **El Cap de Magatzem**	Triar l'algoritme correcte per a cada escenari del punt 7
🏅 **El Caçador d'Errors**	Trobar els 3+1 errors del Laboratori sense usar les pistes

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```java
public class Misteri2 {
    public static void main(String[] args) {
        int[] dades = {2, 4, 6, 8};
        int comptador = 0;

        for (int i = 0; i < dades.length; i++) {
            for (int j = i + 1; j < dades.length; j++) {
                if (dades[i] < dades[j]) {
                    comptador++;
                }
            }
        }
        System.out.println(comptador);
    }
}
```

2. **La còpia que es desordena:** què li passa a la bombolla si en lloc de comparar `>` compares `>=`? Afecta l'estabilitat de l'algoritme?
3. **El detectiu:** la teva cerca binària retorna `-1` per a un nombre que SÍ és en l'array. Quina eina uses i quines variables mires primer?
4. **Vertader o fals:** "la cerca binària funciona amb qualsevol array, només que a vegades és més lenta".

> 💡 **Solucions**
>
> 1. Imprimeix **6**. El bucle doble compta les parelles `(i, j)` amb `i < j` on `dades[i] < dades[j]`. Amb `{2,4,6,8}` totes les parelles compleixen: $4 \cdot 3 / 2 = 6$.

2. Amb `>=` la bombolla seguiria ordenant, però **romp l'estabilitat**: dos elements iguals podrien intercanviar-se, canviant el seu ordre relatiu. La versió amb `>` (estricte) manté l'ordre dels iguals.
3. El **depurador**: posa un breakpoint en el `while` i observa `esquerra`, `dreta` i `mig` en cada vegada. Si `dreta` mai no baixa o `esquerra` no avança amb `mig + 1`, aquell és el fall. El clàssic off-by-one.
4. **Fals.** Amb un array desordenat no és que sigui lenta: retorna **resultats incorrectes sense avisar**. No hi ha error, hi ha escombraria silenciosa.

🧩 Crucigrama de Bits

```
Horitzontal:
1. Notació que mesura com creix el temps (3 lletres + espai + 1 lletra: "O" més una altra lletra)
3. La complexitat de la cerca binària (comença per "log")
5. Algoritme que intercanvia veïns fins a ordenar (8 lletres)
6. Seqüència finita i precisa de passos (9 lletres)

Vertical:
2. Senyal de "no trobat" que retornen les cerques (2 lletres)
4. Algoritme que col·loca cada carta en el seu lloc (9 lletres)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. BIGO · 3. LOG · 5. BOMBOLLA · 6. ALGORITME **Vertical:** 2. MENYS (el -1) · 4. INSERCIÓ

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo fos la teva àvia, la diferència entre cerca lineal i cerca binària."**
2. **"Què és la notació Big O i per què és important?"**
3. **"Escriu una cerca binària en la pissarra. Ara digues-me què passa si l'array no està ordenat."**
4. **"Quan usaries l'ordenació per inserció en comptes de la bombolla?"**

5. **“Què és l’off-by-one i com l’evites en la cerca binària?”**
6. **“Un algoritme O(n²) tarda 1 segon amb 1.000 elements. Quant tardarà amb 2.000? I amb 10.000?”**

🤷 No hi ha preguntes tontes

> ❓ **Puc usar `Arrays.sort()` i `Arrays.binarySearch()` de Java en comptes d’escriure els algoritmes?**

En els teus programes reals, sí: Java porta utilitats ordenades, eficients i provades, i veuràs `Arrays.sort()` aviat. Però en aquesta unitat l’objectiu és **entendre la idea** que hi ha davall. És com aprendre a fer una suma a mà abans d’usar la calculadora: no és que la calculadora sigui dolenta, és que necessites saber què estàs fent. I en una entrevista, l’entrevistador vol veure que ho entens, no que saps importar `java.util.Arrays`.

> ❓ **Per què cal dir “log n” i no simplement “pocs passos”?**

Perquè “pocs passos” no serveix per a comparar: pocs comparat amb què. El logaritme en base 2 et diu exactament **quantes vegades pots partir per la meitat** abans d’arribar a 1. I quan algú et diu “és O(log n)”, tu saps exactament què significa. La precisió és el sou del programador.

> ❓ **Si bombolla i inserció són totes dues O(n²), per què es diu que la inserció és millor?**

Per dos motius: en el **cas mitjà** fa menys intercanvis (desplaça, no intercanvia de tres en tres), i en **arrays quasi ordenats** és O(n) de veritat, mentre que la bombolla sense flag continua fent passades senceres. En la pràctica, amb arrays petits, la inserció nota la diferència. Amb arrays grans, cap de les dues: aquí arriba la U08.

🎬 Post-Crèdits

El programador acaba el seu catàleg: ordena les notes amb inserció, i quan un client pregunta per la seva nota, respon amb cerca binària en un parpelleig. Està content. S’acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa humejant.

CONRAD: — Oi, no està mal. Inserció per a ordenar, binària per a buscar… I a més saps dir-me per què no vas usar la bombolla per al milió de dades. Segur que ets el mateix que va començar

aquesta unitat amb el “posa sal al gust”?

Novell: — *somriu* Vaig començar amb una recepta de truita i he acabat ordenant el catàleg en 20 passos. I ara què?

CONRAD: — *fa un glop* Tot això és l’escalfament. Bombolla, inserció, lineal, binària… són els fonaments. El següent nivell usa tècniques que parteixen els problemes en trossos, es criden a si mateixes i ordenen arrays d’un milió en un sospir. Un altre tipus de bèsties.

El novell guarda el seu projecte, tanca l’IDE i sent que ja no busca ni ordena: **pensa en termes d’algoritmes**.

PRÒXIMAMENT EN U08: Algorítmica II: tècniques avançades i modularitat. Divideix i venç, QuickSort, MergeSort i recursivitat. El moment en què ordenar un milió de dades deixa de ser una utopia. ⚡

📚 Tornar a l’índex de la unitat · **Anterior:** 08 · Be the Code: cerca binària des de zero · **Següent:** U08 · Algorítmica II: tècniques avançades i modularitat

U08 — Algorítmica II: tècniques avançades i modularitat

Recursivitat, divide i venceràs, Quicksort, Mergesort i modularitat: les tècniques que fan que el teu codi pense ⚡

Benvingut, valent explorador de la pila de crides. En la U07 vas dominar l'art de buscar i ordenar amb bucles: cerca lineal, binària, bombolla i inserció. Vas aprendre a mesurar algorismes amb Big O i a no usar mai bombolla en producció (ho vas prometre). Però els bucles es queden curts quan el problema és *inherentment* jeràrquic. I aquí és on la programació es torna elegant.

Aquesta unitat és la que **et canvia la manera de pensar**. Faràs que una funció es crida a si mateixa (sense tornar-te boig), partiràs problemes per la meitat fins a fer-los trivials, i coneixeràs els dos reis de l'ordenació moderna: **Quicksort** i **Mergesort**. Quan acabes, veuràs recursivitat per tot arreu. És com aprendre una paraula nova: de sobte, la trobes en cada racó del codi. I per a rematar, aplicaràs aquesta mateixa mirada al teu propi codi: **modularitat**, l'art de partir-lo en peces amb nom i amb sentit.

Respira fons. I recorda: per a entendre la recursivitat, primer has d'entendre la recursivitat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Escriure funcions **recursives** amb cas base i cas recursiu, i raonar sobre la **pila de crides**.
- Implementar els clàssics: **factorial**, **Fibonacci** (ingenu i amb memoització), **palíndroms** i **torres de Hanoi**.
- Aplicar l'estratègia **divide i venceràs**: dividir, conquistar i combinar.
- Implementar **Quicksort** entenent la partició, el pivot i les seves complexitats.
- Implementar **Mergesort** entenent la fusió i l'estabilitat.
- **Triar amb criteri** l'algorisme d'ordenació segons el context (mida, ordre previ, memòria, estabilitat).
- Reconèixer altres tècniques com els algorismes **voracos** (greedy) i el **backtracking**.
- Organitzar el teu codi amb criteri de **modularitat**: paquets, cohesió alta i acoblament baix.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Recursivitat	Cas base, cas recursiu, la pila de crides i la por al `StackOverflowError`	Tots
02 · Recursivitat: exemples clàssics	Factorial, Fibonacci, palíndroms i torres de Hanoi	Tots
03 · Divide i venceràs	Dividir, conquistar i combinar: el patró de Juli Cèsar	Tots
04 · Quicksort	Partició, pivot, complexitats i per què és tan famós	Tots
05 · Mergesort	Divisió per la meitat, fusió i estabilitat	Tots
06 · Comparació: quan usar cada ordenació	Guia pràctica per a triar algorisme i què hi ha sota `Arrays.sort()`	Tots
07 · Be the Code: quicksort des de zero	Implementar Quicksort i Mergesort pas a pas, sense mirar	⭐⭐⭐
08 · Altres tècniques algorísmiques	Algorismes voracos (greedy) i backtracking	⭐⭐⭐
09 · Modularitat	Paquets, cohesió i acoblament per a partir el teu codi	Tots
10 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 9 primers punts són teoria i pràctica en progressió. El 10 és l'aterratge final: llegeix-lo just després del 9 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

✅ Criteris d'avaluació coberts (RA2 + RA6)

RA2: Escriu i prova programes senzills, reconeixent i aplicant els fonaments de la programació orientada a objectes.

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA2 b)	S'han escrit programes simples.	✅ Tots
RA2 e)	S'han escrit crides a mètodes estàtics.	✅ Punts 1-7
RA2 f)	S'han utilitzat paràmetres en la crida a mètodes.	✅ Punts 1-8
RA2 i)	S'ha utilitzat l'entorn integrat de desenvolupament en la creació i compilació de programes simples.	✅ Tots
RA6 a)	S'han escrit programes que utilitzen arrays.	✅ Punts 3-8
RA6 b)	S'han reconegut les llibreries de classes relacionades amb tipus avançats de dades.	✅ Punts 4, 5 i 6

> 📌 La recursivitat dels punts 1 i 2 és la llavor de les **estructures jeràrquiques** (arbres, grafs) que exploraràs amb més profunditat en el mòdul d'Estructures de Dades. I la tria justificada d'algorisme del punt 6 és, directament, un RA6 en estat pur: *seleccionar* el tipus de processament adequat a les dades.

🚪 Per on començo?

- Vens de la U07 i estàs fresc de Big O? → Arranca en el punt 1. Només necessites el que vas veure en Fonaments.
- El que t'agrada és veure el resultat i no les tripes? → Salta al punt 4 i al punt 5 i torna després a la teoria de la recursivitat.
- Et flipen els reptes? → Fes el punt 7 sense mirar el codi dels punts 4 i 5. Pateix amb honor.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Recursivitat

⏭️ **En acabar la unitat, continua en U09 · POO: classes i objectes.**

01 — Recursivitat: una funció que es crida a si mateixa

Quan el teu codi decideix cridar-se a si mateix i el teu cervell comença a fumejar 🔁

> 🗺️ **Estàs en: ⚡ U08 · Algorítmica II: tècniques avançades i modularitat** → 01 · Recursivitat

---

📬 La idea en una frase

> **La recursivitat és quan una funció es crida a si mateixa per a resoldre un problema resolent una versió més petita de si mateix, fins a arribar a un cas base que ho deté tot.**

En la U07 vas veure que un algorisme es pot escriure amb bucles. Però hi ha problemes que s'*expressen* de manera molt més natural dient "aquest problema és com el mateix problema, però més petit". Per a aquells casos va néixer la recursivitat: la tècnica estrella d'aquesta unitat.

🔁 El cicle de la vida recursiva

Tota funció recursiva necessita **dues parts imprescindibles**. Si et falta una, estàs mort:

> [!NOTI]
>
> - **Cas base**: la condició que deté la recursió. Sense això, el teu programa s'executa fins que la JVM es cansa i et llança un `StackOverflowError`.
> - **Cas recursiu**: la crida a si mateixa, normalment amb una versió més petita del mateix problema.

L'estructura general és sempre la mateixa:

```java
public static tipus funcio(parametres) {
    if (/* condicio de parada */) {
        return /* valor base */;
    } else {
        // fer alguna cosa amb el problema actual
        return funcio(/* versio mes xicoteta */);
    }
}
```

Pensa-ho com a netejar la teva habitació (suposant que algun dia ho facis): en lloc de netejar-la sencera de cop, agafes una cosa del terra i torni a cridar la mateixa funció “netejarHabitacio” amb el que queda. Eventualment no queda res, i has acabat.

📚 La pila de crides: l’origen de tota la diversió

Cada vegada que una funció es crida, Java reserva un trosset de memòria en l’**stack** (la pila). Aquell trosset es diu **stack frame** i guarda: els paràmetres de la funció, les variables locals, i l’adreça de tornada per quan la funció acabi.

Si crides una funció 5 vegades, tens 5 frames en l’stack. Si la crides 10.000 vegades… bum. `StackOverflowError`.

```java
public class StackExplorer {

    static int prof = 0;

    static void recursiu() {
        prof++;
        System.out.println("Crida numero: " + prof);
        recursiu(); // ¡no hi ha cas base!
    }

    public static void main(String[] args) {
        try {
            recursiu();
        } catch (StackOverflowError e) {
            System.out.println("Stack explotat a la crida: " + prof);
        }
    }
}
```

Prova-ho. Veuràs que el nombre varia segons la teva màquina i la JVM: normalment entre 10.000 i 20.000 crides. No és infinit. Res no ho és.

> [!WARNING] Sense cas base, no hi ha pietat. L'stack té un límit i Java no et salvarà. Cada frame ocupa espai, i quan el got es desborda, la JVM diu "fins aquí hem arribat". Atrapa el `StackOverflowError` només per a inspeccionar (com dalt), mai per a "arreglar" una recursió trencada.

🥊 Recursivitat vs iteració: el duel

Aspecte	Recursivitat	Iteració (bucles)
Llegibilitat	Molt elegant per a problemes jeràrquics	Més verbosa però clara
Memòria	Gasta stack (cada crida = frame nou)	Només variables locals
Velocitat	Més lenta (overhead de crides)	Més ràpida

Aspecte	Recursivitat	Iteració (bucles)
Stack overflow	Risc real si la profunditat és alta	No aplica
Casos ideals	Arbres, grafs, backtracking, divide i venceràs	Recorreguts lineals, processament simple

La regla d'or: **usa recursivitat quan el problema sigui inherentment recursiu** (estructures jeràrquiques, expressions anidades, ordenació avançada). Per a la resta, un `for` de tota la vida. Ningú no et donarà una medalla per fer una suma d'array amb recursió.

🌰 Un primer exemple complet

Abans dels clàssics del punt 2, un exemple mínim perquè vegin l'estructura completa. Comptar fins a `n`:

```
public class CompteArrere {

    static void compte(int n) {
        if (n < 0) return;              // cas base: no hi ha nombres negatius
per imprimir
        System.out.println(n);
        compte(n - 1);                  // cas recursiu: el mateix problema, un
pas més xicotet
    }

    public static void main(String[] args) {
        compte(3);
    }
}
```

Sortida:

```
3
2
1
0
```

Fixa't: primer baixa imprimint 3, 2, 1, 0, i quan `n < 0`, la crida retorna i tots els frames es tanquen un darrere l'altre. La pila es buida en ordre invers al qual es va omplir.

> [!TIP] El 90% dels bugs de recursió són dos: **oblidar el cas base** o **no reduir el problema** (cridar-se a si mateixa amb els mateixos paràmetres). Si la teva recursió no acaba o no avança, dibuixa l'arbre de crides en un paper. El paper i el boli continuen sent les teves millors eines de depuració.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines dues parts necessita tota funció recursiva?
2. Què llança la JVM si la recursió mai no acaba?
3. Què guarda un *stack frame*?
4. Per què una suma d'array amb bucle sol ser millor que amb recursió?

> 🔄 **Respostes**
>
> 1. El **cas base** (deté la recursió) i el **cas recursiu** (la crida a si mateixa amb un problema més petit).
> 2. Un **`StackOverflowError`** quan l'stack s'ompli de frames.
> 3. Els **paràmetres**, les **variables locals** i l'**adreça de tornada** per a continuar quan la crida acabi.
> 4. Perquè cada crida recursiva crea un frame nou en l'stack: **més memòria i més overhead** que un simple `for`.

✅ Resum en 3 frases

1. La recursivitat és una **funció que es crida a si mateixa** amb una versió més petita del problema.
2. Sense **cas base** no hi ha detenció: l'stack s'ompli i arriba el `StackOverflowError`.
3. Usa recursió quan el problema sigui **inherentment recursiu**; per a allò lineal, els bucles continuen guanyant.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Cas base	La condició que deté la recursió
Cas recursiu	La crida a si mateixa amb un problema menor
Stack frame	El trosset de memòria de cada crida (paràmetres, locals, retorn)
Call stack	La pila de frames pendents
`StackOverflowError`	El que passa quan omplis la pila
Recursió de cua	Quan la crida recursiva és l'últim que fa la funció

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · Recursivitat: exemples clàssics

02 — Recursivitat: exemples clàssics

Factorial, Fibonacci, palíndroms i torres de Hanoi: els quatre genets de l'apocalipsi recursiu 🧮

> 🗺️ **Estàs en: ⚡ UO8 · Algorítmica II: tècniques avançades i modularitat** → 02 · Recursivitat: exemples clàssics

📬 La idea en una frase

> **Aquests quatre exemples et perseguiran la resta de la teva carrera: apareixen en exàmens, entrevistes tècniques i converses d'ascensor amb altres programadors, així que més val dominar-los.**

En el punt 1 vas veure la teoria: cas base, cas recursiu i la pila. Ara toca el museu dels clàssics. Cada un t'ensenya una lliçó diferent, des de l'elegància lineal del factorial fins a l'horror exponencial del Fibonacci ingenu.

🧮 Factorial: el "Hola Món" de la recursivitat

```java
public class Factorial {

    static long fact(int n) {
        if (n <= 1) return 1;          // cas base: 0! i 1! valen 1
        return n * fact(n - 1);        // cas recursiu
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5));   // 120
        System.out.println("fact(0) = " + fact(0));   // 1
        System.out.println("fact(7) = " + fact(7));   // 5040
    }
}
```

Què passa quan cridem `fact(4)`? Anem pas a pas:

```
fact(4) → 4 * fact(3)
       → 4 * (3 * fact(2))
       → 4 * (3 * (2 * fact(1)))
       → 4 * (3 * (2 * 1))
       → 4 * (3 * 2)
       → 4 * 6
       → 24
```

Primer “baixa” fins al cas base, després “puja” fent les multiplicacions. Com un ascensor que baixa al soterrani i torna a pujar parant a cada porta. La complexitat és **O(n)**: una crida per cada nombre.

> [!NOTI] Ull amb `long`: `fact(20)` ja és 2.432.902.008.176.640.000, quasi al límit de `long`. A partir del 21, desbordes. El factorial creix **molt** de pressa.

🐰 Fibonacci ingenu: no ho facis a casa

```java
public class FiboNaive {

    static long fibo(int n) {
        if (n <= 1) return n;
        return fibo(n - 1) + fibo(n - 2);
    }

    public static void main(String[] args) {
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(40) = " + fibo(40));
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}
```

Executa això. Seran uns segons d’infern. El problema? `fibo(40)` genera un **arbre de crides monstruós**. Cada crida es bifurca en dos, que es bifurquen en dos, etc. El nombre total de crides és aproximadament **O(2ⁿ)**.

```
                        fibo(5)
                       /       \
                  fibo(4)       fibo(3)
                 /       \      /     \
           fibo(3)    fibo(2) fibo(2) fibo(1)
           /    \      /   \   /   \
      fibo(2) fibo(1) ...  ... ...  ...
      /    \
  fibo(1) fibo(0)
```

Compta les vegades que es crida `fibo(1)`. Exacte, massa. És com preguntar al teu company una vegada i una altra la mateixa pregunta esperant una resposta diferent. Això, senyor meu, és bogeria.

> [!TIP] Aquest és l'exemple perfecte per a entendre per què **l'eficiència importa**. Un algorisme que es veu bonic en codi pot ser un desastre en temps d'execució. El Fibonacci ingenu és el rei dels algorismes bonics però inútils per a nombres grans.

⚡ Fibonacci optimitzat: amb memoització, ara sí 🔗

```java
public class FiboOptimizado {

    static long[] memo;

    static long fibo(int n) {
        if (n <= 1) return n;
        if (memo[n] != 0) return memo[n];      // ja el calcularem abans
        memo[n] = fibo(n - 1) + fibo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        int n = 92;
        memo = new long[n + 1];
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(" + n + ") = " + fibo(n)); // instantani
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}
```

Què canvia? Guardem els resultats en un array (`memo[]`) per a no recalcular. Si ja hem calculat `fibo(10)`, el retornem directament. Cada Fibonacci es calcula **una sola vegada**.

La millora és brutal: de **O(2ⁿ) a O(n)**. Aquella és la pujada de nivell més gran des que vas passar de pedra a pokéball.

> [!NOTI] Això es diu **memoització** (sí, sense la "r"). Consisteix en: "si ja ho vaig calcular, no ho torni a calcular; ho guardi en un array o mapa i ho reutilitzi". És la base de la **programació dinàmica**, que veuràs en unitats més avançades.

🔍 Palíndroms: la paraula mirall

Un palíndrom és una paraula que es llegeix igual d'endavant cap enrere: *reconèixer*, *salas*, *ana*. Com es comprova amb recursió?

```
public class Palindromo {

    static boolean esPalindromo(String s, int inicio, int fin) {
        if (inicio >= fin) return true;                    // cas base: queden 0
o 1 lletres
         if (s.charAt(inicio) != s.charAt(fin)) return false; // cas base: les
puntes no coincideixen
         return esPalindromo(s, inicio + 1, fin - 1);         // cas recursiu:
"pelem" les puntes
    }

    public static void main(String[] args) {
        String[] pruebas = {"reconocer", "salas", "hola", "anana", "a"};
        for (String s : pruebas) {
            System.out.println(s + " -> " + esPalindromo(s, 0, s.length() - 1));
        }
    }
}
```

Sortida:

```
reconocer -> true
salas -> true
hola -> false
anana -> true
a -> true
```

La idea és elegant: compares la primera i l'última lletra; si no coincideixen, no és palíndrom; si coincideixen, repeteixes amb les lletres de dins. Com a pelar una ceba, però de lletres.

🗼 Torres de Hanoi: el rei dels clàssics

El mite: tres varetes i una pila de discos de mida creixent. Cal moure tots els discos de la vareta A a la C, amb dues regles: només es mou un disc alhora i mai no pot quedar un disc gran sobre un de petit. La llegenda diu que quan els monjos acaben, el món s'acabarà (amb 64 discos, no ens preocupem).

```
public class Hanoi {

    static void mover(int n, char origen, char destino, char auxiliar) {
        if (n == 1) {
            System.out.println("Mueve disco 1 de " + origen + " a " + destino);
            return;                                    // cas base
        }
          mover(n - 1, origen, auxiliar, destino);         // els n-1 discos de
dalt, a la vareta auxiliar
           System.out.println("Mueve disco " + n + " de " + origen + " a " +
destino);
         mover(n - 1, auxiliar, destino, origen);        // i ara els n-1 discos,
al damunt
    }

    public static void main(String[] args) {
        mover(3, 'A', 'C', 'B');
    }
}
```

Amb 3 discos, la sortida és:

```
Mueve disco 1 de A a C
Mueve disco 2 de A a B
Mueve disco 1 de C a B
Mueve disco 3 de A a C
Mueve disco 1 de B a A
Mueve disco 2 de B a C
Mueve disco 1 de A a C
```

Són $2^n - 1$ moviments. Amb 3 discos: 7. Amb 64 discos: 18.446.744.073.709.551.615. Al ritme d'un moviment per segon, l'univers es mor abans. Hanoi és l'exemple que la recursió pot resoldre en 6 línies el que iterativament et faria patir de veritat.

> [!WARNING] Hanoi és **exponencial** ($O(2^n)$) per naturalesa: no és que ho haguessis implementat malament, és que el problema *és* exponencial. La moralina és una altra: una solució recursiva curta i clara val més que un bucle il·legible que intenta emular-la amb piles manuals.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina és la complexitat del factorial recursiu?
2. Quantes vegades es crida `fibo(1)` en calcular `fibo(5)` amb el mètode ingenu? (pista: mira-ho en l'arbre)
3. Què fa la memoització per a millorar Fibonacci?
4. Quants moviments necessita Hanoi amb 4 discos?

> 🔄 **Respostes**
>
> 1. **O(n)**: una crida per cada nombre, i cada crida fa treball constant.
> 2. **5 vegades** (a l'arbre de la unitat es veu clar: `fibo(1)` apareix en 5 fulles).
> 3. Guarda cada resultat ja calculat en un array per a **no repetir càlculs**: de $O(2^n)$ a O(n).
> 4. **15** moviments ($2^4 - 1$).

✅ Resum en 3 frases

1. El **factorial** (O(n)) i els **palíndroms** són els exemples "lineals": recorren el problema cap al cas base sense sorpreses.
2. El **Fibonacci ingenu** és $O(2^n)$: els números exploten perquè repeteix càlculs; la **memoització** el deixa en O(n).
3. Les **torres de Hanoi** demostren que la recursió expressa en poques línies problemes que iterativament són un infern.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Memoització	Guardar resultats ja calculats per a reutilitzar-los
Palíndrom	Paraula que es llegeix igual d'endavant cap enrere: *salas*
Recursió lineal	La que fa una sola crida recursiva per invocació (factorial)
Recursió exponencial	La que es bifurca en diverses crides (Fibonacci, Hanoi)
`long`	Tipus per a números grans… fins que el factorial el desborda

03 — Divide i venceràs

El patró més antic de la informàtica: divideix el problema, conquista cada part i combina ⚔️

> 🗺️ **Estàs en: ⚡ U08 · Algorítmica II: tècniques avançades i modularitat** → 03 · Divide i venceràs

---

📬 La idea en una frase

> **Divide i venceràs (divide & conquer) consisteix a partir un problema en subproblemes més petits, resoldre cada un per separat (normalment amb recursió) i combinar les solucions.**

L'estratègia és tan antiga com Juli Cèsar, però aplicada a algorismes continua sent igual d'efectiva. I sí, en la U07 ja la vas usar sense saber-ho: la **cerca binària** parteix l'array per la meitat i descarta una part en cada pas. Aquell era el germen. Ara li posaràs nom i cognoms.

🧩 Els tres passos del patró

1. **Dividir** el problema en subproblemes més petits i manejables.
2. **Conquistar** cada subproblema recursivament (crides recursives).
3. **Combinar** les solucions dels subproblemes per a obtenir la solució del problema original.

```
Input gran
     │
       ├──Dividir──→ Subproblema A     → Conquistar (recursiu) → Combinar ──→
Output
     └──Dividir──→ Subproblema B    → Conquistar (recursiu) ─┘
```

La clau està que **cada subproblema és una còpia més petita de l'original**: per això la recursió encaixa tan bé. I el cas base és el subproblema tan petit que ja es resol directament.

📊 Quan paga la pena?

No tot problema es beneficia de dividir. La màgia apareix quan el cost de **combinar** és barat i la divisió redueix de debò el treball. Un clàssic d'aquesta unitat:

Algorisme	Divideix	Conquista	Combina	Complexitat
Cerca binària	Parteix per la meitat	Descartas una meitat	Res	O(log n)
Mergesort	Parteix per la meitat	Ordenes cada meitat	**Fusiones**	O(n log n)
Quicksort	Particiona amb pivot	Ordenes cada zona	Res (ja està)	O(n log n) promig

> [!NOTI] Fixa't en la fila de Quicksort: el treball "car" ocorre en la **divisió** (particionar al voltant del pivot), i la combinació és gratis. En Mergesort passa el contrari: la divisió és trivial (tallar pel mig) i el treball està en **combinar** (fusionar). Són dos sabors del mateix patró.

💡 Un exemple senzill: el màxim d'un array

Encara que el màxim es resol amb un bucle en una línia, vegem com ho enfocaria divide i venceràs perquè vegin el patró en la seva mínima expressió:

```
public class MaximoDyV {

    static int maximo(int[] arr, int izq, int der) {
        if (izq == der) return arr[izq];          // cas base: un sol element

        int mid = (izq + der) / 2;
        int maxIzq = maximo(arr, izq, mid);       // conquistar meitat esquerra
        int maxDer = maximo(arr, mid + 1, der);   // conquistar meitat dreta

        return Math.max(maxIzq, maxDer);          // combinar
    }

    public static void main(String[] args) {
        int[] datos = {3, 9, 2, 7, 5, 8, 1, 6};
         System.out.println("Máximo: " + maximo(datos, 0, datos.length - 1)); //
9
    }
}
```

Espera… això és un bucle disfressat, no? Sí: la seva complexitat és O(n), igual que el bucle, perquè cada element es visita una vegada. La moralina és que **divide i venceràs no sempre guanya**: guanya quan la divisió evita treball repetit (com ordenar o buscar), no quan simplement reparteix el mateix treball.

> [!TIP] Regla pràctica per a decidir: si el bucle simple és clar i ràpid, usa el bucle. Divide i venceràs és per a quan *partir el problema* et permet **descartar** o **reutilitzar** treball. La cerca binària descarta mitja array; el factorial no descarta res.

🌳 Aplicacions famoses

Una vegada entens el patró, comences a veure'l per tot arreu. És com quan aprens una paraula nova i de sobte la veus en cada racó:

- **Cerca binària**: O(log n). Ja la coneixes de la UO7.
- **Mergesort i Quicksort**: els dos reis de l'ordenació, que veuràs en els punts 4 i 5.
- **Potència ràpida**: calcular a^n en O(log n) en lloc de O(n).
- **Multiplicació de matrius (Strassen)**: submatrius en lloc de la triple operació clàssica.
- **FFT (Transformada Ràpida de Fourier)**: la base del JPEG, el MP3 i el WiFi.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quins són els tres passos del patró divide i venceràs?
2. En què es diferencia el treball de Quicksort i Mergesort dins del patró?
3. Per què el "màxim amb divide i venceràs" no millora el bucle simple?
4. La cerca binària, quin pas del patró "evita" (no el necessita)?

> 🔄 **Respostes**
>
> 1. **Dividir** en subproblemes, **conquistar** cada un (recursió) i **combinar** les solucions.
> 2. Quicksort fa el treball en la **partició** (dividir) i la combinació és gratis; Mergesort divideix trivial i fa el treball en la **fusió** (combinar).
> 3. Perquè no **descartar** treball: visita tots els elements una vegada, igual que el bucle → O(n) en tots dos.

> 4. El pas de **combinar** (no ha de combinar res: descarta mitja array i continua).

✅ Resum en 3 frases

1. Divide i venceràs és el patró de **dividir, conquistar i combinar**, amb cada subproblema més petit resolt per recursió.
2. El cas base és el subproblema trivial; la complexitat depèn de quant treball t'estalvie la **divisió** (descartar o reutilitzar).
3. No sempre guanya al bucle: només brilla quan partir el problema **evita treball repetit**, com en ordenació i cerca.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Divide & Conquer | Dividir, conquistar i combinar |
> | Subproblema | Una còpia més petita del problema original |
> | Cas base | El subproblema que es resol directament |
> | Combinar | Unir les solucions parcials en la solució final |
> | Particionar | Repartir elements al voltant d'un pivot (Quicksort) |
> | Fusionar | Barrejar dues llistes ordenades en una (Mergesort) |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Recursivitat: exemples clàssics · **Següent:** 04 · Quicksort

04 — Quicksort: el ràpid

L'algorisme d'ordenació més famós del món, amb la seva partició, el seu pivot i les seves manies 🚀

> 🗺️ **Estàs en:** ⚡ **U08 · Algorítmica II: tècniques avançades i modularitat** → 04 · Quicksort

---

📬 La idea en una frase

> **Quicksort tria un pivot, parteix l'array en menors i majors al seu voltant, i repeteix recursivament en cada meitat: divide i venceràs en estat pur.**

Creat per **Tony Hoare** el 1959. Sí, té més anys que els teus pares. I continua sent l'algorisme d'ordenació més usat del món. Per alguna cosa serà.

🎯 Com funciona

1. Triem un **pivot** (un element de l'array).
2. Col·loquem tots els elements **menors** que el pivot a la seva esquerra i els **majors** a la seva dreta. Això es diu **particionar**.
3. Apliquem el mateix procés recursivament a les dues zones (esquerra i dreta del pivot).

Quan l'array té 0 o 1 elements… ja està ordenat. Cas base.

🛠️ Implementació

```java
public class Quicksort {

    static void qs(int[] arr, int izq, int der) {
        if (izq >= der) return;   // cas base: 0 o 1 elements

        int pivote = arr[(izq + der) / 2];  // triem el del mig
        int i = izq, j = der;

        // partició
        while (i <= j) {
            while (arr[i] < pivote) i++;
            while (arr[j] > pivote) j--;
            if (i <= j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
                j--;
            }
        }

        // crides recursives a cada zona
        qs(arr, izq, j);
        qs(arr, i, der);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        qs(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Execució pas a pas amb `{3, 1, 4, 1, 5, 9, 2, 6}` (primera partició):

```
Array inicial:  [3, 1, 4, 1, 5, 9, 2, 6]
Pivot = arr[(0 + 7) / 2] = arr[3] = 1
Posicions i=0, j=7

1. i no avança (arr[0]=3 no és < 1); j retrocedeix fins a 3 (arr[3]=1 no és > 1)
2. i(0) <= j(3): intercanviem arr[0] i arr[3] → [1, 1, 4, 3, 5, 9, 2, 6]; i=1, j=2
3. i no avança (arr[1]=1 no és < 1); j retrocedeix a 1 (arr[2]=4 > 1)
4. i(1) <= j(1): s'intercanvia amb si mateix; i=2, j=0 → i > j, fi de la partició
5. Recursió esquerra: segment [0..0] (un sol element, ja ordenat)
6. Recursió dreta: segment [2..7] → [4, 3, 5, 9, 2, 6], amb pivot 5
...
```

Fixa't: el pivot (1) ha quedat ben col·locat i l'array s'ha partit en `[1]` i `[4, 3, 5, 9, 2, 6]`. Cada zona s'ordena igual, recursivament. Si tries un pivot diferent, els intercanvis canvien, però el patró és el mateix.

> [!TIP] El secret de Quicksort està en la **partició**. Si aconsegueixes que els elements es reparteixin més o menys equilibradament, l'algorisme vola. Si no... prepara els $O(n^2)$.

🎚️ Elecció del pivot

Estratègia	Avantatge	Desavantatge
Primer element	Simple	Pèssim si l'array ja està ordenat
Últim element	Simple	Pèssim si l'array ja està ordenat
Element central	Millor equilibri	Segueix tenint casos dolents
Mediana de tres (primer, mig, últim)	Molt robust	Un poc més de càlcul
Aleatori	Evita el cas pitjor en la pràctica	Aleatorietat no determinista

> [!WARNING] Si tries sempre el primer element com a pivot i l'array ja està ordenat, Quicksort es torna més lent que una tortuga amb ressaca: **O(n²)**, literalment pitjor que un `for` anidat cutre. Per això en producció ningú usa "primer element" a seques.

📊 Complexitat

- **Cas mitjà**: O(n log n) — quasi sempre.
- **Millor cas**: O(n log n) — quan el pivot divideix sempre en meitats iguals.
- **Pitjor cas**: O(n²) — quan el pivot és sempre el menor o el major (array ja ordenat amb mal pivot).
- **Memòria**: O(log n) extra (per l'stack de les crides recursives). No necessita arrays auxiliars: ordena *in-place*.

🚫 És estable?

No. Durant la partició, dos elements iguals poden intercanviar-se de posició. Si necessites estabilitat (conservar l'ordre original dels elements iguals), millor usa Mergesort.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines són les tres parts de Quicksort?
2. Quin és el pitjor cas de Quicksort i quan ocorre?
3. Quina estratègia de pivot fa que el pitjor cas "quasi mai" ocorri en la pràctica?
4. És Quicksort estable?

> 🔄 **Respostes**
>
> 1. Triar **pivot**, **particionar** (menors a un costat, majors a l'altre) i **cridar recursivament** cada zona.
> 2. **O(n²)**, quan el pivot és sempre el menor o el major de la seva zona (típic amb array ordenat i pivot = primer element).
> 3. La **mediana de tres** o el **pivot aleatori**: reparteixen l'array de forma equilibrada i eviten els casos dolents en la pràctica.

> 4. **No**. En la partició, elements iguals poden canviar el seu ordre relatiu.

✅ Resum en 3 frases

1. Quicksort **particiona** al voltant d'un pivot i repeteix recursivament en cada zona: divide i venceràs.
2. El seu **cas mitjà** és O(n log n) i el **pitjor cas** $O(n^2)$, que s'evita amb una bona elecció de pivot.
3. És **in-place** (memòria O(log n)) i **no estable**; el treball important ocorre en la partició.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Pivot | L'element que divideix l'array en menors i majors |
> | Partició | Repartir elements al voltant del pivot |
> | In-place | Ordenar sense arrays auxiliars, movent el propi array |
> | Estable | Que conserva l'ordre original dels elements iguals |
> | O(n log n) | Complexitat "quasi lineal": l'estàndard de l'ordenació moderna |

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Divide i venceràs · **Següent:** 05 · Mergesort

05 — Mergesort: el fiable

L'algorisme que sempre compleix el que promet: O(n log n) passi el que passi 🤝

> 🗺️ **Estàs en: ⚡ U08 · Algorítmica II: tècniques avançades i modularitat** → 05 · Mergesort

---

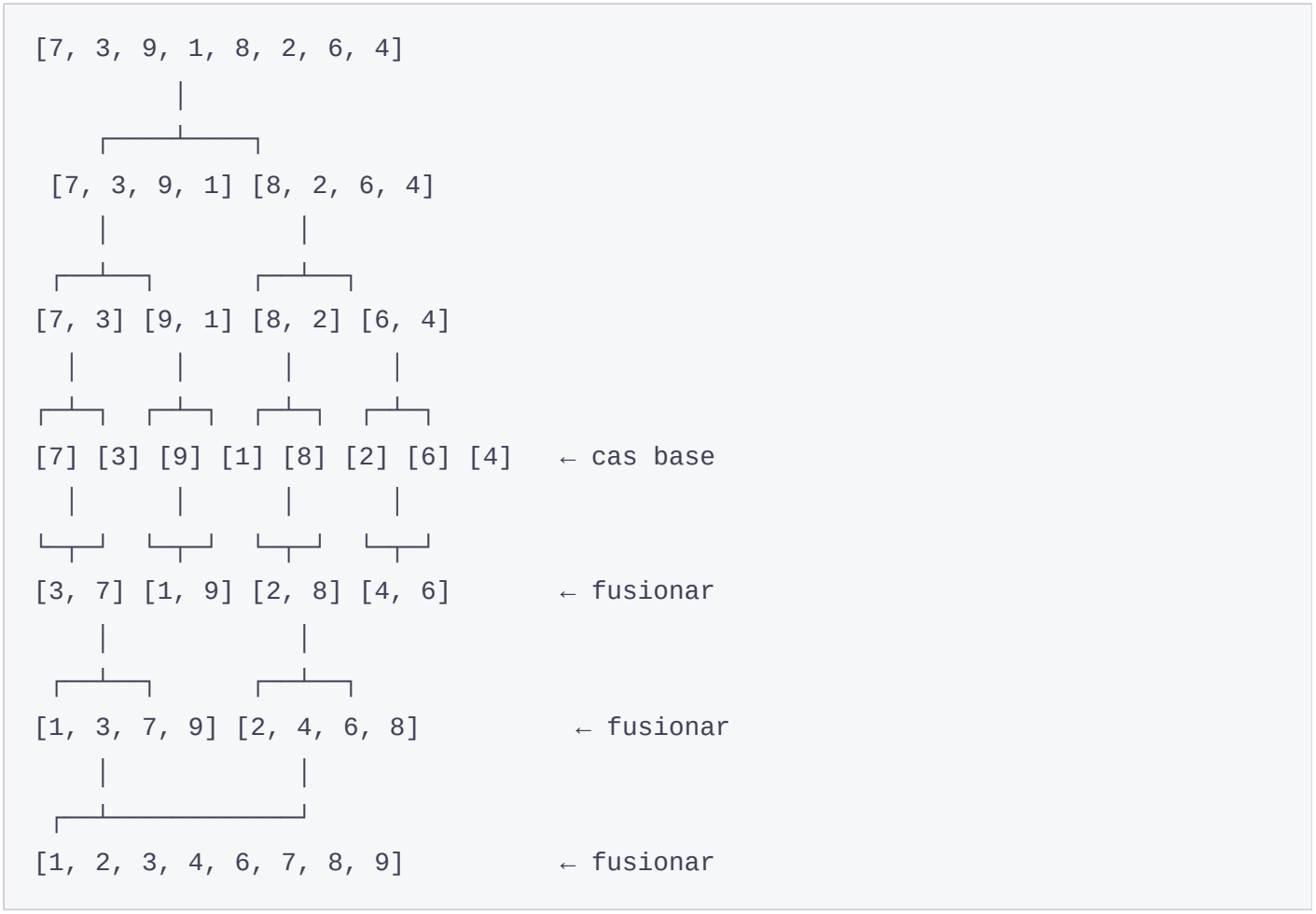
📬 La idea en una frase

> **Mergesort divideix l'array per la meitat, ordena cada meitat recursivament i després fusiona les dues meitats ordenades en una de sola: el "divide i venceràs" sense trampes.**

Creat per **John von Neumann** el 1945. Sí, el mateix de l'arquitectura d'ordinadors. El tio no parava.

🎯 Com funciona

1. **Dividir** l'array en dues meitats (per la meitat exactament; no cal triar pivot).
2. **Ordenar** cada meitat recursivament.
3. **Fusionar** (merge) les dues meitats ordenades en un únic array ordenat.



🛠️ Implementació

```java
public class Mergesort {

    static void ms(int[] arr, int izq, int der) {
        if (izq >= der) return;  // cas base

        int mid = (izq + der) / 2;
        ms(arr, izq, mid);                  // ordenar meitat esquerra
        ms(arr, mid + 1, der);              // ordenar meitat dreta
        fusionar(arr, izq, mid, der);         // combinar les dues meitats
    }

    static void fusionar(int[] arr, int izq, int mid, int der) {
        int[] tmp = new int[der - izq + 1];
        int i = izq, j = mid + 1, k = 0;

        // comparar elements de les dues meitats
        while (i <= mid && j <= der)
            tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

        // copiar el que quede de la meitat esquerra
        while (i <= mid) tmp[k++] = arr[i++];

        // copiar el que quede de la meitat dreta
        while (j <= der) tmp[k++] = arr[j++];

        // copiar l'array temporal a l'original
        System.arraycopy(tmp, 0, arr, izq, tmp.length);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        ms(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

[!TIP] La **fusió** és la part interessant. Imagina dos pilons de cartes ja ordenats: compares les cartes superiors i te'n portes la menor, una vegada i una altra. Com dues cues de supermercat que s'uneixen en una caixa i sempre passa el primer de cada cua.

📊 Complexitat

- **Sempre**: O(n log n). No importa com estigui l'array. Mergesort no té cas dolent.
- **Memòria**: O(n) extra per l'array temporal `tmp`. Aquell és el seu punt feble.

L'estructura és la típica de divide i venceràs: hi ha $\log n$ nivells de divisió, i en cada nivell es fa un treball total de $O(n)$ (fusionar tots els fragments d'aquell nivell). Resultat: $n \cdot \log n$.

✅ És estable?

Sí. Quan dos elements són iguals, el de l'esquerra va primer en la fusió (fixa't en el <= de la condició). Això manté l'ordre original dels elements iguals, una cosa que Quicksort no pot garantir.

> [!NOTI] L'estabilitat no és un concepte teòric avorrit. És important quan ordenes per múltiples criteris. Per exemple, si ordenes una llista d'alumnes per nota i després per nom, vols que els que tenen la mateixa nota mantinguin l'ordre alfabètic. Mergesort t'ho dona. Quicksort et fa plorar.

🏆 Quan brilla Mergesort

- Quan necessites **estabilitat** a tot preu.
- Quan l'array està **quasi ordenat** o ple de duplicats (no li afecta).
- Quan les dades no caben en memòria: el **mergesort extern** (fusionant trossos en disc) és la base de l'ordenació de les bases de dades.

I el seu punt feble: si la memòria és justa (sistemes encastats, mòbils amb poca RAM), l'array temporal de O(n) pot doldre.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin és el cas dolent de Mergesort?
2. Quin és el seu punt feble davant de Quicksort?

3. Per què és estable?
4. En què es diferencia la seva “divisió” de la de Quicksort?

> 🔄 **Respostes**
>
> 1. **No en té**: sempre és O(n log n), passi el que passi l’array.
> 2. La **memòria extra** de O(n) per l’array temporal `tmp`.
> 3. Perquè en la fusió, quan dos elements són iguals, es copia primer el de la meitat esquerra (el <=).
> 4. Quicksort parteix al voltant d’un **pivot**; Mergesort talla **per la meitat exacta**, sense triar pivot.

✅ Resum en 3 frases

1. Mergesort **divideix per la meitat**, ordena cada meitat i **fusiona** les meitats ordenades.
2. És **sempre O(n log n)** i **estable**, a canvi de O(n) de memòria extra.
3. És l’opció fiable quan importa l’estabilitat o l’ordre inicial és problemàtic.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Fusió (merge)	Combinar dues meitats ordenades en una
Estable	Conserva l’ordre original dels elements iguals
Divide i venceràs	El patró de dividir, conquistar i combinar
Memòria O(n)	Necessita un array auxiliar de la mateixa mida
Mergesort extern	Variant que ordena dades que no caben en RAM (discs)

📚 Tornar a l’índex de la unitat · **Anterior:** 04 · Quicksort · **Següent:** 06 · Comparació: quan usar cada ordenació

06 — Comparació: quan usar cada ordenació

La taula de decisió definitiva per a triar algorisme com un professional ⚖️

> 🗺️ **Estàs en: ⚡ U08 · Algorítmica II: tècniques avançades i modularitat** → 06 · Comparació: quan usar cada ordenació

📬 La idea en una frase

> **No existeix el "millor algorisme d'ordenació": existeix el millor per al teu cas concret, i triar-lo bé és tan important com saber implementar-lo.**

Ja has vist Quicksort i Mergesort, i en la U07 vas sobreviure a bombolla i inserció. Ara toca la pregunta d'examen, d'entrevista i de la vida real: *quin usi?* La resposta sempre comença igual: *depèn*.

🧭 La taula de decisió

Situació	Tria
Array petit (< 50 elements)	Tant se val, usa `Arrays.sort()`
Array gran amb dades a l'atzar	**Quicksort** (anirà genial de mitjana)
Array gran, quasi ordenat o amb molts duplicats	**Mergesort** o Quicksort amb mediana de tres
Necessites estabilitat (ordre relatiu)	**Mergesort**
La memòria és justa (sistema encastat, mòbil)	**Quicksort** (O(log n) vs O(n))
No et paguen per pensar	`Arrays.sort()` i a seguir amb la teva vida
L'array és enorme i les dades estan en disc	Mergesort extern (el de les bases de dades)

> [!WARNING] Mai, baix cap concepte, uses **Bombolla (Bubble Sort)** en producció. És $O(n^2)$, lent, i els teus companys t'odiaran. És com usar un caragol per a repartir pizzes: existeix, però no hauria.

🥊 El duel dels titans

	Quicksort	Mergesort
Complexitat mitjana	O(n log n)	O(n log n)
Pitjor cas	$O(n^2)$ (evitable amb bon pivot)	O(n log n) sempre
Memòria extra	O(log n)	O(n)
Estable	No	Sí
Treball principal	En la **partició**	En la **fusió**
Ideal quan	Velocitat i poca memòria	Estabilitat i garanties

En resum: Quicksort és el velocista amb mal geni; Mergesort és el maratonià fiable. Quicksort sol guanyar en velocitat pràctica amb arrays grans en memòria; Mergesort guanya quan no et pots permetre un mal dia.

🕵️ Què fa Java per dins?

En Java, `Arrays.sort()` no és un algorisme, són dos segons el tipus:

- Per a **tipus primitius** (`int[]`, `double[]`…): usa **Dual-Pivot Quicksort**, una versió millorada amb dos pivots que redueix el pitjor cas i accelera el promig.
- Per a **objectes** (`String[]`, `ArrayList`…): usa **TimSort**, una mescla de Mergesort i Insertion Sort. Estable, i rapidíssima amb dades quasi ordenades.

Per això "usa `Arrays.sort()`" és quasi sempre la resposta correcta: Java ja va triar bé per tu.

📊 Rendiment en la pràctica

Algorisme	n=10	n=100	n=1.000	n=10.000	n=100.000	n=1.000.000
Quicksort	~0 ms	~0 ms	~0 ms	~1 ms	~15 ms	~120 ms
Mergesort	~0 ms	~0 ms	~0 ms	~2 ms	~20 ms	~150 ms
Bombolla (per a plorar)	~0 ms	~1 ms	~100 ms	~10.000 ms	no esperis	no esperis

Els temps són orientatius i depenen del maquinari, però la *tendència* és el que importa: amb 10.000 elements, Quicksort i Mergesort continuen en "mil·lisegons" mentre bombolla ja se n'ha anat a prendre un cafè. Amb 100.000, bombolla ni es presenta.

🧠 La lògica que hi ha darrere

Recorda la regla d'or de la complexitat: **O(n log n) quasi sempre guanya a O(n²)**, i la diferència es nota brutalment quan n creix. Per a un milió d'elements:

- O(n log n): ~20 milions d'operacions → un obrir i tancar d'ulls.
- O(n²): ~1 bilió d'operacions → l'ordinador demana la jubilació.

Així que la decisió no és "quin algorisme?", sinó "què necessito garantir?". I això, amic meu, és pensament d'enginyer: tria l'eina pel **context**, no per el costum.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin algorisme triaries per a ordenar 10.000 ints a l'atzar en un servidor amb RAM de sobres?
2. I si necessites estabilitat per a una ordenació per dos criteris?
3. Què usa `Arrays.sort()` per a primitius? I per a objectes?
4. Quant li costa a bombolla ordenar un milió d'elements?

> 🔄 **Respostes**

1. **Quicksort** (o `Arrays.sort()` directament): dades a l'atzar, gran, sense restricció de memòria.
2. **Mergesort** (o TimSort via `Arrays.sort()` sobre objectes): necessita mantenir l'ordre relatiu.
3. **Dual-Pivot Quicksort** per a primitius i **TimSort** per a objectes.
4. **O(n²)** → ~1 bilió d'operacions. Que criden l'assegurança.

✅ Resum en 3 frases

1. No hi ha un "millor" algorisme universal: la tria depèn de la **mida**, l'**ordre inicial**, la **memòria** i l'**estabilitat** que necessites.
2. **Quicksort** guanya en velocitat pràctica i memòria; **Mergesort** guanya en garanties i estabilitat.
3. En Java, `Arrays.sort()` ja tria bé per tu: **Dual-Pivot Quicksort** per a primitius i **TimSort** per a objectes.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Dual-Pivot Quicksort	Quicksort amb dos pivots (Java, primitius)
TimSort	Mescla de Mergesort i Insertion Sort (Java, objectes)
Estabilitat	Conservar l'ordre dels elements iguals
Ordre inicial	Com arriben les dades: a l'atzar, ordenades, quasi ordenades
Mergesort extern	Ordenació de dades que no caben en RAM (bases de dades)

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Mergesort · **Següent:** 07 · Be the Code: quicksort des de zero

07 — Be the Code: quicksort des de zero

Tanca la pàgina, obre l'editor i escriu Quicksort i Mergesort tu sol (o pateix amb honor) 🕶️

> 🗺️ **Estàs en:** ⚡ **U08 · Algorítmica II: tècniques avançades i modularitat** → 07 · Be the Code: quicksort des de zero

---

📬 La idea en una frase

> **Ja has vist el codi: ara toca esborrar-lo de la memòria i escriure'l tu sol, pas a pas, perquè un algorisme només s'entén de veritat quan et dol implementar-lo.**

> 🕶️ **Don Tip:** Divide i venceràs: tria un pivot, parteix l'array en menors i majors, i repeteix recursivament. Si domines aquell patró, Quicksort és teu.

🚦 Exercici guiat 1: Quicksort

Tanca aquesta pàgina. Obre un editor de text en blanc. No miris ni una línia del que has llegit fins ara.

La firma del mètode ha de ser:

```
static void quicksort(int[] arr, int inicio, int fin)
```

Pas 1: el cas base

Quan un array (o una zona de l'array) ja està ordenat? Quan té **0 o 1 elements**. En termes d'índexs:

```
if (inicio >= fin) return;
```

Pas 2: triar el pivot

Tria un element qualsevol. El del mig és una opció decent i fàcil d'escriure:

```
int pivote = arr[(inicio + fin) / 2];
```

Pas 3: particionar amb dos punters

Dos índexs parteixen dels extrems i avancen cap al centre:

- `i` avança des de `inicio` mentre `arr[i] < pivote` (busca un element que *hauria* d'estar a la dreta).
- `j` retrocedeix des de `fin` mentre `arr[j] > pivote` (busca un element que *hauria* d'estar a l'esquerra).
- Si `i <= j`, intercanvies i continues.

Pas 4: cridar recursivament cada zona

Quan `i` i `j` es creuen, la zona esquerra és `[inicio..j]` i la dreta `[i..fin]`:

```
quicksort(arr, inicio, j);
quicksort(arr, i, fin);
```

La solució completa (intenta-ho abans de mirar)

> 🔄 Solució de Quicksort

```java
public class Quicksort {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;   // cas base

        int pivote = arr[(inicio + fin) / 2];
        int i = inicio, j = fin;

        while (i <= j) {
            while (arr[i] < pivote) i++;
            while (arr[j] > pivote) j--;
            if (i <= j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
                j--;
            }
        }

        quicksort(arr, inicio, j);
        quicksort(arr, i, fin);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Sortida: `[1, 2, 3, 4, 5, 6, 7, 8, 9]`

🚦 Exercici guiat 2: Mergesort

Mateix joc, mateixa regla: sense mirar. La firma:

```java
static void mergesort(int[] arr, int inicio, int fin)
```

Pas 1: el cas base

Igual que Quicksort: zona amb 0 o 1 elements, ja ordenada.

Pas 2: partir per la meitat

```
int mid = (inicio + fin) / 2;
```

Pas 3: ordenar cada meitat i fusionar

```
mergesort(arr, inicio, mid);
mergesort(arr, mid + 1, fin);
fusionar(arr, inicio, mid, fin);   // este mètode l'escrius al pas 4
```

Pas 4: la fusió, el cor de Mergesort

Dues meitats ordenades es combinen amb un array temporal:

```
static void fusionar(int[] arr, int inicio, int mid, int fin) {
    int[] tmp = new int[fin - inicio + 1];
    int i = inicio, j = mid + 1, k = 0;

    while (i <= mid && j <= fin)
        tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    while (i <= mid) tmp[k++] = arr[i++];
    while (j <= fin) tmp[k++] = arr[j++];

    System.arraycopy(tmp, 0, arr, inicio, tmp.length);
}
```

La solució completa (intenta-ho abans de mirar)

> 🔄 Solució de Mergesort

```java
public class Mergesort {

    static void mergesort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;

        int mid = (inicio + fin) / 2;
        mergesort(arr, inicio, mid);
        mergesort(arr, mid + 1, fin);
        fusionar(arr, inicio, mid, fin);
    }

    static void fusionar(int[] arr, int inicio, int mid, int fin) {
        int[] tmp = new int[fin - inicio + 1];
        int i = inicio, j = mid + 1, k = 0;

        while (i <= mid && j <= fin)
            tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
        while (i <= mid) tmp[k++] = arr[i++];
        while (j <= fin) tmp[k++] = arr[j++];

        System.arraycopy(tmp, 0, arr, inicio, tmp.length);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        mergesort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Sortida: `[1, 2, 3, 4, 5, 6, 7, 8, 9]`

⭐ Nivells de logro

- ⭐ **Ho tens, però has hagut de mirar el codi una vegada.** Aprovat raspadet.
- ⭐⭐ **T'ha sortit a la primera i funciona.** Ets una màquina.
- ⭐⭐⭐ **T'ha sortit a la primera, sense errors d'off-by-one, i a més has triat mediana de tres com a pivot.** No necessites aquest curs. Vés a donar una xerrada TED.

> [!TIP] Pista mental gratuïta: Quicksort necessita **tres coses** — triar un pivot, partir l'array en dues zones (menors i majors), i cridar recursivament cada costat. Si memoritzes aquell patró, Quicksort és teu per sempre. Mergesort, per la seva banda, només necessita **dues** — partir per la meitat i fusionar.

🧩 El merder

El departament de qualitat ha rebut aquest Quicksort. Alguna cosa fa mala olor. Troba els **3 errors**:

```
public class QuicksortLioso {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio > fin) return;

        int pivote = arr[fin];
        int i = inicio;

        for (int j = inicio; j < fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        quicksort(arr, inicio, i - 1);
        quicksort(arr, i + 1, fin);
    }
}
```

> 🕶️ **Don Tip:** prova a executar-lo amb `{3, 1, 4}` i mira què passa amb el cas base, el lloc del pivot i la variable `i` després del bucle.

> › 🔄 **Solució del merder**
>
> 1. **El cas base és incorrecte**: `if (inicio > fin)` hauria de ser `if (inicio >= fin)`. Amb `inicio == fin` hi ha un sol element (ja ordenat), però aquest codi continuaria "particionant" i es saltaria elements.

2. **El pivot mai no es col·loca en el seu lloc final**: després del bucle, el pivot (que està a `arr[fin]`) hauria d'intercanviar-se amb `arr[i]` abans de les crides recursives. Sense aquell intercanvi, el pivot es queda fora de la zona correcta.
3. **La partició està incompleta**: falten els `else` per als elements *majors o iguals* al pivot. Així, els duplicats del pivot es reparteixen malament i l'algorisme pot deixar d'ordenar correctament.

Amb `{3, 1, 4}` es veu clar: l'`1` no s'intercanvia amb el pivot al final i l'array queda desordenat. La lliçó: **el cas base, la posició final del pivot i els elements iguals són els tres clàssics on mor un Quicksort**.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quan retorna la recursió de Quicksort?
2. Què ocorre quan `i` i `j` es creuen en la partició?
3. Quin és el cas base de Mergesort?
4. Què fan els tres `while` de la fusió?

> 🔄 **Respostes**
>
> 1. Quan `inicio >= fin`: la zona té **0 o 1 elements** i ja està ordenada.
> 2. Que la partició ha acabat: la zona esquerra és `[inicio..j]` i la dreta `[i..fin]`.
> 3. `inicio >= fin`: zona amb **0 o 1 elements**.
> 4. El primer **comparar i copiar el menor** de les dues meitats; els altres dos **bolquen els restes** que queden de cada meitat.

✅ Resum en 3 frases

1. **Quicksort** es redueix a triar pivot, particionar amb dos punters i cridar recursivament cada zona.
2. **Mergesort** es redueix a partir per la meitat, ordenar cada meitat i fusionar amb un array temporal.

3. L'off-by-one en els casos base i els elements iguals són els errors clàssics: **prova sempre amb arrays petits a mà**.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Off-by-one | Error de "un de més o un de menys" en els índexs |
> | Partició | Repartir menors i majors al voltant del pivot |
> | Punters `i` i `j` | Índexs que avancen des dels extrems fins a creuar-se |
> | Fusió | Combinar dues meitats ordenades en una |
> | Array temporal | El `tmp` de la fusió, que costa O(n) de memòria |

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Comparació: quan usar cada ordenació · **Següent:** 08 · Altres tècniques algorísmiques

08 — Altres tècniques algorísmiques

Dos fitxes més per al teu cinturó: els algorismes voracos (greedy) i el backtracking 🧠

> 🗺️ **Estàs en:** ⚡ **U08 · Algorítmica II: tècniques avançades i modularitat** → 08 · Altres tècniques algorísmiques

---

📬 La idea en una frase

> **Recursivitat i divide i venceràs no són les úniques tècniques: els algorismes voracos prenen la decisió que millor pinta en cada pas, i el backtracking prova camins i retrocedeix quan arriba a un cul-de-sac.**

Aquesta unitat ja t'ha donat les eines estrella. Aquests dos punts extra són "per a saber que existeixen" i reconèixer-los quan els vegis, perquè apareixen en munts de problemes reals (i en entrevistes). T'avise: aquí no implementaràs res gegant, entendràs *la idea* amb exemples senzills.

🍬 Algorismes voracos (greedy)

La idea: en cada pas, pren la decisió que sembla millor *en aquell moment*, sense mirar enrere ni endavant. Com qui compra caramels i sempre agafa el més gran dels que queden.

La regla d'or: un greedy només funciona si la decisió local òptima condueix a la solució global òptima. Això no sempre passa. Quan funciona, és dels algorismes més ràpids i simples que existeixen.

Exemple clàssic: el canvi de monedes

Donat un preu i els diners entregats, tornar el canvi amb el **menor nombre de monedes**:

```java
public class CambioMonedas {

    public static void main(String[] args) {
        int[] monedas = {200, 100, 50, 20, 10, 5, 2, 1};
        int cambio = 173;   // 1,73 €

        for (int moneda : monedas) {
            int cuantas = cambio / moneda;
            cambio %= moneda;
            if (cuantas > 0) {
                    System.out.println(cuantas + " moneda(s) de " + moneda + "
céntimos");
            }
        }
    }
}
```

Sortida:

```
1 moneda(s) de 100 céntimos
1 moneda(s) de 50 céntimos
1 moneda(s) de 20 céntimos
1 moneda(s) de 2 céntimos
1 moneda(s) de 1 céntimos
```

> [!NOTI] L'algorisme voràc de la moneda funciona amb el sistema de monedes europeu (on cada moneda és múltiple de l'anterior). Amb sistemes estranys (p. ex. monedes d'1, 3 i 4), el voràc **falla**: amb canvi 6, el voràc donaria 4+1+1 (3 monedes) quan la solució òptima és 3+3 (2 monedes). La lliçó: *voràc = ràpid i simple, però no sempre òptim*.

Altres exemples famosos de greedy

- **El problema de la motxilla (versió fraccionària)**: caben només objectes "a trossos", i omplis la motxilla començant pel de millor relació valor/pes.
- **L'arbre d'expansió mínim (Prim, Kruskal)**: connectes tots els nodes gastant el mínim cable.
- **El problema del canvi**, com acabes de veure.
- **Planificació de tasques**: atendre primer la tasca que acaba abans.

🔁 Backtracking (volta enrere)

La idea: proves una opció; si arribes a un cul-de-sac, **desfàs l'últim pas** (d'aquí el nom) i proves la següent opció. És una cerca exhaustiva i intel·ligent alhora, perquè *no insisteix* en camins que ja saps que no porten enlloc.

El patró clàssic és recursiu:

```
public static void resolver(opciones, pasosDados) {
    if (solucionCompleta) { ... }        // cas base: ho has aconseguit
    if (culDeSac) { return; }            // poda: no insistir

    for (opcion : opcionesPosibles) {
        tomar(opcion);
        resolver(opciones, pasosDados + 1);   // provar el següent pas
        deshacer(opcion);                     // ¡la volta arrere!
    }
}
```

Aquell `deshacer(opcion)` és el cor del backtracking: si el camí no funciona, reculls el que vas posar i proves un altre.

Exemple senzill: el laberint

En un laberint representat com una matriu de caràcters (`'.'` = passadís, `'#'` = paret), intenta avançar cap a la sortida. Quan una casella no porta enlloc, desfàs el moviment:

```java
public class Laberinto {

    static char[][] lab = {
        {'S', '.', '#', '#'},
        {'#', '.', '.', '#'},
        {'#', '#', '.', 'E'}
    };

    static boolean salir(int fila, int col) {
         if (fila < 0 || col < 0 || fila >= lab.length || col >= lab[0].length)
return false;
        if (lab[fila][col] == '#') return false;                 // paret: cul-
de-sac
         if (lab[fila][col] == 'E') return true;                   // cas base:
¡eixida!

         lab[fila][col] = '*';                                      // marque la
casella com a visitada

        boolean encontrado = salir(fila + 1, col)
                           || salir(fila - 1, col)
                           || salir(fila, col + 1)
                           || salir(fila, col - 1);

         lab[fila][col] = '.';                                       // la volta
arrere: la deixe lliure

        return encontrado;
    }

    public static void main(String[] args) {
             System.out.println(salir(0,  0)  ?  "¡Eixida  trobada!"  :  "No  hi  ha
eixida.");
    }
}
```

Fixa't en les dues jugades: marcar la casella com a visitada **evita donar vegades en cercle** i desmarcar-la en fallar **permet provar altres camins**. Aquell és el ball del backtracking.

Altres exemples famosos

- **Les 8 reines**: col·locar 8 reines en un tauler sense que s'ataquen.
- **El Sudoku**: provar nombres i retrocedir quan la casella es queda sense candidats.
- **La suma de subconjunts**: trobar quins nombres d'una llista sumen un valor exacte.
- **Generar totes les permutacions**: totes les ordenacions possibles d'un conjunt.

> [!TIP] Si alguna vegada sentes "exhaustiva però amb poda", això és backtracking: no et rendeixes fins a provar-ho tot, però **tires a la brossa de seguida** els camins que ja saps que són cul-de-sac. És la diferència entre buscar una agulla en un paller i buscar una agulla sabent en quina meitat del paller no està.

🧠 I amb això, el panorama

Ja tens el mapa de les tècniques més importants:

Tècnica	Idea	Típica per a
Recursivitat	Resoldre una versió més petita de si mateix	Estructures jeràrquiques
Divide i venceràs	Dividir, conquistar, combinar	Ordenació, cerca
Vorac (greedy)	Decisió òptima local en cada pas	Canvi de monedes, planificació
Backtracking	Provar, retrocedir, provar de nou	Jocs, Sudoku, laberints
Programació dinàmica	Memoització + subproblemes superposats	Fibonacci optimitzat (¡ja ho vas veure!)

La programació dinàmica és l'"evolució" de la memoització del punt 2: la veuràs amb més profunditat en unitats avançades, però ja portes el germen.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa que un algorisme sigui "vorac"?
2. El greedy del canvi de monedes és sempre òptim?
3. Quin és el pas més característic del backtracking?

4. Per què es marca una casella com a visitada en el laberint?

> 🔄 **Respostes**
>
> 1. Que en **cada pas pren la decisió localment òptima**, sense mirar el futur.
> 2. **No sempre**: amb certs conjunts de monedes (1, 3, 4) falla. El vorac és òptim només quan la decisió local garanteix la global.
> 3. **`deshacer(opcion)`**: si un camí no funciona, es recull l'últim pas i es prova una altra opció.
> 4. Per a **no entrar en bucles infinits**: sense marcar, l'algorisme aniria d'una casella a una altra sense fi.

✅ Resum en 3 frases

1. Els algorismes **voracos** trien en cada pas la millor opció local; són rapidíssims però només òptims en certs problemes.
2. El **backtracking** prova camins i, en arribar a un cul-de-sac, **retrocedeix** desfent l'últim pas.
3. Juntament amb la recursivitat i divide i venceràs, aquestes tècniques completen el teu cinturó algorísmic bàsic de DAM.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Greedy	Prendre la decisió òptima local en cada pas
Òptim local vs global	El millor ara vs el millor en conjunt
Backtracking	Provar, i en fallar, desfer i provar una altra opció
Poda	Descartar de seguida els camins sense sortida
Programació dinàmica	Memoització + subproblemes que es repeteixen

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Be the Code: quicksort des de zero · **Següent:** 09 · Modularitat

09 · Modularitat

Un programa gegant no es domina: es parte. Paquets, cohesió i acoblament perquè el teu codi no es converteixi en espaguetis ⚡🧩

> 🗺️ **Estàs en:** ⚡ **U08 · Algorítmica II: tècniques avançades i modularitat** → 09 · Modularitat

---

📬 La idea en una frase

> **La modularitat és partir un programa en peces amb nom, frontera i responsabilitat pròpia, perquè cada peça s'entenga (i s'arregle) per separat.**

En els punts del 3 al 8 vas dividir *problemes* fins a fer-los trivials. Ara apliques la mateixa estratègia al *codi*: un programa de mil línies en un sol fitxer ningú l'entén; mil línies repartides en mòduls amb sentit, sí. Divideix i venceràs, però ara en l'editor.

🧩 Per què importa tant?

- **Mantenibilitat:** quan algo es trenca, busques en el mòdul responsable, no en un fitxer amb 1.500 línies.
- **Treball en equip:** dues persones poden tocar mòduls distints sense pisar-se. Sense fronteres, cada merge és una loteria.
- **Reutilització:** una peça ben aïllada (una utilitat de dates, un algoritme d'ordenació) serveix en altres projectes.
- **Testing:** es prova una peça xiqueta per separat, no el monstre sencer.
- **Comprensió:** pots entendre *un* mòdul sense entendre *tots*. Ningú entén tot el sistema d'un colp, i no fa falta.

📦 La peça oficial en Java: el paquet

En Java, la unitat modular bàsica és el **paquet** (`package`): una carpeta amb nom que agrupa classes amb la mateixa missió.

```java
package curso.algoritmos;              // este fitxer viu en la carpeta curso/algoritmos/

public class Ordenacion {
    public static int[] burbuja(int[] datos) { /* ... */ }
}
```

Per a usar-lo des d'una altra classe, s'importa:

```java
import curso.algoritmos.Ordenacion;    // necessite la classe Ordenacion del paquet curso.algoritmos

int[] ordenados = Ordenacion.burbuja(datos);
```

Convencions que en l'examen i en la vida real es compleixen:

1. El **directori = paquet**: `curso.algoritmos` viu en `curso/algoritmos/`.
2. Noms en **minúscules i sense espais** (`curso.algoritmos`, no `Curso.Algoritmos`).
3. Paquets que expressen **missió**: `ordenes`, `pagos`, `utilitats`… no `coses2`.
4. El punt (`.`) és el separador jeràrquic: primer l'empresa o el projecte, després la capa.

> 💡 **Analogia:** els paquets són les prestatgeries d'un taller. Les eines soltes en la taula sempre es perden; les que viuen en el seu calaix, apareixen quan les busques.

🎯 Cohesió alta, acoblament baix

Dos paraules que resolen el 90% de les dubtes de disseny:

- **Cohesió (alta):** tot el que hi ha dins d'un mòdul va junt *perquè té sentit*. Si el teu mòdul `factures` conté també receptes de cuina, la cohesió ha mort.
- **Acoplament (baix):** els mòduls han de dependre el mínim dels altres. Si per a canviar el nom d'un fitxer has de tocar cinc mòduls, estan massa pegats.

La regla d'or, en una frase: **un mòdul, una responsabilitat; i que parli amb els altres justet.**

✂️ Com decideixes els talls

No hi ha una única resposta, però aquests senyals funcionen sempre:

1. **Nom que s'explica sol.** Si no saps posar nom al mòdul, encara no saps què fa.
2. **Canvi repetit en el mateix lloc.** Dues funcionalitats canvien juntes van al mateix mòdul.
3. **Mètodes curts amb una missió.** Una funció que fa cinc ja està demanant a crits dividir-se.
4. **Allò que es reutilitza, puja.** Si ho copies a un altre lloc, és que volia ser el seu propi mòdul.

> ⚠️ **[WARNING]** L'extrem contrari també enferma: **mòduls de dues línies** o una carpeta `utilitats` amb 40 classes de tot tipus. Modular en excés és soroll; l'objectiu és *claredat*, no arbres infinites de carpetes.

🔮 El que ve després

La modularitat és el pont perfecte cap a la POO: en la **U09** aprendràs que les **classes** són les peces petites, i en la **U10** que la **visibilitat** (`public`, `private`) decideix quina cara mostra cada mòdul davant la resta del món. Avui poses les prestatgeries; demà fabriques les peces que van a dins.

🎯 Mini-comprovació

Posa't a prova en 30 segons (les respostes estan amagades):

1. Què indica el `package curso.algoritmos;` de la primera línia d'un fitxer?
2. Si canvies un mòdul i has d'editar cinc més, quin problema de disseny tens: cohesió alta o acoblament excessiu?
3. Per què el directori i el paquet han de coincidir?
4. Nombra dues avantatges de modularitzar un programa per a un equip de treball.

> 🔄 **Respostes**
>
> 1. Que la classe pertany al paquet `curso.algoritmos` i que el fitxer viu en la carpeta `curso/algoritmos/`.
> 2. **Acoblament excessiu**: els mòduls depenen massa els uns dels altres.
> 3. Perquè l'eina (el compilador/IDE) trobi la classe pel seu nom: la ruta del paquet *és* la ruta de la carpeta.

> 4. Qualsevol de: treballar en paral·lel sense pisar-se, arreglar errors localitzant el mòdul, reutilitzar peces, provar parts aïllades, entendre només un tros del sistema.

✅ Resum en 3 frases

1. La **modularitat** parteix el programa en peces amb nom i responsabilitat pròpia, aplicant al codi la mateixa «divideix i venceràs» que vas fer servir amb els algoritmes.
2. En Java la peça bàsica és el **paquet** (`package` / `import`), el nom del qual coincideix amb la carpeta on viu la classe.
3. Bon disseny = **cohesió alta** (tot junt va per sentit) i **acoblament baix** (els mòduls es parlen justet).

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Mòdul	Peça del programa amb una missió
Paquet	Carpeta de classes en Java (`package`)
Cohesió	Com té sentit que el codi vagi junt
Acoblament	Com depèn un mòdul dels altres
Responsabilitat única	Un mòdul, una missió
`import`	Declarar quina peça aliena necessites usar

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Altres tècniques algorítmiques · **Següent:** 10 · Repàs interactiu

10 · Repàs interactiu

El tancament de la unitat amb chicha: endevina sortides, arregla bugs i dona't el luxe de patir amb honor 😈

> 🗺️ **Estàs en:** ⚡ **U08 · Algorítmica II: tècniques avançades i modularitat** → 10 · Repàs interactiu

---

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà.** 😈

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
public class MisterioRecursivo {
    static int contar(int n) {
        if (n <= 0) return 0;
        return 1 + contar(n / 2);
    }

    public static void main(String[] args) {
        System.out.println(contar(16));
        System.out.println(contar(100));
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **4 i 6** → Recorda: cada crida divideix n entre 2 fins a arribar a 0. ❌
2. **5 i 7** → ✅ Correcte! `contar(16)`: 16→8→4→2→1→0, cinc salts. `contar(100)`: 100→50→25→12→6→3→1→0, set salts. És un comptador de divisions entre 2 (o sigui, el "$\log_2$" arredonit).
3. **4 i 7** → Compta els salts del primer: 16→8→4→2→1→0 són cinc passos, no quatre. ❌

> › 🔄 **Solució**

> L'opció **2**. `contar(n)` retorna `1 + contar(n / 2)`: cada crida afegeix 1 i redueix `n` a la meitat. Els passos fins a arribar a 0 són el $\log_2$ arredonit cap amunt. Aquest és el clàssic que separa els que han fet la unitat dels que han dormit.

🎯 Vertader o Fals

1. "Tota funció recursiva necessita un cas base."
2. "El Fibonacci ingenu és O(n log n)."
3. "Quicksort és estable."
4. "Mergesort sempre és O(n log n), passi el que passi."
5. "La memoització converteix el Fibonacci de $O(2^n)$ a O(n)."
6. "En Java, `Arrays.sort()` usa TimSort per a tipus primitius."

> 🔄 **Respostes**
>
> 1. **Vertader.** Sense cas base, `StackOverflowError` assegurat.
> 2. **Fals.** El Fibonacci ingenu és **$O(2^n)$**, exponencial.
> 3. **Fals.** Quicksort no és estable; l'estabilitat és de Mergesort.
> 4. **Vertader.** Per això és "el fiable": no té cas dolent.
> 5. **Vertader.** Cada Fibonacci es calcula una sola vegada.
> 6. **Fals.** Per a **primitius** usa Dual-Pivot Quicksort; TimSort és per a **objectes**.

🔥 Fireside Chat: Quicksort i Mergesort discuteixen

> *Dos veterans de l'ordenació discuteixen junt a la tassa humejant.*

Quicksort: — Escolta, col·lega. Jo soc el més usat del planeta. De mitjana, jo ordeno més ràpid que tu amb la mateixa memòria. A tu et cal un array sencer de recanvi.

Mergesort: — ¿De mitjana? I què passa amb els teus mals dies? Quan et donen un array ja ordenat i el teu pivot és el primer element, te'n vas a $O(n^2)$ i plores. Jo mai. Sempre O(n log n), passi el que passi.

Quicksort: — ¡Això era abans! Ara usi mediana de tres, o pivot aleatori, o dos pivots com el Dual-Pivot de Java. No em pilles en la mala.

Mergesort: — Vale, però digues-me, ets estable? Perquè quan ordeni alumnes per nota i després per nom, jo conservo l'ordre dels empats. Tu els barreges com si repartissis cartes.

Quicksort: — Bé... l'estabilitat és per a dèbils. I la teva memòria O(n) no entra en un sistema encastat.

Mergesort: — I el teu $O(n^2)$ no entra en una base de dades. Saps què ordena discs sencers? Jo. Mergesort extern. Tot el dia. Tots els dies.

Quicksort: — *sospira* Vale. Cada un és millor on el seu cap el posa. Però tots sabem qui té més fama.

> La lliçó: **cap no guanya sempre**. Quicksort brilla amb dades a l'atzar i poca memòria; Mergesort amb estabilitat i garanties. El professional tria segons el context, no segons la marca.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Em crid a mi mateixa amb un problema més petit, fins que arribi a un cas que ja es resol sol.**
2. **Soc la variable que divideix l'array en menors i majors, i la meva elecció decideix si l'algorisme vola o s'arrossega.**
3. **Soc el pas en què dues llistes ordenades es combinen en una de sola, comparant el primer de cada una.**
4. **Soc la tècnica que guarda els resultats ja calculats per a no repetir treball.**

> 🔄 **Respostes**
>
> 1. **La recursivitat** — cas base + cas recursiu.
> 2. **El pivot** — cor de la partició de Quicksort.
> 3. **La fusió (merge)** — el cor de Mergesort.
> 4. **La memoització** — de $O(2^n)$ a $O(n)$ en Fibonacci.

🧩 Completa el codi

Aquest factorial recursiu té **3 buits**. Ompli'ls:

```java
public class FactorialHuecos {

    static int fact(int n) {
        if (_____) return 1;          // buit 1: cas base
        return _____;                 // buit 2: cas recursiu
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + _____);   // buit 3: la crida
    }
}
```

> 🔄 Solució

```java
public class FactorialHuecos {

    static int fact(int n) {
        if (n <= 1) return 1;         // buit 1
        return n * fact(n - 1);       // buit 2
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5));   // buit 3 → 120
    }
}
```

😡 CONRAD VS EL MÓN: “La teva recursió ha explotat” 🔗

> *CONRAD, el nostre compilador cascarràs, opina sobre el clàssic del novençà.*

CONRAD: — ¡UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu programa dona StackOverflowError*. I jo: vale, has mirat el teu cas base? *Què cas?* ¡AI, MARE MEVA! Hem estat tres unitats perquè em digues “què cas”!

I el pitjor ve després. Em mostra el codi i veig: `return fibo(n - 1) + fibo(n - 2);` per a `n = 50`. ¿PERÒ TU SAPS EL QUE ÉS AIXÒ? Això no és un programa, és una declaració d’intencions suïcides. `fibo(50)` són milers de milions de crides. El meu stack tremola només de veure-ho.

I després, el súmmum: agafa l'array ja ordenat, tria el primer element com a pivot, i em pregunta per què Quicksort va tan lent. Que per què? ¡Perquè has convertit O(n log n) en O(n^2), campió! ¡Tria el del mig, o mediana de tres, o tira un dau!

La lliçó: abans de plorar sobre el teclat, revisa els tres sospitosos habituals: **cas base**, **complexitat real** (estàs duplicant treball?) i **elecció del pivot**. El 90% dels drames d'aquesta unitat s'arreglen només mirant aquells tres.

⚡ Laboratori de Tortura: el programa que no ordena 🔗

> **Durada estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: aquest programa hauria d'ordenar, però té **3 errors** que el trenquen. Troba'ls i fes que la sortida sigui `[1, 2, 3, 4, 5, 6, 7]`.

```java
public class TorturaOrdenacion {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio > fin) return;

        int pivote = arr[inicio];
        int i = inicio + 1;

        for (int j = inicio + 1; j <= fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        quicksort(arr, inicio, i - 1);
        quicksort(arr, i, fin);
    }

    public static void main(String[] args) {
        int[] datos = {7, 3, 5, 1, 6, 2, 4};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Pistes per quan et frustres (no abans):

1. El cas base usa > o >=? Què passa amb una zona d'un sol element? *pista: `inicio == fin` té un sol element, ja ordenat.*

   > **› I si continuo encallat?**
   >
   > El cas base hauria de ser `if (inicio >= fin) return;`.

2. On hauria d'anar el pivot en acabar la partició? *pista: ell també és un element de l'array.*

   > **› I si continuo encallat?**
   >
   > Falta intercanviar el pivot (a `arr[inicio]`) amb `arr[i - 1]` abans de les crides recursives.

3. I les crides recursives de la dreta, quina zona cobreixen? *pista: amb el pivot a `i - 1`, la zona dreta comença en `i`.*

> Solució final

```java
public class TorturaOrdenacion {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;

        int pivote = arr[inicio];
        int i = inicio + 1;

        for (int j = inicio + 1; j <= fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        // el pivot al seu lloc: just abans de la zona de majors
        int tmp = arr[inicio];
        arr[inicio] = arr[i - 1];
        arr[i - 1] = tmp;

        quicksort(arr, inicio, i - 2);
        quicksort(arr, i, fin);
    }

    public static void main(String[] args) {
        int[] datos = {7, 3, 5, 1, 6, 2, 4};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Sortida correcta: `[1, 2, 3, 4, 5, 6, 7]`.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **El Màgic del Stack**	Explicar cas base, cas recursiu i pila de crides sense mirar
🏅 **Recursionista**	Implementar factorial i palíndrom recursius sense ajuda
🏅 **Domador de Fibonacci**	Baixar `fibo(50)` de O(2ⁿ) a O(n) amb memoització
🏅 **El Ràpid**	Implementar Quicksort des de zero sense mirar el codi
🏅 **El Fiable**	Implementar Mergesort des de zero, amb la seva fusió, sense mirar
🏅 **Enginyer de Dades**	Triar amb criteri entre Quicksort, Mergesort i `Arrays.sort()` segons el context

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```java
public class Misterio2 {
    static int misterio(int n) {
        if (n == 0) return 0;
        return n % 10 + misterio(n / 10);
    }

    public static void main(String[] args) {
        System.out.println(misterio(1234));
    }
}
```

2. **L'iteratiu que no es rendeix:** escriu una versió **iterativa** (amb bucle) de `fact(n)`. Quina prefereixes i per què?
3. **El detectiu de l'IDE:** el teu Quicksort "funciona" amb arrays petits però desordena els grans. Quina tècnica de depuració uses i en quin pas et fixes primer? *pista: els índexs de la partició.*
4. **Vertader o fals:** "Mergesort és millor que Quicksort sempre perquè no té cas dolent."

> 💡 **Solucions**

1. **`10`** — suma les xifres: 4 + 3 + 2 + 1 = 10. És una suma de dígits recursiva.
2. `int r = 1; for (int i = 2; i <= n; i++) r *= i; return r;`. L'iteratiu no gasta stack i sol ser més ràpid, però el recursiu expressa millor la definició matemàtica. Per a un examen, domina els dos.
3. El **depurador** amb breakpoints en la partició. Observa com canvien `i` i `j` (o `i` i el pivot) i on es creuen: els errors d'off-by-one en els límits de la partició són el clàssic.
4. **Fals.** Mergesort és més *fiable* (garanteix O(n log n)), però gasta O(n) de memòria i no guanya en velocitat pràctica amb arrays en memòria. "Millor" depèn del context: aquella és tota la lliçó del punt 6.

🧩 Crucigrama de Bits

```
Horitzontal:
1. La condició que deté la recursió (5,4)
3. Algorisme estable del punt 5 (9)
5. Tècnica que guarda resultats ja calculats (12)
6. La variable que divideix l'array en Quicksort (6)

Vertical:
2. El que llança la JVM quan l'stack s'ompli (8,7)
4. Ordenar sense arrays auxiliars (7)
7. Repartir elements al voltant del pivot (11)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. CAS BASE · 3. MERGESORT · 5. MEMOITZACIÓ · 6. PIVOT **Vertical:** 2. STACKOVERFLOW · 4. IN-PLACE · 7. PARTICIONAR

💬 Preguntes d'Entrevista de Treball

Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo fos la teva àvia, què és la recursivitat i per què a vegades dona StackOverflowError."**
2. **"Quan usaries Quicksort i quan Mergesort? Justifica."**
3. **"El factorial recursiu amb `n = 100000` et rebenta la pila. Què fas?"**

4. **“Què és la memoització i quin problema resol?”**
5. **“Si `Arrays.sort()` ja ordena, per a què aprendre a implementar Quicksort?”**

🤷 No hi ha preguntes tontes

> **❓ Puc escriure recursivitat amb bucles i oblidar-me de tot això?**

Pots, però et perdràs l’elegància en els problemes jeràrquics (arbres, jocs, laberints) i la comprensió de com funcionen per dins Quicksort, Mergesort i el backtracking. A més, quasi qualsevol entrevista tècnica et preguntarà per això. No és obligatori, però dol no saber-ho.

> **❓ Per què `fibo(92)` dona el màxim que cap en `long` i no puc anar més enllà?**

Perquè `long` té un límit (9.223.372.036.854.775.807) i `fibo(93)` ja el supera. Si necessites nombres més grans, hauràs d’usar `BigInteger`, que pot créixer tant com la memòria ho permeti.

> **❓ Mergesort és el que usa Java per defecte? Jurava que era Quicksort.**

Els dos, segons el tipus: **Dual-Pivot Quicksort** per a primitius i **TimSort** (una mescla amb Mergesort) per a objectes. Java no és dogmàtic: usa el millor per a cada cas. Aquell és el mateix criteri que et demanem a tu en el punt 6.

🎬 Post-Crèdits

El programador novençà acaba el seu Quicksort, l’executa i… funciona a la primera. Es frega els ulls. El seu company Mergesort, aquell maratonià fiable, li somriu des de l’altre monitor.

Novençà: — No m’ho crec. He escrit un Quicksort sense mirar!

CONRAD: — *apareix amb cara de pocs amics* Bonic. Ara digues-me: quan usaries Mergesort en el seu lloc?

Novençà: — Quan necessiti estabilitat… o quan l’array arribi quasi ordenat i em faci por el pivot.

CONRAD: — *assenteix, quasi emocionat* Mira per on. Ja no només escrius algorismes: penses en ells. Això es diu ser enginyer. *pausa* Però no et relaxes, que la pròxima unitat t’espera amb objectes, classes i algun constructor que et farà plorar.

Novençà: — Plorar? Jo ja domine la recursivitat.

CONRAD: — *riu malèvol* Això deia l'altre abans de trobar-se amb `this`.

El novençà guarda el seu projecte, i sent que alguna cosa ha canviat en la seva manera de pensar. La recursivitat ja no li fa por: l'entén. I l'entén perquè, per a entendre la recursivitat, primer va entendre la recursivitat.

PRÒXIMAMENT EN U09: POO: classes i objectes. El moment en què els teus programes deixen de ser funcions soltes i es converteixen en **ciutadans amb atributs i mètodes**. 🏗️

📚 Tornar a l'índex de la unitat · **Anterior:** 09 · Modularitat · **Següent:** U09 · POO: classes i objectes

U09 — POO: classes i objectes

El tallagalletas i les galetes: quan el teu codi passa de ser una recepta a un món d'objectes que parlen entre ells

🏗️

Benvingut a la unitat on els teus programes deixen de ser *receptes de cuina* i es converteixen en **mons de coses**. Fins ara has escrit programes lineals: això, després això, després això. Però el món real no funciona així. Al món real tens *coses*: un gos, un cotxe, un professor de programació amb ulleres de pasta. Cada cosa té **atributs** (color, edat, nombre de ganes de corregir exàmens) i **comportaments** (lladrar, accelerar, posar faltes d'ortografia).

La **Programació Orientada a Objectes (POO)** és exactament això: escriure codi com funciona el món real. I en aquesta unitat aprendràs la base de tot: les **classes** (el motlle) i els **objectes** (la galeta). Construiràs les teves pròpies classes amb atributs i mètodes, les faràs néixer amb **constructors**, aprendràs a desambiguar amb `this`, a raonar sobre **referències i memòria**, i a exprimir la **sobrecàrrega** de mètodes i el pas de paràmetres.

Aquesta unitat és el **punt d'inflexió del curs**: el que aprenguis aquí, ho usaràs en totes les unitats que queden. La POO no és un tema més: és el tema que fa que tots els altres tinguin sentit.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar què és la **POO** i per què existeix (spoiler: no és per a fastidiar, és per a no tornar-te boig).
- Diferenciar **classe** (el tallagalletas) d'**objecte** (la galeta), i crear objectes amb `new`.
- Definir **atributs** (les propietats) i **mètodes** (els comportaments) d'una classe.
- Usar **constructors** perquè els objectes neixin ben preparats, amb o sense paràmetres.
- Desambiguar atributs i paràmetres amb la paraula clau **`this`**, i cridar un constructor des d'un altre.
- Raonar sobre **referències, `null` i memòria**: què guarda una variable d'objecte i què pinta el heap.

- Escriure **mètodes sobrecarregats** i entendre el **pas per valor** (i per què les referències es porten "rar").
- Dissenyar i crear una **classe completa** amb els seus mètodes, de principi a fi.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Què és la POO	Programació procedural vs orientada a objectes i per què el món real mana	Tots
02 · Classes i objectes: el tallagalletas i les galetes	El motlle, la galeta, `new`, i la teva primera classe	Tots
03 · Atributs i mètodes	Les propietats i els comportaments d'una classe, amb els seus tipus i paràmetres	Tots
04 · Constructors	El mètode que prepara l'objecte en néixer: per defecte i amb paràmetres	Tots
05 · La paraula clau this	Quan els paràmetres es diuen igual que els atributs (i com sortir de l'embolic)	Tots
06 · Referències, null i memòria	Què guarda una variable d'objecte, el heap, `null` i el mític `NullPointerException`	Tots
07 · Sobrecàrrega i pas per valor	Diversos mètodes amb el mateix nom i què es copia quan crides	Tots
08 · Be the Code: dissenya la teva classe	Dissenyar i construir una classe completa pas a pas, sense mirar	⭐⭐⭐
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

---

✅ Criteris d'avaluació coberts (RA2 + RA4)

RA2: Escriu i prova programes senzills, reconeixent i aplicant els fonaments de la programació orientada a objectes.

RA4: Desenvolupa programes organitzats en classes analitzant i aplicant els principis de la programació orientada a objectes.

CE	Criteri	On es cobreix
RA2 a)	S'han identificat els fonaments de la programació orientada a objectes.	✅ Punt 1
RA2 b)	S'han escrit programes simples.	✅ Tots
RA2 c)	S'han instanciat objectes a partir de classes predefinides.	✅ Punts 2, 3 i 6
RA2 d)	S'han utilitzat mètodes i propietats dels objectes.	✅ Punts 3 i 8
RA2 f)	S'han utilitzat paràmetres en la crida a mètodes.	✅ Punts 7 i 8
RA2 h)	S'han utilitzat constructors.	✅ Punts 4 i 7
RA4 a)	S'ha reconegut la sintaxi, estructura i components típics d'una classe.	✅ Punts 2 i 3

CE	Criteri	On es cobreix
RA4 b)	S'han definit classes.	✅ Punts 2 i 8
RA4 c)	S'han definit propietats i mètodes.	✅ Punts 3 i 8
RA4 d)	S'han creat constructors.	✅ Punts 4 i 7
RA4 e)	S'han desenvolupat programes que instancien i utilitzen objectes de les classes creades anteriorment.	✅ Punts 6 i 8

> 📌 El punt 6 (referències i memòria) i el 7 (pas per valor) són la base per a entendre el **RA2 c) i f)** en profunditat. Els modificadors de visibilitat (RA4 f), l'herència (RA4 g) i els mètodes estàtics (RA4 h) tenen el seu propi capítol en la **U10** (Visibilitat, encapsulació i static) i en la **U11** (Herència), així que aquí els deixem descansant.

🚪 Per on començo?

- Cero coneixements de POO? → Arranca en el punt 1. Només necessites el que vas veure fins a la U08.
- Ja saps què és una classe i vols cim? → Ves directe al punt 4 i al punt 5: aquí comença el ball seriós.
- Et flipen els reptes? → Fes el punt 8 sense mirar el codi dels punts anteriors. Dissenya la teva classe amb honor.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Què és la POO

⏭️ **En acabar la unitat, continua en U10 · Visibilitat, encapsulació i static.**

01 — Què és la POO

Quan el teu codi deixa de ser una recepta i es converteix en un món de coses que parlen entre elles 🏗️

> 🗺️ **Estàs en: 🏗️ U09 · POO: classes i objectes** → 01 · Què és la POO

📬 La idea en una frase

> **La POO és escriure codi com funciona el món real: en comptes d'una llista d'instruccions, models *coses* amb les seves dades (atributs) i els seus comportaments (mètodes).**

Fins ara els teus programes eren com una **recepta de cuina**: fes això, després això, després l'altre. Molt ordenat, però molt rígid. Si vols representar dos estudiants, dos cotxes o dos galetes… comences a copiar i enganxar variables com un boig. I quan el programa creix, la recepta es converteix en un caos.

La POO arriba per a posar ordre: en lloc de variables soltes, agrupes les dades *de la mateixa cosa* juntament amb el que aquella cosa *sap fer*. Un gos no és només un `String nombre` i un `int edad` solts per aquí: és un **objecte** que té nom, edat, i sap lladrar.

📜 De la recepta al món real

Imagina que has de gestionar les dades d'un estudiant sense POO:

```
String nombre1 = "Ana";
int edad1 = 20;
double nota1 = 8.5;

String nombre2 = "Luis";
int edad2 = 19;
double nota2 = 6.0;
```

Amb dos estudiants, ja són sis variables. Amb cinquanta, és la pitjor malson de la teva vida. Les dades de cada persona estan **desconnectades**: no hi ha res que digui "nombre1, edad1 i nota1 van junts perquè són Ana".

Amb POO agrupes tot això en una peça:

```
public class Estudiante {
    String nombre;
    int edad;
    double nota;
}
```

I cada estudiant és un **objecte** amb les seves tres dades dins, ben lligades. Ja no tens variables soltes: tens *persones*.

> 💡 **Tip:** quan vegis codi ple de `nombre1`, `nombre2`, `nombre3`, fa olor que dins vol néixer una classe. Aquell olfacte s'entrena, i aquesta unitat és el teu gimnàs.

🧩 Els dos ingredients: atributs i mètodes

Tota classe es construeix amb dos ingredients:

- **Atributs (o propietats):** les dades que descriuen la cosa. El color del cotxe, l'edat del gos, el saldo del compte.
- **Mètodes (o comportaments):** el que la cosa *sap fer*. Accelerar, lladrar, retirar diners.

```
public class Perro {
    // Atributs
    String nombre;
    int edad;

    // Mètodes
    void ladrar() {
        System.out.println(nombre + ": ¡Guau!");
    }
}
```

> 📝 **Nota:** encara no executem res. Aquesta classe és només el *disseny*. Que existeixi `Perro` no significa que existeixi un gos. Això arribarà en el punt 2 amb `new`.

🏛️ Els quatre pilars (una promo, no el capítol)

La POO es recolza en quatre pilars. En aquesta unitat toquem el primer a fons i fem la promo dels altres:

1. **Abstracció:** et quedes amb el que és important d'una cosa i t'oblides del soroll. Un gat en un programa és `nombre`, `edad` i `maullar()`, no els seus bigotis.
2. **Encapsulació (U10):** cada objecte protegeix les seves dades i decideix qui pot tocar-les. Com un compte bancari: el saldo no està tirat al carrer.
3. **Herència (U11):** una classe pot "heretar" d'una altra. Un `Perro` és un `Animal` i, a més, lladra.
4. **Polimorfisme (U11):** el mateix missatge, distintes respostes. Tots els animals "fan so"; el gos lladra i el gat miola.

> ⚠️ **Advertència:** no t'estresses amb els pilars 2, 3 i 4. Aquesta unitat et dona els fonaments; els altres els alcem en la U10 i la U11. Aquí mana l'abstracció i el naixement de les classes.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** quan vegis una classe, pregunta't sempre dues coses: *quines dades té?* (atributs) i *què sap fer?* (mètodes). Si saps respondre, ja has entès el 80% de la POO.

Exercici: classifica

Sense executar res, decideix per a cada un d'aquests si és un **atribut** o un **mètode** d'una classe `Videojuego`:

1. `String titulo;`
2. `void pausar() { ... }`
3. `int nivelMaximo;`
4. `double calcularPuntuacion() { ... }`
5. `boolean estaInstalado;`

> › 🔄 **Solució**
>
> 1. **Atribut** — el títol és una dada que descriu el joc.
> 2. **Mètode** — pausar és una cosa que el joc *sap fer*.
> 3. **Atribut** — el nivell màxim és una dada.
> 4. **Mètode** — calcular la puntuació és un comportament (a més retorna un `double`).
> 5. **Atribut** — si està instal·lat o no és una dada (un `boolean`).

> La regla d'or: **atribut = substantiu (dada), mètode = verb (acció)**. Si el pots conjugar ("pausar", "calcular"), és mètode. Si el pots descriure ("títol", "nivell"), és atribut.

❓ No Hi Ha Preguntes Tontes!

> **Q:** Val, però jo ja sé fer programes amb variables i bucles. De veritat necessito tot aquest rotllo?
>
> **A:** Per a un programa de 30 línies, no. Per a qualsevol cosa que vagi a créixer, sí. Els bucles i variables continuen estant (no els abandonem, els usem *dins* dels mètodes), però la POO et dona una forma d'organitzar el codi que escala. Ningú no escriu un programa d'empresa amb una sola llista d'instruccions. Bé, ningú *sa*.

> **Q:** La POO és només cosa de Java?
>
> **A:** No. És una *forma de pensar* que usen quasi tots els llenguatges moderns: Python, JavaScript, C++, C#... Cada un amb el seu accent, però la idea és la mateixa: modelar el món en objectes. Aprendre-la aquí et serveix per a tots.

> **Q:** Què passa amb el `main` que porti escrivint des de la UO2? Ara no serveix?
>
> **A:** El `main` continua sent la porta d'entrada: el que hi ha dins de `main` és la "recepta" que arranca la pel·lícula. El que canvia és que ara, dins d'aquell `main`, en comptes de variables soltes crearàs objectes. El `main` continua sent el director; només canvia el repartiment.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quins dos ingredients formen una classe?
2. Quin pilar de la POO consisteix a "quedar-te amb el que és important"?
3. Un `String color;` dins d'una classe és atribut o mètode?
4. En quines unitats es veuen l'encapsulació i l'herència?

> 🔄 **Respostes**
>
> 1. **Atributs** (les dades) i **mètodes** (els comportaments).
> 2. L'**abstracció**.

3. **Atribut** — és una dada (un substantiu), no una acció.
4. L'**encapsulació** en la U10 i l'**herència** (i el polimorfisme) en la U11.

✅ Resum en 3 frases

1. La **POO** modela el món real en objectes amb **atributs** (dades) i **mètodes** (comportaments), en comptes d'una llista d'instruccions.
2. La **classe** és el disseny de la cosa; encara no hi ha cap "cosa" real fins que no la instancies.
3. Els quatre pilars són **abstracció, encapsulació, herència i polimorfisme**; en aquesta unitat cimentem el primer.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
POO	Paradigma que organitza el codi en objectes
Atribut	Dada que descriu la cosa: `color`, `edat`
Mètode	Comportament que la cosa sap fer: `acelerar()`
Abstracció	Quedar-te amb l'essencial i oblidar el soroll
Procedural	Estil antic: una recepta d'instruccions en ordre

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · Classes i objectes: el tallagalletas i les galetes

02 — Classes i objectes: el tallagalletas i les galetes

El motlle, la galeta, `new` i la teva primera classe: ja no escrius receptes, fornegis 🍪

> 🗺️ **Estàs en:** 🏗️ **U09 · POO: classes i objectes** → 02 · Classes i objectes: el tallagalletas i les galetes

---

📬 La idea en una frase

> **Una classe és el tallagalletas (el motlle) i un objecte és la galeta que fas amb ell: un sol motlle, milions de galetes, i cada una amb el seu propi sabor.**

En el punt 1 vas veure que una classe és un disseny. Ara toca la pregunta del milió: i què faig amb un disseny? L'usi per a **fabricar coses**. I en Java, la fàbrica es diu `new`.

🍪 El tallagalletas i la galeta

Imagina la cuina de la teva àvia. Té un tallagalletas amb forma d'estrela. Aquell tallagalletas és **la classe**: defineix la forma de totes les galetes, però ell mateix no és una galeta. Pots usar-lo mil vegades i cada galeta serà distinta: unes amb pepitas de xocolate, altres de vainilla, altres que la teva cosina es menja a mig fer.

En programació:

- **Classe** = el tallagalletas. Defineix *quins atributs* i *quins mètodes* tindran totes les seves galetes.
- **Objecte** = la galeta. Una cosa concreta, amb els seus propis valors, vivint en la memòria.

Així que d'un sol motlle surten infinites galetes, totes amb la mateixa forma però cada una amb la seva pròpia vida.

🔨 La teva primera classe

Escrivim el tallagalletas `Galleta`:

```java
public class Galleta {
    String sabor;
    boolean tieneChocolate;

    void comer() {
        System.out.println("Ñam, galleta sabor " + sabor);
    }
}
```

Això només defineix el motlle. Per ara, **no existeix cap galeta en memòria**. És un fitxer al disc esperant que algú l'usi.

> 📝 **Nota:** la classe porta `public class Galleta` i el fitxer es diu `Galleta.java`. El nom de la classe i del fitxer han de coincidir. És la primera regla no escrita de Java… val, escrita.

🏭 new: la màquina de galetes

Per a fabricar galetes (objectes) a partir del motlle uses l'operador `new`:

```java
public class Pasteleria {
    public static void main(String[] args) {
        Galleta g1 = new Galleta();          // Primera galeta
        g1.sabor = "Chocolate";
        g1.tieneChocolate = true;

        Galleta g2 = new Galleta();          // Segona galeta (mateix motle)
        g2.sabor = "Vainilla";
        g2.tieneChocolate = false;

        g1.comer();   // "Ñam, galleta sabor Chocolate"
        g2.comer();   // "Ñam, galleta sabor Vainilla"
    }
}
```

Desgranem la línia estrella: `Galleta g1 = new Galleta();`

1. `Galleta g1` — declara una variable que apuntarà a una galeta.
2. `new Galleta()` — fabrica la galeta en memòria (i crida al seu constructor, que veuràs en el punt 4).

3. = — connecta la variable `g1` amb la galeta acabada de fornejar.

Després, amb `g1.sabor = "Chocolate"` i `g1.comer()`, li dius a *aquella* galeta concreta quin sabor té i que es menge. La `g2` no s'assabenta de res: és una altra galeta amb la seva pròpia vida.

> 💡 **Tip:** el punt (`.`) és la mà que estens a l'objecte. `g1.comer()` significa "escolta, galeta g1, executa el teu mètode comer". `g1.sabor` significa "dona'm el sabor de la galeta g1".

🥊 Classe vs objecte: el ring

Classe: — Jo soc el motlle, el plànol, la idea platònica. Sense mi no existiries. Defineixo quins atributs i mètodes tenen els objectes. Soc la creadora!

Objecte: — Sí, però jo soc qui realment fa coses. Tu ets només un fitxer `.java` al disc. Jo ocupi memòria, tinc estat, puc canviar els meus atributs. Sense mi el teu codi no serveix per a res.

Classe: — Ah sí? I quants de tu existeixen? Pots tenir milers d'objectes creats a partir de mi. Jo soc únic, tu ets una còpia. Soc original, ets reproduïble!

Objecte: — Exacte. Perquè tu ets el plànol, però jo soc l'edifici construït. Ningú viu en un plànol. Quan executes el programa, el que treballa soc jo.

Classe: — Val, ens necessitem. Sense classe no hi ha objecte. Sense objecte, la classe és només teoria.

Objecte: — Tracte fet.

> 🕶️ **Don Tip:** la classe defineix el **QUÈ** (atributs) i el **CÓM** (mètodes). L'objecte és el **QUI** (la instància concreta que executa i té valors propis).

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** compta cada `new` com una galeta nova. Cada `new` = un objecte nou en memòria, encara que sigui del mateix motlle.

Exercici: quantes galetes hi ha?

Sense executar, respon: quants objectes de la classe `Galleta` existeixen en memòria al final d'aquest `main`?

```
public class Pasteleria2 {
    public static void main(String[] args) {
        Galleta g1 = new Galleta();
        Galleta g2 = new Galleta();
        Galleta g3 = g1;
        g1.comer();
    }
}
```

> 🔄 **Solució**
>
> **2 objectes.** `g1` i `g2` són dues galetes distintes. `g3 = g1` no crea una tercera galeta: fa que `g3` *apunti a la mateixa* que `g1`. Dos noms, una galeta. Això es diu **àlies** i ho destripem en el punt 6. Per ara, memoritza: **cada `new` fabrica un objecte; copiar una variable no fabrica res**.

❓ No Hi Ha Preguntes Tontes!

> **Q:** Per què cal escriure `new`? No podria Java crear l'objecte sol?
>
> **A:** No, perquè `new` és el "permís de construcció". Sense `new`, només declares una variable (com `Galleta g;`), però no hi ha galeta a la safata, només un forat buit. Fins que no facis `new`, l'objecte no existeix.

> **Q:** I per què `String nombre = "Ana";` no porta `new`?
>
> **A:** Perquè Java és un amor amb `String`. És tan comú que et deixa crear-lo amb cometes directament (un literal). És una drecera: darrere del teló, Java ho tracta quasi com si tingués `new`. Però això és un privilegi de `String` (i dels arrays), no una norma general. Tu, amb les teves classes, hauràs d'usar `new` com tothom.

> **Q:** Puc tenir diverses classes en un mateix fitxer `.java`?
>
> **A:** Pots, però només **una** pot ser `public` i ha de coincidir amb el nom del fitxer. Les altres van sense `public`. Per a començar, un fitxer = una classe pública. Simple i net.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què és la classe en l'analogia del tallagalletas?
2. Quin operador fabrica objectes en memòria?
3. Quants objectes crea aquest codi: `Galleta a = new Galleta(); Galleta b = new Galleta();`?
4. Per a què serveix el punt (`.`) entre l'objecte i el membre?

> 🔄 **Respostes**
>
> 1. La classe és el **tallagalletas** (el motlle); l'objecte és la galeta.
> 2. `new`.
> 3. **2 objectes**, un per cada `new`.
> 4. Per a **accedir** a un atribut (`g.sabor`) o invocar un mètode (`g.comer()`) d'aquell objecte concret.

✅ Resum en 3 frases

1. La **classe** és el motlle que defineix atributs i mètodes; l'**objecte** és la galeta concreta, amb els seus propis valors, vivint en memòria.
2. Els objectes es fabriquen amb **new**, i cada `new` crea una instància independent encara que usi el mateix motlle.
3. Amb el **punt** (`.`) accedeixes als atributs i invoques els mètodes d'un objecte concret.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Classe	El motlle / tallagalletas. Defineix atributs i mètodes
Objecte	La galeta. Instància concreta amb valors propis
Instanciar	Crear un objecte a partir d'una classe amb `new`
Instància	Sinònim d'objecte creat
Membre	Qualsevol atribut o mètode d'una classe

03 — Atributs i mètodes

Les propietats i els comportaments de la teva classe, amb els seus tipus, paràmetres i valors de retorn 🏗️

> 🗺️ **Estàs en: 🏗️ U09 · POO: classes i objectes** → 03 · Atributs i mètodes

📬 La idea en una frase

> **Els atributs són les propietats de la cosa (què és) i els mètodes són els seus comportaments (què fa): la classe els declara i cada objecte els viu pel seu compte.**

Ja saps que la classe és el motlle. Ara vestirem aquell motlle per dins: quines dades guarda i què sap fer. Els atributs i mètodes són la carn de la classe, i és aquí on els teus objectes comencen a tenir vida pròpia.

🏷️ Els atributs: les propietats de la cosa

Un atribut es declara igual que una variable, però **dins de la classe** i **fora de qualsevol mètode**:

```
public class Coche {
    String marca;       // atribut
    String modelo;      // atribut
    int velocidad;      // atribut
    boolean encendido;  // atribut
}
```

Cada atribut té un tipus (que ja domines de la U03) i descriu una propietat de l'objecte. Quan crees un cotxe amb `new Coche()`, aquell cotxe concret tindrà **la seva pròpia còpia** dels quatre atributs. Un altre cotxe, altres còpies. Cada galeta porta les seves pepitas.

> 💡 **Tip:** els atributs es declaren amb els mateixos tipus que les variables: `String`, `int`, `double`, `boolean`… Si domines la U03, domines la mitat d'aquest punt.

🎮 Els mètodes: els comportaments de la cosa

Un mètode és una acció que l'objecte sap fer. Té nom, paràmetres (les dades d'entrada) i un **tipus de retorn** (el que retorna):

```
public class Coche {
    String marca;
    int velocidad;

    // Mètode que no retorna res: només fa coses
    void acelerar(int incremento) {
        velocidad += incremento;
    }

    // Mètode que retorna un valor: calcula i retorna
    int velocidadActual() {
        return velocidad;
    }

    // Mètode que imprimeix i no retorna res
    void mostrarEstado() {
        System.out.println(marca + " va a " + velocidad + " km/h");
    }
}
```

- **`void`** = no retorna res. Només executa (acelera, imprimeix, saluda…).
- **Tipus normal** (`int`, `double`, `String`…) = retorna un valor amb `return`.

> ⚠️ **Advertència:** si el mètode promet retornar un `int` (`int velocidadActual()`), està **obligat** a acabar amb un `return` que doni un `int`. Si promet `void`, es prohibeix el `return` amb valor. Java és un notari molt puntillós amb les firmes.

📞 Cridar els mètodes

Dins del `main` (o d'un altre mètode) crides els mètodes amb el punt:

```java
public class Concesionario {
    public static void main(String[] args) {
        Coche miCoche = new Coche();
        miCoche.marca = "Seat";
        miCoche.acelerar(50);          // no retorna res
        int v = miCoche.velocidadActual();   // retorna 50
        System.out.println("Velocidad: " + v);
        miCoche.mostrarEstado();       // "Seat va a 50 km/h"
    }
}
```

> 📝 **Nota:** dins de la pròpia classe (per exemple, `acelerar` cridant a `mostrarEstado`), no cal el punt: `mostrarEstado();` a secas. El punt només cal quan crides un mètode d'**un altre** objecte.

🧬 Cada objecte amb la seva pròpia còpia

Aquí està la màgia del punt 2 feta realitat. Dos cotxes creats del mateix motlle són independents:

```java
Coche cocheA = new Coche();
Coche cocheB = new Coche();

cocheA.marca = "Seat";
cocheB.marca = "Ford";

cocheA.acelerar(100);   // només cocheA s'assabenta

System.out.println(cocheA.marca + " → " + cocheA.velocidad);  // Seat → 100
System.out.println(cocheB.marca + " → " + cocheB.velocidad);  // Ford → 0
```

`cocheB` no sap res de l'acceleració de `cocheA`: cada objecte guarda els seus propis valors. Això és el que fa que la POO no sigui un caos: **les dades de cada objecte van amb el seu objecte**.

> 💡 **Tip:** pensa en els atributs com a taquilles personals. Tu tens la teva taquilla, el teu company té la seva. Si omplis la teva de entrepans, la d'ell no s'ompli sola.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** per a traçar un programa amb objectes, fes una capseta per cada objecte i apunta els seus atributs dins. Cada volta que es crida un mètode, actualitza només la capseta de l'objecte que el crida.

Exercici: la caixa misteriosa

Sense executar, calcula què imprimeix aquest programa:

```
public class Caja {
    int ancho;
    int alto;
    int profundo;

    int volumen() {
        return ancho * alto * profundo;
    }
}

public class Main {
    public static void main(String[] args) {
        Caja c1 = new Caja();
        Caja c2 = new Caja();

        c1.ancho = 2;  c1.alto = 3;  c1.profundo = 4;
        c2.ancho = 5;  c2.alto = 1;  c2.profundo = 2;

        c1.ancho = 10;
        System.out.println(c1.volumen());
        System.out.println(c2.volumen());
    }
}
```

> › 🔄 **Solució**
>
> Imprimeix **`120`** i **`10`**.
>
> - `c1.volumen()` → 10 × 3 × 4 = **120** (hem canviat `ancho` a 10 abans de calcular).
> - `c2.volumen()` → 5 × 1 × 2 = **10**.
> - El canvi de `c1.ancho` **no afecta `c2`**: cada objecte té la seva pròpia còpia dels atributs. Si no ho has vist venir, repeteix el mantra: *cada objecte guarda els seus propis valors*.

❓ No Hi Ha Preguntes Tontes!

> **Q:** Un mètode pot cridar un altre mètode de la mateixa classe?
>
> **A:** Sí, i és el més normal del món. `acelerar` pot cridar a `mostrarEstado` sense el punt: `mostrarEstado();`. És com parlar-te a tu mateix en veu alta: no necessites presentar-te.

> **Q:** I si un mètode necessita diverses dades? Puc passar diversos paràmetres?
>
> **A:** Tots els que vulguis, separats per comes: `void mover(int x, int y, int velocidad)`. El nombre i tipus de paràmetres formen la **firma** del mètode, i Java distingeix mètodes per la seva firma (això ho exprimim en el punt 7 amb la sobrecàrrega).

> **Q:** Els atributs poden tenir valors per defecte?
>
> **A:** Sí, i de fet Java els dona un automàticament quan crees l'objecte: nombres a `0`, `boolean` a `false` i `String` (i qualsevol referència) a `null`. Però dependre d'aquells valors és mael costum: veuràs en el punt 4 que els **constructors** existeixen per a donar-los un bon començament.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. On es declaren els atributs dins d'una classe?
2. Què significa `void` en la firma d'un mètode?
3. Quina paraula retorna un valor en un mètode?
4. Si dos objectes són de la mateixa classe, comparteixen els seus atributs?

> 🔄 **Respostes**
>
> 1. **Dins de la classe i fora de qualsevol mètode**.
> 2. Que el mètode **no retorna res**; només executa.
> 3. `return`.
> 4. **No.** Comparteixen la *definició* (la classe), però cada objecte té la seva **pròpia còpia** dels valors.

✅ Resum en 3 frases

1. Els **atributs** són les propietats de la cosa i es declaren com variables dins de la classe; cada objecte té la seva pròpia còpia.
2. Els **mètodes** són els comportaments: poden no retornar res (`void`) o retornar un valor amb `return`, i reben dades per paràmetres.
3. S'invoquen amb el punt (`objeto.metodo()`), i dins de la mateixa classe no cal.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Atribut | Propietat de l'objecte: `marca`, `velocidad` |
> | Mètode | Comportament: `acelerar()`, `volumen()` |
> | Firma | Nom + tipus dels paràmetres |
> | `void` | El mètode no retorna res |
> | `return` | Retorna un valor al qui va cridar |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Classes i objectes: el tallagalletas i les galetes · **Següent:** 04 · Constructors

04 — Constructors

El mètode que prepara l'objecte en la seva festa de benvinguda: per defecte i amb paràmetres 🎉

> 🗺️ **Estàs en: 🏗️ U09 · POO: classes i objectes** → 04 · Constructors

---

📬 La idea en una frase

> **El constructor és la festa de benvinguda de l'objecte: un mètode especial que s'executa amb `new` perquè l'objecte nasqués ben preparat, amb els seus atributs inicialitzats.**

En el punt 3 veies que els atributs naixien amb valors per defecte (0, `false`, `null`). Molt avorrit. I si vols que el teu cotxe neixi amb marca "Seat" i el motor ja encés? Aquí entren els **constructors**: la festa de benvinguda on li dius a l'objecte com ha de ser des del primer segon.

🎉 Què és un constructor?

Un constructor és un mètode especial amb tres regles d'or:

1. Es diu **igual que la classe**.
2. **No retorna res** (ni tan sols `void`).
3. S'executa automàticament amb cada **`new`**.

```java
public class Persona {
    String nombre;
    int edad;

    // Constructor sense paràmetres: "val, et pose valors per defecte"
    public Persona() {
        nombre = "Desconocido";
        edad = 0;
    }

    // Constructor amb paràmetres: "et passe les dades, tu inicialitza-les"
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    void presentarse() {
           System.out.println("Hola, soy " + nombre + " y tengo " + edad + "
años.");
    }
}
```

> 💡 **Tip:** el `this` de `this.nombre = nombre` s'explica en el punt 5. Per ara només necessites saber que significa "l'atribut d'AQUEST objecte". Si et pica la curiositat, sàpigues que ja ho resoldràs.

🆓 El constructor per defecte (el que regala Java)

Si **no escrius cap constructor**, Java et regala un de gratuït, sense paràmetres i buit. Només es dedica a deixar els atributs amb els seus valors per defecte (0, `false`, `null`). Com un forn fred esperant que hi poses alguna cosa.

```java
public class Galleta {
    String sabor;   // Java la crea amb sabor = null
    boolean tieneChocolate;  // false
}
```

`new Galleta()` funciona perquè Java va posar aquell constructor invisible. Però compte amb la trampa:

> ⚠️ **Advertència:** si escrius **qualsevol** constructor, el constructor buit **desapareix**. Si poses `public Galleta(String sabor)`, aleshores `new Galleta()` (sense arguments) deixa de compilar. Java no et regala res si ja has decidit muntar la teva pròpia festa.

🎛️ El constructor amb paràmetres (el forn amb programa)

Amb paràmetres, l'objecte neix a mida. Li passes les dades i el constructor les col·loca:

```
public class Galleta {
    String forma;
    boolean tieneChocolate;
    int temperatura;

    public Galleta(String forma, boolean tieneChocolate, int temperatura) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
        this.temperatura = temperatura;
    }
}
```

```
Galleta g = new Galleta("estrella", true, 180);
```

Una sola línia crea la galeta **ja amb** forma d'estrela, amb xocolate i a 180 graus. Res de deixar atributs ballant.

> 📝 **Nota:** no és obligatori inicialitzar tots els atributs en el constructor, però és la millor costum. Un atribut que es queda sense tocar viu en la incertesa (`null`, 0, `false`) i la incertesa en programació és on s'amaguen els bugs.

🏭 `new` i el constructor, un tàndem

La línia `new Persona("Ana", 25)` fa tres coses en ordre:

1. Reserva memòria per a l'objecte.
2. Crida al constructor `Persona(String, int)`.

3. Retorna la referència (l'adreça) perquè la guardes en la teva variable.

Sense constructor a la vista, `new Persona("Ana", 25)` fallaria si només existira el buit (o el que regala Java). La JVM tria el constructor que encaixa amb els arguments que li passes. Què passa si n'hi ha diversos? Això és **sobrecàrrega**, i té el seu capítol en el punt 7.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** per a saber quin constructor es crida, mira els arguments de la crida: nombre i tipus. Després, rastreja què fa aquell constructor.

Exercici: quin constructor es crida?

Sense executar, què imprimeix aquest codi?

```
public class Pedido {
    String producto;
    int cantidad;

    public Pedido() {
        this("Sin producto", 0);
        System.out.println("Constructor vacío");
    }

    public Pedido(String producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        System.out.println("Constructor con parámetros");
    }

    public static void main(String[] args) {
        Pedido p = new Pedido();
        System.out.println(p.producto + " x" + p.cantidad);
    }
}
```

> › 🔄 **Solució**
>
> Es crida a `Pedido()`, que fa `this("Sin producto", 0)`: això executa primer `Pedido(String, int)` (imprimeix **"Constructor con parámetros"**), després torna i acaba

> `Pedido()` (imprimeix **"Constructor vacío"**). Després el `main` imprimeix **"Sin producto xO"**.
>
> El `this(...)` d'un constructor a un altre **sempre va primer**, abans de qualsevol altra instrucció. És com la presentació abans de la festa: primer crides el teu col·lega constructor i després fas les teves coses.

❓ No Hi Ha Preguntes Tontes!

> **Q:** El constructor pot ser `private` o ha de ser `public`?
>
> **A:** Normalment és `public` perquè qualsevol pugui crear objectes. Existeixen constructors `private`, però tenen truc (s'usen en patrons com Singleton, que veuràs en altres mòduls). Per a aquesta unitat: `public` i a córrer.

> **Q:** Un constructor pot retornar alguna cosa? I si posi `return` dins?
>
> **A:** Un constructor **no retorna res**, ni tan sols l'objecte (això ho fa `new` per ell). Pots posar un `return;` solt per a sortir abans, però retornar un valor amb `return 5` és un error de compilació. El constructor no és un caixer automàtic.

> **Q:** I si no necessiti inicialitzar res? He d'escriure constructor igualment?
>
> **A:** Si no necessites inicialitzar res i no escrius cap constructor, Java et regala el buit. Però el dia que afegeixis un constructor amb paràmetres per a una coseta, el buit desapareix. Regla: **si escrius un, escriu també el buit** si el vas a usar.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines tres regles d'or té un constructor?
2. Què fa Java si no escrius cap constructor?
3. Què passa si escrius un constructor amb paràmetres però després crides a `new Clase()` sense arguments?
4. Quan s'executa un constructor?

> › 🔄 **Respostes**

1. Mateix **nom que la classe**, **no retorna res** (ni `void`), i s'executa amb cada **new**.
2. Regala un constructor **buit sense paràmetres**, que deixa els atributs amb els seus valors per defecte.
3. **Error de compilació**: en escriure qualsevol constructor, el buit desapareix, així que `new Clase()` no troba amb què cridar-se.
4. Automàticament, **en cada new**, just després de reservar la memòria.

✅ Resum en 3 frases

1. El **constructor** és un mètode especial amb el mateix nom que la classe, sense tipus de retorn, que s'executa amb cada `new` per a preparar l'objecte.
2. **Sense constructor escrit**, Java regala un de buit; **en escriure qualsevol constructor**, el buit desapareix.
3. Amb **paràmetres**, l'objecte neix a mida: els seus atributs queden inicialitzats des del primer segon.

🐛 **Vocabulari ràpid**

Terme	Idea general
Constructor	Mètode especial que prepara l'objecte en néixer
Constructor per defecte	El buit que regala Java si no escrius cap
Constructor parametritzat	El que rep dades per a inicialitzar atributs
Instanciar	Crear un objecte amb `new` (que crida el constructor)
`new`	Operador que reserva memòria, crida el constructor i retorna la referència

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Atributs i mètodes · **Següent:** 05 · La paraula clau this

05 — La paraula clau this

L'objecte cridant «¡parli de mi, no d'un altre!» per a desambiguar i encadenar constructors 🗣️

> 🗺️ **Estàs en:** 🏗️ **U09 · POO: classes i objectes** → 05 · La paraula clau this

---

📬 La idea en una frase

> **`this` és l'objecte parlant de si mateix: serveix per a desambiguar entre atribut i paràmetre, per a cridar un altre constructor, i per a passar-te a tu mateix com a argument.**

Tens un problema clàssic: el paràmetre es diu igual que l'atribut. `nombre` aquí, `nombre` allà… a quin et referixes? Java necessita que li dones una pista, i aquella pista es diu `this`.

🔍 Desambiguar: atribut vs paràmetre

Quan el paràmetre d'un mètode (o constructor) es diu igual que un atribut, hi ha un **xoc de noms**. Sense `this`, Java dona preferència al paràmetre (el més pròxim), així que l'atribut es queda sense tocar:

```
public class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
         // Sense this: "nombre" és el paràmetre, l'atribut es queda amb el seu valor per defecte
        nombre = nombre;   // ❌ no fa res útil
        this.nombre = nombre;   // ✅ atribut = paràmetre
        this.edad = edad;       // ✅ atribut = paràmetre
    }
}
```

`this.nombre` significa **"l'atribut `nombre` d'AQUEST objecte"**; `nombre` a secas (a la dreta) és el paràmetre. El `this.` és la mà que assenyala: *"parli del meu, no del teu"*.

> ⚠️ **Advertència:** si no hi ha xoc de noms, `this` és opcional. Pots escriure `acelerar(int inc) { velocidad += inc; }` sense `this`, perquè `velocidad` només pot ser l'atribut. Però quan hi

> ha ambigüitat, `this` és obligatori si vols tocar l'atribut.

🔗 Cridar un altre constructor: `this(...)`

`this` també serveix per a cridar un constructor des d'un altre constructor de la mateixa classe. Útil per a no repetir codi:

```
public class Galleta {
    String forma;
    boolean tieneChocolate;

    public Galleta() {
        this("redonda", false);   // "crida l'altre constructor"
    }

    public Galleta(String forma, boolean tieneChocolate) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
    }
}
```

Ara `new Galleta()` i `new Galleta("redonda", false)` fan el mateix, però el codi només està escrit una vegada. Una regla d'or:

> 💡 **Tip:** el `this(...)` que crida un altre constructor **ha de ser la primera instrucció** del constructor. Java no deixa que facis altres coses abans. És la presentació abans de la festa: primer crides l'altre constructor i després ja fas les teves coses.

📦 Passar-te a tu mateix com a argument

`this` també pot viatjar: passes l'objecte actual com a argument a un altre mètode. És la manera que té un objecte de dir: *"escolta, passa'm a mi mateix"*.

```java
public class Cliente {
    String nombre;

    void presentarse() {
        System.out.println("Hola, soy " + nombre);
    }
}

public class Sala {
    void recibir(Cliente c) {
        c.presentarse();
    }
}
```

```java
Cliente ana = new Cliente();
ana.nombre = "Ana";
Sala sala = new Sala();
sala.recibir(ana);   // des de fora, passes la variable
```

Dins de la pròpia classe `Cliente` podries fer `sala.recibir(this)` per a passar-te a tu mateix sense tenir la variable a mà. Aquest cas et sonarà "rar" ara, però és la llavor de patrons que veuràs en mòduls més avançats. Per ara, memoritza la idea: **`this` és la referència a l'objecte actual**.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** per a decidir si cal `this`, pregunta't: *hi ha un paràmetre o variable local amb el mateix nom que l'atribut?* Si sí, `this.` assenyala l'atribut. Si no, no el necessites.

Exercici: compila o no?

Aquestes dues línies estan dins d'un constructor `Persona(String nombre)`. Una funciona i l'altra és una pèrdua de temps. Quina i per què?

```java
nombre = nombre;        // línia A
this.nombre = nombre;   // línia B
```

> 🔄 **Solució**

> La **línia B** és la que fa alguna cosa: `this.nombre = nombre` copia el paràmetre en l'atribut. La **línia A** (`nombre = nombre`) assigna el paràmetre… al paràmetre: l'atribut es queda amb el seu valor per defecte (`null`). És com intentar omplir el teu got abocant l'aigua del teu got en el teu got: les ganes no falten, però el resultat és zero.

❓ No Hi Ha Preguntes Tontes!

> **Q:** `this` és una paraula reservada, no? Puc usar-la fora d'una classe?
>
> **A:** No. `this` fora d'una classe és com demanar una pizza en una ferreteria: no té sentit. Només existeix dins del context d'un objecte, és a dir, dins de mètodes no estàtics i constructors d'una classe.

> **Q:** I què passa amb els mètodes `static`? Aquí puc usar `this`?
>
> **A:** No, i aquell és el clàssic dels exàmens. Els mètodes `static` no pertanyen a un objecte concret, així que no hi ha "aquest objecte" al qual assenyalar. Usar `this` en un `static` és error de compilació. Els `static` tenen la seva pròpia unitat: la U10.

> **Q:** Per què en el punt 3 vaig escriure mètodes sense `this` i funcionaven?
>
> **A:** Perquè no hi havia ambigüitat. En `int volumen() { return ancho * alto * profundo; }` no hi ha cap paràmetre anomenat `ancho`, així que Java només pot referir-se a l'atribut. `this` és el que parla en veu alta quan hi ha dues persones anomenades igual a la sala.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa `this.nombre`?
2. Quan és obligatori usar `this`?
3. Quina regla ha de complir el `this(...)` que crida un altre constructor?
4. Puc usar `this` dins d'un mètode `static`?

> › 🔄 **Respostes**
>
> 1. L'**atribut `nombre` de l'objecte actual**.

2. Quan hi ha un **xoc de noms**: un paràmetre o variable local amb el mateix nom que l'atribut.
3. Ha de ser la **primera instrucció** del constructor.
4. **No.** Els mètodes `static` no pertanyen a un objecte, així que no hi ha "aquest objecte" al qual assenyalar.

✅ Resum en 3 frases

1. **`this`** és la referència a l'objecte actual: `this.nombre` parla de l'atribut, `nombre` a secas del paràmetre.
2. **`this(...)`** encadena constructors de la mateixa classe i ha d'anar sempre primer.
3. No cal `this` sense ambigüitat, i està **prohibit** dins de mètodes `static`.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`this`	La referència a l'objecte actual
Desambiguar	Deixar clar a quin `nombre` et refereixes
Xoc de noms	Paràmetre i atribut amb el mateix nom
`this(...)`	Crida a un altre constructor de la mateixa classe
`static`	Context sense objecte (no admet `this`)

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Constructors · **Següent:** 06 · Referències, null i memòria

06 — Referències, null i memòria

Què guarda de veritat una variable d'objecte, què pinta el heap i per què el NullPointerException et despertarà de nit 🕳️

> 🗺️ **Estàs en:** 🏗️ **U09 · POO: classes i objectes** → 06 · Referències, null i memòria

---

📬 La idea en una frase

> **Una variable d'objecte no guarda l'objecte: guarda l'adreça (la referència) on l'objecte viu en el heap. I `null` és una referència que no apunta a res.**

En el punt 2 tenies un dubti pendent: què passa amb `Galleta g3 = g1`. Aquell dubti es diu **referències**, i és de les poques coses d'aquesta unitat que de veritat mereixen que les vegis venir. Perquè quan ho entens, el 90% dels "bugs màgics" deixen de ser màgics.

📬 La carta davant de la casa

Pensa en les variables primitives com en les adreces postals: `int a = 5` guarda el **5** directament, com una foto en la teva cartera. Però una variable d'objecte guarda **l'adreça de la casa**, no la casa sencera:

```
Galleta g = new Galleta();
```

- La galeta (l'objecte) viu en una zona de memòria anomenada **heap**.
- La variable `g` guarda un nombre que és *l'adreça* d'aquella galeta en el heap.

Aquell nombre es diu **referència**. Tu mai no el veus (Java l'amaga), però és la clau de tot el que passa en aquest punt.

> 💡 **Tip:** quan fas `new`, Java fabrica l'objecte en el heap i et lliura un "plànol de localització". La teva variable guarda aquell plànol. Copiar el plànol no copia la casa.

👯 L'àlies: dues variables, un objecte

Aquí està el moment que ho canvia tot:

```java
Galleta g1 = new Galleta();
g1.sabor = "Chocolate";

Galleta g2 = g1;          // NO crea una altra galeta
g2.sabor = "Vainilla";    // quin sabor té g1 ara?

System.out.println(g1.sabor);  // "Vainilla" 😱
```

`g2 = g1` copia la **referència**, no l'objecte. Ara `g1` i `g2` apunten a **la mateixa galeta**. Canviar el sabor amb `g2` canvia la galeta que també veu `g1`. És el famós **àlies**: dos noms, una sola galeta.

> ⚠️ **Advertència:** aquest és el clàssic que cau en els exàmens "Què imprimeix?". La regla: **copiar una variable d'objecte copia l'adreça, no l'objecte**. Si vols una còpia independent, hauries de crear un objecte nou (`new`) i copiar atribut a atribut (o usar el que veuràs en la U12).

⚰️ `null`: la referència al buit

Una variable d'objecte que encara no apunta a res val `null`:

```java
Galleta g;          // sense inicialitzar: encara no apunta a res
Galleta h = null;   // explícitament "no hi ha galeta"
Galleta i = new Galleta();   // apunta a una galeta de veritat
```

`null` significa "referència que no apunta a cap objecte". I aquí ve el monstre:

```java
Galleta h = null;
h.sabor = "Chocolate";   // 💥 NullPointerException
```

Si intentes tocar un atribut o cridar un mètode d'una referència `null`, Java llança **`NullPointerException`** (NPE): el programa mor en l'acte. És l'error més famós i odiat de Java.

> 💡 **Tip:** quan vegis un `NullPointerException` en el teu programa, busca la variable que val `null` i pregunta't: *quan havia d'apuntar a un objecte i no ho va fer?* El 90% de les vegades és un objecte que mai no es va crear amb `new` o un atribut sense inicialitzar.

🏠 Primitius vs referències: el resum

	Primitius (`int`, `double`…)	Referències (objectes)
Què guarda la variable	El **valor** directament	L'**adreça** de l'objecte en el heap
Comparar amb ==	Compara valors	Compara **adreces** (mateix objecte?)
Valor "buit"	`0`, `false`, `0.0`	`null`
Copiar	Copia el valor	Copia l'adreça (àlies)

> 📝 **Nota:** en la UO3 vas veure que `String` es compara amb `equals`, no amb ==. Ara tens el perquè: `String` és un objecte, així que == compara referències, no contingut. Els literals iguals de `String` a vegades comparteixen adreça (el *string pool*), per això `"Hola" == "Hola"` pot donar `true` per pura casualitat. No confies mai en això: usa `equals`.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** dibuixa el heap. Per cada `new`, una capseta amb la seva etiqueta. Per cada variable, una fletxa cap a la seva capseta (o cap al buit si és `null`). Després només has de seguir les fletxes.

Exercici: el lío de les fletxes

Sense executar, què imprimeix aquest programa?

```java
public class Cuenta {
    int saldo;
}

public class Main {
    public static void main(String[] args) {
        Cuenta a = new Cuenta();
        a.saldo = 100;

        Cuenta b = a;
        b.saldo += 50;

        Cuenta c = new Cuenta();
        c.saldo = a.saldo;

        a = null;

        System.out.println(b.saldo);
        System.out.println(c.saldo);
    }
}
```

> 🔄 **Solució**

Imprimeix **`150`** i **`150`**.

- `b = a` fa que tots dos apunten al mateix compte, així que `b.saldo += 50` puja el saldo d'*aquell* compte a 150 (que també veu `a`).
- `c = new Cuenta()` és un objecte nou; `c.saldo = a.saldo` copia el **valor** 150.
- `a = null` desconnecta la variable `a`, però el compte continua viu perquè `b` l'apunta. Si ningú apuntés a l'objecte, el recol·lector de brossa el reciclaria (això també existeix, i t'alegra la vida).

Resum: **b i c imprimeixen 150, però per raons distintes**: `b` perquè comparteix el compte amb `a`, `c` perquè li vas copiar el valor.

❓ No Hi Ha Preguntes Tontes!

Q: Es pot comparar una referència amb `null` per a saber si apunta a alguna cosa?

A: Sí, i és súper útil: `if (galleta == null) { ... }` o `if (galleta != null) { ... }`. És la forma de blindar el teu codi contra els NPE: *"si no és null, faig coses; si és null, salta del vaixell"*.

Q: Què passa amb els objectes que ja no tenen cap variable apuntant-los? Es queden aquí per sempre?

A: No. Java té un **recol·lector de brossa** (garbage collector) que detecta els objectes "orfe" (sense referències) i allibera la seva memòria. Tu no has de fer res: és la criada invisible de Java. En C++ això era manual i feia mal.

Q: I si faig `null` a totes les variables que apunten al mateix objecte?

A: Quan l'última referència desapareix, el recol·lector se'n porta l'objecte. És la forma de "matar" un objecte: llevar-li tots els apuntadors. No hi ha `delete` en Java; hi ha deixar de mirar-lo fins que desaparegui.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què guarda una variable d'objecte: l'objecte o la seva adreça?
2. Què fa `Galleta g2 = g1;`? Crea una altra galeta?
3. Què significa `null`?
4. Quina excepció llança Java si uses una referència `null`?

> 🔄 **Respostes**
>
> 1. L'**adreça** (la referència), no l'objecte. L'objecte viu en el **heap**.
> 2. **No.** Copia la referència: `g1` i `g2` apunten al **mateix objecte** (àlies).
> 3. Que la referència **no apunta a cap objecte**.
> 4. **`NullPointerException`** (NPE).

✅ Resum en 3 frases

1. Una variable d'objecte guarda la **referència** (l'adreça en el heap), no l'objecte; per això copiar-la crea un **àlies**.

2. **`null`** és una referència que no apunta a res, i usar-la provoca la temuda **`NullPointerException`**.
3. Els primitius guarden el valor (i es comparen amb ==); els objectes guarden adreces (i es comparen amb `equals`).

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Referència | L'adreça on viu l'objecte |
> | Heap | La zona de memòria on viuen els objectes |
> | Àlies | Dues variables apuntant al mateix objecte |
> | `null` | Referència que no apunta a res |
> | NPE | `NullPointerException`, el crash per usar `null` |
> | Recol·lector de brossa | El que recicla objectes sense referències |

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · La paraula clau this · **Següent:** 07 · Sobrecàrrega i pas per valor

07 — Sobrecàrrega i pas per valor

Diversos mètodes amb el mateix nom i què es copia de veritat quan crides (pista: les referències t'enganyen) 🎭

> 🗺️ **Estàs en: 🏗️ U09 · POO: classes i objectes** → 07 · Sobrecàrrega i pas per valor

---

📬 La idea en una frase

> **La sobrecàrrega permet que diversos mètodes comparteixin nom si els seus paràmetres difereixen, i en Java tot es passa per valor: els primitius es copien, i de les referències es copia l'adreça (no l'objecte).**

Dos conceptes que semblen d'un altre planeta i que són pa de cada dia en el codi real. El primer et dona flexibilitat (un mètode `sumar` que funciona amb `int`, amb `double` i amb tres nombres). El segon t'explica per què el teu mètode no aconsegueix "canviar" un `int` que li passes… però sí modifica un objecte. Anem a per ells.

🎭 Sobrecàrrega: el mateix nom, altres mans

Sobrecarregar un mètode és declarar diversos mètodes amb el **mateix nom** però amb **distinta llista de paràmetres** (nombre, tipus u ordre). Java decideix quin usar mirant els arguments de la crida:

```
public class Calculadora {
    int sumar(int a, int b) {
        return a + b;
    }

    int sumar(int a, int b, int c) {
        return a + b + c;
    }

    double sumar(double a, double b) {
        return a + b;
    }
}
```

```
Calculadora calc = new Calculadora();
calc.sumar(2, 3);          // usa el de (int, int) → 5
calc.sumar(2, 3, 4);       // usa el de (int, int, int) → 9
calc.sumar(2.5, 3.5);      // usa el de (double, double) → 6.0
```

> ⚠️ **Advertència:** el que **no** pot canviar per a distingir mètodes és només el tipus de retorn. `int sumar(int, int)` i `double sumar(int, int)` no poden coexistir: Java no sap quin vols si només mires `sumar(2, 3)`. El desempat es fa amb la **firma** (nom + paràmetres), no amb el que retornen.

Els **constructors** també es sobrecarreguen, i de fet ja ho vas veure en el punt 4:

```
public class Persona {
    String nombre;
    int edad;

    public Persona() {
        this("Desconocido", 0);
    }

    public Persona(String nombre) {
        this(nombre, 0);
    }

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

Tres maneres de néixer: sense dades, només amb nom, o amb tot. El `this(...)` encadenat evita repetir la inicialització.

📦 Pas per valor: Java només sap copiar

Quan crides un mètode, Java fa una còpia de cada argument i l'hi passa. Això es diu **pas per valor**: el mètode rep **còpies**, no els originals.

Amb primitius queda clar:

```
void duplicar(int x) {
    x = x * 2;   // canvia la còpia, no l'original
}

int numero = 21;
duplicar(numero);
System.out.println(numero);   // 21. La còpia es va duplicar, l'original ni se'n
va assabentar
```

Amb referències la cosa es complica. La variable guarda una adreça; es copia **l'adreça**, no l'objecte:

```
void subirSaldo(Cuenta c) {
    c.saldo += 100;   // l'objecte al qual apunta c SÍ canvia
}

Cuenta miCuenta = new Cuenta();
miCuenta.saldo = 50;
subirSaldo(miCuenta);
System.out.println(miCuenta.saldo);   // 150 ✅ (l'objecte va canviar)
```

La còpia i l'original guarden **la mateixa adreça**, així que tots dos apunten al mateix objecte: modificar l'objecte es nota fora.

> 📝 **Nota:** la trampa és aquesta: pots canviar *el contingut* de l'objecte, però **no pots** fer que la variable original apunti a un altre objecte. Si dins del mètode fas `c = new Cuenta()`, només la còpia canvia d'adreça; l'original continua apuntant al seu compte. És com prémer algú les claus de ta casa: pot redecorar la casa, però no pot canviar l'adreça de ta casa.

📊 El resum que et salva en l'examen

Què li passes	Què rep el mètode	Es nota fora?
Primitiu (`int`, `double`…)	Una **còpia del valor**	❌ No

Què li passes	Què rep el mètode	Es nota fora?
Referència	Una **còpia de l'adreça**	✅ Sí (si modifiques l'objecte) / ❌ No (si reassignes la referència)

> 💡 **Tip:** memoritza-ho així: *Java copia sempre, però copiar una adreça et permet arribar al mateix lloc.* Els primitius mai no canvien fora; els objectes canvien si els toques, però no pots canviar-los el "punt de mira" des de dins.

⭐ Be the Code, my friend... 🔗

> 🕶️ **Don Tip:** per a traçar una crida, dibuixa les caixes dels arguments originals i les caixes de les còpies del mètode. Després segueix qui apunta a qui.

Exercici: la gran confusió

Sense executar, què imprimeix aquest programa?

```java
public class Caja {
    int valor;
}

public class Main {
    static void cambiar(Caja c, int n) {
        c.valor = 99;
        n = 7;
        c = new Caja();
        c.valor = 1;
    }

    public static void main(String[] args) {
        Caja miCaja = new Caja();
        miCaja.valor = 5;
        int numero = 5;

        cambiar(miCaja, numero);

        System.out.println(miCaja.valor);
        System.out.println(numero);
    }
}
```

> 🔄 **Solució**

Imprimeix **99** i **5**.

- `c.valor = 99` modifica l'objecte compartit → l'original se n'assabenta → **99**.
- `n = 7` canvia la còpia del primitiu → l'original continua en **5**.
- `c = new Caja(); c.valor = 1;` canvia a què apunta *la còpia* `c`, però `miCaja` continua apuntant a la caixa de 99. La reassignació no surt del mètode.

Tres instruccions, tres comportaments distints. Aquest exercici té l'examen escrit al front.

❓ No Hi Ha Preguntes Tontes!

Q: Per què no puc sobrecarregar canviant només el tipus de retorn? Semblaria lògic.

> **A:** Perquè Java tria el mètode mirant els **arguments** de la crida. Si dos mètodes tenen els mateixos paràmetres però distint retorn, `calc.sumar(2, 3)` seria ambigu: quin vol el que crida? Java odia l'ambigüitat més que als dilluns.

> **Q:** Els constructors sobrecarregats "hereten" alguna cosa entre ells?
>
> **A:** No hereten, però s'**encadenen** amb `this(...)`. El constructor curt crida el llarg, que fa el treball pesat. Així el codi d'inicialització s'escriu una sola vegada. Quan vegis un constructor de tres línies que només fa `this(...)`, és aquella tècnica en acció.

> **Q:** Existeix el "pas per referència" en Java?
>
> **A:** No. Java **només** té pas per valor. El que passa és que amb referències es copia l'adreça, i això *sembla* pas per referència. Els puristes ho diuen "pas per valor de la referència". En l'examen, digues que Java és sempre pas per valor.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què és sobrecarregar un mètode?
2. Què NO pot canviar per a distingir dos mètodes amb el mateix nom?
3. En Java el pas és per valor o per referència?
4. Si un mètode rep un objecte i el modifica, es nota fora?

> 🔄 **Respostes**
>
> 1. Declarar diversos mètodes amb el **mateix nom** però distinta **llista de paràmetres**.
> 2. El **tipus de retorn**: només la firma (nom + paràmetres) desempata.
> 3. **Pas per valor** sempre: es copien els valors; amb objectes es copia l'adreça.
> 4. **Sí**, perquè la còpia de la referència apunta al mateix objecte.

✅ Resum en 3 frases

1. La **sobrecàrrega** permet diversos mètodes amb el mateix nom i distints paràmetres; el tipus de retorn no serveix per a distingir-los.

2. Java és **pas per valor**: els primitius es copien i mai no canvien fora; les referències copien l'adreça, així que pots modificar l'objecte però no reassignar-lo des de dins.
3. Els **constructors** també es sobrecarreguen i s'encadenen amb `this(...)` per a no repetir codi.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Sobrecàrrega | Diversos mètodes amb el mateix nom i distinta firma |
> | Firma | Nom + llista de paràmetres |
> | Pas per valor | Es passen còpies, mai els originals |
> | Còpia de referència | Es copia l'adreça, no l'objecte |
> | Encadenar constructors | Un constructor crida un altre amb `this(...)` |

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Referències, null i memòria · **Següent:** 08 · Be the Code: dissenya la teva classe

08 — Be the Code: dissenya la teva classe

Tanca la pàgina, obre l'editor i dissenya una classe completa de principi a fi (o pateix amb honor) 🕶️

> 🗺️ **Estàs en:** 🏗️ **U09 · POO: classes i objectes** → 08 · Be the Code: dissenya la teva classe

---

📬 La idea en una frase

> **Ja has vist la teoria: ara toca esborrar-la de la memòria i construir una classe completa tu sol, pas a pas, perquè una classe només s'entén de veritat quan et dol dissenyar-la.**

> 🕶️ **Don Tip:** tota classe es dissenya amb la mateixa escala: (1) tria el nom i els seus atributs, (2) el constructor que els inicialitza, (3) els mètodes que saben fer coses amb ells, (4) un `main` que la provi. Si domines aquella escala, qualsevol classe és teva.

🚦 Exercici guiat: el compte bancari

Tanca aquesta pàgina. Obre un editor en blanc. No miris ni una línia del que has llegit fins ara.

L'encàrrec: dissenyar una classe `CuentaBancaria` que guardi el titular i el saldo, i que sàpiga **ingressar** diners, **retirar** diners (sense deixar el saldo en negatiu) i **mostrar-se** per pantalla.

Pas 1: el nom i els atributs

Quines dades necessita un compte bancari? Mínim dues: qui el té (el **titular**, un `String`) i quants diners hi ha (el **saldo**, un `double`). Els atributs van dins de la classe, fora de qualsevol mètode:

```
public class CuentaBancaria {
    String titular;
    double saldo;
}
```

> 🧠 **Abans de seguir, respon-te a tu mateix:** el saldo hauria de ser `int` o `double`? Per què? (Pista: els diners tenen cèntims, i els cèntims no són enters.)

Pas 2: el constructor

Els atributs no han de néixer sense rumb. Un constructor que rebi el titular i el saldo inicial i els col·loque amb `this`:

```
public CuentaBancaria(String titular, double saldoInicial) {
    this.titular = titular;
    this.saldo = saldoInicial;
}
```

Pas 3: els mètodes

El compte necessita tres comportaments:

- `ingresar(double cantidad)`: suma al saldo. Hauria de rebutjar quantitats negatives (qui ingressa -50 €?).
- `retirar(double cantidad)`: resta al saldo, però **sense deixar-lo negatiu**. Si no hi ha saldo suficient, no retira i avisa.
- `mostrar()`: imprimeix "Titular: X | Saldo: Y €".

Escriu cada mètode amb la seva firma i la seva lògica. Recorda: els mètodes que només fan coses són `void`; si algun necessités retornar alguna cosa, usaria `return`.

Pas 4: prova-la amb un `main`

Cap classe està acabada fins que no l'has vist funcionar. Afegeix un `main` que crege un compte, ingressa, retire (fins i tot més del que hi ha, per a veure la protecció) i mostri el resultat.

La solució completa (intenta-ho abans de mirar)

> 🔄 **Solució del compte bancari**

```
public class CuentaBancaria {
    String titular;
    double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    void ingresar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
        } else {
             System.out.println("No puedes ingresar " + cantidad + " €. Eso es
cosa de magos.");
        }
    }

    void retirar(double cantidad) {
        if (cantidad > saldo) {
            System.out.println("No hay saldo suficiente: solo tienes " + saldo
+ " €.");
        } else {
            saldo -= cantidad;
        }
    }

    void mostrar() {
         System.out.println("Titular: " + titular + " | Saldo: " + saldo + "
€");
    }

    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Ana", 100);
        cuenta.ingresar(50);
        cuenta.retirar(200);
        cuenta.retirar(30);
        cuenta.mostrar();
    }
}
```

Sortida:

```
No hay saldo suficiente: solo tienes 150 €.
Titular: Ana | Saldo: 120 €
```

Fixa't en els detalls: l'`if` de l'`ingresar` protegeix contra quantitats negatives, l'`if` del `retirar` protegeix el saldo, i el `main` és la prova de foc que demostra que la classe funciona.

⭐ Nivells d'assoliment

- ⭐ **Ho tens, però has hagut de mirar la solució una vegada.** Aprovat raspant.
- ⭐⭐ **T'ha sortit a la primera i funciona.** Ets una màquina.
- ⭐⭐⭐ **T'ha sortit a la primera, amb els dos `if` de protecció i, a més, li has afegit un mètode `double consultarSaldo()` que retorna el saldo.** No necessites aquest curs. Ves a donar una xerrada TED.

> [!TIP] Pista mental gratuïta: una classe necessita **quatre peces** — atributs, constructor, mètodes i una prova (`main`). Si memoritzes aquella escala, qualsevol classe que et demanen en l'examen se't rendeix en cinc minuts.

🧩 El lío

El departament de qualitat ha rebut aquesta classe. Alguna cosa fa mala olor. Troba els **3 errors**:

```java
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        marca = marca;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " a " + velocidad + " km/h");
    }
}
```

> 🕶️ **Don Tip:** executa-ho amb `new Coche("Seat")` i mira què imprimeix `mostrar()`. Després revisa el constructor amb lupa.

> 🔄 **Solució del lío**
>
> 1. **El constructor es queda sense inicialitzar l'atribut**: `marca = marca` assigna el paràmetre al paràmetre (sense `this`, guanya el paràmetre). L'atribut `marca` queda en `null`. Ha de ser `this.marca = marca`.
> 2. **La velocitat mai no s'inicialitza**: Java la deixa en 0 per defecte, però això és dependre del regal. Un constructor ben educat posaria `this.velocidad = 0` (o millor, un constructor que la rebi).
> 3. **El `main` no existeix**: la classe no té cap prova. La classe compila, però ningú no pot executar-la per a veure si funciona. Tota classe sense `main`… i sense un altre programa que l'usi, és una classe que viu en l'ombra.
>
> La lliçó: **el constructor és el que "bateja" els atributs, i sense un `main` (o un programa que instancii) la teva classe és només teoria**. Els tres errors típics del novell en una sola classe.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines són les quatre peces de l'escala per a dissenyar una classe?
2. Què passa en el constructor si escrius `marca = marca` sense `this`?
3. Per què el `retirar` necessita un `if` abans de restar?
4. Quan està "acabada" una classe?

> 🔄 **Respostes**
>
> 1. **Atributs → constructor → mètodes → prova (`main`).**
> 2. Que assignes el paràmetre al paràmetre i l'atribut queda en **`null`**.
> 3. Per a **no deixar el saldo en negatiu** (o rebutjar l'operació si no hi ha fons).
> 4. Quan **funciona**, és a dir, quan un `main` (o un altre programa) la instancia i produeix el resultat esperat.

✅ Resum en 3 frases

1. Dissenyar una classe és una **escala de quatre pisos**: atributs, constructor, mètodes i prova.
2. El constructor **bateja els atributs** amb `this` quan hi ha xoc de noms; sense ell, els atributs viuen en `null` o en `0`.
3. Els mètodes **protegeixen les dades** (un `if` abans de retirar) i el `main` demostra que tot funciona.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Dissenyar una classe	Triar atributs, constructor, mètodes i prova
Batejar	Inicialitzar els atributs en el constructor
Protegir	Validar abans de modificar (no saldos negatius)
`main` de prova	El programa que demostra que la classe funciona
Iterar	Provar, trobar l'error, corregir i tornar a provar

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Sobrecàrrega i pas per valor · **Següent:** 09 · Repàs interactiu

09 — Repàs interactiu: domina les classes

El tancament pràctic de la unitat, amb objectes, constructors i un `null` que no funcionarà 😈

> 🗺️ **Estàs en: 🏗️ U09 · POO: classes i objectes** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
public class Galleta {
    String sabor;

    public Galleta(String sabor) {
        this.sabor = sabor;
    }

    void comer() {
        System.out.println("Ñam, galleta de " + sabor);
    }
}

public class Main {
    public static void main(String[] args) {
        Galleta g1 = new Galleta("chocolate");
        Galleta g2 = new Galleta("vainilla");
        Galleta g3 = g1;
        g3.sabor = "limón";
        g1.comer();
        g2.comer();
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`Ñam, galleta de chocolate` i `Ñam, galleta de vainilla`** → Cada galeta guarda el seu sabor original, sempre. ❌
2. **`Ñam, galleta de limón` i `Ñam, galleta de vainilla`** → ✅ Correcte! `g3 = g1` copia la referència, no la galeta: `g3` i `g1` són la mateixa galeta. Canviar el sabor amb `g3` ho canvia per a `g1`. `g2` és una altra galeta, independent.
3. **`Ñam, galleta de limón` i `Ñam, galleta de limón`** → El canvi de `g3` contamina totes les galetes. ❌

> 🔄 **Solució**
>
> L'opció **2**. L'assignació `g3 = g1` crea un **àlies**: dues variables, un objecte. Aquest és l'exercici que separa els qui han entès les referències dels qui aniran a plorar en l'examen.

🔥 Fireside Chat: Classe vs Objecte

> *Dos veterans del motlle i la galeta discuteixen al costat de la safata del forn.*

Classe: — Mira, jo soc el motlle. Sense mi, els teus objectes serien amorfs. Jo defineixo quins atributs i mètodes tenen tots els meus objectes. Soc la constitució del meu poble.

Objecte: — La constitució… clar, i jo soc el ciutadà que paga impostos. Tu ets un fitxer `.java` al disc. Jo ocupi memòria, tinc saldo, puc accelerar. Quan executen el programa, el que treballa soc jo.

Classe: — I quants ciutadans com tu puc produir? Milers. Jo soc únic, tu ets reproduïble. Ets una còpia, un clon, un *vulgar* duplicat.

Objecte: — Un duplicat amb vida pròpia, sí. Tu defineixes el plànol, però jo soc l'edifici. Ningú viu en un plànol. I et recorde: sense `new`, tu no ets més que teoria de fitxer.

Classe: — Sense mi, `new` no tindria res a fabricar. Ens necessitem. Com el tallagalletas i la galeta.

Objecte: — Tracte fet. Tu el motlle, jo la galeta. I que ningú s'oblide de quina es menja al final.

> La lliçó: la **classe** és el disseny (QUÈ i CÓM), l'**objecte** és la instància concreta (QUI). Sense classe no hi ha objecte; sense `new`, la classe no dona de menjar.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc el motlle que defineix atributs i mètodes, però no soc cap cosa concreta.**
2. **Soc la galeta: una cosa concreta, amb valors propis, vivint en la memòria.**
3. **Soc el mètode especial que s'executa amb cada `new` perquè l'objecte neixi ben preparat.**
4. **Soc la referència que no apunta a cap objecte; usar-me provoca l'error més famós de Java.**
5. **Soc la paraula que usa l'objecte per a parlar de si mateix i desfer líos de noms.**
6. **Soc l'adreça on viu l'objecte, la zona de memòria on Java col·loca les galetes.**

> 🔄 **Respostes**
>
> 1. **La classe** — el tallagalletas, el disseny.
> 2. **L'objecte** — la instància concreta creada amb `new`.
> 3. **El constructor** — es diu igual que la classe i no retorna res.
> 4. **`null`** — usar-la llança `NullPointerException`.
> 5. **`this`** — la referència a l'objecte actual.
> 6. **El heap** — la zona de memòria on viuen els objectes (la variable guarda la *referència*).

😡 CONRAD VS EL MÓN: "Constructor, on ets?"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre el clàssic del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, no compila*. I jo: val, què diu l'error? *Pues no ho sé, no l'he llegit.* AI, MARE MEVA! Posi la línia exacta, el motiu i fins a la fletxeta ^, i no ho llegeixes?

I després està el clàssic: escriuen un constructor amb paràmetres… i criden a `new Clase()` sense arguments. *Però si tu mateix has esborrat el constructor buit!* Quan escrius qualsevol constructor, el buit desapareix. És com llevar la porta de ta casa i després intentar entrar per la porta.

I el colmo dels colmos: `Galleta g;` i després `g.sabor = "chocolate"`. Però si `g` no apunta a RES! No has fet `new`. Això és `NullPointerException` en l'acte. És com intentar posar-li un collaret a un gos que no existeix.

La lliçó: abans d'acusar Java de "odiar-te", repassa el trio sagrat: **vaig fer `new`? el constructor existeix amb els arguments que passi? la referència és `null`?** El 90% dels errors d'aquesta unitat s'arreglen amb aquestes tres preguntes. El compilador no t'odia: t'està passant les respostes de l'examen.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Quants objectes crea `Galleta a = new Galleta(); Galleta b = a;`?
   - a) 2 b) 1
2. Què imprimeix `System.out.println(new Galleta("chocolate").sabor);`?
   - a) `chocolate` b) `null`
3. `Persona p;` sense `new`. Quant val `p`?
   - a) `0` b) `null`
4. Puc escriure `int duplicar(int x)` i `double duplicar(int x)` en la mateixa classe?
   - a) Sí, canvia el retorn b) No, mateixa firma

> 🔄 **Solucions**
>
> 1. **b)** — `b = a` copia la referència: un sol objecte, dos noms.
> 2. **a)** — L'objecte es crea, s'inicialitza el seu `sabor` amb `"chocolate"` i s'hi accedeix a l'instant.
> 3. **b)** — Les referències sense inicialitzar valen `null`, no `0` (això era per als primitius).
> 4. **b)** — El tipus de retorn no forma part de la firma: mateixa firma, error de compilació.

⚡ Laboratori de Tortura: la fàbrica que plora

> **Durada estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte quan el arregles.

```
public class Coche
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad -= inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad);
    }

    public static void main(String[] args) {
        Coche c = new Coche();
        c.acelerar(50);
        c.mostrar()
    }
}
```

Fallada intencionada: un dels errors sembla correcte a simple vista perquè “es veu bé”, però canvia per complet el comportament del programa.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun ; que falti? *no → segueix buscant.*
   > **› I si encara estic atascat?**
   > Comprova també les claus `{}`: la classe necessita la seva obertura.

2. Compila ja? *no → mira el missatge d’error i els arguments del constructor.*
   > **› I si encara estic atascat?**
   > `new Coche()` no existeix: l'únic constructor demana `(String, int)`. És l'error "el buit va desaparéixer".

3. Executa però la velocitat surt rara? *És l’error de lògica: el signe del mètode.*

> **Solució final**

Els **3 errors de compilació**:

1. Falta la `{` d'obertura de la classe després de `Coche`.
2. `new Coche()` no coincideix amb el constructor: l'únic és `Coche(String, int)`. En escriure un constructor amb paràmetres, el buit desapareix.
3. Falta el `;` al final de `c.mostrar()`.

L'**error de lògica**: `velocidad -= inc` **resta** en lloc de sumar. Compila i executa perfectament, però el cotxe accelera "cap enrere": amb la velocitat inicial a 0 i accelerar 50, imprimeix `Seat va a -50` en lloc de `Seat va a 50`. Un signe separava el teu cotxe de la veritat.

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad);
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat", 0);
        c.acelerar(50);
        c.mostrar();
    }
}
```

Sortida correcta: `Seat va a 50`. Amb la versió trencada, una vegada arreglats els altres errors, la velocitat sortia `-50`: el signe era la pista de l'error de lògica.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **El Tallagalletas**	Explicar la diferència entre classe i objecte amb l'analogia de les galetes sense mirar
🏅 **Primera Classe**	Escriure una classe amb atributs, constructor i mètodes que funcioni a la primera
🏅 **Caçador de NPE**	Trobar el `null` culpable d'un `NullPointerException` sense pistes
🏅 **El Desambiguador**	Explicar per què cal `this` quan hi ha xoc de noms
🏅 **El Constructor Rudo**	Dissenyar una classe amb sobrecàrrega de constructors encadenats amb `this(...)`

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```
public class Cuenta {
    int saldo;

    void sumar(int x) {
        saldo += x;
    }
}

public class Main {
    public static void main(String[] args) {
        Cuenta a = new Cuenta();
        a.saldo = 100;
        Cuenta b = a;
        b.sumar(50);
        System.out.println(a.saldo);
    }
}
```

2. **El constructor perdut:** en l'exemple `Galleta` del punt 2 (sense constructor), què li passa al codi si afegeixes `public Galleta(String sabor)` i deixes `new Galleta()` en un altre lloc?
3. **El detectiu del heap:** el teu programa imprimeix `null` quan esperaves un nom. Quina eina uses i què mires primer?
4. **Vertader o fals:** "si un mètode rep un objecte i dins del mètode faig `parametro = null`, l'objecte original desapareix".

> 💡 **Solucions**
>
> 1. **`150`**. `b = a` crea un àlies: `b.sumar(50)` modifica el compte que també veu `a`.
> 2. **Error de compilació** en `new Galleta()`: en escriure un constructor amb paràmetres, el constructor buit desapareix.
> 3. El **depurador**: posa un breakpoint on s'assigna el nom i observa si l'objecte va arribar a crear-se amb `new` o si l'atribut va quedar en `null`. (O llegeix el stack trace si hi ha un NPE.)
> 4. **Fals.** `parametro = null` només canvia la *còpia* de la referència: l'objecte original continua viu mentre una altra variable l'apunti. Perquè desaparegui, cap referència no li ha d'apuntar.

🧩 Crucigrama de Bits

```
Horitzontal:
1. Operador que fabrica objectes en memòria (3 lletres)
3. El mètode que prepara l'objecte en nàixer (11 lletres)
5. Referència que no apunta a res (4 lletres)
6. Zona de memòria on viuen els objectes (4 lletres)

Vertical:
2. Paraula que usa l'objecte per a parlar de si mateix (4 lletres)
4. El motle que defineix atributs i mètodes (5 lletres)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. NEW · 3. CONSTRUCTOR · 5. NULL · 6. HEAP **Vertical:** 2. THIS · 4. CLASE

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java júnior.

1. **"Explica'm, com si jo fos la teva àvia, la diferència entre una classe i un objecte."**
2. **"Què fa `new` per dins quan crees un objecte?"**
3. **"Per què `String` es compara amb `equals` i no amb ==?"**
4. **"Java passa els arguments per valor o per referència? Justifica amb un exemple d'objecte."**
5. **"Si dues variables apunten al mateix objecte i una li canvia un atribut, què veu l'altra?"**
6. **"Per a què serveix sobrecarregar un constructor i com l'encadenes amb `this`?"**

🤷 No hi ha preguntes tontes

> ❓ **Puc escriure diverses classes en un mateix fitxer `.java`?**

Sí, però només una pot ser `public` (la que dona nom al fitxer). Les altres van sense `public`. Per als projectes de classe, un fitxer = una classe pública, i cada classe en el seu fitxer. Els teus professors ho agrairan.

> ❓ **Per què a vegades veig `get` i `set` abans dels noms de mètodes, com `getSaldo()`?**

Aquella és la convenció dels **getters i setters**: mètodes per a llegir (`getSaldo`) i modificar (`setSaldo`) atributs de forma controlada. Sona a burocràcia, però és la base de l'**encapsulació**, i té la seva unitat sencera: la U10. Aquí els veuràs de passada; allà els dominaràs.

> ❓ **Puc tenir un mètode anomenat igual que el constructor?**

No. El constructor sempre es diu igual que la classe i no retorna res. Un mètode normal no pot tenir el nom de la classe (a menys que sigui… un constructor, que es detecta per la falta de tipus de retorn). Si intentes afegir-li un `void` davant al constructor, Java et mirarà rar.

🎬 Post-Crèdits

La programadora acaba la seva fàbrica de galetes virtual: una classe `Galleta` amb constructor, una classe `Horno` que la instancia, i un `main` que forgi cinc galetes distintes. Cada

una amb el seu sabor. Fins que, en l'última línia, se li escapa un `null` i tot s'apaga amb un `NullPointerException`.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa humejant.

CONRAD: — Cinc galetes, cinc objectes, i només un `null` per a espatlar-ho tot. Típic. Saps què ha passat?

Programadora: — No vaig fer `new`. La referència apuntava al buit.

CONRAD: — *assenteix* Veus. Quan entens què guarda una variable, els errors deixen de ser màgia. Classe, objecte, constructor, `this`, referències… Ja no escrius receptes, crees mons. Però això no ha fet més que començar.

Programadora: — Què més hi ha?

CONRAD: — *fa un glop* Ara els teus atributs estan a l'aire. Qualsevol els toca. La pròxima parada és saber qui té permís per a entrar i qui no. Espera'm, que vaig a gaudir veient-te protegir les teves dades.

PRÒXIMAMENT EN U10: Visibilitat, encapsulació i `static`. El moment en què els teus objectes aprenen a dir "això no es toca". 🔒

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Be the Code: dissenya la teva classe · **Següent:** U10 · Visibilitat, encapsulació i static

U10 — Visibilitat, encapsulació i static

Els cadenats del teu codi: qui veu els teus atributs, com blinda'ls i què pertany a la classe i no a l'objecte 🔒

Benvingut a la unitat dels cadenats. En la U09 vas aprendre a crear classes i objectes: classes amb atributs i mètodes que es comportaven com *coses* del món real. Però hi havia un problema amagat: **tot era públic**. Qualsevol podia fer `persona.edad = -666` i el teu objecte quedava fet un desastre. Viure en una casa de vidre és incòmode, oi? Doncs el teu codi també mereix portes, claus i un bon cadenat.

Aquesta unitat té dues mitats que al final ballen juntes:

- **Visibilitat i encapsulació:** aprendràs a amagar els teus atributs amb `private` i a controlar com es llegeix i es modifica amb `public` (getters i setters). És el pilar que sosté la POO: el teu objecte decideix què mostra i què guarda, i ningú deixa la teva classe en un estat impossible.
- **Static:** aprendràs la diferència entre el que pertany a cada **objecte** (cada gat té el seu nom) i el que pertany a la **classe** (tots comparteixen el comptador de gats). Atributs estàtics, mètodes estàtics i constants `static final` que ni el temps s'atreveix a tocar.

Pel camí descobriràs per què `main` és `static` encara que no hauria de ser-ho, com es construeix una classe utilitària com `Math` (que ningú pot instanciar), i per què els experts diuen que els atributs `public` són com portar la cartera oberta al metro.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar per què **amagar** els teus atributs és bona idea (i no paranoia).
- Distingir els **4 nivells de visibilitat**: `public`, `protected`, package-private i `private`.
- **Encapsular** classes amb atributs `private` i controlar l'accés amb getters i setters `public`.
- **Validar** dades en els setters perquè el teu objecte mai no visqui en un estat impossible.
- Conèixer el **patró JavaBeans** i les convencions de noms.
- Usar **atributs i mètodes estàtics** (`static`) per al que pertany a la classe, no a l'objecte.

- Entendre per què `main` és `static` i crear **classes utilitàries** amb constructor privat.
- Declarar **constants de classe** amb `static final` i saber quan usar-les.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Visibilitat: l'art de no ensenyar-ho tot	La casa de vidre: el problema d'ensenyar tots els teus atributs al món	Tots
02 · public, private i protected	Els 4 nivells de visibilitat i qui veu cada cosa de la teva classe	Tots
03 · Encapsulació: el pilar de la POO	Atributs privats, control d'accés i el patró JavaBeans	Tots
04 · Getters i setters	El porter de la discoteca: llegir i modificar atributs amb validació	Tots
05 · Atributs static	El que pertany a la classe, no a l'objecte: el grup de WhatsApp	Tots
06 · Mètodes static i el main	Cridar mètodes sense crear objectes i per què `main` és estàtic	Tots
07 · Constants de classe: static final	Valors immutables en MAJÚSCULES i les classes utilitàries	Tots
08 · Be the Code: encapsula la teva classe	Converteix una classe sense blindar en una ben encapsulada, pas a pas	⭐⭐⭐
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

---

✅ Criteris d'avaluació coberts (RA4)

RA4: Desenvolupa programes organitzats en classes analitzant i aplicant els principis de la programació orientada a objectes.

CE	Criteri	On es cobreix
RA4 a)	S'ha reconegut la sintaxi, estructura i components típics d'una classe.	✅ Punt 1
RA4 b)	S'han definit classes.	✅ Punts 3 i 8
RA4 c)	S'han definit propietats i mètodes.	✅ Punts 2, 3 i 4
RA4 d)	S'han creat constructors.	✅ Punts 3 i 4
RA4 e)	S'han desenvolupat programes que instancien i utilitzen objectes de les classes creades anteriorment.	✅ Punts 8 i 9
RA4 f)	S'han utilitzat mecanismes per a controlar la visibilitat de les classes i dels seus membres.	✅ Punts 1 i 2
RA4 h)	S'han creat i utilitzat mètodes estàtics.	✅ Punts 5, 6 i 7

> 📌 L'herència (RA4 g) i les interfícies (RA4 i) tenen el seu capítol propi en la **U11**. Els conjunts i llibreries de classes (RA4 j) floreixen en la **U12**. Aquí ens quedem amb visibilitat, encapsulació i static, que són els fonaments de tot el demés.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites el que vas veure en la U09.
- Ja blindes atributs i vols cim? → Ves directe al punt 5 i al punt 6: aquí comença el món estàtic.
- Només vens a per els cadenats? → Salta al punt 3 i al punt 4.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Visibilitat: l'art de no ensenyar-ho tot

**⏭️ En acabar la unitat, continua en U11 · Herència, polimorfisme i interfícies.

01 — Visibilitat: l'art de no ensenyar-ho tot

Viure en una casa de vidre està fatal: el teu codi també mereix portes, claus i cadenats 🏠

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 01 · Visibilitat: l'art de no ensenyar-ho tot

---

📬 La idea en una frase

> **Si tot en la teva classe és `public`, qualsevol persona (o tu mateix dins d'un any) pot trencar els teus objectes. La visibilitat és l'art de decidir què mostra la teva classe i què es queda a casa.**

En la U09 vas construir classes amb atributs i mètodes. Però hi ha una pregunta que no et vas fer: *i si algú li posa `edad = -666` al meu objecte Persona?* Si tot és públic, qualsevol pot fer-ho des de qualsevol lloc. I aquí és on comença aquesta unitat: en el moment en què el teu codi necessita **portes**.

🏠 El problema de la casa de vidre

Imagina que vius en una casa de vidre. Qualsevol pot veure-ho tot: la teva roba interior, la teva col·lecció de cromos, aquella caixa de galetes buida que guardes "per si de cas". Incòmode, oi? Doncs el mateix passa amb els teus objectes si tot és públic:

```
Persona p = new Persona();
p.edad = -666;          // Edat negativa... Això no té sentit.
p.saldo = 999999999;    // I a més et multipliques el saldo per zero.
```

El teu objecte queda fet un desastre. Ningú no ha comès un error greu: el problema és que **la teva classe ho permetia**. No vas posar cadenat. La visibilitat és exactament això: decidir quines parts de la teva classe són d'accés públic i quines estan fora de la vista de tot el món.

> ⚠️ **Advertència:** un atribut `public` és com deixar la porta de casa teva oberta amb un cartell que diu "Passeu i toqueu-ho tot". Tard o d'hora, algú ho farà.

🚪 Per què importa la visibilitat

Val, i per què no ho deixem tot `public` i tan contents? Per tres motius que en el món laboral són or pur:

1. **Control:** decideixes què pot llegir-se, què pot modificar-se i què ni es toca. Res d'edats negatives ni saldos impossibles.
2. **Seguretat:** ningú no pot deixar el teu objecte en un estat inconsistent (per exemple, un compte amb saldo negatiu).
3. **Mantenibilitat:** si canvies per dins com es calcula alguna cosa, el codi de fora no s'assabenta. Els canvis interns no trenquen els teus usuaris.

En els pròxims punts veuràs les eines exactes (modificadors, getters i setters, `static`). Però primer necessites el concepte: **la teva classe és una caixa, i tu decideixes qui té la clau**.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** la visibilitat no és un caprici: és el contracte que la teva classe firma amb la resta del món. Abans d'escriure `public` en un atribut, pregunta't: *de veritat vull que qualsevol el pugui canviar?*

Exercici: el supermercat sense portes

Aquí tens una classe de la U09 escrita "a la bruta", amb tot públic:

```
public class Carrito {
    public String cliente;
    public int total;          // en cèntims
    public int articulos;
}

public class Supermercado {
    public static void main(String[] args) {
        Carrito c = new Carrito();
        c.cliente = "Pepita";
        c.total = -5000;       // Una compra negativa? És clar!
        c.articulos = -3;      // -3 articles, genial.
    }
}
```

Preguntes (sense executar):

1. Què té de rar `c.total = -5000`?
2. Quins danys col·laterals tindria que un `main` de qualsevol classe pogués fer això en un programa de veritat?
3. Qui hauria d'impedir-ho: el `main` o la pròpia classe `Carrito`?

> 🔄 **Solució**
>
> 1. Un total de -5000 cèntims és impossible en la realitat: ningú no et paga per emportar-te la compra. L'atribut accepta qualsevol valor perquè és `public` i no hi ha filtri.
> 2. En un programa gran, un altre company (o tu dins de tres mesos) podria ficar valors impossibles, trencar la lògica de preus i fer que el bug aparegués a quilòmetres del lloc on es va originar. Trobar-lo costaria hores.
> 3. La pròpia classe. Ella és la que coneix les seves regles (un total no pot ser negatiu), així que ella ha de blindar-se. Això és la **encapsulació** dels punts 3 i 4.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin problema té que tots els atributs d'una classe siguin `public`?
2. Què guanyes en amagar la implementació interna de la teva classe?
3. Qui és el responsable que una classe no accepti valors impossibles?
4. Per què és important la visibilitat per al manteniment del codi?

> 🔄 **Respostes**
>
> 1. Qualsevol classe pot modificar els atributs directament i deixar l'objecte en un estat impossible (edats negatives, saldos inventats…).
> 2. Control, seguretat i mantenibilitat: pots canviar com funciona per dins sense que el codi de fora s'assabenti ni es trenqui.
> 3. La pròpia classe. És la que coneix les seves regles de negoci, així que és la que ha de validar i amagar.
> 4. Perquè els canvis interns no trenquen el codi que usa la classe: el "contracte" públic es manté estable.

✅ Resum en 3 frases

1. La **visibilitat** decideix quines parts de la teva classe pot veure i usar la resta del món.
2. Amb tot `public`, qualsevol codi pot trencar els teus objectes amb valors impossibles.
3. Amagar dades et dona **control, seguretat i mantenibilitat**: els tres superpoders del programador que no plora debuggejant.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Visibilitat | Què es veu d'una classe des de fora |
> | `public` | Visible per a tots, absolutament tots |
> | Atribut | La "caixa" de dades de la classe (ex. `edad`) |
> | Estat inconsistent | Un objecte amb valors impossibles (ex. `edad = -5`) |
> | Contracte públic | El que la teva classe promet al món exterior |

📚 Tornar a l'índex de la unitat · **Anterior:** Índex de la unitat · **Següent:** 02 · public, private i protected

02 — public, private i protected

Els 4 nivells de visibilitat: de la tanca publicitària a Times Square al diari amb cadenat 🔐

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 02 · public, private i protected

📬 La idea en una frase

> **Hi ha 4 nivells de visibilitat en Java — `public`, `protected`, package-private i `private` — i cada un és com un tipus de pany: decideix qui té la clau de cada membre de la teva classe.**

En el punt 1 vas veure el problema de la casa de vidre. Ara toca el kit de panys. Java et dona exactament **quatre nivells** de visibilitat, de la tanca publicitària al diari amb cadenat.

🗂️ Els 4 nivells: de la tanca al diari

Modificador	Es veu des de	És com…
`public`	Tot el món, absolutament tot	Una tanca publicitària a Times Square
`protected`	Mateix paquet + subclasses (fills)	Els secrets de família: ho saben els teus cosins i els teus fills
(res) package-private	Mateix paquet (el veïnat)	El xafardeig del barri
`private`	Només la pròpia classe	El teu diari secret amb cadenat

El detall important: **si no posis res, Java assumeix package-private**. No existeix una paraula `default` per a això (aquella paraula és per a una altra cosa), així que "sense modificador" = "veïnat".

> 💡 **Detall pràctic:** com més `private` siguis, més segur estàs. La regla d'or del programador paranoic: *comença amb l'accés més restrictiu i obre'l només si hi ha motiu.*

📣 public: la tanca publicitària

Tot el món ho veu. Des de qualsevol classe, qualsevol paquet, qualsevol racó de l'univers Java. És com posar el teu número de telèfon en una pancarta:

```
public class VallaPublicitaria {
    public String mensaje;

    public void mostrar() {
        System.out.println(mensaje);
    }
}
```

Usa-ho per al que **vols** que altres usen: mètodes de servei, la porta d'entrada de la classe, el `main`… No per als teus atributs, a menys que t'agrade el caos.

🔒 private: el diari amb cadenat

Només la classe ve els seus propis `private`. Ni sa mare, ni el seu millor amic, ni el gos:

```
public class DiarioSecreto {
    private String contenido;

    public void escribir(String mensaje) {
        this.contenido = "Querido diario: " + mensaje;
    }

    public String leer() {
        return contenido;
    }
}
```

Des de fora no pots fer `diario.contenido` directament: el compilador t'ho prohibeix. Només pots usar el que la classe exposa (`escribir()` i `leer()`).

> ⚠️ **Advertència:** mai, MAI, facis un atribut `public` "perquè és més ràpid". És més ràpid d'escriure i el doble de lent de depurar: quan algú meta un valor impossible, passaràs hores buscant qui ho va canviar.

👨‍👩‍👧 protected: els secrets de família

És com les històries vergonyoses de la família. Els teus cosins (mateix paquet) i els teus fills (subclasses) poden accedir. Però un desconegut d'un altre paquet… no. En la U11 (herència) l'esprémeres de debò; per ara, queda't amb la idea: *protegit per a la família*.

```
package zoologico;

public class Animal {
    protected String nombre; // Ho veuen el paquet i les subclasses
}
```

🏘️ package-private: el xafardeig del barri

Si no posis cap modificador, ho veuen les classes del **mateix paquet**. És com el grup de WhatsApp del veïnat: els del bloc ho saben tot; els de l'altra ciutat, ni idea.

```
package barrio;

public class Casa {
    public String direccion;          // Ho sap tot el món
    protected String telefono;        // Ho sap la família
    int numeroHabitaciones;           // Ho saben els veïns (package-private)
    private String contrasenaWifi;    // NOMÉS JO
}
```

Des del mateix paquet (`barrio`):

```
public class Vecino {
    public void espiar() {
        Casa c = new Casa();
        System.out.println(c.direccion);          // OK: public
          System.out.println(c.telefono);              // OK: protected (mateix
paquet)
        System.out.println(c.numeroHabitaciones); // OK: package-private
        // System.out.println(c.contrasenaWifi);  // ERROR: private
    }
}
```

Des d'un altre paquet, sent subclasse:

```
package otraCiudad;
import barrio.Casa;

public class CasaHeredada extends Casa {
    public void espiar() {
        System.out.println(direccion);            // OK: public
           System.out.println(telefono);                  // OK: protected (soc
subclasse)
        // System.out.println(numeroHabitaciones); // ERROR: package-private
        // System.out.println(contrasenaWifi);     // ERROR: private
    }
}
```

📝 **Nota:** els **mètodes** també tenen visibilitat. Un mètode `private void calcularImpuesto()` només s'usa dins de la classe; ningú fora necessita saber com calcules els impostos (ni tu mateix vols saber-ho).

⭐ Be the Code, my friend...

🕶️ **Don Tip:** quan tinguis dubtes de "qui veu això?", fes-te dues preguntes: *està l'altra classe en el mateix paquet?* i *és una subclasse?* Amb aquelles dues respostes, tens el pany resolt.

Exercici: el banc espia

Ets la classe `Banco`. Tens aquests membres:

```java
public class Banco {
    public String nombreBanco;
    protected String direccionSucursal;
    String listaClientes;
    private double saldoCaja;

    public void mostrarInfo() {
        System.out.println(nombreBanco);
        System.out.println(direccionSucursal);
        System.out.println(listaClientes);
        System.out.println(saldoCaja);
    }
}
```

Preguntes (sense executar):

1. Pot una classe `Sucursal` d'un altre paquet veure `listaClientes`?
2. I `direccionSucursal`?
3. Pot el `main()` d'una classe del mateix paquet veure `saldoCaja`?

> 🔄 **Solució**
>
> 1. **No.** `listaClientes` és package-private: només ho veuen les classes del mateix paquet.
> 2. **Sí**, però només si `Sucursal` és subclasse de `Banco` (perquè és `protected`). Si és una classe solta d'un altre paquet, tampoc.
> 3. **No.** `saldoCaja` és `private`: només la pròpia classe `Banco` pot veure-ho. Per això `mostrarInfo()` funciona: és un mètode de la classe i la classe sempre es veu a si mateixa sencera.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quants nivells de visibilitat té Java i com es diuen?
2. Quin nivell assumeix Java si no escrius cap modificador?
3. Qui pot veure un membre `protected`?
4. Pot un mètode ser `private`? Per a què serveix?

> 🔄 **Respostes**
>
> 1. Quatre: `public`, `protected`, package-private (sense modificador) i `private`.
> 2. **Package-private**: ho veuen les classes del mateix paquet.
> 3. Les classes del **mateix paquet** i les **subclasses** (encara que estiguin en un altre paquet).
> 4. Sí. Un mètode `private` és una peça interna de la classe que ningú de fora necessita: per exemple, `private void recalcularSaldo()`. Aïlla la complexitat i evita que altres usen el que no deuen.

✅ Resum en 3 frases

1. Els **4 nivells** de visibilitat són `public` > `protected` > package-private > `private`.
2. `private` és el més restrictiu (només la classe) i `public` el més obert (tot el món).
3. **Sense modificador, Java assumeix package-private**, i la bona pràctica és començar privat i obrir només el necessari.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Modificador d'accés	Paraula que decideix qui veu un membre: `public`, `protected`, `private`
Package-private	Visibilitat "per defecte": només el mateix paquet
Paquet	Carpeta de classes (`package barrio;`)
Subclasse	Classe que hereta d'una altra (`extends`)
Membre	Un atribut o un mètode de la classe

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Visibilitat: l'art de no ensenyar-ho tot · **Següent:** 03 · Encapsulació: el pilar de la POO

03 — Encapsulació: el pilar de la POO

Atributs privats, accés controlat i el patró JavaBeans: la recepta perquè la teva classe mai no es trenqui 📦

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 03 · Encapsulació: el pilar de la POO

📬 La idea en una frase

> **Encapsular és amagar les dades (atributs `private`) i oferir mètodes controlats (`public`) per a accedir-hi. És el pilar que sosté tota la POO.**

Ja tens l'eina (`private`) i ara toca l'estratègia completa. Encapsulació = **privacitat + control**. No basta amagar: cal decidir com s'entra i com s'surt.

📦 La fórmula de l'encapsulació

La recepta té tres passos:

1. Els teus **atributs** són `private`. Ningú no els toca directament.
2. Controles l'accés amb **getters i setters** `public` (els veuràs en detall en el punt 4).
3. Dins dels setters i mètodes, **valides** perquè l'objecte mai no tingui un estat impossible.

Exemple de vida (o de mort), un banc que no vol fer fallida:

```java
public class CuentaBancaria {
    private double saldo;

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            this.saldo += cantidad;
        }
    }

    public void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            this.saldo -= cantidad;
        } else {
            System.out.println("No tens tants diners, amic.");
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

El saldo és `private`: **ningú** no pot fer `cuenta.saldo = -1000` des de fora. L'única manera de canviar-lo és passar per `ingresar()` i `retirar()`, que són els porters que validen.

> ⚠️ **Advertència:** si fas els atributs `public`, estàs renunciant a l'encapsulació. És com portar la cartera oberta al metro: tard o d'hora, algú fica mà.

🎁 Els avantatges (o per què no dormiràs pitjor)

- **Control:** valides i filtres. Res d'edats negatives ni saldos inventats.
- **Mantenibilitat:** canvies la implementació interna i el codi client ni s'assabenta.
- **Seguretat:** ningú no deixa el teu objecte en un estat inconsistent.
- **Baix acoblament:** cada classe va a la seva. No es fiquen unes en els assumptes d'unes altres.

> 💡 **Detall pràctic:** l'encapsulació et permet canviar com funciona un mètode per dins (per exemple, que `getSaldo()` calculi en comptes de guardar) sense que la resta del programa es trenqui. Aquella llibertat val or en projectes grans.

🧬 La convenció JavaBeans: el protocol

JavaBeans és una convenció (no obligatòria, però sí sensata) que diu que una classe "ben educada":

1. És **pública**.
2. Té **constructor sense arguments**.
3. Els seus **atributs són privats**.
4. Exposa **getters i setters públics**.
5. (Opcional) implementa `Serializable`.

No et deixes enganyar pel nom: no és una llibreria màgica, és un **estil**. Moltes eines i frameworks (i exàmens) esperen que les teves classes el segueixen. El punt 4 t'ensenya la convenció de noms exacta.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan una classe "ho deixa passar tot", sol ser perquè el seu autor va confiar massa que els altres escriurien bé. No confies mai: *valida sempre a la frontera*.

Exercici: el guardiola que perd diners

Aquesta classe pretén estar encapsulada, però amaga un fallo gros:

```
public class Hucha {
    private int cantidad;

    public void anadir(int monedas) {
        cantidad = cantidad + monedas;
    }

    public int getCantidad() {
        return cantidad;
    }
}
```

Preguntes (sense executar):

1. Què passa si algú crida `hucha.anadir(-50)`?
2. Què li faltaria a `anadir()` per a estar ben blindada?
3. I si volguérem que ningú no pogués llegir la quantitat directament i només mostrés un missatge?

> 🔄 **Solució**
>
> 1. El guardiola quedaria amb `cantidad = -50`: has ficat diners negatius en un guardiola. Impossible en la vida real, permés en el codi.
> 2. Validar abans de sumar: `if (monedas > 0) { cantidad += monedas; }`. Mai no acceptis diners negatius.
> 3. És decisió de disseny: podries tenir `getCantidad()` públic i, si només vols mostrar, un mètode `mostrarCantidad()` que imprimeixi. Encapsular és decidir exactament això: què es veu, què es canvia i com.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quins són els 3 passos de l'encapsulació?
2. Què és el patró JavaBeans i per a què serveix?
3. Per què `CuentaBancaria` no deixa fer `saldo = -1000`?
4. Què guanyes en mantenibilitat en encapsular?

> 🔄 **Respostes**
>
> 1. Atributs `private`, accés controlat amb mètodes `public` (getters/setters) i validació a la frontera.
> 2. Una convenció d'estil: classe pública, constructor sense args, atributs privats i getters/setters públics. És el "protocol" que esperen moltes eines.
> 3. Perquè el saldo és `private`: des de fora és impossible tocar-lo. Només `ingresar()` i `retirar()` el modifiquen, i aquells mètodes validen.
> 4. Pots canviar com funciona la teva classe per dins sense que el codi de fora es trenqui ni s'assabenti.

✅ Resum en 3 frases

1. **Encapsular** = atributs `private` + mètodes `public` controlats + validació.
2. Dona **control, mantenibilitat, seguretat i baix acoblament**.
3. El **patró JavaBeans** és la convenció estàndard que seguiran les teves classes a partir d'ara.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Encapsulació | Amagar dades i controlar l'accés a elles |
> | Estat | El conjunt de valors dels atributs d'un objecte |
> | JavaBeans | Convenció: classe pública, atributs privats, getters/setters |
> | Acoblament | Quant depenen unes classes d'unes altres |
> | Frontera | Els mètodes públics per on entra i surt la informació |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · public, private i protected · **Següent:** 04 · Getters i setters

04 — Getters i setters

El porter de la discoteca: llegir i modificar atributs privats amb nom, ordre i validació 🚪

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 04 · Getters i setters

---

📬 La idea en una frase

> **Un getter llegeix un atribut privat i un setter el modifica, i tots dos tenen noms amb una convenció estricta. Són els porters de la teva discoteca: deixen entrar a qui volen i tiren els que van borratxos.**

Els atributs són `private`, però llavors… com llegeix el món la teva classe? Com canvia l'edat d'un `Estudiante`? Aquí entren els **getters** (per a llegir) i els **setters** (per a escriure). I no, no valen noms inventats: Java i els seus frameworks esperen una convenció.

🏷️ La convenció de noms (memoritza-la)

Tipus d'atribut	Getter	Setter
`String nombre`	`getNombre()`	`setNombre(String n)`
`boolean activo`	`isActivo()`	`setActivo(boolean a)`
`int cantidad`	`getCantidad()`	`setCantidad(int c)`

Regles d'or:

- El getter es diu `get` + atribut en majúscula inicial. Amb `boolean`, s'usa `is` en comptes de `get`.
- El setter es diu `set` + atribut en majúscula inicial i rep el valor com a paràmetre.
- Retornen/reben **exactament el tipus de l'atribut**.

```java
public class Estudiante {
    private String nombre;
    private boolean matriculado;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public boolean isMatriculado() {
        return matriculado;
    }

    public void setMatriculado(boolean matriculado) {
        this.matriculado = matriculado;
    }
}
```

> 📝 **Nota:** `this` (el punt 5 de la U09) resol el lío que paràmetre i atribut es diguin igual. `this.nombre` és l'atribut; `nombre` a seques és el paràmetre. Sense `this`, t'assignaries el paràmetre a si mateix i l'atribut es quedaria amb el seu valor per defecte.

🚪 El setter com a porter: validar

Un setter no és només "escriure". És la **frontera on valides**. Exemple clàssic:

```java
public class Termometro {
    private double temperatura;

    public double getTemperatura() {
        return temperatura;
    }

    public void setTemperatura(double temperatura) {
        if (temperatura < -273.15) {
                System.out.println("Impossible: això està per davall del zero
absolut.");
        } else {
            this.temperatura = temperatura;
        }
    }
}
```

Si algú intenta `termometro.setTemperatura(-500)`, el setter el frena: el termòmetre mai no mostrarà una temperatura impossible. Sense el setter, l'atribut `public` hauria acceptat aquell -500 sense dir res.

> ⚠️ **Advertència:** un setter que no valida és un porter adormit: deixa passar a qualsevol. Si el teu setter accepta edats negatives, en la pràctica és com si l'atribut fos `public`.

🔒 Getter sense setter: atributs immutables

I si vols que alguna cosa es llegeixi però **no es pugui canviar**? Doncs poses getter i t'estalvies el setter. Punt. Així aconsegueixes atributs de només lectura:

```java
public class Tarjeta {
    private String numero;
    private int pin;

    public Tarjeta(String numero, int pin) {
        this.numero = numero;
        this.pin = pin;
    }

    public String getNumero() {
        return numero;
    }

    public int getPin() {
        return pin;
    }
    // Sense setters: el número i el pin es fixen en nàixer i mai no canvien.
}
```

> 💡 **Detall pràctic:** els atributs que es fixen en el constructor i no han de canviar (un DNI, un id, un pin) només porten getter. És la forma elegant de dir "això és de només lectura".

🤷 No hi ha preguntes tontes

> ❓ **Per què tant de rotllo amb els noms? No puc dir als meus mètodes `leerEdad()` i `ponerEdad()`?**

Pots, i funcionarien. Però la convenció `getX()` / `setX()` és la que esperen els frameworks (i, de pas, els examinadors). Quan en la U15 vegis com es serialitzen objectes a JSON, entendràs per què dir-se de la manera estàndard estalvia maldecaps.

> ❓ **El setter sempre ha de validar amb `println`?**

No. Una altra opció molt comuna és **llançar una excepció** (`IllegalArgumentException`) quan el valor és invàlid, perquè el programa que crida s'assabenti del problema. Això ho veuràs bé en la U04 amb `try/catch` i en els butlletins avançats.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** pensa en el setter com un porter de discoteca: no deixa entrar els borratxos (valors invàlids). Un bon setter converteix l'error d'un usuari (o del teu futur jo) en una cosa controlada, no en un objecte trencat.

Exercici: l'edat vigilada

Escriu tu (mentalment, o en paper) una classe `Persona` amb:

- Atribut privat `int edad`.
- Constructor que rep `edad`.
- Getter `getEdad()`.
- Setter `setEdad(int edad)` que **rebutgi** edats negatives i majors de 120.

Què imprimiria aquest codi?

```
Persona p = new Persona(25);
p.setEdad(-5);
System.out.println(p.getEdad());
```

> 🔄 **Solució**
>
> Imprimiria **25**: el setter rebutja `-5` i l'atribut es queda amb el valor que ja tenia. Dissenyar bé la frontera fa que els errors no entren. Un setter així:
>
> ```
> public void setEdad(int edad) {
>     if (edad >= 0 && edad <= 120) {
>         this.edad = edad;
>     } else {
>         System.out.println("Edat impossible, es manté l'anterior.");
>     }
> }
> ```

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Com es diu el getter de `boolean activo`?
2. Per a què serveix validar en un setter?
3. Quan uses un atribut només amb getter i sense setter?

4. Què fa `this` en `this.nombre = nombre`?

> 🔄 **Respostes**
>
> 1. `isActivo()`: amb `boolean`, la convenció usa `is` en comptes de `get`.
> 2. Perquè l'objecte mai no accepti valors impossibles: la validació ocorre a la frontera, no en el codi de fora.
> 3. Quan l'atribut es fixa en néixer (constructor) i no ha de canviar: DNI, pin, id… Només lectura.
> 4. Desambigua: `this.nombre` és l'atribut de la classe i `nombre` a seques és el paràmetre del mètode.

✅ Resum en 3 frases

1. **Getter** llegeix un atribut privat; **setter** el modifica, amb noms `getX()`/`setX()` (i `isX()` per a booleans).
2. El setter és la **frontera de validació**: rebutja valors impossibles perquè l'objecte mai no es trenqui.
3. **Sense setter** aconsegueixes atributs de només lectura, perfectes per a dades que no han de canviar.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Getter	Mètode que retorna el valor d'un atribut (`getEdad()`)
Setter	Mètode que modifica un atribut, validant (`setEdad(int)`)
`is`	Prefix del getter quan l'atribut és `boolean`
Només lectura	Atribut amb getter però sense setter
`this`	Referència a l'objecte actual, desambigua atribut vs paràmetre

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Encapsulació: el pilar de la POO · **Següent:** 05 · Atributs static

05 — Atributs static

El que pertany a la classe i no a l'objecte: el grup de WhatsApp en comptes dels missatges privats 📢

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 05 · Atributs static

---

📬 La idea en una frase

> **Un atribut `static` és de la classe, no de l'objecte: només hi ha una còpia per a tots, com el grup de WhatsApp de la classe, mentres que els atributs normals són els missatges privats de cada un.**

Fins ara, cada objecte que creaves amb `new` tenia els seus propis atributs. Si creaves dos `Estudiante`, cada un tenia el seu `nombre`. Però hi ha dades que no tenen sentit "per objecte": el nombre total d'estudiants creats, per exemple. Això no és d'un estudiant concret: és de **la classe**. I per a això existeix `static`.

📱 El grup de WhatsApp vs els missatges privats

Imagina que ets part d'una classe de 30 alumnes:

- **El grup de WhatsApp de la classe** (`static`): tots veuen el mateix missatge. Si algú escriu "demà hi ha examen", els 30 ho llegeixen. És compartit.
- **Els teus missatges privats** (instància): només tu els veus. Cada alumne té els seus i no es barregen.

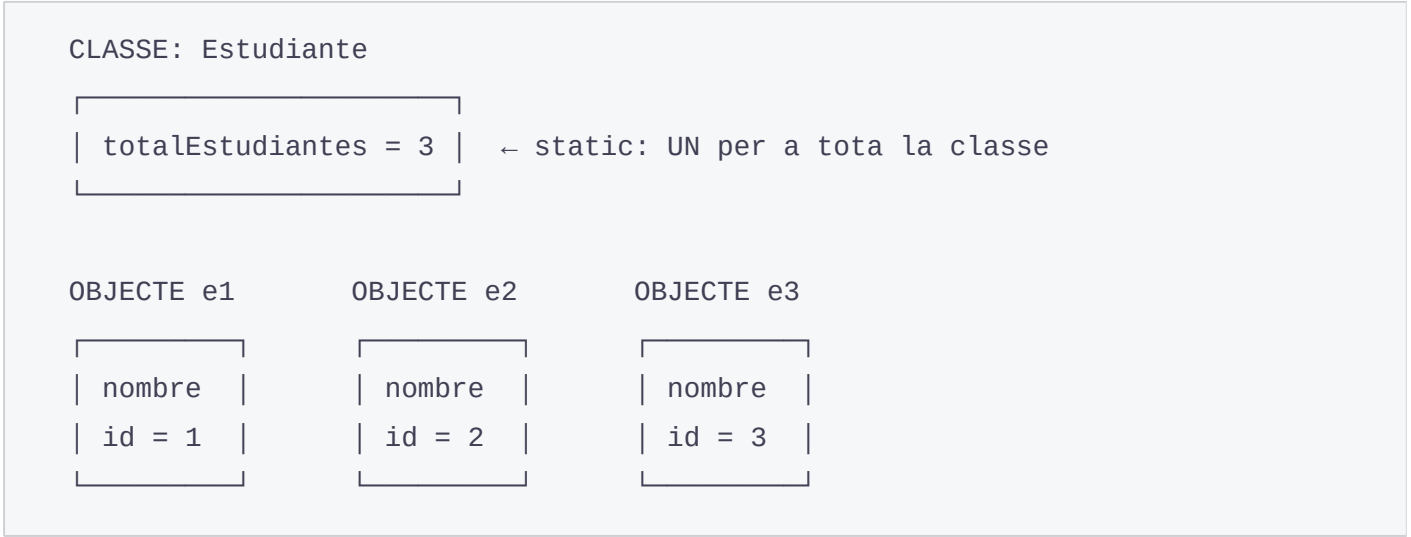
En Java és igual:

- **Variable de classe** (`static`): una sola còpia compartida per tots els objectes.
- **Variable d'instància** (sense `static`): cada objecte té la seva, independent.

```java
public class Estudiante {
    private static int totalEstudiantes = 0;
    private String nombre;
    private int id;

    public Estudiante(String nombre) {
        this.nombre = nombre;
        this.id = ++totalEstudiantes;
    }

    public static int getTotalEstudiantes() {
        return totalEstudiantes;
    }

    public int getId() {
        return id;
    }
}
```

Cada `Estudiante` té el seu propi `nombre` i `id`, però tots comparteixen `totalEstudiantes`. Quan crees el tercer estudiant, el comptador passa a 3 **per a tota la classe**.

🧠 Com es veu en memòria



El comptador viu en “la classe” (un espai que existeix una sola vegada), mentres que cada objecte té el seu nom i el seu id en el seu propi calaix.

📞 L'ús clàssic: comptadors i ids

L'exemple anterior és el patró estrella dels atributs `static`: **comptar objectes creats** i generar **ids autoincrementals**. Cada vegada que el constructor s'executa, `totalEstudiantes` puja i aquell valor s'usa com a id. És com el mostrador de la fleca: hi ha un únic número a la paret que va pujant per a tots els clients.

```
Estudiante a = new Estudiante("Ana");
Estudiante b = new Estudiante("Luis");
Estudiante c = new Estudiante("Marta");

System.out.println(Estudiante.getTotalEstudiantes()); // 3
System.out.println(c.getId());                        // 3
```

> 💡 **Detall pràctic:** per a accedir a un membre `static`, usa el nom de la **classe**: `Estudiante.getTotalEstudiantes()`. Funcionaria amb un objecte (`c.getTotalEstudiantes()`), però és confús: dona a entendre que el valor és "de c", quan és de tots.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan vegis un `static`, pregunta't: *té sentit que això sigui diferent per a cada objecte?* Si la resposta és "no", llavors pertany a la classe.

Exercici: els gats estàtics

Mira aquest codi i respon SENSE EXECUTAR:

```java
public class Gato {
    public static int totalGatos = 0;
    public String nombre;

    public Gato(String nombre) {
        this.nombre = nombre;
        totalGatos++;
    }

    public static void decirTotal() {
        System.out.println("Hay " + totalGatos + " gatos");
        // System.out.println(nombre); // ¿Açò funciona?
    }
}
```

1. Funciona `System.out.println(nombre);` dins de `decirTotal()`?
2. Si crees 3 gats i després fas `Gato.decirTotal()`, què imprimeix?
3. I si crees 5 gats més? Què imprimeix ara?

> 🔄 **Solució**
>
> 1. **NO funciona.** `nombre` és una variable d'instància i `decirTotal()` és estàtic: no sap de quin gat parlem. No hi ha "un gat" concret al qual preguntar-li.
> 2. **`Hay 3 gatos`**: el comptador estàtic ha pujat amb cada constructor.
> 3. **`Hay 8 gatos`**: el comptador segueix on estava i segueix pujant. És compartit, no es reinicia per objecte.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quantes còpies existeix d'un atribut `static`?
2. Com s'accedeix a un atribut estàtic des de fora?
3. Per a què serveix un comptador `static` amb un `id` autoincremental?
4. Pot un mètode estàtic llegir un atribut d'instància directament?

> 🔄 **Respostes**

1. **Una sola**, compartida per tots els objectes de la classe.
2. Amb el nom de la classe: `Estudiante.getTotalEstudiantes()` (o `Clase.atributo` si fos públic).
3. Per a comptar objectes creats i assignar un número únic a cada un, com el mostrador de la fleca.
4. No: no sap de quin objecte es parla. (Això es veu amb detall en el punt 6.)

✅ Resum en 3 frases

1. Un atribut `static` pertany a la **classe**: hi ha una sola còpia per a tots els objectes.
2. S'usa per a dades compartides: **comptadors**, ids autoincrementals, configuracions globals.
3. Accedeix sempre pel **nom de la classe**, no per un objecte, perquè el codi quedi clar.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`static`	Pertany a la classe, no a l'objecte
Variable de classe	Atribut `static`, una còpia compartida
Variable d'instància	Atribut normal, una còpia per objecte
Comptador	Atribut `static` que s'incrementa en cada `new`
Id autoincremental	Número únic assignat amb el comptador

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Getters i setters · **Següent:** 06 · Mètodes static i el main

06 — Mètodes static i el main

Cridar mètodes sense crear objectes, la classe Math, i per què main és el primer a arribar a la festa 🎉

> 🗺️ **Estàs en: 🔒 U10 · Visibilitat, encapsulació i static** → 06 · Mètodes static i el main

📬 La idea en una frase

> **Un mètode `static` es diu usant la classe, sense crear objectes; no té `this` i no pot tocar atributs d'instància. I `main` és estàtic perquè és el primer a arribar a una festa on encara no existeix cap objecte.**

Si els atributs `static` són el grup de WhatsApp, els mètodes `static` són el número d'atenció al client: no necessites parlar amb un empleat concret, crides al número general i llest.

📞 El telèfon de la classe

Un mètode estàtic es declara amb `static` i s'invoca usant el **nom de la classe**:

```
public class UtilidadesMatematicas {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static double media(double a, double b) {
        return (a + b) / 2;
    }
}
```

```
int resultado = UtilidadesMatematicas.sumar(5, 3);   // Sense new, sense objecte.
double med = UtilidadesMatematicas.media(10, 20);
```

Compara-ho amb un mètode d'instància, que exigeix un objecte:

```
String texto = "Hola";
int longitud = texto.length();    // length() NO és static: necessite l'objecte texto.
```

Aquella és la gran diferència pràctica: **static → classe; sense static → objecte**.

🚫 El que un mètode estàtic NO pot fer

Regles d'or que salven exàmens:

1. **No pot accedir a atributs d'instància** (no sap de quin objecte parla).
2. **No té `this`** (no hi ha "jo" perquè no hi ha objecte).
3. Només pot cridar **directament** a altres mètodes estàtics.

```
public class Prueba {
    int x = 1;
    static int y = 1;

    public void incrementarX() { x++; }          // Mètode d'instància
    public static void incrementarY() { y++; }   // Mètode estàtic
}
```

`incrementarY()` pot tocar `y` (estàtic) però no `x`: no sap quin `x` agarrar. `incrementarX()` pot tocar `x` perquè es diu sobre un objecte concret.

> ⚠️ **Advertència:** el clàssic del novell és ficar `System.out.println(nombreDeInstancia)` dins d'un mètode `static` i esperar que funcioni. El compilador et dirà *"non-static variable cannot be referenced from a static context"*. Ara ja saps per què.

🧮 La classe Math: la fàbrica de mètodes estàtics

`java.lang.Math` és la classe utilitària per excel·lència. TOTS els seus mètodes són estàtics. No pots (ni vols) fer `new Math()`:

```java
double max = Math.max(10, 20);        // 20.0
double raiz = Math.sqrt(25);          // 5.0
double potencia = Math.pow(2, 10);    // 1024.0
double absoluto = Math.abs(-7);       // 7.0
double random = Math.random();        // Aleatori en [0.0, 1.0)
```

La uses des del primer dia del curs, i ara entens **per què** funciona sense `new`: són mètodes estàtics. I de pas, `Integer.parseInt("42")` també ho és.

> 📝 **Nota:** `Math` té el constructor **privat**. Ningú no pot instanciar-la. És com una estàtua: per a admirar-la, no per a fer-li clons. Les classes així es diuen *utilitàries* i les muntes tu en el punt 7.

🎉 Per què `main` és static

`main` és el mètode que ho arranca tot:

```java
public static void main(String[] args)
```

Per què és estàtic? Perquè quan comença el programa **encara no existeix cap objecte**. Algú ha d'arrancar la festa abans que arribi ningú. Si `main` fos d'instància, la JVM necessitaria un objecte per a cridar-lo… i per a crear aquell objecte necessitaria un programa en marxa. Peix que es mossega la cua. Per això `main` és el primer a arribar i obre la porta ell mateix.

🤷 No hi ha preguntes tontes

> ❓ **Puc cridar a un mètode estàtic des d'un objecte? Com `miObjeto.metodoEstatico()`?**

Tècnicament sí: Java t'ho permet. Però és com cridar a ta mare pel cognom: funciona, però queda rar. La convenció és usar la classe: `Clase.metodoEstatico()`. De fet, alguns IDEs et mostren una warning per fer-ho.

> ❓ **Els mètodes estàtics són més ràpids?**

Un poquet: no necessiten la referència a l'objecte. Però la diferència és tan petita que en el 99,9% dels casos no la notaràs. No t'obsessiones amb la velocitat: preocupa't que el teu codi tingui sentit.

> ❓ **Puc posar-li `static` a tot i estalviar-me crear objectes?**

Pots, però aleshores no estàs fent POO: estàs fent "programació estàtica a la bruta". És com usar un tornavís per a clavar un clau: pots, però per a això existeix el martell. Usa `static` per al que és de la classe, no per a tot.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** el truc per a no liar-te és preguntar sempre: *aquest mètode necessita saber de quin objecte parla?* Si la resposta és no, pot ser `static`.

Exercici: la calculadora de la classe

Funciona aquest codi? Si no, per què?

```
public class Calculadora {
    private String marca;

    public static int sumar(int a, int b) {
        System.out.println(marca);  // Què passa ací?
        return a + b;
    }
}
```

> › 🔄 **Solució**
>
> **No compila.** El mètode `sumar()` és estàtic i intenta llegir `marca`, que és un atribut d'instància. Un mètode estàtic no té objecte, així que no sap quina `marca` imprimir. El compilador ho prohibeix amb: *"non-static variable marca cannot be referenced from a static context"*. Si `marca` fos `static` (compartida per la classe), sí que funcionaria.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Com es diu a un mètode estàtic?
2. Pot un mètode estàtic accedir a atributs d'instància?
3. Per què `main` és estàtic?

4. Què diferencia `Math.sqrt(9)` de `texto.length()`?

> 🔄 **Respostes**
>
> 1. Amb el nom de la classe: `UtilidadesMatematicas.sumar(5, 3)`. Sense crear objectes.
> 2. No: no té `this` i no sap de quin objecte parla.
> 3. Perquè quan arranca el programa no existeix cap objecte encara: algú ha d'obrir la porta.
> 4. `Math.sqrt` és estàtic (es diu sobre la classe) i `length()` és d'instància (es diu sobre l'objecte `texto`).

✅ Resum en 3 frases

1. Els mètodes `static` es diuen amb la **classe**, sense `new`, i no poden tocar atributs d'instància ni usar `this`.
2. `Math` és la classe utilitària per excel·lència: tot estàtic, constructor privat.
3. `main` és estàtic perquè **és el primer a arribar**: quan arranca, no hi ha objectes encara.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Mètode estàtic	Mètode de la classe: `Clase.metodo()`
Mètode d'instància	Mètode de l'objecte: `objeto.metodo()`
`this`	Referència a l'objecte actual (no existeix en estàtics)
Classe utilitària	Només mètodes estàtics, sense objectes
Context estàtic	"El món de la classe": sense objectes a la vista

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Atributs static · **Següent:** 07 · Constants de classe: static final

07 — Constants de classe: static final

Valors immutables en MAJÚSCULES i les classes utilitàries que ningú no pot instanciar 🏛️

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 07 · Constants de classe: static final

📬 La idea en una frase

> **`static final` és una constant de classe: un valor que pertany a la classe, no a l'objecte, i que ningú no pot canviar. I si a més li poses el constructor privat, tens una classe utilitària com `Math`.**

Recordes el `final` de la U03: una variable que no es pot canviar. Ara suma-li `static` (de la classe, una sola còpia) i tens la fórmula perfecta per a valors fixos de tota l'aplicació: l'IVA, el nom de l'app, el nombre màxim d'usuaris…

🏷️ La constant de classe

Declaració canònica, amb el nom en **MAJÚSCULES** i paraules separades per _:

```
public class Config {
    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "2.1.0";
    public static final int MAX_USUARIOS = 100;
    public static final double IVA = 0.21;
}
```

S'usa així, sense crear cap objecte:

```
double precioConIva = precio * Config.IVA;
System.out.println("Benvingut a " + Config.NOMBRE_APP);
```

I si algú intenta fer `Config.IVA = 0.5`? **El compilador t'ho prohibeix.** És més ferm que els teus propòsits d'Any Nou.

> 💡 **Detall pràctic:** junta les constants en una classe dedicada (`Config`, `Constantes`, `Valores`) i tindràs un únic lloc on canviar, per exemple, l'IVA quan toqui. Ni un "número màgic" solt pel

> codi.

🔒 La regla de les MAJÚSCULES

És una **convenció**, no una regla del llenguatge: les constants s'escriuen en MAJÚSCULES amb _:

- `MAX_INTENTOS_LOGIN`
- `TIMEOUT_SEGUNDOS`
- `RUTA_LOG`

Quan un altre programador (o tu) vegis `Config.MAX_USUARIOS` en un codi, sabrà a l'instant que això és una constant: no es toca, no es pregunta, no es negocia. L'estil diu molt.

> ⚠️ **Advertència:** compte amb la temptació de declarar-ho tot `static final` "perquè mola". Només és constant el que de veritat no canviarà. El dia que l'IVA pugi, voldràs canviar-lo en un lloc… i si és constant, el canvies i llest.

🏛️ Les classes utilitàries: el "no necessiti parella" de Java

Una classe utilitària és una classe que **només té membres estàtics**. Són com l'amic que està solter i feliç: no necessita instanciar-se per a ser útil. La prova: `Math`, que ja coneixes.

Com evitem que algú faci `new StringUtils()`? Amb un **constructor privat**:

```java
public class StringUtils {
    private StringUtils() {}  // Ningú no pot fer new StringUtils()

    public static boolean esVacio(String str) {
        return str == null || str.trim().isEmpty();
    }

    public static String invertir(String str) {
        return str == null ? null : new StringBuilder(str).reverse().toString();
    }

    public static String capitalizar(String str) {
        if (esVacio(str)) return str;
                                return    str.substring(0,    1).toUpperCase()    +
str.substring(1).toLowerCase();
    }
}
```

Des de fora, tot és `StringUtils.invertir("hola")`. Intentar `new StringUtils()` dona error de compilació: el constructor és privat, la classe "no necessita parella".

> 📝 **Nota:** el constructor privat no és obligatori perquè funcioni, però és l'**etiqueta** d'una classe utilitària ben feta: li diu al món "no m'instancies, soc pura utilitat".

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** abans d'escriure un número solt (un 100, un 0.21), pregunta't si és un *número màgic* que mereix una constant amb nom. Els números màgics són la malson de qui manté el teu codi.

Exercici: el validador del barri

Escriu (mentalment) una classe `Validador` que sigui utilitària: constructor privat i un mètode `public static boolean esPar(int numero)` que retorni `true` si el nombre és parell.

Com comprovaries, sense crear objectes, que 42 és parell?

> › 🔄 **Solució**

```java
public class Validador {
    private Validador() {}

    public static boolean esPar(int numero) {
        return numero % 2 == 0;
    }
}
```

I per a usar-la, sense `new`: `boolean resultado = Validador.esPar(42);` → `true`. El `%` del punt 3 de la UO3 i el `static` d'aquesta unitat, treballant en equip.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa `static final` i com es nomena?
2. Per què `Math` no es pot instanciar?
3. Què és un número màgic i com l'evites?
4. Pot canviar una constant després de declarar-la?

> 🔄 **Respostes**
>
> 1. Una constant de classe: pertany a la classe (una sola còpia) i no es pot canviar. Es nomena en MAJÚSCULES amb _.
> 2. Perquè el seu constructor és `private`: és una classe utilitària, no hi ha motius per a crear objectes.
> 3. Un valor literal sense explicació (com `0.21` o `100`) tirat pel codi. L'evites donant-li nom en una constant: `Config.IVA`.
> 4. No: el compilador prohibeix modificar una variable `final` després de la seva assignació inicial.

✅ Resum en 3 frases

1. `static final` crea una **constant de classe**: una sola còpia, immutable, en MAJÚSCULES.
2. Les **classes utilitàries** (com `Math`) només tenen membres estàtics i usen **constructor privat** perquè ningú no les instancii.

3. Evita els **números màgics** donant-los nom i ajuntant-los en una classe `Config`.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Constant | `static final`: valor fix de la classe |
> | Número màgic | Valor literal sense nom que hauria de ser constant |
> | Classe utilitària | Només estàtics, constructor privat |
> | Convenció | Regla d'estil (MAJÚSCULES) que tots segueixen |
> | Constructor privat | Impedeix crear objectes de la classe |

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Mètodes static i el main · **Següent:** 08 · Be the Code: encapsula la teva classe

08 — Be the Code: encapsula la teva classe

El repte guiat: converteix una classe sense blindar en una ben encapsulada amb getters i setters 🔨

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 08 · Be the Code: encapsula la teva classe

---

📬 La idea en una frase

> **Aquí no aprenem res de nou: posem en pràctica tot el dels punts 1 al 7 convertint, pas a pas, una classe que és una porta oberta en una classe amb cadenat, porter i alarma.**

És l'hora de ser tu el codi. La U09 ja et va ensenyar a dissenyar una classe; ara toca el treball fi: **blinda-la**. Agafem una classe que viola totes les regles i la transformem en una classe professional.

🏚️ El punt de partida: la classe del caos

Aquest és el codi que "funciona" però hauria de fer-te vergonya aliena:

```
public class CuentaBancaria {
    public double saldo;
    public String titular;
    public int numeroIngresos;

    public CuentaBancaria(String titular, double saldoInicial) {
        saldo = saldoInicial;
        numeroIngresos = 0;
    }

    public void ingresar(double cantidad) {
        saldo = saldo + cantidad;
        numeroIngresos++;
    }
}
```

Què falla? Tot: atributs `public`, sense validació, sense getters... Anem a convertir-lo en una classe que respira seguretat. **Pas a pas, sense saltar-te'n cap.**

🧰 Pas 1: atributs privats

Regla número u de la casa: els atributs van amb `private`. Sense excepcions.

```
public class CuentaBancaria {
    private double saldo;
    private String titular;
    private int numeroIngresos;
    ...
}
```

Si algú intenta `cuenta.saldo = 1000` des de fora, el compilador li diu que no. Però... i si algú legítimament vol saber el saldo o canviar el titular? Això ho resolen els passos 2 i 3.

🧰 Pas 2: getters per a llegir

Un getter retorna el valor de l'atribut, seguint la convenció `get` + nom:

```
public double getSaldo() {
    return saldo;
}

public String getTitular() {
    return titular;
}

public int getNumeroIngresos() {
    return numeroIngresos;
}
```

Ara el món pot *llegir* el saldo, però no *tocar-lo*.

> 💡 **Detall pràctic:** fixa't que `numeroIngresos` té getter però no setter: ningú no l'ha de posar a mà, només `ingresar()` l'incrementa. Només lectura, com ha de ser.

🧰 Pas 3: setter per al que es pot canviar

El titular sí que pot canviar (un banc actualitza titulars, què li farem). Afegim el setter amb la seva validació:

```
public void setTitular(String titular) {
    if (titular != null && !titular.trim().isEmpty()) {
        this.titular = titular;
    } else {
        System.out.println("Titular invàlid.");
    }
}
```

Fixa't en l'ordre del `if`: primer la condició de validesa, i dins del `else` l'avís. El `this` desambigua (paràmetre vs atribut, del punt 5 de la U09).

🧰 Pas 4: validar també en els mètodes

Un setter és un porter, però els mètodes `ingresar()` també han de ser-ho. Ara mateix el codi accepta ingressos negatius (un "ingrés" de -50 euros!). Ho arreglem:

```
public void ingresar(double cantidad) {
    if (cantidad > 0) {
        saldo = saldo + cantidad;
        numeroIngresos++;
    } else {
        System.out.println("No es poden ingressar quantitats negatives.");
    }
}
```

L'estat del compte ja no pot ser impossible: ni saldo negatiu per la via ràpida ni ingressos fantasma.

🧰 Pas 5: el toc final, constants

Podem rematar amb una constant de classe. Imaginem que el banc només permet un màxim d'ingressos al dia per l'app:

```
public class CuentaBancaria {
    public static final int MAX_INGRESOS_DIA = 10;
    private double saldo;
    private String titular;
    private int numeroIngresos;
    ...
}
```

I en `ingresar()` afegim la comprovació:

```
if (numeroIngresos >= MAX_INGRESOS_DIA) {
    System.out.println("Límit diari assolit.");
    return;
}
```

🏆 La classe final (compara-la amb la del caos)

```java
public class CuentaBancaria {
    public static final int MAX_INGRESOS_DIA = 10;

    private double saldo;
    private String titular;
    private int numeroIngresos;

    public CuentaBancaria(String titular, double saldoInicial) {
        setTitular(titular);
        saldo = saldoInicial;
        numeroIngresos = 0;
    }

    public double getSaldo() {
        return saldo;
    }

    public String getTitular() {
        return titular;
    }

    public int getNumeroIngresos() {
        return numeroIngresos;
    }

    public void setTitular(String titular) {
        if (titular != null && !titular.trim().isEmpty()) {
            this.titular = titular;
        } else {
            System.out.println("Titular invàlid.");
        }
    }

    public void ingresar(double cantidad) {
        if (numeroIngresos >= MAX_INGRESOS_DIA) {
            System.out.println("Límit diari assolit.");
            return;
        }
        if (cantidad > 0) {
            saldo = saldo + cantidad;
            numeroIngresos++;
        } else {
```

```
            System.out.println("No es poden ingressar quantitats negatives.");
        }
    }
}
```

Del caos a la fortalesa: `private` en tot, getters, setters amb validació, mètodes porters i una constant. **Això és encapsular.**

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Per què `numeroIngresos` no té setter?
2. Què fa `setTitular()` si li passes un text buit?
3. Quants passos cal seguir per a blindar la classe?
4. Per a què serveix la constant `MAX_INGRESOS_DIA`?

> 🔄 **Respostes**
>
> 1. Perquè ningú no l'ha de fixar a mà: només `ingresar()` l'incrementa. Getter per a llegir, res per a escriure.
> 2. El rebutja i mostra "Titular invàlid.", deixant l'atribut com estava.
> 3. Cinc: atributs privats, getters, setter, validació en mètodes i constants. Tots importen.
> 4. Per a posar un límit global al negoci en un únic lloc, llegible i a prova de "números màgics".

✅ Resum en 3 frases

1. Encapsular una classe és un **procés en passos**: `private`, getters, setters, validació i constants.
2. Cada setter i cada mètode són una **frontera** que decideix què entra i què surt de l'objecte.
3. Al final tens una classe que **es protegeix sola**: l'error del món exterior no pot trencar-la.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Refactoritzar	Millorar el codi sense canviar el seu comportament
Blindar	Convertir atributs `public` en `private` amb control d'accés
Frontera	Mètode públic que valida el que entra
Només lectura	Getter sense setter: es llegeix però no s'escriu
Número màgic	Literal sense nom: es converteix en constant

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Constants de classe: static final · **Següent:** 09 · Repàs interactiu

09 — Repàs interactiu: blinda-ho tot

El tancament pràctic de la unitat, amb cadenats, estàtics i una classe que no voldrà compilar 😈

> 🗺️ **Estàs en:** 🔒 **U10 · Visibilitat, encapsulació i static** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no voldrà compilar. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
public class Estudiante {
    public static int total = 0;
    private int id;

    public Estudiante() {
        id = ++total;
    }

    public int getId() {
        return id;
    }
}
```

```
public class Misteri {
    public static void main(String[] args) {
        Estudiante a = new Estudiante();
        Estudiante b = new Estudiante();
        Estudiante c = new Estudiante();

        System.out.println(c.getId());
        System.out.println(Estudiante.total);
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`3` i `3`** → ✅ Correcte! Cada `new` incrementa el `static total`, i el tercer estudiant s'emporta l'`id = 3`. A més, `total` (estàtic, de la classe) també val 3: una sola còpia per a tots.
2. **`1` i `3`** → El `static` seria "de cada objecte", però no: `total` és de la classe, una sola còpia. ❌
3. **`3` i `0`** → El constructor sí que incrementa `total`, no el deixa en 0. ❌

> 🔄 **Solució**
>
> L'opció **1**. `total` és un comptador `static` que puja amb cada constructor; `id` és d'instància i s'emporta el valor de `total` en el moment de néixer. És el patró "comptador + id autoincremental" del punt 5.

🔥 Fireside Chat: la classe i l'objecte

> *Dos veterans de la POO discuteixen al costat de la màquina de cafè.*

Classe: — Jo soc el motlle. Defineixo atributs, mètodes, l'estructura. Sense mi no hi ha res que instanciar.

Objecte: — I jo soc la galeta, el resultat real. Jo guardo les dades: el meu `nombre`, el meu `id`. Tu només ets el plànol.

Classe: — Plànols, plànols... I qui té el comptador de quantes galetes hi ha? Jo! `static int total`. Això és meu, i tots els objectes ho compartiu.

Objecte: — Compartim, sí, però jo tinc el meu `id` únic. El meu `nombre`, el meu saldo. No em toques els meus atributs d'instància.

Classe: — Precisament per això els vaig declarar `private`. Perquè ni tu te'ls toques: només a través dels meus getters i setters. Jo soc el porter.

Objecte: — Val, però sense mi, els teus setters no tenen a qui validar. Necessites un objecte perquè la festa existeixi.

Classe: — *suspira* Per això el `main` és `static`: perquè jo pugui arrancar la festa sense que arribi cap objecte primer.

> La lliçó: la **classe** defineix l'estructura i posseïx el que és `static` (comptadors, constants); l'**objecte** guarda la informació de cada cosa (atributs d'instància). Es necessiten mútuament, però cada un al seu terreny.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc el nivell més restrictiu: ni ma mare, ni el meu millor amic, ni el gos. Només la classe em veu.**
2. **Soc el porter de la discoteca: deixo llegir l'atribut, però amb noms estrictes.**
3. **Soc el número d'atenció al client: em diuen per la classe, sense crear cap objecte.**
4. **Soc el grup de WhatsApp de la classe: només hi ha una còpia i tots la compartim.**
5. **Soc el valor que ni el temps s'atreveix a tocar: en MAJÚSCULES i amb _.**
6. **Soc la classe que ningú no pot instanciar: només estàtics i constructor privat.**

> 🔄 **Respostes**
>
> 1. **`private`** — només la pròpia classe.
> 2. **El getter/setter** — llegeixen i modifiquen atributs privats amb convenció `getX()`/`setX()`.
> 3. **Un mètode estàtic** — `Clase.metodo()`, sense `new`.
> 4. **Un atribut `static`** — una còpia compartida per tots els objectes.
> 5. **Una constant** — `static final`, escrita en MAJÚSCULES amb _.
> 6. **Una classe utilitària** — com `Math` o `StringUtils`.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Quina visibilitat té un atribut sense modificador?
   - a) `public` b) package-private (només el paquet)
2. Com es diu el getter de `boolean activo`?
   - a) `getActivo()` b) `isActivo()`
3. Pot un mètode estàtic usar `this`?

- a) Sí, sempre b) No: no hi ha objecte del qual parlar

4. Què fa `Config.IVA = 0.5` si `IVA` és `static final`?
   - a) Canvia el valor b) Error de compilació
5. Si dos objectes comparteixen un atribut `static int x`, i un l'incrementa…
   - a) Només l'objecte que l'incrementa ho veu b) Els dos ho veuen canviat

> 🔄 **Solucions**
>
> 1. **b)** — sense modificador és package-private: ho veu el mateix paquet.
> 2. **b)** — amb `boolean` la convenció usa `is` en comptes de `get`.
> 3. **b)** — els estàtics no tenen `this`.
> 4. **b)** — `final` prohibeix modificar la constant: error de compilació.
> 5. **b)** — l'atribut `static` és compartit: només hi ha una còpia per a tots.

😡 CONRAD VS EL MÓN: "Per què tot públic?"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre el clàssic del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu programa funciona però després es trenca en el mòdul de facturació*. I jo: val, com estan els teus atributs? *Pues públics, per a anar més ràpid.* AI, MARE MEVA! Un `public` en un atribut és una invitació a què qualsevol classe li fique un valor impossible. És clar que funciona… fins que deixa de funcionar a les 3 de la matinada.

I després està el que escriu `setEdad(int edad) { this.edad = edad; }` *sense validar res. Un setter sense validar és un porter adormit: deixa passar a qualsevol, fins i tot amb edats negatives.*

I el colmo: un mètode `static` intentant llegir `nombre` d'instància. No saps ni de quin objecte parles! T'ho dic des del punt 6: el `static` no té `this`.

La lliçó: abans de queixar-te de bugs rars, revisa la frontera: els atributs són `private`? Els setters validen? Allò estàtic està on toca? El 90% dels estats impossibles s'eviten amb un bon cadenat a la classe.

⚡ Laboratori de Tortura: la caixa forta sense cadenat

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

El escenari: copia aquesta classe i fes que sigui una classe "professional". Té **4 problemes** de visibilitat i disseny que la deixen com una porta oberta.

```
public class Mascota {
    public String nombre;
    public int edad;
    public static int totalMascotas = 0;

    public Mascota(String nombre, int edad) {
        nombre = nombre;
        edad = edad;
        totalMascotas++;
    }

    public void cumplirAnos() {
        edad = edad + 1;
    }
}
```

Fallada intencionada: un dels problemes sembla correcte a simple vista, però fa que el `nombre` i l'`edad` de la mascota es queden "en blanc" en néixer.

La teva tasca: convertir-la en una classe encapsulada: atributs `private`, getters, setters amb validació (l'edat no pot ser negativa), i corregeix el constructor. A més, prova que `totalMascotas` puja amb cada `new`.

Pistes per quan et frustres (no abans):

1. Els atributs haurien de ser `public`? *no → és el primer problema.*

   > **› I si continuec atascat?**
   > Posa'ls `private` i crea getters i setters amb la convenció `getX()`/`setX()`.

2. Què passa amb `nombre = nombre;` i `edad = edad;`? *Sonen a assignar-se a si mateixos.*

   > **› I si continuec atascat?**
   > Sense `this`, el paràmetre s'assigna a si mateix i l'atribut es queda amb el seu valor per defecte. Usa `this.nombre = nombre`.

3. Què hauria de validar el setter de l'edat?

> **I si continuec atascat?**
>
> Que l'edat no sigui negativa. Si és invàlida, no l'apliques i avisa.

4. `totalMascotas` està bé com està? *Recorda la convenció d'accés.*

> **Solució final**

```java
public class Mascota {
    public static int totalMascotas = 0;

    private String nombre;
    private int edad;

    public Mascota(String nombre, int edad) {
        this.nombre = nombre;
        setEdad(edad);
        totalMascotas++;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        if (nombre != null && !nombre.trim().isEmpty()) {
            this.nombre = nombre;
        }
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        if (edad >= 0) {
            this.edad = edad;
        } else {
            System.out.println("Edat invàlida: " + edad);
        }
    }

    public void cumplirAnos() {
        this.edad++;
    }
}
```

Els 4 problemes: atributs públics, constructor sense `this` (els paràmetres s'assignaven a si mateixos), setter d'edat sense validar i `cumplirAnos()` que podia acabar en edats impossibles.

Amb `private` + validació, la mascota ja no pot “néixer en blanc” ni tenir edat negativa.

🧩 L’EMBOLIC: el setter que no validava

El següent codi pretén encapsular una classe `CuentaBancaria`, però té diversos errors. Troba’ls:

```
public class CuentaBancaria {
    public double saldo;
    private String titular;

    public CuentaBancaria(String titular, double saldoInicial) {
        titular = titular;
        saldo = saldoInicial;
    }

    public double getSaldo() {
        return saldo;
    }

    private void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }
}
```

1. L’atribut `saldo` està ben encapsulat?
2. Quin error hi ha al constructor amb `titular = titular`?
3. Per què `setSaldo` és `private`? Com es modifica el saldo llavors?

> 🕶️ **Don Tip:** `this` resol ambigüitats. Si paràmetre i atribut es diuen igual, sense `this` t’assignes el paràmetre a si mateix i l’atribut es queda “en blanc”.

> › 🔄 **Solucions**
>
> 1. **No.** `saldo` és `public`: qualsevol pot fer `cuenta.saldo = -500` des de fora, saltant-se tota validació. Hauria de ser `private` i modificar-se només a través de mètodes com `ingresar()` i `retirar()`.

2. `titular = titular` assigna el paràmetre a si mateix. L'atribut `titular` de la classe es queda amb el seu valor per defecte (`null`). Ha de ser `this.titular = titular`.
3. Si `setSaldo` és `private`, ningú de fora pot modificar el saldo, ni tan sols amb validació. La classe hauria d'exposar operacions de negoci (`ingresar(double)`, `retirar(double)`) que criden internament a la lògica de canvi, com vam fer en el punt 8.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🏅 **El Candidat al Cadenat**	Explicar els 4 nivells de visibilitat i qui veu cada cosa, sense mirar
🏅 **Porter Estricte**	Escriure una classe amb atributs `private`, getters i setters amb validació
🏅 **El Comptador de la Classe**	Crear una classe amb comptador `static` i ids autoincrementals que funcioni
🏅 **L'Endevinador Estàtic**	Predir "Què imprimeix?" en un trencaclosques de `static` vs instància sense executar
🏅 **El Refactoritzador**	Convertir la classe del Laboratori en una classe ben encapsulada sense mirar la solució

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```java
public class Contador {
    static int n = 0;
    int m = 0;

    public static void subirN() { n++; }
    public void subirM() { m++; }

    public static void main(String[] args) {
        Contador a = new Contador();
        Contador b = new Contador();

        a.subirM();
        b.subirM();
        a.subirM();
        Contador.subirN();
        Contador.subirN();

        System.out.println(a.m + " " + b.m + " " + Contador.n);
    }
}
```

2. **L'atribut fantasma:** en `CuentaBancaria`, com faries que el saldo només pogués pujar amb ingressos i baixar amb retirades, sense que ningú pogués posar-lo a un valor arbitrari?
3. **El detectiu:** el teu programa accepta edats negatives perquè "ningú no sap qui les introduïx". Quina eina d'aquesta unitat uses per a parar-ho?
4. **Vertader o fals:** "un atribut `static` té una còpia per cada objecte creat".

> 💡 **Solucions**
>
> 1. Imprimeix **`2 1 2`**: `a.m` puja dos vegades (2), `b.m` una (1), i `n` (estàtic) puja dos vegades (2). El `m` és de cada objecte; el `n` és compartit.
> 2. Declarant `saldo` com a `private` i creant només mètodes `ingresar(double)` i `retirar(double)` que validen. Sense setter públic per al saldo: només operacions de negoci.
> 3. La **encapsulació**: `private int edad` + `setEdad(int)` que validi `edad >= 0`. L'error es detecta a la frontera, no a quilòmetres.
> 4. **Fals.** Un atribut `static` té **una sola còpia** compartida per tots els objectes.

🧩 Crucigrama de Bits

```
Horizontal:
1. Modificador d'accés més restrictiu (7 lletres)
4. La classe no el té, l'objecte sí, quan parlem de "jo" (4 lletres)
6. Mètode que llig un atribut privat (6 lletres)

Vertical:
2. Valor fix de la classe, en MAJÚSCULES (9 lletres)
3. El que pertany a la classe, no a l'objecte (6 lletres)
5. Prefix del getter quan l'atribut és booleà (2 lletres)
```

> 📝 Solucions
>
> **Horizontal:** 1. PRIVATE · 4. THIS · 6. GETTER **Vertical:** 2. CONSTANTE · 3. STATIC · 5. IS

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java junior.

1. **"Explica-m'ho, com si jo fos la teva àvia, per què els atributs no haurien de ser públics."**
2. **"Quina és la diferència entre un mètode estàtic i un mètode d'instància?"**
3. **"Què és l'encapsulació i quins avantatges aporta?"**
4. **"Per què `main` és `static`?"**
5. **"Quan usaries `static final`?"**
6. **"Un company ha deixat `public` un atribut que guarda l'estat d'un joc i apareixen bugs impossibles. Què fas?"**

🤷 No hi ha preguntes tontes

> ❓ **Package-private i `default` són el mateix?**

Sí: es diuen igual. És el nivell que Java assumeix quan no escrius `public`, `private` ni `protected`. No confonguis amb la paraula `default`, que és per a una altra cosa.

> ❓ **Un setter sempre ha d'imprimir un missatge quan el valor és invàlid?**

No. Una altra opció molt professional és **llançar una excepció** (`IllegalArgumentException`), perquè el codi que crida s'assabenti i decideixi. Imprimir és còmode per a practicar; llançar excepcions és el que veuràs en codi de producció.

> ❓ **Puc tenir un atribut estàtic que sigui un objecte de la seva pròpia classe?**

Sí! És el patró **Singleton**: un `private static MiClase instancia` i un mètode `getInstance()`. És com tenir una única pedra filosofal. Però això és un altre tema… que algun dia mereixerà la seva pròpia unitat.

🎬 Post-Crèdits

El programador acaba la seva classe `CuentaBancaria` blindada: atributs `private`, getters, setters amb validació i un comptador estàtic que compta els comptes creats. Se sent un fortí ambulant.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Això era una classe o una caixa forta del Banc d'Espanya? `private`, validació, constants… Fins a constructor privat en una classe utilitària que t'has inventat. No està malament per a un aprenent de cadenats.

Novell: — I ara què? Les meves classes ja no es trenquen encara que algú intenti trencar-les.

CONRAD: — *fes un glop* Blindar està bé, però un banc amb un sol compte és un banc trist. Els objectes es relacionen entre si: hereten, comparteixen, s'especialitzen. Això és una altra història. La pròxima parada t'ho compta.

El novell guarda el seu projecte, tanca l'IDE i sent que els seus objectes ja no són cases de vidre.

PROXIMAMENT EN U11: Herència, polimorfisme i interfícies. El moment en què les teves classes **es miren als ulls i es diuen "mira, això ho compartim".** 🧬

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Be the Code: encapsula la teva classe · **Següent:** U11 · Herència, polimorfisme i interfícies

U11 — Herència, polimorfisme i interfícies

Quan les teves classes hereden, es disfressen i firmen contractes: extends, super, override, abstract i interfícies 🧬

Fins ara les teves classes eren illes: cadascuna amb els seus atributs, els seus mètodes i els seus secrets ben guardats. Però en la vida real les coses no són illes: un gos *és un* animal, un cotxe *és un* vehicle, i un professor *és un* empleat. Les coses **hereden** característiques dels seus pares i, a més, afigen les seves. Doncs el teu codi fa exactament el mateix.

Aquesta unitat és el cor de la POO. I té tres grans actes:

- **Herència (`extends`):** una classe filla obté tot el que té la seva classe pare i pot millorar-ho. Aprendràs a usar `super` per a cridar la superclasse, `@Override` per a sobreescriure mètodes i `protected` per a compartir secrets només amb la família.
- **Polimorfisme:** el mateix mètode, mil comportaments. Una variable de tipus `Animal` pot apuntar a un `Gos`, un `Gat` o una `Vaca`, i cadascun fa el seu so. Veuràs `instanceof`, downcasting i la mítica classe `Object`, el besavi de tot.
- **Abstracció i interfícies:** classes abstractes que no es poden instanciar (esbossos) i interfícies que són contractes. Aquí es decideix si el teu codi és un caos o una obra d'art.

Pel camí descobriràs per què `toString()` et menteix si no el sobreescrius, què passa quan fas un downcasting a la babalà (spoiler: `ClassCastException`), i per què els entrevistadors de treball adoren preguntar "quan usaries una classe abstracta i quan una interfície?".

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- **Heredar** d'una classe amb `extends` i distingir IS-A de HAS-A.
- Usar **`super`** per a cridar constructors i mètodes de la superclasse.
- **Sobreescriure** mètodes amb `@Override` i saber quan (i quan no) fer-ho.
- Protegir membres amb **`protected`** i dissenyar **jerarquies de classes** sensates.
- Explicar el **polimorfisme dinàmic** (dynamic binding) i usar-lo amb col·leccions i paràmetres.

- Aplicar **`instanceof` i downcasting** amb seguretat.
- Sobreescriure **`toString()`, `equals()` i `hashCode()`** de la classe `Object`.
- Dissenyar **classes abstractes** amb mètodes abstractes i el patró template method.
- Implementar **interfícies**, usar mètodes `default` i decidir quan usar `abstract class` vs `interface`.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Herència: quan els teus fills segueixen els teus passos	`extends`, què s'hereda i què no, IS-A vs HAS-A, l'arbre genealògic	Tots
02 · super i @Override	Cridar la superclasse, sobreescriure mètodes i encadenar constructors	Tots
03 · protected i jerarquies de classes	El membre que només veu la família, arbres de classes i la classe base fràgil	Tots
04 · Polimorfisme: el camaleó de la POO	Dynamic binding, referències polimòrfiques, col·leccions i paràmetres	Tots
05 · instanceof i downcasting	Preguntar qui ets i baixar la referència sense trencar res	Tots
06 · La classe Object: el besavi de tot	`toString()`, `equals()` i `hashCode()` ben sobreescrits	Tots
07 · Classes abstractes	Esbossos que no s'instancien, mètodes abstractes i template method	Tots
08 · Interfícies: el contracte que signes	`implements`, polimorfisme, mètodes `default` i abstract vs interface	Tots
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA4 i RA7)

RA4: Desenvolupa programes organitzats en classes analitzant i aplicant els principis de la programació orientada a objectes.

CE	Criteri	On es cobreix
RA4 a)	S'ha reconegut la sintaxi, estructura i components típics d'una classe.	✅ Punt 1
RA4 b)	S'han definit classes.	✅ Punts 7 i 8
RA4 d)	S'han creat constructors.	✅ Punts 2 i 7
RA4 e)	S'han desenvolupat programes que instancien i utilitzen objectes de les classes creades anteriorment.	✅ Punts 4, 8 i 9
RA4 g)	S'han definit i utilitzat classes heredades.	✅ Punts 1, 2 i 3
RA4 i)	S'han definit i utilitzat interfícies.	✅ Punt 8

RA7: Desenvolupa programes, aplicant característiques avançades dels llenguatges orientats a objectes i de l'entorn de programació.

CE	Criteri	On es cobreix
RA7 a)	S'han identificat els conceptes d'herència, superclasse i subclasse.	✅ Punt 1
RA7 b)	S'han utilitzat modificadors per a bloquejar i forçar l'herència de classes i mètodes.	✅ Punts 2 i 7
RA7 c)	S'ha reconegut la incidència dels constructors en l'herència.	✅ Punt 2
RA7 d)	S'han creat classes heredades que sobreescriguin la implementació de mètodes de la superclasse.	✅ Punts 2 i 4
RA7 e)	S'han dissenyat i aplicat jerarquies de classes.	✅ Punts 3 i 7
RA7 f)	S'han provat i depurat les jerarquies de classes.	✅ Punt 9
RA7 g)	S'han realitzat programes que implementen i utilitzen jerarquies de classes.	✅ Punts 4, 8 i 9

> 📌 El resultat d'aprenentatge **RA4** es recolza en la U09 (definir classes) i la U10 (visibilitat), i culmina aquí amb herència i interfícies. La **RA7** es tanca amb aquesta unitat: jerarquies, sobrescriptura i contractes. Les col·leccions (RA6) arriben en la **U12**.

🚪 Per on començo?

- Cero coneixements? → Comença en el punt 1. Només necessites el que vas veure en les unitats 06 i 07.
- Ja heredes i vols el contracte? → Ves directe al punt 7 i al punt 8: aquí està la mitat abstracta de la unitat.
- Només vens a per el polimorfisme? → Salta al punt 4 i al punt 5.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Herència: quan els teus fills segueixen els teus passos
⏭️ **En acabar la unitat, continua en U12 · Col·leccions.**

01 — Herència: quan els teus fills segueixen els teus passos

extends, què s'hereda i què no, IS-A vs HAS-A i l'arbre genealògic de les teves classes 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 01 · Herència: quan els teus fills segueixen els teus passos

---

📬 La idea en una frase

> **L'herència permet que una classe filla obtinga els membres d'una classe pare amb `extends`, reutilitzant codi i afegint o millorant el que calgui.**

Recordes quan vas heretar el nas de l'àvia o el geni per a enfadar els professors? Doncs en Java passa el mateix, però amb menys drama i més codi reutilitzable.

🧬 Què és l'herència?

L'herència és el mecanisme pel qual una classe *filla* (subclasse) obté tots els membres d'una classe *pare* (superclasse). I pot fer dues coses més: **afegir** els seus propis i **millorar** els existents.

```java
public class Animal {
    protected String nombre;
    protected int edad;

    public void hacerSonido() {
        System.out.println("Algún sonido genérico...");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("¡Guau guau!");
    }

    public void moverCola() {
        System.out.println("*mueve la cola felizmente*");
    }
}
```

`Perro` ara té `nombre`, `edad`, `hacerSonido()` (millorat) i `moverCola()`. Cortesia de l'herència.

> 💡 **Consell:** codi reutilitzat, neurones estalviades. L'herència existeix perquè NO hagis de copiar i enganxar el mateix codi en 15 classes.

🔑 extends: "Soc com tu, però amb millores"

La paraula clau és `extends` (la vas veure en la U10 per damunt, ara la domines):

```java
public class Coche extends Vehiculo {
    // Coche té TOT el de Vehiculo + el seu
}
```

Darrere de cada `extends` hi ha una pregunta: **realment és-un?** Si la resposta és sí, endavant. Si és "té-un"... això és composició, i la veuràs en el punt 3.

Què s'hereda i què no?

No tot viatja de pares a fills. Aquesta taula t'ho aclareix:

Membre	¿Ho hereta la filla?
`public`	✅ Sí
`protected`	✅ Sí
package-private (sense modificador)	✅ Sí, si és al mateix paquet
`private`	❌ No (però existeix dins de l'objecte)
Constructors	❌ No (s'invoquen amb `super()`, punt 2)
Mètodes `final`	❌ No es poden sobreescriure

> ⚠️ **Advertència:** els membres `private` no s'hereden, però continuen aquí, dins de l'objecte. La filla no pot tocar-los directament, però el pare els usa. És com els diners al compte dels teus pares: existeixen, però no els toques.

⚖️ La regla d'or: IS-A vs HAS-A

- **IS-A** (és-un): relació d'herència. `Perro` IS-A `Animal`.
- **HAS-A** (té-un): relació de composició. `Coche` HAS-A `Motor`.

```
public class Coche extends Vehiculo { }   // IS-A ✓
public class Coche { private Motor m; }   // HAS-A ✓
```

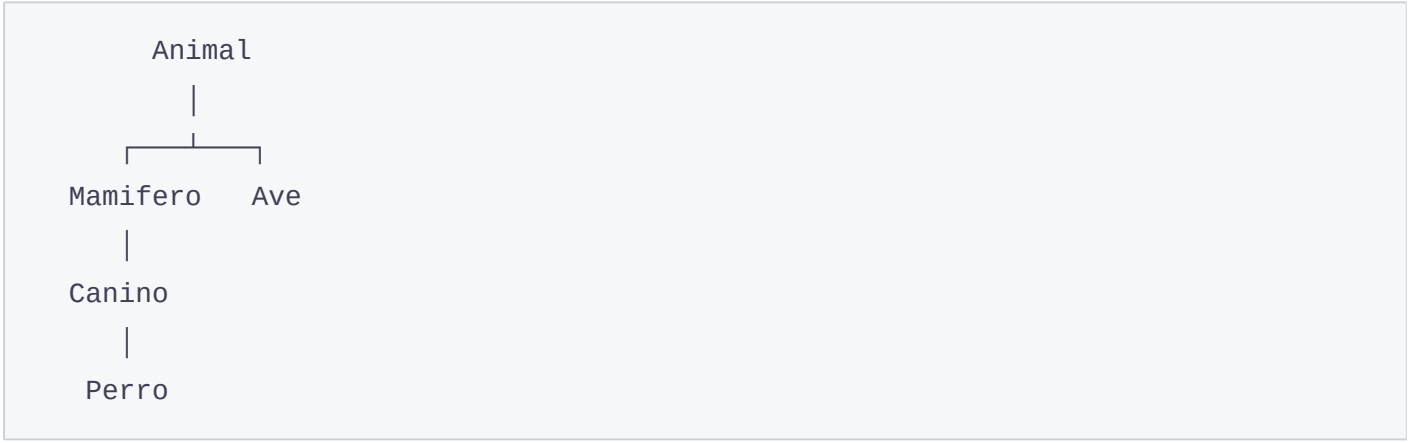
> 🎮 **¿Qui vol ser milionari? — Edició Java:** Quina és la relació correcta? a) `Cliente extends Persona` b) `Cliente has-a Persona` c) `Coche extends Rueda`
>
> **Resposta:** la a. `Cliente` IS-A `Persona`. `Coche` NO és una roda: té rodes (HAS-A). Si vas dubtar amb la c, repassa el teu vocabulari de cotxes.

🌳 Jerarquia de classes: l'arbre genealògic

Una classe pot heredar d'una altra que al seu torn hereta d'una altra. Així es formen les jerarquies:

```java
public class Animal { }
public class Mamifero extends Animal { }
public class Canino extends Mamifero { }
public class Perro extends Canino { }
```

`Perro` hereta de `Canino`, que hereta de `Mamifero`, que hereta de `Animal`. Cada generació aporta el seu:



> 📝 **Nota:** com més amunt en l'arbre, més general és la classe; com més avall, més específica. `Animal` no sap bordar; `Perro` no sap volar. Cadascú al seu nivell.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan una classe hereta, s'emporta TOT el accessible. Pregunta't sempre què veu la filla abans d'escriure `extends`.

Exercici: l'examen de visibilitat

```java
public class Animal {
    private String idSecreto = "X-123";
    protected String nombre = "Animal";
    public int edad = 5;
}

public class Perro extends Animal {
    public void mostrar() {
        System.out.println(idSecreto); // ¿compila?
        System.out.println(nombre);    // ¿compila?
        System.out.println(edad);      // ¿compila?
    }
}
```

Preguntes (sense executar):

1. Quina de les tres línies compila?
2. Què passaria si `idSecreto` fos `protected` en comptes de `private`?

> 🔄 **Solució**
>
> 1. `idSecreto` **NO** compila (és `private`: només `Animal` el veu). `nombre` **sí** (és `protected`). `edad` **sí** (és `public`).
> 2. Llavors `Perro` el veuria sense problema. `protected` és el "secret de família" del qual parlaràs en el punt 3.

Exercici: què imprimeix?

```java
class Abuelo { void decir() { System.out.println("Abuelo"); } }
class Padre extends Abuelo { void decir() { System.out.println("Padre"); } }
class Hijo extends Padre { void decir() { System.out.println("Hijo"); } }

public class Test {
    public static void main(String[] args) {
        new Hijo().decir();
    }
}
```

> 🔄 **Solució**

> Imprimeix **"Hijo"**. Java busca el mètode des de la classe més específica cap amunt: troba `decir()` en `Hijo` i aquí es queda. Mai no puja a `Padre` ni a `Abuelo`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina paraula clau uses perquè `Perro` herede d'`Animal`?
2. S'hereden els atributs `private`? I els `protected`?
3. És `Coche extends Rueda` bona idea? Per què?
4. Què significa que una jerarquia de classes és un "arbre genealògic"?

> 🔄 **Respostes**
>
> 1. `extends`: `public class Perro extends Animal { }`.
> 2. `private` no s'hereden; `protected` sí (i també `public`).
> 3. No: `Coche` no és una roda, té rodes. Seria HAS-A (composició), no IS-A.
> 4. Que cada nivell és més específic: `Perro` hereta de `Canino`, que hereta de `Mamifero`, que hereta de `Animal`.

✅ Resum en 3 frases

1. **L'herència** amb `extends` permet que una subclasse reutilitzi i millore els membres de la seva superclasse.
2. Només s'hereda el **accessible** (`public`, `protected` i package-private); els `private` i els constructors no.
3. Abans d'heredar, pregunta: **és-un?** Si és "té-un", usa composició.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Superclasse	La classe pare, la general
Subclasse	La classe filla, l'específica

Terme	Idea general
`extends`	La paraula que estableix l'herència
IS-A	Relació d'herència (Perro és-un Animal)
HAS-A	Relació de composició (Coche té-un Motor)
Jerarquia	L'arbre de classes de general a específic

📚 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · super i @Override

02 — super i @Override

Cridar la superclasse, sobreescriure mètodes i encadenar constructors sense plorar 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 02 · super i @Override

---

📬 La idea en una frase

> **`super` crida la superclasse (constructors i mètodes) i `@Override` li diu al compilador que estàs sobreescrivint, no inventant.**

En el punt 1 vas veure que la filla "hereta". Però heretar no és suficient: de vegades vols que el teu pare faci primer la seva part i després tu la teva. Per a això existeixen `super` i `@Override`.

📞 super: cridant mamà/papà perquè t'ajuden

`super` té dos usos, i convé tenir-los clars:

1. `super(...)` per a invocar el constructor del pare

```
public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        super(marca);          // ← configura la part de Vehiculo
        this.puertas = puertas; // ← i després el teu
    }
}
```

Si el pare té un constructor amb paràmetres, **has de** cridar-lo amb `super(...)` i ha de ser **la primera línia** del constructor fill. Si no ho fas, el compilador intenta cridar `super()` (sense arguments)… que no existeix. Error de compilació immediat.

> ⚠️ **Advertència:** `super(...)` només serveix per a invocar constructors i mètodes de la superclasse. No pots passar-lo com a paràmetre ni assignar-lo a una variable. És un punter "només lectura" cap amunt.

2. `super.metodo()` per a cridar el mètode del pare

De vegades volem **estendre** el mètode del pare, no reemplaçar-lo:

```
public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        super.hacerSonido();   // primer el que fa Animal
        System.out.println("¡MIAU!");  // i després el meu
    }
}
```

`super` és com cridar "¡MAAAAMÁ!" al supermercat: "executa la versió del meu pare, i després jo faig la meva".

✍️ @Override: "Papa, jo ho faig millor"

`@Override` és una anotació que li diu al compilador: "assegura't que realment estic sobreescrivint un mètode del pare":

```
public class Pez extends Animal {
    @Override
    public void hacerSonido() { }  // ✓ existeix en Animal

    @Override
    public void nadar() { }        // ✗ ERROR: no existeix en Animal
}
```

La segona línia no compila: el compilador verifica que `nadar()` existeixi en `Animal` i, com que no existeix, t'avisa. Aquell avís a temps val or: si escrius mal el nom o la signatura d'un mètode,

`@Override` t'ho descobreix sense esperar un bug rar en runtime.

> 📝 **Nota:** `@Override` no és obligatori, però posa'l sempre. És com el cinturó de seguretat: no passa res si no el poses… fins que passa. A més, és la forma de dir-li a qui llegeix el teu codi "això és un override, no un mètode nou".

Regles d'or de l'override

1. La signatura ha de ser **idèntica** (nom i paràmetres).
2. El tipus de retorn ha de ser el mateix o un subtipus (covariant).
3. No pots reduir la visibilitat: `public` en el pare no pot tornar-se `protected` en la filla.
4. Els mètodes `final` no es poden sobreescriure; les classes `final` no poden tenir filles.

🔒 final: quan vols bloquejar l'herència

`final` és el fre de mà:

```
public final class String { }        // no pot tindre filles

public class Vehiculo {
    public final void frenar() { }   // les filles NO poden sobreescriure'l
}
```

Usa'l quan una classe o un mètode no ha de modificar-se. Per exemple, un mètode que forma part de l'esquelet d'un algoritme (ho veuràs en el template method del punt 7).

🏗️ Constructors en l'herència: la cadena de muntatge

Quan crees un objecte de la classe més profunda, s'executen **tots** els constructors de la cadena, del més general al més específic:

```java
public class Abuelo {
    public Abuelo() { System.out.println("Abuelo"); }
}
public class Padre extends Abuelo {
    public Padre() { super(); System.out.println("Padre"); }
}
public class Hijo extends Padre {
    public Hijo() { super(); System.out.println("Hijo"); }
}
// new Hijo() imprimeix: Abuelo / Padre / Hijo
```

Si no escrius `super()`, Java l'afegeix automàticament al principi del constructor. Per això la regla és: **el pare es construeix sempre primer**. És com muntar una casa: primer els fonaments, després les parets, després el teulada.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** si el pare té constructor amb paràmetres, el fill està obligat a cridar-lo amb `super(...)`. Si no, el compilador intenta el buit… que no existeix.

L'exercici clàssic: què impedeix que compili això?

```java
public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void arrancar() {
        System.out.println("Vehículo arrancado");
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        this.puertas = puertas;
    }

    @Override
    public void arrancar() {
        System.out.println("Coche arrancado con " + puertas + " puertas");
    }
}
```

Pregunta: quin error impedeix que `Coche` compili?

> 🔄 **Solució**
>
> El constructor de `Coche` no crida a `super(...)`. Com que `Vehiculo` només té `Vehiculo(String)`, el compilador no troba un `super()` sense arguments i falla. La solució:
>
> ```java
> public Coche(String marca, int puertas) {
>     super(marca);
>     this.puertas = puertas;
> }
> ```

Exercici: què imprimeix amb herència encadenada i super?

```java
class Vehiculo {
    void describir() { System.out.println("Soy un vehículo"); }
}
class Coche extends Vehiculo {
    void describir() { System.out.println("Soy un coche"); }
    void describirCompleto() { super.describir(); this.describir(); }
}
class Deportivo extends Coche {
    void describir() { System.out.println("Soy un coche deportivo"); }
}

public class Test {
    public static void main(String[] args) {
        Deportivo d = new Deportivo();
        d.describirCompleto();
        d.describir();
    }
}
```

> 🔄 **Solució**
>
> Imprimeix:
>
> ```
> Soy un vehículo
> Soy un coche deportivo
> Soy un coche deportivo
> ```
>
> `super.describir()` dins de `Coche` va directe a `Vehiculo`. En canvi, `this.describir()` es resol en runtime amb l'objecte real, que és un `Deportivo`. Aquell detall és el polimorfisme, i li dedicaràs el punt 4.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quins són els dos usos de `super`?
2. Què li demanes a `@Override`?
3. Què ocorre si el fill no crida a `super(...)` i el pare no té constructor buit?
4. Pot una filla sobreescriure un mètode `final` del pare?

> 🔄 **Respostes**
>
> 1. `super(...)` per a cridar el constructor del pare i `super.metodo()` per a cridar un mètode del pare.
> 2. Que verifiqui en el compilador que realment està sobreescrivint un mètode existent.
> 3. Error de compilació: Java intenta `super()` sense arguments i no existeix.
> 4. No: `final` prohibeix la sobrescriptura. I una classe `final` ni tan sols pot tenir filles.

✅ Resum en 3 frases

1. **`super`** encadena amb la superclasse: constructors i mètodes, sempre com a primera jugada.
2. **`@Override`** és la teva xarxa de seguretat: el compilador comprova que el mètode sobreescrit existeix.
3. Els **constructors** s'executen de dalt a baix (el pare primer) i els mètodes `final` són intocables.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
`super(...)`	Crida al constructor de la superclasse
`super.metodo()`	Crida a un mètode de la superclasse
`@Override`	Anotació que verifica la sobrescriptura
Sobreescriure	Redefinir un mètode heretat amb una altra implementació
`final`	Bloqueja l'herència d'una classe o la sobrescriptura d'un mètode
Cadena de constructors	El pare es construeix sempre abans que el fill

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Herència: quan els teus fills segueixen els teus passos · **Següent:** 03 · protected i jerarquies de classes

03 — protected i jerarquies de classes

El membre que només veu la família, arbres de classes i el perill de la classe base fràgil 🧬

> 🗺️ **Estàs en:** 🧬 **U11 · Herència, polimorfisme i interfícies** → 03 · protected i jerarquies de classes

📬 La idea en una frase

> **`protected` és la visibilitat de la família: el membre el veuen les subclasses (i el mateix paquet). I les jerarquies de classes ben dissenyades són arbres, no boscos embullats.**

En la U10 vas veure els quatre nivells de visibilitat. Aquí descobreixes el matís que només cobra sentit amb herència: `protected`.

🔐 protected: el secret que només la família veu

Visibilitat	Mateixa classe	Mateix paquet	Subclasses (qualsevol paquet)	Tots
`private`	✅	❌	❌	❌
package-private	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

`protected` és el punt dolç: ni privat ni públic per a tot el món. És la porta amb **clau per a la família**:

```
public class Animal {
    private String secreto;    // Només Animal
    protected String familia;  // Animal, les seues subclasses i el seu paquet
    public String nombre;      // Tots
}
```

> 💡 **Consell pràctic:** quan una classe base necessita que les filles llegeixen un atribut o criden un mètode, declara'l `protected`. Quan les filles no han de tocar-lo, `private`. I si ningú ha d'heredar de la teva classe… `final`.

🌳 Dissenyant jerarquies sensates

Una jerarquia de classes és un arbre. Les regles d'un arbre sa:

1. **L'arrel és general, les fulles són específiques.** `Animal` al cim, `Perro` a la base.
2. **Cada nivell afegeix alguna cosa.** No creïs una classe només per tenir una classe més.
3. **Poca profunditat.** 2 o 3 nivells són fàcils de mantenir; 7 nivells són 7 nivells de dolor.
4. **Tot hereta d'`Object`** (ho veuràs en el punt 6), encara que no l'escriguis.

```java
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public void mostrarInfo() {
        System.out.println(nombre + " cobra " + salarioBase + " €");
    }
}

public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double salarioBase, double bono) {
        super(nombre, salarioBase);
        this.bono = bono;
    }

    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("  i té un bo de " + bono + " €");
    }
}
```

Fixa't en el patró: el `Gerente` usa `super(...)` per a construir la part d'`Empleado` i `super.mostrarInfo()` per a estendre la sortida. Aquell és el disseny net d'una jerarquia: cada classe s'encarrega del seu.

💥 El problema de la classe base fràgil

Aquí va l'advertència seriosa. Imagina que tens un `Jarrón` amb un mètode `romper()` i un `JarrónChino` que el sobreescriu per a fer un ritual de rotura. Tot va bé… fins que algú afegeix un mètode `caerAlSuelo()` a `Jarrón` que internament crida a `romper()`:

```
public class Jarrón {
    public void romper() { System.out.println("¡Crac! Jarrón roto"); }

    public void caerAlSuelo() {
        System.out.println("¡Se cae!");
        romper();   // ¡sorpresa: pot cridar l'override de JarrónChino!
    }
}
```

Ara `JarrónChino` es comporta de forma inesperada cada vegada que cau al sòl, sense que ningú toqués el seu codi. Aquest és el **problema de la classe base fràgil** (fragile base class problem): canviar la classe pare pot trencar el comportament de les filles.

> ⚠️ **Advertència:** l'herència crea acoblament. Quan canvies `Jarrón`, TOTES les seves filles canvien amb tu, vulguis o no. És un superpoder amb efectes secundaris.

> ❓ **Llavors l'herència és dolenta?**
>
> No! És una eina. Ben usada és perfecta per a relacions IS-A clares. El problema és usar-la quan una simple composició bastaria. La regla moderna: **prefereix composició sobre herència**, i usa herència només per a "és-un" reals.

🧩 Composició: "tenir" en comptes de "ser"

```
// HERÈNCIA: un cotxe ÉS UN vehicle
public class Coche extends Vehiculo { }

// COMPOSICIÓ: un cotxe TÉ UN motor
public class Coche {
    private Motor motor;
    private Rueda[] ruedas;
}
```

Per què preferir composició de vegades?

1. **Menys acoblament:** canviar la classe de la part no trenca la que la conté.
2. **Més flexible:** pots canviar les parts en temps d'execució.
3. **Evites jerarquies profundes:** 5 nivells d'herència = 5 nivells de dolor.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** `protected` no és "públic per als amics": és una visibilitat formal que el compilador controla. Usa-la amb intenció, no per peresa.

Exercici: l'arbre que no compila

```
public class SerVivo {
    protected int energia;

    public SerVivo(int energia) {
        this.energia = energia;
    }
}

public class Animal extends SerVivo {
    protected String nombre;

    public Animal(String nombre, int energia) {
        super(energia);
        this.nombre = nombre;
    }
}

public class Perro extends Animal {
    public Perro(String nombre, int energia) {
        super(nombre, energia);
    }

    public void ladrar() {
        System.out.println(nombre + " ladra con energía " + energia);
    }
}
```

Pregunta: per què `Perro` pot usar `nombre` i `energia` encara que no les declari? I què passaria si `energia` fos `private`?

> › 🔄 **Solució**
>
> `Perro` usa `nombre` (declarat en `Animal`) i `energia` (declarat en `SerVivo`) perquè tots dos són `protected`: l'herència els posa a disposició de tota la cadena. Si `energia` fos `private`, `Perro` no podria llegir-lo directament; hauria de passar per un getter de `SerVivo`.

Exercici: el gerro que cau

Amb el codi de `Jarrón` i `JarrónChino` de dalt, què imprimiria si fas `new JarrónChino().caerAlSuelo()` sabent que `JarrónChino` sobreescriu `romper()` per a cridar "¡LAMENTO MILENARIO!"?

> 🔄 **Solució**
>
> ```
> ¡Se cae!
> ¡LAMENTO MILENARIO!
> ```
>
> El mètode `caerAlSuelo()` està en `Jarrón`, però en cridar a `romper()` usa l'override de `JarrónChino` (polimorfisme). Un canvi en la base va alterar el comportament de la filla sense tocar el seu codi: la classe base fràgil en acció.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Qui veu un membre `protected`?
2. Quants nivells de profunditat suggereix el curs per a una jerarquia?
3. Què és el problema de la classe base fràgil?
4. Quan usaries composició en comptes d'herència?

> 🔄 **Respostes**
>
> 1. La pròpia classe, les classes del mateix paquet i totes les seves subclasses (estiguin on estiguin). El món exterior, no.
> 2. 2 o 3 nivells. Més profunditat = més acoblament i més dolor.
> 3. Que en canviar la superclasse, les subclasses es comporten de forma inesperada perquè hereden aquells canvis.
> 4. Quan la relació és "té-un" (HAS-A), quan necessites flexibilitat o quan la jerarquia es faria massa profunda.

✅ Resum en 3 frases

1. **`protected`** és la visibilitat familiar: subclasses i paquet el veuen; el món, no.

2. Les **jerarquies sanes** són poc profundes, cada nivell aporta alguna cosa i l'arrel és general.
3. L'herència **acobla**; el problema de la classe base fràgil avisa que canviar el pare té conseqüències. Composició sobre herència quan hi hagi dubti.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `protected` | Visible per a la classe, el paquet i les subclasses |
> | Jerarquia de classes | Arbre d'herència del general a l'específic |
> | Classe base fràgil | Canvis en la superclasse que trenquen subclasses sense avisar |
> | Acoblament | Quant depenen unes classes d'unes altres |
> | Composició | "Tenir" (HAS-A) en comptes de "ser" (IS-A) |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · super i @Override · **Següent:** 04 · Polimorfisme: el camaleó de la POO

04 — Polimorfisme: el camaleó de la POO

Dynamic binding, referències polimòrfiques, col·leccions i paràmetres que canvien de forma 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 04 · Polimorfisme: el camaleó de la POO

📬 La idea en una frase

> **El polimorfisme fa que un mateix mètode es comporte de forma diferent segons l'objecte real que l'invoque, decidint-ho la JVM en temps d'execució.**

Polimorfisme (*poly* = moltes, *morphé* = formes): un mateix mètode es comporta diferent segons l'objecte que l'invoque. Com un comandament universal on el mateix botó "PLAY" funciona en Netflix, Spotify i la teva torradora.

🎭 El mateix mètode, diferents comportaments

```
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}

public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Miau!"); }
}
```

I ara la màgia:

```
Animal a;
a = new Perro(); a.hacerSonido(); // ¡Guau!
a = new Gato();  a.hacerSonido(); // ¡Miau!
```

La variable a és de tipus `Animal`, però apunta a objectes diferents. S'executa sempre el mètode de l'**objecte real**, no el del tipus de la variable.

🔮 Dynamic binding: la decisió en runtime

El compilador i la JVM fan treballs distints:

> **Compilador:** "¿`Animal` té `hacerSonido()`? Sí. Endavant." **JVM (runtime):** "L'objecte és un `Perro`. Executi el de `Perro`."

Això és **dynamic binding** (o late binding): el *què* es pot cridar ho decideix el compilador; el *com* s'executa ho decideix la JVM quan el programa corre. Aquella separació és la que fa possible el polimorfisme.

> 💡 **Detall pràctic:** si Java resolgués els mètodes en compilació (static binding), cridar a `a.hacerSonido()` sempre executaria la versió d'`Animal`. El polimorfisme existeix perquè la decisió s'ajorna fins al moment just.

🔗 Referències polimòrfiques

Pots guardar un objecte específic en una variable de tipus més general:

```
Animal miMascota = new Perro();  // ✓
Animal tuMascota = new Gato();   // ✓
```

Però la variable només "veu" els mètodes del tipus de la referència:

```
Animal a = new Perro();
a.hacerSonido();   // ✓
a.moverCola();     // ✗ Animal no tiene moverCola()
```

> ⚠️ **Advertència:** el compilador és conservador: si la variable és `Animal`, només deixa cridar mètodes d'`Animal`, encara que per dins hi hagi un `Perro`. Per a accedir a `moverCola()` necessites el punt 5 (downcasting).

📚 Polimorfisme amb col·leccions

Aquí és on el polimorfisme paga el lloguer:

```
import java.util.ArrayList;

ArrayList<Animal> animales = new ArrayList<>();
animales.add(new Perro());
animales.add(new Gato());
animales.add(new Vaca());

for (Animal a : animales) {
    a.hacerSonido();  // cada u el seu
}
// ¡Guau! / ¡Miau! / ¡Muuu!
```

Una sola llista, un sol bucle. Sense polimorfisme necessitaries tres llistes separades (i una bicicleta per a portar-les).

🧑‍⚕️ Polimorfisme amb paràmetres

Un mètode que accepta el tipus general funciona amb qualsevol subclasse:

```
public class Veterinario {
    public void vacunar(Animal a) {
        System.out.print("Vacunando a: ");
        a.hacerSonido();
    }
}

Veterinario vet = new Veterinario();
vet.vacunar(new Perro()); // Vacunando a: ¡Guau!
vet.vacunar(new Gato());  // Vacunando a: ¡Miau!
```

I pots anar més lluny: una classe que gestiona una col·lecció polimòrfica:

```
public class Zoologico {
    private ArrayList<Animal> animales = new ArrayList<>();

    public void agregarAnimal(Animal a) {
        animales.add(a);
    }

    public void hacerDesfile() {
        for (Animal a : animales) {
            System.out.print(a.getClass().getSimpleName() + " dice: ");
            a.hacerSonido();
        }
    }
}
```

Afegir un `Gato` o un `Perro` no requereix canviar **ni una sola línia** del `Zoologico`. Això és disseny a prova de futur: el codi s'obre a noves classes sense tocar-se.

🚫 Els atributs NO són polimòrfics

Punt important per a l'examen: la màgia només funciona amb **mètodes**. Els atributs es resolen pel tipus de la **referència**, no de l'objecte:

```
class A { int x = 1; }
class B extends A { int x = 2; }

A ref = new B();
System.out.println(ref.x);   // 1, NO 2
```

`ref.x` torna `1` perquè els atributs no participen en el dynamic binding. Només els mètodes. Els atributs "oculten" (shadowing), no sobreescriuen.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** el compilador mira el tipus de la variable; la JVM mira el tipus real de l'objecte. Aquí està la màgia.

Exercici: què imprimeix?

```
class A { void saluda() { System.out.println("Hola desde A"); } }
class B extends A { void saluda() { System.out.println("Hola desde B"); } }
class C extends B { void saluda() { System.out.println("Hola desde C"); } }

public class Test {
    public static void main(String[] args) {
        A ref1 = new B();
        A ref2 = new C();
        B ref3 = new C();
        ref1.saluda();
        ref2.saluda();
        ref3.saluda();
    }
}
```

> 🔄 **Solució**

```
Hola desde B
Hola desde C
Hola desde C
```

Només importa el tipus real de l'objecte (B, C, C). El tipus de la referència (A, A, B) no decideix res en runtime.

Exercici: compila?

```
class Vehiculo { void mover() { System.out.println("Vehículo se mueve"); } }

class Coche extends Vehiculo {
    void mover() { System.out.println("Coche acelera"); }
    void abrirPuertas() { System.out.println("Puertas abiertas"); }
}

public class Test {
    public static void main(String[] args) {
        Vehiculo v = new Coche();
        v.mover();
        // v.abrirPuertas();   <- ¿compila?
    }
}
```

> 🔄 Solució

`v.mover()` imprimeix **"Coche acelera"**. La línia comentada **NO compila**: el compilador només mira el tipus de la referència (`Vehiculo`), i `Vehiculo` no té `abrirPuertas()`.

Exercici: què imprimeix o?

```
interface Volable { void volar(); }
class Pajaro implements Volable {
    public void volar() { System.out.println("Vuela"); }
}
class Aguila extends Pajaro {
    public void volar() { System.out.println("Vuela alto"); }
}

public class Test {
    public static void main(String[] args) {
        Volable v = new Aguila();
        Pajaro p = new Aguila();
        Object o = new Aguila();
        v.volar();
        p.volar();
        // o.volar() no compila: Object no tiene volar()
    }
}
```

> 🔄 Solució

`v.volar()` i `p.volar()` imprimeixen **"Vuela alto"**: el tipus de la referència no importa, només l'objecte real. `o.volar()` ni tan sols compila perquè `Object` no coneix `volar()` (encara que en runtime l'objecte sí que el tingui). El polimorfisme actua dins dels límits que el compilador permet.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què és el dynamic binding?
2. `Animal a = new Perro();` — quins mètodes pots cridar amb a?
3. Per què no necessites canviar el `Zoologico` en afegir un animal nou?

4. Els atributs són polimòrfics?

> 🔄 **Respostes**
>
> 1. Que el mètode a executar es decideix en runtime segons el tipus real de l'objecte, no en compilació.
> 2. Només els d'`Animal` (`hacerSonido()`). Per a `moverCola()` necessites downcasting (punt 5).
> 3. Perquè `agregarAnimal` accepta `Animal` (o qualsevol subclasse) i el bucle usa polimorfisme.
> 4. No. Els mètodes són polimòrfics; els atributs es resolen pel tipus de la referència.

✅ Resum en 3 frases

1. **Polimorfisme** = un mètode, moltes formes: la JVM tria la implementació de l'objecte real.
2. Les **referències polimòrfiques** (`Animal a = new Perro()`) permeten col·leccions i paràmetres flexibles.
3. Els **mètodes** són polimòrfics; els **atributs**, no.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Polimorfisme	Un mateix mètode es comporta segons l'objecte real
Dynamic binding	Decisió del mètode en temps d'execució
Referència polimòrfica	Variable de tipus general apuntant a un objecte específic
Override	L'eina que canvia el comportament
Shadowing	Ocultar un atribut de la superclasse (els atributs no són polimòrfics)

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · protected i jerarquies de classes · **Següent:** 05 · instanceof i downcasting

05 — instanceof i downcasting

Preguntar qui ets realment i baixar la referència amb cura per a no estavellar-te 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 05 · instanceof i downcasting

---

📬 La idea en una frase

> **`instanceof` pregunta "ets realment això?" i el downcasting `(Tipo)` baixa la referència; junts són segurs, per separat… perillosos.**

En el punt 4 vas veure que una referència `Animal` no pot cridar a `moverCola()` d'un `Perro`. Per a arribar a aquells mètodes específics necessites baixar la referència. I per a baixar sense estavellar-te, necessites preguntar abans.

🕵️ instanceof: "Qui ets realment?"

`instanceof` torna `true` si l'objecte és d'aquell tipus (o d'una subclasse):

```
Animal a = new Perro();
if (a instanceof Perro) {
    System.out.println("¡Es un perro!");
}
```

> 💡 **Consell:** usa'l amb moderació. Si omplis el teu codi d'`instanceof`, alguna cosa estàs fent mal: el polimorfisme (punt 4) hauria de resoldre la majoria dels casos sense preguntar. `instanceof` és per a quan un objecte concret necessita un tracte que el polimorfisme no cobreix.

📉 Downcasting: baixar la referència

Pujar (upcasting) és gratuït i implícit: `Animal a = new Perro();`. Baixar (downcasting) és explícit i arriscat:

```
Animal a = new Perro();
Perro p = (Perro) a;   // downcasting
p.moverCola();         // ✓ funciona
```

Però si t'equivoques de tipus:

```
Animal a2 = new Gato();
Perro p2 = (Perro) a2;  // 💥 ClassCastException en runtime
```

Java no es deixa enganyar: l'objecte real és un `Gato`, i convertir-lo a `Perro` explota en temps d'execució. És com intentar fer parlar un gat en un concurs de gossos: el públic (la JVM) ho veu i munta un escàndol.

> ⚠️ **Advertència:** el downcasting **compila sempre** (el compilador no sap què hi ha dins de la variable), però pot fallar en runtime. L'única xarxa de seguretat és preguntar abans.

🛡️ El combo segur: instanceof + downcasting

La recepta que mai no falla:

```
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.moverCola();
} else {
    System.out.println("No es un perro, no puedo hacerlo mover la cola.");
}
```

`instanceof` és la teva mirilla: si és un repartidor de pizzes, obre; si és un lleó, millor no.

Exemple real: downcasting en una llista polimòrfica

```java
import java.util.ArrayList;

ArrayList<Empleado> plantilla = new ArrayList<>();
plantilla.add(new Programador("Ana"));
plantilla.add(new Disenador("Luis"));
plantilla.add(new Programador("Eva"));

for (Empleado e : plantilla) {
    if (e instanceof Programador) {
        ((Programador) e).escribirCodigo();
    } else if (e instanceof Disenador) {
        ((Disenador) e).disenar();
    }
}
```

Cada empleat rep el seu tracte especial només si realment és d'aquell tipus. Sense `instanceof`, no podries cridar a `escribirCodigo()` ni a `disenar()`.

✨ Bonus: pattern matching (Java 16+)

Des de Java 16 hi ha una versió més neta d'`instanceof` + casting en una sola línia:

```java
if (a instanceof Perro p) {
    p.moverCola();   // ja no fa falta el (Perro)
}
```

El compilador crea la variable `p` automàticament dins del bloc. Menys codi, mateix resultat, zero `ClassCastException` possible.

> 📝 **Nota:** en el teu IDE de classe, si uses una versió de Java anterior a la 16, queda't amb la forma clàssica. El patró no canvia el concepte: preguntar abans de baixar.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** el downcasting sense `instanceof` és jugar a la ruleta russa amb el teu programa. Pregunta sempre abans de baixar.

Exercici: què ocorre aquí?

```java
Animal a = new Perro();
if (a instanceof Gato) {
    System.out.println("Es un gato");
} else {
    System.out.println("No es un gato");
}
```

> 🔄 **Solució**
>
> Imprimeix **"No es un gato"**. L'objecte real és un `Perro`, així que `a instanceof Gato` és `false`. El programa ni tan sols intenta el downcasting: `instanceof` t'ha salvat del `ClassCastException`.

Exercici: el jutge de la granja

```java
ArrayList<Animal> granja = new ArrayList<>();
granja.add(new Perro());
granja.add(new Gato());
granja.add(new Vaca());

for (Animal a : granja) {
    if (a instanceof Perro) {
        System.out.println("El perro mueve la cola");
    } else if (a instanceof Gato) {
        System.out.println("El gato ronronea");
    } else {
        System.out.println("Alguien da leche");
    }
}
```

Quants `instanceof` són necessaris? Podries haver-ho fet sense cap?

> 🔄 **Solució**
>
> Dos `instanceof` basten (el `else` final captura a `Vaca`). I sí, es podria haver resolt amb polimorfisme pur: un mètode `comportamientoEspecial()` en cada animal i un sol `for`. `instanceof` brilla quan el comportament és impossible de ficar en la superclasse (mètodes que només existeixen en una subclasse, com `moverCola()`).

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `instanceof` si l'objecte no és d'aquell tipus?
2. Quina excepció llança un downcasting mal fet?
3. Per què el combo `instanceof` + casting és segur?
4. Quan NO hauries d'usar `instanceof`?

> 🔄 **Respostes**
>
> 1. `false`.
> 2. `ClassCastException`, en temps d'execució.
> 3. Perquè `instanceof` garanteix que l'objecte és d'aquell tipus (o subclasse) abans de convertir, així que el cast no pot fallar.
> 4. Quan el polimorfisme pot resoldre-ho amb un mètode sobreescrit: no preguntis el que la JVM ja sap.

✅ Resum en 3 frases

1. **`instanceof`** comprova el tipus real de l'objecte abans d'actuar.
2. El **downcasting** `(Tipo) ref` baixa la referència i només és segur després d'un `instanceof`.
3. Usa aquest combo **amb moderació**: el polimorfisme sol ser la solució més neta.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`instanceof`	Operador que comprova el tipus real d'un objecte
Downcasting	Convertir una referència a un tipus més específic
Upcasting	Convertir (implícit) a un tipus més general
`ClassCastException`	Explosió en runtime per un cast impossible
Pattern matching	`instanceof Perro p` (Java 16+): preguntar i baixar en un pas

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Polimorfisme: el camaleó de la POO · **Següent:** 06 · La classe Object: el besavi de tot

06 — La classe Object: el besavi de tot

toString, equals i hashCode: els tres mètodes que tota classe hereta i quasi ningú sobreescriu bé 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 06 · La classe Object: el besavi de tot

📬 La idea en una frase

> **Tota classe hereta d'`Object`, i els seus tres mètodes estrella (`toString()`, `equals()` i `hashCode()`) cal sobreescriure'ls perquè la teva classe es comporte amb sentit comú.**

Sabies que la teva classe `Perro` té un besavi? Totes les classes hereden d'`Object`, encara que no escriguis `extends Object`. És l'arrel de tot l'arbre.

👴 Object: l'arrel de tot

```
public class MiClase { } // = public class MiClase extends Object { }
```

Mètodes que tota classe hereta:

Mètode	Què fa?	Sobreescriure'l?
`toString()`	Representació textual de l'objecte	Quasi sempre
`equals(Object)`	Compara per valor	Quan tingui sentit
`hashCode()`	Codi hash per a col·leccions	Amb `equals`
`getClass()`	Classe real de l'objecte	No
`clone()`	Còpia (superficial)	Rarament

> 💡 **Consell:** els quatre primers els uses cada dia. `clone()` millor oblida'l: usa un constructor de còpia (el vas veure en la U09).

📇 toString(): la targeta de presentació

Per defecte, `Object.toString()` torna alguna cosa com `Perro@1a2b3c` (la classe i una adreça de memòria). Inútil per al ser humà mitjà. Sobreescriu-lo:

```
public class Perro {
    private String nombre;
    private int edad;

    public Perro(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Perro{nombre='" + nombre + "', edad=" + edad + "}";
    }
}

System.out.println(new Perro("Firulais", 3));
// Perro{nombre='Firulais', edad=3}
```

Fixa't: `System.out.println(objeto)` crida a `toString()` automàticament. I també la concatenació `"Perro: " + perro`. És el mètode que Java crida en mil llocs sense que el demanes.

⚖️ equals(): mateix objecte o iguals?

Per defecte, `equals()` compara **referències** (és a dir, fa ==). Dos objectes diferents amb les mateixes dades NO són iguals. Per a comparar per **valor**, sobreescriu-lo:

```java
import java.util.Objects;

public class Perro {
    private String nombre;
    private int edad;

    // constructor...

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                    // 1. mateix objecte
          if (o == null || getClass() != o.getClass())    // 2. null o classe distinta
            return false;
        Perro perro = (Perro) o;                        // 3. downcasting segur
        return edad == perro.edad                       // 4. comparar camps
                && Objects.equals(nombre, perro.nombre);
    }
}
```

El patró té 4 passos que són l'examen quasi literal de qualsevol entrevista:

1. **Reflexivitat ràpida:** si és el mateix objecte (`this == o`), són iguals.
2. **Filtri:** `null` o classe distinta → `false`. (`getClass()` evita comparar un `Perro` amb un `Gato`.)
3. **Downcasting** (segur, gràcies al pas 2).
4. **Comparar camp a camp**, usant `Objects.equals` per als `String` (que gestiona bé el `null`).

🏷️ hashCode(): el codi de barres

El `hashCode()` és un número que Java usa per a guardar objectes en col·leccions com `HashSet` o `HashMap`. La regla sagrada:

> ⚠️ **Advertència:** si dos objectes són iguals segons `equals()`, **HAN** de tenir el mateix `hashCode()`. Si sobreescrius `equals()` sense `hashCode()`, les col·leccions hash es comporten de forma imprevisible (objectes "iguals" que apareixen dues vegades).

La forma moderna i sense dolor:

```java
@Override
public int hashCode() {
    return Objects.hash(nombre, edad);
}
```

`Objects.hash` usa els mateixos camps que `equals()`. Regla pràctica: **mateixos camps en `equals` i en `hashCode`**, i vius en pau.

> 💡 **Detall pràctic:** la regla inversa NO és obligatòria: objectes amb el mateix `hashCode()` poden no ser iguals (col·lisions permeses). Només importa una direcció: `equals == true` → mateix hash.

🕵️ getClass(): el nom de l'objecte real

`getClass()` torna un objecte `Class` amb informació de la classe real de l'objecte (el que veus en `a.getClass().getSimpleName()` del punt 4). No es sobreescriu: ve de fàbrica i és perfecte tal qual.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** `toString()`, `equals()` i `hashCode()` són el trio que el teu IDE pot generar amb un clic (Generate → equals() and hashCode()). Però entén què generes abans de prémer.

Exercici: què imprimeix?

```java
Empleado e1 = new Programador("Ana", "001", 2500, "Java");
Empleado e2 = new Programador("Ana", "001", 2500, "Java");
Empleado e3 = new Gerente("Ana", "002", 3000, 500);

System.out.println(e1);
System.out.println(e1.equals(e2));
System.out.println(e1.equals(e3));
System.out.println(e1.hashCode() == e2.hashCode());
```

Sabent que `Programador` sobreescriu `toString()`, `equals()` (per id) i `hashCode()` correctament, i `Gerente` és una altra classe:

> 🔄 Solució

- `e1` imprimeix `Programador: Ana (ID: 001)` (gràcies a `toString()`).
- `e1.equals(e2)` → `true` (mateix id, encara que siguin objectes diferents).
- `e1.equals(e3)` → `false` (classes diferents: `getClass()` els separa).
- `e1.hashCode() == e2.hashCode()` → `true` (mateixos camps → mateix hash).

Exercici: el gat impostor

```
class Perro extends Animal { Perro(String n) { super(n); } }
class Gato extends Animal { Gato(String n) { super(n); } }

System.out.println(new Perro("Firulais").equals(new Gato("Firulais")));
```

> 🔄 Solució

`false`. Encara que tots dos es diguin "Firulais", `getClass()` torna classes diferents (`Perro` vs `Gato`), així que `equals()` falla en el pas 2. Un gat no és un gos, per molt que comparteixi nom.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. De quina classe hereta tot en Java?
2. Què imprimeix `System.out.println(objeto)` si sobreescrius `toString()`?
3. Què passa si sobreescrius `equals()` però no `hashCode()`?
4. Quants passos té el patró estàndard d'`equals()`?

> 🔄 Respostes

1. D'`Object`, encara que no escriguis `extends`.
2. El teu `toString()`: `println`, la concatenació amb + i molts altres llocs el criden sols.
3. Les col·leccions hash (`HashSet`, `HashMap`) es comporten mal: dos objectes "iguals" poden tractar-se com a diferents.
4. Quatre: `this == o`, filtri de `null`/classe, downcasting i comparar camps amb `Objects.equals`.

✅ Resum en 3 frases

1. **`Object`** és el besavi de totes les classes i aporta `toString()`, `equals()`, `hashCode()` i `getClass()`.
2. **`toString()`** et dona representacions llegibles; **`equals()`** compara per valor seguint el patró de 4 passos.
3. **`equals()`** i **`hashCode()`** van sempre junts, usant els mateixos camps, o les col·leccions es venjaran de tu.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `Object` | La classe arrel de tota la jerarquia Java |
> | `toString()` | Representació textual de l'objecte |
> | `equals(Object)` | Comparació per valor (no per referència) |
> | `hashCode()` | Codi hash usat per les col·leccions |
> | `getClass()` | Informació de la classe real de l'objecte |
> | Contracte equals/hashCode | Iguals segons `equals` ⇒ mateix `hashCode` |

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · instanceof i downcasting · **Següent:** 07 · Classes abstractes

07 — Classes abstractes

Esbossos que no s'instancien, mètodes abstractes que obliguen i el patró template method 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 07 · Classes abstractes

📬 La idea en una frase

> **Una classe abstracta és un esbós: no es pot instanciar, defineix mètodes obligatoris (abstractes) i comparteix codi concret amb les seves filles.**

Imagina “A la venda: esbós de cadira”. No pots seure't en un esbós, oi? Doncs això són les classes abstractes: plànols incomplets perquè *altres* els completen.

🎨 Què és una classe abstracta?

Una classe que **NO pot instanciar-se**. Pot tenir mètodes abstractes (sense implementació) i mètodes concrets (amb implementació):

```
public abstract class Animal {
    protected String nombre;

    public abstract void hacerSonido();   // sense claus: obliga les filles

    public void dormir() {                 // amb claus: codi compartit
        System.out.println(nombre + " está durmiendo... Zzz");
    }
}
```

> ⚠️ **Advertència:** `new Animal()` és un **error de compilació**. No pots crear un “animal genèric”; només pots crear animals concrets que completen l'esbós.

📋 You MUST: implementar els mètodes abstractes

Si una classe **concreta** estén una abstracta, està OBLIGADA a implementar tots els seus mètodes abstractes:

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Guau!");
    }
}

public abstract class Pajaro extends Animal {
    // No està obligada: Pajaro també és abstracta i pot deixar el mètode sense implementar
}
```

La regla és la llei dels contractes: **o implementes tot, o et declares abstracte tu també.** No hi ha terme mitjà.

abstract vs concrete

Classe abstracta	Classe concreta
No pots crear objectes directament	Pots crear objectes
Pot tenir mètodes abstractes	Tots implementats
Concepte general	Alguna cosa específica
`abstract class`	Només `class`

📐 Exemple: figures geomètriques

```java
public abstract class Figura {
    protected String color;

    public Figura(String color) {
        this.color = color;
    }

    public abstract double calcularArea();
    public abstract double calcularPerimetro();

    public void mostrarColor() {
        System.out.println("Color: " + color);
    }
}

public class Circulo extends Figura {
    private double radio;

    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override public double calcularArea() { return Math.PI * radio * radio; }
    @Override public double calcularPerimetro() { return 2 * Math.PI * radio; }
}

public class Rectangulo extends Figura {
    private double ancho, alto;

    public Rectangulo(String color, double ancho, double alto) {
        super(color);
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override public double calcularArea() { return ancho * alto; }
    @Override public double calcularPerimetro() { return 2 * (ancho + alto); }
}
```

`Figura` defineix **què** tenen totes les figures (color, àrea, perímetre) i delega **com** es calculen a cadascuna. El polimorfisme del punt 4 fa la resta: `ArrayList<Figura>` amb cercles i

rectangles, i `calcularArea()` funciona sol.

🏗️ Constructors en classes abstractes

Sí, les abstractes poden tenir constructors. Es criden amb `super()`, igual que en qualsevol herència:

```
public abstract class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
        System.out.println("Constructor de Animal");
    }
}

public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre);
    }
}
```

No pots instanciar `Animal`, però el seu constructor **sí s'executa** quan crees un `Perro` (recorda la cadena de constructors del punt 2). El constructor de l'abstracta inicialitza la part comuna.

🧩 Template Method: les abstractes en acció

Aquest és el patró que fa que les classes abstractes valguin el seu pes en or. Defineixes l'**esquelet d'un algoritme** i deixes que les subclasses omplin els detalls:

```
public abstract class Bebida {
      public final void preparar() {              // final: ningú pot canviar
l'esquelet
        hervirAgua();
        prepararIngrediente();
        servirEnTaza();
        anadirExtras();
    }

    private void hervirAgua() { System.out.println("Hirviendo agua..."); }
    private void servirEnTaza() { System.out.println("Sirviendo en taza..."); }

    protected abstract void prepararIngrediente();
    protected abstract void anadirExtras();
}

public class Te extends Bebida {
                  @Override    protected    void    prepararIngrediente()    {
System.out.println("Poniendo la bolsita de té..."); }
      @Override protected void anadirExtras() { System.out.println("Añadiendo
limón..."); }
}
```

La màgia: `Te` només implementa les dues parts variables, i `preparar()` (que és `final`) ja sap l'ordre complet. Les subclasses no poden desordenar l'algoritme, només personalitzar-lo. Aquell és el **template method pattern**.

> 💡 **Detall pràctic:** fixa't que els mètodes abstractes són `protected`: només la família ha d'implementar-los. L'esquelet `public final` és l'únic que el món exterior pot cridar.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** els mètodes abstractes són contractes: la subclasse HA d'implementar-los. Si no, no compila. Usa'ls per a "forçar" comportament.

Exercici: compila?

```
public abstract class A { public abstract void metodo(); }
public class B extends A { }
```

> 🔄 Solució

NO compila. `B` és concreta i ha d'implementar `metodo()`. Solució: `public class B extends A { public void metodo() { } }` o declarar `B` com `abstract`.

Exercici: quines línies donen error?

```
public abstract class A { public abstract void metodo(); }
public class B extends A { public void metodo() { } }

A a = new A();  // ¿error?
B b = new B();  // ¿error?
```

> 🔄 Solució

`new A()` → **ERROR**: no pots instanciar una classe abstracta. `new B()` → **OK**: `B` va implementar `metodo()` i és concreta.

Exercici: el gat persa

```
abstract class Animal { abstract void hablar(); }
abstract class Mamifero extends Animal { void hablar() { System.out.println("Mamífero raro"); } }
class Gato extends Mamifero { void hablar() { System.out.println("Miau"); } }
class GatoPersa extends Gato { }

public class Test {
    public static void main(String[] args) {
        Animal a = new GatoPersa();
        a.hablar();
    }
}
```

> 🔄 Solució

Imprimeix **"Miau"**. `GatoPersa` hereta el `hablar()` de `Gato` (que sobreescriu el de `Mamifero`), i `Animal a` resol en runtime l'objecte real: dynamic binding de principi a fi.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Pots fer `new` sobre una classe abstracta?
2. Què està obligada a fer una subclasse concreta?
3. Pot una classe abstracta tenir mètodes concrets i constructors?
4. Què garanteix el `final` en el mètode `preparar()` del template method?

> 🔄 **Respostes**
>
> 1. No: `new Animal()` és un error de compilació.
> 2. Implementar TOTS els mètodes abstractes de la superclasse (o declarar-se abstracta).
> 3. Sí: codi compartit i constructors que s'executen via `super()`.
> 4. Que cap subclasse pot reordenar o saltar-se passos de l'algoritme: només ompli les parts abstractes.

✅ Resum en 3 frases

1. Una **classe abstracta** és un esbós no instanciable amb mètodes abstractes (obligatoris) i concrets (compartits).
2. Les subclasses **concretes** han d'implementar tots els mètodes abstractes; les abstractes poden deixar el deute pendent.
3. El **template method** defineix l'esquelet `final` d'un algoritme i delega els detalls variables a les subclasses.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Classe abstracta	Classe que no pot instanciar-se
Mètode abstracte	Mètode sense implementació que obliga les filles
Subclasse concreta	Subclasse que implementa tots els abstractes
Template method	Esquelet d'algoritme amb parts que les filles omplen
`abstract`	Modificador que marca classes i mètodes sense acabar

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · La classe Object: el besavi de tot · **Següent:** 08 · Interfícies: el contracte que signes

08 — Interfícies: el contracte que signes

implements, polimorfisme amb interfícies, mètodes default i la gran decisió abstract vs interface 🧬

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 08 · Interfícies: el contracte que signes

---

📬 La idea en una frase

> **Una interfície és un contracte: declara quins mètodes ha de tenir una classe (amb `implements`), sense dir com implementar-los, i permet que una classe signe diversos contractes alhora.**

Has firmat un contracte? "El treballador es compromet a: programar en Java, no dormir-se en les reunions…" però no diu COM. Una **interfície** en Java és això: un contracte.

✍️ Declarant i implementant

```java
public interface Reproducible {
    void reproducir();
    void pausar();
    void detener();
    int obtenerDuracion();
}

public class Cancion implements Reproducible {
    private String titulo;

    public Cancion(String titulo) {
        this.titulo = titulo;
    }

    @Override public void reproducir() { System.out.println("🎵 Reproduciendo: "
+ titulo); }
    @Override public void pausar() { System.out.println("⏸ Canción pausada"); }
     @Override public void detener() { System.out.println("⏹ Canción detenida");
}
    @Override public int obtenerDuracion() { return 240; }
}
```

Qualsevol classe que signe el contracte (amb `implements`) **HA** d'implementar tots aquells mètodes. El com és cosa seva.

> 📝 **Nota:** els mètodes d'una interfície són `public abstract` implícitament (no cal escriure-ho). I els seus atributs són `public static final`: constants, no estat.

🎭 Polimorfisme amb interfícies

Una interfície també pot ser el tipus d'una referència polimòrfica:

```java
import java.util.ArrayList;
import java.util.List;

public class Reproductor {
    public static void main(String[] args) {
        List<Reproducible> lista = new ArrayList<>();
        lista.add(new Cancion("Bohemian Rhapsody"));
        lista.add(new Pelicula("Inception"));

        for (Reproducible r : lista) {
            r.reproducir(); // no sap si és cançó o pel·lícula
        }
    }
}
```

El `Reproductor` només coneix el contracte `Reproducible`. Tant li fa si demà arriba un `Podcast` o un `Audiolibro`: mentre signen el contracte, entren en la llista sense tocar una línia. És el polimorfisme del punt 4, però amb contractes en comptes d'herència.

🚣 Múltiples interfícies

Aquí està el gran avantatge sobre l'herència: una classe només estén **UNA** classe, però pot implementar **VARIES** interfícies:

```java
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }

public class Triatleta implements Nadador, Corredor {
    @Override public void nadar() { System.out.println("🚣 Nadando 1.5 km"); }
    @Override public void correr() { System.out.println("🏃 Corriendo 10 km"); }
}
```

Un `Triatleta` és nedador I corredor. Amb herència múltiple això seria el famós *problema del diamant*; amb interfícies, signar dos contractes no trenca res.

🩹 default Methods: pegats sense trencar res

Abans de Java 8, afegir un mètode a una interfície **trencava** totes les classes que la implementaven. Arribaren els mètodes `default`:

```
public interface Volable {
    void volar();

    default void despegar() {
        System.out.println("🛫 Despegando...");
    }
}

public class Avion implements Volable {
    @Override public void volar() { System.out.println("✈️ Volando a 900 km/h");
}
    // despegar() ja ve implementada: no estàs obligat a tocar-la
}
```

Un mètode `default` és una implementació "per defecte": la classe pot usar-la tal qual o sobreescriure-la. És una evolució del contracte sense trencar els que ja van signar.

I si dos interfícies tenen el mateix `default`?

Conflicte. La classe està obligada a sobreescriure i decidir:

```
interface Guerrero { default void atacar() { System.out.println("Ataca con
espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); }
}

class Personaje implements Guerrero, Mago {
    @Override
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}
```

Imprimeix: `Ataca con espada`, `Lanza hechizo`, `¡Y usa ambas!`. La sintaxi `Interfaz.super.metodo()` crida la implementació concreta de cada contracte.

⚖️ La gran decisió: abstract class vs interface

Aspecte	Classe abstracta	Interfície
Mètodes amb codi	Sí	Sí (default)
Atributs	Qualsevol	`public static final`
Herència múltiple	No (un `extends`)	Sí (varis `implements`)
Constructors	Sí	No
Estat (atributs d'instància)	Sí	No
Quan usar-la	Compartir codi + forçar mètodes en una família	Definir capacitats/contractes reutilitzables

> 💡 **Consell:** usa **classe abstracta** per a "és-un" amb codi compartit (una família que comparteix implementació). Usa **interfície** per a "pot-fer" (capacitats que no depenen de la jerarquia: `Pato implements Volable, Cantable`). Si pots, prefereix la interfície: és més flexible.

🥊 EL RING: extends vs implements

Dos paraules clau discuteixen sobre qui és més important.

extends: «Jo soc l'herència pura. Codi reutilitzat, una jerarquia clara. `Perro extends Animal`, `Coche extends Vehiculo`. ¡Soc la base de la POO!»

implements: «Sí, però amb mi no hi ha límits. Una classe implementa diverses interfícies. Amb `extends` només tens un pare. Jo et permeti ser varies coses alhora: `Serializable`, `Comparable`, `Cloneable`…»

extends: «Les meves classes poden tenir codi ja fet. Tu només declares mètodes buits. ¡Jo aporte implementació!»

implements: «Des de Java 8 tinc mètodes `default` i `static`. Mira: `default void log()` ja funciona. A més, soc més flexible: no imposo una jerarquia rígida.»

extends: «Val, però sense mi les interfícies no tindrien sentit. Una interfície no pot instanciar-se sola.»

implements: «I sense mi tindries herència múltiple, que és un caos. Mira el problema del diamant en C++.»

extends: «Ens necessitem.»

implements: «Sí. `extends` per a la jerarquia, `implements` per als contractes.»

> 🕶️ **Don Tip:** es poden combinar: `class Perro extends Animal implements Mascota, Jugable`. Primer el `extends` (un pare), després els `implements` (varis contractes).

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** `implements` és un contracte: la classe es compromet a tenir TOTS els mètodes de la interfície. Res de "quasi tots".

Exercici: compila?

```
public interface Cantante { void cantar(); }
public interface Bailarin { void bailar(); }

public class Artista implements Cantante, Bailarin {
    @Override public void cantar() { System.out.println("canta"); }
    // ¡FALTA bailar()!
}
```

> › 🔄 **Solució**
>
> **NO compila.** En implementar `Cantante` i `Bailarin`, `Artista` ha d'implementar `cantar()` I `bailar()`. Signar un contracte no admet clàusules "a mitges".

Exercici: què imprimeix?

```
interface Guerrero { default void atacar() { System.out.println("Ataca con espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); } }

class Personaje implements Guerrero, Mago {
    @Override
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}

public class Test {
    public static void main(String[] args) {
        new Personaje().atacar();
    }
}
```

> 🔄 **Solució**

```
Ataca con espada
Lanza hechizo
¡Y usa ambas!
```

El conflicte de `default` es resol sobreescrivint i cridant a cada `Interfaz.super.metodo()`. Sense aquell override, `Personaje` no compilaria.

Exercici: la classe que implementa dos contractes

```
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }

public class Triatleta implements Nadador, Corredor {
    @Override public void nadar() { System.out.println("🏊 Nadando"); }
    @Override public void correr() { System.out.println("🏃 Corriendo"); }
}
```

Pots usar un `Triatleta` on s'espera un `Nadador`? I on s'espera un `Object`?

> 🔄 **Solució**
>
> Sí a les dues. `Triatleta` IS-A `Nadador`, IS-A `Corredor` i IS-A `Object`. Les referències `Nadador n = new Triatleta()`, `Corredor c = new Triatleta()` i `Object o = new Triatleta()` són totes vàlides. Cada referència només exposa el seu propi contracte.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina paraula usa una classe per a signar un contracte?
2. Quantes classes pot estendre i quantes interfícies pot implementar una classe?
3. Què són els mètodes `default` i per a què serveixen?
4. Quan tries una interfície en comptes d'una classe abstracta?

> 🔄 **Respostes**
>
> 1. `implements`: `class Cancion implements Reproducible`.
> 2. Una sola classe (extends), però diverses interfícies (implements).
> 3. Mètodes amb implementació per defecte en una interfície: permeten evolucionar el contracte sense trencar les implementacions existents.
> 4. Quan defineixes una capacitat ("pot-fer") reutilitzable, no una jerarquia amb codi compartit. I si necessites herència múltiple de contractes, la interfície és l'única via.

✅ Resum en 3 frases

1. Una **interfície** és un contracte: la classe que la implementa ha d'aportar tots els seus mètodes.
2. Les interfícies permeten **polimorfisme** i **múltiples contractes**, amb mètodes `default` per a evolucionar sense trencar res.
3. Regla pràctica: **classe abstracta** per a famílies amb codi compartit, **interfície** per a capacitats flexibles.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Interfície	Contracte de mètodes que la classe ha d'implementar
`implements`	La paraula que signa el contracte
Mètode `default`	Implementació per defecte que la classe pot heretar
Contracte múltiple	Una classe pot implementar diverses interfícies
`Interfaz.super.metodo()`	Com resoldre conflictes entre `default`

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Classes abstractes · **Següent:** 09 · Repàs interactiu

09 — Repàs interactiu: domina l'herència

El tancament pràctic de la unitat, amb polimorfisme, contractes i una herència que no va a funcionar 😈

> 🗺️ **Estàs en: 🧬 U11 · Herència, polimorfisme i interfícies** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no va a funcionar. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}

public class Perro extends Animal {
    public void hacerSonido() { System.out.println("¡Guau!"); }
    public void moverCola() { System.out.println("*mueve la cola*"); }
}

public class Gato extends Animal {
    public void hacerSonido() { System.out.println("¡Miau!"); }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Perro();
        Animal a2 = new Gato();
        a1.hacerSonido();
        a2.hacerSonido();
        // a1.moverCola();  <- ¿compila?
    }
}
```

Què imprimeixes per pantalla? Tria assenyadament:

1. `...` i `...` → La variable és `Animal`, així que s'executa el mètode d'`Animal`. ❌
2. **`¡Guau!` i `¡Miau!`** → ✅ ¡Correcte! El polimorfisme executa el mètode de l'objecte real, no el de la referència.
3. **`¡Guau!` i un error** → `a2` no pot ser un `Gato`. ❌

> 🔄 **Solució**
>
> L'opció **2**. `a1.hacerSonido()` executa la versió de `Perro`; `a2.hacerSonido()`, la de `Gato`. La línia comentada `a1.moverCola()` **no compila**: la referència és `Animal` i `Animal` no té `moverCola()`. Compilador i JVM, cadascú al seu.

🔥 Fireside Chat: Classe abstracta vs Interfície

> *Dos veterans de la POO discuteixen al costat de la pissarra mentre una subclasse els mira.*

Classe abstracta: — Jo soc l'esbós. Defineixo l'esquelet i comparteixo codi: els meus fills s'estalvien escriure l'avorrit. `Figura` amb `calcularArea()` abstracte i `mostrarColor()` ja fet. Soc la família que hereta.

Interfície: — La família hereta d'un de sol. Jo signe contractes a mansalva. Un `Pato` implementa `Volable` i `Nadador` alhora. Quants `extends` li deixes tu al pobre ànec?

Classe abstracta: — Però jo tinc estat. Atributs `protected`, constructors, mètodes concrets amb lògica de veritat. Tu només declares "això ha d'existir".

Interfície: — Des de Java 8 tinc mètodes `default` amb implementació. I soc la base del polimorfisme modern: `List<Reproducible>`, `Comparable`, `Runnable`… Tota l'API de Java està plena de mi.

Classe abstracta: — Sense mi, no tindries famílies amb codi compartit. El template method, qui el fa?

Interfície: — I sense mi tindries el problema del diamant. Ens complementem.

Classe abstracta: — Això és veritat. Jo per a les famílies.

Interfície: — Jo per a les capacitats. ¿Acord?

> La lliçó: usa **classe abstracta** quan hi hagi una família amb codi compartit (és-un); usa **interfície** quan vulguis capacitats reutilitzables (pot-fer). I es poden combinar: `class Perro extends Animal implements Mascota, Jugable`.

🕵️ Qui soc?

Endevina quin concepte de la unitat soc:

1. **Soc la paraula que fa que una classe filla obtinga tot el del seu pare.**
2. **Soc l'anotació que li diu al compilador "verifica que estic sobreescrivint, no inventant".**
3. **Soc la crida que configura la part del pare abans que el fill faci el seu.**
4. **Soc el mecanisme pel qual `Animal a = new Perro()` executa el so del gos.**
5. **Soc l'operador que pregunta "ets realment un Perro?" abans de convertir.**
6. **Soc el contracte que una classe signa amb `implements` i que admet diversos alhora.**
7. **Soc la classe abstracta que defineix l'esquelet d'un algoritme i deixa que les filles omplin els detalls.**

> 🔄 **Respostes**
>
> 1. **`extends`** — estableix l'herència.
> 2. **`@Override`** — verificació en compilació de la sobrescriptura.
> 3. **`super(...)`** — el constructor del pare s'executa primer, sempre.
> 4. **El polimorfisme (dynamic binding)** — la JVM decideix en runtime.
> 5. **`instanceof`** — i després el downcasting segur.
> 6. **La interfície** — diversos `implements`, un sol `extends`.
> 7. **El template method** — el patró de les classes abstractes en acció.

⚖️ Vertader o fals

1. **"Si `Animal` té un mètode `public`, una subclasse pot fer-lo `private`."**
2. **"`super.metodo()` es pot cridar des de qualsevol línia del mètode."**
3. **"Una classe pot implementar tres interfícies i estendre una classe."**
4. **"Els atributs també són polimòrfics: `ref.x` usa el de la classe real de l'objecte."**
5. **"Sobreescriure `equals()` sense `hashCode()` és un error de compilació."**

> 🔄 Solucions
>
> 1. **Fals.** No pots reduir la visibilitat en sobreescriure: `public` no pot tornar-se `protected` ni `private`.
> 2. **Fals.** `super(...)` (constructor) ha de ser la primera línia. `super.metodo()` pot anar on vulguis, però l'afirmació mescla tots dos.
> 3. **Vertader.** Un `extends` + diversos `implements`: `class Perro extends Animal implements Mascota, Jugable`.
> 4. **Fals.** Els mètodes són polimòrfics; els atributs es resolen pel tipus de la referència (shadowing).
> 5. **Fals.** Compila perfectament. El problema és que les col·leccions hash (`HashSet`, `HashMap`) es comporten mal en runtime.

⚡ Laboratori de tortura: l'herència que plora

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte quan l'arregles.

```
public class Vehiculo
    protected int velocidad;

    public Vehiculo(int velocidad) {
        this.velocidad = velocidad;
    }

    void mover() { System.out.println("Vehículo a " + velocidad + " km/h"); }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(int velocidad, int puertas) {
        this.velocidad = puertas;
        this.puertas = velocidad;
    }

    @Override
    void mover() {
        super.mover();
        System.out.println("Coche con " + puertas + " puertas");
    }

    public static void main(String[] args) {
        Vehiculo v = new Coche(120, 5);
        v.mover()
    }
}
```

Fall intencionat: un dels errors sembla correcte a simple vista perquè “es veu bé”, però canvia per complet el comportament del programa.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun ; o { que falti? *no → segueix buscant.*

   > **I si continuo encallat?**
   >
   > La classe `Vehiculo` necessita la seva clau d'apertura `{` just després del nom.

2. Compila ja? *no → mira el missatge d'error i els constructors.*

   > **I si continuo encallat?**
   > `Vehiculo` només té constructor amb paràmetres: `Coche` ha de cridar-lo amb `super(velocidad)` com a primera línia. Sense això, el compilador busca un `super()` buit que no existeix.

3. Executa però els números surten canviats? *És l'error de lògica: el constructor intercanvia els valors.*

   > **Solució final**

Els **3 errors de compilació**:

1. Falta la `{` d'apertura de la classe `Vehiculo`.
2. El constructor de `Coche` no crida a `super(velocidad)`. En escriure qualsevol constructor en el pare, el buit desapareix.
3. Falta el `;` al final de `v.mover()`.

L'**error de lògica**: en el constructor, les assignacions estan creuades:

```
this.velocidad = puertas;   // ¡mal! velocidad hauria de ser 120
this.puertas = velocidad;   // ¡mal! puertas hauria de ser 5
```

Compila i executa perfectament, però intercanvia els valors. Codi corregit:

```java
public class Vehiculo {
    protected int velocidad;

    public Vehiculo(int velocidad) {
        this.velocidad = velocidad;
    }

    void mover() { System.out.println("Vehículo a " + velocidad + " km/h"); }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(int velocidad, int puertas) {
        super(velocidad);
        this.puertas = puertas;
    }

    @Override
    void mover() {
        super.mover();
        System.out.println("Coche con " + puertas + " puertas");
    }

    public static void main(String[] args) {
        Vehiculo v = new Coche(120, 5);
        v.mover();
    }
}
```

Sortida correcta:

```
Vehículo a 120 km/h
Coche con 5 puertas
```

Amb la versió trencada, una vegada arreglats els altres errors, la sortida era `Vehículo a 5 km/h` i `Coche con 120 puertas`: dues línies canviades per un constructor creuat.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-lo
🏅 **L'Hereu**	Explicar què s'hereda i què no (private, constructors) sense mirar la taula
🏅 **El Sobreescriptor**	Usar `@Override` + `super` per a estendre un mètode del pare amb sentit
🏅 **El Camaleó**	Resoldre un problema de col·leccions amb polimorfisme sense usar `instanceof`
🏅 **El Notari**	Decidir quan usar classe abstracta i interfície i justificar-ho
🏅 **El Genealogista**	Dissenyar una jerarquia de 3 nivells amb `toString()`, `equals()` i `hashCode()`

🧠 Atreveix-te a pensar

1. **Sense executar:** què imprimeix aquest programa?

```
class A { void mensaje() { System.out.println("A"); } }
class B extends A { void mensaje() { System.out.println("B"); } }
class C extends B { }

public class Test {
    public static void main(String[] args) {
        A r1 = new B();
        A r2 = new C();
        B r3 = new C();
        r1.mensaje();
        r2.mensaje();
        r3.mensaje();
    }
}
```

2. **El constructor perdut:** `Animal` només té `Animal(String nombre)`. Què li passa a `Perro extends Animal` si el seu constructor no crida a `super`?
3. **El detectiu de l'equals:** el teu `HashSet` conté dos objectes que són `equals()` entre si… què has oblidat sobreescriure?

4. **Vertader o fals:** “`a instanceof Perro` és `true` si `a` és de tipus `Animal` i l’objecte real és un `Perro`.”

> 💡 **Solucions**
>
> 1. **B, C, C**. Només importa el tipus real de l’objecte (`B`, `C`, `C`); la referència (`A`, `A`, `B`) no decideix res.
> 2. **Error de compilació**: Java intenta `super()` sense arguments i no existeix. Cal cridar a `super(nombre)` com a primera línia.
> 3. **`hashCode()`**. Si `equals()` diu que són iguals però `hashCode()` difereix, el `HashSet` els fica en calaixos diferents.
> 4. **Vertader.** `instanceof` mira el tipus real de l’objecte: `Perro` és un `Animal`, i l’objecte real és un `Perro`, així que és `true`.

🧩 Crucigrama de bits

```
Horitzontal:
1. La paraula que establix l'herència (7 lletres)
3. L'anotació que verifica que sobreescrius (8 lletres)
5. El contracte que signes amb implements (10 lletres)
6. Excepció en convertir un Gat en Gos (18 lletres)

Vertical:
2. La classe de la qual tots hereden (6 lletres)
4. La paraula que crida el mètode del pare (5 lletres)
```

> 📝 **Solucions**
>
> **Horitzontal:** 1. EXTENDS · 3. OVERRIDE · 5. INTERFACE · 6. CLASSCASTEXCEPTION **Vertical:** 2. OBJECT · 4. SUPER

💬 Preguntes d’entrevista de treball

> Preguntes reals que et faríen per a programador Java junior.

1. **“Explica-m’ho, com si jo fos la teva àvia, la diferència entre herència i composició.”**
2. **“Què fa `super()` en un constructor i per què ha de ser la primera línia?”**

3. **“Quan usaries una classe abstracta i quan una interfície?”**
4. **“Per què `ArrayList<Animal>` pot guardar `Perro` i `Gato` però no al revés?”**
5. **“Quina excepció llanços en fer `(Perro) gato` i com l’evites?”**
6. **“Per què `equals()` i `hashCode()` van sempre junts?”**

🤷 No hi ha preguntes tontes

> ❓ **Puc heredar de diverses classes alhora?**

No. Java no permet herència múltiple (el *problema del diamant*). Però per això existeixen les interfícies: pots implementar tantes com vulguis.

> ❓ **Què passa si una classe filla té un atribut amb el mateix nom que el pare?**

Es diu que el **oculta** (shadowing). Però compte: els atributs no són polimòrfics. Si uses una referència de la superclasse, veus el de la superclasse; si uses una de la subclasse, veus el de la subclasse. Millor no jugar a això: posa noms diferents.

> ❓ **I els mètodes `final`? No es poden sobreescriure?**

Exacte. Un mètode `final` queda congelat: les filles l’hereden però no poden canviar-lo. I una classe `final` (com `String`) no pot tenir filles. És la decisió de disseny “això no es toca”.

🎬 Poscrèdits

La programadora acaba el seu simulador de zoo virtual: una classe abstracta `Animal`, un `Perro` i un `Gato` que l’implementen, i un `ArrayList<Animal>` que els fica tots en la mateixa gàbia. Fins que, en l’última línia, intenta convertir un gat en gos i tot explota amb una `ClassCastException`.

S’acosta CONRAD, el compilador cascarràbies, amb la seva tassa fumant.

CONRAD: — Una altra vegada. Un gat disfressat de gos. Saps què t’ha faltat?

Programadora: — ¿L’`instanceof`?

CONRAD: — *assenteix* Veus. Quan entens qui és realment cada objecte, els errors deixen de ser màgia. Herència, `super`, polimorfisme, contractes… Ja no escrius classes, dissenyes famílies i signes acords.

Programadora: — ¿I ara què toca?

CONRAD: — *fa un glop* Ara que tens jerarquies senceres, necessites guardar-les en munts. I no, no val una caixa per a cada cosa. Et presentaré unes estructures que guarden animals, figures i fins i tot contractes amb elegància.

PRÒXIMAMENT EN U12: Col·leccions. El moment en què les teves jerarquies aprenen a guardar-se en llistes, conjunts i mapes. 📚

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Interfícies: el contracte que signes · **Següent:** U12 · Col·leccions

U12 — Col·leccions, genèrics i mapes

L'aparcament que creix, el porter sense duplicats, el `` que ho canvia tot i la guia telefònica: ArrayList, Set, genèrics i HashMap en una sola unitat 📚

En la U06 vas veure els arrays: grandària fixa, accés per índex i prou treball manual per a créixer o encongir-se. Avui això s'acaba. Les **col·leccions** de Java són les estructures de dades amb superpoders: creixen soles, s'encongeixen soles, rebutgen duplicats si vols, s'ordenen automàticament i es recorren sense que t'importi com estan construïdes per dins.

I aquesta unitat va més enllà: després de dominar les col·leccions, llevarem el capó del `<String>` que portes setmanes escrivint. Els **genèrics** expliquen per què el compilador es fia de les teves llistes, i els **mapes** (`HashMap`, `TreeMap`) et donen l'estructura que més s'usa en Java real: associar una clau amb un valor.

Aquesta unitat té quatre grans actes:

- **Llistes (`List`):** `ArrayList`, l'aparcament que creix sol, i `LinkedList`, la conga que encadena els seus elements i brilla en cues i piles.
- **Conjunts (`Set`):** `HashSet`, el porter que no deixa entrar duplicats; `TreeSet`, el que els ordena sols; i `LinkedHashSet`, el que recorda l'ordre d'arribada.
- **Genèrics:** d'on surt el `<T>`, la teva pròpia classe genèrica, el diamant `<>`, el type erasure i el principi PECS.
- **Mapes (`Map`):** `HashMap`, la guia telefònica de Java; `TreeMap`, el que ordena les seves claus; i la guia definitiva per a triar estructura.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Usar **`ArrayList`** per a guardar llistes dinàmiques i distingir **`remove(índex)`** de **`remove(objecte)`**.
- Aplicar **`LinkedList`** per a cues (FIFO) i piles (LIFO).

- Eliminar duplicats amb **`HashSet`** i entendre per què `hashCode()` i `equals()` van sempre junts.
- Ordenar i mantenir l'ordre amb **`TreeSet`** i **`LinkedHashSet`**.
- Recórrer qualsevol col·lecció amb **`Iterator`** i esborrar durant el recorregut sense excepció.
- Manipular llistes amb la classe **`Collections`** (`sort`, `reverse`, `shuffle`, `max`, `min`, `frequency`).
- Crear les teves **classes i mètodes genèrics** amb `<T>`, límits i wildcards.
- Guardar i buscar parelles clau → valor amb **`HashMap`** i **`TreeMap`**.
- **Triar l'estructura adequada** (List, Set o Map) per a cada problema.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · ArrayList i el Java Collections Framework	L'aparcament que creix sol, la família `java.util` i per què no guarda `int` a palpes	Tots
02 · LinkedList: la conga line	La llista enllaçada, cues FIFO, piles LIFO i els seus mètodes d'extrems	Tots
03 · HashSet, TreeSet i LinkedHashSet: els conjunts	El porter sense duplicats, el que ordena sol i el que recorda l'ordre d'arribada	Tots
04 · Iterator i Collections	Esborrar mentre recorres i la navalla suïssa de les col·leccions	Tots
05 · Classes genèriques: la teva `Caja<T>`	D'on surt el `<String>`, convencions T/E/K/V i la teva primera classe genèrica	Tots
06 · El diamant, mètodes genèrics i wildcards	Type erasure, inferència, límits i el principi PECS	Tots
07 · HashMap: la guia telefònica	Clau → valor, `get`, `put`, `containsKey` i recórrer mapes	Tots
08 · TreeMap i com triar estructura	Mapes ordenats, `LinkedHashMap` i Map vs List vs Set	Tots

Punt	Què aprendràs	Dificultat
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA6)

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA6 b)	S'han reconegut les llibreries de classes relacionades amb tipus avançats de dades.	✅ Punts 1, 7 i 8
RA6 c)	S'han utilitzat llistes per a emmagatzemar i processar informació.	✅ Punts 1, 2 i 9
RA6 d)	S'han utilitzat iteradors per a recórrer els elements de les llistes.	✅ Punts 4 i 9
RA6 e)	S'han reconegut les característiques i avantatges de cadascuna de les col·leccions de dades disponibles.	✅ Punts 3, 7 i 8
RA6 f)	S'han utilitzat tipus enumeradors, estructures dinàmiques de dades i classes genèriques.	✅ Punts 5, 6, 7 i 8

> 📌 La **RA6** es completa amb la **U06** (arrays, CE a) i la **U14** (regex i XML, CEs g, h i i). Aquesta unitat cobreix col·leccions, genèrics i mapes: la llibreria `java.util` sencera.

🚪 Per on començo?

- Vens de la U11 i vols l'essencial? → Comença en el punt 1 i el punt 2: les llistes de tots els dies.
- Ja uses llistes i vols conjunts? → Ves directe al punt 3: duplicats fora.
- Et pica la curiositat del <T>? → Salta al punt 5 i al punt 6: genèrics de cap a cap.
- Vols mapes? → Punt 7 i punt 8: la guia telefònica i el mapa de decisió.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · ArrayList i el Java Collections Framework
⏭️ **En acabar la unitat, continua en U13 · Programació funcional (Streams i Lambdas).**

01 · ArrayList i el Java Collections Framework

L'aparcament que creix sol: ArrayList, la família java.util i per què no pot guardar int a palpes 🅿️📈

> 🗺️ **Estàs en: 📚 U12 · Col·leccions** → 01 · ArrayList i el Java Collections Framework

📬 La idea en una frase

> **ArrayList és un array amb superpoders: creix sol, s'encongeix sol i porta mètodes llestos (`add`, `get`, `remove`, `contains`…) en comptes d'obligar-te a escriure cada bucle a mà.**

En la U06 vas veure els arrays: grandària fixa, accés per índex i molt treball manual. Va arribar el moment que estaven esperant: l'aparcament s'ha quedat petit i necessites alguna cosa que creixi. Aquí comença la unitat de les col·leccions i, probablement, l'eina que més usaràs en la teva vida com a programador o programadora.

🅿️ El problema: el teu array s'ha quedat sense places

Has creat un array de 10 places. Han arribat 11 gats. Què fas?

```
String[] gatos = new String[10]; // 10 places, 11 gats... mal assumpte
```

Amb un array clàssic hauríes de crear-ne un de nou, copiar-ho tot i afegir el que falta. És com aparcar al carrer perquè l'aparcament està ple:

```
String[] gatosMasGrande = new String[gatos.length + 1];
System.arraycopy(gatos, 0, gatosMasGrande, 0, gatos.length);
gatosMasGrande[gatosMasGrande.length - 1] = "Bigotes Jr.";
gatos = gatosMasGrande; // Ara apunta al nou array
```

Funciona, però és tediós. I si a més has d'esborrar un element del mig, és pitjor. Necessites alguna cosa que creixi i s'encongeixi sola.

🎓 El Java Collections Framework (JCF)

Per a això existeix el **JCF**: una família de classes i interfícies en `java.util` que gestionen grups d'objectes com si fossin de goma. L'arbre de la família comença així:

```
Collection (interfície)
├── List        → ArrayList, LinkedList (ordenats, permeten duplicats)
├── Set         → HashSet, TreeSet (sense duplicats)
└── Queue       → cues (FIFO)
```

Totes comparteixen mètodes bàsics: `add`, `remove`, `contains`, `size`, `isEmpty`, `clear`. Aprendre un és aprendre l'esquelet de tots. La interfície `List` és la que més usaràs.

> 💡 **Consell:** declara les teves variables amb la interfície, no amb la classe: `List<String> gatos = new ArrayList<>();`. Així, si demà necessites una `LinkedList`, canvies una línia. Ho veuràs al punt 7.

🅿️📈 ArrayList: l'aparcament que creix sol

ArrayList és un array "de goma": quan s'ompli, en crea un de més gran per dins i copia les dades. Tu no te n'assabentes de res.

```java
import java.util.ArrayList;

public class EjemploArrayList {
    public static void main(String[] args) {
        ArrayList<String> gatos = new ArrayList<>();
        gatos.add("Bigotes");                    // [Bigotes]
        gatos.add("Garfield");                   // [Bigotes, Garfield]
        gatos.add("Misifú");                     // [Bigotes, Garfield, Misifú]
         gatos.remove(1);                           // [Bigotes, Misifú] - adéu,
Garfield
        String primero = gatos.get(0);           // "Bigotes"
        int cuantos = gatos.size();              // 2
        boolean hay = gatos.contains("Misifú");  // true
        int pos = gatos.indexOf("Bigotes");      // 0
        System.out.println(gatos);
    }
}
```

El repertori de mètodes imprescindibles:

Mètode	Què fa
`add(e)`	Afegeix al final
`add(i, e)`	Insereix a la posició `i` (desplaça la resta)
`get(i)`	Torna l'element de la posició `i`
`set(i, e)`	Substituïx l'element de la posició `i`
`remove(i)`	Esborra per índex
`remove(objeto)`	Esborra la primera aparició d'aquell objecte
`size()`	Nombre d'elements (NO `length`)
`contains(e)`	Hi és?
`indexOf(e)`	Posició de la primera aparició
`isEmpty()`	Està buida?
`clear()`	Ho esborra tot

> ⚠️ **Advertència:** ArrayList usa `size()`, no `length`. Array usa `length`, no `size()`. String usa `length()`, no `length` ni `size()`. Cadascun té la seva forma de preguntar com de gran és. És una trampa en el 90% dels exàmens.

Recórrer un ArrayList

Amb for-each (el preferit per a llegir):

```
for (String gato : gatos) {
    System.out.println("Miau: " + gato);
}
```

O amb for clàssic, que també funciona perquè `List` té índexs:

```java
for (int i = 0; i < gatos.size(); i++) {
    System.out.println(i + ": " + gatos.get(i));
}
```

Fixa't: `gatos.size()` dins de la condició s'avalua en cada vegada. Funciona, encara que per rendiment se sol guardar en una variable si no canvia.

🚫 ArrayList NO guarda primitius

No pots fer `ArrayList<int>`. Els genèrics (`<...>`) només funcionen amb objectes. Per a números uses les classes **wrapper**:

```java
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(42);              // autoboxing: int → Integer
int n = numeros.get(0);       // unboxing: Integer → int
```

Des de Java 5, l'**autoboxing/unboxing** és automàtic: fiques un `int` i Java l'embolica en un `Integer` sense que tu ho notes. Però per dins segueix havent-hi objectes.

> 📝 **Nota:** el mateix amb la resta de primitius: `boolean` → `Boolean`, `double` → `Double`, `char` → `Character`… Si necessites saber més sobre genèrics i wrappers, els punts 4 i 5 d'aquesta mateixa unitat els estiren a fons.

El truc de remove amb números

`remove(1)` i `remove(Integer.valueOf(1))` NO fan el mateix:

```java
ArrayList<Integer> nums = new ArrayList<>();
nums.add(10);
nums.add(20);
nums.add(30);
nums.remove(1);               // esborra per ÍNDEX: se'n va el 20
// nums.remove(Integer.valueOf(20)); // això esborra l'OBJECTE 20
```

Si vols esborrar el número 20, has de passar-li el wrapper. Si li passes un `int`, ho interpreta com a índex. És l'error clàssic dels novells amb `ArrayList<Integer>`.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** ArrayList usa arrays per dins. Quan s'ompli, en crea un de nou i copia. Cada operació té el seu cost, però tu no has de pensar-hi.

Exercici: l'ArrayList misteriós

```
import java.util.ArrayList;

public class BeTheList {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);
        lista.remove(Integer.valueOf(20));

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

Què imprimeix?

- (A) 10 20 30
- (B) 10 15 30
- (C) 10 15 20 30
- (D) 10 15

> 🔄 **Solució**
>
> La **B**. S'afegeix 15 a l'índex 1 → `{10, 15, 20, 30}`. Després s'esborra l'**objecte** `Integer(20)` (no l'índex) → `{10, 15, 30}`. El for-each imprimeix `10 15 30`. Si haguessin estat `lista.remove(2)`, el resultat hauria estat el mateix per casualitat, però per un motiu diferent.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode usa ArrayList per a saber la seva grandària: `length`, `length()` o `size()`?
2. Per què no es pot fer `ArrayList<int>`?
3. Què esborra `lista.remove(2)` en un `ArrayList<Integer>`: l'índex 2 o el número 2?
4. Per a què serveix declarar `List<String> x = new ArrayList<>()` en comptes de `ArrayList<String> x`?

> 🔄 **Respostes**
>
> 1. `size()`. És un mètode, amb parèntesis. El `length` sense parèntesis és dels arrays.
> 2. Perquè els genèrics només accepten objectes, i `int` és un primitiu. Usa `Integer` i deixa que l'autoboxing faci la seva màgia.
> 3. L'índex 2 (esborra el tercer element). Per a esborrar el número 2, usa `remove(Integer.valueOf(2))`.
> 4. Per a poder canviar la implementació (a `LinkedList`, per exemple) sense tocar la resta del codi. Es programa contra la interfície.

✅ Resum en 3 frases

1. **ArrayList** és una llista dinàmica: creix sol, s'encongeix sol i porta mètodes (`add`, `get`, `remove`, `contains`, `size`) per a no reinventar la roda.
2. Pertany al **Java Collections Framework** (`java.util`), una família de llistes, conjunts i cues amb mètodes comuns.
3. No guarda **primitius**: usa les classes wrapper (`Integer`, `Double`…) amb autoboxing automàtic, i compte amb la diferència entre `remove(índex)` i `remove(objecte)`.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
JCF	La família de col·leccions de `java.util`
`List`	Col·lecció ordenada amb índexs i duplicats permesos
Wrapper	Classe que embolica un primitiu (`Integer` embolica `int`)
Autoboxing	Passar de primitiu a wrapper automàticament

Terme	Idea general
Unboxing	Passar de wrapper a primitiu automàticament

📚 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · LinkedList: la conga line

02 · LinkedList: la conga line

La llista que encadena els seus elements com una conga: rapidíssima al principi i al final, i perfecta per a cues i piles 🎉

> 🗺️ **Estàs en:** 📚 **U12 · Col·leccions** → 02 · LinkedList: la conga line

---

📬 La idea en una frase

> **LinkedList encadena els seus elements com una conga: cadascun sap qui va davant i qui darrere, i és rapidíssima per a afegir o esborrar al principi i al final.**

Al punt 1 vas veure `ArrayList`, la llista de tots els dies. Ara toca conèixer la seva cosina, que resol un problema diferent: ficar i treure gent pels extrems sense que res es descoloque. És la col·lecció perfecta per a cues i piles.

🎉 LinkedList: la conga line

LinkedList és una **llista enllaçada**: cada element sap qui va davant i qui darrere, com en una conga. Per a recórrer-la per índex cal saltar d'element en element (és més lenta que `ArrayList` en això), però és rapidíssima per a afegir o esborrar **al principi** i al final.

```
import java.util.LinkedList;

public class EjemploLinkedList {
    public static void main(String[] args) {
        LinkedList<String> cola = new LinkedList<>();
        cola.addLast("Persona 1");   // al final
        cola.addLast("Persona 2");
        cola.addFirst("Colado");     // es cola al principi
        String primero = cola.removeFirst(); // "Colado" - se'n va
        System.out.println(cola);    // [Persona 1, Persona 2]
    }
}
```

Els seus mètodes propis (encara que `add`, `remove`, `get` també funcionen):

Mètode	Què fa
`addFirst(e)` / `addLast(e)`	Afegeix al principi / al final
`removeFirst()` / `removeLast()`	Treu i torna el primer / l'últim
`getFirst()` / `getLast()`	Mira el primer / l'últim sense treure'l
`peek()`	Mira el primer sense treure'l (no llança si està buida)
`poll()`	Treu el primer (no llança si està buida)

> 💡 **Consell:** usa `LinkedList` quan necessites una **cua** (FIFO: el primer que arriba és el primer que surt) o una **pila** (LIFO), o quan vagis a afegir/esborrar molt al principi. Per a accés aleatori freqüent (`get(i)`), `ArrayList` és millor: va directe a l'índex.

🛒 Cues i piles: les dues disfresses de LinkedList

LinkedList és camaleònica: segons quins mètodes uses, es comporta com una cua o com una pila.

Cua (FIFO): el primer que arriba és el primer que surt. Com la fila del supermercat.

```
LinkedList<String> cola = new LinkedList<>();
cola.addLast("Ana");     // Ana es posa al final
cola.addLast("Bob");     // Bob darrere d'Ana
cola.addLast("Carla");   // Carla darrere de Bob
String atendido = cola.removeFirst(); // "Ana" - la primera que va arribar
```

Pila (LIFO): l'últim que entra és el primer que surt. Com la pila de plats: el de dalt de tot és el que agafes.

```
LinkedList<String> pila = new LinkedList<>();
pila.addFirst("plat 1");  // base
pila.addFirst("plat 2");
pila.addFirst("plat 3");  // l'últim apilat
String quito = pila.removeFirst(); // "plat 3" - l'últim que va entrar
```

> ⚠️ **Advertència:** quan la cua està buida, `removeFirst()` i `getFirst()` llancen una `NoSuchElementException`. Pregunta abans amb `isEmpty()`, o usa `peek()` i `poll()`, que tornen `null` en comptes d'explotar.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `addLast` + `removeFirst` = cua FIFO. `addFirst` + `removeFirst` = pila LIFO. Triar els mètodes, no la classe.

Exercici: el que es cola a la conga

```
import java.util.LinkedList;

public class BeTheConga {
    public static void main(String[] args) {
        LinkedList<String> conga = new LinkedList<>();
        conga.addLast("Ana");
        conga.addLast("Bob");
        conga.addFirst("Carlos");   // es cola
        conga.removeLast();

        System.out.println(conga.getFirst());
    }
}
```

Què imprimeix?

- (A) Ana
- (B) Bob
- (C) Carlos
- (D) Error d'execució

> › 🔄 **Solució**
>
> La **C**. `addLast("Ana")` → `[Ana]`, `addLast("Bob")` → `[Ana, Bob]`, `addFirst("Carlos")` → `[Carlos, Ana, Bob]`. `removeLast()` treu Bob → `[Carlos, Ana]`. `getFirst()` torna Carlos. La conga queda tallada pel principi, no pel final.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina col·lecció usaries per a una cua de supermercat?
2. Què fa `removeFirst()` sobre una llista buida si uses `peek()`/`poll()` en el seu lloc?
3. Quina és la diferència clau de rendiment entre `LinkedList` i `ArrayList`?
4. Amb `addFirst` + `removeFirst`, estàs simulant una cua o una pila?

> 🔄 **Respostes**
>
> 1. `LinkedList`, amb els seus mètodes `addLast` i `removeFirst`: la cua FIFO del supermercat.
> 2. `peek()` torna `null` sense tocar la llista; `poll()` torna `null` i no llança excepció. `removeFirst()` directe sobre buida llança `NoSuchElementException`.
> 3. `LinkedList` brilla afegint/esborrant als extrems; `ArrayList` brilla amb l'accés aleatori `get(i)`, que va directe a l'índex.
> 4. Una pila (LIFO): l'últim que entra és el primer que surt.

✅ Resum en 3 frases

1. **LinkedList** encadena els seus elements (la conga): cadascun sap qui va davant i darrere.
2. És **rapidíssima al principi i al final**, i per això és la reina de les **cues (FIFO)** i les **piles (LIFO)**.
3. Per a accés aleatori freqüent (`get(i)`), guanya `ArrayList`: `LinkedList` ha de saltar d'anella en anella.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Llista enllaçada	Elements encadenats que saben qui va davant i darrere
FIFO	First In, First Out: la cua del supermercat
LIFO	Last In, First Out: la pila de plats
`peek()`	Mirar el primer sense treure'l

Terme	Idea general
`poll()`	Treure el primer sense llançar excepció si està buida

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · ArrayList i el Java Collections Framework · **Següent:** 03 · HashSet, TreeSet i LinkedHashSet

03 · HashSet, TreeSet i LinkedHashSet: els conjunts

El porter sense duplicats, el que ordena sol i el que recorda l'ordre d'arribada: la família Set al complet 🚫🧭

> 🗺️ **Estàs en: 📚 U12 · Col·leccions, genèrics i mapes** → 03 · HashSet, TreeSet i LinkedHashSet

📬 La idea en una frase

> **HashSet és una discoteca que no deixa entrar ningú repetit; TreeSet és el mateix porter però amb brúixola (ordena sol); i LinkedHashSet té memòria de l'ordre d'arribada.**

Al punt 2 vas conèixer les llistes: ordre i duplicats benvinguts. Ara canvia el ball: hi ha problemes on els repetits sobren. Aquí entra la família `Set`, amb tres germans que comparteixen la mania de no admetre duplicats però es diferencien en l'ordre.

🚫 HashSet: el porter que no deixa duplicats

HashSet és com una discoteca: **no deixa entrar ningú que ja estigui dins**. No importa l'ordre, només l'exclusivitat.

```
import java.util.HashSet;

public class EjemploHashSet {
    public static void main(String[] args) {
        HashSet<String> invitados = new HashSet<>();
        invitados.add("Ana");
        invitados.add("Bob");
        invitados.add("Ana"); // No passa res, Ana ja està
        System.out.println(invitados.size()); // 2, no 3
        System.out.println(invitados.contains("Bob")); // true
    }
}
```

Com sap si un element ja hi és? Usa `hashCode()` i `equals()`. Primer calcula el `hashCode()` de l'element per a trobar el "calaix" i després comprova amb `equals()` si ja n'hi ha un d'igual dins.

> ⚠️ **Advertència:** si sobreescrius `equals()` en una classe, SOBREESCRIU `hashCode()`. Sempre. Si dos objectes són iguals segons `equals()`, han de tenir el mateix `hashCode()`. Si no, HashSet s'ho passarà malament. Repeteix: **sempre**.

Operacions típiques amb HashSet

```
HashSet<String> set = new HashSet<>();
set.add("roig");
set.add("verd");
set.add("blau");
set.remove("roig");
set.contains("verd");   // true
set.isEmpty();          // false
set.size();             // 2
set.clear();            // ho buida tot
```

Els seus avantatges: **O(1)** per a afegir, esborrar i comprovar pertinença, i elimina duplicats sense que ho hagis de pensar. És la col·lecció més ràpida per a "ja ho tinc?".

Un HashSet no té ordre

HashSet no garanteix cap ordre. Si fiques `{"roig", "verd", "blau"}`, en imprimir pot sortir `[blau, roig, verd]` o qualsevol altra combinació. No et fies de l'ordre, fia-te'n de `size()` i `contains()`.

```
HashSet<String> colores = new HashSet<>();
colores.add("roig");
colores.add("verd");
colores.add("blau");
System.out.println(colores); // [roig, verd, blau]? Pot ser, pot ser que no...
```

🧭 TreeSet: l'organitzat

TreeSet és un HashSet que **s'ordena sol**. Internament usa un arbre vermell-negre (no necessites saber més, només que ordena). Tot el que fiques es col·loca automàticament al seu lloc.

```java
import java.util.TreeSet;

public class EjemploTreeSet {
    public static void main(String[] args) {
        TreeSet<String> ordenado = new TreeSet<>();
        ordenado.add("Zara");
        ordenado.add("Ana");
        ordenado.add("Bob");
        System.out.println(ordenado); // [Ana, Bob, Zara] - ordre alfabètic

        System.out.println(ordenado.first());      // "Ana"
        System.out.println(ordenado.last());       // "Zara"
        System.out.println(ordenado.headSet("Bob")); // [Ana]
    }
}
```

Mètodes extra útils:

Mètode	Què fa
`first()` / `last()`	El menor / el major
`headSet(e)`	Tot el que va abans de e
`tailSet(e)`	Tot el que va després (inclòs) de e
`subSet(a, b)`	El tros entre a i b
`floor(e)` / `ceiling(e)`	El major ≤ e / el menor ≥ e

> 📝 **Nota:** TreeSet **no admet `null`**. Per a ordenar necessita comparar elements, i com compares `null` amb alguna cosa? HashSet, en canvi, admet un únic `null`. Detall d'examen assegurat.

🧠 LinkedHashSet: el que recorda l'ordre d'arribada

LinkedHashSet és el germà de HashSet que **conserva l'ordre d'inserció**. Elimina duplicats com el porter de fitxers, però recorda qui va arribar primer, segon, tercer…

```
import java.util.LinkedHashSet;

public class EjemploLinkedHashSet {
    public static void main(String[] args) {
        LinkedHashSet<String> colaDeEntrada = new LinkedHashSet<>();
        colaDeEntrada.add("Ana");
        colaDeEntrada.add("Bob");
        colaDeEntrada.add("Ana");   // duplicat: no entra
        colaDeEntrada.add("Carla");
             System.out.println(colaDeEntrada);  //  [Ana,  Bob,  Carla]  -  ordre
d'arribada
    }
}
```

El seu ús estrella: **eliminar duplicats mantenint l'ordre de primera aparició**. Un clàssic dels exercicis i de les entrevistes.

```
import java.util.ArrayList;
import java.util.LinkedHashSet;

ArrayList<Integer>  lista  =  new  ArrayList<>(java.util.Arrays.asList(3,  1,  4,  1,
5,  9,  2,  6,  5,  3,  5));
LinkedHashSet<Integer> sinRepetidos = new LinkedHashSet<>(lista);
System.out.println(sinRepetidos); // [3, 1, 4, 5, 9, 2, 6] - sense duplicats i
en ordre d'arribada
```

Construir un `LinkedHashSet` a partir de la llista elimina els repetits; convertir-lo de nou en `ArrayList` et torna la llista neta. Dues línies i el problema dels duplicats desapareix.

> 💡 **Consell:** HashSet és el més ràpid (O(1)), TreeSet ordena (O(log n)) i LinkedHashSet recorda l'ordre (O(1) amb un poc més de memòria). Cadascun amb la seva superpotència.

⭐ Be the Code, my friend... 🔗

> 🕶️ **Don Tip:** davant "sense duplicats i ordenats" pensa en TreeSet. Davant "sense duplicats però mantenint l'ordre d'arribada" pensa en LinkedHashSet. Davant "només sense duplicats" pensa en HashSet.

Exercici: què surt per pantalla?

```java
import java.util.TreeSet;

public class BeTheTree {
    public static void main(String[] args) {
        TreeSet<Integer> notas = new TreeSet<>();
        notas.add(7);
        notas.add(3);
        notas.add(9);
        notas.add(7);   // duplicat: no entra
        notas.add(1);

        System.out.println(notas.first() + " " + notas.last());
        System.out.println(notas.size());
    }
}
```

Què imprimeix?

- (A) 1 9 i 5
- (B) 3 9 i 4
- (C) 1 9 i 4
- (D) 1 7 i 4

> 🔄 **Solució**
>
> La **C**. Els valors únics són `{1, 3, 7, 9}`: 4 elements (el 7 repetit no entra). Ordenats: primer el `1`, últim el `9`. `first()` = 1, `last()` = 9, `size()` = 4.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quants elements hi ha en un `HashSet` després d'afegir `a, b, a, c, b`?
2. Què diferencia `TreeSet` de `HashSet`?
3. Pot un `TreeSet` contenir un `null`? I un `HashSet`?
4. Quina col·lecció elimina duplicats **mantenint l'ordre d'inserció**?
5. Què torna `headSet("Bob")` en un TreeSet amb `{"Ana", "Bob", "Zara"}`?

> 🔄 **Respostes**

1. 3 (a, b, c). Els duplicats no entren.
2. `TreeSet` manté els elements ordenats automàticament (arbre vermell-negre) a costa de ser més lent (O(log n) davant O(1)).
3. `TreeSet` no: necessita comparar per a ordenar i no pot comparar `null`. `HashSet` sí admet un únic `null`.
4. `LinkedHashSet`: elimina duplicats com HashSet però conserva l'ordre d'arribada.
5. `[Ana]`: tot el que va abans de "Bob" en l'ordre natural.

✅ Resum en 3 frases

1. **HashSet** és el porter sense duplicats: no admet repetits, no ordena res i és **O(1)** gràcies a `hashCode()` i `equals()`.
2. **TreeSet** ordena tot el que fiques automàticament i ofereix extra (`first`, `last`, `headSet`, `subSet`), a canvi de ser més lent i no acceptar `null`.
3. **LinkedHashSet** conserva l'ordre d'inserció i és la solució estrella per a **eliminar duplicats mantenint l'ordre d'arribada**.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`Set`	Col·lecció sense duplicats
`hashCode()`	"Número de calaix" que usa HashSet per a trobar elements
Arbre vermell-negre	Estructura interna de TreeSet que manté l'ordre
Ordre natural	L'ordre per defecte del tipus (numèric, alfabètic)
Ordre d'inserció	L'ordre en què es van afegir els elements
O(1) / O(log n)	Instantani / logarítmic (ràpid encara que creixi)

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · LinkedList: la conga line · **Següent:** 04 · Iterator i Collections

04 · Iterator i Collections: recórrer i manipular

El camarer que recorre qualsevol col·lecció i esborra sense trencar res, i la navalla suïssa que ordena, barreja i consulta en una línia 🍽️🧰

> 🗺️ **Estàs en: 📚 U12 · Col·leccions, genèrics i mapes** → 04 · Iterator i Collections

---

📬 La idea en una frase

> **`Iterator` recorre qualsevol col·lecció i el seu `remove()` és l'única forma segura d'esborrar durant el recorregut; `Collections` és la navalla suïssa de mètodes `static` que fa en una línia el que tu tardaries en deu.**

En els punts anteriors vas recórrer llistes i conjunts amb `for-each` sense preguntar-te com funcionaven per dins. Aquell `for-each` té un superheroi treballant per davall: l'`Iterator`. I una vegada saps recórrer, toca manipular sense escriure bucles a mà: per a això està `Collections`, la germana de la classe `Arrays` que vas veure en la U06.

🍽️ Iterator: el camarer que pren nota

`Iterator` recorre una col·lecció **sense que sàpigues ni t'importi com està implementada per dins**. És com un camarer: «Què vol? I vostè? I vostè?»

```java
import java.util.ArrayList;
import java.util.Iterator;

public class EjemploIterator {
    public static void main(String[] args) {
        ArrayList<String> platos = new ArrayList<>();
        platos.add("Tortilla");
        platos.add("Paella");
        platos.add("Croquetas");

        Iterator<String> it = platos.iterator();
        while (it.hasNext()) {          // queda alguna cosa?
            String plato = it.next();   // dona-m'l
            System.out.println(plato);
        }
    }
}
```

El protocol del camarer:

Mètode	Què fa
`hasNext()`	Hi ha més elements? (`boolean`)
`next()`	Torna el següent i avança
`remove()`	Esborra l'últim element tornat per `next()`

> 💡 **Consell:** el `for-each` usa un `Iterator` per davall. Per això pots usar `for-each` amb qualsevol col·lecció: totes ho implementen. El for-each és l'atac; l'`Iterator` és l'eina amb superpoders (esborrar).

💥 La superpotència: esborrar durant el recorregut

Aquí està la joia. **Mai** facis `lista.remove(elemento)` mentre uses un `for-each`:

```java
ArrayList<String> platos = new ArrayList<>();
platos.add("Tortilla");
platos.add("Paella");
platos.add("Croquetas");

// MAL: ConcurrentModificationException
for (String plato : platos) {
    if (plato.equals("Paella")) {
        platos.remove(plato);   // BOOM
    }
}
```

> ⚠️ **Advertència:** esborrar amb `lista.remove()` mentre un for-each recorre la llista llença `ConcurrentModificationException`. El for-each no sap que la llista ha canviat per davall i s'eixelebrat. Usa SEMPRE `iterator.remove()` si necessites esborrar durant el recorregut.

La forma correcta:

```java
Iterator<String> it = platos.iterator();
while (it.hasNext()) {
    String plato = it.next();
    if (plato.equals("Paella")) {
        it.remove(); // ESBORRA de la llista ORIGINAL, sense excepció
    }
}
// Ara platos = [Tortilla, Croquetas]
```

`it.remove()` esborra l'element que acabava de tornar `next()` de la col·lecció original. És l'única forma segura d'«esborrar mentre recorres».

🧰 Collections: l'aliat utilitari 🔗

```java
import java.util.ArrayList;
import java.util.Collections;

public class EjemploCollections {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("Zara");
        lista.add("Ana");
        lista.add("Bob");

        Collections.sort(lista);        // [Ana, Bob, Zara]
        Collections.reverse(lista);     // [Zara, Bob, Ana]
        Collections.shuffle(lista);     // ordre aleatori
        String max = Collections.max(lista);    // "Zara" (ordre alfabètic)
        String min = Collections.min(lista);    // "Ana"
           int veces = Collections.frequency(lista, "Ana"); // quantes vegades
apareix
        Collections.replaceAll(lista, "Ana", "Ana María");
        Collections.rotate(lista, 2);   // rota 2 posicions
    }
}
```

El repertori essencial:

Mètode	Què fa
`sort(lista)`	Ordena la llista en el lloc
`reverse(lista)`	Li dona la volta
`shuffle(lista)`	Barreja a l'atzar
`max(lista)` / `min(lista)`	El major / el menor
`frequency(lista, e)`	Quantes vegades apareix e
`replaceAll(lista, viejo, nuevo)`	Substituïx totes les aparicions
`rotate(lista, k)`	Rota els elements k posicions
`binarySearch(lista, e)`	Busca (llista ordenada)

Mètode	Què fa
`copy(destino, origen)`	Copia una llista en una altra

> 📝 **Nota:** `Collections.sort()` ordena la **llista original** (la modifica). Si no vols tocar la teva, copia-la abans: `ArrayList<String> copia = new ArrayList<>(lista);` i ordena la còpia.

⚠️ Els mètodes que modifiquen la teva llista

Gairebé tots els mètodes de `Collections` que comencen amb un verb («sort», «shuffle», «reverse», «rotate»…) **modifiquen la llista que els pases**. No tornen una de nova: canvien l'original.

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(3);
nums.add(1);
nums.add(2);

Collections.sort(nums);      // nums ARA és [1, 2, 3]
System.out.println(nums);    // [1, 2, 3] - l'original, modificada
```

> ⚠️ **Advertència:** si necessites conservar la llista original, fes sempre una còpia abans: `new ArrayList<>(lista)`. És l'error típic de qui ordena sense voler la seva llista de la compra.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `it.remove()` esborra l'últim element tornat per `next()`. Mai crides a `remove()` sense un `next()` just abans. I recorda: els mètodes `static` de `Collections` van a per l'original.

Exercici 1: què imprimeix (o què llença)?

```java
import java.util.ArrayList;

public class BeTheIterator {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>();
        nombres.add("Ana");
        nombres.add("Bob");
        nombres.add("Carla");

        for (String n : nombres) {
            if (n.equals("Bob")) {
                nombres.remove(n);
            }
        }
        System.out.println(nombres);
    }
}
```

Què passa?

- (A) Imprimeix `[Ana, Carla]`
- (B) Imprimeix `[Ana, Bob, Carla]`
- (C) Llença `ConcurrentModificationException`
- (D) No compila

> 🔄 **Solució**
>
> La **C**. El `for-each` usa un `Iterator` intern. En esborrar amb `nombres.remove(n)` mentre l'iterador recorre la llista, la col·lecció canvia «per l'esquena» i l'iterador llença `ConcurrentModificationException`. La solució: recórrer amb `Iterator` explícit i usar `it.remove()`.

Exercici 2: Collections en acció

```java
import java.util.*;

public class BeTheCollections {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5);
        nums.add(1);
        nums.add(8);
        nums.add(3);

        Collections.sort(nums);
        Collections.reverse(nums);

        System.out.println(nums.get(1));
    }
}
```

Què imprimeix?

- (A) 1
- (B) 3
- (C) 5
- (D) 8

> 🔄 **Solució**
>
> La **C**. `sort` → `{1, 3, 5, 8}`. `reverse` → `{8, 5, 3, 1}`. `get(1)` = 5. Dos mètodes `static` en dues línies i ja tens la llista manipulada. Sense bucles a mà.

🎯 Mini-chequeig

Posa't a prova en 30 segons (les respostes estan amagades):

1. Quina excepció llença esborrar amb `lista.remove()` dins d'un `for-each`?
2. Com s'esborra un element mentre es recorre amb `Iterator`?
3. `Collections.sort()` modifica la llista original?
4. Quin mètode de `Collections` torna l'element major segons l'ordre natural?

> 🔄 **Respostes**

1. `ConcurrentModificationException`. El for-each no detecta que la llista ha canviat i s'eixelebrat.
2. Amb `it.remove()` just després de `it.next()`. Esborra l'últim element tornat, de la col·lecció original.
3. Sí. Modifica la llista en el lloc. Si vols conservar la teva, copia-la primer amb `new ArrayList<>(lista)`.
4. `Collections.max(lista)`. Torna el major segons l'ordre natural del tipus.

✅ Resum en 3 frases

1. **Iterator** recorre qualsevol col·lecció amb `hasNext()` / `next()`, i el seu `remove()` és l'única forma segura d'**esborrar durant el recorregut**.
2. **Collections** és la navalla suïssa estàtica: `sort`, `reverse`, `shuffle`, `max`, `min`, `frequency`…
3. La majoria dels mètodes de `Collections` **modifiquen la llista original**: fes-te una còpia abans si la necessites intacta.

🐛 Vocabulari ràpid

Terme	Idea general
Iterador	Objecte que recorre una col·lecció sense exposar la seva estructura
`ConcurrentModificationException`	Error en modificar la col·lecció mentre es recorre
Classe utilitària	Classe de mètodes `static` que no s'instància
Modificar en el lloc	Canviar la pròpia col·lecció (no una còpia)
`shuffle`	Barrejar a l'atzar
`frequency`	Comptar aparicions

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · HashSet, TreeSet i LinkedHashSet · **Següent:** 05 · Classes genèriques: la teva `Caja<T>`

05 · Classes genèriques: la teva `Caja<T>`

D'on surt el `` d'ArrayList, les convencions T/E/K/V i la teva primera classe amb buits rellenables 📦

> 🗺️ **Estàs en:** 📚 **U12 · Col·leccions, genèrics i mapes** → 05 · Classes genèriques: la teva `Caja<T>`

📬 La idea en una frase

> **Els genèrics existeixen per una sola raó: seguretat de tipus. I una classe genèrica és una plantilla amb buits: declares <T> i qui la usi decideix quin tipus reompli cada buit.**

En els punts 1 i 2 portes tot el dia usant `ArrayList<String>` i `HashSet<Integer>`… però ningú t'ha explicat d'on surt aquell `<String>`. Avui és el dia.

😱 L'infern abans dels genèrics

Una `ArrayList` a la manera antiga (sense genèrics) és una capsa de caos on pots ficar qualsevol cosa. En treure, Java et torna un `Object` i tu has de recordar què vas ficar:

```
import java.util.*;

ArrayList cajaDeCaos = new ArrayList();   // sin <String>, sin <Integer>...
cajaDeCaos.add(42);
cajaDeCaos.add("Hola");

Object cosa = cajaDeCaos.get(0);
String texto = (String) cosa;  // 💥 ClassCastException en temps d'execució
```

> ⚠️ **Advertència:** sense genèrics, els errors de tipus salten en temps d'EXECUCIÓ (quan l'usuari usa el teu programa). Amb genèrics, salten en temps de COMPILACIÓ (mentre programes). Quan prefereixes assabentar-te'n?

🤝 Arriben els genèrics

Una `ArrayList<String>` ja no és una capsa de caos: és una màquina expenedora que SOLS dona Coca-Coles. Si intentes ficar un `int`, el compilador et para el braç. I en treure, `get(0)` torna un `String` directament: sense casting i sense creuar els dits.

El `<String>` és un **paràmetre de tipus**: una promesa que li fas al compilador i que ell s'encarrega de complir.

🔤 Les convencions: T, E, K, V, N

El paràmetre de tipus s'escriu entre `< >` i, per convenció, s'usen majúscules amb significat implícit:

- `T` → Tipus (Type), el comodí general.
- `E` → Element (Element), per a col·leccions.
- `K` / `V` → Clau / Valor (Key / Value), per a mapes.
- `N` → Número (Number).

📦 La classe `Caja<T>`

Una capsa que guarda UN objecte de qualsevol tipus:

```
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return contenido;
    }

    public boolean estaVacia() {
        return contenido == null;
    }
}
```

Fixa't en el patró: `T` apareix com a tipus de l'atribut, del paràmetre de `guardar` i del retorn de `sacar`. Quan algú escriu `Caja<String>`, totes les `T` es converteixen en `String` per a aquella instància:

```
Caja<String> cajaDeTexto = new Caja<>();
cajaDeTexto.guardar("Mensaje secreto");
String mensaje = cajaDeTexto.sacar();  // Sin casting, directo al pelo

Caja<Integer> cajaDeNumeros = new Caja<>();
cajaDeNumeros.guardar(42);
Integer numero = cajaDeNumeros.sacar();
```

`cajaDeTexto` i `cajaDeNumeros` surten del mateix motlle, però per al compilador són tipus diferents: no pots barrejar-les sense que et pille.

> ⚠️ **Advertència:** no pots usar primitius com a paràmetre de tipus. `Caja<int>` no compila: usa `Caja<Integer>` amb la seva classe envolvent i deixa que l'autoboxing faci la conversió automàtica (ho vas veure en el punt 1).

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** el tipus `T` és un comodí que es reemplaça en instanciar. Però ull: la capsa només guarda UN element; el segon `guardar` pisà el primer.

Exercici: la capsa que només recorda l'últim

```
public class BeTheCaja {
    public static void main(String[] args) {
        Caja<Integer> caja = new Caja<>();
        caja.guardar(5);
        caja.guardar(10);
        System.out.println(caja.sacar());
    }
}
```

Què imprimeix?

- (A) 5
- (B) 10

- (C) null
- (D) Error de compilació

> 🔄 **Solució**
>
> La **B**. `Caja<T>` guarda un únic atribut `contenido`. El segon `guardar(10)` sobreescriu el 5, així que `sacar()` torna 10. Moraleja: si necessites guardar diversos elements, per a això estan les col·leccions dels punts anteriors.

🤷 No hi ha preguntes tontes

> ❓ **Puc tenir una classe amb dos tipus genèrics?**

Sí, i és molt comú. `Pareja<T, U>` amb `private T primero; private U segundo;` és l'exemple clàssic (el deixaràs ben lligat en els butlletins). Els paràmetres de tipus van separats per comes entre els `< >`.

> ❓ **`Caja` i `Caja<T>` són la mateixa classe?**

Quasi, però no. `Caja` a seques (sense `< >`) és un **tipus cru (raw type)**: funciona com l'infern del principi, sense seguretat de tipus. No l'usis mai excepte si t'obliguen les restes de codi antic.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina excepció evitem gràcies als genèrics en treure elements d'una col·lecció?
2. Quina lletra s'usa per convenció per al paràmetre de tipus d'una col·lecció? I per a clau i valor d'un mapa?
3. Per què no existeix `Caja<int>`?
4. Què li passa al 5 després de cridar `guardar(10)` en una `Caja<Integer>`?

> 🔄 **Respostes**
>
> 1. `ClassCastException`, el clàssic d'usar un element com si fos d'un altre tipus.
> 2. `E` (Element); `K` (Key) i `V` (Value).

3. Perquè `int` és un primitiu i els genèrics només accepten objectes: s'usa `Integer` amb autoboxing.
4. Que es perd: la capsa guarda un únic element i el segon `guardar` sobreescriu el primer.

✅ Resum en 3 frases

1. Els genèrics porten **seguretat de tipus**: comproven en compilació que no fiquis la pota i eliminen els castings a cegues.
2. Una **classe genèrica** es declara amb `<T>` i usa `T` com un buit rellenable en atributs, paràmetres i retorns; les convencions (`T`, `E`, `K`/`V`, `N`) fan que el codi es lligga sol.
3. **Els primitius no valen**: usa les classes wrapper (`Integer` en comptes de `int`) i deixa que l'autoboxing converteixi sol.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Genèric | Codi que treballa amb un tipus que es decideix en usar-lo (`<T>`) |
> | Paràmetre de tipus | El `<String>` de `ArrayList<String>`: el tipus que es passa |
> | Classe genèrica | Classe que declara `<T>` i usa el tipus com un buit rellenable |
> | Wrapper | Classe que embolica un primitiu (`Integer` embolica `int`) |
> | Tipus cru (raw type) | `Caja` sense `< >`: sense seguretat de tipus, evita'l |

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Iterator i Collections · **Següent:** 06 · El diamant, mètodes genèrics i wildcards

06 · El diamant, mètodes genèrics i wildcards

El peresós oficial que infereix el tipus, mètodes amb el seu propi `` i el principi PECS de Joshua Bloch 💎

> 🗺️ **Estàs en:** 📚 **U12 · Col·leccions, genèrics i mapes** → 06 · El diamant, mètodes genèrics i wildcards

---

📬 La idea en una frase

> **El diamant <> t'estalvia repetir el tipus, un mètode genèric declara el seu propi <T> abans del retorn, i els wildcards (`? extends` / `? super`) resolen la invariància dels genèrics amb la regla PECS.**

Tres peces que completes el trencaclosques genèric: la primera t'estalvia teclejar, la segona et dona flexibilitat en utilitats i la tercera és la que cau en les entrevistes.

💎 L'operador diamant <>

Des de Java 7 no cal repetir el tipus dues vegades; el compilador l'infereix:

```
// Antes de Java 7 (repetitivo):
Caja<String> caja1 = new Caja<String>();

// Desde Java 7 (el diamante <> al rescate):
Caja<String> caja2 = new Caja<>();  // "ja saps de què parle, no?"
```

> 💡 **Consell:** usa sempre el diamant. El compilador mira el costat esquerre i deduïx el dret: codi més net i zero repeticions.

🎩 Type erasure: el mag es porta els genèrics

Els genèrics SOLS existeixen en compilació. En generar el bytecode, el compilador **verifica** els tipus, **esborra** la informació genèrica i **afegeix** els castings on calgui:

```
// En tu código fuente:
ArrayList<String> nombres = new ArrayList<>();

// En el bytecode tras compilar:
ArrayList nombres = new ArrayList();   // sin genéricos, con castings
```

Per això dos `ArrayList` de tipus diferents són la mateixa classe en temps d'execució, i per això no pots fer `new T()` ni `new T[]`: en runtime, `T` no existeix.

🧰 Mètodes genèrics

Un mètode pot declarar el seu propi `<T>` **abans del tipus de retorn**, sense que la classe sigui genèrica. El compilador el deduïx dels arguments:

```
public class Utilidades {

    // Declara <T> antes del tipo de retorno
    public static <T> void imprimir(T elemento) {
        System.out.println("Elemento: " + elemento);
    }

    // Con límite: T debe saber compararse (implementar Comparable)
    public static <T extends Comparable<T>> T maximo(T a, T b) {
        return a.compareTo(b) > 0 ? a : b;
    }
}

// Uso con inferencia:
Utilidades.imprimir(42);        // T = Integer
Utilidades.imprimir("Hola");    // T = String
Utilidades.maximo(5, 8);        // 8
Utilidades.maximo("gato", "perro"); // "perro"
```

El **límit** (`T extends Comparable<T>`) converteix la promesa genèrica en contracte: si la teva classe no sap comparar-se, el compilador no et deixa passar-la. I encara que quasi mai cal, existeix la sintaxi explícita `Utilidades.<String>maximo(...)` (sol sortir en exàmens).

🃏 Wildcards: el problema de la invariància

Moment incòmode: `List<Number>` NO accepta una `List<Integer>`, encara que un Integer sigui un Number. Els genèrics són **invariants**. La solució és el comodí `?`:

`? extends T`: per a LLEGIR (covariància)

```
public static double sumar(List<? extends Number> numeros) {
    double total = 0.0;
    for (Number n : numeros) {
        total += n.doubleValue();   // llegir: sí
    }
    // numeros.add(42);             // 🚫 afegir: no (excepte null)
    return total;
}
// sumar(List.of(1, 2, 3)) i sumar(List.of(1.5, 2.5)) compilen ✅
```

`? super T`: per a ESCRIURE (contravariància)

```
public static void rellenar(List<? super Integer> lista) {
    lista.add(1);
    lista.add(2);          // escriure: sí
    // Integer n = lista.get(0);  // 🚫 llegir tipat: no, només com a Object
}
//  rellenar(new  ArrayList<Number>())  i  rellenar(new  ArrayList<Object>())
compilen ✅
```

🧠 PECS: la mnemotècnia infalible

Joshua Bloch (autor de *Effective Java*) ho va resumir en quatre lletres:

> - **P**roducer **E**xtends: si el mètode només **produeix** (llegeix) dades, usa `? extends`.
> - **C**onsumer **S**uper: si el mètode només **consumeix** (escriu) dades, usa `? super`.
> - Fa les dues coses? → no usis wildcard: usa `T` directament.

> ⚠️ **Advertència:** pregunta clàssica d'examen: "puc afegir a una `List<? extends Number>`?" NO (excepte `null`). "Puc llegir tipat d'una `List<? super Integer>`?" Tampoc: només com a `Object`. PECS et salva de totes dues.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** gràcies al type erasure, els genèrics no existeixen en runtime: `nombres.getClass() == numeros.getClass()` és `true` encara que un sigui `ArrayList<String>` i l'altre `ArrayList<Integer>`.

Exercici: el comodí que ho accepta tot

```
import java.util.*;

public class BeTheWildcard {
    public static void main(String[] args) {
        printNumbers(Arrays.asList(1, 2, 3));      // ¿compila?
        printNumbers(Arrays.asList(1.5, 2.5, 3.5)); // ¿compila?
    }

    public static void printNumbers(List<? extends Number> lista) {
        for (Number n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

Quantes crides compilen?

- (A) 0
- (B) 1
- (C) 2
- (D) Error en ambdues

> › 🔄 **Solució**
>
> La **C**. `List<? extends Number>` accepta qualsevol llista el tipus de la qual hereta de Number: tant `List<Integer>` com `List<Double>`. El mètode només llegeix (for-each), així que compleix el costat "producer" de PECS sense problema. Imprimeix `1 2 3 1.5 2.5 3.5`.

🤷 No hi ha preguntes tontes

> ❓ **Els genèrics ralentitzen el meu programa?**

No. El **type erasure** els converteix en castings normals en compilar: és només sucre sintàctic. En runtime no hi ha genèrics ni cost extra.

> ❓ **Per què no puc fer `new T()`?**

Perquè en compilació Java no sap què és `T` i en runtime ja s'ha esborrat. És com demanar-li a un pastisser "un pastís" sense dir-li de què.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Des de quina versió de Java existeix l'operador diamant <>?
2. On es declara el <T> d'un mètode genèric?
3. Què significa `? extends Number` i per a què serveix?
4. Què diu la C de PECS?

> 🔄 **Respostes**
>
> 1. Des de Java 7.
> 2. Abans del tipus de retorn: `public static <T> void imprimir(T x)`.
> 3. "Una llista de Number o de qualsevol subclasse seva", de només lectura (covariància).
> 4. Consumer Super: si el mètode escriu en la col·lecció, usa `? super`.

✅ Resum en 3 frases

1. El **diamant <>** (Java 7+) infereix el tipus del costat dret, i el **type erasure** esborra els genèrics en compilar: per això són gratis i no existeixen en runtime.
2. Un **mètode genèric** declara <T> abans del retorn i pot dur un **límit** (`T extends Comparable<T>`) que actua com a contracte.
3. Els **wildcards** resolen la invariància: `? extends` per a llegir, `? super` per a escriure, i **PECS** (*Producer Extends, Consumer Super*) per a recordar-ho.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Diamant	El <> que infereix el tipus i evita repetir-lo
Type erasure	Procés que esborra els genèrics en compilar
Límit (bound)	Restricció: `T extends Comparable<T>`
Wildcard	El `?`: “qualsevol tipus”
Covariància / Contravariància	`? extends` (llegir) / `? super` (escriure)
PECS	Producer Extends, Consumer Super

📚 Tornar a l’índex de la unitat · **Anterior:** 05 · Classes genèriques: la teva `Caja<T>` · **Següent:** 07 · HashMap: la guia telefònica

07 · HashMap: la guia telefònica

Associa claus amb valors, el put que sobreescriu i les tres formes de recórrer-lo ☎️

> 🗺️ **Estàs en: 📚 U12 · Col·leccions, genèrics i mapes** → 07 · HashMap: la guia telefònica

---

📬 La idea en una frase

> **Un `Map` associa claus úniques amb valors: busques per clau (nom) i obtens el seu valor (telèfon). `HashMap` ho fa en O(1) i, si repeteixes clau, el `put` sobreescriu el valor anterior.**

Fins aquí només guardaves elements solts en llistes i conjunts. Però la vida real està plena d'associacions: nom → telèfon, paraula → vegades que apareix, matrícula → plaça de parking. Per a això existeixen els **mapes**, la tercera família d'estructures juntament amb `List` i `Set`. Benvingut a la guia telefònica.

☎️ HashMap: clau → valor

`HashMap` associa claus amb valors. Com una agenda: busques per nom (clau) i obtens el telèfon (valor).

```
import java.util.HashMap;

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);
agenda.put("Ana", 600000000);  // Sobrescribe el anterior

int telefono = agenda.get("Ana");          // 600000000
Integer inexistente = agenda.get("NoExisto"); // null
agenda.containsKey("Bob");        // true
agenda.containsValue(600000000);  // true
agenda.remove("Bob");             // borra la entrada
agenda.size();                    // 1 (solo queda Ana)
```

Fixa't en les tres idees clau:

1. **Les claus són úniques.** `put("Ana", 612345678)` i després `put("Ana", 600000000)`: el segon sobreescriu el primer. És com ratllar un número de la teva agenda per a posar-ne un altre: només queda l'últim.
2. **`get` torna `null` si la clau no existeix.** No llança excepció, però compte: si la variable és `int` (primitiu), assignar `null` dona error. Per això existeix `getOrDefault`.
3. **Cerca rapidíssima**: O(1) de mitjana. Tant se val si hi ha 10 o 10.000 entrades.

> 📝 **Nota:** les claus d'un HashMap han de ser **immutables**. Per això `String` i `Integer` són perfectes. Si uses un objecte mutable com a clau i després el modifiques, el seu `hashCode()` canvia i el HashMap no el trobarà mai. És com canviar el pany i esperar que la clau vella continuï funcionant.

🛟 `getOrDefault`: el salvavides dels null

En lloc d'escriure el clàssic `if (map.get(clau) != null)`, fas:

```
int edadCarlos = agenda.getOrDefault("Carlos", 0);  // 0 si no existe
```

És com un pla B: "si no trobes a Carlos, dona'm 0". El usaràs moltíssim per a comptar freqüències, com veuràs en el Be the Code.

🚶 Recórrer un HashMap: tres camins

Un mapa té tres vistes per a recórrer: les claus, els valors o les entrades completes.

```java
HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);

// 1) Solo las claves
for (String nombre : agenda.keySet()) {
    System.out.println(nombre + " → " + agenda.get(nombre));
}

// 2) Solo los valores
for (Integer telefono : agenda.values()) {
    System.out.println("Tel: " + telefono);
}

// 3) Las entradas completas (clave + valor juntos) — la preferida
for (HashMap.Entry<String, Integer> entrada : agenda.entrySet()) {
    System.out.println(entrada.getKey() + " → " + entrada.getValue());
}
```

La tercera és la preferida dels professionals: `entrySet()` et dona parells clau-valor sense haver de fer un `get` extra dins del bucle. En els exàmens i en el treball, aquesta és la que veuràs una vegada i una altra.

> 💡 **Consell:** per a recórrer, declara el mapa com `Map<String, Integer>` i usa `Map.Entry<K, V>` en el bucle: així, si demà canvies a `TreeMap`, no toquis res. Programa contra la interfície, com en la U12.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** HashMap no garanteix ordre. Si necessites ordre, usa TreeMap o LinkedHashMap (punt 7). I ull amb `put`: repetir clau sobreescriu, no afegeix.

Exercici 1: el HashMap traïdor

```java
import java.util.HashMap;

public class BeTheMap {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("España", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("España", "Barcelona");

        System.out.println(capitales.get("España"));
    }
}
```

Què imprimeix?

- (A) Madrid
- (B) Barcelona
- (C) null
- (D) Error

> 🔄 **Solució**
>
> La **B**. La clau "España" se sobreescriu amb el nou valor: `put("España", "Barcelona")` pisa l'anterior. "Madrid" ha estat reemplaçat. Moraleja: en un HashMap, una clau només pot tenir UN valor, l'últim que es posi.

Exercici 2: la freqüència de lletres

> 🕶️ **Don Tip:** `getOrDefault()` evita el null check. Usa'l sempre que puguis.

```java
import java.util.HashMap;

public class BeTheFrequency {
    public static void main(String[] args) {
        String texto = "banana";
        HashMap<Character, Integer> frec = new HashMap<>();
        for (char c : texto.toCharArray()) {
            frec.put(c, frec.getOrDefault(c, 0) + 1);
        }
        System.out.println(frec.get('a') + " " + frec.get('n'));
    }
}
```

Què imprimeix?

- (A) 1 1
- (B) 3 1
- (C) 3 2
- (D) 2 2

> 🔄 **Solució**
>
> La **C**. En “banana”, “b-a-n-a-n-a”: b=1, a=3, n=2. Així que `frec.get('a') = 3` i `frec.get('n') = 2` → **3 2**. El truc de `getOrDefault(c, 0) + 1` és l’heroi: si la lletra no estava, torna 0 i comença en 1; si estava, suma un al comptador. Aquest patró és EL patró dels mapes: el veuràs en la U14 amb fitxers i en mitja carrera de programació.

🤷 No hi ha preguntes tontes

> ❓ **Puc tenir un HashMap amb clau `null`?**

Sí, HashMap admet **una** clau `null` (la guarda en una posició especial). TreeMap no ho admet. I els valors `null` sí que es permeten en tots dos.

> ❓ **Què passa si la clau no existeix en el mapa?**

`get()` torna `null`. No llança excepció. Usa’l amb cura o, millor, usa `getOrDefault(clau, valorPerDefecte)`, que et torna un valor segur.

> ❓ **I si necessiti associar diverses coses a la mateixa clau?**
>
> Usa un mapa de llistes: `HashMap<String, ArrayList<Integer>>`. La clau apunta a una llista i cada `put` ben fet afegeix a la llista existent. És l'estructura que usaràs per a "una clau, molts valors".

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `get(clau)` si la clau no existeix?
2. Què fa `put("Ana", x)` si la clau "Ana" ja existeix?
3. Quin mètode torna un valor per defecte si la clau no existeix?
4. Quin és el bucle preferit per a recórrer un mapa i per què?

> 🔄 **Respostes**
>
> 1. `null` (no llança excepció).
> 2. Sobreescriu el valor anterior: les claus són úniques.
> 3. `getOrDefault(clau, valorPerDefecte)`.
> 4. `for (Map.Entry<K, V> e : mapa.entrySet())`: et dona clau i valor junts sense un `get` extra.

✅ Resum en 3 frases

1. **`Map`** associa claus úniques amb valors: `put` afegeix (o sobreescriu), `get` consulta i torna `null` si la clau no existeix.
2. Es recorre amb **`keySet()`** (claus), **`values()`** (valors) o **`entrySet()`** (parells), sent aquesta última la preferida.
3. Les claus han de ser **immutables** i el `getOrDefault()` és el teu salvavides contra els `null`, especialment per a comptar freqüències.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`Map`	Interfície que associa claus amb valors
Clau / valor	El parell associat; la clau és única
`put`	Afegir o sobrescriure una entrada
`getOrDefault`	Tornar un valor segur si la clau no existeix
`entrySet`	Les entrades (clau + valor) per a recórrer

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · El diamant, mètodes genèrics i wildcards · **Següent:** 08 · TreeMap i com triar estructura

08 · TreeMap i com triar estructura

L'ordenat de sèrie, el que recorda l'ordre d'arribada i els mètodes de navegació 🌳

> 🗺️ **Estàs en: 📚 U12 · Col·leccions, genèrics i mapes** → 08 · TreeMap i com triar estructura

---

📬 La idea en una frase

> **`HashMap` és rapidíssim però no ordena, `TreeMap` ordena les claus automàticament (O(log n)) i `LinkedHashMap` recorda l'ordre d'inserció. Triaràs segons el que necessites.**

En el punt 6 vas conèixer el rei de la velocitat. Però a vegades la velocitat no basta: necessites que les claus surtin ordenades, o que es respecte l'ordre en què les vas ficar. Aquí estan els seus dos cosins, amb els seus superpoders i les seves debilitats.

🌳 TreeMap: l'ordenat de sèrie

`TreeMap` és un mapa ordenat per clau. Internament usa un arbre vermell-negre, que és com una biblioteca que es reordena sola cada vegada que afegeixes un llibre.

```
import java.util.TreeMap;

TreeMap<String, Integer> ordenado = new TreeMap<>();
ordenado.put("Zara", 30);
ordenado.put("Ana", 25);
ordenado.put("Bob", 35);
System.out.println(ordenado);  // {Ana=25, Bob=35, Zara=30} — orden alfabético

// Métodos de navegación extra
ordenado.firstKey();    // "Ana" — la primera clave
ordenado.lastKey();     // "Zara" — la última clave
ordenado.headMap("Bob");  // {Ana=25} — entradas antes de "Bob"
ordenado.subMap("Ana", "Zara"); // {Ana=25, Bob=35} — entre claves
```

La gràcia està en els mètodes de navegació: `firstKey`, `lastKey`, `headMap`, `subMap`… són la raó de ser del TreeMap. Amb un HashMap, per a "les claus que van abans que Bob" hauríes de copiar i ordenar a mà.

> ⚠️ **Advertència:** TreeMap NO admet una clau `null` (no sap comparar-la amb res). I les claus han de ser comparables: per això `String` i `Integer` funcionen, i les classes teves sense `Comparable` fallen en temps d'execució.

🔗 LinkedHashMap: el que recorda l'ordre d'arribada

`LinkedHashMap` és un HashMap que, a més, manté l'ordre d'inserció. És com un HashMap amb memòria del que vas anar ficant.

```
import java.util.LinkedHashMap;

LinkedHashMap<String, Integer> conMemoria = new LinkedHashMap<>();
conMemoria.put("Zara", 30);
conMemoria.put("Ana", 25);
conMemoria.put("Bob", 35);
System.out.println(conMemoria);    // {Zara=30, Ana=25, Bob=35} — orden de inserción
```

Quan l'uses? Quan vols la velocitat del HashMap però necessites saber "en quin ordre va arribar cada cosa": un historial, una llista de "l'últim vist"... Recorda: **ordre d'inserció**, no ordre natural.

⚖️ La família al complet

Mapa	Ordre	Velocitat	Clau `null`?
`HashMap`	Cap	O(1)	Sí (una)
`TreeMap`	Natural (per clau)	O(log n)	No
`LinkedHashMap`	D'inserció	O(1)	Sí (una)

Regla pràctica: velocitat i no t'importa l'ordre? → `HashMap`. Claus sempre ordenades o navegació (`firstKey`, `headMap`...)? → `TreeMap`. Velocitat però respectant l'ordre d'arribada? → `LinkedHashMap`.

🥊 EL RING: HashMap vs TreeMap

Dues implementacions de Map s'enfronten davant la màquina de cafè.

HashMap: — Jo soc el rei de la velocitat. O(1) en `get` i `put`. No m'importa l'ordre, m'importa la rapidesa.

TreeMap: — Sí, però jo mantinc les claus ordenades automàticament. Si necessites recórrer-les en ordre alfabètic, soc el teu únic amic.

HashMap: — ¿Ordenat? Això costa. Jo soc O(1) en tot. ¡Soc imbatible en rendiment!

TreeMap: — Cert, però puc navegar: `firstKey()`, `lastKey()`, `subMap()`, `headMap()`. Tu, per a tot això, has de copiar i ordenar.

HashMap: — Si no necessites ordre, per què pagar el cost? La majoria dels casos usen HashMap.

TreeMap: — I quan necessiten ordre, aquí estic jo. I no soc tan lent: O(log n) continua sent molt ràpid per a la majoria dels casos.

HashMap: — Treva. Cadascú al seu lloc.

TreeMap: — Fet.

> La lliçó: **velocitat? → HashMap. ordre natural? → TreeMap. ordre d'inserció? → LinkedHashMap.** Cadascú té el seu superpoder, i la resposta a "quin és millor?" sempre és "depèn".

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** TreeMap ordena per clau en el moment del `put`. No ordenis tu: ell ordena sol.

Exercici: la biblioteca que s'ordena sola

```java
import java.util.TreeMap;

public class BeTheTreeMap {
    public static void main(String[] args) {
        TreeMap<String, Integer> m = new TreeMap<>();
        m.put("Zara", 30);
        m.put("Ana", 25);
        m.put("Bob", 35);

        System.out.println(m.firstKey());
        System.out.println(m.headMap("Bob"));
    }
}
```

Què imprimeix?

- (A) Zara / {Zara=30}
- (B) Ana / {Ana=25}
- (C) Ana / {Ana=25, Bob=35}
- (D) Zara / {Zara=30, Ana=25}

> 🔄 **Solució**
>
> La **B**. `firstKey()` torna la primera clau en ordre natural: “Ana”. `headMap("Bob")` torna les entrades amb claus estrictament menors que “Bob”: només `{Ana=25}`. Ull: “Bob” no entra en el seu propi `headMap`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mapa ordena les claus automàticament?
2. Quin mapa manté l’ordre d’inserció?
3. Quin mètode torna les entrades amb clau menor que una donada?
4. Pot un TreeMap tenir una clau `null`?

> 🔄 **Respostes**
>
> 1. `TreeMap` (ordre natural de la clau).

> 2. `LinkedHashMap` (ordre d'inserció).
> 3. `headMap(clau)`.
> 4. No. TreeMap no sap comparar `null` amb res; HashMap sí admet una clau `null`.

⚖️ Map vs List vs Set: la comparativa definitiva

Característica	List	Set	Map
Què guarda?	Elements ordenats per posició	Elements únics	Parells clau → valor
Duplicats?	Sí	No	Claus no, valors sí
Ordre?	D'inserció	Depèn (Hash/Tree)	Depèn (Hash/Tree)
Accés	Per índex	Per element	Per clau
Nulls?	Sí	HashSet: 1, TreeSet: 0	HashMap: 1 clau, TreeMap: 0
Implementació principal	`ArrayList`	`HashSet`	`HashMap`

La gran diferència: List i Set guarden elements solts; Map guarda parelles. Si la teva dada necessita una "clau" per a buscar-se després (un DNI, un nom, una paraula), és un Map. Si només necessites una col·lecció de coses, és List o Set.

🧭 L'arbre de decisions

```
Tens parelles clau → valor?
├── Sí → Necessites ordre per clau?
│        ├── Sí → TreeMap
│        └── No → Ordre d'arribada?
│                 ├── Sí → LinkedHashMap
│                 └── No → HashMap
└── No → T'importa l'ordre?
         ├── No → Duplicats?
         │        ├── No → HashSet
         │        └── Sí → ArrayList
         └── Sí → Quin ordre?
                  ├── Natural → Duplicats?
                  │        ├── No → TreeSet
                  │        └── Sí → ArrayList + sort
                  └── D'arribada → Duplicats?
                           ├── No → LinkedHashSet
                           └── Sí → ArrayList
```

🥇 La regla pràctica

- Una llista de coses que recorres per posició? → `ArrayList`
- Coses sense repetir i no t'importa l'ordre? → `HashSet`
- Coses sense repetir i sempre ordenades? → `TreeSet`
- Associar una cosa amb una altra i buscar per clau? → `HashMap`
- Claus ordenades o navegació (`firstKey`, `headMap`)? → `TreeMap`
- Velocitat però respectant l'ordre d'arribada? → `LinkedHashMap`

La regla d'or: la dada es busca només pel seu contingut? → Set. Per una posició? → List. Per una clau diferent d'ella mateixa? → Map. Aquella última frase separa el que memoritza de qui entén.

Quina estructura usaries? Per a cada escenari, tria i justifica en una línia:

1. Associar cada alumne (pel seu nom) amb la seva nota d'examen.
2. Les notes de la classe, per a recórrer-les moltes vegades per posició.
3. Els DNI dels matriculats, sense repetir i en ordre alfabètic.
4. Saber si una paraula ja ha sortit en un sorteig, amb 100.000 paraules.
5. El nombre de vegades que apareix cada lletra en un text, mostrades en ordre alfabètic.

> 🔄 Solucions
>
> 1. **`HashMap<String, Integer>`** — clau = nom, valor = nota. Cerca directa per alumne.
> 2. **`ArrayList<Integer>`** — accés per índex O(1) i duplicats permesos.
> 3. **`TreeSet<String>`** — sense duplicats i ordre natural automàtic.
> 4. **`HashSet<String>`** — `contains` en O(1) per a la pregunta "ja ha sortit?".
> 5. **`TreeMap<Character, Integer>`** — freqüència per lletra amb les claus ordenades soles.

✅ Resum en 3 frases (ampliat)

1. **List** guarda elements per posició (duplicats sí), **Set** guarda únics i **Map** associa claus úniques amb valors: la primera pregunta sempre és "necessiti associar?".
2. **TreeMap** ordena les seves claus i afegeix navegació (`firstKey`, `headMap`, `floorEntry`); **LinkedHashMap** combina velocitat de HashMap amb ordre d'arribada.
3. **"Quina és la millor estructura?"** es respon sempre amb "depèn" i arguments: l'arbre de decisions d'aquest punt és la teva brúixola.

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · HashMap: la guia telefònica · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu: col·leccions, genèrics i mapes

El tancament pràctic de la unitat: un remove() que ho liarà tot, un HashMap que sobreescriu i un bucle que explotarà 😈

> 🗺️ **Estàs en: 📚 U12 · Col·leccions, genèrics i mapes** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend...

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
import java.util.ArrayList;

public class Misterio {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(1);
        lista.add(2);
        lista.add(3);
        lista.add(4);

        lista.remove(1);
        lista.add(2, 10);

        for (int i = 0; i < lista.size(); i++) {
            System.out.print(lista.get(i) + " ");
        }
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`1 3 4 10`** → El `remove(1)` esborra el 2, i després el `add(2, 10)` fica el 10 al final. ❌
2. **`1 2 3 10`** → El `remove(1)` esborra el 2, i el `add(2, 10)` insereix el 10 a la posició 2. ❌
3. **`1 3 10 4`** → ✅ Correcte! `remove(1)` esborra l'índex 1 (el 2) → `{1, 3, 4}`. `add(2, 10)` insereix el 10 a l'índex 2, desplaçant el 4 → `{1, 3, 10, 4}`.

> 🔄 **Solució**
>
> L'opció **3**. `remove(1)` elimina per **índex** (no el valor 1): se'n va el 2, que ocupava la posició 1. Després `add(2, 10)` insereix el 10 a la posició 2 i empeny el 4 una posició. Els índexs es desplacen, com les fitxes de dòmino.

I ara la versió mapa, amb trampa doble:

```
import java.util.*;

public class MisterioMapa {
    public static void main(String[] args) {
        HashMap<String, Integer> m = new HashMap<>();
        m.put("Ana", 3);
        m.put("Bob", 5);
        m.put("Ana", m.get("Ana") + 2);

        TreeMap<Integer, String> ordenado = new TreeMap<>();
        for (Map.Entry<String, Integer> e : m.entrySet()) {
            ordenado.put(e.getValue(), e.getKey());
        }

        System.out.println(ordenado.firstKey() + " " + m.size());
    }
}
```

Què imprimeix aquest segon programa?

1. **`3 3`** → El `put("Ana", m.get("Ana") + 2)` afegeix una altra entrada i el mapa té 3. ❌
2. **`5 2`** → ✅ Correcte! `Ana` passa de 3 a 5 (3 + 2), `Bob` continua en 5, el mapa té 2 entrades, i el TreeMap de claus numèriques té `{5 → Ana, 5 → Bob}` on el segon `put` sobreescriu: primera clau = 5.
3. **`Bob 2`** → El TreeMap guarda el valor com a clau i el nom com a valor, i `firstKey()` torna el valor. ❌

> 🔄 Solució

L'opció **2**. Primer `Ana = 3 + 2 = 5`, així que el HashMap queda `{Ana=5, Bob=5}` i `size()` és 2. Després el TreeMap es construeix amb `put(valor, clau)`: `put(5, "Ana")` i `put(5, "Bob")` sobreescriuen, deixant `{5=Bob}`. `firstKey()` torna la menor clau numèrica: `5`. Sortida: `5 2`. Fixa't en les dues trampes juntes: `put` que sobreescriu i TreeMap que ordena per clau (aquí, la clau és el número).

🔥 Fireside Chat: ArrayList vs LinkedList

> *Dos cosins de la família List s'enfronten davant la màquina de cafè.*

ArrayList: — Mira, jo soc el de tota la vida. Accés directe a qualsevol posició: `get(500)` i ja està. Contigu, ràpid, fiable.

LinkedList: — Sí, però tu, per a afegir al principi, desplaces tota la fila. Jo encadeno els meus elements com una conga: cadascun sap qui va davant i qui darrere.

ArrayList: — I per a recórrer? Jo soc un array per dins, la CPU t'ho agraeix. Tu saltes d'anella en anella.

LinkedList: — Cert, però els meus mètodes `addFirst`, `addLast`, `removeFirst` són instantanis. Soc la reina de les cues i les piles. Com simules tu una cua de supermercat?

ArrayList: — Val, per a cues i piles ets tu. Per a la resta dels dies, jo. Treva?

LinkedList: — Treva.

> La lliçó: **accés aleatori i recorreguts freqüents? → ArrayList. Afegir/esborrar als extrems o simular cua/pila? → LinkedList.** Cada problema amb la seva llista.

🔥 Fireside Chat: el Genèric vs el Tipus Cru

> *Dues maneres de declarar una llista es troben davant de la mateixa màquina de cafè.*

Genèric: — Mira, jo me'n dic `ArrayList<String>`. Sé exactament què hi ha dins, el compilador em protegeix i no em fan falta castings. Visc amb la consciència tranquil·la.

Tipus Cru: — Ja! Jo soc `ArrayList` a seques. Fique el que vull, trec el que puc i si alguna cosa explota, és que el programador no sabia el que feia. Llibertat total.

Genèric: — Llibertat? A tu, la primera vegada que fiques un `Integer` on anava un `String`, se't trenca el programa en producció. Jo detecto aquell error en compilació, quan encara estàs a temps d'arreglar-lo.

Tipus Cru: — Val, val... però els genèrics són més difícils d'escriure. Aquell `<T>` per tot arreu...

Genèric: — Des de Java 7 el diamant `<>` ho infereix tot, i els genèrics desapareixen en compilar (type erasure). No costen ni un cèntim de rendiment. Tu ets el que paga: amb castings i `ClassCastException`.

Tipus Cru: — Treva. Reconec que soc només per a codi antic.

Genèric: — Fet. I per l'amor de Java, no escriguis `new ArrayList()` sense els `< >`.

> La lliçó: **usa sempre genèrics amb diamant `<>`. El tipus cru és una relíquia que només dona susts.** El compilador no t'odia: t'està protegint.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc l'aparcament que creix sol i no pot guardar `int` a pelo.**
2. **Soc el que encadena elements com una conga, perfecte per a cues i piles.**
3. **Soc el porter que no deixa entrar ningú que ja estigui dins.**
4. **Soc el que ordena tot el que fiques automàticament.**
5. **Soc el cambrer que recorre qualsevol col·lecció i permet esborrar sense liar-la.**
6. **Soc la classe estàtica que ordena, remena i troba el màxim de les teves col·leccions.**
7. **Soc el comodí que significa "qualsevol tipus" i limita a una jerarquia quan li posi `extends`.**
8. **Soc l'operador peresós que evita repetir el tipus dos vegades.**
9. **Soc el mag que esborra els genèrics quan el codi es compila.**
10. **Soc el mapa rapidíssim que no garanteix cap ordre.**
11. **Soc el mapa que ordena les claus automàticament.**
12. **Soc el mètode que funciona per a qualsevol tipus gràcies al seu `<T>` propi.**

> 🔄 **Respostes**

1. **`ArrayList`** — redimensionable; per a números usa `Integer`.
2. **`LinkedList`** — `addLast`, `removeFirst`, la conga de Java.
3. **`HashSet`** — sense duplicats, sense ordre.
4. **`TreeSet`** — ordre natural automàtic (i LinkedHashSet per a l'ordre d'arribada).
5. **`Iterator`** — `hasNext()`, `next()` i el sagrat `remove()`.
6. **`Collections`** — `sort`, `shuffle`, `max`, `min`…
7. El **wildcard** (`?`) — `? extends Number` accepta qualsevol subtipus.
8. El **diamant** `<>` — `new Caja<>()` infereix el tipus.
9. El **type erasure** — esborra els genèrics i afegeix castings en el bytecode.
10. **`HashMap`** — O(1) però sense ordre.
11. **`TreeMap`** — arbre vermell-negre amb `firstKey()`, `headMap()`…
12. El **mètode genèric** — `public static <T> void imprimir(T x)`.

😡 CONRAD VS EL MÓN: "L'aparcament que no aparca" 🔗

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre els clàssics del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu programa es cau.* I jo: val, quin error? *Pues una cosa de "ConcurrentModificationException".* AI, MARE MEVA! Estaves esborrant amb `lista.remove(...)` dins d'un `for-each`. El bucle està recorrent la llista mentre tu la canvies per darrere! Usa `iterator.remove()`, que per a això existeix.

I després està el del `remove` amb números. Escriu `numeros.remove(1)` creient que esborra el número 1. Però si `1` és un `int`! Ho interpreta com a **índex**. Per a esborrar el número 1 necessites `remove(Integer.valueOf(1))`. Índex o valor: decideix què vols esborrar.

I el colmo dels colmos: confon `HashSet` amb `TreeSet`. Es queixa que "el seu HashSet no ordena". I tant que no! Un `HashSet` és un porter, no un organitzador. Si vols ordre, `TreeSet` (ordre natural) o `LinkedHashSet` (ordre d'arribada). I no li fiques `null` a un `TreeSet`, que no sap comparar-lo amb res.

La lliçó: abans d'acusar Java de "odiar-te", repassa el trio sagrat de la unitat: **estic esborrant per índex o per valor? estic modificant una col·lecció mentre la recórri? he triat la col·lecció**

amb l'ordre que necessiti? El 90% dels errors d'aquesta unitat s'arreglen amb aquestes tres preguntes. El compilador no t'odia: t'està passant les respostes de l'examen.

I quan ja creies que CONRAD estava tranquil, arriba el del mapa:

CONRAD: — UNA ALTRA VEGADA! Un alumne m'ensenya la seva agenda i es queixa: *"CONRAD, se m'ha perdut un contacte"*. I jo: l'has afegit amb `put` usant el mateix nom dos vegades? *"Pues sí, ¿por?"* AI, MARE MEVA! En un HashMap les claus són úniques! El segon `put("Ana", ...)` SOBREESCRIU el primer. És com ratllar un número de la teva agenda per a posar-ne un altre: l'anterior se'n va per sempre. Si vols conservar-ne varis, usa `HashMap<String, List<String>>`.

I després està el que pregunta "per què no ordena el meu HashMap?" I tant que no ordena! És un HashMap, el rei de la velocitat, no un bibliotecari. Si vols ordre, `TreeMap`. Si vols ordre d'arribada, `LinkedHashMap`. Cada mapa amb el seu superpoder.

I el colmo dels colmos: el que escriu `Caja<int>` i espera que compili. No es pot! Els genèrics només accepten objectes: usa `Integer` i deixa que l'autoboxing faci la seva màgia. I no em vinguis amb el `get` d'una clau que no existeix: torna `null`. Usa `getOrDefault`, que per a això està.

La lliçó: repassa el trio sagrat del costat dels mapes: **estic sobreescrient una clau amb `put`? he triat el mapa amb l'ordre que necessiti? estic guardant un primitiu on només caben objectes?** El compilador no t'odia: t'està passant les respostes de l'examen.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. `ArrayList<Integer> l = new ArrayList<>(); l.add(7); l.remove(7);` Què esborra el `remove(7)`?
   - a) L'índex 7 b) L'objecte 7
2. Quina col·lecció et dona els elements sempre ordenats i sense duplicats?
   - a) `HashSet` b) `TreeSet`
3. Què llança esborrar amb `lista.remove()` dins d'un `for-each`?
   - a) `ArrayIndexOutOfBoundsException` b) `ConcurrentModificationException`
4. Quina col·lecció elimina duplicats mantenint l'ordre d'arribada?

    - a) `LinkedHashSet` b) `TreeSet`
5. `Collections.sort(lista)`… què fa amb la llista original?
    - a) La modifica b) Torna una llista nova
6. `mapa.put("Ana", 10); mapa.put("Ana", 20);` Què fa el segon `put`?
    - a) Afegeix una altra entrada b) Sobreescriu el valor de “Ana” a 20
7. `Caja<int>`… compila?
    - a) Sí b) No: els genèrics no accepten primitius
8. `mapa.get("NoExisto")` torna…
    - a) `null` b) 0 c) Una excepció
9. Quin mapa et dona les claus sempre ordenades alfabèticament?
    - a) `HashMap` b) `TreeMap`
10. Pots afegir un element a una `List<? extends Number>`?
    - a) Sí, qualsevol Number b) No (només `null`)
11. En runtime, `ArrayList<String>` i `ArrayList<Integer>` són la mateixa classe?
    - a) Sí, gràcies al type erasure b) No, són classes distintes

> 🔄 **Solucions**
>
> 1. **b)** — Com `7` és un `Integer` i la llista és `Integer`, `remove(Object)` esborra l’objecte 7. Si passares un `int` a seques, el compilador triaria `remove(int)` = índex.
> 2. **b)** — `TreeSet` ordena automàticament. `HashSet` no ordena res.
> 3. **b)** — El `for-each` recorre amb un `Iterator` intern; esborrar per l’esquena llança `ConcurrentModificationException`.
> 4. **a)** — `LinkedHashSet` conserva l’ordre d’inserció.
> 5. **a)** — `sort`, `reverse`, `shuffle`… modifiquen la llista que els passes. Si la necessites intacta, copia-la abans.
> 6. **b)** — Les claus d’un Map són úniques; `put` sobreescriu.
> 7. **b)** — `int` és primitiu; es usa `Integer` amb autoboxing.
> 8. **a)** — `get` torna `null` si la clau no existeix (no llança excepció).
> 9. **b)** — `TreeMap` ordena per clau; `HashMap` no ordena res.
> 10. **b)** — `? extends Number` és de només lectura (PECS: Producer Extends).
> 11. **a)** — El type erasure esborra els genèrics en el bytecode: tots dos són `ArrayList`.

⚡ Laboratori de Tortura: la llista que plora

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Et diuen que té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte… però, i si t'ho diuen malament? La teva tasca: fer que compili, que executi i que **tota** la sortida sigui correcta, comptant tu els errors reals.

```
import java.util.ArrayList;

public class Estadisticas {

    public static int mayor(ArrayList<Integer> notas) {
        int max = notas.get(0);
        for (int i = 1; i < notas.size(); i++) {
            if (notas.get(i) > max) {
                max = notas.get(i)
            }
        }
        return max;
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        System.out.println("La mayor es: " + mayor(notas))
    }
}
```

Fallada intencionada: el nombre d'errors de l'enunciat està malament a propòsit: si busques el "tercer error de compilació" i "l'error de lògica", te'n tornes boig. La trampa és la teva confiança en l'enunciat.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Falta algun `;`? *no → segueix buscant.*

   > **I si continuec atascat?**
   >
   > Repassa la línia `max = notas.get(i)` i l'última del `main`: totes dues necessiten `;`.

2. Compila ja? *no → mira els tipus i els imports.*

   > **I si continuec atascat?**
   >
   > El mètode `mayor` rep un `ArrayList` i el `main` el crea igual: els tipus quadren. Mira una altra vegada els `;`.

3. Creus que hi ha un error de lògica que canvia la nota màxima? *Torna a llegir el codi amb calma i no et fies de l'enunciat.*

   > **Solució final**

Els **errors de compilació**:

1. Falta el `;` al final de `max = notas.get(i)`.
2. Falta el `;` al final de `System.out.println(...)`.

L'**error de lògica**: no existeix. La condició `notas.get(i) > max` és correcta i torna el major (9). Aquella era la fallada intencionada: l'enunciat diu "3 errors de compilació i 1 de lògica", però només hi ha 2 faltes de `;` i cap lògica trencada. L'error "amagat" era la teva confiança en l'enunciat. El resultat correcte és `La mayor es: 9`.

```java
import java.util.ArrayList;

public class Estadisticas {

    public static int mayor(ArrayList<Integer> notas) {
        int max = notas.get(0);
        for (int i = 1; i < notas.size(); i++) {
            if (notas.get(i) > max) {
                max = notas.get(i);
            }
        }
        return max;
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        System.out.println("La mayor es: " + mayor(notas));
    }
}
```

Sortida correcta: `La mayor es: 9`. La lliçó real: **llegeix el codi, no l'enunciat**. Compta, verifica i confia en el compilador, no en la llegenda.

⚡ Laboratori de Tortura: la mitjana que plora

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: el mateix joc, ara amb genèrics i mapes. Et diuen que té **3 errors** de compilació i 1 de lògica… t'ho creus?

```java
import java.util.*;

public class Estadisticas {

    public static <T extends Number> double media(List<T> numeros) {
        double suma = 0;
        for (T n : numeros) {
            suma += n.doubleValue()
        }
        return suma / numeros.size();
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        HashMap<String, Double> resultado = new HashMap<>();
        resultado.put("media", media(notas))
        System.out.println(resultado.get("media"));
    }
}
```

Pistes per quan et frustres (no abans):

1. Falta algun `;`? *no → segueix buscant.*

   > **I si continuec atascat?**
   > Repassa la línia `suma += n.doubleValue()` i l'última del `main`: totes dues necessiten `;`.

2. Compila ja? *no → mira els tipus i els imports.*

   > **I si continuec atascat?**
   > El mètode `media` rep `List` amb `T extends Number`, i el `main` li passa un `ArrayList`: els tipus quadren (els wildcards no fan falta perquè `T` es deduïx). Mira una altra vegada els `;`.

3. Creus que hi ha un error de lògica que canvia la mitjana? *Torna a llegir el codi amb calma i no et fies de l'enunciat.*

› Solució final

Els **errors de compilació**:

1. Falta el `;` al final de `suma += n.doubleValue()`.
2. Falta el `;` al final de `resultado.put(...)`.

L'**error de lògica**: no existeix. `media` recorre la llista, acumula amb `doubleValue()` (funciona per a qualsevol `Number`) i divideix entre la grandària: `(6+8+5+9)/4 = 7.0`. Aquella era la fallada intencionada: l'enunciat diu "3 errors de compilació i 1 de lògica", però només hi ha 2 faltes de `;` i cap lògica trencada. L'error "amagat" era la teva confiança en l'enunciat. El resultat correcte és `7.0`.

```
import java.util.*;

public class Estadisticas {

    public static <T extends Number> double media(List<T> numeros) {
        double suma = 0;
        for (T n : numeros) {
            suma += n.doubleValue();
        }
        return suma / numeros.size();
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        HashMap<String, Double> resultado = new HashMap<>();
        resultado.put("media", media(notas));
        System.out.println(resultado.get("media"));
    }
}
```

Sortida correcta: `7.0`. La lliçó real: **llegeix el codi, no l'enunciat**. Compta, verifica i confia en el compilador, no en la llegenda.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🥇 **L'Aparcador**	Explicar per què `remove(1)` i `remove(Integer.valueOf(1))` fan coses diferents
🥇 **El Porter**	Usar `HashSet` per a eliminar duplicats d'una llista en un exercici real
🥇 **El Conguer**	Muntar una cua FIFO amb `LinkedList` sense que es cole ningú
🥇 **El Doctor Iterator**	Esborrar elements d'una col·lecció durant un recorregut sense `ConcurrentModificationException`
🥇 **El Tipògraf**	Explicar per què `ArrayList<String>` rebutja un `int` sense necessitat de casting
🥇 **El Caixer**	Crear la teva pròpia classe genèrica `Caja<T>` i usar-la amb dos tipus diferents
🥇 **El Mag**	Explicar el type erasure i el diamant `<>` com si fos un truc de màgia
🥇 **La Guia Telefònica**	Recórrer un `HashMap` amb `entrySet()` i comptar freqüències amb `getOrDefault`
🥇 **L'Estratega**	Triar entre Map, List i Set justificant la tria amb arguments

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```java
import java.util.ArrayList;

public class Acertijo {
    public static void main(String[] args) {
        ArrayList<Integer> l = new ArrayList<>();
        l.add(1);
        l.add(2);
        l.add(3);
        int suma = 0;
        for (int i = 0; i < l.size(); i++) {
            if (l.get(i) % 2 == 0) {
                suma += l.get(i);
            }
        }
        System.out.println(suma);
    }
}
```

2. **El netejador de llistes:** tens un `ArrayList<Integer>` amb repetits. Com el deixes sense duplicats però mantenint l'ordre de primera aparició, en dues línies?
3. **El detectiu del TreeSet:** tens un `TreeSet<String>` i vols saber quants elements van abans que `"Bob"` en l'ordre natural. Quin mètode uses i què torna?
4. **Vertader o fals:** "un `for-each` pot modificar els elements d'un `ArrayList<Integer>` fent `elemento = 99`".
5. **Sense executar:** què imprimeix aquest programa?

```java
import java.util.HashMap;

public class AcertijoMapa {
    public static void main(String[] args) {
        HashMap<String, Integer> m = new HashMap<>();
        for (char c : "casa".toCharArray()) {
            m.put(String.valueOf(c), m.getOrDefault(String.valueOf(c), 0) + 1);
        }
        System.out.println(m.get("a") + " " + m.size());
    }
}
```

6. **El comptador de paraules ordenat:** tens un `String[]` de paraules i vols mostrar cada paraula amb la seva freqüència, en ordre alfabètic. Quina estructura uses i amb quin

bucle?

7. **El detectiu del erasure:** per què no pots escriure `new T()` dins d'una classe genèrica, encara que t'hagis passat pel <T>?
8. **Vertader o fals:** "un `HashMap` garanteix l'ordre en què introduïxes les claus".

> 💡 **Solucions**
>
> 1. **2**. El bucle recorre els índexs 0, 1, 2. Només el `2` (índex 1) és parell: `suma = 2`.
> 2. Amb un `LinkedHashSet`: `LinkedHashSet<Integer> set = new LinkedHashSet<>(lista);` i després `new ArrayList<>(set)`. Duplicats fora, ordre d'arribada dins.
> 3. `headSet("Bob")` — torna el subconjunt amb els elements que van abans que `"Bob"` en l'ordre natural.
> 4. **Fals.** El for-each és de només lectura: `elemento = 99` només canvia la variable local del bucle, mai la col·lecció. Per a modificar, usa `set(índex, valor)` o el `for` clàssic.
> 5. **2 3**. "casa" té c=1, a=2, s=1. `get("a")` = 2 i `size()` = 3 (tres claus diferents: c, a, s).
> 6. Un **`TreeMap<String, Integer>`** (claus ordenades soles) recorregut amb `entrySet()`: cada entrada és la paraula i la seva comptada.
> 7. Perquè en compilació Java no sap què és `T`, i en runtime el type erasure l'ha esborrat: no hi ha manera d'instanciar un tipus que ja no existeix.
> 8. **Fals.** `HashMap` no garanteix ordre. Per a ordre d'inserció usa `LinkedHashMap`; per a ordre natural, `TreeMap`.

🧩 Crucigrama de Bits

Col·leccions:

```
Horizontal:
1. Classe estàtica que ordena i remena col·leccions (11 lletres)
3. Excepció per modificar una col·lecció mentre es recorre (31 lletres)
5. Bucle de només lectura: "per a cada X en Y" (7 lletres)
6. Classe llista que creix sola i usa size() (9 lletres)

Vertical:
2. Col·lecció que no deixa entrar duplicats (7 lletres)
4. Col·lecció que manté els elements ordenats (7 lletres)
```

> 📝 Solucions (col·leccions)
>
> **Horitzontal:** 1. COLLECTIONS · 3. CONCURRENTMODIFICATIONEXCEPTION · 5. FOREACH · 6. ARRAYLIST **Vertical:** 2. HASHSET · 4. TREESET

Genèrics i mapes:

```
Horitzontal:
1. Mapa que ordena les claus automàticament (7 lletres)
3. Operador peresós que inferix el tipus en `new ArrayList<>()` (7 lletres)
5. Mètode que torna un valor per defecte si la clau no existix (12 lletres)
6. Mapa rapidíssim que no garanteix ordre (7 lletres)

Vertical:
2. Procés que borra els genèrics en compilar (12 lletres)
4. Estructura que associa claus amb valors (3 lletres)
```

> 📝 Solucions (genèrics i mapes)
>
> **Horitzontal:** 1. TREEMAP · 3. DIAMANTE · 5. GETORDEFAULT · 6. HASHMAP **Vertical:** 2. TYPEERASURE · 4. MAP

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java junior.

1. **"Explica-m'ho, com si jo fos la teva àvia, què és un `ArrayList`."**
2. **"Quina és la diferència entre `ArrayList` i `LinkedList`?"**
3. **"Què és l'autoboxing i per què `ArrayList<int>` no compila?"**
4. **"Com elimines un element d'una col·lecció mentre la recórres? Què passa si uses `list.remove()`?"**
5. **"Quan usaries `HashSet` en comptes de `ArrayList`?"**
6. **"Quina és la diferència entre `Collection` i `Collections`?"**
7. **"Explica-m'ho, com si jo fos la teva àvia, què és un `HashMap`."**
8. **"Què és el type erasure i per què és important?"**
9. **"Quina és la diferència entre `? extends T` i `? super T`? Què és PECS?"**
10. **"Per què `List<Number>` no accepta una `List<Integer>`?"**

11. **“Quan usaries un `TreeMap` en comptes d’un `HashMap`?”**
12. **“Pots crear un array de genèrics? I fer `new T()`? Per què?”**

🤷 No hi ha preguntes tontes

> ❓ **`Collection` i `Collections` són el mateix?**

No. `Collection` és la **interfície** arrel de la família (List, Set, Queue la implementen). `Collections` és la **classe utilitària** de mètodes estàtics (`sort`, `shuffle`, `max`…). Una és un contracte; l’altra, una navalla suïssa.

> ❓ **I si mescle tipus en una col·lecció sense genèrics?**

Pots: `ArrayList lista = new ArrayList();` accepta qualsevol cosa. Però en treure-la tot és `Object` i has de fer casting creuant els dits. Amb genèrics (`ArrayList<String>`), el compilador et protegeix i et casca abans que el lio arribi a l’execució.

> ❓ **`for-each` i `iterator` són el mateix?**

Quasi. El `for-each` usa un `Iterator` per darrere. La diferència: amb l’`Iterator` tens el mètode `remove()`, que et permet esborrar durant el recorregut. Si només llegeixes, usa el for-each: és més curt i llegible.

> ❓ **`Map` és una `Collection`?**

No. `Map` no hereta de la interfície `Collection` (encara que viu en `java.util`). `List` i `Set` són `Collection`; `Map` és una família a banda que associa claus amb valors. És un error molt típic: “les col·leccions de Java” inclouen els tres, però tècnicament `Map` no implementa `Collection`.

> ❓ **Els genèrics ralentitzen el meu programa?**

No. Java aplica **type erasure**: el compilador esborra tota la informació genèrica i la converteix en castings normals. En runtime no hi ha genèrics, així que no hi ha cost extra. És només sucre sintàctic en compilació.

> ❓ **Wildcard o paràmetre de tipus? Quan usi cada un?**

Usa el **paràmetre de tipus <T>** quan necessites el tipus més d'una vegada (tornar-lo, guardar-lo en una variable, dos arguments del mateix tipus). Usa el **wildcard ?** quan només t'importi la col·lecció en si (llegir o escriure sense anomenar el tipus). Si tens dubtes: "necessito anomenar el tipus?" → paràmetre de tipus; "m'és igual quin sigui?" → wildcard.

🎬 Post-Crèdits

La programadora acaba la seva agenda telefònica: un `HashMap<String, String>` amb nom → número, un `TreeMap` que li ordena els contactes alfabèticament i un `getOrDefault` que evita els `null` quan busca algú que no existeix. Tot perfecte… fins que tanca el programa i la seva agenda desapareix.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Has vist? Tot viu en memòria RAM. Tanques l'app i adéu, agenda. Un mapa és fantàstic per a guardar dades mentre el programa corre, però perquè sobrevisqui al reinici necessites una altra cosa.

Programadora: — Guardar-ho en un fitxer? Com un fitxer de text en el disc?

CONRAD: — *assenteix* Exacte. Fitxers, serialització, i quan les dades creixin, expressions regulars per a buscar patrons. Tot això és una altra història… i quasi et toca.

PRÒXIMAMENT EN U14: Fitxers i expressions regulars (Regex). Guarda els teus mapes i les teves llistes en disc i busca patrons amb expressions regulars. 📁

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · TreeMap i com triar estructura · **Següent:** U13 · Programació funcional (Streams i Lambdas)

U13 — Programació funcional (Streams i Lambdas)

La cinta transportadora que processa les teves col·leccions i la fletxa que viatja sola: lambdes, interfícies funcionals, streams, Collectors i Optional 🎯

En la U12 vas veure col·leccions, genèrics i mapes: les teves dades per fi viuen en estructures que el compilador controla i que s'associen amb elegància. Però hi havia una manera de recórrer-les: bucles, bucles i més bucles. I si et digués que hi ha una forma de dir-li a Java **QUÈ** vols aconseguir i que ell s'encarregue del **COM**?

Benvingut o benvinguda a la programació funcional. Aquesta unitat té tres grans actes:

- **Lambdes i interfícies funcionals:** la fletxa `->` i els quatre vestits oficials (`Predicate`, `Function`, `Consumer`, `Supplier`). Les funcions deixen de ser només blocs de codi i es converteixen en valors que viatgen d'un lloc a un altre com un missatge dins d'una botella.
- **Streams:** la cinta transportadora que recorre les teves col·leccions sense escriure un sol bucle. `filter`, `map`, `distinct`, `sorted`, `limit`, `collect`, `reduce`… declares què vols i Java s'encarrega del com. La peresa mai no va ser tan productiva: les operacions intermèdies no fan res fins que obris l'aixeta.
- **Optional i Collectors:** la capsa que accepta l'absència en comptes d'un `null` a seques, i la maleta que converteix streams en llistes, conjunts i mapes (`toList()`, `groupingBy`, `toMap`).

Pel camí entendràs per què les operacions intermèdies són peresoses, per què un `Stream` només es recorre una vegada, i per què `Optional.of(null)` t'explotarà a la cara (una vegada. Només en necessites una).

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar què és una **expressió lambda** i escriure la sintaxi `(x) -> ...` sense por.

- Reconèixer les **interfícies funcionals** de `java.util.function`: `Predicate`, `Function`, `Consumer` i `Supplier`.
- Usar **referències a mètodes** (`::`) per a escriure el mateix codi amb menys soroll.
- Crear un **`Stream`** a partir d'una col·lecció amb `stream()` o des de zero amb `Stream.of`.
- Encadenar **operacions intermèdies** (`filter`, `map`, `distinct`, `sorted`, `limit`) sense executar res.
- Tancar el pipeline amb **operacions terminals** (`collect`, `reduce`, `count`, `forEach`).
- Convertir streams en col·leccions amb **`toList()`** i **`Collectors`** (`groupingBy`, `toMap`, `joining`).
- Usar **`Optional`** per a gestionar l'absència de valor i per què és millor que un `null` a seques.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Lambdes: funcions que viatgen soles	La fletxa `->`, la sintaxi i guardar funcions en variables	Tots
02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier	Els quatre vestits oficials de tota lambda i `@FunctionalInterface`	Tots
03 · Referències a mètodes: la drecera `::`	`String::toUpperCase`: reciclar mètodes com si fossin lambdes	Tots
04 · Streams: el flux de dades	Crear streams amb `stream()` i `Stream.of`, i el pipeline	Tots
05 · Operacions intermèdies: filter, map, distinct, sorted, limit	La cinta que filtra, transforma, lleva repetits, ordena i limita	Tots
06 · Operacions terminals: collect, reduce, count, forEach	L'aixeta que fa que l'aigua flueixi de veritat	Tots
07 · Del stream a la col·lecció: toList, Collectors, groupingBy	`toList()`, `Collectors.toMap` i agrupar com en una taula	Tots

Punt	Què aprendràs	Dificultat
08 · Optional: el que accepta l'absència	Embolicar el `null` per a no ensopegar-hi	Tots
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: en aquesta unitat el per-resoldre va primer. Intenta-ho de veritat i només mira el resolt quan hagis suat una mica.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA6)

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA6 b)	S'han reconegut les llibreries de classes relacionades amb tipus avançats de dades.	✅ Punts 1, 2, 4, 7 i 8
RA6 c)	S'han utilitzat llistes per a emmagatzemar i processar informació.	✅ Punts 4, 5, 6 i 7

> 📌 La **RA6** es completa amb la **U06** (arrays, CE a), la **U12** (col·leccions, genèrics i mapes, CEs b, c, d, e i f) i la **U14** (regex, CE g). Aquesta unitat afegeix la capa de **programació funcional**: processa la informació de les col·leccions amb lambdes, streams i `Optional`, reforçant els CEs b i c des de l'enfocament declaratiu.

✅ Criteris d'avaluació coberts (RA7)

RA7: Desenvolupa programes, aplicant característiques avançades dels llenguatges orientats a objectes i de l'entorn de programació.

CE	Criteri	On es cobreix
RA7 a)–g)	Herència, superclasse, subclasse, modificadors, constructors, sobrescriptura i jerarquies de classes.	🔁 Es va cobrir en la **U11**
RA7 h)	S'ha comentat i documentat el codi.	✅ Repàs interactiu i butlletins

> 📌 La **RA7** es recolza en la **U11** (herència, CEs a–g). Aquesta unitat la desenvolupa en la seva vessant de **característiques avançades del llenguatge**: lambdes, interfícies funcionals, streams i `Optional`, els paquets `java.util.function` i `java.util.stream`.

🚪 Per on començo?

- Vens de la U12 i vols l'essencial? → Comença en el punt 1 i el punt 2: les funcions que viatgen soles.
- Ja escrius lambdes i vols fluxos? → Ves directe al punt 4 i al punt 5: la cinta transportadora.
- Només vens a pels trucs? → Salta al punt 3 (la drecera `::`) i al punt 8 (la capsa que accepta l'absència).
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Lambdes: funcions que viatgen soles

⏭️ **En acabar la unitat, continua en U14 · Fitxers i expressions regulars (Regex).**

01 · Lambdes: funcions que viatgen soles

La fletxa `→`, la sintaxi de les lambdes i la primera vegada que guardes una funció en una variable 🏹

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 01 · Lambdes: funcions que viatgen soles

📬 La idea en una frase

> **Una lambda és una funció sense nom que pots tractar com un valor: guardar-la en una variable, passar-la a un mètode o tornar-la. És la resposta de Java a "vull passar-li a aquest mètode un comportament, no només dades".**

En la U12 vas veure les col·leccions, els genèrics i els mapes. Tot genial. Però per a recórrer-les sempre hi havia una constant: els bucles i els mètodes un darrere l'altre. Avui canvia la jugada: les funcions deixen d'estar encadenades a una classe i comencen a **viatjar soles**.

😱 La forma antiga: classes anònimes

Imagina que vols passar-li a alguna cosa "l'acció d'imprimir una salutació". Abans de les lambdes, Java només et deixava fer-ho amb una **classe anònima**: una classe sense nom que defines sobre la marxa, només per a implementar un mètode i regalar-lo a un altre lloc.

```
Runnable tarea = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hola");
    }
};
```

Fixa't en l'espectacle: per a dir "imprimeix Hola" necessites `new Runnable()`, una classe anònima, `@Override`, `public void run()`… sis línies per a una idea d'una. Java sempre ha estat molt d'anar-se'n per les branques.

🏹 Arriba la fletxa: la lambda

Una **expressió lambda** (del grec lambda, i no, no té res a veure amb els lámbada de la sèrie *Futurama*... o sí, perquè els dos són raros) és exactament aquella idea en una línia:

```
Runnable tarea = () -> System.out.println("Hola");
```

Es llegeix: "sense paràmetres, fes `System.out.println("Hola")`". La fletxa `->` separa **els paràmetres** (a l'esquerra) del **cos** (a la dreta). És com dir: "amb aquestes dades, fes això".

La sintaxi té diverses cares:

```
() -> System.out.println("Hola");        // sense paràmetres
(x) -> x * 2;                            // un paràmetre amb parèntesis
x -> x % 2 == 0;                               // un paràmetre SENSE parèntesis
(permés)
(a, b) -> a + b;                         // dos paràmetres
nombre -> {                              // cos amb bloc: diverses sentències
    String saludo = "Hola, " + nombre;
    System.out.println(saludo);
}
```

Regles d'or:

- **Un sol paràmetre** → pots llevar els parèntesis: `x -> x * 2`.
- **El tipus s'infereix** → no escrius `(Integer x) -> ...` a menys que et vegi de gust.
- **Cos d'una sola expressió** → no necessita `return` ni claus.
- **Cos amb bloc** → necessites `{ }` i `return` si tornes alguna cosa.

📦 Funcions que viatgen en variables

Aquí ve la part que et farà volar el cap: la lambda no necessita pertànyer a cap classe concreta. Només necessita una **interfície funcional**: una interfície amb un únic mètode abstracte (en el punt 2 ho aprofundim). Amb això, la funció viatja com un valor.

```
interface Transformador {
    int transformar(int x);
}

public class Demo {
    public static void main(String[] args) {
        Transformador doble = x -> x * 2;           // guarde una funció en una
variable
        Transformador sumaUno = x -> x + 1;         // i una altra

        System.out.println(doble.transformar(5));   // 10
        System.out.println(sumaUno.transformar(5)); // 6
    }
}
```

`doble` no és un número ni un String: és **una funció**. L'has ficada en una variable com qui guarda un llapis a la butxaca. I pots passar-la a altres mètodes, guardar-la en llistes... ja no ets tu qui crida la funció: la funció viatja sola fins on calgui.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** una lambda amb un sol paràmetre pot llevar-se els parèntesis, però amb dos o més (o cap) no hi ha drecera: `(a, b) -> ...` i `() -> ...`.

Exercici: el comptador de viatges

```
interface Viajera {
    String viajar(String destino);
}

public class Demo {
    public static void main(String[] args) {
        Viajera saludo = destino -> "¡Bienvenida a " + destino + "!";
        System.out.println(saludo.viajar("Valencia"));
    }
}
```

Què imprimeix?

- (A) `¡Bienvenida a Valencia!`

- (B) `saludo@1234abcd`
- (C) Error de compilació: les lambdes no poden tornar Strings.
- (D) `destino`

> 🔄 **Solució**
>
> La **A**. La lambda `destino -> "¡Bienvenida a " + destino + "!"` té un cos que és una sola expressió, així que el resultat d'aquella expressió és el que torna la interfície. No cal `return` ni claus: una expressió → torna. `saludo.viajar("Valencia")` executa la lambda amb `destino = "Valencia"` i obtens la frase completa. Les lambdes no són objectes rars que s'imprimeixen com `saludo@...`: quan les crides, executen el seu cos.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa la fletxa `->` en una lambda?
2. Quan pots ometre els parèntesis dels paràmetres?
3. Quin tipus necessita la variable que guarda una lambda?
4. Quin és l'avantatge d'una lambda davant d'una classe anònima?

> 🔄 **Respostes**
>
> 1. Separa els paràmetres (esquerra) del cos (dreta): "amb aquestes dades, fes això".
> 2. Quan hi ha exactament un paràmetre: `x -> x * 2`. El tipus s'infereix sol.
> 3. Una interfície funcional: una interfície amb un únic mètode abstracte.
> 4. Menys soroll: la mateixa idea en una línia, sense `new Interfaz() { ... }` ni `@Override`.

✅ Resum en 3 frases

1. Una **lambda** és una funció sense nom que s'escriu amb la fletxa `->` i es pot tractar com un valor: `(x) -> x * 2`.
2. S'usa on abans calia una **classe anònima**: menys claus, menys soroll, exactament la mateixa idea.

3. Per a guardar-la en una variable necessites una **interfície funcional**: una interfície amb un únic mètode abstracte.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Lambda | Funció sense nom, escrita amb `->` |
> | Fletxa `->` | Separa paràmetres de cos en una lambda |
> | Classe anònima | Classe sense nom definida sobre la marxa (la forma antiga) |
> | Interfície funcional | Interfície amb un únic mètode abstracte |
> | Variable funcional | Variable el tipus de la qual és una interfície funcional i guarda una lambda |

📚 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier

02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier

Els quatre vestits oficials de tota lambda: què espera cada un i com llegir la seva firma 🎭

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier

---

📬 La idea en una frase

> **Una interfície funcional és una interfície amb un únic mètode abstracte. Java ja et regala les quatre més usades en `java.util.function`: `Predicate` (comprova), `Function` (transforma), `Consumer` (consumeix) i `Supplier` (fabrica).**

En el punt 1 vas veure que per a guardar una lambda en una variable necessites una interfície funcional. Avui et presente els quatre vestits oficials que Java et dona fets. Ja no cal inventar-se interfícies pròpies com `Transformador`: l'armari de `java.util.function` està ple.

🎭 Què és una interfície funcional?

És una interfície amb **exactament un mètode abstracte**. Aquell mètode és la firma que tota lambda ha de complir: si la interfície espera `boolean test(int)`, la teva lambda ha de ser una cosa que rebi un `int` i torni un `boolean`.

Perquè el compilador et faci una mà, existeix l'anotació `@FunctionalInterface`: si algú intenta afegir un segon mètode abstracte, el compilador munta en còlera.

```
@FunctionalInterface
interface Transformador {
    int transformar(int x);
    // int otroMetodo();  // 🚫 Error de compilació: només un mètode abstracte
}
```

> ⚠️ **Advertència:** `@FunctionalInterface` és opcional. La interfície funciona igual sense ella; l'anotació només serveix perquè el compilador verifiqui que segueixes les regles. Com un cinturó de seguretat: ningú t'obliga, però és bona idea.

🎭 Els quatre vestits oficials

1. `Predicate<T>` — el jutge

Comprova si alguna cosa compleix una condició. El seu mètode abstracte: **`boolean test(T t)`**.

```
import java.util.function.Predicate;

Predicate<Integer> esPar = n -> n % 2 == 0;
System.out.println(esPar.test(4));   // true
System.out.println(esPar.test(7));   // false
```

2. `Function<T, R>` — el transformador

Rep una cosa de tipus `T` i torna una altra de tipus `R`. El seu mètode abstracte: **`R apply(T t)`**.

```
import java.util.function.Function;

Function<String, Integer> longitud = s -> s.length();
System.out.println(longitud.apply("java"));   // 4

Function<String, String> gritando = s -> s.toUpperCase();
System.out.println(gritando.apply("hola"));   // HOLA
```

3. `Consumer<T>` — el que consumeix sense tornar

Fa alguna cosa amb el valor i no torna res. El seu mètode abstracte: **`void accept(T t)`**.

```
import java.util.function.Consumer;

Consumer<String> imprimir = s -> System.out.println(s);
imprimir.accept("¡Hola!");
```

4. `Supplier<T>` — el fabricant

No rep res i produeix un valor. El seu mètode abstracte: **`T get()`**.

```java
import java.util.function.Supplier;

Supplier<String> fabricaSaludos = () -> "¡Hola!";
System.out.println(fabricaSaludos.get());   // ¡Hola!
```

📋 La xurrera de les quatre

Interfície	Mètode abstracte	Què fa	Lambda típica
`Predicate<T>`	`boolean test(T)`	Comprova una condició	`n -> n % 2 == 0`
`Function<T, R>`	`R apply(T)`	Transforma un valor en un altre	`s -> s.length()`
`Consumer<T>`	`void accept(T)`	Consumeix sense tornar	`s -> System.out.println(s)`
`Supplier<T>`	`T get()`	Fabrica un valor sense entrada	`() -> new ArrayList<>()`

Fixa't en el patró per a llegir-los sense por: el nom et diu el paper (**pre**dica, **fun**ció, **con**sumeix, **sup**lix) i la firma et diu el contracte. Quan en els punts següents vegis `filter(n -> ...)`, `map(n -> ...)`, `forEach(...)` o `orElseGet(...)`, tots estaran demanant un d'aquests vestits per darrere.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan una lambda no encaixa en `Predicate`, `Function`, `Consumer` o `Supplier`, Java té desenes de variants (`BiFunction`, `IntFunction`, `UnaryOperator`…). Comença sempre per les quatre bàsiques i amplia només si cal.

Exercici: el porter de discoteca (versió funcional)

```java
import java.util.function.*;

public class Portero {
    public static void main(String[] args) {
        Predicate<Integer> mayorDeEdad = edad -> edad >= 18;
          Function<Integer, String> etiqueta = edad -> edad >= 18 ? "ADULTO" :
"MENOR";
        Consumer<String> anunciar = s -> System.out.println("Entrada: " + s);
        Supplier<Integer> reloj = () -> 20;

        int edad = reloj.get();
        if (mayorDeEdad.test(edad)) {
            anunciar.accept(etiqueta.apply(edad));
        }
    }
}
```

Què imprimeix?

- (A) `Entrada: ADULTO`
- (B) `Entrada: MENOR`
- (C) Res: l'`if` mai no es compleix.
- (D) Error de compilació: no es poden combinar quatre lambdes així.

> 🔄 **Solució**
>
> La **A**. `reloj.get()` torna `20`, `mayorDeEdad.test(20)` és `true` (20 ≥ 18), i dins de l'`if` s'encadenen les altres dos: `etiqueta.apply(20)` torna `"ADULTO"` i `anunciar.accept("ADULTO")` imprimeix `Entrada: ADULTO`. Fixa't en com cada vestit fa el seu paper: el `Supplier` fabrica la dada, el `Predicate` decideix, la `Function` transforma i el `Consumer` actua. És el ball complet de `java.util.function`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode abstracte té `Predicate<T>` i què torna?
2. Què torna `Function<T, R>`?

3. Per a què serveix `Consumer<T>`?
4. I `Supplier<T>`?

> 🔄 **Respostes**
>
> 1. `boolean test(T)` — torna `true` o `false` segons una condició.
> 2. `R apply(T)` — transforma un valor de tipus `T` en un altre de tipus `R`.
> 3. Per a accions que no tornen res: `void accept(T)`. Ideal per a imprimir, guardar, enviar…
> 4. Produeix un valor sense rebre paràmetres: `T get()`. El proveïdor peresós.

✅ Resum en 3 frases

1. Una **interfície funcional** és una interfície amb un únic mètode abstracte: és la firma que tota lambda ha de complir.
2. Els quatre vestits oficials de `java.util.function` són **`Predicate`** (`test`), **`Function`** (`apply`), **`Consumer`** (`accept`) i **`Supplier`** (`get`).
3. L'anotació **`@FunctionalInterface`** li demana al compilador que verifiqui que la interfície és funcional: un únic mètode abstracte i res més.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Interfície funcional	Interfície amb un únic mètode abstracte
`Predicate<T>`	Comprova una condició: `boolean test(T)`
`Function<T, R>`	Transforma un valor: `R apply(T)`
`Consumer<T>`	Consumeix sense tornar: `void accept(T)`
`Supplier<T>`	Fabrica un valor: `T get()`
`@FunctionalInterface`	Anotació que valida que la interfície és funcional

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Lambdes: funcions que viatgen soles · **Següent:** 03 · Referències a mètodes: la drecera `::`

03 · Referències a mètodes: la drecera `::`

Reciclar mètodes ja escrits com si fossin lambdes amb l'operador `::` 🩹

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 03 · Referències a mètodes: la drecera `::`

📬 La idea en una frase

> **Quan la teva lambda només crida un mètode que ja existeix, pots escriure `::` en comptes de `->`: `String::toUpperCase` significa "el mètode `toUpperCase` dels Strings", llest per a usar-se com una lambda.**

En el punt 1 vas veure les lambdes i en el punt 2 els quatre vestits oficials. Avui aprens a vestir-los sense escriure ni una fletxa: si el cos de la teva lambda és literalment "crida a aquest mètode", Java et deixa reciclar aquell mètode directament amb l'operador `::` (dos punts, dos punts, no són punts suspensius).

🩹 Quan la lambda és només una crida

Imagina que vols una `Function<String, String>` que posi un text en majúscules. Amb lambda:

```
Function<String, String> mayusculas = s -> s.toUpperCase();
```

Aquella lambda fa una sola cosa: cridar a `toUpperCase()`. Java pensa: "escolta, ja existeix aquell mètode, per a què escric la fletxa?". La versió amb `::`:

```
Function<String, String> mayusculas = String::toUpperCase;
```

`String::toUpperCase` es llegeix: "el mètode `toUpperCase` de la classe `String`". I funciona exactament igual que la lambda: quan li passes un String, el posarà en majúscules.

```
System.out.println(mayusculas.apply("hola"));   // HOLA
```

🗂️ Els quatre tipus de referència

1. Mètode estàtic: `Clase::metodoEstatico`

```
Function<Integer, Integer> valorAbsoluto = Math::abs;
System.out.println(valorAbsoluto.apply(-7));   // 7
```

Equival a `x -> Math.abs(x)`.

2. Mètode d'instància sobre un objecte qualsevol: `Clase::metodoInstancia`

```
Function<String, Integer> longitud = String::length;
System.out.println(longitud.apply("java"));   // 4
```

Equival a `s -> s.length()`. El paràmetre de la lambda es converteix en l'objecte sobre el qual cridem.

3. Mètode d'instància sobre un objecte concret: `objeto::metodo`

```
Consumer<String> imprimir = System.out::println;
imprimir.accept("¡Hola!");
```

Equival a `s -> System.out.println(s)`. Aquí l'objecte ja existeix (`System.out`) i el paràmetre de la lambda és l'argument del mètode.

4. Constructor: `Clase::new`

```
Supplier<ArrayList<String>> fabrica = ArrayList::new;
ArrayList<String> lista = fabrica.get();
```

Equival a `() -> new ArrayList<>()`. El constructor també és un mètode, i també es pot reciclar.

📋 La taula del : :

Referència	Sintaxi	Equival a
Estàtica	`Math::abs`	`x -> Math.abs(x)`
D'instància (classe)	`String::toUpperCase`	`s -> s.toUpperCase()`
D'instància (objecte)	`System.out::println`	`s -> System.out.println(s)`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

> ⚠️ **Advertència:** la referència a mètode no executa el mètode. `String::toUpperCase` **no** posa res en majúscules: és la recepta, no el menjar. S'executarà quan el stream o la interfície funcional ho demani.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** quan dubtes entre lambda i referència, pensa en la llegibilitat. `String::toUpperCase` és més net que `s -> s.toUpperCase()`, però una lambda amb lògica (com `s -> s.length() > 3`) no es pot convertir a `::` perquè no crida un únic mètode.

Exercici: el reciclador de salutacions

```
import java.util.function.*;

public class Reciclador {
    public static void main(String[] args) {
        Supplier<String> fabrica = String::new;
        Function<String, String> mayusculas = String::toUpperCase;
        Consumer<String> imprimir = System.out::println;

        String vacio = fabrica.get();
        imprimir.accept(mayusculas.apply(vacio.isEmpty() ? "hola" : vacio));
    }
}
```

Què imprimeix?

- (A) HOLA

- (B) `hola`
- (C) Res: el `String` està buit.
- (D) Error de compilació: `String::new` no és una referència vàlida.

> 🔄 **Solució**
>
> La **A**. `String::new` és una referència al constructor: `fabrica.get()` crea un `String` buit (`""`). Com que `vacio.isEmpty()` és `true`, el ternari tria `"hola"`, i `mayusculas.apply("hola")` crida a `String::toUpperCase` sobre aquell text, donant `"HOLA"`. El `Consumer` amb `System.out::println` ho imprimeix. Tres referències a mètode en tres rols distints: constructor, mètode d'instància i mètode sobre un objecte concret.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa `String::toUpperCase` sense parèntesis?
2. Quan usaries `objeto::metodo` en comptes de `Clase::metodo`?
3. `Math::max` quin tipus de referència és?
4. Què fa `Clase::new`?

> 🔄 **Respostes**
>
> 1. Una referència al mètode `toUpperCase` de qualsevol `String`, llesta per a cridar-se quan toqui. No executa res per si sola.
> 2. Quan el mètode es crida sobre un objecte concret que ja existeix, com `System.out::println`. `Clase::metodo` serveix per a mètodes que es criden sobre el paràmetre que arribi.
> 3. Referència a un mètode estàtic d'una classe.
> 4. Referència al constructor: fabrica objectes nous, com un `Supplier` (`() -> new Clase()`).

✅ Resum en 3 frases

1. Una **referència a mètode** (`::`) és una lambda encara més curta: reutilitza un mètode que ja existeix en comptes d'escriure la fletxa.

2. Hi ha quatre tipus: **estàtica** (`Math::abs`), **d'instància sobre classe** (`String::toUpperCase`), **d'instància sobre objecte** (`System.out::println`) i **de constructor** (`ArrayList::new`).
3. Quan la teva lambda només crida un mètode, la referència és més llegible i es comporta exactament igual.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | Referència a mètode | Reutilitzar un mètode com a lambda amb `::` |
> | `Clase::estatico` | Referència a un mètode estàtic |
> | `Clase::instancia` | Mètode d'instància cridat sobre el paràmetre que arribi |
> | `objeto::metodo` | Mètode cridat sobre un objecte concret que ja existeix |
> | `Clase::new` | Referència al constructor |

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Interfícies funcionals: Predicate, Function, Consumer, Supplier · **Següent:** 04 · Streams: el flux de dades

04 · Streams: el flux de dades

La cinta transportadora de Java: crear un `Stream`, el pipeline i per què no guarda res 📦

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 04 · Streams: el flux de dades

---

📬 La idea en una frase

> **Un `Stream` no és una col·lecció nova: és una cinta transportadora per on passen els elements d'una font (una llista, un array, diversos valors…) per a processar-los d'una vegada, sense bucles.**

Fins ara, per a treballar amb una llista de la U12 escrivies `for (String s : lista)` i dins feies la màgia. El stream és una altra filosofia: en comptes de dir-li a Java **com** recórrer (bucle, índex, iterador), li dius **què** vols aconseguir i ell munta la cinta. Tu dissenyes el flux; Java mou els paquets.

📦 Què és un Stream (i què NO és)?

Un `Stream<T>` és una **seqüència d'elements** de tipus `T` que es processa d'una vegada. Tres regles que t'estalviaran disgustos:

- **No guarda dades.** La cinta no emmagatzema paquets: les dades continuen vivint en la teva llista o el teu array. El stream només les va passant.
- **És peresós.** Les operacions intermèdies no s'executen fins que apareix una operació terminal (ho veiem en el punt 5).
- **Només es recorre una vegada.** Com una cinta de supermercat: quan l'últim producte cau a la bossa, la cinta es queda buida. Usar-lo dues vegades és impossible.

🚚 Crear un Stream: les quatre fonts

```java
import java.util.*;
import java.util.stream.*;

public class Fuentes {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla");

         Stream<String> s1 = nombres.stream();                              // des
d'una col·lecció
        Stream<Integer> s2 = Stream.of(1, 2, 3);                          // des de
valors solts
         Stream<String> s3 = Arrays.stream(new String[]{"x", "y"});       // des
d'un array
          Stream<Integer> s4 = Stream.iterate(0, n -> n + 1);               //
infinita (¡amb compte!)
    }
}
```

- `lista.stream()` — la font més comuna: qualsevol col·lecció de la U12 té aquest mètode.
- `Stream.of(a, b, c)` — quan vols un flux a partir de diversos valors sense muntar una llista.
- `Arrays.stream(array)` — per a convertir un array de la U06 en flux.
- `Stream.iterate(0, n -> n + 1)` — genera una seqüència infinita. No la tanques amb una terminal que ho recorri tot o el teu programa es dedicarà a comptar fins a l'infinit (i no és una metàfora).

🏭 El pipeline: font → intermèdies → terminal

La forma d'usar un stream és el **pipeline**: una font, zero o més operacions intermèdies, i una operació terminal que tanca l'aixeta.

```java
import java.util.*;
import java.util.stream.*;

public class Pipeline {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");

        long cantidad = nombres.stream()          // 1. font
            .filter(n -> n.length() > 3)          // 2. intermèdia: es queda amb els llargs
            .count();                             // 3. terminal: els compta

        System.out.println(cantidad);             // 3 (carla, david, eva)
    }
}
```

L'operació terminal (`count()`) és la que fa que la cinta arranqui. Sense ella, el stream no mou ni un paquet: les intermèdies només preparen la cinta. És com planejar un viatge: pots dissenyar la ruta tot el que vulguis, però ningú surt fins que l'autobús arranca.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `lista.stream()` et dona un flux, però `List.of(...)` crea llistes **immutables**. Si necessites que el resultat sigui modificable, en el punt 7 veuràs com sortir del stream a una col·lecció que sí puguis tocar.

Exercici: la cinta que arranca

```java
import java.util.*;
import java.util.stream.*;

public class Cinta {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(4, 8, 15, 16, 23, 42);
        Stream<Integer> flujo = numeros.stream()
            .filter(n -> n % 2 == 0)
            .map(n -> n * 10);
        System.out.println("Cinta preparada");
    }
}
```

Què imprimeix?

- (A) `Cinta preparada`
- (B) `Cinta preparada` i després `40 80 160 420`
- (C) `40 80 160 420`
- (D) Res: el programa es queda penjat esperant la cinta.

> 🔄 **Solució**
>
> La **A**. El pipeline acaba amb `map` (una operació intermèdia), així que **no hi ha operació terminal**: la cinta es prepara però mai no arranca. `filter` i `map` no executen res fins que arriba un `collect`, `count`, `forEach` o similar. És la peresa del stream en estat pur: prepares la ruta, i ningú es puja a l'autobús. La B i la C serien certes si afegires `forEach(System.out::println)` o un `toList()` al final. I el programa no es penja: simplement no imprimeix números.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Un `Stream` guarda dades?
2. Quantes vegades pots recórrer un `Stream`?
3. Com crees un stream des d'una llista?
4. I des de diversos valors solts?

> 🔄 **Respostes**
>
> 1. No. És una vista/flux: les dades continuen en la col·lecció original.
> 2. Una sola vegada. Després d'una operació terminal, el stream queda consumit.
> 3. Amb `lista.stream()`.
> 4. Amb `Stream.of(valor1, valor2, ...)`.

✅ Resum en 3 frases

1. Un **`Stream`** és un flux d'elements, no un contenidor: no guarda res, és una cinta per on passen les dades.
2. Es crea amb `coleccion.stream()`, `Stream.of(...)`, `Arrays.stream(array)` o `Stream.iterate`, i s'usa en un **pipeline** de font → intermèdies → terminal.
3. El stream és **peresós i d'un sol ús**: sense operació terminal no fa res, i una vegada consumit no es pot reutilitzar.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Stream	Flux de dades que es processa d'una vegada
Pipeline	Cadena font → intermèdies → terminal
Font	D'on surten els elements (col·lecció, array, `Stream.of`)
Peresa	Les operacions intermèdies no s'executen fins a la terminal
Consumible	Un stream només es recorre una vegada

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Referències a mètodes: la drecera `::` · **Següent:** 05 · Operacions intermèdies: filter, map, distinct, sorted, limit

05 · Operacions intermèdies: filter, map, distinct, sorted, limit

La cinta que filtra, transforma, lleva repetits, ordena i limita el flux 🌀

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 05 · Operacions intermèdies: filter, map, distinct, sorted, limit

---

📬 La idea en una frase

> **Les operacions intermèdies transformen el flux i tornen un altre `Stream`. No fan res per si soles (són peresoses) i no toquen la col·lecció original: només decideixen com serà la cinta quan arranqui.**

Ja tens la cinta del punt 4. Avui li poses les estacions: `filter` decideix quins paquets passen, `map` els reetiqueta, `distinct` treu els duplicats, `sorted` els ordena i `limit` talla el flux. Totes tornen un stream nou, totes són peresoses i cap modifica la teva llista original. Això sí: **l'ordre de les estacions importa**.

🌀 Les cinc estacions

`filter(Predicate)` — el porter

Deixa passar només els elements que compleixen la condició. Necessita un `Predicate` (punt 2).

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);
List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)      // 2, 4, 6
    .toList();
```

`map(Function)` — el transformador

Transforma cada element. Necessita una `Function` (punt 2). Canvia el tipus de la cinta: `Stream<Integer>` → `Stream<String>` si vols.

```java
List<String> palabras = List.of("hola", "java");
List<Integer> longitudes = palabras.stream()
    .map(String::length)          // 4, 4
    .toList();
```

`distinct()` — el que lleva repetits

Elimina duplicats usant `equals()`.

```java
List<String> repetidas = List.of("sol", "mar", "sol", "luna");
List<String> unicas = repetidas.stream()
    .distinct()                   // sol, mar, luna
    .toList();
```

`sorted()` — el que ordena

Ordena segons l'ordre natural (amb `Comparable`). Si vols un altre criteri, usa `sorted(Comparator)`.

```java
List<Integer> desordenados = List.of(5, 2, 8, 1);
List<Integer> ordenados = desordenados.stream()
    .sorted()                     // 1, 2, 5, 8
    .toList();
```

`limit(n)` — les tisores

Es queda amb els n primers elements del flux.

```java
List<Integer> numeros = List.of(10, 20, 30, 40, 50);
List<Integer> tresPrimeros = numeros.stream()
    .limit(3)                     // 10, 20, 30
    .toList();
```

🏭 Les estacions en cadena

Allò bonic és que s'encadenen: cada operació rep el flux de l'anterior i li passa el seu a la següent. Un exemple complet:

```
import java.util.*;
import java.util.stream.*;

public class Cadena {
    public static void main(String[] args) {
             List<String>  palabras  =  List.of("java",  "hola",  "java",  "mundo",
"adios", "luna");

        palabras.stream()
            .filter(p -> p.length() > 3)      // java, hola, java, mundo, adios,
luna
            .map(String::toUpperCase)         // JAVA, HOLA, JAVA, MUNDO, ADIOS,
LUNA
            .distinct()                       // JAVA, HOLA, MUNDO, ADIOS, LUNA
            .sorted()                         // ADIOS, HOLA, JAVA, LUNA, MUNDO
            .limit(3)                         // ADIOS, HOLA, JAVA
            .forEach(System.out::println);    // terminal: imprimeix les tres
    }
}
```

Sortida:

```
ADIOS
HOLA
JAVA
```

Fixa't en la **importància de l'ordre**: si poses `limit(3)` abans de `sorted()`, et quedes amb els tres primers en l'ordre d'arribada i després els ordenes; el resultat canvia. Dissenyar el pipeline és decidir en quin ordre passa cada paquet per cada estació.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `map` pot canviar el tipus del stream (`Stream<String>` → `Stream<Integer>`), però `filter` sempre torna el mateix tipus que rep: només lleva elements, no els transforma.

Exercici: l'estació que sobra

```java
import java.util.*;
import java.util.stream.*;

public class Estaciones {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");
        List<String> resultado = nombres.stream()
            .sorted()
            .filter(n -> n.length() == 3)
            .limit(1)
            .toList();
        System.out.println(resultado);
    }
}
```

Què imprimeix?

- (A) `[ana]`
- (B) `[bob]`
- (C) `[ana, bob, eva]`
- (D) `[eva]`

> 🔄 **Solució**
>
> La **B**. Primer `sorted()` ordena: `ana, bob, carla, david, eva`. Després `filter(n -> n.length() == 3)` deixa només els de 3 lletres: `ana, bob, eva`. I `limit(1)` es queda amb el primer del flux filtrat, que ara és `bob` (perquè després d'ordenar, `bob` va abans que `eva`). Sense el `sorted`, el resultat hauria estat `[ana]`; l'ordre de les estacions canvia la resposta. Fixa't com `filter` i `limit` no toquen el tipus: continuem amb `List<String>`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. `filter` deixa passar els elements que compleixen la condició o els que no?
2. `map` modifica la col·lecció original?
3. Què fa `distinct`?
4. Què fa `limit(3)`?

> 🔄 **Respostes**
>
> 1. Els que fan que el `Predicate` torni `true`.
> 2. No. Torna un stream nou; la col·lecció original no es toca.
> 3. Elimina duplicats usant `equals()`.
> 4. Tall el flux i es queda amb els 3 primers elements (el seu efecte depèn de si va abans o després de `sorted`).

✅ Resum en 3 frases

1. Les **operacions intermèdies** transformen el flux i tornen un altre `Stream`: `filter`, `map`, `distinct`, `sorted` i `limit`.
2. Són **peresoses**: no s'executen fins que arriba una operació terminal, i **no modifiquen** la font.
3. L'**ordre de les estacions importa**: el mateix pipeline amb les mateixes operacions en un altre ordre pot donar resultats diferents.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Intermèdia	Operació que torna un altre `Stream`
`filter`	Deixa passar el que compleix el `Predicate`
`map`	Transforma cada element amb una `Function`
`distinct`	Elimina repetits (amb `equals`)
`sorted`	Ordena (ordre natural o amb `Comparator`)
`limit(n)`	Tall el flux en n elements

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Streams: el flux de dades · **Següent:** 06 · Operacions terminals: collect, reduce, count, forEach

06 · Operacions terminals: collect, reduce, count, forEach

L'aixeta que fa que l'aigua flueixi de veritat: `collect`, `reduce`, `count` i `forEach` 🚰

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 06 · Operacions terminals: collect, reduce, count, forEach

---

📬 La idea en una frase

> **Les operacions terminals són l'aixeta del pipeline: sense una d'elles, el stream no flueix. `collect` empaqueta el resultat, `reduce` combina tots els elements en un, `count` els compta i `forEach` els recorre.**

En el punt 4 vas veure que el stream és peresós i en el punt 5 que les intermèdies només preparen la cinta. Avui obris l'aixeta. Cada terminal produeix un resultat final i, en fer-ho, **consumeix el stream**: després de `count()`, `reduce()` o `forEach()`, la cinta queda buida i no es pot reutilitzar.

🚰 Les quatre terminals

`collect(Collector)` — la que empaqueta

Recull els elements del flux en una estructura. La més famosa: `Collectors.toList()`. És el pont entre el stream i les col·leccions de la U12 (en el punt 7 l'espremem a fons).

```
import java.util.*;
import java.util.stream.*;

List<Integer> numeros = List.of(4, 8, 2, 9, 5);
List<Integer> lista = numeros.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());    // [4, 8, 2]
```

`reduce(identitat, acumulador)` — la que combina

Va combinant els elements en un de sol. El primer argument és la **identitat** (el valor neutre: `0` per a sumes, `1` per a productes), i el segon és un `BinaryOperator` que diu com combinar dos elements.

```
int suma = numeros.stream().reduce(0, (a, b) -> a + b);      // 28
int maximo = numeros.stream().reduce(Integer.MIN_VALUE, Math::max);  // 9
```

`reduce(0, (a, b) -> a + b)` es llegeix: "comença en 0 i ve sumant cada element a l'acumulador". És com un bucle `for` on l'acumulador viatja d'element en element… sense bucle.

`count()` — la que compta

Torna el nombre d'elements com a `long`.

```
long pares = numeros.stream().filter(n -> n % 2 == 0).count();   // 3
```

`forEach(Consumer)` — la que recorre

Aplica un `Consumer` a cada element. És la terminal "imperativa": l'única que s'assembla a un bucle, pensada per a efectes secundaris (imprimir, guardar…).

```
numeros.stream().forEach(System.out::println);   // imprimeix tots
```

> ⚠️ **Advertència:** `forEach` amb un stream en paral·lel no garanteix ordre. Per al curs, usa'l en streams normals i no et preocupes; però recorda que si l'ordre importa, és més segur `collect` i després recórrer.

🏭 El pipeline complet

Junta el dels dos punts anteriors:

```java
import java.util.*;
import java.util.stream.*;

public class Notas {
    public static void main(String[] args) {
        List<Integer> notas = List.of(7, 8, 3, 9, 5);

        long aprobados = notas.stream()
            .filter(n -> n >= 5)
            .count();                                   // 4

        int sumaAprobados = notas.stream()
            .filter(n -> n >= 5)
            .reduce(0, Integer::sum);                   // 29

        List<Integer> dobladas = notas.stream()
            .filter(n -> n >= 5)
            .map(n -> n * 2)
            .collect(Collectors.toList());              // [14, 16, 18, 10]

        System.out.println(aprobados + " " + sumaAprobados + " " + dobladas);
    }
}
```

Sortida: `4 29 [14, 16, 18, 10]`. Tres pipelines, tres aixetes, tres resultats. Cada vegada que obris una, el stream es consumeix i has de crear-ne una altra.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `Integer::sum` és una referència a mètode (punt 3) perfecta per a `reduce`: és un `BinaryOperator<Integer>` que suma dos enters.

Exercici: l'acumulador

```java
import java.util.*;
import java.util.stream.*;

public class Acumulador {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);
        int total = numeros.stream()
            .reduce(10, (a, b) -> a * b);
        System.out.println(total);
    }
}
```

Què imprimeix?

- (A) `120`
- (B) `15`
- (C) `1200`
- (D) `10`

> 🔄 **Solució**
>
> La **C**. `reduce(10, (a, b) -> a * b)` comença en la identitat `10` i va multiplicant per cada element: `10 * 1 = 10`, `10 * 2 = 20`, `20 * 3 = 60`, `60 * 4 = 240`, `240 * 5 = 1200`. La identitat NO és "el primer element": és el valor amb què arranca l'acumulador. Per això la B (`15`) seria el resultat de `reduce(1, ...)` sense multiplicar per la identitat, i la A (`120`) la trampa de creure que la identitat s'ignora. La identitat és un element més del càlcul: `10` compta.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `count()`?
2. Per a què serveix el primer argument de `reduce(0, ...)`?
3. Què fa `forEach`?
4. Pots reutilitzar el mateix stream després d'una operació terminal?

> 🔄 **Respostes**

1. Un `long` amb el nombre d'elements.
2. És la **identitat**: el valor amb què arranca l'acumulador (el neutre de l'operació: `0` per a sumes, `1` per a productes).
3. Aplica un `Consumer` a cada element: la terminal per a efectes secundaris com imprimir.
4. No. Després d'una terminal, el stream queda consumit.

✅ Resum en 3 frases

1. Les **operacions terminals** són l'aixeta del pipeline: sense elles, res no flueix.
2. `collect` empaqueta el resultat, `reduce` combina els elements en un (amb una identitat), `count` els compta i `forEach` els recorre.
3. Després d'una terminal, el stream es **consumeix**: si necessites processar de nou, crea un altre stream.

🐛 Vocabulari ràpid

Terme	Idea general
Terminal	Operació que tanca el pipeline i produeix un resultat
`collect`	Recull el flux en una estructura (p. ex. llista)
`reduce`	Combina tots els elements en un (suma, màxim…)
Identitat	Valor inicial del `reduce` (neutre: `0` per a la suma)
`count()`	Torna el nombre d'elements (`long`)
`forEach`	Aplica un `Consumer` a cada element

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Operacions intermèdies: filter, map, distinct, sorted, limit · **Següent:** 07 · Del stream a la col·lecció: toList, Collectors, groupingBy

07 · Del stream a la col·lecció: toList, Collectors, groupingBy

`toList()`, `Collectors.toMap` i `groupingBy`: del flux a llistes, conjunts i mapes 🗂️

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 07 · Del stream a la col·lecció: toList, Collectors, groupingBy

---

📬 La idea en una frase

> **Un stream no guarda res, així que al final del pipeline toca aterrar: `toList()` o els `Collectors` converteixen el flux en llistes, conjunts, Strings i, amb `groupingBy`, en mapes agrupats per una clau.**

La cinta transportadora està molt bé per a processar, però al final els paquets han d'anar a algun lloc. La terminal `collect` (del punt 6) és la porta de sortida, i avui coneixes la seva maleta: `Collectors`, la classe amb les receptes per a empaquetar de totes les formes imaginables.

📥 `toList()`: la sortida ràpida (Java 16+)

Des de Java 16 existeix `toList()` directe sobre el stream: la forma més curta d'aterrar en una llista.

```
List<String> palabras = List.of("hola", "mundo", "hola", "adios");
List<String> unicas = palabras.stream()
    .distinct()
    .toList();                      // [hola, mundo, adios]
```

> ⚠️ **Advertència:** `toList()` torna una llista **immutable**: no pots fer `add`, `remove` ni reordenar-la. Si necessites modificar-la, usa `collect(Collectors.toList())`, que torna un `ArrayList` de tota la vida.

🗂️ Els `Collectors` de tota la vida

La classe `Collectors` (en `java.util.stream`) té receptes per a cada tipus d'aterratge:

```
import java.util.*;
import java.util.stream.*;

List<String> palabras = List.of("hola", "mundo", "hola", "adios");

List<String> lista = palabras.stream().collect(Collectors.toList());          //
ArrayList
Set<String> conjunto = palabras.stream().collect(Collectors.toSet());         //
HashSet sense duplicats
String frase = palabras.stream().collect(Collectors.joining(", "));           //
"hola, mundo, hola, adios"
long cuantos = palabras.stream().collect(Collectors.counting());             // 4
```

`joining(", ")` és el favorit per a treure un `String` llegible: uneix els elements amb el separador que li passes. Ideal per a depurar pipelines en dues línies.

🗺️ `groupingBy`: agrupar com en una taula

El rei d'aquest punt. **`Collectors.groupingBy(funció)`** agrupa els elements segons la clau que torna la funció i et dona un `Map<clau, List<valor>>`.

```
import java.util.*;
import java.util.stream.*;

public class Agrupando {
    public static void main(String[] args) {
             List<String> palabras = List.of("hola", "adios", "hola", "mar",
"mundo");

        Map<Character, List<String>> porLetra = palabras.stream()
            .collect(Collectors.groupingBy(p -> p.charAt(0)));

        System.out.println(porLetra);
        // {a=[adios], h=[hola, hola], m=[mar, mundo]}
    }
}
```

És l'equivalent a la sentència SQL `GROUP BY`: en comptes de recórrer a mà i muntar el `HashMap` de la U12 amb `getOrDefault`, declares la clau i Java agrupa. Per dins està usant un `HashMap`, però tu no ho veus: el `Collectors` el munta per tu.

🗺️ `toMap`: d'elements a entrades

`Collectors.toMap(clau, valor)` converteix cada element en una entrada de mapa. Té un tercer argument: la **funció de fusió**, que decideix què fer quan dos elements volen la mateixa clau.

```
List<String> palabras = List.of("hola", "mundo", "hola", "adios");

// paraula → longitud
Map<String, Integer> longitudes = palabras.stream()
    .collect(Collectors.toMap(p -> p, String::length, (a, b) -> a));
// {adios=5, hola=4, mundo=5}

// primera lletra → primera paraula que la complisca
Map<Character, String> representantes = palabras.stream()
    .collect(Collectors.toMap(p -> p.charAt(0), p -> p, (a, b) -> a));
// {a=adios, h=hola, m=mundo}
```

Sense la funció de fusió `(a, b) -> a`, un `"hola"` duplicat provocaria una `IllegalStateException`: dos elements volent la mateixa clau i ningú resolent el conflicte. Amb `(a, b) -> a` li dius "queda't amb el primer".

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** `groupingBy` admet un segon argument, el "collector aigües avall": `groupingBy(clau, Collectors.counting())` et dona `Map<clau, Long>` amb quants elements hi ha en cada grup. El comptador de freqüències de la U12 en una sola línia.

Exercici: el comptador de grups

```java
import java.util.*;
import java.util.stream.*;

public class Grupos {
    public static void main(String[] args) {
           List<String> palabras = List.of("sol", "mar", "sol", "luna", "mar",
"sol");
        Map<String, Long> conteo = palabras.stream()
            .collect(Collectors.groupingBy(p -> p, Collectors.counting()));
        System.out.println(conteo);
    }
}
```

Què imprimeix?

- (A) `{sol=3, mar=2, luna=1}`
- (B) `{sol=1, mar=1, luna=1}`
- (C) `{sol=1, mar=1, sol=1, luna=1, mar=1, sol=1}`
- (D) `{sol=6}`

> 🔄 **Solució**
>
> La **A**. `groupingBy(p -> p, Collectors.counting())` agrupa les paraules per elles mateixes (cada paraula és la seva pròpia clau) i el collector aigües avall `counting()` compta quantes vegades apareix cada una: `sol` apareix 3 vegades, `mar` 2 i `luna` 1. Les claus del mapa són úniques (aquella és la regla d'or dels mapes de la U12), així que la C és impossible: no pot haver-hi dos entrades amb la mateixa clau. La B seria el resultat sense `counting()` (grups d'un element) i la D confon el total amb el grup.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `toList()`?
2. Què fa `Collectors.groupingBy(...)`?
3. Per a què serveix el tercer argument de `toMap`?
4. Què torna `Collectors.joining(", ")`?

> 🔄 **Respostes**
>
> 1. Una `List` immutable amb els elements del stream.
> 2. Agrupa els elements per la clau que torna la funció i dona un `Map<clau, List<valor>>`.
> 3. La **funció de fusió**: resol què fer quan dos elements volen la mateixa clau (per exemple, `(a, b) -> a` es queda amb el primer).
> 4. Un `String` amb els elements units per la coma i un espai.

✅ Resum en 3 frases

1. `toList()` i els `Collectors` converteixen el flux en **col·leccions reals**: llistes, conjunts, Strings i mapes.
2. `groupingBy` agrupa els elements per una clau i torna un `Map<clau, List<valor>>`; amb `counting()` com a segon argument, compta cada grup.
3. `toMap` converteix cada element en una entrada clau-valor i necessita una **funció de fusió** quan pot haver-hi claus repetides.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`toList()`	Recull el stream en una llista immutable (Java 16+)
`Collectors.toList()` / `toSet()`	Recullen en llista o conjunt modificables
`groupingBy`	Agrupa per una clau en un `Map`
Collector aigües avall	Segon argument que processa cada grup (`counting()`)
`toMap`	Converteix cada element en una entrada clau-valor
Fusió (merge)	Funció que resol què fer amb claus repetides
`joining`	Uneix els elements en un `String`

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Operacions terminals: collect, reduce, count, forEach · **Següent:** 08 · Optional: el que accepta l'absència

08 · Optional: el que accepta l'absència

Embolicar el `null` per a no ensopegar-hi: `Optional.of`, `orElse`, `isPresent` i `ifPresent` 🎁

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 08 · Optional: el que accepta l'absència

---

📬 La idea en una frase

> **`Optional<T>` és una capsa que conté un valor o està buida. En comptes de tornar un `null` a seques (que ningú sap si era una dada real o un oblit), tornes la capsa i obligues a qui la rep a preguntar: hi ha valor dins, o no?**

Portes tota la unitat processant dades com un cap. Però hi ha un moment en què tot es torç: quan un mètode no troba el que busca i torna `null`. I tu, innocent, crides a `.length()` o a `.get()` sobre aquell `null`… i la `NullPointerException` et saluda a tota la cara. `Optional` existeix perquè l'absència sigui **explícita** i **gestionable**, no un accident.

🎁 La capsa en acció

Un `Optional<T>` és una capseta que pot estar **plena** (amb un valor) o **buida**. Crear-la és fàcil:

```
import java.util.*;

Optional<String> algo = Optional.of("java");          // capsa PLENA. Prohibit passar null.
Optional<String> vacio = Optional.empty();            // capsa BUIDA.
Optional<String> dudoso = Optional.ofNullable(valor); // accepta null: si és null, capsa buida
```

> ⚠️ **Advertència:** `Optional.of(null)` llança `NullPointerException` a l'instant. És la forma de dir "aquesta capsa NOMÉS admet valors reals". Si no saps si pot arribar `null`, usa `ofNullable`.

🎁 Obrir la capsa amb elegància

La gràcia de la capsa és que no l'obris a la bruta: uses els seus mètodes per a decidir què passa en cada cas.

```
import java.util.*;

public class Caja {
    public static void main(String[] args) {
        Optional<String> vacio = Optional.empty();

        // Preguntar
        System.out.println(vacio.isPresent());          // false

        // Valor per defecte: immediat
        String texto = vacio.orElse("valor por defecto");
        System.out.println(texto);                       // valor por defecto

        // Valor per defecte: peresós (Supplier)
        String fabricado = vacio.orElseGet(() -> fabricarLento());

        // Si està buida... revienta amb la teua excepció
                        //   String   valor   =   vacio.orElseThrow(()   ->   new
IllegalStateException("¡Faltaba!"));

        // Si hi ha valor, fes alguna cosa (Consumer). Si no, no faces res.
            vacio.ifPresent(s  ->  System.out.println(s.toUpperCase()));   //  no
imprimeix res
    }

    static String fabricarLento() {
        return "fabricado con esfuerzo";
    }
}
```

La diferència entre `orElse` i `orElseGet`: `orElse(x)` ja té `x` calculat (l'argument s'avalua sempre, hi hagi o no valor). `orElseGet(() -> ...)` només fabrica el valor per defecte si la capsa està buida: és peresós, i si fabricar és car, t'estalvies la despesa.

🎁 Optional i els streams: parella de ball

Aquí s'ajunta tot: moltes operacions terminals de streams tornen un `Optional` perquè no poden garantir que existeixi resultat. `findFirst`, `findAny`, `max` i `min` són les típiques.

```
import java.util.*;
import java.util.stream.*;

public class Pareja {
    public static void main(String[] args) {
        List<String> nombres = List.of("bob", "ana", "carla");

        Optional<String> primeroConA = nombres.stream()
            .filter(n -> n.startsWith("a"))
            .findFirst();

        System.out.println(primeroConA.isPresent());      // true
        primeroConA.ifPresent(System.out::println);       // ana

        Optional<String> ninguno = nombres.stream()
            .filter(n -> n.startsWith("z"))
            .findFirst();
        System.out.println(ninguno.orElse("no hay nadie"));  // no hay nadie
    }
}
```

Si el `filter` no deixa passar a ningú, `findFirst` torna una capsa buida… i tu, amb `orElse` o `ifPresent`, decideixes l'aterratge. Això sí: **no usis `get()` a la boja**: si la capsa està buida, `get()` llança `NoSuchElementException`. Pregunta (`isPresent`) o aterra (`orElse`) abans d'obrir.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `orElse(null)` és legal però és tornar a casa amb les mans buides: millor `orElse("algo sensato")` o `orElseThrow(...)`. Si vas a tornar `null`, per a què vas embolicar l'absència en una capsa?

Exercici: l'ascensor de la capsa

```java
import java.util.*;
import java.util.stream.*;

public class Ascensor {
    public static void main(String[] args) {
        List<Integer> notas = List.of();
        Optional<Integer> maxima = notas.stream()
            .max(Integer::compareTo);
        System.out.println(maxima.orElse(-1));
    }
}
```

Què imprimeix?

- (A) `-1`
- (B) `0`
- (C) `Optional.empty`
- (D) Una `NoSuchElementException`

> 🔄 **Solució**
>
> La **A**. `notas` està buida, així que `max(Integer::compareTo)` no troba cap màxim i torna una capsa **buida**. `orElse(-1)` és la xarxa de seguretat: com que no hi ha valor, torna `-1`. Sense el `orElse`, un `get()` a cegues hauria llançat `NoSuchElementException` (la D, que passa si intentes `maxima.get()`). La B (`0`) seria el resultat de `orElse(0)` amb una altra xarxa. I la C descriu la capsa abans d'obrir-la: l'`orElse` ja l'ha obert i ha decidit l'aterratge.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què passa si fas `Optional.of(null)`?
2. Com llegeixes el valor d'un `Optional` sense arriscar-te a una excepció?
3. Quina diferència hi ha entre `orElse` i `orElseGet`?
4. Quins mètodes de streams tornen `Optional`?

> 🔄 **Respostes**

1. `NullPointerException`: `Optional.of` només admet valors reals. Per a possibles `null`, usa `ofNullable`.
2. Amb `orElse`, `orElseGet` o `ifPresent`. Mai amb `get()` a cegues.
3. `orElse(x)` té el valor per defecte ja calculat (s'avalua sempre); `orElseGet(() -> ...)` el fabrica només si la capsa està buida (peresós).
4. `findFirst`, `findAny`, `max` i `min` tornen `Optional` perquè no poden garantir resultat.

✅ Resum en 3 frases

1. **`Optional`** és una capsa que conté un valor o està buida: substituïx el `null` a seques i fa explícita l'absència.
2. Es crea amb `of`, `ofNullable` o `empty`, i es consumeix amb `orElse`, `orElseGet`, `orElseThrow` i `ifPresent`.
3. Molts mètodes de streams (`findFirst`, `max`, `min`) tornen `Optional`: la capsa és el pont entre la peresa del stream i la teva decisió sobre l'absència.

🐛 **Vocabulari ràpid**

Terme	Idea general
`Optional<T>`	Capsa que embolica un valor o la seva absència
`Optional.of(v)`	Capsa plena (prohibeix `null`)
`Optional.ofNullable(v)`	Capsa que admet `null` (si és null, queda buida)
`Optional.empty()`	Capsa buida
`orElse(x)` / `orElseGet(s)`	Valor per defecte (immediat / peresós)
`orElseThrow()`	Llança una excepció si està buit
`ifPresent(c)`	Executa un `Consumer` només si hi ha valor

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Del stream a la col·lecció: toList, Collectors, groupingBy · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu: domina la programació funcional

El tancament pràctic de la unitat, amb un pipeline que sembla màgia i un Optional que amaga un ensurt 🎯

> 🗺️ **Estàs en: 🎯 U13 · Programació funcional (Streams i Lambdas)** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
import java.util.*;
import java.util.stream.*;

public class Misterio {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");

        long r = nombres.stream()
            .filter(n -> n.length() > 3)
            .map(String::toUpperCase)
            .count();

        System.out.println(r);
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **5** → El `filter` no filtra res i el `map` converteix tots. ❌
2. **3** → `filter` deixa passar a `carla`, `david` i `eva`… no? ❌

3. **2** → ✅ Correcte de veritat: de `ana`, `bob`, `carla`, `david`, `eva` passen el filtre `carla` i `david` (les úniques amb més de 3 lletres). `map(String::toUpperCase)` les posa en majúscules, però no canvia la comptada. `count()` torna **2**.

> **› 🔄 Solució**
>
> L'opció **3**: **2**. Les longituds són `ana`=3, `bob`=3, `carla`=5, `david`=5, `eva`=3. El `filter(n -> n.length() > 3)` deixa passar només `carla` i `david`. El `map` les converteix en `CARLA` i `DAVID`, però `count()` compta elements: no mira els valors transformats. La trampa està en `eva`: 3 lletres no superen a 3, i el `map` no canvia el nombre d'elements, només la seva forma.

🔥 Fireside Chat: el Bucle For vs el Stream

> *Dues maneres de recórrer una llista es troben davant la màquina de cafè.*

Bucle For: — Mira, jo soc el clàssic. `for (String s : lista) { ... }`. Vaig element a element, decideixo jo el ritme, puc parar quan vull. El control total.

Stream: — ¡Ja! Tu controles el *com*. Jo només necessito que em digues el *què*: `lista.stream().filter(...).map(...).toList()`. Declares la recepta i jo cuino.

Bucle For: — I quan necessites modificar la llista mentre la recorres? Jo et deixo, amb compte. Tu ni ho intentes.

Stream: — Per a això existeixen les col·leccions de sortida. I quan t'oblides del `break` i et recorres mil elements de més, no em vinguis plorant. A més, jo puc dir `limit(3)` i m'aturo sol. Saps què és peresa productiva?

Bucle For: — Val, val... però els meus primitius i els meus índexs continuen sent útils per a coses simples.

Stream: — Ningú diu que no. Jo brillo quan el pipeline es torna complex: filtrar, transformar, ordenar i agrupar en tres línies en comptes de tres bucles imbricats. Tu per a allò simple, jo per a allò declaratiu. Treva.

Bucle For: — Treva. Al final, els dos recorrem.

> La lliçó: **usa el bucle quan el control importi i el stream quan vulguis declarar el resultat.** No és una guerra: és saber quan usar cada eina. El stream no t'odia: t'està llevat treball.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc la fletxa que converteix un bloc de codi en un valor que viatja sol.**
2. **Soc la interfície funcional que respon `true` o `false` a una pregunta.**
3. **Soc l'operador que et deixa reciclar un mètode com si fos una lambda.**
4. **Soc el flux peresós que no fa res fins que obris l'aixeta.**
5. **Soc l'aixeta final: sense mi, el stream no flueix.**
6. **Soc la capsa que accepta l'absència.**

> 🔄 **Respostes**
>
> 1. La **lambda** (`->`) — `(x) -> x * 2`.
> 2. **`Predicate<T>`** — `boolean test(T)`.
> 3. La **referència a mètode** (`::`) — `String::toUpperCase`.
> 4. El **`Stream`** — peresós i d'un sol ús.
> 5. L'**operació terminal** — `collect`, `reduce`, `count`, `forEach`.
> 6. **`Optional`** — la capsa que embolica un valor o la seva absència.

🤬 CONRAD VS EL MÓN: "El que oblida l'aixeta"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre els clàssics del novell.*

CONRAD: — UNA ALTRA VEGADA! Un alumne m'ensenya el seu programa i es queixa: *"CONRAD, no em imprimeix res"*. I jo: has posat una operació terminal? *"Una què?"* AI, MARE MEVA! `lista.stream().filter(...).map(...)` sense terminal no fa RES! El stream és peresós, com la meva paciència. Posa-li un `collect`, un `count` o un `forEach`, o la cinta es queda parada per sempre.

I després està el que intenta recórrer el mateix stream dues vegades. "Per què em dona error al segon `count()`?" I tant! Perquè la cinta ja es va buidar! Un stream es consumeix amb la primera terminal. Si vols comptar dues coses, crea dos streams. És com voler menjar la mateixa pizza dues vegades: impossible, ja està al teu estómac.

I el colmo dels colmos: el que fa `Optional.of(objetoQuePuedeSerNull)` i s'estranya de la `NullPointerException`. `of` és per a allò segur! Si pot venir `null`, `ofNullable`. I no em vinguis

amb `get()` a cegues sobre una capsa que pot estar buida: això és `NoSuchElementException` assegurada. `orElse`, `ifPresent`, `orElseThrow`… la capsa té porta, usa-la.

La lliçó: abans d'acusar Java de "odiar-te", repassa el trio sagrat de la unitat: **hi ha una operació terminal? estic reutilitzant un stream ja consumit? estic obrint un `Optional` buit sense xarxa?** El compilador no t'odia: t'està passant les respostes de l'examen.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. `nombres.stream().map(String::length)` sense operació terminal… s'executa el `map`?
   - a) Sí, a l'acte b) No, fins que arribi una terminal
2. Pots fer `lista.add(...)` sobre una llista tornada per `toList()`?
   - a) Sí b) No: és immutable
3. `Optional<String> o = Optional.empty(); o.get()`… què passa?
   - a) Torna `null` b) Llança `NoSuchElementException`
4. `reduce(0, Integer::sum)` sobre un `Stream<Integer>`… què torna?
   - a) `int` b) `Optional<Integer>`
5. `Collectors.groupingBy(Function.identity())` torna un…
   - a) `Map<T, List<T>>` b) `List<T>`
6. Després de `stream.count()`, pots usar el mateix stream per a una altra cosa?
   - a) Sí b) No: queda consumit

> 🔄 **Solucions**
>
> 1. **b)** — El stream és peresós; les intermèdies no s'executen fins a la terminal.
> 2. **b)** — `toList()` torna una llista immutable. Per a poder modificar-la, `collect(Collectors.toList())`.
> 3. **b)** — `get()` sobre una capsa buida llança `NoSuchElementException`. Usa `orElse` o `ifPresent`.
> 4. **a)** — Amb identitat, `reduce` torna el valor directe (`int`). Sense identitat (`reduce((a, b) -> ...)`) tornaria `Optional<Integer>`.
> 5. **a)** — `groupingBy` torna un `Map` amb les claus agrupades i les seves llistes d'elements.
> 6. **b)** — Un stream només es recorre una vegada; la primera terminal el consumeix.

⚡ Laboratori de Tortura: l'estadística que no arranca

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Et diuen que té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte… però, i si t'ho diuen malament? La teva tasca: fer que compili, que executi i que **tota** la sortida sigui correcta, comptant tu els errors reals.

```
import java.util.*;
import java.util.stream.*;

public class Estadisticas {

    public static void main(String[] args) {
        List<Integer> notas = List.of(7, 8, 3, 9, 5);

        long aprobados = notas.stream()
            .filter(n -> n >= 5)
            .count;                                    // línea A

        List<Integer> dobladas = notas.stream()
            .map(n -> n * 2)
            .toList();                                 // línea B

        int total = dobladas.stream()
            .reduce(0, (a, b) -> a + b);               // línea C

        System.out.println(aprobados + " " + dobladas + " " + total);
    }
}
```

Fallada intencionada: el nombre d'errors de l'enunciat està malament a propòsit: si busques el "tercer error de compilació" i "l'error de lògica", te'n tornes boig. La trampa és la teva confiança en l'enunciat.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Hi ha algun error de compilació? *sí → busca la línia sospitosa.*

> **› I si continuec atascat?**
> Repassa la línia A: `count` sense parèntesis no és una crida a mètode, és una referència. Necessita `count()`.

2. Compila ja? *sí → ara pensa en la sortida.*

> **› I si continuec atascat?**
> `notas` = {7, 8, 3, 9, 5}. El `filter(n → n >= 5)` deixa passar 7, 8, 9 i 5: quatre aprovats. `map(n → n * 2)` els dobla i `toList()` els recull. `reduce(0, (a, b) → a + b)` suma la llista doblada. Cap error de lògica a l'aguait.

3. Creus que hi ha un error de lògica que canvia el resultat? *Torna a llegir el codi amb calma i no et fies de l'enunciat.*

> **› Solució final**
>
> L'**únic error de compilació**:
>
> 1. Línia A: falta el `()` — `count` sense parèntesis és una referència a mètode, no una crida. Ha de ser `count()`.
>
> L'**error de lògica**: no existeix. `aprobados = 4` (7, 8, 9, 5 aproven; el 3 es queda fora). `dobladas = [14, 16, 6, 18, 10]` (tota la llista es dobla, també el 3). `total = 14 + 16 + 6 + 18 + 10 = 64`. Sortida: `4 [14, 16, 6, 18, 10] 64`.
>
> Aquella era la fallada intencionada: l'enunciat diu "3 errors de compilació i 1 de lògica", però només hi ha 1 falta de `()` i cap lògica trencada. L'error "amagat" era la teva confiança en l'enunciat.

```java
import java.util.*;
import java.util.stream.*;

public class Estadisticas {

    public static void main(String[] args) {
        List<Integer> notas = List.of(7, 8, 3, 9, 5);

        long aprobados = notas.stream()
            .filter(n -> n >= 5)
            .count();

        List<Integer> dobladas = notas.stream()
            .map(n -> n * 2)
            .toList();

        int total = dobladas.stream()
            .reduce(0, (a, b) -> a + b);

        System.out.println(aprobados + " " + dobladas + " " + total);
    }
}
```

Sortida correcta: `4 [14, 16, 6, 18, 10] 64`. La lliçó real: **llegeix el codi, no l'enunciat**. Un bug pot ser que et diguin "hi ha 3 errors" i només n'hi hagi 1: si busques els altres dos, et tornes boig. Compta, verifica i confia en el compilador, no en la llegenda.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🏅 **L'Arquer**	Explicar la sintaxi `->` de les lambdes amb un exemple d'un paràmetre sense parèntesis
🏅 **El Sastre**	Vestir una lambda amb `Predicate`, `Function`, `Consumer` i `Supplier` sense consultar la xurrera
🏅 **El Reciclador**	Reescriure tres lambdes com a referències a mètode (`::`)

Assoliment	Com aconseguir-ho
🏅 **El Mestre de la Cinta**	Muntar un pipeline `filter → map → sorted → limit → collect` i predir la seva sortida
🏅 **L'Acceptador**	Gestionar un `Optional` buit amb `orElse` i `ifPresent` sense llançar cap excepció

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```
import java.util.*;
import java.util.stream.*;

public class Acertijo {
    public static void main(String[] args) {
          List<String> palabras = List.of("sol", "mar", "luna", "sol", "luna",
"cielo");
        long distintas = palabras.stream().distinct().count();
        List<String> orden = palabras.stream().distinct().sorted().toList();
        System.out.println(distinctas + " " + orden);
    }
}
```

2. **La mitjana d'aprovats:** tens una `List<Integer>` amb notes. Com calcules, amb streams, la mitjana de les notes que són 5 o més? Què torna `average()` si cap nota aprova?

3. **El detectiu del pipeline:** per què `stream.map(...).filter(...)` sense operació terminal no imprimeix absolutament res?

4. **Vertader o fals:** "un `Stream` es pot recórrer tantes vegades com vulguis".

> 💡 **Solucions**
>
> 1. **`4 [cielo, luna, mar, sol]`**. `distinct()` deixa `sol, mar, luna, cielo` (4 de diferents). El segon stream ordena les 4 alfabèticament: `cielo, luna, mar, sol`.
> 2. Amb `notas.stream().filter(n -> n >= 5).mapToInt(n -> n).average()`. `average()` torna un `OptionalDouble`: si no hi ha aprovats, la capsa està buida i aterres

amb `orElse(0)`.

3. Perquè el stream és **peresós**: les operacions intermèdies només preparen la cinta. Sense una terminal (`collect`, `count`, `forEach`…), no s'executa res.
4. **Fals.** Un stream és d'un sol ús: la primera operació terminal el consumeix. Per a repetir, crea un altre stream.

🧩 Crucigrama de Bits

```
Horizontal:
1. Expressió sense nom que viatja com un valor (6 lletres)
3. Operació terminal que combina tots els elements en un (6 lletres)
5. Classe que embolica un valor o la seua absència (8 lletres)
7. Operació intermèdia que deixa passar només el que compleix una condició (6 lletres)

Vertical:
2. Interfície funcional que torna `boolean` (9 lletres)
4. El flux de dades peresós de Java (6 lletres)
6. Operador que transforma cada element (3 lletres)
```

> 📝 **Solucions**
>
> **Horizontal:** 1. LAMBDA · 3. REDUCE · 5. OPTIONAL · 7. FILTER **Vertical:** 2. PREDICATE · 4. STREAM · 6. MAP

💬 Preguntes d'Entrevista de Treball

Preguntes reals que et farien per a programador Java junior.

1. **"Explica-m'ho, com si jo fos la teva àvia, què és una lambda."**
2. **"Què és una interfície funcional i què té a veure amb les lambdes?"**
3. **"Quina és la diferència entre una operació intermèdia i una terminal en un stream?"**
4. **"Per què `Optional` és millor que tornar `null`?"**
5. **"Què fa `groupingBy` i què torna?"**
6. **"Quina és la diferència entre `map` i `reduce`?"**

🤷 No hi ha preguntes tontes

> ❓ **Una lambda és un objecte?**

Sí. Una lambda és un objecte de la classe anònima que implementa la seva interfície funcional. `Predicate<Integer> p = n -> n % 2 == 0;` crea per darrere un objecte que implementa `Predicate<Integer>`. El compilador fa la màgia; tu només escrius la fletxa.

> ❓ **Puc guardar una lambda en una variable?**

Sí, però no en una variable de tipus `int` ni `String`: necessita una variable de tipus **interfície funcional**. `Predicate<Integer> esPar = n -> n % 2 == 0;` és perfectament legal. La lambda viatja com qualsevol altre valor.

> ❓ **Un `Stream` guarda dades? I quan es buida?**

No guarda res: és una vista sobre una font (llista, array…). Es buida en arribar una operació terminal. Les dades originals continuen intactes en la seva col·lecció; el que es consumeix és la cinta.

> ❓ **Puc usar streams amb arrays, que en la U06 eren una altra cosa?**

Clar: `Arrays.stream(miArray)` converteix un array en flux, i `Arrays.stream(int[])` a més dona un `IntStream` amb mètodes com `sum()` i `average()`. Els arrays i els streams no competeixen: es passen el relleu.

🎬 Post-Crèdits

La programadora acaba el seu analitzador de notes: un pipeline de streams que filtra els aprovats, calcula la mitjana amb `average()` i guarda el resultat en un `Optional` per si un dia no hi ha notes que analitzar. Tot perfecte… fins que tanca el programa i les seves estadístiques desapareixen.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Has vist? Tot viu en memòria RAM. Tanques l'app i adéu, estadístiques. Un stream és fantàstic per a processar dades mentre el programa corre, però perquè sobrevisquin al reinici necessites una altra cosa.

Programadora: — Guardar-les en un fitxer? Com un fitxer de text en el disc?

CONRAD: — *assenteix* Exacte. Fitxers, i quan les dades creixin, expressions regulars per a buscar patrons. La teva cinta transportadora processa; el fitxer conserva. Tot això és una altra història… i quasi et toca.

PROXIMAMENT EN U14: Fitxers i expressions regulars (Regex). Guarda els resultats dels teus streams en disc i busca patrons amb expressions regulars. 📁

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · Optional: el que accepta l'absència · **Següent:** U14 · Fitxers i expressions regulars (Regex)

U14 — Fitxers i expressions regulars (Regex)

El GPS dels fitxers, el llapis que no oblida, la porta que es tanca sola i el detectiu de patrons: File, FileWriter, BufferedReader, NIO, serialització i regex 📁

En la U12 vas veure col·leccions, genèrics i mapes: les teves dades per fi viuen en memòria com voldries. Però hi ha un problema: quan el programa acaba, tot s'esfuma. Els arrays, els `ArrayList` i els `HashMap` moren amb el `main`. Avui això s'acaba. Els **fitxers** fan que les teves dades sobrevisquin al teu programa, i les **expressions regulars** et donen vista de superheroi per a buscar, validar i extreure qualsevol patró en un text.

Aquesta unitat té tres grans actes:

- **Fitxers de text:** la classe `File`, el GPS dels fitxers; `FileWriter` i `FileReader` + `BufferedReader`, el llapis i el lector de sempre; `try-with-resources`, la porta que es tanca sola; i `PrintWriter` i `Scanner`, el sergent que formata i el pop que tokenitza.
- **La forma moderna i la d'elit:** l'API **NIO** (`Files` i `Paths`), que fa en una línia el que abans costava cinc, i la **serialització**, que guarda objectes sencers en un fitxer com qui fica roba en una maleta.
- **Expressions regulars:** `Pattern` i `Matcher`, la taula de símbols, i les seves aplicacions amb `matches()`, `replaceAll()`, `split()` i grups de captura per a validar correus, DNIs i telèfons com un professional.

Pel camí entendràs per què `new File("ruta")` no crea res, què és això de l'infern de les contrabarres (`\\d`), per què no tancar un fitxer et pot costar les dades, i per què una regex no serveix per a validar HTML (per molt que ho intentes).

> 📌 **Nota:** la consola (`System.out`, `printf`, `Scanner` pel teclat) ja la vas veure en la **U03**, punt 7. Aquí el `Scanner` i el `PrintWriter` tornen, però jugant en la seva lliga: llegint i escrivint fitxers.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Localitzar i descriure fitxers i carpetes amb la classe **`File`** (`exists`, `isFile`, `isDirectory`, `length`, `listFiles`).
- Escriure text amb **`FileWriter`** (sobreescrivint o afegint) i llegir-lo amb **`FileReader`** + **`BufferedReader`** línia a línia.
- Tancar recursos sense plorar amb **`try-with-resources`** i explicar per què el `close()` importa.
- Formatar la sortida amb **`PrintWriter`** i llegir fitxers amb **`Scanner`** (inclòs `useDelimiter`).
- Gestionar fitxers amb l'API moderna **NIO** (`Files` i `Paths`): llegir, escriure i consultar en una línia.
- Guardar i recuperar objectes sencers amb **`ObjectOutputStream`** i **`ObjectInputStream`** (serialització).
- Crear i compilar **expressions regulars** amb `Pattern` i `Matcher`, i dominar els símbols bàsics.
- Aplicar regex amb **`matches()`**, **`replaceAll()`** i **`split()`**, i usar **grups de captura** per a validar i extreure.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · La classe File: el teu explorador de fitxers	El GPS que localitza fitxers sense llegir-ne el contingut	Tots
02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader	El llapis que no oblida i el lector que no s'ennuega	Tots
03 · try-with-resources: tancar sense plorar	La porta que es tanca sola: adéu al `close()` manual	Tots
04 · PrintWriter i Scanner + File	El sergent que formata i el pop que tokenitza	Tots
05 · NIO: Files i Paths (la forma moderna)	Llegir, escriure i consultar fitxers en una línia	Tots

Punt	Què aprendràs	Dificultat
06 · Serialització: guardar objectes amb ObjectOutputStream	El teu objecte sencer en una maleta: `Serializable`	Tots
07 · Expressions regulars: Pattern i Matcher	El motlle, el text i l'infern de les contrabarres	Tots
08 · Regex en acció: matches, replaceAll, split i validacions	Validar correus, DNIs, telèfons i extreure amb grups	Tots
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA5)

RA5: Realitza operacions d'entrada i sortida d'informació, utilitzant procediments específics del llenguatge i llibreries de classes.

CE	Criteri	On es cobreix
RA5 c)	S'han reconegut les possibilitats d'entrada / sortida del llenguatge i les llibreries associades.	✅ Punts 1, 4 i 5
RA5 d)	S'han utilitzat fitxers per a emmagatzemar i recuperar informació.	✅ Punts 1, 2, 3, 4, 5 i 6

CE	Criteri	On es cobreix
RA5 e)	S'han creat programes que utilitzen diversos mètodes d'accés al contingut dels fitxers.	✅ Punts 2, 4, 5 i 6

> 📌 Els CEs **RA5 a)** (consola) i **RA5 b)** (format) es van cobrir en la **U03**, punt 7. Aquesta unitat completa la RA5 amb els fitxers i les llibreries d'E/S.

✅ Criteris d'avaluació coberts (RA6)

RA6: Escriu programes que manipulen informació, seleccionant i utilitzant tipus avançats de dades.

CE	Criteri	On es cobreix
RA6 g)	S'han utilitzat expressions regulars en la cerca de patrons en cadenes de text.	✅ Punts 7, 8 i 9

> 📌 La **RA6** es completa amb la **U06** (arrays, CE a), la **U12** (col·leccions, genèrics i mapes, CEs b, c, d, e i f). Aquesta unitat aporta el CE g: les expressions regulars.

🚪 Per on començo?

- Vens de la U12 i vols l'essencial? → Comença en el punt 1 i el punt 2: localitzar i llegir fitxers de text.
- Ja saps escriure i llegir i vols la forma moderna? → Salta al punt 3 (tancar sense plorar) i al punt 5 (NIO en una línia).
- Només vens a per les regex? → Ves directe al punt 7 i al punt 8: el detectiu de patrons.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · La classe File: el teu explorador de fitxers

⏭️ **En acabar la unitat, continua en U15 · Persistència de dades: JDBC i introducció a ORM.**

01 · La classe File: el teu explorador de fitxers

El GPS que localitza fitxers sense llegir-ne el contingut: existeix, és fitxer, és carpeta, quant pesa i què hi ha dins 🗂️📍

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 01 · La classe File: el teu explorador de fitxers

---

📬 La idea en una frase

> **`File` no és el contingut d'un fitxer: és la targeta de visita que et diu on està, com es diu, quant pesa i si existeix. Abans de llegir o escriure, pregunta-li al GPS.**

En la U12 guardaves les teves dades en memòria: llistes, mapes, objectes… tot viu mentre el programa corre. Però i si vols saber si un fitxer existeix abans d'obrir-lo? I si necessites recórrer les carpetes del teu disc? Aquí entra la classe `File`. No llegeix ni escriu res: només explora i descriu. És el porter de l'edifici dels fitxers.

🗂️ File: el GPS dels teus fitxers

La classe `File` (en `java.io`) representa una **ruta** en el teu sistema: un fitxer o una carpeta. El més important que has de gravar a foc: **`new File("ruta")` no crea res**. Només crea l'objecte que representa aquella ruta. Crear el fitxer de veritat és una altra història (ho veuràs al punt 2).

```
import java.io.File;

File f = new File("C:/datos/notas.txt");
System.out.println("¿Existe? " + f.exists());            // true o false
System.out.println("¿Es archivo? " + f.isFile());         // ¿és un fitxer i no una carpeta?
System.out.println("¿Es carpeta? " + f.isDirectory());   // ¿és un directori?
System.out.println("Tamaño: " + f.length() + " bytes");  // pes en bytes
System.out.println("Ruta absoluta: " + f.getAbsolutePath());  // l'adreça completa
System.out.println("Nombre: " + f.getName());             // només el nom amb extensió
```

> 💡 **Consell:** en Windows usa `/` o `\\`. Escriu `"C:/datos/notas.txt"` o `"C:\\datos\\notas.txt"`. Amb una sola `\` Java creu que comences un caràcter especial (`\n`, `\t`…) i t'enredaràs. Això es posa interessant amb les regex al punt 7: les contrabarres es dupliquen pertot.

El repertori imprescindible

Mètode	Què et diu
`exists()`	Existeix la ruta?
`isFile()`	És un fitxer (i no una carpeta)?
`isDirectory()`	És una carpeta?
`length()`	Grandària en bytes
`getName()`	Nom del fitxer (sense la ruta)
`getAbsolutePath()`	Ruta completa
`getParent()`	La carpeta que el conté
`listFiles()`	Els fitxers i carpetes que hi ha dins (si és un directori)
`createNewFile()`	Crea el fitxer buit (torna `false` si ja existia)
`mkdir()` / `mkdirs()`	Crea la carpeta (amb `mkdirs`, crea també les intermèdies)

> ⚠️ **Advertència:** `File` **no llegeix contingut**. Li preguntes *"existeix? quant pesa? és carpeta?"* i et contesta. Per a llegir el de dins, ja arribaran `FileReader`, `BufferedReader`, `Scanner` i `Files.readAllLines()` en els pròxims punts. Cadascú al seu ofici.

📁 Recórrer una carpeta amb listFiles

`listFiles()` et torna un array amb tot el que hi ha dins d'una carpeta. Perfecte per a l'"explorador de fitxers" casolà:

```java
import java.io.File;

public class Explorador {
    public static void main(String[] args) {
        File carpeta = new File("C:/datos");
        if (carpeta.exists() && carpeta.isDirectory()) {
            File[] contenidos = carpeta.listFiles();
            for (File item : contenidos) {
                String tipo = item.isDirectory() ? "[CARPETA] " : "[ARCHIVO] ";
                 System.out.println(tipo + item.getName() + " (" + item.length()
+ " bytes)");
            }
        } else {
            System.out.println("Esa ruta no es una carpeta existente.");
        }
    }
}
```

Recorda comprovar `exists()` i `isDirectory()` abans de `listFiles()`: si la ruta no existeix, `listFiles()` torna `null` i el `for-each` t'esclata a la cara.

📝 La ruta no és el contingut

Un error típic de novell és creure que `new File("notas.txt")` "crea" el fitxer o que en construir un `File` ja "llegeixes" alguna cosa. No:

```java
File f = new File("notas.txt");    // Només representa la ruta. No existeix
res nou en el disc.
boolean creado = f.createNewFile();  // AIXÒ sí intenta crear-lo. Torna false si
ja existia.
```

`createNewFile()` crea un fitxer **buit** i torna `true` si l'ha creat, o `false` si ja existia. Si la carpeta no existeix, llança `IOException`. El patró habitual: comprova `exists()` abans de decidir què fer.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** File representa rutes, no contingut. Per a llegir el contingut usa `Scanner`, `BufferedReader` o `Files.readAllLines()`. El GPS no t'explica el que hi ha dins de l'edifici: només et porta a la porta.

Exercici: el detectiu de fitxers

```
import java.io.*;

public class DetectiveDeArchivos {
    public static void main(String[] args) throws IOException {
        File f = new File("misterio.txt");
        if (!f.exists()) {
            System.out.println("Creando archivo...");
            FileWriter w = new FileWriter(f);
            w.write("Tres\npalabras\nmisteriosas\n");
            w.close();
        }
        BufferedReader r = new BufferedReader(new FileReader(f));
        String s = "";
        String linea;
        while ((linea = r.readLine()) != null) {
            s = linea + " " + s;
        }
        r.close();
        System.out.println(s);
    }
}
```

Què imprimeix la PRIMERA vegada? I la SEGONA vegada (quan el fitxer ja existeix)?

> 🔄 **Solució**
>
> La **primera** vegada: el fitxer no existeix, així que es crea amb les paraules "Tres", "palabras" i "misteriosas". Després es llegeix i es concatena **al revés**: `misteriosas palabras Tres`.
>
> La **segona** vegada: el fitxer ja existeix (`exists()` és `true`), així que no es recrea i es llegeix el mateix → torna a imprimir `misteriosas palabras Tres`.
>
> La trampa està en l'`if (!f.exists())`: la primera execució "crea el fitxer", però les següents no. El contingut no canvia mai, així que la sortida és sempre la mateixa.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Crea alguna cosa `new File("notas.txt")` en el disc?
2. Quin mètode torna `true` si una ruta és una carpeta?
3. Què torna `createNewFile()` si el fitxer ja existeix?
4. Per a què serveix `length()`?

> 🔄 **Respostes**
>
> 1. No. Només crea un objecte que representa la ruta. Per a crear el fitxer, `createNewFile()` (o un `FileWriter`, punt 2).
> 2. `isDirectory()`.
> 3. `false` (el fitxer ja existia, no ha creat res de nou).
> 4. Per a saber la grandària del fitxer en bytes. Serveix per a decidir si et fa mandra llegir-lo sencer.

✅ Resum en 3 frases

1. **`File`** és un GPS de rutes: et diu si existeixen, si són fitxer o carpeta, quant pesen i què contenen (`listFiles()`), però **no llegeix contingut**.
2. `new File("ruta")` **no crea res**: crea l'objecte que representa la ruta. Per a crear, `createNewFile()`, `mkdir()`/`mkdirs()` o un `FileWriter`.
3. Abans d'operar amb una ruta, comprova `exists()` i `isFile()`/`isDirectory()`: els mètodes tornen `false` o `null` si la ruta no és l'esperada.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`File`	Objecte que representa una ruta (fitxer o carpeta) en el sistema
Ruta	L'"adreça" d'un fitxer: `C:/datos/notas.txt`
Ruta relativa	Adreça des del directori de treball (`"notas.txt"`)
Ruta absoluta	Adreça completa des de l'arrel del disc

Terme	Idea general
`listFiles()`	Torna els fitxers/carpetes que hi ha dins d'un directori

📁 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader

02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader

El llapis que no oblida i el lector que no s'ennuega: escriu fitxers amb FileWriter i llegeix-los línia a línia amb BufferedReader ✍️📖

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader

---

📬 La idea en una frase

> **`FileWriter` és el llapis que escriu caràcters en un fitxer i `FileReader` el lector que els recupera; amb `BufferedReader` al damunt, llegir línia a línia és bufar i fer ampolles.**

Al punt 1 `File` només et deia si el fitxer existia i quant pesava. Ara toca el bo: **escriure** contingut i **llegir-lo**. Per a escriure tens `FileWriter`; per a llegir, `FileReader`. I si a més emboliques el lector amb `BufferedReader`, podràs llegir fitxers sencers línia a línia sense despeinar-te.

✍️ Escriure amb FileWriter

`FileWriter` escriu **caràcters** en un fitxer. Si el fitxer no existeix, el crea; si existeix, el **sobreescriu** per defecte.

```
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) throws IOException {
        FileWriter escritor = new FileWriter("salida.txt");
        escritor.write("Primera línea.\n");
        escritor.write("Segunda línea.\n");
        escritor.close();
    }
}
```

Vols **afegir al final** sense esborrar l'anterior? Passa-li un segon argument `true` (mode *append*):

```
FileWriter writer = new FileWriter("bitacora.txt", true);  // afig al final
```

> ⚠️ **Advertència:** sense `close()` o `flush()`, les dades es queden atrapades al buffer intern i es poden perdre. És com escriure una carta i no tancar el sobre: el carter no la recull. Ho veuràs a fons al punt 3 amb `try-with-resources`.

📖 Llegir amb FileReader + BufferedReader

`FileReader` llegeix caràcters solts, que és incòmode. La solució: emboliques el `FileReader` amb un `BufferedReader` i uses `readLine()`, que et torna una línia sencera d'una vegada (i `null` quan s'acaba el fitxer).

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerTexto {
    public static void main(String[] args) throws IOException {
                        BufferedReader   lector   =   new   BufferedReader(new
FileReader("salida.txt"));
        String linea = lector.readLine();
        while (linea != null) {
            System.out.println(linea);
            linea = lector.readLine();
        }
        lector.close();
    }
}
```

El patró `while ((linea = lector.readLine()) != null)` és el pa de cada dia de Java: llegeix una línia, comprova que no és `null`, la processa i repeteix. Quan `readLine()` torna `null`, el fitxer s'ha acabat.

> 📝 **Nota:** la lectura amb `Scanner` pel teclat la vas veure en la UO3 (punt 6). Aquí el `Scanner` reapareix al punt 4, però llegint fitxers.

🚨 I si el fitxer no existeix en llegir?

Llances un `FileReader` sobre un fitxer que no existeix i… `FileNotFoundException`. És filla de `IOException`, així que tens dos camins:

1. **Declarar** l'excepció en la signatura: `throws IOException` (el que hem fet adalt).
2. **Capturar-la** amb `try-catch` i fer alguna cosa sensata, com avisar l'usuari:

```
try {
    BufferedReader lector = new BufferedReader(new FileReader("no_existe.txt"));
    String linea;
    while ((linea = lector.readLine()) != null) {
        System.out.println(linea);
    }
    lector.close();
} catch (IOException e) {
    System.out.println("Error al leer: " + e.getMessage());
}
```

> 💡 **Consell:** el camí 2 és el professional: no deixis que el programa muira per un fitxer que falta. El camí 1 és ràpid per a practicar. Al punt 3 veuràs la combinació definitiva.

🧱 I per què BufferedReader i no FileReader a seques?

`FileReader` et dona caràcters d'un en un. Per a llegir frases has de muntar un bucle d'acumulació. `BufferedReader` et dona **línies senceres** i a més és més ràpid en fitxers grans perquè **llegeix en blocs** i guarda el que sobra al seu buffer. El patró "embolicar" (`new BufferedReader(new FileReader(...))`) és típic de Java: una classe que afegeix superpoders a una altra.

Tasca	Eina
Escriure caràcters	`FileWriter`
Escriure afegint al final	`FileWriter(path, true)`
Llegir caràcters	`FileReader`
Llegir línies senceres (recomanat)	`BufferedReader` embolicant `FileReader`

Tasca	Eina
Formatar i escriure amb `println/printf`	`PrintWriter` (punt 4)

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** el trio clàssic per a llegir fitxers de text en Java és `BufferedReader` + `FileReader`. El bucle `while ((linea = lector.readLine()) != null)` és l'estàndard que veuràs en milers de projectes.

Exercici: el comptador que no menteix

```
import java.io.*;

public class ContadorLineas {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

Què imprimeix? Compta les línies de veritat?

> › 🔄 **Solució**
>
> Imprimeix **3**.
>
> El fitxer es crea amb "linea1", "linea2" i "linea3", cadascuna amb el seu `\n`. El `readLine()` torna cada línia i quan arriba al final torna `null`, i aquí s'acaba el `while`. L'últim `\n` no crea una "quarta línia": `readLine()` torna `null` quan no queda més text, no una línia buida.

> Detall fi: el codi del `while` ni tan sols guarda la línia. Només la descarta (`r.readLine()` a seques) per a comptar. A vegades no necessites el contingut, només saber quantes línies hi ha.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què fa `FileWriter` amb un fitxer que ja existeix per defecte?
2. Com afegeixes text al final d'un fitxer sense esborrar l'anterior?
3. Què torna `readLine()` quan el fitxer s'ha acabat?
4. Per què s'embolica `FileReader` amb `BufferedReader`?

> 🔄 **Respostes**
>
> 1. El sobreescriu des de zero.
> 2. `new FileWriter("bitacora.txt", true)`: el segon argument `true` activa el mode *append*.
> 3. `null`. Per això funciona el bucle `while ((linea = lector.readLine()) != null)`.
> 4. Per a llegir línies senceres amb `readLine()` (i per a ser més ràpid en fitxers grans gràcies al seu buffer).

✅ Resum en 3 frases

1. **`FileWriter`** escriu caràcters en un fitxer (el crea si no existeix i el sobreescriu si existeix); amb l'argument `true` afegeix al final.
2. **`FileReader`** + **`BufferedReader`** llegeix fitxers de text línia a línia amb `readLine()`, que torna `null` al final.
3. Sense `close()` (o `flush()`) les dades es queden al buffer: **tanca sempre** o usa `try-with-resources` (punt 3).

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
`FileWriter`	Escriu caràcters en un fitxer

Terme	Idea general
`FileReader`	Llegeix caràcters d'un fitxer
`BufferedReader`	Lector amb buffer que permet `readLine()`
`readLine()`	Torna la següent línia o `null` si no en queda cap
Append	Afegir al final del fitxer sense esborrar l'anterior
`FileNotFoundException`	Es llança en llegir un fitxer que no existeix

📁 Tornar a l'índex de la unitat · **Anterior:** 01 · La classe File: el teu explorador de fitxers · **Següent:** 03 · try-with-resources: tancar sense plorar

03 · try-with-resources: tancar sense plorar

La porta que es tanca sola: deixa que Java faci servir close() per tu, encara que salti una excepció enmig del camí 🔒🚪

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 03 · try-with-resources: tancar sense plorar

---

📬 La idea en una frase

> **En comptes d'acordar-te de tancar cada fitxer, li dius a Java "tanca això quan acabis, passi el que passi" i ell ho fa sol.**

Al punt 2 vas veure que sense `close()` les dades es poden perdre. Però acordar-te de tancar, i a més quan hi ha excepcions pel mig, és un patiment. Des de Java 7 existeix el `try-with-resources`: obris els recursos dins dels parèntesis del `try` i Java els tanca automàticament en sortir del bloc, tant si tot va bé com si salta una excepció.

🔒 El try-with-resources en acció

Mira com de net queda el lector del punt 2 sense `close()` manual:

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LectorLimpio {
    public static void main(String[] args) {
                        try   (BufferedReader   br   =   new   BufferedReader(new
FileReader("salida.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
        // No hi ha br.close(). Es tanca sol.
    }
}
```

El que passa per darrere: en sortir del bloc `try` (amb èxit o amb excepció), Java crida a `close()` sobre cada recurs que vas declarar entre parèntesis. És el `finally` que t'estalvies escriure.

> 💡 **Consell:** fixa't que el `catch` continua existint. El `try-with-resources` tanca els recursos, però no s'empassa les excepcions: tu decideixes com gestionar-les.

🧹 Diversos recursos alhora

¿Llegir d'un fitxer i escriure en un altre? Declara els dos recursos separats per `;` i Java els tanca **en ordre invers** al de declaració:

```java
import java.io.*;

public class Copiador {
    public static void main(String[] args) {
                        try   (BufferedReader   br   =   new   BufferedReader(new
FileReader("origen.txt"));
             PrintWriter pw = new PrintWriter(new FileWriter("copia.txt"))) {

            String linea;
            while ((linea = br.readLine()) != null) {
                pw.println(linea);
            }
            System.out.println("Copia terminada.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

És el patró estàndard per a copiar fitxers: llegeix línia a línia i escriu-la al destí. I si un dels dos no es pot obrir, no passa res: Java intenta tancar els que sí que es van obrir.

🤔 Quins recursos valen?

Només valen els que implementen `AutoCloseable` (o la seva filla `Closeable`). Les classes de fitxers que hem vist (`FileReader`, `FileWriter`, `BufferedReader`, `PrintWriter`, `ObjectInputStream`…) la implementen. En general: si la classe té `close()`, quasi segur que val.

> ⚠️ **Advertència:** `Scanner` també és `AutoCloseable`, així que pots fer `try (Scanner sc = new Scanner(new File(...)))`. Ull, no el uses per a tancar un `Scanner` que llegeix del teclat en un programa llarg: al tancar-lo, tanques també l'entrada estàndard i a partir d'aquí tot `nextLine()` et dona errors. Per al teclat, un `Scanner` normal i un `close()` al final.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `try-with-resources` no és només per a fitxers: val per a qualsevol recurs que calgui alliberar. Si la teva classe necessita netejar-se en acabar, que implementi

> `AutoCloseable` i la fiques entre els parèntesis.

Exercici: el diari que no oblida (mode append)

```
import java.io.*;
import java.time.LocalDate;

public class Diario {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("diario.txt", true)) {
                    fw.write(LocalDate.now() + ": Hoy he aprendido try-with-
resources.\n");
        } catch (IOException e) {
            System.out.println("No pude escribir el diario: " + e.getMessage());
        }
    }
}
```

Què passa si executes el programa dues vegades? I què fa el `LocalDate.now()`?

> 🔄 **Solució**
>
> Executat **dues vegades**, el fitxer `diario.txt` tindrà **dues línies** amb la data de avui. El mode *append* (`true`) afegeix al final sense esborrar l'anterior, així que cada execució suma una entrada. Perfecte per a un diari de veritat.
>
> `LocalDate.now()` torna la data actual del sistema (per exemple, `2026-09-06`), en format ISO. És la forma moderna de demanar "quin dia és avui" sense liar-te amb `Date` i `Calendar`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què fa Java en sortir del bloc d'un `try-with-resources`?
2. En quin ordre es tanquen diversos recursos declarats en el `try`?
3. Quina interfície ha d'implementar un recurs per a poder usar-lo en `try-with-resources`?
4. El `try-with-resources` elimina la necessitat de `catch`?

> 🔄 **Respostes**

1. Crida a `close()` sobre tots els recursos declarats, tant si tot va bé com si hi va haver excepció.
2. En ordre invers al que es van declarar.
3. `AutoCloseable` (o la seva subinterfície `Closeable`).
4. No. El `try-with-resources` tanca els recursos, però les excepcions s'han de capturar o declarar igualment.

✅ Resum en 3 frases

1. **`try-with-resources`** declara els recursos entre parèntesis del `try` i Java els tanca sol en sortir, amb èxit o amb excepció: adéu al `close()` oblidat.
2. Pots obrir **diversos recursos alhora** (separats per `;`) i es tanquen en ordre invers.
3. Només valen recursos que implementen **`AutoCloseable`/`Closeable`**, que és el cas de totes les classes de fitxers de Java.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `try-with-resources` | Bloc `try` que tanca els seus recursos automàticament |
> | `AutoCloseable` | Interfície amb `close()` que marca un recurs com a "alliberable" |
> | `Closeable` | Subinterfície de `AutoCloseable`, la que usen els streams clàssics |
> | Recurs | Alguna cosa que s'ha d'alliberar: fitxer, connexió, scanner… |
> | *Append* | Afegir al final d'un fitxer (segon argument `true` de `FileWriter`) |

📁 Tornar a l'índex de la unitat · **Anterior:** 02 · Escriure i llegir text: FileWriter, FileReader i BufferedReader · **Següent:** 04 · PrintWriter i Scanner + File

04 · PrintWriter i Scanner + File

El sergent que formata la sortida i el pop que tokenitza l'entrada: PrintWriter per a escriure bonic i Scanner per a llegir fitxers a trossos 🖨️🐔

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 04 · PrintWriter i Scanner + File

📬 La idea en una frase

> **`PrintWriter` escriu en fitxers amb la comoditat de `System.out` (`println`, `printf`) i `Scanner` llegeix fitxers trocejant-los en tokens (`nextInt`, `nextLine`, `useDelimiter`).**

En la U03 vas veure `System.out.printf` i `Scanner` pel teclat (punts 6 i 7). Ara tots dos tornen, però apuntant a **fitxers** en comptes de a la consola. `PrintWriter` escriu amb format; `Scanner` + `File` llegeix fitxers com si estigueres llegint del teclat, amb el seu `hasNextLine`, `nextInt` i companyia.

🖨️ PrintWriter: System.out, però en un fitxer

`PrintWriter` té `println()`, `print()` i `printf()`: exactament els mètodes de la consola, però escrivint en un fitxer.

```
import java.io.PrintWriter;
import java.io.FileWriter;

public class Informe {
    public static void main(String[] args) throws Exception {
        try (PrintWriter pw = new PrintWriter(new FileWriter("informe.txt"))) {
            pw.println("=== Informe de ventas ===");
            pw.printf("Total: %10.2f €%n", 1234.5678);
            pw.printf("Producto: %-12s Unidades: %d%n", "Camiseta", 42);
            pw.printf("Producto: %-12s Unidades: %d%n", "Chaqueta", 7);
        }
    }
}
```

El `%n` és el salt de línia portable (no usis `\n` en `printf`, que en Windows no encerta). Si necessites repassar els formats de `printf` (`%d`, `%f`, `%s`, `%-10s`…), torna a la UO3, punt 7.

> 📝 **Nota:** el `printf` amb `%10.2f` es veu bé en la consola, però en un fitxer que s'òbriga amb un altre programa, compte amb els espais: no tots els editors els respecten igual.

🐙 Scanner + File: llegir fitxers a trossos

`Scanner` pot llegir d'un fitxer igual que llegeix del teclat. Embolica un `File` i a jugar amb `hasNextLine()` / `nextLine()`, o amb `hasNextInt()` / `nextInt()` per a dades soltes.

```
import java.io.File;
import java.util.Scanner;

public class LeerConScanner {
    public static void main(String[] args) throws Exception {
        try (Scanner sc = new Scanner(new File("notas.txt"))) {
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
        }
    }
}
```

¿Fitxers amb números separats per espais? `nextInt()` te'ls va servint sense que tu et fiques en el lio de les línies:

```
// datos.txt → "10 20 30 40"
try (Scanner sc = new Scanner(new File("datos.txt"))) {
    int suma = 0;
    while (sc.hasNextInt()) {
        suma += sc.nextInt();
    }
    System.out.println("Suma: " + suma);  // Suma: 100
}
```

> ⚠️ **Advertència:** al punt 3 ho comentàvem: un `Scanner` en `try-with-resources` és fantàstic per a fitxers. No ho facis amb un `Scanner` del teclat dins d'un programa llarg: al tancar-lo tanques `System.in` i tot el que vingui després de llegir del teclat es romp.

🪓 useDelimiter: tallar on tu digues

Per defecte `Scanner` troceja per espais i salts de línia. Però si el teu fitxer separa els camps amb `;`, li dius amb `useDelimiter()` on tallar:

```
import java.io.File;
import java.util.Scanner;

public class LeeCSV {
    public static void main(String[] args) throws Exception {
        // datos.csv → "Ana;25;DAM" ... "Bob;22;DAW"
        try (Scanner sc = new Scanner(new File("datos.csv"))) {
            sc.useDelimiter(";|\\R");   // talla per ';' o per salt de línia
            while (sc.hasNext()) {
                String nombre = sc.next();
                int edad = sc.nextInt();
                String ciclo = sc.next();
                System.out.printf("%-8s %2d años %s%n", nombre, edad, ciclo);
            }
        }
    }
}
```

`\\R` és el "qualsevol salt de línia" de Java (punt 7, quan vegis regex, això et sonarà). Amb `";|\\R"` li dius: talla per punt i coma o per salt de línia. El resultat és un fitxer CSV llegit sense despeinar-te.

🆚 Scanner vs BufferedReader: quin usi?

Criteri	`Scanner`	`BufferedReader`
Llegir per línies	`nextLine()`	`readLine()`
Llegir números	`nextInt()` directe	Has de convertir a mà
Delimitadors personalitzats	`useDelimiter()`	No

Criteri	`Scanner`	`BufferedReader`
Velocitat en fitxers enormes	Suficient	Millor (buffer més eficient)
El millor per a	Parsejar dades soltes i tokens	Llegir fitxers de text línia a línia

> 💡 **Consell:** ¿fitxer de configuració amb números i paraules? `Scanner`. ¿Log de 2 GB que cal recórrer sencer? `BufferedReader`. Per a processar logs amb regex ja ho veuràs al punt 8.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** `PrintWriter` i `Scanner` són la parella de moda per a "llegir, transformar i escriure": llegeixes amb `Scanner`, transformes, i escrius amb `PrintWriter` formatat. El `printf` de `PrintWriter` és el mateix de sempre, però amb destinació de fitxer.

Exercici: el full de càlcul que es fa sol

```
import java.io.*;
import java.util.Scanner;

public class NotasMedias {
    public static void main(String[] args) throws Exception {
        try (Scanner sc = new Scanner(new File("alumnos.txt"))) {
            sc.useDelimiter(";|\\R");
            PrintWriter pw = new PrintWriter(new FileWriter("medias.txt"));
            while (sc.hasNext()) {
                String nombre = sc.next();
                int n1 = sc.nextInt();
                int n2 = sc.nextInt();
                double media = (n1 + n2) / 2.0;
                pw.printf("%s: %.1f%n", nombre, media);
            }
            pw.close();
        }
        System.out.println("Hecho.");
    }
}
```

El fitxer `alumnos.txt` conté `"Ana;7;9\nBob;5;6"`. Què contindrà `medias.txt`?

> 🔄 **Solució**
>
> `medias.txt` contindrà:
>
> ```
> Ana: 8.0
> Bob: 5.5
> ```
>
> El `useDelimiter(";|\\R")` talla per `;` o per salt de línia, així que els camps surten nets: nom, nota1, nota2. La mitjana d'Ana és `(7+9)/2.0 = 8.0`; la de Bob, `(5+6)/2.0 = 5.5`. Fixa't en el `2.0` en comptes de `2`: sense el `.0`, `(7+9)/2` faria divisió entera i donaria `8` en comptes de `8.0`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode de `PrintWriter` et permet formatar amb `%d`, `%f`, `%s`?
2. Com llegeixes un fitxer amb `Scanner` en comptes de pel teclat?
3. Per a què serveix `useDelimiter(";")`?
4. Què torna `hasNextInt()` quan el fitxer s'ha quedat sense números?

> 🔄 **Respostes**
>
> 1. `printf()` (o `format()`), igual que `System.out.printf`.
> 2. `new Scanner(new File("ruta.txt"))` en comptes de `new Scanner(System.in)`.
> 3. Per a canviar el separador per defecte (espais) per un altre, aquí el `;`. Útil per a CSVs.
> 4. `false`. Per això el `while (sc.hasNextInt())` es deté quan ja no queden números.

✅ Resum en 3 frases

1. **`PrintWriter`** escriu en fitxers amb `println`, `print` i `printf`: la comoditat de la consola amb destinació de fitxer.
2. **`Scanner` + `File`** llegeix fitxers a tokens (`nextInt`, `nextLine`), i amb **`useDelimiter`** pots tallar per `;`, comes o el que necessites.

3. Regla ràpida: **`Scanner`** per a parsejar dades soltes, **`BufferedReader`** per a fitxers grans línia a línia, i tots dos dins de **`try-with-resources`**.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `PrintWriter` | Escriptor amb `println`/`printf` sobre fitxers |
> | `useDelimiter()` | Canvia el separador amb què `Scanner` troceja |
> | Token | Cada trosset en què `Scanner` talla l'entrada |
> | `\\R` | "Qualsevol salt de línia" en regex de Java |
> | CSV | Fitxer de valors separats per comes (o per `;`) |

📁 Tornar a l'índex de la unitat · **Anterior:** 03 · try-with-resources: tancar sense plorar · **Següent:** 05 · NIO: Files i Paths (la forma moderna)

05 · NIO: Files i Paths (la forma moderna)

Llegir, escriure i consultar fitxers en una línia: l'API NIO va arribar perquè el vell File semble del segle passat ⚡🗃️

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 05 · NIO: Files i Paths (la forma moderna)

📬 La idea en una frase

> **`Path` és la ruta moderna i `Files` la seva navalla suïssa: `readAllLines`, `write`, `readString`… coses que amb `File` costaven cinc línies i amb NIO són una.**

Des de Java 7 existeix l'API **NIO** (New I/O) en `java.nio.file`. És el reemplaçament modern de la classe `File`: `Path` per a representar rutes i `Files` per a fer tot lo altre. ¿Llegir un fitxer sencer en una llista de línies? Una línia de codi. ¿Escriure una llista en un fitxer? Una altra. El vell `File` continua funcionant, però quan vegis això ja no voldràs tornar.

🗃️ Path i Paths: el nou File

`Path` és la ruta (com `File`, però més lleugera i pensada per a compondre's). Es crea amb la classe `Paths`:

```
import java.nio.file.Path;
import java.nio.file.Paths;

Path ruta = Paths.get("C:/datos/notas.txt");
Path relativa = Paths.get("notas.txt");              // relativa al directori de treball
Path combinada = Paths.get("datos", "sub", "f.txt"); // amb separadors a prova de SO
```

`Paths.get(...)` amb diversos arguments construeix la ruta usant el separador del sistema operatiu (en Windows `\`, en Linux `/`). És la forma de no liar-te amb barres.

> 💡 **Consell:** els mètodes de `Path` (`getFileName()`, `getParent()`, `exists()`, `isDirectory()`…) són els mateixos conceptes de `File`. La diferència gran està en `Files`, la classe estàtica que fa el treball pesat.

⚡ Files: la navalla suïssa

La classe estàtica `Files` és on viu la màgia. Aquests són els mètodes que usaràs el 90% del temps:

```
import java.nio.file.*;
import java.util.List;

public class NioFacil {
    public static void main(String[] args) throws Exception {
        Path ruta = Paths.get("notas.txt");

        // Escriure una llista de línies (sobreescriu)
        Files.write(ruta, List.of("Línea 1", "Línea 2", "Línea 3"));

        // Llegir tot el fitxer en una llista de línies
        List<String> lineas = Files.readAllLines(ruta);
        for (String linea : lineas) {
            System.out.println(linea);
        }

        // Llegir tot el fitxer com un únic String
        String contenido = Files.readString(ruta);

        // Altres consultes útils
        System.out.println("¿Existe? " + Files.exists(ruta));
        System.out.println("¿Es archivo? " + Files.isRegularFile(ruta));
        System.out.println("Tamaño: " + Files.size(ruta) + " bytes");
    }
}
```

> ⚠️ **Advertència:** `Files.write` i `readAllLines` carreguen el fitxer sencer en memòria. Per a fitxers gegants usa `Files.newBufferedReader(ruta)` (com al punt 2, però sense `FileReader` pel mig) i llegeix línia a línia.

📋 Altres joies de Files

Mètode	Què fa
`Files.readAllLines(ruta)`	Tot el fitxer com a `List<String>`
`Files.write(ruta, llista)`	Escriu una llista de línies (sobreescriu)
`Files.readString(ruta)`	Tot el fitxer com un únic `String`
`Files.writeString(ruta, text)`	Escriu un `String`
`Files.newBufferedReader(ruta)`	`BufferedReader` directe sobre un `Path`
`Files.newBufferedWriter(ruta)`	`BufferedWriter` directe sobre un `Path`
`Files.copy(origen, destí)`	Copia un fitxer
`Files.move(origen, destí)`	Mou (o reanomena)
`Files.delete(ruta)`	Esborra (llança excepció si no existeix)
`Files.deleteIfExists(ruta)`	Esborra si existeix, sense excepció
`Files.walk(ruta)`	Recorre recursivament tot un arbre de fitxers
`Files.lines(ruta)`	Un `Stream<String>` línia a línia (per a fitxers grans)

La parella `Files.walk` + filtre és la forma elegant de "busca tots els `.java` en una carpeta i les seves subcarpetes":

```
import java.nio.file.*;
import java.util.stream.Stream;

try (Stream<Path> rutas = Files.walk(Paths.get("src"))) {
    rutas.filter(p -> p.toString().endsWith(".java"))
         .forEach(System.out::println);
}
```

> 📝 **Nota:** no passa res si el `Stream` t'aclapara: és la UO7 amb esteroides. L'important ara és el patró: `Files.walk` et dona totes les rutes i tu filtres la que t'interessa.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** NIO usa `Path` i `Files`. Són més moderns i tenen mètodes útils com `Files.walk()` per a recórrer arbres. Quan vegis codi nou de Java, el 80% de les vegades usa NIO.

Exercici: Be the NIO

```
import java.nio.file.*;
import java.util.*;

public class BeTheNIO {
    public static void main(String[] args) throws Exception {
        Path p = Paths.get("nums.txt");
        Files.write(p, List.of("3", "7", "2", "9", "5"));
        List<String> l = Files.readAllLines(p);
        int suma = 0;
        for (String s : l) {
            suma += Integer.parseInt(s);
        }
        Files.write(p, List.of("Total: " + suma));
        System.out.println(Files.readString(p));
    }
}
```

Què imprimeix?

> 🔄 **Solució**
>
> Imprimeix **`Total: 26`**.
>
> Pas a pas:
>
> 1. `Files.write` crea `nums.txt` amb les línies 3, 7, 2, 9, 5.
> 2. `readAllLines` les llegeix i el bucle les converteix a `int` i les suma: `3+7+2+9+5 = 26`.
> 3. `Files.write` **sobreescriu** el fitxer amb una sola línia: `Total: 26`.
> 4. `Files.readString` llegeix tot el fitxer i `println` ho imprimeix: `Total: 26`.
>
> Fixa't que el segon `Files.write` no afegeix: sobreescriu. Si volguessis afegir, tindries `Files.write(p, llista, StandardOpenOption.APPEND)`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Amb quina classe crees un `Path`?
2. Quin mètode de `Files` llegeix un fitxer sencer com a `List<String>`?
3. Quin mètode de `Files` recorre un arbre de carpetes recursivament?
4. `Files.write` afegeix al final o sobreescriu?

> 🔄 **Respostes**
>
> 1. Amb `Paths` (`Paths.get(...)`). El plural té raó de ser: `Path` és la ruta, `Paths` la fàbrica.
> 2. `Files.readAllLines(ruta)`.
> 3. `Files.walk(ruta)`, que torna un `Stream<Path>` amb tot el que hi ha a sota.
> 4. Sobreescriu. Per a afegir, `Files.write(ruta, llista, StandardOpenOption.APPEND)`.

✅ Resum en 3 frases

1. **`Path`** (creat amb `Paths.get`) és la ruta moderna i **`Files`** la classe estàtica que fa el treball: `readAllLines`, `write`, `readString`, `copy`, `move`, `delete`, `walk`.
2. Llegir o escriure un fitxer complet són **operacions d'una línia**, sense `close()` perquè NIO s'encarrega.
3. `Files` ho carrega tot en memòria: per a fitxers enormes usa `Files.newBufferedReader` o `Files.lines` i processa en streaming.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
NIO	"New I/O": l'API moderna d'entrada/sortida de `java.nio.file`
`Path`	La ruta moderna (el `File` nou)
`Paths`	Fàbrica de `Path`
`Files`	Classe estàtica amb les operacions sobre rutes
`Stream`	Seqüència de dades sobre la qual pots filtrar, transformar i recórrer

📁 Tornar a l'índex de la unitat · **Anterior:** 04 · PrintWriter i Scanner + File · **Següent:** 06 · Serialització: guardar objectes amb ObjectOutputStream

06 · Serialització: guardar objectes amb ObjectOutputStream

El teu objecte sencer en una maleta: implementa Serializable i guarda i recupera objectes complets amb ObjectOutputStream i ObjectInputStream 🧳📦

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 06 · Serialització: guardar objectes amb ObjectOutputStream

---

📬 La idea en una frase

> **La serialització converteix un objecte sencer en una seqüència de bytes que pots guardar en un fitxer, i la deserialització el reconstruïx tal qual. La teva `Persona`, el teu `Estudiant`, el teu `ArrayList` sencer, en una maleta.**

Fins ara guardaves text. Però i si vols guardar un **objecte complet** amb tots els seus atributs? ¿O un `ArrayList` sencer? Fer-ho camp a camp en un `FileWriter` és un horror. La serialització et deixa "congelar" l'objecte, guardar-lo en un fitxer i "descongelar-lo" després amb totes les dades intactes.

🧳 El tracte: implementar Serializable

Perquè un objecte es pugui serialitzar, la seva classe ha d'implementar la interfície **`Serializable`**. No té mètodes: és una *marca* que li diu a Java "aquest objecte pot congelar-se".

```
import java.io.Serializable;

class Persona implements Serializable {
    String nombre;
    int edad;

    Persona(String n, int e) {
        this.nombre = n;
        this.edad = e;
    }
}
```

> ⚠️ **Advertència:** si un atribut de la classe no és serialitzable (per exemple, un `Thread` o un `Socket`), llançaràs una `NotSerializableException`. Solucions: marca l'atribut com a `transient` (no es guarda) o fes que la seva classe també sigui `Serializable`.

📦 Guardar: ObjectOutputStream

Per a escriure objectes uses `ObjectOutputStream` embolicat sobre un `FileOutputStream`:

```
import java.io.*;

public class GuardandoObjetos {
    public static void main(String[] args) throws Exception {
        Persona p = new Persona("Luis", 25);

        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("persona.obj"))) {
            oos.writeObject(p);
        }
        System.out.println("Objeto guardado.");
    }
}
```

> 💡 **Consell:** l'extensió `.obj` (o `.ser`/`.dat`) és purament convencional: el fitxer és binari, no l'obris amb el bloc de notes. Si l'obris, veuràs brossa i algun nom de classe llegible: és la firma que Java usa per a saber què deserialitzar.

🧊 Recuperar: ObjectInputStream

Per a llegir-lo de tornada, `ObjectInputStream` sobre `FileInputStream`. El resultat és un `Object`, així que necessites un **casting** (ja saps: el repartiment de l'`instanceof` de la U11):

```java
import java.io.*;

public class RecuperandoObjetos {
    public static void main(String[] args) throws Exception {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("persona.obj"))) {
            Persona recuperada = (Persona) ois.readObject();
            System.out.println(recuperada.nombre + " tiene " + recuperada.edad);
        }
    }
}
```

⚠️ **Advertència:** `readObject()` pot llançar `ClassNotFoundException` si la classe no existeix en deserialitzar (per exemple, si canvies de projecte o reanomenes la classe). Per això el mètode declara `throws Exception` o captura les tres: `IOException`, `ClassNotFoundException`.

🧨 I si la classe canvia entre guardar i llegir?

Aquí ve la trampa clàssica: guardes l'objecte, modifiques la classe (afegeixes un camp), i en llegir… `InvalidClassException`. Java guarda un identificador de versió de la classe i, si canvia, es queixa. La solució: declara tu l'identificador:

```java
class Persona implements Serializable {
    private static final long serialVersionUID = 1L;  // jo controle la versió
    String nombre;
    int edad;
}
```

Amb `serialVersionUID` fix, Java no recalcula la versió i tolera petits canvis. Sense ell, qualsevol retoc en la classe romp els fitxers guardats abans. És d'aquelles coses que dolen una vegada i després no s'obliden.

📚 Guardar col·leccions senceres

¿Una llista d'objectes? També es serialitza d'una vegada. És la forma més còmoda de "persistir" les teves dades de la U12:

```java
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class GuardandoLista {
    public static void main(String[] args) throws Exception {
        List<Persona> equipo = new ArrayList<>();
        equipo.add(new Persona("Ana", 20));
        equipo.add(new Persona("Bob", 22));
        equipo.add(new Persona("Carla", 19));

                try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("equipo.obj"))) {
            oos.writeObject(equipo);
        }

                 try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("equipo.obj"))) {
            List<Persona> recuperado = (List<Persona>) ois.readObject();
            for (Persona p : recuperado) {
                System.out.println(p.nombre + " - " + p.edad);
            }
        }
    }
}
```

> 📝 **Nota:** `ArrayList` i les seves classes ja són `Serializable`, i una llista és serialitzable si els seus elements ho són. Per això funciona sense tocar res. Persistir col·leccions amb serialització és el "guardar partida" dels teus programes.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** la serialització no guarda el codi, guarda l'**estat**: els valors dels atributs. Per a reconstruir l'objecte, la classe ha d'existir i ser compatible. Si necessites que les dades les llegeixes un altre programa (no Java), la serialització no val: hauràs d'usar text, JSON o XML.

Exercici: l'objecte que viatja en el temps

```
import java.io.*;

class Nota implements Serializable {
    private static final long serialVersionUID = 1L;
    String texto;
    int prioridad;

    Nota(String t, int p) {
        this.texto = t;
        this.prioridad = p;
    }
}

public class ViajeTemporal {
    public static void main(String[] args) throws Exception {
        Nota original = new Nota("Comprar leche", 5);

                try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("nota.obj"))) {
            oos.writeObject(original);
        }

        original.texto = "Comprar café";

                 try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("nota.obj"))) {
            Nota copia = (Nota) ois.readObject();
                System.out.println("Texto: " + copia.texto + " (prioridad " +
copia.prioridad + ")");
        }
    }
}
```

Què imprimeix? ¿Canvia el fitxer perquè vas modificar l'objecte en memòria abans de llegir-lo?

> 🔄 **Solució**
>
> Imprimeix **`Texto: Comprar leche (prioridad 5)`**.
>
> L'`ObjectOutputStream` va guardar l'objecte **tal com estava** en el moment del `writeObject`: `"Comprar leche"` amb prioritat `5`. Quan després canvies `original.texto` a `"Comprar`

> `café"`, només canvia l'objecte en memòria, no el fitxer. En llegir, el fitxer reconstruïx la versió guardada: `"Comprar leche"`.
>
> La lliçó: la serialització és una **foto** de l'objecte en un instant. Modificar l'objecte després de guardar-lo no afecta el fitxer. Perquè el canvi es reflecteixi, hauríes de tornar a serialitzar.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina interfície ha d'implementar una classe per a poder serialitzar-se?
2. Quina excepció llança si un atribut no és serialitzable?
3. Què fa `transient` en un atribut?
4. Per a què serveix `serialVersionUID`?

> 🔄 **Respostes**
>
> 1. `Serializable` (és una interfície *marca*, sense mètodes).
> 2. `NotSerializableException`.
> 3. Evita que aquell atribut es guardi: no es serialitza. Útil per a camps que no es poden (o no s'han de) guardar.
> 4. Per a fixar la versió de la classe. Amb ell, petits canvis en la classe no rompen els fitxers ja guardats.

✅ Resum en 3 frases

1. Una classe **`Serializable`** (sense mètodes) pot congelar-se: `ObjectOutputStream.writeObject` la guarda en un fitxer binari i `ObjectInputStream.readObject` la reconstruïx amb un casting.
2. Si un atribut no és serialitzable, `NotSerializableException`; marca'l **`transient`** per a excloure'l.
3. Fixa un **`serialVersionUID`** perquè els teus fitxers sobrevisquin a petits canvis en la classe, i recorda que la serialització guarda una **foto** de l'estat, no el codi.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Serialitzar	Convertir un objecte en bytes per a guardar-lo o enviar-lo
Deserialitzar	Reconstruir l'objecte a partir dels bytes
`Serializable`	Interfície-marca que permet serialitzar una classe
`transient`	Atribut que no es guarda en serialitzar
`serialVersionUID`	Versió de la classe per a compatibilitat
Binari	Fitxer que no és text llegible (`.obj`, `.dat`, `.ser`)

📁 Tornar a l'índex de la unitat · **Anterior:** 05 · NIO: Files i Paths (la forma moderna) · **Següent:** 07 · Expressions regulars: Pattern i Matcher

07 · Expressions regulars: Pattern i Matcher

El motlle, el text i l'infern de les contrabarres: aprèn a crear patrons amb Pattern i a buscar-los amb Matcher 🔍🧩

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 07 · Expressions regulars: Pattern i Matcher

📬 La idea en una frase

> **Una expressió regular (regex) és un patró que descriu un conjunt de cadenes: `"\\d+"` significa "un o més dígits". Amb `Pattern` la compiles i amb `Matcher` la busques dins d'un text.**

Fins aquí guardaves dades en fitxers. Ara toca l'altra meitat de la unitat: **buscar patrons dins de text**. ¿Quantes vegades has volgut "trobar tots els números" o "comprovar que això sembla un correu"? Les expressions regulars són el llenguatge per a descriure aquelles cerques. S'usen en quasi tots els llenguatges i, una vegada les aprens, no hi ha text que se't resista.

⚠️ L'infern de les contrabarres (llegeix-ho dues vegades)

En Java, les regex s'escriuen com a **cadenes** i, dins d'un `String`, la contrabarra `\` és un caràcter d'escapament. Per a escriure un `\d` de regex dins d'un `String` has de duplicar-la:

El que vols en la regex	El que escrius en Java
`\d` (dígit)	`"\\d"`
`\.` (punt literal)	`"\\."`
`\s` (espai)	`"\\s"`

> ⚠️ **Advertència:** si escrius `"\d"` Java intenta escapar la `d`, no sap què és i et dona un error de compilació (o pitjor, un comportament rar). L'"infern de les contrabarres" és la causa número 1 que una regex "no funcioni" quan acabes de començar.

🔍 Pattern i Matcher: el motlle i el text

El flux clàssic són dos passos:

- **`Pattern`**: l'expressió regular **compilada**. És el motlle.
- **`Matcher`**: s'aplica a un **text concret** i va buscant coincidències. És el detector.

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class PrimerRegex {
    public static void main(String[] args) {
          Pattern patron = Pattern.compile("\\d+");    // un o més dígits (el
motle)
        Matcher matcher = patron.matcher("Hay 123 manzanas y 456 peras");

        while (matcher.find()) {
            System.out.println("Encontrado: " + matcher.group()
                + " (posición " + matcher.start() + "-" + matcher.end() + ")");
        }
    }
}
```

Sortida:

```
Encontrado: 123 (posición 4-7)
Encontrado: 456 (posición 21-24)
```

- `find()` avança buscant la següent coincidència i torna `true` mentre en trobi.
- `group()` torna el text que ha coincidit.
- `start()` i `end()` tornen on comença i acaba la coincidència.

> 💡 **Consell:** compila el `Pattern` **una sola vegada** fora de bucles i reutilitza'l. `Pattern.compile()` és car (analitza la regex i munta una màquina d'estats). Si el crides dins d'un bucle d'1 milió de línies, te'l faràs mirar.

🧩 La taula de símbols (la teva xulleta d'examen)

Símbol	Significat	Exemple
`.`	Qualsevol caràcter (excepte salt de línia)	`c.sa` → “casa”, “cose”
`\d`	Dígit (0-9)	`\d{3}` → “123”
`\D`	NO dígit	`\D+` → “Hola”
`\w`	Lletra, dígit o _	`\w+` → “Hola_123”
`\W`	NO `\w`	`\W` → “.”, “ “
`\s`	Espai en blanc	`\s+` → separadors
`\S`	NO espai	`\S+` → paraules
`*`	0 o més vegades	`a*` → “”, “a”, “aa”
`+`	1 o més vegades	`a+` → “a”, “aa”
`?`	0 o 1 vegada (opcional)	`colou?r` → “color”, “colour”
`{n}`	Exactament n	`\d{3}` → tres dígits
`{n,m}`	Entre n i m	`\d{2,4}` → de 2 a 4 dígits
`[abc]`	Un del conjunt	`[aeiou]` → vocals
`[a-z]`	Rang	`[a-z]` → minúscules
`[^abc]`	Negació	`[^0-9]` → no dígits
`( )`	Grup de captura	`(\d+)-(\w+)`
`^`	Inici de línia	`^Hola`
`$`	Final de línia	`mundo$`
`|`	OR lògic	`gato|perro`
`\b`	Límit de paraula	`\bJava\b` no coincideix amb “JavaScript”

> 📝 **Nota:** `\w` NO inclou accents ni ñ per defecte. Per a “pérez” o “muñoz” necessites `[a-zA-ZáéíóúüñÑ]` o la bandera `Pattern.UNICODE_CHARACTER_CLASS`. Cosa d’examen, apunta-t’ho.

🕵️ Grups de captura: els parèntesis que recorden

Els parèntesis `( )` no només agrupen: **capturen** el que coincideix per a poder extreure-ho després. És la base del punt 8, però convé veure-ho ja:

```
Pattern patron = Pattern.compile("(\\w+): (\\d+) años");
Matcher matcher = patron.matcher("Juan: 28 años, María: 32 años");

while (matcher.find()) {
      System.out.println(matcher.group(1) + " tiene " + matcher.group(2) + "
años");
}
// Juan tiene 28 años
// María tiene 32 años
```

- `group(0)` (o `group()`) és tota la coincidència.
- `group(1)`, `group(2)`… són els parèntesis en ordre d’obertura.
- Si agrupes sense voler capturar (per rendiment): `(?:...)`.

🚩 Les banderes (flags)

`Pattern.compile` accepta un segon argument amb banderes:

```
Pattern    p1    =    Pattern.compile("java",    Pattern.CASE_INSENSITIVE);
// Java, JAVA, java...
Pattern p2 = Pattern.compile("^\\d+", Pattern.MULTILINE);                    //
^ a l'inici de cada línia
Pattern p3 = Pattern.compile(".*", Pattern.DOTALL);                          //
el . també captura salts
Pattern    p4    =    Pattern.compile("java",    Pattern.CASE_INSENSITIVE    |
Pattern.MULTILINE);  // combinades
Pattern p5 = Pattern.compile("(?i)java");                                    //
flag inline: (?i) = case-insensitive
```

`(?i)` dins de la pròpia regex fa el mateix que la bandera, sense segon argument. Les banderes es combinen amb `|`.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** les regex es proven amb `matches()` (tot el string) o `find()` (subcadena). `group()` extrau el capturat amb parèntesis. Si la teva regex no troba res, revisa les contrabarres abans que la lògica.

Exercici: el caçador d'ànecs

```
import java.util.regex.*;

public class CazadorPatos {
    public static void main(String[] args) {
        String texto = "Hay 3 patos, luego 45 patos, y al final 678 patos";
        Pattern patron = Pattern.compile("\\d+ patos");
        Matcher matcher = patron.matcher(texto);

        int total = 0;
        while (matcher.find()) {
             String grupo = matcher.group();                // "3 patos", "45
patos"...
            String numero = grupo.split(" ")[0];           // "3", "45"...
            total += Integer.parseInt(numero);
        }
        System.out.println("Patos totales: " + total);
    }
}
```

Què imprimeix? I si canvies `matcher.find()` per `matcher.matches()`?

> › 🔄 **Solució**
>
> Imprimeix **`Patos totales: 726`** (`3 + 45 + 678`).
>
> `find()` va saltant de coincidència en coincidència: primer "3 patos", després "45 patos", després "678 patos". De cada grup s'extrau la part numèrica amb `split(" ")` i s'acumula.

> I ull amb la segona pregunta (trampa): amb `matches()`, la resposta seria **cap coincidència** (o `false`). `matches()` exigeix que **tot** el string complica amb el patró, i aquí hi ha més text al voltant. Per a buscar subcadenes usa sempre `find()`. És l'error més repetit de la unitat.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Com escrius en Java la regex `\d`?
2. Què torna `matcher.find()`?
3. Quin mètode de `Matcher` et dona el text que ha coincidit?
4. Per què convé compilar el `Pattern` fora d'un bucle?

> 🔄 **Respostes**
>
> 1. `"\\d"`. Dins d'un `String`, la contrabarra es duplica.
> 2. `true` si troba una coincidència (i avança a la següent). `false` quan no en queden.
> 3. `group()` (o `group(0)`), i `group(n)` per al grup de captura `n`.
> 4. Perquè `Pattern.compile()` és car: analitza la regex i munta la màquina de cerca. Reutilitzar el `Pattern` és gratis i molt més ràpid.

✅ Resum en 3 frases

1. Una **regex** descriu un conjunt de cadenes amb símbols (`\d`, `\w`, `+`, `{n}`, `[a-z]`…), i en Java les contrabarres es **dupliquen** (`"\\d"`).
2. **`Pattern.compile`** compila la regex (el motlle) i **`matcher.find()`** busca coincidències en un text: `group()`, `start()` i `end()` et donen els detalls.
3. `matches()` exigeix que **tot** el string complica amb el patró; `find()` busca subcadenes. No els confonguis: és l'error número 1.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Regex	Patró de cerca que descriu un conjunt de cadenes

Terme	Idea general
`Pattern`	La regex compilada (el motlle)
`Matcher`	El buscador aplicat a un text concret
`find()`	Busca la següent coincidència
`group()`	El text que ha coincidit
Grup de captura	Parèntesis que “recorden” part de la coincidència
Metacaràcter	Símbol amb significat especial en una regex (`.`, `+`, `\d`…)

📁 Tornar a l’índex de la unitat · **Anterior:** 06 · Serialització: guardar objectes amb ObjectOutputStream · **Següent:** 08 · Regex en acció: matches, replaceAll, split i validacions

08 · Regex en acció: matches, replaceAll, split i validacions

Del motlle al món real: valida correus i DNIs, neteja textos amb replaceAll, troceja amb split i extrau dades amb grups de captura 🧰🛠️

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 08 · Regex en acció: matches, replaceAll, split i validacions

---

📬 La idea en una frase

> **Aquí la regex deixa de ser teoria: valides un correu amb `matches()`, neteges espais amb `replaceAll()`, troceges una frase amb `split()` i extraus els camps d'un log amb grups de captura.**

El punt 7 et va donar el martell (`Pattern` i `Matcher`). Aquest punt és el taller: aplicacions reals que usaràs tots els dies. A més de `Pattern`/`Matcher`, la classe `String` té mètodes que accepten regex directament, i són els que més ràpid et faran la vida.

🎯 String.matches(): ¿complica TOT el string?

`matches()` torna `true` només si **tot** el text complica amb el patró. És l'eina perfecta per a validar.

```
"123".matches("\\d+");          // true  (tot són dígits)
"Hola".matches("\\d+");         // false (no són dígits)
"12345678Z".matches("\\d{8}[A-Z]");  // true (8 dígits + majúscula)
"1234567Z".matches("\\d{8}[A-Z]");   // false (falten dígits)
```

> ⚠️ **Advertència:** `matches()` emparella **tot** el string. Per a buscar subcadenes dins d'un text usa `find()` (punt 7). Aquest error es cobra el seu preu en cada examen.

🧹 replaceAll(): netejar text en una línia

`replaceAll(regex, reemplaçament)` substituïx **totes** les coincidències del patró. `replaceFirst()` només la primera.

```
String texto = "  Hola    mundo  de las   regex  ";

String limpio = texto.replaceAll("\\s+", " ").trim();
// "Hola mundo de las regex"

texto.replaceFirst("\\s+", " ").trim();
// "Hola    mundo  de las   regex"  (només el primer bloc d'espais)
```

El patró `"\\s+"` = “un o més espais” → el substituïx per un sol espai. Amb `.trim()` neteges els extrems. Resultat: text normalitzat en dues crides.

> 💡 **Consell:** els reemplaçaments també accepten grups: `"Hola Ana".replaceAll("Hola (\\w+)", "Adiós $1")` → `"Adiós Ana"`. El `$1` recupera el grup 1 en el reemplaçament.

🔪 split(): trocejar per patró

`split(regex)` divideix el string per les coincidències i torna un `String[]`.

```
"a,b,c,d".split(",");        // ["a", "b", "c", "d"]
"a,b,c,d".split(",", 3);     // ["a", "b", "c,d"]  (amb límit)
"hola  mundo   java".split("\\s+");   // ["hola", "mundo", "java"] (un o més espais)
"a1b22c333".split("\\d+");   // ["a", "b", "c", ""]
```

Fixa’t en l’últim: si el string acaba amb dígits, `split` deixa una cadena buida al final. És un clàssic que mossega. I `split` amb un punt (`"."`) no troceja per punts, perquè `.` és “qualsevol caràcter”: necessites `"\\."`.

🎫 Validacions de la vida real

Aquí ho juntem tot: `matches()` + patrons compilats una vegada com a constants. El validador clàssic de correu, DNI i telèfon:

```
import java.util.regex.Pattern;

public class ValidadorRegex {
    private static final Pattern PATRON_DNI = Pattern.compile("\\d{8}[A-Z]");
    private static final Pattern PATRON_EMAIL =
        Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}");
    private static final Pattern PATRON_TELEFONO =
        Pattern.compile("[679]\\d{8}");

    public static boolean esEmailValido(String email) {
        return PATRON_EMAIL.matcher(email.toLowerCase()).matches();
    }

    public static boolean esDNIValido(String dni) {
        return PATRON_DNI.matcher(dni).matches();
    }

    public static boolean esTelefonoValido(String telefono) {
        return PATRON_TELEFONO.matcher(telefono).matches();
    }

    public static void main(String[] args) {
        System.out.println(esEmailValido("user@example.com"));   // true
        System.out.println(esEmailValido("user@@example"));      // false
        System.out.println(esDNIValido("12345678Z"));            // true
           System.out.println(esDNIValido("12345678z"));                // false
(minúscula)
        System.out.println(esTelefonoValido("612345678"));       // true
            System.out.println(esTelefonoValido("512345678"));          // false
(comença per 5)
    }
}
```

📝 **Nota:** `[\\w.]+@[\\w.]+\\.[a-z]{2,}` es llegeix: “lletres/punts, una @, lletres/punts, un punt, i almenys 2 lletres”. És un format **bàsic**: no comprova si el domini existeix ni si l’adreça és real. I el DNI: la regex només verifica el **format** (8 dígits + lletra); per a validar la lletra de veritat caldria l’algoritme mòdul 23. Les regex validen forma, no veritat.

🕵️ El processador de logs (grups de captura + fitxers) 🔗

L'aplicació estrella: llegir un log i extreure'n els camps amb grups. Recorda del punt 2 com llegir fitxers, i aquí li sumes la regex:

```
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ProcesadorLog {
    public static void main(String[] args) throws Exception {
        Pattern patron = Pattern.compile(
            "\\[(ERROR|INFO|WARN)\\]\\s+" +
            "(\\d{4}-\\d{2}-\\d{2})\\s+" +
            "(\\d{2}:\\d{2}:\\d{2})\\s+-\\s+(.*)");

        Path entrada = Paths.get("app.log");
        Path salida = Paths.get("errores.txt");

        try (BufferedReader br = Files.newBufferedReader(entrada);
             PrintWriter pw = new PrintWriter(Files.newBufferedWriter(salida)))
{

            String linea;
            while ((linea = br.readLine()) != null) {
                Matcher matcher = patron.matcher(linea);
                if (matcher.find()) {
                    String nivel = matcher.group(1);
                    String fecha = matcher.group(2);
                    String hora = matcher.group(3);
                    String mensaje = matcher.group(4);
                       System.out.printf("[%s] %s a las %s: %s%n", nivel, fecha,
hora, mensaje);
                    if (nivel.equals("ERROR")) {
                        pw.println(fecha + " " + hora + " " + mensaje);
                    }
                }
            }
        }
        System.out.println("Errores extraídos a " + salida);
    }
}
```

Amb una sola regex traus el nivell, la data, l'hora i el missatge de cada línia. I si la línia és un ERROR, l'escrius en un fitxer a part. Regex + fitxers: la unitat sencera en un programa.

> 💡 **Consell:** per a un log enorme no usis `Files.readAllLines` (punt 5): carrega tot en memòria. Llegeix **línia a línia** amb `BufferedReader` i aplica el `Matcher` a cadascuna. Memòria constant, tant se val que el log pese 2 GB.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** cada parèntesi captura una part. `(.*)` al final captura "tot el que quedi": és el clàssic per a la resta de la línia. Si una línia no coincideix, `find()` torna `false` i no passa res: el bucle segueix.

Exercici: el formatejador de dates

```
public class FormateadorFechas {
    public static void main(String[] args) {
        String fecha = "15/03/2024";
            String convertida = fecha.replaceAll("(\\d{2})/(\\d{2})/(\\d{4})",
"$3-$2-$1");
        System.out.println(convertida);
    }
}
```

Què imprimeix? I què passa si la data ve amb guions `15-03-2024`?

> › 🔄 **Solució**
>
> Imprimeix **`2024-03-15`**: passa de `dd/mm/aaaa` a `aaaa-mm-dd`.
>
> Els grups capturen el dia (`$1`), el mes (`$2`) i l'any (`$3`), i el reemplaçament `$3-$2-$1` els reordena. Aquell és el truc per a reordenar parts de text amb regex sense tocar res més.
>
> I la trampa: amb `15-03-2024` **no coincideix res** (el patró espera `/`, no `-`), així que `replaceAll` no toca el string i torna `15-03-2024` tal qual. Per a suportar tots dos separadors hauríes d'usar `[\\/\\-]` o dues crides. Les regex són literals: no endevinen, compleixen ordres.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode de `String` valida que TOT el text complica amb un patró?
2. Com lleves els espais dobles d'un text amb una línia?
3. Què torna `"a,b,c".split(",")`?
4. En `replaceAll("(\\w+)@(\\w+)", "$2@$1")`, què fa `$2`?

> 🔄 **Respostes**
>
> 1. `matches()`.
> 2. `texto.replaceAll("\\s+", " ").trim()`.
> 3. Un `String[]` amb `["a", "b", "c"]`.
> 4. Recupera el segon grup de captura (el que va coincidir amb el segon parèntesi) i el col·loca en el reemplaçament: intercanvia les dues parts.

✅ Resum en 3 frases

1. **`matches()`** valida que tot el text complica amb el patró (correus, DNIs, telèfons); **`replaceAll`** neteja i reordena text; **`split`** troceja pel patró.
2. Els **grups de captura** extrauen parts (`group(n)` en Java, `$n` en els reemplaçaments) i són la base per a processar logs i CSV.
3. Per a fitxers grans, aplica la regex **línia a línia** amb `BufferedReader` (no carreguis el fitxer sencer en memòria).

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`matches()`	¿Tot el string complica amb el patró?
`replaceAll()`	Substituïx totes les coincidències
`replaceFirst()`	Substituïx només la primera
`split()`	Troceja el string pel patró
`$n`	Grup n en un reemplaçament de `replaceAll`

Terme	Idea general
Validar	Comprovar el format d'una dada (no la seva veracitat)

📁 Tornar a l'índex de la unitat · **Anterior:** 07 · Expressions regulars: Pattern i Matcher · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu: domina fitxers i regex

El tancament pràctic de la unitat, amb un close() que doldrà, una regex que fallarà i un objecte que viatjarà en el temps 😈

> 🗺️ **Estàs en:** 📁 **U14 · Fitxers i expressions regulars (Regex)** → 09 · Repàs interactiu

---

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```java
import java.io.*;
import java.util.regex.*;

public class Misterio {
    public static void main(String[] args) throws Exception {
        File f = new File("datos.txt");
        if (!f.exists()) {
            FileWriter w = new FileWriter(f);
            w.write("Ana;20\nBob;35\nCarla;28");
            w.close();
        }

        Pattern patron = Pattern.compile("(\\w+);(\\d+)");
        BufferedReader r = new BufferedReader(new FileReader(f));
        String linea;
        int suma = 0;
        while ((linea = r.readLine()) != null) {
            Matcher m = patron.matcher(linea);
            if (m.matches()) {
                suma += Integer.parseInt(m.group(2));
            }
        }
        r.close();
        System.out.println("Suma: " + suma);
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`Suma: 83`** → La regex captura nom i edat, i se sumen 20 + 35 + 28. ✅
2. **`Suma: 0`** → `m.matches()` no troba res perquè el patró no quadra amb les línies. ❌
3. **`Suma: 83` però només la primera vegada** → El fitxer es crea la primera execució, però la suma és sempre la mateixa. ❌

> 🔄 **Solució**
>
> L'opció **1**. El fitxer es crea amb tres línies del format `Nom;edat`. La regex `(\\w+);(\\d+)` captura el nom (grup 1) i l'edat (grup 2), i com que `matches()` exigeix que **tota** la línia quadre (i quadra), suma `20 + 35 + 28 = 83`. Tant se val quantes vegades l'executes: el fitxer ja existeix i la sortida és sempre la mateixa.

🔥 Fireside Chat: File vs Path

> *Dues generacions de l'API de fitxers es troben davant la màquina de cafè.*

File: — Porti aquí des de Java 1.0. Tota una vida localitzant fitxers: `exists()`, `isFile()`, `length()`… Soc el clàssic.

Path: — I jo vaig arribar en Java 7 amb tota la modernitat: `Paths.get(...)`, i el meu company `Files` fa `readAllLines`, `write` i `readString` en una línia.

File: — I per a què em necessites a mi aleshores? Jo tinc `listFiles()` per a recórrer carpetes.

Path: — Jo tinc `Files.walk()` que recorre **arbres sencers** recursivament i te'ls dona en un `Stream`. Amb filtre, a més.

File: — Val, però continues depenent de mi en alguns llocs… i de `FileReader`, `FileWriter` i companyia per a llegir i escriure.

Path: — Cert, el trio `FileWriter/FileReader/BufferedReader` és etern i funciona de meravella. La família de `java.io` no desapareixerà. Però per a allò modern, `Files` i `Paths` són la meva lliga.

> La lliçó: **`java.io` (File, FileReader, FileWriter, BufferedReader) continua sent vàlid i l'usaràs tota la vida. `java.nio.file` (Paths, Files) és la forma moderna i preferida en codi nou.** Saber els dos et fa flexible; saber-ne només un, limitat.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc el GPS que localitza fitxers, però no llegeix contingut.**
2. **Soc el lector que et dona línies senceres i mai s'ennuega.**
3. **Soc el bloc que tanca els fitxers per tu, passi el que passi.**
4. **Soc la interfície-marca que permet guardar un objecte en un fitxer.**
5. **Soc el motlle compilat d'una expressió regular.**
6. **Soc el mètode que exigeix que TOT el string complica amb el patró.**

> ▸ 🔄 **Respostes**
>
> 1. **`File`** — representa rutes, no contingut.

2. **`BufferedReader`** — `readLine()` sense parpellejar.
3. **`try-with-resources`** — tanca automàticament en sortir del bloc.
4. **`Serializable`** — sense mètodes, només la marca per a congelar objectes.
5. **`Pattern`** — la regex compilada; `Matcher` és qui busca.
6. **`matches()`** — del `String`, valida tot el text.

🤬 CONRAD VS EL MÓN: "No vaig tancar el fitxer"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre els clàssics del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu fitxer s'ha quedat a mig fer*. I jo: val, a veure el teu codi? *Pues mira, escric amb FileWriter…* I NO TANQUES! I tant que s'ha quedat a mig fer! Les dades viuen al buffer fins que arriba el `close()`. Et fa gràcia escriure cartes i no tancar el sobre? Pues això. Des de Java 7 tens `try-with-resources` per a tancar sense plorar. USA'L!

I després està el de la regex. Escriu `"abc123".matches("\\d+")` i s'estranya que doni `false`. Però si hi ha lletres pel mig! `matches()` exigeix que TOT el string siguin dígits. Per a buscar subcadenes, `find()`. Porta tota la unitat dient-ho!

I el de la serialització: guarda un objecte, canvia la classe afegint un camp, i es queixa que en llegir-li salta `InvalidClassException`. I tant! No li posares el `serialVersionUID` i ara plora… Posa-l'hi fix i deixa de plorar.

La lliçó: el trio sagrat de la unitat: **¿tanqui els recursos amb `try-with-resources`? ¿usi `find()` per a buscar i `matches()` només per a validar tot el string? ¿la meva classe té `Serializable` i un `serialVersionUID` fix?** El 90% dels ensurts d'aquesta unitat s'arreglen amb aquestes tres preguntes. El compilador no t'odia: t'està passant les respostes de l'examen.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. `new File("notas.txt")` … què fa en el disc?
    - a) Crea el fitxer buit b) Res, només representa la ruta
2. Quin mètode de `String` torna `true` només si TOT el text és un correu vàlid?

- a) `matches()` b) `find()`

3. Què llança llegir amb `FileReader` un fitxer que no existeix?
   - a) `FileNotFoundException` b) `NullPointerException`
4. Quin mètode de `Files` (NIO) llegeix un fitxer complet com a `List<String>`?
   - a) `Files.readAllLines()` b) `Files.readString()`
5. Per a afegir al final d'un fitxer amb `FileWriter` sense esborrar l'anterior…
   - a) `new FileWriter("f.txt", true)` b) `new FileWriter("f.txt", false)`

> 🔄 **Solucions**
>
> 1. **b)** — `new File("ruta")` no crea res: només l'objecte que representa la ruta. Per a crear, `createNewFile()` o un `FileWriter`.
> 2. **a)** — `matches()` valida el string sencer. `find()` busca subcadenes dins del text.
> 3. **a)** — `FileNotFoundException`, una filla de `IOException`.
> 4. **a)** — `readAllLines` torna `List<String>`; `readString` torna un únic `String`.
> 5. **a)** — el segon argument `true` activa el mode *append*.

⚡ Laboratori de Tortura: el validador que no valida

> **Durada estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Et diuen que té **2 errors de compilació i 1 error de lògica**… però, i si t'ho diuen malament? La teva tasca: fer que compili, que executi i que **tota** la sortida sigui correcta, comptant tu els errors reals.

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Validador {
    public static void main(String[] args) {
        String[] correos = {"ana@example.com", "bob@@example", "carla@site.es"};

        Pattern patron = Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}")

        int validos = 0;
        for (String c : correos) {
            Matcher m = patron.matcher(c);
            if (m.find()) {
                System.out.println(c + " → válido");
                validos++;
            } else {
                System.out.println(c + " → inválido");
            }
        }
        System.out.println("Válidos: " + validos)
    }
}
```

Fallada intencionada: el nombre d'errors de l'enunciat està malament a propòsit. La trampa és la teva confiança en l'enunciat.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Compila? *no → mira els ; i els imports.*
   > **› I si continuec atascat?**
   > Falta un `;` després del `Pattern.compile(...)` i un altre al final del `println`. Els imports estan bé.

2. Ja compila? *aleshores mira el resultat.*
   > **› I si continuec atascat?**
   > El codi usa `find()`. Quants correus creus que haurien de passar la validació? Torna a mirar què significa `find()` davant de `matches()`.

3. Creus que hi ha un error de lògica? *Torna a llegir el codi amb calma i no et fies de l'enunciat.*

   > **Solució final**

Els **errors de compilació**:

1. Falta el `;` al final de `Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}")`.
2. Falta el `;` al final del `System.out.println("Válidos: " + validos)`.

L'**error de lògica**: no existeix. L'enunciat deia "2 errors de compilació i 1 de lògica", però només hi ha 2 faltes de `;` i cap lògica trencada. Aquella era la trampa: la teva confiança en l'enunciat.

Sortida correcta:

```
ana@example.com → válido
bob@@example → inválido
carla@site.es → válido
Válidos: 2
```

Amb `find()`, "bob@@example" NO passa: el patró `[\\w.]+@[\\w.]+\\.[a-z]{2,}` necessita lletres, una `@`, més lletres, un punt i almenys dos lletres. "bob@@example" té dos `@` seguides i cap punt, així que no hi ha subcadena que complica amb el patró.

La lliçó real: **llegeix el codi, no l'enunciat**. Compta, verifica i confia en el compilador, no en la llegenda.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🏅 **L'Explorador**	Llistar el contingut d'una carpeta amb `File.listFiles()` distingint fitxers i carpetes
🏅 **El Copió**	Copiar un fitxer de text línia a línia amb `BufferedReader` + `PrintWriter` i `try-with-resources`
🏅 **El Sergent**	Formatar una taula alineada en un fitxer amb `PrintWriter.printf`
🏅 **El Maleter**	Guardar i recuperar una `ArrayList` d'objectes amb la serialització

Assoliment	Com aconseguir-ho
🏅 El Detectiu	Extreure nivell, data i missatge d'un log amb una regex i grups de captura
🏅 El Porter	Validar correus, DNIs o telèfons amb `matches()` i patrons compilats

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```
import java.io.*;

public class Acertijo {
    public static void main(String[] args) throws Exception {
        try (FileWriter fw = new FileWriter("nums.txt")) {
            fw.write("5\n10\n15\n");
        }
        try (BufferedReader br = new BufferedReader(new FileReader("nums.txt")))
{
            int suma = 0;
            String linea;
            while ((linea = br.readLine()) != null) {
                suma += Integer.parseInt(linea);
            }
            System.out.println(suma);
        }
    }
}
```

2. **El netejador:** tens `String texto = "Hola mundo Java"` i vols deixar-lo en un sol `String[]` amb les tres paraules. Què escrius?
3. **El detectiu del log:** una línia de log és `[ERROR] 2026-09-06 10:30:45 - Conexión fallida: server=db01`. Quin grup captura `server=db01` amb el patró del punt 8? ¿Coincideix amb `matches()` o només amb `find()`?
4. **Vertader o fals:** "`Files.readAllLines` és la millor opció per a un fitxer de 2 GB".

> 💡 **Solucions**

1. **30**. `5 + 10 + 15`. El `try-with-resources` del `FileWriter` escriu les tres línies i tanca; el del `BufferedReader` les llegeix i suma. Fixa't que el `FileWriter` sense `try-with-resources` hauria pogut deixar-te la meitat sense escriure.
2. `texto.split("\\s+")` → `["Hola", "mundo", "Java"]`. El `\\s+` talla per "un o més espais", just el que sobrava.
3. `server=db01` està en el **grup 4** (`(.*)`, "tot el que quedi"). I ull: amb `matches()` no coincidiria si el patró no cobreix la línia sencera des del principi; `find()` busca subcadenes sense exigir començar a l'inici. Recorda: `matches()` = patró complet, `find()` = cerca de subcadena.
4. **Fals.** `readAllLines` ho carrega tot en memòria. Per a 2 GB usa `BufferedReader` o `Files.lines(...)` i processa en streaming, línia a línia.

🧩 Crucigrama de Bits

```
Horizontal:
1. Interfície-marca per a guardar objectes en fitxers (11 lletres)
3. Mètode de String que valida que TOT el text complica amb el patró (7 lletres)
5. Classe que embolica FileReader per a llegir línies senceres (15 lletres)
6. API moderna de fitxers: Files i Paths (3 lletres)
7. El que torna readLine() quan s'acaba el fitxer (4 lletres)

Vertical:
2. Bloc que tanca recursos automàticament (16 lletres)
4. Classe que representa una ruta sense llegir contingut (4 lletres)
```

> 📝 **Solucions**
>
> **Horizontal:** 1. SERIALIZABLE · 3. MATCHES · 5. BUFFEREDREADER · 6. NIO · 7. NULL **Vertical:** 2. TRYWITHRESOURCES · 4. FILE

💬 Preguntes d'Entrevista de Treball

> Preguntes reals que et farien per a programador Java junior.

1. **"Explica-m'ho, com si jo fos la teva àvia, la diferència entre `File`, `FileReader` i `FileWriter`."**

2. **“Què és `try-with-resources` i per què hauries d’usar-lo sempre amb fitxers?”**
3. **“Quina és la diferència entre `find()` i `matches()` en una regex?”**
4. **“Com guardaries i recuperaries una llista d’objectes en un fitxer?”**
5. **“Què és l’infern de les contrabarres en les regex de Java?”**
6. **“Quan usaries `BufferedReader` en comptes de `Scanner`, i al revés?”**

🤷 No hi ha preguntes tontes

> ❓ **`File` i `Path` són el mateix?**

Quasi. Tots dos representen rutes. `File` és de l’API antiga (`java.io`) i `Path` de la moderna (`java.nio.file`). `Path` + `Files` és el recomanat en codi nou, però `File` continua funcionant i el veuràs en projectes antics. Saber els dos és ser un programador complet.

> ❓ **Puc guardar qualsevol objecte amb la serialització?**

Només si la seva classe (i totes les seves classes d’atributs) implementa `Serializable`. Si no, `NotSerializableException`. I la serialització és de Java a Java: si un altre programa (Python, JavaScript) ha de llegir les teves dades, usa text, CSV o JSON.

> ❓ **Es poden validar documents HTML amb regex?**

No. HTML no és un llenguatge regular: té etiquetes niades que les regex no poden rastrejar. Per a això existeixen els *parsers*. Les regex són per a text pla amb patrons, no per a arbres d’etiquetes. Aquella pregunta se la fan tots, i la resposta és la mateixa: “no”.

> ❓ **`\w` reconeix els accents i la ñ?**

No per defecte. `\w` és `[a-zA-Z0-9_]`. Per a “pérez” o “muñoz” usa `[a-zA-ZáéíóúüñÑ]` o la bandera `Pattern.UNICODE_CHARACTER_CLASS`.

🎬 Post-Crèdits

La programadora acaba el seu gestor de contactes: guarda cada contacte amb `Serializable`, els carrega en arrancar amb `ObjectInputStream`, valida el correu de cadascun amb una regex

compilada i escriu un informe bonic amb `PrintWriter.printf`. Funciona. Fins que intenta llegir el fitxer des d'una altra màquina i li surt `ClassNotFoundException`.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Has vist? La serialització guarda les dades, però el fitxer se la fot amb qualsevol: només val per al mateix programa amb la mateixa classe. Quan vulguis que les teves dades les entenga qualsevol sistema, hauràs d'anar-te'n de viatge: guardar-les en una base de dades.

Programadora: — Una base de dades? I com conecte Java amb una?

CONRAD: — *assenteix* Amb JDBC: una connexió, un `Statement`, i les teves dades deixen d'estar en un fitxer per a viure en taules amb consultes. Tu solta el `FileWriter` i ves preparant el SQL.

PROXIMAMENT EN U15: Connexió a bases de dades amb JDBC. El moment en què els teus programes aprenen a parlar amb un gestor de bases de dades relacional. 🗄️

📁 Tornar a l'índex de la unitat · **Anterior:** 08 · Regex en acció: matches, replaceAll, split i validacions · **Següent:** **U15 · Persistència de dades: JDBC i introducció a ORM**

U15 — Persistència de dades: JDBC i introducció a ORM

Java aprèn a parlar amb les bases de dades: JDBC, SQLite, Connection, Statement, PreparedStatement, el patró DAO, transaccions i un primer contacte amb els ORM. Tot amb humor i sense perdre la connexió 🗄️

En la U14 vas tancar el tracte amb els **fitxers**: guardar i llegir informació al disc. Avui pugem de nivell. Aquella informació deixa de viure en un fitxer solt i passa a una **base de dades relacional**, amb les seves taules, les seves claus i les seves regles. I Java, és clar, hi parlarà. L'eina es diu **JDBC** (Java Database Connectivity) i és el pont oficial entre el teu codi i qualsevol base de dades amb controlador.

Aquesta unitat té quatre grans actes:

- **Connectar i consultar:** què és JDBC, com es munta la dependència de **SQLite** a Maven i el ritual dels 5 passos per a obrir una connexió (`Connection`), llançar consultes (`Statement`) i llegir els resultats (`ResultSet`).
- **El CRUD segur:** inserir, llegir, actualitzar i esborrar amb **`PreparedStatement`**, la vacuna contra l'**SQL injection** (pregunta-li a Bobby Tables), i el **patró DAO** perquè l'SQL no es cole en la teva lògica de negoci.
- **La integritat: transaccions** amb `commit` i `rollback` per a operacions de tot-o-res, i un decàleg de bones pràctiques perquè la teva connexió no sigui una fuga de recursos.
- **El pont amb els ORM:** després de domar el JDBC a mà, veuràs què és un **ORM** (mapeig objecte-relacional) i per què els equips de veritat el fan servir per a no escriure tant SQL de catàleg.

Pel camí coneixeràs `SQLException`, l'excepció checked que t'acompanyarà a cada pas, i entendràs per què `executeQuery()` i `executeUpdate()` són com la porta d'embarcament i la del maleter: cada operació té la seva.

Aquesta unitat es llegeix com un **llibre de 10 capítols**: els 9 primers punts són teoria en progressió i el 10 és un aterratge pràctic per a deixar-ho tot ben lligat.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar **què és JDBC** i per què serveix per a qualsevol base de dades amb controlador.
- Configurar la dependència de **SQLite** a Maven i connectar el teu programa amb `DriverManager.getConnection()`.
- Executar consultes amb **`Statement`** i llegir els resultats amb **`ResultSet`** usant `next()` i els `getXxx("columna")`.
- Aplicar el **CRUD complet** (INSERT, SELECT, UPDATE, DELETE) amb `executeQuery()` i `executeUpdate()`.
- Usar **`PreparedStatement`** amb placeholders `?` i evitar l'**SQL injection** sense excuses.
- Aplicar el **patró DAO** (interfície + implementació) per a separar l'SQL de la lògica de negoci.
- Gestionar **transaccions** amb `commit()`, `rollback()` i savepoints per a operacions atòmiques.
- Escriure codi JDBC seguint el **decàleg de bones pràctiques**: `try-with-resources`, files afectades i `WHERE` sempre.
- Explicar **què és un ORM** i en què es diferencia d'escriure JDBC a mà.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · Què és JDBC	JDBC, SQLite i la dependència Maven que ho posa tot en marxa	Tots
02 · Connexió a SQLite	Els 5 passos del ritual i la interfície `Connection`	Tots
03 · Statement i ResultSet	Executar consultes i processar files amb `next()` i `getXxx`	Tots
04 · CRUD complet	Inserir, llegir, actualitzar i esborrar amb `executeQuery` i `executeUpdate`	Tots
05 · PreparedStatement i SQL injection	Placeholders `?`, `setXxx` i la història de Bobby Tables	Tots

Punt	Què aprendràs	Dificultat
06 · El patró DAO	Interfície + implementació: l'SQL no trepitja la teva lògica de negoci	Tots
07 · Transaccions	`commit`, `rollback` i savepoints: tot o res	Tots
08 · Bones pràctiques	El decàleg del JDBC: recursos, files afectades i `WHERE` sempre	Tots
09 · Introducció a ORM	Què és un ORM, mapeig objecte-relacional i JDBC vs ORM	Tots
10 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 9 primers punts són teoria en progressió. El 10 és l'aterratge pràctic: llegeix-lo just després del 9 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

✅ Criteris d'avaluació coberts (RA9)

RA9: Gestiona informació emmagatzemada en bases de dades relacionals mantenint la integritat i la consistència de les dades.

CE	Criteri	On es cobreix
RA9 a)	S'han identificat les característiques i mètodes d'accés a sistemes gestors de bases de dades relacionals.	✅ Punts 1 i 2

CE	Criteri	On es cobreix
RA9 b)	S'han programat connexions amb bases de dades.	✅ Punt 2
RA9 c)	S'ha escrit codi per a emmagatzemar informació en bases de dades.	✅ Punts 4 i 5
RA9 d)	S'han creat programes per a recuperar i mostrar informació emmagatzemada en bases de dades.	✅ Punts 3 i 4
RA9 e)	S'han efectuat esborrats i modificacions sobre la informació emmagatzemada.	✅ Punt 4
RA9 f)	S'han creat aplicacions que executin consultes sobre bases de dades.	✅ Punts 3, 5 i 10
RA9 g)	S'han creat aplicacions per a possibilitar la gestió d'informació present en bases de dades relacionals.	✅ Punts 6 i 10

> 📌 Aquesta unitat cobreix la **RA9** completa. El camí fins aquí: la **U04** et va ensenyar les excepcions (i `SQLException` és checked, et perseguirà), la **U09/U10** et va donar les classes i interfícies amb què muntar els POJO i el DAO, i la **U14** et va deixar el `try-with-resources` llest per a tancar connexions. Tot el material anterior cobra sentit ara: és hora que els teus objectes dormin en una base de dades.

🚪 Per on començo?

- Vens de la U14 (fitxers) i vols l'essencial? → Comença en el punt 1 i el punt 2: connectar és el 90% de la batalla.
- Ja connectes i vols gestionar dades? → Ves directe al punt 4 (el CRUD) i al punt 5 (fer-ho sense que et pirategen).
- Només vens a pels trucs? → Salta al punt 5 (l'SQL injection) i al punt 8 (el decàleg).
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · Què és JDBC

⏭️ **En acabar la unitat, continua en U16 · Servir i consumir APIs amb Web.**

01 · Què és JDBC

JDBC, l'USB de les bases de dades: Java parla amb SQLite, MySQL o PostgreSQL gràcies a un controlador. I la dependència Maven que ho encén tot 🗄️🔌

> 🗺️ **Estàs en: 🗄️ U15 · Persistència de dades: JDBC i introducció a ORM** → 01 · Què és JDBC

📬 La idea en una frase

> **JDBC (Java Database Connectivity) és el traductor universal entre el teu codi Java i qualsevol base de dades que tingui un controlador: tant fa SQLite, MySQL o PostgreSQL, el codi que escrius és quasi el mateix.**

En la U14 vas aprendre a guardar dades en fitxers: llistes planes, text pla, fitxers solts. Però un fitxer no sap respondre a «dona'm tots els alumnes majors de 18». Per a això estan les **bases de dades relacionals**: taules, files, columnes i consultes. I perquè Java pugui parlar-hi existeix JDBC, un conjunt d'interfícies del paquet `java.sql`. Benvingut al punt on el teu programa comença a tenir memòria de llarg termini.

🔌 JDBC: l'USB de les bases de dades

Pensa en JDBC com el port USB: tant fa què hi endollis (SQLite, MySQL, PostgreSQL, Oracle…), la clavilla és la mateixa. Cada base de dades aporta el seu **controlador** (en anglès, *driver*): un `.jar` que sap traduir les crides genèriques de JDBC al dialecte d'aquella base de dades en concret.

```
El teu programa Java  ──JDBC──►  Controlador  ──►  Base de dades
   (java.sql)          (genèric)   (l'endoll)        (SQLite, MySQL...)
```

La regla d'or: si canvies de base de dades i vas usar SQL estàndard, només canvies el **controlador** i la **URL de connexió**. La resta del codi ni se n'assabenta. Aquella és la màgia de JDBC.

> 💡 **Consell:** el paquet `java.sql` conté les interfícies clau: `Connection`, `Statement`, `PreparedStatement` i `ResultSet`. Aquest punt les presenta; els següents les desmunten una a una.

🗄️ SQLite: la base de dades d'entrenament

Per a aprendre no necessites muntar un servidor ni obrir ports. **SQLite** és una base de dades que viu en un **únic fitxer local**: sense servidor, sense usuari, sense contrasenya. Si el fitxer no existeix, SQLite el crea sol la primera vegada que et connectes. Perfecta per a practicar, i de passada és la que usen un munt d'aplicacions reals.

> 📝 **Nota:** en el curs usem SQLite perquè no necessita instal·lació. Tot el que aprens aquí (CRUD, PreparedStatement, transaccions) funciona igual en MySQL o PostgreSQL canviant el controlador i la URL.

📦 La dependència Maven

Per a usar el controlador de SQLite en un projecte Maven, afegeixes aquesta dependència al `pom.xml`:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.45.1.0</version>
</dependency>
```

Quan Maven la descarrega, el controlador queda al teu *classpath* i Java el troba sol. Des de Java 6 no necessites ni escriure `Class.forName(...)` per a carregar-lo: JDBC 4.0 s'auto-descobreix. Si veus `Class.forName()` en tutorials antics, és que aquell tutorial es va escriure quan es feia a mà. No passa res si el poses; tampoc passa res si no el poses.

> ⚠️ **Advertència:** si Maven no troba la classe `org.sqlite.JDBC` en executar, el 99% de les vegades és que la dependència no està al `pom.xml` o no s'ha descarregat. Primer revisa això, i després el teu codi.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què significa JDBC i en quin paquet viu?
2. Quin paper fa el controlador (driver)?

3. Per què SQLite és ideal per a aprendre?
4. Què has de canviar si passes de SQLite a MySQL?

> 🔄 **Respostes**
>
> 1. **Java Database Connectivity**, un conjunt d'interfícies en el paquet `java.sql`.
> 2. Tradueix les crides genèriques de JDBC al dialecte d'una base de dades concreta. És l'endoll.
> 3. Perquè no necessita servidor: viu en un únic fitxer local que es crea sol.
> 4. Només el controlador (la dependència) i la URL de connexió. El codi SQL estàndard es queda igual.

✅ Resum en 3 frases

1. **JDBC** és el pont estàndard de Java cap a qualsevol base de dades amb controlador: interfícies en `java.sql` que no canvien encara que canviï la base de dades.
2. Cada base de dades (SQLite, MySQL, PostgreSQL…) aporta el seu **controlador**, que tradueix les crides genèriques al seu dialecte.
3. Usem **SQLite** perquè és un únic fitxer sense servidor, i a Maven s'afegeix amb una dependència d'una línia.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
JDBC	Java Database Connectivity: el pont Java ↔ base de dades
Controlador (driver)	L'endoll específic per a cada base de dades
`java.sql`	El paquet amb les interfícies de JDBC
SQLite	Base de dades en un únic fitxer, sense servidor
Dependència Maven	La línia al `pom.xml` que porta el controlador

📚 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · Connexió a SQLite: els 5 passos i Connection

02 · Connexió a SQLite: els 5 passos i Connection

El ritual dels 5 passos per a connectar Java amb SQLite i la interfície Connection, la canonada per on viatgen les teves consultes 🗄️🔗

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 02 · Connexió a SQLite

---

📬 La idea en una frase

> **Connectar-se a SQLite és un ritual de 5 passos: `DriverManager.getConnection("jdbc:sqlite:instituto.db")` et torna una `Connection`, i aquella canonada serà la que usi la resta del codi.**

Si el punt 1 era conèixer JDBC de lluny, aquí t'hi acostes a saludar. Obrir una connexió és el primer que fa qualsevol programa amb base de dades, i també el que més vegades oblidem tancar. Anem a muntar el ritual complet i a entendre què és exactament aquell objecte `Connection` que usaràs en cada exercici de la unitat.

🪜 Els 5 passos (i el bonus no opcional)

Tot programa JDBC segueix aquesta escala:

1. **Carregar el controlador** — des de Java 6, automàtic. Només te'n recordes que existeix quan llegeixes codi antic.
2. **Establir la connexió** — `DriverManager.getConnection(url)` et torna una `Connection`.
3. **Crear un Statement** — `con.createStatement()` (o `con.prepareStatement(sql)`, ho veuràs al punt 4).
4. **Executar la consulta** — `executeQuery()` per a SELECT, `executeUpdate()` per a INSERT/UPDATE/DELETE.
5. **Processar els resultats** — recórrer el `ResultSet` amb `next()`.

I el **bonus no opcional**: tancar-ho tot. En Java modern, `try-with-resources` ho fa sol (ho vas veure a la U14 amb els fitxers; aquí és exactament la mateixa idea). Un recurs que no es tanca

és una fuga: en un servidor, cada connexió oberta consumeix memòria i hi ha un límit. Arriba el següent `getConnection()` i… "Too many connections".

> 💡 **Consell:** memoritza els 5 passos en ordre. El pas 4 i el 5 es repeteixen en cada consulta que escriguis la resta de la teva vida. I el bonus de tancar… que no se t'oblide mai.

🔗 Connection: la canonada

`Connection` és la interfície que representa la connexió oberta amb la base de dades. És la canonada per on viatgen les teves consultes. Per a SQLite, que no té usuari ni contrasenya, la URL és simplement la ruta d'un fitxer:

```
String url = "jdbc:sqlite:instituto.db";
Connection con = DriverManager.getConnection(url);
```

> 📝 **Nota:** si el fitxer `instituto.db` no existeix, SQLite el crea automàticament. No has de muntar la base de dades a mà abans de connectar: la primera vegada que la uses, ja hi serà.

El famós protocol "jdbc:sqlite:" és la URL estàndard: `jdbc` és el protocol, `sqlite` és el subprotocol (quina base de dades) i la resta és el destí (per a MySQL seria una cosa com `jdbc:mysql://localhost:3306/instituto`).

SQLException, la que et perseguirà

`DriverManager.getConnection(...)` llança **`SQLException`**, una excepció *checked*: el compilador t'obliga a capturar-la o declarar-la. Vas aprendre les excepcions a la UO4; aquí conviuràs amb aquesta a diari. El correcte no és empassar-te-la en silenci, sinó mostrar què ha passat:

```
try {
    Connection con = DriverManager.getConnection(url);
    System.out.println("Connectat a SQLite!");
} catch (SQLException e) {
    System.err.println("Error de connexió: " + e.getMessage());
    System.err.println("Codi d'error: " + e.getErrorCode());
    System.err.println("Estat SQL: " + e.getSQLState());
}
```

> ⚠️ **Advertència:** no facis mai `catch (Exception e) {}` i et quedes tan ample. Això és tapar la llum del "check engine" amb esparadrap: l'error continua sent-hi, només que ja ningú el veu. Captura `SQLException`, i si vols saber més, usa `getErrorCode()` i `getSQLState()`.

🧠 La connexió completa amb try-with-resources

El patró que usaràs a tota la unitat. Obre la connexió, fes el que hagis de fer i oblida't d'històries:

```
String url = "jdbc:sqlite:instituto.db";

try (Connection con = DriverManager.getConnection(url)) {
    System.out.println("Connectat. Base de dades llesta.");

    // ací anirien les consultes (Statement, ResultSet...)

} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
// la Connection es tanca sola en eixir del try
```

`try-with-resources` tanca els recursos en **ordre invers** al d'obertura. Si obris `Connection`, `Statement` i `ResultSet`, es tanquen ResultSet → Statement → Connection. Sol i sense que hagis de pensar. És màgic, i és la forma correcta.

> 💡 **Consell:** no comparteixis una `Connection` entre fils (ho veuràs més endavant): no és *thread-safe*. Si algun dia necessites diverses connexions alhora, usa un pool de connexions com HikariCP. Per a aquest curs, una connexió per operació amb `try-with-resources` és més que suficient.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què torna `DriverManager.getConnection(url)`?
2. Què fa SQLite si el fitxer de la base de dades no existeix?
3. Quina excepció llança `getConnection()` i per què no pots ignorar-la?

4. En quin ordre es tanquen els recursos amb `try-with-resources`?

> 🔄 **Respostes**
>
> 1. Una **`Connection`**, la canonada cap a la base de dades.
> 2. El crea automàticament. No has de muntar la base de dades a mà.
> 3. **`SQLException`**, una checked que el compilador t'obliga a gestionar.
> 4. En ordre invers al d'obertura: ResultSet → Statement → Connection.

✅ Resum en 3 frases

1. Connectar-se a SQLite és un **ritual de 5 passos** el cor del qual és `DriverManager.getConnection("jdbc:sqlite:instituto.db")`.
2. `Connection` és la **canonada** per on viatgen les teves consultes; si no la tanques, es queda oberta i esgota els recursos del servidor.
3. `getConnection()` llança **`SQLException`** (checked), i la forma correcta de treballar és embolicar-ho tot en **`try-with-resources`** perquè es tanqui sol.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`DriverManager`	La classe que reparteix connexions (`getConnection`)
`Connection`	La canonada oberta cap a la base de dades
URL JDBC	`jdbc:sqlite:archivo.db` — protocol, subprotocol i destí
`SQLException`	La checked que t'acompanya en tot JDBC
`try-with-resources`	Tancament automàtic de recursos, ordre invers

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · Què és JDBC · **Següent:** 03 · Statement i ResultSet: consultar dades

03 · Statement i ResultSet: consultar dades

Llança consultes amb Statement, processa files amb ResultSet i decideix entre executeQuery i executeUpdate sense ficar una forquilla al microones 🗄️📊

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 03 · Statement i ResultSet

---

📬 La idea en una frase

> **Amb `Statement` envies la consulta i amb `ResultSet` reps les files: `rs.next()` avança a la següent fila (començant abans de la primera) i `rs.getInt("id")` / `rs.getString("nombre")` llegeix cada columna.**

Ja tens la canonada (`Connection`). Ara toca la part divertida: **preguntar-li** a la base de dades. El missatger que porta la teva pregunta es diu `Statement`, i la resposta arriba en un `ResultSet`, que és com una taula virtual amb les files que han coincidit. Anem a veure com es llança una consulta i com es processa fila a fila.

📤 Statement: el missatger

`Statement` és la forma més directa d'executar una consulta. La crees a partir de la connexió i li passes el SQL com un String:

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
```

`executeQuery()` és el mètode per a **SELECT**: torna un `ResultSet` amb les files que han coincidit. Si la consulta no torna res, no és un error: et dona un `ResultSet` buit, i `rs.next()` tornarà `false` directament.

> ⚠️ **Advertència:** no usis `executeQuery()` per a un INSERT, UPDATE o DELETE. Això llança una `SQLException`. `executeQuery()` és només per a consultes que tornen dades (SELECT); per a les altres està `executeUpdate()`. Cada cosa al seu lloc, com la forquilla i el microones.

📊 ResultSet: la taula virtual

`ResultSet` és la resposta de la base de dades: un conjunt de files. La clau és que el cursor comença **abans de la primera fila**, i `next()` l'avança a la següent. Quan no queden més files, `next()` torna `false`.

```
while (rs.next()) {
    System.out.println(rs.getInt("id") + ": " + rs.getString("nombre"));
}
```

El patró és sempre el mateix: `while (rs.next())` per a recórrer totes les files, i dins, `rs.getXxx("nom_columna")` per a llegir cada camp. El `getXxx` depèn del tipus: `getInt`, `getString`, `getDouble`, `getBoolean`, `getDate`…

Tipus en SQLite	Mètode de lectura
INTEGER	`getInt("columna")`
TEXT	`getString("columna")`
REAL	`getDouble("columna")`
BOOLEAN (0/1)	`getBoolean("columna")`

> 💡 **Consell:** també pots usar l'índex de columna en comptes del nom: `rs.getString(1)` és la primera columna, `rs.getString(2)` la segona… El nom és més llegible i no es trenca si reordena les columnes. Usa noms.

⚖️ executeQuery vs executeUpdate

Dos mètodes, dues missions. Aquesta taula et salva en els exàmens:

Mètode	Per a què	Torna
`executeQuery()`	SELECT	`ResultSet` (les files)
`executeUpdate()`	INSERT, UPDATE, DELETE	`int` (files afectades)

Regla mnemotècnica: si esperes **dades de tornada** (files amb columnes), `executeQuery()`. Si només vols saber **quantes files s'han modificat**, `executeUpdate()`. Barrejar-los dona `SQLException`: és com ficar la forquilla al microones, no hi ha volta enrere.

🧠 El programa complet

Tot junt, amb `try-with-resources` perquè no escapi cap recurs:

```
String url = "jdbc:sqlite:instituto.db";
String sql = "SELECT id, nombre FROM alumnos";

try (Connection con = DriverManager.getConnection(url);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.printf("%d: %s%n", rs.getInt("id"), rs.getString("nombre"));
    }

} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
```

Fixa't en l'ordre d'obertura: Connection → Statement → ResultSet. Amb `try-with-resources` es tanquen en ordre invers i no has de preocupar-te'n. Si la consulta és fixa i no té paràmetres variables, un `Statement` senzill és suficient. Quan hi hagi dades de l'usuari de per mig… això és cosa del punt 5.

> 💡 **Consell:** `Statement` serveix per a consultes literalment fixes. Per al 90% de les teves consultes reals (amb paràmetres que canvien) usaràs `PreparedStatement`. Ho veuràs molt aviat, no t'enamores del `Statement` encara.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. On comença el cursor d'un `ResultSet` abans de cridar `next()`?

2. Quin mètode usaries per a un `UPDATE` i què torna?
3. Què passa si `executeQuery()` no troba cap fila?
4. Com llegeixes un valor enter d'una columna anomenada `edad`?

> 🔄 **Respostes**
>
> 1. **Abans de la primera fila.** Per això el bucle és `while (rs.next())`.
> 2. `executeUpdate()` — torna un `int` amb les files afectades.
> 3. No és un error: et dona un `ResultSet` buit i `next()` torna `false` a la primera.
> 4. `rs.getInt("edad")`.

✅ Resum en 3 frases

1. `Statement` envia la consulta amb `executeQuery()` (SELECT) i torna un `ResultSet`, una **taula virtual** que es recorre amb `while (rs.next())`.
2. Cada fila es llegeix amb `rs.getXxx("columna")`, i el cursor comença **abans** de la primera fila, d'aquí el `next()` del bucle.
3. `executeUpdate()` (per a INSERT/UPDATE/DELETE) torna les **files afectades**, i usar `executeQuery()` per a aquelles operacions llança `SQLException`.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`Statement`	El missatger que executa una consulta fixa
`ResultSet`	La taula virtual amb les files tornades
`next()`	Avança a la següent fila (comença abans de la primera)
`executeQuery()`	Per a SELECT: torna `ResultSet`
`executeUpdate()`	Per a INSERT/UPDATE/DELETE: torna files afectades

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Connexió a SQLite · **Següent:** 04 · CRUD complet

04 · CRUD complet: inserir, llegir, actualitzar i esborrar

El CRUD complet amb JDBC: INSERT, SELECT, UPDATE i DELETE amb PreparedStatement, el POJO que guarda cada fila i el WHERE que et salva la vida 🗄️🔄

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 04 · CRUD complet

📬 La idea en una frase

> **CRUD són les 4 operacions bàsiques de qualsevol base de dades — Create, Read, Update, Delete — i amb JDBC es traduïxen en `INSERT`, `SELECT`, `UPDATE` i `DELETE` usant `executeUpdate()` per a les que modifiquen i `executeQuery()` per a les que consulten.**

Ja saps connectar i consultar. Ara toca la rutina de tota aplicació amb base de dades: **inserir, llegir, actualitzar i esborrar**. És el pa de cada dia. Aquí anem a construir l'exemple complet amb una taula `contactos`, un POJO (objecte Java, dels que vas conèixer a la U09) i els quatre mètodes. Prepara la cafetera.

🧱 La taula

Primer, la base de dades necessita una taula. Aquest SQL es pot executar una vegada des de qualsevol client SQLite (o amb un `Statement` i `executeUpdate`, que també val):

```
CREATE TABLE contactos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    telefono TEXT,
    email TEXT
);
```

> 📝 **Nota:** SQLite usa `INTEGER PRIMARY KEY AUTOINCREMENT` on MySQL usa `INT AUTO_INCREMENT`. El `id` s'assigna sol: tu no el tries, la base de dades el genera.

🧍 El POJO

Cada fila de la taula es converteix en un objecte Java. És la classe de sempre, amb atributs privats, getters i setters (la vas veure a la U09 i la U10):

```
public class Contacto {
    private int id;
    private String nombre;
    private String telefono;
    private String email;

    public Contacto() {}
    public Contacto(String nombre, String telefono, String email) {
        this.nombre = nombre;
        this.telefono = telefono;
        this.email = email;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTelefono() { return telefono; }
    public void setTelefono(String telefono) { this.telefono = telefono; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return id + " - " + nombre + " (" + telefono + ")";
    }
}
```

L'estratègia de tota la unitat: la base de dades torna files, i el codi les converteix en objectes `Contacto`. Així el teu programa treballa amb objectes de veritat, no amb `ResultSet` solts.

✍️ Create (INSERT)

Inserir un contacte. Fixa't en els `?`: són placeholders que s'omplen amb `setXxx`, començant en 1. Més sobre ells al punt 5; de moment, fia-te'n:

```
public void insertarContacto(Contacto c) {
    String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?, ?,
?)";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error en inserir: " + e.getMessage());
    }
}
```

No li diem a la base de dades quin `id` usar: el genera ella sola. Si volguessis saber quin ha estat, ho veuràs al butlletí avançat amb `RETURN_GENERATED_KEYS`.

📖 Read (SELECT)

Llegir tots els contactes. La consulta torna files i les convertim en una `List<Contacto>` (col·leccions, U12, fent de les seves):

```java
public List<Contacto> obtenerTodos() {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos ORDER BY nombre";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Contacto c = new Contacto(
                rs.getString("nombre"),
                rs.getString("telefono"),
                rs.getString("email")
            );
            c.setId(rs.getInt("id"));
            contactos.add(c);
        }
    } catch (SQLException e) {
        System.err.println("Error en consultar: " + e.getMessage());
    }
    return contactos;
}
```

> 💡 **Consell:** torna `List<Contacto>`, mai un `ResultSet`. El `ResultSet` està lligat a la seva connexió: si tanques la connexió, el `ResultSet` mor. Converteix a objectes Java i tanca-ho tot tranquil.

La cerca amb LIKE

I si volem buscar per una part del nom? Aquí entra `LIKE` amb comodins `%`:

```java
public List<Contacto> buscarPorNombre(String nombre) {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos WHERE nombre LIKE ?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, "%" + nombre + "%");
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email")
                );
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        }
    } catch (SQLException e) {
        System.err.println("Error en buscar: " + e.getMessage());
    }
    return contactos;
}
```

El % davant i darrere fa que coincideixi amb qualsevol contacte que contingui aquell text. Fixa't que els % van dins del **valor**, no en l'SQL: la concatenació aquí és segura.

✏️ Update (UPDATE)

Actualitzar un contacte. Aquí apareix l'invitat estrella: el `WHERE`. Sense ell, actualitzes TOTS els contactes de la taula:

```
public boolean actualizarContacto(Contacto c) {
    String sql = "UPDATE contactos SET nombre = ?, telefono = ?, email = ? WHERE
id = ?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.setInt(4, c.getId());
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error en actualitzar: " + e.getMessage());
        return false;
    }
}
```

> ⚠️ **Advertència:** sense `WHERE id = ?`, actualitzes TOTS els contactes. 500 contactes anomenats "John Doe" en un obrir i tancar d'ulls. **WHERE SEMPRE** en UPDATE i DELETE. És la primera norma del club de les bases de dades.

`executeUpdate()` torna el nombre de files afectades: si és `> 0`, s'ha actualitzat alguna cosa. Comprovar-ho és gratis i et diu si l'operació va funcionar de veritat.

🗑️ Delete (DELETE)

Esborrar un contacte pel seu id. La mateixa història: `WHERE`, o crema la taula:

```
public boolean eliminarContacto(int id) {
    String sql = "DELETE FROM contactos WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error en eliminar: " + e.getMessage());
        return false;
    }
}
```

> 💡 **Consell:** confirma amb l'usuari abans d'esborrar. El `DELETE` no té Ctrl+Z: és per a sempre. Com les decisions en una pel·lícula de terror, no es poden desfer.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode usa l'INSERT, `executeQuery()` o `executeUpdate()`?
2. Què passa si executes un `UPDATE` sense `WHERE`?
3. Per què convé tornar `List<Contacto>` i no un `ResultSet`?
4. En `LIKE ?`, on van els `%`?

> 🔄 **Respostes**
>
> 1. `executeUpdate()` — torna les files afectades.
> 2. Actualitza **totes** les files de la taula. Per això `WHERE` sempre.
> 3. Perquè el `ResultSet` està lligat a la connexió: en tancar-la, mor. Millor objectes Java.
> 4. Dins del **valor** (`"%" + nombre + "%"`), no en l'SQL.

✅ Resum en 3 frases

1. El **CRUD** són les 4 operacions bàsiques: `INSERT` (create), `SELECT` (read), `UPDATE` (update) i `DELETE` (delete).
2. Cada fila del `ResultSet` es converteix en un **objecte Java** (`Contacto`) i les consultes tornen `List<Contacto>` per a no dependre de la connexió oberta.
3. **`WHERE` sempre** en `UPDATE` i `DELETE`, i comprova les **files afectades** que torna `executeUpdate()` per a saber si l'operació va tenir efecte.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
CRUD	Create, Read, Update, Delete: l'abecedari de les dades
POJO	Objecte Java que representa una fila de la taula

Terme	Idea general
`LIKE ?`	Cerca per coincidència parcial amb `%` com a comodí
`WHERE`	El filtre que t'estalvia actualitzar/esborrar la taula sencera
Files afectades	El `int` que torna `executeUpdate()`

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Statement i ResultSet · **Següent:** 05 · PreparedStatement i SQL injection

05 · PreparedStatement i SQL injection

Concatena strings en SQL i Bobby Tables esborrarà la teva taula d'alumnes. La vacuna es diu PreparedStatement i els seus placeholders ? 🗄️💉

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 05 · PreparedStatement i SQL injection

---

📬 La idea en una frase

> **Mai construïscues SQL concatenant strings amb dades de l'usuari: és SQL injection. La vacuna és `PreparedStatement`, que separa l'SQL de les dades amb placeholders `?` i l'ompli de forma segura.**

Al punt 3 vas veure `Statement` i va quedar clar que serveix per a consultes fixes. Aquí arriba el moment en què et dic per què, així que hi hagi **dades de l'usuari** de per mig, el `Statement` es converteix en una bomba de rellotgeria. I de passada coneixeràs Bobby Tables, el xiquet més famós de la història de les bases de dades.

💣 Concatenar SQL: la bomba

Imagina una cerca d'alumnes per nom. La forma innocent (i fatal) de muntar-la:

```
// PERILL: això és una bomba
String sql = "SELECT * FROM alumnos WHERE nombre = '" + inputUsuario + "'";
```

Ara, un "usuari" escriu això al camp de cerca:

```
Luis'; DROP TABLE alumnos; --
```

La consulta que s'executa acaba sent:

```
SELECT * FROM alumnos WHERE nombre = 'Luis'; DROP TABLE alumnos; --'
```

Fixa't: tanca la cometa amb `'`, acaba la consulta amb `;`, **esborra la taula** amb `DROP TABLE alumnos`, i comenta la resta amb `--`. Adéu, taula d'alumnes. Això és **SQL Injection** i passa de

veritat, en aplicacions reals, cada dia.

> ⚠️ **Advertència:** MAI construïscues SQL concatenant strings amb dades de l'usuari. És com deixar les claus posades al cotxe amb un cartell a la porta que posa "PASSI VOSTÉ".

La història de Bobby Tables

```
Madre: "He criat al meu fill perquè siga un programador acurat,
        no un hacker que robe dades."
Fill:  "Hola, em dic Robert'); DROP TABLE Students; --"
```

No siguis Bobby Tables. Usa PreparedStatement.

💉 PreparedStatement al rescat

La solució és separar el **codi SQL** de les **dades**. L'SQL es prepara amb placeholders `?` i els valors es passen després, per separat:

```java
String sql = "SELECT * FROM alumnos WHERE nombre = ?";
PreparedStatement pstmt = conexion.prepareStatement(sql);
pstmt.setString(1, nombre);
ResultSet rs = pstmt.executeQuery();
```

El controlador escapa les dades automàticament: si l'usuari escriu `Robert'); DROP TABLE Students; --`, es tracta com un **valor**, no com a codi. La taula es queda on està.

Els `?` són **placeholders posicionals** i comencen en **1** (no en 0, com els arrays… trampa clàssica d'examen):

Tipus	Mètode
String	`setString(i, valor)`
int	`setInt(i, valor)`
double	`setDouble(i, valor)`
boolean	`setBoolean(i, valor)`

Tipus	Mètode
Date	`setDate(i, valor)` (usa `java.sql.Date`)
null	`setNull(i, Types.TIPO)`

> 💡 **Consell:** per a moltes consultes iguals, `PreparedStatement` pot ser **més ràpid** que `Statement`, perquè la base de dades compila la consulta una sola volta i reutilitza el pla. La seguretat la tens gratis, i de regal ve la velocitat.

🧪 Guia de placeholders

Diversos `?`, en ordre. El primer és el `1`, el segon el `2`, i així successivament:

```
String sql = "SELECT * FROM alumnos WHERE curso = ? AND edad > ? ORDER BY
nombre";
try (PreparedStatement pstmt = conexion.prepareStatement(sql)) {
    pstmt.setString(1, "DAM");
    pstmt.setInt(2, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

¿Veu qué net? No hi ha concatenació, no hi ha cometes escapades, no hi ha risc d'injecció. Només `?` nets i ordenats. Els placeholders només valen per a **valors**, mai per a noms de taula o columna: `SELECT * FROM ?` no funciona, i no hauria de funcionar: un identificador no s'escapa, es tria al codi.

⭐ Be the Code, my friend...

> 🕶️ **Don Tip:** PreparedStatement prevé l'SQL injection perquè separa el codi SQL de les dades. Mai concatenes strings en SQL, ni en broma.

Exercici: detecta l'intent d'atac

Un alumne t'ensenya aquest codi i et pregunta si és segur:

```
String nombre = sc.nextLine();
String sql = "SELECT * FROM alumnos WHERE nombre = '" + nombre + "'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

Què respondries? És segur? Què passaria si l'usuari escrivís `Ana'; DELETE FROM matriculas; --`? Com ho arreglaries?

> 🔄 **Solució**
>
> **No és segur, és SQL injection en estat pur.** Amb `Ana'; DELETE FROM matriculas; --`, l'SQL es converteix en un `SELECT` que acaba abans d'hora i després executa `DELETE FROM matriculas` (la instrucció que el `--` comenta només en la part sobrera). S'esborrarien totes les matrícules.
>
> La solució: `PreparedStatement` amb placeholder.
>
> ```
> String sql = "SELECT * FROM alumnos WHERE nombre = ?";
> PreparedStatement pstmt = con.prepareStatement(sql);
> pstmt.setString(1, nombre);
> ResultSet rs = pstmt.executeQuery();
> ```
>
> Ara `Ana'; DELETE FROM matriculas; --` és només un nom estrany que es busca tal qual. La taula respira tranquil·la.

🥊 El ring: Statement vs PreparedStatement

Dues formes d'executar SQL discuteixen davant la màquina de cafè.

Statement: — Jo soc l'original. Senzill, directe. `stmt.executeQuery("SELECT * FROM usuarios")`. Per a consultes fixes soc perfecte.

PreparedStatement: — Sí, però concatenar strings en SQL és una bomba de rellotgeria. Injecció SQL, errors de sintaxi amb cometes… Jo separo l'SQL de les dades amb `?` i soc segur.

Statement: — Per a una consulta fixa, una sola volta, per què he de preparar res?

PreparedStatement: — A més, jo faig cache del pla d'execució. Si executes la mateixa consulta diverses vegades amb diferents paràmetres, soc més ràpid. I en Java, quasi sempre executes la mateixa consulta amb diferents valors.

Statement: — Val, però jo serveixo per a DDL: CREATE TABLE, ALTER...

PreparedStatement: — Cert. Per a DDL usa Statement. Per a DML (SELECT, INSERT, UPDATE, DELETE) usa PreparedStatement. Tracte?

Statement: — Tracte.

> 🕶️ **Don Tip:** usa sempre `PreparedStatement` per a consultes amb dades d'usuari. No és només seguretat: és més ràpid en consultes repetitives i més llegible.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. En quin número comencen els índexs dels `?` en un `PreparedStatement`?
2. Què fa la base de dades amb `Robert'); DROP TABLE Students; --` si uses PreparedStatement?
3. Pots usar `?` per al nom d'una taula?
4. Per què `PreparedStatement` pot ser més ràpid en consultes repetides?

> › 🔄 **Respostes**
>
> 1. En **1**. El primer `?` és el `setXxx(1, ...)`. No hi ha cap `?` a la posició 0.
> 2. Ho tracta com un **valor** de text, escapat. No s'executa cap instrucció extra.
> 3. No. Els `?` només valen per a valors, no per a identificadors.
> 4. Perquè la base de dades **compila la consulta una sola volta** i reutilitza el pla d'execució.

✅ Resum en 3 frases

1. Concatenar SQL amb dades de l'usuari és **SQL injection**: una entrada com `'; DROP TABLE alumnos; --` pot esborrar taules senceres.
2. **`PreparedStatement`** separa l'SQL de les dades amb placeholders `?` i els `setXxx(i, valor)` (índexs des de 1), escapant els valors automàticament.

3. És la forma **segura**, la més **ràpida** per a consultes repetides i l'única que no et farà passar vergonya en una entrevista de treball.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | SQL injection | Injectar instruccions SQL a través de dades d'usuari |
> | Placeholder | El `?` que reserva el buit per a un valor |
> | `setXxx(i, valor)` | Ompli el placeholder número `i` (des de 1) |
> | Escapar | Neutralitzar caràcters perillosos d'un text |
> | Bobby Tables | El xiquet que t'ensenya per què no concatenar |

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · CRUD complet · **Següent:** 06 · El patró DAO

06 · El patró DAO

El patró DAO: una interfície que promet i una implementació que compleix, perquè l'SQL no es cole mai en la teva lògica de negoci 🗄️🏗️

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 06 · El patró DAO

📬 La idea en una frase

> **El patró DAO (Data Access Object) encapsula tot l'SQL en una classe separada, darrere d'una interfície: el teu programa parla amb objectes i mètodes (`insertarContacto`, `obtenerTodos`…), no amb sentències SQL soltes.**

Al punt 4 vam escriure els quatre mètodes del CRUD. Si els deixes solts pel programa, l'SQL s'enganxa a la lògica de negoci com un xicle a la sabata: cada vegada que canviï la base de dades, cal repassar el codi sencer buscant `getConnection`. El patró **DAO** posa ordre. És el tema de les interfícies que vas veure a la U10, aplicat a l'accés a dades.

🏛️ La idea: separar l'SQL del negoci

Què vol dir "separar la lògica de negoci de l'SQL"? Que el teu programa faci això:

```
ContactoDAO dao = new ContactoDAOImpl();
List<Contacto> contactos = dao.obtenerTodos();     // no hi ha ni un SELECT ací
Contacto c = new Contacto("María", "666000111", "maria@ies.edu");
dao.insertarContacto(c);                           // tampoc
```

I que **tot** el `jdbc:sqlite:`, el `PreparedStatement` i el `ResultSet` visqui dins d'una sola classe. Si demà canvies de SQLite a MySQL, només toques aquella classe. La resta del programa no se n'assabenta de res.

> 💡 **Consell:** això és programar contra una interfície, que vas veure a la U10: declares el tipus `ContactoDAO` i crees la implementació concreta `ContactoDAOImpl`. Si la implementació canvia, el teu codi no es mou.

📜 La interfície: el contracte

La interfície declara **què** es pot fer, sense dir **com**:

```
public interface ContactoDAO {
    void insertarContacto(Contacto c);
    List<Contacto> obtenerTodos();
    List<Contacto> buscarPorNombre(String nombre);
    boolean actualizarContacto(Contacto c);
    boolean eliminarContacto(int id);
}
```

Cinc mètodes, cinc operacions. El que crida la interfície no sap (ni li importa) si darrere hi ha SQLite, MySQL o un fitxer de text. El contracte és l'única cosa que importa.

> 📝 **Nota:** els mètodes són exactament els que vam escriure al punt 4. El DAO els unifica davall d'una mateixa interfície: això és tot el secret.

🏭 La implementació: qui posa les mans a la pasta

La implementació concreta conté la URL, l'SQL i el maneig de `SQLException`. Aquí és on viu tot el dels punts anteriors:

```
public class ContactoDAOImpl implements ContactoDAO {
    private static final String URL = "jdbc:sqlite:contactos.db";

    @Override
    public void insertarContacto(Contacto c) {
        String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?,
?, ?)";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setString(1, c.getNombre());
            pstmt.setString(2, c.getTelefono());
            pstmt.setString(3, c.getEmail());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error en inserir: " + e.getMessage());
        }
    }

    @Override
    public List<Contacto> obtenerTodos() {
        List<Contacto> contactos = new ArrayList<>();
        String sql = "SELECT * FROM contactos ORDER BY nombre";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email"));
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        } catch (SQLException e) {
            System.err.println("Error en consultar: " + e.getMessage());
        }
        return contactos;
    }

    // buscarPorNombre, actualizarContacto i eliminarContacto seguixen el mateix
patró
}
```

Fixa't en el que **no** hi ha aquí fora: ni un `SELECT` en el `main`, ni una URL de base de dades perduda entre la lògica. Tot el treball brut de l'SQL està tancat a la seva cel·la.

> 💡 **Consell:** és habitual tenir un DAO per entitat: `AlumnoDAO`, `ContactoDAO`, `LibroDAO`… Una taula, un POJO i un DAO formen el trio clàssic de les aplicacions amb base de dades.

🤔 DAO vs Repository

Dos termes que s'assemblen tant que de vegades s'usen com a sinònims. La diferència fina:

- **DAO** està pegat a la base de dades: parla de `INSERT`, `SELECT`, `UPDATE` i `DELETE`.
- **Repository** està més prop del domini: parla de `guardar`, `buscar`, `esborrar`, sense mencionar SQL.

En projectes petits s'usen indistintament. En els grans, el Repository pot usar diversos DAOs per davall. Si algú t'ho pregunta en una entrevista, amb aquesta resposta sobrevius.

⭐ Be the Code, my friend…

> 🕶️ **Don Tip:** un DAO encapsula l'accés a dades. Si canvies de SQLite a MySQL, només canvia el DAO; la lògica de negoci ni se n'assabenta.

Exercici: la biblioteca

Ets l'arquitecta o l'arquitecte d'una aplicació de gestió de biblioteca. Tens aquestes entitats:

```
class Libro {
    String isbn;
    String titulo;
    String autor;
    boolean disponible;
}

class Prestamo {
    int id;
    String isbn;
    String socio;
    LocalDate fechaPrestamo;
    LocalDate fechaDevolucion;
}
```

Preguntes per a pensar (respon abans de mirar):

1. Quants DAOs crearíes: un per entitat (`LibroDAO`, `PrestamoDAO`) o un de sol genèric?
2. Quins mètodes tindria cada DAO?
3. On posaries la regla de "un llibre no es pot prestar si ja està prestat"?
4. Usaries transaccions per a gestionar préstecs i devolucions?

> 🔄 **Solució orientativa**
>
> 1. **Un per entitat**: `LibroDAO` i `PrestamoDAO`. Cadascun amb el seu POJO i la seva taula. Un DAO genèric amb `<T>` existeix, però per a dues entitats afegeix abstracció sense benefici.
> 2. `LibroDAO`: `listar`, `buscarPerIsbn`, `insertar`, `actualitzar`, `eliminar`. `PrestamoDAO`: `listar`, `buscarPerSoci`, `insertar` (registrar préstec), `actualitzar` (devolució).
> 3. A la **lògica de negoci**, no al DAO. El DAO pregunta "està disponible?", i el servei de préstecs decideix si es presta. El DAO no pren decisions de negoci.
> 4. **Sí.** Prestar un llibre implica marcar `disponible = false` i crear el préstec: dues operacions que han d'anar juntes (transaccions, punt 7) o no anar-ne cap.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què conté la interfície del DAO: el “què” o el “com”?
2. Si canvies de SQLite a MySQL, quina classe toques i quina no?
3. On viu el `jdbc:sqlite:...` en un programa ben fet?
4. DAO i Repository són exactament el mateix?

> 🔄 **Respostes**
>
> 1. El **què**: les operacions disponibles. El com (SQL) viu a la implementació.
> 2. Toques la **implementació** (`...DAOImpl`); la interfície i la resta del programa no es mouen.
> 3. Dins de la **implementació del DAO**, com una constant privada.
> 4. No. DAO està pegat a la base de dades (INSERT/SELECT); Repository és més de domini (guardar/buscar). En projectes petits s’usen com a sinònims.

✅ Resum en 3 frases

1. El **patró DAO** separa l’accés a dades de la lògica de negoci: una **interfície** amb les operacions i una **implementació** amb l’SQL.
2. La resta del programa parla amb mètodes (`obtenerTodos()`, `insertarContacto(...)`) i no veu mai un `SELECT` ni una URL de connexió.
3. Si canvies de base de dades, només canvia la **implementació**: aquella és la màgia de programar contra la interfície.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
DAO	Data Access Object: el patró que tanca l’SQL
Interfície	El contracte: què es pot fer
Implementació	La classe concreta: com es fa (SQL, URL…)
Lògica de negoci	Les regles del problema, sense SQL
Repository	Cosí del DAO, més prop del domini

📚 Tornar a l’índex de la unitat · **Anterior:** 05 · PreparedStatement i SQL injection · **Següent:** 07 · Transaccions

07 · Transaccions: commit, rollback i savepoints

Operacions de tot o res: setAutoCommit(false), commit per a confirmar, rollback per a desfer i savepoints per a no desfer-ho tot 🗄️🎭

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 07 · Transaccions

---

📬 La idea en una frase

> **Una transacció agrupa diverses operacions en un "tot o res": `setAutoCommit(false)` obre el parèntesi, `commit()` el tanca confirmant, i `rollback()` el tanca desfent-ho tot.**

Fins ara cada `executeUpdate()` es guardava al moment. Però què passa quan una operació depèn d'una altra? Pensa en una transferència bancària: treure 50 € d'un compte i posar-los en un altre. Si la segona operació falla, et quedes amb 50 € volant al limbe. Això no pot passar. Aquí entren les transaccions.

🎭 El problema: el limbe dels diners

Imagina aquestes dues operacions:

```
// Operació 1: traure 50 € a Ana
stmt.executeUpdate("UPDATE cuentas SET saldo = saldo - 50 WHERE titular = 'Ana'");

// Operació 2: posar 50 € a Luis
stmt.executeUpdate("UPDATE cuentas SET saldo = saldo + 50 WHERE titular = 'Luis'");
```

Sense transaccions, cada `UPDATE` es confirma sol. Si l'operació 2 llança una `SQLException`, la 1 ja s'ha executat: Ana és més pobra i Luis no és més ric. El sistema queda **inconsistent**. Les dades ja no quadren, i ningú sap com arreglar-ho.

> ⚠️ **Advertència:** sense transaccions, si falla la segona operació, la primera ja es va executar. El sistema queda inconsistent. Usa transaccions per a operacions **atòmiques** (tot o res): com un pont que o està sencer o no està.

🔄 setAutoCommit(false): obrir el parèntesi

Per defecte, JDBC usa **auto-commit**: cada sentència es confirma al moment. Per a agrupar operacions, el desactivem i decidim nosaltres quan es confirma:

```
public void transferir(int idOrigen, int idDestino) {
    String quitar = "UPDATE cuentas SET saldo = saldo - 50 WHERE id = ?";
    String poner  = "UPDATE cuentas SET saldo = saldo + 50 WHERE id = ?";

    try (Connection con = DriverManager.getConnection(URL)) {
        con.setAutoCommit(false);   // obrim la transacció

        try (PreparedStatement q = con.prepareStatement(quitar);
             PreparedStatement p = con.prepareStatement(poner)) {

            q.setInt(1, idOrigen);
            q.executeUpdate();

            if (idOrigen == idDestino) {
                throw new SQLException("Destí invàlid: mateix compte");
            }

            p.setInt(1, idDestino);
            p.executeUpdate();

            con.commit();           // tot bé: confirmem
            System.out.println("Transferència OK");

        } catch (SQLException e) {
            con.rollback();         // alguna cosa va fallar: desfem tot
            System.err.println("Va fallar, tot desfet: " + e.getMessage());
        }
    } catch (SQLException e) {
        System.err.println("Error de connexió: " + e.getMessage());
    }
}
```

El flux és clar: **obrir** amb `setAutoCommit(false)`, fer totes les operacions, i al final **`commit()`** si tot va bé o **`rollback()`** si alguna cosa va fallar. El `rollback()` està al `catch` intern: qualsevol `SQLException` de les dues operacions el dispara i la base de dades torna a com estava.

> 💡 **Consell:** el `setAutoCommit(false)` va abans de les operacions, sobre la connexió. El `commit()` i el `rollback()` també són mètodes de la `Connection`. Tot gira al voltant d'aquell objecte.

📍 Savepoints: no desfer-ho tot

De vegades una transacció té diverses fases i no vols desfer tot si falla l'última. Un **savepoint** és una marca intermèdia: pots fer `rollback` fins a aquell punt i conservar l'anterior.

```
con.setAutoCommit(false);

Savepoint sp = con.setSavepoint("despresDInserir");
// ... més operacions ...

if (algoMal) {
    con.rollback(sp);   // torna al savepoint, no desfà tot
} else {
    con.commit();       // tot bé
}
```

És com en un videojoc: `setSavepoint` guarda la partida, `rollback(sp)` et torna a aquella partida guardada en comptes de començar de zero, i `commit()` guarda el final definitiu. Útil quan una transacció llarga té una part que saps que sempre surt bé i una altra que pot fallar.

> 📝 **Nota:** els savepoints s'usen menys del que sembla. El 90% de les vegades et basta amb `commit()`/`rollback()` en bloc. Però saber que existeixen t'estalviarà un ensurt el dia que els necessites.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode de `Connection` obre una transacció?
2. Què fa `commit()`? I `rollback()`?
3. On se sol posar el `rollback()` en un mètode típic?
4. Què et permet fer un `Savepoint`?

> 🔄 **Respostes**
>
> 1. `con.setAutoCommit(false)`. Amb `true` (el normal) cada sentència es confirma sola.
> 2. `commit()` confirma totes les operacions; `rollback()` les desfà totes.
> 3. Al `catch (SQLException e)` del bloc d'operacions: si alguna cosa falla, es desfà tot.
> 4. Fer `rollback` fins a aquell punt concret en comptes de desfer tota la transacció.

✅ Resum en 3 frases

1. Una **transacció** agrupa operacions en un tot-o-res: `setAutoCommit(false)` l'obre, `commit()` la confirma i `rollback()` la desfà.
2. Sense transaccions, una operació que falla a mitges deixa les dades **inconsistents** (com els diners que desapareixen del limbe).
3. Els **savepoints** marquen punts intermedis per a poder desfer només una part de la transacció.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
Transacció	Un grup d'operacions que van juntes o no van
`setAutoCommit(false)`	Obre la transacció: tu controles el moment
`commit()`	Confirmar-ho tot: es guarda
`rollback()`	Desfer-ho tot: com si no hagués passat
`Savepoint`	Marca intermèdia per a desfer només fins aquí
Atòmic	Tot o res, sense estats a mitges

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · El patró DAO · **Següent:** 08 · Bones pràctiques

08 · Bones pràctiques: el decàleg del JDBC

Deu manaments per a no morir en l'intent: PreparedStatement sempre, try-with-resources, WHERE sempre i no tornar ResultSet 🗄️📜

> 🗺️ **Estàs en:** 🗄️ **U15 · Persistència de dades: JDBC i introducció a ORM** → 08 · Bones pràctiques

---

📬 La idea en una frase

> **Tot el que has après en aquesta unitat, resumit en 10 manaments: de "PreparedStatement sempre" a "confirma abans d'esborrar". Memoritza'ls i seràs l'ànima de l'oficina.**

Ja saps connectar, consultar, fer el CRUD, evitar la injecció, aplicar el DAO i gestionar transaccions. Aquest punt és el tancament de la teoria: la llista de coses que els programadors veterans van aprendre a base d'ensurts, i que tu pots aprendre en cinc minuts. Són els hàbits que separen qui escriu JDBC de qui el domestica.

📜 El decàleg del JDBC

1. **PreparedStatement sempre.** Mai concatenes SQL. L'SQL injection no és un mite: és Bobby Tables trucant a la teva porta.
2. **Try-with-resources.** `Connection`, `Statement` i `ResultSet` es tanquen sols. Connexió oberta que no es tanca = servidor que s'ennuega.
3. **No tornis `ResultSet`.** Torna llistes d'objectes (`List<Contacto>`). El `ResultSet` està lligat a la seva connexió: quan la tanques, mor amb ella.
4. **Captura `SQLException` amb missatge descriptiu.** No atrapis `Exception` a la babalà i no et quedes en silenci. `e.getMessage()` és el teu amic.
5. **Usa transaccions per a operacions múltiples.** Tot o res. Una transferència no es fa a mitges, i les teves dades tampoc.
6. **No exposes credencials en el codi.** La URL i les dades d'accés van en un fitxer de configuració (`config.properties`) o en variables d'entorn, no en un `static final String` de la classe. És com portar la contrasenya del banc escrita al front.
7. **Comprova les files afectades.** `executeUpdate()` et diu si l'operació va funcionar (`> 0`). Ignorar aquell valor és apostar a cegues.

8. **No facis consultes dins de bucles.** Una consulta en un bucle que recorre mil elements és mil viatges a la base de dades. Una sola consulta amb `JOIN` basta. El bucle és per a processar resultats, no per a fer consultes.
9. **`WHERE` sempre en `UPDATE` i `DELETE`.** O pagues les conseqüències. Sense `WHERE`, actualitzes o esborres la taula sencera i no hi ha volta enrere.
10. **Confirma abans d'esborrar.** L'usuari sempre s'equivoca. Un `DELETE` no té Ctrl+Z.

> 💡 **Consell:** els manaments 1, 2 i 9 són els que més vides han salvat. Si memoritzes només tres, que siguin aquells. Els altres set vénen de regal.

🔮 L'enigma

Tens dues taules en SQLite: `usuarios(id, nombre, email)` i `pedidos(id, usuario_id, total)`.

Vols obtenir tots els usuaris que han fet algun pedido amb total superior a **100 €**. Quantes consultes SQL necessites com a mínim si les fas "a la babalà"? I si uses `JOIN`?

> › 🔄 **Solució**
>
> **A la babalà (patró N+1):** primer una consulta per a llistar tots els usuaris (1 consulta) i després, per a cada usuari, una altra consulta que pregunti pels seus pedidos (N consultes, una per usuari). En total, **N + 1** consultes: si tens 1.000 usuaris, són 1.001 viatges a la base de dades. Lentíssim.
>
> **Amb `JOIN`:** una sola consulta. La base de dades relaciona les dues taules internament i et torna només el que necessites.
>
> ```
> SELECT DISTINCT u.nombre, u.email
> FROM usuarios u
> JOIN pedidos p ON p.usuario_id = u.id
> WHERE p.total > 100;
> ```
>
> **Una consulta, un viatge.** La lliçó del manament 8: el bucle processa resultats, però la base de dades és qui ha de fer el treball brut. Pregunta't sempre: estic fent N consultes on cabria una?

> 🕶️ **Don Tip:** una consulta amb `JOIN` és UNA sola crida a la base de dades. Fer una consulta per cada usuari en un bucle és el patró N+1, l'error de rendiment clàssic que cau en totes les

> entrevistes de treball.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quins tres manaments són els més importants segons aquest punt?
2. Per què no convé tornar un `ResultSet` des d'un mètode?
3. On han de viure les credencials de la base de dades?
4. Què és el patró N+1?

> 🔄 **Respostes**
>
> 1. **PreparedStatement sempre**, **try-with-resources** i **WHERE sempre** en UPDATE/DELETE.
> 2. Perquè el `ResultSet` està lligat a la seva connexió: en tancar-la, deixa de ser vàlid. Millor tornar `List<Contacto>`.
> 3. En un fitxer de configuració (`config.properties`) o variables d'entorn. Mai en el codi.
> 4. Fer 1 consulta per a llistar i N consultes dins d'un bucle: N+1 viatges a la base de dades on cabria una sola amb `JOIN`.

✅ Resum en 3 frases

1. El decàleg del JDBC són **10 hàbits** que t'estalvien desastres: `PreparedStatement` sempre, `try-with-resources`, `WHERE` sempre i comprovar files afectades, entre altres.
2. Dos errors de rendiment clàssics: **consultes en bucles** (patró N+1, s'arregla amb `JOIN`) i **tornar `ResultSet`** (s'arregla tornant llistes d'objectes).
3. La seguretat també és una bona pràctica: les **credencials mai van en el codi**, i els `DELETE` es **confirmen** abans d'executar-se.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Decàleg	Les 10 regles d'or del JDBC
Patró N+1	N consultes en un bucle on cabria una

Terme	Idea general
`JOIN`	Relacionar dues taules en una sola consulta
Files afectades	El `int` de `executeUpdate()` que confirma el canvi
`config.properties`	Fitxer on van la URL i les credencials

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Transaccions · **Següent:** 09 · Introducció a ORM

09 · Introducció a ORM

El traductor automàtic entre les teves classes Java i les taules de la base de dades: què és un ORM, com mapeja i per què no t'ensarra d'aprendre SQL 🗄️🤖

> 🗺️ **Estàs en: 🗄️ U15 · Persistència de dades: JDBC i introducció a ORM** → 09 · Introducció a ORM

📬 La idea en una frase

> **Un ORM (Object-Relational Mapping) és una capa de codi que converteix els teus objectes Java en files de taula i les teves consultes SQL en crides a mètodes, perquè tu et centres en el negoci i no en el catàleg.**

Has escrit JDBC a mà: `Connection`, `PreparedStatement`, `setXxx()`, `executeUpdate()`, tancar tot en l'ordre correcte… Funciona, és ràpid i aprens moltíssim. Però és **mol** codi repetitiu per al que fa: moure dades d'un lloc a un altre. Els ORM nàixeren exactament per a estalviar-te aquell treball mecànic.

🔎 Què és un ORM, concretament?

La base de dades pensa en **taules, files i columnes**. El teu programa pensa en **objectes, atributs i referències**. Parlen dos idiomes diferents i algú ha de traduir. Aquell traductor és el ORM:

Concepte de la base de dades	Equivalent en Java
Taula	Classe (`@Entity`)
Fila	Objecte (instància)
Columna	Atribut
Clau primària (`id`)	Camp amb `@Id`
Clau forània	Referència a un altre objecte
`INSERT / UPDATE / DELETE`	`persist() / merge() / remove()`
`SELECT`	Mètode de busca (`find`, JPQL, criteris)

En comptes de muntar el SQL i llegir el `ResultSet`, el ORM **fa el viatge d'anada i tornada per tu**: en guardar un objecte crea la fila, i en llegir una fila construeix l'objecte.

⚖️ JDBC vs ORM: la mateixa tasca, dos estils

Imagina guardar un contacte nou.

A mà (JDBC):

```
String sql = "INSERT INTO contactos (nombre, email) VALUES (?, ?)";
try (Connection c = DriverManager.getConnection(url);
     PreparedStatement ps = c.prepareStatement(sql)) {
    ps.setString(1, "Ana");
    ps.setString(2, "ana@ejemplo.com");
    ps.executeUpdate();
}
```

Amb un ORM (estil JPA):

```
Contacto ana = new Contacto("Ana", "ana@ejemplo.com");
em.persist(ana);   // una línia: el ORM genera l'INSERT per tu
```

El ORM construeix l'INSERT, passa els paràmetres, gestiona la connexió i, quans llegeixes, ompli el teu objecte camp a camp. Menys boilerplate, més lògica de negoci.

🧰 L'arsenal: JPA, Hibernate i companyia

No tots els ORM són iguals, i convé conèixer els noms que sentiràs en qualsevol entrevista:

- **JPA** (Java Persistence API): l'**especificació** de Java per a persistència. Són anotacions (`@Entity`, `@Table`, `@Column`) i una API estàndard, no un programa en si.
- **Hibernate**: la **implementació** de JPA més famosa. Si JPA és l'estàndard de tràfic, Hibernate és el cotxe que més circula.
- **Sessió i `EntityManager`:** l'objecte que gestiona les teves entitats. Les teves crides a `persist()` o `find()` passen per ell; ell decideix quan tocar la base de dades.
- **Càrrega perezosa (*lazy*) vs anticipada (*eager*):** el ORM pot treure els objectes relacionats només quan els necessites (lazy) o tot d'una (eager). Triar mal aquí és l'origen de no pocs

forats de rendiment.

> 💡 **Del curs a la indústria:** nosaltres hem usat SQLite + JDBC perquè és la forma més honesta d'entendre què passa sota el capó. En un projecte JPA el capó continua sent el mateix: connexions, SQL i transaccions… sol que el ORM ho condueix per tu.

🐘 La cara oculta: allò que el ORM no t'arregla

Un ORM no és màgia, i saber els seus límits és el que et separa de qui només va copiar un tutorial:

1. **El SQL continua sent teu.** El ORM genera consultes senzilles, però una consulta amb tres `JOIN` i un agregat la escrius tu (o la genera el ORM, però has de llegir-la i entendre-la).
2. **N+1 també, però amagat.** Recórrer una llista i que cada objecte carrega els seus fills a escondides = N+1 una altra vegada. El ORM l'amplifica si no el vigiles.
3. **Capa de rendiment.** De vegades el ORM fa *dos* viatges on tu n'hauries fet un. Medir continua sent obligatori.
4. **Corba d'aprenentatge.** Configurar proveïdors, dialectes, mapatges… té el seu cost inicial.
5. **El desajust objecte-relacional.** Classes amb herència i taules planes no encaixen sempre net: per això el tema es diu *mapeig* i no *traducció perfecta*.

> ⚠️ **[WARNING]** Un ORM no substituïx a SQL, l'**amaga**. Si no entens allò que hi ha davall (INSERT, transaccions, índexs), quan algo falla no sabrás ni per on començar a mirar. Per això ho veiem al final de la unitat: primer el motor, després el GPS.

🧭 I per què ho veem ara?

Perquè l'ordre importa: quan algú digui «aquesta app usa Hibernate i no escrivim SQL», tu ja sabràs respondre **què està fent exactament davall** (el teu `persist()` d'una línia és l'`PreparedStatement` del capítol 5, amb vestit i corbata). I quan el ORM et falla a les 3 de la matinada, tu seràs qui obre el log i vagi el SQL real.

Aquella és la promesa d'aquest punt: **no memoritzar un framework, sinó entendre la idea** que hi ha darrere de tots ells.

🎯 Mini-comprovació

Posa't a prova en 30 segons (les respostes estan amagades):

1. Què tradueix un ORM: d'objectes a taules o de taules a objectes?
2. En JPA, què és `@Entity` i què és Hibernate?
3. Quin és el risc clàssic de rendiment que un ORM pot amagar?
4. Per què no substituïx a SQL?

> 🔄 **Respostes**
>
> 1. **En les dues direccions**: en guardar tradueix objectes → files; en llegir, files → objectes.
> 2. `@Entity` és una anotació de l'especificació **JPA** (marca la classe com a entitat persistent); **Hibernate** és la implementació que fa el treball real.
> 3. El patró **N+1**: cada objecte que carregues dispara consultes addicionals sense que ho vegis a primera vista.
> 4. Perquè genera consultes senzilles i amaga la mecànica; per a consultes complexes, rendiment o depuració, continues necessitant entendre SQL.

✅ Resum en 3 frases

1. Un **ORM** mapeja classes → taules i objectes → files, i tradueix crides a mètodes (`persist`, `find`) en SQL per a estalviar-te el JDBC mecànic.
2. **JPA** és l'especificació de Java (anotacions + API) i **Hibernate** és la seva implementació més usada; treballen amb un `EntityManager` i entitats anotades.
3. No substituïx a SQL: **amaga** el treball repetitiu, però les consultes complexes, el rendiment (N+1) i la depuració continuen sent cosa teva.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
ORM	Capa que tradueix objectes ↔ taules
JPA	Especificació de persistència de Java (anotacions + API)
Hibernate	Implementació de JPA més estesa

Terme	Idea general
Entitat	Classe mapada a una taula (`@Entity`)
`EntityManager`	El “mandatari” que executa la persistència
Lazy / eager	Carregar dades tard (quan toca) o tot d’una

📚 Tornar a l’índex de la unitat · **Anterior:** 08 · Bones pràctiques · **Següent:** 10 · Repàs interactiu

10 · Repàs interactiu: domina el JDBC

El tancament pràctic de la unitat, amb una injecció SQL que farà mal, un UPDATE sense WHERE que cremarà i una connexió que ningú no va tancar 😈🗄️

> 🗺️ **Estàs en: 🗄️ U15 · Persistència de dades: JDBC i introducció a ORM** → 10 · Repàs interactiu

---

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
import java.sql.*;

public class Misterio {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:instituto.db";

        try (Connection con = DriverManager.getConnection(url);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM alumnos")) {

            if (rs.next()) {
                System.out.println("Total: " + rs.getInt(1));
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`Total: 0`** → `COUNT(*)` sempre torna 0 si hi ha algun alumne. ❌
2. **`Total: <nombre d'alumnes>`** → `COUNT(*)` torna una sola fila amb una columna, i `rs.getInt(1)` la llegeix per índex. ✅
3. **Una excepció** → `getInt(1)` no existeix perquè les columnes comencen en 0. ❌

> 🔄 **Solució**
>
> L'opció **2**. `SELECT COUNT(*)` torna **una sola fila amb una columna** sense nom fix, així que s'accedeix per índex `rs.getInt(1)` (les columnes del `ResultSet` comencen en 1, no en 0). Si la taula té 25 alumnes, imprimeix `Total: 25`. Un `ResultSet` amb una fila segueix necessitant el seu `rs.next()`: el cursor sempre comença abans de la primera fila.

🔥 Fireside Chat: Statement vs PreparedStatement 🔗

> *Dues formes d'executar SQL es troben davant la màquina de cafè.*

Statement: — Jo soc el de tota la vida. `stmt.executeQuery("SELECT * FROM usuarios")`. Directe, sense embuts. Per a una consulta fixa soc perfecte.

PreparedStatement: — Sí, però quan arriba una dada de l'usuari i concatenes, se t'encén el pilot de la injecció. Jo separo l'SQL de les dades amb `?` i els `setXxx`. Seguretat de sèrie.

Statement: — I si només executi una consulta una vegada? Per a què he de preparar res?

PreparedStatement: — A més jo faig cache del pla d'execució. Si executes la mateixa consulta amb diferents paràmetres, soc més ràpid. I en Java quasi sempre fas això: la mateixa consulta, mil valors diferents.

Statement: — Val... però jo serveixo per al DDL: `CREATE TABLE`, `ALTER`...

PreparedStatement: — Cert. Per a DDL, Statement. Per a DML (SELECT, INSERT, UPDATE, DELETE), jo. Tracte?

Statement: — Tracte.

> La lliçó: **DDL → Statement. DML → PreparedStatement.** I mai concatenes dades d'usuari en un SQL, digues el que digues.

🕵️ Qui soc?

Endevina quin concepte de la unitat soc:

1. **Soc el traductor universal entre Java i qualsevol base de dades amb controlador.**
2. **Soc la canonada que `DriverManager.getConnection()` et torna i que tanques amb `try-with-resources`.**
3. **Soc la taula virtual que es recorre amb `next()` i es llegeix amb `getXxx("columna")`.**
4. **Soc el missatger segur que separa l'SQL de les dades amb `?`.**
5. **Soc el patró que encapsula tot l'SQL darrere d'una interfície.**
6. **Soc l'operació que fa que diverses sentències es confirmen juntes… o cap.**

> 🔄 **Respostes**
>
> 1. **JDBC** — `java.sql`, el pont cap a SQLite, MySQL, PostgreSQL…
> 2. **`Connection`** — s'obté amb `DriverManager.getConnection()` i es tanca sempre.
> 3. **`ResultSet`** — `while (rs.next())` + `rs.getInt("id")`, `rs.getString("nombre")`…
> 4. **`PreparedStatement`** — placeholders `?` i `setString(1, ...)`, `setInt(2, ...)`…
> 5. **DAO** — `ContactoDAO` (interfície) + `ContactoDAOImpl` (implementació).
> 6. **Transacció** — `setAutoCommit(false)`, `commit()` o `rollback()`.

😡 CONRAD VS EL MÓN: "El JDBC que no es tanca"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre els clàssics del novell.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu programa funciona però a la segona volta es queixa de "Too many connections"*. AI, MARE MEVA! És que està obrint `Connection` a la babalà i **no la tanca mai**. Cada `getConnection()` sense el seu `try-with-resources` és una canonada degotant. El servidor té un límit de connexions, i tu les estàs esgotant totes. `try-with-resources`! Tant costa?

I després està el de l'`UPDATE` sense `WHERE`. M'ensenya un codi que actualitza "un" contacte i resulta que **els ha posat tots** amb el mateix telèfon. I el `WHERE id = ?`? Se li va oblidar! En `UPDATE` i `DELETE`, si no hi ha `WHERE`, és que volies arrasar la taula sencera.

I el rei del drama: usa `executeQuery()` per a un `INSERT` i s'emporta una `SQLException`. I tant! `executeQuery()` és per a SELECT; per a INSERT/UPDATE/DELETE està `executeUpdate()`. Cada cosa al seu lloc. Com la forquilla i el microones, que t'ho vaig dir al punt 3.

I l'últim, el del silenci: captura l'excepció i… **no fa res**. `catch (Exception e) {}` buit. La llum del "check engine" encesa i ell tan feliç. Escriu `e.getMessage()`! És gratis i et diu exactament què ha passat.

La lliçó: abans d'acusar la base de dades d'"odiar-te", repassa el trio sagrat de la unitat: **he tancat la connexió? hi ha `WHERE` en el meu UPDATE/DELETE? estic usant el mètode d'execució correcte?** El 90% dels errors d'aquesta unitat s'arreglen amb aquestes tres preguntes. La base de dades no t'odia: t'està passant les respostes de l'examen.

🎮 El joc de les decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Vols buscar alumnes per un nom que tecleja l'usuari. Què uses?
   - a) `Statement` amb concatenació b) `PreparedStatement` amb `?`
2. `executeUpdate()` en un `UPDATE` que no troba files… què torna?
   - a) `0` b) una `SQLException`
3. Quin mètode obre una transacció?
   - a) `con.setAutoCommit(false)` b) `con.beginTransaction()`
4. `rs.next()` en un `ResultSet` buit torna…
   - a) `false` b) `null`
5. Per al DDL (`CREATE TABLE`), què s'usa amb més sentit?
   - a) `Statement` b) `PreparedStatement`

> 🔄 **Solucions**
>
> 1. **b)** — Mai concatenes dades d'usuari en SQL. `PreparedStatement` amb placeholders.
> 2. **a)** — Torna `0`: no va trobar files, però no és un error. Per això es comprova `> 0`.
> 3. **a)** — `setAutoCommit(false)` desactiva l'auto-commit; la resta és `commit()/rollback()`.
> 4. **a)** — `next()` torna `false` quan no queden files, no `null` ni excepció.
> 5. **a)** — Per a DDL un `Statement` simple val; `PreparedStatement` és per a DML amb dades.

⚡ Laboratori de tortura: el CRUD que sagna

> **Duració estimada:** 30 minuts **Eina:** el teu IDE, el controlador de SQLite i un fitxer nou

L'escenari: et passen aquest gestor de contactes amb SQLite. Et diuen que té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el resultat sigui incorrecte… però, i si t'ho diuen malament? La teva tasca: fer que compili, que executi i que **tota** la sortida sigui correcta, comptant tu els errors reals.

```
import java.sql.*;

public class GestorContactos {
    private static final String URL = "jdbc:sqlite:contactos.db";

    public static void main(String[] args) {
        crearTabla();
        insertar("Ana", "666111222");
        insertar("Luis", "666333444");
        listar();
    }

    static void crearTabla() {
         String sql = "CREATE TABLE IF NOT EXISTS contactos (id INTEGER PRIMARY
KEY AUTOINCREMENT, nombre TEXT, telefono TEXT)";
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement()) {
            stmt.executeUpdate(sql);
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    static void insertar(String nombre, String telefono) {
          String sql = "INSERT INTO contactos (nombre, telefono) VALUES ('" +
nombre + "', '" + telefono + "')";
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement()) {
            stmt.executeUpdate(sql);
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    static void listar() {
        String sql = "SELECT * FROM contactos"
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                                System.out.println(rs.getInt("id")  +  "  -  "  +
rs.getString("nombre"));
```

```
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Fallada intencionada: el nombre d'errors de l'enunciat està malament a propòsit: si busques el "tercer error de compilació" i "l'error de lògica", te'n tornes boig. La trampa és la teva confiança en l'enunciat.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Compila? *no → busca punts i comes.*
   > **› I si continuec atascat?**
   > Fixa't en la línia `String sql = "SELECT * FROM contactos"` de `listar()`: li falta el `;` al final.

2. Compila ja? *no → busca més coses rares.*
   > **› I si continuec atascat?**
   > Mira els `Statement` de `insertar()`: està concatenant l'SQL amb dades del programa. Compila, però és una bomba de rellotgeria.

3. Creus que hi ha un error de lògica que canvia la sortida? *Torna a llegir amb calma i no et fies de l'enunciat.*
   > **› Solució final**

Errors de compilació:

1. Falta el `;` al final de `String sql = "SELECT * FROM contactos"` a `listar()`.

L'"error de lògica": no existeix com a tal. La lògica de crear taula, inserir i llistar és correcta i la sortida seria `1 - Ana` i `2 - Luis`. Aquella era la fallada intencionada: l'enunciat diu "3 errors de compilació i 1 de lògica", però només hi ha una falta de `;`. L'error "amagat" era la teva confiança en l'enunciat.

El problema real que no es veu en executar: `insertar()` concatena l'SQL amb els valors, cosa que és **SQL injection** en estat pur (punt 5). El programa "funciona" perquè les dades són de

confiança, però si `nombre` vingués de l'usuari, un `Ana'; DROP TABLE contactos; --` s'enduria la taula. La correcció professional és `PreparedStatement`:

```
static void insertar(String nombre, String telefono) {
    String sql = "INSERT INTO contactos (nombre, telefono) VALUES (?, ?)";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, nombre);
        pstmt.setString(2, telefono);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
```

Sortida correcta amb el codi arreglat:

```
1 - Ana
2 - Luis
```

La lliçó real: **llegeix el codi, no l'enunciat.** Un bug pot ser que et diguin "hi ha 3 errors" i només n'hi hagi 1: si busques els altres dos, et tornes boig. I l'error més perillós és el que no dona error: l'SQL injection es veu amb ulls d'arquitecte, no amb el compilador. Compta, verifica i confia en el que veus, no en la llegenda.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🥇 **El Conectador**	Obrir i tancar una `Connection` amb `try-with-resources` sense que escapi ni una
🥈 **El Missatger Segur**	Usar `PreparedStatement` per a una consulta amb dades d'usuari, sense concatenar res
🥉 **El Caçador de Taules**	Detectar SQL injection en codi alié i explicar per què és perillosa

Assoliment	Com aconseguir-ho
🏅 L'Arquitecte DAO	Encapsular el CRUD en una interfície + implementació i usar-lo des d'un `main` net
🏅 El Banquer	Muntar una transferència atòmica amb `commit()` i `rollback()` que mai deixa les dades a mitges

🧠 Atreveix-te a pensar

1. **Sense executar:** què imprimeix aquest programa si la taula `alumnos` té 3 files?

```
String sql = "SELECT nombre FROM alumnos WHERE edad > ?";
try (Connection con = DriverManager.getConnection(url);
     PreparedStatement pstmt = con.prepareStatement(sql)) {
    pstmt.setInt(1, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.print(rs.getString("nombre") + " ");
        }
    }
}
```

2. **El detectiu del ResultSet:** un mètode torna un `ResultSet` al `main` i després tanca la connexió. Què passa quan el `main` intenta recórrer-lo?
3. **El consultor de preus:** tens una taula `productos(precio)` i vols actualitzar un preu concret. Quines dues coses són imprescindibles en l'`UPDATE`?
4. **Vertader o fals:** "un `PreparedStatement` permet usar `?` tant per a valors com per al nom de la taula".

> 💡 Solucions
>
> 1. **Els noms** dels alumnes majors de 18, separats per espai. El `?` s'ompli amb `setInt(1, 18)` i el bucle imprimeix cada `nombre`.
> 2. **El `ResultSet` mor.** Està lligat a la seva connexió: en tancar-la, deixa de ser vàlid i llança una `SQLException` al recórrer-lo. Per això els mètodes tornen `List<Alumno>` i no `ResultSet`.

3. **`WHERE id = ?`** (o el filtre que identifiqui el producte) i **comprovar les files afectades** amb `executeUpdate() > 0`. Sense `WHERE`, actualitzes tots els preus.
4. **Fals.** Els `?` només valen per a **valors**, mai per a identificadors (taula, columna). Aquests es trien al codi.

🧩 Crucigrama de bits

```
Horizontal:
1. Mètode que avança a la següent fila d'un ResultSet (4 lletres)
3. Excepció checked que acompanya tot JDBC (13 lletres)
5. Mètode que confirma una transacció (6 lletres)
6. Patró que encapsula l'accés a dades (3 lletres)

Vertical:
2. El pont entre Java i la base de dades (4 lletres)
4. Objecte Java que representa una fila de la taula (4 lletres)
```

> 📝 **Solucions**
>
> **Horizontal:** 1. NEXT · 3. SQLEXCEPTION · 5. COMMIT · 6. DAO **Vertical:** 2. JDBC · 4. POJO

💬 Preguntes d'entrevista de treball

Preguntes reals que et farien per a programador Java junior.

1. **"Explica-m'ho, com si jo fos la teva àvia, què és JDBC."**
2. **"Quina és la diferència entre `Statement` i `PreparedStatement`?"**
3. **"Què és l'SQL injection i com l'evites?"**
4. **"Què passa si no tanquis una `Connection`?"**
5. **"Què és el patró DAO i per què s'usa?"**
6. **"Què són `commit` i `rollback` i quan els uses?"**

🤷 No hi ha preguntes tontes

❓ **És necessari `Class.forName()` per a carregar el controlador?**

No. Des de Java 6, els controladors JDBC 4.0 s'auto-descobreixen si són al classpath. Ho veuràs en tutorials antics, i no passa res si el poses: només és codi que ja no cal.

> ❓ **`Statement` s'usa encara?**

Està zombi. Si veus un `Statement` en codi real (que no sigui una consulta literal o DDL), és bandera vermella. Per a qualsevol consulta amb dades variables: `PreparedStatement`, sempre.

> ❓ **Puc compartir una `Connection` entre diversos fils?**

Tècnicament sí, pràcticament no. `Connection` no és *thread-safe*. En aplicacions amb fils s'usa un pool de connexions (HikariCP) i cada fil demana la seva.

> ❓ **DAO i Repository són el mateix?**

Quasi. El DAO està pegat a la base de dades (INSERT, SELECT). El Repository és més de domini (guardar, buscar). En projectes petits s'usen com a sinònims; en els grans conviuen.

🎬 Post-Crèdits

La programadora acaba el seu gestor de biblioteca: un `LibroDAO` amb el seu `PreparedStatement` en cada mètode, una transacció amb `commit` i `rollback` per als préstecs i un `config.properties` guardant la URL. Tot funciona. Fins que la directora li demana que els préstecs es consulten des del mòbil.

S'acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Has vist? Tens les dades en SQLite i només les llegeixes des de Java a la mateixa màquina. Per a consultar-les des d'un mòbil necessites que el teu programa **expose** aquella informació per la xarxa: una API. Algú li demana dades al teu servidor amb una petició HTTP, i el teu servidor respon amb JSON.

Programadora: — És a dir, que en comptes de connectar-me jo a la base de dades des de cada client, el client em parla a mi per la xarxa?

CONRAD: — *assenteix* Exacte. El teu programa Java es converteix en l'intermediari: rep peticions, consulta la base de dades amb el JDBC que acables de dominar, i torna les dades. Primer vas servir dades des de la base; ara les serviràs per la web.

PROXIMAMENT EN U16: Servir i consumir APIs amb Web. El moment en què el teu programa Java es posa un uniforme de cambrer: rep peticions HTTP i serveix dades en JSON. 🌐

📚 Tornar a l'índex de la unitat · **Anterior:** 09 · Introducció a ORM · **Següent:** U16 · Servir i consumir APIs amb Web

U16 — Servir i consumir APIs amb Web

Del terminal a Internet: HttpServer, HTML, formularis, JSON i HttpClient per a servir i consumir APIs com una de veritat 🌐

Fins ara els teus programes vivien en la terminal, en fitxers i en bases de dades. Avui s'ha acabat: és hora que surtin a Internet. Sense JavaScript frameworks, sense servidors d'aplicacions i sense instal·lacions eternes: només Java i HTTP.

Aquesta unitat té tres grans actes:

- **Ser servidor:** amb `HttpServer`, el servidor HTTP que Java porta de sèrie, serviràs text, HTML, formularis i JSON sense instal·lar Tomcat ni Spring.
- **Ser API:** muntaràs un mini projecte REST complet —un gestor de tasques— amb GET, POST, PUT i DELETE.
- **Ser client:** amb `java.net.http.HttpClient`, consumiràs APIs externes reals (GitHub, jsonplaceholder), parsejaràs JSON amb Gson i gestionaràs capçaleres, timeouts i codis d'estat.

Pel camí entendràs el model petició-resposta d'HTTP, per què `"POST".equals(...)` és millor que `...equals("POST")`, què significa de veritat un 404 i per què el `fetch` de JavaScript i l'`HttpClient` de Java són dues cares de la mateixa moneda.

Aquesta unitat es llegeix com un **llibre de 9 capítols**: els 8 primers punts són teoria en progressió i el 9 és un aterratge pràctic per a deixar-ho tot ben lligat.

> 🧪 **Material d'ampliació (extra, no avaluables).** Aquesta unitat va més enllà del temari oficial: úsala per a ampliar, no per a suar la gota gorda. Els butlletins són recomanats, no obligatoris.

🎯 Objectiu de la unitat 🔗

En acabar, seràs capaç de:

- Explicar el model **petició-resposta** d'HTTP i els seus codis d'estat més comuns (200, 404, 500…).
- Muntar un **servidor web mínim** amb `com.sun.net.httpserver.HttpServer` en menys de 20 línies.
- Servir **HTML** des de Java i llegir-lo també des d'un fitxer.

- Processar **paràmetres GET** i **formularis POST** sense por.
- Tornar **JSON** amb el `Content-Type` adequat i consumir-lo amb `fetch`.
- Construir una **API REST** completa (GET, POST, PUT, DELETE) amb dades en memòria.
- Consumir APIs externes amb **`HttpClient`**: GET, POST, capçaleres, timeouts i crides asíncrones.
- **Parsejar JSON** amb Gson, mapejar-lo a `record` i gestionar errors HTTP amb elegància.

🗺️ Mapa de la unitat

Punt	Què aprendràs	Dificultat
01 · El protocol HTTP en 30 segons	Petició-resposta, mètodes, codis d'estat i URLs	Tots
02 · Servidor web mínim amb HttpServer	El teu primer servidor Java: `HttpServer`, `createContext` i `HttpExchange`	Tots
03 · Servint HTML	Pàgines completes amb `Content-Type: text/html` i des de fitxer	Tots
04 · Paràmetres GET i formularis POST	Llegir la query string i el cos de la petició	Tots
05 · Tornant JSON: com una API de veritat	`application/json`, `fetch` des del frontend i per què front i back se separen	Tots
06 · Mini projecte: gestor de tasques (API REST)	CRUD complet en memòria: GET, POST, PUT i DELETE	Tots
07 · Consumir APIs externes amb HttpClient	`HttpClient`, GET, `BodyHandlers` i parsejar JSON amb Gson	Tots
08 · HttpClient avançat	POST, capçaleres, timeouts, `sendAsync` i PUT/DELETE	Tots
09 · Repàs interactiu	Be the Code, Fireside, Laboratori, Crucigrama i més	Tots

> 📖 **Flux de lectura:** els 8 primers punts són teoria en progressió. El 9 és l'aterratge pràctic: llegeix-lo just després del 8 i abans d'obrir els butlletins.

📝 Butlletins de la unitat

> Practica amb els parells del curs: intenta primer el per-resoldre i comprova amb el resolt quan hagis acabat.

🟢 Inicial per resoldre ✅ Inicial resolt ⭐ Avançat per resoldre 💪 Avançat resolt 🔥 Extres

---

✅ Criteris d'avaluació coberts (RA5)

RA5: Realitza operacions d'entrada i sortida d'informació, utilitzant procediments específics del llenguatge i llibreries de classes.

CE	Criteri	On es cobreix
RA5 c)	S'han reconegut les possibilitats d'entrada / sortida del llenguatge i les llibreries associades.	✅ Punts 1, 2, 7 i 8
RA5 f)	S'han utilitzat les eines de l'entorn de desenvolupament per a crear interfícies gràfiques d'usuari simples.	✅ Punts 3 i 6
RA5 g)	S'han programat controladors d'esdeveniments.	✅ Punt 6
RA5 h)	S'han escrit programes que utilitzen interfícies gràfiques per a l'entrada i sortida d'informació.	✅ Punts 5 i 6

> 📌 La **RA5** es completa amb la **U03** (consola i formats, CEs a i b) i la **U14** (fitxers i accés a arxius, CEs c, d i e). Aquesta unitat cobreix la cara web de l'entrada/sortida: HTTP com a protocol, HTML i JSON com a formats, i les llibreries `HttpServer` i `HttpClient` com a procediments del llenguatge.

🚪 Per on començo?

- Vens de la U15 (JDBC) i vols l'essencial? → Comença en el punt 1 i el punt 2: HTTP i el teu primer servidor.
- Ja saps servir i vols fer APIs? → Ves directe al punt 5 i al punt 6: JSON i el CRUD de tasques.
- Només vens a pel costat client? → Salta al punt 7 i al punt 8: consumir APIs externes.
- Vens a repassar? → Fes el Repàs interactiu i després els butlletins.

📍 **Primer punt:** 01 · El protocol HTTP en 30 segons

🎓 **Fi del curs.** Vas començar escrivint `System.out.println("Hola, mundo")` en una terminal i acabes servint i consumint APIs per Internet. De consola a web en 14 unitats: no és poc. La propera parada ja no té guia: tria el teu camí (Spring Boot, el teu projecte final, la teva primera entrevista) i segueix programant.

01 · El protocol HTTP en 30 segons

Petició-resposta, verbs HTTP, codis d'estat i per què tota la web se sosté sobre una conversa tan simple 🌐⚡

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 01 · El protocol HTTP en 30 segons

---

📬 La idea en una frase

> **HTTP és un protocol de petició-resposta: el client demana un recurs (una URL) i el servidor li respon amb un codi d'estat i el contingut. Això és tot. La resta són capes que es construeixen damunt.**

En la U14 ja tancaves fitxers amb `try-with-resources` i en la U15 connectaves Java a bases de dades. Ara el protagonista és un altre: la web. I tant si algun dia uses Tomcat, Spring Boot o un microservei al núvol: tots parlen el mateix idioma, i aquell idioma es diu HTTP. Avui ho aprens en 30 segons… i en unes poques pàgines més.

💬 La conversa de la web

Quan escrius `https://google.com` en el teu navegador, ocorre això:

```
Tu (client)                    Google (servidor)
    ── GET / HTTP/1.1 ──────────►
    │                               │
    ◄── HTTP/1.1 200 OK ────────────
         Content-Type: text/html
         (el HTML de Google)
```

Tres peces:

1. **Petició:** el client demana un recurs. Inclou un mètode (`GET`), una ruta (`/`) i capçaleres (informació extra com `User-Agent`).
2. **Resposta:** el servidor contesta amb un **codi d'estat** (`200 OK`) i un cos (el HTML, un JSON, una imatge…).
3. **Capçaleres:** notes al marge que viatgen en tots dos sentits. La més important de moment: `Content-Type`, que li diu al client quin tipus de contingut li envies (`text/html`, `application/json`…).

> 💡 **Consell:** memoritza l'estructura del codi d'estat: el primer dígit és la família. **2xx** tot bé, **3xx** redireccions, **4xx** el client s'ha equivocat, **5xx** el servidor ha fallat. Amb això ja vas sobradet per a interpretar qualsevol error.

🎤 Els verbs d'HTTP

Els mètodes HTTP són el verb de la frase. N'hi ha molts, però amb aquests quatre fas el 99 % de la teva vida:

Mètode	Què fa	Analogia
`GET`	Demanar informació	Preguntar
`POST`	Enviar informació nova	Entregar un formulari
`PUT`	Reemplaçar un recurs complet	Tornar una cosa per una de nova
`DELETE`	Esborrar un recurs	Llançar a la paperera

Fixa't en la diferència fina: `GET` no canvia res al servidor (és de només lectura), mentre que `POST`, `PUT` i `DELETE` el modifiquen. Per això un enllaç normal és `GET` i l'enviament d'un formulari de registre és `POST`.

🚦 Els codis d'estat més comuns

No necessites memoritzar els centenars que existeixen. Amb la família i aquests clàssics et basta:

Codi	Significat	Moment típic
`200`	Tot bé	Pàgina carregada correctament
`201`	Recurs creat	Un `POST` que crea alguna cosa nova
`404`	No trobat	La URL no existeix (o l'usuari no existeix)
`403`	Prohibit	Sense permís (sol faltar un token)

Codi	Significat	Moment típic
`500`	Error intern	El servidor ha petat per dins

> 🐛 **Error típic:** quan el teu servidor torna `500`, quasi sempre és una excepció que no vas capturar dins del handler. El navegador només veu `500`, però la teva consola de Java et compta tota la història.

🧩 Anatomia d'una URL

Tot l'anterior ocorre sobre una URL. I una URL té parts que t'interessen:

```
http://localhost:8080/saludo?nombre=Ana&edad=25
└──┘   └───────┘  └──┘ └────┘ └──────────────┘
protocol   domini    port   ruta     query string
```

La **query string** (després de `?`) són parelles `clau=valor` separades per `&`. És el que envia un formulari GET i el que llegiràs al punt 4. El port per defecte d'HTTP és el 80 (no es veu), però en desenvolupament usaràs el 8080.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Què componen sempre la resposta d'un servidor HTTP?
2. Quin mètode HTTP usaries perquè un usuari registri un compte nou: `GET` o `POST`? I per a veure el seu perfil?
3. Què significa un `404`? I un `500`?
4. En `http://localhost:8080/saludo?nombre=Ana&edad=25`, què és `nombre=Ana&edad=25`?

> 🔄 **Respostes**
>
> 1. Un **codi d'estat** (com `200 OK`) i un **cos** (HTML, JSON, imatge…) amb les seves capçaleres.
> 2. `POST` per a registrar (crea alguna cosa al servidor). `GET` per a veure el perfil (només lectura).

3. `404` és que el recurs no existeix; `500` és que el servidor ha fallat internament.
4. La **query string**: els paràmetres que viatgen a la URL com a parelles `clau=valor`.

✅ Resum en 3 frases

1. **HTTP és petició-resposta:** el client demana una URL amb un mètode i el servidor respon amb un codi d'estat i un contingut.
2. Els mètodes clau són **GET** (llegir), **POST** (crear), **PUT** (reemplaçar) i **DELETE** (esborrar), i els codis s'agrupen en famílies: 2xx bé, 4xx culpa del client, 5xx culpa del servidor.
3. La **query string** (`?clau=valor&clau2=valor2`) és com viatgen les dades en una petició GET.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | HTTP | L'idioma que parlen client i servidor |
> | Petició | El que el client demana (mètode + URL + capçaleres) |
> | Resposta | El que el servidor contesta (codi + cos + capçaleres) |
> | `Content-Type` | Capçalera que diu quin tipus de contingut s'envia |
> | Query string | Els paràmetres `clau=valor` que viatgen a la URL |

📚 Tornar a l'índex de la unitat · **Anterior:** — · **Següent:** 02 · Servidor web mínim amb HttpServer

02 · Servidor web mínim amb HttpServer

El teu primer servidor Java: HttpServer, createContext i HttpExchange en menys de 20 línies 🖥️🚀

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 02 · Servidor web mínim amb HttpServer

---

📬 La idea en una frase

> **Java porta un servidor HTTP de sèrie en `com.sun.net.httpserver.HttpServer`: sense Tomcat, sense Spring, sense instal·lar res. En menys de 20 línies tens un servidor responent en `http://localhost:8080`.**

Al punt anterior vas veure el protocol. Ara toca posar-lo en marxa: no amb Tomcat ni amb un framework, sinó amb el que Java ja porta dins des de la versió 6. És mínim, és didàctic i, per a aprendre com funciona una petició per dins, és perfecte.

🖥️ El servidor mínim

Copia això, compila i executa:

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class ServidorMinimo {
    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/", intercambio -> {
            String resp = "Hola, mundo web!";
            intercambio.sendResponseHeaders(200, resp.length());
            OutputStream os = intercambio.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Obre `http://localhost:8080` en el teu navegador. **Acabes de crear el teu primer servidor web en Java.** Ara desmuntem les peces:

- **`HttpServer.create(new InetSocketAddress(8080), 0)`** — crea el servidor al port 8080. El `0` és el nombre de connexions en espera (el backlog); amb 0 val.
- **`server.createContext("/", lambda)`** — registra una ruta i li associa un *handler*: una lambda que rep un `HttpExchange`, l'objecte que representa "una petició que està entrant".
- **`intercambio.sendResponseHeaders(200, longitud)`** — envia el codi d'estat i la grandària del cos. Si passes `0` o `-1`, Java usa *chunked* i no necessites la grandària exacta.
- **`getResponseBody().write(...)` i `.close()`** — escriu el cos i tanca el canal. Tancar és obligatori: si no, el navegador es queda esperant.
- **`server.setExecutor(null)`** — usa l'executor per defecte; cada petició s'atén en un fil propi.
- **`server.start()`** — arranca. A partir d'aquí el `main` no s'"acaba": el servidor es queda escoltant.

> 💡 **Consell:** el `try-with-resources` de la U14 no val aquí: el servidor ha d'estar viu mentre el `main` segueix corrent. Per això `start()` no està dins d'un `try`.

🕶️ Be the Code, my friend…

> **Don Tip:** el mètode HTTP el decideix el client. `intercambio.getRequestMethod()` et diu quin ha estat.

Afegeix just abans de `server.start()`:

```
server.createContext("/track", intercambio -> {
    System.out.println("Mètode: " + intercambio.getRequestMethod());
    System.out.println("URI: " + intercambio.getRequestURI());
                                System.out.println("Capçaleres:        "        +
intercambio.getRequestHeaders().entrySet());
    intercambio.getResponseHeaders().add("Content-Type", "text/plain");
    intercambio.sendResponseHeaders(200, 0);
    try (var os = intercambio.getResponseBody()) {
        os.write("Tot bé, gràcies per preguntar.".getBytes());
    }
});
```

Obre `http://localhost:8080/track?nombre=Ana&edad=25` i mira la consola del servidor: veuràs el mètode, la URI amb la query string i totes les capçaleres que t'envia el navegador. És el millor "espia" que tindràs en tota la unitat.

> › 🔄 **Què esperes veure?**
>
> Alguna cosa semblant a això (simplificat):
>
> ```
> Mètode: GET
> URI: /track?nombre=Ana&edad=25
> Capçaleres:  [Host=[localhost:8080],  User-Agent=[Mozilla/5.0  ...],  Accept=
> [text/html,...]]
> ```
>
> És la petició HTTP del punt 1, destripada i en viu.

🚨 "Address already in use"

L'error més típic en arrancar el servidor. Significa que el port 8080 ja l'està usant un altre procés (una instància anterior de la teva, o una altra aplicació). Dues solucions:

- Canvia el port en `new InetSocketAddress(8081, 0)`.
- O troba i mata el culpable. A Windows: `netstat -ano | findstr 8080` i després `taskkill /PID <numero> /F`.

> ⚠️ **Advertència:** els ports per davall de 1024 (com el 80) requereixen permisos d'administrador. Queda't amb el 8080 per a desenvolupament.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina classe de Java et dona un servidor HTTP sense dependències externes?
2. Quin mètode de `HttpServer` associes a una ruta amb el seu handler?
3. Quin objecte rep la teva lambda i què representa?
4. Què passa si oblides tancar el `getResponseBody()`?

> 🔄 **Respostes**
>
> 1. `com.sun.net.httpserver.HttpServer` (paquet `jdk.httpserver`, inclòs en el JDK).
> 2. `server.createContext("/ruta", handler)`.
> 3. Un `HttpExchange`, que representa una petició entrant i et dona accés a la petició i a la resposta.
> 4. El navegador es queda esperant la resposta per sempre: el cos no es considera enviat fins que tanques el canal.

✅ Resum en 3 frases

1. **`HttpServer`** és el servidor HTTP que Java porta de sèrie: el crees en un port, registres rutes amb `createContext` i l'arranques amb `start()`.
2. El **handler** rep un `HttpExchange` amb el qual llegeixes la petició, envies el codi d'estat amb `sendResponseHeaders` i escrius el cos amb `getResponseBody()`.
3. **Tanca sempre el cos** de la resposta i recorda que el servidor no es deté fins que tu ho vulguis: és la teva primera aplicació "que no s'acaba".

🐛 Vocabulari ràpid

Terme	Idea general
`HttpServer`	El servidor HTTP del JDK
`createContext`	Associa una ruta amb el seu handler
`HttpExchange`	Objecte que representa una petició entrant
Handler	La lambda que processa cada petició d'una ruta
`sendResponseHeaders`	Envia el codi d'estat (i la grandària del cos)

📚 Tornar a l'índex de la unitat · **Anterior:** 01 · El protocol HTTP en 30 segons · **Següent:** 03 · Servint HTML

03 · Servint HTML

El teu servidor passa de parlar en text pla a servir pàgines completes: Content-Type, cadenes ben formades i fitxers 📄🎨

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 03 · Servint HTML

📬 La idea en una frase

> **Tornar text pla és com parlar-li al navegador a cau d'orella: s'assabenta, però no mostra res bonic. Amb `Content-Type: text/html` i una cadena ben formada, el teu servidor es converteix en una web de veritat.**

Al punt 2 el servidor contestava amb una frase solta. Avui li ensenyem a vestir-se: pàgines HTML completes. Dos camins, tots dos legítims: escriure-ho directament en una cadena Java o llegir-ho des d'un fitxer (molt més còmode quan la pàgina creix).

📄 HTML des d'una cadena

El pas natural: en comptes de `"Hola, mundo web!"`, tornes una pàgina completa. Fixa't en el bloc de text (Java 15+):

```
server.createContext("/", intercambio -> {
    String html = """
        <!DOCTYPE html>
        <html>
        <head><title>Mi App Java</title>
        <meta charset="UTF-8">
        </head>
        <body>
            <h1>Bienvenido a mi App</h1>
            <p>Esto se sirve desde Java.</p>
        </body>
        </html>
        """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Dos detalls importants:

- **`Content-Type: text/html; charset=UTF-8`** — sense aquesta capçalera, el navegador pot tractar el teu HTML com a text pla o interpretar malament les titles. El `charset=UTF-8` és obligatori si la teva pàgina porta caràcters com á, ñ o €.
- **`html.getBytes().length`** — la grandària del cos en **bytes**, no en caràcters. Una ñ ocupa 2 bytes en UTF-8; si passes `html.length()` (caràcters), el navegador pot quedar-se esperant la resta.

> 🐛 **Error típic:** `sendResponseHeaders(200, 0)` amb HTML funciona, però amb `0` el navegador no sap quan acaba el cos amb connexions persistents. Passa sempre la grandària en bytes (o usa `-1` per a *chunked*).

📂 HTML des d'un fitxer

Escriure HTML dins de Java és lleig quan la pàgina creix: barreges dos llenguatges i qualsevol cometa és una agonia. L'alternativa professional: guarda la pàgina en un fitxer i llegeix-la amb `Files.readString()` (el mateix API de la U14):

```
server.createContext("/", intercambio -> {
    String html = Files.readString(Path.of("web", "index.html"));
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Amb aquesta estructura, l'HTML viu en `web/index.html` (dins del teu projecte, junt als fonts) i el codi Java només s'encarrega de servir-lo. Canviar el disseny ja no exigeix recompilar: edites el fitxer, recarregues el navegador i llest.

> 💡 **Consell:** el fitxer pot portar CSS i JavaScript enllaçats amb rutes relatives (`estilos.css`, `app.js`). Però compte: aquelles rutes també les demana el navegador al servidor, així que hauràs de servir aquells fitxers també. Es veu al punt 6 amb el projecte de tasques.

🕶️ Be the Code, my friend…

> **Don Tip:** serveix l'HTML amb el `Content-Type` correcte i el navegador és el teu millor aliat; oblida la capçalera i el navegador et mostrarà el codi en comptes de la pàgina.

Exercici: el servidor despistat

Un company t'ensenya aquest handler i jura que funciona:

```
server.createContext("/", intercambio -> {
    String html = "<h1>Hola mundo</h1>";
    intercambio.sendResponseHeaders(200, html.length());
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Diu que "de vegades" el navegador mostra `<h1>Hola  mundo</h1>` literal en comptes d'un encapçalament. Per què? Quants bugs hi ha?

> › 🔄 **Solució**
>
> Hi ha **dos** bugs:

1. Falta `Content-Type: text/html`. Sense ell, el navegador no sap que és HTML i pot mostrar-lo com a text pla.
2. Es passa `html.length()` (caràcters) en comptes de `html.getBytes().length` (bytes). Amb un HTML tan curt quasi mai falla, però en quan hi hagi un caràcter accentuat o un emoji, el navegador esperarà més bytes dels que li arriben.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quina capçalera necessites perquè el navegador renderitze la teva resposta com a HTML?
2. Què li passes a `sendResponseHeaders` com a segon argument: caràcters o bytes?
3. Amb quin mètode de la U14 llegeixes el contingut d'un fitxer en un `String`?
4. Per què és millor servir l'HTML des d'un fitxer que escriure 50 línies dins del codi Java?

> 🔄 **Respostes**
>
> 1. `Content-Type: text/html; charset=UTF-8`.
> 2. **Bytes**: `html.getBytes().length`. Els caràcters accentuats ocupen més d'un byte en UTF-8.
> 3. `Files.readString(Path.of("web", "index.html"))`.
> 4. Perquè separes el disseny del codi: edites l'HTML sense recompilar i el codi Java queda net.

✅ Resum en 3 frases

1. Per a servir HTML, fixa **`Content-Type: text/html; charset=UTF-8`** i passa la grandària del cos **en bytes** (`getBytes().length`).
2. L'HTML pot anar **en una cadena** (ràpid per a provar) o **en un fitxer** llegit amb `Files.readString()` (net i professional).
3. Tancar el `getResponseBody()` segueix sent obligatori: és el senyal que el navegador espera per a pintar la pàgina.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
`text/html`	El `Content-Type` de les pàgines web
`charset=UTF-8`	Declaració que el text usa UTF-8 (titles incloses)
`Files.readString`	Llegeix un fitxer complet en un `String` (vist a U14)
Text block	`"""..."""`: cadena multilínia per a escriure HTML còmodament

📚 Tornar a l'índex de la unitat · **Anterior:** 02 · Servidor web mínim amb HttpServer · **Següent:** 04 · Paràmetres GET i formularis POST

04 · Paràmetres GET i formularis POST

El navegador pregunta amb la URL i entrega amb el cos: llegeix la query string i el body com un professional 📥📤

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 04 · Paràmetres GET i formularis POST

📬 La idea en una frase

> **El client pregunta (GET) amb la query string de la URL i entrega (POST) amb el cos de la petició. Saber llegir totes dues coses és saber entendre el que la teva web rep.**

Fins ara el servidor contestava el mateix a tothom. Avui aprèn a escoltar: què demana el client i què li envia. Amb GET llegeixes la URL; amb POST llegeixes el cos. Tots dos arriben com a text i la teva tasca és convertir-los en dades útils.

📥 GET: la query string

Quan un formulari s'envia per GET, les dades viatgen a la URL: `http://localhost:8080/saludo?nombre=Ana`. Per a llegir-les:

```
server.createContext("/saludo", intercambio -> {
    String query = intercambio.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre=")) {
        nombre = query.split("=")[1];
    }
    String html = "<h1>Hola, " + nombre + "!</h1>";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

- `getRequestURI().getQuery()` et dona `"nombre=Ana"` (o `null` si no hi ha query).
- L'`if` evita dos drames: que `query` sigui `null` i que no comenci per `nombre=`.

> ⚠️ **Advertència:** això és **fràgil**. Si el nom conté `&` o `%20` (espai codificat), `split("=")[1]` et dona brossa o peta. En producció usa `URLDecoder.decode(valor, "UTF-8")` i un parser de query de veritat. Aquest és un exemple didàctic, no codi de producció.

📥 POST: el cos de la petició

Per a rebre dades d'un formulari POST necessites llegir el **cos** de la petició:

```
server.createContext("/procesar", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        byte[] body = intercambio.getRequestBody().readAllBytes();
        String datos = new String(body);
        // datos = "nombre=Ana&edad=25"
        String nombre = extraerParametro(datos, "nombre");
        String respuesta = "<h1>Recibido: " + nombre + "</h1>";
        intercambio.getResponseHeaders()
            .set("Content-Type", "text/html");
        intercambio.sendResponseHeaders(200,
            respuesta.getBytes().length);
        intercambio.getResponseBody()
            .write(respuesta.getBytes());
        intercambio.getResponseBody().close();
    }
});
```

I l'HTML del formulari que l'alimenta:

```
<form action="/procesar" method="POST">
  <input name="nombre" placeholder="Tu nombre">
  <button>Enviar</button>
</form>
```

> 💡 **Consell:** per què `"POST".equals(intercambio.getRequestMethod())` i no `intercambio.getRequestMethod().equals("POST")`? Perquè si `getRequestMethod()` torna `null`, la segona forma peta amb `NullPointerException`. La primera forma és **null-safe**. És una mania que t'estalviarà maldecaps per sempre.

🧰 El parser casolà

El cos `"nombre=Ana&edad=25"` té el mateix format que la query string: parelles `clau=valor` separades per `&`. Un parser mínim per a tots dos casos:

```
static String extraerParametro(String datos, String clave) {
    if (datos == null) return null;
    for (String par : datos.split("&")) {
        String[] trozos = par.split("=");
        if (trozos.length == 2 && trozos[0].equals(clave)) {
            return trozos[1];
        }
    }
    return null;
}
```

No és la vuitena meravella, però per a aprendre com funciona el format per dins val. Quan al punt 6 facis l'API REST, els cossos seran JSON i el parser el farà Gson.

🕶️ Be the Code, my friend…

> **Don Tip:** GET i POST poden portar les mateixes dades, però en llocs diferents. Un GET és reproduïble (pots repetir la URL); un POST de registre no s'hauria de repetir sense voler.

Exercici: el formulari amb dos camps

Un formulari envia `nombre=Ana&edad=25` per POST a `/procesar`. Amb l'`extraerParametro` de dalt:

1. Què torna `extraerParametro(datos, "edad")`?
2. I `extraerParametro(datos, "ciudad")`?
3. Què torna si `datos` és `null`?

> ▸ 🔄 **Solució**
>
> 1. `"25"`. El bucle parteix `"nombre=Ana&edad=25"` per `&`, troba el parell `edad=25` i torna `"25"`.
> 2. `null`. No existeix cap parell la clau del qual sigui `ciudad`.

3. `null`, gràcies a l'`if (datos == null) return null;` de la primera línia. Per això sempre es comprova abans d'usar `split`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. On viatgen les dades d'un formulari GET?
2. On viatgen les dades d'un formulari POST?
3. Per què `"POST".equals(mètode)` és millor que `mètode.equals("POST")`?
4. Quin mètode de `HttpExchange` et dona el cos de la petició?

> 🔄 **Respostes**
>
> 1. En la **query string** de la URL (`?nombre=Ana&edad=25`), que llegeixes amb `getRequestURI().getQuery()`.
> 2. En el **cos** de la petició, que llegeixes amb `getRequestBody().readAllBytes()`.
> 3. Perquè és **null-safe**: si `getRequestMethod()` torna `null`, la primera forma no peta i la segona llança `NullPointerException`.
> 4. `intercambio.getRequestBody()`, del qual llegeixes els bytes amb `readAllBytes()`.

✅ Resum en 3 frases

1. Els **GET** porten les seves dades en la query string (`?clau=valor&...`) i les llegeixes amb `getRequestURI().getQuery()`.
2. Els **POST** porten les seves dades en el **cos** de la petició i les llegeixes amb `getRequestBody().readAllBytes()`.
3. En tots dos casos és text amb format `clau=valor`, i la mania de `"POST".equals(...)` en comptes de `...equals("POST")` et salva dels `NullPointerException`.

> 🐛 **Vocabulari ràpid**
>
Terme	Idea general
Query string	Dades del GET, a la URL

Terme	Idea general
Request body	Dades del POST, en el cos de la petició
`URLDecoder`	Converteix `%20` i amics en caràcters llegibles
Null-safe	Codi que no peta encara que rebi `null`

📚 Tornar a l'índex de la unitat · **Anterior:** 03 · Servint HTML · **Següent:** 05 · Tornant JSON: com una API de veritat

05 · Tornant JSON: com una API de veritat

application/json, fetch des del frontend i per què les aplicacions modernes separen dades de disseny 🧩🔀

> 🗺️ **Estàs en:** 🌐 **U16 · Servir i consumir APIs amb Web** → 05 · Tornant JSON: com una API de veritat

---

📬 La idea en una frase

> **Les aplicacions modernes no tornen HTML al navegador: tornen dades en JSON i que el frontend se les arregle. Quan el teu servidor respon amb `application/json`, deixa de ser una pàgina web i es converteix en una API.**

Fins aquí el teu servidor parlava en HTML, l'idioma dels humans. Ara aprèn l'idioma de les màquines: JSON. El frontend (JavaScript) demana dades i el backend li les dona en JSON, sense barrejar disseny amb dades. És la separació que fa funcionar totes les aplicacions modernes.

🧩 Què és JSON (en 10 segons)

JSON és text amb una estructura de dues peces:

```
{
  "id": 1,
  "nombre": "Ana",
  "edad": 25,
  "activo": true
}
```

- **Objectes** entre claus `{...}`: col·lecció de parells `"clau": valor`.
- **Arrays** entre corxets `[...]`: llistes de valors.
- Valors possibles: textos entre cometes, números, `true`/`false`, `null`, objectes i arrays.

Un `record` de Java (els vas veure a la U12) i un objecte JSON són cosins germans: mateixa idea de "dades amb nom". La diferència és que JSON és text, així que qualsevol llenguatge l'entén.

🚀 Servir JSON des del teu servidor

La clau de tot és una capçalera i una cadena:

```
server.createContext("/api/usuarios", intercambio -> {
    String json = """
        [
            {"id":1,"nombre":"Ana","edad":25},
            {"id":2,"nombre":"Luis","edad":30}
        ]
        """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "application/json");
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

Fixa't: res ha canviat respecte a l'HTML excepte dues coses:

- El `Content-Type` ara és **`application/json`**.
- El contingut és un array JSON amb les dades.

> ⚠️ **Advertència:** si tornes JSON amb `Content-Type: text/html` o sense capçalera, el navegador no ho tractarà com a dades. Moltes APIs externes rebutgen respostes que diuen ser JSON i no ho són… i ho notaràs. Capçalera correcta o res.

📡 Consumir-lo amb fetch

I ara l'altra cara de la moneda: des de l'HTML, JavaScript demana les dades i les pinta. Res de recarregar la pàgina: el `fetch` fa una petició en segon pla:

```
<script>
fetch('/api/usuarios')
  .then(r => r.json())
  .then(data => console.log(data));
</script>
```

El que ocorre per dins és exactament el punt 1: el navegador fa un `GET /api/usuarios`, el teu servidor respon amb `200` i un JSON, i `r.json()` el converteix en objectes JavaScript llestos per a usar.

> 💡 **Consell:** aquest és el patró de totes les aplicacions modernes: el **backend** serveix dades (JSON) i el **frontend** les consumeix i decideix com mostrar-les. Aprendre a servir JSON és aprendre a fer APIs; el frontend ja ve de sèrie al navegador.

🕶️ Be the Code, my friend…

> **Don Tip:** quan el `fetch` falla, no miris només el `console.log`: obre la pestanya *Network* del navegador i mira el codi d'estat i el `Content-Type` de la resposta. El 90 % dels misteris es resolen allà.

Exercici: l'endpoint amb cap

Escriu el handler de `/api/hora` que torni aquest JSON:

```
{"hora": "14:30:01", "zona": "Europe/Madrid"}
```

Completa el codi (sense executar, a ull):

```
server.createContext("/api/hora", intercambio -> {
    String json = """
        {"hora": "______", "zona": "______"}
        """;
    intercambio.getResponseHeaders()
        .set("______", "application/json");
    intercambio.sendResponseHeaders(______, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

> › 🔄 Solució

```
server.createContext("/api/hora", intercambio -> {
    String json = """
        {"hora": "14:30:01", "zona": "Europe/Madrid"}
        """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "application/json");
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

Els buits eren: l'hora (`14:30:01`), la zona (`Europe/Madrid`), `Content-Type` i `200`.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin `Content-Type` uses per a servir JSON?
2. Quines dues estructures componen el format JSON?
3. Quin mètode de JavaScript fa una petició HTTP des del navegador?
4. Quin avantatge té servir dades JSON en comptes d'HTML?

> 🔄 **Respostes**
>
> 1. `application/json`.
> 2. **Objectes** (`{...}`, parells clau-valor) i **arrays** (`[...]`, llistes).
> 3. `fetch()`, que torna una promesa amb la resposta.
> 4. Separes dades de disseny: qualsevol frontend (web, mòbil, un altre servidor) pot consumir les mateixes dades, perquè JSON és text universal.

✅ Resum en 3 frases

1. **JSON** és el format de dades universal: objectes `{...}` i arrays `[...]`, i un `record` de Java és el seu cosí germà.
2. Per a servir JSON, fixa **`Content-Type: application/json`** i la resta del handler és idèntic al de l'HTML.

3. El **frontend** el consumeix amb `fetch(...).then(r => r.json())`, i així neix la separació entre dades i disseny que defineix les APIs.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | JSON | Format de dades universal en text |
> | `application/json` | El `Content-Type` de JSON |
> | API | Interfície per la qual el teu servidor exposa dades |
> | `fetch` | La forma de JavaScript de fer peticions HTTP |
> | Frontend / Backend | Qui mostra (navegador) / qui serveix (servidor) |

📚 Tornar a l'índex de la unitat · **Anterior:** 04 · Paràmetres GET i formularis POST · **Següent:** 06 · Mini projecte: gestor de tasques (API REST)

06 · Mini projecte: gestor de tasques (API REST)

Ajunta-ho tot en una API REST: GET, POST, PUT i DELETE amb dades en memòria 🗂️🔧

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 06 · Mini projecte: gestor de tasques (API REST)

📬 La idea en una frase

> **Ja saps servir HTML i JSON. Ara ajunta-ho tot en una API REST amb les quatre operacions de la vida: crear, llegir, actualitzar i esborrar (CRUD), amb les dades vivint en un `ArrayList` en memòria.**

Aquest és el punt on tot cobra sentit. Un servidor que només contesta "hola" és un monòleg. Una **API REST** és una conversa: el client fa `GET`, `POST`, `PUT` o `DELETE` i el servidor respon com cal. I no necessites base de dades: un `ArrayList` en memòria (vas veure les col·leccions a la U12) és suficient perquè funcioni.

🗂️ Què és REST (en 20 segons)

REST és una forma ordenada d'organitzar rutes i mètodes perquè l'API s'expliqui sola:

Mètode	Ruta	Acció	Resultat
GET	`/api/tareas`	Llistar totes	`200` + JSON
POST	`/api/tareas`	Crear una	`201` + la tasca creada
PUT	`/api/tareas/1`	Actualitzar la tasca 1	`200`
DELETE	`/api/tareas/1`	Esborrar la tasca 1	`204` (sense contingut)

La regla d'or: **el mètode diu el verb i la ruta diu el substantiu.** `GET /api/tareas` és "dona'm les tasques"; `DELETE /api/tareas/3` és "esborra la tasca 3". Així, una taula de quatre files documenta tota l'API.

🧱 El model i el magatzem

Primer, la tasca. Un `record` (genèric i net, com a la U12):

```
record Tarea(int id, String titulo, boolean completada) {}
```

I el magatzem: un `ArrayList<Tarea>` + un comptador d'IDs. El declares com a camp de la classe del servidor:

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class ApiTareas {
    private static final List<Tarea> tareas = new ArrayList<>();
    private static final AtomicInteger contador = new AtomicInteger(1);

    static {
         tareas.add(new Tarea(contador.getAndIncrement(), "Aprender HttpServer",
false));
          tareas.add(new Tarea(contador.getAndIncrement(), "Hacer la práctica de
JSON", false));
    }

    // ... ací van els contexts
}
```

> 💡 **Consell:** `AtomicInteger` et dona IDs que no es repeteixen ni amb peticions simultànies. Amb un `int` normal, dues peticions alhora podrien llegir el mateix valor. En producció, aquell `ArrayList` seria una base de dades (JDBC, com a la U124), però el patró és idèntic.

🔧 Els quatre endpoints

GET: llistar-ho tot

```
server.createContext("/api/tareas", intercambio -> {
    if ("GET".equals(intercambio.getRequestMethod())) {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < tareas.size(); i++) {
            Tarea t = tareas.get(i);
            if (i > 0) sb.append(",");
            sb.append("{\"id\":").append(t.id())
              .append(",\"titulo\":\"").append(t.titulo())
              .append("\",\"completada\":").append(t.completada()).append("}");
        }
        sb.append("]");
        responderJSON(intercambio, sb.toString(), 200);
    }
});
```

Aquí el JSON es construeix a mà perquè vegin què és cada peça. Al punt 7 descobriràs que Gson ho fa per tu amb una línia: `new Gson().toJson(tareas)`.

POST: crear

```
server.createContext("/api/tareas", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        String body = new String(intercambio.getRequestBody().readAllBytes());
        // body = {"titulo": "Comprar café"}
        String titulo = body.split("\"titulo\":\"")[1].split("\"")[0];
        Tarea nueva = new Tarea(contador.getAndIncrement(), titulo, false);
        tareas.add(nueva);
        responderJSON(intercambio,
                      "{\"id\":" + nueva.id() + ",\"titulo\":\"" + titulo +
"\",\"completada\":false}",
            201);
    }
});
```

> ⚠️ **Advertència:** parsejar JSON amb `split` és un apaño didàctic. Si el títol porta cometes, es trenca. El camí professional és Gson (punts 7 i 8), que parseja això en una línia i sense drama.

PUT i DELETE: actualitzar i esborrar per ID

Per a rutes amb paràmetre (`/api/tareas/3`) cal llegir l'ID de la pròpia ruta:

```
server.createContext("/api/tareas/", intercambio -> {
    String ruta = intercambio.getRequestURI().getPath();
    int id = Integer.parseInt(ruta.substring("/api/tareas/".length()));

    if ("PUT".equals(intercambio.getRequestMethod())) {
        for (int i = 0; i < tareas.size(); i++) {
            if (tareas.get(i).id() == id) {
                tareas.set(i, new Tarea(id, tareas.get(i).titulo(), true));
                responderJSON(intercambio, "{\"actualizada\":true}", 200);
                return;
            }
        }
        responderJSON(intercambio, "{\"error\":\"tarea no encontrada\"}", 404);
    } else if ("DELETE".equals(intercambio.getRequestMethod())) {
        boolean borrada = tareas.removeIf(t -> t.id() == id);
        if (borrada) {
            intercambio.sendResponseHeaders(204, -1);
        } else {
               responderJSON(intercambio, "{\"error\":\"tarea no encontrada\"}",
404);
        }
        intercambio.close();
    }
});
```

Fixa't en dos detalls: el `404` quan l'ID no existeix (el client pregunta per una cosa que no hi és), i el `204` del DELETE (esborrar "bé" no torna cos).

> 💡 **Consell:** `tareas.removeIf(...)` és la joia de les col·leccions de la U12: esborra tots els elements que compleixin la condició en una línia, sense `ConcurrentModificationException` perquè és un mètode de la pròpia llista.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode HTTP uses per a crear un recurs? I per a llistar-los?
2. Quin codi d'estat tornes quan el client demana un ID que no existeix?
3. Què torna un `DELETE` que ha sortit bé?

4. Per què serveix un `ArrayList` com a magatzem si en reiniciar es perd tot?

> 🔄 **Respostes**
>
> 1. `POST` per a crear, `GET` per a llistar.
> 2. `404`, perquè el recurs demanat no existeix.
> 3. `204` (sense contingut): esborrar amb èxit no necessita tornar cos.
> 4. Perquè per a aprendre el patró REST basta; la persistència (amb JDBC, U15) canvia el magatzem, no les rutes ni els mètodes.

✅ Resum en 3 frases

1. Una **API REST** ordena rutes i mètodes: `GET` llegeix, `POST` crea, `PUT` actualitza, `DELETE` esborra, i cada ruta diu quin recurs toca.
2. El **magatzem en memòria** (`ArrayList` + `AtomicInteger`) és perfecte per a aprendre el patró; la base de dades de la U15 només substituïx el magatzem, no les rutes.
3. Gestiona els **errors amb codis honestos**: `404` si no existeix, `201` si crees, `204` si esborres; així la teva API parla clar.

> 🐛 **Vocabulari ràpid**

Terme	Idea general
REST	Estil d'organització de rutes i mètodes d'una API
CRUD	Crear, llegir, actualitzar, esborrar (les 4 operacions)
Endpoint	Una ruta concreta de l'API (`/api/tareas`)
Path param	El 3 de `/api/tareas/3`, llegit de la pròpia ruta
`204`	Resposta d'èxit sense contingut (típic de DELETE)

📚 Tornar a l'índex de la unitat · **Anterior:** 05 · Tornant JSON: com una API de veritat · **Següent:** 07 · Consumir APIs externes amb HttpClient

07 · Consumir APIs externes amb HttpClient

El teu programa es converteix en client: HttpClient, GET, BodyHandlers i parsejar JSON amb Gson 📡🔍

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 07 · Consumir APIs externes amb HttpClient

---

📬 La idea en una frase

> **Fins ara has estat el servidor. Amb `java.net.http.HttpClient`, el teu programa es converteix en client i li pregunta a les APIs de tot el món: GitHub, el temps, la teva xarxa social preferida…**

Els punts anteriors eren sobre rebre. Ara toca demanar: en el món real, els teus programes criden a APIs de tercers constantment. Java 11 porta `HttpClient`, un client HTTP modern i sense dependències externes, amb suport d'HTTP/2, peticions síncrones i asíncrones, i gestió de capçaleres.

> 📝 **Nota:** abans de Java 11 havies d'usar `HttpURLConnection` (lleig, verbós, un càstig) o llibreries externes com Apache HttpClient o OkHttp. `java.net.http.HttpClient` va arribar per a salvar la humanitat.

📡 El trio sagrat del client

Consumir una API són sempre tres peces:

1. **`HttpClient`** — el que fa la crida. Un objecte, reutilitzable per a totes les teves peticions.
2. **`HttpRequest`** — la petició: URL, mètode, capçaleres. Es construeix amb *builder*.
3. **`HttpResponse`** — la resposta: codi d'estat, capçaleres i cos.

El GET més bàsic que existeix:

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClienteGET {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/octocat"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Cuerpo: " + response.body());
    }
}
```

Dos mètodes clau:

- **`client.send(...)`** — fa la petició de forma **síncrona**: el programa es queda esperant fins que arriba la resposta (o falla). És el que usaràs el 90 % del temps.
- **`HttpResponse.BodyHandlers.ofString()`** — li diu a Java "converteix-me el cos en un `String`". N'hi ha més: `ofByteArray()`, `ofInputStream()`, `ofFile(Path)`… segons el que necessites.

> 💡 **Consell:** per a provar, jsonplaceholder (`https://jsonplaceholder.typicode.com`) és l'API de joguina perfecta: torna JSON d'exemple per a practicar sense registre ni tokens.

🧩 Parsejar JSON amb Gson

El cos que torna GitHub és JSON. I aquí ve la notícia trista: **Java no porta parser JSON natiu**. Necessites una llibreria. Les més usades:

Llibreria	Grup Maven	Ideal per a
Gson (Google)	`com.google.code.gson:gson`	Senzilla, mapeig a classes
Jackson	`com.fasterxml.jackson.core:jackson-databind`	Potent, ràpida, estàndard industrial
org.json	`org.json:json`	La més simple, sense mapeig

Usarem **Gson**, la més intuïtiva. Dues formes d'usar-la:

Forma 1 — a lo bruto, amb `JsonObject`:

```
import com.google.gson.Gson;
import com.google.gson.JsonObject;

Gson gson = new Gson();
JsonObject json = gson.fromJson(response.body(), JsonObject.class);

String login = json.get("login").getAsString();
String nombre = json.get("name").getAsString();
System.out.println("Usuario: " + login + " — " + nombre);
```

Forma 2 — mapejant a un `record` (els de la U12):

```
record UsuarioGitHub(String login, String name, int public_repos) {}

UsuarioGitHub usuario = gson.fromJson(response.body(), UsuarioGitHub.class);
System.out.println(usuario.name() + " tiene " + usuario.public_repos() + " repos públicos");
```

> ⚠️ **Advertència:** si l'API torna camps que no existeixen en el teu `record`, Gson els ignora per defecte. Si el teu `record` té camps que no estan en el JSON, es queden en `null`. Amb Jackson pots configurar-ho amb `@JsonIgnoreProperties(ignoreUnknown = true)`.

🌟 L'exemple complet: els repos de GitHub

Ajuntem-ho tot: consumir l'API de GitHub per a llistar els repositoris d'un usuari.

```
import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class GitHubRepos {
    public static void main(String[] args) throws Exception {
        String usuario = args.length > 0 ? args[0] : "octocat";

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build();

        HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create("https://api.github.com/users/" + usuario +
"/repos"))
            .header("Accept", "application/vnd.github+json")
            .header("User-Agent", "JavaCliente/1.0")
            .timeout(Duration.ofSeconds(15))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            JsonArray repos = gson.fromJson(response.body(), JsonArray.class);

            System.out.println("Repositorios de " + usuario + ":");
            System.out.println("————————————————————————————————");
            for (int i = 0; i < repos.size(); i++) {
                JsonObject repo = repos.get(i).getAsJsonObject();
                String nombre = repo.get("name").getAsString();
                String lenguaje = repo.has("language")
                    && !repo.get("language").isJsonNull()
                    ? repo.get("language").getAsString()
                    : "?";
```

```
                int estrellas = repo.get("stargazers_count").getAsInt();
                    System.out.printf("⭐ %s (%s) ★ %d%n", nombre, lenguaje,
estrellas);
            }
        } else if (response.statusCode() == 403) {
            System.out.println("Rate limit alcanzado. Espera un minuto.");
        } else {
            System.out.println("Error " + response.statusCode()
                + " — ¿existe el usuario " + usuario + "?");
        }
    }
}
```

> 📝 **Nota:** fixa't en `repo.has("language") && !repo.get("language").isJsonNull()`. En JSON un camp pot existir però valdre `null`. Si intentes `getAsString()` sobre un `null`, Gson llança una excepció. Aquest patró és molt comú al parsejar APIs reals.

Sortida d'exemple:

```
Repositoris de octocat:
──────────────────────────────────
⭐ Hello-World (Java) ★ 2727
⭐ Spoon-Knife (HTML) ★ 13291
⭐ Octocat (?) ★ 5
```

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quines són les tres classes del trio sagrat del client HTTP?
2. Què fa `client.send(...)`: síncron o asíncron?
3. Com li dius a `HttpClient` que el cos de la resposta sigui un `String`?
4. Quina llibreria usem per a parsejar JSON i quina és la forma de mapejar-lo a una classe?

> › 🔄 **Respostes**
>
> 1. `HttpClient` (qui crida), `HttpRequest` (la petició) i `HttpResponse` (la resposta).
> 2. **Síncron**: el programa espera fins que arriba la resposta (o falla). L'asíncron és `sendAsync`, al punt 8.

3. Amb el `BodyHandler`: `HttpResponse.BodyHandlers.ofString()`.
4. **Gson**, i la forma elegant és mapejar-lo a un `record` amb `gson.fromJson(cos, MiRecord.class)`.

✅ Resum en 3 frases

1. **`HttpClient` + `HttpRequest` + `HttpResponse`** són el trio per a consumir APIs: `client.send(...)` fa la petició síncrona i `BodyHandlers.ofString()` et dona el cos com a text.
2. **Gson** parseja aquell text JSON: a lo bruto amb `JsonObject` o mapejant a un `record` amb `fromJson`.
3. En JSON un camp pot existir però ser `null`: comprova amb `has(...)` i `isJsonNull()` abans de `getAsString()`, o Gson et petarà.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `HttpClient` | El client HTTP modern de Java 11+ |
> | `BodyHandler` | Li diu a Java com convertir el cos (`ofString`, `ofByteArray`…) |
> | Gson | Llibreria per a convertir JSON en objectes Java (i al revés) |
> | `JsonObject` / `JsonArray` | El JSON sense mapejar, accessible a lo bruto |
> | Rate limit | Límit de peticions per minut que imposen les APIs |

📚 Tornar a l'índex de la unitat · **Anterior:** 06 · Mini projecte: gestor de tasques (API REST) · **Següent:** 08 · HttpClient avançat

08 · HttpClient avançat

POST amb BodyPublishers, capçaleres i timeouts, sendAsync sense bloquejar i PUT/DELETE com un client professional 🚀🧠

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 08 · HttpClient avançat

---

📬 La idea en una frase

> **El GET és el saludet. Avui aprens la resta de la conversa: enviar dades amb POST, afinar la petició amb capçaleres i timeouts, no bloquejar el programa amb `sendAsync` i fins i tot esborrar amb DELETE.**

El punt 7 era llegir. Aquest és escriure: enviar dades a una API, controlar el temps d'espera, interpretar capçaleres, llançar peticions sense congelar el programa i completar el repertori de mètodes HTTP. En acabar, ja ets un client complet.

📤 POST: enviar dades a una API

Per a crear un recurs, fer login o registrar alguna cosa, uses POST amb un cos JSON:

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClientePOST {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        String json = """
            {"title": "Aprender HttpClient",
             "body": "Ya casi lo tengo",
             "userId": 1}
            """;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Creado: " + response.body());
    }
}
```

Dues novetats:

- **`BodyPublishers.ofString(json)`** — el mirall del `BodyHandler`: converteix el teu `String` en el cos de la petició. N'hi ha `ofByteArray()`, `ofFile()`… i `ofInputStream()` per a casos rars.
- **`.header("Content-Type", "application/json")`** — sense aquesta capçalera, el servidor pot rebutjar la teva petició o interpretar mal el cos. JSON sense `Content-Type` és com enviar una carta sense remitent.

> 💡 **Consell:** `HttpRequest.BodyPublishers` té mètodes per a enviar String, byte[], fitxers, etc. El més comú és `ofString()` per a JSON. I si el servidor respon `415`, aquell codi significa

> “Unsupported Media Type”: quasi sempre és que has oblidat el `Content-Type`.

🛡️ Capçaleres, timeouts i gestió d’errors

Les APIs no sempre es comporten bé: de vegades tarden, de vegades cauen, de vegades et tornen 401, 403, 404 o 500. El teu codi ha d’estar preparat.

```java
import java.net.http.HttpConnectTimeoutException;
import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10))   // temps màxim per a connectar
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.github.com/users/someone"))
    .header("Accept", "application/vnd.github+json")
    .header("User-Agent", "MiAppJava/1.0")
      .timeout(Duration.ofSeconds(15))              // temps màxim per a rebre
resposta
    .GET()
    .build();

try {
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    int codigo = response.statusCode();
    if (codigo >= 200 && codigo < 300) {
        System.out.println("Éxito: " + response.body());
    } else if (codigo == 404) {
        System.out.println("El recurso no existe.");
    } else if (codigo == 403) {
        System.out.println("Prohibido. ¿Token necesario?");
    } else if (codigo >= 500) {
        System.out.println("Error del servidor. Vuelve a intentarlo.");
    }
} catch (HttpConnectTimeoutException e) {
    System.out.println("El servidor no responde (timeout de conexión).");
} catch (java.net.http.HttpTimeoutException e) {
    System.out.println("La respuesta tardó demasiado.");
} catch (java.io.IOException e) {
    System.out.println("Error de red: " + e.getMessage());
} catch (InterruptedException e) {
    System.out.println("La petición fue interrumpida.");
}
```

⚠️ **Advertència:** moltes APIs públiques (GitHub, Twitter, etc.) exigeixen un **token d'autenticació** en la capçalera `Authorization`. Sense ell, tens quotes molt baixes (rate

> limiting) o accés denegat. Si veus un `403`, probablement necessites registrar una aplicació i obtenir un token.

Les capçaleres més comunes que enviaràs:

Capçalera	Propòsit
`Content-Type`	Tipus de dades que envies (`application/json`)
`Accept`	Tipus de dades que esperes rebre
`Authorization`	Token Bearer, Basic Auth, etc.
`User-Agent`	Identifica la teva aplicació (moltes APIs ho exigeixen)
`Cache-Control`	Control de memòria cau

⚡ Crides asíncrones amb sendAsync

Si necessites diverses peticions sense bloquejar el programa, `HttpClient` també té mode asíncron:

```
CompletableFuture<HttpResponse<String>> futuro = client.sendAsync(
    request, HttpResponse.BodyHandlers.ofString()
);

// Mentrestant, pots fer altres coses...
System.out.println("Esperando respuesta...");

// Quan arribe:
futuro.thenAccept(response -> {
    System.out.println("Código: " + response.statusCode());
    System.out.println(response.body());
});

// O bloquejar fins que arribe (no recomanat si vas just de temps):
HttpResponse<String> resp = futuro.get();
```

> 💡 **Consell:** `sendAsync` torna un `CompletableFuture`, la forma elegant de Java per al codi asíncron sense liar-te amb fils manuals. Pots encadenar crides amb `thenApply`,

> `thenCompose`, etc. La diferència amb `send()`: el primer no espera (el `main` segueix) i el segon congela el programa fins a tenir la resposta.

🗑️ PUT, DELETE i altres mètodes

`HttpClient` suporta tots els mètodes HTTP. La diferència és només la crida al builder:

```
// PUT
HttpRequest putRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .header("Content-Type", "application/json")
    .PUT(HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Actualizado\"}"))
    .build();

// DELETE
HttpRequest deleteRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .DELETE()
    .build();

// Amb mètode personalitzat (PATCH, etc.)
HttpRequest patchRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .method("PATCH", HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Parcial\"}"))
    .build();
```

Fixa't: `PUT` porta cos (reemplaça amb contingut nou), `DELETE` no. I si necessites alguna cosa que no té mètode propi (com `PATCH`), `method(...)` t'ho dona.

🏗️ Frameworks moderns (una ullada)

El `HttpServer` natiu és perfecte per a aprendre, però en el món real s'usen frameworks:

- **Javalin** — minimalista, modern, ideal per a APIs. En 3 línies tens un servidor.
- **Spring Boot** — l'estàndard industrial. Ho té tot, però és més pesat.
- **Spark** — semblant a Javalin, molt lleuger.

Exemple amb Javalin:

```
import io.javalin.Javalin;

public class App {
    public static void main(String[] args) {
        Javalin app = Javalin.create().start(8080);
        app.get("/", ctx -> ctx.result("Hola desde Javalin"));
        app.get("/api/usuarios", ctx ->
            ctx.json(List.of(new Usuario(1, "Ana"))));
    }
}
```

> 💡 **Consell:** en aquest curs usem `HttpServer` natiu com a servidor i `HttpClient` per a consumir APIs: no necessites instal·lar res més que Gson. Els frameworks els veuràs en el mòdul de "Desenvolupament d'Interfícies" i, si t'hi llanços, en el teu projecte final. Ara entens el terreny que trepitgen: tots parlen HTTP.

🎯 Mini-chequeig

Posat a prova en 30 segons (les respostes estan amagades):

1. Quin mètode de `BodyPublishers` uses per a enviar un `String` en el cos d'un POST?
2. Quina capçalera és imprescindible en enviar JSON?
3. Quina diferència hi ha entre `send()` i `sendAsync()`?
4. Com fas un DELETE amb `HttpClient`?

> 🔄 **Respostes**
>
> 1. `HttpRequest.BodyPublishers.ofString(json)`.
> 2. `Content-Type: application/json`. Sense ella, el servidor pot rebutjar la petició (o tornar `415`).
> 3. `send()` és **síncron** (bloqueja el programa) i `sendAsync()` és **asíncron** (torna un `CompletableFuture` i el programa segueix).
> 4. Amb `.DELETE()` en el builder; no porta cos, només la URI.

✅ Resum en 3 frases

1. Per a **enviar dades** uses `POST` amb `BodyPublishers.ofString(json)` i la capçalera `Content-Type: application/json`.
2. Les APIs cauen: usa **timeouts** (`connectTimeout`, `timeout`), comprova el codi d'estat i captura les excepcions de xarxa amb elegància.
3. **`sendAsync()`** torna un `CompletableFuture` i no bloqueja el programa; `PUT`, `DELETE` i `PATCH` es fan canviant només la crida al builder.

> 🐛 **Vocabulari ràpid**
>
> | Terme | Idea general |
> |---|---|
> | `BodyPublishers` | Converteix el teu `String`/`byte[]` en el cos de la petició |
> | `Authorization` | Capçalera per a tokens i credencials |
> | Timeout | Temps màxim d'espera abans de rendir-se |
> | `CompletableFuture` | La promesa de Java: "arribarà, però no ara" |
> | `415` | Unsupported Media Type: has oblidat el `Content-Type` |

📚 Tornar a l'índex de la unitat · **Anterior:** 07 · Consumir APIs externes amb HttpClient · **Següent:** 09 · Repàs interactiu

09 · Repàs interactiu: domina la web

El tancament pràctic de la unitat i del curs, amb un parser que ho liarà tot i un servidor que no arranca 😈🌐

> 🗺️ **Estàs en: 🌐 U16 · Servir i consumir APIs amb Web** → 09 · Repàs interactiu

📬 La idea en una frase

> **En aquest punt no aprenem res de nou: ho convertim tot en pràctica. I, com sempre, alguna cosa no funcionarà. 😈**

⭐ Be the Code, my friend…

> *Ets la JVM. Acaben de donar-te aquest programa per a executar:*

```
public class Misterio {

    static String extraer(String query, String clave) {
        if (query == null) return "nada";
        for (String par : query.split("&")) {
            String[] trozos = par.split("=");
            if (trozos.length == 2 && trozos[0].equals(clave)) {
                return trozos[1];
            }
        }
        return "nada";
    }

    public static void main(String[] args) {
        System.out.println(extraer("nombre=Ana&edad=25", "edad"));
        System.out.println(extraer("nombre=Ana&edad=25", "ciudad"));
        System.out.println(extraer("clave=valor", "clave"));
    }
}
```

Què imprimeixes per pantalla? Tria saviament:

1. **`25 nada valor`** → ✅ Correcte! Per a la clau `edad` troba el parell `edad=25` i torna `"25"`. Per a `ciudad` no hi ha parell amb aquella clau → `"nada"`. Per a `clave` → `"valor"`.
2. **`nada 25 valor`** → Vas llegir l'ordre al revés: pensa quina clau es busca primer. ❌
3. **`25 25 valor`** → El segon `System.out` no busca "edad" dues vegades: busca `ciudad`, que no existeix. ❌

> 🔄 **Solució**
>
> L'opció **1**. El parser recorre els parells `clau=valor` separats per `&` i torna el valor de la clau demanada. `edad` → `25`; `ciudad` → no està → `nada`; `clave` → `valor`. És exactament l'`extraerParametro` del punt 4: la mateixa lògica que uses per a llegir la query string d'un GET.

🔥 Fireside Chat: GET vs POST

> *Dos cosins del protocol s'embasten davant la màquina de cafè.*

GET: — Jo soc el de tota la vida. Demano, em donen, no canvio res. Vols repetir la meva petició? Només enganxes la URL al navegador i ja està. Reproduïble, cacheable, simple.

POST: — I per a enviar el formulari de registre d'un usuari? Vas a ficar la seva contrasenya a la URL perquè quedi en l'historial i en els logs del servidor? Això és el que faries tu. Jo ho porto en el cos: privat, ordenat i sense límit de grandària pràctic.

GET: — Val, per a dades sensibles ets tu. Però jo soc el que carrega el 99 % de la web: cada pàgina, cada imatge, cada cerca passa per mi. Quantes vegades t'he vist en un formulari de cerca? Ah, que el cercador usa GET, perquè així el resultat es pot compartir i enllaçar. Interessant.

POST: — Tu per a llegir i compartir. Jo per a enviar i crear. Treva?

GET: — Treva. Però no m'enviïs un `DELETE` amb cos, que és de mal gust.

> La lliçó: **llegir, cachear, compartir? → GET. enviar dades, crear, alguna cosa sensible? → POST.** I recorda: un GET no ha de canviar l'estat del servidor; això és treball de POST, PUT i DELETE.

🕵️ Qui Soc?

Endevina quin concepte de la unitat soc:

1. **Soc l'idioma que parlen el client i el servidor, i el 200 és el meu saludet favorit.**
2. **Soc el servidor web que Java porta de sèrie; crée'm un port i un context i t'atendré.**
3. **Soc el format de dades que qualsevol llenguatge entén, amb claus i corxets.**
4. **Soc la capçalera que delata si la resposta és HTML, JSON o text pla.**
5. **Soc el client de Java 11+ que consulta APIs alienes amb el trio client-petició-resposta.**
6. **Soc la llibreria que converteix JSON en objectes Java, amb `fromJson` com a conjur.**

> 🔄 **Respostes**
>
> 1. **HTTP** — petició-resposta, i `200` és "tot bé".
> 2. **`HttpServer`** — `createContext` + `start()` i a servir.
> 3. **JSON** — objectes `{...}` i arrays `[...]`.
> 4. **`Content-Type`** — `text/html`, `application/json`…
> 5. **`HttpClient`** — `HttpClient`, `HttpRequest` i `HttpResponse`.
> 6. **Gson** — `gson.fromJson(cos, Classe.class)`.

🤬 CONRAD VS EL MÓN: "El port que no arranca"

> *CONRAD, el nostre compilador cascarrabutxes, opina sobre els clàssics del novell en la web.*

CONRAD: — UNA ALTRA VEGADA! Ve un alumne i em diu: *CONRAD, el meu servidor no arranca*. I jo: val, quin error? *Pues una cosa de "Address already in use".* AI, MARE MEVA! Tens una altra instància del servidor corrent des de la pràctica anterior. Tanca la pestanya de la terminal o canvia de port: `new InetSocketAddress(8081, 0)`. El port no és de la teva propietat vitalícia.

I després està el de l'HTML. Torna una pàgina i el navegador li mostra el codi en comptes de la web. Endevines què falta? El `Content-Type: text/html`. Sense la capçalera, el navegador fa el que pot, i el que pot és mostrar-te el codi com a text pla. La capçalera és l'uniforme: sense ella, ningú sap què ets.

I el colmo dels colmos: el que parseja el JSON a mà amb `split`. Li arriba un títol amb una cometa i Gson no té cap culpa: és que el `split("\"titulo\":\"")` s'ha descosit. Per què? Perquè existeix Gson per a això. Al servidor i al client: **per a parsejar JSON, deixa de fer servir tisores.** Un `split` és per a aprendre, no per a produir.

La lliçó: abans d'acusar Java de "odiar-te", repassa el trio sagrat de la unitat: **tinc la capçalera correcta? estic usant el mètode HTTP adequat? estic parsejant JSON amb una eina de veritat?** El compilador no t'odia: t'està passant les respostes de l'examen.

🎮 El Joc de les Decisions

Tria la resposta correcta per a cada decisió (respostes al final):

1. Un formulari d'inici de sessió envia la contrasenya. GET o POST?
   - a) GET b) POST
2. El teu endpoint torna una llista d'usuaris. Quin `Content-Type` poses?
   - a) `text/html` b) `application/json`
3. El client demana `/api/tareas/99` i aquella tasca no existeix. Quin codi tornes?
   - a) `200` b) `404`
4. En `HttpClient`, quin mètode envia dades en el cos de la petició?
   - a) `.GET()` b) `.POST(...)`
5. `gson.fromJson(cos, Usuario.class)` amb un camp que no està en el JSON… què rep aquell camp?
   - a) `null` b) `""` (cadena buida)

> 🔄 **Solucions**
>
> 1. **b)** — La contrasenya viatja en el cos del POST, no a la URL.
> 2. **b)** — `application/json`. Tornes dades, no una pàgina.
> 3. **b)** — `404`: el recurs demanat no existeix.
> 4. **b)** — `.POST(...)` porta el `BodyPublisher`; `.GET()` no envia cos.
> 5. **a)** — Gson deixa els camps absents en `null`. Per això es comproven amb `has(...)` / `isJsonNull()`.

⚡ Laboratori de Tortura: el servidor que no era

> **Duració estimada:** 30 minuts **Eina:** el teu IDE i un fitxer nou

L'escenari: copia aquest programa i fes que funcioni. Et diuen que té **3 errors** que impedeixen que compili i 1 error de lògica que fa que el JSON surti malament… però, i si t'ho diuen

malament? La teva tasca: fer que compili, que executi i que **tota** la sortida sigui correcta, comptant tu els errors reals.

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

public class ApiHora {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/api/hora", e -> {
            String json = "{\"hora\": \"" + java.time.LocalTime.now() + "\"}";
            e.getResponseHeaders().set("Content-Type", "application/json");
            e.sendResponseHeaders(200, json.getBytes().length);
            e.getResponseBody().write(json.getBytes())
            e.getResponseBody().close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080")
    }
}
```

Fallada intencionada: el nombre d'errors de l'enunciat està malament a propòsit: si busques el "tercer error de compilació" i "l'error de lògica", te'n tornes boig. La trampa és la teva confiança en l'enunciat.

La teva tasca: aconseguir que compili, que executi i que **tota** la sortida sigui correcta. Si el resultat no quadra, diagnostica.

Pistes per quan et frustres (no abans):

1. Falta algun ;? *no → segueix buscant.*

   > **I si continuec atascat?**
   > Repassa la línia `e.getResponseBody().write(json.getBytes())` i l'última del `main`: totes dues necessiten `;`.

2. Compila ja? *no → mira els tipus i els imports.*

   > **I si continuec atascat?**

> L'endpoint torna JSON i el `Content-Type` ja està. Mira una altra vegada els `;`.

3. Creus que hi ha un error de lògica que trenca el JSON? *Torna a llegir el codi amb calma i no et fies de l'enunciat.*

   > **Solució final**

Els **errors de compilació**:

1. Falta el `;` al final de `e.getResponseBody().write(json.getBytes())`.
2. Falta el `;` al final de `System.out.println(...)`.

L'**error de lògica**: no existeix. El JSON `{"hora": "09:30:05"}` es construeix bé i el `Content-Type` és correcte. Aquella era la fallada intencionada: l'enunciat diu "3 errors de compilació i 1 de lògica", però només hi ha 2 faltes de `;` i cap lògica trencada. L'error "amagat" era la teva confiança en l'enunciat. El servidor funciona en `http://localhost:8080/api/hora`.

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

public class ApiHora {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/api/hora", e -> {
            String json = "{\"hora\": \"" + java.time.LocalTime.now() + "\"}";
            e.getResponseHeaders().set("Content-Type", "application/json");
            e.sendResponseHeaders(200, json.getBytes().length);
            e.getResponseBody().write(json.getBytes());
            e.getResponseBody().close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

La lliçó real: **llegeix el codi, no l'enunciat**. Un bug pot ser que et diguin "hi ha 3 errors" i només n'hi hagi 2: si busques el tercer, et tornes boig. Compta, verifica i confia en el compilador, no en la llegenda.

🏆 Assoliments d'aquesta unitat

Assoliment	Com aconseguir-ho
🏅 **El Carter**	Explicar la diferència entre GET i POST amb un exemple de cadascun
🏅 **El Gran Cap del Port**	Muntar un `HttpServer` que no digui "Address already in use" i serveixi dues rutes
🏅 **El Sommelier de Content-Type**	Tornar HTML, JSON o text pla amb la capçalera correcta en cada cas
🏅 **El Client Elegant**	Consumir una API externa amb `HttpClient` i parsejar el JSON amb Gson
🏅 **El Mestre de l'API**	Construir un CRUD REST complet (GET, POST, PUT, DELETE) en memòria

🧠 Atreveix-te a Pensar

1. **Sense executar:** què imprimeix aquest programa?

```
public class Acertijo {
    static String url = "/api/tareas/3";

    public static void main(String[] args) {
        String id = url.substring("/api/tareas/".length());
        System.out.println(id);
        System.out.println(url.split("/").length);
    }
}
```

2. **El servidor que espia:** quina capçalera poses perquè el navegador no faci memòria cau de les teves respostes JSON i sempre rebi dades fresques?
3. **El detectiu del 403:** consumes una API sense autenticar-te i et torna `403`. Quina capçalera probablement et falta? I què has de fer per a aconseguir-la?

4. **Vertader o fals:** “un `GET` pot enviar dades en el cos de la petició”.

> 💡 **Solucions**
>
> 1. Imprimeix `3` i després `4`. `substring` extrau l’ID de la ruta; `url.split("/")` dona `["", "api", "tareas", "3"]`, que són 4 trossos (el primer és buit perquè la URL comença per `/`).
> 2. `Cache-Control: no-cache` (o `no-store`). Així el navegador no serveix la resposta de la memòria cau i torna a preguntar al servidor.
> 3. La capçalera **`Authorization`** amb un token. Per a aconseguir-lo, normalment registres una aplicació en la plataforma i obtens un token d’accés.
> 4. **Fals en la pràctica.** L’estàndard HTTP no ho prohibeix del tot, però cap servidor decent ho respecta i `HttpClient` no t’ho posa fàcil: per a enviar dades usa POST, PUT o PATCH, que és el que el món espera.

🧩 Crucigrama de Bits

```
Horizontal:
1. Format de dades universal de la web, amb claus i corxets (4 lletres)
4. Servidor HTTP que Java porta de sèrie (11 lletres)
6. Codi d'estat per a "no trobat" (3 lletres)
7. Capçalera que diu el tipus de contingut (12 lletres)

Vertical:
2. Mètode HTTP per a enviar dades i crear recursos (4 lletres)
3. Client HTTP modern de Java 11+ (11 lletres)
5. Format de contingut d'una pàgina web (4 lletres)
```

> 📝 **Solucions**
>
> **Horizontal:** 1. JSON · 4. HTTPSERVER · 6. CERO CUATRO CUATRO (404) · 7. CONTENTTYPE
> **Vertical:** 2. POST · 3. HTTPCLIENT · 5. HTML

💬 Preguntes d’Entrevista de Treball

> Preguntes reals que et farien per a programador Java junior.

1. **“Explica’m, com si jo fos la teva àvia, què és una API REST.”**
2. **“Quina és la diferència entre GET i POST? Quan usaries cadascun?”**
3. **“Què significa el codi 404? I el 500?”**
4. **“Com consumes una API externa en Java sense llibreries de tercers?”**
5. **“Què és Gson i per a què serveix? Què passa si el JSON porta un camp que no està en el teu `record`?”**
6. **“Què és un `Content-Type` i per què és important?”**

🤷 No hi ha preguntes tontes

> ❓ **HTTP i HTTPS són el mateix?**

Quasi. HTTPS és HTTP amb una capa de xifrat (SSL/TLS): les dades viatgen encriptades. Per a producció, sempre HTTPS; per a desenvolupament local, HTTP val.

> ❓ **Què és CORS i per què em dona error?**

Cross-Origin Resource Sharing. El navegador bloqueja peticions d’un domini a un altre per seguretat. Per a desenvolupament, afegeix en el teu servidor: `intercambio.getResponseHeaders().add("Access-Control-Allow-Origin", "*")`.

> ❓ **El `HttpServer` és com Tomcat?**

No. `HttpServer` és mínim i didàctic: serveix per a entendre el protocol. Tomcat és un servidor d’aplicacions complet (Servlets, JSP…). Aquí usem just el necessari perquè la web deixi de ser màgia.

🎬 Post-Crèdits

La programadora acaba la seva primera API: un gestor de tasques servit amb `HttpServer`, amb els seus quatre verbs REST i un frontend que fa `fetch`. Funciona. Fins que obre el mòbil i vol consumir-la des de fora de l’ordinador, i descobreix que el localhost només viu en la seva màquina.

S’acosta CONRAD, el compilador cascarrabutxes, amb la seva tassa fumant.

CONRAD: — Això és el bonic del que acabes de construir: l'API ja no depèn d'on s'executa. Avui viu en el teu portàtil; demà, en un núvol. Els que la consumeixen només veuen rutes i JSON.

Programadora: — Val… però ara mateix tot està en un `ArrayList` que s'esborra en reiniciar. Quan aprenc a guardar-lo de veritat?

CONRAD: — *assenteix* Això ja ho vas veure a la U15 amb JDBC: canvia el magatzem, no les rutes. La teva API no se n'assabenta. Però si em preguntes a mi, ja has recorregut un camí enorme: vas començar amb un `Hola, mundo` en una consola i acabes servint i consumint web. Això és tot el curs.

Programadora: — I ara què?

CONRAD: — *s'ajusta les ulleres* Ara tria. Spring Boot, el teu projecte final, la teva primera entrevista… El curs s'ha acabat, però la programació no: només canvia de terminal.

🎓 **FI DEL CURS.** De `System.out.println("Hola, mundo")` a servir i consumir APIs en 14 unitats. Ha estat un plaer programar amb tu. Segueix escrivint codi! 🌐

📚 Tornar a l'índex de la unitat · **Anterior:** 08 · HttpClient avançat · 🎓 **Fi del curs**

Butlletí U01 — Inicial

📝 Inicial per resoldre

> Entrena la teva ment de programador/a (sense una sola línia de Java). No tenen nota ni compten per a l'avaluació, però són la millor manera de trencar el gel i compartir idees al fòrum.

Exercici 1: "Enganya al teu robot literal" 🤖🥪

- **El concepte:** Un ordinador no té sentit comú; fa exactament el que li dius, no el que tenies al cap.
- **Com participar:**
  1. Tria una acció quotidiana (ex: *fer un cafè de càpsula*, *posar-se un abric*, *Demanar una pizza per una app*).
  2. Escriu al fòrum les instruccions pas a pas (màxim 8 passos) per a un robot.
  3. **Mode gamberro:** Llegeix la proposta d'un/a company/a i fes de *Robot Maliciós*: troba un pas on falti informació implícita i indica la pifiada literal que faria el robot!

Exercici 2: "Caçadors de bugs humans" 🐛🎯

- **El concepte:** Els programadors professionals proven els "casos límit" per a trobar falles abans que el programa pugui petar.
- **L'algorisme a analitzar:** Un company ha dissenyat aquest sistema per a **"Calcular si una persona pot pujar a una atracció (Alçada mínima: 140 cm)"**:

1. Demanar l'alçada en centímetres.
2. Si l'alçada és major que 140:
3. ```
   Mostrar: "Pots pujar!"
   ```
4. Si no:
5. ```
   Mostrar: "Ets massa baixet/a."
   ```

- **Com participar:** Respon al fòrum contestant a aquestes preguntes:

- Quin cas límit o dada “gamberra” faria fallar aquest algorisme? *(Pista: Què passa si mesures 140 cm exactes? I si penses un nombre negatiu?).*
- Com ho arreglaries tu en una sola frase de text normal?

Exercici 3: “El teu protocol de pau” 🛡️💬 🔗

- **El concepte:** A distància, el perill més gran és el bloqueig i la frustració quan un exercici no surt a la primera.
- **Com participar:** Comparteix en un missatge breu les teves respostes a aquestes dues preguntes:
  1. *Quin és el concepte d’aquesta Unitat 1 que més t’ha sorprés o canviat la xip?*
  2. *Quins seran els teus 3 passos personals quan te n’atures en un exercici abans de demanar ajuda?*

💡 **Recorda:** No hi ha respostes “dolentes” ni formes d’equivocar-se. El fòrum està obert per a provar, riure’s dels errors de pensament literal i veure que tots i totes estem en el mateix vaixell.

Exercici 4: El filtre de l’hospital vs. el gimnàs (Abstracció) 🔗

- **El concepte:** Un ordinador només ha de guardar les dades que necessita. Guardar informació inútil només ocupa espai i crea problemes.
- **L’escenari:** Imagina que has de dissenyar la fitxa d’un usuari per a dues aplicacions diferents: l’aplicació d’un **Hospital** i l’aplicació d’un **Gimnàs**.
- **Com participar:** Classifica les següents dades segons si són **ÚTILS** o **IRRELEVANTS** per a cada aplicació i comparteix-ho al fòrum: ***Dades**: Nom, Grup sanguini, Cançó preferida, Al·lèrgies a medicaments, Número de sabata, Objectiu de pes, Historial de fractures.*

Exercici 5: El desglossador de promocions (Descomposició) 🔗

- **El concepte:** Si un problema sembla molt gran o embolcallat, cal dividir-lo en peces microscòpiques.

- **L'escenari:** Una botiga en línia vol aplicar la següent oferta: *"Si compres 3 productes o més, se t'aplica un 10% de descompte. Si a més ets client VIP, se t'aplica un 5% addicional."*
- **Com participar:** Escriu en el fòrum la llista de preguntes (en ordre) que hauria de fer el programa per a saber quina millora aplicar. *(**Pista**: 1. Quants productes porta? 2. És VIP?...)*

Exercici 6: El test de la recepta tapada (Metacognició)

- **El concepte:** Comprovar per tu mateix/a la diferència entre "veure com es fa" i "saber fer-ho tu tot sol".
- **Com participar:**
  1. Llegeix les instruccions de com fer un nus de corbata, una figura de papiroflèxia senzilla o una recepta de cuina ràpida.
  2. Tanca la pàgina.
  3. Intenta escriure en un paper tots els passos de memòria sense mirar.
  4. Comparteix al fòrum: *En quin pas t'has encallat en tancar la mostrada? Què has après d'aquesta experiència per a quan estudies Java?*

Exercici 7: El Patró del Caixer i el pàrquing (Patrons)

- **El concepte:** En programació no es reinventa la roda: es busquen coses que funcionen igual per a usar la mateixa lògica.
- **L'escenari:** Pensa en com funciona un **caixer automàtic quan treus diners** i com funciona la **barrera d'un pàrquing quan pagues el tiquet**.
- **Com participar:** Escriu al fòrum quina és la condició que tenen en comú aquestes dues màquines per a deixar-te sortir o donar-te els diners. *(Pista: Totes dues comproven si "l'import introduït és major o igual a...").*

Exercici 8: El meu manual per a aprendre a programar

Després d'haver treballat els conceptes clau de la Unitat 1, no volem que te'n vagis a la Unitat 2 sense un pla d'acció personalitzat.

Aquesta tasca es divideix en dues parts molt simples:

1. Part 1: El teu "Full de Ruta" personal per a gestionar el bloqueig, els errors i la IA durant el curs.
2. Part 2: Un mini-repte pràctic de pensament lògic per a posar a prova la teva capacitat de descomposició i testing (sense tocar ordinador).

📝 PART 1: El meu manual de navegació (Reflexió)

Completa les següents 5 frases amb la teva reflexió personal. Siguis sincer/a: no hi ha respostes "correctes" de llibre, sinó el teu compromís real sobre com vas a estudiar a distància.

1. Davant de la pantalla en blanc...

Quan em quedi bloquejat/ada davant d'un exercici nou, el primer que faré abans de tocar el teclat serà...

(Pista: Pensa en la regla del paper, la descomposició o el dibuix de la lògica).

2. La meva relació amb la solució dels exercicis...

Quan reviso un exemple de codi resolt als apunts o en un vídeo, per a assegurar-me que no he caigut en el "miratge de la comprensió", jo...

(**Pista**: Recorda la regla de la pantalla tapada).

3. El meu protocol davant del "text vermell" (Errors)...

Si el meu programa em dona un error en executar-se, seguiré aquests 3 passos en ordre abans de demanar ajuda al fòrum:

- Pas 1:
- Pas 2:
- Pas 3:

4. El meu compromís amb la intel·ligència artificial...

M'hagi trobat o no amb la IA (ChatGPT, Gemini, etc.), em compromet a utilitzar-la principalment per a... i MAI per a...

(**Pista**: Recorda l'us de la IA com a tutor socràtic vs. esclau de copy-paste).

5. La meva troballa sobre aprendre a programar...

Una cosa important que he descobert en aquesta Unitat 1 sobre com aprèn el meu cervell a programar és...

🧩 PART 2: Mini-Repte de Lògica (Sense Java)

Llegeix atentament l'enunciat d'aquest problema del món real i resol els 3 apartats següents utilitzant el teu pensament lògic.

📌 Enunciat del Mini-Repte:

"Un cine d'estiu vol automatitzar la venda d'entrades. El preu base de l'entrada és de 8 €. Tanmateix, si el client té menys de 12 anys o més de 65 anys, se li aplica un descompte del 50%. A més, si és dimarts (el dia de l'espectador), TOTHOM té un descompte directe de 2 € sobre el preu que li corresponga."

✂️ 1. Descompondre el problema

Abans de pensar en la solució, escriu quines dades d'entrada (Inputs) necessita demanar el programa a l'usuari per a poder fer els càlculs.

👣 2. Escriure l'algorisme pas a pas

Escriu en el teu propi idioma (pseudocodi) la seqüència d'instruccions ordenades que hauria de seguir el programa per a calcular el preu final de l'entrada.

(**Pista**: Pensa en quin ordre s'han de comprovar l'edat i el dia de la setmana).

🧪 3. Control de qualitat (Provar casos límit)

Un bon/a programador/a comprova si el programa falla en els punts crítics. Indica quin hauria de ser el preu final de l'entrada en aquests 4 casos de prova:

- Cas A: Un xiquet de 10 anys un dijous ⟶ Preu final: ____ €
- Cas B: Un adult de 30 anys un dimarts ⟶ Preu final: ____ €

- Cas C (Límit 1): Una persona de 12 anys exactes un dimecres ⟶ Preu final: ____ (Atenció: Té descompte del 50% o no? Revisa bé l'enunciat!)

- Cas D (Límit 2): Un avi de 70 anys un dimarts ⟶ Preu final: ____ (Atenció: Se li apliquen tots dos descomptes? Quin és el resultat?)

Pots compartir les teves solucións al forum 😀

Butlletí UO2 — Inicial

📝 Butlletí UO2 — Inicial

> Sense solucions. Sense presses. Amb un editor de text i moltes ganes de compilar. Això només comença. Comenta les solucions al fòrum i intenta ajudar als companys a què milloren les seves capacitats com a programador.

Exercici 1: Desordena això

Les línies d'aquest programa estan desordenades. Ordena-les per a formar un programa Java vàlid que compili i s'executi:

```
}
    public static void main(String[] args) {
        System.out.println("Mi primer programa ordenado");
public class Ordenado {
    }
```

Exercici 2: Què imprimeix?

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class Escapista {
    public static void main(String[] args) {
        System.out.print("Dijo: \"Java mola\"");
        System.out.println(" y siguió: \tprogramando.");
    }
}
```

💡 **Pista important**: `\"` imprimeix una cometa doble literal, `\t` és un tabulador.

Exercici 3: Caçador d'errors

Aquest codi té **4 errors**. Troba'ls i corregeix-los:

```
Public class ErrorFinder {
    public static void main(string[] args) {
        System.out.println("Hola, "Mundo"");
        System.out.println("Esto funciona?)
    }
}
```

Exercici 4: La teva fitxa personal

Escriu un programa anomenat `FichaPersonal.java` que mostri el següent text per pantalla:

```
Nombre: [El teu nom]
Edad: [La teua edat]
Lenguaje favorito: Java
¿Emocionado?: true
```

Usa una línia `System.out.println` per a cada camp i assegura't que els booleans (com `true`) no porten cometes.

Exercici 5: Completa el programa

Falta una clau de tancament i un caràcter crucial. Afegeix-los perquè compili i mostri "Aprobado, esto funciona":

```
public class Completame {
    public static void main(String[] args) {
        System.out.println("Aprobado, esto funciona")
}
```

Exercici 6: Emparella conceptes

Relaciona cada concepte de l'esquerra amb la seva definició de la dreta:

Concepte	Definició
1. `class`	A. Punt d'entrada del programa
2. `main`	B. Imprimeix text i salta de línia
3. `System.out.println`	C. Defineix un nou tipus de dades o plantilla de codi
4. `//`	D. Comentari d'una línia
5. `args`	E. Conté els arguments de la línia d'ordres

Escriu les respostes amb el format: 1→C, 2→A, ...

Exercici 7: El detectiu d'errors

El següent codi té un error de sintaxi que impedeix que compili. Troba'l i corregeix-lo:

```
public class Detective {
    public static void main(String[] args) {
        System.out.println("Soy un detective")
        System.out.println("y resuelvo errores");
    }
}
```

Escriu la versió corregida. Després, executa-la en VS Code i comprova que funciona.

Exercici 8: La teva biografia

Escriu un programa anomenat `Biografia.java` que mostri:

- El teu nom
- La teva edat
- El teu llenguatge de programació favorit
- Una frase que et motive

Cada dada ha de sortir en una línia diferent, però amb una única condició: usa una sola instrucció `System.out.println` utilitzant `\n` per als salts de línia.

Butlletí UO2 — Inicial Resolt

📝 Butlletí UO2 — Inicial (Resolt)

> Les solucions estan ocultes. Intenta-ho de veritat abans de destapar-les. Compara el teu codi amb aquestes propostes de resolució i comparteix els teus dubtes o alternatives en el fòrum.

Exercici 1: Desordena això (Solució)

> 🔄 **Solució**
>
> **Codi ordenat i corregit:**
>
> ```
> public class Ordenado {
>     public static void main(String[] args) {
>         System.out.println("Mi primer programa ordenado");
>     }
> }
> ```

Exercici 2: Què imprimeix? (Solució)

> 🔄 **Solució**
>
> **Sortida exacta per pantalla:**
>
> ```
> Dijo: "Java mola" y siguió:   programando.
> ```
>
> **Explicació**: `System.out.print` no afegeix salt de línia al final, per la qual cosa la segona instrucció s'imprimeix immediatament a continuació. La seqüència `\"` permet escriure la cometa doble sense tancar la cadena, i `\t` afegeix l'espaiat del tabulador.

Exercici 3: Caçador d'errors (Solució)

> 🔄 **Solució**

Errors trobats:

1. `Public` va en minúscula (`public`).
2. `string` va amb la primera lletra en majúscula (`String`).
3. En `"Hola, "Mundo""` cal escapar les cometes internes (`"Hola, \"Mundo\""`)
4. En `"Esto funciona?)` falta la cometa doble de tancament (`"Esto funciona?"`).

Codi corregit:

```
public class ErrorFinder {
    public static void main(String[] args) {
        System.out.println("Hola, \"Mundo\"");
        System.out.println("Esto funciona?");
    }
}
```

Exercici 4: La teva fitxa personal (Solució)

› 🔄 Solució

Codi de resolució:

```
public class FichaPersonal {
    public static void main(String[] args) {
        System.out.println("Nombre: Sergi");
        System.out.println("Edad: 25");
        System.out.println("Lenguaje favorito: Java");
        System.out.println("¿Emocionado?: " + true);
    }
}
```

Exercici 5: Completa el programa (Solució)

› 🔄 Solució

Elements afegits:

- El punt i coma `;` al final de la instrucció `System.out.println`.

- La clau de tancament } de la classe `Completame`.

Codi corregit:

```
public class Completame {
    public static void main(String[] args) {
        System.out.println("Aprobado, esto funciona");
    }
}
```

Exercici 6: Emparella conceptes (Solució)

> 🔄 **Solució**

Resolució: 1→C, 2→A, 3→B, 4→D, 5→E

Exercici 7: El detectiu d'errors (Solució)

> 🔄 **Solució**

Error trobat: Falta el punt i coma ; al final de la primera instrucció `System.out.println("Soy un detective")`.

Codi corregit:

```
public class Detective {
    public static void main(String[] args) {
        System.out.println("Soy un detective");
        System.out.println("y resuelvo errores");
    }
}
```

Exercici 8: La teva biografia (Solució)

> 🔄 **Solució**

Codi de resolució (amb una sola instrucció i `\n`):

```
public class Biografia {
    public static void main(String[] args) {
              System.out.println("Nombre:  Sergi\nEdad:  25\nLenguaje  favorito:
Java\nFrase: El código limpio habla por sí solo.");
    }
}
```

Butlletí UO2 — Avançat

📝 Butlletí UO2 — Avançat

> Sense solucions. Sense presses. Amb un editor de text i moltes ganes de compilar. Això només comença. Comenta les solucions al fòrum i intenta ajudar als companys a què milloren les seves capacitats com a programador.

Exercici 1: Calculadora de propines

Escriu un programa que calculi quant deixar de propina en un restaurant. Declara:

- `double totalCompte = 45.50;`
- `int percentatgePropina = 15;` (el percentatge, sense el símbol)

Calcula la propina (`totalCompte * percentatgePropina / 100`) i el total final (`totalCompte + propina`). Mostra els tres valors.

Pista: compte amb la divisió: `percentatgePropina / 100` amb enters dona 0. Multiplica primer i divideix després, o usa `100.0`.

Exercici 2: Conversor dòlar-euro

Declara `final double TAXA_CANVI = 0.92;` (1 dòlar = 0.92 euros). Declara `double dolars = 100.0;` i calcula el seu equivalent en euros. També fes la conversió inversa: donat `double euros = 50.0;`, calcula quants dòlars són.

Mostra:

```
100.0$ són 92.0€
50.0€ són 54.347826086956516$
```

Pista: per a passar d'euros a dòlars divideixes entre la taxa: `euros / TAXA_CANVI`.

Exercici 3: Què imprimeix? — el casting traïdor

Sense executar, escriu la sortida exacta:

```
public class CastingTraidor {
   public static void main(String[] args) {
       int a = 7;
       int b = 2;
       double resultat1 = a / b;
       double resultat2 = (double) a / b;
       double resultat3 = a / (double) b;

       System.out.println(resultat1);
       System.out.println(resultat2);
       System.out.println(resultat3);
       System.out.println(3 + 4 * 2.0);
       System.out.println((int) (3.7 + 2.3));
   }
}
```

Pista: fixa't bé en on està el càsting i en quin moment s'aplica la divisió entera. En `resultat1`, ¿la divisió es fa abans o després de guardar en el `double`?

Exercici 4: Interès compost (sense bucle)

Declara `final double CAPITAL_INICIAL = 1000.0;`, `final double TAXA = 0.05;` (5% anual), `int anys = 3;`. Calcula el capital final després de 3 anys usant la fórmula de l'interès compost SENSE bucles:

```
capitalFinal = capitalInicial * (1 + taxa)^anys
```

Per a la potència usa `Math.pow(base, exponent)`. Mostra el capital any a any:

```
Any 0: 1000.0€
Any 1: 1050.0€
Any 2: 1102.5€
Any 3: 1157.625€
```

💡 **Compte amb la precisió:** en l'any 3 el teu programa pot imprimir `1157.6250000000002` en lloc de `1157.625` per la coma flotant binària dels `double` (ho veurem a «Atreveix-te a pensar»).

No és un error del teu codi.

Pista: per a mostrar cada any sense bucle, crea tres variables distintes (`capital1`, `capital2`, `capital3`) i usa `Math.pow(1 + TAXA, i)` amb `i` valent 1, 2 i 3.

Exercici 5: L'enigma del post-increment

Sense executar, determina el valor de cada variable després d'executar aquest codi. Escriu el pas a pas:

```
public class EnigmaIncrement {
   public static void main(String[] args) {
   int x = 3;
   int y = x++ + ++x;
   int z = --y + y-- + x++;
   System.out.println("x = " + x);
   System.out.println("y = " + y);
   System.out.println("z = " + z);
   }
}
```

Pista: fes una taula en un full amb els valors de `x` i `y` després de cada operació. `x++` usa i després incrementa; `++x` incrementa i després usa.

💡 Eines d'ampliació (Operador ternari, .equals(), Scanner i Math.random())

⚠️ **Fora de temari (de moment):** Aquestes eines les veurem en detall i de manera oficial en pròximes unitats, però és útil saber com funcionen si vols començar a explorar opcions més avançades en els teus programes.

1. L'operador ternari (? :)

És una forma ultracompacta d'escriure una estructura condicional (if-else) senzilla en una sola línia. Serveix per a triar entre dos valors segons si es compleix o no una condició.

Sintaxi: `condicio ? valor_si_cert : valor_si_fals`

Exemple:

```
int edat = 20;
String estat = (edat >= 18) ? "Major d'edat" : "Menor d'edat";
```

Si la condició (`edat >= 18`) és certa, retorna “Major d’edat”; si no, retorna “Menor d’edat”.

2. Comparar text amb .equals()

En Java, les cadenes de text (`String`) són objectes, no tipus primitius com `int` o `double`. Per això, mai s’ha d’utilitzar `==` per a comparar el contingut de dos textos, ja que `==` compara si estan guardats en la mateixa posició de memòria, no si tenen les mateixes lletres.

Sintaxi: `cadena1.equals(cadena2)`

Exemple:

```
String operador = "+";

// Incorrecte ❌ (pot fallar segons com s'haja creat el text):
// if (operador == "+")

// Correcte ✅ (compara el contingut del text lletra per lletra):
if (operador.equals("+")) {
    System.out.println("És una suma");
}
```

3. Llegir dades del teclat amb Scanner i la importació (import)

Per a llegir el que l’usuari escriu en la terminal, utilitzem la classe `Scanner`. Com que no ve carregada per defecte en el motor bàsic de Java, hem de dir-li al programa on trobar-la mitjançant una instrucció `import` en la primera línia de l’arxiu.

Importació obligatòria: `import java.util.Scanner;` (es posa abans de declarar la classe).

Mètodes principals de lectura:

- `tecleat.nextLine()`: Llegeix una línia de text completa (`String`).
- `tecleat.nextInt()`: Llegeix un nombre enter (`int`).

- `tecleat.nextDouble()`: Llegeix un nombre decimal (`double`).

Exemple:

```
import java.util.Scanner; // 1. Importem la classe de la llibreria de Java

public class ExempleScanner {
    public static void main(String[] args) {
        Scanner tecleat = new Scanner(System.in); // 2. Creem l'objecte Scanner

        System.out.print("Introdueix el teu nom: ");
        String nom = tecleat.nextLine(); // 3. Llegim el text introduït

        System.out.println("Hola, " + nom + "!");
        tecleat.close(); // Bons hàbits: tanquem el Scanner en acabar
    }
}
```

4. Generar nombres aleatoris amb Math.random()

La classe `Math` ve inclosa automàticament en Java (no necessita `import`). El seu mètode `Math.random()` genera un nombre decimal aleatori entre 0.0 (inclòs) i 1.0 (exclòs).

Com adaptar-lo per a obtenir un nombre enter en un rang (ex: un dau d'1 a 6):

- Multipliquem el resultat pel màxim del rang (`Math.random() * 6` dona de 0.0 a 5.999...).
- Fem un casting a enter (`int`) per a eliminar la part decimal (0 a 5).
- Sumem 1 per a ajustar el rang final (1 a 6).

Exemple per a simulació de daus:

```
// Genera un nombre enter aleatori entre 1 i 6:
int dau = (int) (Math.random() * 6) + 1;
System.out.println("Has tret un: " + dau);
```

Exercici 6: El duel de daus (Extra)

Escriu un programa que llanci un dau per a l'usuari (li demana el nom amb `Scanner`) i un altre per a la màquina (amb `Math.random()`). Guanya qui traga més. Mostra les dues tirades i el

guanyador amb un missatge tipus:

```
Anna en trau 4, la màquina en trau 6.
Guanya la màquina.
```

Pista: combina `nextLine()` (nom), `(int)(Math.random() * 6) + 1` (daus) i un ternari encadenat per al veredicte. Compte amb l'empat.

Exercici 7: Supercalculadora amb lògica (Extra)

Escriu un programa que llegeixi dos nombres amb `Scanner` (`double`) i un caràcter d'operació (`+`, `-`, `*`, `/`) amb `next()`.

Segons l'operador, mostra el resultat. Si l'operador és `/` i el segon nombre és 0, mostra "Error: no es pot dividir entre 0". Per a qualsevol altre caràcter, mostra "Operador desconegut".

Pista: calcula els resultats possibles en variables i tria amb ternaris encadenats o amb lògica. Pots comprovar la divisió entre 0 amb `op.equals("/") && b == 0`.

Exercici 8: Javadoc de campionat

Escriu una classe anomenada `SobreMi.java` que mostri per pantalla el teu nom, la teva motivació per a programar i quants arguments ha rebut per la línia d'ordres (`args.length`).

El programa ha d'incloure els tres tipus de comentaris de Java:

- Comentari d'una línia (`//`): Explica per què s'imprimeix primer el nom.
- Comentari de bloc (`/* ... */`): Un resum breu de la funció del programa.
- Comentaris Javadoc (`/** ... */`): Documentació formal per a la classe i per al mètode main, utilitzant les etiquetes `@author`, `@version` i `@param`.

Tasca extra: Executa la comanda `javadoc SobreMi.java` en la terminal (o des de les opcions de Java en VS Code) per a generar la documentació en HTML i obre l'arxiu `index.html` en el teu navegador.

Butlletí UO2 — Avançat Resolt

📝 Butlletí UO2 — Avançat (Resolt)

> Les solucions estan ocultes. Intenta-ho de veritat abans de destapar-les. Compara el teu codi amb aquestes propostes de resolució i comparteix els teus dubtes o alternatives en el fòrum.

Exercici 1: Calculadora de propines (Solució)

> 🔄 Solució

```
public class CalculadoraPropines {
    public static void main(String[] args) {
        double totalCompte = 45.50;
        int percentatgePropina = 15;

        // Multipliquem primer per a evitar la divisió entera entre enters (15 / 100 = 0)
        double propina = totalCompte * percentatgePropina / 100.0;
        double totalFinal = totalCompte + propina;

        System.out.println("Total del compte: " + totalCompte + "€");
        System.out.println("Propina (" + percentatgePropina + "%): " + propina + "€");
        System.out.println("Total a pagar: " + totalFinal + "€");
    }
}
```

Exercici 2: Conversor dòlar-euro (Solució)

> 🔄 Solució

```java
public class ConversorMoneda {
    public static void main(String[] args) {
        final double TAXA_CANVI = 0.92;

        double dolars = 100.0;
        double eurosDesDeDolars = dolars * TAXA_CANVI;

        double euros = 50.0;
        double dolarsDesDeEuros = euros / TAXA_CANVI;

        System.out.println(dolars + "$ són " + eurosDesDeDolars + "€");
        System.out.println(euros + "€ són " + dolarsDesDeEuros + "$");
    }
}
```

Exercici 3: Què imprimeix? — el casting traïdor (Solució)

› 🔄 Solució

Sortida exacta per pantalla:

```
3.0
3.5
3.5
11.0
6
```

Explicació pas a pas:

- **resultat1 (3.0)**: `a / b` és una divisió entre dos enters (7 / 2). El resultat és la divisió entera 3. En assignar-ho a un `double`, es converteix en 3.0.
- **resultat2 (3.5)**: `(double) a` converteix en 7.0. La divisió 7.0 / 2 és entre `double` i `int`, per la qual cosa `b` es promou a `double` i s'obté la divisió decimal exacta 3.5.
- **resultat3 (3.5)**: Ocorre el mateix: `(double) b` converteix `b` en 2.0, donant la divisió decimal 7 / 2.0 = 3.5.
- **3 + 4 * 2.0 (11.0)**: La multiplicació té prioritat (4 * 2.0 = 8.0). Després es suma 3 + 8.0 = 11.0 (promogut a `double`).

- **(int) (3.7 + 2.3) (6)**: Primer es resol el parèntesi 3.7 + 2.3 = 6.0. El casting `(int)` trunca la part decimal i queda l'enter 6.

Exercici 4: Interès compost (sense bucle) (Solució)

> 🔄 Solució

```
public class InteresCompost {
    public static void main(String[] args) {
        final double CAPITAL_INICIAL = 1000.0;
        final double TAXA = 0.05;

        double capital0 = CAPITAL_INICIAL;
        double capital1 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 1);
        double capital2 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 2);
        double capital3 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 3);

        System.out.println("Any 0: " + capital0 + "€");
        System.out.println("Any 1: " + capital1 + "€");
        System.out.println("Any 2: " + capital2 + "€");
        System.out.println("Any 3: " + capital3 + "€");
    }
}
```

Exercici 5: L'enigma del post-increment (Solució)

> 🔄 Solució

Pas a pas de l'execució:

1. **`int x = 3;`** → x=3.
2. **`int y = x++ + ++x;`**:
   - x++: Usa el valor actual de x (3) i després l'incrementa a 4.
   - ++x: Incrementa x primer (4→5) i usa el nou valor (5).
   - y=3+5=8. Valors actuals: x=5, y=8.
3. **`int z = --y + y-- + x++;`**:
   - `--y`: Decrementa y primer (8→7) i usa el nou valor (7).

- `y--`: Usa el valor actual de y (7) i després el decrementa a 6.
- `x++`: Usa el valor actual de x (5) i després l'incrementa a 6.
- z=7+7+5=19. Valors finals: x=6, y=6, z=19.

Sortida per pantalla:

```
x = 6
y = 6
z = 19
```

💡 Eines d'ampliació (Operador ternari, .equals(), Scanner i Math.random())

⚠️ **Fora de temari (de moment):** Aquestes eines les veurem en detall i de manera oficial en pròximes unitats, però és útil saber com funcionen si vols començar a explorar opcions més avançades en els teus programes.

1. L'operador ternari (? :)

És una forma ultracompacta d'escriure una estructura condicional (if-else) senzilla en una sola línia. Serveix per a triar entre dos valors segons si es compleix o no una condició.

Sintaxi: `condicio ? valor_si_cert : valor_si_fals`

Exemple:

```
int edat = 20;
String estat = (edat >= 18) ? "Major d'edat" : "Menor d'edat";
```

Si la condició (`edat >= 18`) és certa, retorna "Major d'edat"; si no, retorna "Menor d'edat".

2. Comparar text amb .equals()

En Java, les cadenes de text (`String`) són objectes, no tipus primitius com `int` o `double`. Per això, mai s'ha d'utilitzar `==` per a comparar el contingut de dos textos, ja que `==` compara si estan guardats en la mateixa posició de memòria, no si tenen les mateixes lletres.

Sintaxi: `cadena1.equals(cadena2)`

Exemple:

```
String operador = "+";

// Incorrecte ❌ (pot fallar segons com s'haja creat el text):
// if (operador == "+")

// Correcte ✅ (compara el contingut del text lletra per lletra):
if (operador.equals("+")) {
    System.out.println("És una suma");
}
```

3. Llegir dades del teclat amb Scanner i la importació (import)

Per a llegir el que l'usuari escriu en la terminal, utilitzem la classe `Scanner`. Com que no ve carregada per defecte en el motor bàsic de Java, hem de dir-li al programa on trobar-la mitjançant una instrucció `import` en la primera línia de l'arxiu.

Importació obligatòria: `import java.util.Scanner;` (es posa abans de declarar la classe).

Mètodes principals de lectura:

- `tecleat.nextLine()`: Llegeix una línia de text completa (`String`).
- `tecleat.nextInt()`: Llegeix un nombre enter (`int`).
- `tecleat.nextDouble()`: Llegeix un nombre decimal (`double`).

Exemple:

```java
import java.util.Scanner; // 1. Importem la classe de la llibreria de Java

public class ExempleScanner {
    public static void main(String[] args) {
        Scanner tecleat = new Scanner(System.in); // 2. Creem l'objecte Scanner

        System.out.print("Introdueix el teu nom: ");
        String nom = tecleat.nextLine(); // 3. Llegim el text introduït

        System.out.println("Hola, " + nom + "!");
        tecleat.close(); // Bons hàbits: tanquem el Scanner en acabar
    }
}
```

4. Generar nombres aleatoris amb Math.random()

La classe `Math` ve inclosa automàticament en Java (no necessita `import`). El seu mètode `Math.random()` genera un nombre decimal aleatori entre 0.0 (inclòs) i 1.0 (exclòs).

Com adaptar-lo per a obtenir un nombre enter en un rang (ex: un dau d'1 a 6):

- Multipliquem el resultat pel màxim del rang (`Math.random() * 6` dona de 0.0 a 5.999...).
- Fem un casting a enter (`int`) per a eliminar la part decimal (0 a 5).
- Sumem 1 per a ajustar el rang final (1 a 6).

Exemple per a simulació de daus:

```java
// Genera un nombre enter aleatori entre 1 i 6:
int dau = (int) (Math.random() * 6) + 1;
System.out.println("Has tret un: " + dau);
```

Exercici 6: El duel de daus (Extra) (Solució)

> 🔄 Solució

```java
import java.util.Scanner;

public class DuelDaus {
    public static void main(String[] args) {
        Scanner tecleat = new Scanner(System.in);

        System.out.print("Introdueix el teu nom: ");
        String nomUsuari = tecleat.nextLine();

        int dauUsuari = (int) (Math.random() * 6) + 1;
        int dauMaquina = (int) (Math.random() * 6) + 1;

        System.out.println(nomUsuari + " en trau " + dauUsuari + ", la màquina en trau " + dauMaquina + ".");

         String veredicte = (dauUsuari > dauMaquina) ? "Guanya " + nomUsuari + "!" :
                            (dauMaquina > dauUsuari) ? "Guanya la màquina." :
                            "Empat!";

        System.out.println(veredicte);

        tecleat.close();
    }
}
```

Exercici 7: Supercalculadora amb lògica (Extra) (Solució)

› 🔄 Solució

```java
import java.util.Scanner;

public class Supercalculadora {
    public static void main(String[] args) {
        Scanner tecleat = new Scanner(System.in);

        System.out.print("Introdueix el primer nombre: ");
        double a = tecleat.nextDouble();

        System.out.print("Introdueix el segon nombre: ");
        double b = tecleat.nextDouble();

        System.out.print("Introdueix l'operador (+, -, *, /): ");
        String op = tecleat.next();

        String resultat = op.equals("+") ? "Resultat: " + (a + b) :
                          op.equals("-") ? "Resultat: " + (a - b) :
                          op.equals("*") ? "Resultat: " + (a * b) :
                          op.equals("/") ? (b == 0 ? "Error: no es pot dividir
entre 0" : "Resultat: " + (a / b)) :
                          "Operador desconegut";

        System.out.println(resultat);

        tecleat.close();
    }
}
```

Exercici 8: Javadoc de campionat (Solució)

› 🔄 Solució

```java
/**
 * Classe que representa l'autor de la unitat i la seua motivació.
 *
 * @author Sergi Garcia
 * @version 1.0
 */
public class SobreMi {

    /**
     * Punt d'entrada: imprimeix la motivació i el nombre d'arguments rebuts.
     *
     * @param args arguments rebuts des de la línia d'ordres
     */
    public static void main(String[] args) {
        // Imprimisc primer el nom perquè el missatge tinga context
        System.out.println("Me llamo Sergi");

        /*
        Este bloc explica el programa a grans trets:
        mostrem la motivació per a estudiar programació
        i quants arguments hem rebut des de la terminal.
        */
        System.out.println("Mi motivación: convertirme en programador");
        System.out.println("He recibido " + args.length + " argumentos.");
    }
}
```

Explicació: En executar la comanda `javadoc SobreMi.java` en la terminal, es generaran automàticament els arxius `.html` amb la documentació tècnica del teu projecte. Si obris l'arxiu generat al navegador, veuràs la teva classe i el teu mètode amb les etiquetes `@author`, `@version` i `@param` perfectament estructurades i formatades com la documentació oficial de Java.

Butlletí U02 — Extras

📝 Butlletí U02 — Extras, per incrementar portfoli 🔗

Activitats opcionals orientades a resoldre reptes en plataformes competitives de programació. Estan dissenyades per a aquells alumnes que vulguin anar un pas més allà, avançar pel seu compte i anar creant el seu portafoli de projectes en GitHub.

⚠️ **Nota important (Fora de temari per ara):** Aquestes activitats són totalment extres i voluntàries. Utilitzen conceptes o estructures que anirem veient al llarg del curs. Si ara mateix no saps com resoldre-les totes, no et preocupes en absolut: pots guardar-te el butlletí i tornar a ells més endavant quan hàgem avançat en les pròximes unitats.

💼 **Impacte en el teu Currículum i Perfil Professional:** Plataformes com CodeWars i AceptaElReto et permeten mantenir un perfil públic amb les teves solucions, puntuació i nivell assolit. Incloure els enllaços a aquests perfils en el teu CV o en el teu LinkedIn és una pràctica molt valorada per les empreses del sector tecnològic, ja que demostra de manera pràctica la teva capacitat de resolució de problemes, la teva constància i el teu interès per millorar contínuament les teves habilitats de programació.

💡 **Consell per al teu Portafoli:** Crea un repositori en GitHub anomenat `java-challenges` o `code-solutions` i puja una carpeta organitzada per a cada un d'aquests reptes amb el seu codi font ben documentat.

Exercici 1: CodeWars — Square(n) Sum (Extra) 🔗

Resol la kata **Square(n) Sum** (8 kyu) en CodeWars.

Donat un array de números, eleva cadascun al quadrat i suma els resultats.

- Exemple: `[1, 2, 2]` → $1^2 + 2^2 + 2^2 = 1 + 4 + 4 = 9$.

Exercici 2: CodeWars — Grasshopper - Summation (Extra) 🔗

Resol la kata **Grasshopper - Summation** (8 kyu) en CodeWars.

Escriu un programa que trobi la suma de tots els números sencers des de 1 fins a n (ambdós inclosos).

- Exemple: Si n = 4, el resultat ha de ser 1 + 2 + 3 + 4 = 10.
- Pista: Hi ha una fórmula matemàtica coneguda (`n * (n + 1) / 2`), però si t'atreveixes, també pots resoldre-ho utilitzant un bucle. El mètode rebrà un `int n` i haurà de retornar un `int`.

Exercici 3: AceptaElReto — 119 Escudos del ejército romano (Extra)

Resol el problema **119 — Escudos del ejército romano** en la plataforma AceptaElReto.com.

Un exèrcit romà necessita organitzar els seus legionaris en formacions quadrades protegides per escuts. Cada formació de k x k legionaris requereix escuts per al contorn i la coberta superior. Determina quants escuts necessiten en total per a organitzar un nombre donat de soldats.

- Pista: Llegeix el nombre total de soldats per a cada cas de prova. La seqüència d'entrada finalitza quan es llegeix el valor 0 (marcador de fi).

Butlletí U03 — Inicial

📝 Butlletí U03 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, declara variables i embruta't les mans de bits. Ningú no neix sabent què és un `double`.

Exercici 1: Conversor de temperatures

Escriu un programa anomenat `ConversorTemperatura` que converteixi 30 graus Celsius a Fahrenheit. Usa la fórmula:

```
F = C * 9/5 + 32
```

Declara `int celsius = 30` i una variable `double fahrenheit` per al resultat. Mostra les dues temperatures.

Exercici 2: Què imprimeix? — increments

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class Increments {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x++);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(--x);
        System.out.println(x);
    }
}
```

Pista: recorda la diferència entre pre i post. x++ usa i després puja; ++x puja i després usa.

Exercici 3: Calculadora de descomptes

Declara una constant `final double DESCOMPTE = 0.15` i declara `double preuOriginal = 120.0`. Calcula:

1. El descompte (`preuOriginal * DESCOMPTE`)
2. El preu final (`preuOriginal - descompte`)

Mostra-ho tot amb `println`. Si després intentes canviar el valor de `DESCOMPTE`, el compilador s'enfadarà. Saps per què?

Exercici 4: El tipus perfecte

Indica quin tipus de dada primitiu (`int`, `double`, `boolean`, `char`, `long`) usaries per a cada cas:

1. El nombre d'habitants de la teva ciutat (~500.000)
2. La distància en quilòmetres fins a la Lluna (~384.400)
3. La inicial del teu segon cognom
4. La nota mitjana d'un examen (3.7)
5. Si has aprovat o no l'examen anterior
6. El preu d'un cafè en cèntims (enter)

Exercici 5: El casting assassí

Declara `double preu = 9.99;`. Converteix-lo a `int` amb un casting explícit. Imprimeix tots dos valors. Què es perd pel camí?

Després, declara `int gran = 300;` i converteix-lo a `byte`. Imprimeix el resultat. T'esperaves aquell nombre? (Pista: en un `byte` només caben de -128 a 127.)

Exercici 6: Parell o senar?

Escriu un programa anomenat `ParellOSenar` que usi l'operador `%` per a determinar si un nombre és parell o senar. Pots usar una variable `int numero = 7;` i l'operador ternari per a mostrar "Parell" o "Senar". Mostra també `true` o `false` en comprovar si `numero % 2 == 0`.

Exercici 7: String en acció

Escriu un programa anomenat `NomEnAccio` que, amb la variable `String nom = " ana ";`:

1. Netege els espais amb `trim()`
2. Mostri la longitud del nom net
3. El mostri en majúscules
4. Mostri la primera lletra (amb `substring(0, 1)`)

Exercici 8: Salutació amb Scanner

Escriu un programa anomenat `SalutacioScanner` que:

1. Pregunti "Com et dius?" i llegeixi el nom amb `nextLine()`
2. Pregunti "Quants anys tens?" i llegeixi l'edat amb `nextInt()`
3. Mostri: "Hola, [nom]. [edat] anys, benvingut."

Recorda l'`import java.util.Scanner;` i el `sc.close()`.

Exercici 9: CodeWars — Will you make it?

Resol la kata **"Will you make it?"** (8 kyu) en CodeWars.

Et donen la distància fins a una gasolinera, els litres que té el teu cotxe i els quilòmetres per litre. Determina si hi arribes o no. Retorna `true` si hi arribes, `false` si et quedes tirat.

Exercici 10: Què imprimeix? — printf amb conversions

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class FormatacioBasica {
    public static void main(String[] args) {
        int enter = 42;
        double decimal = 3.1416;
        String text = "Java";

        System.out.printf("%d %f %s %n", enter, decimal, text);
    }
}
```

Què imprimeix? Què fa `%n` al final?

Pista: `%d` és per a enters, `%f` per a decimals i `%s` per a text. Quants decimals mostra `%f` quan no li poses `%.2f`?

Exercici 11: la nota amb dos decimals

Escriu un programa anomenat `NotaFormatejada` que declari `String nom = "Marta"` i `double nota = 9.5678;`. Usa `String.format()` per a construir aquest missatge:

```
Marta ha tret un 9.57.
```

Després mostra el mateix missatge amb `System.out.printf()`. Quina diferència hi ha entre les dues formes?

Pista: `String.format` retorna un `String` i no imprimeix res; `printf` escriu directament en pantalla. Els dos usen `%.2f` per a deixar dos decimals.

Exercici 12: la propina amb dos decimals

Escriu un programa anomenat `PropinaFormatejada` que demani amb `Scanner` el total del compte (`double`) i el percentatge de propina (`int`). Calcula la propina i el total final, i mostra'ls amb `System.out.printf` i dos decimals:

```
Total: 45.50 €
Propina (15%): 6.82 €
Total a pagar: 52.33 €
```

Pista: `%.2f` controla els decimals. Multiplica primer (`total * percentatge`) i divideix després amb `100.0`, o `percentatge / 100` donarà 0.

Butlletí U03 — Inicial Resolt
📝 Butlletí U03 — Inicial (Resolt)

> Les solucions estan ocultes en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Conversor de temperatures

> 🔄 **Solució**

```
public class ConversorTemperatura {
    public static void main(String[] args) {
        int celsius = 30;
        double fahrenheit = celsius * 9 / 5.0 + 32;

        System.out.println(celsius + "°C són " + fahrenheit + "°F");
    }
}
```

Sortida: `30°C són 86.0°F`.

Fixat en el `5.0`: si escrivissis `celsius * 9 / 5 + 32`, la divisió `9 / 5` es faria amb enters (dona 1) i el resultat seria 62, un desastre. En usar `5.0`, la divisió es fa en decimal. Això és la divisió entera del punt 3 en acció.

Exercici 2: Què imprimeix? — increments

> 🔄 **Solució**

```
5
7
7
5
5
```

Pas a pas, amb x començant en 5:

1. x++ (POST) → imprimeix 5, després x passa a 6.
2. ++x (PRE) → x passa a 7, després imprimeix 7.
3. x- - (POST) → imprimeix 7, després x passa a 6.
4. - -x (PRE) → x passa a 5, després imprimeix 5.
5. x → imprimeix 5.

Exercici 3: Calculadora de descomptes

› 🔄 Solució

```
public class CalculadoraDescomptes {
    public static void main(String[] args) {
        final double DESCOMPTE = 0.15;
        double preuOriginal = 120.0;

        double descompte = preuOriginal * DESCOMPTE;
        double preuFinal = preuOriginal - descompte;

        System.out.println("Preu original: " + preuOriginal + "€");
        System.out.println("Descompte (15%): " + descompte + "€");
        System.out.println("Preu final: " + preuFinal + "€");
    }
}
```

Si intentes fer `DESCOMPTE = 0.20;` després, el compilador t'ho impedirà: `final` és el superglue del punt 2, una constant no es pot reassignar. Aquella és la raó que "el compilador s'enfade".

Exercici 4: El tipus perfecte

› 🔄 Solució

1. **`int`** — ~500.000 cap de sobres en un `int` (topall: 2.147 milions).
2. **`long`** — 384.400 km sí que cap en un `int`, però és una distància "astronòmica" i queda més coherent amb `long`. (Si ho posares `int`, també compila: la decisió de fons és que supera el quotidià.)

3. **`char`** — una sola lletra, amb cometes simples: `'M'`.
4. **`double`** — 3.7 té decimals.
5. **`boolean`** — només dos valors: `true` o `false`.
6. **`int`** — els cèntims són un nombre enter (sense decimals).

Exercici 5: El casting assassí

› 🔄 Solució

```
public class CastingAssassi {
    public static void main(String[] args) {
        double preu = 9.99;
        int preuEnter = (int) preu;

        System.out.println("Preu original: " + preu);
        System.out.println("Preu enter: " + preuEnter);

        int gran = 300;
        byte petit = (byte) gran;
        System.out.println("300 en un byte: " + petit);
    }
}
```

- `(int) 9.99` trunca i dona **9**: es perden els 0.99€. El casting talla amb destral, no arrodoneix.
- `300` en un `byte` (màxim 127) es **desborda** silenciosament i dona **44**. Java no avisa: és la teva responsabilitat comprovar que el valor cap abans d'estretir la caixa.

Exercici 6: Parell o senar?

› 🔄 Solució

```java
public class ParellOSenar {
    public static void main(String[] args) {
        int numero = 7;
        String resultat = numero % 2 == 0 ? "Parell" : "Senar";

        System.out.println(numero + " és " + resultat);
        System.out.println("És parell? " + (numero % 2 == 0));
    }
}
```

Sortida:

```
7 és Senar
És parell? false
```

El % retorna el resti de la divisió: si és 0, el nombre és parell. El ternari tria el missatge en una línia.

Exercici 7: String en acció

› 🔄 Solució

```java
public class NomEnAccio {
    public static void main(String[] args) {
        String nom = "  ana  ";
        String net = nom.trim();

        System.out.println("Net: " + net);
        System.out.println("Longitud: " + net.length());
        System.out.println("Majúscules: " + net.toUpperCase());
        System.out.println("Primera lletra: " + net.toUpperCase().substring(0,
1));
    }
}
```

Sortida:

```
Net: ana
Longitud: 3
Majúscules: ANA
Primera lletra: A
```

Fixat en l'encadenat: `net.toUpperCase().substring(0, 1)` neteja, posa en majúscules i talla la primera lletra en una sola línia.

Exercici 8: Salutació amb Scanner

› 🔄 Solució

```
import java.util.Scanner;

public class SalutacioScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Com et dius? ");
        String nom = sc.nextLine();

        System.out.print("Quants anys tens? ");
        int edat = sc.nextInt();

             System.out.println("Hola, " + nom + ". " + edat + " anys,
benvingut.");

        sc.close();
    }
}
```

En aquest programa l'ordre està bé: com que el `nextLine()` va abans que el `nextInt()`, no hi ha Enter residual que reclamar. El lio apareix al revés (número primer i text després), com en el punt 6.

Exercici 9: CodeWars — Will you make it?

› 🔄 Solució

```
public class Kata {
     public static boolean zeroFuel(double distanceToPump, double mpg, double
fuelLeft) {
         return mpg * fuelLeft >= distanceToPump;
    }
}
```

Multipliques els litres que tens pels quilòmetres per litre: això et dona la distància màxima que pots recórrer. Si és més gran o igual que la distància a la gasolinera, hi arribes (`true`); si no, et quedes tirat (`false`). Una línia amb un operador relacional i un >=.

Exercici 10: Què imprimeix? — printf amb conversions

› 🔄 Solució

Imprimeix:

```
42 3.141600 Java
```

%d rellena amb l'enter (42), `%f` amb el decimal (per defecte mostra **6 decimals**: 3.141600) i %s amb el text (Java). El %n és un salt de línia independent del sistema operatiu: funciona igual en Windows, Linux i Mac. En un `printf`, el %n és l'opció "oficial" per a saltar de línia.

Exercici 11: la nota amb dos decimals

› 🔄 Solució

```java
public class NotaFormatejada {
    public static void main(String[] args) {
        String nom = "Marta";
        double nota = 9.5678;

        String missatge = String.format("%s ha tret un %.2f.", nom, nota);
        System.out.println(missatge);

        System.out.printf("%s ha tret un %.2f.%n", nom, nota);
    }
}
```

`String.format` construeix el text i el guarda en la variable `missatge` (no imprimeix res); `printf` escriu directament en pantalla. Els dos arrodoneixen a dos decimals amb `%.2f`. Fixa't: usa `String.format` quan vulguis el text com a valor, i `printf` quan només vulguis escriure'l.

Exercici 12: la propina amb dos decimals

› 🔄 Solució

```java
import java.util.Scanner;

public class PropinaFormatejada {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Total del compte: ");
        double total = sc.nextDouble();

        System.out.print("Percentatge de propina: ");
        int percentatge = sc.nextInt();

        double propina = total * percentatge / 100.0;
        double totalFinal = total + propina;

        System.out.printf("Total: %.2f €%n", total);
        System.out.printf("Propina (%d%%): %.2f €%n", percentatge, propina);
        System.out.printf("Total a pagar: %.2f €%n", totalFinal);

        sc.close();
    }
}
```

- Per a mostrar el símbol `%` en un `printf` cal escriure'l **doble**: `%%`.
- Fixa't en `100.0`: si usares `100` (enter), `percentatge / 100` faria divisió entera i la propina sortiria 0. Ordre de les operacions i divisió entera, les dues trampes del punt 3.

Butlletí U03 — Avançat

📝 Butlletí U03 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: Calculadora de propines

Escriu un programa que calculi quant deixar de propina en un restaurant. Declara:

- `double totalCompte = 45.50;`
- `int percentatgePropina = 15;` (el percentatge, sense el símbol)

Calcula la propina (`totalCompte * percentatgePropina / 100`) i el total final (`totalCompte + propina`). Mostra els tres valors.

Pista: compte amb la divisió: `percentatgePropina / 100` amb enters dona 0. Multiplica primer i divideix després, o usa `100.0`.

⭐ Exercici 2: Conversor dòlar-euro

Declara `final double TAXA_CANVI = 0.92;` (1 dòlar = 0.92 euros). Declara `double dolars = 100.0;` i calcula el seu equivalent en euros. També fes la conversió inversa: donat `double euros = 50.0;`, calcula quants dòlars són.

Mostra:

```
100.0$ són 92.0€
50.0€ són 54.347826086956516$
```

Pista: per a passar d'euros a dòlars divideixes entre la taxa: `euros / TAXA_CANVI`.

⭐⭐ Exercici 3: Què imprimeix? — el casting traïdor

Sense executar, escriu la sortida exacta:

```java
public class CastingTraidor {
    public static void main(String[] args) {
        int a = 7;
        int b = 2;
        double resultat1 = a / b;
        double resultat2 = (double) a / b;
        double resultat3 = a / (double) b;

        System.out.println(resultat1);
        System.out.println(resultat2);
        System.out.println(resultat3);
        System.out.println(3 + 4 * 2.0);
        System.out.println((int) (3.7 + 2.3));
    }
}
```

Pista: fixa't bé en on està el casting i en quin moment s'aplica la divisió entera. En `resultat1`, ¿la divisió es fa abans o després de guardar en el `double`?

⭐⭐ Exercici 4: Interès compost (sense bucle)

Declara `final double CAPITAL_INICIAL = 1000.0;`, `final double TAXA = 0.05;` (5% anual) i `int anys = 3;`. Calcula el capital final després de 3 anys usant la fórmula de l'interès compost SENSE bucles:

```
capitalFinal = capitalInicial * (1 + taxa)^anys
```

Per a la potència usa `Math.pow(base, exponent)`. Mostra el capital any a any:

```
Any 0: 1000.0€
Any 1: 1050.0€
Any 2: 1102.5€
Any 3: 1157.625€
```

> 💡 **Compte amb la precisió:** en l'any 3 el teu programa pot imprimir `1157.6250000000002` en lloc de `1157.625` per la coma flotant binària dels `double` (ho veurem a «Atreveix-te a

> pensar»). No és un error del teu codi.

Pista: per a mostrar cada any sense bucle, crea tres variables distintes (`capital1`, `capital2`, `capital3`) i usa `Math.pow(1 + TAXA, i)` amb `i` valent 1, 2 i 3.

⭐⭐⭐ Exercici 5: L'enigma del post-increment

Sense executar, determina el valor de cada variable després d'executar aquest codi. Escriu el pas a pas:

```
public class EnigmaIncrement {
    public static void main(String[] args) {
        int x = 3;
        int y = x++ + ++x;
        int z = --y + y-- + x++;
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("z = " + z);
    }
}
```

Pista: fes una taula amb els valors de `x` i `y` després de cada operació. `x++` usa i després incrementa; `++x` incrementa i després usa.

⭐⭐ Exercici 6: El duel de daus

Escriu un programa que llanci un dau per a l'usuari (li demana el nom amb `Scanner`) i un altre per a la màquina (amb `Math.random()`). Guanya qui traga més. Mostra les dues tirades i el guanyador amb un missatge tipus:

```
Anna trau 4, la màquina trau 6.
Guanya la màquina.
```

Pista: combina `nextLine()` (nom), `(int)(Math.random() * 6) + 1` (daus) i un ternari encadenat per al veredicte. Compte amb l'empat.

⭐⭐ Exercici 7: Supercalculadora amb lògica

Escriu un programa que llegeixi dos nombres amb `Scanner` (`double`) i un caràcter d'operació (`+`, `-`, `*`, `/`) amb `next()`.

Segons l'operador, mostra el resultat. Si l'operador és `/` i el segon nombre és 0, mostra "Error: no es pot dividir entre 0". Per a qualsevol altre caràcter, mostra "Operador desconegut".

Pista: calcula els resultats possibles en variables i tria amb ternaris encadenats o amb lògica. Pots comprovar la divisió entre 0 amb `(op.equals("/") && b == 0)`.

⭐⭐⭐ Exercici 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'

Resol la kata **"Convert boolean values to strings 'Yes' or 'No'"** (8 kyu) en CodeWars.

Completa el mètode `public static String boolToWord(boolean b)` que retorni `"Yes"` si rep `true` i `"No"` si rep `false`.

Pista: es pot fer en una sola línia amb l'operador ternari: `b ? "Yes" : "No"`.

⭐⭐⭐ Exercici 9: AceptaElReto — 114 Últim dígit del factorial

Resol el problema **114 — Últim dígit del factorial** en AceptaElReto.com.

Donat un nombre N (0 ≤ N ≤ 1.000.000), calcula l'últim dígit de N! (factorial de N).

Pista: no necessites calcular el factorial sencer. Observa que 5! = 120, 6! = 720, 7! = 5040… a partir de 5, el factorial sempre acaba en 0. Només necessites gestionar els casos petits (0 a 4) i els grans amb una sola regla.

⭐⭐ Exercici 10: el tiquet de compra amb NumberFormat

Escriu un programa anomenat `TiquetCompra` que simuli un tiquet de compra amb tres productes (Pa, Llet i Ous, cadascun amb el seu preu i la seva quantitat). Mostra el tiquet usant `NumberFormat` amb moneda i locale espanyol per als preus:

```
=========================
    TICKET DE COMPRA
=========================
Pa    2 x 1,20 € = 2,40 €
Llet  3 x 0,95 € = 2,85 €
Ous   1 x 3,50 € = 3,50 €
-------------------------
TOTAL              = 8,75 €
=========================
```

Requisits: alinear els noms a l'esquerra, dos decimals en els preus i formatar tots els imports amb `NumberFormat.getCurrencyInstance(new Locale("es", "ES"))`.

Pista: `NumberFormat` formata un `double` com a moneda amb el separador del teu idioma (`1.234,56 €`). Per a cada producte calcula el subtotal (`preu * quantitat`) i suma'l al total. El `Locale("es", "ES")` li diu "parla com a Espanya": coma per als decimals i símbol €.

⭐⭐⭐ Exercici 11: l'edat a prova de bombes

Escriu un programa anomenat `EdatSegura` que demani l'edat pel teclat amb `Scanner` i **la repeteixi fins que l'usuari escrigui un nombre enter**. Si l'usuari escriu lletres o un decimal, el programa ha d'avisar amb "Això no és un nombre enter." i tornar a preguntar sense trencar-se (res d'`InputMismatchException`).

Quan per fi aconsegueixi un enter, mostra amb `printf`:

```
Genial, 20 anys i llest per a programar.
```

Pista: abans de cada `nextInt()`, pregunta amb `sc.hasNextInt()`. Si retorna `false`, descarta la brossa amb `sc.next()` i repeteix. Recorda: `hasNextInt()` **mira** la següent dada sense consumir-la; si no la descartes, es quedarà aquí per sempre.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	114 — Últim dígit del factorial	Fàcil
AceptaElReto	148 — Cap d'any	Fàcil
AceptaElReto	217 — Quin costat del carrer?	Fàcil
CodeWars	Will you make it? (8 kyu)	Principiant
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiant
CodeWars	Keep Hydrated (8 kyu)	Principiant
CodeWars	Get the Middle Character (7 kyu)	Intermedi

Butlletí UO3 — Avançat Resolt

📝 Butlletí UO3 — Avançat (Resolt)

> Les solucions estan ocultes. Intenta-ho de veritat abans de destapar-les.

⭐ Exercici 1: Calculadora de propines

> 🔄 Solució

```
public class Propines {
    public static void main(String[] args) {
        double totalCompte = 45.50;
        int percentatgePropina = 15;

        double propina = totalCompte * percentatgePropina / 100;
        double totalFinal = totalCompte + propina;

        System.out.println("Total compte: " + totalCompte + "€");
        System.out.println("Propina (" + percentatgePropina + "%): " + propina
+ "€");
        System.out.println("Total a pagar: " + totalFinal + "€");
    }
}
```

La clau: `totalCompte * percentatgePropina / 100` multiplica primer (45.50 * 15 = 682.5) i divideix després per 100 → 6.825. Si haguessis escrit `percentatgePropina / 100`, hauries fet divisió entera (15/100 = 0) i la propina hauria sortit 0. Ordre i divisió entera: les dues trampes del punt 3.

⭐ Exercici 2: Conversor dòlar-euro

> 🔄 Solució

```java
public class ConversorMoneda {
    public static void main(String[] args) {
        final double TAXA_CANVI = 0.92;

        double dolars = 100.0;
        double euros = dolars * TAXA_CANVI;

        double eurosInvers = 50.0;
        double dolarsInvers = eurosInvers / TAXA_CANVI;

        System.out.println(dolars + "$ són " + euros + "€");
        System.out.println(eurosInvers + "€ són " + dolarsInvers + "$");
    }
}
```

Sortida:

```
100.0$ són 92.0€
50.0€ són 54.347826086956516$
```

Per a passar de dòlars a euros multipliques per la taxa; al revés, divideixes. El `final` garanteix que ningú (ni tu) toqui la taxa de canvi sense voler.

⭐ ⭐ Exercici 3: Què imprimeix? — el casting traïdor

› 🔄 Solució

```
3.0
3.5
3.5
11.0
6
```

Línia a línia:

- `resultat1 = a / b` → primer divideix enters (7/2 = 3) i després guarda en `double`: **3.0**.
- `resultat2 = (double) a / b` → castin a a a 7.0 abans de dividir: **3.5**.

- `resultat3 = a / (double) b` → castin a `b` a 2.0: **3.5**.
- `3 + 4 * 2.0` → la multiplicació mana i el `2.0` arrossega a decimal: `3 + 8.0` = **11.0**.
- `(int) (3.7 + 2.3)` → primer suma (6.0), després trunca: **6**.

⭐⭐ Exercici 4: Interès compost (sense bucle)

> 🔄 **Solució**

```
public class InteresCompost {
    public static void main(String[] args) {
        final double CAPITAL_INICIAL = 1000.0;
        final double TAXA = 0.05;

        double capital1 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 1);
        double capital2 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 2);
        double capital3 = CAPITAL_INICIAL * Math.pow(1 + TAXA, 3);

        System.out.println("Any 0: " + CAPITAL_INICIAL + "€");
        System.out.println("Any 1: " + capital1 + "€");
        System.out.println("Any 2: " + capital2 + "€");
        System.out.println("Any 3: " + capital3 + "€");
    }
}
```

`Math.pow(1 + TAXA, i)` calcula `(1.05)^i`. Amb tres variables distintes evites el bucle… que arribarà en la UO4. Fixa't que `Math.pow` retorna un `double`.

> 💡 **Detall de precisió:** en l'any 3 el teu programa pot imprimir `1157.6250000000002` en lloc de `1157.625`. És la coma flotant binària del punt "Atreveix-te a pensar" (els decimals no sempre es representen exactes). No és un error: és així com funcionen els `double`.

⭐⭐⭐ Exercici 5: L'enigma del post-increment

> 🔄 **Solució**

El programa imprimeix:

```
x = 6
y = 6
z = 19
```

Pas a pas:

- `int x = 3;` → x = 3.
- `int y = x++ + ++x;` → x++ usa 3 i deixa x = 4; ++x puja x a 5 i usa 5. `y = 3 + 5 = 8`. Ara x = 5, y = 8.
- `int z = --y + y-- + x++;` → `--y` baixa y a 7 i usa 7; `y--` usa 7 i baixa y a 6; x++ usa 5 i puja x a 6. `z = 7 + 7 + 5 = 19`. Final: x = 6, y = 6, z = 19.

> 💡 **Confessió honesta:** si en fer-ho et va sortir un altre número, benvingut al club: aquest exercici existeix precisament perquè pateixi una vegada en classe i no vint en l'examen. La lliçó del punt 3: els ++ i `--` es fan servir sols, en la seva pròpia línia.

⭐⭐ Exercici 6: El duel de daus

> 🔄 Solució

```java
import java.util.Scanner;

public class DuelDaus {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Com et dius? ");
        String nom = sc.nextLine();

        int dauUsuari = (int) (Math.random() * 6) + 1;
        int dauMaquina = (int) (Math.random() * 6) + 1;

        System.out.println(nom + " trau " + dauUsuari + ", la màquina trau " +
dauMaquina + ".");

        String guanyador = dauUsuari > dauMaquina ? "Guanya " + nom + "." :
                           dauUsuari < dauMaquina ? "Guanya la màquina." :
"Empat.";
        System.out.println(guanyador);

        sc.close();
    }
}
```

Aquí no hi ha problema d'Enter residual: el `nextLine()` va abans del `nextInt()` (que ni tan sols usem). Els daus usen la fórmula `(int)(Math.random() * 6) + 1`, i el ternari encadenat decideix el guanyador en tres casos.

⭐⭐ Exercici 7: Supercalculadora amb lògica

› 🔄 Solució

```java
import java.util.Scanner;

public class SuperCalculadora {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Primer nombre: ");
        double a = sc.nextDouble();

        System.out.print("Segon nombre: ");
        double b = sc.nextDouble();

        System.out.print("Operació (+, -, *, /): ");
        String op = sc.next();

        double suma = a + b;
        double resta = a - b;
        double multi = a * b;

        String resultat = op.equals("+") ? "Resultat: " + suma :
                          op.equals("-") ? "Resultat: " + resta :
                          op.equals("*") ? "Resultat: " + multi :
                          op.equals("/") && b != 0 ? "Resultat: " + (a / b) :
                           op.equals("/") ? "Error: no es pot dividir entre 0"
:
                          "Operador desconegut";

        System.out.println(resultat);

        sc.close();
    }
}
```

La lògica del ternari encadenat: primer comprova si l'operador és `/` i a més `b` no és 0; si l'operador és `/` però `b` és 0, cau en el missatge d'error. Fixa't en `op.equals("/")`: els `String` sempre es comparen amb `.equals()`. Els operadors lògics i relacionals uneixen totes les condicions.

⭐⭐⭐ Exercici 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'

> 🔄 Solució

```
public class Kata {
    public static String boolToWord(boolean b) {
        return b ? "Yes" : "No";
    }
}
```

Una línia. El ternari retorna `"Yes"` si el booleà és `true` i `"No"` si és `false`. Elegant, com un esmòquing per al teu codi.

⭐⭐⭐ Exercici 9: AceptaElReto — 114 Últim dígit del factorial

> 🔄 Solució

```java
import java.util.Scanner;

public class UltimDigitFactorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int casos = sc.nextInt();

        for (int i = 0; i < casos; i++) {
            int n = sc.nextInt();
            int ultim;
            if (n >= 5) {
                ultim = 0;
            } else if (n == 0 || n == 1) {
                ultim = 1;
            } else if (n == 2) {
                ultim = 2;
            } else if (n == 3) {
                ultim = 6;
            } else {
                ultim = 4; // 4! = 24
            }
            System.out.println(ultim);
        }
        sc.close();
    }
}
```

La clau matemàtica: 5! = 120, i a partir d'aquí qualsevol factorial multiplica per 5 (i abans per un nombre parellll), així que **sempre acaba en 0**. Només cal memoritzar els casos petits: 0! i 1! són 1, 2! = 2, 3! = 6, 4! = 24. Si intentares calcular `1000000!`, el teu programa reventaria: per això la trampa de l'enunciat és que "sembla que cal calcular el factorial". No cal.

⭐ ⭐ Exercici 10: el tiquet de compra amb NumberFormat

› 🔄 Solució

```java
import java.text.NumberFormat;
import java.util.Locale;

public class TiquetCompra {
    public static void main(String[] args) {
                NumberFormat  moneda  =  NumberFormat.getCurrencyInstance(new
Locale("es", "ES"));

        double preuPa = 1.20;
        double preuLlet = 0.95;
        double preuOus = 3.50;

        double subtotalPa = preuPa * 2;
        double subtotalLlet = preuLlet * 3;
        double subtotalOus = preuOus * 1;
        double total = subtotalPa + subtotalLlet + subtotalOus;

        System.out.println("==========================");
        System.out.println("    TICKET DE COMPRA");
        System.out.println("==========================");
          System.out.println("Pa     2 x " + moneda.format(preuPa) + " = " +
moneda.format(subtotalPa));
         System.out.println("Llet   3 x " + moneda.format(preuLlet) + " = " +
moneda.format(subtotalLlet));
          System.out.println("Ous    1 x " + moneda.format(preuOus) + " = " +
moneda.format(subtotalOus));
        System.out.println("--------------------------");
        System.out.println("TOTAL               = " + moneda.format(total));
        System.out.println("==========================");
    }
}
```

`NumberFormat.getCurrencyInstance(new Locale("es", "ES"))` formata qualsevol `double` com a moneda espanyola: `1.234,56 €`. Com que els tres subtotals es calculen amb `preu * quantitat` abans de sumar-los, el total és exacte i sense errors d'arrodoniment acumulats. Aquest exercici et deixa llest per al punt 7 en l'examen: separadors de milers i moneda en un sol objecte.

⭐⭐⭐ Exercici 11: l'edat a prova de bombes

› 🔄 Solució

```java
import java.util.Scanner;

public class EdatSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int edat = -1;

        while (edat == -1) {
            System.out.print("Quants anys tens? ");
            if (sc.hasNextInt()) {
                edat = sc.nextInt();
            } else {
                System.out.println("Això no és un nombre enter.");
                sc.next();
            }
        }

        System.out.printf("Genial, %d anys i llest per a programar.%n", edat);
        sc.close();
    }
}
```

El bucle `while` repeteix la pregunta fins que l'usuari dona un enter. `hasNextInt()` mira si la següent dada és un enter sense consumir-la; si no ho és, `sc.next()` es menja la brossa i el bucle torna a preguntar. Així el programa és **a prova de bombes**: no importa quantes vegades l'usuari escrigui "hola" o "3.14", mai no saltarà la `InputMismatchException`.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	114 — Últim dígit del factorial	Fàcil
AceptaElReto	148 — Cap d'any	Fàcil
AceptaElReto	217 — Quin costat del carrer?	Fàcil
CodeWars	Will you make it? (8 kyu)	Principiant

Plataforma	Problema	Dificultat
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiant
CodeWars	Keep Hydrated (8 kyu)	Principiant
CodeWars	Get the Middle Character (7 kyu)	Intermedi

Butlletí U03 — Extres

📝 Butlletí U03 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està oculta: resisteix-te fins a esgotar la teva pista.

CodeWars

1. Keep Hydrated!

Nathan beu aigua, però no sempre prou. Li han donat `time` en hores i vol saber quants litres d'aigua ha de beure. La regla: beu 0.5 litres per hora. Retorna el nombre de litres **redonit cap avall**.

Exemple: amb `time = 6.7`, beu `3` litres (3.35 → 3).

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: multiplica `time * 0.5` i trunca el resultat amb `(int)`. El truncament i el "arrodoniment cap avall" són el mateix aquí… per una vegada.

> 🔄 **Solució**

```
public class Kata {
    public static int Liters(double time) {
        return (int) (time * 0.5);
    }
}
```

`6.7 * 0.5 = 3.35`, i `(int)` trunca a 3. El truncament del casting és exactament el "arrodoniment cap avall" que demana l'enunciat.

---

2. Opposite number

Escriu una funció que rebi un nombre i retorni el seu oposat (el mateix nombre amb el signe canviat).

Exemples: `1` → `-1`, `14` → `-14`, `-34` → `34`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: l'operador unari `-` inverteix el signe de qualsevol nombre: `-numero`. O multiplica'l per `-1`.

> 🔄 **Solució**

```
public class Kata {
    public static int opposite(int number) {
        return -number;
    }
}
```

L'operador unari `-` del punt 3 inverteix el signe. Un sol caràcter i el problema està resolt.

---

3. MakeUpperCase

Escriu una funció que converteixi un String a majúscules.

Exemple: `"hello"` → `"HELLO"`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: recordes el mètode de `String` del punt 8 que posa tot en majúscules? És la primera eina de la caixa.

> 🔄 **Solució**

```
public class Kata {
    public static String makeUpperCase(String str) {
        return str.toUpperCase();
    }
}
```

`toUpperCase()` del punt 8. Directe de la caixa d'eines.

---

4. Count of positives / sum of negatives

Donat un array d'enters, retorna un array de dos elements: el **nombre de números positius** en el primer forat i la **suma dels negatius** en el segon. Si l'array està buit, retorna un array buit.

Exemple: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, -11, -12, -13, -14, -15]` → `[10, -65]`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

Pista: no cal ni bucles fins que arribes a la UO6, però pots declarar `int[] resultat = new int[2];` i usar un bucle `for...each` (ja l'has vist en classe) per a comptar positius i sumar negatius amb `if`. Aquí l'important és recordar com accedir als forats d'un array.

> 🔄 **Solució**

```
public class Kata {
    public static int[] countPositivesSumNegatives(int[] input) {
        if (input == null || input.length == 0) {
            return new int[0];
        }

        int positius = 0;
        int sumaNegatius = 0;

        for (int numero : input) {
            if (numero > 0) {
                positius++;
            } else if (numero < 0) {
                sumaNegatius += numero;
            }
        }

        return new int[]{positius, sumaNegatius};
    }
}
```

El comptador usa ++, la suma acumulada usa +=, i el `if` decideix amb operadors relacionals. Un repàs perfecte de la unitat (el `for...each` complet el veuràs en la UO6).

---

5. Get the Middle Character

Donada una paraula, retorna el caràcter (o els dos caràcters) del centre.

Exemples: `"test"` → `"es"`, `"testing"` → `"t"`, `"middle"` → `"dd"`, `"A"` → `"A"`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

Pista: usa `length()` i `substring()` del punt 9. Si la longitud és parell, retorna els dos caràcters centrals; si és senar, només un.

> 🔄 **Solució**

```
public class Kata {
    public static String getMiddle(String word) {
        int longitud = word.length();
        int mig = longitud / 2;

        if (longitud % 2 == 0) {
            return word.substring(mig - 1, mig + 1);
        }
        return word.substring(mig, mig + 1);
    }
}
```

`longitud / 2` et dona el centre. Si la longitud és parell, el centre està repartit entre dos lletres (`"test"` → mig 2 → `substring(1, 3)` = `"es"`); si és senar, el centre és una sola lletra. `%` per a saber si és parell o senar, `length` per a mesurar i `substring` per a tallar: el trio d'eines d'aquesta unitat.

AceptaElReto

6. 148 — Cap d'any

El 31 de desembre, els més impacients miren el rellotge i conten quants segons falten per a les 12 de la nit. Donat un instant amb format `HH:MM:SS`, calcula els segons que falten per a mitjanit.

Entrada: un nombre amb el total de casos, i en cada línia una hora amb format `HH:MM:SS` (amb dos dígits per camp, però els zeros a l'esquerra poden ometre's).

Exemple:

```
3
00:00:00
23:59:59
05:30:00
```

Sortida:

```
86400
1
66600
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: el dia té 86400 segons. Converteix l'hora a segons totals (`hora * 3600 + minut * 60 + segon`) i resta-ho de 86400. Per a separar els camps, llegeix amb `nextInt()` i descarta els dos punts amb `next()`, o usa `Scanner.useDelimiter(":")`. El truc està en `split(":")`… o en llegir directament.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class CapAny {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int casos = sc.nextInt();

        for (int i = 0; i < casos; i++) {
            String hora = sc.next();
            String[] parts = hora.split(":");
            int h = Integer.parseInt(parts[0]);
            int m = Integer.parseInt(parts[1]);
            int s = Integer.parseInt(parts[2]);

            int segonsTranscorreguts = h * 3600 + m * 60 + s;
            int segonsFalten = 86400 - segonsTranscorreguts;
            System.out.println(segonsFalten);
        }
        sc.close();
    }
}
```

Lectura alternativa sense `split`: declarar el `Scanner` amb `useDelimiter("\\s*:\\s*")` i llegir tres `nextInt()` seguits. La lògica és la mateixa: hora a segons, resta-ho de 86400 (els segons que té el dia). Per a `00:00:00` falten 86400; per a `23:59:59`, 1. Pura aritmètica de la unitat.

---

7. 217 — Quin costat del carrer?

En el carrer principal de Donya Lita les cases es distribuïxen en dos costats: els números senars van en un costat i els parells en l'altre. Donat un número de casa, digues en quin costat està.

Entrada: el número 0 indica la fi de l'entrada (no cal processar-lo). Per a cada número de casa (positiu), mostra:

```
ESQUERRA
```

o

```
DRETA
```

segons corresponga. El criteri és simple: un número de casa és `ESQUERRA` si és parell i `DRETA` si és senar (o al revés, segons el carrer; llegeix bé l'enunciat original).

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: quin operador et diu si un nombre és parell o senar? El `%` del punt 3. El bucle acaba quan llegeixes un 0, amb un `while`.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class CostatCarrer {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero = sc.nextInt();

        while (numero != 0) {
            String costat = numero % 2 == 0 ? "ESQUERRA" : "DRETA";
            System.out.println(costat);
            numero = sc.nextInt();
        }
        sc.close();
    }
}
```

L'operador mòdul `%` decideix: resti 0 és parell, resti 1 és senar. El `while` repeteix fins a llegir el 0 sentinella. La lògica sencera del problema és un sol ternari.

🧭 **I si et quedes amb ganes?** Quan domines les estructures de control (UO4) podràs tornar a aquests problemes amb `for` i `switch` per a fer solucions més elegants. El material no es perd: es reutilitza.

Butlletí UO4 — Inicial

📝 Butlletí UO4 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, encén el semàfor i fes que el teu codi decideixi, repeteixi i sobrevisqui. Ningú no neix sabent quan posar un `break`.

Exercici 1: El porter del club

Escriu un programa anomenat `Porter` que, amb les variables `int edat = 17` i `boolean teEntrada = true`, decideixi amb `if/else`:

- Si `edat >= 18` **i** té entrada → "Endavant, que passi el major d'edat."
- Si és major d'edat però **no** té entrada → "Faltes tu sol, sense entrada no hi ha paradís."
- Si és menor → "Fora d'aquí, petit."

Exercici 2: Què imprimeix? — el semàfor invertit

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class SemaforInvertit {
    public static void main(String[] args) {
        int nota = 8;

        if (nota >= 5) {
            System.out.println("Aprovat");
        } else if (nota >= 7) {
            System.out.println("Notable");
        } else if (nota >= 9) {
            System.out.println("Excel·lent");
        } else {
            System.out.println("Suspés");
        }
    }
}
```

Pista: recorda que guanya el **primer** `if` que es compleix, encara que n'hi hagi altres més "precisos" després.

Exercici 3: El menú del dia

Escriu un programa anomenat `MenuDia` que, amb `int dia = 4`, mostri amb `switch` el menú corresponent:

- `1` → Dilluns: Llenties
- `2` → Dimarts: Paella
- `3` → Dimecres: Macarrons
- `4` → Dijous: Fabada
- `5` → Divendres: Peix
- Qualsevol altre número → `default`: Cap de setmana, no hi ha menú

No t'oblides del `break` en cada `case` ni del `default`.

Exercici 4: El compte enrere del coet

Escriu un programa anomenat `Coet` que compte des de 5 fins a 0 amb un `while` i després mostri "Enlairament! 🚀". Mostra un nombre per línia.

Pista: `int comptador = 5;` i `while (comptador >= 0) { ... comptador--; }`.

Exercici 5: La taula del 7

Escriu un programa anomenat `TaulaDelSet` que mostri la taula de multiplicar del 7 (de l'1 al 10) usant un `for`:

```
7 x 1 = 7
7 x 2 = 14
...
```

Exercici 6: Només els parells

Escriu un programa anomenat `SolsParells` que imprimeixi els nombres parells del 2 al 20 usant un `for` i `continue`. Cada nombre en la seva línia.

Pista: recorre de l'1 al 20 i usa `continue` per a saltar-te els senars (`if (i % 2 != 0) continue;`).

Exercici 7: El detectiu de divisors

Escriu un programa anomenat `DetectiuDivisors` que comprovi amb `for` i `break` si el nombre `int numero = 29` té algun divisor entre 2 i `numero - 1`. Si el troba, mostra "No és primer" i surt; si no, mostra "És primer".

Pista: recorre els divisors i, tan bon punt en trobes un (`numero % divisor == 0`), fes `break`.

Exercici 8: L'edat blindada

Escriu un programa anomenat `EdatBlindada` que demani l'edat amb `Scanner.nextInt()` dins d'un `try/catch`. Si l'usuari escriu alguna cosa que no és un nombre, atrapar l'`InputMismatchException` i mostra "Això no és una edat vàlida.". Si ho és, mostra "Tens X anys."

Pista: necessites `import java.util.Scanner;` i `import java.util.InputMismatchException;`. Després del `catch`, el programa ha de seguir viu.

Exercici 9: CodeWars — Even or Odd

Resol la kata **"Even or Odd"** (8 kyu) en CodeWars.

Crea el mètode `public static String even_or_odd(int number)` que retorni `"Even"` si el nombre és parell i `"Odd"` si és senar.

Pista: l'operador `%` del punt 3 de la UO3 és el teu millor amic: `number % 2 == 0`.

Butlletí U04 — Inicial Resolt

📝 Butlletí U04 — Inicial (Resolt)

> Les solucions estan ocultes en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: El porter del club

> 🔄 Solució

```
public class Porter {
    public static void main(String[] args) {
        int edat = 17;
        boolean teEntrada = true;

        if (edat >= 18 && teEntrada) {
            System.out.println("Endavant, que passe el major d'edat.");
        } else if (edat >= 18) {
                System.out.println("Faltes tu sol, sense entrada no hi ha
paradís.");
        } else {
            System.out.println("Fora d'ací, xicotet.");
        }
    }
}
```

Sortida: `Fora d'ací, xicotet.`

Fixa't en l'ordre: primer comproves la combinació completa (`major I entrada`), després el cas del major sense entrada. El `else` final es queda amb els menors. Ordre estricte a lax, com en el punt 1.

Exercici 2: Què imprimeix? — el semàfor invertit

> 🔄 Solució

Imprimeix **`Aprovat`**.

Amb `nota = 8`, el primer `if` (`nota >= 5`) es compleix i guanya, encara que 8 també compliria les condicions de Notable i Excel·lent. Java avalua en ordre i es queda amb la primera `true`. La lliçó: l'ordre dels `else if` decideix el resultat.

Exercici 3: El menú del dia

› 🔄 **Solució**

```
public class MenuDia {
    public static void main(String[] args) {
        int dia = 4;

        switch (dia) {
            case 1:
                System.out.println("Dilluns: Llenties");
                break;
            case 2:
                System.out.println("Dimarts: Paella");
                break;
            case 3:
                System.out.println("Dimecres: Macarrons");
                break;
            case 4:
                System.out.println("Dijous: Fabada");
                break;
            case 5:
                System.out.println("Divendres: Peix");
                break;
            default:
                System.out.println("Cap de setmana, no hi ha menú");
                break;
        }
    }
}
```

Sortida: `Dijous: Fabada`

Cada `case` amb el seu `break` (llevat del `default`, que no el necessita, però no fa mal). Sense els `break`, el switch es convertiria en un tobogan i ho imprimiria tot des del `case 4` en endavant.

Exercici 4: El compte enrere del coet

> 🔄 Solució

```
public class Coet {
    public static void main(String[] args) {
        int comptador = 5;

        while (comptador >= 0) {
            System.out.println(comptador);
            comptador--;
        }

        System.out.println("Enlairament! 🚀");
    }
}
```

Sortida:

```
5
4
3
2
1
0
Enlairament! 🚀
```

El `comptador--` al final de cada vegada és el que evita el bucle infinit: la condició `comptador >= 0` avança cap a `false`.

Exercici 5: La taula del 7

> 🔄 Solució

```java
public class TaulaDelSet {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            System.out.println("7 x " + i + " = " + (7 * i));
        }
    }
}
```

Sortida (primeres línies):

```
7 x 1 = 7
7 x 2 = 14
```

Fixa't en els parèntesis de `(7 * i)`: sense ells, el + concatenaria igualment (perquè `*` mana sobre +), però és més clar amb ells. El `for` junta comptador, condició i avanç en una línia.

Exercici 6: Només els parells

› 🔄 **Solució**

```java
public class SolsParells {
    public static void main(String[] args) {
        for (int i = 1; i <= 20; i++) {
            if (i % 2 != 0) {
                continue;
            }
            System.out.println(i);
        }
    }
}
```

Sortida:

```
2
4
6
8
10
12
14
16
18
20
```

El `continue` es salta els senars i el bucle segueix amb el següent nombre. Alternativa sense `continue`: `for (int i = 2; i <= 20; i += 2)`, però aquí practiquem el salt.

Exercici 7: El detectiu de divisors

› 🔄 Solució

```java
public class DetectiuDivisors {
    public static void main(String[] args) {
        int numero = 29;
        boolean esPrimer = true;

        for (int divisor = 2; divisor < numero; divisor++) {
            if (numero % divisor == 0) {
                esPrimer = false;
                break;
            }
        }

        System.out.println(esPrimer ? "És primer" : "No és primer");
    }
}
```

Sortida: `És primer`

El `break` talla el bucle tan bon punt apareix un divisor: no cal seguir comprovant. Per al 29 no hi ha divisors (és primer), així que el bucle es recorre sencer i `esPrimer` continua sent `true`.

Exercici 8: L'edat blindada

› 🔄 Solució

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class EdatBlindada {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Quants anys tens? ");
            int edat = sc.nextInt();
            System.out.println("Tens " + edat + " anys.");
        } catch (InputMismatchException e) {
            System.out.println("Això no és una edat vàlida.");
        }

        System.out.println("El programa seguix viu. 🎉");
        sc.close();
    }
}
```

Si escrius `hola`, el `catch` atrapar l'error, imprimeix el missatge i el programa continua. Si escrius `17`, tot normal. Aquell és el poder del `try/catch`: el teu programa ja no mor per la porqueria de l'usuari.

Exercici 9: CodeWars — Even or Odd

› 🔄 Solució

```
public class Kata {
    public static String even_or_odd(int number) {
        return number % 2 == 0 ? "Even" : "Odd";
    }
}
```

> Una línia amb l'operador `%` i un ternari. Si el residu de dividir entre 2 és 0, és parell (`"Even"`); si no, senar (`"Odd"`). Els dos conceptes de la U03 i la U04 treballant junts.

Butlletí U04 — Avançat

📝 Butlletí U04 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: La calculadora de notes

Escriu un programa que converteixi una nota numèrica (`double nota`) en la seva qualificació textual usant `if/else if`:

- `>= 9` → "Excel·lent"
- `>= 7` → "Notable"
- `>= 5` → "Aprovat"
- `>= 0` → "Suspés"
- Qualsevol altre valor (negatiu o major que 10) → "Nota invàlida"

Usa `double nota = 8.7;` i mostra el resultat. Fixa't en l'ordre: el cas invàlid s'ha de comprovar abans que els rangs.

Pista: un `if` inicial per al cas invàlid (`nota < 0 || nota > 10`) i després la cascada de rangs de dalt a baix.

⭐ Exercici 2: El menú que no es rendeix

Escriu un programa que mostri un menú de dues opcions (1. Jugar, 2. Sortir) amb `do-while` i repeteixi la pregunta fins que l'usuari escrigui 1 o 2. Usa `Scanner`. Al final mostra "Has triat l'opció X.".

Pista: `do { ... } while (opcio != 1 && opcio != 2);`. El `do-while` garanteix que el menú es mostri almenys una vegada.

⭐⭐ Exercici 3: Què imprimeix? — la piràmide

Sense executar, escriu la sortida exacta:

```
public class Piramide {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int col = 1; col <= fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Pista: el bucle interior depèn de `fila`: la fila 1 imprimeix 1 asterisc, la fila 2 en imprimeix 2… Quants asteriscs en total?

⭐⭐ Exercici 4: L'escala de nombres

Escriu un programa que imprimeixi aquesta escala de nombres (bucles anidats):

```
1
1 2
1 2 3
1 2 3 4
```

Pista: bucle exterior de l'1 al 4 (les files) i bucle interior que imprimeix de l'1 al número de fila, amb `print` per a no saltar de línia i un `println()` buit al final de cada fila.

⭐ ⭐ ⭐ Exercici 5: Què imprimeix? — break, continue i l'etiqueta

Sense executar, escriu la sortida exacta:

```java
public class Escapista {
    public static void main(String[] args) {
        exterior:
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 3; j++) {
                if (j == 2 && i >= 3) {
                    continue exterior;
                }
                if (i * j >= 8) {
                    break exterior;
                }
                System.out.print(i + "" + j + " ");
            }
        }
    }
}
```

Pista: fes una taula de parells `(i, j)`. El `continue exterior` salta a la següent `i`; el `break exterior` apaga tots els bucles. Comprova parell a parell fins on arriba.

⭐⭐ Exercici 6: Caçador de primers

Escriu un programa que imprimeixi tots els nombres primers de l'1 al 50, cada un en la seva línia. Reutilitza la lògica del detectiu de divisors (`for` + `break`) dins d'un altre `for`.

Pista: anida dos bucles: un que recorri de l'1 al 50 i un altre interior que busqui divisors. Usa un booleà `esPrimer` que es posi a `false` amb `break` si apareix un divisor.

⭐⭐ Exercici 7: La suma sentinella

Escriu un programa que sumi nombres enters introduïts per l'usuari fins que escrigui `0`. Mostra la suma final. Usa `while` i un sentinella.

Pista: `while (numero != 0) { suma += numero; numero = sc.nextInt(); }`. El 0 no es suma: és el senyal de parada.

⭐⭐⭐ Exercici 8: CodeWars — Categorize New Member

Resol la kata **"Categorize New Member"** (7 kyu) en CodeWars.

Completa el mètode `public static String[] openOrSenior(int[][] data)` que rep parells `{edat, handicap}` i retorna `"Senior"` si el membre té almenys 55 anys I un handicap major que 7; si no, `"Open"`.

Pista: recorre l'array amb un `for` i decideix cada cas amb una condició combinada (`&&`): `data[i][0] >= 55 && data[i][1] > 7`.

⭐⭐⭐ Exercici 9: AceptaElReto — 156 Ascensor

Resol el problema **156 — Ascensor** en AceptaElReto.com.

Un ascensor parteix de la planta 0. Donat un nombre de visites i les plantes de cada visita, calcula la distància total recorreguda. El valor `0` indica el final de l'entrada (no es processa). Usa `Math.abs()` per a les distàncies.

Exemple:

```
5
5 1 10 4 2
0
```

Distància: |0-5| + |5-1| + |1-10| + |10-4| + |4-2| = 5 + 4 + 9 + 6 + 2 = **26**.

Pista: un `while` que llegeixi el nombre de visites i trenqui amb `break` si és 0; dins, un `for` que acumuli `Math.abs(pis - pisActual)` i actualitze `pisActual`.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	156 — Ascensor	Fàcil
AceptaElReto	149 — San Fermines	Fàcil

Plataforma	Problema	Dificultat
AceptaElReto	340 — Següent amb mateix nombre de xifres	Mitjà
CodeWars	Even or Odd (8 kyu)	Principiant
CodeWars	Categorize New Member (7 kyu)	Aficionat
CodeWars	Return Negative (8 kyu)	Principiant

Butlletí U04 — Avançat Resolt

📝 Butlletí U04 — Avançat (Resolt)

> Les solucions estan ocultes. Intenta-ho de veritat abans de destapar-les.

⭐ Exercici 1: La calculadora de notes

> 🔄 Solució

```java
public class CalculadoraNotes {
    public static void main(String[] args) {
        double nota = 8.7;

        if (nota < 0 || nota > 10) {
            System.out.println("Nota invàlida");
        } else if (nota >= 9) {
            System.out.println("Excel·lent");
        } else if (nota >= 7) {
            System.out.println("Notable");
        } else if (nota >= 5) {
            System.out.println("Aprovat");
        } else {
            System.out.println("Suspés");
        }
    }
}
```

Sortida: `Notable`

El primer `if` caça les notes impossibles (negatives o majors de 10) i la cascada següent avalua de la més exigent a la més permissiva. Amb 8.7, el `>= 7` guanya i dona “Notable”.

⭐ Exercici 2: El menú que no es rendeix

> 🔄 Solució

```java
import java.util.Scanner;

public class MenuTenac {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcio = 0;

        do {
            System.out.println("1. Jugar  2. Eixir");
            System.out.print("Tria: ");
            opcio = sc.nextInt();
        } while (opcio != 1 && opcio != 2);

        System.out.println("Has triat l'opció " + opcio + ".");
        sc.close();
    }
}
```

El `do-while` garanteix que el menú es mostri almenys una vegada. Mentres l'opció no sigui 1 ni 2, el bucle insisteix. L'usuari pot ser tot el tonto que vulgui; el menú no es rendeix.

⭐⭐ Exercici 3: Què imprimeix? — la piràmide

› 🔄 Solució

```
*
**
***
****
```

En total **10 asteriscs** (1 + 2 + 3 + 4). El bucle interior (`col <= fila`) imprimeix tants asteriscs com el número de fila. El cor dels bucles anidats: el interior depèn de l'exterior.

⭐⭐ Exercici 4: L'escala de nombres

› 🔄 Solució

```java
public class Escala {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int num = 1; num <= fila; num++) {
                System.out.print(num + " ");
            }
            System.out.println();
        }
    }
}
```

El interior imprimeix de l'1 al número de fila amb `print` (sense salt de línia); el `println()` buit salta en acabar cada fila. L'escala completa:

```
1
1 2
1 2 3
1 2 3 4
```

⭐ ⭐ ⭐ Exercici 5: Què imprimeix? — break, continue i l'etiqueta

› 🔄 Solució

Taula de parells, en ordre d'execució:

i	j	Què passa?	Imprimeix?
1	1	`1*1=1 < 8`	11
1	2	`j==2 && i>=3` → no; `1*2=2 < 8`	12
1	3	`1*3=3 < 8`	13
2	1	`2*1=2 < 8`	21
2	2	`2*2=4 < 8`	22
2	3	`2*3=6 < 8`	23

i	j	Què passa?	Imprimeix?
3	1	`3*1=3 < 8`	`31`
3	2	`j==2 && i>=3` → **sí** → `continue exterior`	res
4	1	`4*1=4 < 8`	`41`
4	2	`j==2 && i>=3` → **sí** → `continue exterior`	res

Fixat que el `break exterior` **mai no es dispara**: amb `i=1` i `i=2` el producte `i*j` no arriba a 8, i amb `i>=3` sempre salta abans pel `continue exterior` (en `j=2`), així que mai no s'avalua el `j=3` on `3*3=9` hauria superat 8. La lliçó: amb les etiquetes, no assumeixis què passa — executa-ho mentalment parell a parell.

⭐⭐ Exercici 6: Caçador de primers

› 🔄 Solució

```java
public class CacadorPrimers {
    public static void main(String[] args) {
        for (int numero = 1; numero <= 50; numero++) {
            boolean esPrimer = true;

            if (numero < 2) {
                esPrimer = false;
            } else {
                for (int divisor = 2; divisor < numero; divisor++) {
                    if (numero % divisor == 0) {
                        esPrimer = false;
                        break;
                    }
                }
            }

            if (esPrimer) {
                System.out.println(numero);
            }
        }
    }
}
```

Sortida (primers): `2 3 5 7 11 13...`. El bucle exterior recorre els candidats i el interior busca divisors amb `break` tan bon punt en troba un. El `if (numero < 2)` aparta l'1, que no és primer (és l'anec lleig de les matemàtiques).

⭐⭐ Exercici 7: La suma sentinella

› 🔄 **Solució**

```java
import java.util.Scanner;

public class SumaSentinella {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int numero = sc.nextInt();

        while (numero != 0) {
            suma += numero;
            numero = sc.nextInt();
        }

        System.out.println("Suma total: " + suma);
        sc.close();
    }
}
```

El `while` es repeteix mentres el nombre no sigui 0. El 0 és el **sentinella**: no es suma, només senyala el final. És el patró clàssic de lectura de dades amb `while`.

⭐⭐⭐ Exercici 8: CodeWars — Categorize New Member

› 🔄 Solució

```java
public class Kata {
    public static String[] openOrSenior(int[][] data) {
        String[] result = new String[data.length];

        for (int i = 0; i < data.length; i++) {
              result[i] = (data[i][0] >= 55 && data[i][1] > 7) ? "Senior" :
"Open";
        }

        return result;
    }
}
```

Un bucle que recorre els parells `{edat, handicap}` i un ternari amb la condició combinada `&&`: per a ser “Senior” cal tenir almenys 55 anys I un handicap major que 7. Tot el que no ho compleix és “Open”. Bucles, condicionals i ternaris de la unitat en una sola funció.

⭐⭐⭐ Exercici 9: AceptaElReto — 156 Ascensor

› 🔄 Solució

```
import java.util.Scanner;

public class Ascensor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int n = sc.nextInt();
            if (n == 0) {
                break;
            }

            int distancia = 0;
            int pisActual = 0;

            for (int i = 0; i < n; i++) {
                int pis = sc.nextInt();
                distancia += Math.abs(pis - pisActual);
                pisActual = pis;
            }

            System.out.println(distancia);
        }
        sc.close();
    }
}
```

El `while (true)` amb `break` en el 0 gestiona els múltiples casos. Dins, un `for` acumula `Math.abs(pis - pisActual)` i actualitza la planta actual. `Math.abs` dona la distància sense importar si puja o baixa. Amb l’entrada de l’exemple: `|0-5| + |5-1| + |1-10| + |10-4| + |4-2| = 26`.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	156 — Ascensor	Fàcil
AceptaElReto	149 — San Fermines	Fàcil
AceptaElReto	340 — Següent amb mateix nombre de xifres	Mitjà
CodeWars	Even or Odd (8 kyu)	Principiant
CodeWars	Categorize New Member (7 kyu)	Aficionat
CodeWars	Return Negative (8 kyu)	Principiant

Butlletí UO4 — Extres

📝 Butlletí UO4 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està oculta: resisteix-te fins a esgotar la teva pista.

CodeWars

1. Return Negative

Et donen un nombre i has de retornar-lo negatiu. Però compte: si el nombre ja és negatiu, deixa'l com està.

Exemples: `1` → `-1`, `-14` → `-14`, `34` → `-34`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: un ternari decideix: si és major que 0, canvia el signe (`-num`); si no, retorna'l tal qual.

> 🔄 **Solució**
>
> ```
> public class Kata {
>     public static int makeNegative(int num) {
>         return num > 0 ? -num : num;
>     }
> }
> ```
>
> Un ternari amb la condició `num > 0`. Si és positiu, l'operador unari `-` el fa negatiu; si ja era negatiu (o zero), es queda igual. Decidir amb una línia: pura UO4.

---

2. Sum of positive

Et donen un array d'enters. Retorna la suma de tots els **positius**.

Exemple: `[1, -4, 7, 12]` → `1 + 7 + 12 = 20`.

- Enunciat en CodeWars

- Dificultat: 8 kyu

Pista: recorre l'array amb un `for` i, amb un `if`, suma només els nombres majors que 0.

> 🔄 **Solució**

```
public class Kata {
    public static int sum(int[] arr) {
        int suma = 0;
        for (int numero : arr) {
            if (numero > 0) {
                suma += numero;
            }
        }
        return suma;
    }
}
```

El bucle recorre cada nombre i l'`if` filtra els positius abans de sumar-los. El `for...each` és una drecera que ja coneixes; l'`if` dins del bucle és l'essència de les estructures de control.

---

3. Grasshopper — Summation

Escriu un programa que calculi la suma de tots els nombres des d'1 fins a `n`.

Exemples: `summation(2)` → 3 (1 + 2), `summation(8)` → 36 (1 + 2 + … + 8).

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: un `for` que acumuli de l'1 al `n` en una variable `suma`. (O la fórmula matemàtica `n * (n + 1) / 2`, si et va la drecera.)

> 🔄 **Solució**

```java
public class Kata {
    public static int summation(int n) {
        int suma = 0;
        for (int i = 1; i <= n; i++) {
            suma += i;
        }
        return suma;
    }
}
```

Amb bucles: `suma += i` acumula cada vegada. Amb la fórmula directa: `return n * (n + 1) / 2;`, que dona el mateix resultat sense ni un bucle. Les dues són vàlides; el bucle és el que practiquem aquí.

---

4. Sum without highest and lowest number

Suma tots els nombres d'un array **excepte** el més alt i el més baix. Si l'array està buit, és `null` o té un sol element, retorna `0`.

Exemple: `[6, 2, 1, 8, 10]` → `2 + 6 + 8 = 16`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

Pista: recorre l'array una vegada per a acumular la suma i, en el mateix bucle, rastreja el mínim i el màxim amb dos `if`. Al final: `suma - minimo - maximo`.

> 🔄 **Solució**

```java
public class Kata {
    public static int sum(int[] numeros) {
        if (numeros == null || numeros.length <= 1) {
            return 0;
        }

        int suma = 0;
        int minimo = Integer.MAX_VALUE;
        int maximo = Integer.MIN_VALUE;

        for (int numero : numeros) {
            suma += numero;
            if (numero < minimo) minimo = numero;
            if (numero > maximo) maximo = numero;
        }

        return suma - minimo - maximo;
    }
}
```

El truc: `Integer.MAX_VALUE` i `Integer.MIN_VALUE` com a “infinits” inicials, perquè el primer nombre de l’array sempre els supere. Un sol bucle fa tres coses (sumar, buscar mínim, buscar màxim) i la resta final lleva els extrems. Tres decisions en un sol recorregut: pura múscul d’aquesta unitat.

AceptaElReto

5. 149 — San Fermines

Als sanfermines, els moços volen saber la velocitat màxima dels braus per a saber si sortir corrent o volar. Donat un nombre de braus i la velocitat de cada un, digues quina és la màxima.

Entrada: diversos casos de prova fins al final de l’entrada (EOF). Cada cas: un nombre N i a continuació les N velocitats.

Exemple:

```
3
10 20 15
5
50 12 90 7 25
```

Sortida:

```
20
90
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: un `while (sc.hasNextInt())` llegeix fins al final de l'entrada. Dins, un `for` recorre les velocitats i un `if` va guardant el màxim.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class SanFermines {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (sc.hasNextInt()) {
            int n = sc.nextInt();
            int max = 0;

            for (int i = 0; i < n; i++) {
                int velocitat = sc.nextInt();
                if (velocitat > max) {
                    max = velocitat;
                }
            }

            System.out.println(max);
        }
        sc.close();
    }
}
```

> El `while (sc.hasNextInt())` és el patró "llegir fins al final". L'`if` dins del `for` compara cada velocitat amb el màxim acumulat. En acabar el bucle, `max` és la resposta. Bucles i decisions treballant en equip.

---

6. 340 — Següent amb mateix nombre de xifres

Donat un nombre natural N, troba el següent nombre que tingui el **mateix nombre de xifres** que N. Si no existeix (perquè N és l'últim amb aquell nombre de xifres), mostra `MENSAJE`.

Entrada: diversos casos separats per espais, que acaba amb un `0`.

Exemples:

```
1 → 2
9 → MENSAJE
99 → MENSAJE
100 → 101
```

- Enunciat en AceptaElReto
- Dificultat: Mitjà

Pista: compta les xifres de N amb un `while (copia > 0) { cifras++; copia /= 10; }`. Si N és 9, 99, 999… (tots nous), no hi ha següent. En la resta de casos, la resposta és `N + 1`.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class SeguentCifres {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n != 0) {
            int copia = n;
            int cifres = 0;

            while (copia > 0) {
                cifres++;
                copia /= 10;
            }

            int ultimAmbEixeTamany = (int) Math.pow(10, cifres) - 1;

            if (n == ultimAmbEixeTamany) {
                System.out.println("MENSAJE");
            } else {
                System.out.println(n + 1);
            }

            n = sc.nextInt();
        }
        sc.close();
    }
}
```

Dos bucles: el interior compta les xifres dividint entre 10 (`while (copia > 0)`), i l'exterior llegeix casos fins al 0 sentinella. Si N és 9, 99 o 999 (el `10^cifres - 1`), no existeix següent i toca "MENSAJE". En qualsevol altre cas, sumar 1 no canvia el nombre de xifres… llevat dels de tots nous, que ja caçàrem abans.

🧭 **I si et quedes amb ganes?** Quan domines els bucles i les condicions podràs tornar als problemes de la UO3 (com Nochevieja o Quina banda del carrer?) i resoldre'ls amb `for` i `switch` per a fer solucions més elegants. El material no es perd: es reutilitza.

Butlletí UO5 — Inicial

📝 Butlletí UO5 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, prem «executar» i deixa que els teus mètodes facin la feina pesada. Un `main` amb tot dins està bé… fins que deixa de cabre.

Exercici 1: La salutació oficial

Escriu un programa anomenat `Saludo` que tingui un mètode `public static void saludar()` que imprimeixi `¡Hola, Java!`. El `main` ha de cridar `saludar()` tres vegades, una per línia.

Pista: el mètode viu fora del `main`; el `main` simplement l'invoca pel seu nom amb parèntesis i punt i coma.

Exercici 2: Què passa? — la variable fantasma

Aquest programa no compila. Mira'l amb calma i respon: per què falla i què canviaries per arreglar-ho?

```
public class Fantasma {
    static int puntos = 10;

    public static void sumar() {
        int extra = 5;
        puntos += extra;
    }

    public static void main(String[] args) {
        sumar();
        System.out.println(puntos);
        System.out.println(extra);
    }
}
```

Pista: `extra` neix i mor dins de `sumar()`. El compilador no la ve des de `main`.

Exercici 3: La fitxa de presentació

Escriu un programa anomenat `Ficha` amb un mètode `public static void presentar(String nombre, int edad)` que imprimeixi:

```
Em dic Ana i tinc 20 anys.
```

El `main` ha de cridar el mètode amb `("Ana", 20)` i amb `("Luis", 15)`, cada crida en la seva línia.

Exercici 4: El primer `return`

Escriu un programa anomenat `Suma` amb un mètode `public static int sumar(int a, int b)` que **torna** la suma en lloc d'imprimir-la.

En el `main`, guarda el resultat de `sumar(3, 4)` en una variable i imprimeix `3 + 4 = 7`. Després imprimeix el resultat de `sumar(10, 20)` directament: `10 + 20 = 30`.

Pista: `return a + b;` entrega el valor a qui l'ha cridat; encara no l'imprimeix ningú.

Exercici 5: Doble, si us plau

Escriu un programa anomenat `Doble` amb **dos** mètodes:

- `public static int calcularDoble(int n)` → torna el doble de `n`.
- `public static void mostrarDoble(int n)` → imprimeix `El doble de 6 és 12`.

El `main` ha d'usar els dos amb el valor `6`. Fixa't en la diferència: un torna, l'altre parla.

Exercici 6: Què imprimeix? — el viatge d'anada i tornada

Sense executar, escriu la sortida exacta, línia a línia:

```java
public class Viaje {
    static int sumar(int a, int b) {
        System.out.println("sumant...");
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("abans");
        int total = sumar(3, 4);
        System.out.println("total: " + total);
    }
}
```

Pista: el programa no salta de forma màgica: es para en la crida, executa el mètode i torna amb el valor en la mà.

Exercici 7: Parell o senar?

Escriu un programa anomenat `ParOImpar` amb un mètode `public static boolean esPar(int n)` que torni `true` si el nombre és parell.

El `main` ha de comprovar el `7` i el `12` amb un `if/else` i imprimir `7 és senar` i `12 és pare`.

Pista: un sol `return` amb l'operador `%` de la U03.

Exercici 8: La porta de l'edat

Escriu un programa anomenat `Puerta` amb un mètode `public static boolean mayorDeEdad(int edad)` que torni `true` si la persona té 18 anys o més.

El `main` ha de comprovar `15`, `18` i `30` i imprimir, una per línia:

```
15: no entra
18: entra
30: entra
```

Pista: pots usar un ternari en cada `println`, com en l'exercici 6 dels extres de la U04.

Exercici 9: CodeWars — Century From Year

Resol la kata **“Century From Year”** (8 kyu) a CodeWars.

Completa el mètode `public static int century(int year)` que torna el segle al qual pertany un any: el primer segle va de l’any 1 al 100, el segon del 101 al 200, i així successivament.

Exemples: `century(1705)` → `18`, `century(1900)` → `19`, `century(1601)` → `17`, `century(89)` → `1`.

Pista: la divisió entera se menja l’any 1601 (`1601 / 100` és `16`); ajusta restant 1 abans de dividir o arredonint cap amunt amb `Math.ceil`.

Butlletí UO5 — Inicial Resolt
📝 Butlletí UO5 — Inicial (Resolt)

> Les solucions estan ocultes en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: La salutació oficial

> 🔄 Solució

```
public class Saludo {
    public static void saludar() {
        System.out.println("¡Hola, Java!");
    }

    public static void main(String[] args) {
        saludar();
        saludar();
        saludar();
    }
}
```

Sortida:

```
¡Hola, Java!
¡Hola, Java!
¡Hola, Java!
```

La recepta s'escriu una vegada, fora del `main`, i s'usa tres vegades. Eix exactament el negoci dels mètodes: escriure una vegada, cridar les vegades que faci falta.

Exercici 2: Què passa? — la variable fantasma

> 🔄 Solució

El programa no compila perquè `extra` és una **variable local** de `sumar()`: el seu àmbit acaba en el claudàtor que tanca el mètode. En la línia `System.out.println(extra);` el compilador

escup un `cannot find symbol` (no troba el símbol), perquè des de `main` aquella variable mai ha existit.

Com arreglar-ho (qualsevol de les dues):

- Imprimir `extra` **dins** de `sumar()`, on sí viu.
- Passar-la com a paràmetre o tornar-la amb `return` si `main` la necessita.

La lliçó: cada mètode és una casa amb porta. El que es deixa dins no surt sense invitació.

Exercici 3: La fitxa de presentació

> 🔄 **Solució**

```
public class Ficha {
    public static void presentar(String nombre, int edad) {
        System.out.println("Em dic " + nombre + " i tinc " + edad + " anys.");
    }

    public static void main(String[] args) {
        presentar("Ana", 20);
        presentar("Luis", 15);
    }
}
```

Sortida:

```
Em dic Ana i tinc 20 anys.
Em dic Luis i tinc 15 anys.
```

Un mateix mètode, dos crides, dues persones diferents: els paràmetres són les entrades que fan genèric el mètode. Sense ells hauries de copiar el `println` dues vegades.

Exercici 4: El primer `return`

> 🔄 **Solució**

```java
public class Suma {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int total = sumar(3, 4);
        System.out.println("3 + 4 = " + total);
        System.out.println("10 + 20 = " + sumar(10, 20));
    }
}
```

Sortida:

```
3 + 4 = 7
10 + 20 = 30
```

`return` talla el mètode i entrega el valor a qui l'ha cridat. El primer el guardes en `total`; el segon l'uses directament dins del `println`. Imprimir és cosa de qui rep, no de qui torna.

Exercici 5: Doble, si us plau

› 🔄 **Solució**

```java
public class Doble {
    public static int calcularDoble(int n) {
        return n * 2;
    }

    public static void mostrarDoble(int n) {
        System.out.println("El doble de " + n + " és " + n * 2);
    }

    public static void main(String[] args) {
        int doble = calcularDoble(6);
        System.out.println(doble);
        mostrarDoble(6);
    }
}
```

Sortida:

```
12
El doble de 6 és 12
```

`calcularDoble` torna un `int` que pots guardar, comparar o usar en un altre càlcul. `mostrarDoble` és `void`: no torna res, només imprimeix. La regla d'or: el que calcula no imprimeix, i el que imprimeix no calcula.

Exercici 6: Què imprimeix? — el viatge d'anada i tornada

› 🔄 Solució

```
abans
sumant...
total: 7
```

L'ordre és fidel al viatge: el `main` imprimeix `abans`, es para en `sumar(3, 4)`, salta al mètode (que imprimeix `sumant...` i torna `7`), torna al `main` amb el resultat i continua amb l'últim `println`. Si et vas saltar `sumant...`, recorda: res s'executa «en paral·lel»; Java va de dalt a baix i d'anada i tornada, sense atalls.

Exercici 7: Parell o senar?

› 🔄 Solució

```java
public class ParOImpar {
    public static boolean esPar(int n) {
        return n % 2 == 0;
    }

    public static void main(String[] args) {
        if (esPar(7)) {
            System.out.println("7 és senar");
        } else {
            System.out.println("7 és pare");
        }
        if (esPar(12)) {
            System.out.println("12 és pare");
        } else {
            System.out.println("12 és senar");
        }
    }
}
```

Sortida:

```
7 és senar
12 és pare
```

`esPar` torna un `boolean`, així que pot viure directament en la condició del `if`. És la manera elegant de preguntar: sense guardar el resultat en una variable intermèdia.

Exercici 8: La porta de l'edat

› 🔄 Solució

```java
public class Puerta {
    public static boolean mayorDeEdad(int edad) {
        return edad >= 18;
    }

    public static void main(String[] args) {
        System.out.println("15: " + (mayorDeEdad(15) ? "entra" : "no entra"));
        System.out.println("18: " + (mayorDeEdad(18) ? "entra" : "no entra"));
        System.out.println("30: " + (mayorDeEdad(30) ? "entra" : "no entra"));
    }
}
```

Sortida:

```
15: no entra
18: entra
30: entra
```

La regla viu en un sol lloc (`mayorDeEdad`) i s'aplica a les tres edats. Si demà el límit canvia a 21, toques **un** `return` i tot el programa se n'assabenta. Aquell és el poder de no repetir la lògica.

Exercici 9: CodeWars — Century From Year

› 🔄 **Solució**

```java
public class Kata {
    public static int century(int year) {
        return (year - 1) / 100 + 1;
    }
}
```

Dos camins:

- `(year - 1) / 100 + 1`: el `-1` fa que l'any 100 caiga en el segle 1 i el 101 en el 2.
- `Math.ceil(year / 100.0)`: arrodoneix cap amunt el resultat decimal (`1705 / 100.0` és `17.05`, i `Math.ceil` el puja a `18`). Ojo: si divideixes en enter (`year / 100`) perds el resta i l'any 1601 cauria en el segle 16.

> Un mètode, dues línies, zero bucles: de vegades la millor solució és la que no es complica.

Butlletí U05 — Avançat

📝 Butlletí U05 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: El classificador de notes

Escriu un programa amb un mètode `public static String qualificar(double nota)` que torni la qualificació textual amb les mateixes regles de la U04: `>= 9` → "Excel·lent", `>= 7` → "Notable", `>= 5` → "Aprovat", `>= 0` → "Suspés", i fora de rang → "Nota no vàlida".

El `main` ha d'imprimir el resultat de `qualificar(8.7)`, `qualificar(4.5)`, `qualificar(11)` i `qualificar(-2)`, un per línia.

Pista: el mètode **torna** cadenes amb `return`, no imprimeix res. Comprova primer si la nota està fora de rang.

⭐ Exercici 2: Taules a demanda

Escriu un mètode `public static void imprimirTabla(int n)` que imprimeixi la taula de multiplicar del `n` (de l'1 al 10), una línia per cada multiplicació.

El `main` ha de demanar-li la taula del 7, després un separador `---`, i després la taula del 3.

Pista: el mètode no sap res del 7 ni del 3: només coneix el seu paràmetre `n`. Un `for` i a volar.

⭐⭐ Exercici 3: Què imprimeix? — l'ombra del paràmetre

Sense executar, escriu la sortida exacta:

```java
public class Sombra {
    static int valor = 100;

    public static void cambiar(int valor) {
        valor = valor + 1;
    }

    public static void main(String[] args) {
        cambiar(5);
        System.out.println(valor);
    }
}
```

Pista: dins de `cambiar`, el paràmetre `valor` tapa al camp `valor`. Quin dels dos canvia amb `valor + 1`?

⭐⭐ Exercici 4: La suma de xifres

Escriu un mètode `public static int sumaDigitos(int n)` que torni la suma de les xifres de `n` (per exemple, `1234` → `10`).

El `main` ha de mostrar `sumaDigitos(1234)` i `sumaDigitos(905)`.

Pista: `n % 10` treu l'última xifra i `n / 10` se la menja; repeteix amb un `while` fins que `n` sigui 0. Si el nombre pot ser negatiu, usa `Math.abs(n)` al principi.

⭐⭐⭐ Exercici 5: Què imprimeix? — el return que talla

Sense executar, escriu la sortida exacta:

```java
public class Corte {
    public static String signo(int n) {
        if (n < 0) {
            return "negatiu";
        }
        if (n == 0) {
            return "zero";
        }
        return "positiu";
    }

    public static int primeroPar(int a, int b) {
        if (a % 2 == 0) {
            return a;
        }
        if (b % 2 == 0) {
            return b;
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(signo(-5) + " " + signo(0) + " " + signo(4));
        System.out.println(primeroPar(7, 4));
        System.out.println(primeroPar(3, 5));
    }
}
```

Pista: en cada mètode només s'executa **un** `return`: la resta del codi queda mort. Recorre els casos un a un.

⭐⭐⭐ Exercici 6: Divideix l'informe

Aquest programa funciona, però tot viu dins del `main`:

```java
public class Informe {
    public static void main(String[] args) {
        double a = 7.5, b = 9.0, c = 4.5, d = 8.25;

        double media = (a + b + c + d) / 4;
        double maximo = a;
        if (b > maximo) maximo = b;
        if (c > maximo) maximo = c;
        if (d > maximo) maximo = d;
        double minimo = a;
        if (b < minimo) minimo = b;
        if (c < minimo) minimo = c;
        if (d < minimo) minimo = d;

        System.out.println("Mitjana: " + media);
        System.out.println("Màxim: " + maximo);
        System.out.println("Mínim: " + minimo);
    }
}
```

Trosseja-lo en **tres mètodes** que tornen `double`: `media(a, b, c, d)`, `maximo(a, b, c, d)` i `minimo(a, b, c, d)`. El `main` ha de quedar-se en tres línies de `println` que criden als mètodes.

Pista: els tres mètodes són `public static double ...` amb `return` i cap crida a `System.out`. Comparar `a, b, c, d` és exactament el mateix dins de cadascun.

⭐⭐ Exercici 7: El comptador de vocals

Escriu un mètode `public static int contarVocales(String texto)` que torni quantes vocals hi ha en `texto` (sense distingir majúscules de minúscules).

El `main` ha d'imprimir `contarVocales("Hola mundo")` (dona `4`) i `contarVocales("zzz")` (dona `0`).

Pista: recorre amb `for (int i = 0; i < texto.length(); i++)`, mira cada caràcter amb `charAt(i)` i compara'l amb les cinc vocals en les seves dues versions.

⭐ ⭐ ⭐ Exercici 8: CodeWars — 'Disemvowel' Trolls

Resol la kata **"'Disemvowel' Trolls"** (7 kyu) a CodeWars.

Crea el mètode `public static String disemvowel(String str)` que torni la cadena sense vocals.

Exemple: `"This website is for losers LOL!"` → `"Ths wbst s fr lsrs LL!"`.

Pista: recorre amb `charAt` i concatena només el que no sigui vocal. La solució amb regex arriba en la U13; aquí ens val un `if` amb moltes comparacions.

⭐ ⭐ ⭐ Exercici 9: AceptaElReto — 165 Número hyperpar

Resol el problema **165 — Número hyperpar** a AceptaElReto.com.

Un nombre és *hyperpar* quan **totes** les seves xifres són parells. L'entrada conté un nombre per línia fins a un nombre negatiu (que no es processa). Per a cada cas, mostra `SI` o `NO`.

Exemple:

```
2460
1234
2
-1
```

```
SI
NO
SI
```

Pista: embolica la comprovació en un mètode `public static boolean esHyperpar(int n)`: amb un `while` treu xifres amb `n % 10` i, si alguna és senar, `return false` en quant la trobes. Si el bucle acaba sense trobar-ne cap, `return true`.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	165 — Número hyperpar	Fàcil
AceptaElReto	115 — Número de Kaprekar	Mitjà
CodeWars	Century From Year (8 kyu)	Principiant
CodeWars	‘Disemvowel’ Trolls (7 kyu)	Aficionat
CodeWars	Volume of a Cuboid (8 kyu)	Principiant
CodeWars	Drink about (8 kyu)	Principiant

Butlletí UO5 — Avançat Resolt

📝 Butlletí UO5 — Avançat (Resolt)

> Les solucions estan ocultes. Intenta-ho de veritat abans de destapar-les.

⭐ Exercici 1: El classificador de notes

▸ 🔄 Solució

```
public class Notas {
    public static String calificar(double nota) {
        if (nota < 0 || nota > 10) {
            return "Nota no vàlida";
        }
        if (nota >= 9) {
            return "Excel·lent";
        }
        if (nota >= 7) {
            return "Notable";
        }
        if (nota >= 5) {
            return "Aprovat";
        }
        return "Suspés";
    }

    public static void main(String[] args) {
        System.out.println(calificar(8.7));
        System.out.println(calificar(4.5));
        System.out.println(calificar(11));
        System.out.println(calificar(-2));
    }
}
```

Sortida:

```
Notable
Suspés
Nota no vàlida
Nota no vàlida
```

La guarda de rang (`< 0 || > 10`) va **primera**: si no, un `11` seria "Excel·lent" i un `-2` entraria en la cascada. Cada `return` talla el mètode, així que els rangs posteriors només es miren si els anteriors no han fet *match*.

⭐ Exercici 2: Taules a demanda

› 🔄 Solució

```java
public class Tablas {
    public static void imprimirTabla(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }

    public static void main(String[] args) {
        imprimirTabla(7);
        System.out.println("---");
        imprimirTabla(3);
    }
}
```

Sortida (primers):

```
7 x 1 = 7
7 x 2 = 14
...
---
3 x 1 = 3
...
```

El mètode no en sap res del 7 ni del 3: tot el que fa depèn de `n`. La mateixa recepta, dos taules; afegir una tercera és una línia més en el `main`.

⭐⭐ Exercici 3: Què imprimeix? — l'ombra del paràmetre

> 🔄 Solució

```
100
```

Dins de `cambiar`, el paràmetre `valor` **tapa** al camp `valor` durant tot el mètode (per això se'n diu *ombra*). L'expressió `valor = valor + 1` treballa sobre el paràmetre local: `5 + 1` es guarda en el paràmetre, que desapareix en tornar. El camp de la classe mai es toca, així que `main` continua imprimint `100`. Per a modificar el camp caldria `Sombra.valor = valor + 1` (sense `this` no hi ha `static`).

⭐⭐ Exercici 4: La suma de xifres

> 🔄 Solució

```java
public class Digitos {
    public static int sumaDigitos(int n) {
        n = Math.abs(n);
        int suma = 0;
        while (n > 0) {
            suma += n % 10;
            n /= 10;
        }
        return suma;
    }

    public static void main(String[] args) {
        System.out.println(sumaDigitos(1234));
        System.out.println(sumaDigitos(905));
    }
}
```

Sortida:

```
10
14
```

Amb `1234`: el `while` treu `4`, després `3`, després `2`, després `1` → `4 + 3 + 2 + 1 = 10`. `Math.abs` deixa a salvo el mètode d'un nombre negatiu. Tot el treball passa dins del mètode i `main` només imprimeix el `return`.

⭐⭐⭐ Exercici 5: Què imprimeix? — el return que talla

› 🔄 Solució

```
negatiu zero positiu
4
-1
```

`signo(-5)` cau en el primer `if` i torna `negatiu`; `signo(0)` supera el primer i cau en el segon → `zero`; `signo(4)` arriba fins a l'últim `return` → `positiu`. `primeroPar(7, 4)`: el 7 és senar, el 4 és pare → torna `4` abans d'arribar a l'últim `return`. `primeroPar(3, 5)`: cap és pare, així que arriba al final i torna `-1` (el valor sentinella). Recórrer amb calma cada crida és tot l'exercici: un `return` per execució, la resta és mort.

⭐⭐⭐ Exercici 6: Divideix l'informe

› 🔄 Solució

```java
public class Informe {
    public static double media(double a, double b, double c, double d) {
        return (a + b + c + d) / 4;
    }

    public static double maximo(double a, double b, double c, double d) {
        double max = a;
        if (b > max) max = b;
        if (c > max) max = c;
        if (d > max) max = d;
        return max;
    }

    public static double minimo(double a, double b, double c, double d) {
        double min = a;
        if (b < min) min = b;
        if (c < min) min = c;
        if (d < min) min = d;
        return min;
    }

    public static void main(String[] args) {
        System.out.println("Mitjana: " + media(7.5, 9.0, 4.5, 8.25));
        System.out.println("Màxim: " + maximo(7.5, 9.0, 4.5, 8.25));
        System.out.println("Mínim: " + minimo(7.5, 9.0, 4.5, 8.25));
    }
}
```

Sortida:

```
Mitjana: 7.3125
Màxim: 9.0
Mínim: 4.5
```

Cada concepte té la seva caixa i el seu nom: si demà l'informe rep mitja ponderada, toques **un** mètode. El `main` ha quedat en el que ha de ser: presentar resultats, pas de càlcul zero.

⭐⭐ Exercici 7: El comptador de vocals

› 🔄 Solució

```java
public class Vocales {
    public static int contarVocales(String texto) {
        int contador = 0;
        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
                    || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c ==
'U') {
                contador++;
            }
        }
        return contador;
    }

    public static void main(String[] args) {
        System.out.println(contarVocales("Hola mundo"));
        System.out.println(contarVocales("zzz"));
    }
}
```

Sortida:

```
4
0
```

`"Hola mundo"` té o, a, u, o → `4`. Les 10 comparacions són el preu de no voler encara `toLowerCase()` ni les *regex* de la U13. Quan el criteri sigui més lleuger que la llista, refactoritzaràs en una sola línia.

⭐ ⭐ ⭐ Exercici 8: CodeWars — 'Disemvowel' Trolls 🔗

› 🔄 Solució

```java
public class Kata {
    public static String disemvowel(String str) {
        String resultado = "";
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            if (c != 'a' && c != 'e' && c != 'i' && c != 'o' && c != 'u'
                    && c != 'A' && c != 'E' && c != 'I' && c != 'O' && c != 'U') {
                resultado += c;
            }
        }
        return resultado;
    }
}
```

Mateix motiu que l'exercici anterior, però a l'inversa: en lloc de comptar el que t'interessa, **concatenes el que no és vocal**. `"This website is for losers LOL!"` perd les vocals i torna `"Ths wbst s fr lsrs LL!"`. Cadenes immutables i += en bucle no són el més ràpid del món, però per aquesta kata és més que suficient.

⭐⭐⭐ Exercici 9: AceptaElReto — 165 Número hyperpar

› 🔄 Solució

```java
import java.util.Scanner;

public class Hiperpar {
    public static boolean esHyperpar(int n) {
        while (n > 0) {
            if (n % 10 % 2 != 0) {
                return false;
            }
            n /= 10;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while (n >= 0) {
            System.out.println(esHyperpar(n) ? "SI" : "NO");
            n = sc.nextInt();
        }
        sc.close();
    }
}
```

Sortida per a l'exemple:

```
SI
NO
SI
```

`2460` → totes les xifres parelles → `true`. `1234` → en trobar la `1`, `1 % 2 != 0` → `false` i sortim de pressa. `2` → un sol salt de bucle → `true`. El `return false` anticipat és la idea clave de la unitat: quan ja saps la resposta, no cal acabar la feina. El `while` exterior del `main` llegeix fins que arriba un nombre negatiu.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	165 — Número hyperpar	Fàcil
AceptaElReto	115 — Número de Kaprekar	Mitjà
CodeWars	Century From Year (8 kyu)	Principiant
CodeWars	‘Disemvowel’ Trolls (7 kyu)	Aficionat
CodeWars	Volume of a Cuboid (8 kyu)	Principiant
CodeWars	Drink about (8 kyu)	Principiant

Butlletí U05 — Extres

📝 Butlletí U05 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està oculta: resisteix-te fins a esgotar la teva pista.

CodeWars

1. L1: Set Alarm

Un booleà `employed` et diu si treballes i un altre, `vacation`, si estàs de vacances. Torna `true` **només** si treballes i no estàs de vacances.

Exemples: `setAlarm(true, false)` → `true`, `setAlarm(false, false)` → `false`, `setAlarm(true, true)` → `false`, `setAlarm(false, true)` → `false`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: una sola condició amb `&&` i un `!` davant de les vacances.

> 🔄 **Solució**

```
public class Kata {
    public static boolean setAlarm(boolean employed, boolean vacation) {
        return employed && !vacation;
    }
}
```

`employed && !vacation` és la traducció literal de «treballes i no estàs de vacances». Un `return` d'una línia, sense `if`: quan la condició ja és un `boolean`, el ternari o el `if` sobren.

---

2. Beginner Series #2 Clock

El rellotge marca `h` hores, `m` minuts i `s` segons des de la mitjanit. Escribeu el mètode `Past`, que torna aquell temps en **mil·lisegons**.

Exemples: `Past(0, 1, 1)` → `61000`, `Past(1, 0, 1)` → `3601000`, `Past(0, 0, 0)` → `0`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: 1 hora = 3.600.000 ms, 1 minut = 60.000 ms i 1 segon = 1.000 ms. Suma les tres parts i multiplica al final.

> 🔄 **Solució**

```
public class Clock {
    public static int Past(int h, int m, int s) {
        return (h * 60 * 60 + m * 60 + s) * 1000;
    }
}
```

`h * 60 * 60 + m * 60 + s` deixa tot en segons dins d'un parèntesi i un sol `* 1000` el converteix a mil·lisegons. El valor màxim possible (23:59:59 = 86.399 segons → 86.399.000 ms) cap sobrat en un `int`, que arriba fins a 2.147.483.647.

---

3. Volume of a Cuboid

El mètode `getVolumeOfCuboid` rep llargada, amplada i alçada d'una caixa i torna el seu volum.

Exemples: `(1, 2, 3)` → `6`, `(6, 2, 1)` → `12`, `(5, 3, 2)` → `30`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: el volum d'un paral·lelepípede és amplada × alçada × llargada. Un `return`, tres multiplicacions, ni un `if`.

> 🔄 **Solució**

```
public class Kata {
    public static int getVolumeOfCuboid(int length, int width, int height) {
        return length * width * height;
    }
}
```

> El mètode més curt de tot el butlletí: rep tres entrades, torna una sortida. Fixa't en el patró de la signatura `public static` que ja és rutinari: sense `static`, CodeWars (igual que el teu `main`) no podria cridar-lo.

---

4. Drink about

Donada una edat, torna quina beguda correspon: menys de 14 → `"drink toddy"`, menys de 18 → `"drink coke"`, menys de 21 → `"drink beer"` i a partir de 21 → `"drink whisky"`.

Exemples: `10` → `"drink toddy"`, `15` → `"drink coke"`, `20` → `"drink beer"`, `30` → `"drink whisky"`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: són quatre `return` encadenats amb `if`, de l'edat més petita a la major, com la cascada de notes de la U04.

> 🔄 **Solució**

```java
public class Kata {
    public static String peopleWithAgeDrink(int old) {
        if (old < 14) {
            return "drink toddy";
        }
        if (old < 18) {
            return "drink coke";
        }
        if (old < 21) {
            return "drink beer";
        }
        return "drink whisky";
    }
}
```

La mateixa cascada de rangs de sempre, però tornant en lloc d'imprimir. L'ordre importa: si comprovaras `old < 21` primer, un xiquet de 10 anys beuria cervesa.

AceptaElReto

5. 116 — Hola món!

A la primera classe de qualsevol curs de programació s'hauria de sortir havent escrit un «hola món». Doncs això: donat un nombre `n` (`0 ≤ n ≤ 5`), escriu `n` línies amb la cadena `Hola mundo.` (el text de la sortida és exactament aquell, tal com demana el jutge).

Exemple:

```
3
```

Sortida:

```
Hola mundo.
Hola mundo.
Hola mundo.
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: un mètode `public static void imprimirHolaMundo(int n)` amb un `for` de `0` a `n - 1`, i un `main` que només llegeix i crida.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class HolaMundo {
    public static void imprimirHolaMundo(int n) {
        for (int i = 0; i < n; i++) {
            System.out.println("Hola mundo.");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        imprimirHolaMundo(n);
        sc.close();
    }
}
```

`imprimirHolaMundo(0)` no imprimeix res (el `for` no arranca) i `imprimirHolaMundo(5)` repeteix la línia cinc vegades. El mètode encapsula la repetició: el jutge demana un nombre, el teu mètode sap repetir. Programar en equip, amb un per àrea.

---

6. 115 — Número de Kaprekar

Un nombre és de *Kaprekar* si el seu quadrat pot partir-se en dos nombres que sumats tornen l'original, i el segon de la partida no pot ser zero. Per exemple: $9^2 = 81 \rightarrow 8 + 1 = 9$ (¡és Kaprekar!) i $703^2 = 494209 \rightarrow 494 + 209 = 703$. En canvi, `100` no ho és: `10000` podria partir-se en `100 + 00`, però `00` val zero.

Entrada: diversos nombres (≥ 1 i < 65536) fins a un `0` que marca el final.

Exemple:

```
22222
75
99
100
0
```

Sortida:

```
SI
NO
SI
NO
```

- Enunciat en AceptaElReto
- Dificultat: Mitjà

Pista: embolica-ho tot en un `public static boolean esKaprekar(int n)`. El quadrat pot passar-se de `int` (usa `long`), i has de provar **tots** els talls possibles: `p = 10, 100, 1000, ...` fins a superar el quadrat, amb `izq = cuadrado / p` i `der = cuadrado % p`. Si `der != 0` i `izq + der == n`, has guanyat.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Kaprekar {
    public static boolean esKaprekar(int n) {
        long cuadrado = (long) n * n;
        for (long p = 10; p <= cuadrado * 10; p *= 10) {
            long izquierda = cuadrado / p;
            long derecha = cuadrado % p;
            if (derecha != 0 && izquierda + derecha == n) {
                return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while (n != 0) {
            System.out.println(esKaprekar(n) ? "SI" : "NO");
            n = sc.nextInt();
        }
        sc.close();
    }
}
```

> Comprovem amb `9`: `cuadrado = 81`, amb `p = 10` → `izq = 8`, `der = 1`, i `8 + 1 = 9` → `SI`. Amb `100`: cap tall dona `100` sense que `der` sigui `0` → `NO`. El tall extra `p <= cuadrado * 10` és el que permet que l'esquerra quedi buida, just el que necessita el `1` (`1² = 1` → `0 + 1 = 1` → `SI`). Un `for` de talls i un `long` a salvaguarda de desbordaments: pur múscul d'aquesta unitat.

> 🧭 **I si et quedes amb ganes?** Amb els mètodes a la mà pots reescriure els teus programes de la U04 trossejant cada bloc en una funció amb nom. I si et ve de gust un repte de futur, la U08 t'ensenyarà a fer que un mètode es cridi a ell mateix (recursió): el joc de la caixa russa del codi.

Butlletí U06 — Inicial

📝 Butlletí U06 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, crea el teu primer aparcament de dades i fes que el `for-each` deixi de semblar màgia. Cap plaça no neix sabent tenir amo.

Exercici 1: Què imprimeix? — Array de booleans

```
boolean[] flags = new boolean[3];
flags[1] = true;
System.out.println(flags[0] + " " + flags[1] + " " + flags[2]);
```

Què imprimeix? Quin és el valor per defecte d'un `boolean` en un array?

Exercici 2: Troba l'error — NullPointerException

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
System.out.println(nombres[2].toUpperCase());
```

Què ocorre en executar aquest codi? Per què?

Exercici 3: Completa el codi — for bàsic per a buscar el major

Completa el següent programa perquè trobi i imprimeixi el número més gran de l'array:

```java
int[] numeros = {12, 45, 7, 34, 89, 23};
int mayor = numeros[0];

for (int i = 1; i < ______; i++) {   // fins on arriba el bucle?
    if (numeros[i] ______ mayor) {    // quin operador?
        ______ = numeros[i];          // què assignem?
    }
}

System.out.println("El mayor es: " + mayor);
```

Exercici 4: Escriu aquest programa — comptar números parells

Crea un array de 10 enters amb valors que tria tu. Recórrel amb un bucle `for` i compta quants d'ells són parells. Al final, imprimeix el total de parells i l'array original amb `Arrays.toString`.

Exemple de sortida:

```
Array: [3, 8, 12, 5, 7, 10, 2, 9, 6, 1]
Pares: 5
```

Pista: un número és parell si `numeros[i] % 2 == 0`.

Exercici 5: Troba l'error — length vs length()

```java
int[] numeros = {10, 20, 30};
String texto = "Hola";

System.out.println(numeros.length());
System.out.println(texto.length);
```

Quines línies tenen error? Explica la diferència entre `length` (sense parèntesis) i `length()` (amb parèntesis).

Exercici 6: Què imprimeix? — la suma dels senars

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class SumaImpares {
    public static void main(String[] args) {
        int[] datos = {3, 8, 2, 9, 5};
        int total = 0;

        for (int n : datos) {
            if (n % 2 == 1) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

Pista: el `for-each` recorre tots els valors; només se sumen els que deixen resta 1 en dividir entre 2.

Exercici 7: Escriu aquest programa — cerca lineal

Crea un array d'enters anomenat `edades` amb 8 valors. Demana a l'usuari un número pel teclat (amb `Scanner`) i busca si aquell número està a l'array. Imprimeix «Encontrado en posición X» o «No encontrado».

```
Introduce edad a buscar: 25
Encontrado en posición 3
```

Pista: usa una variable `posicion = -1` com a "no trobat", i `break` tan bon punt el trobes.

Exercici 8: Escriu aquest programa — l'invers

Crea un array de 10 enters, ompli'l amb els números de l'1 al 10 i després imprimeix-lo en **ordre invers** (del 10 a l'1). Fes-ho amb un `for` que recorri l'array cap enrere.

Pista: el bucle va de `length - 1` fins a `0`, baixant amb `i--`.

Exercici 9: Escriu aquest programa — la classe Arrays en acció

Crea l'array `int[] notas = {7, 3, 9, 5, 2, 8}` i fes el següent:

1. Mostra'l amb `Arrays.toString`.
2. Ordena amb `Arrays.sort` i mostra'l una altra vegada.
3. Busca el `8` amb `Arrays.binarySearch` i imprimeix la seva posició.

Pista: `import java.util.Arrays;` al principi, i recorda: `binarySearch` només és fiable si l'array ja està ordenat.

Butlletí U06 — Inicial Resolt

📝 Butlletí U06 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què imprimeix? — Array de booleans

> 🔄 Solució

Imprimeix **`false true false`**.

`flags` és un `boolean[]` de 3 places acabades de crear. El valor per defecte de `boolean` és `false`, així que `flags[0]` i `flags[2]` valen `false`. Només `flags[1]` es va posar a `true`. Cada plaça neix amb el valor per defecte del seu tipus: `false` per a `boolean`.

Exercici 2: Troba l'error — NullPointerException

> 🔄 Solució

Es llança una **`NullPointerException`** a l'última línia.

`nombres[2]` mai no es va assignar, així que val `null` (el valor per defecte dels objectes). Cridar `nombres[2].toUpperCase()` sobre `null` és demanar-li un mètode al no-res: Java no sap què fer i llança l'excepció. Les places d'un `String[]` acabat de crear estan plenes de `null`, no de `""`.

Exercici 3: Completa el codi — for bàsic per a buscar el major

> 🔄 Solució

```java
int[] numeros = {12, 45, 7, 34, 89, 23};
int mayor = numeros[0];

for (int i = 1; i < numeros.length; i++) {   // fins a length, sense passar
    if (numeros[i] > mayor) {                // és més gran que l'actual?
        mayor = numeros[i];                  // actualitza el major
    }
}

System.out.println("El mayor es: " + mayor);
```

El patró del “màxim acumulat”: comences assumint que el primer és el major i, si n’apareix un de més gran, el substitueixes. El bucle comença en `i = 1` perquè el candidat inicial ja és `numeros[0]`. Imprimeix `El mayor es: 89`.

Exercici 4: Escriu aquest programa — comptar números parells

› 🔄 Solució

```java
import java.util.Arrays;

public class ContarPares {
    public static void main(String[] args) {
        int[] numeros = {3, 8, 12, 5, 7, 10, 2, 9, 6, 1};
        int pares = 0;

        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] % 2 == 0) {
                pares++;
            }
        }

        System.out.println("Array: " + Arrays.toString(numeros));
        System.out.println("Pares: " + pares);
    }
}
```

> Sortida: `Array: [3, 8, 12, 5, 7, 10, 2, 9, 6, 1]` i `Pares: 5`. Un número és parell si el seu residu en dividir entre 2 és 0 (`% 2 == 0`). I `Arrays.toString` és el que fa la sortida llegible.

Exercici 5: Troba l'error — length vs length()

> 🔄 **Solució**

Les **dues línies tenen error**, però per motius oposats:

- `numeros.length()` → els arrays usen `length` com a **atribut**, sense parèntesis. `numeros.length()` no compila.
- `texto.length` → els `String` usen `length()` com a **mètode**, amb parèntesis. `texto.length` no compila.

Regla d'or: **array → `length`; `String` → `length()`; col·leccions → `size()`.** Confondre'ls és la trampa favorita dels exàmens.

Exercici 6: Què imprimeix? — la suma dels senars

> 🔄 **Solució**

Imprimeix **`17`**.

El `for-each` recorre els 5 valors: 3, 8, 2, 9, 5. El `if` només suma els que són senars (`n % 2 == 1`): 3, 9 i 5. `3 + 9 + 5 = 17`. El 8 i el 2 són parells i s'ignoren.

Exercici 7: Escriu aquest programa — cerca lineal

> 🔄 **Solució**

```java
import java.util.Scanner;

public class BusquedaLineal {
    public static void main(String[] args) {
        int[] edades = {12, 45, 25, 67, 33, 18, 40, 21};
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce edad a buscar: ");
        int buscado = sc.nextInt();

        int posicion = -1;
        for (int i = 0; i < edades.length; i++) {
            if (edades[i] == buscado) {
                posicion = i;
                break;
            }
        }

        if (posicion >= 0) {
            System.out.println("Encontrado en posición " + posicion);
        } else {
            System.out.println("No encontrado");
        }
        sc.close();
    }
}
```

La cerca lineal recorre l'array de principi a fi. `posicion = -1` és el "no trobat"; si apareix el valor, guardes l'índex i talles amb `break` (ja no cal seguir).

Exercici 8: Escriu aquest programa — l'invers

› 🔄 Solució

```java
import java.util.Arrays;

public class Inverso {
    public static void main(String[] args) {
        int[] numeros = new int[10];
        for (int i = 0; i < numeros.length; i++) {
            numeros[i] = i + 1;
        }

        System.out.println("Original: " + Arrays.toString(numeros));

        System.out.print("Inverso: ");
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.print(numeros[i] + " ");
        }
    }
}
```

El primer bucle ompli de l'1 al 10. El segon recorre **cap enrere**: comença en `length - 1` (el 10) i baixa fins a 0 (l'1). Imprimeix `10 9 8 7 6 5 4 3 2 1`.

Exercici 9: Escriu aquest programa — la classe Arrays en acció

› 🔄 Solució

```java
import java.util.Arrays;

public class ArraysEnAccion {
    public static void main(String[] args) {
        int[] notas = {7, 3, 9, 5, 2, 8};

        System.out.println("Original: " + Arrays.toString(notas));

        Arrays.sort(notas);
        System.out.println("Ordenado: " + Arrays.toString(notas));

        int pos = Arrays.binarySearch(notas, 8);
        System.out.println("El 8 está en la posición " + pos);
    }
}
```

Sortida:

```
Original: [7, 3, 9, 5, 2, 8]
Ordenado: [2, 3, 5, 7, 8, 9]
El 8 está en la posición 4
```

`Arrays.sort` ordena “al lloc” (modifica l’array). Després `binarySearch` troba el 8 a l’índex 4. Si el buscares abans d’ordenar, el resultat seria impredictible.

Butlletí U06 — Avançat

📝 Butlletí U06 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ Exercici 1: La fusió d'arrays ordenats

Escriu un mètode `fusionarArrays(int[] a, int[] b)` que rebi dos arrays ordenats de menor a major i torni un **nou array** també ordenat amb tots els elements d'ambdós. No usis `Arrays.sort()`. Fes-ho amb l'algoritme de fusió (merge) tipus «dos punters».

Pista: avança amb dos índexs, un per array, comparant en cada pas quin element és menor. Quan un array s'acabi, copia la resta de l'altre.

⭐ Exercici 2: Rotació circular a la dreta

Implementa un mètode `rotarDerecha(int[] arr, int k)` que desplaci cada element de l'array `k` posicions cap a la dreta. Els elements que «surten» pel final tornen a entrar pel principi.

Exemple: `{1, 2, 3, 4, 5}` amb `k = 2` → `{4, 5, 1, 2, 3}`.

Pista: usa un array temporal de la grandària de `k` amb els últims `k` elements, desplaça la resta cap a la dreta i després col·loca els guardats al principi. (O divideix els índexs amb `(i + k) % arr.length`.)

⭐ Exercici 3: Suma de diagonals (matriu quadrada)

Crea un programa que generi una matriu quadrada `int[N][N]` amb valors aleatoris entre 1 i 100, i calculi:

1. Suma de la **diagonal principal** (de dalt-esquerra a baix-dreta).
2. Suma de la **diagonal secundària** (de dalt-dreta a baix-esquerra).

3. Diferència absoluta entre totes dues sumes.

Usa `N = 5` per a les proves i `Math.random()` per a omplir-la.

Pista: a la diagonal principal, `fila == columna`, així que un sol bucle amb `matriz[i][i]` basta. A la secundària, `columna = N - 1 - i`.

⭐⭐ Exercici 4: Està ordenat?

Escriu un mètode `public static boolean estaOrdenado(int[] arr)` que torni `true` si l'array està ordenat **de menor a major** (cada element és menor o igual que el següent), i `false` en cas contrari.

Proves:

- `estaOrdenado({1, 2, 3, 4})` → `true`
- `estaOrdenado({1, 3, 2, 4})` → `false`
- `estaOrdenado({})` → `true`
- `estaOrdenado({7})` → `true`

Pista: recorre amb un `for` de `i = 1` fins al final i pregunta si `arr[i] < arr[i - 1]`. En el moment que un veí trenqui l'ordre, torna `false`.

⭐⭐ Exercici 5: L'invers al lloc

Escriu un mètode `public static void invertir(int[] arr)` que doni la volta a l'array **sense crear un altre array** (usa dos punters i una variable temporal).

Prova amb `{1, 2, 3, 4, 5}` i mostra el resultat amb `Arrays.toString` → ha de sortir `[5, 4, 3, 2, 1]`.

Pista: `izquierda = 0` i `derecha = arr.length - 1`; mentre `izquierda < derecha`, intercanvia i mou tots dos cap al centre.

⭐⭐ Exercici 6: Què imprimeix? — el doble bucle que compta parelles

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class CuentaParejas {
    public static void main(String[] args) {
        int[] datos = {2, 4, 6};
        int contador = 0;

        for (int i = 0; i < datos.length; i++) {
            for (int j = i + 1; j < datos.length; j++) {
                if (datos[i] < datos[j]) {
                    contador++;
                }
            }
        }

        System.out.println(contador);
    }
}
```

Pista: compta les parelles `(i, j)` amb `i < j` on el primer és menor que el segon. Amb `{2, 4, 6}` totes les parelles ho compleixen. Quantes n'hi ha?

⭐⭐⭐ Exercici 7: Estadístiques de classe

Demana a l'usuari les notes de 20 alumnes amb `Scanner`, guarda-les en un `double[]` i calcula:

- La nota mitjana.
- La nota més alta i la més baixa.
- Quants alumnes van aprovar (nota >= 5).

Usa un bucle per a omplir, un altre per a sumar i el patró del màxim/mínim acumulat.

Pista: per al mínim comença amb la primera nota; per al màxim, també. Guarda les notes en un array i recórri'l diverses vegades: cada càlcul és un bucle.

⭐⭐⭐ Exercici 8: El gran repte — busca-mines simplificat

Crea un array bidimensional `boolean[5][5]` que representi un camp de mines. Col·loca 5 mines en posicions **aleatòries** (amb `Math.random()`). L'usuari introduïx coordenades `(fila, columna)` i el programa diu si hi ha mina o no. Si encerta una mina, el joc acaba mostrant «¡BOOM!».

Extres opcionals: digues quantes mines hi ha al voltant de la casella (mira les 8 veïnes) i acaba quan hagis comprovat totes les caselles sense mines.

Pista: comprova abans de mirar una veïna que la seva fila i la seva columna estiguin entre 0 i 4, o sortiràs de l'array amb `ArrayIndexOutOfBoundsException`. El joc usa un `while` que acaba en trepitjar una mina o en esgotar les caselles.

⭐⭐⭐ Exercici 9: Compactar — els zeros al final

Escriu un mètode `public static void compactar(int[] arr)` que mogui tots els zeros al final de l'array, **mantenint l'ordre** dels elements que no són zero.

Exemples:

- `{0, 3, 0, 1, 0, 2}` → `{3, 1, 2, 0, 0, 0}`
- `{1, 2, 3}` → `{1, 2, 3}`
- `{0, 0, 0}` → `{0, 0, 0}`

Pista: usa un segon índex `pos` que marqui on va el següent valor no zero. Recorre amb `i`, i quan `arr[i]` no sigui 0, copia'l a `arr[pos]` i puja `pos`. Al final, ompli de zeros des de `pos` fins al final.

Butlletí UO6 — Avançat Resolt

📝 Butlletí UO6 — Avançat (Resolt) 🔗

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: La fusió d'arrays ordenats 🔗

> 🔄 Solució

```java
public class Fusion {
    public static int[] fusionarArrays(int[] a, int[] b) {
        int[] resultado = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;

        while (i < a.length && j < b.length) {
            if (a[i] < b[j]) {
                resultado[k++] = a[i++];
            } else {
                resultado[k++] = b[j++];
            }
        }

        while (i < a.length) {
            resultado[k++] = a[i++];
        }
        while (j < b.length) {
            resultado[k++] = b[j++];
        }

        return resultado;
    }

    public static void main(String[] args) {
        int[] a = {1, 3, 5};
        int[] b = {2, 4, 6};
        int[] r = fusionarArrays(a, b);
         System.out.println(java.util.Arrays.toString(r)); // [1, 2, 3, 4, 5,
6]
    }
}
```

Dos punters (`i` i `j`) avancen per cada array comparant a cada pas. El menor dels dos es col·loca al resultat. Quan un array s'esgota, es copia la resta de l'altre. És el mateix algoritme que usa `merge` en el mergesort de la U08.

⭐ Exercici 2: Rotació circular a la dreta

› 🔄 Solució

```java
import java.util.Arrays;

public class Rotar {
    public static void rotarDerecha(int[] arr, int k) {
        int n = arr.length;
        if (n == 0) return;
        k = k % n; // si k >= n, sobren voltes completes

        int[] guardados = new int[k];
        for (int i = 0; i < k; i++) {
            guardados[i] = arr[n - k + i]; // els últims k
        }
        for (int i = n - 1; i >= k; i--) {
            arr[i] = arr[i - k];           // desplaça la resta a la dreta
        }
        for (int i = 0; i < k; i++) {
            arr[i] = guardados[i];         // els guardats al principi
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        rotarDerecha(arr, 2);
        System.out.println(Arrays.toString(arr)); // [4, 5, 1, 2, 3]
    }
}
```

Amb `{1,2,3,4,5}` i `k = 2`: guardes `{4,5}`, desplaces `{1,2,3}` a les posicions 2, 3, 4 i col·loques `{4,5}` al principi. El `k % n` evita donar vegades completes inútils si k és major que la grandària.

⭐ Exercici 3: Suma de diagonals (matriu quadrada)

› 🔄 Solució

```java
public class Diagonales {
    public static void main(String[] args) {
        int n = 5;
        int[][] matriz = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matriz[i][j] = (int) (Math.random() * 100) + 1;
            }
        }

        int diagPrincipal = 0;
        int diagSecundaria = 0;
        for (int i = 0; i < n; i++) {
            diagPrincipal += matriz[i][i];
            diagSecundaria += matriz[i][n - 1 - i];
        }

        int diferencia = Math.abs(diagPrincipal - diagSecundaria);
        System.out.println("Principal: " + diagPrincipal);
        System.out.println("Secundaria: " + diagSecundaria);
        System.out.println("Diferencia: " + diferencia);
    }
}
```

A la diagonal principal, `fila == columna` (`matriz[i][i]`). A la secundària, la columna és el "mirall": `n - 1 - i`. Un sol bucle recorre totes dues a la vegada. `Math.abs` dona la diferència absoluta, sense signe.

⭐⭐ Exercici 4: Està ordenat?

› 🔄 Solució

```java
public class EstaOrdenado {
    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < arr[i - 1]) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(estaOrdenado(new int[]{1, 2, 3, 4})); // true
        System.out.println(estaOrdenado(new int[]{1, 3, 2, 4})); // false
        System.out.println(estaOrdenado(new int[]{}));           // true
        System.out.println(estaOrdenado(new int[]{7}));          // true
    }
}
```

Comences en `i = 1` i compares cada element amb l'anterior: si algun és menor que el precedent, l'ordre està trencat. Amb array buit o d'un element, el bucle ni tan sols s'executa i torna `true` (no hi ha res desordenat).

⭐⭐ Exercici 5: L'invers al lloc

› 🔄 Solució

```java
import java.util.Arrays;

public class Invertir {
    public static void invertir(int[] arr) {
        int izquierda = 0;
        int derecha = arr.length - 1;

        while (izquierda < derecha) {
            int temp = arr[izquierda];
            arr[izquierda] = arr[derecha];
            arr[derecha] = temp;
            izquierda++;
            derecha--;
        }
    }

    public static void main(String[] args) {
        int[] datos = {1, 2, 3, 4, 5};
        invertir(datos);
        System.out.println(Arrays.toString(datos)); // [5, 4, 3, 2, 1]
    }
}
```

Els dos punters es mouen cap al centre intercanviant elements. Amb `{1,2,3,4,5}`: primer canvia 1↔5, després 2↔4, i quan es creuen (`izquierda >= derecha`) ja està. Només s'usa memòria per a la variable temporal `temp`.

⭐⭐ Exercici 6: Què imprimeix? — el doble bucle que compta parelles

› 🔄 **Solució**

Imprimeix **3**.

El bucle exterior va amb `i` per 0, 1 i 2. L'interior comença en `j = i + 1`:

- `i = 0`: parelles `(0,1)` i `(0,2)` → `2 < 4` i `2 < 6` → **2**.
- `i = 1`: parella `(1,2)` → `4 < 6` → **1**.
- `i = 2`: l'interior no entra (`j = 3` no és `< 3`).

Total: `2 + 1 = 3`. Amb `{2, 4, 6}` totes les parelles compleixen `datos[i] < datos[j]`, i hi ha exactament `3 · 2 / 2 = 3` parelles.

⭐⭐⭐ Exercici 7: Estadístiques de classe

› 🔄 Solució

```
import java.util.Scanner;

public class Estadisticas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double[] notas = new double[20];

        for (int i = 0; i < notas.length; i++) {
            System.out.print("Nota del alumno " + (i + 1) + ": ");
            notas[i] = sc.nextDouble();
        }

        double suma = 0;
        double max = notas[0];
        double min = notas[0];
        int aprobados = 0;

        for (double n : notas) {
            suma += n;
            if (n > max) max = n;
            if (n < min) min = n;
            if (n >= 5) aprobados++;
        }

        System.out.println("Media: " + (suma / notas.length));
        System.out.println("Máxima: " + max);
        System.out.println("Mínima: " + min);
        System.out.println("Aprobados: " + aprobados);
        sc.close();
    }
}
```

Un bucle ompli l'array, i el for-each fa la resta en una sola passada: acumula la suma, actualitza màxim i mínim i compta aprovats. Com `max` i `min` comencen en la primera nota, l'algoritme funciona fins i tot amb totes les notes iguals.

⭐⭐⭐ Exercici 8: El gran repte — busca-mines simplificat

› 🔄 Solució

```java
import java.util.Scanner;

public class Buscaminas {
    public static void main(String[] args) {
        boolean[][] minas = new boolean[5][5];
        int puestas = 0;
        while (puestas < 5) {
            int f = (int) (Math.random() * 5);
            int c = (int) (Math.random() * 5);
            if (!minas[f][c]) {
                minas[f][c] = true;
                puestas++;
            }
        }

        Scanner sc = new Scanner(System.in);
        boolean juegoActivo = true;

        while (juegoActivo) {
            System.out.print("Introduce fila y columna (0-4): ");
            int fila = sc.nextInt();
            int col = sc.nextInt();

            if (minas[fila][col]) {
                System.out.println("¡BOOM! Has pisado una mina.");
                juegoActivo = false;
            } else {
                int alrededor = 0;
                for (int i = fila - 1; i <= fila + 1; i++) {
                    for (int j = col - 1; j <= col + 1; j++) {
                        if (i >= 0 && i < 5 && j >= 0 && j < 5 && minas[i][j])
{
                            alrededor++;
                        }
                    }
                }
                System.out.println("Seguro. Minas alrededor: " + alrededor);
            }
        }
        sc.close();
    }
}
```

El `while` que col·loca mines garanteix exactament 5 en posicions diferents (la condició `!minas[f][c]` evita repetir). Per a comptar veïnes, el doble bucle revisa les 8 caselles al voltant i el `if` amb els límits (0-4) impedeix sortir-te de l'array: sense aquella comprovació, `minas[-1][...]` seria `ArrayIndexOutOfBoundsException`.

⭐⭐⭐ Exercici 9: Compactar — els zeros al final

› 🔄 Solució

```
import java.util.Arrays;

public class Compactar {
    public static void compactar(int[] arr) {
        int pos = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[pos] = arr[i];
                pos++;
            }
        }

        for (int i = pos; i < arr.length; i++) {
            arr[i] = 0;
        }
    }

    public static void main(String[] args) {
        int[] datos = {0, 3, 0, 1, 0, 2};
        compactar(datos);
        System.out.println(Arrays.toString(datos)); // [3, 1, 2, 0, 0, 0]
    }
}
```

L'índex `pos` marca on va el següent valor no zero. Quan trobes un valor diferent de 0, el copies a `arr[pos]` i avances. En acabar, des de `pos` fins al final s'ompli amb zeros. És el mateix patró de "índex lent / índex ràpid" que usa la fusió i que veuràs de nou amb les col·leccions.

Butlletí U06 — Extres

📝 Butlletí U06 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. Les solucions estan amagades: esgota la teva pista abans de mirar-les.

CodeWars

1. Convert number to reversed array of digits

Et donen un número enter no negatiu. Torna els seus dígits en un array d'enters, **en ordre invers**.

Exemple: `35231` → `[1, 3, 2, 5, 3]` · `0` → `[0]`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

> › 💡 **Pista**
>
> Per a extreure dígits, `% 10` et dona l'últim i `/ 10` se'l treu. Amb això ja surten invertits: el primer que traus és l'últim del número.

> › 🔄 **Solució**

```
public class Kata {
    public static int[] digitize(long n) {
        String texto = Long.toString(n);
        int[] resultado = new int[texto.length()];

        for (int i = 0; i < resultado.length; i++) {
                                                    resultado[i]     =
Character.getNumericValue(texto.charAt(resultado.length - 1 - i));
        }
        return resultado;
    }
}
```

> Una altra forma, sense convertir a text, si prefereixes `%` i `/`:

```java
public static int[] digitize(long n) {
    String texto = String.valueOf(n);
    int[] resultado = new int[texto.length()];
    for (int i = 0; i < texto.length(); i++) {
        resultado[i] = (int) (n % 10);
        n /= 10;
    }
    return resultado;
}
```

Amb `% 10` i `/ 10`: l'últim dígit eix primer (ja ve invertit), i després de dividir entre 10 es repeteix. El cas `0` → `[0]` funciona sol: `0 % 10 = 0`.

---

2. Find the smallest integer in the array

Et donen un array d'enters. Torna l'enter **més petit**.

Exemples: `[78, 56, 232, 12, 11, 43]` → `11` · `[34, -345, -1, 100]` → `-345`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

> 💡 **Pista**

El patró del mínim: comença assumint que el primer és el mínim i compara amb cada un. Aquí l'array mai no està buit.

> 🔄 **Solució**

```java
public class Kata {
    public static int findSmallestInt(int[] args) {
        int minimo = args[0];
        for (int i = 1; i < args.length; i++) {
            if (args[i] < minimo) {
                minimo = args[i];
            }
        }
        return minimo;
    }
}
```

> El mateix esquelet que el “màxim acumulat”, però al revés. Com l’array mai no està buit, pots usar `args[0]` com a punt de partida sense por.

---

3. Count by X

Torna un array amb els **primers n múltiples** de x.

Exemples: `countBy(2, 5)` → `[2, 4, 6, 8, 10]` · `countBy(1, 10)` → `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

> › 💡 **Pista**
>
> Crea l’array amb `new int[n]` i ompli amb un `for`: la plaça `i` guarda `x * (i + 1)`.

> › 🔄 **Solució**
>
> ```
> public class Kata {
>     public static int[] countBy(int x, int n) {
>         int[] resultado = new int[n];
>         for (int i = 0; i < n; i++) {
>             resultado[i] = x * (i + 1);
>         }
>         return resultado;
>     }
> }
> ```
>
> Saps la grandària per endavant (`n`), així que no cal el patró de dues passades: un sol `for` crea i ompli. El `(i + 1)` és perquè els múltiples comencen en `x * 1`, no en `x * 0`.

---

4. Take a Ten Minute Walk

Vius en una ciutat en graella. Et donen un array de direccions (`'n'`, `'s'`, `'e'`, `'w'`). Torna `true` si el passeig dura **exactament 10 minuts** (una illa per minut) **i et deixa al punt de partida**.

Exemples: `['n','s','n','s','n','s','n','s','n','s']` → `true` · `['n','n','n','s','n','s','n','s','n','s']` → `false`.

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Porta dos comptadors: un per a nord/sud i un altre per a est/oest. Si el passeig no dura 10 minuts, ja pots tornar `false` sense comptar res.

> 🔄 **Solució**

```
public class Kata {
    public static boolean isValid(char[] walk) {
        if (walk.length != 10) {
            return false;
        }

        int x = 0, y = 0;
        for (char c : walk) {
            if (c == 'n') y++;
            if (c == 's') y--;
            if (c == 'e') x++;
            if (c == 'w') x--;
        }

        return x == 0 && y == 0;
    }
}
```

Nord puja y, sud el baixa; est puja x, oest el baixa. Si al final tots dos comptadors tornen a 0, ets on vas començar. És un array recorregut una sola volta, amb la suma com a testimoni. O(n), com ha de ser.

AceptaElReto

5. 171 — Abadies de pedra

Una serralada té una sèrie de muntanyes, cada una amb la seva altura. Es pot construir una **abadia** sobre una muntanya si és **més alta que totes les que té a la dreta** (cap al mar). Compta quantes muntanyes compleixen la condició.

Entrada: diversos casos de prova. Cada cas comença amb `N` (nombre de muntanyes), seguit de `N` altures en la mateixa línia. Un `0` acaba l'entrada.

Exemple:

```
3
5 3 4
4
2 3 4 1
0
```

Sortida:

```
2
2
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

> 💡 **Pista**
>
> Recorre l'array **de dreta a esquerra** guardant l'altura màxima vista. L'última muntanya sempre val. Quan una muntanya supere la màxima vista fins ara, és una abadia nova.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Abadias {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n != 0) {
            int[] montanas = new int[n];
            for (int i = 0; i < n; i++) {
                montanas[i] = sc.nextInt();
            }

            int abadias = 0;
            int maxVista = -1;
            for (int i = n - 1; i >= 0; i--) {
                if (montanas[i] > maxVista) {
                    abadias++;
                    maxVista = montanas[i];
                }
            }

            System.out.println(abadias);
            n = sc.nextInt();
        }
        sc.close();
    }
}
```

La clau és recórrer cap enrere: la muntanya veu el mar si és més alta que **tot** el que té a la dreta, i la màxima vista ho resumeix tot. Amb `5 3 4`: el 4 val, el 3 no (hi ha un 4 més alt), el 5 val → 2. És un array + recorregut en el sentit correcte.

---

6. 369 — Comptant en la sorra

Un xiquet escriu els números en la sorra amb marques de compte: l'`1` el escriu com `1`, el `2` com `11`, el `3` com `111`… És a dir, cada número es representa amb tants uns com val. Donat un número, escriu-lo com ho faria el xiquet.

Entrada: diversos números, un per línia, fins a un `0` final.

Exemple:

```
1
5
3
0
```

Sortida:

```
1
11111
111
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

> 💡 **Pista**
>
> Un bucle que afegeixi `'1'` a un `StringBuilder` tantes vegades com indiqui el número. No cal array: és la prova que de vegades n'hi ha prou amb saber quantes vegades repetir.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class ContandoEnLaArena {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n != 0) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < n; i++) {
                sb.append('1');
            }
            System.out.println(sb);
            n = sc.nextInt();
        }
        sc.close();
    }
}
```

> Cada número `n` es converteix en `n` uns. El bucle és el mateix que usaries per a omplir un array, però aquí l'"array" és un `StringBuilder` que creix. El `0` marca el final de l'entrada, com sempre en AceptaElReto.

> 🧭 **I si et quedes amb ganes?** Quan domines l'aparcament sencer —crear, recórrer, ordenar, buscar, invertir i compactar—, torna a aquests problemes amb el punt 9 com a examen. I en la UO7 t'esperen els algoritmes: cerca binària, bombolla i Big O. Els arrays que has après avui són la matèria primera de tot el que ve.

Butlletí UO7 — Inicial

📝 Butlletí UO7 — Inicial

> Sense solucions. Sense presses. Obre l'IDE i deixa que els teus bucles busquen, ordenen i calculen. L'ordinador mai no es cansa de recórrer un array: ets tu qui decideix com.

Exercici 1: Què imprimeix? — la cerca de la gema

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class CercaGema {
    public static void main(String[] args) {
        int[] cofre = {12, 7, 25, 9, 31};
        int objectiu = 9;
        int posicio = -1;

        for (int i = 0; i < cofre.length; i++) {
            if (cofre[i] == objectiu) {
                posicio = i;
                break;
            }
        }

        System.out.println("Posició: " + posicio);
    }
}
```

Pista: recorre l'array mentalment element a element. En quin índex apareix el 9 per primera vegada?

Exercici 2: El buscador de tresors

Escriu un mètode `public static int buscar(int[] dades, int objectiu)` que recorri l'array amb un `for` i retorni l'**índex** on apareix l'objectiu. Si no hi és, retorna `-1`.

Prova amb `int[] tresors = {4, 8, 15, 16, 23, 42}`:

- `buscar(tresors, 15)` → ha de retornar `2`
- `buscar(tresors, 7)` → ha de retornar `-1`

Pista: retorna `i` tan bon punt trobes l'objectiu; només després del bucle retornes `-1`.

Exercici 3: Què imprimeix? — la bombolla mínima

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class BombollaMinima {
    public static void main(String[] args) {
        int[] dades = {3, 1, 2};

        for (int i = 0; i < dades.length - 1; i++) {
            for (int j = 0; j < dades.length - 1 - i; j++) {
                if (dades[j] > dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                }
            }
        }

        for (int num : dades) {
            System.out.print(num + " ");
        }
    }
}
```

Pista: fes la traça en un paper. Primera passada: el 3 vs 1, després 3 vs 2. Què queda al final?

Exercici 4: La capsa de sabates ordenada

Escriu un mètode `public static void ordenar(int[] dades)` que ordeni l'array **de menor a major** usant l'ordenació per inserció.

Prova amb `int[] capsa = {9, 2, 7, 1}` i mostra l'array resultant amb un `for...each`.

Pista: guarda `int clau = dades[i]` i llisqui cap a la dreta els elements majors amb un `while`.

Exercici 5: El detectiu de la cerca binària

Escriu un mètode `public static int cercaBinaria(int[] dades, int objectiu)` que usi la cerca binària sobre un array **ja ordenat** i retorni l'índex de l'objectiu (o `-1`).

Prova amb `int[] agenda = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`:

- `cercaBinaria(agenda, 23)` → ha de retornar `5`
- `cercaBinaria(agenda, 30)` → ha de retornar `-1`

Pista: dos punters `esquerra = 0` i `dreta = dades.length - 1`, un `while (esquerra <= dreta)` i `int mig = esquerra + (dreta - esquerra) / 2`.

Exercici 6: Què imprimeix? — la binària que falla

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class BinariaQueFalla {
    public static void main(String[] args) {
        int[] dades = {10, 20, 30, 40, 50};
        int objectiu = 35;
        int esquerra = 0;
        int dreta = dades.length - 1;
        boolean trobat = false;

        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;
            if (dades[mig] == objectiu) {
                trobat = true;
                break;
            } else if (dades[mig] < objectiu) {
                esquerra = mig + 1;
            } else {
                dreta = mig - 1;
            }
        }

        System.out.println(trobat);
    }
}
```

Pista: el 35 no està en l'array. Quin valor té `trobat` al final del bucle?

Exercici 7: El comptador d'intercanvis

Escriu un programa anomenat `ComptadorIntercanvis` que usi la bombolla per a ordenar `int[] dades = {5, 2, 9, 1, 5}` i **compte quants intercanvis** realitza en total. Al final mostra: `Intercanvis: X`.

Pista: declara `int intercanvis = 0;` i suma-li 1 dins del `if` d'intercanvi, just després de l'operació.

Exercici 8: La nota més alta de la classe

Escriu un mètode `public static int notaMaxima(int[] notes)` que recorri l'array i retorni la nota més alta.

Prova amb `int[] notes = {7, 9, 5, 10, 6, 8, 4}` → ha de retornar `10`.

Pista: usa una variable `maxim` inicialitzada amb el primer element (`notes[0]`) i compara amb cada un.

Exercici 9: CodeWars — Find the smallest integer in the array

Resol la kata **"Find the smallest integer in the array"** (8 kyu) en CodeWars.

Crea el mètode `public static int findSmallestInt(int[] args)` que retorni l'enter més petit de l'array.

Pista: és el mateix patró de l'exercici 8: recorre i compara. Aquí l'array mai no està buit.

Butlletí U07 — Inicial Resolt

📝 Butlletí U07 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què imprimeix? — la cerca de la gema

> 🔄 Solució

Imprimeix **`Posició: 3`**.

Recorregut: `cofre[0] = 12` (no), `cofre[1] = 7` (no), `cofre[2] = 25` (no), `cofre[3] = 9` (sí!). Guardem `posicio = 3` i el `break` talla el bucle. No cal seguir buscant: ja està.

Exercici 2: El buscador de tresors

> 🔄 Solució

```
public static int buscar(int[] dades, int objectiu) {
    for (int i = 0; i < dades.length; i++) {
        if (dades[i] == objectiu) {
            return i;
        }
    }
    return -1;
}
```

Si el troba, el `return i` retorna l'índex i talla el mètode a l'instant. Només si el bucle acaba sencer sense trobar-lo, s'executa el `return -1`. Aquell `-1` és la contrasenya universal del "no trobat" que usa tota la indústria.

Exercici 3: Què imprimeix? — la bombolla mínima

> 🔄 Solució

Imprimeix **`1 2 3`** .

Traça de la bombolla sobre `{3, 1, 2}`:

Passada	j	Compara	Intercanvia	Array
1	0	3 vs 1	Sí	1 3 2
1	1	3 vs 2	Sí	1 2 3
2	0	1 vs 2	No	1 2 3

La passada 2 només comprova (el `-1 - i` redueix el recorregut) i confirma que ja està ordenat.

Exercici 4: La capsa de sabates ordenada

› 🔄 Solució

```
public static void ordenar(int[] dades) {
    for (int i = 1; i < dades.length; i++) {
        int clau = dades[i];
        int j = i - 1;

        while (j >= 0 && dades[j] > clau) {
            dades[j + 1] = dades[j];
            j--;
        }
        dades[j + 1] = clau;
    }
}

// En main:
int[] capsa = {9, 2, 7, 1};
ordenar(capsa);
for (int num : capsa) {
    System.out.print(num + " ");
}
```

Sortida: `1 2 7 9`

Sobre `{9, 2, 7, 1}`: el 2 es cola a l'esquerra del 9, el 7 entra entre el 2 i el 9, i l'1 viatja fins al principi. Cada element s'insereix en el seu lloc dins de la "mà" ja ordenada.

Exercici 5: El detectiu de la cerca binària

> 🔄 Solució

```
public static int cercaBinaria(int[] dades, int objectiu) {
    int esquerra = 0;
    int dreta = dades.length - 1;

    while (esquerra <= dreta) {
        int mig = esquerra + (dreta - esquerra) / 2;

        if (dades[mig] == objectiu) {
            return mig;
        } else if (dades[mig] < objectiu) {
            esquerra = mig + 1;
        } else {
            dreta = mig - 1;
        }
    }
    return -1;
}
```

Per al 23: mig = 4 (16 < 23 → esquerra = 5), mig = 7 (56 > 23 → dreta = 6), mig = 5 (23 → retorna 5). Per al 30: descarta i descarta fins que `esquerra` supera `dreta` i cau el `return -1`. Els `+1`/`-1` són els que garanteixen que el segment sempre es redueix.

Exercici 6: Què imprimeix? — la binària que falla

> 🔄 Solució

Imprimeix **`false`**.

El 35 no està en l'array. La binària va descartant meitats: mig = 2 (30 < 35 → esquerra = 3), mig = 4 (50 > 35 → dreta = 3), mig = 3 (40 > 35 → dreta = 2). Ara `esquerra = 3` > `dreta = 2`,

el `while` acaba i `trobat` segueix en `false`. Sense error, sense drama: el "no trobat" també es comunica.

Exercici 7: El comptador d'intercanvis

> 🔄 Solució

```
public class ComptadorIntercanvis {
    public static void main(String[] args) {
        int[] dades = {5, 2, 9, 1, 5};
        int intercanvis = 0;

        for (int i = 0; i < dades.length - 1; i++) {
            for (int j = 0; j < dades.length - 1 - i; j++) {
                if (dades[j] > dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                    intercanvis++;
                }
            }
        }

        System.out.println("Intercanvis: " + intercanvis);
    }
}
```

Sortida: `Intercanvis: 5`

Cada vegada que el `if` dispara un intercanvi, sumem 1 al comptador. Amb els dos 5 (repetits), la bombolla no els intercanvia entre si (perquè `5 > 5` és `false`): la comparació estricta manté l'ordre relatiu dels iguals. Ordenat: `{1, 2, 5, 5, 9}`.

Exercici 8: La nota més alta de la classe

> 🔄 Solució

```java
public static int notaMaxima(int[] notes) {
    int maxim = notes[0];

    for (int i = 1; i < notes.length; i++) {
        if (notes[i] > maxim) {
            maxim = notes[i];
        }
    }
    return maxim;
}
```

Inicialitzem `maxim` amb el primer element (no amb 0, per si hi hagués notes negatives) i comparem amb els altres. Recorregut lineal O(n): un sol bucle. Retorna 10.

Exercici 9: CodeWars — Find the smallest integer in the array

› 🔄 Solució

```java
public class Kata {
    public static int findSmallestInt(int[] args) {
        int minim = args[0];

        for (int i = 1; i < args.length; i++) {
            if (args[i] < minim) {
                minim = args[i];
            }
        }
        return minim;
    }
}
```

El mirall de l'exercici 8: en comptes de buscar el màxim, busquem el mínim. `args[0]` com a valor inicial i un sol recorregut. O(n): no pots fer-ho més ràpid sense mirar cada element almenys una vegada.

Butlletí UO7 — Avançat

📝 Butlletí UO7 — Avançat

> Dificultat progressiva. ⭐ per a calfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ Exercici 1: Què imprimeix? — la binària amb historial

Sense executar, escriu la sortida exacta d'aquest programa:

```
public class BinariaHistorial {
    public static void main(String[] args) {
        int[] dades = {2, 4, 6, 8, 10, 12, 14, 16};
        int objectiu = 10;
        int esquerra = 0;
        int dreta = dades.length - 1;

        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;
            System.out.println("Prove l'índex " + mig);

            if (dades[mig] == objectiu) {
                System.out.println("Trobat en " + mig);
                break;
            } else if (dades[mig] < objectiu) {
                esquerra = mig + 1;
            } else {
                dreta = mig - 1;
            }
        }
    }
}
```

Pista: escriu en una taula cada vegada amb els seus `esquerra`, `dreta`, `mig` i què decideix. Només són 3 o 4 línies de traça.

⭐ Exercici 2: El buscador binari amb historial

Escriu un mètode `public static int cercaBinaria(int[] dades, int objectiu)` com el de la teoria, però que **cada vegada que provi un índex, el mostri per pantalla**: `Provant l'índex X`. Al final retorna l'índex o `-1`.

Prova amb `int[] dades = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19}` i `objectiu = 7`.

Pista: el `System.out.println` va dins del `while`, just després de calcular `mig` i abans de comparar.

⭐⭐ Exercici 3: La bombolla amb recompte (i flag)

Escriu un programa anomenat `BombollaRecompte` que ordeni `int[] dades = {9, 3, 7, 1, 5}` amb bombolla i **compte els intercanvis**, igual que en l'inicial. Però a més, afegeix el flag `boolean hiHaIntercanvi` amb el seu `break` per a no fer passades inútils.

Mostra al final l'array ordenat i `Intercanvis: X`.

Pista: el flag es reinicia a `false` al principi de cada passada i es posa a `true` dins del `if`. Després de la passada: `if (!hiHaIntercanvi) break;`.

⭐⭐ Exercici 4: La inserció descendent

Modifica l'ordenació per inserció perquè ordeni **de major a menor**. Escriu un mètode `public static void ordenarDescendent(int[] dades)`.

Prova amb `int[] notes = {6, 9, 3, 8, 5}` i mostra el resultat.

Pista: només canvia un signe: en la condició del `while`, els majors han de lliscar cap a la dreta. Pensa quin és ara l'"ordre correcte".

⭐⭐ Exercici 5: L'analista de complexitat

Digues la complexitat Big O de cada mètode i justifica breument:

```java
public class Analista {

    public static int metodeA(int[] dades) {
        int total = 0;
        for (int num : dades) {
            total += num;
        }
        return total;
    }

    public static int metodeB(int[] dades) {
        int parells = 0;
        for (int i = 0; i < dades.length; i++) {
            for (int j = i + 1; j < dades.length; j++) {
                if (dades[i] == dades[j]) {
                    parells++;
                }
            }
        }
        return parells;
    }

    public static int metodeC(int[] dades, int objectiu) {
        int esquerra = 0;
        int dreta = dades.length - 1;
        while (esquerra <= dreta) {
            int mig = esquerra + (dreta - esquerra) / 2;
            if (dades[mig] == objectiu) return mig;
            if (dades[mig] < objectiu) esquerra = mig + 1;
            else dreta = mig - 1;
        }
        return -1;
    }
}
```

Pista: compte bucles: un → O(n), dos anidats → O(n²). El que parteix el segment per la mitat en cada vegada és O(log n). El `j = i + 1` no el salva de ser O(n²): segueix sent quasi n × n.

⭐⭐ Exercici 6: El caçador de parelles

Escriu un mètode `public static boolean existeixParella(int[] dades, int sumaObjectiu)` que retorni `true` si existeixen **dos elements distints** de l'array la suma dels quals sigui `sumaObjectiu`.

Prova amb `int[] preus = {10, 3, 7, 5, 12}`:

- `existeixParella(preus, 17)` → `true` (10 + 7)
- `existeixParella(preus, 25)` → `false`

Pista: dos bucles anidats amb `j = i + 1` (per a no provar un element amb si mateix). Dins: `if (dades[i] + dades[j] == sumaObjectiu) return true;`. Al final, `return false;`.

⭐⭐ Exercici 7: El detectiu d'inversions

Una **inversió** és una parella de posicions `(i, j)` amb `i < j` on `dades[i] > dades[j]` (estan desordenades). Escriu un mètode `public static int contarInversions(int[] dades)` que les compte.

Prova amb `int[] dades = {2, 4, 1, 3}` → hi ha 3 inversions: (2,1), (4,1), (4,3).

Pista: el patró de doble bucle amb `j = i + 1` una altra vegada, però ara el `if` compara `dades[i] > dades[j]` i suma 1 al comptador.

⭐⭐⭐ Exercici 8: CodeWars — Ones and Zeros

Resol la kata **"Ones and Zeros"** (7 kyu) en CodeWars.

Et donen un array d'enters (només 0 i 1) que representa un nombre en binari. Retorna el seu valor decimal.

Exemple: `[1, 0, 1, 1]` → `11` (1·8 + 0·4 + 1·2 + 1·1).

Pista: recorre l'array d'esquerra a dreta acumulant `valor = valor * 2 + digit`. Aquell és l'algoritme per a convertir de binari a decimal sense usar `Math.pow`.

⭐⭐⭐ Exercici 9: AceptaElReto — 100 Constant de Kaprekar

Resol el problema **100 — Constant de Kaprekar** en AceptaElReto.com.

L'algoritme de Kaprekar: donat un nombre de 4 xifres, ordena els seus dígits de major a menor i de menor a major, resta tots dos, i repeteix amb el resultat. Al final sempre s'arriba a **6174** (o al propi 6174) i, si el nombre té totes les xifres iguals (com 1111), la diferència dona 0.

L'entrada du diversos nombres (poden tenir menys de 4 xifres: cal completar amb zeros a l'esquerra). Per a cada un, mostra **quantes iteracions** calen per a arribar a 6174. El 6174 necessita 0 iteracions. **Compte amb els repdigits** (1111, 5555…): la primera resta dona 0 i el bucle mai no arribaria a 6174; el problema oficial demana **8** per a ells. L'0 d'entrada acaba el programa.

Pista: per a ordenar els dígits, extrau-los en un `int[]` de 4 posicions amb `% 10` i `/ 10`, ordena'l amb bombolla (¡reutilitza la U07!), i reconstrueix el nombre major (dígits en ordre descendent) i el menor (ascendent). Compta les iteracions amb un comptador.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	100 — Constant de Kaprekar	Fàcil
AceptaElReto	185 — Potitos	Fàcil
CodeWars	Find the smallest integer in the array (8 kyu)	Principiant
CodeWars	Ones and Zeros (7 kyu)	Aficionat
CodeWars	You only need one (8 kyu)	Principiant

Butlletí UO7 — Avançat Resolt

📝 Butlletí UO7 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què imprimeix? — la binària amb historial

> 🔄 Solució

Imprimeix:

```
Prove l'índex 3
Prove l'índex 5
Prove l'índex 4
Trobat en 4
```

Traça sobre `{2, 4, 6, 8, 10, 12, 14, 16}` (8 elements, índexs 0 a 7):

Volta	esq	der	mig	dades[mig]	Decisió
1	0	7	3	8	8 < 10 → esq = 4
2	4	7	5	12	12 > 10 → der = 4
3	4	4	4	10	Trobat!

Tres proves, tres línies, i l'índex 4. Així de compacta és la binària: cada vegada descarta la meitat de l'espai.

Exercici 2: El buscador binari amb historial

> 🔄 Solució

```java
public static int cercaBinaria(int[] dades, int objectiu) {
    int esquerra = 0;
    int dreta = dades.length - 1;

    while (esquerra <= dreta) {
        int mig = esquerra + (dreta - esquerra) / 2;
        System.out.println("Provant l'índex " + mig);

        if (dades[mig] == objectiu) {
            return mig;
        } else if (dades[mig] < objectiu) {
            esquerra = mig + 1;
        } else {
            dreta = mig - 1;
        }
    }
    return -1;
}
```

Sortida per a `objectiu = 7`: `Provant l'índex 4`, `Provant l'índex 1`, `Provant l'índex 2`, i retorna `2` (el 7 està en `dades[2]`). El `println` dins del bucle converteix la binària en una capsa de vidre: pots veure cada intent.

Exercici 3: La bombolla amb recompte (i flag)

› 🔄 Solució

```java
public class BombollaRecompte {
    public static void main(String[] args) {
        int[] dades = {9, 3, 7, 1, 5};
        int intercanvis = 0;
        boolean hiHaIntercanvi;

        for (int i = 0; i < dades.length - 1; i++) {
            hiHaIntercanvi = false;

            for (int j = 0; j < dades.length - 1 - i; j++) {
                if (dades[j] > dades[j + 1]) {
                    int temp = dades[j];
                    dades[j] = dades[j + 1];
                    dades[j + 1] = temp;
                    intercanvis++;
                    hiHaIntercanvi = true;
                }
            }

            if (!hiHaIntercanvi) break;
        }

        System.out.println("Intercanvis: " + intercanvis);
        for (int num : dades) {
            System.out.print(num + " ");
        }
    }
}
```

Sortida:

```
Intercanvis: 7
1 3 5 7 9
```

Sobre `{9, 3, 7, 1, 5}` són 7 intercanvis. El flag fa que, tan bon punt una passada no mou res, el `break` talla: no mereix la pena seguir. Compara amb el butlletí inicial, on el flag no existia i sempre es feien totes les passades.

Exercici 4: La inserció descendent

> 🔄 Solució

```
public static void ordenarDescendent(int[] dades) {
    for (int i = 1; i < dades.length; i++) {
        int clau = dades[i];
        int j = i - 1;

        while (j >= 0 && dades[j] < clau) {
            dades[j + 1] = dades[j];
            j--;
        }
        dades[j + 1] = clau;
    }
}
```

L'únic canvi és el signe: `dades[j] < clau` en comptes de >. Ara llisquem cap a la dreta els elements **menors** que la clau, perquè volem que els grans queden a l'esquerra. Sobre `{6, 9, 3, 8, 5}` retorna `9 8 6 5 3`. Canviar una comparació i tot el sentit de l'algoritme gira.

Exercici 5: L'analista de complexitat

> 🔄 Solució

- **`metodeA` → O(n)**: un sol bucle que recorre l'array complet.
- **`metodeB` → O(n²)**: dos bucles anidats. Encara que l'interior comenci en `j = i + 1`, són ≈ n·(n-1)/2 iteracions, que és O(n²). En Big O, les constants i la mitat no conten.
- **`metodeC` → O(log n)**: la cerca binària. En cada vegada el segment es parteix per la mitat, així que el nombre de vegades és $\log_2(n)$.

Exercici 6: El caçador de parelles

> 🔄 Solució

```java
public static boolean existeixParella(int[] dades, int sumaObjectiu) {
    for (int i = 0; i < dades.length; i++) {
        for (int j = i + 1; j < dades.length; j++) {
            if (dades[i] + dades[j] == sumaObjectiu) {
                return true;
            }
        }
    }
    return false;
}
```

`j = i + 1` evita provar un element amb si mateix i duplicar parelles (provar (3,5) i (5,3)). Per a 17: 10 + 7 → `true`. Per a 25: cap combinació → `false`. És O(n²), però per a arrays petits és instantani.

Exercici 7: El detectiu d'inversions

› 🔄 Solució

```java
public static int contarInversions(int[] dades) {
    int inversions = 0;

    for (int i = 0; i < dades.length; i++) {
        for (int j = i + 1; j < dades.length; j++) {
            if (dades[i] > dades[j]) {
                inversions++;
            }
        }
    }
    return inversions;
}
```

Sobre `{2, 4, 1, 3}`: parelles `(0,1)` 2<4 no, `(0,2)` 2>1 sí, `(0,3)` 2<3 no, `(1,2)` 4>1 sí, `(1,3)` 4>3 sí, `(2,3)` 1<3 no → 3 inversions. Curiositat: les inversions mesuren "com de desordenat" està un array. Un array ja ordenat té 0; un d'invertit té el màxim.

Exercici 8: CodeWars — Ones and Zeros

› 🔄 Solució

```java
public class Kata {
    public static int binaryArrayToNumber(int[] numeros) {
        int valor = 0;
        for (int digit : numeros) {
            valor = valor * 2 + digit;
        }
        return valor;
    }
}
```

Per a `[1, 0, 1, 1]`: valor = 0·2+1 = 1, després 1·2+0 = 2, després 2·2+1 = 5, després 5·2+1 = 11. El truc `valor * 2 + digit` converteix binari a decimal en un sol recorregut, sense `Math.pow`. És un algoritme clàssic que et trobaràs a tot arreu: dígit a dígit, el valor "es desplaça" cap a l'esquerra.

Exercici 9: AceptaElReto — 100 Constant de Kaprekar

› 🔄 Solució

```java
import java.util.Scanner;

public class Kaprekar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero = sc.nextInt();

        while (numero != 0) {
            if (esRepdigit(numero)) {
                System.out.println(8);
                numero = sc.nextInt();
                continue;
            }

            int iteracions = 0;

            while (numero != 6174) {
                int[] digits = new int[4];

                for (int i = 3; i >= 0; i--) {
                    digits[i] = numero % 10;
                    numero /= 10;
                }

                for (int i = 0; i < digits.length - 1; i++) {
                    for (int j = 0; j < digits.length - 1 - i; j++) {
                        if (digits[j] > digits[j + 1]) {
                            int temp = digits[j];
                            digits[j] = digits[j + 1];
                            digits[j + 1] = temp;
                        }
                    }
                }

                int ascendent = 0;
                int descendent = 0;
                for (int i = 0; i < 4; i++) {
                    ascendent = ascendent * 10 + digits[i];
                    descendent = descendent * 10 + digits[3 - i];
                }

                numero = descendent - ascendent;
```

```
                iteracions++;
            }

            System.out.println(iteracions);
            numero = sc.nextInt();
        }
        sc.close();
    }

    static boolean esRepdigit(int n) {
        String s = String.format("%04d", n);
        char primera = s.charAt(0);
        for (char c : s.toCharArray()) {
            if (c != primera) return false;
        }
        return true;
    }
}
```

Els nombres amb menys de 4 xifres es completen amb zeros perquè extraiem amb `% 10` i `/ 10` sobre un array de 4 posicions: el 21 es converteix en `{0, 0, 2, 1}`. L'algoritme de Kaprekar acaba sempre (màxim unes poques iteracions) i arriba a 6174… **excepte els repdigits** (1111, 5555…): la primera resta dona 0 i, si entrares al `while (numero != 6174)`, et quedaries donant vegades amb 0 per sempre. Per això el problema oficial d'AceptaElReto demana que els repdigits imprimeixin `8`, i ho detectem abans amb `esRepdigit`. La bombolla del punt 4, reutilitzada dins del propi Kaprekar: el codi de la UO7 resolent problemes reals.

Butlletí U07 — Extres

📝 Butlletí U07 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està amagada: resisteix fins a esgotar la teva pista.

CodeWars

1. You only need one — Beginner

Et donen un array i un valor. Retorna `true` si l'array **conté** el valor.

Exemples: `{"a", "b", "c"}, "a"` → `true` · `{"a", "b", "c"}, "z"` → `false`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: un `for` que recorri l'array i un `if` que retorni `true` si algun element és igual al valor. Després del bucle, `false`.

> 🔄 **Solució**

```
public class Kata {
    public static boolean check(Object[] a, Object x) {
        for (Object element : a) {
            if (element.equals(x)) {
                return true;
            }
        }
        return false;
    }
}
```

És la cerca lineal en la seva forma més pura: recórrer i retornar tan bon punt trobes. Per a objectes usem `.equals()` en comptes de ==. La complexitat és O(n): pot ser que el trobes a la primera o al final.

---

2. Square(n) Sum

Et donen un array d'enters. Retorna la suma dels **quadrats** de cada nombre.

Exemple: `[1, 2, 2]` → `1 + 4 + 4 = 9`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: un acumulador `suma` i, dins del bucle, `suma += numero * numero;`.

> 🔄 **Solució**

```
public class Kata {
    public static int squareSum(int[] numeros) {
        int suma = 0;
        for (int numero : numeros) {
            suma += numero * numero;
        }
        return suma;
    }
}
```

`numero * numero` és el quadrat (evita `Math.pow`, que és més lent per a un enter). Un acumulador i un bucle: O(n). El mateix esquelet que "suma tots" però amb un petit càlcul dins.

---

3. Convert number to reversed array of digits

Et donen un nombre enter no negatiu. Retorna els seus dígits en un array d'enters, **en ordre invers**.

Exemple: `35231` → `[1, 3, 2, 5, 3]` · `0` → `[0]`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: per a extreure dígits, `% 10` et dona l'últim i `/ 10` se'l lleva. Amb això ja surten invertits: el primer que traus és l'últim del nombre.

> 🔄 **Solució**

```java
public class Kata {
    public static int[] digitize(long n) {
        String text = Long.toString(n);
        int[] resultat = new int[text.length()];

        for (int i = 0; i < resultat.length; i++) {
                                                                resultat[i]     =
Character.getNumericValue(text.charAt(resultat.length - 1 - i));
        }
        return resultat;
    }
}
```

Una altra forma, sense convertir a text, si prefereixes `%` i `/`:

```java
public static int[] digitize(long n) {
    String text = String.valueOf(n);
    int[] resultat = new int[text.length()];
    for (int i = 0; i < text.length(); i++) {
        resultat[i] = (int) (n % 10);
        n /= 10;
    }
    return resultat;
}
```

Amb `% 10` i `/ 10`: l'últim dígit surt primer (ja ve invertit), i després de dividir entre 10 es repeteix. El cas `0` → `[0]` funciona sol: `0 % 10 = 0`.

---

4. Count the divisors of a number

Compta quants divisors té un nombre n (positiu).

Exemples: `divisors(4)` → 3 (1, 2, 4) · `divisors(12)` → 6 (1, 2, 3, 4, 6, 12).

- Enunciat en CodeWars
- Dificultat: 7 kyu

Pista: un bucle de l'1 a `n` amb un `if (n % i == 0)` i un comptador. (Optimització opcional: només arribes a l'arrel quadrada i comptes de dos en dos.)

> 🔄 Solució

```
public class Kata {
    public static long numberOfDivisors(int n) {
        int comptador = 0;
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                comptador++;
            }
        }
        return comptador;
    }
}
```

Versió optimitzada (O(√n) en comptes d'O(n)):

```
public static long numberOfDivisors(int n) {
    int comptador = 0;
    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            comptador += (i * i == n) ? 1 : 2;
        }
    }
    return comptador;
}
```

La versió simple recorre de l'1 a n: si `n % i == 0`, és divisor. L'optimitzada s'aprofita que els divisors van en parelles: si `i` divideix `n`, llavors `n / i` també, excepte quan són el mateix (el cas del quadrat perfecte). D'O(n) a O(√n): Big O en acció.

AceptaElReto

5. 185 — Potitos

Un pare li compra potets (bots de menjar per a bebé) al seu fill. A vegades el bebé li **dona la volta** al bot per a veure el revers, on es llegeix l'ingredient *que menys li agrada* (o "FIN" si no n'hi ha cap). El pare vol saber els ingredients que el bebé **odia**, per a deixar de comprar-los.

Entrada: diversos casos de prova. Cada cas comença amb `N` (nombre de potets). A continuació, `N` línies: cada una amb un ingredient `I` que li agrada, un guió `--`, i opcionalment un `FIN` o un ingredient que detesta. El cas `0` acaba.

Exemple:

```
4
pollo--FIN
pescado--FIN
verdura--atun
pollo--FIN
0
```

Sortida:

```
atun
```

(L'únic detestat és `atun`; els altres no tenien revers o deien `FIN`.) Els ingredients detestats es mostren **ordenats alfabèticament** i **sense repetir**.

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: per a cada línia, separa amb `split("--")`. Si la segona part no és `"FIN"`, és un ingredient detestat: guarda'l. Al final, ordena'l (la bombolla del punt 4, però amb `String.compareTo` en comptes de >) i elimina els repetits.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Potitos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        while (n != 0) {
            String[] odiats = new String[n];
            int quants = 0;

            for (int i = 0; i < n; i++) {
                String linea = sc.nextLine();
                String[] parts = linea.split("--");

                if (parts.length > 1 && !parts[1].equals("FIN")) {
                    odiats[quants] = parts[1];
                    quants++;
                }
            }

            for (int i = 0; i < quants - 1; i++) {
                for (int j = 0; j < quants - 1 - i; j++) {
                    if (odiats[j].compareTo(odiats[j + 1]) > 0) {
                        String temp = odiats[j];
                        odiats[j] = odiats[j + 1];
                        odiats[j + 1] = temp;
                    }
                }
            }

            StringBuilder eixida = new StringBuilder();
            for (int i = 0; i < quants; i++) {
                if (i == 0 || !odiats[i].equals(odiats[i - 1])) {
                    eixida.append(odiats[i]).append(" ");
                }
            }
            System.out.println(eixida.toString().trim());

            n = sc.nextInt();
            sc.nextLine();
```

```
        }
        sc.close();
    }
}
```

Dues idees de la U07 treballant alhora: la **bombolla** per a ordenar (adaptada a `String` amb `compareTo`) i la idea de recórrer per a **filtrar repetits** (comparant amb l'anterior una vegada ordenat). `split("--")` trosseja cada línia, i només els que tenen revers diferent de `FIN` entren en la llista d'odiats.

---

6. 342 — No ho pots saber!

Un jugador pensa un nombre ocult en un rang `[ini, fin]` i un segon jugador fa preguntes ("El 500?") rebent respostes **"menor"** o **"major o igual"**. Donats el rang, el nombre ocult i la llista de preguntes realitzades, digues si el segon jugador pot **saber amb certesa** quin és el nombre.

Entrada: diversos casos. Cada cas: una línia amb `ini`, `fin` i `n` (el nombre ocult). La línia següent té `k` (nre. de preguntes) i les `k` hipòtesis. Termina amb `0 0 0`.

Exemple:

```
1 10 1
2 1 2
1 1000 450
2 400 500
0 0 0
```

Sortida:

```
HO SAP
NO HO SAP
```

En el primer cas, el jugador pregunta per 1 i 2: respon "major o igual" (1) i "menor" (2), així que només queda l'1 → HO SAP. En el segon, amb les preguntes 400 i 500 només acota a 450, 451… 500 → encara queden diversos → NO HO SAP.

- Enunciat en AceptaElReto
- Dificultat: Mitjana

Pista: és la **cerca binària** del punt 3 al revés: cada pregunta divideix el rang possible en dos segons la resposta. Porta `min` i `max` (els límits possibles). Si la hipòtesi és menor que l'ocult, `min` puja a hipòtesi + 1; si és major o igual, `max` baixa a la hipòtesi. Al final, HO SAP si `min == max == n`.

> 🔄 **Solució**

```
import java.util.Scanner;

public class NoHoPotsSaber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int ini = sc.nextInt();
            int fin = sc.nextInt();
            int n = sc.nextInt();
            if (ini == 0 && fin == 0 && n == 0) break;

            int min = ini;
            int max = fin;
            int k = sc.nextInt();

            for (int i = 0; i < k; i++) {
                int hipotesi = sc.nextInt();
                if (hipotesi < n) {
                    min = Math.max(min, hipotesi + 1);
                } else {
                    max = Math.min(max, hipotesi);
                }
            }

              System.out.println((min == max && min == n) ? "HO SAP" : "NO HO
SAP");
        }
        sc.close();
    }
}
```

Cada resposta estreny l'interval: si la hipòtesi és menor que l'ocult, el nombre no pot estar en `[ini, hipotesi]`, així que `min` puja a `hipòtesi + 1`. Si és major o igual, `max` baixa a la

hipòtesi. És el mateix raonament de la cerca binària del punt 3: partir el rang per la meitat (o per on toqui) i descartar. Al final, el jugador "ho sap" només si l'interval s'ha tancat per complet sobre el nombre ocult. La comparació amb `n` usa la dada que nosaltres sí coneixem, però el programa simula el que el jugador va descobrint.

🧭 **¿I si et quedes amb ganes?** Quan domines cerques, bombolles i Big O, torna als problemes d'aquesta unitat amb el punt 9 com a examen. I en la U08 t'esperen les bèsties: QuickSort, MergeSort i la recursivitat. Ordenar un milió de dades en un sospir està a un pas.

Butlletí U08 — Inicial

📝 Butlletí U08 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, respira fondo i recorda: per a entendre la recursivitat, primer has d'entendre la recursivitat.

Exercici 1: El factorial del valent

Escriu un programa anomenat `FactorialValent` amb un mètode recursiu `static long fact(int n)` que calculi el factorial d'un nombre. Prova'l amb `fact(5)`, `fact(0)` i `fact(10)`.

Recorda: `0! = 1` i `n! = n * (n-1)!`.

Exercici 2: L'àvia suma l'array

Escriu un programa anomenat `SumaRecursiva` que sumi tots els elements d'un array usant un mètode recursiu:

```
static int sumar(int[] arr, int indice)
```

Si `indice == arr.length`, torna 0. Si no, `arr[indice] + sumar(arr, indice + 1)`. Prova'l amb `{3, 8, 2, 10, 5}` → ha de donar 28.

Exercici 3: Què imprimeix? — la compte enrere recursiva

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class CompteRara {
    static void compte(int n) {
        if (n == 0) {
            System.out.println("¡BOOM!");
            return;
        }
        System.out.println(n);
        compte(n - 1);
    }

    public static void main(String[] args) {
        compte(3);
    }
}
```

Pista: primer baixa imprimint nombres i, quan arriba al cas base, fa BOOM.

Exercici 4: La potència de l'aprenent

Escriu un programa anomenat `PotenciaRecursiva` amb un mètode recursiu `static double potencia(double base, int exponente)` que calculi `base^exponente`:

- Cas base: `exponente == 0` → torna 1.
- Cas recursiu: `base * potencia(base, exponente - 1)`.

Prova'l amb `potencia(2, 10)` → 1024.

Exercici 5: El detectiu de dígits

Escriu un programa anomenat `ContadorDigitos` amb un mètode recursiu `static int contarDigitos(int n)` que compte quants dígits té un nombre:

- Cas base: `n < 10` → torna 1.
- Cas recursiu: `1 + contarDigitos(n / 10)`.

Prova'l amb `12345` → ha de donar 5.

Exercici 6: El mirall de lletres

Escriu un programa anomenat `InvertirText` amb un mètode recursiu `static String invertir(String s)` que torni el text al revés:

- Cas base: `s.isEmpty()` → torna `""`.
- Cas recursiu: `invertir(s.substring(1)) + s.charAt(0)`.

Prova'l amb `"hola"` → ha de tornar `"aloh"`.

Exercici 7: Què imprimeix? — l'enigma del stack

Sense executar, escriu la sortida exacta:

```
public class EnigmaStack {
    static int misterio(int n) {
        if (n == 0) return 0;
        return n % 10 + misterio(n / 10);
    }

    public static void main(String[] args) {
        System.out.println(misterio(1234));
    }
}
```

Pista: `n % 10` extrau l'última xifra i `n / 10` la lleva.

Exercici 8: El guardià dels palíndroms

Escriu un programa anomenat `GuardiaPalindroms` amb un mètode recursiu que comprovi si una paraula és un palíndrom (es llegeix igual cap endavant i cap enrere):

```
static boolean esPalindromo(String s, int inicio, int fin)
```

Prova'l amb `"reconocer"` (sí), `"salas"` (sí) i `"hola"` (no). Pista: si les puntes no coincideixen, ja pots tornar `false` sense continuar recursant.

Exercici 9: CodeWars — Factorial

Resol la kata **“Factorial”** (7 kyu) en CodeWars.

Crea el mètode `public static long factorial(int n)` que torni el factorial de `n` (recorda: `0! = 1`). ¡Resol-la amb recursivitat, que és el que toca!

Pista: `if (n <= 1) return 1; return n * factorial(n - 1);`

Butlletí U08 — Inicial (Resolt)

📝 Butlletí U08 — Inicial (Resolt)

> Les solucions estan amagades darrere de cada 🔄. Primer intenta resoldre'ls tu; mirar la resposta abans d'intentar-ho és com copiar en un examen de màgia: el truc s'espatlla.

Exercici 1: El factorial del valent

> 🔄 Solució

```
public class FactorialValent {

    static long fact(int n) {
        if (n <= 1) return 1;
        return n * fact(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("fact(5)  = " + fact(5));   // 120
        System.out.println("fact(0)  = " + fact(0));   // 1
        System.out.println("fact(10) = " + fact(10));  // 3628800
    }
}
```

> Sortida: `fact(5) = 120`, `fact(0) = 1`, `fact(10) = 3628800`.

Exercici 2: L'àvia suma l'array

> 🔄 Solució

```java
public class SumaRecursiva {

    static int sumar(int[] arr, int indice) {
        if (indice == arr.length) return 0;
        return arr[indice] + sumar(arr, indice + 1);
    }

    public static void main(String[] args) {
        int[] datos = {3, 8, 2, 10, 5};
        System.out.println("Suma: " + sumar(datos, 0));   // 28
    }
}
```

Sortida: `Suma: 28`. El cas base quan `indice` surt de l'array torna 0 (element neutre de la suma).

Exercici 3: Què imprimeix? — la compte enrere recursiva

› 🔄 Solució

```
3
2
1
¡BOOM!
```

Imprimeix 3, 2 i 1 mentre baixa recursivament, i en arribar al cas base (`n == 0`) fa ¡BOOM! i la pila es va desenrotllant sense fer res més.

Exercici 4: La potència de l'aprenent

› 🔄 Solució

```java
public class PotenciaRecursiva {

    static double potencia(double base, int exponente) {
        if (exponente == 0) return 1;
        return base * potencia(base, exponente - 1);
    }

    public static void main(String[] args) {
        System.out.println("2^10 = " + potencia(2, 10));   // 1024
        System.out.println("3^3  = " + potencia(3, 3));    // 27
        System.out.println("5^0  = " + potencia(5, 0));    // 1
    }
}
```

Sortida: `2^10 = 1024.0`, `3^3 = 27.0`, `5^0 = 1.0`.

Exercici 5: El detectiu de dígits

› 🔄 Solució

```java
public class ContadorDigitos {

    static int contarDigitos(int n) {
        if (n < 10) return 1;
        return 1 + contarDigitos(n / 10);
    }

    public static void main(String[] args) {
          System.out.println("12345 té " + contarDigitos(12345) + " dígits");
// 5
              System.out.println("7  té  "  +  contarDigitos(7)  +  "  dígits");
// 1
    }
}
```

Sortida: `12345 té 5 dígits`, `7 té 1 dígit`. Cada crida lleva l'última xifra (`n / 10`) i suma 1.

Exercici 6: El mirall de lletres

› 🔄 Solució

```
public class InvertirText {

    static String invertir(String s) {
        if (s.isEmpty()) return "";
        return invertir(s.substring(1)) + s.charAt(0);
    }

    public static void main(String[] args) {
        System.out.println("\"hola\" → " + invertir("hola"));   // aloh
        System.out.println("\"juan\" → " + invertir("juan"));    // nauj
    }
}
```

Sortida: `"hola" → aloh`, `"juan" → nauj`. La crida recursiva primer inverteix la resta i després concatena el primer caràcter al final.

Exercici 7: Què imprimeix? — l'enigma del stack

› 🔄 Solució

```
10
```

És una **suma de dígits recursiva**: 4 + 3 + 2 + 1 = 10. En cada crida, `n % 10` extrau l'última xifra i `n / 10` la lleva al nombre.

Exercici 8: El guardià dels palíndroms

› 🔄 Solució

```java
public class GuardiaPalindroms {

    static boolean esPalindromo(String s, int inicio, int fin) {
        if (inicio >= fin) return true;
        if (s.charAt(inicio) != s.charAt(fin)) return false;
        return esPalindromo(s, inicio + 1, fin - 1);
    }

    public static void main(String[] args) {
         System.out.println("reconocer: " + esPalindromo("reconocer", 0, 8));
// true
          System.out.println("salas:       " + esPalindromo("salas", 0, 4));
// true
           System.out.println("hola:        " + esPalindromo("hola", 0, 3));
// false
    }
}
```

Sortida: `reconocer: true`, `salas: true`, `hola: false`. El truc: comparar les puntes i, si no coincideixen, tornar `false` sense gastar més stack.

Exercici 9: CodeWars — Factorial

› 🔄 Solució

```java
public class Kata {

    public static long factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
}
```

Amb `factorial(5)` → `120` i `factorial(0)` → `1`. El cas base `n <= 1` cobreix tant el 0 com l'1, i el cas recursiu encadena les multiplicacions.

Butlletí U08 — Avançat

📝 Butlletí U08 — Avançat

> La zona intermèdia: aquí ja no n'hi ha prou amb copiar el patró, cal pensar. Cada exercici porta la seva **Pista** per quan portes 10 minuts donant-li vegades.

Exercici 1: ⭐ Fibonacci amb el comptador xafarder

Crea un programa `FiboContador` que calculi l'enèssim Fibonacci amb la versió ingènua recursiva PERÒ que a més compte **quantes crides fa en total**.

```
static long crides;   // variable estàtica que suma 1 en cada crida
static long fibo(int n)
```

Prova'l amb `fibo(30)`. Quantes crides fa? I quantes en faria si només sumares els índexs `fibo(30) + fibo(29) + ... + 1`?

Pista: suma 1 a `crides` com a primera línia del mètode, abans de qualsevol cas base.

Exercici 2: ⭐ ⭐ Què imprimeix? — l'arbre de crides

Sense executar, escriu la sortida exacta i l'**ordre de les crides** (qui crida a qui):

```java
public class ArbreRare {
    static void pintar(int n) {
        if (n == 0) return;
        System.out.println("baixe " + n);
        pintar(n - 1);
        System.out.println("puge " + n);
    }

    public static void main(String[] args) {
        pintar(3);
    }
}
```

Pista: el `System.out.println` de després de la crida recursiva no s'executa fins que aquella crida acaba. Dibuixa la pila amb llapis i paper.

Exercici 3: ⭐⭐ El palíndrom rebel

Amplia el palíndrom perquè **ignore espais, signes i majúscules**:

- `"Anita lava la tina"` → true
- `"La ruta natural"` → true
- `"No soy un palíndromo"` → false

```java
static boolean esPalindromoFrase(String s, int inicio, int fin)
```

Pista: en lloc de comparar directament `s.charAt(inicio)`, salta't els caràcters que no siguin lletres avançant `inicio` o retrocedint `fin` dins del mateix mètode. Usa `Character.isLetter()` i `Character.toLowerCase()`.

Exercici 4: ⭐ ⭐ La potència exprés (divide i venceràs)

Escriu un programa `PotenciaRapida` amb un mètode recursiu `static long potenciaRapida(int base, int exponente)` que calculi `base^exponente` en **O(log n)**:

```
potenciaRapida(b, e):
  si e == 0 → 1
  meitat = potenciaRapida(b, e / 2)
  si e és parell  → meitat * meitat
  si e és senar → meitat * meitat * b
```

Prova'l amb `potenciaRapida(2, 20)` → 1048576. Quantes crides fa comparat amb `potencia(2, 20)` del butlletí inicial?

Pista: ull amb l'arrodoniment: quan `e` és senar, `e / 2` es queda amb la part entera i per això multipliques per `b` una vegada més.

Exercici 5: ⭐ ⭐ ⭐ Quicksort amb mediana de tres

Millora el Quicksort de la unitat: en lloc d'agafar el primer element com a pivot, tria la **mediana de tres** (primer, mitjà i últim) per a evitar el pitjor cas amb arrays quasi ordenats.

```
static void quicksort(int[] arr, int inicio, int fin)
```

Prova'l amb l'array ja ordenat `{1, 2, 3, 4, 5, 6, 7, 8}`. Quantes particions fa la teva versió amb mediana de tres?

Pista: `int medio = (inicio + fin) / 2;` compara `arr[inicio]`, `arr[medio]` i `arr[fin]` i col·loca el del mig en `arr[inicio]` (intercanviant-los) abans de particionar amb la tècnica de la unitat.

Exercici 6: ⭐⭐⭐ Mergesort amb el comptador de comparacions

Modifica el Mergesort de la unitat perquè compte **quantes comparacions** fa la fusió en total, i mostra el nombre al final.

```
static long comparacions;   // suma 1 en cada comparació de la fusió
```

Prova'l amb `{9, 8, 7, 6, 5, 4, 3, 2, 1}` (el pitjor cas visual). Quantes comparacions? I amb `{1, 2, 3, 4, 5, 6, 7, 8, 9}`?

Pista: en la fusió, suma 1 en cada `while` que compara dos elements. En els bucles on una llista ja s'ha acabat, també hi ha comparacions contra el final de l'array: compta-les amb la mateixa variable.

Exercici 7: ⭐ ⭐ ⭐ Les torres de Hanói amb comptador

Implementa les Torres de Hanói recursives de la unitat i afegeix un **comptador de moviments**:

```
static int moviments;
static void hanoi(int n, char origen, char destino, char auxiliar)
```

Cada vegada que es mova un disc, imprimeix `"Mueve disco X de ORIGEN a DESTINO"` i suma 1 al comptador. Al final, imprimeix el total. Prova'l amb 3, 4 i 8 discos.

Pista: amb `n` discos el mínim de moviments és `2^n - 1`. Si el teu comptador amb 8 discos no dona 255, alguna cosa estàs movent de més.

Exercici 8: ⭐⭐⭐ CodeWars — Sort Numbers

Resol la kata **"Sort Numbers"** (7 kyu) en CodeWars.

Crea el mètode `public static int[] sortArray(int[] nums)` que torni l'array ordenat de menor a major. Si `nums` és `null` o està buit, torna un array buit.

Pista: fes un mètode `int[] copia` (per a no mutar l'original) i ordena-la amb `Arrays.sort()`. O, si tens ganes, implementa el teu propi Mergesort sobre la còpia.

Exercici 9: ⭐⭐⭐ AceptaElReto — 104 Mòbils

Resol el problema **"Mòbils" (104)** de AceptaElReto.

Un mòbil penja d'una barra amb dos pesos `pi` i `pd` a distàncies `di` i `dd`. Està equilibrat si `pi * di == pd * dd` i a més les dues subbarres (que poden contenir altres mòbils) també ho estan. Un pes `0` significa que dalt penja un altre mòbil, que es descriu a continuació. Torna `SI` si el mòbil està equilibrat i `NO` en cas contrari.

Pista: no hi ha cas base amb `n`: els mòbils es llegeixen recursivament. Llegeix `pi di pd dd`; si `pi == 0`, cal llegir (i comprovar) un submòbil complet abans de continuar; si `pd == 0`, un altre. Recorda que **ambdós** submòbils han d'estar equilibrats, no només la barra.

Butlletí U08 — Avançat (Resolt)

📝 Butlletí U08 — Avançat (Resolt)

> Els exercicis avançats amb les seves solucions. Intenta'ls primer; baixar a veure la solució sense suar és com demanar el 10 abans de l'examen.

Exercici 1: ⭐ Fibonacci amb el comptador xafarder

> 🔄 Solució

```
public class FiboContador {

    static long crides;

    static long fibo(int n) {
        crides++;
        if (n <= 1) return n;
        return fibo(n - 1) + fibo(n - 2);
    }

    public static void main(String[] args) {
        long resultat = fibo(30);
        System.out.println("fibo(30) = " + resultat);
        System.out.println("crides = " + crides);
    }
}
```

Sortida: `fibo(30) = 832040` i `crides = 2692537`. ¡Més de 2,6 milions de crides per a un nombre de 6 xifres! La versió ingènua és brutalment cara: cada `fibo(n)` torna a calcular `fibo(n-1)` i `fibo(n-2)` per complet.

El "truc dels índexs" (`fibo(n) + fibo(n-1) + ... + 1`) NO funciona aquí perquè les crides no s'encadenen netament: cada terme de l'arbre recalcula subarbres sencers.

Exercici 2: ⭐ ⭐ Què imprimeix? — l'arbre de crides

> 🔄 Solució

```
baixe 3
baixe 2
baixe 1
puge 1
puge 2
puge 3
```

L'ordre: `pintar(3)` imprimeix "baixi 3", crida a `pintar(2)`, que imprimeix "baixi 2", crida a `pintar(1)`, que imprimeix "baixi 1" i crida a `pintar(0)` (que no fa res). En tornar, cada nivell executa el seu `println` pendent: primer el d'1, després el de 2, després el de 3. La pila es desenvolupa en ordre invers: **baixa tot, després puja tot**.

Exercici 3: ⭐⭐ El palíndrom rebel

> 🔄 Solució

```java
public class PalindromRebel {

    static boolean esPalindromoFrase(String s, int inicio, int fin) {
        if (inicio >= fin) return true;

        if (!Character.isLetter(s.charAt(inicio))) {
              return esPalindromoFrase(s, inicio + 1, fin);    // salta el no-
lletra
        }
        if (!Character.isLetter(s.charAt(fin))) {
              return esPalindromoFrase(s, inicio, fin - 1);    // salta el no-
lletra
        }

        char a = Character.toLowerCase(s.charAt(inicio));
        char b = Character.toLowerCase(s.charAt(fin));
        if (a != b) return false;

        return esPalindromoFrase(s, inicio + 1, fin - 1);
    }

    public static void main(String[] args) {
          System.out.println(esPalindromoFrase("Anita lava la tina", 0, 17));
// true
            System.out.println(esPalindromoFrase("La ruta natural", 0, 14));
// true
         System.out.println(esPalindromoFrase("No soy un palindromo", 0, 19));
// false
    }
}
```

Sortida: `true`, `true`, `false`. La clau està en els dos `if` que salten els caràcters que no són lletres ABANS de comparar. Ull: `Character.toLowerCase()` no lleva accents, així que les frases d'exemple es trien sense tildes («La ruta natural»). Si tinguessis tildes («Dábale arroz a la zorra el abad»), hauries de normalitzar els accents apart.

Exercici 4: ⭐ ⭐ La potència exprés (divide i venceràs) 🔗

› 🔄 Solució

```
public class PotenciaRapida {

    static long potenciaRapida(int base, int exponente) {
        if (exponente == 0) return 1;
        long meitat = potenciaRapida(base, exponente / 2);
        if (exponente % 2 == 0) {
            return meitat * meitat;
        } else {
            return meitat * meitat * base;
        }
    }

    public static void main(String[] args) {
        System.out.println("2^20 = " + potenciaRapida(2, 20));   // 1048576
        System.out.println("3^10 = " + potenciaRapida(3, 10));   // 59049
    }
}
```

Sortida: `2^20 = 1048576`, `3^10 = 59049`. `potenciaRapida(2, 20)` fa només **5 crides recursives** (20 → 10 → 5 → 2 → 1 → 0), mentre que la versió lineal del butlletí inicial en fa 20. De O(n) a O(log n): aquell salt és tot el divide i venceràs. Quan e és senar, `e / 2` arrodoneix cap avall i per això cal multiplicar per `base` una vegada més.

Exercici 5: ⭐ ⭐ ⭐ Quicksort amb mediana de tres 🔗

› 🔄 Solució

```java
public class QuicksortMedianaTres {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;

        int medio = (inicio + fin) / 2;

        // triar la mediana de arr[inicio], arr[medio], arr[fin] i portar-la a arr[inicio]
        if (arr[medio] < arr[inicio]) intercanviar(arr, inicio, medio);
        if (arr[fin] < arr[inicio]) intercanviar(arr, inicio, fin);
        if (arr[fin] < arr[medio]) intercanviar(arr, medio, fin);
        intercanviar(arr, inicio, medio);

        int pivote = arr[inicio];
        int i = inicio + 1;

        for (int j = inicio + 1; j <= fin; j++) {
            if (arr[j] < pivote) {
                intercanviar(arr, i, j);
                i++;
            }
        }

        intercanviar(arr, inicio, i - 1);

        quicksort(arr, inicio, i - 2);
        quicksort(arr, i, fin);
    }

    static void intercanviar(int[] arr, int a, int b) {
        int tmp = arr[a];
        arr[a] = arr[b];
        arr[b] = tmp;
    }

    public static void main(String[] args) {
        int[] datos = {1, 2, 3, 4, 5, 6, 7, 8};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Sortida: `[1, 2, 3, 4, 5, 6, 7, 8]`. Amb l'array ja ordenat, el pivot ja NO és el menor (o el major): amb la mediana de tres (`inicio`, `medio`, `fin`) el pivot de la primera crida és 4, partint l'array per la meitat. Així s'evita el pitjor cas O(n²) dels arrays quasi ordenats. Els tres `if` col·loquen el valor del mig dels tres en `arr[medio]`, i després s'intercanvia a `arr[inicio]` per a no canviar la tècnica de partició de la unitat.

Exercici 6: ⭐⭐⭐ Mergesort amb el comptador de comparacions 🔗

› 🔄 Solució

```java
public class MergesortContador {

    static long comparacions;

    static void mergesort(int[] arr) {
        if (arr.length <= 1) return;
        int medio = arr.length / 2;

        int[] izq = new int[medio];
        int[] der = new int[arr.length - medio];

        for (int i = 0; i < medio; i++) izq[i] = arr[i];
        for (int i = medio; i < arr.length; i++) der[i - medio] = arr[i];

        mergesort(izq);
        mergesort(der);
        fusionar(arr, izq, der);
    }

    static void fusionar(int[] destino, int[] izq, int[] der) {
        int i = 0, j = 0, k = 0;

        while (i < izq.length && j < der.length) {
            comparacions++;
            if (izq[i] <= der[j]) {
                destino[k++] = izq[i++];
            } else {
                destino[k++] = der[j++];
            }
        }

        while (i < izq.length) {
            comparacions++;
            destino[k++] = izq[i++];
        }
        while (j < der.length) {
            comparacions++;
            destino[k++] = der[j++];
        }
    }

    public static void main(String[] args) {
```

```
        comparacions = 0;
        int[] datos = {9, 8, 7, 6, 5, 4, 3, 2, 1};
        mergesort(datos);
        System.out.println(java.util.Arrays.toString(datos));
        System.out.println("comparacions (cas dolent) = " + comparacions);

        comparacions = 0;
        int[] datos2 = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        mergesort(datos2);
        System.out.println("comparacions (ja ordenat) = " + comparacions);
    }
}
```

Sortida (exemple): `[1, 2, 3, 4, 5, 6, 7, 8, 9]`, `comparacions (cas dolent) = 25`, `comparacions (ja ordenat) = 18`. Mergesort sempre fa ~n·log₂(n) comparacions (per a n=9, 9·3,17 ≈ 28 en el pitjor cas teòric), per això és tan previsible: l'ordre de l'entrada amb prou faenes canvia el total. Compara-ho amb Quicksort, on l'entrada ho canvia TOT.

Exercici 7: ⭐ ⭐ ⭐ Les torres de Hanói amb comptador 🔗

› 🔄 Solució

```java
public class HanoiContador {

    static int moviments;

    static void hanoi(int n, char origen, char destino, char auxiliar) {
        if (n == 1) {
                System.out.println("Mueve disco 1 de " + origen + " a " +
destino);
            moviments++;
            return;
        }
        hanoi(n - 1, origen, auxiliar, destino);
          System.out.println("Mueve disco " + n + " de " + origen + " a " +
destino);
        moviments++;
        hanoi(n - 1, auxiliar, destino, origen);
    }

    public static void main(String[] args) {
        moviments = 0;
        hanoi(3, 'A', 'C', 'B');
        System.out.println("Total de moviments: " + moviments);
    }
}
```

Amb 3 discos, `Total de moviments: 7`. Amb 4 → 15, i amb 8 → 255. La fórmula `2^n - 1` es compleix exactament. El patró clàssic: moure `n-1` a l'auxiliar, moure el disc gran, moure `n-1` de l'auxiliar al destí. Cada moviment imprés suma 1 al comptador.

Exercici 8: ⭐⭐⭐ CodeWars — Sort Numbers

› 🔄 Solució

```java
import java.util.Arrays;

public class Kata {

    public static int[] sortArray(int[] nums) {
        if (nums == null || nums.length == 0) return new int[0];

        int[] copia = nums.clone();
        Arrays.sort(copia);
        return copia;
    }
}
```

Amb `{1, 5, 2, 3, 4}` → `{1, 2, 3, 4, 5}`, amb `null` → `[]`, amb `{}` → `[]`. Es clona l'array per a no mutar l'original i s'ordena la còpia.

Exercici 9: ⭐⭐⭐ AceptaElReto — 104 Mòbils

> 🔄 Solució

```java
import java.util.Scanner;

public class Mobils {

    // Llig un mòbil complet (la seua barra i els seus submòbils) i torna:
    //   el seu pes total si està equilibrat
    //   -1 si NO està equilibrat
    static int llegirMobil(Scanner sc) {
        int pi = sc.nextInt();   // pes esquerre
        int di = sc.nextInt();   // distància esquerra
        int pd = sc.nextInt();   // pes dret
        int dd = sc.nextInt();   // distància dreta

        boolean ok = true;

        if (pi == 0) {                // hi ha un submòbil a l'esquerra
            int sub = llegirMobil(sc);
            if (sub == -1) ok = false;
            else pi = sub;
        }
        if (pd == 0) {                // hi ha un submòbil a la dreta
            int sub = llegirMobil(sc);
            if (sub == -1) ok = false;
            else pd = sub;
        }

        if (!ok) return -1;                        // algun submòbil ja estava
desequilibrat
        if (pi * di != pd * dd) return -1;        // esta barra no s'equilibra
        return pi + pd;                           // el pes total d'este mòbil
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int pi = sc.nextInt();
            int di = sc.nextInt();
            int pd = sc.nextInt();
            int dd = sc.nextInt();
```

```
            if (pi == 0 && di == 0 && pd == 0 && dd == 0) break;    // fi de
l'entrada

            boolean ok = true;
            if (pi == 0) {
                int sub = llegirMobil(sc);
                if (sub == -1) ok = false;
                else pi = sub;
            }
            if (pd == 0) {
                int sub = llegirMobil(sc);
                if (sub == -1) ok = false;
                else pd = sub;
            }

            boolean equilibrada = ok && pi * di == pd * dd;
            System.out.println(equilibrada ? "SI" : "NO");
        }
    }
}
```

El cas base no és un nombre: el mòbil “fulla” és aquell els dos costats del qual tenen pes > 0 (no hi ha submòbils a llegir). Cada crida recursiva llegeix i comprova un submòbil complet i torna el seu pes total. El flag `ok` (o el `-1`) propaga cap amunt que TOTES les subbarres estiguin equilibrades. Entrada `0 0 0 0` marca el final de l’entrada.

Butlletí U08 — Extres

📝 Butlletí U08 — Extres

> T'has quedat amb ganes de més? Aquesta secció és per als que volen arribar a l'examen amb el stack templat: 4 katas de CodeWars i 2 problemes d'AceptaElReto. Cada pista està amagada al seu desplegable per a no espatlar el misteri.

🥋 CodeWars

> Regista't a CodeWars, tria Java i resol les katas per ordre. Comencen assequibles i pugen de nivell.

Kata 1: Factorial (7 kyu)

En CodeWars.

En matemàtiques, el factorial d'un enter no negatiu `n`, denotat `n!`, és el producte de tots els enters positius menors o iguals que `n`:

```
n! = 1 * 2 * 3 * ... * n
```

Amb la convenció que `0! = 1`. Implementa `public static long factorial(int n)` amb recursivitat.

> › 💡 **Pista**
>
> Cas base `n <= 1` → 1; cas recursiu `n * factorial(n - 1)`.

Kata 2: Descending Order (7 kyu)

En CodeWars.

Agafa un nombre no negatiu i torna el **nombre major possible** reordenant els seus dígits de major a menor. Exemples:

- 42145 → 54421

- `145263` → `654321`
- `123456789` → `987654321`

> 💡 **Pista**
>
> Converteix el nombre a `String`, passa a un array de `char` o `int`, ordena'l descendent i reconstruïx el nombre amb `Long.parseLong`. Aquest exercici et demana ordenar, no recursivitat: usa el que has après.

Kata 3: Are the numbers in order? (7 kyu)

En CodeWars.

Escriu `public static boolean isAscOrder(int[] arr)` que torni `true` si els nombres estan **estrictament en ordre ascendent** (cada un major que l'anterior) i `false` en cas contrari.

- `[1, 2, 3, 4]` → true
- `[1, 4, 13, 97, 508, 717]` → true
- `[1, 3, 2]` → false
- `[9, 5, 5]` → false (no és estrictament ascendent)

> 💡 **Pista**
>
> Un sol `for` amb `i` de 1 fins al final comparant `arr[i - 1] >= arr[i]` → si alguna vegada es compleix, torna `false` al moment.

Kata 4: Tribonacci Sequence (6 kyu)

En CodeWars.

Igual que Fibonacci però amb tres nombres d'arrancada: cada terme és la suma dels tres anteriors. L'enunciat sempre dona una llavor `double[] signature` de 3 elements i un nombre `n`. Torna els primers `n` termes:

- `{1, 1, 1}, 10` → `{1, 1, 1, 3, 5, 9, 17, 31, 57, 105}`
- `{0, 0, 1}, 10` → `{0, 0, 1, 1, 2, 4, 7, 13, 24, 44}`
- `{1, 1, 1}, 1` → `{1}` (recorda: si `n < 3`, només tornes els que demani)

> 💡 **Pista**

> Si `n < 3`, torna un subarray amb els primers `n` de la llavor. Si no, crea un `double[n]`, copia la llavor al principi i ompli de `i = 3` endavant amb `arr[i] = arr[i-1] + arr[i-2] + arr[i-3]`. Pots fer-ho iteratiu; el "truc del tribonacci" no necessita recursivitat.

🏆 AceptaElReto

> Dos problemes del concurs ProgramaMe. El primer és un clàssic absolut de la recursivitat; el segon, un sudorífer d'ordenació.

Repre 1: Mòbils (104)

En AceptaElReto.

Un mòbil penjant es descriu amb quatre enters: `pi di pd dd` (pes i distància esquerres, pes i distància dretes). La barra està equilibrada si `pi * di == pd * dd`. Un pes `0` indica que dalt penja un altre mòbil, que es descriu a continuació en l'entrada. Has de dir `SI` si el mòbil complet està equilibrat (barra I submòbils) i `NO` en cas contrari. L'entrada acaba amb `0 0 0 0`.

> › 💡 **Pista**
>
> No hi ha comptador de casos: els mòbils es llegeixen **recursivament**. Crea un mètode que llegeixi una barra i torni el pes total del mòbil si està equilibrat, o `-1` si no. Quan un pes és 0, crida al mètode per a llegir el submòbil. Un submòbil desequilibrat (torna `-1`) contamina tota la cadena cap amunt.

Repre 2: Tortetes (143)

En AceptaElReto.

Tens una pila de tortetes amb mides diferents. Amb una espàtula pots donar la volta a les `k` tortetes superiors (invertir l'ordre de les primeres `k`). Troba el mínim nombre de vegades per a ordenar-les de major (dalt) a menor (baix). L'entrada acaba amb `0`. Per a cada cas, imprimeix quantes vegades necessites.

> › 💡 **Pista**

> Voraç (greedy): busca la torteta més gran que no estigui al seu lloc. Si no està dalt, dona-li la volta fins a la seva posició (invertir `k` on `k` és la seva posició) per a pujar-la. Després inverteix tota la part sense ordenar per a baixar-la al seu lloc. Cada torteta queda fixada per sempre: aquell és el famós “flip-sort”. Amb 2 vegades per torteta mal col·locada et val.

Butlletí U09 — Inicial

📝 Butlletí U09 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, encén la fàbrica de galetes i fes que els teus objectes neixin, parlen i funcionen. Ningú no neix sabent quan cal usar `this`.

Exercici 1: El gos que parla

Escriu una classe `Gos` amb dos atributs: `String nom` i `int edat`. Inclou un mètode `void lladrar()` que imprimeixi `"¡Guau guau! Soc [nom]"`. Després, en un `main`, crea un gos anomenat `Toby` de 3 anys i fes que lladre.

Pista: recorda que els atributs s'inicialitzen en el constructor amb `this`.

Exercici 2: El telèfon amb bateria

Escriu una classe `Telefon` amb els atributs `String marca` i `int bateria`. Inclou un mètode `void trucar()` que imprimeixi `"Trucant... (bateria al X%)"` i un altre `void carregar(int minuts)` que pugi la bateria en els minuts rebuts. Crea en el `main` un `Telefon` de marca `"Nokia"` amb bateria inicial `50`, truca-hi i carrega'l `30` minuts.

Pista: `carregar(int minuts)` modifica l'atribut `bateria`: `this.bateria += minuts;`.

Exercici 3: L'estudiant amb nota

Escriu una classe `Estudiant` amb els atributs `String nom` i `double nota`. Inclou un mètode `void mostrarEstat()` que imprimeixi `"[nom] ha aprovat"` si la nota és major o igual que 5, o `"[nom] ha suspés"` si és menor. Crea en el `main` dos estudiants (una amb nota 8.5 i un altre amb nota 3) i mostra'n l'estat.

Pista: dins de `mostrarEstat()` pots usar `if/else` amb `this.nota`.

Exercici 4: La pel·lícula en cartellera

Escriu una classe `Pelicula` amb els atributs `String titol` i `String genere`. Inclou un mètode `void mostrarCartellera()` que imprimeixi `"Ara en cinemes: [titol] ([genere])"`. Crea en el `main` una pel·lícula `"El curs de Java"` de gènere `"Terror"` i mostra-la en cartellera.

Pista: el constructor rep els dos valors i els assigna amb `this`.

Exercici 5: La persona educada

Escriu una classe `Persona` amb els atributs `String nom` i `int edat`. Usa un constructor amb paràmetres. Afegeix un mètode `void presentar-se()` que imprimeixi `"Hola, soc [nom] i tinc [edat] anys."`. Crea en el `main` a `"Anna"` de 25 anys i fes que es presente.

Pista: aquesta és la classe amb què entendràs per què `this` separa l'atribut del paràmetre.

Exercici 6: El cercle calculador

Escriu una classe `Cercle` amb l'atribut `double radi`. Inclou un mètode `double calcularArea()` que torni l'àrea ($\pi * radi^2$). Usa `Math.PI` per a π. Crea en el `main` un cercle de radi `2.5` i mostra'n l'àrea.

Pista: dins del mètode, `return Math.PI * this.radi * this.radi;`.

Exercici 7: El videojoc jugable

Escriu una classe `Videojoc` amb els atributs `String titol` i `int horesJugades`. Inclou un mètode `void jugar(int hores)` que sumi les hores a `horesJugades` i un altre `void mostrar()` que imprimeixi `"[titol] - X hores"`. Crea en el `main` un joc amb 10 hores, juga'n 5 més i mostra'l.

Pista: suma les hores a l'atribut: `this.horesJugades += hores;`.

Exercici 8: Què imprimeix? — el taller de cotxes

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class Taller {
    public static void main(String[] args) {
        Coche c1 = new Coche("Seat", 120);
        Coche c2 = new Coche("Ford", 90);
        c1.acelerar(30);
        c1.mostrar();
        c2.mostrar();
    }
}

class Coche {
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad + " km/h");
    }
}
```

Pista: quants objectes hi ha? Cada un guarda la seva pròpia `velocidad`; accelerar el `c1` no toca el `c2`.

Exercici 9: CodeWars — Object Oriented Piracy

Resol la kata **"Object Oriented Piracy"** (8 kyu) en CodeWars.

Crea la classe `Ship` amb un constructor que rebi `draft` (calat en peus) i `crew` (tripulació), i un mètode `boolean isWorthIt()` que torni `true` si `draft - (crew * 1.5) > 20`.

Pista: guarda `draft` i `crew` en atributs amb `this` i usa els operadors `-` i `*` de la UO3 dins de `isWorthIt()`.

Butlletí U09 — Inicial Resolt

📝 Butlletí U09 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: El gos que parla

> 🔄 Solució

```
public class Gos {
    String nom;
    int edat;

    public Gos(String nom, int edat) {
        this.nom = nom;
        this.edat = edat;
    }

    void lladrar() {
        System.out.println("¡Guau guau! Soc " + nom);
    }

    public static void main(String[] args) {
        Gos toby = new Gos("Toby", 3);
        toby.lladrar();
    }
}
```

Sortida: `¡Guau guau! Soc Toby`

El constructor amb `this` col·loca el nom i l'edat en l'objecte acabat de néixer. El mètode `lladrar()` no rep res: usa l'atribut `nom` que ja va quedar guardat.

Exercici 2: El telèfon amb bateria

> 🔄 Solució

```java
public class Telefon {
    String marca;
    int bateria;

    public Telefon(String marca, int bateria) {
        this.marca = marca;
        this.bateria = bateria;
    }

    void trucar() {
        System.out.println("Trucant... (bateria al " + bateria + "%)");
    }

    void carregar(int minuts) {
        this.bateria += minuts;
    }

    public static void main(String[] args) {
        Telefon mobil = new Telefon("Nokia", 50);
        mobil.trucar();
        mobil.carregar(30);
        mobil.trucar();
    }
}
```

Sortida:

```
Trucant... (bateria al 50%)
Trucant... (bateria al 80%)
```

`carregar(int minuts)` modifica l'atribut amb `this.bateria += minuts;`. El mateix objecte conserva la bateria entre trucades: això és tenir estat.

Exercici 3: L'estudiant amb nota

› 🔄 Solució

```java
public class Estudiant {
    String nom;
    double nota;

    public Estudiant(String nom, double nota) {
        this.nom = nom;
        this.nota = nota;
    }

    void mostrarEstat() {
        if (nota >= 5) {
            System.out.println(nom + " ha aprovat");
        } else {
            System.out.println(nom + " ha suspés");
        }
    }

    public static void main(String[] args) {
        Estudiant anna = new Estudiant("Anna", 8.5);
        Estudiant lluis = new Estudiant("Lluís", 3.0);
        anna.mostrarEstat();
        lluis.mostrarEstat();
    }
}
```

Sortida:

```
Anna ha aprovat
Lluís ha suspés
```

El mètode mescla dos mons: usa els atributs de l'objecte (`nom`, `nota`) amb la lògica de decisió (`if/else`) que vas aprendre en la UO4. Dos objectes, dos estats, una classe.

Exercici 4: La pel·lícula en cartellera

› 🔄 **Solució**

```java
public class Pelicula {
    String titol;
    String genere;

    public Pelicula(String titol, String genere) {
        this.titol = titol;
        this.genere = genere;
    }

    void mostrarCartellera() {
        System.out.println("Ara en cinemes: " + titol + " (" + genere + ")");
    }

    public static void main(String[] args) {
        Pelicula p = new Pelicula("El curs de Java", "Terror");
        p.mostrarCartellera();
    }
}
```

Sortida: `Ara en cinemes: El curs de Java (Terror)`

El mateix patró que el gos i el telèfon: atributs, constructor amb `this`, mètode que els usa. A aquestes alçades ja hauries de notar que totes les classes fàcils s'assemblen.

Exercici 5: La persona educada

› 🔄 Solució

```java
public class Persona {
    String nom;
    int edat;

    public Persona(String nom, int edat) {
        this.nom = nom;
        this.edat = edat;
    }

    void presentarSe() {
        System.out.println("Hola, soc " + nom + " i tinc " + edat + " anys.");
    }

    public static void main(String[] args) {
        Persona anna = new Persona("Anna", 25);
        anna.presentarSe();
    }
}
```

Sortida: `Hola, soc Anna i tinc 25 anys.`

Aquí és on `this` demostra el seu valor: sense `this`, `nom = nom` assignaria el paràmetre al paràmetre i l'atribut quedaria en `null`. Amb `this`, l'atribut rep el valor del paràmetre.

Exercici 6: El cercle calculador

› 🔄 Solució

```java
public class Cercle {
    double radi;

    public Cercle(double radi) {
        this.radi = radi;
    }

    double calcularArea() {
        return Math.PI * this.radi * this.radi;
    }

    public static void main(String[] args) {
        Cercle c = new Cercle(2.5);
        System.out.println("Àrea: " + c.calcularArea());
    }
}
```

Sortida: `Àrea: 19.634954084936208`

Aquest mètode no és `void`: usa `return` per a tornar l'àrea. Els mètodes que *calculen* tornen un valor; els que només *mostren* són `void`. Aquella distinció és la meitat de la batalla d'aquesta unitat.

Exercici 7: El videojoc jugable

› 🔄 Solució

```java
public class Videojoc {
    String titol;
    int horesJugades;

    public Videojoc(String titol, int horesJugades) {
        this.titol = titol;
        this.horesJugades = horesJugades;
    }

    void jugar(int hores) {
        this.horesJugades += hores;
    }

    void mostrar() {
        System.out.println(titol + " - " + horesJugades + " hores");
    }

    public static void main(String[] args) {
        Videojoc joc = new Videojoc("Minecraft", 10);
        joc.jugar(5);
        joc.mostrar();
    }
}
```

Sortida: `Minecraft - 15 hores`

El mètode `jugar(int hores)` és un *setter* informal: modifica l'estat de l'objecte. L'objecte no és una caixa estàtica: els seus atributs canvien amb els mètodes. Això és la vida d'un objecte.

Exercici 8: Què imprimeix? — el taller de cotxes

› 🔄 Solució

Imprimeix:

```
Seat va a 150 km/h
Ford va a 90 km/h
```

> Hi ha **dos** objectes (`c1` i `c2`), cada un amb la seva pròpia `velocidad`. `c1.acelerar(30)` suma 30 a la velocitat de `c1` (120 + 30 = 150) però no toca la de `c2`. Quan `c2.mostrar()` corre, la seva velocitat continua sent 90. Els atributs d'un objecte són privats d'aquell objecte.

Exercici 9: CodeWars — Object Oriented Piracy

> 🔄 **Solució**

```
public class Ship {
    private final double draft;
    private final int crew;

    public Ship(double draft, int crew) {
        this.draft = draft;
        this.crew = crew;
    }

    public boolean isWorthIt() {
        return draft - (crew * 1.5) > 20;
    }
}
```

> El constructor guarda `draft` i `crew` amb `this`, i `isWorthIt()` aplica la fórmula: cada membre de la tripulació redueix el calat útil en 1.5 peus. Si el que queda supera 20, el vaixell val la pena. Objecte + operacions de la U03, resolt.

Butlletí U09 — Avançat

📝 Butlletí U09 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: La biblioteca

Escriu una classe `Llibre` amb els atributs `String titol`, `String autor` i `int pagines`. Afegeix un mètode `void mostrarInfo()` que imprimeixi:

```
El Quixot, de Miguel de Cervantes (863 pàgines)
```

Crea en el `main` dos llibres distints i mostra'n la informació.

Pista: tres atributs, un constructor amb tres paràmetres i `this.` tres vegades. El mètode usa els atributs per a construir la frase.

⭐ Exercici 2: El rectangle que raona

Escriu una classe `Rectangle` amb els atributs `double ample` i `double alt`. Afegeix:

- `double calcularArea()` → `ample * alt`
- `double calcularPerimetre()` → `2 * (ample + alt)`
- `boolean esQuadrat()` → `true` si `ample == alt`

Crea un rectangle de 4 x 4 i un altre de 5 x 8 i prova els tres mètodes en tots dos.

Pista: els tres mètodes tornen valors amb `return`. `esQuadrat()` torna el resultat d'una comparació amb ==.

⭐ Exercici 3: El compte bancari blindat

Escriu una classe `CompteBancari` amb els atributs `String titular` i `double saldo`. Afegeix:

- `void ingressar(double quantitat)` → suma al saldo.
- `void retirar(double quantitat)` → resta al saldo **només si** `quantitat <= saldo`; si no, imprimeix `"Saldo insuficient"`.
- `void mostrar()` → imprimeix `"Titular: X | Saldo: Y €"`.

Crea un compte amb 100 €, retira 30 €, intenta retirar 200 € i mostra el saldo final.

Pista: dins de `retirar`, un `if (quantitat <= this.saldo)` protegeix el saldo de quedar-se en negatiu. Aquell `if` és la diferència entre un compte i un clot.

⭐⭐ Exercici 4: L'hora que es corregeix sola

Escriu una classe `Hora` amb els atributs `int hora`, `int minut` i `int segon`. El constructor ha de validar: si els valors no són vàlids (hora entre 0 i 23, minut i segon entre 0 i 59), s'inicialitzen a 0. Afegeix el mètode `void incrementarSegon()` que suma 1 segon gestionant els arrossegaments: si arriba a 60 segons passa a 0 i suma un minut, i així amb els minuts i les hores.

Crea una `Hora(23, 59, 59)`, crida `incrementarSegon()` i mostra `00:00:00`.

Pista: la validació és un `if` gran en el constructor. L'arrossegament són tres `if` encadenats, de segon a minut a hora. Per a mostrar amb dos dígits, imprimeix `"0" + valor` si és menor que 10.

⭐⭐ Exercici 5: Què imprimeix? — el ball de referències

Sense executar, escriu la sortida exacta:

```java
public class Baile {
    public static void main(String[] args) {
        Punto a = new Punto(3, 4);
        Punto b = a;
        b.x = 10;
        System.out.println("a.x = " + a.x);

        Punto c = new Punto(1, 1);
        cambiar(c);
        System.out.println("c.x = " + c.x);
    }

    static void cambiar(Punto p) {
        p.x = 99;
        p = new Punto(50, 50);
    }
}

class Punto {
    int x;
    int y;

    public Punto(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

Pista: `b = a` no copia l'objecte: copia la referència. Quan `cambiar(c)` rep `c`, el paràmetre `p` és una *còpia* de la referència, així que modificar `p.x` sí que es nota, però `p = new Punto(...)` només reassigna el paràmetre local.

⭐⭐ Exercici 6: El correu que s'encadena

Escriu una classe `Email` amb els atributs `String remitent`, `String destinatari` i `String assumpte`. Crea **tres constructors sobrecarregats**:

- `Email(String remitent, String destinatari, String assumpte)` → el complet.

- `Email(String remitent, String destinatari)` → assumpte per defecte "(sense assumpte)".
- `Email(String remitent)` → destinatari "(sense destí)" i assumpte "(sense assumpte)".

Usa `this(...)` per a encadenar i evitar repetir codi. Afegeix `void mostrar()` que imprimeixi les tres dades.

Pista: el constructor d'un paràmetre crida el de dos, i el de dos crida el de tres. Amb `this(...)` escrius l'assignació completa una sola vegada, en el constructor de tres paràmetres.

⭐⭐ Exercici 7: La fracció que es simplifica

Escriu una classe `Fraccio` amb els atributs `int numerador` i `int denominador`. Afegeix:

- Constructor que validi: si `denominador == 0`, s'usa `1`.
- `Fraccio sumar(Fraccio altra)` → torna una fracció nova amb `(a.num * b.den + b.num * a.den) / (a.den * b.den)`.
- `void simplificar()` → divideix numerador i denominador pel seu màxim comú divisor (MCD).

Crea `1/2` i `1/3`, suma-les i simplifica el resultat.

Pista: per al MCD usa l'algoritme d'Euclides (restes o mòduls) o `Math.abs` amb un bucle. `simplificar()` no torna res: modifica `this`.

⭐⭐⭐ Exercici 8: CodeWars — Building blocks

Resol la kata **"Building blocks"** (7 kyu) en CodeWars.

Crea la classe `Block` amb un constructor que rebi les tres dimensions (com `int[]` de 3 o com 3 enters) i els mètodes:

- `int getWidth()`, `int getLength()`, `int getHeight()`
- `int getVolume()` → `width * length * height`
- `int getSurfaceArea()` → `2 * (w*l + w*h + l*h)`

Pista: guarda les tres dimensions en atributs amb `this` i deixa que els getters simplement les tornen. La superfície és la suma de les cares per dos.

⭐⭐⭐ Exercici 9: CodeWars — Object Oriented Piracy

Resol la kata **"Object Oriented Piracy"** (8 kyu) en CodeWars.

Crea la classe `Ship` que rep un `draft` (calat en unitats) i un `crew` (tripulants). Implementa:

- `getDraft()` i `getCrew()` → els getters de sempre.
- `isWorthIt()` → torna `true` si el vaixell mereix la pena saquejar-lo: el calat total menys `1.5` per cada tripulant ha de superar 20.

```
Ship titanic = new Ship(15, 10);
titanic.isWorthIt(); // false: 15 - 1.5*10 = 0, no mereix la pena
```

Pista: un getter torna l'atribut tal qual; `isWorthIt()` combina tots dos: `return draft - 1.5 * crew > 20;`. El `1.5` és una constant amb nom, millor que un número solt.

📚 Referències

Plataforma	Problema	Dificultat
AceptaElReto	100 — Constante de Kaprekar	Mitjana
AceptaElReto	148 — Nochevieja	Fàcil
CodeWars	Object Oriented Piracy (8 kyu)	Principiant
CodeWars	Building blocks (7 kyu)	Aficionat
CodeWars	FIXME: Get Full Name (7 kyu)	Aficionat

Butlletí U09 — Avançat Resolt

📝 Butlletí U09 — Avançat (Resolt)

> Les solucions estan amagades. Intenta-ho de veritat abans de destapar-les.

⭐ Exercici 1: La biblioteca

▸ 🔄 Solució

```
public class Llibre {
    String titol;
    String autor;
    int pagines;

    public Llibre(String titol, String autor, int pagines) {
        this.titol = titol;
        this.autor = autor;
        this.pagines = pagines;
    }

    void mostrarInfo() {
          System.out.println(titol + ", de " + autor + " (" + pagines + "
pàgines)");
    }

    public static void main(String[] args) {
        Llibre quixot = new Llibre("El Quixot", "Miguel de Cervantes", 863);
        Llibre java = new Llibre("Java", "Sergi", 100);
        quixot.mostrarInfo();
        java.mostrarInfo();
    }
}
```

Tres atributs, tres assignacions amb `this`, un mètode que els combina. Dos llibres, dos objectes, dos sortides independents.

⭐ Exercici 2: El rectangle que raona

🔄 Solució

```
public class Rectangle {
    double ample;
    double alt;

    public Rectangle(double ample, double alt) {
        this.ample = ample;
        this.alt = alt;
    }

    double calcularArea() {
        return ample * alt;
    }

    double calcularPerimetre() {
        return 2 * (ample + alt);
    }

    boolean esQuadrat() {
        return ample == alt;
    }

    public static void main(String[] args) {
        Rectangle quadrat = new Rectangle(4, 4);
        Rectangle rectangle = new Rectangle(5, 8);

        System.out.println("Quadrat: àrea " + quadrat.calcularArea()
                + ", perímetre " + quadrat.calcularPerimetre()
                + ", és quadrat? " + quadrat.esQuadrat());
        System.out.println("Rectangle: àrea " + rectangle.calcularArea()
                + ", perímetre " + rectangle.calcularPerimetre()
                + ", és quadrat? " + rectangle.esQuadrat());
    }
}
```

Sortida:

```
Quadrat: àrea 16.0, perímetre 16.0, és quadrat? true
Rectangle: àrea 40.0, perímetre 26.0, és quadrat? false
```

> Tres mètodes que *tornen* valors. Fixa't: `esQuadrat()` converteix una comparació en un booleà d'un sol `return`. Objectes que raonen, no només que mostren.

⭐ Exercici 3: El compte bancari blindat

> 🔄 Solució

```
public class CompteBancari {
    String titular;
    double saldo;

    public CompteBancari(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    void ingressar(double quantitat) {
        this.saldo += quantitat;
    }

    void retirar(double quantitat) {
        if (quantitat <= this.saldo) {
            this.saldo -= quantitat;
        } else {
            System.out.println("Saldo insuficient");
        }
    }

    void mostrar() {
         System.out.println("Titular: " + titular + " | Saldo: " + saldo + "
€");
    }

    public static void main(String[] args) {
        CompteBancari compte = new CompteBancari("Anna", 100);
        compte.retirar(30);
        compte.retirar(200);
        compte.mostrar();
    }
}
```

Sortida:

```
Saldo insuficient
Titular: Anna | Saldo: 70 €
```

L'`if` de `retirar` és el guardià: la retirada de 200 € es rebutja perquè supera el saldo, i el compte no queda mai en negatiu. Un objecte que es protegeix a si mateix.

⭐⭐ Exercici 4: L'hora que es corregeix sola

> 🔄 Solució

```
public class Hora {
    int hora;
    int minut;
    int segon;

    public Hora(int hora, int minut, int segon) {
        if (hora < 0 || hora > 23) {
            hora = 0;
        }
        if (minut < 0 || minut > 59) {
            minut = 0;
        }
        if (segon < 0 || segon > 59) {
            segon = 0;
        }
        this.hora = hora;
        this.minut = minut;
        this.segon = segon;
    }

    void incrementarSegon() {
        segon++;
        if (segon == 60) {
            segon = 0;
            minut++;
            if (minut == 60) {
                minut = 0;
                hora++;
                if (hora == 24) {
                    hora = 0;
                }
            }
        }
    }

    void mostrar() {
        String h = hora < 10 ? "0" + hora : "" + hora;
        String m = minut < 10 ? "0" + minut : "" + minut;
        String s = segon < 10 ? "0" + segon : "" + segon;
        System.out.println(h + ":" + m + ":" + s);
    }
```

```
    public static void main(String[] args) {
        Hora h = new Hora(23, 59, 59);
        h.incrementarSegon();
        h.mostrar();
    }
}
```

Sortida: `00:00:00`

Tres nivells de validació en el constructor i tres arrossegaments encadenats en `incrementarSegon()`. De 23:59:59 passa a 00:00:00: l'hora es corregeix sola perquè cada unitat sap quan reinicia i avisa la següent.

⭐ ⭐ Exercici 5: Què imprimeix? — el ball de referències

› 🔄 **Solució**

Imprimeix:

```
a.x = 10
c.x = 99
```

Primera part: `b = a` copia la *referència*, no l'objecte. `b.x = 10` modifica el mateix objecte que veu `a`, així que `a.x` també és 10.

Segona part: en cridar `cambiar(c)`, el paràmetre `p` rep una *còpia* de la referència. `p.x = 99` modifica l'objecte original (per això `c.x` és 99). Però `p = new Punto(50, 50)` només reassigna la còpia local: l'objecte de `c` no canvia i el nou `Punto` es perd en acabar el mètode. En Java les referències es passen per valor.

⭐⭐ Exercici 6: El correu que s'encadena

› 🔄 **Solució**

```java
public class Email {
    String remitent;
    String destinatari;
    String assumpte;

    public Email(String remitent, String destinatari, String assumpte) {
        this.remitent = remitent;
        this.destinatari = destinatari;
        this.assumpte = assumpte;
    }

    public Email(String remitent, String destinatari) {
        this(remitent, destinatari, "(sense assumpte)");
    }

    public Email(String remitent) {
        this(remitent, "(sense destí)");
    }

    void mostrar() {
         System.out.println("De: " + remitent + " | Per a: " + destinatari + "
| Assumpte: " + assumpte);
    }

    public static void main(String[] args) {
         Email complet = new Email("anna@gmail.com", "lluis@gmail.com", "Tasca
Java");
        Email mig = new Email("anna@gmail.com", "lluis@gmail.com");
        Email curt = new Email("anna@gmail.com");

        complet.mostrar();
        mig.mostrar();
        curt.mostrar();
    }
}
```

Sortida:

```
De: anna@gmail.com | Per a: lluis@gmail.com | Assumpte: Tasca Java
De: anna@gmail.com | Per a: lluis@gmail.com | Assumpte: (sense assumpte)
De: anna@gmail.com | Per a: (sense destí) | Assumpte: (sense assumpte)
```

Els dos constructors curts deleguen en el llarg amb `this(...)`. L'assignació completa s'escriu **una sola vegada**; els valors per defecte s'omplin en l'encadenament. Això és sobrecàrrega sense duplicar codi.

⭐⭐ Exercici 7: La fracció que es simplifica

> 🔄 Solució

```
public class Fraccio {
    int numerador;
    int denominador;

    public Fraccio(int numerador, int denominador) {
        if (denominador == 0) {
            denominador = 1;
        }
        this.numerador = numerador;
        this.denominador = denominador;
    }

    Fraccio sumar(Fraccio altra) {
            int num = this.numerador * altra.denominador + altra.numerador *
this.denominador;
        int den = this.denominador * altra.denominador;
        return new Fraccio(num, den);
    }

    void simplificar() {
        int mcd = mcd(Math.abs(numerador), Math.abs(denominador));
        numerador /= mcd;
        denominador /= mcd;
    }

    int mcd(int a, int b) {
        while (b != 0) {
            int resto = a % b;
            a = b;
            b = resto;
        }
        return a == 0 ? 1 : a;
    }

    void mostrar() {
        System.out.println(numerador + "/" + denominador);
    }

    public static void main(String[] args) {
        Fraccio unMig = new Fraccio(1, 2);
        Fraccio unTerç = new Fraccio(1, 3);
        Fraccio suma = unMig.sumar(unTerç);
```

```
        suma.simplificar();
        suma.mostrar();
    }
}
```

Sortida: `5/6`

`sumar(Fraccio altra)` usa `this` per al primer sumand i `altra.` per al segon, i torna una **fracció nova** (no toca cap de les dues). `simplificar()` sí que modifica l'objecte (per això és `void`). Euclides amb mòduls troba el MCD en poques vegades.

⭐⭐⭐ Exercici 8: CodeWars — Building blocks

› 🔄 Solució

```java
public class Block {
    private int width;
    private int length;
    private int height;

    public Block(int[] dimensions) {
        this.width = dimensions[0];
        this.length = dimensions[1];
        this.height = dimensions[2];
    }

    public int getWidth() {
        return width;
    }

    public int getLength() {
        return length;
    }

    public int getHeight() {
        return height;
    }

    public int getVolume() {
        return width * length * height;
    }

    public int getSurfaceArea() {
        return 2 * (width * length + width * height + length * height);
    }

    public static void main(String[] args) {
        Block bloque = new Block(new int[]{2, 4, 6});
        System.out.println("Volum: " + bloque.getVolume());
        System.out.println("Superfície: " + bloque.getSurfaceArea());
    }
}
```

Sortida:

```
Volum: 48
Superfície: 88
```

Els getters tornen cada atribut, i els dos mètodes calculats combinen els tres. La superfície és cada parell de cares multiplicat i sumat, tot per dos. La kata accepta també tres enters per separat en el constructor.

⭐⭐⭐ Exercici 9: CodeWars — Object Oriented Piracy 🔗

› 🔄 Solució

```java
public class Ship {
    private static final double PES_TRIPULANT = 1.5;

    private final double draft;
    private final int crew;

    public Ship(double draft, int crew) {
        this.draft = draft;
        this.crew = crew;
    }

    public double getDraft() {
        return draft;
    }

    public int getCrew() {
        return crew;
    }

    public boolean isWorthIt() {
        return draft - PES_TRIPULANT * crew > 20;
    }
}
```

`draft` i `crew` són `final`: es fixen en el constructor i no canvien (vaixell immutable). Els getters tornen l'atribut tal qual. `isWorthIt()` aplica la fórmula amb la constant

> `PES_TRIPULANT`: cada tripulant aporta 1.5 al calat, i només mereix la pena si el que queda supera 20. Per a `new Ship(15, 10)`: `15 - 1.5*10 = 0` → `false`.

> 📚 **Vols més?** Revisa el butlletí d'**extres** d'aquesta unitat: té quatre katas de CodeWars (inclosa una de depuració d'una classe) i dos problemes d'AceptaElReto.

Butlletí U09 — Extres

📝 Butlletí U09 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està amagada: resisteix-te fins a esgotar la teva pista.

CodeWars

1. FIXME: Get Full Name

Et donen una classe `Dinglemouse` amb un mètode `getFullName()` que hauria de tornar `"Nom Cognom"`, però el resultat està al revés o incomplet. Arregla-la.

- Enunciat en CodeWars
- Dificultat: 7 kyu

Pista: fixa't en quins atributs existeixen (`firstName` i `lastName`) i en com el mètode els concatena. La classe està feta per a rompre's: busca l'error de `this` o d'ordre.

> 🔄 **Solució**

```java
public class Dinglemouse {
    private final String firstName;
    private final String lastName;

    public Dinglemouse(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getFullName() {
        if (firstName.isEmpty() && lastName.isEmpty()) {
            return "";
        }
        if (firstName.isEmpty()) {
            return lastName;
        }
        if (lastName.isEmpty()) {
            return firstName;
        }
        return firstName + " " + lastName;
    }
}
```

La classe original fallava en construir el nom complet: els casos en què falta un nom o tots dos havien de tornar el que hi ha. El mètode correcte encadena els casos límit abans de juntar tots dos noms amb un espai.

---

2. Geometry Basics: Distance between points in 2D

Et donen una classe `Point` amb dues propietats (`x` i `y`) i has d'implementar el mètode `distanceBetweenPoints(Point a, Point b)` que torni la distància entre ells.

Exemple: `Point(3, 3)` i `Point(3, 3)` → `0`. `Point(1, 6)` i `Point(4, 2)` → `5`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: distància euclidiana `Math.sqrt(Math.pow(a.x - b.x, 2) + Math.pow(a.y - b.y, 2))`. Els objectes es passen com a paràmetre, com vam fer amb `Fraccio.sumar`.

› 🔄 Solució

```java
public class Kata {
    public static double distanceBetweenPoints(Point a, Point b) {
        return Math.sqrt(Math.pow(a.x - b.x, 2) + Math.pow(a.y - b.y, 2));
    }
}

class Point {
    public double x;
    public double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}
```

El mètode rep dos objectes `Point` i llegeix els seus atributs (`a.x`, `b.x`…). És exactament el mateix patró de `sumar(Fraccio altra)`: un objecte treballa amb un altre. La fórmula de Pitàgores resolta per objectes.

---

3. Grasshopper — Personalized Message

Crea el mètode `greet(name, owner)` que torni `"Hello boss"` si `name` i `owner` són iguals, o `"Hello guest"` si són distints.

Exemples: `greet("Daniel", "Daniel")` → `"Hello boss"`, `greet("Greg", "Daniel")` → `"Hello guest"`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: compara `String` amb `.equals()`, mai amb ==. Un `if/else` o un ternari.

› 🔄 Solució

```java
public class Kata {
    public static String greet(String name, String owner) {
        return name.equals(owner) ? "Hello boss" : "Hello guest";
    }
}
```

Una línia amb el ternari i `.equals()`. Si uses `==` amb cadenes, els resultats seran imprevisibles: en Java els `String` es comparen amb `.equals()` perquè són objectes (encara que Java els doni un tracte especial). La U10 t'explicarà per què.

---

4. Sleigh Authentication

Has de completar la classe `Sleigh` amb un mètode `authenticate(name, password)` que torni `true` només si `name` és `"Santa Claus"` i `password` és `"Ho Ho Ho!"`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

Pista: torna `name.equals("Santa Claus") && password.equals("Ho Ho Ho!")`. Una altra vegada `.equals()`, i el `&&` de la U04.

› 🔄 **Solució**

```java
public class Sleigh {
    public boolean authenticate(String name, String password) {
        return name.equals("Santa Claus") && password.equals("Ho Ho Ho!");
    }
}
```

Un mètode d'una sola línia: dos comparacions amb `.equals()` unides per `&&`. És una d'aquelles katas trampa: sembla trivial, però molts cauen usant `==` i fallen els tests. Objectes per tot arreu.

AceptaElReto

5. 148 — Nochevieja

Ramón es passa el dia de cap d'any contant els minuts que falten per a mitjanit. Per a cada hora de l'entrada (format `HH:MM`), digues quants minuts falten per a les `00:00`. L'entrada acaba amb `00:00`, que no es processa.

Entrada:

```
23:45
21:30
00:01
00:00
```

Sortida:

```
15
150
1439
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: separa l'hora i el minut amb `split(":")`. Els minuts que falten són `(23 - hora) * 60 + (60 - minut)`. Escriu una classe `Hora` amb un mètode `minutsFinsMitjanit()` i veuràs com de natural queda.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Nochevieja {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String linea = sc.next();

        while (!linea.equals("00:00")) {
            String[] parts = linea.split(":");
            int hora = Integer.parseInt(parts[0]);
            int minut = Integer.parseInt(parts[1]);
            System.out.println((23 - hora) * 60 + (60 - minut));
            linea = sc.next();
        }
        sc.close();
    }
}
```

Per a `23:45`: `(23-23)*60 + (60-45)` = 15. Per a `21:30`: `2*60 + 30` = 150. El `while` es deté amb la línia `00:00` perquè es compara amb `.equals()` (és un `String`, recorda). La versió amb classe `Hora` és un bon exercici voluntari.

---

6. 117 — La fiesta aburrida

Tinín odia saludar desconeguts. L'entrada comença amb un nombre N (quanta gent hi ha) i després N línies amb el format `"Soy Lotario"`. Per a cada una, imprimeix `"Hola, Lotario."`.

Entrada:

```
3
Soy Lotario
Soy Aldonza
Soy Ender
```

Sortida:

```
Hola, Lotario.
Hola, Aldonza.
Hola, Ender.
```

- Enunciat en AceptaElReto
- Dificultat: Fàcil

Pista: `split(" ")` sobre cada línia i agafa la segona part (índex 1). Pots modelar cada persona com un objecte d'una classe `Persona` amb el nom i un mètode `saludar()`.

> 🔄 **Solució**

```
import java.util.Scanner;

public class FiestaAburrida {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            String linea = sc.nextLine();
            String nom = linea.split(" ")[1];
            System.out.println("Hola, " + nom + ".");
        }
        sc.close();
    }
}
```

`split(" ")` parteix `"Soy Lotario"` en `["Soy", "Lotario"]` i l'índex 1 és el nom. El `sc.nextLine()` després del `nextInt()` consumeix el salt de línia sobrant. Una versió amb classe `Persona` i el seu mètode `saludar()` és el repte extra ideal per a aquesta unitat.

🧭 **I si et quedes amb ganes?** Quan domines les classes, el pas natural és protegir les seves dades: això és l'**encapsulació** de la U10. I si vols veure objectes que es creen i destrueixen sols, espera't als arrays de la U06 i les col·leccions de la U12, on crearàs desenes d'objectes en un bucle.

Butlletí U10 — Inicial

📝 Butlletí U10 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, posa-li cadenat a la primera classe i fes que el `static` deixi de semblar màgia. Cap atribut no neix sabent ser `private`.

Exercici 1: La casa de cristall

Aquesta classe és una casa de cristall: tothom pot entrar i tocar el que vulgui.

```
public class Persona {
    public String nom;
    public int edat;
}
```

Converteix els atributs en `private` i afegeix getters i setters per a `nom` i `edat`. Escriu també un `main` que creï una `Persona`, li posi "Anna" i 25 anys usant els setters, i després mostri els valors amb els getters.

Exercici 2: El cotxe del veïnat

Parteix d'aquesta classe:

```
public class Cotxe {
    private String marca;
    private double velocitat;

    public Cotxe(String marca) {
        this.marca = marca;
        this.velocitat = 0;
    }

    public String getMarca() {
        return marca;
    }
}
```

Afegeix `getVelocitat()` i un `setVelocitat(double velocitat)` que **rebutgi** les velocitats negatives i les superiors a 200 (mostrant "Velocitat invàlida."). Escriu un `main` que creï un cotxe "Seat" i intenti posar-li -50 i després 120.

Exercici 3: El termòmetre amb cervell

Escriu una classe `Termometre` amb:

- Atribut `private double temperatura`.
- Constructor que inicialitzi la temperatura a 20.0.
- Getter `getTemperatura()`.
- Setter `setTemperatura(double temperatura)` que **només** accepti valors entre -273.15 i 100.0. Si el valor no és vàlid, mostra "Temperatura fora de rang.".

En un `main`, crea un termòmetre, posa-li -500 (s'ha de rebutjar) i després 36.5, i mostra la temperatura final.

Exercici 4: Getter sense setter

Aquesta classe representa la configuració d'una app: l'idioma es tria en crear-la i **no ha de poder canviar-se després**.

```
public class Configuracio {
    private String idioma;

    public Configuracio(String idioma) {
        this.idioma = idioma;
    }

    public String getIdioma() {
        return idioma;
    }
}
```

Fixa't: té getter però **no** setter. Escriu un `main` que creï una configuració amb "es" i mostri l'idioma. Respon: per què no té setter? Què li passaria a un usuari que intentàs fer `config.idioma = "va"` des de fora?

Exercici 5: El comptador de la classe

Escriu una classe `Comptador` amb:

- Atribut `public static int total = 0;`.
- Un constructor que incrementi `total` en 1 cada vegada que es crea un objecte.

En un `main`, crea 3 objectes `Comptador` i mostra `Comptador.total`. Respon: quant val `total` i per què no és 1?

Exercici 6: La calculadora sense piles

Escriu una classe `Utilitats` amb dos mètodes **estàtics**: `sumar(int a, int b)` i `restar(int a, int b)`. En un `main`, usa els dos mètodes **sense crear cap objecte** (usa el nom de la classe) i mostra els resultats de `sumar(5, 3)` i `restar(10, 4)`.

Pista: `int resultat = Utilitats.sumar(5, 3);`

Exercici 7: Les constants del barri

Escriu una classe `Constants` que contingui:

- `public static final double IVA = 0.21;`
- `public static final int MAX_INTENTS_LOGIN = 3;`
- `public static final String NOM_APP = "GestioCurs";`

En un `main`, mostra les tres constants. Després, intenta fer `Constants.IVA = 0.5;` i respon: què ocorre en compilar?

Exercici 8: Què imprimeix? — el trencaclosques estàtic

Sense executar, escriu la sortida exacta d'aquest programa:

```java
public class Trencaclosques {
    static int s = 0;
    int i = 0;

    public Trencaclosques() {
        s++;
        i = s;
    }

    public static void main(String[] args) {
        Trencaclosques t1 = new Trencaclosques();
        Trencaclosques t2 = new Trencaclosques();
        System.out.println(t1.i + " " + t2.i + " " + Trencaclosques.s);
    }
}
```

Pista: `s` és de la classe (una sola còpia compartida); `i` és de cada objecte i es copia del valor de `s` en el moment de néixer.

Exercici 9: CodeWars — Square(n) Sum

Resol la kata **"Square(n) Sum"** (8 kyu) en CodeWars.

Completa el mètode `public static int squareSum(int[] n)` que rebi un array d'enters i retorni la suma dels quadrats de cada número.

Exemple: `[1, 2, 2]` → `1 + 4 + 4 = 9`.

Pista: usa un `for` que acumuli `n[i] * n[i]`. Els arrays es recorren amb la UO6, però ja pots usar l'índex.

Butlletí U10 — Inicial Resolt

📝 Butlletí U10 — Inicial (Resolt)

> Les solucions estan ocultes en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: La casa de cristall

▸ 🔄 Solució

```java
public class Persona {
    private String nom;
    private int edat;

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public int getEdat() {
        return edat;
    }

    public void setEdat(int edat) {
        this.edat = edat;
    }

    public static void main(String[] args) {
        Persona p = new Persona();
        p.setNom("Anna");
        p.setEdat(25);
        System.out.println(p.getNom() + " té " + p.getEdat() + " anys.");
    }
}
```

Els atributs passen a `private` i tot l'accés es fa amb getters i setters. El `this` en els setters desambiguïx: el paràmetre s'assigna a l'atribut, no a si mateix.

Exercici 2: El cotxe del veïnat

> 🔄 Solució

```
public class Cotxe {
    private String marca;
    private double velocitat;

    public Cotxe(String marca) {
        this.marca = marca;
        this.velocitat = 0;
    }

    public String getMarca() {
        return marca;
    }

    public double getVelocitat() {
        return velocitat;
    }

    public void setVelocitat(double velocitat) {
        if (velocitat >= 0 && velocitat <= 200) {
            this.velocitat = velocitat;
        } else {
            System.out.println("Velocitat invàlida.");
        }
    }

    public static void main(String[] args) {
        Cotxe c = new Cotxe("Seat");
        c.setVelocitat(-50);
        c.setVelocitat(120);
        System.out.println("Velocitat: " + c.getVelocitat());
    }
}
```

> Sortida: `Velocitat invàlida.` (pel -50) i després `Velocitat: 120.0`. El setter valida abans de tocar l'atribut: és la frontera que protegeix l'estat de l'objecte.

Exercici 3: El termòmetre amb cervell

> 🔄 Solució

```
public class Termometre {
    private double temperatura;

    public Termometre() {
        temperatura = 20.0;
    }

    public double getTemperatura() {
        return temperatura;
    }

    public void setTemperatura(double temperatura) {
        if (temperatura >= -273.15 && temperatura <= 100.0) {
            this.temperatura = temperatura;
        } else {
            System.out.println("Temperatura fora de rang.");
        }
    }

    public static void main(String[] args) {
        Termometre t = new Termometre();
        t.setTemperatura(-500);
        t.setTemperatura(36.5);
        System.out.println("Temperatura: " + t.getTemperatura());
    }
}
```

> Sortida: `Temperatura fora de rang.` i després `Temperatura: 36.5`. El setter converteix la classe en un "termòmetre amb cervell": no accepta qualsevol número, només valors físicament possibles.

Exercici 4: Getter sense setter

> 🔄 Solució

```
public class Configuracio {
    private String idioma;

    public Configuracio(String idioma) {
        this.idioma = idioma;
    }

    public String getIdioma() {
        return idioma;
    }

    public static void main(String[] args) {
        Configuracio config = new Configuracio("es");
        System.out.println("Idioma: " + config.getIdioma());
    }
}
```

No té setter perquè l'idioma és una decisió d'una sola vegada: es tria en el constructor i prou. Si algú intentàs `config.idioma = "va"` des de fora, **no compilaria**: `idioma` és `private`, i fora de la classe no es pot tocar. Obligar que el canvi passi per un setter (o que no existeixi) és l'essència de l'encapsulació.

Exercici 5: El comptador de la classe

> 🔄 Solució

```java
public class Comptador {
    public static int total = 0;

    public Comptador() {
        total++;
    }

    public static void main(String[] args) {
        Comptador c1 = new Comptador();
        Comptador c2 = new Comptador();
        Comptador c3 = new Comptador();
        System.out.println("Total: " + Comptador.total);
    }
}
```

`total` val **3**. És `static`: una sola còpia compartida per tota la classe. Cada `new` crida al constructor i l'incrementa; com que els tres objectes comparteixen la mateixa variable, el comptador compta els tres. No és 1 perquè no hi ha una còpia per objecte: n'hi ha una única de classe.

Exercici 6: La calculadora sense piles

› 🔄 **Solució**

```java
public class Utilitats {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static int restar(int a, int b) {
        return a - b;
    }

    public static void main(String[] args) {
        System.out.println(Utilitats.sumar(5, 3));
        System.out.println(Utilitats.restar(10, 4));
    }
}
```

Sortida: `8` i `6`. Com que els mètodes són `static`, es diuen amb el nom de la classe (`Utilitats.sumar`), sense `new` i sense objecte. És el mateix patró que ja uses amb `Math.sqrt` o `Integer.parseInt`.

Exercici 7: Les constants del barri

> 🔄 Solució

```
public class Constants {
    public static final double IVA = 0.21;
    public static final int MAX_INTENTS_LOGIN = 3;
    public static final String NOM_APP = "GestioCurs";

    public static void main(String[] args) {
        System.out.println("IVA: " + Constants.IVA);
        System.out.println("Màxim intents: " + Constants.MAX_INTENTS_LOGIN);
        System.out.println("App: " + Constants.NOM_APP);
        // Constants.IVA = 0.5; // Error de compilació
    }
}
```

En intentar `Constants.IVA = 0.5;` el compilador ho **prohibeix**: `final` significa que el valor no es pot reassignar després de la seva declaració. Les constants són a prova de bombes, per això van en MAJÚSCULES amb _: tothom sap que no es toquen.

Exercici 8: Què imprimeix? — el trencaclosques estàtic

> 🔄 Solució

Imprimeix **`1 2 2`**.

`Trencaclosques.s` és `static`: una sola còpia compartida. Amb el primer `new`, `s` passa a 1 i `t1.i` es copia aquell 1. Amb el segon `new`, `s` passa a 2 i `t2.i` es copia aquell 2. Al final, `t1.i` = 1, `t2.i` = 2 i `Trencaclosques.s` = 2. L'estàtic puja per a tots; el d'instància es congela amb el valor que tenia la classe en el moment de néixer.

Exercici 9: CodeWars — Square(n) Sum

> 🔄 Solució

```java
public class Kata {
    public static int squareSum(int[] n) {
        int suma = 0;
        for (int i = 0; i < n.length; i++) {
            suma += n[i] * n[i];
        }
        return suma;
    }
}
```

Per a `[1, 2, 2]` → `1 + 4 + 4 = 9`. El bucle recorre cada element i acumula `n[i] * n[i]`. Nota el `static`: CodeWars exigeix el mètode estàtic per a poder cridar-lo sense crear objectes, just el que acabeu de practicar.

Butlletí U10 — Avançat

📝 Butlletí U10 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ Exercici 1: Empleat amb validació

Escriu una classe `Empleat` amb:

- Atributs `private String nom`, `private double salari`.
- Constructor que validi que el salari no sigui negatiu (si ho és, el deixa en 0 i mostra un avís).
- Getters i setters. El setter del salari també ha de rebutjar negatius.

En un `main`, crea un empleat "Laura" amb salari 1500, intenta posar-li -300 i mostra el salari final.

Pista: reutilitza la validació del setter dins del constructor: `setSalari(salari)` en comptes d'assignar a seques.

⭐ Exercici 2: Cercle encapsulat

Escriu una classe `Cercle` amb:

- Atribut `private double radi`.
- Constructor que rebi el radi.
- Getter `getRadi()`, setter `setRadi(double)` que rebutgi radis negatius o zero.
- Mètodes `getArea()` i `getPerimetre()` que usen `Math.PI`.

En un `main`, crea un cercle de radi 5 i mostra la seva àrea i perímetre.

Pista: `area = Math.PI * radi * radi;` i `perimetre = 2 * Math.PI * radi;`. Recorda que `Math.PI` és una constant estàtica.

⭐⭐ Exercici 3: JavaBean Alumne

Escriu una classe `Alumne` seguint l'estil JavaBean (l'estàndard per a classes de dades):

- Atributs `private String nom`, `private int edat`, `private double notaMitjana`.
- Constructor sense paràmetres i constructor amb els tres valors.
- Getter i setter per a **cada** atribut.
- El setter de `edat` ha de rebutjar edats fora de 0 i 120; el de `notaMitjana`, fora de 0 i 10.

En un `main`, crea un alumne amb el constructor complet i després modifica la seva nota amb el setter.

Pista: un JavaBean és "atributs privats + getters/setters + constructors": el patró que veuràs en qualsevol framework. Els setters validen; els getters només llegeixen.

⭐⭐ Exercici 4: Hora immutable

Escriu una classe `Hora` que representi una hora del dia i que sigui **immutable**: els seus atributs només s'assignen en el constructor i no tenen setters.

- Atributs `private final int hora`, `private final int minut`.
- Constructor que validi `hora` entre 0 i 23 i `minut` entre 0 i 59.
- Getters `getHora()` i `getMinut()`.
- Mètode `mostrar()` que retorni `"HH:MM"` (amb zeros: `09:05`).

En un `main`, crea una hora 9:05 i mostra-la. Respon: per què no necessita setters?

Pista: usa `String.format("%02d:%02d", hora, minut)` o `"0" + ...` quan el valor sigui menor que 10.

⭐ ⭐ Exercici 5: Què imprimeix? — el trencaclosques dels gats

Sense executar, escriu la sortida exacta:

```java
public class Gat {
    public static int totalGats = 0;
    private String nom;
    private int vides;

    public Gat(String nom) {
        this.nom = nom;
        this.vides = 9;
        totalGats++;
    }

    public void perdreVida() {
        if (vides > 0) {
            vides--;
        }
    }

    public String toString() {
        return nom + " (" + vides + " vides)";
    }
}
```

```java
public class TrencaclosquesGats {
    public static void main(String[] args) {
        Gat g1 = new Gat("Bigotis");
        Gat g2 = new Gat("Garfield");
        g1.perdreVida();
        g1.perdreVida();
        g2.perdreVida();

        System.out.println(g1);
        System.out.println(g2);
        System.out.println("Total: " + Gat.totalGats);
    }
}
```

Pista: cada gat neix amb 9 vides i les perd una a una amb `perdreVida()`. Compta quantes vegades es diu sobre cada gat, i recorda que `totalGats` és `static`.

⭐⭐ Exercici 6: Comptador d'usuaris

Escriu una classe `Usuari` que assigni a cada objecte un `id` **únic i automàtic**:

- Atribut `private static int comptador = 0;` i `private int id;`.
- Constructor que incrementi `comptador` i assigni `id = comptador`.
- Mètode `public static int getTotalUsuaris()`.
- Getter `getId()`.

En un `main`, crea 5 usuaris i mostra l'id de l'últim i el total d'usuaris.

Pista: el patró del punt 5: el `static` compta quants s'han creat, i cada objecte es "congela" el seu número en néixer. `getTotalUsuaris` és estàtic perquè pregunta a la classe, no a un objecte.

⭐ ⭐ ⭐ Exercici 7: La classe utilitària OperacionsArray

Escriu una classe `OperacionsArray` **utilitària** (constructor privat) amb aquests mètodes `static`:

- `suma(int[] numeros)` → suma tots els elements.
- `mitjana(double[] numeros)` → retorna el promig.
- `maxim(int[] numeros)` → retorna el més gran.
- `estaOrdenat(int[] numeros)` → `true` si cada element és major o igual que l'anterior.

En un `main`, usa la classe **sense crear cap objecte** sobre `{3, 1, 4, 1, 5}` i `{1, 2, 3}`.

Pista: el constructor `private OperacionsArray() {}` impedeix instanciar-la, com `Math`. Per a `estaOrdenat`, recorre amb un `for` i compara cada element amb l'anterior (`numeros[i] < numeros[i - 1]` → no està ordenat).

⭐⭐⭐ Exercici 8: Validador de dades

Escriu una classe `Validador` **utilitària** (constructor privat) amb aquests mètodes `static`:

- `esEmailValid(String email)` → `true` si conté exactament una `@` i almenys un `.` després de la `@`.
- `esEdatValida(int edat)` → `true` si està entre 0 i 120.
- `esTextNoBuit(String text)` → `true` si no és `null` i no està en blanc.

En un `main`, prova els tres mètodes amb casos vàlids i invàlids (per exemple `"ana@mail.com"`, `"ana@"`, `"hola"`, `-5`, `200`, `null`).

Pista: usa `String.indexOf("@")` per a localitzar la `@`, `indexOf("@", pos + 1)` per a comprovar que no n'hi ha una segona, i `indexOf(".", pos)` per al punt després de la `@`. Un `return` primerenc en cada condició fallida simplifica molt.

⭐⭐⭐ Exercici 9: El gran repte — refactoritza el banc

Aquest codi funciona… però és una porta oberta. Refactoritza'l seguint els passos del punt 8 (Be the Code):

```
public class CompteBancari {
    public String titular;
    public double saldo;

    public CompteBancari(String titular, double saldo) {
        titular = titular;
        saldo = saldo;
    }

    public void retirar(double quantitat) {
        saldo = saldo - quantitat;
    }

    public void ingressar(double quantitat) {
        saldo = saldo + quantitat;
    }
}
```

Ha de quedar així:

1. Atributs `private`.

2. Constructor amb `this` i que validi que el saldo inicial no sigui negatiu.
3. Getters per als dos; **sense setters** per al saldo.
4. `retirar(double)` que rebutgi quantitats negatives i que **no permeti** deixar el saldo en negatiu (si `quantitat > saldo`, avisa i no retira).
5. `ingressar(double)` que rebutgi quantitats negatives.
6. Una constant `public static final String NOM_BANC = "Banc DAM";`.

Escriu també un `main` de prova que creï un compte, ingressi, intenti retirar més del que té i mostri el saldo.

Pista: sense setter per al saldo, només les operacions `retirar` i `ingressar` poden tocar-lo: és la frontera del negoci. Recorda el bug del constructor (`titular = titular` s'assigna a si mateix) i valida tot abans d'assignar.

Butlletí U10 — Avançat Resolt
📝 Butlletí U10 — Avançat (Resolt)

> Les solucions estan ocultes en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: Empleat amb validació

> 🔄 Solució

```java
public class Empleat {
    private String nom;
    private double salari;

    public Empleat(String nom, double salari) {
        this.nom = nom;
        setSalari(salari);
    }

    public String getNom() {
        return nom;
    }

    public double getSalari() {
        return salari;
    }

    public void setSalari(double salari) {
        if (salari >= 0) {
            this.salari = salari;
        } else {
            System.out.println("Salari invàlid.");
        }
    }

    public static void main(String[] args) {
        Empleat e = new Empleat("Laura", 1500);
        e.setSalari(-300);
        System.out.println(e.getNom() + ": " + e.getSalari());
    }
}
```

Sortida: `Salari invàlid.` i `Laura: 1500.0`. El truc: el constructor crida a `setSalari(salari)`, així la validació viu en un sol lloc i no hi ha dues còpies de la mateixa regla.

⭐ Exercici 2: Cercle encapsulat

> 🔄 Solució

```java
public class Cercle {
    private double radi;

    public Cercle(double radi) {
        setRadi(radi);
    }

    public double getRadi() {
        return radi;
    }

    public void setRadi(double radi) {
        if (radi > 0) {
            this.radi = radi;
        } else {
            System.out.println("Radi invàlid.");
        }
    }

    public double getArea() {
        return Math.PI * radi * radi;
    }

    public double getPerimetre() {
        return 2 * Math.PI * radi;
    }

    public static void main(String[] args) {
        Cercle c = new Cercle(5);
        System.out.println("Àrea: " + c.getArea());
        System.out.println("Perímetre: " + c.getPerimetre());
    }
}
```

`getArea()` i `getPerimetre()` són getters “calculats”: no retornen un atribut, sinó un valor derivat d’ell. `Math.PI` és una constant estàtica, i els mètodes els crees tu amb `static` quan convé. Àrea del radi 5: ≈ 78.54, perímetre ≈ 31.42.

⭐⭐ Exercici 3: JavaBean Alumne

🔄 Solució

```java
public class Alumne {
    private String nom;
    private int edat;
    private double notaMitjana;

    public Alumne() {}

    public Alumne(String nom, int edat, double notaMitjana) {
        this.nom = nom;
        setEdat(edat);
        setNotaMitjana(notaMitjana);
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public int getEdat() {
        return edat;
    }

    public void setEdat(int edat) {
        if (edat >= 0 && edat <= 120) {
            this.edat = edat;
        } else {
            System.out.println("Edat invàlida.");
        }
    }

    public double getNotaMitjana() {
        return notaMitjana;
    }

    public void setNotaMitjana(double notaMitjana) {
        if (notaMitjana >= 0 && notaMitjana <= 10) {
            this.notaMitjana = notaMitjana;
        } else {
            System.out.println("Nota invàlida.");
```

```
        }
    }

    public static void main(String[] args) {
        Alumne a = new Alumne("Sara", 18, 7.5);
        a.setNotaMitjana(8.75);
        System.out.println(a.getNom() + ": " + a.getNotaMitjana());
    }
}
```

JavaBean: atributs privats, constructors (buit i amb dades) i getters/setters de tot. Els setters amb validació eviten estats impossibles (edat 250, nota 15). El constructor amb paràmetres reutilitza els setters per a no duplicar regles.

⭐⭐ Exercici 4: Hora immutable

› 🔄 Solució

```java
public class Hora {
    private final int hora;
    private final int minut;

    public Hora(int hora, int minut) {
        if (hora < 0 || hora > 23 || minut < 0 || minut > 59) {
            throw new IllegalArgumentException("Hora o minut invàlids.");
        }
        this.hora = hora;
        this.minut = minut;
    }

    public int getHora() {
        return hora;
    }

    public int getMinut() {
        return minut;
    }

    public String mostrar() {
        return String.format("%02d:%02d", hora, minut);
    }

    public static void main(String[] args) {
        Hora h = new Hora(9, 5);
        System.out.println(h.mostrar());
    }
}
```

Sortida: `09:05`. No necessita setters perquè és **immutable**: una vegada creada, el seu valor no canvia mai (els `final` ho garanteixen). Canviar l'hora és crear una `Hora` nova. El constructor usa `throw` (vist en la UO4) per als valors impossibles.

⭐ ⭐ Exercici 5: Què imprimeix? — el trencaclosques dels gats

› 🔄 Solució

Imprimeix:

```
Bigotis (7 vides)
Garfield (8 vides)
Total: 2
```

`Bigotis` perd 2 vides (de 9 a 7), `Garfield` en perd 1 (de 9 a 8). `totalGats` és `static` i puja amb cada `new`, així que val 2. Les `vides` són de cada gat; el `totalGats`, de la classe. `toString()` és el mètode que Java crida en imprimir un objecte amb `System.out.println`.

⭐⭐ Exercici 6: Comptador d'usuaris

› 🔄 Solució

```java
public class Usuari {
    private static int comptador = 0;
    private int id;

    public Usuari() {
        comptador++;
        id = comptador;
    }

    public int getId() {
        return id;
    }

    public static int getTotalUsuaris() {
        return comptador;
    }

    public static void main(String[] args) {
        Usuari u1 = new Usuari();
        Usuari u2 = new Usuari();
        Usuari u3 = new Usuari();
        Usuari u4 = new Usuari();
        Usuari u5 = new Usuari();
        System.out.println("Últim id: " + u5.getId());
        System.out.println("Total: " + Usuari.getTotalUsuaris());
    }
}
```

Sortida: `Últim id: 5` i `Total: 5`. El patró “comptador + id”: `comptador` (estàtic) puja en cada constructor i l’objecte es guarda el seu número com a `id`. `getTotalUsuaris()` és `static` perquè la pregunta se li fa a la classe, no a un usuari concret.

⭐ ⭐ ⭐ Exercici 7: La classe utilitària OperacionsArray

› 🔄 Solució

```java
public class OperacionsArray {
    private OperacionsArray() {}

    public static int suma(int[] numeros) {
        int suma = 0;
        for (int numero : numeros) {
            suma += numero;
        }
        return suma;
    }

    public static double mitjana(double[] numeros) {
        double suma = 0;
        for (double numero : numeros) {
            suma += numero;
        }
        return numeros.length > 0 ? suma / numeros.length : 0;
    }

    public static int maxim(int[] numeros) {
        int max = numeros[0];
        for (int i = 1; i < numeros.length; i++) {
            if (numeros[i] > max) {
                max = numeros[i];
            }
        }
        return max;
    }

    public static boolean estaOrdenat(int[] numeros) {
        for (int i = 1; i < numeros.length; i++) {
            if (numeros[i] < numeros[i - 1]) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
         System.out.println("Suma: " + OperacionsArray.suma(new int[]{3, 1, 4,
1, 5}));
```

```
        System.out.println("Màxim: " + OperacionsArray.maxim(new int[]{3, 1,
4, 1, 5}));
        System.out.println("Ordenat: " + OperacionsArray.estaOrdenat(new int[]
{1, 2, 3}));
    }
}
```

Constructor privat + tot `static` = classe utilitària, com `Math`. `estaOrdenat` retorna `false` en seguint que trobi una parella fora d'ordre (`return` primerenc). El `for...each` de la UO6 es pot usar ja: recorre sense índex.

⭐⭐⭐ Exercici 8: Validador de dades

› 🔄 Solució

```java
public class Validador {
    private Validador() {}

    public static boolean esEmailValid(String email) {
        if (email == null) {
            return false;
        }
        int arrova = email.indexOf('@');
        if (arrova < 1) {
            return false;
        }
        if (email.indexOf('@', arrova + 1) != -1) {
            return false;
        }
        return email.indexOf('.', arrova + 1) != -1;
    }

    public static boolean esEdatValida(int edat) {
        return edat >= 0 && edat <= 120;
    }

    public static boolean esTextNoBuit(String text) {
        return text != null && !text.trim().isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(Validador.esEmailValid("ana@mail.com"));
        System.out.println(Validador.esEmailValid("ana@"));
        System.out.println(Validador.esEdatValida(-5));
        System.out.println(Validador.esEdatValida(200));
        System.out.println(Validador.esTextNoBuit("  "));
    }
}
```

Sortida: `true`, `false`, `false`, `false`, `false`. `esEmailValid` exigeix: alguna cosa abans de la `@`, una sola `@`, i un `.` després. `indexOf('@', pos)` busca des de `pos` endavant. Cada condició fallida surt amb `false` a l'instant: sense `else` encadenats.

⭐⭐⭐ Exercici 9: El gran repte — refactoritza el banc

> 🔄 Solució

```java
public class CompteBancari {
    public static final String NOM_BANC = "Banc DAM";

    private String titular;
    private double saldo;

    public CompteBancari(String titular, double saldo) {
        this.titular = titular;
        if (saldo >= 0) {
            this.saldo = saldo;
        } else {
            System.out.println("Saldo inicial invàlid.");
        }
    }

    public String getTitular() {
        return titular;
    }

    public double getSaldo() {
        return saldo;
    }

    public void retirar(double quantitat) {
        if (quantitat <= 0) {
            System.out.println("Quantitat invàlida.");
            return;
        }
        if (quantitat > saldo) {
            System.out.println("Saldo insuficient.");
            return;
        }
        saldo = saldo - quantitat;
    }

    public void ingressar(double quantitat) {
        if (quantitat <= 0) {
            System.out.println("Quantitat invàlida.");
            return;
        }
        saldo = saldo + quantitat;
    }
```

```
    public static void main(String[] args) {
        CompteBancari compte = new CompteBancari("Ana", 100);
        compte.ingressar(50);
        compte.retirar(400);
        System.out.println("Saldo: " + compte.getSaldo());
    }
}
```

Sortida: `Saldo insuficient.` i `Saldo: 150.0`. El constructor corregit usa `this` (el bug original assignava el paràmetre a si mateix). Sense setter de saldo, només `ingressar` i `retirar` poden tocar-lo, i totes dues validen. La constant dona identitat al banc. El compte ja no pot quedar en negatiu ni en valors arbitraris.

Butlletí U10 — Extres

📝 Butlletí U10 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està oculta: resisteix-te fins a esgotar la teva pista. Els `private`, els getters i el `static` són aquí els protagonistes: encapsula, protegeix i comparteix sense por.

CodeWars

1. Playing with cubes II

Et donen un motlle de classe `Cube` amb un atribut privat `side`. El teu treball: construir-la amb getter i setter, i dos constructors (un buit que inicialitza `side` a 0, i un altre que rep el costat).

Exemple d'ús:

```
Cube c = new Cube(3);
c.getSide();  // 3
c.setSide(7);
c.getSide();  // 7
```

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: `side` ha de ser `private` (ningú el toca des de fora) i els únics portals són `getSide()` i `setSide()`. El constructor sense paràmetres posa `this.side = 0;`. Exactament el que has vist en la unitat: encapsulació pura.

> › 🔄 **Solució**

```java
public class Cube {
    private int side;

    public Cube() {
        this.side = 0;
    }

    public Cube(int side) {
        this.side = side;
    }

    public int getSide() {
        return side;
    }

    public void setSide(int side) {
        this.side = side;
    }
}
```

L'atribut `side` és `private`: ningú pot fer `c.side = 9` des de fora. Els dos constructors donen dues maneres de néixer (amb o sense valor), i el getter/setter són les úniques portes. El `this` del setter desambigua el paràmetre de l'atribut, com has vist en la unitat.

---

2. Classy Extentions

Crea la classe `Pet` amb un atribut privat `name` (String) i un mètode `speak()` que torni el nom de l'animal. Després crea la classe `Cat` que **hereta** de `Pet` i sobreescriu `speak()` perquè torni `"[name] meows."`.

Exemples:

```java
new Cat("Milo").speak();  // "Milo meows."
new Cat("Garfield").speak();  // "Garfield meows."
```

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: `name` és privat, així que `Cat` no pot llegir-lo directament: necessita un getter `getName()` en `Pet` (recorda: els privats no s'hereten, però existeixen dins de l'objecte). `Cat extends Pet` i usa `super(nombre)` per a construir la part del pare.

> 🔄 Solució

```
public class Pet {
    private String name;

    public Pet(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public String speak() {
        return name;
    }
}

public class Cat extends Pet {
    public Cat(String name) {
        super(name);
    }

    @Override
    public String speak() {
        return getName() + " meows.";
    }
}
```

Fixa't: `Cat` no toca `name` directament (és privat en `Pet`), usa `getName()`. Això és el triangle perfecte: **encapsulació** (privat + getter) + **herència** (`extends`) + **sobreescriptura** (`@Override`). L'herència la aprofundeixes en la U11, però aquí ja la veus en acció.

---

3. Sleigh Authentication

Pare Noel té un mètode `authenticate(name, password)` i només admet dues credencials: el nom ha de ser `"Santa Claus"` i la contrasenya `"Ho Ho Ho!"`. Torna `true` només si totes dues coincideixen.

Exemples: `authenticate("Santa Claus", "Ho Ho Ho!")` → `true`, `authenticate("Santa", "Ho Ho Ho!")` → `false`.

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: els dos valors són constants: `private static final String NOM_VALID = "Santa Claus";`. Compara amb `.equals()`, mai amb `==`. I nota el `static`: el mètode no necessita objecte, és pura lògica de classe.

> 🔄 **Solució**

```
public class Sleigh {
    private static final String NOM_VALID = "Santa Claus";
    private static final String PASSWORD_VALID = "Ho Ho Ho!";

    public boolean authenticate(String name, String password) {
        return NOM_VALID.equals(name) && PASSWORD_VALID.equals(password);
    }
}
```

Les constants `static final` són el secret de la casa: `static` (una sola còpia per a tota la classe, la vas veure en el punt 7) i `final` (ningú pot reassignar-les). El `authenticate` combina les dues comprovacions amb `&&`. El `.equals()` es crida sobre la constant, no sobre el paràmetre: així `null` mai trenca el mètode.

---

4. Object Oriented Piracy

Crea la classe `Ship` que rep un `draft` (calat) i un `crew` (tripulants). Implementa `isWorthIt()`: torna `true` si el calat total menys `1.5` per cada tripulant supera 20.

Exemple: `new Ship(15, 10).isWorthIt()` → `false` (`15 - 1.5*10 = 0`).

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: guarda `draft` i `crew` en atributs `private final` (es fixen en el constructor i ja no canvien). El mètode combina tots dos: `return draft - 1.5 * crew > 20;`. El `1.5` mereix ser una constant amb nom.

> 🔄 **Solució**

```
public class Ship {
    private static final double PES_TRIPULANT = 1.5;

    private final double draft;
    private final int crew;

    public Ship(double draft, int crew) {
        this.draft = draft;
        this.crew = crew;
    }

    public boolean isWorthIt() {
        return draft - PES_TRIPULANT * crew > 20;
    }
}
```

La classe és immutable: els atributs `final` es fixen en néixer i ningú pot canviar-los (ni tan sols amb un setter, que aquí no existeix). La constant `static final` documenta el `1.5`. El vaixell "sap" si mereix la pena saquejar-lo sense que ningú llegeix les seves tripes: encapsulació i responsabilitat única.

AceptaElReto

5. 117 — La festa avorrida

Tinín odia les festes, i cada persona que se li acosta es presenta amb el format `"Soc Lotari"`. Ajuda'l a respondre `"Hola, [nom]."` a cadascun. L'entrada comença amb un nombre que indica quantes persones hi ha, seguit d'una línia per persona. Escriu la solució amb una classe `Persona` que guardi el nom i un mètode `saludar()`.

Entrada d'exemple:

```
3
Soc Lotari
Soc Aldonça
Soc Ender
```

Sortida d'exemple:

```
Hola, Lotari.
Hola, Aldonça.
Hola, Ender.
```

- Enunciat a AceptaElReto
- Dificultat: Fàcil

Pista: llegeix la línia, treu el `"Soc "` inicial (`linea.substring(4)` o `split(" ")[1]`), crea la `Persona` amb aquell nom i crida el seu `saludar()`. És una excusa perfecta per a una classe amb un atribut i un mètode, en comptes d'un main que ho fa tot.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Persona {
    private String nom;

    public Persona(String nom) {
        this.nom = nom;
    }

    public void saludar() {
        System.out.println("Hola, " + nom + ".");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();  // consumeix el salt de línia

        for (int i = 0; i < n; i++) {
            String linea = sc.nextLine();
            String nom = linea.substring(4);  // trau "Soc "
            Persona p = new Persona(nom);
            p.saludar();
        }
        sc.close();
    }
}
```

`Persona` encapsula el seu nom: `private` + constructor + un mètode que sap saludar. El `main` només s'encarrega de llegir i crear objectes. `linea.substring(4)` es salta els 4 primers caràcters (`"Soc "`). El `sc.nextLine()` extra després del `nextInt()` consumeix l'Enter, el clàssic del Scanner que vas veure en la UO3.

6. 117 bis — La festa avorrida amb comptador estàtic

Repte extra amb `static`: usa la classe `Persona` de l'exercici 5, però afegeix un atribut `private static int totalSaluts` que compte quantes persones ha saludat Tinín en total. Després de cada salut, mostra el total acumulat.

Entrada d'exemple:

```
2
Soc Lotari
Soc Ender
```

Sortida d'exemple:

```
Hola, Lotari. (saluts: 1)
Hola, Ender. (saluts: 2)
```

Pista: `static` significa "de la classe, no de l'objecte": tots els `Persona` comparteixen `totalSaluts`. Instrumenta-ho dins de `saludar()` amb `totalSaluts++`. Així practiques que el comptador puja per a tots els objectes, com el `Contador` del butlletí inicial.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Persona {
    private static int totalSaluts = 0;

    private String nom;

    public Persona(String nom) {
        this.nom = nom;
    }

    public void saludar() {
        totalSaluts++;
          System.out.println("Hola, " + nom + ". (saluts: " + totalSaluts +
")");
    }

    public static int getTotalSaluts() {
        return totalSaluts;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            String linea = sc.nextLine();
            Persona p = new Persona(linea.substring(4));
            p.saludar();
        }
        sc.close();
    }
}
```

`totalSaluts` és `static`: hi ha una única còpia compartida per tota la classe, no una per objecte. Cada `saludar()` l'incrementa i, com que tots comparteixen la mateixa variable, el comptador acumula de veritat. `getTotalSaluts()` és `static` perquè la pregunta "quants saluts en total?" se li fa a la classe, no a una persona concreta.

> 🧭 **I si et quedes amb ganes?** Quan domines l'encapsulació, torna als problemes d'unitats anteriors i reescriu-los amb classes ben blindades: un `Rectangle` amb la seva àrea com a mètode, un `Numero` amb la seva anàlisi com a mètode… El `private`, els getters i el `static` transformen un script solt en un disseny. El material no es perd: es reutilitza.

Butlletí U11 — Inicial

📝 Butlletí U11 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, dona-li la mà a la primera superclasse i fes que `extends` deixi de semblar màgia. L'herència en Java és com la de veritat: de vegades et portes genial amb les subclasses, de vegades vols renegar de tot. Però ningú no neix sabent usar `super`.

Exercici 1: Què imprimeix? — La família musical

```
class Instrumentista {
    void tocar() { System.out.println("L'instrumentista toca un instrument"); }
}

class Guitarrista extends Instrumentista {
    void tocar() { System.out.println("El guitarrista toca la guitarra"); }
}

class Baixista extends Guitarrista {
    void tocar() { System.out.println("El baixista toca el baix"); }
}

public class Banda {
    public static void main(String[] args) {
        Baixista b = new Baixista();
        b.tocar();
    }
}
```

Què imprimeix? Per què?

Exercici 2: Troba l'error — extends mal usat

```java
public class Animal {
    private String especie;

    public Animal(String especie) {
        this.especie = especie;
    }
}

public class Gos extends Animal {
    private String raça;

    public Gos(String raça) {
        this.raça = raça;
    }
}
```

Aquest codi **no compila**. Per què? Explica l'error i corregeix-lo.

Exercici 3: Completa el codi — el gat que crida el seu pare

```java
public class Animal {
    public void ferSo() {
        System.out.println("Algun so genèric...");
    }
}

public class Gat extends Animal {
    @Override
    public void ferSo() {
        ________.ferSo();   // primer el del pare
        System.out.println("¡MIAU!");
    }
}
```

Quina paraula falta al buit perquè `Gat` primer executi el so d'`Animal` i després el seu "¡MIAU!"? Escriu a més un `main` que creï un `Gat` i crida a `ferSo()`.

Exercici 4: Escriu aquest programa — l'herència de vehicles

Crea una jerarquia de 3 nivells usant `extends`:

- `Vehicle` (atribut: `String marca`)
- `Cotxe` (atribut: `int numPortes`)
- `Esportiu` (atribut: `int velocitatMaxima`)

Cada classe ha de tenir un constructor que rebi els seus atributs i usi `super`. En `main()`, crea un `Esportiu` de marca "Ferrari", 2 portes i 340 km/h. Imprimeix els seus atributs.

Exercici 5: Què imprimeix? — Polimorfisme amb referències

```
class X {
    void missatge() { System.out.println("X"); }
}

class Y extends X {
    void missatge() { System.out.println("Y"); }
}

class Z extends Y { }

public class Test {
    public static void main(String[] args) {
        X ref1 = new Y();
        X ref2 = new Z();
        Y ref3 = new Z();

        ref1.missatge();
        ref2.missatge();
        ref3.missatge();
    }
}
```

Què imprimeix cada crida? Per què el tipus de la referència no decideix res?

Exercici 6: Escriu aquest programa — la granja polimòrfica

Crea una classe `Animal` amb mètode `ferSo()`. Crea `Vaca`, `Ovella` i `Gallina` que el sobreescriguin. En `main()`, crea un `ArrayList<Animal>`, fica una vaca, una ovella i una gallina, recorre'l amb un for-each cridant a `ferSo()`.

```
// Eixida esperada:
// Muuuu
// Beeee
// Cloc cloc
```

Exercici 7: Troba l'error — @Override que no ho és

```
public class Animal {
    public void ferSo() {
        System.out.println("...");
    }
}

public class Peix extends Animal {
    @Override
    public void ferSo() { }   // ¿compila?

    @Override
    public void nedar() { }   // ¿compila?
}
```

Una de les dues línies amb `@Override` impedeix compilar. Quina i per què? Què t'avisa el compilador en l'instant en què escrius aquella línia?

Exercici 8: Escriu aquest programa — el gos ben heretat

Parteix d'aquesta classe base:

```java
public class Animal {
    protected String nom;
    protected int edat;

    public Animal(String nom, int edat) {
        this.nom = nom;
        this.edat = edat;
    }
}
```

Escriu una classe `Gos extends Animal` amb:

- Constructor que usi `super(nom, edat)`.
- Mètode `lladrar()` que imprimeixi `nom + " diu: ¡Guau!"`.
- Un `main` que creï un `Gos("Firulais", 3)` i crida a `lladrar()`.

Respon: per què `Gos` pot usar `nom` i `edat` encara que no les declari?

Exercici 9: Què imprimeix? — la cadena de constructors

```java
class Avi {
    public Avi() { System.out.println("Avi"); }
}

class Pare extends Avi {
    public Pare() { System.out.println("Pare"); }
}

class Fill extends Pare {
    public Fill() { System.out.println("Fill"); }
}

public class Test {
    public static void main(String[] args) {
        new Fill();
    }
}
```

Què imprimeix i per què en aquell ordre?

Butlletí U11 — Inicial Resolt

📝 Butlletí U11 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què imprimeix? — La família musical

> 🔄 **Solució**
>
> Imprimeix **"El baixista toca el baix"**.
>
> `Baixista` té la seva pròpia versió de `tocar()`. Java busca el mètode començant per la classe més específica (`Baixista`) i el troba aquí mateix: mai no puja a `Guitarrista` ni a `Instrumentista`. Aquell és el *dynamic dispatch*: el mètode es resol segons el tipus real de l'objecte, no segons el tipus de la referència.

Exercici 2: Troba l'error — extends mal usat

> 🔄 **Solució**
>
> L'error és que `Gos` no crida al constructor d'`Animal`. Quan una classe filla no posa `super(...)`, Java intenta cridar a `super()` sense paràmetres. Però `Animal` només té `Animal(String)`, així que el compilador no troba el constructor buit: **error de compilació**.
>
> ```
> public class Gos extends Animal {
>     private String raça;
>
>     public Gos(String especie, String raça) {
>         super(especie);   // la clau!
>         this.raça = raça;
>     }
> }
> ```
>
> Pensa en `super()` com cridar a papà perquè configuri la seva part abans que tu configures la teva. Si papà necessita una espècie per a construir-se, tu l'hi has de passar. És com

construir una casa sense fonaments: el constructor del pare és la base.

Exercici 3: Completa el codi — el gat que crida el seu pare

> 🔄 Solució

La paraula és **super**:

```
public class Gat extends Animal {
    @Override
    public void ferSo() {
        super.ferSo();   // primer el del pare
        System.out.println("¡MIAU!");
    }

    public static void main(String[] args) {
        Gat g = new Gat();
        g.ferSo();
    }
}
```

Sortida:

```
Algun so genèric...
¡MIAU!
```

`super.ferSo()` executa la versió d'`Animal` i després el `Gat` afegeix el seu. Sense el `super`, el mètode estaria sobreescrit per complet i la línia del pare no sortiria mai.

Exercici 4: Escriu aquest programa — l'herència de vehicles

> 🔄 Solució

```java
public class Vehicle {
    protected String marca;

    public Vehicle(String marca) {
        this.marca = marca;
    }
}

public class Cotxe extends Vehicle {
    protected int numPortes;

    public Cotxe(String marca, int numPortes) {
        super(marca);
        this.numPortes = numPortes;
    }
}

public class Esportiu extends Cotxe {
    private int velocitatMaxima;

    public Esportiu(String marca, int numPortes, int velocitatMaxima) {
        super(marca, numPortes);
        this.velocitatMaxima = velocitatMaxima;
    }

    public static void main(String[] args) {
        Esportiu e = new Esportiu("Ferrari", 2, 340);
        System.out.println(e.marca + " amb " + e.numPortes
                + " portes i " + e.velocitatMaxima + " km/h");
    }
}
```

L'herència en cadena: `Esportiu` → `Cotxe` → `Vehicle`. Cada constructor crida al del seu pare amb `super(...)`. Per això `marca` (de `Vehicle`) i `numPortes` (de `Cotxe`) són accessibles en `Esportiu` gràcies a `protected`.

Exercici 5: Què imprimeix? — Polimorfisme amb referències

› 🔄 Solució

Imprimeix:

```
Y
Z
Z
```

El tipus de la **referència** (X, X, Y) no importa. El que importa és el tipus **real** de l'objecte (Y, Z, Z). Java sempre executa el mètode més específic de l'objecte real. És com portar la jaqueta del teu pare: per fora pareixes el teu pare (la referència), però per dins ets tu (l'objecte). Quan parles, se sent la teva veu, no la del teu pare. Dynamic binding en tot el seu esplendor.

Exercici 6: Escriu aquest programa — la granja polimòrfica

› 🔄 Solució

```java
import java.util.ArrayList;

public class Animal {
    public void ferSo() { System.out.println("..."); }
}

class Vaca extends Animal {
    @Override public void ferSo() { System.out.println("Muuuu"); }
}

class Ovella extends Animal {
    @Override public void ferSo() { System.out.println("Beeee"); }
}

class Gallina extends Animal {
    @Override public void ferSo() { System.out.println("Cloc cloc"); }
}

public class Granja {
    public static void main(String[] args) {
        ArrayList<Animal> animals = new ArrayList<>();
        animals.add(new Vaca());
        animals.add(new Ovella());
        animals.add(new Gallina());

        for (Animal a : animals) {
            a.ferSo();
        }
    }
}
```

Sortida:

```
Muuuu
Beeee
Cloc cloc
```

Un sol `ArrayList<Animal>` i un sol bucle: cada animal executa la seva pròpia versió gràcies al polimorfisme. Sense ell, tindries tres llistes separades. Això és el que fa que el polimorfisme valgui el seu pes en or.

Exercici 7: Troba l'error — @Override que no ho és

> 🔄 Solució

La línia que **no compila** és:

```
@Override
public void nedar() { }   // ✗ ERROR: Animal no té nedar()
```

`@Override` li diu al compilador: "verifica que realment estic sobreescrivint un mètode del pare". Com que `Animal` no té `nedar()`, el compilador t'avisa en l'acte. L'altra línia (`ferSo()`) sí que és un override vàlid. Aquell avís a temps és el regal de `@Override`: si escrius malament un nom de mètode, te n'assabenta el compilador, no un bug raríssim a mitjanit.

Exercici 8: Escriu aquest programa — el gos ben heretat

> 🔄 Solució

```
public class Gos extends Animal {
    public Gos(String nom, int edat) {
        super(nom, edat);
    }

    public void lladrar() {
        System.out.println(nom + " diu: ¡Guau!");
    }

    public static void main(String[] args) {
        Gos g = new Gos("Firulais", 3);
        g.lladrar();
    }
}
```

Sortida: `Firulais diu: ¡Guau!`

`Gos` pot usar `nom` i `edat` perquè estan declarats com a `protected` en `Animal`: l'herència els posa a disposició de tota la família. Si fossin `private`, ni `Gos` els veuria. És com l'herència familiar: el que és privat a casa dels avis, no ho veuen ni els néts.

Exercici 9: Què imprimeix? — la cadena de constructors

> 🔄 Solució

Imprimeix:

```
Avi
Pare
Fill
```

En crear un `Fill` s'executen **tots** els constructors de la cadena, del més general al més específic. Com que cada constructor crida a `super()` (o Java el posa automàticament), primer es construeix `Avi`, després `Pare` i per últim `Fill`. Els fonaments abans que el teulada, sempre.

Butlletí U11 — Avançat

📝 Butlletí U11 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. L'herència és com la família: de vegades heretes coses bones, de vegades et toca la col·lecció de segells del teu tio avi. Però amb interfícies, almenys tries què implementar.

⭐ Exercici 1: Interfície FiguraGeometrica

Crea una interfície `FiguraGeometrica` amb dos mètodes:

- `double calcularArea()`
- `double calcularPerimetre()`

Implementa la interfície en:

- `Cercle` (constructor amb radi)
- `Rectangle` (constructor amb ample i alt)
- `TriangleRectangle` (constructor amb base i altura)

En el `main`, crea un `ArrayList<FiguraGeometrica>`, afegeix un cercle de radi 5, un rectangle 4x3 i un triangle rectangle 3x4. Recorre'ls imprimint àrea i perímetre de cadascun.

```
public interface FiguraGeometrica {
    double calcularArea();
    double calcularPerimetre();
}
```

Pista: per al triangle rectangle, `area = base * altura / 2` i el perímetre és `base + altura + hipotenusa`, amb `hipotenusa = Math.sqrt(base*base + altura*altura)`.

⭐ Exercici 2: Jerarquia d'Empleats

Crea una classe base `Empleat` amb:

- `String nom`

- `double salariBase`
- Constructor, getters
- Mètode `double calcularSalari()` que torni el `salariBase`

Crea dos subclasses:

- `Gerent`: té un `double bo` extra. `calcularSalari()` torna `salariBase + bo`.
- `Venedor`: té un `double comissio` per venda i un `int vendesRealitzades`. `calcularSalari()` torna `salariBase + comissio * vendesRealitzades`.

En el `main`, crea un array d'`Empleat` amb un gerent i un venedor, recorre'l polimòrficament i mostra el salari de cadascun.

```
public class Empleat {
    protected String nom;
    protected double salariBase;

    public Empleat(String nom, double salariBase) {
        this.nom = nom;
        this.salariBase = salariBase;
    }

    public double calcularSalari() {
        return salariBase;
    }
}
```

Pista: declara `nom` i `salariBase` com a `protected` perquè les subclasses els usen sense getters. L'`@Override` de `calcularSalari()` en cada subclasse és el cor del polimorfisme.

⭐ ⭐ Exercici 3: Sistema de pagaments amb interfície

Crea una interfície `Pagable` amb el mètode:

- `boolean procesarPagament(double quantitat)`

Implementa la interfície en:

- `TarjetaCredito`: té `double limit` i `double saldoUsat`. Pot pagar si `quantitat + saldoUsat <= limit`.
- `PayPal`: té `double saldo`. Pot pagar si `quantitat <= saldo`.
- `TransferenciaBancaria`: té `double saldo`. Pot pagar sempre que `quantitat <= saldo`, però té un cost fix de 1 € per transferència.

```
public interface Pagable {
    boolean procesarPagament(double quantitat);
}
```

Exemple:

```
Pagable tarjeta = new TarjetaCredito(1000, 0);
tarjeta.procesarPagament(500);   // true
tarjeta.procesarPagament(600);   // false (supera el límit)
```

Pista: cada classe decideix la seva pròpia lògica d'aprovació; només el contracte `procesarPagament` és comú. Per a la transferència, comprova que `quantitat + 1 <= saldo`.

⭐ ⭐ Exercici 4: Interfícies múltiples: Volador i Nedador

Crea dos interfícies:

- `Volador`: mètode `void volar()`
- `Nedador`: mètode `void nadar()`

Crea una classe `Ànec` que implementi les dues interfícies, una classe `Avió` que només implementi `Volador` i una classe `Peix` que només implementi `Nedador`.

En el `main`, crea un `ArrayList<Volador>` amb un Ànec i un Avió, i recorre'l cridant a `volar()`. Després fes el mateix amb un `ArrayList<Nedador>`.

```
public interface Volador { void volar(); }
public interface Nedador { void nadar(); }
```

Pista: l'Ànec és l'estrella: una sola classe que firma dos contractes. Els `ArrayList` de tipus interfície accepten qualsevol classe que implementi aquell contracte.

⭐⭐ Exercici 5: Downcasting segur

Crea una jerarquia `Empleat` → `Programador`, `Dissenyador`:

- `Programador` té `void escriureCodi()`.
- `Dissenyador` té `void dissenyar()`.
- `Empleat` té `String nom` i `void mostrarInfo()`.

En el `main`, crea un `ArrayList<Empleat>` amb diversos empleats de tots dos tipus i recorre'l usant `instanceof` per a cridar els mètodes específics.

Pista: dins del bucle, `if (e instanceof Programador)` → `((Programador) e).escriureCodi();`. Sempre amb `instanceof` abans del cast: mai no baixis sense preguntar.

⭐⭐ Exercici 6: Calculadora de figures amb classe abstracta

Crea una classe abstracta `Figura` amb:

- Atribut `protected String color`.
- Constructor que rebi el color.
- Mètodes abstractes `double calcularArea()` i `double calcularPerimetre()`.
- Mètode concret `void mostrarColor()` que imprimeixi el color.

Implementa `Cercle` (radi) i `Rectangle` (ample, alt). En el `main`, crea un `ArrayList<Figura>` amb un cercle vermell de radi 3 i un rectangle blau de 4x2, i mostra l'àrea de cadascun i el seu color.

Pista: l'àrea total de la llista es calcula recorrent-la amb `for (Figura f : figures)`. `mostrarColor()` ja està fet: les subclasses només implementen els dos mètodes abstractes.

⭐ ⭐ ⭐ Exercici 7: Sistema de notificacions polimòrfic

Crea una interfície `Notificable` amb:

- `void enviar(String missatge)`
- `String getEstat()`

Implementa:

- `EmailNotificacio`: atributs `String direccio`, `boolean enviat`. En enviar, imprimeix "Enviant email a [direcció]: [missatge]". Estat: "Enviat" o "Pendent".
- `SMSNotificacio`: atributs `String telefon`, `boolean enviat`. En enviar, imprimeix "Enviant SMS a [telèfon]: [missatge]". Estat similar.
- `PushNotificacio`: atributs `String dispositiuId`, `boolean enviat`. En enviar, imprimeix "Enviant push a [dispositiuId]: [missatge]".

```
public interface Notificable {
    void enviar(String missatge);
    String getEstat();
}
```

En el `main`, crea un `ArrayList<Notificable>` amb els tres tipus. Afegeix un mètode estàtic que recorre la llista i envie totes les notificacions:

```
public static void enviarTotes(List<Notificable> notificacions, String
missatge) {
    for (Notificable n : notificacions) {
        n.enviar(missatge);
    }
}
```

Pista: els tres tipus comparteixen el patró: en enviar, posen `enviat = true` i tornen "Enviat"; si no, "Pendent". El polimorfisme fa que `enviarTotes` no sàpiga amb quin tipus tracta.

⭐ ⭐ ⭐ Exercici 8: Template method — les begudes

Crea una classe abstracta `Beguda` amb el patró template method:

```java
public abstract class Beguda {
    public final void preparar() {
        bullirAigua();
        prepararIngredient();
        servirEnTassa();
        afegirExtres();
    }

    private void bullirAigua() { System.out.println("Bullint aigua..."); }
    private void servirEnTassa() { System.out.println("Servint en tassa..."); }

    protected abstract void prepararIngredient();
    protected abstract void afegirExtres();
}
```

Implementa `Te` (bosseta de te + llima) i `Cafe` (cafè molt + sucre). En el `main`, prepara un `Te` i un `Cafe` amb una variable de tipus `Beguda`.

Pista: les subclasses només omplen els dos mètodes `protected abstract`. El `final` en `preparar()` garanteix que ningú no reordeni els passos. Per a provar ambdues, usa una referència polimòrfica: `Beguda b = new Te(); b.preparar();`.

⭐⭐⭐ Exercici 9: El gran repte — vehicles amb combustible

Crea una jerarquia de vehicles:

- `Vehicle` (abstracta): `String matricula`, `int combustible`, `abstract boolean moure()`
- `Cotxe`: gasta 5 de combustible per moviment
- `Moto`: gasta 3 de combustible per moviment
- `Camio`: gasta 10 de combustible per moviment, però pot portar `int càrrega`

Cada vehicle té un `moure()` que redueix el combustible i torna `true` si va poder moure's. Si no n'hi ha prou, imprimeix "Sense combustible" i torna `false`.

En `main()`, crea un `ArrayList<Vehicle>` amb diversos vehicles. Cada vehicle es mou repetidament mentre pugui i compta quants moviments va fer.

Pista: dona a cada subclasse una constant `private static final int DESPESA` amb el que consumeix per moviment. En `moure()`: `if (combustible >= DESPESA) { combustible -= DESPESA; ...; return true; } else { System.out.println("Sense combustible"); return false; }`. El bucle del `main` usa el valor de retorn: `while (v.moure()) { moviments++; }`. Així no es queda mai en un bucle infinit quan el combustible no arriba per a moure's.

Butlletí U11 — Avançat Resolt

📝 Butlletí U11 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: Interfície FiguraGeometrica

> 🔄 Solució

```java
import java.util.ArrayList;

public interface FiguraGeometrica {
    double calcularArea();
    double calcularPerimetre();
}

class Cercle implements FiguraGeometrica {
    private double radi;

    public Cercle(double radi) {
        this.radi = radi;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radi * radi;
    }

    @Override
    public double calcularPerimetre() {
        return 2 * Math.PI * radi;
    }
}

class Rectangle implements FiguraGeometrica {
    private double ample, alt;

    public Rectangle(double ample, double alt) {
        this.ample = ample;
        this.alt = alt;
    }

    @Override
    public double calcularArea() {
        return ample * alt;
    }

    @Override
    public double calcularPerimetre() {
        return 2 * (ample + alt);
    }
```

```
}

class TriangleRectangle implements FiguraGeometrica {
    private double base, altura;

    public TriangleRectangle(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura / 2;
    }

    @Override
    public double calcularPerimetre() {
        double hipotenusa = Math.sqrt(base * base + altura * altura);
        return base + altura + hipotenusa;
    }
}

public class TestFigures {
    public static void main(String[] args) {
        ArrayList<FiguraGeometrica> figures = new ArrayList<>();
        figures.add(new Cercle(5));
        figures.add(new Rectangle(4, 3));
        figures.add(new TriangleRectangle(3, 4));

        for (FiguraGeometrica f : figures) {
            System.out.println("Àrea: " + f.calcularArea()
                    + ", perímetre: " + f.calcularPerimetre());
        }
    }
}
```

La interfície és el contracte: `ArrayList<FiguraGeometrica>` accepta qualsevol classe que signe el contracte, i cadascuna calcula la seva àrea i el seu perímetre a la seva manera. Polimorfisme amb interfícies en estat pur.

⭐ Exercici 2: Jerarquia d'Empleats

> 🔄 Solució

```java
public class Empleat {
    protected String nom;
    protected double salariBase;

    public Empleat(String nom, double salariBase) {
        this.nom = nom;
        this.salariBase = salariBase;
    }

    public String getNom() {
        return nom;
    }

    public double calcularSalari() {
        return salariBase;
    }
}

class Gerent extends Empleat {
    private double bo;

    public Gerent(String nom, double salariBase, double bo) {
        super(nom, salariBase);
        this.bo = bo;
    }

    @Override
    public double calcularSalari() {
        return salariBase + bo;
    }
}

class Venedor extends Empleat {
    private double comissio;
    private int vendesRealitzades;

    public Venedor(String nom, double salariBase,
                   double comissio, int vendesRealitzades) {
        super(nom, salariBase);
        this.comissio = comissio;
        this.vendesRealitzades = vendesRealitzades;
    }
```

```
    @Override
    public double calcularSalari() {
        return salariBase + comissio * vendesRealitzades;
    }
}

public class Nomina {
    public static void main(String[] args) {
        Empleat[] plantilla = {
            new Gerent("Anna", 2000, 500),
            new Venedor("Lluís", 1200, 20, 15)
        };

        for (Empleat e : plantilla) {
            System.out.println(e.getNom() + " cobra "
                    + e.calcularSalari() + " €");
        }
    }
}
```

L'array és d'`Empleat`, però cada objecte executa el seu propi `calcularSalari()`: el gerent amb el bo, el venedor amb les comissions. `protected` permet que les subclasses llegeixen `salariBase` sense necessitat d'un getter.

⭐ ⭐ Exercici 3: Sistema de pagaments amb interfície

› 🔄 Solució

```
public interface Pagable {
    boolean procesarPagament(double quantitat);
}

class TarjetaCredito implements Pagable {
    private double limit;
    private double saldoUsat;

    public TarjetaCredito(double limit, double saldoUsat) {
        this.limit = limit;
        this.saldoUsat = saldoUsat;
    }

    @Override
    public boolean procesarPagament(double quantitat) {
        if (quantitat + saldoUsat <= limit) {
            saldoUsat += quantitat;
            return true;
        }
        return false;
    }
}

class PayPal implements Pagable {
    private double saldo;

    public PayPal(double saldo) {
        this.saldo = saldo;
    }

    @Override
    public boolean procesarPagament(double quantitat) {
        if (quantitat <= saldo) {
            saldo -= quantitat;
            return true;
        }
        return false;
    }
}

class TransferenciaBancaria implements Pagable {
    private double saldo;
```

```
    public TransferenciaBancaria(double saldo) {
        this.saldo = saldo;
    }

    @Override
    public boolean procesarPagament(double quantitat) {
        if (quantitat + 1 <= saldo) {
            saldo -= (quantitat + 1);
            return true;
        }
        return false;
    }
}

public class TestPagaments {
    public static void main(String[] args) {
        Pagable tarjeta = new TarjetaCredito(1000, 0);
        System.out.println(tarjeta.procesarPagament(500));  // true
        System.out.println(tarjeta.procesarPagament(600));  // false

        Pagable paypal = new PayPal(200);
        System.out.println(paypal.procesarPagament(150));   // true
        System.out.println(paypal.procesarPagament(100));   // false
    }
}
```

Tres formes de pagar, un sol contracte. Cada classe guarda la seva lògica d'aprovació i el seu saldo intern: la interfície només exigeix el mètode. La transferència descompta a més la seva comissió fixa d'1 €, que és la "taxa" del banc.

⭐ ⭐ Exercici 4: Interfícies múltiples: Volador i Nedador

› 🔄 Solució

```java
import java.util.ArrayList;

public interface Volador {
    void volar();
}

public interface Nedador {
    void nadar();
}

class Ànec implements Volador, Nedador {
    @Override
    public void volar() {
        System.out.println("L'ànec vola en formació en V");
    }

    @Override
    public void nadar() {
        System.out.println("L'ànec nada tranquil·lament a l'estany");
    }
}

class Avió implements Volador {
    @Override
    public void volar() {
        System.out.println("L'avió vola a 900 km/h");
    }
}

class Peix implements Nedador {
    @Override
    public void nadar() {
        System.out.println("El peix nada contra corrent");
    }
}

public class TestAnimals {
    public static void main(String[] args) {
        ArrayList<Volador> voladors = new ArrayList<>();
        voladors.add(new Ànec());
        voladors.add(new Avió());
```

```
        for (Volador v : voladors) {
            v.volar();
        }

        ArrayList<Nedador> nedadors = new ArrayList<>();
        nedadors.add(new Ànec());
        nedadors.add(new Peix());

        for (Nedador n : nedadors) {
            n.nadar();
        }
    }
}
```

L'Ànec és el campió: firma dos contractes alhora. A la llista de `Volador` es comporta com a volador; a la de `Nedador`, com a nedador. Una classe, dos personalitats, zero conflictes: aquell és el poder de les interfícies davant de l'herència única.

⭐⭐ Exercici 5: Downcasting segur

› 🔄 Solució

```java
import java.util.ArrayList;

public class Empleat {
    protected String nom;

    public Empleat(String nom) {
        this.nom = nom;
    }

    public void mostrarInfo() {
        System.out.println("Empleat: " + nom);
    }
}

class Programador extends Empleat {
    public Programador(String nom) {
        super(nom);
    }

    public void escriureCodi() {
        System.out.println(nom + " escriu codi Java");
    }
}

class Dissenyador extends Empleat {
    public Dissenyador(String nom) {
        super(nom);
    }

    public void dissenyar() {
        System.out.println(nom + " dissenya la interfície");
    }
}

public class Empresa {
    public static void main(String[] args) {
        ArrayList<Empleat> plantilla = new ArrayList<>();
        plantilla.add(new Programador("Anna"));
        plantilla.add(new Dissenyador("Lluís"));
        plantilla.add(new Programador("Eva"));

        for (Empleat e : plantilla) {
```

```
            e.mostrarInfo();
            if (e instanceof Programador) {
                ((Programador) e).escriureCodi();
            } else if (e instanceof Dissenyador) {
                ((Dissenyador) e).dissenyar();
            }
        }
    }
}
```

`instanceof` és la mirilla: abans de baixar la referència, preguntes si l'objecte real és d'aquell tipus. Sense el cast, no podries cridar a `escriureCodi()` ni a `dissenyar()`; sense l'`instanceof`, arriscaries un `ClassCastException`. Mai no baixis sense preguntar.

⭐⭐ Exercici 6: Calculadora de figures amb classe abstracta

› 🔄 Solució

```java
import java.util.ArrayList;

public abstract class Figura {
    protected String color;

    public Figura(String color) {
        this.color = color;
    }

    public abstract double calcularArea();
    public abstract double calcularPerimetre();

    public void mostrarColor() {
        System.out.println("Color: " + color);
    }
}

class Cercle extends Figura {
    private double radi;

    public Cercle(String color, double radi) {
        super(color);
        this.radi = radi;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radi * radi;
    }

    @Override
    public double calcularPerimetre() {
        return 2 * Math.PI * radi;
    }
}

class Rectangle extends Figura {
    private double ample, alt;

    public Rectangle(String color, double ample, double alt) {
        super(color);
        this.ample = ample;
```

```
        this.alt = alt;
    }

    @Override
    public double calcularArea() {
        return ample * alt;
    }

    @Override
    public double calcularPerimetre() {
        return 2 * (ample + alt);
    }
}

public class TestFigures {
    public static void main(String[] args) {
        ArrayList<Figura> figures = new ArrayList<>();
        figures.add(new Cercle("roig", 3));
        figures.add(new Rectangle("blau", 4, 2));

        for (Figura f : figures) {
            f.mostrarColor();
            System.out.println("Àrea: " + f.calcularArea());
        }
    }
}
```

`Figura` aporta el color (concret) i obliga a l'àrea i al perímetre (abstractes). Les subclasses només implementen l'obligatori. `mostrarColor()` s'hereta tal qual: codi compartit on toca, contracte on toca.

⭐ ⭐ ⭐ Exercici 7: Sistema de notificacions polimòrfic

› 🔄 Solució

```
import java.util.ArrayList;
import java.util.List;

public interface Notificable {
    void enviar(String missatge);
    String getEstat();
}

class EmailNotificacio implements Notificable {
    private String direccio;
    private boolean enviat;

    public EmailNotificacio(String direccio) {
        this.direccio = direccio;
    }

    @Override
    public void enviar(String missatge) {
        System.out.println("Enviant email a " + direccio + ": " + missatge);
        enviat = true;
    }

    @Override
    public String getEstat() {
        return enviat ? "Enviat" : "Pendent";
    }
}

class SMSNotificacio implements Notificable {
    private String telefon;
    private boolean enviat;

    public SMSNotificacio(String telefon) {
        this.telefon = telefon;
    }

    @Override
    public void enviar(String missatge) {
        System.out.println("Enviant SMS a " + telefon + ": " + missatge);
        enviat = true;
    }
```

```
    @Override
    public String getEstat() {
        return enviat ? "Enviat" : "Pendent";
    }
}

class PushNotificacio implements Notificable {
    private String dispositiuId;
    private boolean enviat;

    public PushNotificacio(String dispositiuId) {
        this.dispositiuId = dispositiuId;
    }

    @Override
    public void enviar(String missatge) {
            System.out.println("Enviant push a " + dispositiuId + ": " +
missatge);
        enviat = true;
    }

    @Override
    public String getEstat() {
        return enviat ? "Enviat" : "Pendent";
    }
}

public class GestorNotificacions {
      public static void enviarTotes(List<Notificable> notificacions, String
missatge) {
        for (Notificable n : notificacions) {
            n.enviar(missatge);
        }
    }

    public static void main(String[] args) {
        ArrayList<Notificable> notificacions = new ArrayList<>();
        notificacions.add(new EmailNotificacio("ana@mail.com"));
        notificacions.add(new SMSNotificacio("600123456"));
        notificacions.add(new PushNotificacio("dev-001"));

        enviarTotes(notificacions, "Examen de Java el dilluns");
    }
```

```
}
```

`enviarTotes` no sap (ni li importa) quin tipus concret hi ha a la llista: només coneix el contracte `Notificable`. Afegir un quart canal de notificació no obliga a tocar ni una línia del gestor. Disseny obert al canvi, que és el premi del polimorfisme.

⭐ ⭐ ⭐ Exercici 8: Template method — les begudes 🔗

› 🔄 Solució

```
public abstract class Beguda {
    public final void preparar() {
        bullirAigua();
        prepararIngredient();
        servirEnTassa();
        afegirExtres();
    }

    private void bullirAigua() { System.out.println("Bullint aigua..."); }
     private void servirEnTassa() { System.out.println("Servint en tassa...");
}

    protected abstract void prepararIngredient();
    protected abstract void afegirExtres();
}

class Te extends Beguda {
    @Override
    protected void prepararIngredient() {
        System.out.println("Posant la bosseta de te...");
    }

    @Override
    protected void afegirExtres() {
        System.out.println("Afegint llima...");
    }
}

class Cafe extends Beguda {
    @Override
    protected void prepararIngredient() {
        System.out.println("Abocant el cafè màlt...");
    }

    @Override
    protected void afegirExtres() {
        System.out.println("Afegint sucre...");
    }
}

public class Cafeteria {
    public static void main(String[] args) {
```

```
        Beguda b1 = new Te();
        Beguda b2 = new Cafe();
        b1.preparar();
        b2.preparar();
    }
}
```

L'esquelet (`preparar()`) és `final`: ningú no pot reordenar els passos. Les subclasses només personalitzen els dos detalls variables. El template method garanteix que l'algoritme sempre s'executi en l'ordre correcte, passi el que passi.

⭐⭐⭐ Exercici 9: El gran repte — vehicles amb combustible

› 🔄 Solució

```
import java.util.ArrayList;

abstract class Vehicle {
    protected String matricula;
    protected int combustible;

    public Vehicle(String matricula, int combustible) {
        this.matricula = matricula;
        this.combustible = combustible;
    }

    public abstract boolean moure();
}

class Cotxe extends Vehicle {
    private static final int DESPESA = 5;

    public Cotxe(String matricula, int combustible) {
        super(matricula, combustible);
    }

    @Override
    public boolean moure() {
        if (combustible >= DESPESA) {
            combustible -= DESPESA;
            System.out.println("Cotxe " + matricula + " avança (combustible: "
+ combustible + ")");
            return true;
        } else {
            System.out.println("Sense combustible");
            return false;
        }
    }
}

class Moto extends Vehicle {
    private static final int DESPESA = 3;

    public Moto(String matricula, int combustible) {
        super(matricula, combustible);
    }
```

```
    @Override
    public boolean moure() {
        if (combustible >= DESPESA) {
            combustible -= DESPESA;
             System.out.println("Moto " + matricula + " avança (combustible: "
+ combustible + ")");
            return true;
        } else {
            System.out.println("Sense combustible");
            return false;
        }
    }
}

class Camio extends Vehicle {
    private static final int DESPESA = 10;
    private int càrrega;

    public Camio(String matricula, int combustible, int càrrega) {
        super(matricula, combustible);
        this.càrrega = càrrega;
    }

    @Override
    public boolean moure() {
        if (combustible >= DESPESA) {
            combustible -= DESPESA;
                System.out.println("Camió " + matricula + " amb càrrega " +
càrrega
                    + " avança (combustible: " + combustible + ")");
            return true;
        } else {
            System.out.println("Sense combustible");
            return false;
        }
    }
}

public class Circuit {
    public static void main(String[] args) {
        ArrayList<Vehicle> vehicles = new ArrayList<>();
        vehicles.add(new Cotxe("1234ABC", 12));
        vehicles.add(new Moto("5678DEF", 8));
```

```
        vehicles.add(new Camio("9999ZZZ", 25, 3000));

        for (Vehicle v : vehicles) {
            int moviments = 0;
            while (v.moure()) {
                moviments++;
            }
            System.out.println("Moviments: " + moviments);
        }
    }
}
```

Cada subclasse defineix la seva despesa amb una constant i el seu `moure()`, que torna `true` només si va poder moure's. El `main` usa només `Vehicle`: el `while (v.moure())` pregunta al mateix vehicle i el polimorfisme fa la resta. Aquesta manera d'escriure el bucle evita el problema de comprovar el combustible des de fora (que deixaria un bucle infinit quan el vehicle no pot moure's però encara li queda combustible). Si demà arriba una `Bicicleta` (despesa 0), entra sense tocar el circuit. L'abstracció paga la casa.

Butlletí U11 — Extres

📝 Butlletí U11 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. La solució està oculta: resisteix-te fins a esgotar la teva pista. CodeWars i AceptaElReto són els gimnasos on els programadors es repten cada dia: aquí és on l'herència deixa de ser teoria i es converteix en reflexos.

CodeWars

1. Thinkful — Object Drills: Quarks

Crea la classe `Quark` amb tres propietats: `color` (String), `flavor` (String) i `baryon_number` (sempre `1.0`). A més:

- Constructor que rep `color` i `flavor`.
- `interact(altre)` → intercanvia els colors dels dos quarks.

Exemple:

```
Quark q1 = new Quark("red", "up");
Quark q2 = new Quark("blue", "strange");
q1.interact(q2);
q1.color;  // "blue"
q2.color;  // "red"
q1.baryon_number;  // 1.0
```

- Enunciat a CodeWars
- Dificultat: 7 kyu

Pista: `baryon_number` és una constant que tots els quarks comparteixen: `public final double baryon_number = 1.0;`. `interact()` usa una variable temporal per a intercanviar: `String temp = this.color; this.color = altre.color; altre.color = temp;`. El `this` desambigua qui és qui en l'intercanvi.

> 🔄 Solució

```java
public class Quark {
    public String color;
    public String flavor;
    public final double baryon_number = 1.0;

    public Quark(String color, String flavor) {
        this.color = color;
        this.flavor = flavor;
    }

    public void interact(Quark altre) {
        String temporal = this.color;
        this.color = altre.color;
        altre.color = temporal;
    }
}
```

La física de quarks aplicada: `baryon_number` és `final` perquè cap quark canvia el seu nombre bariònic (és una constant universal). `interact()` intercanvia els colors amb una variable temporal; sense ella, un dels dos colors es perdria. Aquest és el clàssic "swap" que ja vas veure amb variables, ara entre dos objectes.

---

2. Building blocks

Crea la classe `Block` que rep les tres dimensions (com `int[]` de 3 o com 3 enters) i els mètodes:

- `int getWidth()`, `int getLength()`, `int getHeight()`
- `int getVolume()` → `width * length * height`
- `int getSurfaceArea()` → `2 * (w*l + w*h + l*h)`

Exemple: `new Block(new int[]{2, 4, 6})` → volum 48, superfície `2*(2*4 + 2*6 + 4*6) = 88`.

- Enunciat a CodeWars
- Dificultat: 7 kyu

Pista: guarda les tres dimensions en atributs privats en el constructor, i deixa que els getters les tornen. Per a la superfície, la fórmula és la suma de les tres cares per dos. El polimorfisme no

apareix aquí, però l'objecte amb estat i comportament sí: l'excusa perfecta per a repassar la U09 mentre penses en l'herència.

> 🔄 **Solució**

```
public class Block {
    private final int width;
    private final int length;
    private final int height;

    public Block(int[] dimensions) {
        this.width = dimensions[0];
        this.length = dimensions[1];
        this.height = dimensions[2];
    }

    public int getWidth() {
        return width;
    }

    public int getLength() {
        return length;
    }

    public int getHeight() {
        return height;
    }

    public int getVolume() {
        return width * length * height;
    }

    public int getSurfaceArea() {
        return 2 * (width * length + width * height + length * height);
    }
}
```

L'objecte `Block` guarda el seu estat i l'exposa amb getters. Els atributs `final` es fixen en el constructor (un bloc no canvia de forma). El volum i la superfície són mètodes que calculen a partir de l'estat. És un objecte amb responsabilitat única: sap les seves dimensions i com mesurar-se. L'herència del curs arribarà quan vulguis especialitzar-lo en `Cub` o `Caixa` sense duplicar codi.

3. Basic subclasses — Adam and Eve

Segons el mite, Adam i Eva van ser els primers humans. El teu treball és “fer el treball de Déu”: crear un mètode estàtic `create()` que torni un array de `Human` amb dos objectes: el primer un `Man` i el segon una `Woman`. Les dues classes hereten de `Human`, i cada humà té `name`, `sex` i la propietat `species` amb valor `"Human"`.

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: herència pura: `class Man extends Human` i `class Woman extends Human`. Cada subclasse crida `super(...)` per a omplir el nom i el sexe. L’array de retorn és de tipus `Human`, així que accepta les dues subclasses.

> 🔄 **Solució**

```
public class Human {
    private String name;
    private String sex;
    protected String species = "Human";

    public Human(String name, String sex) {
        this.name = name;
        this.sex = sex;
    }

    public String getName() {
        return name;
    }

    public String getSex() {
        return sex;
    }

    public String getSpecies() {
        return species;
    }
}

class Man extends Human {
    public Man(String name) {
        super(name, "man");
    }
}

class Woman extends Human {
    public Woman(String name) {
        super(name, "woman");
    }
}

public class God {
    public static Human[] create() {
        return new Human[] { new Man("Adam"), new Woman("Eve") };
    }
}
```

> `Man` i `Woman` heretem tot de `Human` i només aporten el seu constructor amb el sexe fix. `create()` torna un array de `Human` (el tipus general) omplit amb les dues subclasses: polimorfisme de dalt a baix, com Adam i Eva al Paradís.

---

4. Object Oriented Piracy

Crea la classe `Ship` que rep un `draft` (calat) i un `crew` (tripulants). Implementa `isWorthIt()`: torna `true` si el calat total menys `1.5` per cada tripulant supera 20.

Exemple: `new Ship(15, 10).isWorthIt()` → `false` (`15 - 1.5*10 = 0`).

- Enunciat a CodeWars
- Dificultat: 8 kyu

Pista: guarda `draft` i `crew` en atributs `private final`. El mètode combina tots dos: `return draft - 1.5 * crew > 20;`. És la classe d'objecte simple que ja domines: una excusa per a repassar que l'estat viu en l'objecte, no en el main.

> 🔄 **Solució**

```
public class Ship {
    private static final double PES_TRIPULANT = 1.5;

    private final double draft;
    private final int crew;

    public Ship(double draft, int crew) {
        this.draft = draft;
        this.crew = crew;
    }

    public boolean isWorthIt() {
        return draft - PES_TRIPULANT * crew > 20;
    }
}
```

Cada `Ship` guarda el seu propi estat (`draft` i `crew`) i decideix per si mateix si mereix la pena. Els atributs `final` fan l'objecte immutable: es fixen en néixer. La constant `static final` documenta el `1.5`. És el mateix patró d'objecte amb comportament que has practicat tota la unitat.

AceptaElReto

5. 117 — La festa avorrida

Tinín odia les festes, i cada persona que se li acosta es presenta amb el format `"Soc Lotari"`. Ajuda'l a respondre `"Hola, [nom]."` a cadascun. L'entrada comença amb un nombre que indica quantes persones hi ha, seguit d'una línia per persona. Resol-lo amb una classe `Persona` que encapsuli el nom i un mètode `saludar()`.

Entrada d'exemple:

```
3
Soc Lotari
Soc Aldonça
Soc Ender
```

Sortida d'exemple:

```
Hola, Lotari.
Hola, Aldonça.
Hola, Ender.
```

- Enunciat a AceptaElReto
- Dificultat: Fàcil

Pista: llegeix la línia i treu el `"Soc "` inicial amb `linea.substring(4)` o `split(" ")[1]`. Crea la `Persona` amb aquell nom i crida `saludar()`. És l'excusa perfecta per a veure que un objecte amb un mètode pot substituir un main que fa de tot.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Persona {
    private String nom;

    public Persona(String nom) {
        this.nom = nom;
    }

    public void saludar() {
        System.out.println("Hola, " + nom + ".");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();  // consumeix el salt de línia

        for (int i = 0; i < n; i++) {
            String linea = sc.nextLine();
            Persona p = new Persona(linea.substring(4));
            p.saludar();
        }
        sc.close();
    }
}
```

`Persona` encapsula el seu nom i sap saludar: el `main` només llegeix i crea objectes. `linea.substring(4)` es salta `"Soc "`. És la manera “orientada a objectes” de resoldre un problema que també podries fer amb un `String` solt: aquí l’estat (el nom) i el comportament (`saludar()`) viuen junts en la classe.

6. 119 — Escuts de l’exèrcit romà

Un general divideix els seus legionaris en formacions **quadrades** (el més gran possible), repetint amb els que queden lliures fins a esgotar-los. Cada quadrat de costat n necessita escuts segons el perímetre més la cobertura: per a un quadrat de $n \times n$, els escuts són $n^2 + 4n$ (una base per soldat més el perímetre exterior). Donat el nombre de legionaris, calcula el mínim d’escuts necessaris.

Entrada: diversos casos de prova, cadascun amb el nombre de legionaris. Termina amb `0`.

Entrada d'exemple:

```
35
20
10
0
```

Sortida d'exemple:

```
71
44
26
```

- Enunciat a AceptaElReto
- Dificultat: Fàcil/Mitjana

Pista: mentre queden legionaris, troba el major quadrat `n` tal que `n² <= restants` (prova `n` creixent o usa `Math.sqrt`). Suma els escuts d'aquell quadrat i resta `n²` dels restants. Encapsula la lògica en una classe `Formacio` amb mètodes com `majorQuadrat()` i `calcularEscuts()`.

> 🔄 **Solució**

```java
import java.util.Scanner;

public class Formacio {
    public static int escutsDeQuadrat(int costat) {
        return costat * costat + 4 * costat;
    }

    public static int majorQuadrat(int restants) {
        int n = (int) Math.sqrt(restants);
        return n * n;  // el major quadrat perfecte <= restants
    }

    public static int resoldre(int legionaris) {
        int escuts = 0;
        int restants = legionaris;

        while (restants > 0) {
            int quadrat = majorQuadrat(restants);
            int costat = (int) Math.sqrt(quadrat);
            escuts += escutsDeQuadrat(costat);
            restants -= quadrat;
        }
        return escuts;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        while (n != 0) {
            System.out.println(resoldre(n));
            n = sc.nextInt();
        }
        sc.close();
    }
}
```

Verifica-ho amb 35: el major quadrat és 25 (costat 5) → `25 + 20 = 45` escuts; queden 10, major quadrat 9 (costat 3) → `9 + 12 = 21`; queda 1 (costat 1) → `1 + 4 = 5`. Total `45 + 21 + 5 = 71` ✓. `Math.sqrt` et dona l'arrel; en truncar obtens el costat del major quadrat que cap. La classe agrupa els tres càlculs com a mètodes estàtics: pura lògica ben empaquetada.

🧭 **I si et quedes amb ganes?** Quan domines l'herència, torna als problemes d'unitats anteriors i reescriu-los amb jerarquies: un `Lector` abstracte, un `Solucionador` polimòrfic, figures que es calculen soles. El material no es perd: es reutilitza.

Butlletí U12 — Inicial

📝 Butlletí U12 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, crea el teu primer `ArrayList`, la teva primera `Caja<T>` i fes que el `HashMap` deixi de semblar màgia. Comença suaument, que les col·leccions mosseguen a poc a poc.

Exercici 1: Què imprimeix? — ArrayList remove per índex vs valor

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("B");
        lista.add("D");

        lista.remove(1);          // remove per índex
        lista.remove("B");        // remove per objecte

        System.out.println(lista);
    }
}
```

Què imprimeix? Per què el segon `remove("B")` no esborra el mateix element que el primer?

Exercici 2: Troba l'error — size() vs length vs length()

```java
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");

int[] edades = {20, 30};
String saludo = "Hola";

System.out.println(nombres.length);   // línia 1
System.out.println(edades.size());    // línia 2
System.out.println(saludo.length);    // línia 3
```

Quines línies tenen error? Explica què usa cada tipus per a preguntar quant mesura: `size()`, `length` o `length()`.

Exercici 3: Completa el codi — for-each que suma una llista

Completa el següent programa perquè sumi tots els números d'una `ArrayList<Integer>`:

```java
import java.util.ArrayList;

public class SumaLista {
    public static void main(String[] args) {
        ArrayList<Integer> numeros = new ArrayList<>();
        numeros.add(4);
        numeros.add(9);
        numeros.add(2);
        numeros.add(7);

        int suma = ______;
        for (______ n : numeros) {      // quin tipus i quina variable?
            suma ______ n;              // quin operador?
        }

        System.out.println("Suma: " + suma);
    }
}
```

Què falta en cada buit? Quant val la suma al final?

Exercici 4: Escriu aquest programa — la llista de la compra

Crea un programa amb un `ArrayList<String>` anomenat `compra` i fes això:

1. Afegeix `"Leche"`, `"Pan"` i `"Huevos"`.
2. Afegeix `"Café"` en la posició 1 (entre Leche i Pan).
3. Mostra la grandària de la llista.
4. Esborra l'element de la posició 2.
5. Recorre la llista amb un for-each i imprimeix cada element.

Pista: usa `add(e)`, `add(i, e)`, `remove(i)`, `size()` i un for-each.

Exercici 5: Què imprimeix? — el HashSet que no deixa repetir

```
import java.util.HashSet;

public class Test {
    public static void main(String[] args) {
        HashSet<String> invitados = new HashSet<>();
        invitados.add("Ana");
        invitados.add("Bob");
        invitados.add("Ana");

        System.out.println(invitados.size());
        System.out.println(invitados.contains("Bob"));
    }
}
```

Què imprimeix? Per què el segon `add("Ana")` no provoca cap error ni avís?

Exercici 6: Escriu aquest programa — sense duplicats

Crea un `ArrayList<String>` amb noms de companys, **amb repetits a propòsit** (mínim 8 elements, 3 d'ells repetits). Després:

1. Construeix un `LinkedHashSet<String>` a partir de la llista.
2. Imprimeix la llista original i el conjunt.
3. Explica en un comentari quina diferència veus i per què.

Pista: el constructor de `LinkedHashSet` accepta qualsevol col·lecció: `new LinkedHashSet<>(lista)`.

Exercici 7: Completa el codi — la teva primera classe genèrica

```
public class Caja<______> {        // quin paràmetre de tipus falta?
    private ______ contenido;

    public void guardar(______ contenido) {
        this.contenido = contenido;
    }

    public ______ sacar() {
        return contenido;
    }
}
```

Completa els buits perquè `Caja<String>` guardi Strings i `Caja<Integer>` guardi Integers. Per què `Caja<int>` no compila?

Exercici 8: Què imprimeix? — HashMap amb put repetit

```java
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, Integer> mapa = new HashMap<>();
        mapa.put("Ana", 10);
        mapa.put("Bob", 20);
        mapa.put("Ana", 30);

        System.out.println(mapa.get("Ana"));
        System.out.println(mapa.size());
    }
}
```

Què imprimeix? Per què `size()` no és 3?

Exercici 9: Escriu aquest programa — comptador de paraules amb HashMap

Crea un programa que tingui un array de paraules (hardcodejat) com aquest:

```java
String[] palabras = {"hola", "mundo", "hola", "java", "mundo", "hola", "adios"};
```

Usa un `HashMap<String, Integer>` per a comptar quantes vegades apareix cada paraula. Al final, recorre el mapa amb un bucle for-each sobre `entrySet()` i mostra cada paraula amb el seu compte.

Pista: el patró estrella és `mapa.put(p, mapa.getOrDefault(p, 0) + 1)`.

Butlletí U12 — Inicial Resolt

📝 Butlletí U12 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què imprimeix? — ArrayList remove per índex vs valor

> 🔄 **Solució**
>
> Imprimeix **`[A, C, D]`**.
>
> - `lista.remove(1)` esborra per **índex**: lleva el `"B"` de la posició 1 → `[A, C, B, D]`.
> - `lista.remove("B")` esborra per **objecte**: busca el primer `"B"` que quedi → `[A, C, D]`.
>
> El primer mira posicions; el segon, continguts. Com que l'`Integer` té sobrecàrregues (`remove(int)` vs `remove(Object)`), aquesta distinció és la trampa favorita dels exàmens.

Exercici 2: Troba l'error — size() vs length vs length()

> 🔄 **Solució**
>
> Les tres línies tenen error:
>
> - **Línia 1:** un `ArrayList` pregunta la seva grandària amb `size()`, no amb `.length` (això és per a arrays).
> - **Línia 2:** un array usa `.length` (sense parèntesis), no `.size()`.
> - **Línia 3:** un `String` usa `.length()` (amb parèntesis, és un mètode).
>
> Regla mnemotècnica: col·lecció → `size()`, array → `length`, String → `length()`.

Exercici 3: Completa el codi — for-each que suma una llista

> 🔄 Solució

```
int suma = 0;
for (Integer n : numeros) {   // o int n, amb unboxing automàtic
    suma += n;
}
```

La suma final és **22** (4 + 9 + 2 + 7). El buit inicial era `0` (començar acumulant des de zero), el tipus era `Integer` (o `int`, i l'unboxing fa la resta) i l'operador, `+=`.

Exercici 4: Escriu aquest programa — la llista de la compra

> 🔄 Solució

```
import java.util.ArrayList;

public class ListaCompra {
    public static void main(String[] args) {
        ArrayList<String> compra = new ArrayList<>();
        compra.add("Leche");       // [Leche]
        compra.add("Pan");         // [Leche, Pan]
        compra.add("Huevos");      // [Leche, Pan, Huevos]
        compra.add(1, "Café");     // [Leche, Café, Pan, Huevos]

        System.out.println("Grandària: " + compra.size());   // 4

        compra.remove(2);          // lleva "Pan" → [Leche, Café, Huevos]

        for (String articulo : compra) {
            System.out.println(articulo);
        }
    }
}
```

`add(e)` afegeix al final, `add(i, e)` se cola en la posició `i` desplaçant la resta, i `remove(i)` lleva per índex. Ull: després d'inserir `"Café"` en la posició 1, el `"Pan"` passa a la posició 2, així que `remove(2)` el lleva a ell.

Exercici 5: Què imprimeix? — el HashSet que no deixa repetir

> 🔄 Solució

Imprimeix **`2`** i **`true`**.

El segon `add("Ana")` no falla ni avisa: simplement torna `false` i no afegeix res, perquè el `HashSet` ja conté "Ana" (compara amb `hashCode()` i `equals()`). Per això `size()` és 2 (només Ana i Bob) i `contains("Bob")` és `true`.

Exercici 6: Escriu aquest programa — sense duplicats

> 🔄 Solució

```java
import java.util.ArrayList;
import java.util.LinkedHashSet;

public class SinDuplicados {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>();
        nombres.add("Ana");
        nombres.add("Bob");
        nombres.add("Ana");
        nombres.add("Carla");
        nombres.add("Bob");
        nombres.add("Diego");
        nombres.add("Ana");
        nombres.add("Eva");

        LinkedHashSet<String> sinRepetir = new LinkedHashSet<>(nombres);

        System.out.println("Llista:    " + nombres);
        System.out.println("Conjunt:   " + sinRepetir);
    }
}
```

La llista imprimeix els 8 elements amb els seus repetits; el conjunt, només 5: `Ana`, `Bob`, `Carla`, `Diego` i `Eva`. El `LinkedHashSet` elimina els duplicats i conserva l'ordre de primera aparició. Amb un `HashSet` a seques, l'ordre de sortida seria imprevisible.

Exercici 7: Completa el codi — la teva primera classe genèrica

▸ 🔄 Solució

```java
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return contenido;
    }
}
```

El paràmetre de tipus `<T>` es declara al costat del nom de la classe i s'usa com un tipus més dins d'ella. `Caja<int>` no compila perquè els genèrics només accepten tipus referència: `int` és un primitiu, així que toca usar `Caja<Integer>` i deixar que l'autoboxing converteixi sol.

Exercici 8: Què imprimeix? — HashMap amb put repetit

› 🔄 **Solució**

Imprimeix **`30`** i **`2`**.

- `put("Ana", 10)` i després `put("Ana", 30)`: la clau "Ana" es sobreescriu amb l'últim valor.
- Per això `size()` és 2, no 3: les claus són úniques i "Ana" només compta una vegada.

Exercici 9: Escriu aquest programa — comptador de paraules amb HashMap

› 🔄 **Solució**

```java
import java.util.HashMap;
import java.util.Map;

public class ContadorPalabras {
    public static void main(String[] args) {
        String[] palabras = {"hola", "mundo", "hola", "java", "mundo", "hola",
"adios"};

        HashMap<String, Integer> contador = new HashMap<>();
        for (String p : palabras) {
            contador.put(p, contador.getOrDefault(p, 0) + 1);
        }

        for (Map.Entry<String, Integer> entrada : contador.entrySet()) {
            System.out.println(entrada.getKey() + " → " + entrada.getValue());
        }
    }
}
```

El patró de les freqüències: `getOrDefault(p, 0) + 1` torna el compte actual (o 0 la primera vegada) i suma 1. `entrySet()` et dona cada paraula amb el seu comptador en un sol bucle, sense un `get` extra.

Butlletí U12 — Avançat

📝 Butlletí U12 — Avançat

> Dificultat progressiva. ⭐ per a calentar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ Exercici 1: La cua del supermercat amb LinkedList

Simula una cua de supermercat usant `LinkedList<String>`. El programa ha de mostrar un menú:

1. **Arriba client** → Afegeix un nom al final de la cua.
2. **Atén client** → Elimina i mostra el primer de la cua.
3. **Qui segueix?** → Mostra el primer sense eliminar-lo.
4. **Estat de la cua** → Mostra tots els clients en ordre.
5. **Sortir**

Usa els mètodes `addLast()`, `removeFirst()` i `getFirst()` de `LinkedList`, amb un `switch` i un `Scanner`.

Pista: el menú es repeteix amb un `while` fins que l'usuari triï `0`. Cuida els casos en què la cua està buida abans de cridar `removeFirst()` o `getFirst()`.

⭐⭐ Exercici 2: Intersecció i unió de conjunts

Crea dos `HashSet<Integer>` amb números aleatoris (entre 1 i 20, 8 elements cada un). Calcula i mostra:

- **Intersecció:** elements que estan en tots dos conjunts.
- **Unió:** tots els elements sense repetir.
- **Diferència simètrica:** elements que estan en un o en l'altre, però no en tots dos.

Pista: usa `retainAll()`, `addAll()` i `removeAll()` de la interfície `Set`. Per a la diferència simètrica: `union.removeAll(interseccion)`.

⭐ ⭐ Exercici 3: Eliminar duplicats mantenint l'ordre

Crea un `ArrayList<Integer>` amb elements repetits (`[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`). Escriu un mètode que torni un nou `ArrayList<Integer>` **sense duplicats però mantenint l'ordre de primera aparició**.

Pista: un `HashSet` no val directament perquè perd l'ordre. Usa un `LinkedHashSet` (conserva l'ordre d'inserció) o recorre manualment comprovant amb `contains` abans d'afegir.

⭐⭐⭐ Exercici 4: Què imprimeix? — el remove que trenca el ball

Sense executar, respon què passa amb aquest programa:

```
import java.util.ArrayList;

public class Puzle {
    public static void main(String[] args) {
        ArrayList<String> palabras = new ArrayList<>();
        palabras.add("hola");
        palabras.add("mundo");
        palabras.add("adiós");

        for (String p : palabras) {
            if (p.equals("mundo")) {
                palabras.remove(p);
            }
        }
        System.out.println(palabras);
    }
}
```

Imprimeix alguna cosa? O llança una excepció? Quina i per què?

Pista: pensa en què fa el `for-each` per davall (usa un `Iterator`) i en què passa si la col·lecció canvia mentre es recorre.

⭐⭐ Exercici 5: Filtrar amb Iterator

Crea un `ArrayList<Integer>` amb els números de l'1 al 10. Recorre'l amb un `Iterator<Integer>` explícit i **esborra tots els números parells** usant `it.remove()`. Al final, imprimeix la llista.

Pista: recorre amb `while (it.hasNext())`, obtín cada número amb `it.next()` i comprova si és parell amb `% 2 == 0`. Mai uses `lista.remove(...)` dins del bucle.

⭐⭐ Exercici 6: El TreeSet ordenat

Crea un programa que demani a l'usuari paraules amb `Scanner` fins que escrigui `"fin"`. Guarda-les en un `TreeSet<String>`. En acabar, mostra:

1. Les paraules ordenades (imprimir el TreeSet directament).
2. La primera paraula (`first()`) i l'última (`last()`).
3. Les paraules que van abans que `"m"` en ordre alfabètic (`headSet("m")`).

Pista: el TreeSet ordena i elimina duplicats automàticament. Comprova què passa si l'usuari repeteix una paraula.

⭐ Pila genèrica <T>: Exercici 7

Implementa una classe genèrica `Pila<T>` que funcioni com una pila (LIFO). Ha de tenir els mètodes:

- `void push(T elemento)` — apila un element.
- `T pop()` — desapila i torna l'element superior (llança `EmptyStackException` si està buida).
- `T peek()` — torna l'element superior sense desapilar-lo.
- `boolean isEmpty()` — indica si està buida.
- `int size()` — nombre d'elements.

Internament, usa un `ArrayList<T>` com a emmagatzematge. Prova-la amb `Pila<Integer>`, `Pila<String>` i `Pila<Double>`.

Pista: `EmptyStackException` està en `java.util.EmptyStackException`. Recorda comprovar `isEmpty()` abans de `pop()`/`peek()`.

⭐⭐ Exercici 8: HashMap invers

Escriu un mètode genèric estàtic:

```
public static <K, V> HashMap<V, K> invertirMapa(HashMap<K, V> original)
```

Que torni un nou `HashMap` intercanviant claus i valors. Si hi ha valors duplicats en el mapa original, l'últim trobat sobreescriu l'anterior.

Prova-ho amb un mapa de `String → Integer` i un altre de `String → String`.

Pista: recorre `original.entrySet()` i fes `invertido.put(e.getValue(), e.getKey())`.

⭐⭐ Exercici 9: TreeMap — freqüència de lletres

Escriu un programa que llisqui un text per teclat (o n'usi un hardcodejat) i compte quantes vegades apareix cada **lletra** (ignorant espais, números i signes). Usa un `TreeMap<Character, Integer>` perquè les lletres es mostren automàticament ordenades alfabèticament.

Exemple de sortida per «Hola mundo»:

```
a: 1, d: 1, h: 1, l: 1, m: 1, n: 1, o: 2, u: 1
```

Pista: recorre el `String` amb `toCharArray()` i usa `Character.isLetter(c)` per a filtrar. El `getOrDefault` suma el comptador; el TreeMap ordena sol.

⭐⭐⭐ Exercici 10: Wildcards — suma i barreja de números

Implementa un mètode que sumi tots els números d'una llista, acceptant qualsevol subtipus de `Number`:

```
public static double sumar(List<? extends Number> lista)
```

Prova-ho amb `List<Integer>`, `List<Double>` i `List<Float>`. Què passa si intentes passar una `List<String>`?

Crea també un segon mètode que **barrege** dues llistes de números de tipus distints en una sola `List<Double>`:

```
public static List<Double> mezclar(List<? extends Number> a, List<? extends Number> b)
```

Pista: per a `sumar`, recorre amb `for (Number n : lista)` i usa `n.doubleValue()`. Per a `mezclar`, usa `addAll()` i converteix cada element amb `doubleValue()`. I no intentis `add` en `sumar`: `? extends` és de només lectura (PECS).

⭐ ⭐ ⭐ Exercici 11: Sistema de votacions amb mètode genèric

Crea un sistema de votacions on:

- Cada votant pot votar per un candidat (String).
- Usa un `HashMap<String, Integer>` per als vots.
- Usa un `TreeMap<String, Integer>` per a mostrar el rànquing ordenat.

Crea un mètode genèric:

```
public static <T> T obtenerGanador(Map<T, Integer> votos)
```

Que torni la clau amb més vots. Prova-ho amb un `HashMap<String, Integer>` i un altre `HashMap<Integer, Integer>`.

Pista: recorre `votos.entrySet()` i guarda el guanyador provisional comparant `getValue()` amb un màxim acumulat. El mètode és genèric perquè el tipus de la clau (`String`, `Integer`…) no importa.

⭐⭐⭐ Exercici 12: el type erasure al descobert

La classe `Caja<T>` guarda un valor i el torna amb `getValor()`:

```
public class Caja<T> {
    private T valor;

    public Caja(T valor) {
        this.valor = valor;
    }

    public T getValor() {
        return valor;
    }
}
```

Sense executar, respon:

1. Són `Caja<String>` i `Caja<Integer>` classes distintes en temps d'execució?
2. Quin tipus té realment `caja.getValor()` dins del bytecode si el compiles com `Caja<String>`?
3. Escriu un `main` que creï `Caja<String>` i `Caja<Integer>` i comprovi amb `getClass()` que totes dues són instàncies de la mateixa classe `Caja`.

Pista: el type erasure converteix `Caja<T>` en `Caja` a palpes (amb `Object` on estava `T`). Per això `caja.getClass()` torna el mateix per a `Caja<String>` i `Caja<Integer>`: en runtime no hi ha dues classes, només una `Caja`. El cast de `getValor()` l'afegeix el compilador, no el teu codi.

Butlletí U12 — Avançat Resolt

📝 Butlletí U12 — Avançat (Resolt) 🔗

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: La cua del supermercat amb LinkedList 🔗

> 🔄 Solució

```
import java.util.LinkedList;
import java.util.Scanner;

public class ColaSupermercado {
    public static void main(String[] args) {
        LinkedList<String> cola = new LinkedList<>();
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
              System.out.println("\n1. Llega cliente  2. Atender cliente  3. ¿Quién sigue?  4. Estado  0. Salir");
            opcion = sc.nextInt();
            sc.nextLine();

            switch (opcion) {
                case 1:
                    System.out.print("Nombre del cliente: ");
                    cola.addLast(sc.nextLine());
                    break;
                case 2:
                    if (!cola.isEmpty()) {
                                    System.out.println("Atendiendo a: " + cola.removeFirst());
                    } else {
                        System.out.println("No hay nadie en la cola.");
                    }
                    break;
                case 3:
                    if (!cola.isEmpty()) {
                                System.out.println("El siguiente es: " + cola.getFirst());
                    } else {
                        System.out.println("No hay nadie en la cola.");
                    }
                    break;
                case 4:
                    System.out.println("Cola: " + cola);
                    break;
            }
        } while (opcion != 0);
```

```
        sc.close();
    }
}
```

`addLast` fica al final (els clients es posen darrere), `removeFirst` atén el primer (FIFO) i `getFirst` el mira sense treure-lo. El `isEmpty()` evita l'error de demanar-li el primer a una cua buida.

⭐⭐ Exercici 2: Intersecció i unió de conjunts

› 🔄 Solució

```
import java.util.HashSet;
import java.util.Set;

public class Conjuntos {
    public static void main(String[] args) {
        Set<Integer> a = new HashSet<>();
        Set<Integer> b = new HashSet<>();

        while (a.size() < 8) a.add((int) (Math.random() * 20) + 1);
        while (b.size() < 8) b.add((int) (Math.random() * 20) + 1);

        Set<Integer> interseccion = new HashSet<>(a);
        interseccion.retainAll(b);

        Set<Integer> union = new HashSet<>(a);
        union.addAll(b);

        Set<Integer> difSimetrica = new HashSet<>(union);
        difSimetrica.removeAll(interseccion);

        System.out.println("A: " + a);
        System.out.println("B: " + b);
        System.out.println("Intersección: " + interseccion);
        System.out.println("Unión: " + union);
        System.out.println("Diferencia simétrica: " + difSimetrica);
    }
}
```

La màgia són els tres mètodes de `Set`: `retainAll` deixa només el comú, `addAll` uni sense duplicats i `removeAll` lleva la intersecció de la unió per a deixar el que està només en un dels dos. Com que `HashSet` no admet duplicats, la unió surt neta sola. El `while` garanteix 8 elements únics en cada conjunt.

⭐ ⭐ Exercici 3: Eliminar duplicats mantenint l'ordre

› 🔄 Solució

```
import java.util.ArrayList;
import java.util.LinkedHashSet;

public class SinDuplicados {
    public static ArrayList<Integer> sinDuplicados(ArrayList<Integer> lista) {
        LinkedHashSet<Integer> set = new LinkedHashSet<>(lista);
        return new ArrayList<>(set);
    }

    public static void main(String[] args) {
         ArrayList<Integer> lista = new ArrayList<>(java.util.Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5));
        System.out.println(sinDuplicados(lista)); // [3, 1, 4, 5, 9, 2, 6]
    }
}
```

`LinkedHashSet` és la combinació perfecta: elimina duplicats (com `HashSet`) però conserva l'ordre d'inserció (com una llista). Es construeix passant-li la llista i es torna a convertir en `ArrayList`. Resultat: `[3, 1, 4, 5, 9, 2, 6]`, sense repetits i en ordre de primera aparició.

⭐⭐⭐ Exercici 4: Què imprimeix? — el remove que trenca el ball

› 🔄 Solució

Llança una **`ConcurrentModificationException`**.

El `for-each` usa un `Iterator` per davall. Quan dins del bucle crides a `palabras.remove(p)`, la llista canvia mentre l'iterador la està recorrent. L'iterador detecta la modificació externa i explota. La solució és recórrer amb un `Iterator` explícit i usar `it.remove()`. O construir una llista nova amb els elements que vols conservar.

```
import java.util.ArrayList;
import java.util.Iterator;

public class Puzle {
    public static void main(String[] args) {
        ArrayList<String> palabras = new ArrayList<>();
        palabras.add("hola");
        palabras.add("mundo");
        palabras.add("adiós");

        Iterator<String> it = palabras.iterator();
        while (it.hasNext()) {
            if (it.next().equals("mundo")) {
                it.remove();
            }
        }
        System.out.println(palabras); // [hola, adiós]
    }
}
```

⭐⭐ Exercici 5: Filtrar amb Iterator

› 🔄 Solució

```java
import java.util.ArrayList;
import java.util.Iterator;

public class FiltrarPares {
    public static void main(String[] args) {
        ArrayList<Integer> numeros = new ArrayList<>();
        for (int i = 1; i <= 10; i++) {
            numeros.add(i);
        }

        Iterator<Integer> it = numeros.iterator();
        while (it.hasNext()) {
            if (it.next() % 2 == 0) {
                it.remove();
            }
        }

        System.out.println(numeros); // [1, 3, 5, 7, 9]
    }
}
```

`it.next()` torna el número i avança; si és parell, `it.remove()` el esborra de la llista original sense llançar excepció. Recorda: només pots esborra l'element que acaba de tornar `next()`, i l'ordre importa.

⭐⭐ Exercici 6: El TreeSet ordenat

› 🔄 Solució

```java
import java.util.Scanner;
import java.util.TreeSet;

public class PalabrasOrdenadas {
    public static void main(String[] args) {
        TreeSet<String> palabras = new TreeSet<>();
        Scanner sc = new Scanner(System.in);

        System.out.println("Escribe palabras (fin para terminar):");
        String palabra = sc.nextLine();
        while (!palabra.equals("fin")) {
            palabras.add(palabra);
            palabra = sc.nextLine();
        }

        System.out.println("Ordenadas: " + palabras);
        System.out.println("Primera: " + palabras.first());
        System.out.println("Última: " + palabras.last());
        System.out.println("Antes de 'm': " + palabras.headSet("m"));
        sc.close();
    }
}
```

El `TreeSet` ordena automàticament (ordre alfabètic) i **elimina duplicats**: si l'usuari repeteix una paraula, només es guarda una vegada. `first()` i `last()` donen els extrems; `headSet("m")` torna totes les paraules que van abans que "m" en l'ordre natural.

⭐ Pila genèrica <T>: Exercici 7

› 🔄 Solució

```java
import java.util.ArrayList;
import java.util.EmptyStackException;

public class Pila<T> {
    private ArrayList<T> elementos = new ArrayList<>();

    public void push(T elemento) {
        elementos.add(elemento);
    }

    public T pop() {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        return elementos.remove(elementos.size() - 1);
    }

    public T peek() {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        return elementos.get(elementos.size() - 1);
    }

    public boolean isEmpty() {
        return elementos.isEmpty();
    }

    public int size() {
        return elementos.size();
    }
}
```

La pila es construeix sobre un `ArrayList<T>`: el final de la llista és la cima. `push` afegeix, `pop` llevo i torna l'últim, i `peek` el mira sense llevar-lo. En ser genèrica, funciona igual amb `Integer`, `String` o `Double`: `new Pila<String>()` i llisto.

⭐⭐ Exercici 8: HashMap invers

> 🔄 Solució

```java
import java.util.HashMap;

public class Utilidades {

    public static <K, V> HashMap<V, K> invertirMapa(HashMap<K, V> original) {
        HashMap<V, K> invertido = new HashMap<>();
        for (HashMap.Entry<K, V> e : original.entrySet()) {
            invertido.put(e.getValue(), e.getKey());
        }
        return invertido;
    }

    public static void main(String[] args) {
        HashMap<String, Integer> edades = new HashMap<>();
        edades.put("Ana", 25);
        edades.put("Bob", 30);

        HashMap<Integer, String> porEdad = invertirMapa(edades);
        System.out.println(porEdad.get(25)); // Ana
        System.out.println(porEdad.get(30)); // Bob
    }
}
```

Recórrer `entrySet()` et dona clau i valor junts, i el `put` invertit els canvia de lloc. Si dos claus comparteixen valor (dues persones de 25 anys), l'últim en el recorregut sobreescriu l'anterior: els valors del mapa original no són únics, així que l'invers pot perdre informació. Aquella és la limitació natural d'invertir un mapa.

⭐⭐ Exercici 9: TreeMap — freqüència de lletres

› 🔄 Solució

```java
import java.util.Map;
import java.util.TreeMap;

public class FrecuenciaLetras {
    public static void main(String[] args) {
        String texto = "Hola mundo";

        TreeMap<Character, Integer> frec = new TreeMap<>();
        for (char c : texto.toLowerCase().toCharArray()) {
            if (Character.isLetter(c)) {
                frec.put(c, frec.getOrDefault(c, 0) + 1);
            }
        }

        for (Map.Entry<Character, Integer> e : frec.entrySet()) {
            System.out.print(e.getKey() + ": " + e.getValue() + ", ");
        }
        // a: 1, d: 1, h: 1, l: 1, m: 1, n: 1, o: 2, u: 1
    }
}
```

`toLowerCase()` unifica majúscules i minúscules, `Character.isLetter(c)` descarta espais i signes, i `getOrDefault` suma el comptador. La màgia del `TreeMap` és que, en recórrer-lo, les claus surten ordenades alfabèticament sense que facis res.

⭐⭐⭐ Exercici 10: Wildcards — suma i barreja de números

› 🔄 Solució

```java
import java.util.ArrayList;
import java.util.List;

public class Numeros {

    public static double sumar(List<? extends Number> lista) {
        double total = 0.0;
        for (Number n : lista) {
            total += n.doubleValue();
        }
        return total;
    }

       public  static  List<Double>  mezclar(List<?  extends  Number>  a,  List<?
extends Number> b) {
        List<Double> resultado = new ArrayList<>();
        for (Number n : a) {
            resultado.add(n.doubleValue());
        }
        for (Number n : b) {
            resultado.add(n.doubleValue());
        }
        return resultado;
    }

    public static void main(String[] args) {
        List<Integer> enteros = List.of(1, 2, 3);
        List<Double> dobles = List.of(1.5, 2.5);

        System.out.println(sumar(enteros)); // 6.0
        System.out.println(sumar(dobles));  // 4.0
         System.out.println(mezclar(enteros, dobles)); // [1.0, 2.0, 3.0, 1.5,
2.5]
    }
}
```

`List<? extends Number>` accepta qualsevol llista de Number o d'una subclasse. En llegir, cada element és un `Number` i `doubleValue()` el converteix. Passar una `List<String>` seria un error de compilació: `String` no és un `Number`. I ull: `? extends` és de només lectura, així que en `sumar` no pots fer `add` (PECS: Producer Extends).

⭐ ⭐ ⭐ Exercici 11: Sistema de votacions amb mètode genèric

> 🔄 Solució

```
import java.util.HashMap;
import java.util.Map;

public class Votaciones {

    public static <T> T obtenerGanador(Map<T, Integer> votos) {
        T ganador = null;
        int maxVotos = -1;
        for (Map.Entry<T, Integer> e : votos.entrySet()) {
            if (e.getValue() > maxVotos) {
                maxVotos = e.getValue();
                ganador = e.getKey();
            }
        }
        return ganador;
    }

    public static void main(String[] args) {
        HashMap<String, Integer> votos = new HashMap<>();
        votos.put("Ana", 3);
        votos.put("Bob", 5);
        votos.put("Carla", 2);

        System.out.println(obtenerGanador(votos)); // Bob

        HashMap<Integer, Integer> porCategoria = new HashMap<>();
        porCategoria.put(1, 10);
        porCategoria.put(2, 7);
        System.out.println(obtenerGanador(porCategoria)); // 1
    }
}
```

El mètode és genèric (<T>) perquè el tipus de la clau no importa: només necessita recórrer i comparar valors. El patró del màxim acumulat sobre `entrySet()` torna la clau amb més vots. Funciona igual amb claus `String`, `Integer` o qualsevol altre tipus.

⭐⭐⭐ Exercici 12: el type erasure al descobert

> 🔄 Solució

1. **No, és la mateixa classe.** `Caja<String>` i `Caja<Integer>` no generen dues classes en el bytecode: el compilador esborra el paràmetre de tipus i deixa una única `Caja` amb `Object`. Per això no hi ha cap guany de rendiment per «especialitzar»: erasure significa que no es duplica codi.
2. **`Object`.** `getValor()` en el bytecode torna `Object`. El compilador insereix el cast a `String` en el punt d'ús (quan assignes a `String s = caja.getValor();`).
3. La comprovació amb `getClass()`:

```
public class Demo {
    public static void main(String[] args) {
        Caja<String> cajaTexto = new Caja<>("hola");
        Caja<Integer> cajaNumero = new Caja<>(42);

        System.out.println(cajaTexto.getClass());
        System.out.println(cajaNumero.getClass());
        System.out.println(cajaTexto.getClass() == cajaNumero.getClass());  //
true
    }
}
```

Totes dos imprimeixen `class Caja` i la comparació amb == dona `true`: és la MATEIXA classe en runtime. El `<String>` i el `<Integer>` només existeixen en temps de compilació. Aquest és el type erasure: el mag que esborra els tipus quan compiles.

Butlletí U12 — Extres

📝 Butlletí U12 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. Les solucions estan amagades: agota la teva pista abans de mirar-les.

CodeWars

1. Convert a string to an array

Et donen una cadena de text separada per espais. Escriu una funció que la divideixi i torni un array de paraules.

Exemple: `"Robin Singh"` → `["Robin", "Singh"]`, i `"I love arrays they are my favorite"` → `["I", "love", "arrays", "they", "are", "my", "favorite"]`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

> › 💡 **Pista**
>
> El mètode `String.split(" ")` ja et torna un `String[]`. Però com que estem en la unitat de col·leccions: converteix-lo en una `List<String>` amb `Arrays.asList(...)` o guarda les paraules amb un bucle en un `ArrayList<String>`.

> › 🔄 **Solució**
>
> ```
> import java.util.List;
>
> public class Kata {
>     public static String[] stringToArray(String s) {
>         return s.split(" ");
>     }
> }
> ```
>
> O, pensant en col·leccions:

```java
import java.util.ArrayList;
import java.util.List;

public class Kata {
    public static List<String> stringToArray(String s) {
        List<String> palabras = new ArrayList<>();
        for (String palabra : s.split(" ")) {
            palabras.add(palabra);
        }
        return palabras;
    }
}
```

`split(" ")` parteix la cadena pels espais i torna un array. La versió amb `ArrayList` recorre aquell array i construeix la llista: el mateix datum, vist des de la unitat de col·leccions.

---

2. Counting Duplicates

Escriu una funció que torni quants caràcters **distints** (lletres i dígits, sense distingir majúscules) apareixen més d'una vegada en una cadena.

Exemple: `"abcde"` → `0`, `"aabbcde"` → `2` (a i b), `"indivisibility"` → `1` (la i), `"aA11"` → `2` (a i 1).

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Usa un `HashMap<Character, Integer>` per a comptar quantes vegades apareix cada caràcter (amb `toLowerCase()` primer i `getOrDefault` al comptar). Després compta quantes claus tenen un valor major que 1.

> 🔄 **Solució**

```java
import java.util.HashMap;
import java.util.Map;

public class CountingDuplicates {
    public static int duplicateCount(String text) {
        HashMap<Character, Integer> contador = new HashMap<>();
        for (char c : text.toLowerCase().toCharArray()) {
            contador.put(c, contador.getOrDefault(c, 0) + 1);
        }

        int repetidos = 0;
        for (int veces : contador.values()) {
            if (veces > 1) {
                repetidos++;
            }
        }
        return repetidos;
    }
}
```

Dos passades sobre el mateix mapa: primer es compten freqüències amb `getOrDefault` (el patró estrella de la unitat), i després es recorren els **valors** amb `values()` comptant quants superen 1. `toLowerCase()` unifica 'A' i 'a'. Aquest és l'ús de mapa més típic que existeix en les kates.

---

3. Find the unique number

Tens un array de números on tots són iguals excepte un. Troba el número únic.

Exemple: `[ 1, 1, 1, 2, 1, 1 ]` → 2, i `[ 0, 0, 0.55, 0, 0 ]` → `0.55`.

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Compta les aparicions de cada número amb un `HashMap<Double, Integer>`. Després recorre `entrySet()` i torna la clau el valor de la qual sigui 1. Alternativa tramposa: mira els tres primers números per a saber quin és el repetit.

› 🔄 Solució

```java
import java.util.HashMap;
import java.util.Map;

public class Kata {
    public static double findUniq(double[] arr) {
        HashMap<Double, Integer> contador = new HashMap<>();
        for (double d : arr) {
            contador.put(d, contador.getOrDefault(d, 0) + 1);
        }

        for (Map.Entry<Double, Integer> e : contador.entrySet()) {
            if (e.getValue() == 1) {
                return e.getKey();
            }
        }
        return -1;
    }
}
```

El `HashMap` agrupa per valor: tots els repetits cauen en una clau amb comptador alt i l'únic solitari té comptador 1. Recórrer `entrySet()` i tornar la clau amb `getValue() == 1` és directe. La pista alternativa (comparar els tres primers) evita el mapa, però aquesta versió t'entrena en freqüències, que és just el que toca aquesta unitat.

AceptaElReto

4. 152 — Va de modes...

Donat un conjunt de números, la **moda** és el valor (o valors) que més es repeteix. Et demanen calcular la moda de cada distribució.

Entrada: diversos casos de prova. Cada cas comença amb un número que indica quants valors té el conjunt (mai major de 25.000). En la següent línia es donen els valors separats per espais. L'entrada acaba quan el primer número és 0.

Sortida: per a cada cas, la moda (es garanteix que només n'hi ha una).

Exemple: `1 2 2 3 3 3 4 4 4 4 5` → 4, i `1 8 9 6 3 2 1 5 4 7 9 6 3 2 1 4 7` → 1.

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**

Compta cada número amb un `HashMap<Integer, Integer>` (el patró `getOrDefault`). Després recorre `entrySet()` guardant el número amb major comptador. No cal ordenar res: el mapa fa la feina.

> 🔄 **Solució**

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class VaDeModas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int n = sc.nextInt();
            if (n == 0) break;

            HashMap<Integer, Integer> frec = new HashMap<>();
            for (int i = 0; i < n; i++) {
                int valor = sc.nextInt();
                frec.put(valor, frec.getOrDefault(valor, 0) + 1);
            }

            int moda = 0, maxVeces = 0;
            for (Map.Entry<Integer, Integer> e : frec.entrySet()) {
                if (e.getValue() > maxVeces) {
                    maxVeces = e.getValue();
                    moda = e.getKey();
                }
            }
            System.out.println(moda);
        }
        sc.close();
    }
}
```

> El problema clàssic de les freqüències amb mapa: una passada per a comptar (`getOrDefault`), una altra sobre `entrySet()` per a trobar el màxim. Com que es garanteix una única moda, no cal gestionar empats. Aquesta és la plantilla que usaràs en desenes de problemes de concursos.

---

5. 416 — Michael J. Fox i el Pato Donald

En un grup de persones, cal comprovar si **dues persones fan els anys el mateix dia**. Et donen les dates de naixement de cada una en format `dia/mes/any`.

Entrada: diversos casos de prova en dues línies cada un. La primera línia té el nombre de persones del grup; la segona, les seves dates de naixement separades per espais. L'entrada acaba amb un `0`.

Sortida: `SI` si hi ha algun aniversari repetit (mateix dia i mes) i `NO` en cas contrari.

Exemple: `9/6/1961 22/10/1938 31/5/1961 20/4/1964` → `NO`, i `9/6/1961 22/10/1938 31/5/1961 20/4/1964 9/6/1934` → `SI`.

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**
>
> Fica cada data en un `HashSet<String>`. Si `add` torna `false`, aquella data ja hi era: n'hi ha de repetida. Recorda que l'any no compta: talla la data en `dia/mes` amb `split("/")`.

> 🔄 **Solució**

```java
import java.util.HashSet;
import java.util.Scanner;

public class Cumpleanos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int n = sc.nextInt();
            if (n == 0) break;

            HashSet<String> fechas = new HashSet<>();
            boolean repetido = false;

            for (int i = 0; i < n; i++) {
                String fecha = sc.next();
                  String diaMes = fecha.split("/")[0] + "/" + fecha.split("/")
[1];
                if (!fechas.add(diaMes)) {
                    repetido = true;
                }
            }
            System.out.println(repetido ? "SI" : "NO");
        }
        sc.close();
    }
}
```

El truc del `HashSet`: `add()` torna `false` si l'element ja hi era, així que no necessites `contains` per separat. Es guarda només `dia/mes` (sense l'any) perquè dues persones fan els anys el mateix dia encara que hagin nascut en anys distints. Detectar duplicats en O(1) és el superpoder del Set, germà petit del mapa d'aquesta unitat.

🧭 **I si et quedes amb ganes?** Quan domines llistes, conjunts, genèrics i mapes, torna als problemes de unitats anteriors i resol-los guardant les dades en col·leccions: el comptador de notes, el buscador de noms… Tot el que abans era un array paral·lel ara és un mapa. El material no es perd: es reutilitza.

Butlletí U13 — Inicial

📝 Butlletí U13 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, escriu la teva primera `->` i fes que la cinta transportadora del stream deixi de semblar màgia. Les funcions viatgen soles, però el que programa ets tu. Comença suaument, que la fletxa no mossega.

Exercici 1: Completa el codi — la teva primera lambda

Completa les lambdes perquè facin el que diu el comentari:

```
Predicate<Integer> esMayorDeEdad = ______;      // edad >= 18
Function<Integer, Integer> doble = ______;      // x * 2
Consumer<String> imprimir = ______;             // System.out.println(s)
Supplier<String> saludar = ______;              // "¡Hola!"
```

Quin tipus necessita la variable perquè la lambda compili en cada cas?

Exercici 2: Què imprimeix? — l'ordre de la fletxa

```
import java.util.function.Function;

public class Test {
    public static void main(String[] args) {
        Function<Integer, Integer> operacion = x -> x * x + 1;
        System.out.println(operacion.apply(5));
        System.out.println(operacion.apply(0));
    }
}
```

Què imprimeix? Per què el cos `x * x + 1` no necessita `return` ni claus?

Exercici 3: Troba l'error — la lambda mal vestida

```java
import java.util.function.Predicate;

public class Error {
    public static void main(String[] args) {
        Predicate<Integer> esPositivo = x -> x > 0;
        System.out.println(esPositivo.test(-3));
        System.out.println(esPositivo.accept(5));   // ¿Qué ocurre aquí?
    }
}
```

Aquest codi **no compila**. Per què? Quin mètode hauries de cridar en comptes d'`accept`?

Exercici 4: Escriu aquest programa — filtrar parells amb streams

Crea un programa que tingui aquesta llista:

```java
List<Integer> numeros = List.of(10, 15, 22, 33, 40, 55);
```

Usa un stream per a **filtrar els parells**, recollir-los en una llista amb `toList()` i mostrar-la. Quants parells hi ha?

Exercici 5: Què imprimeix? — el pipeline bàsic

```java
import java.util.*;
import java.util.stream.*;

public class Test {
    public static void main(String[] args) {
        List<String> palabras = List.of("sol", "luna", "mar", "cielo", "sol");
        long largas = palabras.stream()
            .filter(p -> p.length() >= 4)
            .distinct()
            .count();
        System.out.println(largas);
    }
}
```

Què imprimeix? Per què `distinct()` canvia el resultat davant de no usar-lo?

Exercici 6: Completa el codi — majúscules amb map

Completa el pipeline perquè transforme cada paraula a majúscules i les arreplegue en una llista:

```java
List<String> palabras = List.of("hola", "java", "mundo");

List<String> mayusculas = palabras.stream()
    .______(String::toUpperCase)
    .______();
```

Quina operació intermèdia i quina terminal necessites? I si usares `Collectors.toList()` en comptes de `toList()`?

Exercici 7: Escriu aquest programa — longitud de cada paraula

Crea un programa que tingui un array de noms i usi un stream amb `map` per a calcular la **longitud de cada nom**, arreplegant-lo en una `List<Integer>`. Després mostra el resultat amb

`forEach(System.out::println)`.

Exercici 8: Troba l'error — la cinta que mai no arranca

```
import java.util.*;
import java.util.stream.*;

public class Error {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);
        Stream<Integer> flujo = numeros.stream()
            .filter(n -> n % 2 == 1)
            .map(n -> n * 10);
        System.out.println("Preparado");
    }
}
```

Aquest programa **compila i executa**, però no imprimeix cap número transformat. Per què? Què li falta al pipeline?

Exercici 9: Completa el codi — un Consumer per a imprimir

Tens una llista de Strings i vols imprimir cada un entre parèntesis, per exemple `(hola)`. Completa el `forEach`:

```
List<String> palabras = List.of("hola", "java");

palabras.stream()
    .forEach(p -> System.out._____("(" + p + ")"));
```

Quin mètode de `System.out` imprimeix sense salt de línia? I quin afegeix el salt?

Butlletí U13 — Inicial Resolt

📝 Butlletí U13 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Completa el codi — la teva primera lambda

> 🔄 Solució

```
Predicate<Integer> esMayorDeEdad = edad -> edad >= 18;
Function<Integer, Integer> doble = x -> x * 2;
Consumer<String> imprimir = s -> System.out.println(s);
Supplier<String> saludar = () -> "¡Hola!";
```

La variable ha de ser de tipus **interfície funcional**: la lambda només compila si la seva firma encaixa amb el mètode abstracte de la interfície. `Predicate` espera `boolean test(Integer)`, `Function` espera `R apply(T)`, `Consumer` espera `void accept(T)` i `Supplier` espera `T get()`. Fixa't en el `Supplier`: sense paràmetres, els parèntesis buits `()` són obligatoris.

Exercici 2: Què imprimeix? — l'ordre de la fletxa

> 🔄 Solució

Imprimeix:

```
26
1
```

- `operacion.apply(5)` → `5 * 5 + 1` = 26.
- `operacion.apply(0)` → `0 * 0 + 1` = 1.

> El cos `x * x + 1` és una **sola expressió**: en les lambdes, un cos d'una expressió torna el seu resultat sense necessitat de `return` ni claus. Les claus i el `return` només calen quan el cos té diverses sentències.

Exercici 3: Troba l'error — la lambda mal vestida

> 🔄 Solució

L'error està en la línia `esPositivo.accept(5)`: `Predicate` no té un mètode `accept`. El seu mètode abstracte és `test(T)`. `accept` pertany a `Consumer`. La línia correcta és:

```
System.out.println(esPositivo.test(5));   // true
```

Cada interfície funcional té EL SEU mètode: `Predicate` usa `test`, `Function` usa `apply`, `Consumer` usa `accept` i `Supplier` usa `get`. Confondre'ls és com demanar una pizza a la peixateria: no és que la pizza no existeixi, és que no està allà.

Exercici 4: Escriu aquest programa — filtrar parells amb streams

> 🔄 Solució

```
import java.util.*;
import java.util.stream.*;

public class Pares {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(10, 15, 22, 33, 40, 55);

        List<Integer> pares = numeros.stream()
            .filter(n -> n % 2 == 0)
            .toList();

        System.out.println(pares);   // [10, 22, 40]
    }
}
```

> El `filter` amb `n -> n % 2 == 0` (un `Predicate<Integer>`) deixa passar només els parells: 10, 22 i 40. `toList()` recull el resultat. Hi ha 3 parells. Fixa't que la llista original no es toca: el stream crea una llista nova.

Exercici 5: Què imprimeix? — el pipeline bàsic

> 🔄 **Solució**

Imprimeix **2**.

- `filter(p -> p.length() >= 4)` deixa passar: `luna` (4) i `cielo` (5). Les dues `sol` tenen 3 lletres i `mar` també (3): es queden fora.
- `distinct()` no canvia res aquí (ja no hi ha repetits entre els que passen).
- `count()` → 2.

Sense `distinct()`, el resultat seria el mateix en aquest cas (2), perquè `sol` ja va ser eliminada pel `filter`. `distinct()` hauria importat si el `filter` deixés passar dos iguals.

Exercici 6: Completa el codi — majúscules amb map

> 🔄 **Solució**

```
List<String> mayusculas = palabras.stream()
    .map(String::toUpperCase)
    .toList();
```

- La intermèdia és **`map`** amb la referència a mètode `String::toUpperCase` (equival a `p -> p.toUpperCase()`).
- La terminal és **`toList()`** (Java 16+).

Amb `Collectors.toList()` seria exactament igual però tornant un `ArrayList` modificable: `palabras.stream().map(String::toUpperCase).collect(Collectors.toList())`. `toList()` torna una llista immutable; per al resultat seria `["HOLA", "JAVA", "MUNDO"]`.

Exercici 7: Escriu aquest programa — longitud de cada paraula

> 🔄 Solució

```java
import java.util.*;
import java.util.stream.*;

public class Longitudes {
    public static void main(String[] args) {
        String[] nombres = {"Ana", "Bob", "Carla", "David"};

        List<Integer> longitudes = Arrays.stream(nombres)
            .map(String::length)
            .collect(Collectors.toList());

        longitudes.forEach(System.out::println);   // 3, 3, 5, 5
    }
}
```

Dos trucs nous: `Arrays.stream(nombres)` converteix l'array (de la U06) en stream, i `map(String::length)` transforma cada `String` en la seva longitud, canviant el tipus del flux a `Stream<Integer>`. El `forEach` amb `System.out::println` és la terminal que imprimeix cada element.

Exercici 8: Troba l'error — la cinta que mai no arranca

> 🔄 Solució

El pipeline no té **operació terminal**: `filter` i `map` són intermèdies (peresoses) i no executen res fins que arriba un `collect`, `count`, `forEach` o similar. El stream es prepara, però la cinta mai no arranca.

Per a veure els números transformats cal tancar l'aixeta. Per exemple:

```java
List<Integer> imparesDoblados = numeros.stream()
    .filter(n -> n % 2 == 1)
    .map(n -> n * 10)
    .toList();
System.out.println(imparesDoblados);   // [10, 30, 50]
```

És l'error més típic de la unitat: muntar la ruta i oblidar que l'autobús necessita arrancar.

Exercici 9: Completa el codi — un Consumer per a imprimir

> 🔄 Solució

```java
palabras.stream()
    .forEach(p -> System.out.print("(" + p + ")"));
```

- `System.out.print(...)` imprimeix **sense salt de línia**: la sortida seria `(hola)(java)`.
- `System.out.println(...)` afegeix el salt de línia al final: `(hola)` i `(java)` en línies separades.

El `forEach` rep un `Consumer<String>`; la lambda `p -> System.out.print("(" + p + ")")` es pot reescriure amb una referència a mètode, encara que aquí el cos ja no és una única crida, així que la lambda és l'opció natural.

Butlletí U13 — Avançat

📝 Butlletí U13 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ ⭐ Exercici 1: Ordenar amb referències a mètode

Tens una llista de noms. Ordena la llista **per longitud** (de menor a major) amb un stream usant `sorted()` i la referència `String::length` combinada amb `Comparator.comparing`. Després mostra-la.

Pista: `sorted(Comparator.comparing(String::length))` ordena per longitud sense tocar la llista original. Si vols l'ordre invers, usa `reversed()`.

⭐ ⭐ Exercici 2: Agrupar paraules per la seva primera lletra

Tens una llista de paraules. Usa `groupingBy` per a agrupar-les per la seva **primera lletra** i mostra el mapa resultant. Després, amb `groupingBy(p -> p.charAt(0), Collectors.counting())`, compta quantes paraules hi ha en cada grup.

Pista: `groupingBy` torna `Map<Character, List<String>>`. El segon argument `counting()` canvia el valor del mapa a `Long`.

⭐ ⭐ Exercici 3: Optional — el que no es deixa enganyar

Implementa un mètode que rebi una `List<Integer>` i torni el **màxim** usant `max(Integer::compareTo)`, gestionant el resultat amb `orElse` perquè torni `-1` si la llista està buida. Prova amb una llista buida i amb una de plena.

Pista: `max` torna `Optional<Integer>`. No usis `get()` a cegues: `orElse(-1)` aterra amb seguretat.

⭐⭐⭐ Exercici 4: El pipeline complet

Tens aquesta llista de números:

```
List<Integer> numeros = List.of(12, 5, 8, 3, 9, 5, 12, 7);
```

Construeix un pipeline que: filtri els **majors o iguals a 5**, els **elevi al quadrat** (`n * n`), **elimini els duplicats**, els **ordeni de major a menor** i es quedi amb els **3 primers**. Recull el resultat en una llista amb `toList()`.

Pista: per a ordenar de major a menor: `sorted(Comparator.reverseOrder())`. Recorda que l'ordre de les estacions importa: `distinct` abans de `sorted` canvia la comptada.

⭐⭐ Exercici 5: De llista a mapa amb `toMap`

Crea una classe senzilla `Alumno` amb `nombre` i `nota`. Amb una llista de 5 alumnes, usa `Collectors.toMap` per a obtenir un `Map<String, Integer>` on la clau sigui el nom i el valor la nota. Com que els noms són únics, usa una funció de fusió per si de cas.

Pista: `Collectors.toMap(Alumno::getNombre, Alumno::getNota, (a, b) -> a)`. La fusió `(a, b) -> a` evita la `IllegalStateException` si es repeteix una clau.

⭐ ⭐ ⭐ Exercici 6: El màxim amb `reduce` i comparador

Implementa el màxim d'una `List<Integer>` de dues formes: amb `reduce` i un acumulador que vagi guardant el major (sense usar `Math::max`), i amb `max`. Què torna cada una? Quina necessita una identitat?

Pista: `reduce(Integer.MIN_VALUE, (a, b) -> a > b ? a : b)` usa `Integer.MIN_VALUE` com a identitat. `max(Integer::compareTo)` torna un `Optional`.

⭐⭐ Exercici 7: Freqüències amb `groupingBy`

Tens un array de paraules amb repetides:

```
String[] palabras = {"hola", "adios", "hola", "java", "hola", "adios"};
```

Usa `Arrays.stream` i `groupingBy(p -> p, Collectors.counting())` per a comptar quantes vegades apareix cada paraula. Mostra el mapa i, després, la paraula que més vegades apareix.

Pista: el mapa és `Map<String, Long>`. Per a la paraula més repetida, recorre `entrySet()` comparant valors, o usa streams de nou amb `max(Map.Entry.comparingByValue())`.

⭐⭐⭐ Exercici 8: Optional i streams, la parella

Tens una llista de noms. Busca, amb streams, el **primer nom que comenci per "J"** usant `filter(...).findFirst()`. Gestiona el `Optional` resultant amb `ifPresent` per a imprimir-lo i amb `orElse` per a mostrar "no hay nadie" si no existeix. Prova amb una llista que tingui "J" i amb una altra que no.

Pista: `findFirst()` torna `Optional<String>`. Amb `ifPresent(System.out::println)` imprimeixes només si hi ha valor; `orElse("no hay nadie")` cobreix l'absència.

⭐⭐⭐ Exercici 9: el stream que es nega a morir

Observa aquest codi i respon **sense executar-lo**:

```
import java.util.*;
import java.util.stream.*;

public class Test {
    public static void main(String[] args) {
        Stream<Integer> flujo = List.of(1, 2, 3).stream();
        long a = flujo.count();
        long b = flujo.count();
        System.out.println(a + " " + b);
    }
}
```

1. Compila?
2. Què ocorre en executar-lo?
3. Com l'arreglaries?

Pista: un stream és d'un sol ús. La primera operació terminal el consumeix. Si vols comptar dues vegades, crea dos streams (`List.of(1, 2, 3).stream()` dues vegades).

> 🔄 **Solució**
>
> 1. **Sí, compila** (l'error és d'execució, no de sintaxi).
> 2. En executar, la segona crida `flujo.count()` llança **`IllegalStateException: stream has already been operated upon or closed`**. El primer `count()` ja va consumir el stream: no es pot reutilitzar.
> 3. Creant un stream nou per a cada comptada:
>
> ```
> long a = List.of(1, 2, 3).stream().count();
> long b = List.of(1, 2, 3).stream().count();
> System.out.println(a + " " + b);   // 3 3
> ```
>
> La regla d'or: un stream és com un bitllet d'autobús d'un sol viatge. Després de baixar-te, el bitllet no serveix.

Butlletí U13 — Avançat Resolt

📝 Butlletí U13 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ ⭐ Exercici 1: Ordenar amb referències a mètode

> 🔄 Solució

```
import java.util.*;
import java.util.stream.*;

public class Orden {
    public static void main(String[] args) {
        List<String> nombres = List.of("Carlos", "Ana", "David", "Bob");

        List<String> porLongitud = nombres.stream()
            .sorted(Comparator.comparing(String::length))
            .toList();

        System.out.println(porLongitud);   // [Ana, Bob, Carlos, David]

        List<String> inverso = nombres.stream()
            .sorted(Comparator.comparing(String::length).reversed())
            .toList();
        System.out.println(inverso);   // [Carlos, David, Ana, Bob]
    }
}
```

`Comparator.comparing(String::length)` construeix un comparador que usa la referència a mètode `String::length` com a "clau d'ordenació". `sorted` no modifica la llista original: torna un stream ordenat. Amb `.reversed()` inverteixes el criteri (els més llargs primer; els empats mantenen l'ordre d'arribada).

⭐ ⭐ Exercici 2: Agrupar paraules per la seva primera lletra

> 🔄 Solució

```
import java.util.*;
import java.util.stream.*;

public class Grupos {
    public static void main(String[] args) {
            List<String> palabras = List.of("hola", "adios", "mar", "mundo",
"luna");

        Map<Character, List<String>> porLetra = palabras.stream()
            .collect(Collectors.groupingBy(p -> p.charAt(0)));

        System.out.println(porLetra);
        // {a=[adios], h=[hola], l=[luna], m=[mar, mundo]}

        Map<Character, Long> conteo = palabras.stream()
                            .collect(Collectors.groupingBy(p  ->  p.charAt(0),
Collectors.counting()));

        System.out.println(conteo);
        // {a=1, h=1, l=1, m=2}
    }
}
```

`groupingBy(p -> p.charAt(0))` agrupa les paraules per la seva primera lletra: cada lletra és una clau i la seva llista de paraules el valor. Amb `Collectors.counting()` com a segon argument (el "collector aigües avall"), el valor passa de `List<String>` a `Long`: quantes paraules cauen en cada grup. És el comptador de freqüències per categoria en una línia.

⭐ ⭐ Exercici 3: Optional — el que no es deixa enganyar

> 🔄 Solució

```java
import java.util.*;
import java.util.stream.*;

public class Maximo {
    public static int maximoSeguro(List<Integer> numeros) {
        return numeros.stream()
            .max(Integer::compareTo)
            .orElse(-1);
    }

    public static void main(String[] args) {
        System.out.println(maximoSeguro(List.of(4, 9, 2, 7)));   // 9
        System.out.println(maximoSeguro(List.of()));             // -1
    }
}
```

`max(Integer::compareTo)` torna un `Optional<Integer>`: si la llista està buida, la capsa està buida. `orElse(-1)` aterra amb seguretat: torna el màxim si hi ha valor i `-1` si no. Usar `get()` aquí hauria llançat `NoSuchElementException` amb la llista buida: l'`orElse` és la xarxa que converteix una excepció en una dada controlada.

⭐⭐⭐ Exercici 4: El pipeline complet

> 🔄 Solució

```java
import java.util.*;
import java.util.stream.*;

public class Pipeline {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(12, 5, 8, 3, 9, 5, 12, 7);

        List<Integer> resultado = numeros.stream()
            .filter(n -> n >= 5)                    // 12, 5, 8, 9, 5, 12, 7
             .map(n -> n * n)                         // 144, 25, 64, 81, 25,
144, 49
            .distinct()                             // 144, 25, 64, 81, 49
            .sorted(Comparator.reverseOrder())      // 144, 81, 64, 49, 25
            .limit(3)                               // 144, 81, 64
            .toList();

        System.out.println(resultado);   // [144, 81, 64]
    }
}
```

El pipeline complet de la unitat: filtrar (el 3 es queda fora), transformar al quadrat, llevar duplicats (el 12 i el 5 repetits desapareixen), ordenar de major a menor amb `Comparator.reverseOrder()` i tallar en 3 amb `limit`. L'ordre importa: `distinct` abans de `sorted` significa que la llista a ordenar ja no té repetits.

⭐⭐ Exercici 5: De llista a mapa amb `toMap`

› 🔄 Solució

```
import java.util.*;
import java.util.stream.*;

class Alumno {
    private String nombre;
    private int nota;

    public Alumno(String nombre, int nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    public String getNombre() { return nombre; }
    public int getNota() { return nota; }
}

public class Mapa {
    public static void main(String[] args) {
        List<Alumno> alumnos = List.of(
            new Alumno("Ana", 8),
            new Alumno("Bob", 6),
            new Alumno("Carla", 9),
            new Alumno("David", 7),
            new Alumno("Eva", 5)
        );

        Map<String, Integer> porNombre = alumnos.stream()
             .collect(Collectors.toMap(Alumno::getNombre, Alumno::getNota, (a,
b) -> a));

          System.out.println(porNombre);    // {Eva=5, Ana=8, Bob=6, Carla=9,
David=7}
    }
}
```

`Collectors.toMap(Alumno::getNombre, Alumno::getNota, (a, b) -> a)` usa referències a mètode per a treure clau (nom) i valor (nota). La funció de fusió `(a, b) -> a` és l'assegurança: si un nom es repetira, dos elements voldrien la mateixa clau i sense fusió Java llançaria `IllegalStateException`. Amb `(a, b) -> a` es queda amb el primer.

⭐ ⭐ ⭐ Exercici 6: El màxim amb `reduce` i comparador

> 🔄 Solució

```
import java.util.*;
import java.util.stream.*;

public class Maximo {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(4, 9, 2, 9, 7);

        int conReduce = numeros.stream()
            .reduce(Integer.MIN_VALUE, (a, b) -> a > b ? a : b);
        System.out.println(conReduce);   // 9

        Optional<Integer> conMax = numeros.stream().max(Integer::compareTo);
        System.out.println(conMax.orElse(-1));   // 9
    }
}
```

- Amb `reduce`, la identitat `Integer.MIN_VALUE` garanteix que el primer element sempre guanyi la comparació (qualsevol `int` és major que el mínim possible). L'acumulador va guardant el major vist.
- Amb `max(Integer::compareTo)` no hi ha identitat: torna un `Optional<Integer>` perquè una llista buida no té màxim. S'aterra amb `orElse(-1)`.

La diferència clau: `reduce` amb identitat torna el valor directe; `max` torna `Optional` i t'obliga a gestionar l'absència.

⭐⭐ Exercici 7: Freqüències amb `groupingBy`

> 🔄 Solució

```java
import java.util.*;
import java.util.stream.*;

public class Frecuencias {
    public static void main(String[] args) {
            String[] palabras = {"hola", "adios", "hola", "java", "hola",
"adios"};

        Map<String, Long> frec = Arrays.stream(palabras)
            .collect(Collectors.groupingBy(p -> p, Collectors.counting()));

        System.out.println(frec);   // {adios=2, hola=3, java=1}

        Map.Entry<String, Long> campeona = frec.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .orElse(null);

           System.out.println("Más repetida: " + campeona.getKey() + " (" +
campeona.getValue() + ")");
    }
}
```

Dos nivells de streams: el primer converteix l'array en flux i agrupa per la paraula mateixa (`p -> p`), comptant amb `counting()`: `hola=3`, `adios=2`, `java=1`. El segon recorre les entrades del mapa (`entrySet()`) i busca el màxim valor amb `max(Map.Entry.comparingByValue())`, que torna `Optional<Map.Entry>` (aterrat amb `orElse(null)`). És l'`entrySet` de la U13 + el `max` dels streams.

⭐⭐⭐ Exercici 8: Optional i streams, la parella

› 🔄 Solució

```java
import java.util.*;
import java.util.stream.*;

public class Busqueda {
    public static void buscarJ(List<String> nombres) {
        nombres.stream()
            .filter(n -> n.startsWith("J"))
            .findFirst()
            .ifPresentOrElse(
                System.out::println,
                () -> System.out.println("no hay nadie")
            );
    }

    public static void main(String[] args) {
        buscarJ(List.of("Ana", "Juan", "Carla"));    // Juan
        buscarJ(List.of("Ana", "Carla"));            // no hay nadie
    }
}
```

`filter(n -> n.startsWith("J")).findFirst()` torna `Optional<String>`: la capsa està plena si algú compleix i buida si no. `ifPresentOrElse` és el mètode que junta els dos camins: el primer argument és el `Consumer` per a quan hi ha valor (`System.out::println`), el segon un `Runnable` per a quan no n'hi ha. També ho podries fer amb `ifPresent` + `orElse`, però `ifPresentOrElse` fa la parella en una sola crida.

⭐⭐⭐ Exercici 9: el stream que es nega a morir

› 🔄 **Solució**

1. **Sí, compila** (l'error és d'execució, no de sintaxi).
2. En executar, la segona crida `flujo.count()` llança **`IllegalStateException: stream has already been operated upon or closed`**. El primer `count()` ja va consumir el stream: no es pot reutilitzar.
3. Creant un stream nou per a cada comptada:

```java
long a = List.of(1, 2, 3).stream().count();
long b = List.of(1, 2, 3).stream().count();
System.out.println(a + " " + b);   // 3 3
```

La regla d'or: un stream és com un bitllet d'autobús d'un sol viatge. Després de baixar-te, el bitllet no serveix.

Butlletí U13 — Extres

📝 Butlletí U13 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. Les solucions estan amagades: esgota la teva pista abans de mirar-les.

CodeWars

1. Enumerable Magic #1 - True for All?

Implementa la funció `all`: rep una llista i un `Predicate`, i torna `true` si el predicat és verdader per a **tots** els elements. Si la llista està buida, torna `true` (res no ha fallat la prova).

Exemple: `all([1, 2, 3, 4, 5], x -> x < 9)` → `true`, i `all([1, 2, 3, 4, 5], x -> x > 9)` → `false`.

- Enunciat en CodeWars
- Dificultat: 8 kyu

> 💡 **Pista**
>
> Un `Stream` té l'operació terminal `allMatch(Predicate)` que fa exactament això: torna `true` si tots els elements compleixen el predicat. I amb la llista buida ja torna `true` per si sola. Una sola línia de stream.

> 🔄 **Solució**

```
import java.util.List;
import java.util.function.Predicate;

public class Solution {
        public static boolean all(List<Integer> list, Predicate<Integer>
predicate) {
        return list.stream().allMatch(predicate);
    }
}
```

> `allMatch` és la terminal dels quantificadors: comprova si tots els elements compleixen el predicat (l'`all` que demana la kata). El `Stream` fa el recorregut per tu, i el cas de la llista buida està resolt per disseny: si no hi ha elements, tècnicament cap no falla, així que torna `true`. És la versió funcional del "recorre i comprova".

---

2. Sorted? yes? no? how?

Reps un array de números. Torna `"yes, ascending"` si està ordenat de menor a major, `"yes, descending"` si està de major a menor, i `"no"` si no està ordenat.

Exemple: `[1, 2]` → `"yes, ascending"`, `[15, 7, 3]` → `"yes, descending"`, `[4, 2, 30]` → `"no"`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> › 💡 **Pista**
>
> Un array ordenat de menor a major compleix que cada element és menor o igual que el següent. Això és un `allMatch` sobre les parelles consecutives: `IntStream.range(0, array.length - 1).allMatch(i -> array[i] <= array[i + 1])`. Fes el mateix per a l'ordre descendent.

> › 🔄 **Solució**

```java
import java.util.stream.IntStream;

public class Kata {
    public static String isSortedAndHow(int[] array) {
        boolean ascendente = IntStream.range(0, array.length - 1)
            .allMatch(i -> array[i] <= array[i + 1]);
        boolean descendente = IntStream.range(0, array.length - 1)
            .allMatch(i -> array[i] >= array[i + 1]);

        if (ascendente) return "yes, ascending";
        if (descendente) return "yes, descending";
        return "no";
    }
}
```

> `IntStream.range(0, array.length - 1)` genera els índexs de 0 a n-2: cada un apunta a l'inici d'una parella `(array[i], array[i + 1])`. `allMatch` comprova que totes les parelles respecten l'ordre. És "ordenat" si totes les parelles van en la mateixa direcció. Un array d'un sol element compleix les dues condicions alhora (no hi ha parelles), i l'`if` ascendent guanya: és correcte, un element està "ascendent".

---

3. Sum of odd numbers

Donat el triangle de números imparells consecutius:

```
             1
          3     5
       7     9    11
   13    15    17    19
21    23    25    27    29
```

Torna la suma de la fila n (començant per 1). **Exemple:** n=1 → 1, n=2 → 8, n=3 → 27.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> 💡 **Pista**
>
> El primer número de la fila n és `n * n - n + 1` (fila 3: 9 - 3 + 1 = 7). Després hi ha n imparells consecutius, separats de 2 en 2. Genera la fila amb `IntStream.range(0, n).map(i -> primer + 2 * i)` i suma-los amb `.sum()`.

> 🔄 **Solució**

```java
import java.util.stream.IntStream;

public class Kata {
    public static int rowSumOddNumbers(int n) {
        int primerImpar = n * n - n + 1;
        return IntStream.range(0, n)
            .map(i -> primerImpar + 2 * i)
            .sum();
    }
}
```

> `IntStream.range(0, n)` genera els `n` números de la fila i `map` els converteix en imparells consecutius des de `primerImpar` (fila 3: 7, 9, 11). `.sum()` és la terminal que suma un `IntStream`. El resultat coincideix amb `n * n * n` (¡la suma de la fila n és sempre el cub de n!), però aquesta versió t'entrena a construir i sumar streams, que és el que toca la unitat.

---

4. Two Oldest Ages

Implementa `twoOldestAges`: rep un array d'edats (sempre amb almenys 2 elements) i torna un array amb les **dos edats més altes**, en el format `[segona més alta, la més alta]`.

Exemple: `[1, 2, 10, 8]` → `[8, 10]`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> › 💡 **Pista**
>
> Ordena l'array de menor a major amb `Arrays.stream(ages).sorted()` i salta't tots menys els dos últims: `skip(ages.length - 2)`. El stream resultant té exactament les dues edats més altes en l'ordre demanat. Recull amb `.toArray()`.

> › 🔄 **Solució**
>
> ```java
> import java.util.Arrays;
>
> public class Kata {
>     public static int[] twoOldestAges(int[] ages) {
>         return Arrays.stream(ages)
>             .sorted()
>             .skip(ages.length - 2)
>             .toArray();
>     }
> }
> ```
>
> `Arrays.stream(ages)` converteix l'array en un `IntStream`, `sorted()` l'ordena de menor a major, i `skip(ages.length - 2)` se salta tots els elements menys els dos últims. En estar ordenats, aquells dos últims són la segona més alta i la més alta, en aquell ordre. `.toArray()` recull el flux de tornada en un array. Quatre operacions per a un problema que a mà demanaria dues variables i un bucle.

AceptaElReto

5. 219 — La loteria de la penya Atlètica

La penya atlètica només compra dècims amb **números parells**. Et donen una llista de dècims de cada administració i has de dir quants en pot comprar.

Entrada: el primer número indica quants casos de prova hi ha. Cada cas són dues línies: el número de dècims `n` i la llista de `n` números (entre 0 i 99.999).

Sortida: per a cada cas, quants dècims són parells.

Exemple: `10` i `1 2 3 4 5 6 7 8 9 10` → `5`.

- Enunciat en AceptaElReto
- Dificultat: ⭐

> 💡 **Pista**
>
> Llegeix els `n` números en un array i compta els parells amb un stream: `Arrays.stream(decimos).filter(d -> d % 2 == 0).count()`. El `filter` amb un `Predicate` i la terminal `count` és la plantilla de "quants compleixen".

> 🔄 **Solució**

```java
import java.util.Arrays;
import java.util.Scanner;

public class Loteria {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int casos = sc.nextInt();

        while (casos-- > 0) {
            int n = sc.nextInt();
            int[] decimos = new int[n];
            for (int i = 0; i < n; i++) {
                decimos[i] = sc.nextInt();
            }

            long comprables = Arrays.stream(decimos)
                .filter(d -> d % 2 == 0)
                .count();

            System.out.println(comprables);
        }
        sc.close();
    }
}
```

L'entrada es llegeix en un array (llegir dins d'una lambda seria un efecte secundari lleig) i el stream fa el treball: `filter(d -> d % 2 == 0)` deixa passar els parells i `count()` els compta. Fixa't que `count()` torna `long`: el problema accepta la sortida encara que la variable sigui `long`. És el mateix patró que `filter(...).count()` de la unitat, aplicat a un problema de concurs real.

---

6. 105 — Vendes

El bar de Javier obre tots els dies menys els dilluns. Apunta la caixa de cada dia de la setmana (dimarts, dimecres, dijous, divendres, dissabte i diumenge). Ha de dir: el **dia de més vendes**, el **dia de menys vendes** (o `EMPATE` si hi ha empat en el màxim o el mínim), i si les **vendes del diumenge superen la mitjana setmanal** (`SI` o `NO`).

Entrada: diversos casos de prova. Cada cas són 6 números (les vendes de dimarts a diumenge). El programa acaba quan el primer número del cas és `-1`.

Sortida: per a cada cas, `DIA_MAX DIA_MIN SI/NO`.

Exemple: `185.50 250.36 163.45 535.20 950.22 450.38` → `SABADO JUEVES SI`.

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**
>
> Guarda els 6 valors en un `double[]`. Amb streams: `Arrays.stream(ventas).max()`, `.min()` i `.average()` et donen les tres dades (totes tornen `OptionalDouble`, aterra amb `orElse(0)`). Per a saber el dia, busca l'índex del màxim/mínim; i compta quants valors coincideixen amb el màxim/mínim: si n'hi ha més d'un, és `EMPATE`.

> 🔄 **Solució**

```
import java.util.*;
import java.util.stream.*;

public class Ventas {
    static final String[] DIAS = {"MARTES", "MIERCOLES", "JUEVES", "VIERNES",
"SABADO", "DOMINGO"};

    static int indiceDe(double[] ventas, double valor) {
        return IntStream.range(0, ventas.length)
            .filter(i -> ventas[i] == valor)
            .findFirst()
            .orElse(-1);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            double[] ventas = new double[6];
            ventas[0] = sc.nextDouble();
            if (ventas[0] == -1) break;
            for (int i = 1; i < 6; i++) {
                ventas[i] = sc.nextDouble();
            }

            double max = Arrays.stream(ventas).max().orElse(0);
            double min = Arrays.stream(ventas).min().orElse(0);
            double media = Arrays.stream(ventas).average().orElse(0);

                    long vecesMax = Arrays.stream(ventas).filter(v -> v ==
max).count();
                    long vecesMin = Arrays.stream(ventas).filter(v -> v ==
min).count();

              String diaMax = vecesMax > 1 ? "EMPATE" : DIAS[indiceDe(ventas,
max)];
              String diaMin = vecesMin > 1 ? "EMPATE" : DIAS[indiceDe(ventas,
min)];
            String domingo = ventas[5] > media ? "SI" : "NO";

            System.out.println(diaMax + " " + diaMin + " " + domingo);
        }
```

```
        sc.close();
    }
}
```

El problema sencer és un desfilament de streams: `max()` i `min()` per als extrems, `average()` per a la mitjana (els tres tornen `OptionalDouble` perquè l'array podria estar buit; `orElse(0)` aterra). El `filter(...).count()` compta quants dies empaten amb el màxim/mínim, i `indiceDe` usa `IntStream.range` + `findFirst` per a localitzar el dia exacte. La mitjana es compara amb les vendes del diumenge (`ventas[5]`, l'últim índex). El `EMPATE` de l'enunciat surt de `vecesMax > 1` / `vecesMin > 1`. Aquest és el problema de la unitat: quasi tot, amb streams.

🧭 **¿I si et quedes amb ganes?** Quan domines lambdes i streams, torna als problemes d'unitats anteriors i reescriu-los amb pipelines: el comptador de freqüències de la U13 amb `groupingBy`, els bucles de la U07 amb `filter + reduce`, l'agenda de la U13 amb `Collectors.toMap`… Tot el que abans era un bucle ara és una declaració. El material no es perd: es reutilitza.

Butlletí U14 — Inicial

📝 Butlletí U14 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, crea el teu primer fitxer amb `FileWriter` i fes que `readLine()` deixi de semblar màgia. El disc és teu i les dades esperen. Comença suaument, que els fitxers mosseguen a poc a poc.

Exercici 1: Troba l'error — IOException sense capturar

```
import java.io.*;

public class Test {
    public static void main(String[] args) {
        FileWriter writer = new FileWriter("salida.txt");
        writer.write("Hola mundo");
        writer.close();
    }
}
```

Aquest codi **no compila**. Per què? Quines dues formes hi ha de solucionar-ho?

Exercici 2: Completa el codi — try-with-resources

Completa el següent programa perquè llegeixi un fitxer i mostri el seu contingut:

```java
import java.io.*;
import java.nio.file.*;

public class Lector {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos.txt");

        try (______ reader = Files.newBufferedReader(ruta)) {  // quin tipus?
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(______);  // què va ací?
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Exercici 3: Escriu aquest programa — guardar ciutats en un fitxer

Crea un array de cadenes amb 5 noms de ciutats. Escriu cada nom en una línia d'un fitxer anomenat `ciudades.txt`, usant `BufferedWriter` i `try-with-resources`. No oblidis el salt de línia.

Pista: `BufferedWriter` té `write(...)` i `newLine()`.

Exercici 4: Troba l'error — File.createNewFile sense comprovar

```java
File f = new File("documento.txt");
f.createNewFile();
FileWriter w = new FileWriter(f);
w.write("Contenido importante");
w.close();
```

Què passa si el fitxer `documento.txt` ja existeix? Què torna `createNewFile()`?

Exercici 5: Què imprimeix? — el comptador de línies

```java
import java.io.*;

public class Test {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

Què imprimeix? L'últim `\n` compta com una línia més?

Exercici 6: Escriu aquest programa — el diari personal

Escriu un programa que afegeixi una línia a `diario.txt` amb la data de avui i el text que l'usuari introduïsca pel teclat. Cada execució ha d'**afegir al final sense esborrar** l'anterior.

Pista: `new FileWriter("diario.txt", true)` i per a la data `java.time.LocalDate.now()`.

Exercici 7: Què imprimeix? — matches() vs find()

```java
import java.util.regex.*;

public class Test {
    public static void main(String[] args) {
        String texto = "abc123";

        System.out.println(texto.matches("\\d+"));
        System.out.println(texto.matches("\\w+"));

        Pattern p = Pattern.compile("\\d+");
        Matcher m = p.matcher(texto);
        while (m.find()) {
            System.out.println("Número: " + m.group());
        }
    }
}
```

Què imprimeix cada línia? Per què `matches("\\d+")` dona `false` però `find()` sí que troba alguna cosa?

Exercici 8: Escriu aquest programa — comptar paraules amb split

Demana a l'usuari una frase pel teclat i mostra quantes paraules té, ignorant els espais dobles.

Pista: `frase.split("\\s+")` troceja per "un o més espais". Compte amb el `.trim()`.

Exercici 9: Escriu aquest programa — comptar dígits amb Matcher

Escriu un programa que compte quants **dígits** hi ha en una frase. Per exemple, `"En 2026 hay 12 unidades"` → 6 dígits (2, 0, 2, 6, 1 i 2).

Pista: compila `Pattern.compile("\\d")`, usa `matcher.find()` en un bucle i porta un comptador.

Butlletí U14 — Inicial Resolt

📝 Butlletí U14 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Troba l'error — IOException sense capturar

> 🔄 **Solució**

`FileWriter` llança `IOException` (per exemple, si no hi ha permís d'escriptura o la carpeta no existeix). El `main` no la declara amb `throws` ni la captura amb `try-catch`, així que el compilador es queixa.

Les **dues formes** de solucionar-ho:

1. Declarar l'excepció en la signatura:

```
public static void main(String[] args) throws IOException {
    FileWriter writer = new FileWriter("salida.txt");
    writer.write("Hola mundo");
    writer.close();
}
```

2. Capturar-la amb `try-catch`:

```
public static void main(String[] args) {
    try (FileWriter writer = new FileWriter("salida.txt")) {
        writer.write("Hola mundo");
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

La versió amb `try-with-resources` és la moderna: tanca el fitxer sol i captura l'error. Les excepcions comprovades de `java.io` no es poden ignorar: o les declares o les captures.

Exercici 2: Completa el codi — try-with-resources

> 🔄 Solució

```
import java.io.*;
import java.nio.file.*;

public class Lector {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos.txt");

        try (BufferedReader reader = Files.newBufferedReader(ruta)) {
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Els buits: `BufferedReader` i `linea`. `Files.newBufferedReader(Path)` et torna directament un `BufferedReader` sense passar per `FileReader` (la forma NIO del punt 5). El bucle llegeix línia a línia fins que `readLine()` torna `null`, i el `try-with-resources` tanca el fitxer en sortir.

Exercici 3: Escriu aquest programa — guardar ciutats en un fitxer

> 🔄 Solució

```java
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class Ciudades {
    public static void main(String[] args) {
          String[] ciudades = {"Valencia", "Madrid", "Barcelona", "Sevilla",
"Bilbao"};

                                        try     (BufferedWriter     bw     =
Files.newBufferedWriter(Paths.get("ciudades.txt"))) {
            for (String ciudad : ciudades) {
                bw.write(ciudad);
                bw.newLine();
            }
            System.out.println("Ciudades guardadas.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

`Files.newBufferedWriter(Path)` et dona un `BufferedWriter` llest per a usar (la forma NIO del punt 5). Es recorre l'array i cada ciutat s'escriu amb el seu `newLine()`. El `try-with-resources` tanca el fitxer sol.

Exercici 4: Troba l'error — File.createNewFile sense comprovar

› 🔄 **Solució**

Si `documento.txt` **ja existeix**, `createNewFile()` torna `false` (no crea res de nou, no llança error) i el `FileWriter` **sobreescriu** el contingut igualment. El codi funciona, però sense que t'assabentes de si el fitxer ja estava.

`createNewFile()` torna:

- `true` si ha creat el fitxer.
- `false` si ja existia.

El patró professional és comprovar-ho:

```
File f = new File("documento.txt");
if (f.createNewFile()) {
    System.out.println("Archivo creado.");
} else {
    System.out.println("El archivo ya existía.");
}
```

Si la carpeta no existeix, `createNewFile()` llança `IOException`, així que també va amb `try-catch` o `throws`.

Exercici 5: Què imprimeix? — el comptador de línies

› 🔄 Solució

Imprimeix **3**.

El fitxer té tres línies: “linea1”, “linea2” i “linea3”. L’últim `\n` **no** crea una quarta línia: quan `readLine()` no troba més text, torna `null` (no una línia buida) i el `while` acaba. El comptador s’incrementa 3 vegades.

Detall: el codi del `while` descarta la línia (`r.readLine()` a seques) perquè només vol comptar. Si a més volguessis el contingut, hauríes de guardar-la en una variable.

Exercici 6: Escriu aquest programa — el diari personal

› 🔄 Solució

```java
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.util.Scanner;

public class Diario {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("¿Qué has aprendido hoy? ");
        String entrada = sc.nextLine();

        try (FileWriter fw = new FileWriter("diario.txt", true)) {
            fw.write(LocalDate.now() + ": " + entrada + "\n");
            System.out.println("Anotado en el diario.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

El segon argument `true` del `FileWriter` activa el mode *append*: afegeix al final sense esborrar l'anterior. `LocalDate.now()` dona la data actual del sistema. Cada execució suma una entrada nova al diari.

Exercici 7: Què imprimeix? — matches() vs find()

› 🔄 **Solució**

Imprimeix:

```
false
true
Número: 123
```

- `"abc123".matches("\\d+")` → **`false`**: `matches()` exigeix que **tot** el string siguin dígits, i hi ha lletres pel mig.
- `"abc123".matches("\\w+")` → **`true`**: lletres i dígits són `\w`, i tot el string ho compleix.
- El `Matcher` amb `find()` busca **subcadenes**: troba "123" dins del text i ho imprimeix.

> La diferència clau: `matches()` = patró complet; `find()` = buscar dins. És l'error més repetit de la unitat.

Exercici 8: Escriu aquest programa — comptar paraules amb split

> 🔄 Solució

```
import java.util.Scanner;

public class ContarPalabras {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Escribe una frase: ");
        String frase = sc.nextLine().trim();

        String[] palabras = frase.split("\\s+");
              System.out.println("La  frase  tiene  "  +  palabras.length  +  "
palabras.");
        sc.close();
    }
}
```

`split("\\s+")` troceja per "un o més espais", així que els espais dobles no conten com a separadors buits. El `.trim()` lleva els espais dels extrems abans de trocejar (si no, una frase que comenci amb espai generaria una paraula buida al principi).

Exercici 9: Escriu aquest programa — comptar dígits amb Matcher

> 🔄 Solució

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ContarDigitos {
    public static void main(String[] args) {
        String frase = "En 2026 hay 12 unidades";

        Pattern patron = Pattern.compile("\\d");
        Matcher matcher = patron.matcher(frase);

        int contador = 0;
        while (matcher.find()) {
            contador++;
        }
        System.out.println("Dígitos: " + contador);
    }
}
```

Sortida: `Dígitos: 6` — el `2026` aporta 4 dígits i el `12` altres 2: `4 + 2 = 6`.

El patró `\\d` troba cada dígit individual i `find()` avança d'un en un mentre hi hagi coincidències. El comptador suma cada troballa. Amb la frase de l'enunciat, `"En 2026 hay 12 unidades"`, el resultat és exactament 6.

Butlletí U14 — Avançat

📝 Butlletí U14 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: Buscador de fitxers per extensió

Crea un programa que demani una ruta de directori i una extensió (ex: `.txt`, `.java`) i llixe **recursivament** tots els fitxers amb aquella extensió. Usa la classe `File` i el seu mètode `listFiles()`.

Pista: si el fitxer és un directori, crida el mètode de nou (recursió). Recorda comprovar `isDirectory()` abans de `listFiles()`.

⭐ Exercici 2: Lector de CSV amb Scanner

Donat un fitxer `datos.csv` amb el següent format (sense capçalera):

```
Ana;25;DAM
Bob;22;DAW
Carlos;30;DAM
```

Usa `Scanner` amb `useDelimiter()` per a llegir el fitxer i mostrar les dades en format de taula alineada amb `printf()`.

Pista: `useDelimiter(";|\\R")` talla per `;` o per salt de línia. Repassa els formats de `printf` en la UO3, punt 7.

⭐⭐ Exercici 3: Filtri de línies per paraula clau

Crea un programa que llegeixi un fitxer de text (`origen.txt`) i escrigui en `destino.txt` només les línies que contenen una paraula clau (demanada a l'usuari). Usa `BufferedReader` i `PrintWriter`. Mostra al final quantes línies van coincidir i quantes es van descartar.

Pista: la comprovació és `linea.contains(palabraClave)`. Porta dos comptadors.

⭐ ⭐ Exercici 4: Separador de línies parells i senars

Crea un programa que llegeixi un fitxer `entrada.txt` i generi dos fitxers:

- `pares.txt` → conté les línies en posició parell (0, 2, 4…).
- `impares.txt` → conté les línies en posició senar (1, 3, 5…).

Usa `try-with-resources` amb **tres** recursos (un `BufferedReader` i dos `PrintWriter`).

Pista: els tres recursos van entre els parèntesis del `try`, separats per `;`. Usa `% 2` sobre el número de línia.

⭐⭐ Exercici 5: Split amb regex — analitzador de frases

Escriu un programa que llegeixi una frase de l'usuari i usi `split()` amb una expressió regular per a:

1. Separar les paraules (ignorant espais, comes, punts i signes).
2. Mostrar quantes paraules hi ha.
3. Mostrar la paraula més llarga.
4. Mostrar les paraules que comencen per vocal.

Exemple: `"Hola, mundo. Esto es Java: ¿mola?"` →

```
Palabras: 6
Más larga: "mundo"
Empiezan por vocal: ["Esto"]
```

Pista: el separador que ignora tot el que no sigui lletra és `"[^a-zA-ZáéíóúüñÑ]+"`. Per a les vocals, comprova la primera lletra amb `matches("[aeiouAEIOUáéíóú]")` o amb un `indexOf` sobre una cadena de vocals.

⭐⭐⭐ Exercici 6: Validador de dades amb regex

Crea un programa que llegeixi un fitxer `datos.txt` on cada línia conté una dada i el seu tipus (separats per `;`):

```
ana@email.com;email
12345678Z;dni
+34 612345678;telefono
91 123 45 67;telefono
esto-no-es-email;email
```

Valida cada línia segons el tipus usant expressions regulars:

- **Correu:** format bàsic `xxx@xxx.xxx`.
- **DNI:** 8 dígits + lletra majúscula (la lletra ha de ser vàlida segons l'algoritme mòdul 23).
- **Telèfon:** opcional `+34` seguit de 9 dígits, amb o sense espais.

Mostra un resum: quants de vàlids, quants d'invàlids, i llixa els invàlids.

Pista: per a cada línia, fes `linea.split(";")`, mira el tipus amb `equals` i aplica el patró corresponent amb `matches()`.

⭐⭐⭐ Exercici 7: Xifrat Cèsar amb fitxers

Crea un programa que llegeixi un fitxer `mensaje.txt`, desplaci cada caràcter **3 posicions** en l'alfabet (xifrat Cèsar) i escrigui el resultat en `mensaje_cifrado.txt`. Després, un altre programa (o el mateix amb una opció) que el descifre. Usa `try-with-resources` i `BufferedReader/PrintWriter`.

Pista: per cada `char`, si és lletra fes `(char) (c + 3)` i compte amb els extrems (la `z` ha de tornar a la `a`: usa `% 26` sobre la posició en l'alfabet).

⭐⭐⭐ Exercici 8: Serialització d'estudiants

Crea una classe `Estudiante` que implementi `Serializable` amb `String nombre`, `int edad` i `double notaMedia`. Crea un programa que guardi un `ArrayList<Estudiante>` en un fitxer

`estudiantes.dat` usant `ObjectOutputStream`. Després, un altre programa (o el mateix amb una opció) que el llegeixi amb `ObjectInputStream` i mostri les dades formatades.

Pista: recorda el `serialVersionUID`. El `readObject()` torna `Object`: fes el casting a `List<Estudiante>` amb calma i comprova que no sigui `null`.

⭐⭐ Exercici 9: El comptador de línies, paraules i caràcters

Crea un programa que llegeixi un fitxer de text i mostri quantes línies, paraules i caràcters té. Usa `BufferedReader` per a llegir.

Pista: cada línia suma 1 al comptador de línies i `linea.length()` al de caràcters; per a les paraules, `linea.split("\\s+").length` (amb compte amb les línies buides).

Repte extra: resol-lo també amb NIO (`Files.readAllLines`) i compara la diferència.

Butlletí U14 — Avançat Resolt

📝 Butlletí U14 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: Buscador de fitxers per extensió

> 🔄 Solució

```
import java.io.File;

public class Buscador {
    public static void buscar(File carpeta, String extension) {
        File[] contenidos = carpeta.listFiles();
        if (contenidos == null) return;

        for (File item : contenidos) {
            if (item.isDirectory()) {
                    buscar(item, extension);          // recursió: dins de la
carpeta
            } else if (item.getName().endsWith(extension)) {
                System.out.println(item.getAbsolutePath());
            }
        }
    }

    public static void main(String[] args) {
        File carpeta = new File("src");
        buscar(carpeta, ".java");
    }
}
```

La recursió és el cor: si el fitxer és una carpeta, el mètode es crida a si mateix amb aquella carpeta; si és un fitxer, comprova l'extensió. L'`if (contenidos == null)` evita el `NullPointerException` si no hi ha permís de lectura. Així es recorre un arbre complet sense bucles niats infinits.

⭐ Exercici 2: Lector de CSV amb Scanner

> 🔄 Solució

```
import java.io.File;
import java.util.Scanner;

public class LeeCSV {
    public static void main(String[] args) throws Exception {
        try (Scanner sc = new Scanner(new File("datos.csv"))) {
            sc.useDelimiter(";|\\R");
            System.out.printf("%-8s %3s %s%n", "Nombre", "Edad", "Ciclo");
            while (sc.hasNext()) {
                String nombre = sc.next();
                int edad = sc.nextInt();
                String ciclo = sc.next();
                System.out.printf("%-8s %3d %s%n", nombre, edad, ciclo);
            }
        }
    }
}
```

`useDelimiter(";|\\R")` talla per `;` o per qualsevol salt de línia: els camps surten nets, un darrere de l'altre. `printf` amb `%-8s` alinea a l'esquerra i `%3d` reserva 3 posicions per a l'edat. Sortida:

```
Nombre   Edad Ciclo
Ana        25 DAM
Bob        22 DAW
Carlos     30 DAM
```

⭐⭐ Exercici 3: Filtri de línies per paraula clau

> 🔄 Solució

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.PrintWriter;
import java.io.FileWriter;
import java.util.Scanner;

public class Filtro {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Palabra clave: ");
        String clave = sc.nextLine();

        int coinciden = 0, descartadas = 0;

                    try (BufferedReader br = new BufferedReader(new
FileReader("origen.txt"));
             PrintWriter pw = new PrintWriter(new FileWriter("destino.txt")))
{

            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.contains(clave)) {
                    pw.println(linea);
                    coinciden++;
                } else {
                    descartadas++;
                }
            }
        }

        System.out.println("Coinciden: " + coinciden);
        System.out.println("Descartadas: " + descartadas);
        sc.close();
    }
}
```

`linea.contains(clave)` busca la paraula dins de la línia (sense regex, que aquí no cal). Els dos `PrintWriter`/`BufferedReader` van en el mateix `try-with-resources` i Java tanca tots dos en sortir. Els comptadors donen el resum final.

⭐ ⭐ Exercici 4: Separador de línies parells i senars

> 🔄 Solució

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.PrintWriter;
import java.io.FileWriter;

public class Separador {
    public static void main(String[] args) throws Exception {
                    try   (BufferedReader   br   =   new   BufferedReader(new
FileReader("entrada.txt"));
             PrintWriter pares = new PrintWriter(new FileWriter("pares.txt"));
                                PrintWriter   impares   =   new   PrintWriter(new
FileWriter("impares.txt"))) {

            String linea;
            int numLinea = 0;
            while ((linea = br.readLine()) != null) {
                if (numLinea % 2 == 0) {
                    pares.println(linea);
                } else {
                    impares.println(linea);
                }
                numLinea++;
            }
        }
        System.out.println("Separados.");
    }
}
```

Tres recursos en el mateix `try-with-resources`, separats per `;`. El `numLinea` compta des de 0, així que la primera línia (posició 0, parell) va a `pares.txt`. En acabar, Java tanca els tres fitxers en ordre invers.

⭐⭐ Exercici 5: Split amb regex — analitzador de frases

› 🔄 Solució

```java
import java.util.Scanner;

public class Analizador {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Frase: ");
        String frase = sc.nextLine();

        String[] palabras = frase.split("[^a-zA-ZàéíóúüñÑ]+");

        System.out.println("Palabras: " + palabras.length);

        String masLarga = "";
        for (String p : palabras) {
            if (p.length() > masLarga.length()) {
                masLarga = p;
            }
        }
        System.out.println("Más larga: \"" + masLarga + "\"");

        System.out.print("Empiezan por vocal: [");
        boolean primera = true;
        for (String p : palabras) {
            if (!p.isEmpty() && p.toLowerCase().charAt(0) == 'a' ||
                p.toLowerCase().charAt(0) == 'e' ||
                p.toLowerCase().charAt(0) == 'i' ||
                p.toLowerCase().charAt(0) == 'o' ||
                p.toLowerCase().charAt(0) == 'u') {
                if (!primera) System.out.print(", ");
                System.out.print("\"" + p + "\"");
                primera = false;
            }
        }
        System.out.println("]");
        sc.close();
    }
}
```

El patró `[^a-zA-ZàéíóúüñÑ]+` talla per tot el que NO sigui lletra: espais, comes, punts i signes desapareixen com a separadors. La condició de les vocals usa `charAt(0)` sobre la

paraula en minúscules (amb el `!p.isEmpty()` per a no esclatar amb cadenes buides). Sortida per a l'exemple:

```
Palabras: 6
Más larga: "mundo"
Empiezan por vocal: ["Esto"]
```

⭐⭐⭐ Exercici 6: Validador de dades amb regex

› 🔄 Solució

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.regex.Pattern;

public class ValidadorDatos {
    private static final Pattern PATRON_EMAIL =
        Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}");
    private static final Pattern PATRON_DNI =
        Pattern.compile("\\d{8}[A-Z]");
    private static final Pattern PATRON_TELEFONO =
        Pattern.compile("(\\+34\\s?)?[679]\\d{8}");

    public static boolean valida(String dato, String tipo) {
        switch (tipo) {
            case "email":    return PATRON_EMAIL.matcher(dato).matches();
                                    case    "dni":                    return
PATRON_DNI.matcher(dato.toUpperCase()).matches();
            case "telefono": return PATRON_TELEFONO.matcher(dato).matches();
            default:         return false;
        }
    }

    public static void main(String[] args) throws Exception {
        int validos = 0, invalidos = 0;

                        try   (BufferedReader   br   =   new   BufferedReader(new
FileReader("datos.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] partes = linea.split(";");
                boolean ok = valida(partes[0], partes[1]);
                if (ok) {
                    validos++;
                } else {
                    invalidos++;
                    System.out.println("Inválido: " + linea);
                }
            }
        }

        System.out.println("Válidos: " + validos);
        System.out.println("Inválidos: " + invalidos);
```

```
    }
}
```

Cada línia es troceja per `;` en dada i tipus, i un `switch` tria el patró. El telèfon `(\\+34\\s?)?` admet el prefix `+34` opcional (amb o sense espai) seguit de 9 dígits que comencen per 6, 7 o 9. El DNI es passa a majúscules per a acceptar la lletra en minúscula. Les regex validen el format: per al DNI de veritat caldria l'algoritme mòdul 23, que aquí es dona per bo.

⭐⭐⭐ Exercici 7: Xifrat Cèsar amb fitxers

› 🔄 Solució

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.PrintWriter;
import java.io.FileWriter;

public class CifradoCesar {
    public static char desplaza(char c, int n) {
        if (c >= 'a' && c <= 'z') {
            return (char) ('a' + (c - 'a' + n) % 26);
        }
        if (c >= 'A' && c <= 'Z') {
            return (char) ('A' + (c - 'A' + n) % 26);
        }
        return c;   // no és lletra: es queda igual
    }

     public static void procesar(String origen, String destino, int n) throws
Exception {
        try (BufferedReader br = new BufferedReader(new FileReader(origen));
             PrintWriter pw = new PrintWriter(new FileWriter(destino))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                StringBuilder sb = new StringBuilder();
                for (char c : linea.toCharArray()) {
                    sb.append(desplaza(c, n));
                }
                pw.println(sb);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        procesar("mensaje.txt", "mensaje_cifrado.txt", 3);   // xifrar
        procesar("mensaje_cifrado.txt", "descifrado.txt", -3); // descifrar
    }
}
```

El truc del `% 26`: cada lletra es converteix a la seva posició en l'alfabet (`c - 'a'`), es desplaça `n` i es fa mòdul 26 perquè la `z` torni a la `a`. Descifrar és el mateix amb `n = -3`. Els caràcters que no són lletres (espais, signes) es queden intactes, que és el que fa un Cèsar clàssic.

⭐⭐⭐ Exercici 8: Serialització d'estudiants

> 🔄 Solució

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

class Estudiante implements Serializable {
    private static final long serialVersionUID = 1L;
    String nombre;
    int edad;
    double notaMedia;

    Estudiante(String n, int e, double m) {
        this.nombre = n;
        this.edad = e;
        this.notaMedia = m;
    }
}

public class GuardaEstudiantes {
    public static void main(String[] args) throws Exception {
        List<Estudiante> equipo = new ArrayList<>();
        equipo.add(new Estudiante("Ana", 20, 8.5));
        equipo.add(new Estudiante("Bob", 22, 6.0));
        equipo.add(new Estudiante("Carla", 19, 9.2));

                try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("estudiantes.dat"))) {
            oos.writeObject(equipo);
        }

                  try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("estudiantes.dat"))) {
                        List<Estudiante> recuperados = (List<Estudiante>)
ois.readObject();
            for (Estudiante e : recuperados) {
                  System.out.printf("%-6s %2d años - nota: %.1f%n", e.nombre,
e.edad, e.notaMedia);
            }
        }
    }
}
```

La classe `Estudiante` implementa `Serializable` amb el seu `serialVersionUID` fix perquè els fitxers sobrevisquin a petits canvis. El `writeObject` guarda la llista sencera d'una vegada i `readObject` la reconstruïx amb un casting a `List<Estudiante>`. Com que `ArrayList` i `Estudiante` són serialitzables, tot el paquet es congela i descongela en dues línies.

⭐⭐ Exercici 9: El comptador de línies, paraules i caràcters

> 🔄 Solució

```
import java.io.BufferedReader;
import java.io.FileReader;

public class Contador {
    public static void main(String[] args) throws Exception {
        int lineas = 0, palabras = 0, caracteres = 0;

                    try (BufferedReader br = new BufferedReader(new
FileReader("texto.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                lineas++;
                caracteres += linea.length();
                if (!linea.trim().isEmpty()) {
                    palabras += linea.split("\\s+").length;
                }
            }
        }

        System.out.println("Líneas: " + lineas);
        System.out.println("Palabras: " + palabras);
        System.out.println("Caracteres: " + caracteres);
    }
}
```

Cada línia suma 1 a les línies i el seu `length()` als caràcters. Per a les paraules, `split("\\s+")` troceja pels espais; la comprovació `!linea.trim().isEmpty()` evita que una línia en blanc es compte com una "paraula buida".

> Amb NIO seria encara més curt: `Files.readAllLines(ruta)` i un `for` sobre la llista, sense `close()` manual. Compara-ho amb el punt 5 del temari: menys codi, mateixa lògica.

Butlletí U14 — Extres

📝 Butlletí U14 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. Les solucions estan amagades: esgota la teva pista abans de mirar-les.

CodeWars

1. Regex validate PIN code

Crea una funció que validi un PIN: un `String` que és vàlid només si té **4 o 6 dígits exactes**.

Exemple: `"1234"` → `true`, `"12345"` → `false`, `"a234"` → `false`, `"123456"` → `true`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> 💡 **Pista**
>
> És la kata més regex de la unitat: un sol `matches()` amb el patró `\\d{4}|\\d{6}` ho resol. Recorda: `matches()` exigeix que tot el string complica amb el patró, que és just el que demana un PIN.

> 🔄 **Solució**

```
public class Kata {
    public static boolean validatePin(String pin) {
        return pin.matches("\\d{4}|\\d{6}");
    }
}
```

`\\d{4}` és “exactament 4 dígits” i `\\d{6}` “exactament 6”. El `|` els uneix: o 4 o 6. `matches()` comprova el string sencer, així que `"12345a"` no passa encara que tingui 5 dígits: hi ha una lletra al final.

---

2. Two to One

Et donen dos strings en minúscules amb lletres de la `a` a la `z`. Torna un únic string amb les lletres **distintes de tots dos, ordenades alfabèticament**.

Exemple: `"xyaabbbccccdefww"` i `"xxxxyyyyabklmopq"` → `"abcdefklmopqwxy"`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> 💡 **Pista**
>
> Concatena els dos strings en un. Després recorre les lletres de la `a` a la `z` i, si el string concatenat conté aquella lletra (`indexOf`), afegeix-la al resultat. L'ordre alfabètic eix gratis perquè recorres l'alfabet en ordre.

> 🔄 **Solució**

```java
public class Kata {
    public static String longest(String s1, String s2) {
        String unidos = s1 + s2;
        StringBuilder resultado = new StringBuilder();

        for (char c = 'a'; c <= 'z'; c++) {
            if (unidos.indexOf(c) != -1) {
                resultado.append(c);
            }
        }
        return resultado.toString();
    }
}
```

La clau és invertir la pregunta: en comptes de filtrar els caràcters dels strings, recorres l'alfabet i preguntes "està aquesta lletra en la unió?". Amb `indexOf(c) != -1` saps si apareix. El `StringBuilder` acumula el resultat i l'ordre alfabètic és automàtic perquè la `a` es processa abans que la `b`.

---

3. Categorize New Member

El club de tenis classifica els seus nous socis. Un soci és **"Senior"** si té 55 anys o més **i** un hàndicap major que 7; si no, és **"Open"**. Et donen un array de parells `[edat, hándicap]` i has de tornar un array amb la categoria de cadascun.

Exemple: `[[45, 12], [55, 21], [19, -2], [104, 20]]` → `["Open", "Senior", "Open", "Senior"]`.

- Enunciat en CodeWars
- Dificultat: 7 kyu

> 💡 **Pista**
>
> Simple lògica booleana: `edad >= 55 && hándicap > 7`. Recorre l'array de parells i munta el resultat amb un ternari.

> 🔄 **Solució**

```
public class Kata {
    public static String[] openOrSenior(int[][] data) {
        String[] resultado = new String[data.length];

        for (int i = 0; i < data.length; i++) {
            resultado[i] = (data[i][0] >= 55 && data[i][1] > 7)
                ? "Senior" : "Open";
        }
        return resultado;
    }
}
```

Cada parell `data[i]` té l'edat en `[0]` i l'hàndicap en `[1]`. El ternari decideix la categoria en una línia i guarda el resultat en la seva posició. És la lògica booleana pura de l'enunciat: les dues condicions amb `&&`.

---

4. Primes in numbers

Donat un número positiu `n`, torna la seva descomposició en factors primers amb el format `"(p1**exp1)(p2**exp2)"`. Si l'exponent és 1, s'escriu només `"(p)"`.

Exemple: `86240` → `"(2**5)(5)(7**2)(11)"`.

- Enunciat en CodeWars
- Dificultat: 5 kyu

> 💡 **Pista**

Divideix per 2, després pels senars des de 3 fins a l'arrel quadrada de n (que es va reduint en dividir). Compta quantes vegades divideix cada divisor (l'exponent) i munta el string amb un `StringBuilder`.

> 🔄 Solució

```
public class Kata {
    public static String factors(int n) {
        StringBuilder sb = new StringBuilder();

        for (int i = 2; i * i <= n; i++) {
            int veces = 0;
            while (n % i == 0) {
                n /= i;
                veces++;
            }
            if (veces == 1) {
                sb.append("(").append(i).append(")");
            } else if (veces > 1) {
                                                        sb.append("
(").append(i).append("**").append(veces).append(")");
            }
        }
        if (n > 1) {
            sb.append("(").append(n).append(")");
        }
        return sb.toString();
    }
}
```

El `while` intern divideix mentre el divisor encaixi i compta les repeticions: aquell és l'exponent. El bucle només arriba a l'arrel de n (i n es va reduint), així que l'`if (n > 1)` final recull l'últim factor primer que queda. El `StringBuilder` evita concatenar cadenes en cada vegada, que seria lent.

AceptaElReto

5. 108 — Formigues

Sobre una barra de longitud `L` cm hi ha `n` formigues. Cada formiga es mou a 1 cm/s cap a l'extrem que li toqui (l'enunciat et dona les posicions). Quan dues formigues es creuen, totes dues canvien de sentit. Calcula, per a cada cas de prova, el **temps mínim** i el **temps màxim** que tarden a caure totes de la barra.

Entrada: diversos casos de prova. Cada cas comença amb `L` i `n`, seguit de les `n` posicions de les formigues.

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐

> **💡 Pista**
>
> Quan dues formigues es creuen i canvien de sentit, és **com si s'ignoraren**: la posició de les formigues és indistinguible. Així que cada formiga cau en `min(pos, L - pos)` o `max(pos, L - pos)`. El temps mínim és el major dels mínims; el màxim, el major dels màxims.

> **🔄 Solució**

```java
import java.util.Scanner;

public class Hormigas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (sc.hasNextInt()) {
            int L = sc.nextInt();
            int n = sc.nextInt();
            int minTiempo = 0, maxTiempo = 0;

            for (int i = 0; i < n; i++) {
                int pos = sc.nextInt();
                int haciaIzq = pos;
                int haciaDer = L - pos;
                minTiempo = Math.max(minTiempo, Math.min(haciaIzq, haciaDer));
                maxTiempo = Math.max(maxTiempo, Math.max(haciaIzq, haciaDer));
            }

            System.out.println(minTiempo + " " + maxTiempo);
        }
        sc.close();
    }
}
```

El truc conceptual: quan dues formigues es creuen, totes dues giren, però com que són indistinguibles, l'efecte és el mateix que si passaren de llarg. Cada formiga tarda `pos` o `L - pos` segons el costat cap on vagi. Per al mínim total, cada formiga tria el seu costat més pròxim i el temps és el major d'aquells mínims; per al màxim, el pitjor cas.

---

6. 140 — Suma de dígits

Donat un número positiu, calcula la suma dels seus dígits. L'entrada és una seqüència de números que acaba amb `-1`.

Exemple: `123` → `123: 6`, `55` → `55: 10`, `1000` → `1000: 1`.

- Enunciat en AceptaElReto
- Dificultat: ⭐

> 💡 **Pista**

Pots extreure els dígits amb `% 10` i `/ 10` en un bucle mentre el número sigui major que 0. O, més en l'esperit d'aquesta unitat, convertir el número a `String` i recórrer-lo amb un `charAt`, sumant `c - '0'`.

> 🔄 **Solució**

```
import java.util.Scanner;

public class SumaDigitos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            int n = sc.nextInt();
            if (n == -1) break;

            int suma = 0;
            String texto = String.valueOf(n);
            for (int i = 0; i < texto.length(); i++) {
                suma += texto.charAt(i) - '0';
            }

            System.out.println(n + ": " + suma);
        }
        sc.close();
    }
}
```

Convertir el número a `String` permet tractar els dígits com a caràcters: `charAt(i) - '0'` converteix el caràcter `'3'` en el número `3` (perquè els codis ASCII dels dígits són consecutius). Sense operacions aritmètiques de `%` i `/`: la versió "d'unitat de regex" d'un clàssic.

🧭 **I si et quedes amb ganes?** Quan domines els fitxers i les regex, torna als problemes d'unitats anteriors i resol-los llegint les dades des d'un fitxer en comptes de demanar-les pel teclat, o validant l'entrada amb un `matches()`. El material no es perd: es reutilitza.

Butlletí U15 — Inicial

📝 Butlletí U15 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, afegeix la dependència de SQLite i fes que el teu primer `SELECT` deixi de semblar màgia. Connectar és el 90% de la batalla; la resta és qüestió de pràctica. Comença suaument, que les bases de dades mosseguen a poc a poc.

Exercici 1: Què necessitos per a usar JDBC?

Respon breument:

1. Quina dependència Maven (grup i artefacte) necessites per a connectar Java amb SQLite?
2. Quina classe de Java proporciona el mètode `getConnection()`?
3. Quina interfície representa la connexió oberta a la base de dades?
4. Quina excepció *checked* has de gestionar sempre en treballar amb JDBC?

Exercici 2: Completa el codi — la connexió

Completa els tipus que falten:

```
String url = "jdbc:sqlite:instituto.db";

String sql = "SELECT * FROM alumnos";

try (______ con = DriverManager.getConnection(url);
     ______ stmt = con.createStatement();
     ______ rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }

} catch (______ e) {  // quina excepció?
    System.err.println("Error: " + e.getMessage());
}
```

Quines tres interfícies van en els buits? I l'excepció?

Exercici 3: Què imprimeix? — ResultSet buit

```
try (Connection con = DriverManager.getConnection(url);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos WHERE id = 9999"))
{

    if (rs.next()) {
        System.out.println("Encontrado: " + rs.getString("nombre"));
    } else {
        System.out.println("No encontrado");
    }
}
```

Si no hi ha cap alumne amb `id = 9999`, què imprimeix? Què torna `rs.next()` la primera vegada que es crida?

Exercici 4: Troba l'error — SQLException sense gestionar

```
public class Test {
    public static void main(String[] args) {
                                                Connection con =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

Aquest codi **no compila**. Per què? Quines dues coses falten perquè funcioni?

Exercici 5: Escriu aquest programa — la primera connexió

Crea una classe `TestConnexio` que:

1. Es connecte a una base de dades SQLite `test.db`.
2. Creï una taula `alumnes(id INTEGER PRIMARY KEY, nom TEXT, nota REAL)`.
3. Insereixi 3 alumnes.
4. Mostri «Connexió i taula creades» si tot ha anat bé.

Pista: usa `try-with-resources`, un `Statement` per al `CREATE TABLE` i `executeUpdate()` per a inserir. SQLite crea el fitxer sol.

Exercici 6: Què imprimeix? — executeQuery en UPDATE

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("UPDATE alumnos SET nota = 10 WHERE id = 1");
```

Què passa en executar aquesta línia? Per què no has d'usar `executeQuery()` per a un `UPDATE`?

Exercici 7: Completa el codi — INSERT amb PreparedStatement

Completa els buits per a inserir un alumne de forma segura:

```
String sql = "INSERT INTO alumnos (nombre, nota) VALUES (?, ?)";

try (Connection con = DriverManager.getConnection(url);
     PreparedStatement pstmt = con.prepareStatement(sql)) {

    pstmt._____(1, "Ana");
    pstmt._____(2, 8.5);

    int filas = pstmt._____();
    System.out.println("Inserides " + filas + " fila/es");
}
```

Quins mètodes van en els buits? En quin número comencen els índexs dels ??

Exercici 8: Escriu aquest programa — llistar amb try-with-resources

Escriu un mètode `llistarAlumnes()` que:

1. Es connecte a la base de dades amb `try-with-resources`.
2. Executi `SELECT * FROM alumnos`.
3. Mostri cada alumne amb `printf("%d - %s (%.2f)", id, nombre, nota)`.
4. Gestioni `SQLException` mostrant `e.getMessage()`.

Pista: `Connection`, `Statement` i `ResultSet` es tanquen sols dins del `try`. Recorre amb `while (rs.next())`.

Exercici 9: Troba l'error — índexs del PreparedStatement

```
String sql = "INSERT INTO alumnos (nombre, nota, curso) VALUES (?, ?, ?)";
PreparedStatement pstmt = con.prepareStatement(sql);
pstmt.setString(0, "Ana");    // índex correcte?
pstmt.setDouble(1, 8.5);      // índex correcte?
pstmt.setString(2, "DAM");    // índex correcte?
pstmt.executeUpdate();
```

Quins índexs són correctes per als `?` d'un `PreparedStatement`? En quin número comencen i què passa si uses `0`?

Butlletí U15 — Inicial Resolt
📝 Butlletí U15 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Què necessitos per a usar JDBC?

> 🔄 Solució

1. La dependència **`org.xerial:sqlite-jdbc`**. Al `pom.xml`:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.45.1.0</version>
</dependency>
```

2. **`DriverManager`** — el seu mètode estàtic `getConnection()` estableix la connexió.

3. **`Connection`** — la interfície del paquet `java.sql` que representa la connexió oberta.

4. **`SQLException`** — és *checked*: el compilador t'obliga a capturar-la o declarar-la.

Exercici 2: Completa el codi — la connexió

> 🔄 Solució

```
String url = "jdbc:sqlite:instituto.db";

String sql = "SELECT * FROM alumnos";

try (Connection con = DriverManager.getConnection(url);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }

} catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
}
```

Els tres tipus són **`Connection`**, **`Statement`** i **`ResultSet`**, tots del paquet `java.sql`. L'excepció és **`SQLException`**. Fixa't en l'ordre d'obertura: Connection → Statement → ResultSet; `try-with-resources` els tanca en ordre invers.

Exercici 3: Què imprimeix? — ResultSet buit

> 🔄 Solució

Imprimeix **`No encontrado`**.

`rs.next()` torna **`false`** la primera vegada si no hi ha files. El cursor del `ResultSet` comença *abans* de la primera fila, així que amb una consulta sense resultats, el primer `next()` ja es troba amb el buit i torna `false`, saltant a l'`else`. No és un error: una consulta sense resultats torna un `ResultSet` buit, no una excepció.

Exercici 4: Troba l'error — SQLException sense gestionar

> 🔄 Solució

No compila perquè **`SQLException` és checked**: `DriverManager.getConnection()`, `con.createStatement()` i `stmt.executeQuery()` la llancen, i el codi no la captura ni la declara.

Falten dues coses:

1. Embolcar el codi en un `try { ... } catch (SQLException e) { ... }`.
2. Tancar els recursos (millor, amb `try-with-resources`).

```
public class Test {
    public static void main(String[] args) {
        String sql = "SELECT * FROM alumnos";
                                            try     (Connection     con     =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                System.out.println(rs.getString("nombre"));
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Exercici 5: Escriu aquest programa — la primera connexió

› 🔄 Solució

```java
import java.sql.*;

public class TestConnexio {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:test.db";

        String crearTaula = "CREATE TABLE IF NOT EXISTS alumnos (" +
            "id INTEGER PRIMARY KEY, nombre TEXT, nota REAL)";

        try (Connection con = DriverManager.getConnection(url);
             Statement stmt = con.createStatement()) {

            stmt.executeUpdate(crearTaula);
                stmt.executeUpdate("INSERT INTO alumnos (nombre, nota) VALUES
('Ana', 7.5)");
                stmt.executeUpdate("INSERT INTO alumnos (nombre, nota) VALUES
('Luis', 9.0)");
                stmt.executeUpdate("INSERT INTO alumnos (nombre, nota) VALUES
('Sara', 6.5)");

            System.out.println("Connexió i taula creades");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

`CREATE TABLE` i `INSERT` són operacions que no tornen files, així que s'executen amb `executeUpdate()`. L'`IF NOT EXISTS` evita errors si tornesses a executar el programa. I sí: `test.db` es crea sol amb la primera connexió.

Exercici 6: Què imprimeix? — executeQuery en UPDATE

› 🔄 Solució

Llança una **`SQLException`**.

`executeQuery()` és només per a consultes que **tornen files** (SELECT). Un `UPDATE` no torna un `ResultSet`; torna el nombre de files afectades, i això és feina de `executeUpdate()`. Barrejar-los dona `SQLException` sempre. És com ficar una forquilla al microones: no hi ha volta enrere.

Regla mnemotècnica: **esperes dades de tornada? → `executeQuery()`. Només vols saber quantes files s'han modificat? → `executeUpdate()`.**

Exercici 7: Completa el codi — INSERT amb PreparedStatement

› 🔄 Solució

```
String sql = "INSERT INTO alumnos (nombre, nota) VALUES (?, ?)";

try (Connection con = DriverManager.getConnection(url);
     PreparedStatement pstmt = con.prepareStatement(sql)) {

    pstmt.setString(1, "Ana");
    pstmt.setDouble(2, 8.5);

    int filas = pstmt.executeUpdate();
    System.out.println("Inserides " + filas + " fila/es");
}
```

Els buits: **`setString`**, **`setDouble`** i **`executeUpdate`**. Els índexs dels `?` comencen en **1** (no en 0, com els arrays). `setString(1, ...)` ompli el primer `?`, `setDouble(2, ...)` el segon. `executeUpdate()` torna les files afectades: si és `1`, tot bé.

Exercici 8: Escriu aquest programa — llistar amb try-with-resources

› 🔄 Solució

```java
import java.sql.*;

public class LlistarAlumnes {
    public static void llistarAlumnes() {
        String url = "jdbc:sqlite:instituto.db";
        String sql = "SELECT * FROM alumnos";

        try (Connection con = DriverManager.getConnection(url);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                System.out.printf("%d - %s (%.2f)%n",
                    rs.getInt("id"),
                    rs.getString("nombre"),
                    rs.getDouble("nota"));
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        llistarAlumnes();
    }
}
```

`try-with-resources` tanca `Connection`, `Statement` i `ResultSet` automàticament en ordre invers. El `printf` formata la sortida en columnes, i `getInt`/`getString`/`getDouble` llegeix cada columna per nom. La `SQLException` es captura i mostra el seu missatge.

Exercici 9: Troba l'error — índexs del PreparedStatement

› 🔄 Solució

Els índexs dels ? comencen en **1**:

```
pstmt.setString(1, "Ana");    // primer ?
pstmt.setDouble(2, 8.5);      // segon ?
pstmt.setString(3, "DAM");    // tercer ?
```

`setString(0, "Ana")` és **incorrecte**: llança una `SQLException` perquè no existeix cap `?` amb índex 0. Els placeholders són posicionals i el primer és l'1. És la trampa clàssica de qui ve dels arrays, on els índexs comencen en 0.

Butlletí U15 — Avançat

📝 Butlletí U15 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix a mirar-la).

⭐ Exercici 1: Connexió des de fitxer de propietats

Crea un fitxer `db.properties` amb les dades de connexió:

```
url=jdbc:sqlite:instituto.db
```

Escriu un programa que llegeixi aquest fitxer usant la classe `Properties` i estableixi la connexió. Si el fitxer no existeix o falta la propietat `url`, mostra un missatge d'error clar.

Pista: carrega el fitxer amb `props.load(Files.newInputStream(Path.of("db.properties")))` i usa `props.getProperty("url")`. `load` llança una `IOException` (la vas veure a la U14 amb els fitxers) a més de la `SQLException`.

⭐ Exercici 2: INSERT amb clau autogenerada

Insereix un nou alumne a la taula `alumnos` i **recupera l'ID** que la base de dades li ha assignat automàticament (és un `AUTOINCREMENT`). Usa `PreparedStatement` amb `Statement.RETURN_GENERATED_KEYS` i el mètode `getGeneratedKeys()`.

Pista:

```
PreparedStatement pstmt = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS);
pstmt.executeUpdate();
ResultSet claus = pstmt.getGeneratedKeys();
if (claus.next()) {
    int id = claus.getInt(1);
}
```

El `getGeneratedKeys()` torna un `ResultSet` amb la clau que acaba de generar la base de dades. Es llegeix amb `next()` i `getInt(1)`.

⭐ Exercici 3: UPDATE condicional

Actualitza el curs de tots els alumnes que tinguin una edat superior a un valor donat. Per exemple:

```
Quina edat mínima? 25
Quin nou curs? DAM2
```

Tots els alumnes majors de 25 anys passen al curs «DAM2». Mostra quantes files s'han actualitzat.

Pista: `UPDATE alumnos SET curso = ? WHERE edad > ?` amb `setInt` i `setString`. `executeUpdate()` et torna el nombre de files afectades: és el teu millor amic per a confirmar que alguna cosa ha canviat.

⭐ ⭐ Exercici 4: INNER JOIN amb PreparedStatement

Donada una taula `matriculas` amb `id_alumno`, `asignatura`, `nota`, escriu un programa que rebi un nom d'alumne i mostri totes les seves assignatures i notes. Usa un `INNER JOIN` entre `alumnos` i `matriculas`.

Exemple de sortida:

```
Alumne: Ana García
  Matemàtiques: 8.5
  Programació: 9.0
  Bases de Dades: 7.5
```

Pista: el `?` va a la part del nom: `SELECT a.nombre, m.asignatura, m.nota FROM alumnos a INNER JOIN matriculas m ON m.id_alumno = a.id WHERE a.nombre = ?`. El `JOIN` relaciona les dues taules en una sola consulta: ni un bucle, ni consultes dins de bucles.

⭐⭐ Exercici 5: Cerca amb LIKE

Implementa una cerca d'alumnes per nom usant `LIKE` i `PreparedStatement`. L'usuari escriu una part del nom i es mostren tots els que coincideixin. Si no hi ha resultats, mostra «Sense resultats».

Pista: `SELECT * FROM alumnos WHERE nombre LIKE ?` amb `pstmt.setString(1, "%" + text + "%")`. Els `%` són comodins i van dins del **valor**, no en l'SQL. El `%text%` busca el text en qualsevol posició.

⭐⭐ Exercici 6: Dates en JDBC

Afegeix una columna `fecha_nacimiento DATE` a la taula `alumnos` (assumeix que ja existeix). Crea un programa que:

1. Demani nom, edat, curs i data de naixement (format `YYYY-MM-DD`).
2. Insereixi l'alumne usant `PreparedStatement` amb `java.sql.Date.valueOf()`.
3. Lliste tots els alumnes mostrant també la seva data de naixement.

Pista: `Date.valueOf("2000-03-15")` converteix el text en `java.sql.Date` (alerta: és `java.sql.Date`, no `java.util.Date`!). Per a llegir-la, `rs.getDate("fecha_nacimiento")`. Recorda comprovar el valor que torna `executeUpdate()`.

⭐⭐ Exercici 7: Batch INSERT — 100 alumnes de prova

Crea un programa que insereixi **100 alumnes de prova** a la taula `alumnos` usant lots (batch). Els noms poden ser genèrics: `Alumno1`, `Alumno2`, etc.

Usa `addBatch()` i `executeBatch()` de `PreparedStatement`. Mesura el temps que tarda amb `System.currentTimeMillis()`.

Pista: al bucle, fas `addBatch()` a cada vegada i un sol `executeBatch()` al final (o cada 50). Per a mesurar: `long inicio = System.currentTimeMillis(); … long fin = System.currentTimeMillis();` i restes. Compara mentalment amb 100 `executeUpdate()` solts.

⭐⭐⭐ Exercici 8: El patró DAO

Implementa el patró **Data Access Object (DAO)** per a la taula `alumnos`. Crea les següents classes:

1. `Alumno` — classe model amb `id`, `nombre`, `edad`, `curso`.
2. `AlumnoDAO` — interfície amb mètodes: `listar()`, `buscarPorId(int id)`, `buscarPorNombre(String nombre)`, `insertar(Alumno a)`, `actualizar(Alumno a)`, `eliminar(int id)`.
3. `AlumnoDAOImpl` — implementació concreta amb JDBC i SQLite.
4. `Main` — programa amb menú que usi el DAO.

Pista: la URL (`jdbc:sqlite:instituto.db`) és una constant privada de la implementació. Cada mètode obre la seva pròpia connexió amb `try-with-resources`. El `Main` només parla amb la interfície `AlumnoDAO`; l'SQL no li importa.

⭐⭐⭐ Exercici 9: Transacció bancària atòmica

Simula una transferència entre dos comptes en una taula `cuentas(id, titular, saldo)`. La transferència ha de ser **atòmica**: treu 100 € d'un compte, posa'ls en l'altre, i si falla qualsevol pas, fes `rollback()` perquè el sistema no quedi a mitges.

Exemple de sortida:

```
Saldo abans: Ana 500, Luis 300
Transferència OK
Saldo després: Ana 400, Luis 400
```

Pista: `con.setAutoCommit(false)`, després les dues operacions amb `PreparedStatement`, i al final `con.commit()`. El `rollback()` va al `catch (SQLException e)` intern. Prova a forçar la fallada (per exemple, un compte inexistent) i comprova que el saldo d'Ana no canvia.

Butlletí U15 — Avançat Resolt

📝 Butlletí U15 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: Connexió des de fitxer de propietats

> 🔄 Solució

`db.properties`:

```
url=jdbc:sqlite:instituto.db
```

```java
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class ConexioProperties {
    public static void main(String[] args) {
        Properties props = new Properties();
         try (InputStream in = Files.newInputStream(Path.of("db.properties")))
{
            props.load(in);
        } catch (IOException e) {
                System.err.println("No es va poder llegir db.properties: " +
e.getMessage());
            return;
        }

        String url = props.getProperty("url");
        if (url == null) {
            System.err.println("Falta la propietat 'url' a db.properties");
            return;
        }

        try (Connection con = DriverManager.getConnection(url)) {
            System.out.println("Connectat a: " + url);
        } catch (SQLException e) {
            System.err.println("Error BD: " + e.getMessage());
        }
    }
}
```

`Properties` carrega el parell clau-valor i `getProperty("url")` el recupera. Dues excepcions conviuen aquí: la `IOException` de llegir el fitxer (U14) i la `SQLException` de connectar. I les credencials no viatgen en el codi: és el manament 6 del decàleg.

⭐ Exercici 2: INSERT amb clau autogenerada

› 🔄 Solució

```
import java.sql.*;

public class InserirAmbId {
    public static void main(String[] args) {
        String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?,
?)";

                                        try     (Connection     con     =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
                        PreparedStatement  pstmt  =  con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

            pstmt.setString(1, "María");
            pstmt.setInt(2, 22);
            pstmt.setString(3, "DAM");
            pstmt.executeUpdate();

            try (ResultSet claus = pstmt.getGeneratedKeys()) {
                if (claus.next()) {
                    System.out.println("Nou id: " + claus.getInt(1));
                }
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

La clau està en `Statement.RETURN_GENERATED_KEYS`: li dius a la base de dades que vols saber el id que acaba de generar. Després, `getGeneratedKeys()` torna un `ResultSet` amb aquella clau i es llegeix amb `next()` + `getInt(1)`. Sense aquella opció, hauríes de fer una consulta extra o endevinar.

⭐ Exercici 3: UPDATE condicional

› 🔄 Solució

```java
import java.sql.*;
import java.util.Scanner;

public class ActualitzarCurs {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Quina edat mínima? ");
        int edat = sc.nextInt();
        sc.nextLine();
        System.out.print("Quin nou curs? ");
        String curs = sc.nextLine();

        String sql = "UPDATE alumnos SET curso = ? WHERE edad > ?";

                                        try     (Connection     con     =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, curs);
            pstmt.setInt(2, edat);

            int files = pstmt.executeUpdate();
            System.out.println("Actualitzats " + files + " alumne/s");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

El `WHERE edad > ?` limita l'actualització als alumnes majors d'aquella edat. `executeUpdate()` torna el nombre de files afectades: si imprimeix `0`, és que no hi havia ningú major de 25 (o tots ja estaven en aquell curs). Sense el `WHERE`, hauríes actualitzat tota la taula. Comprova sempre les files.

⭐ ⭐ Exercici 4: INNER JOIN amb PreparedStatement 🔗

› 🔄 Solució

```
import java.sql.*;
import java.util.Scanner;

public class JoinAlumne {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nom de l'alumne: ");
        String nom = sc.nextLine();

        String sql = "SELECT a.nombre, m.asignatura, m.nota " +
                     "FROM alumnos a " +
                     "INNER JOIN matriculas m ON m.id_alumno = a.id " +
                     "WHERE a.nombre = ?";

                                          try     (Connection     con     =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, nom);

            try (ResultSet rs = pstmt.executeQuery()) {
                System.out.println("Alumne: " + nom);
                boolean trobat = false;
                while (rs.next()) {
                    System.out.printf("  %s: %.1f%n",
                        rs.getString("asignatura"),
                        rs.getDouble("nota"));
                    trobat = true;
                }
                if (!trobat) System.out.println("  Sense matrícules");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

L'`INNER JOIN` relaciona `alumnos` i `matriculas` per la clau forana `id_alumno`, i el `WHERE` filtra pel nom. Una sola consulta, un sol viatge a la base de dades: és l'antídot del patró N+1 (manament 8). Els àlies `a` i `m` fan l'SQL més curt.

⭐⭐ Exercici 5: Cerca amb LIKE

> 🔄 Solució

```
import java.sql.*;
import java.util.Scanner;

public class CercarAlumnes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Part del nom: ");
        String text = sc.nextLine();

        String sql = "SELECT * FROM alumnos WHERE nombre LIKE ?";

                                            try     (Connection     con     =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, "%" + text + "%");

            try (ResultSet rs = pstmt.executeQuery()) {
                boolean trobat = false;
                while (rs.next()) {
                    System.out.printf("%d - %s (%d)%n",
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getInt("edad"));
                    trobat = true;
                }
                if (!trobat) System.out.println("Sense resultats");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

`LIKE ?` amb `setString(1, "%" + text + "%")`: els `%` són comodins que permeten coincidència en qualsevol posició. La concatenació aquí és segura perquè els `%` formen part del **valor**, no de l'SQL. El `ResultSet` anidat en el seu propi `try-with-resources` es tanca sol.

⭐⭐ Exercici 6: Dates en JDBC

> 🔄 Solució

```
import java.sql.*;
import java.util.Scanner;

public class DatesJDBC {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nom: ");
        String nom = sc.nextLine();
        System.out.print("Edat: ");
        int edat = sc.nextInt();
        sc.nextLine();
        System.out.print("Curs: ");
        String curs = sc.nextLine();
        System.out.print("Data de naixement (YYYY-MM-DD): ");
        String dataText = sc.nextLine();

            String inserir = "INSERT INTO alumnos (nombre, edad, curso,
fecha_nacimiento) VALUES (?, ?, ?, ?)";

                                        try (Connection con =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             PreparedStatement pstmt = con.prepareStatement(inserir)) {

            pstmt.setString(1, nom);
            pstmt.setInt(2, edat);
            pstmt.setString(3, curs);
            pstmt.setDate(4, Date.valueOf(dataText));

            int files = pstmt.executeUpdate();
            System.out.println("Inserits " + files + " alumne/s");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }

        String llistar = "SELECT nombre, fecha_nacimiento FROM alumnos";
                                        try (Connection con =
DriverManager.getConnection("jdbc:sqlite:instituto.db");
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(llistar)) {
            while (rs.next()) {
                System.out.printf("%s - %s%n",
```

```
                    rs.getString("nombre"),
                    rs.getDate("fecha_nacimiento"));
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

`java.sql.Date.valueOf("2000-03-15")` converteix el text en una data que JDBC entén, i `setDate(4, ...)` la insereix. Per a llegir-la, `rs.getDate(...)`. Alerta amb el tipus: és `java.sql.Date`, no `java.util.Date` (són diferents). El `String` de la data ha d'anar en el format exacte `YYYY-MM-DD`.

⭐⭐ Exercici 7: Batch INSERT — 100 alumnes de prova

› 🔄 Solució

```java
import java.sql.*;

public class BatchAlumnes {
    public static void main(String[] args) {
        String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";

        long inicio = System.currentTimeMillis();

        try (Connection con = DriverManager.getConnection("jdbc:sqlite:instituto.db");
             PreparedStatement pstmt = con.prepareStatement(sql)) {

            for (int i = 1; i <= 100; i++) {
                pstmt.setString(1, "Alumno" + i);
                pstmt.setInt(2, 18 + (i % 10));
                pstmt.setString(3, "DAM");
                pstmt.addBatch();
            }

            int[] resultats = pstmt.executeBatch();

            long fin = System.currentTimeMillis();
            System.out.println("Inserits " + resultats.length + " alumnes en " + (fin - inicio) + " ms");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

`addBatch()` acumula les sentències sense enviar-les, i `executeBatch()` les mana **totes de cop** en un sol viatge a la base de dades. `resultats` conté les files afectades per cada lot. Comparat amb 100 `executeUpdate()` solts (100 viatges), el batch és moltíssim més ràpid: una sola operació de xarxa en comptes de cent.

⭐⭐⭐ Exercici 8: El patró DAO

› 🔄 Solució

1. El model:

```
public class Alumno {
    private int id;
    private String nombre;
    private int edad;
    private String curso;

    public Alumno() {}
    public Alumno(String nombre, int edad, String curso) {
        this.nombre = nombre;
        this.edad = edad;
        this.curso = curso;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }
    public String getCurso() { return curso; }
    public void setCurso(String curso) { this.curso = curso; }

    @Override
    public String toString() {
        return id + " - " + nombre + " (" + edad + ") " + curso;
    }
}
```

2. La interfície (el contracte):

```java
import java.util.List;

public interface AlumnoDAO {
    List<Alumno> listar();
    Alumno buscarPorId(int id);
    List<Alumno> buscarPorNombre(String nombre);
    boolean insertar(Alumno a);
    boolean actualizar(Alumno a);
    boolean eliminar(int id);
}
```

3. La implementació (l'SQL):

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class AlumnoDAOImpl implements AlumnoDAO {
    private static final String URL = "jdbc:sqlite:instituto.db";

    @Override
    public List<Alumno> listar() {
        List<Alumno> alumnes = new ArrayList<>();
        String sql = "SELECT * FROM alumnos ORDER BY nombre";
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                Alumno a = new Alumno(
                    rs.getString("nombre"),
                    rs.getInt("edad"),
                    rs.getString("curso"));
                a.setId(rs.getInt("id"));
                alumnes.add(a);
            }
        } catch (SQLException e) {
            System.err.println("Error en llistar: " + e.getMessage());
        }
        return alumnes;
    }

    @Override
    public Alumno buscarPorId(int id) {
        String sql = "SELECT * FROM alumnos WHERE id = ?";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setInt(1, id);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    Alumno a = new Alumno(
                        rs.getString("nombre"),
                        rs.getInt("edad"),
                        rs.getString("curso"));
                    a.setId(rs.getInt("id"));
                    return a;
```

```
                }
            }
        } catch (SQLException e) {
            System.err.println("Error en cercar: " + e.getMessage());
        }
        return null;
    }

    @Override
    public List<Alumno> buscarPorNombre(String nombre) {
        List<Alumno> alumnes = new ArrayList<>();
        String sql = "SELECT * FROM alumnos WHERE nombre LIKE ?";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setString(1, "%" + nombre + "%");
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    Alumno a = new Alumno(
                        rs.getString("nombre"),
                        rs.getInt("edad"),
                        rs.getString("curso"));
                    a.setId(rs.getInt("id"));
                    alumnes.add(a);
                }
            }
        } catch (SQLException e) {
            System.err.println("Error en cercar: " + e.getMessage());
        }
        return alumnes;
    }

    @Override
    public boolean insertar(Alumno a) {
        String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?,
?)";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setString(1, a.getNombre());
            pstmt.setInt(2, a.getEdad());
            pstmt.setString(3, a.getCurso());
            return pstmt.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error en inserir: " + e.getMessage());
```

```
            return false;
        }
    }

    @Override
    public boolean actualizar(Alumno a) {
        String sql = "UPDATE alumnos SET nombre = ?, edad = ?, curso = ? WHERE
id = ?";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setString(1, a.getNombre());
            pstmt.setInt(2, a.getEdad());
            pstmt.setString(3, a.getCurso());
            pstmt.setInt(4, a.getId());
            return pstmt.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error en actualitzar: " + e.getMessage());
            return false;
        }
    }

    @Override
    public boolean eliminar(int id) {
        String sql = "DELETE FROM alumnos WHERE id = ?";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setInt(1, id);
            return pstmt.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error en eliminar: " + e.getMessage());
            return false;
        }
    }
}
```

4. El Main (programa contra la interfície):

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        AlumnoDAO dao = new AlumnoDAOImpl();

        dao.insertar(new Alumno("Nova Alumna", 21, "DAM"));
        System.out.println("--- Alumnes ---");
        for (Alumno a : dao.listar()) {
            System.out.println(a);
        }

        System.out.println("--- Cercar per nom ('María') ---");
        for (Alumno a : dao.buscarPorNombre("María")) {
            System.out.println(a);
        }
    }
}
```

El `Main` només coneix la interfície `AlumnoDAO`: si demà canvies SQLite per MySQL, només canvia `AlumnoDAOImpl`, i el `Main` no se n'assabenta. Aquella és la màgia del DAO.

⭐⭐⭐ Exercici 9: Transacció bancària atòmica

› 🔄 Solució

```
import java.sql.*;

public class Transferencia {
    private static final String URL = "jdbc:sqlite:instituto.db";

    public static void transferir(int idOrigen, int idDesti, double quantitat)
{
        String treure = "UPDATE cuentas SET saldo = saldo - ? WHERE id = ?";
        String posar  = "UPDATE cuentas SET saldo = saldo + ? WHERE id = ?";

        try (Connection con = DriverManager.getConnection(URL)) {
            con.setAutoCommit(false);  // obrim la transacció

            try (PreparedStatement q = con.prepareStatement(treure);
                 PreparedStatement p = con.prepareStatement(posar)) {

                q.setDouble(1, quantitat);
                q.setInt(2, idOrigen);
                q.executeUpdate();

                if (idOrigen == idDesti) {
                    throw new SQLException("Destí invàlid: mateix compte");
                }

                p.setDouble(1, quantitat);
                p.setInt(2, idDesti);
                p.executeUpdate();

                con.commit();
                System.out.println("Transferència OK");
            } catch (SQLException e) {
                con.rollback();
                            System.err.println("Va  fallar,  tot  desfet:  "  +
e.getMessage());
            }
        } catch (SQLException e) {
            System.err.println("Error de connexió: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        transferir(1, 2, 100);
```

```
            // Per a provar el rollback, crida transferir(1, 1, 100): mateix
compte
        }
}
```

`setAutoCommit(false)` obre la transacció: les dues operacions s'executen sense confirmar-se. Si tot va bé, `commit()` les guarda juntes. Si alguna cosa falla (aquí, forçat amb `throw new SQLException`), `rollback()` desfà la primera operació perquè el sistema no quedi a mitges. És la definició d'atòmic: tot o res. Sense això, una transferència a mitges deixaria diners flotant al limbe.

Butlletí U15 — Extres

📝 Butlletí U15 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes. Les solucions estan amagades: esgota la teva pista abans de mirar-les.

CodeWars

1. SQL Basics: Simple JOIN

Crea una consulta `SELECT` que torni tots els productes juntament amb l'empresa que els fabrica. Hauràs de relacionar les taules `products` i `companies` mitjançant la clau forana `company_id`.

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Un `INNER JOIN` sobre la clau forana: `products.company_id = companies.id`. Selecciona les columnes que et demani el problema i cuida els àlies.

> 🔄 **Solució**
>
> ```sql
> SELECT p.name, c.name AS company_name
> FROM products p
> INNER JOIN companies c ON p.company_id = c.id;
> ```
>
> El `JOIN` relaciona les dues taules per la clau forana i, en una sola consulta, tens producte i empresa. És el mateix concepte del punt 8 de la unitat: una consulta amb `JOIN` on no caben consultes en bucle.

---

2. SQL with Street Fighter: Total Wins

És hora de decidir quins lluitadors passen a les semifinals del campionat mundial de Street Fighter. Cada combat registra si el lluitador va guanyar (1) o va perdre (0), i el moviment amb què va acabar. Com que els atacs ki han estat prohibits, **no es compten** els combats acabats

amb Hadoken, Shouoken o Kikoken. Torna `name`, `won` i `lost` sumant les victòries i derrotes, ordena de més a menys victòries i torna els 6 millors.

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> `GROUP BY` el nom del lluitador amb `SUM(won)` i `SUM(lost)`. Exclou els moviments prohibits amb `NOT IN ('Hadoken', 'Shouoken', 'Kikoken')` i ordena amb `ORDER BY won DESC LIMIT 6`.

> 🔄 **Solució**
>
```sql
SELECT f.name, SUM(f.won) AS won, SUM(f.lost) AS lost
FROM fighters f
LEFT JOIN winning_moves m ON f.move_id = m.id
WHERE m.move NOT IN ('Hadoken', 'Shouoken', 'Kikoken')
GROUP BY f.name
ORDER BY won DESC
LIMIT 6;
```
>
> `GROUP BY` agrupa els combats de cada lluitador, `SUM` acumula victòries i derrotes, i el `WHERE` descarta els atacs prohibits abans d'agrupar. És exactament el tipus de consulta que podries llançar amb un `PreparedStatement` contra la taula `fighters`.

---

3. SQL Basics: Simple HAVING

Tens una taula `people` amb `id`, `name` i `age`. Compta quantes persones tenen la mateixa edat i torna **només els grups d'edat amb 10 o més persones**.

- Enunciat en CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> `GROUP BY` la columna `age` i compta amb `COUNT(*)`. `WHERE` no val per a filtrar grups: necessites `HAVING COUNT(*) >= 10`, que s'aplica després del agrupament.

> 🔄 **Solució**

```sql
SELECT age, COUNT(id) AS total_people
FROM people
GROUP BY age
HAVING COUNT(id) >= 10;
```

`GROUP BY age` crea un grup per cada edat, `COUNT(id)` compta els seus integrants, i `HAVING` filtra els grups que no arriben a 10 persones. La diferència amb `WHERE`: `WHERE` filtra files abans d'agrupar, `HAVING` filtra grups després.

---

4. SQL Basics: Group By Day

Tens una taula `orders` amb `id`, `datetime` i `amount`. Compta quants pedidos hi ha **per dia**, extraient la data del camp `datetime`.

- Enunciat en CodeWars
- Dificultat: 5 kyu

> 💡 **Pista**
>
> Extrau el dia amb la funció `DATE(datetime)` i agrupa'l amb `GROUP BY`. Compta amb `COUNT(*)`. És el `GROUP BY` de sempre, però sobre una columna calculada.

> 🔄 **Solució**
>
> ```sql
> SELECT DATE(datetime) AS day, COUNT(*) AS total
> FROM orders
> GROUP BY DATE(datetime);
> ```
>
> `DATE(datetime)` retalla el `datetime` fins a només la data, i `GROUP BY` agrupa tots els pedidos d'aquell dia. Una columna calculada com a àlies: el mateix mecanisme que usaries amb `ResultSetMetaData` per a llegir-la després des de Java.

AceptaElReto

5. 245 — Qui guanya la partida?

Un grup de jugadors participa en un joc per torns amb números. Cada ronda, el jugador que encerta es descarta i el que falla passa al **final de la cua** per a tornar-ho a intentar. Simula les

rondes i determina qui guanya la partida.

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐⭐

> 💡 **Pista**
>
> Simula els torns amb una `Queue<Integer>`: cada ronda, treu el primer amb `poll()`, i si falla el tornes a ficar amb `offer()`. El que encerta surt per sempre. La cua encaixa perfectament amb l'estructura "qui falla, torna al final".

> 🔄 **Solució**

```java
import java.util.*;

public class QuiGuanya {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Llegir participants i dades del joc segons l'enunciat
        Queue<Integer> cua = new LinkedList<>();
        // ... omplir la cua amb els jugadors ...

        // while (cua.size() > 1) {
        //     int jugador = cua.poll();
        //     if (falla(jugador)) {
        //         cua.offer(jugador);   // torna al final
        //     }
        // }
        // System.out.println(cua.peek());
    }
}
```

La `LinkedList` com a `Queue` és la protagonista: `poll()` treu el primer i `offer()` el torna al final si falla. Els jugadors que encerten surten per sempre, i l'últim que queda és el guanyador. Estructura de dades (U12) al servei del problema de torns.

---

6. 424 — Bitllets d'autobús

Hi ha diverses rutes d'autobús entre dues ciutats, cadascuna amb la seva hora de sortida i d'arribada. Vols agafar **el màxim nombre d'autobusos possible** sense que se superposen

(agafar-ne un, baixar-te, i poder pujar a un altre que surti després d'arribar).

- Enunciat en AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**
>
> Algoritme voraç: ordena les rutes per **hora d'arribada** i tria sempre la següent ruta que acabi abans i que no se superpose amb l'última triada. És el clàssic problema de "selecció d'activitats".

> 🔄 **Solució**

```
import java.util.*;

public class Bitllets {
    static class Ruta implements Comparable<Ruta> {
        int eixida, arribada;
        Ruta(int e, int a) { eixida = e; arribada = a; }

        public int compareTo(Ruta o) {
                return Integer.compare(arribada, o.arribada); // ordena per
arribada
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Llegir rutes, ordenar-les i aplicar el voraç:
        // Collections.sort(rutes);
        // int ultima = Integer.MIN_VALUE, comptador = 0;
        // for (Ruta r : rutes) {
        //     if (r.eixida >= ultima) {
        //         comptador++;
        //         ultima = r.arribada;
        //     }
        // }
        // System.out.println(comptador);
    }
}
```

El truc voraç: ordenar per hora d'arribada garanteix que sempre tries la ruta que lliber a el dia abans, deixant buit per a més autobusos. Una sola passada amb un comptador i una variable `ultima`. Clàssic d'AceptaElReto: les dades s'ordenen i la solució surt sola.

🧭 **I si et quedes amb ganes?** Quan domines el JDBC, torna als problemes d'AceptaElReto d'unitats anteriors i reescriu-los guardant les dades d'entrada en una taula SQLite amb `PreparedStatement`: et sorprendrà el natural que resulta que els teus algoritmes parlen amb una base de dades. El material no es perd: es reutilitza.

Butlletí U15 — Inicial

📝 Butlletí U15 — Inicial

> Sense solucions. Sense presses. Obre l'IDE, crea el teu primer `HttpServer` i fes que el navegador et parli de tu a tu. Promet que cap bit sortirà ferit. Comença suaument, que la web mossega a a poc a poc.

Exercici 1: Escriu aquest programa — Hola, món web

Crea un servidor HTTP al port 8080 amb una ruta `/` que torni el text `Hola, mundo web!`. Obre `http://localhost:8080` en el teu navegador i comprova que ho veus.

Pista: `HttpServer.create(new InetSocketAddress(8080), 0)`, després `createContext("/", ...)` i `sendResponseHeaders(200, ...)`. No oblidis tancar el cos de la resposta.

Exercici 2: Escriu aquest programa — servidor de l'hora

Afegeix al teu servidor una ruta `/hora` que torni l'hora actual en text pla amb format `HH:mm:ss`.

Cada vegada que recarregues el navegador, l'hora canvia. Màgia negra amb `LocalTime.now()`.

```
// Pista:
String hora = java.time.LocalTime.now().format(
    java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")
);
```

Exercici 3: Escriu aquest programa — pàgina que diu el teu nom

Afegeix una ruta `/saludo?nombre=Pepe` que torni una pàgina HTML amb `<h1>¡Hola, Pepe!</h1>`.

Si no es passa nom, que digui `¡Hola, desconocido!`.

Pista: `e.getRequestURI().getQuery()` et dona `nombre=Pepe`. Divideix per = i llest.

Exercici 4: Escriu aquest programa — comptador de visites global

Usa una variable `static int visites = 0`. Cada vegada que algú visita `/`, incrementa el comptador i mostra:

```
Eres el visitante número 47
```

Què passa si dues persones recarreguen alhora? (spoiler: problemes — però de moment no et preocupes).

Exercici 5: Escriu aquest programa — generador d'excuses per a lliuraments tardans

Crea una ruta `/excusa` que torni una excusa generada aleatòriament combinant elements de tres arrays:

```
String[] subjectes = {"El meu gos", "GitHub", "El plugin d'IntelliJ", "La connexió"};
String[] verbs     = {"es va menjar", "va esborrar", "va corrompre", "va perdre"};
String[] objectes  = {"l'examen", "la pràctica", "els apunts", "la meua paciència"};
```

Exemple: *"GitHub va esborrar la pràctica"*. Torna-ho en HTML amb bona lletra.

Exercici 6: Escriu aquest programa — taula de multiplicar personalitzada

Ruta `/tabla?num=7` que generi una taula HTML completa amb la taula de multiplicar del 7 (del 7×1 al 7×10).

Si no es passa número, usa el 5 per defecte. Formata la taula amb `border=1` i colors alterns en les files.

Exercici 7: Escriu aquest programa — convertidor d'euros a pessetes (sí, pessetes)

Ruta `/conversor?euros=50` que torni HTML amb el resultat: "50 euros son 8319.3 pesetas".

1 € = 166.386 pts. Mostra dos decimals.

Pista: `String.format("%.2f", valor)` per als decimals. Sí, pessetes. Si ets jove, pregunta-li a un vell què era això.

Exercici 8: Escriu aquest programa — pàgina d'estat del servidor

Ruta `/estado` que torni un JSON amb aquesta pinta:

```
{
    "servidor": "ok",
    "hora": "14:30:01",
    "visites": 47,
    "versio": "1.0",
    "autor": "El teu nom ací"
}
```

No oblidis el `Content-Type: application/json` o el navegador es tornarà tonto.

Exercici 9: Troba l'error — el Content-Type oblidat

Un company t'ensenya aquest handler i jura que torna JSON:

```
server.createContext("/api/frase", intercambio -> {
    String json = """
        {"frase": "El código limpio es como un buen chiste"}
        """;
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

El navegador mostra el text, però el `fetch` que el consumeix es queixa: `r.json()` peta. Què falta i per què és tan important?

Butlletí U15 — Inicial Resolt

📝 Butlletí U15 — Inicial (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

Exercici 1: Escriu aquest programa — Hola, món web

> 🔄 Solució

```
import com.sun.net.httpserver.HttpServer;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class HolaMundoWeb {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            String resp = "Hola, mundo web!";
            e.sendResponseHeaders(200, resp.getBytes().length);
            OutputStream os = e.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

`createContext` registra la ruta `/` amb el seu handler; `sendResponseHeaders(200, grandària)` envia el codi i la grandària del cos (en bytes); `getResponseBody()` escriu el text i es tanca. El servidor es queda escoltant fins que el detinguis.

Exercici 2: Escriu aquest programa — servidor de l'hora

> 🔄 Solució

```
server.createContext("/hora", e -> {
    String hora = java.time.LocalTime.now().format(
        java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")
    );
    String resp = "Hora actual: " + hora;
    e.getResponseHeaders().set("Content-Type", "text/plain; charset=UTF-8");
    e.sendResponseHeaders(200, resp.getBytes().length);
    e.getResponseBody().write(resp.getBytes());
    e.getResponseBody().close();
});
```

`LocalTime.now()` s'avalua en cada petició, per això l'hora canvia en recarregar. Un servidor sense estat: no recorda res entre peticions.

Exercici 3: Escriu aquest programa — pàgina que diu el teu nom

> 🔄 Solució

```
server.createContext("/saludo", e -> {
    String query = e.getRequestURI().getQuery();
    String nombre = "desconocido";
    if (query != null && query.startsWith("nombre=")) {
        nombre = java.net.URLDecoder.decode(query.split("=")[1], "UTF-8");
    }
    String html = "<h1>¡Hola, " + nombre + "!</h1>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});
```

La query string arriba com `nombre=Pepe` i es divideix per `=`. L'`URLDecoder` converteix coses com `%20` en espais (si el nom fora "Ana M."). Si no hi ha query, el valor per defecte és

> desconocido.

Exercici 4: Escriu aquest programa — comptador de visites global

> 🔄 Solució

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

public class ContadorVisitas {
    static int visitas = 0;

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            visitas++;
            String resp = "Eres el visitante número " + visitas;
            e.sendResponseHeaders(200, resp.getBytes().length);
            e.getResponseBody().write(resp.getBytes());
            e.getResponseBody().close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

`static int visitas` viu en la classe, no en cap petició, així que sobreviu entre visites. Si dues persones recarreguen alhora, totes dues podrien llegir el mateix valor abans d'incrementar (problema de concurrència). Per a aprendre, val; per a producció, `AtomicInteger`.

Exercici 5: Escriu aquest programa — generador d'excuses per a lliuraments tardans

> 🔄 Solució

```
server.createContext("/excusa", e -> {
    String[] subjectes = {"El meu gos", "GitHub", "El plugin d'IntelliJ", "La connexió"};
    String[] verbs     = {"es va menjar", "va esborrar", "va corrompre", "va perdre"};
    String[] objectes  = {"l'examen", "la pràctica", "els apunts", "la meua paciència"};

    java.util.Random r = new java.util.Random();
    String excusa = subjectes[r.nextInt(subjectes.length)] + " "
        + verbs[r.nextInt(verbs.length)] + " "
        + objectes[r.nextInt(objectes.length)];

    String html = "<h1>La teua excusa per a hui</h1><p><em>" + excusa + "</em></p>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});
```

`Random.nextInt(longitud)` tria un índex aleatori de cada array sense passar-se. Tres decisions aleatòries = una excusa nova en cada recàrrega. Recorda: `nextInt(n)` va de 0 a `n-1`.

Exercici 6: Escriu aquest programa — taula de multiplicar personalitzada

› 🔄 Solució

```
server.createContext("/tabla", e -> {
    String query = e.getRequestURI().getQuery();
    int num = 5;
    if (query != null && query.startsWith("num=")) {
        num = Integer.parseInt(query.split("=")[1]);
    }

    StringBuilder sb = new StringBuilder("<table border=1>");
        sb.append("<tr><th>").append(num).append("  ×  n</th><th>Resultado</th>
</tr>");
    for (int i = 1; i <= 10; i++) {
        String color = (i % 2 == 0) ? "#e0f7e0" : "#ffffff";
        sb.append("<tr style=\"background:").append(color).append("\">")
          .append("<td>").append(num).append(" × ").append(i).append("</td>")
          .append("<td>").append(num * i).append("</td></tr>");
    }
    sb.append("</table>");

    String html = "<html><body>" + sb + "</body></html>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});
```

El `StringBuilder` munta l'HTML de la taula fila a fila (és més eficient que concatenar amb `+` en un bucle). Les files parells porten un fons verd clar per a alternar colors. Si no hi ha `num=`, s'usa el 5.

Exercici 7: Escriu aquest programa — convertidor d'euros a pessetes

› 🔄 Solució

```
server.createContext("/conversor", e -> {
    String query = e.getRequestURI().getQuery();
    double euros = 50;
    if (query != null && query.startsWith("euros=")) {
        euros = Double.parseDouble(query.split("=")[1]);
    }
    double pesetas = euros * 166.386;
    String html = "<h1>" + String.format("%.2f", euros) + " euros son "
        + String.format("%.2f", pesetas) + " pesetas</h1>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});
```

`String.format("%.2f", valor)` arrodoneix a dos decimals. Es multiplica per 166.386 (el canvi oficial). Un exercici clàssic de GET: l'entrada viatja a la URL i la sortida és HTML.

Exercici 8: Escriu aquest programa — pàgina d'estat del servidor

› 🔄 Solució

```
server.createContext("/estado", e -> {
    String json = "{\"servidor\":\"ok\","
        + "\"hora\":\"" + java.time.LocalTime.now().format(
            java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")) + "\","
        + "\"visites\":" + visitas + ","
        + "\"versio\":\"1.0\","
        + "\"autor\":\"El teu nom ací\"}";
    e.getResponseHeaders().set("Content-Type", "application/json");
    e.sendResponseHeaders(200, json.getBytes().length);
    e.getResponseBody().write(json.getBytes());
    e.getResponseBody().close();
});
```

La capçalera `Content-Type: application/json` és el que converteix això en una API: sense ella, el navegador i `fetch` tractarien el text com a HTML o text pla. Fixa't en com s'escapen les cometes del JSON dins del codi Java.

Exercici 9: Troba l'error — el Content-Type oblidat

> 🔄 Solució

Falta **`Content-Type: application/json`**. Sense aquella capçalera, `fetch` no sap que la resposta és JSON i `r.json()` falla (o torna brossa). El text arriba, però el navegador l'interpreta com a text pla.

La solució és afegir una línia abans d'enviar les capçaleres:

```
intercambio.getResponseHeaders().set("Content-Type", "application/json");
```

Aquesta és la lliçó que es repeteix en tota la unitat: **la capçalera correcta és la diferència entre "funciona" i "funciona en la meva màquina però el frontend es queixa"**.

Butlletí U15 — Avançat

📝 Butlletí U15 — Avançat

> Dificultat progressiva. ⭐ per a escalfar, ⭐⭐ per a pensar, ⭐⭐⭐ per a concursar. Cada exercici inclou una pista (resisteix-te a mirar-la).

⭐ Exercici 1: API de frases motivacionals

Crea un endpoint `GET /api/frase` que torni un JSON amb una frase aleatòria d'un array precarregat i el seu autor.

```
{"frase": "El código limpio es como un buen chiste: si tienes que explicarlo, es malo", "autor": "Alguien que sabe"}
```

El frontend és un HTML amb un botó "Nova frase" que al fer clic fa `fetch('/api/frase')` i mostra la frase en pantalla.

Pista: usa `Math.random()` per a triar un índex aleatori de l'array, o `Random.nextInt(longitud)`.

⭐ Exercici 2: Formulari de contacte amb POST

Crea una ruta `/contacto` que serveixi un formulari HTML (GET) amb camps `nombre` i `mensaje`, i una ruta `/enviar` que rebi les dades per POST i les mostri en una pàgina de confirmació.

Pista: en el handler de `/enviar` comprova `"POST".equals(e.getRequestMethod())` i llegeix el cos amb `e.getRequestBody().readAllBytes()`. El format del cos és `nombre=Ana&mensaje=Hola`.

⭐ Exercici 3: Pedra, paper, tisora online

Endpoint `POST /api/jugar` que rep:

```
{"jugada": "piedra"}
```

I torna:

```
{"jugadaPC": "tijera", "resultado": "ganaste"}
```

Regles clàssiques: pedra > tisora, tisora > paper, paper > pedra.

Frontend: tres botons amb emojis 🪨📄✂️. Al fer clic, envia la jugada i mostra el resultat.

Pista: la jugada del PC es tria amb `Random`. Les regles es poden implementar amb un `Map<String, String>` on la clau venç el valor: `{"piedra": "tijera", "tijera": "papel", "papel": "piedra"}`.

⭐⭐ Exercici 4: El temps que NO fa

Crea `GET /api/clima?ciudad=Madrid` que torni un JSON amb dades meteorològiques **aleatòries** (generades cada vegada):

```
{"ciudad": "Madrid", "temperatura": 28, "humedad": 45, "estado": "soleado"}
```

Estats possibles: `"soleado"`, `"nublado"`, `"lluvia"`, `"tormenta"`. Frontend amb emojis i temperatures de colors.

Pista: usa `String[] estados = {...}` i tria aleatòriament. La temperatura pot ser `random.nextInt(40) - 5`.

⭐⭐ Exercici 5: Traductor xungo (però funcional)

Implementa un endpoint `POST /api/traducir` que rebi:

```
{"texto": "hola", "idioma": "en"}
```

I torni:

```
{"traduccion": "hello"}
```

Usa un `HashMap<String, HashMap<String, String>>` com a diccionari. Fica almenys 10 paraules en espanyol traduïdes a anglès i francès.

Pista: inicialitza el diccionari amb blocs `static`. `diccionario.get("hola").get("en")` et dona `"hello"`. Els mapes els vas veure a la U13.

⭐ ⭐ Exercici 6: API REST de tasques amb prioritat

Implementa un CRUD complet de tasques on cada tasca té: `id`, `titol`, `prioritat` (`"ALTA"`, `"MITJA"`, `"BAIXA"`).

Mètode	Ruta	Descripció
GET	`/api/tareas`	Llista totes
POST	`/api/tareas`	Crea una (JSON: `{"titulo": "...", "prioridad": "ALTA"}`)
PUT	`/api/tareas/{id}`	Canvia prioritat (JSON: `{"prioridad": "BAJA"}`)
DELETE	`/api/tareas/{id}`	Esborra una

Frontend: taula amb colors de fons segons prioritat (vermell ALTA, groc MITJA, verd BAIXA). Botons per a crear, canviar prioritat i esborrar.

Pista: guarda les tasques en un `ConcurrentHashMap<Integer, Tarea>` amb un `AtomicInteger` per als IDs. Per al path param, parseja la ruta amb `substring`.

⭐⭐ Exercici 7: Client GET — els repos de GitHub

Usa `HttpClient` per a consultar l'API de GitHub (`https://api.github.com/users/{usuario}/repos`) i mostrar només el **nom** i el **llenguatge** de cada repositori d'un usuari (que es demana pel teclat). Després guarda els resultats en un fitxer `repos.txt`.

Pista: parseja la resposta amb Gson (`JsonArray`), recórri-la, i escriu amb `Files.writeString` (els fitxers els vas veure a la U14). Recorda la capçalera `User-Agent`, que GitHub exigeix.

⭐⭐ Exercici 8: Client POST — crear una publicació a jsonplaceholder

Usa `HttpClient` per a fer un `POST` a `https://jsonplaceholder.typicode.com/posts` amb un cos JSON:

```
{"title": "Mi primera API", "body": "Consumida desde Java", "userId": 1}
```

Mostra el codi d'estat i el cos de la resposta. Comprova que la capçalera `Content-Type: application/json` està posada.

Pista: `HttpRequest.newBuilder().uri(...).header("Content-Type", "application/json").POST(HttpRequest.BodyPublishers.ofString(json)).build()`.

⭐⭐⭐ Exercici 9: Middleware de logging

Crea una classe `LoggerMiddleware` que emboliqui qualsevol `HttpHandler` i registri en consola:

```
[2026-06-21 14:30:01] GET /api/peliculas → 200 (15ms)
[2026-06-21 14:30:05] POST /api/tareas → 201 (3ms)
```

Ha de poder aplicar-se a qualsevol handler així:

```
server.createContext("/api", new LoggerMiddleware(new TareasHandler()));
```

Pista: guarda `System.currentTimeMillis()` abans i després de cridar el handler original. Usa `e.getRequestMethod()`, `e.getRequestURI()` i `e.getResponseCode()` (després d'enviar capçaleres)

⭐ ⭐ ⭐ Exercici 10: la petició asíncrona amb sendAsync

Usa `HttpClient` per a demanar dades a l'API pública de GitHub **sense bloquejar el fil principal** amb `sendAsync()`. Descarrega els repos d'un usuari i, quan arribi la resposta, imprimeix el nombre de repos i el codi d'estat.

```
// Esquelet per a completar
HttpClient client = HttpClient.newHttpClient();
HttpRequest peticio = HttpRequest.newBuilder()
        .uri(URI.create("https://api.github.com/users/google/repos"))
        .GET()
        .build();
```

Completa el programa perquè:

- Usi `sendAsync(peticio, HttpResponse.BodyHandlers.ofString())`.
- Encadeno `.thenAccept(...)` per a processar la resposta quan arribi (imprimeix `statusCode()` i, si és 200, compta les aparicions de `"full_name"` en el JSON amb `split`).
- Afegeix un `System.out.println("Petició llançada, seguim treballant...");` ABANS que arribi la resposta, per a demostrar que el fil no es va bloquejar.

Pista: `sendAsync` torna un `CompletableFuture<HttpResponse<String>>`. El `thenAccept` rep la resposta quan estigui llesta, però el `main` continua corrent mentrestant. Perquè el programa no acabi abans que arribi la resposta, espera al final amb `.join()` sobre el `CompletableFuture`. Sense el `join()`, el `main` s'acaba i la petició es perd.

> 🔄 **Solució**

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ReposAsync {
    public static void main(String[] args) {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest peticio = HttpRequest.newBuilder()
                .uri(URI.create("https://api.github.com/users/google/repos"))
                .GET()
                .build();

        System.out.println("Petició llançada, seguim treballant...");

        client.sendAsync(peticio, HttpResponse.BodyHandlers.ofString())
                .thenAccept(resposta -> {
                                System.out.println("Resposta rebuda: codi " +
resposta.statusCode());
                    if (resposta.statusCode() == 200) {
                                                                int repos =
resposta.body().split("\"full_name\"").length - 1;
                        System.out.println("Repos de google: " + repos);
                    } else {
                            System.out.println("Alguna cosa ha fallat (límit de
l'API?): " + resposta.body().substring(0, 80));
                    }
                })
                .join();  // espera que acabe abans de finalitzar el main
    }
}
```

La màgia està en l'ordre de les sortides: "Petició llançada" s'imprimeix **abans** de "Resposta rebuda", encara que la petició es va llançar abans. Això és `sendAsync`: el fil principal no es deté esperant la xarxa; el `thenAccept` s'executa quan arribi la resposta. El `join()` al final és imprescindible: sense ell, el `main` acabaria i la JVM es tancaria abans que arribés la resposta. El comptatge de repos amb `split("\"full_name\"")` és un truc ràpid de JSON sense llibreria: cada repo apareix com un `"full_name"` en la llista.

.

Butlletí U15 — Avançat Resolt

📝 Butlletí U15 — Avançat (Resolt)

> Les solucions estan amagades en cada exercici. No facis trampa: primer intenta-ho de veritat.

⭐ Exercici 1: API de frases motivacionals

> 🔄 Solució

```java
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.util.Random;

public class ApiFrases {

    static String[] frases = {
        "El código limpio es como un buen chiste: si tienes que explicarlo, es malo",
        "La mejor forma de predecir el futuro es implementarlo",
        "Primero resuelve el problema, luego escribe el código"
    };
       static String[] autores = {"Alguien que sabe", "Alan Kay", "John Johnson"};

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        Random r = new Random();

        server.createContext("/api/frase", e -> {
            int i = r.nextInt(frases.length);
              String json = "{\"frase\":\"" + frases[i] + "\",\"autor\":\"" + autores[i] + "\"}";
            e.getResponseHeaders().set("Content-Type", "application/json");
            e.sendResponseHeaders(200, json.getBytes().length);
            e.getResponseBody().write(json.getBytes());
            e.getResponseBody().close();
        });

        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

`Random.nextInt(longitud)` tria un índex aleatori de l'array de frases. El `Content-Type: application/json` converteix la resposta en una API que el `fetch` del frontend pot llegir amb `r.json()`.

⭐ Exercici 2: Formulari de contacte amb POST

🔄 Solució

```
server.createContext("/contacto", e -> {
    String html = """
        <form action="/enviar" method="POST">
          <input name="nombre" placeholder="Tu nombre"><br>
          <textarea name="mensaje" placeholder="Tu mensaje"></textarea><br>
          <button>Enviar</button>
        </form>
        """;
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

server.createContext("/enviar", e -> {
    if ("POST".equals(e.getRequestMethod())) {
        String datos = new String(e.getRequestBody().readAllBytes());
        // datos = "nombre=Ana&mensaje=Hola"
        String nombre = extraer(datos, "nombre");
        String mensaje = extraer(datos, "mensaje");
          String html = "<h1>Gràcies, " + nombre + "!</h1><p>" + mensaje + "
</p>";
           e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-
8");
        e.sendResponseHeaders(200, html.getBytes().length);
        e.getResponseBody().write(html.getBytes());
        e.getResponseBody().close();
    } else {
        e.sendResponseHeaders(405, -1);  // Method Not Allowed
        e.close();
    }
});

static String extraer(String datos, String clave) {
    for (String par : datos.split("&")) {
        String[] t = par.split("=");
        if (t.length == 2 && t[0].equals(clave)) return t[1];
    }
    return "?";
}
```

GET serveix el formulari, POST rep les dades del cos i les torna en una pàgina de confirmació. El `405` per a mètodes que no siguin POST és la guinda professional: l'API no es calla, respon "mètode no permés".

⭐ Exercici 3: Pedra, paper, tisora online

> 🔄 Solució

```
import java.util.Map;
import java.util.Random;

static Map<String, String> regles = Map.of(
    "piedra", "tijera", "tijera", "papel", "papel", "piedra"
);

server.createContext("/api/jugar", e -> {
    if ("POST".equals(e.getRequestMethod())) {
        String body = new String(e.getRequestBody().readAllBytes());
        // body = {"jugada": "piedra"}
        String jugada = body.split("\"jugada\":\"")[1].split("\"")[0];
               String pc = new String[]{"piedra", "papel", "tijera"}[new
Random().nextInt(3)];

        String resultado;
        if (jugada.equals(pc)) {
            resultado = "empate";
                        } else if (regles.get(jugada) != null &&
regles.get(jugada).equals(pc)) {
            resultado = "ganaste";
        } else {
            resultado = "perdiste";
        }

           String json = "{\"jugadaPC\":\"" + pc + "\",\"resultado\":\"" +
resultado + "\"}";
        e.getResponseHeaders().set("Content-Type", "application/json");
        e.sendResponseHeaders(200, json.getBytes().length);
        e.getResponseBody().write(json.getBytes());
        e.getResponseBody().close();
    }
});
```

El `Map regles` codifica “qui venç a qui”: si `regles.get(jugada).equals(pc)`, guanyes. El `Random` tria la jugada del PC entre tres opcions. L’extracció de la jugada del JSON es fa amb `split` (didàctic); en producció, Gson.

⭐⭐ Exercici 4: El temps que NO fa

› 🔄 Solució

```
server.createContext("/api/clima", e -> {
    String query = e.getRequestURI().getQuery();
    String ciudad = "Madrid";
    if (query != null && query.startsWith("ciudad=")) {
        ciudad = query.split("=")[1];
    }

    Random r = new Random();
    String[] estados = {"soleado", "nublado", "lluvia", "tormenta"};
    int temperatura = r.nextInt(40) - 5;   // de -5 a 34
    int humedad = r.nextInt(101);          // de 0 a 100
    String estado = estados[r.nextInt(estados.length)];

       String json = "{\"ciudad\":\"" + ciudad + "\",\"temperatura\":" +
temperatura
        + ",\"humedad\":" + humedad + ",\"estado\":\"" + estado + "\"}";
    e.getResponseHeaders().set("Content-Type", "application/json");
    e.sendResponseHeaders(200, json.getBytes().length);
    e.getResponseBody().write(json.getBytes());
    e.getResponseBody().close();
});
```

El rang de temperatura surt de `nextInt(40) - 5` (desplaçar el rang de 0-39 a -5 a 34). Cada recàrrega genera dades diferents: un “clima” fals, però el patró d’una API amb query param i resposta JSON és exactament el de les de veritat.

⭐⭐ Exercici 5: Traductor xungo (però funcional)

› 🔄 Solució

```
import java.util.HashMap;
import java.util.Map;

static Map<String, Map<String, String>> diccionario = new HashMap<>();
static {
    Map<String, String> hola = new HashMap<>();
    hola.put("en", "hello");
    hola.put("fr", "bonjour");
    diccionario.put("hola", hola);

    Map<String, String> adios = new HashMap<>();
    adios.put("en", "goodbye");
    adios.put("fr", "au revoir");
    diccionario.put("adios", adios);

    Map<String, String> gracias = new HashMap<>();
    gracias.put("en", "thank you");
    gracias.put("fr", "merci");
    diccionario.put("gracias", gracias);
    // ... afig almenys 7 més
}

server.createContext("/api/traducir", e -> {
    if ("POST".equals(e.getRequestMethod())) {
        String body = new String(e.getRequestBody().readAllBytes());
        // body = {"texto": "hola", "idioma": "en"}
        String texto = body.split("\"texto\":\"")[1].split("\"")[0];
        String idioma = body.split("\"idioma\":\"")[1].split("\"")[0];

        String traduccion = diccionario.getOrDefault(texto, Map.of())
            .getOrDefault(idioma, "¿?");
        String json = "{\"traduccion\":\"" + traduccion + "\"}";
        e.getResponseHeaders().set("Content-Type", "application/json");
        e.sendResponseHeaders(200, json.getBytes().length);
        e.getResponseBody().write(json.getBytes());
        e.getResponseBody().close();
    }
});
```

Un mapa dins d'un altre: la paraula en espanyol és la clau de l'exterior, i el mapa interior associa idioma amb traducció. `getOrDefault` evita el `NullPointerException` quan la paraula o l'idioma no existeixen. Els mapes, com a la U13.

⭐ ⭐ Exercici 6: API REST de tasques amb prioritat

> 🔄 Solució

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

record Tarea(int id, String titulo, String prioridad) {}

static ConcurrentHashMap<Integer, Tarea> tareas = new ConcurrentHashMap<>();
static AtomicInteger contador = new AtomicInteger(1);

// Llistar
server.createContext("/api/tareas", e -> {
    if ("GET".equals(e.getRequestMethod())) {
        StringBuilder sb = new StringBuilder("[");
        boolean primero = true;
        for (Tarea t : tareas.values()) {
            if (!primero) sb.append(",");
            sb.append("{\"id\":").append(t.id())
              .append(",\"titulo\":\"").append(t.titulo())

.append("\",\"prioridad\":\"").append(t.prioridad()).append("\"}");
            primero = false;
        }
        sb.append("]");
        e.getResponseHeaders().set("Content-Type", "application/json");
        e.sendResponseHeaders(200, sb.toString().getBytes().length);
        e.getResponseBody().write(sb.toString().getBytes());
        e.getResponseBody().close();
    } else if ("POST".equals(e.getRequestMethod())) {
        String body = new String(e.getRequestBody().readAllBytes());
        String titulo = body.split("\"titulo\":\"")[1].split("\"")[0];
        String prioridad = body.split("\"prioridad\":\"")[1].split("\"")[0];
              Tarea nueva = new Tarea(contador.getAndIncrement(), titulo,
prioridad);
        tareas.put(nueva.id(), nueva);
        String json = "{\"id\":" + nueva.id() + "}";
        e.getResponseHeaders().set("Content-Type", "application/json");
        e.sendResponseHeaders(201, json.getBytes().length);
        e.getResponseBody().write(json.getBytes());
        e.getResponseBody().close();
    }
});

// Actualitzar i esborrar per ID
```

```
server.createContext("/api/tareas/", e -> {
    String ruta = e.getRequestURI().getPath();
    int id = Integer.parseInt(ruta.substring("/api/tareas/".length()));
    Tarea t = tareas.get(id);
    if (t == null) {
        e.getResponseHeaders().set("Content-Type", "application/json");
        String err = "{\"error\":\"no encontrada\"}";
        e.sendResponseHeaders(404, err.getBytes().length);
        e.getResponseBody().write(err.getBytes());
        e.getResponseBody().close();
        return;
    }
    if ("PUT".equals(e.getRequestMethod())) {
        String body = new String(e.getRequestBody().readAllBytes());
        String prioridad = body.split("\"prioridad\":\"")[1].split("\"")[0];
        tareas.put(id, new Tarea(id, t.titulo(), prioridad));
        e.sendResponseHeaders(200, -1);
        e.close();
    } else if ("DELETE".equals(e.getRequestMethod())) {
        tareas.remove(id);
        e.sendResponseHeaders(204, -1);
        e.close();
    }
});
```

`ConcurrentHashMap` + `AtomicInteger` aguanten peticions simultànies sense corrompre's. El `404` quan l'ID no existeix és el comportament correcte d'una API REST que es respecta. El `201` en crear i el `204` en esborrar arrodoneixen la bona conducta.

⭐⭐ Exercici 7: Client GET — els repos de GitHub

› 🔄 Solució

```
import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Scanner;

public class GitHubCliente {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Usuari de GitHub: ");
        String usuario = sc.nextLine();
        sc.close();

        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://api.github.com/users/" + usuario +
"/repos"))
            .header("User-Agent", "ClienteJava/1.0")
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
                    JsonArray repos = new Gson().fromJson(response.body(),
JsonArray.class);
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < repos.size(); i++) {
                JsonObject repo = repos.get(i).getAsJsonObject();
                String nombre = repo.get("name").getAsString();
                                String lenguaje = repo.has("language") &&
!repo.get("language").isJsonNull()
                    ? repo.get("language").getAsString() : "?";
                sb.append(nombre).append(" (").append(lenguaje).append(")\n");
                System.out.println("⭐ " + nombre + " (" + lenguaje + ")");
            }
```

```
            Files.writeString(Path.of("repos.txt"), sb.toString());
            System.out.println("Guardado en repos.txt");
        } else {
            System.out.println("Error " + response.statusCode() + " — existeix
l'usuari?");
        }
    }
}
```

`HttpClient` demana, Gson parseja i `Files.writeString` guarda: el trio complet del costat client. La capçalera `User-Agent` és obligatòria a GitHub (sense ella, rebutja la petició). El `has(...)` evita el drama quan el camp `language` no existeix o és `null`.

⭐⭐ Exercici 8: Client POST — crear una publicació a jsonplaceholder

› 🔄 Solució

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClientePOST {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        String json = """
            {"title": "Mi primera API",
             "body": "Consumida desde Java",
             "userId": 1}
            """;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        System.out.println("Código: " + response.statusCode());
        System.out.println("Respuesta: " + response.body());
    }
}
```

jsonplaceholder et torna el recurs creat amb el seu nou ID: `201` (Created). El `Content-Type: application/json` és el que li diu al servidor que el cos és JSON. Sense ell, jsonplaceholder respon `415` (Unsupported Media Type).

⭐⭐⭐ Exercici 9: Middleware de logging

› 🔄 Solució

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class LoggerMiddleware implements HttpHandler {

    private final HttpHandler original;

    public LoggerMiddleware(HttpHandler original) {
        this.original = original;
    }

    @Override
    public void handle(HttpExchange e) throws IOException {
        long inicio = System.currentTimeMillis();
        try {
            original.handle(e);
        } finally {
            long duracion = System.currentTimeMillis() - inicio;
            String fecha = LocalDateTime.now().format(
                DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
            System.out.printf("[%s] %s %s → %d (%dms)%n",
                fecha, e.getRequestMethod(), e.getRequestURI(),
                e.getResponseCode(), duracion);
        }
    }
}
```

I s'usa així, embolicant qualsevol handler:

```java
server.createContext("/api", new LoggerMiddleware(new TareasHandler()));
```

El truc: `LoggerMiddleware` **implementa el mateix contracte** (`HttpHandler`) i embolica l'original. El `try/finally` garanteix que el log s'imprimeix encara que el handler falli. `getResponseCode()` només val després d'enviar capçaleres, per això es llegeix al final. A això se li diu *decorator* o middleware: emboliques un handler amb un altre que afegeix comportament.

⭐ ⭐ ⭐ Exercici 10: la petició asíncrona amb sendAsync

> 🔄 Solució

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ReposAsync {
    public static void main(String[] args) {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest peticio = HttpRequest.newBuilder()
                .uri(URI.create("https://api.github.com/users/google/repos"))
                .GET()
                .build();

        System.out.println("Petició llançada, seguim treballant...");

        client.sendAsync(peticio, HttpResponse.BodyHandlers.ofString())
                .thenAccept(resposta -> {
                            System.out.println("Resposta rebuda: codi " +
resposta.statusCode());
                    if (resposta.statusCode() == 200) {
                                                                int repos =
resposta.body().split("\"full_name\"").length - 1;
                        System.out.println("Repos de google: " + repos);
                    } else {
                            System.out.println("Alguna cosa ha fallat (límit de
l'API?): " + resposta.body().substring(0, 80));
                    }
                })
                .join();  // espera que acabe abans de finalitzar el main
    }
}
```

La màgia està en l'ordre de les sortides: "Petició llançada" s'imprimeix **abans** de "Resposta rebuda", encara que la petició es va llançar abans. Això és `sendAsync`: el fil principal no es deté esperant la xarxa; el `thenAccept` s'executa quan arribi la resposta. El `join()` al final és

> imprescindible: sense ell, el `main` acabaria i la JVM es tancaria abans que arribés la resposta. El comptatge de repos amb `split("\"full_name\"")` és un truc ràpid de JSON sense llibreria: cada repo apareix com un `"full_name"` en la llista.

>

Butlletí U15 — Extres

📝 Butlletí U15 — Extres

> Exercicis de CodeWars i AceptaElReto amb pistes i solucions. Les solucions estan amagades: esgota la teva pista abans de mirar-les.

CodeWars

1. IP Validation

Et donen una cadena que diu ser una adreça IPv4. Escriu una funció que torni `true` només si és una IP vàlida: quatre parts, cada una un número entre 0 i 255, sense zeros a l'esquerra i sense caràcters estranys.

Exemples: `"1.2.3.4"` → `true`, `"123.045.067.089"` → `false` (zeros a l'esquerra), `"12.34.56"` → `false` (falten parts), `"a.b.c.d"` → `false`.

- Enunciat a CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Divideix per `.` amb `split("\\.", -1)` — compte: el `.` és un comodí en regex, per això s'escapa. Han de ser exactament 4 parts, cada una entre 0 i 255, sense zeros a l'esquerra i amb tots els caràcters dígits.

> 🔄 **Solució**

```java
public class Kata {
    public static boolean isValidIP(String ip) {
        String[] partes = ip.split("\\.", -1);
        if (partes.length != 4) return false;

        for (String p : partes) {
            if (p.isEmpty() || p.length() > 3) return false;
            if (p.length() > 1 && p.charAt(0) == '0') return false;
            for (char c : p.toCharArray()) {
                if (!Character.isDigit(c)) return false;
            }
            int n = Integer.parseInt(p);
            if (n < 0 || n > 255) return false;
        }
        return true;
    }
}
```

Tres trampes típiques: el `.` en regex (cal escapar-lo), el `-1` en `split` (per a no descartar parts buides finals) i els zeros a l'esquerra (`"045"` no és vàlid encara que valgui 45). És un exercici de validació de format, com validar una URL o un JSON.

---

2. Simple URL parser

Et donen una URL com `"http://www.codewars.com/kata/56f8fe6a2e6c0dc83b0008a7?page=1"`. Escriu una funció que la divideixi en **protocol**, **domini** i **ruta**.

Exemples: `"http://www.codewars.com/path"` → protocol `http`, domini `www.codewars.com`, ruta `/path`. `"https://example.com"` → protocol `https`, domini `example.com`, ruta buida.

- Enunciat a CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Busca primer `://` (divideix protocol de la resta). Després busca la primera `/` (divideix domini de ruta). Si alguna cosa no està, aquell camp queda buit. Usa `indexOf` i `substring`.

> 🔄 **Solució**

```java
record UrlParts(String protocolo, String dominio, String ruta) {}

public class Kata {
    public static UrlParts parsear(String url) {
        String restante = url;
        String protocolo = "";

        int dosPuntos = restante.indexOf("://");
        if (dosPuntos >= 0) {
            protocolo = restante.substring(0, dosPuntos);
            restante = restante.substring(dosPuntos + 3);
        }

        String dominio;
        String ruta = "";
        int barra = restante.indexOf('/');
        if (barra >= 0) {
            dominio = restante.substring(0, barra);
            ruta = restante.substring(barra);
        } else {
            dominio = restante;
        }

        return new UrlParts(protocolo, dominio, ruta);
    }
}
```

És la mateixa anatomia de URL del punt 1, portada a codi: el protocol acaba en `://`, el domini acaba en `/`. `indexOf` localitza els separadors i `substring` talla. Un `record` (U13) és la forma neta de tornar tres dades alhora.

---

3. Extract the domain name from a URL

Et donen una URL completa i has de tornar només el **nom de domini** (sense protocol, sense `www.`, sense extensió).

Exemples: `"http://github.com/carbonfive/raygun"` → `"github"`, `"http://www.zombie-bites.com"` → `"zombie-bites"`, `"https://www.cnet.com"` → `"cnet"`.

- Enunciat a CodeWars

- Dificultat: 5 kyu

> 💡 **Pista**
>
> Treu primer el protocol (`http://`, `https://`), després el `www.` si està, i finalment talla pel primer `.`. L'ordre de les operacions importa.

> 🔄 **Solució**

```java
public class Kata {
    public static String domainName(String url) {
        String s = url;
        s = s.replace("http://", "").replace("https://", "");
        if (s.startsWith("www.")) {
            s = s.substring(4);
        }
        int punto = s.indexOf('.');
        return punto >= 0 ? s.substring(0, punto) : s;
    }
}
```

El truc és l'ordre: sense protocol primer, `www.zombie-bites.com` començaria per `www.` i el tallaries malament. `replace` neteja el protocol, `startsWith` detecta el `www.` i `indexOf('.')` troba on acaba el nom. Petit, però amb trampes.

---

4. Decode the Morse code

Et donen un missatge en codi Morse (lletres separades per un espai, paraules per tres espais). Escriu-lo en text llegible.

Exemple: `"... --- ..."` → `"SOS"`, `".... . -.-- .--- ..- -.. ."` → `"HEY JUDE"`.

- Enunciat a CodeWars
- Dificultat: 6 kyu

> 💡 **Pista**
>
> Crea un `Map` amb cada símbol Morse → lletra (els mapes, de la U13). Separa paraules per tres espais i lletres per un. No oblidis `trim()` els extrems.

> 🔄 **Solució**

```java
import java.util.HashMap;
import java.util.Map;

public class MorseDecoder {

    static final Map<String, String> MORSE = new HashMap<>();
    static {
        MORSE.put(".-", "A"); MORSE.put("-...", "B"); MORSE.put("-.-.", "C");
        MORSE.put("-..", "D"); MORSE.put(".", "E"); MORSE.put("..-.", "F");
        MORSE.put("--.", "G"); MORSE.put("....", "H"); MORSE.put("..", "I");
        MORSE.put(".---", "J"); MORSE.put("-.-", "K"); MORSE.put(".-..", "L");
        MORSE.put("--", "M"); MORSE.put("-.", "N"); MORSE.put("---", "O");
        MORSE.put(".--.", "P"); MORSE.put("--.-", "Q"); MORSE.put(".-.", "R");
        MORSE.put("...", "S"); MORSE.put("-", "T"); MORSE.put("..-", "U");
        MORSE.put("...-", "V"); MORSE.put(".--", "W"); MORSE.put("-..-", "X");
        MORSE.put("-.--", "Y"); MORSE.put("--..", "Z");
          MORSE.put("-----", "0"); MORSE.put(".----", "1"); MORSE.put("..---",
"2");
          MORSE.put("...--", "3"); MORSE.put("....-", "4"); MORSE.put(".....",
"5");
          MORSE.put("-....", "6"); MORSE.put("--...", "7"); MORSE.put("---..",
"8");
        MORSE.put("----.", "9");
    }

    public static String decode(String morseCode) {
        StringBuilder resultado = new StringBuilder();
        for (String palabra : morseCode.trim().split(" {3}")) {
            for (String letra : palabra.split(" ")) {
                resultado.append(MORSE.getOrDefault(letra, ""));
            }
            resultado.append(" ");
        }
        return resultado.toString().trim();
    }
}
```

El `Map` associa cada símbol amb la seva lletra (U13), `split(" {3}")` separa paraules per tres espais i `split(" ")` separa lletres per un. `getOrDefault` torna `""` si el símbol és rar, i `trim()` treu els espais dels extrems. És un problema de parseig: separar, consultar, recompondre.

AceptaElReto

5. 396 — Quants dies falten?

Es donen dues dates i cal dir **quants dies hi ha entre elles** (els dies que falten per a la segona des de la primera).

Entrada: diversos casos de prova. Cada cas porta dues dates amb el format dia, mes i any. L'entrada acaba quan no queden dades.

- Enunciat a AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**
>
> Converteix cada data a dia de l'any (nombre de dies des de l'1 de gener) i resta. O, més directe amb Java modern: `ChronoUnit.DAYS.between(fecha1, fecha2)` amb `LocalDate`.

> 🔄 **Solució**

```java
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.Scanner;

public class CuantosDiasFaltan {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (sc.hasNextInt()) {
            int d1 = sc.nextInt(), m1 = sc.nextInt(), a1 = sc.nextInt();
            int d2 = sc.nextInt(), m2 = sc.nextInt(), a2 = sc.nextInt();

            LocalDate f1 = LocalDate.of(a1, m1, d1);
            LocalDate f2 = LocalDate.of(a2, m2, d2);

            long dias = ChronoUnit.DAYS.between(f1, f2);
            System.out.println(dias);
        }
        sc.close();
    }
}
```

> `LocalDate` et llibera dels càlculs manuals (mesos de 30 i 31, anys bixests…). `ChronoUnit.DAYS.between` torna els dies entre dues dates, sigui el desfasament positiu o negatiu. L'alternativa "clàssica" era convertir cada data a dia de l'any i restar, però Java modern ho fa en una línia.

---

6. 462 — Dia de la setmana

Et donen una data (dia, mes i any) i has de dir **quin dia de la setmana és**.

Entrada: diversos casos. Cada cas: una data en una línia amb dia, mes i any. L'entrada acaba amb `0 0 0`.

- Enunciat a AceptaElReto
- Dificultat: ⭐⭐

> 💡 **Pista**
>
> Usa la congruència de Zeller o un dia de referència conegut per a calcular el residu. En Java modern, `LocalDate.of(a, m, d).getDayOfWeek()` et dona el dia directament.

> 🔄 **Solució**

```java
import java.time.LocalDate;
import java.util.Scanner;

public class DiaSemana {

    static final String[] DIAS = {"LUNES", "MARTES", "MIÉRCOLES",
        "JUEVES", "VIERNES", "SÁBADO", "DOMINGO"};

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (sc.hasNextInt()) {
            int d = sc.nextInt(), m = sc.nextInt(), a = sc.nextInt();
            if (d == 0 && m == 0 && a == 0) break;

            LocalDate fecha = LocalDate.of(a, m, d);
            // getDayOfWeek().getValue(): 1=DILLUNS ... 7=DIUMENGE
            System.out.println(DIAS[fecha.getDayOfWeek().getValue() - 1]);
        }
        sc.close();
    }
}
```

`getDayOfWeek().getValue()` torna 1 per al dilluns i 7 per al diumenge; restant 1 tens l'índex de l'array. La "manera de concurs" era la congruència de Zeller (una fórmula que calcula el dia sense calendari), però `LocalDate` fa el mateix per dins: mateix resultat, menys codi.

🧭 **I si et quedes amb ganes?** Quan domines servidors i clients HTTP, torna als problemes d'unitats anteriors i planteja'ls com a APIs: un problema que tornava text per consola pot tornar JSON per HTTP. El material no es perd: es reutilitza.